MACHINE LEARNING

AI · ML · DS / REVISION NOTES

COURSERA · DEEPLEARNING.AI & STANFORD ONLINE · ANDREW NG

Unsupervised Learning, Recommenders, Reinforcement Learning

Machine Learning Specialization, Course 3 — clustering, anomaly detection, recommender systems, RL

46
SECTIONS

503
FIGURES

443
PAGES

Compiled October 2026
github.com/ankitgupta1729/AI-ML-DS

Contents

LECTURE 1 Beyond Supervised Learning 4
LECTURE 2 Unsupervised Learning 6
LECTURE 3 Cluster Centroid 10
LECTURE 4 K-means for Clusters That Are Not Well 28
LECTURE 5 Cost Function and Cluster Centroid 40
LECTURE 6 Random Initialization 50
LECTURE 7 Clusters 59
LECTURE 8 Density Estimation 68
LECTURE 9 Gaussian Distribution Example 83
LECTURE 10 Anomaly Detection Example 94
LECTURE 11 Aircraft Engines Monitoring Example 104
LECTURE 12 Anomaly Detection 116
LECTURE 13 Error Analysis for Anomaly Detection 120
LECTURE 14 Predicting Movie Ratings 138
LECTURE 15 Cost Function 141
LECTURE 16 Collaborative Filtering 152
LECTURE 17 Cost Function for Binary Application 164
LECTURE 18 Mean Normalization 172
LECTURE 19 Derivatives in ML 182
LECTURE 20 Finding Related Items 199
LECTURE 21 Content-based Filtering: Learning to Match 206
LECTURE 22 Neural Network Architecture 216
LECTURE 23 Retrieval Step 226
LECTURE 24 Travel Industry Payday Loans 231
LECTURE 25 Item Features 239
LECTURE 26 PCA: Find New Axis and Coordinates 241
LECTURE 27 Choose an Axis 252
LECTURE 28 Applications of PCA 270

LECTURE 29 Autonomous Helicopter 282
LECTURE 30 Mars Rover Example 292
LECTURE 31 Example of Return 302
LECTURE 32 The Goal of Reinforcement Learning 312
LECTURE 33 Markov Decision Process (MDP) 319
LECTURE 34 State Action Value Function 323
LECTURE 35 State Action Value Function Example 334
LECTURE 36 Bellman Equation 338
LECTURE 37 Expected Return 350
LECTURE 38 Discrete vs Continuous State 368
LECTURE 39 Lunar Lander 371
LECTURE 40 Bellman Equation 379
LECTURE 41 Neural Network 395
LECTURE 42 Learning Algorithm 399
LECTURE 43 Learning Algorithm 409
LECTURE 44 Limitations of Reinforcement Learning 432
LECTURE 1 Machine Learning 434
LECTURE 46 Robotics 435

LECTURE 1

Beyond Supervised Learning

Welcome to this third and final course of this specialization on unsupervised learning, recommender systems, and reinforcement learning. Whereas in the first two courses, we spent a lot of time on supervised learning, in this third and final course, we'll talk about a new set of techniques that goes beyond supervised learning and we'll give you an extra set of powerful tools that I hope you enjoy adding to your tool sets and by the time you finish this course and finish this specialization, I think you'll be well on your way to being an expert in machine learning. Let's take a look. This week we'll start with supervised learning. In particular, you'll learn about clustering algorithms, which is a way of grouping data into clusters, as well as anomaly detection.

Both of these are techniques used by many companies today, in important commercial applications. By the end of this week, you know how these algorithms work and be able to get them to work with yourself as well. In the second week, you will learn about recommender systems. When you go to a online shopping websites or a video streaming website, how does it recommend products or movies to you? Recommender systems is one of the most commercially important machine learning technologies, is moving many billions of dollars worth of value or products or other things around, is one of the technologies that receives surprisingly little attention from academia, despite how important it is.

But in the second week, I hope you learn how these systems work and be able to implement one for yourself. If you are curious about how online ads systems work, the description of recommender systems will also give you a sense for how those large online ad tech companies decide what ads to show you. In the third and final week of this course, you'll learn about reinforcement learning. You may have read in the news about reinforcement learning being great at playing a variety of video games, even outperforming humans. I've also used reinforcement learning many times myself to control a variety of different robots. Even though reinforcement learning is a new and emerging technology, that is, the number of commercial applications of reinforcement learning is not nearly as large as the other techniques covered in this week or in the previous two courses of this specialization, is a technology that is exciting and is opening up a new frontier to what you can get learning algorithms to do.

In the final week, you implement reinforcement learning yourself and use it to land a simulated moon lander.

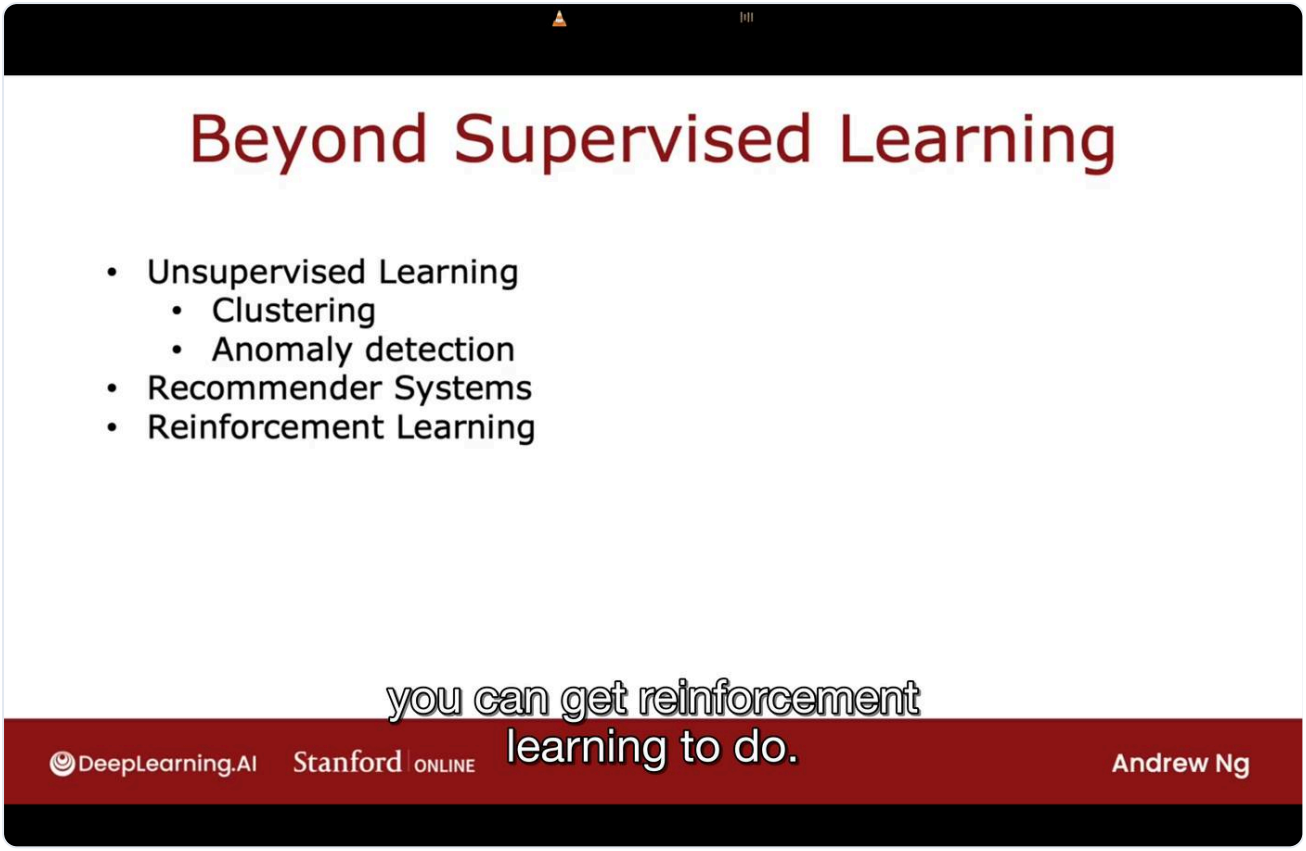


FIGURE 1.1 | Beyond Supervised Learning

When you see that working for yourself with your own code, later in this course, I think you'll be impressed by what you can get reinforcement learning to do. I'm really excited to be here with you to talk about unsupervised learning, recommender systems, and reinforcement learning. Let's go on to the next video where you learn about an important unsupervised learning algorithm called a clustering algorithm.

LECTURE 2

Unsupervised Learning

What is clustering? A clustering algorithm looks at a number of data points and automatically finds data points that are related or similar to each other. Let's take a look at what that means. Let me contrast clustering, which is an unsupervised learning algorithm, with what you had previously seen with supervised learning for binary classification. Given a dataset like this with features x_1 and x_2.

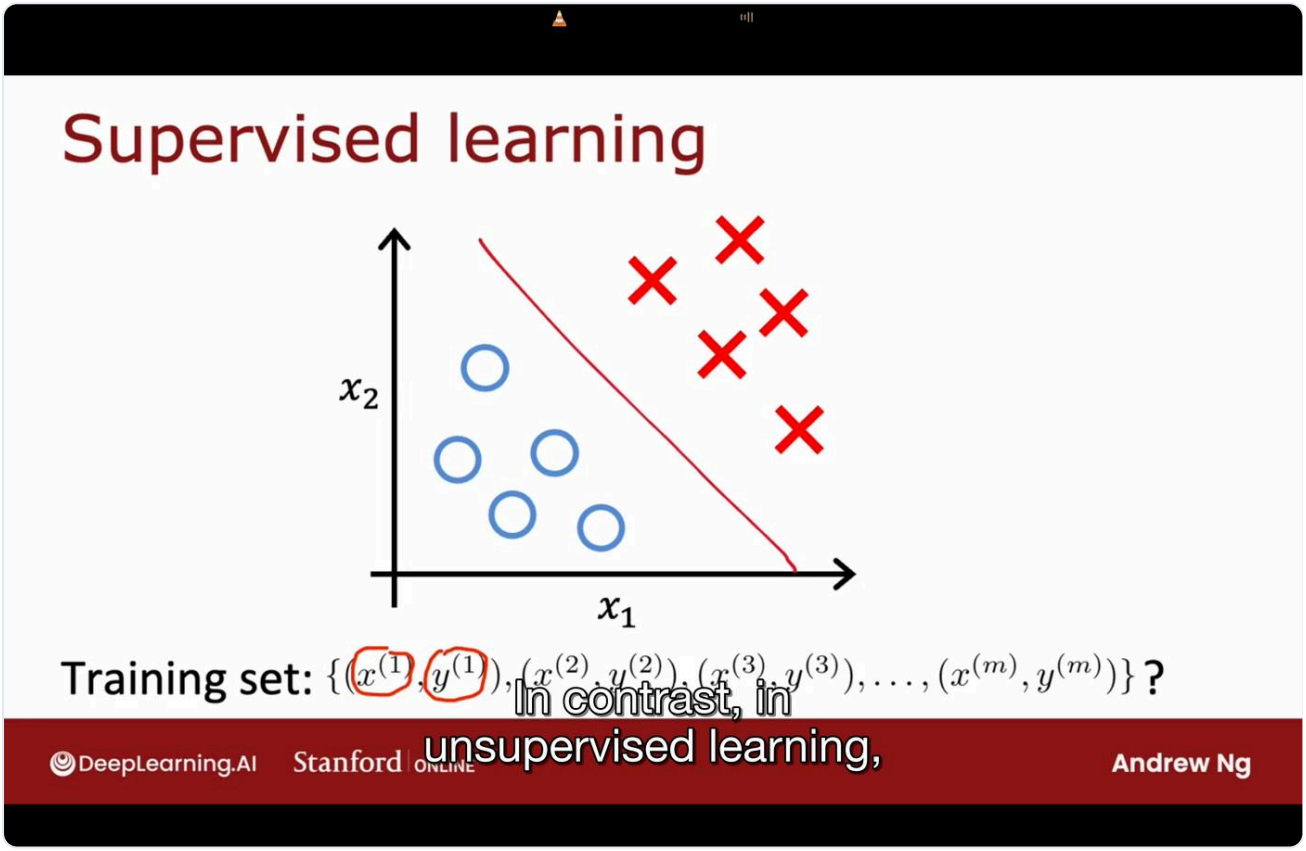


FIGURE 2.1 | Supervised Learning

With supervised learning, we had a training set with both the input features x as well as the labels y. We could plot a dataset like this and fit, say, a logistic regression algorithm or a neural network to learn a decision boundary like that. In supervised learning, the dataset included both the inputs x as well as the target outputs y. In contrast, in unsupervised learning, you are given a dataset like this with just x, but not the labels or the target labels y. That's why when I plot a dataset, it looks like this, with just dots rather than two classes denoted by the x's and the o's.

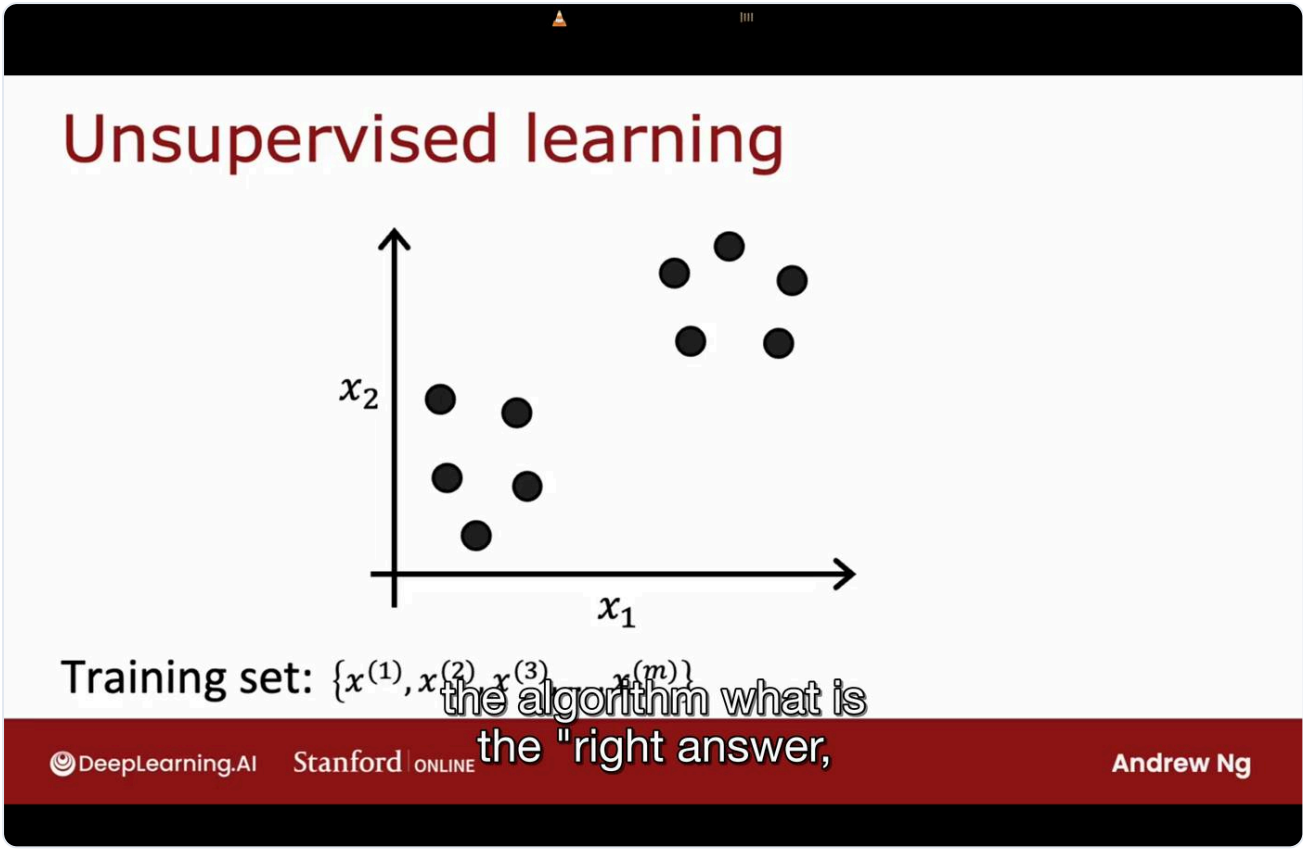


FIGURE 2.2 | Unsupervised Learning

Because we don't have target labels y, we're not able to tell the algorithm what is the "right answer, y" that we wanted to predict.

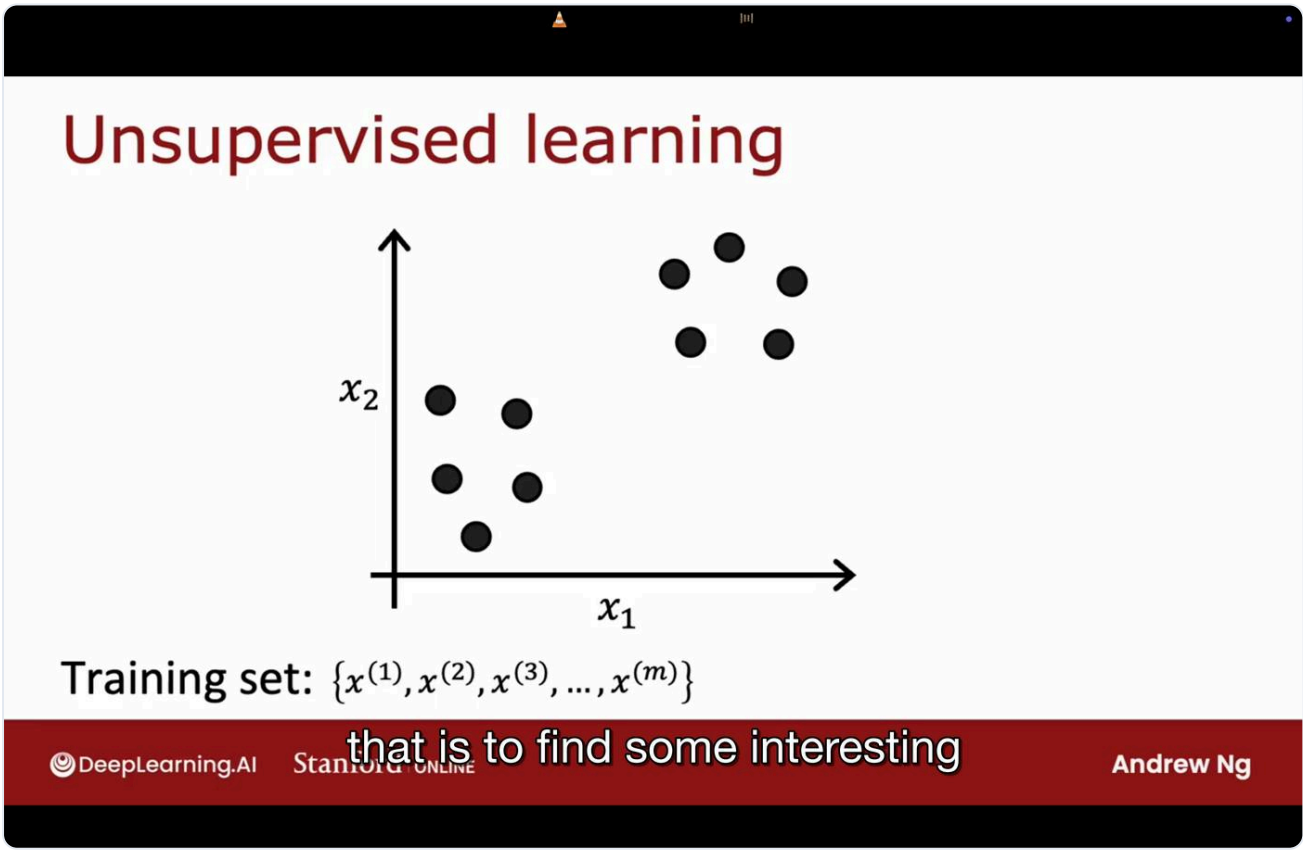


FIGURE 2.3 | Unsupervised Learning

Instead, we're going to ask the algorithm to find something interesting about the data, that is to find some interesting structure about this data. But the first unsupervised learning algorithm that you learn about is called a clustering algorithm, which looks for one particular type of structure in the data. Namely, look at the dataset like this and try to see if it can be grouped into clusters, meaning groups of points that are similar to each other. A clustering algorithm, in this case, might find that this dataset comprises of data from two clusters shown here. Here are some applications of clustering. In the first week of the first course, you heard me talk about grouping similar news articles together, like the story about Pandas or market segmentation, where at deeplearning.ai, we discovered that there are many learners that come here because you may want to grow your skills, or develop your careers, or stay updated with AI and understand how it affects your field of work.

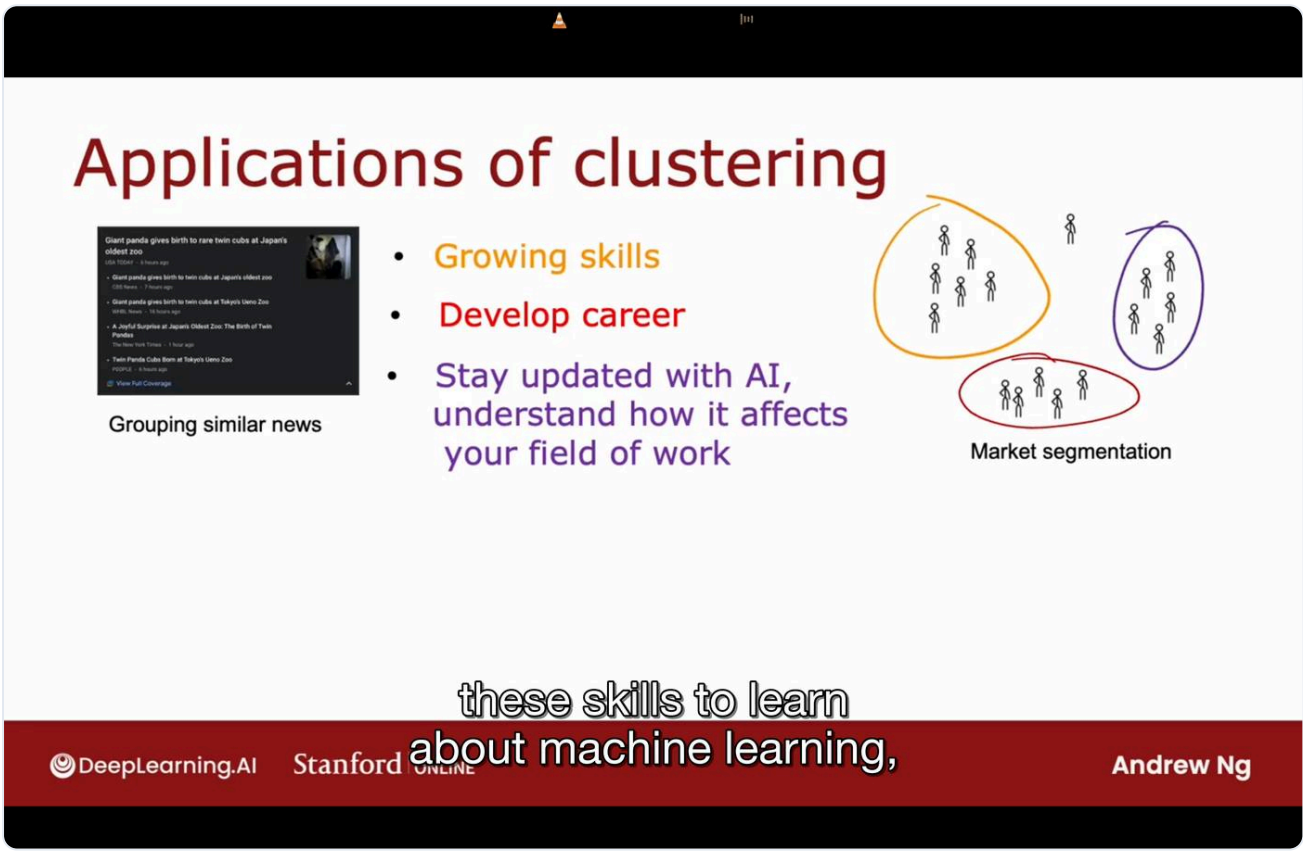


FIGURE 2.4 | Applications of Clustering

We want to help everyone with any of these skills to learn about machine learning, or if you don't fall into one of these clusters, that's totally fine too. I hope deeplearning.ai and Stanford Online's materials will be useful to you as well. Clustering has also been used to analyze DNA data, where you will look at the genetic expression data from different individuals and try to group them into people that exhibit similar traits. I find astronomy and space exploration fascinating.

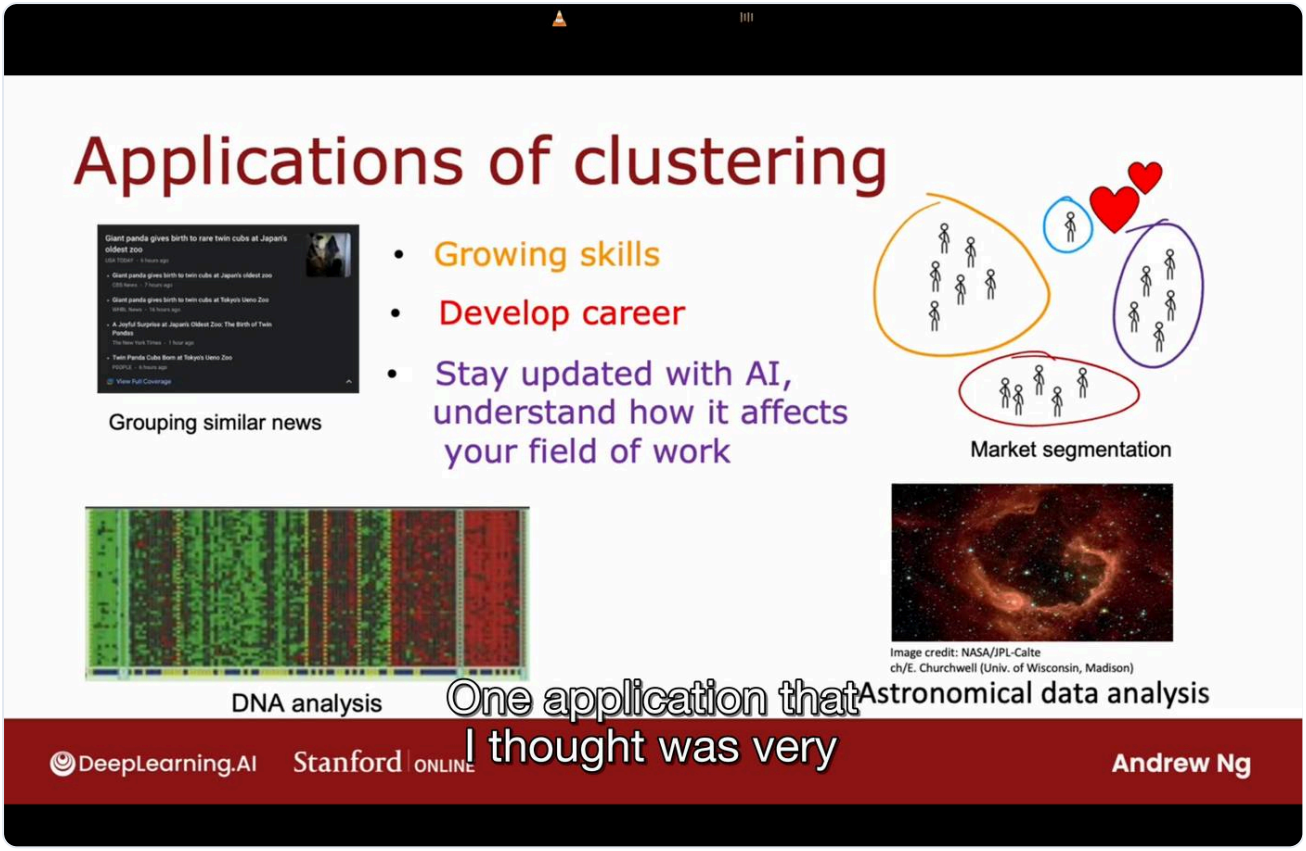


FIGURE 2.5

One application that I thought was very exciting was astronomers using clustering for astronomical data analysis to group bodies in space together for their own analysis of what's going on in space. One of the applications I found fascinating was astronomers using clustering to group bodies together to figure out which ones form one galaxy or which one form coherent structures in space. Clustering today is used for all of these applications and many more. In the next video, let's take a look at the most commonly used clustering algorithm called the k-means algorithm, and let's take a look at how it works.

LECTURE 3

Cluster Centroid

Let's take a look at what the K-means clustering algorithm does. Let me start with an example. Here I've plotted a data set with 30 unlabeled training examples. So there are 30 points. And what we like to do is run K-means on this data set. The first thing that the K-means algorithm does is it will take a random guess at where might be the centers of the two clusters that you might ask it to find. In this example I'm going to ask it to try to find two clusters. Later in this week we'll talk about how you might decide how many clusters to find. But the very first step is it will randomly pick two points, which I've shown here as a red cross and the blue cross, at where might be the centers of two different clusters.

This is just a random initial guess and they're not particularly good guesses.

FIGURE 3.1

But it's a start. One thing I hope you take away from this video is that K-means will repeatedly do two different things. The first is assign points to cluster centroids and the second is move cluster centroids. Let's take a look at what this means. The first of the two steps is it will go through each of these points and look at whether it is closer to the red cross or to the blue cross. The very first thing that K-means does is it will take a random guess at where are the centers of the cluster? And the centers of the cluster are called cluster centroids.

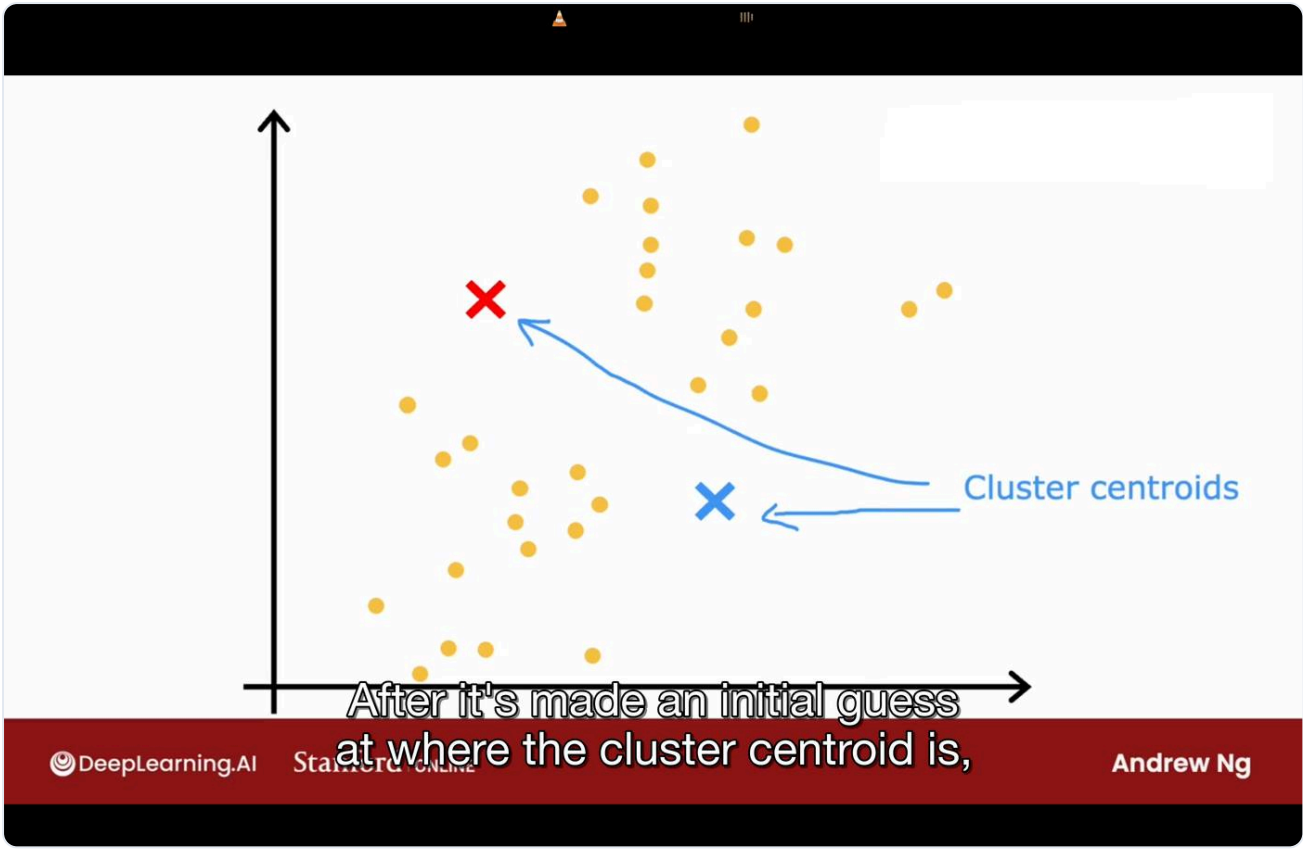


FIGURE 3.2

After it's made an initial guess at where the cluster centroid is, it will go through all of these examples, x(1) through x(30), my 30 data points.

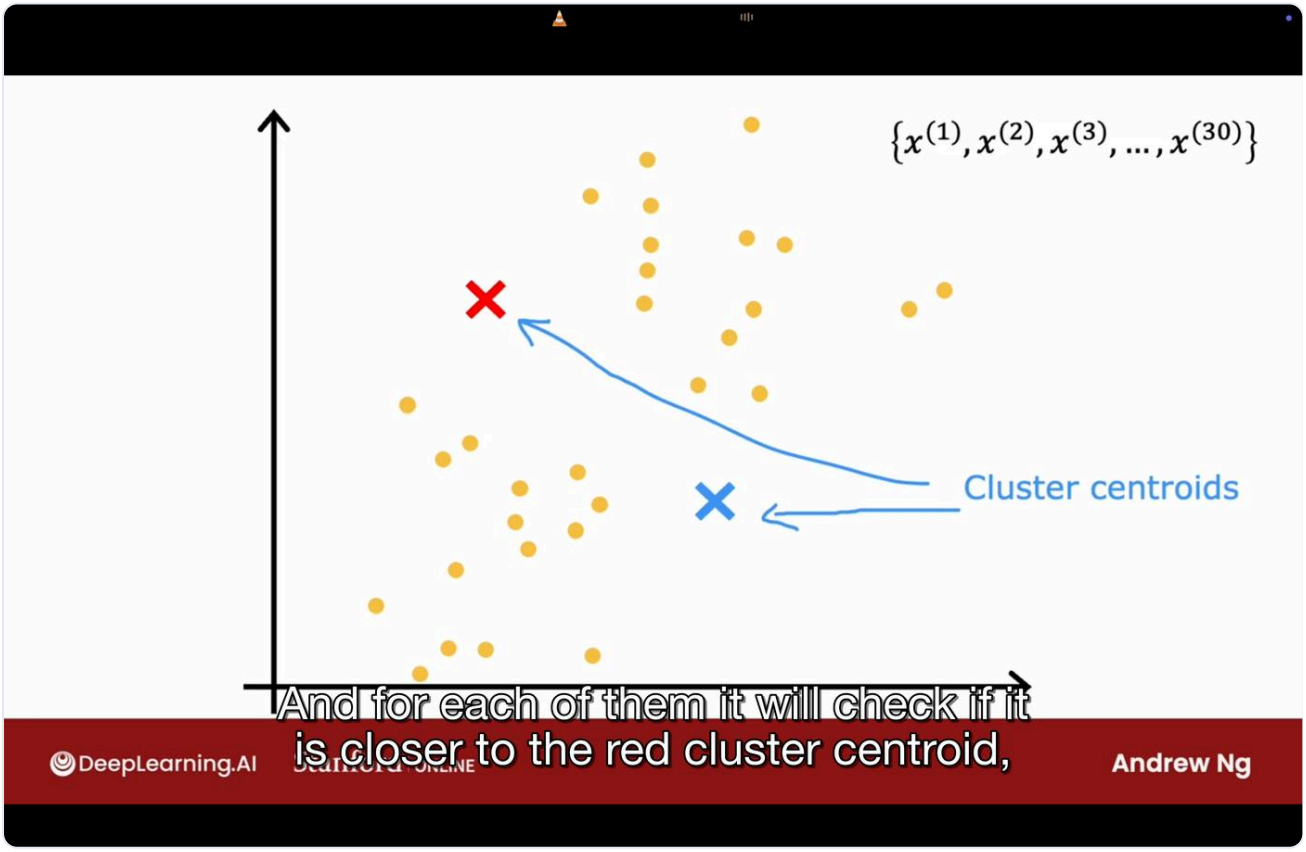


FIGURE 3.3

And for each of them it will check if it is closer to the red cluster centroid, shown by the red cross, or if it's closer to the blue cluster centroid, shown by the blue cross.

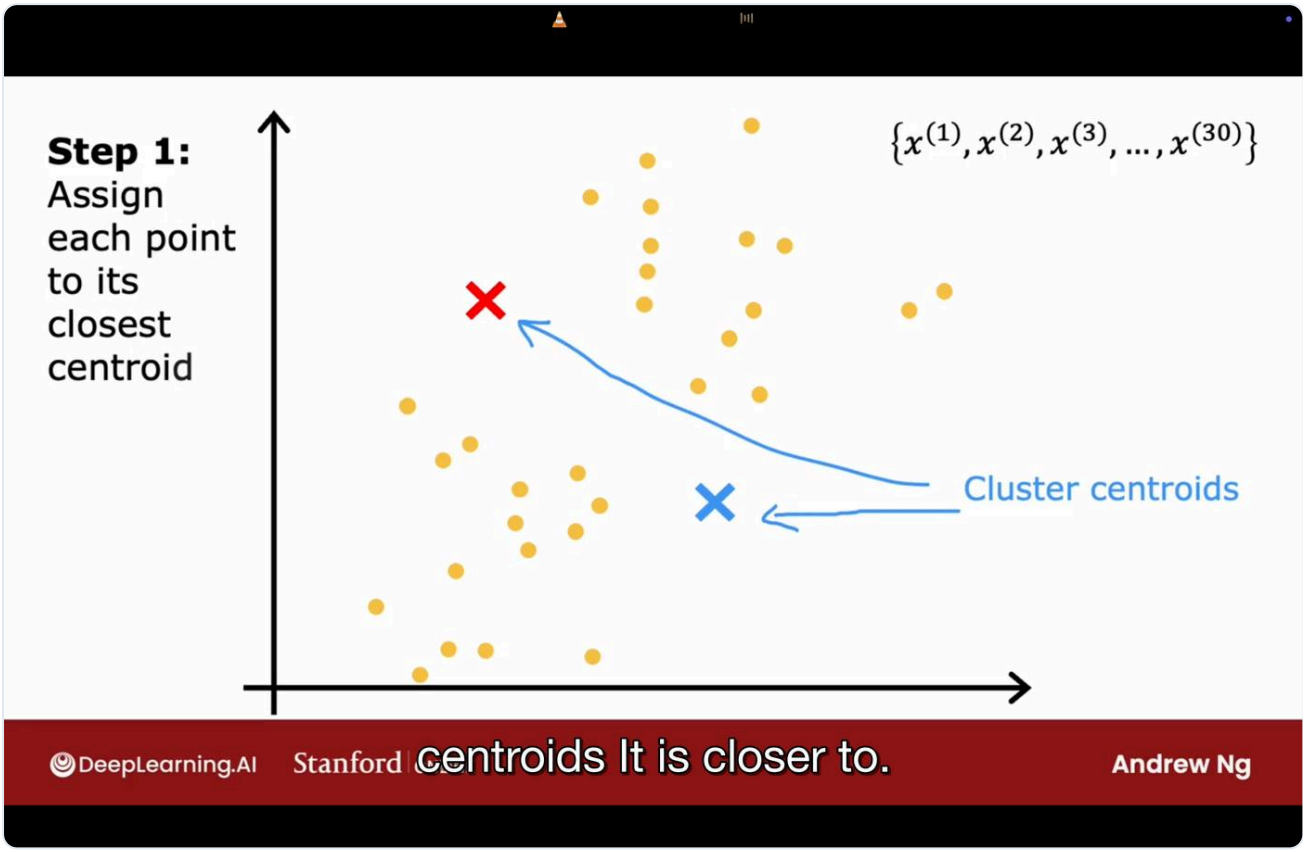


FIGURE 3.4

And it will assign each of these points to whichever of the cluster centroids It is closer to. I'm going to illustrate that by painting each of these examples, each of these little round dots, either red or blue, depending on whether that example is closer to the red or to the blue cluster centroid. So this point up here is closer to the red centroid, which is why it's painted red. Whereas this point down there is closer to the blue cluster centroid, which is why I've now painted it blue.

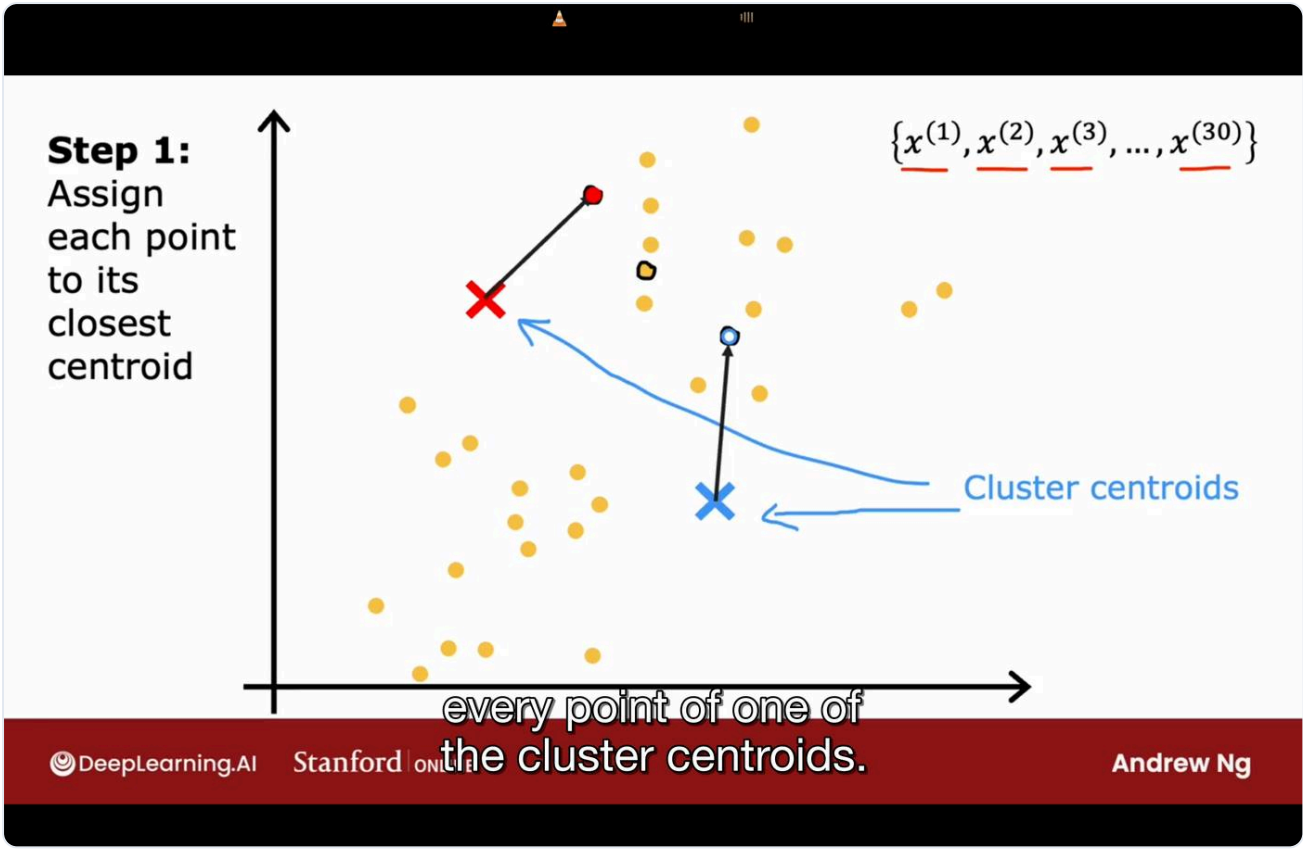


FIGURE 3.5

So that was the first of the two things that K-means does over and over. Which is a sign points to clusters centroids. And all that means is it will associate which I'm illustrating with the color, every point of one of the cluster centroids.

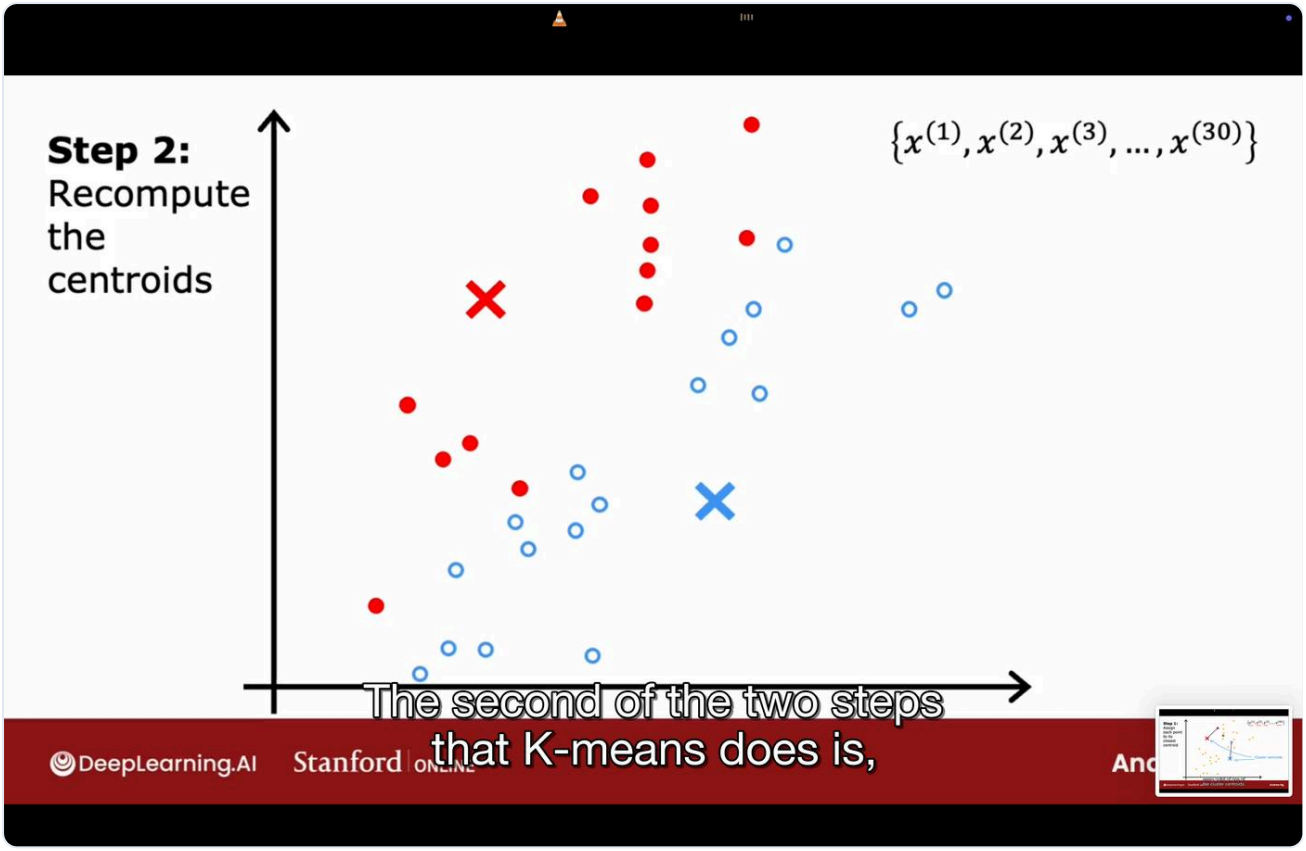


FIGURE 3.6

The second of the two steps that K-means does is, it'll look at all of the red points and take an average of them. And it will move the red cross to whatever is the average location of the red dots, which turns out to be here.

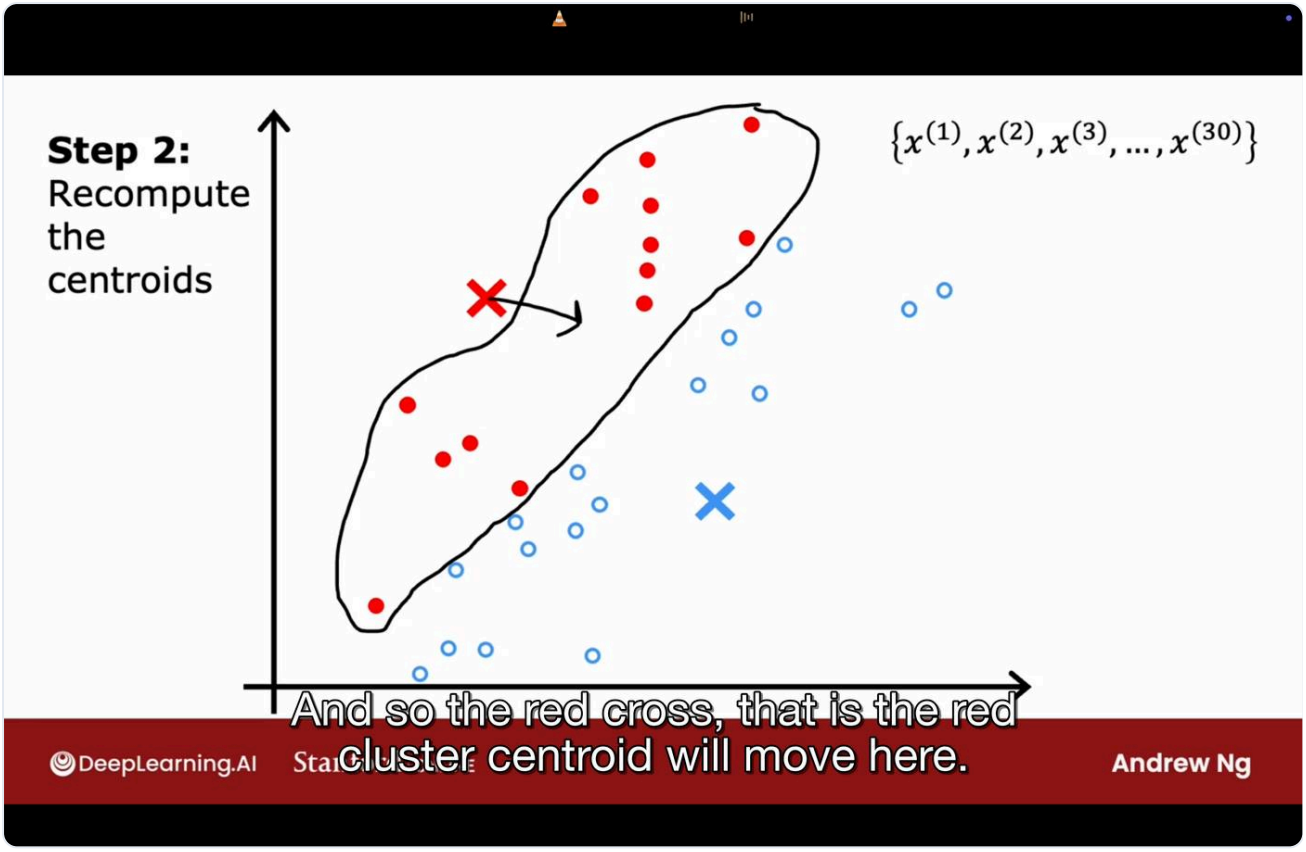


FIGURE 3.7

And so the red cross, that is the red cluster centroid will move here.

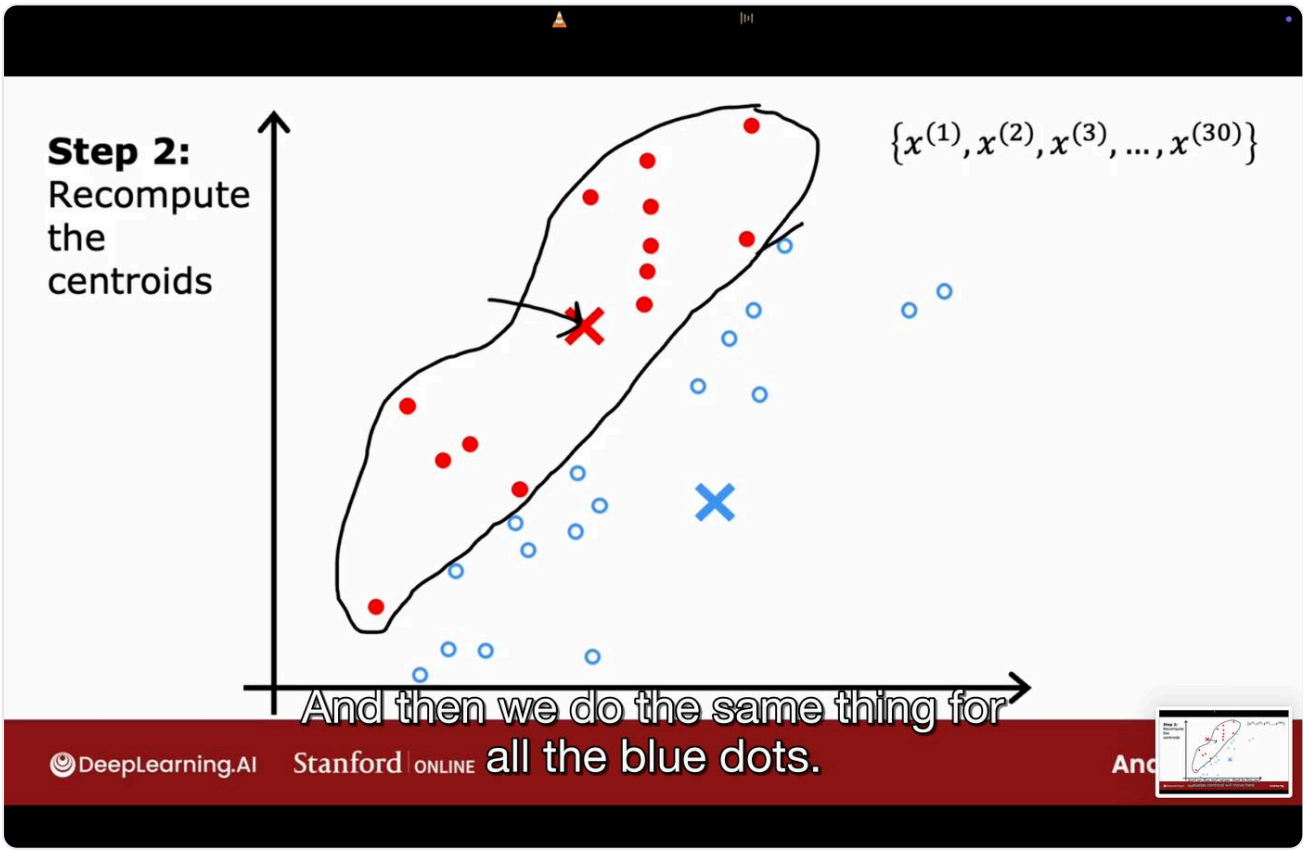


FIGURE 3.8

And then we do the same thing for all the blue dots.

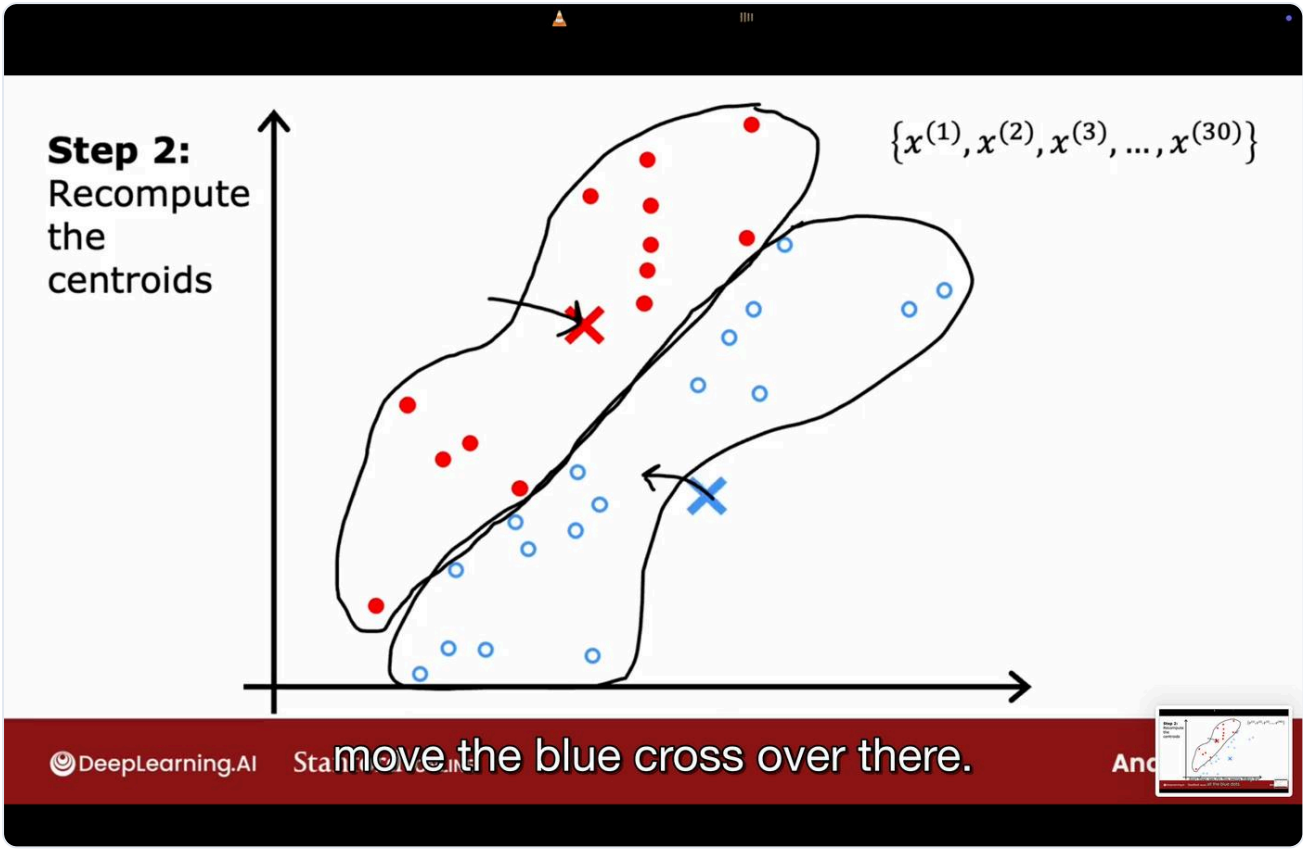


FIGURE 3.9

Look at all the blue dots, and take an average of them, and move the blue cross over there.

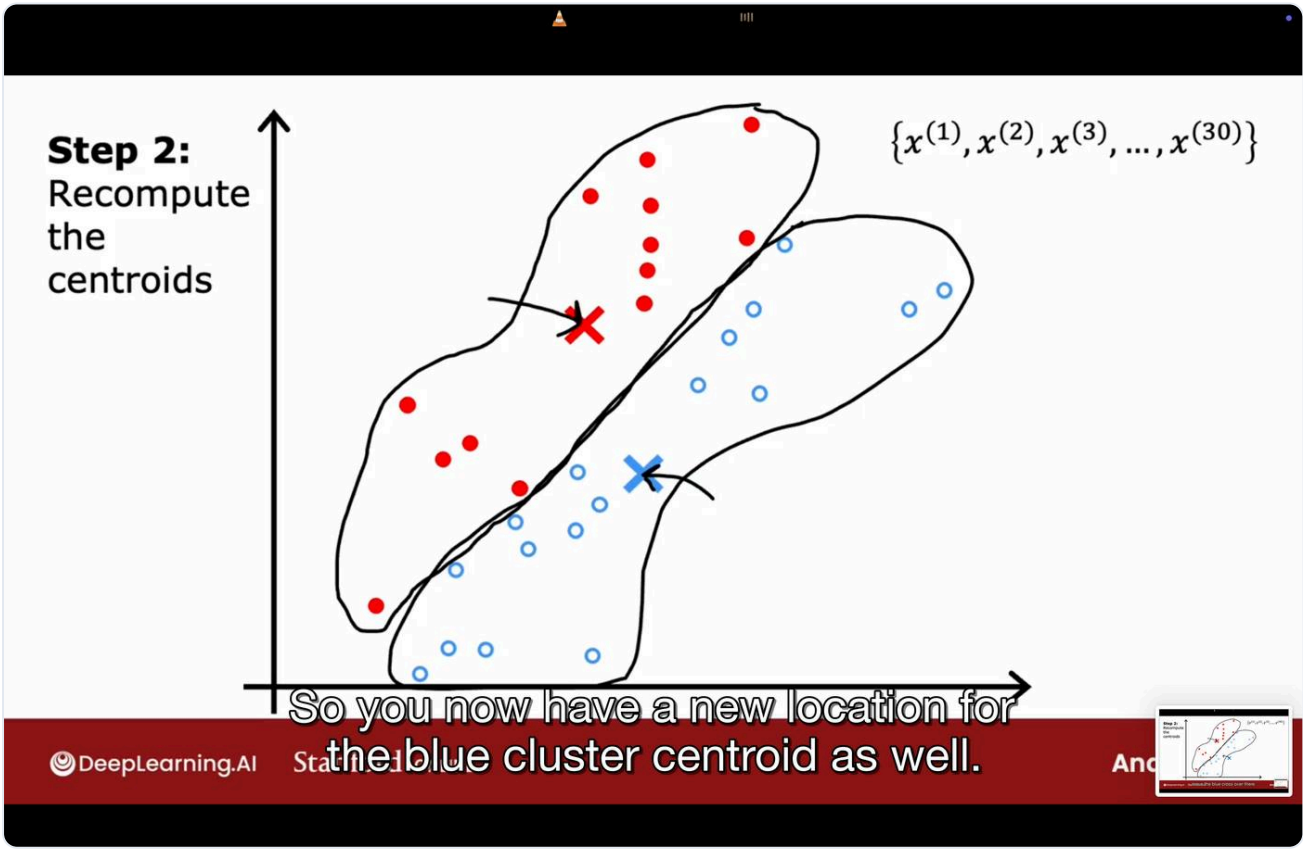


FIGURE 3.10

So you now have a new location for the blue cluster centroid as well. In the next video we'll look at the mathematical formulas for how to do both of these steps. But now that you have these new and hopefully slightly improved guesses for the locations of the to cluster centroids, we'll look through all of the 30 training examples again. And check for every one of them, whether it's closer to the red or the blue cluster centroid for the new locations.

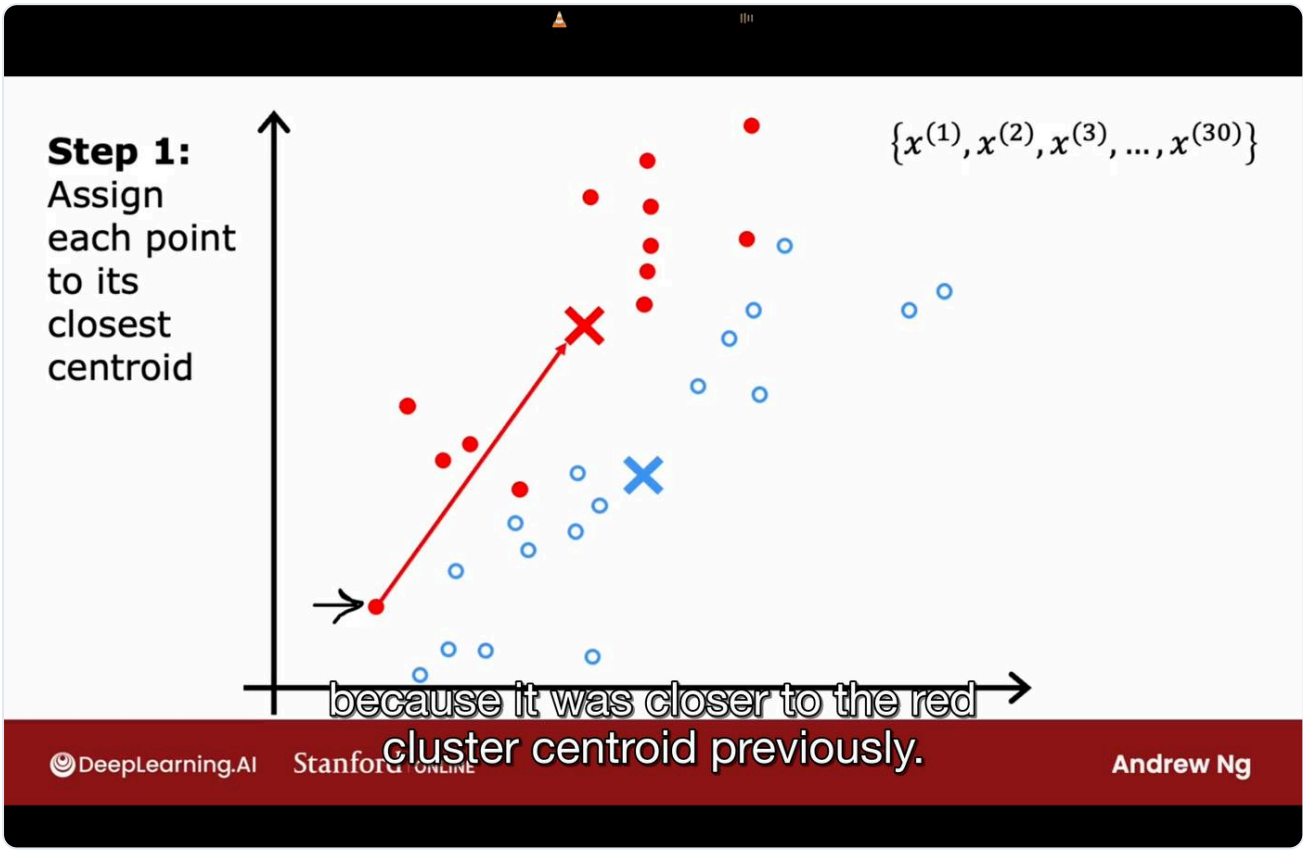


FIGURE 3.11

And then we will associate them which are indicated by the color again, every point to the closer cluster centroid. And if you do that, you see that the field points change color. So for example, this point is colored red, because it was closer to the red cluster centroid previously.

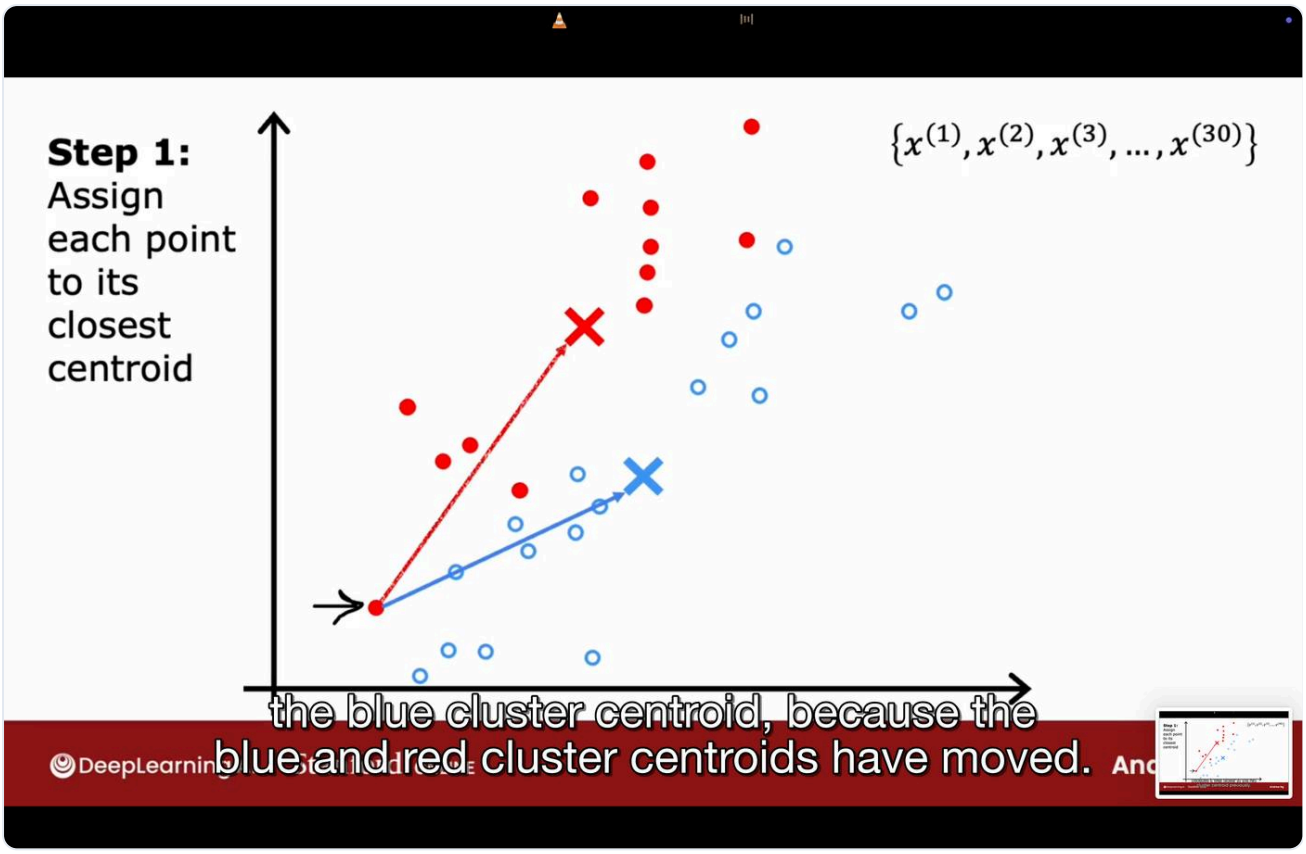


FIGURE 3.12

But if we now look again, it's now actually closer to the blue cluster centroid, because the blue and red cluster centroids have moved.

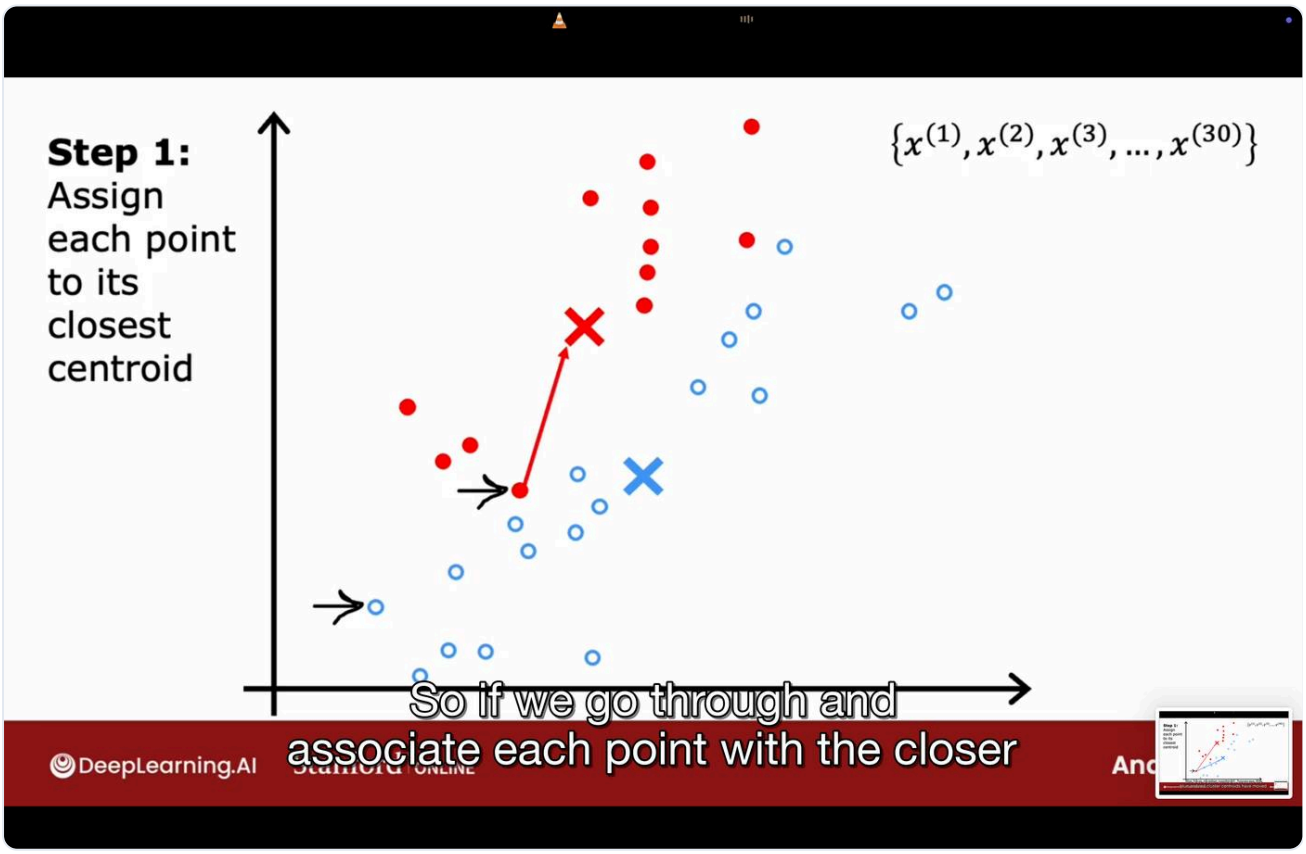


FIGURE 3.13

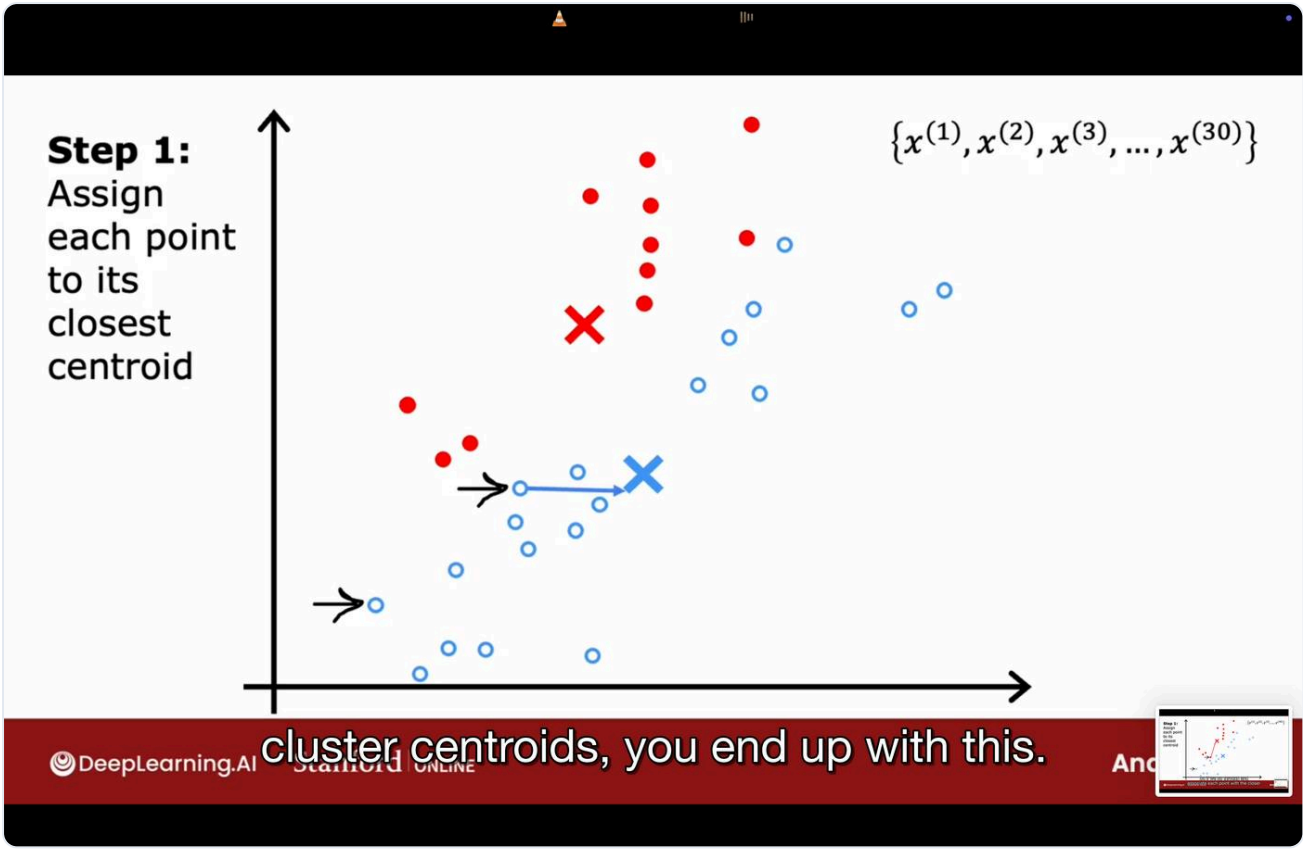


FIGURE 3.14

So if we go through and associate each point with the closer cluster centroids, you end up with this.

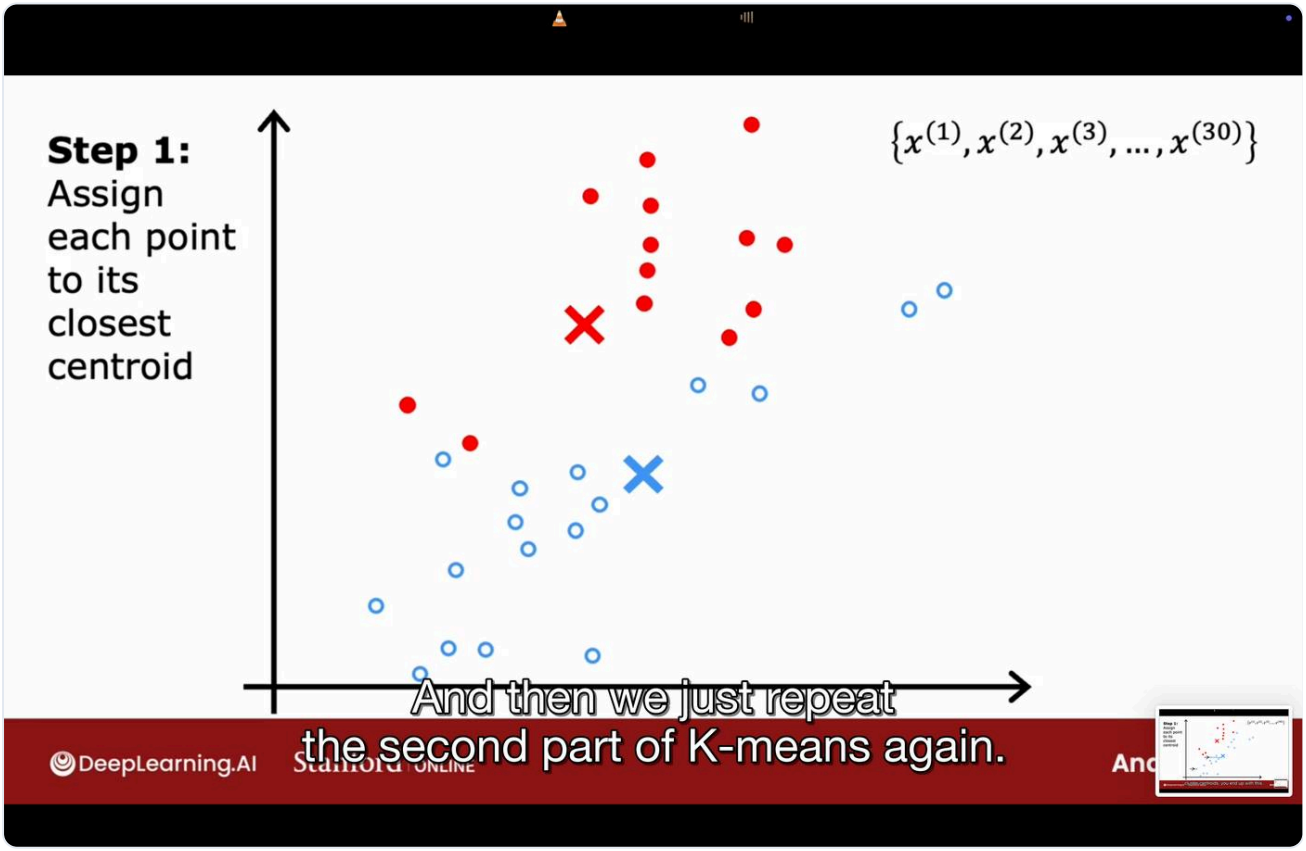


FIGURE 3.15

And then we just repeat the second part of K-means again.

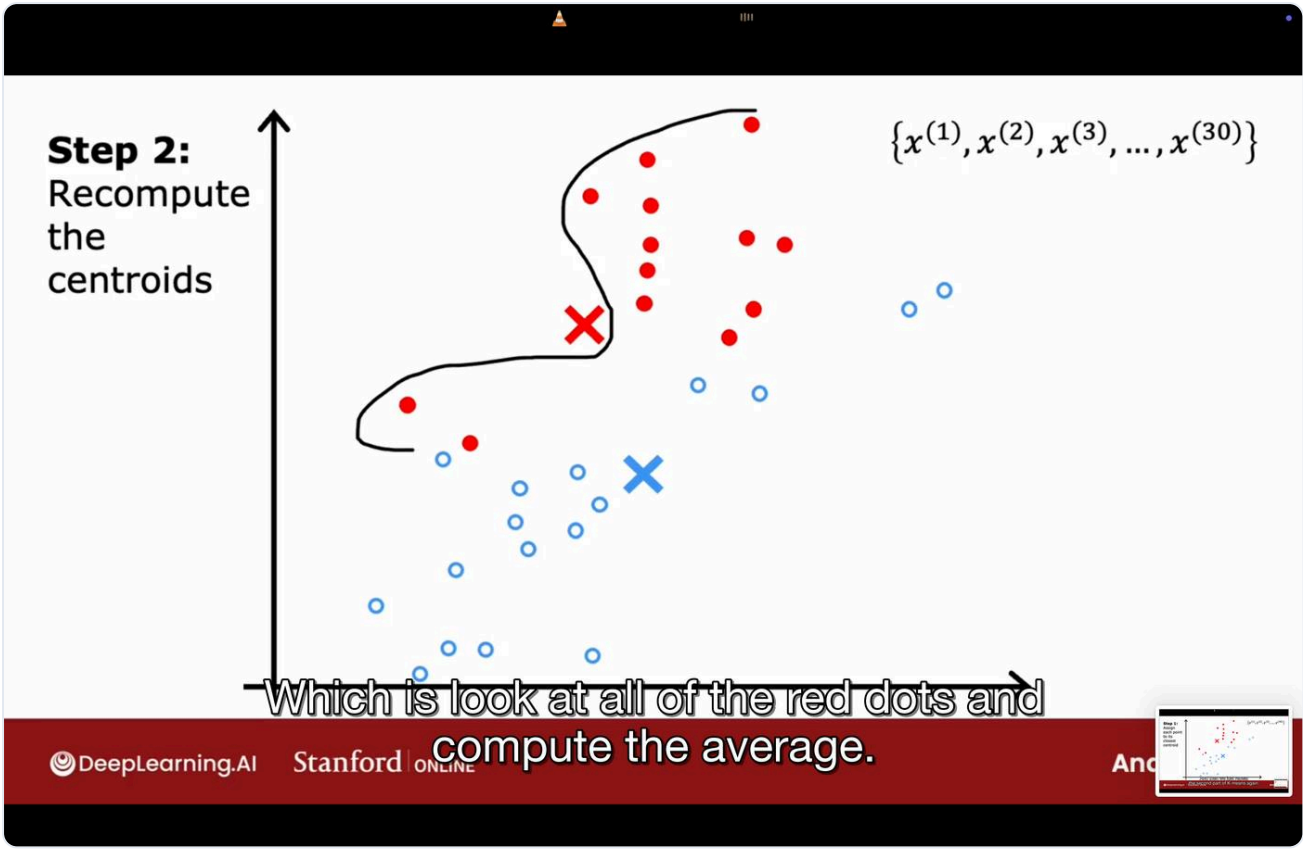


FIGURE 3.16

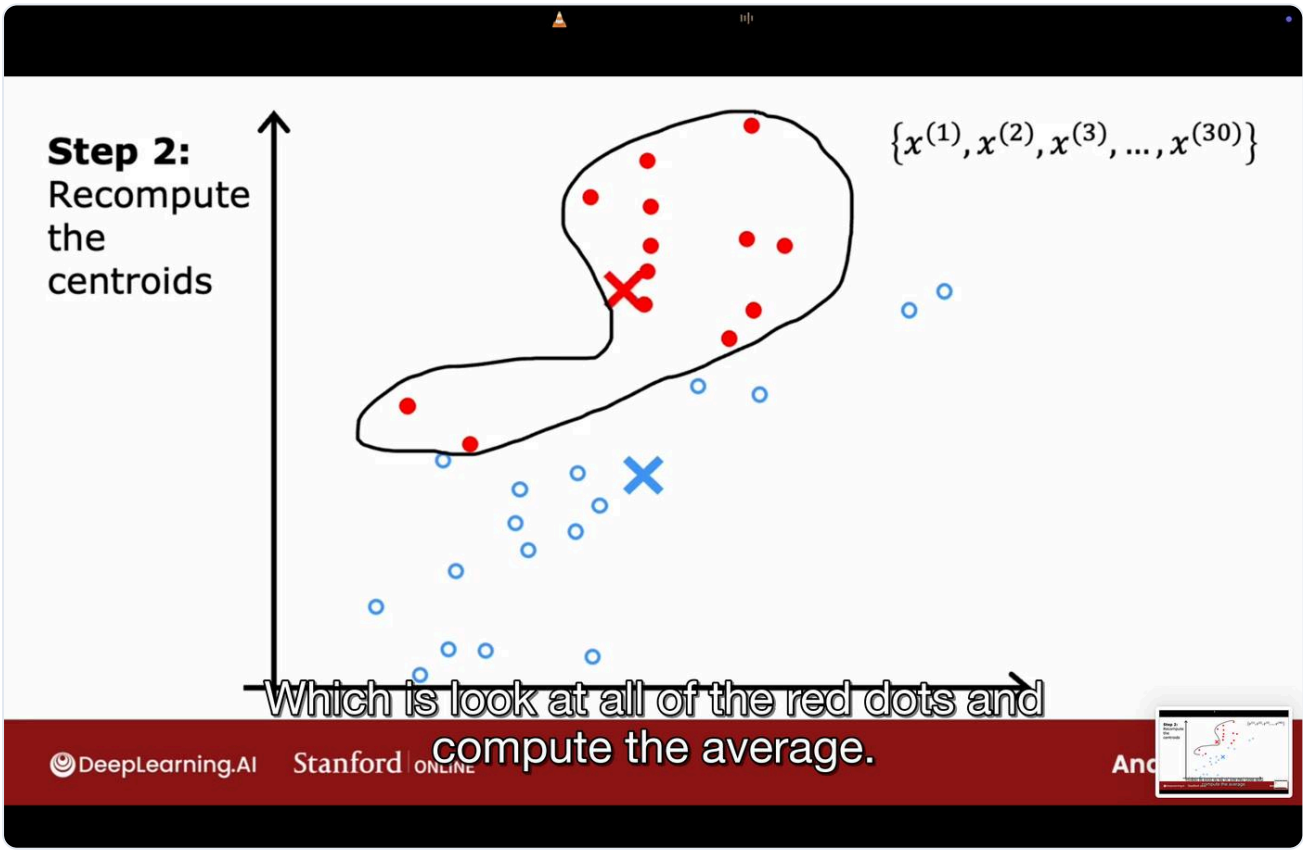


FIGURE 3.17

Which is look at all of the red dots and compute the average.

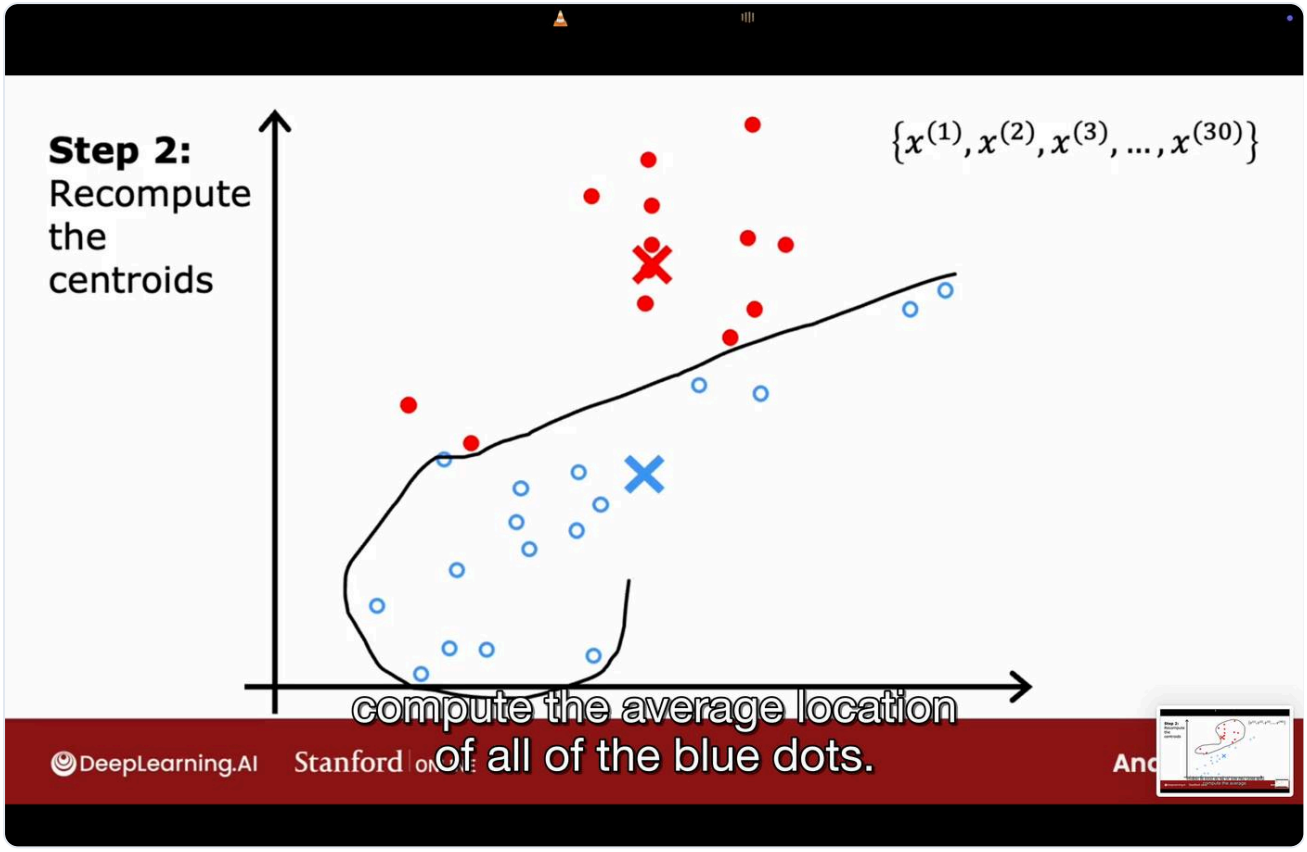


FIGURE 3.18

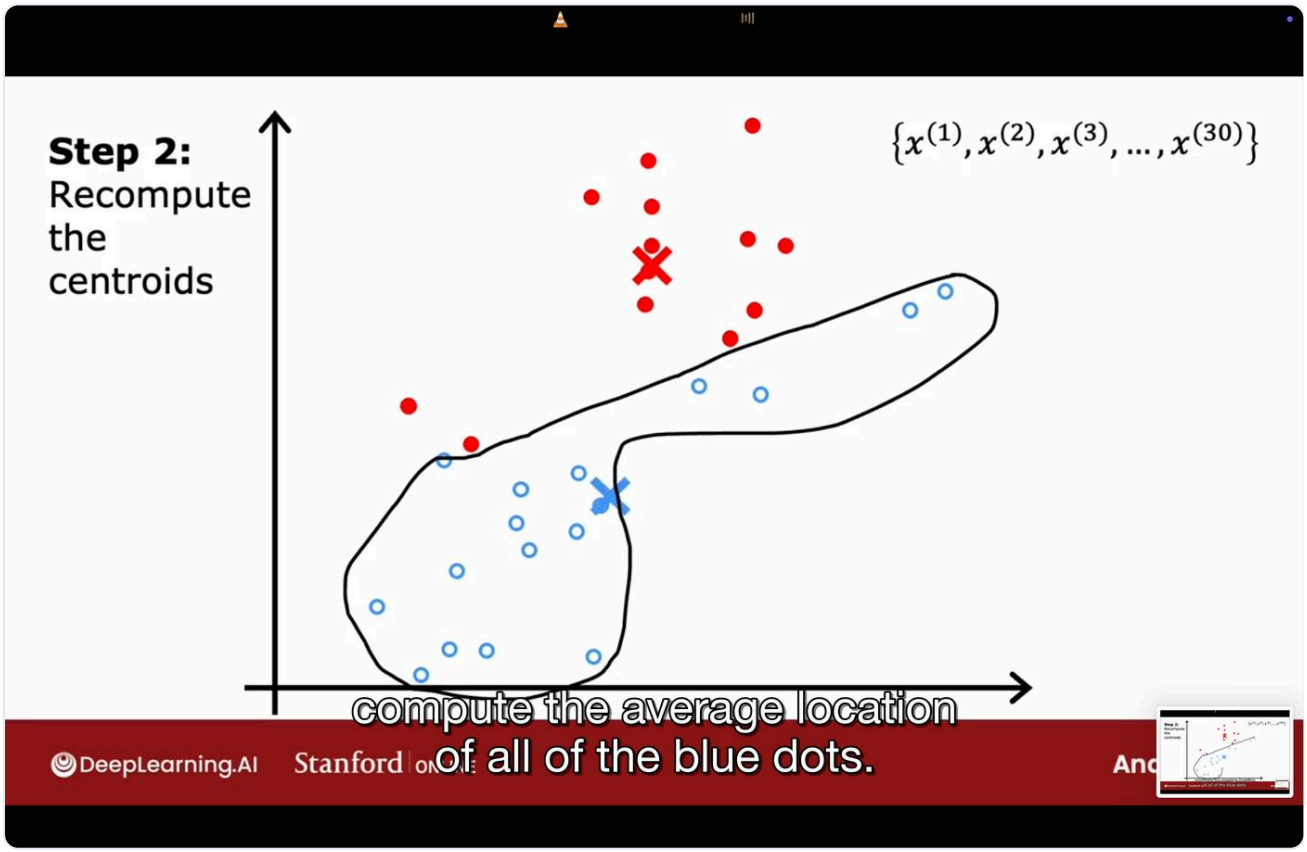


FIGURE 3.19

And also look at all of the blue dots and compute the average location of all of the blue dots.

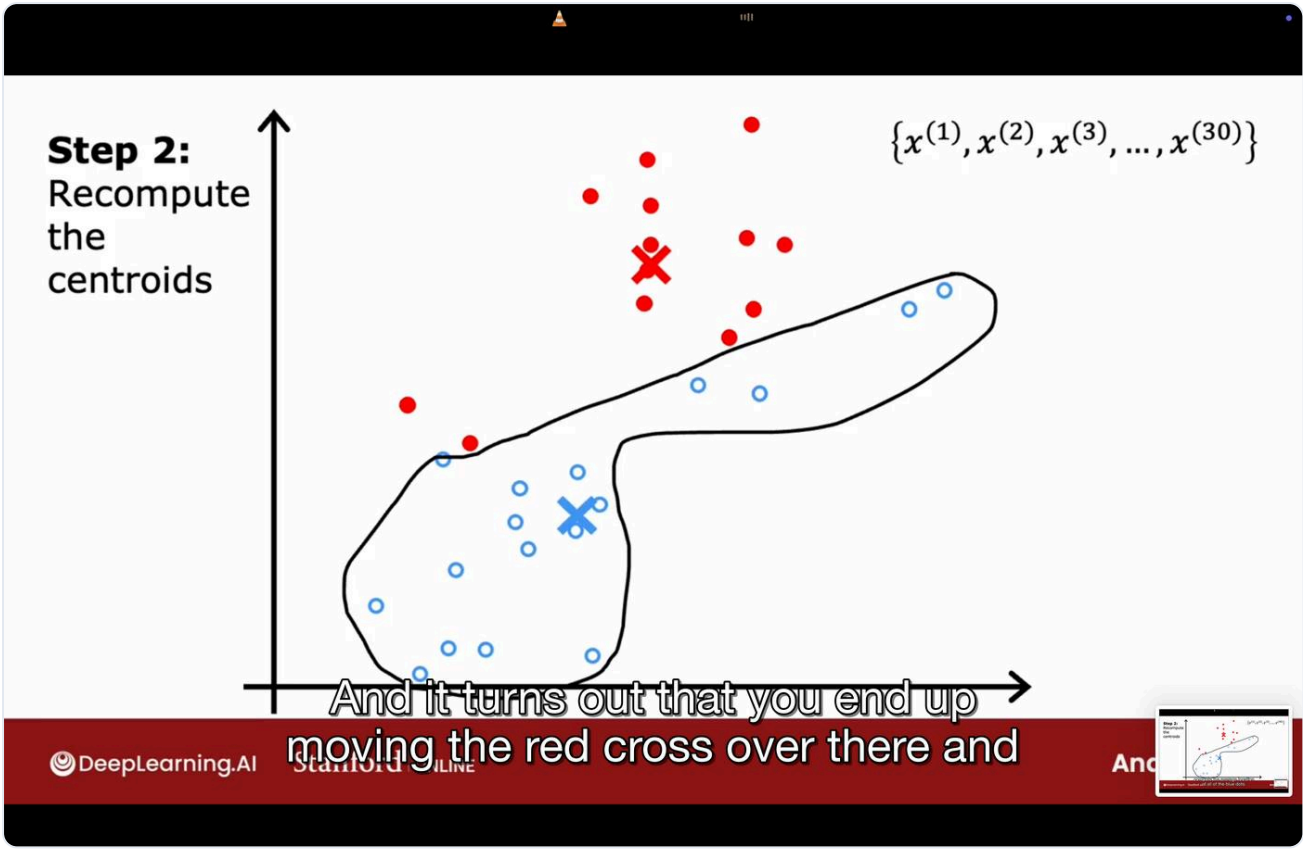


FIGURE 3.20

And it turns out that you end up moving the red cross over there and the blue cross over here. And we repeat. Let's look at all of the points again and we color them, either red or blue, depending on which cluster centroid that is closer to. So you end up with this.

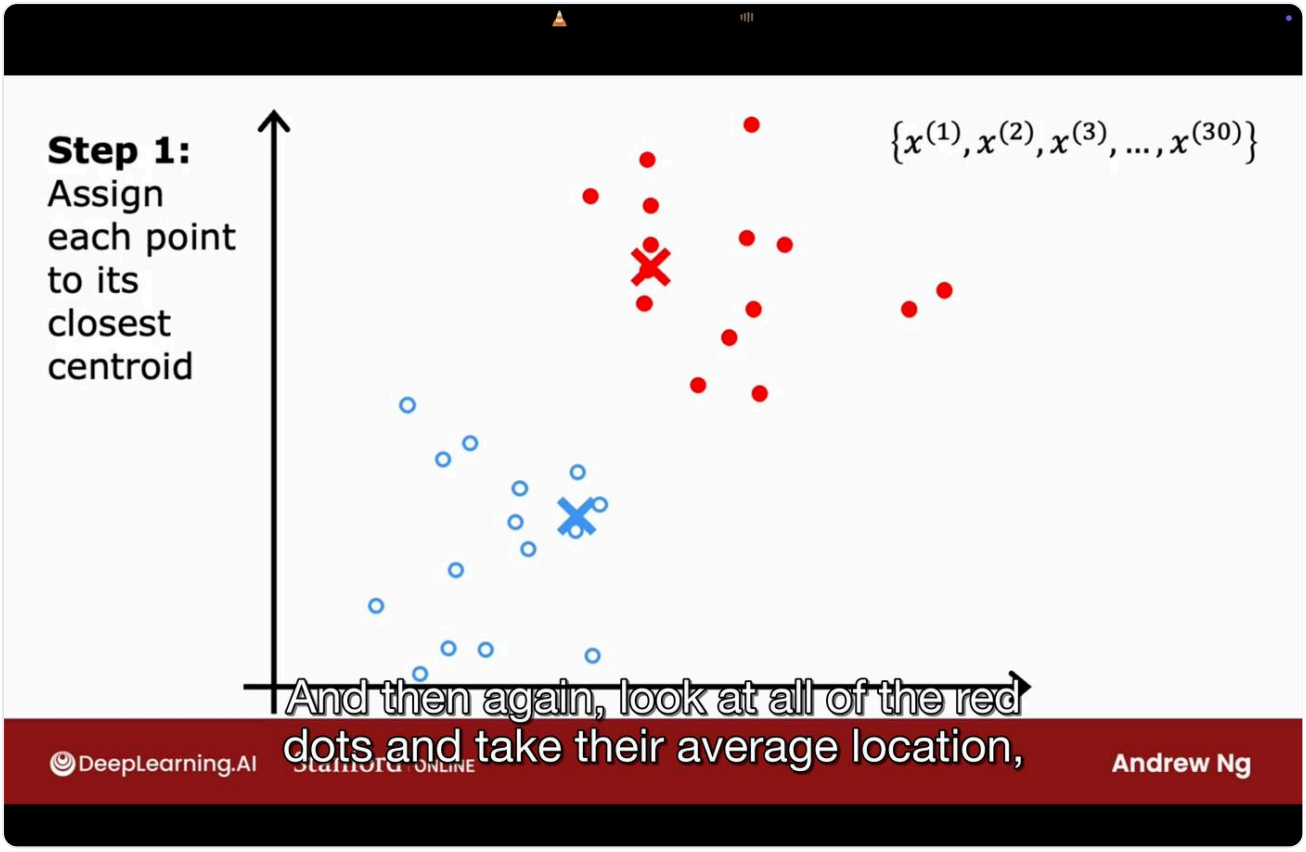


FIGURE 3.21

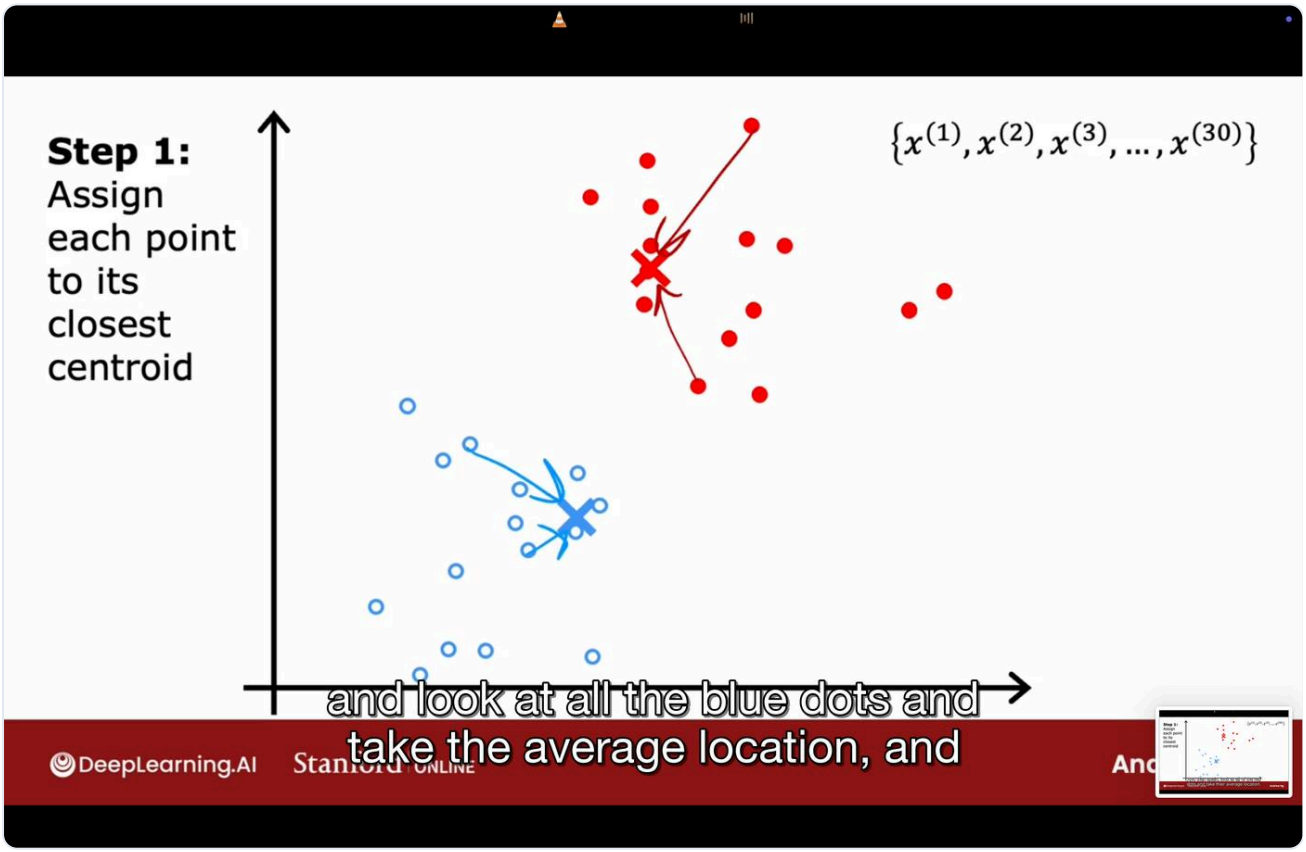


FIGURE 3.22

FIGURE 3.23

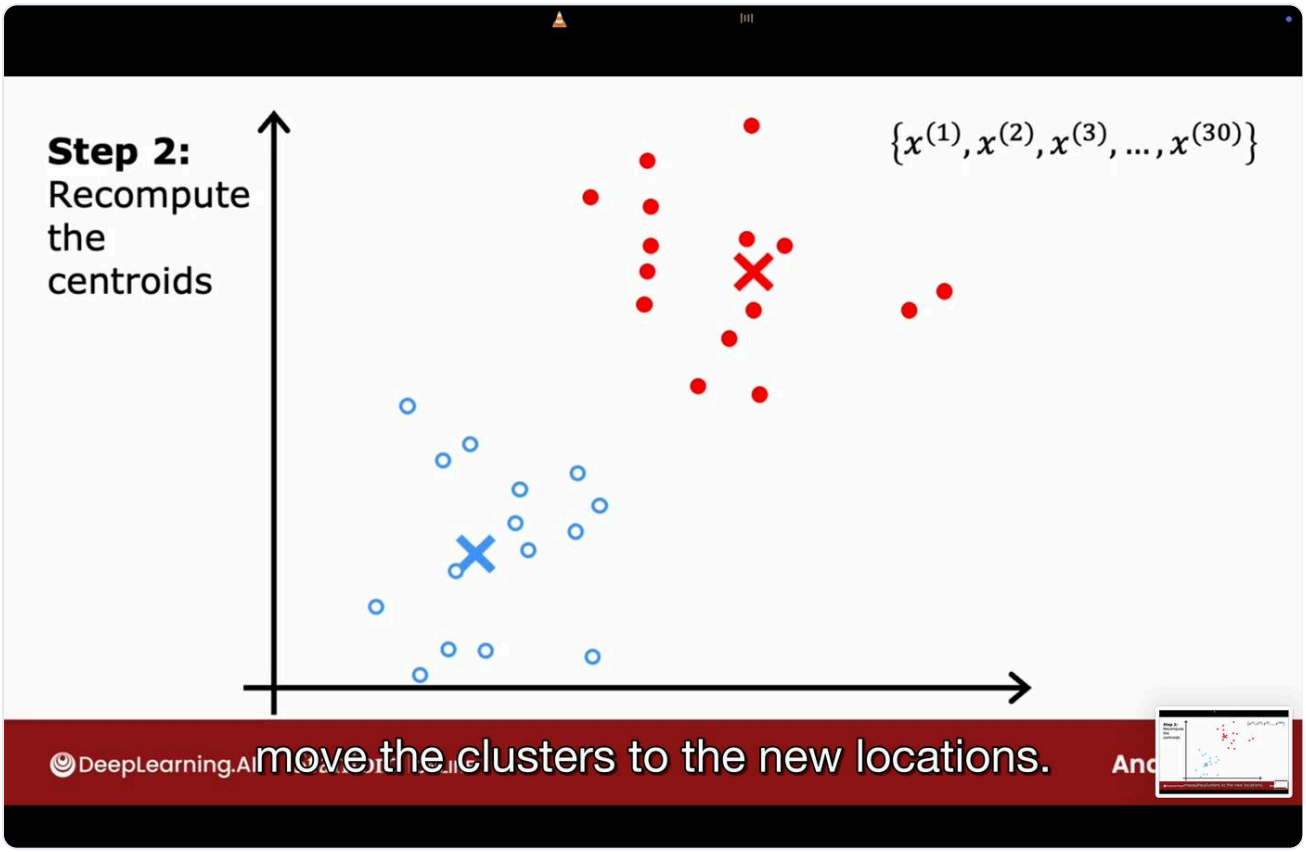


FIGURE 3.24

And then again, look at all of the red dots and take their average location, and look at all the blue dots and take the average location, and move the clusters to the new locations. And it turns out that if you were to keep on repeating these two steps, that is look at each point and assign it to the nearest cluster centroid and then also move each cluster centroid to the mean of all the points with the same color.

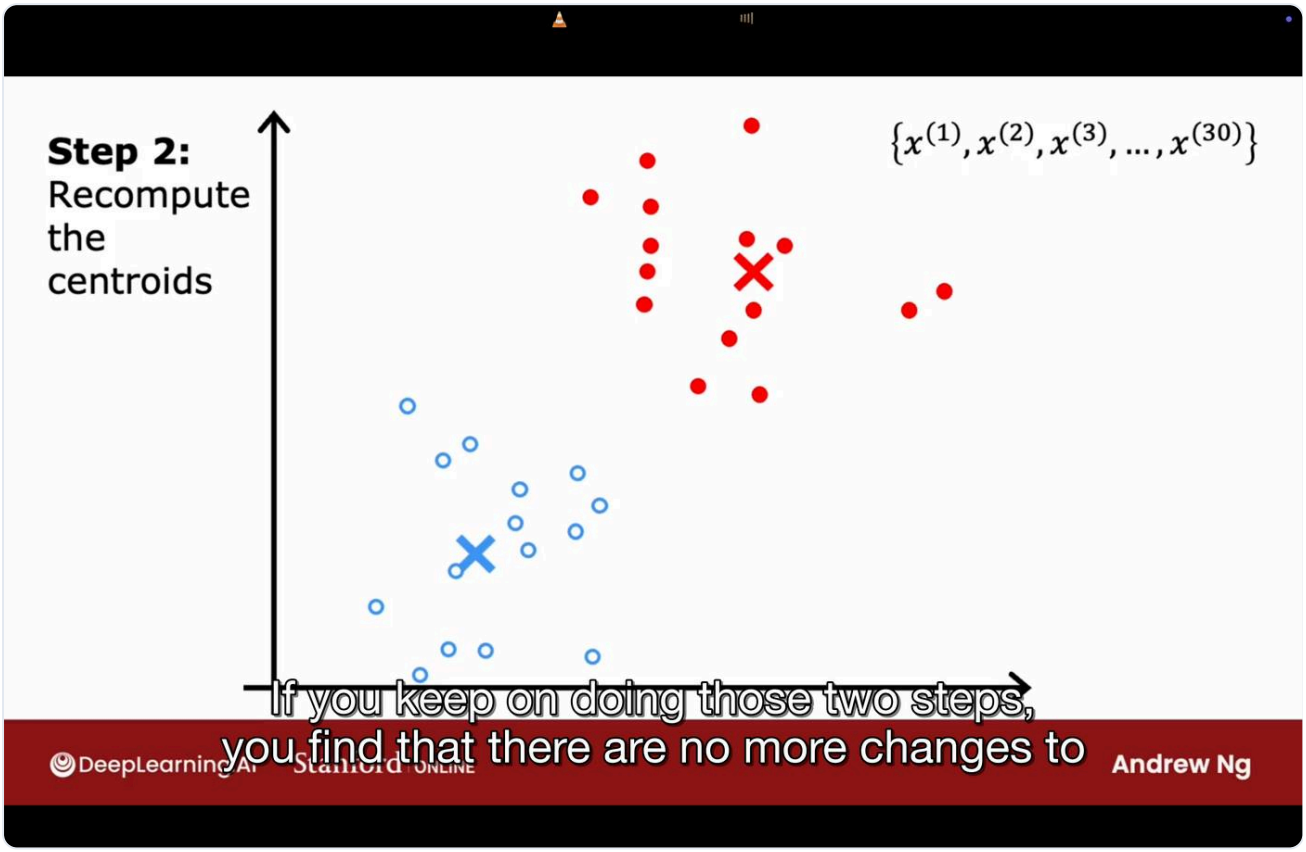


FIGURE 3.25

If you keep on doing those two steps, you find that there are no more changes to the colors of the points or to the locations of the clusters centroids. And so this means that at this point the K-means clustering algorithm has converged. Because applying those two steps over and over, results in no further changes to either the assignment to point to the centroids or the location of the cluster centroids. In this example, it looks like K-means has done a pretty good job.

FIGURE 3.26

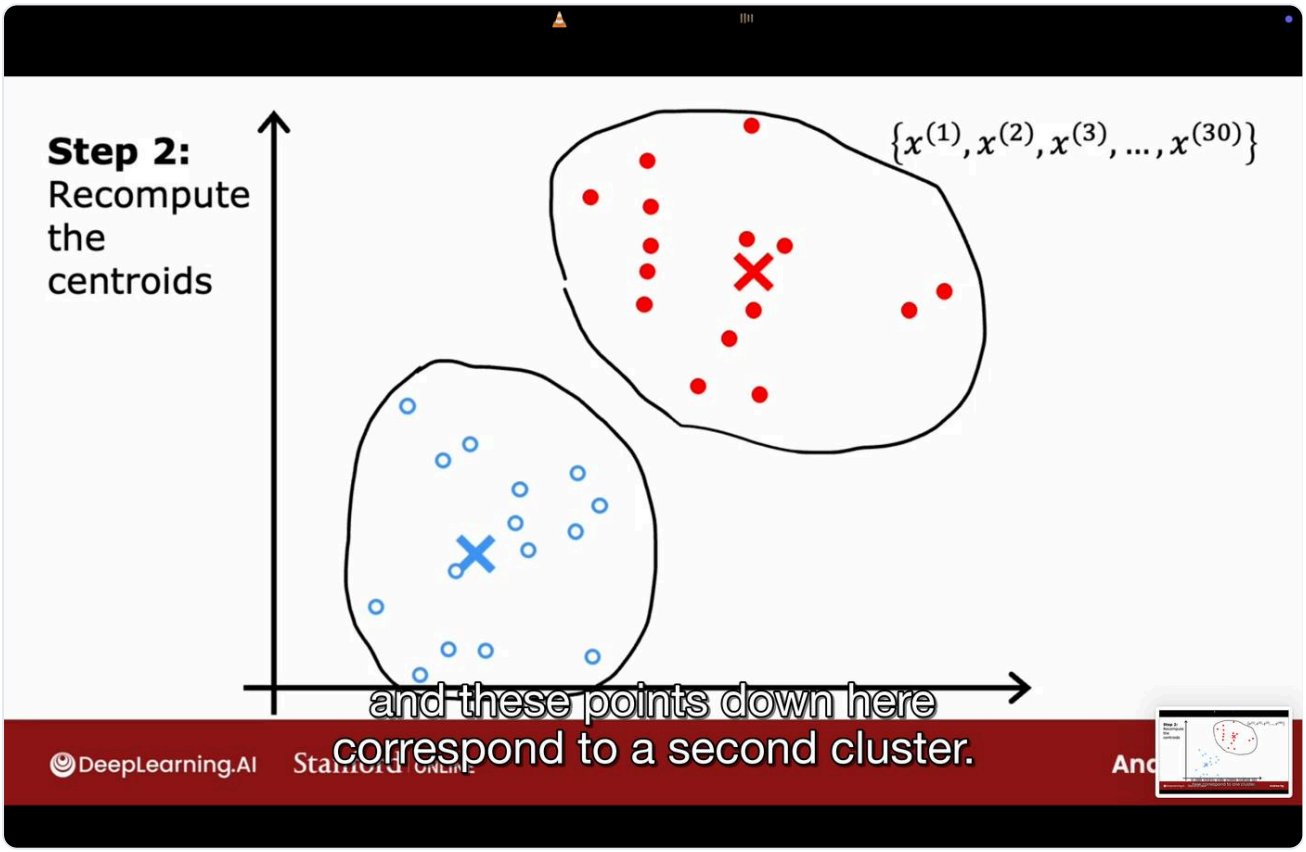


FIGURE 3.27

It has found that these points up here correspond to one cluster, and these points down here correspond to a second cluster. So now you've seen an illustration of how K-means works. The two key steps are, assign every point to the cluster centroid, depending on what cluster centroid is nearest to. And second move each cluster centroid to the average or the mean of all the points that were assigned to it. In the next video, we'll look at how to formalize this and write out the algorithm that does what you just saw in this video. Let's go on to the next video.

LECTURE 4

K-means for Clusters That Are Not Well

In the last video, you saw an illustration of the k-means algorithm running. Now let's write out the K-means algorithm in detail so that you'd be able to implement it for yourself. Here's the K-means algorithm. The first step is to randomly initialize K cluster centroids, Mu 1 Mu 2, through Mu k. In the example that we had, this corresponded to when we randomly chose a location for the red cross and for the blue cross corresponding to the two cluster centroids.

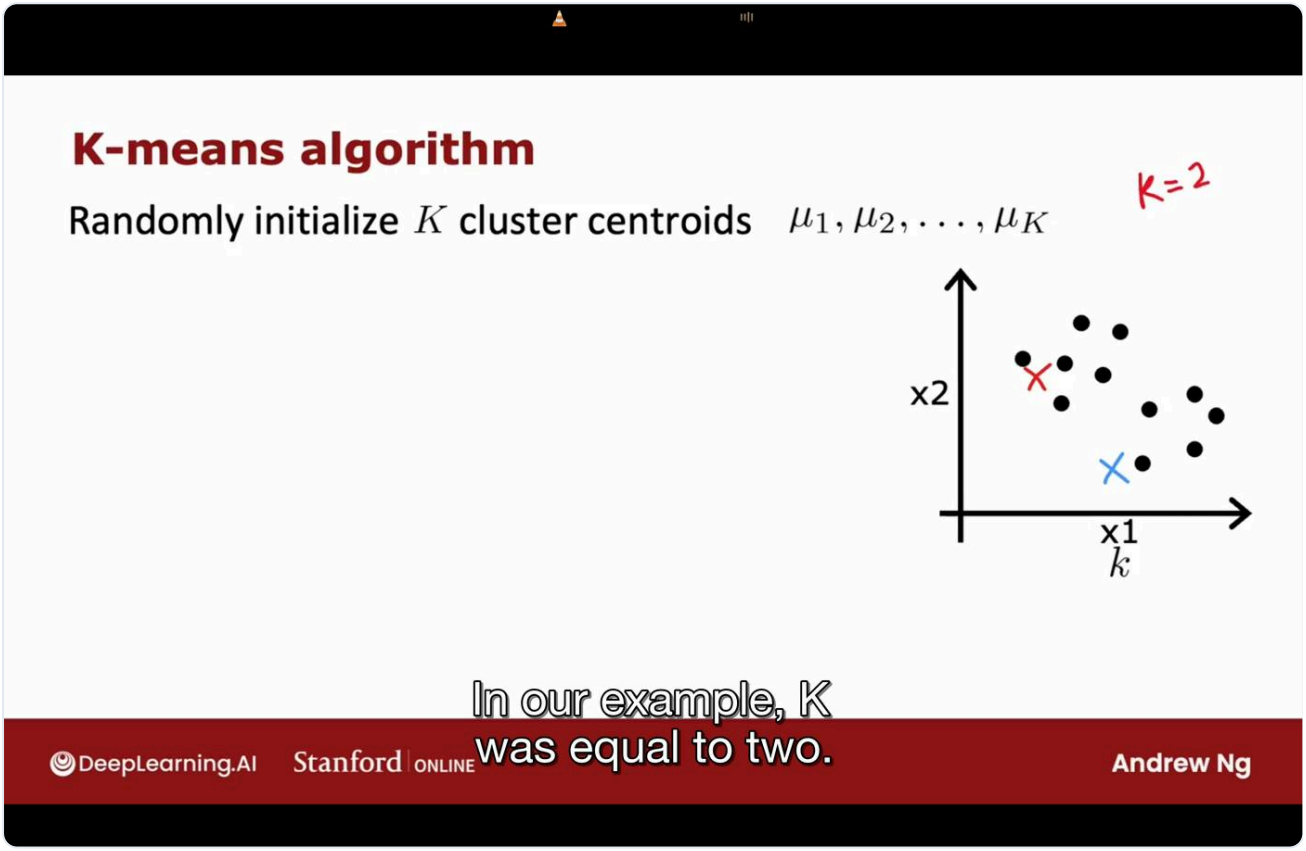


FIGURE 4.1

In our example, K was equal to two. If the red cross was cluster centroid one and the blue cross was cluster centroid two. These are just two indices to denote the first and the second cluster. Then the red cross would be the location of Mu 1 and the blue cross would be the location of Mu 2. Just to be clear, Mu 1 and Mu 2 are vectors which have the same dimension as your training examples, X1 through say X30, in our example. All of these are lists of two numbers or they're two-dimensional vectors or whatever dimension the training data had. We had n equals two features for each of the training examples then Mu 1 and Mu 2 will also be two-dimensional vectors, meaning vectors with two numbers in them.

Having randomly initialized the K cluster centroids, K-means will then repeatedly carry out the two steps that you saw in the last video. The first step is to assign points to clusters, centroids, meaning color, each of the points, either red or blue, corresponding to assigning them to cluster centroids one or two when K is equal to two. Rinse it out enough. That means that we're going to,

for I equals one through m for all m training examples, we're going to set c^i to be equal to the index, which can be anything from one to K of the cluster centroid closest to the training example x^i. Mathematically you can write this out as computing the distance between x^i and Mu k.

In math, the distance between two points is often written like this.

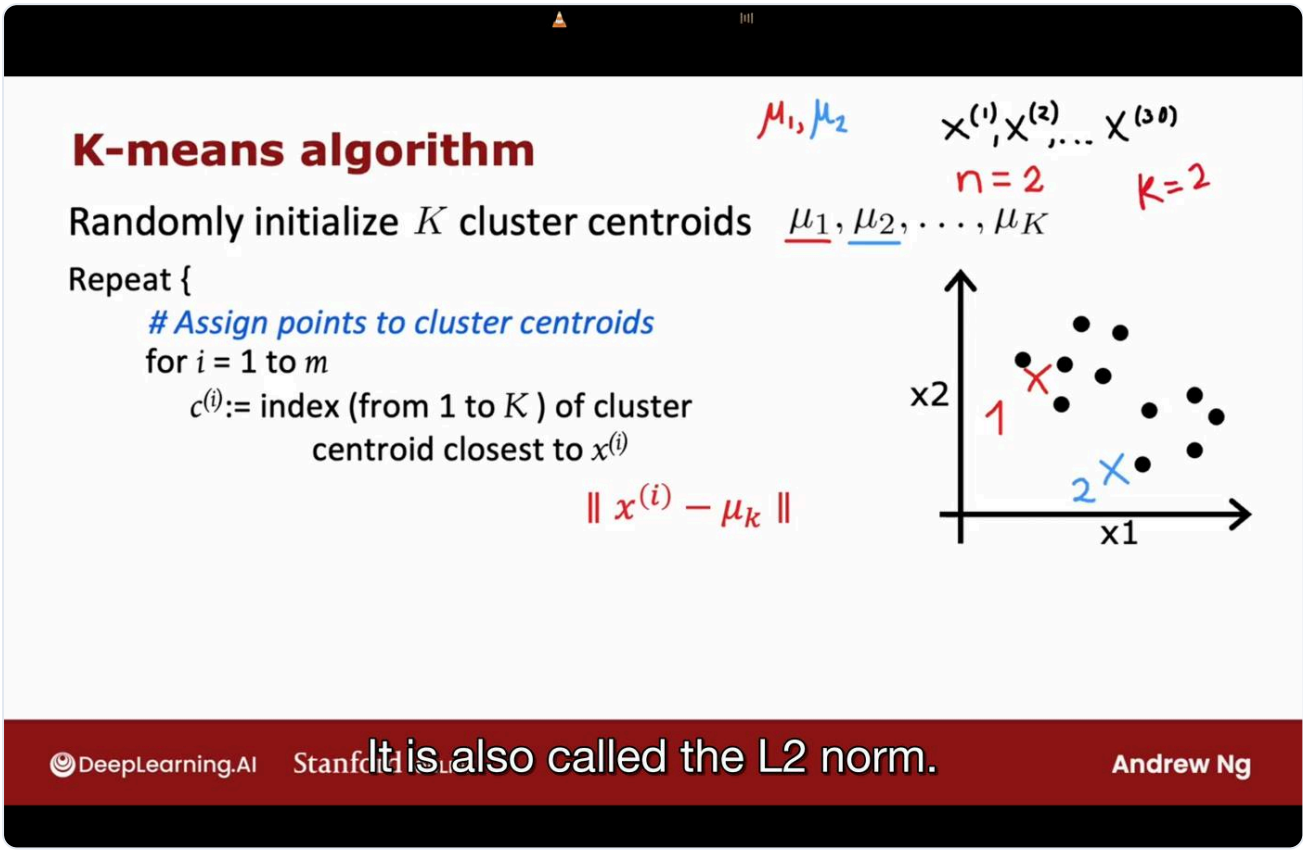


FIGURE 4.2

It is also called the L2 norm. What you want to find is the value of k that minimizes this, because that corresponds to the cluster centroid Mu k that is closest to the training example x^i.

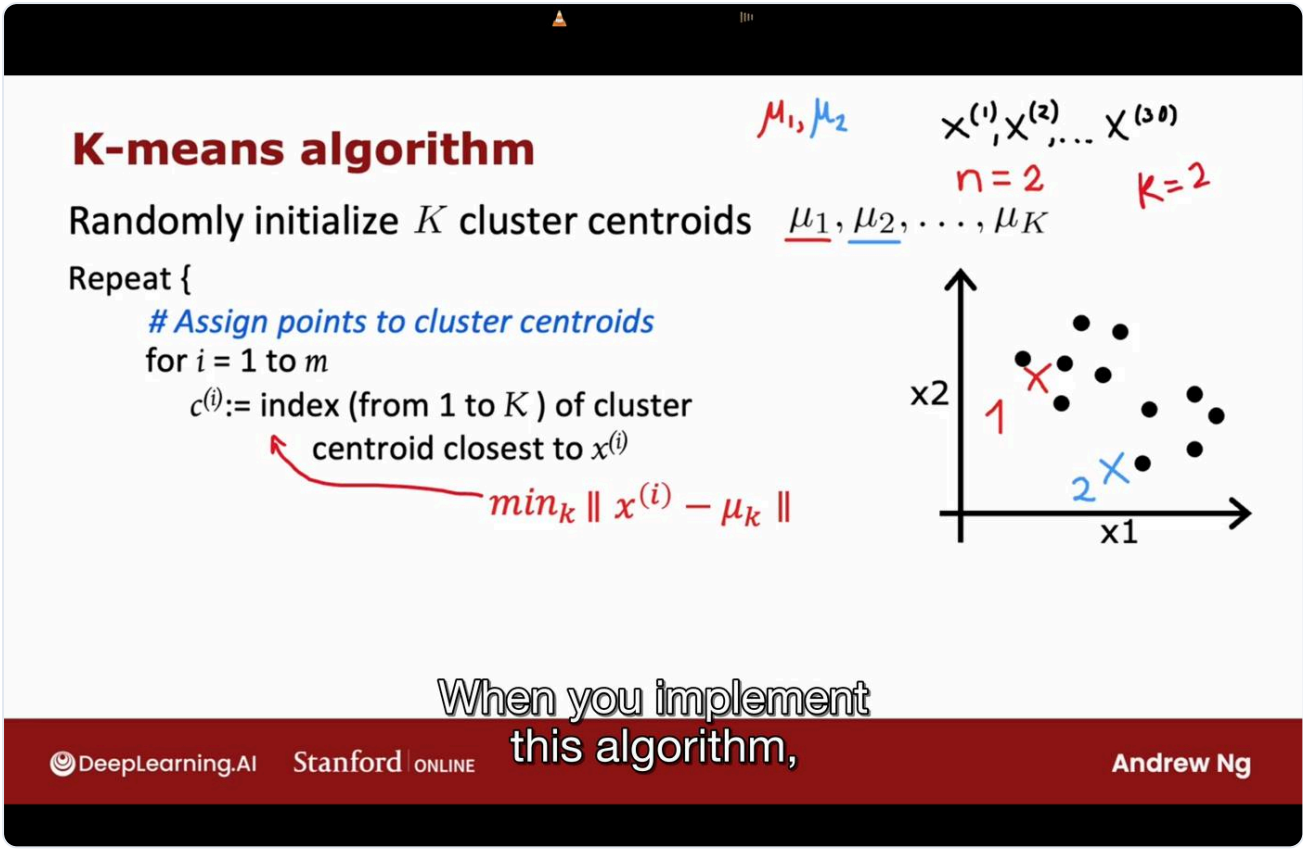


FIGURE 4.3

Then the value of k that minimizes this is what gets set to c^i.

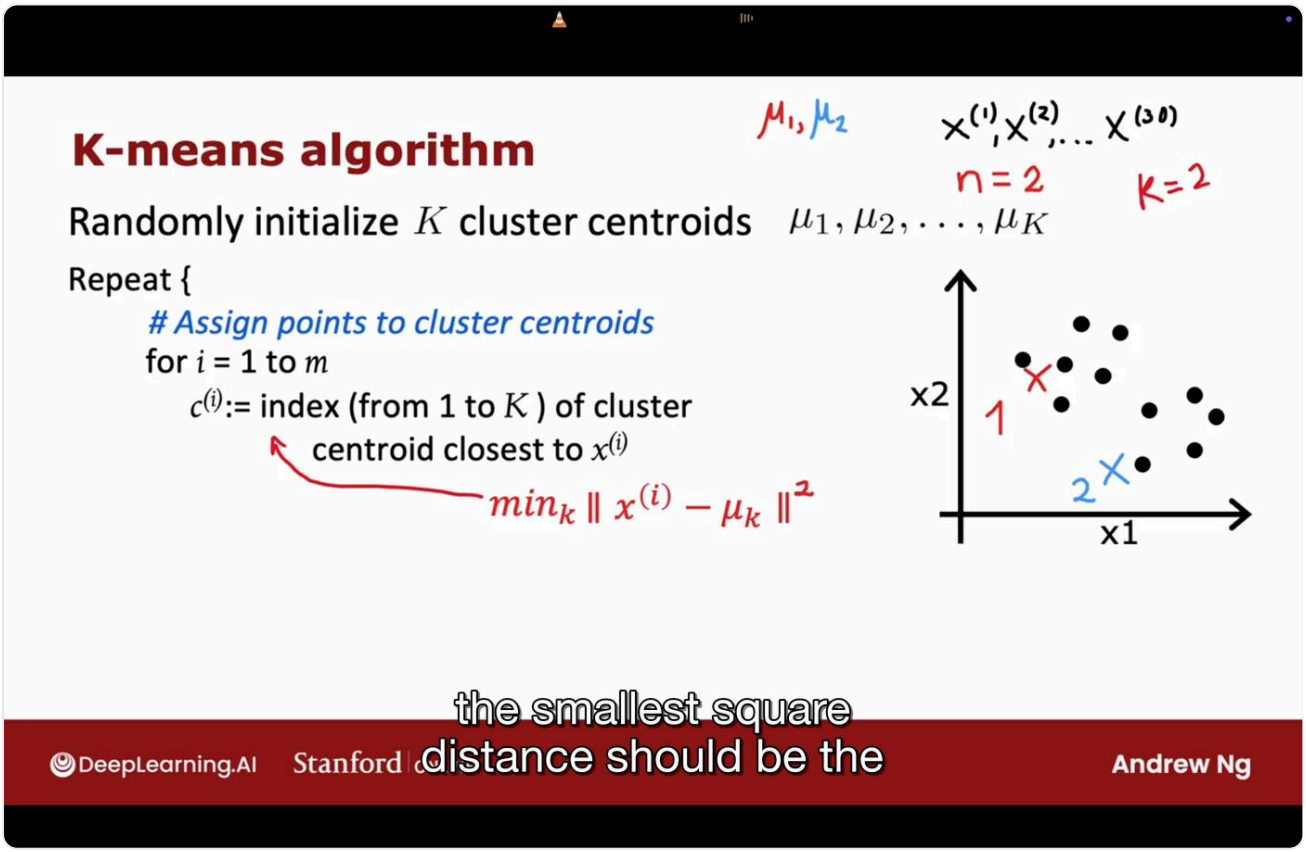


FIGURE 4.4

When you implement this algorithm, you find that it's actually a little bit more convenient to minimize the squared distance because the cluster centroid with the smallest square distance should be the same as the cluster centroid with the smallest distance. When you look at this week's optional labs and practice labs, you see how to implement this in code for yourself. As a concrete example, this point up here is closer to the red or two cluster centroids 1. If this was training example x^1, we will set c^1 to be equal to 1. Whereas this point over here, if this was the 12th training example, this is closer to the second cluster centroids the blue one.

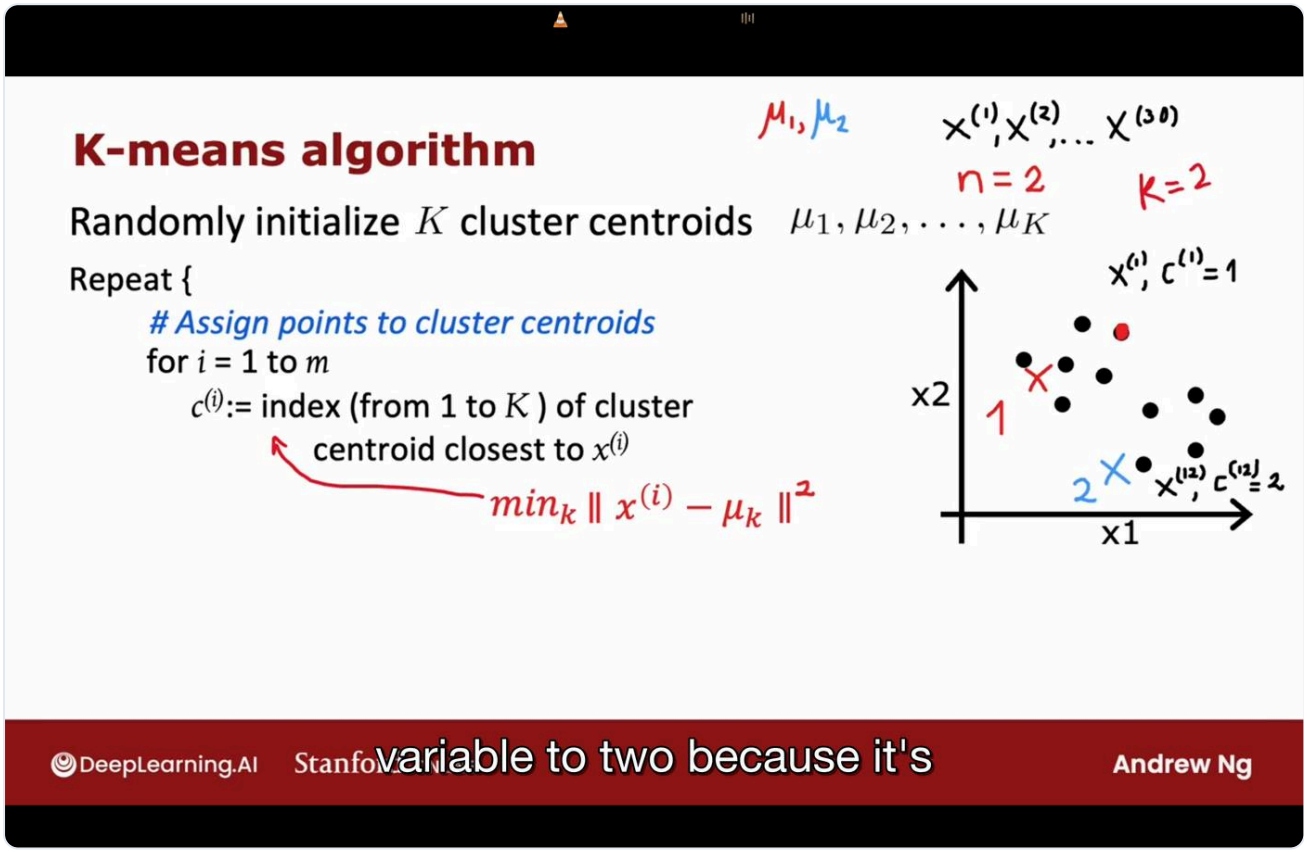


FIGURE 4.5

We will set this, the corresponding cluster assignment variable to two because it's closer to cluster centroid 2. That's the first step of the K-means algorithm, assign points to cluster centroids. The second step is to move the cluster centroids. What that means is for lowercase k equals 1 to capital K, the number of clusters. We're going to set the cluster centroid location to be updated to be the average or the mean of the points assigned to that cluster k.

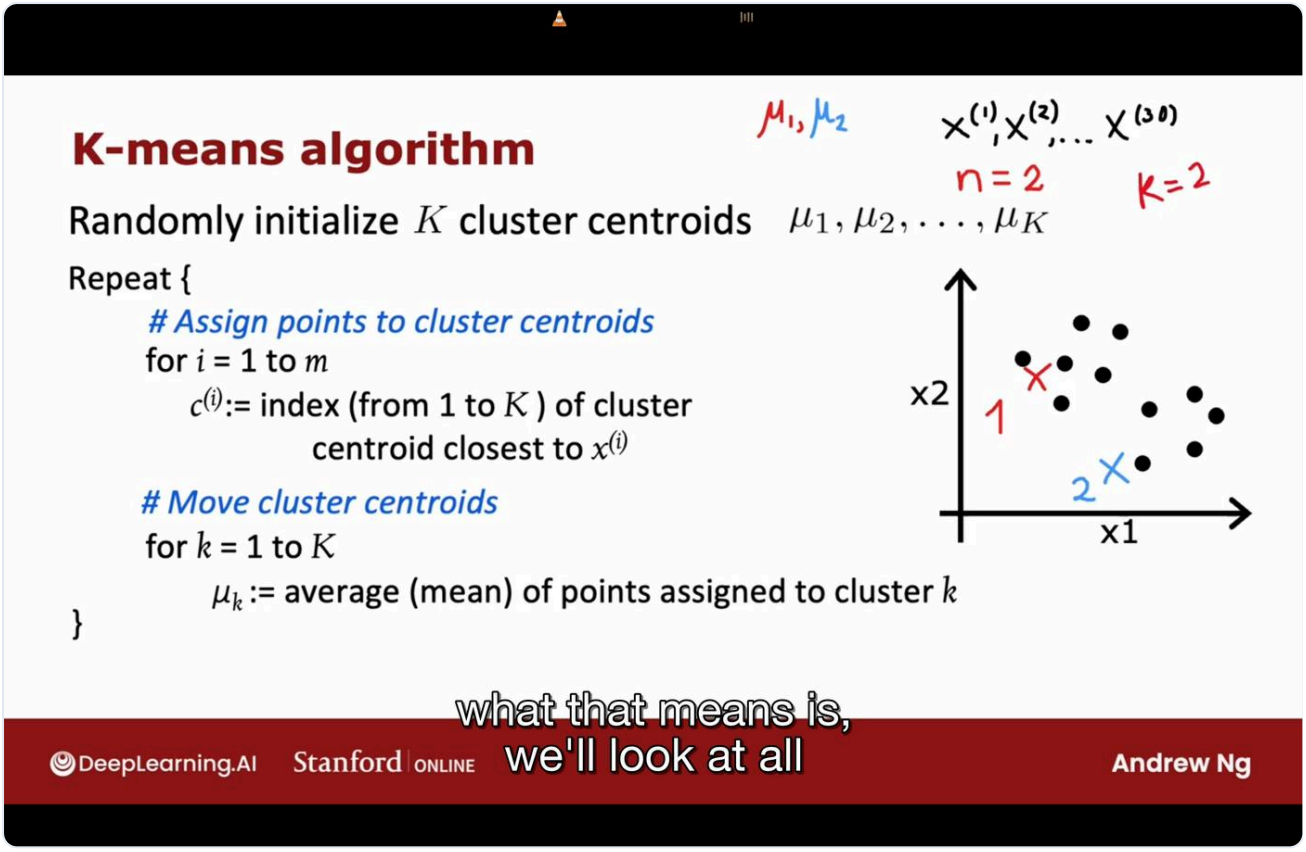


FIGURE 4.6

Concretely, what that means is, we'll look at all of these red points, say, and look at their position on the horizontal axis and look at the value of the first feature x^1, and average that out. Compute the average value on the vertical axis as well.

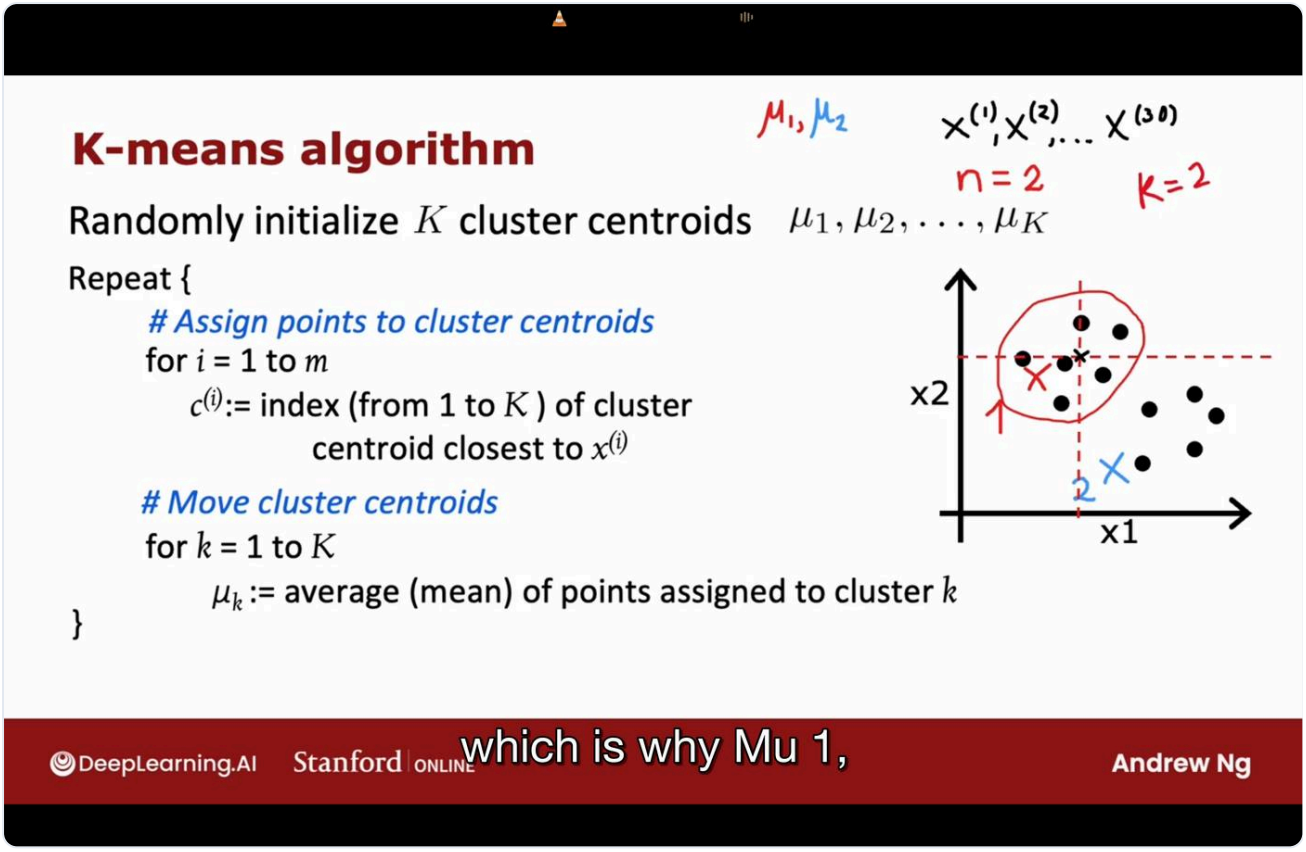


FIGURE 4.7

After computing those two averages, you find that the mean is here, which is why Mu 1, that is the location that the red cluster centroid gets updated as follows. Similarly, we will look at all of the points that were colored blue, that is, with c^i equals 2 and computes the average of the value on the horizontal axis, the average of their feature x1. Compute the average of the feature x2.

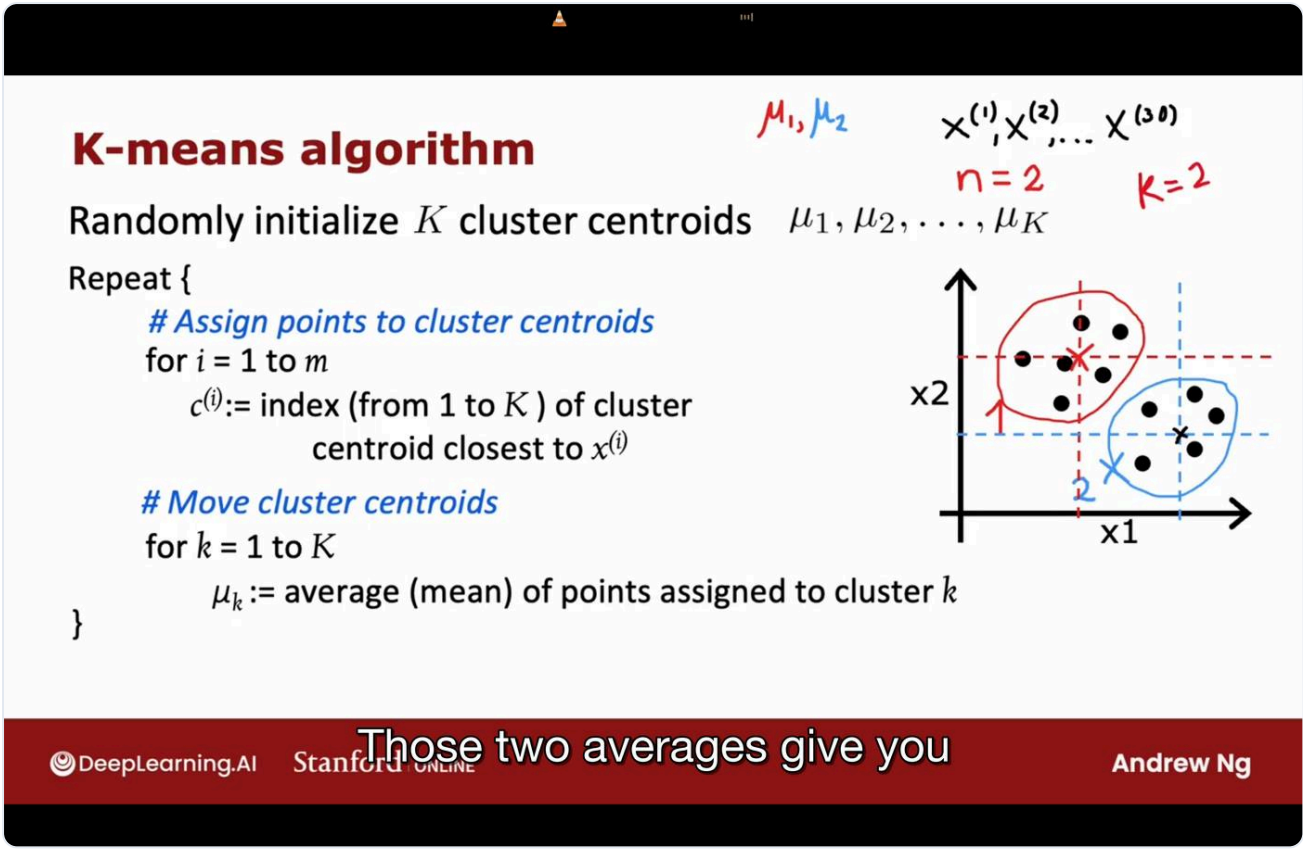


FIGURE 4.8

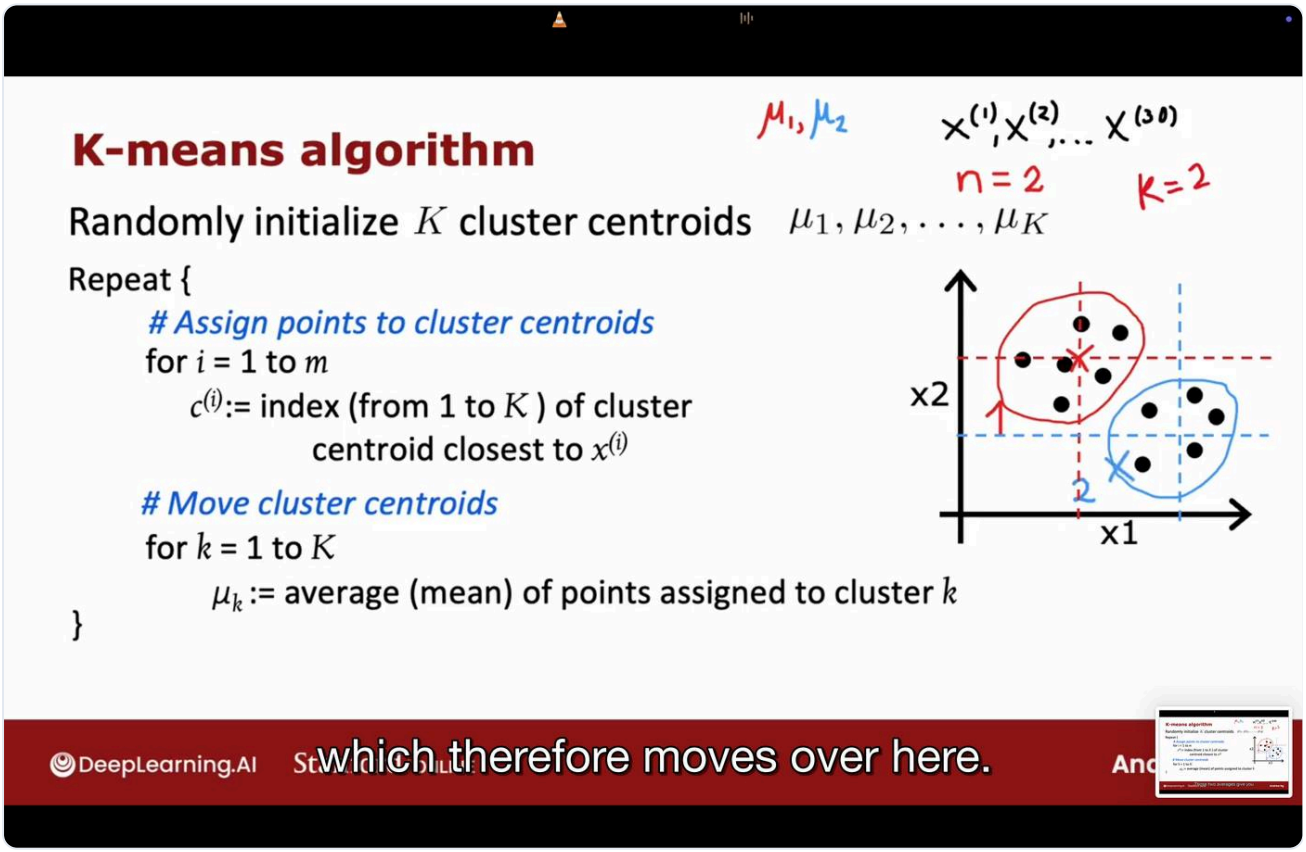


FIGURE 4.9

Those two averages give you the new location of the blue cluster centroid, which therefore moves over here.

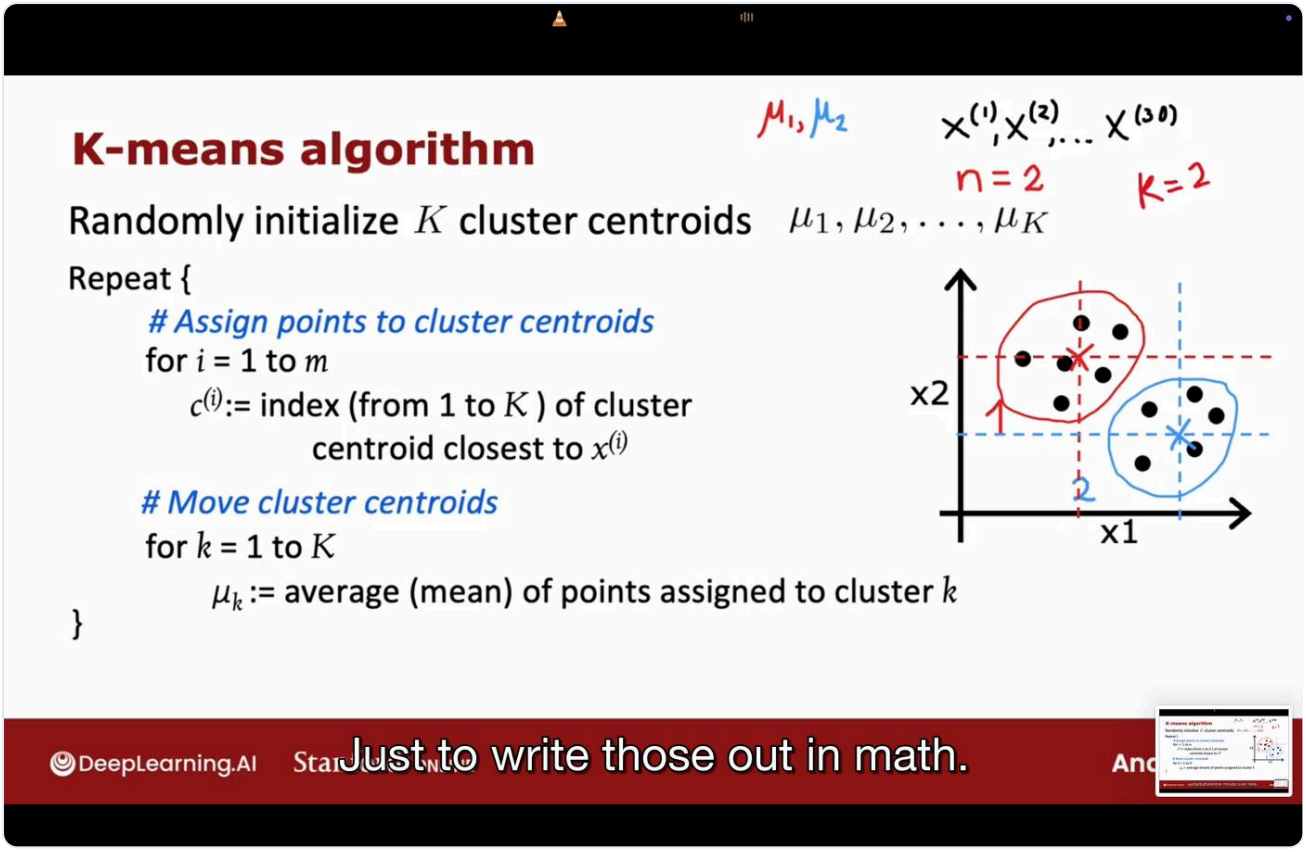


FIGURE 4.10

Just to write those out in math. If the first cluster has assigned to it training examples 1,5,6,10. Just as an example. Then what that means is you will compute the average this way. Notice that x^1, x^5, x^6, and x^10 are training examples.

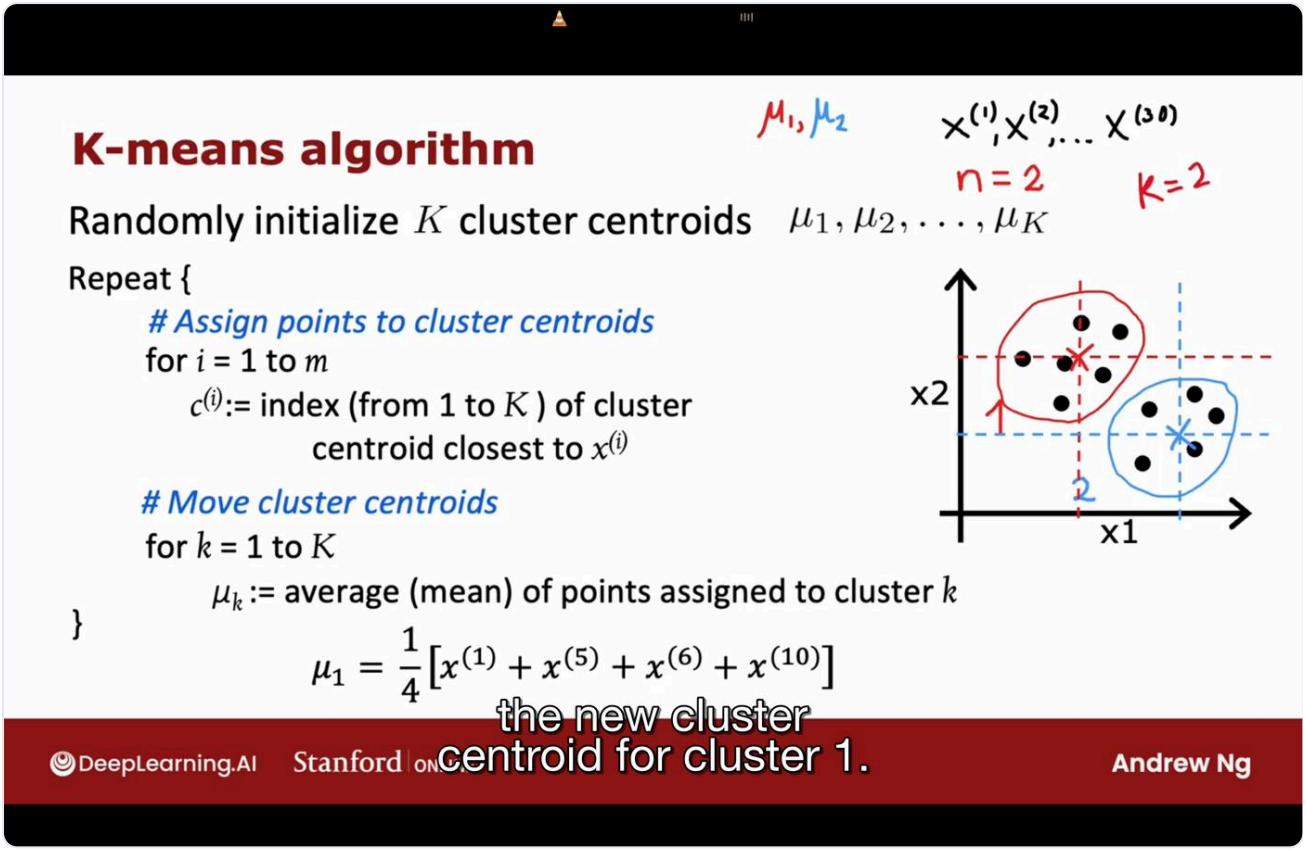


FIGURE 4.11

Four training examples, so we divide by 4 and this gives you the new location of Mu1, the new cluster centroid for cluster 1.

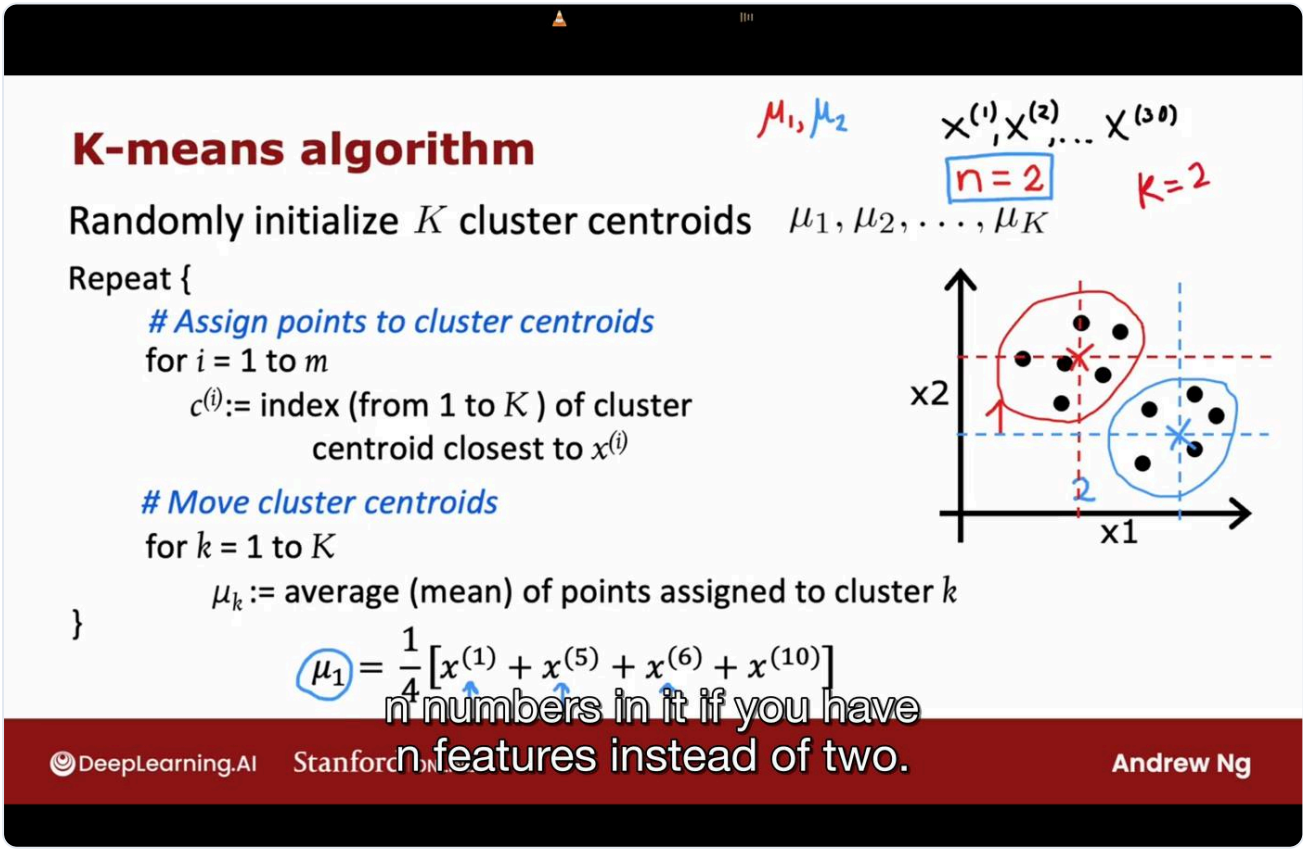


FIGURE 4.12

To be clear, each of these x values are vectors with two numbers in them, or n numbers in them if you have n features, and so Mu will also have two numbers in it or n numbers in it if you have n features instead of two. Now, there is one corner case of this algorithm, which is what happens if a cluster has zero training examples assigned to it. In that case, the second step, Mu k, would be trying to compute the average of zero points. That's not well-defined. If that ever happens, the most common thing to do is to just eliminate that cluster. You end up with K minus 1 clusters.

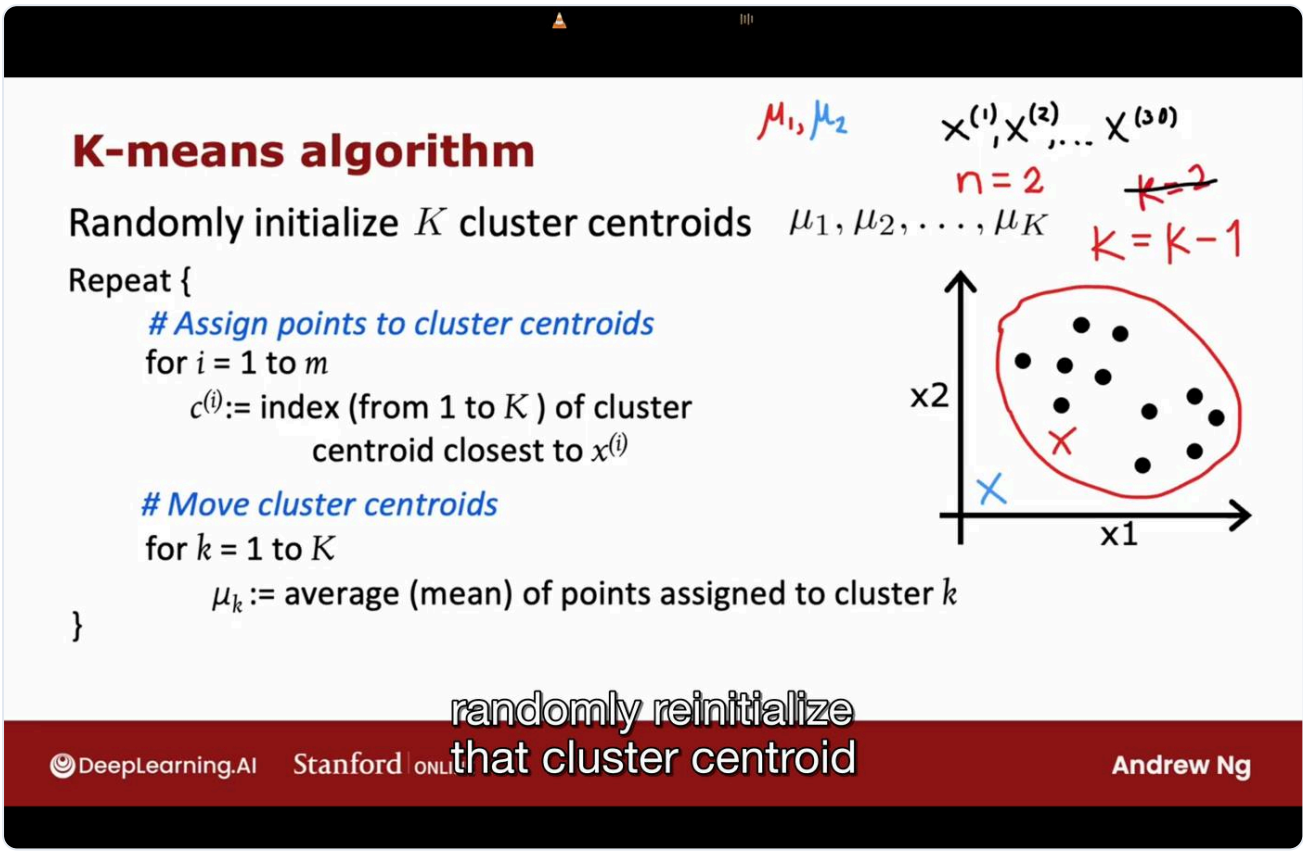


FIGURE 4.13

Or if you really, really need K clusters an alternative would be to just randomly reinitialize that cluster centroid and hope that it gets assigned at least some points next time round. But it's actually more common when running K-means to just eliminate a cluster if no points are assigned to it.

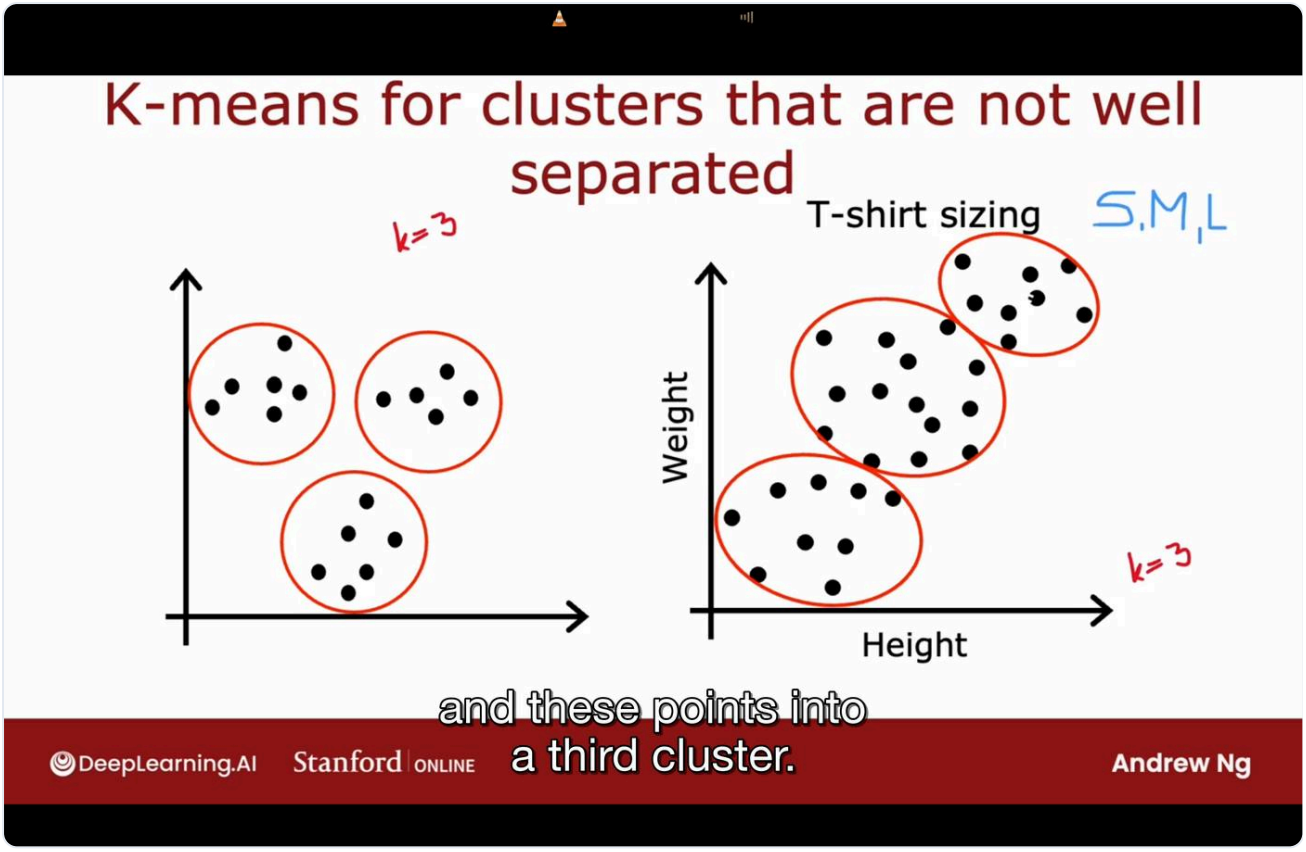


FIGURE 4.14 | K-means for Clusters That Are Not Well

Even though I've mainly been describing K-means for clusters that are well separated. Clusters that may look like this. Where if you asked her to find three clusters, hopefully they will find these three distinct clusters. It turns out that K-means is also frequently applied to data sets where the clusters are not that well separated. For example, if you are a designer and manufacturer of cool t-shirts, and you want to decide, how do I size my small, medium, and large t-shirts. How small should a small be, how large should a large be, and what should a medium-size t-shirt really be? One thing you might do is collect data of people likely to buy your t-shirts based on their heights and weights.

You find that the height and weight of people tend to vary continuously on the spectrum without some very clear clusters. Nonetheless, if you were to run K-means with say, three clusters centroids, you might find that K-means would group these points into one cluster, these points into a second cluster, and these points into a third cluster. If you're trying to decide exactly how to size your small, medium, and large t-shirts, you might then choose the dimensions of your small t-shirt to try to make it fit these individuals well.

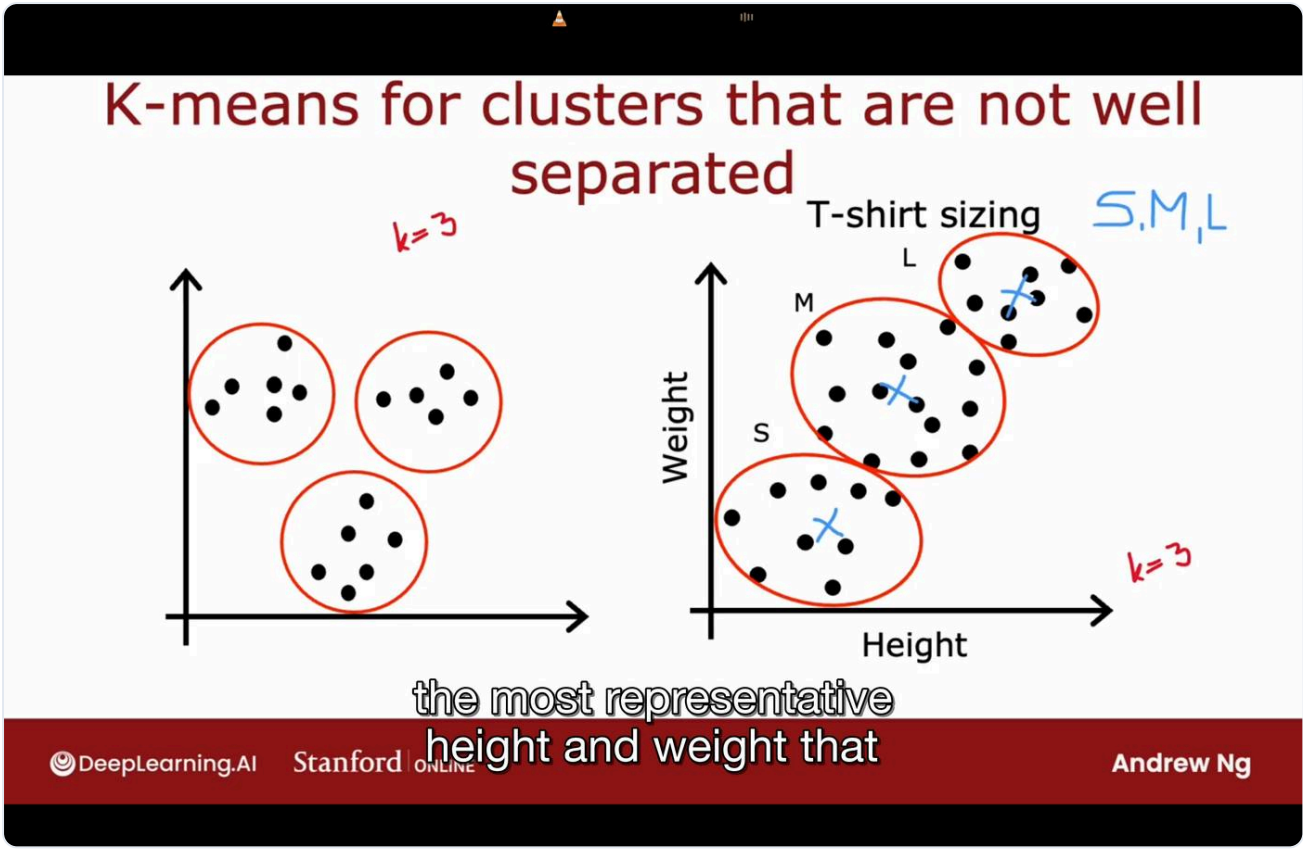


FIGURE 4.15 | K-means for Clusters That Are Not Well

The medium-size t-shirt to try to fit these individuals well, and the large t-shirt to try to fit these individuals well with potentially the cluster centroids giving you a sense of what is the most representative height and weight that you will want your three t-shirt sizes to fit. This is an example of K-means working just fine and giving a useful results even if the data does not lie in well-separated groups or clusters. That was the K-means clustering algorithm. Assign cluster centroids randomly and then repeatedly assign points to cluster centroids and move the cluster centroids. But what this algorithm really doing and do we think this algorithm will converge or they just keep on running forever and never converge.

To gain deeper intuition about the K-means algorithm and also see why we might hope this algorithm does converge, let's go on to the next video where you see that K-means is actually trying to optimize a specific cost function. Let's take a look at that in the next video.

LECTURE 5

Cost Function and Cluster Centroid

In the earlier courses, courses one and two of the specialization, you saw a lot of supervised learning algorithms as taking training set posing a cost function. And then using grading descent or some other algorithms to optimize that cost function. It turns out that the K-means algorithm that you saw in the last video is also optimizing a specific cost function. Although the optimization algorithm that it uses to optimize that is not gradient descent is actually the algorithm that you already saw in the last video. Let's take a look at what all this means. Let's take a look at what is the cost function for K-means, to get started as a reminder this is a notation we've been using whereas CI is the index of the cluster.

So CI is some number from one Su K of the index of the cluster to which training example XI is currently assigned and new K is the location of cluster centroid k. Let me introduce one more piece of notation, which is when lower case K equals CI. So mu subscript CI is the cluster centroid of the cluster to which example XI has been assigned. So for example, if I were to look at some training example C train example 10 and I were to ask What's the location of the clustering centroids to which the 10th training example has been assigned? Well, I would then look up C10. This will give me a number from one to K. That tells me was example 10 assigned to the red or the blue or some other cluster centroid, and then mu subscript C- 10 is the location of the cluster centroid to which extent has been assigned.

So armed with this notation, let me now write out the cost function that K means turns out to be minimizing. The cost function J, which is a function of C1 through CM. These are all the assignments of points to clusters Androids as well as new one through mu capsule K. These are the locations of all the clusters centroid is defined as this expression on the right. It is the average, so one over M some from i equals to m of the squared distance between every training example XI as I goes from one through M it is a square distance between X I.

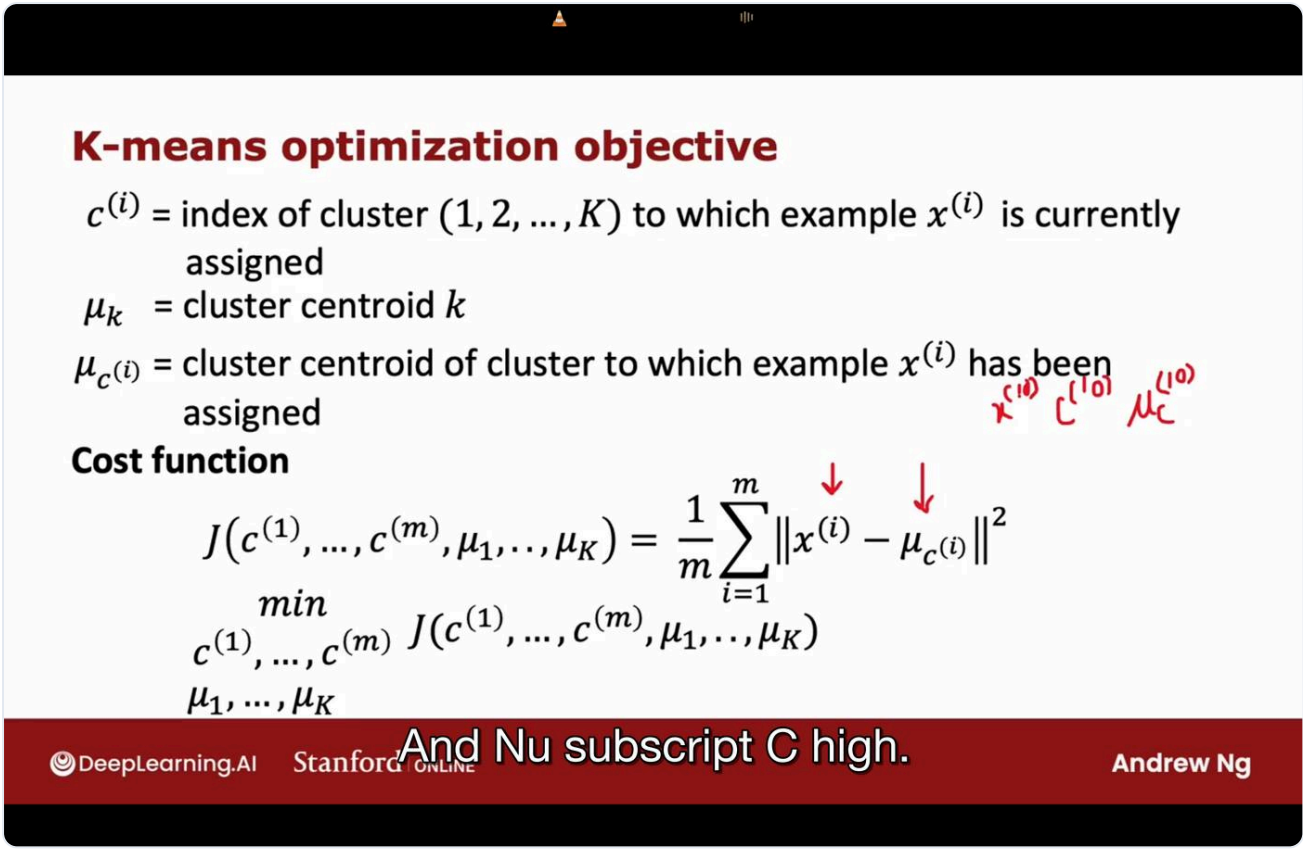


FIGURE 5.1

And Nu subscript C high. So this quantity up here, in other words, the cost function good for K is the average squared distance between every training example XI. And the location of the cluster centroid to which the training example exile has been assigned. So for this example up here we've been measuring the distance between X10 and mu subscript C10. The cluster centroid to which extent has been assigned and taking the square of that distance and that would be one of the terms over here that we're averaging over. And it turns out that what the K means algorithm is doing is trying to find assignments of points of clusters centroid as well as find locations of clusters centroid that minimizes the squared distance.

Visually, here's what you saw part way into the run of K means in the earlier video. And at this step the cost function. If you were to computer it would be to look at everyone at the blue points and measure these distances and computer square. And then also similarly look at every one of the red points and compute these distances and compute the square.

FIGURE 5.2

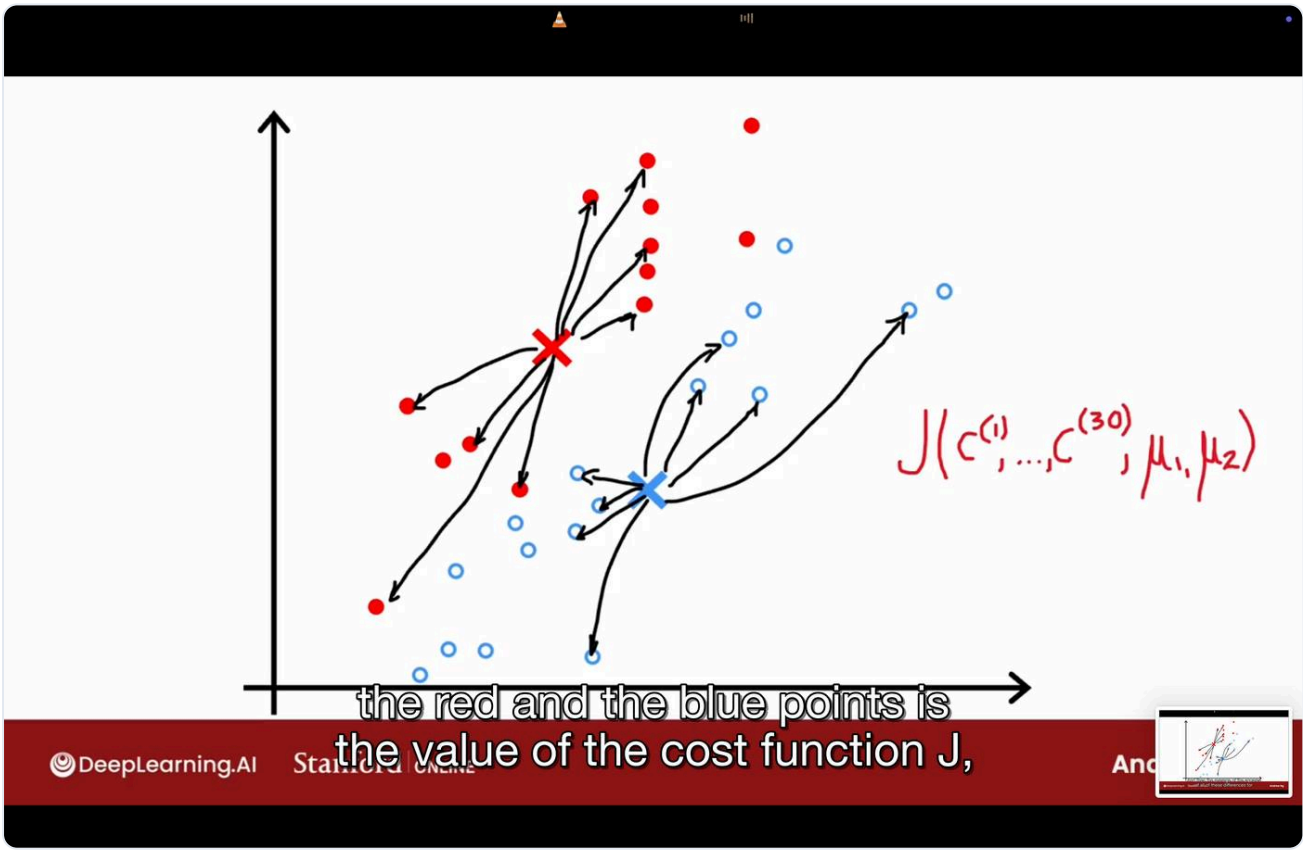


FIGURE 5.3

And then the average of the squares of all of these differences for the red and the blue points is the value of the cost function J, at this particular configuration of the parameters for K-means. And what they will do on every step is try to update the cluster assignments C1 through C30 in this example. Or update the positions of the cluster centroids, U1 and U2. In order to keep on reducing this cost function J.

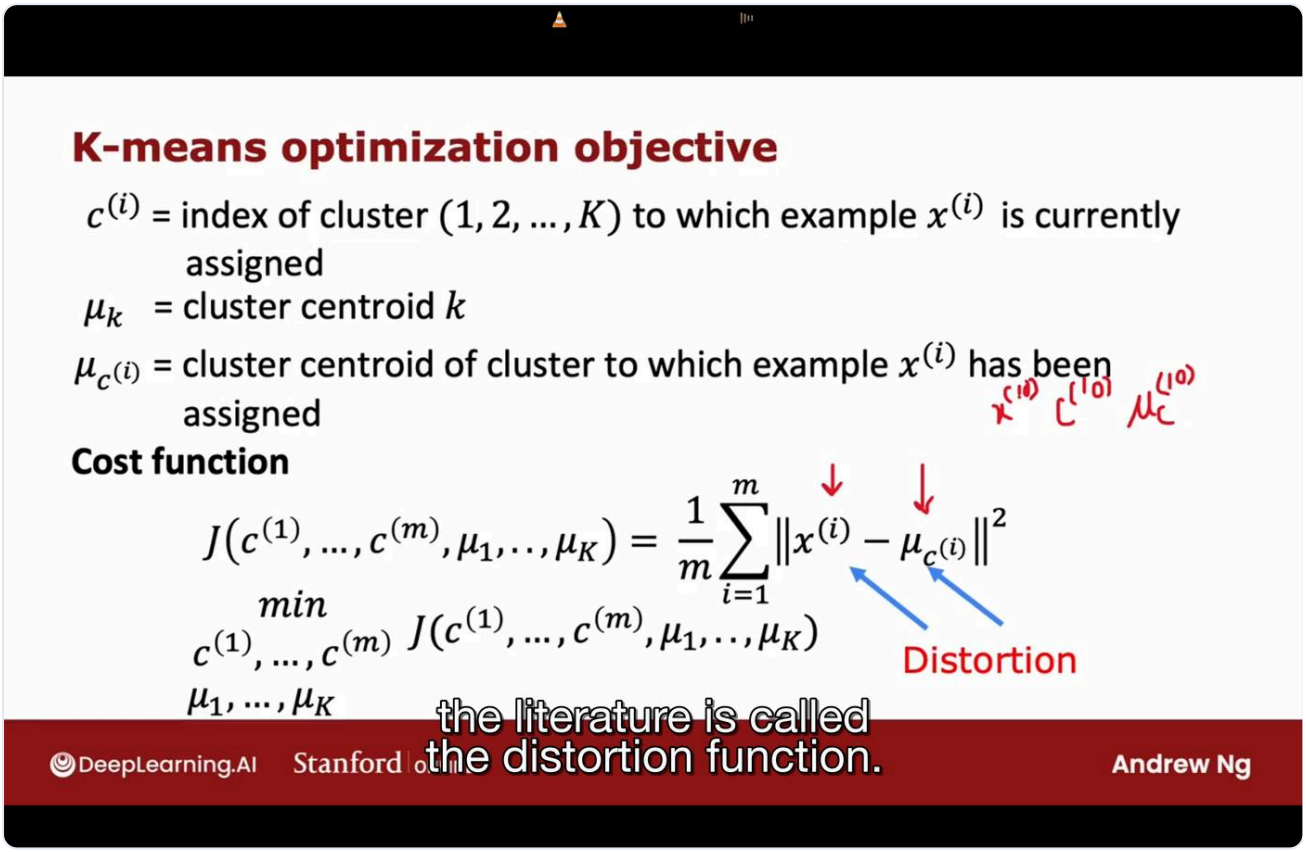


FIGURE 5.4

By the way, this cost function J also has a name in the literature is called the distortion function. I don't know that this is a great name. But if you hear someone talk about the key news algorithm and the distortion or the distortion cost function, that's just what this formula J is computing. Let's now take a deeper look at the algorithm and why the algorithm is trying to minimize this cost function J.

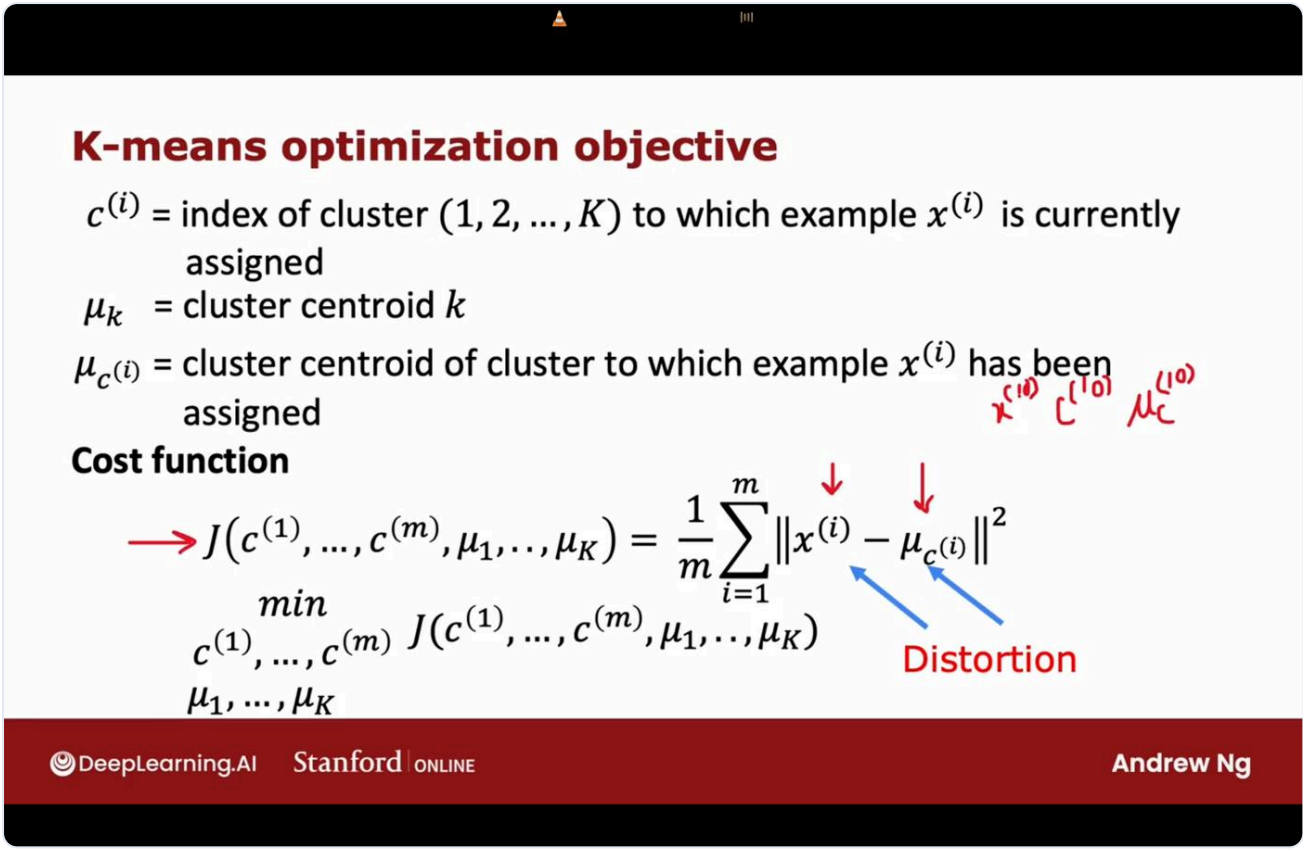


FIGURE 5.5

Or why is trying to minimize the distortion here on top of copied over the cost function from the previous slide. It turns out that the first part of K means where you assign points to cluster centroid. That turns out to be trying to update C1 through CM. To try to minimize the cost function J as much as possible while holding mu one through mu K fix. And the second step, in contrast where you move the custom centroid, it turns out that is trying to leave C1 through CM fix.

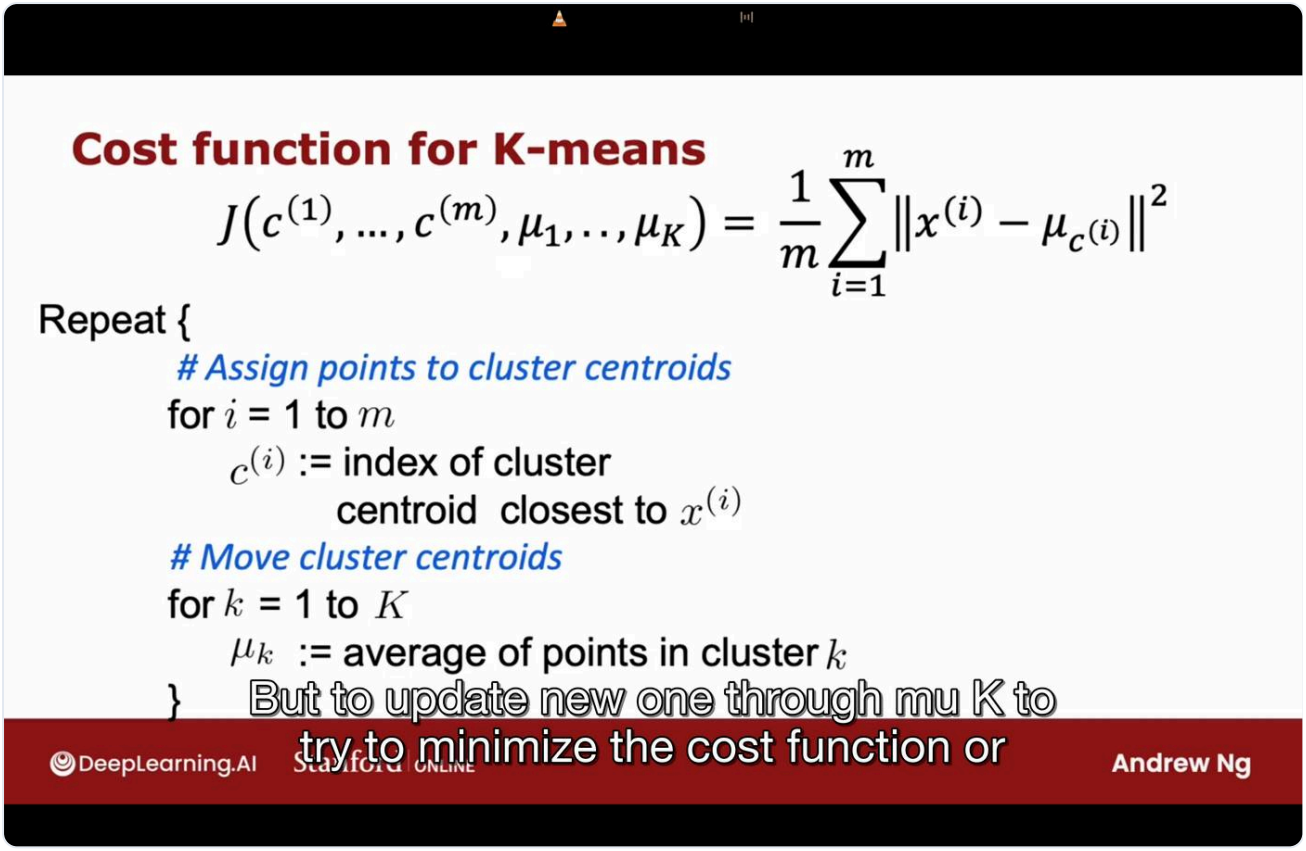


FIGURE 5.6

But to update new one through mu K to try to minimize the cost function or the distortion as much as possible. Let's take a look at why this is the case. During the first step, if you want to choose the values of C1 through CM or save a particular value of Ci to try to minimize this. Well, what would make Xi minus mu CI as small as possible? This is the distance or the square distance between a training example XI. And the location of the class is central to which has been assigned. So if you want to minimize this distance or the square distance, what you should do is assign XI to the closest cluster centroid. So to take a simplified example, if you have two clusters centroid say close to central is one and two and just a single training example, XI.

If you were to sign it to cluster centroid one, this square distance here would be this large distance, well squared.

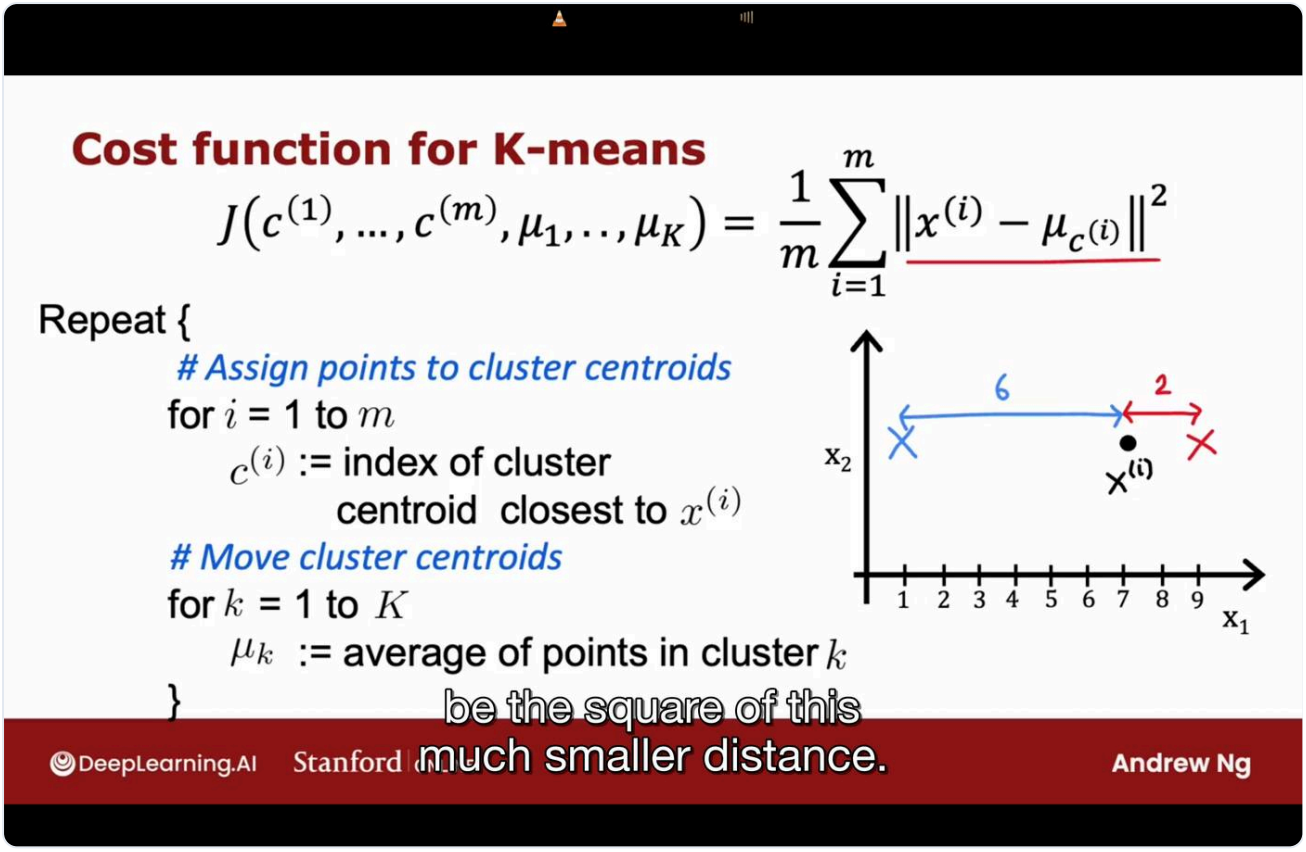


FIGURE 5.7 | Cost Function for K-means 7 M

And if you were to assign it to cluster centroid 2 then this square distance would be the square of this much smaller distance. So if you want to minimize this term, you will take X I and assign it to the closer centroid, which is exactly what the algorithm is doing up here. So that's why the step where you assign points to a cluster centroid is choosing the values for CI to try to minimize J. Without changing, we went through the mu K for now, but just choosing the values of C1 through CM to try to make these terms as small as possible. How about the second step of the K-means algorithm that is to move to clusters centroids? It turns out that choosing mu K to be average and the mean of the points assigned is the choice of these terms mu that will minimize this expression.

To take a simplified example, say you have a cluster with just two points assigned to it shown as follows. And so with the cluster centroid here, the average of the square distances would be a distance of one here squared plus this distance here, which is 9 squared. And then you take the average of these two numbers. And so that turns out to be one half of 1 plus 81, which turns out to be 41.

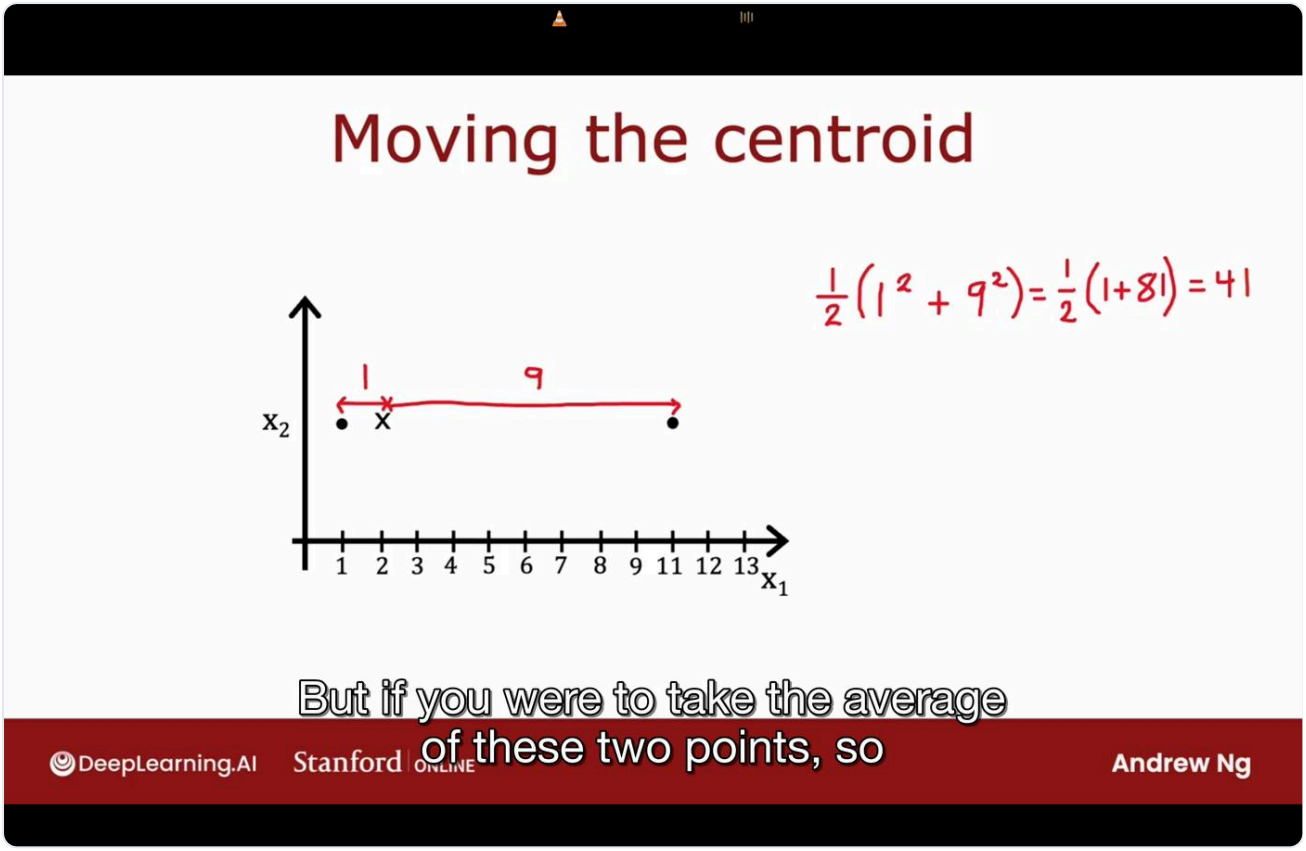


FIGURE 5.8

But if you were to take the average of these two points, so 1+ 11/2, that's equal to 6. And if you were to move the cluster centroid over here to middle than the average of these two square distances, turns out to be a distance of five and five here.

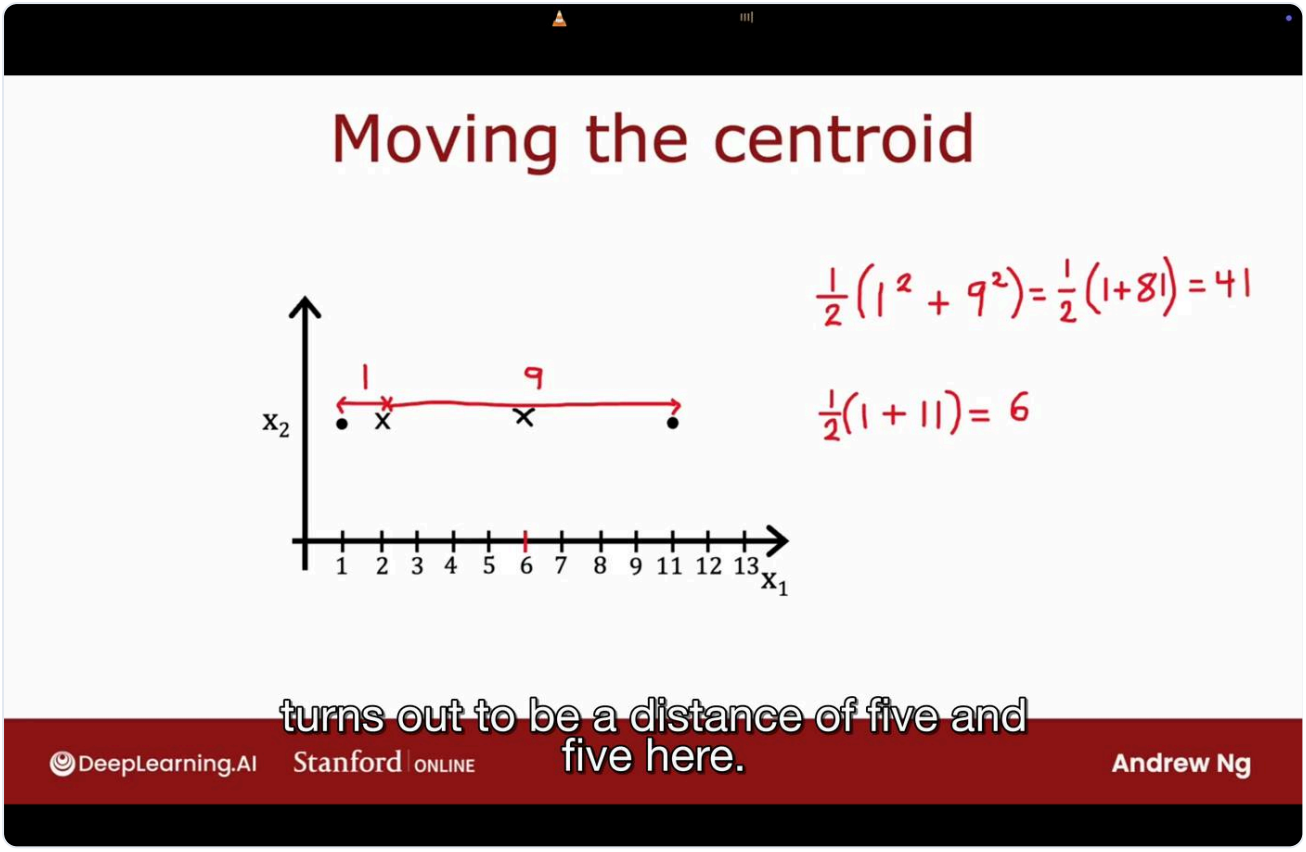


FIGURE 5.9

So you end up with one half of 5 squared plus 5 squared, which is equal to 25.

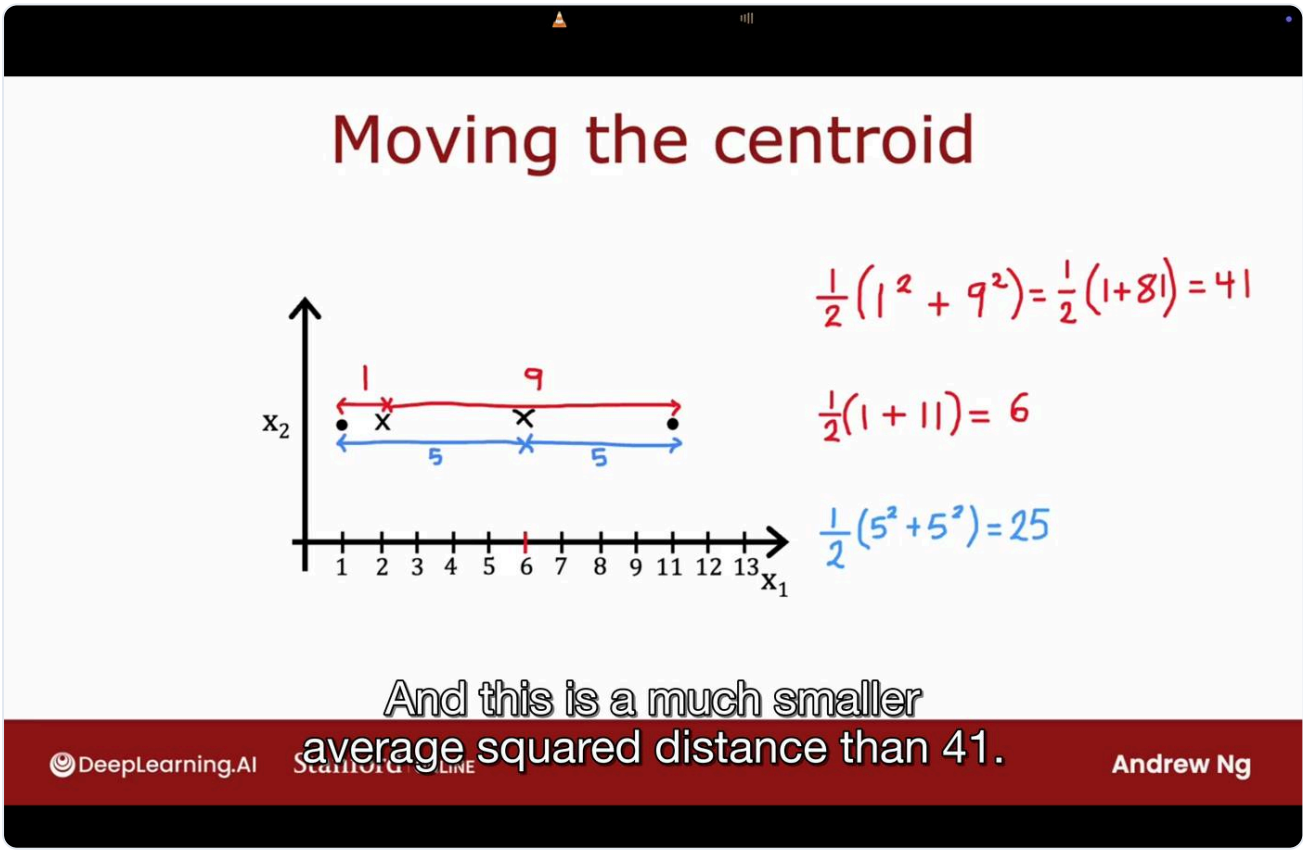


FIGURE 5.10

And this is a much smaller average squared distance than 41. And in fact, you can play around with the location of this cluster centroid and maybe convince yourself that taking this mean location. This average location in the middle of these two training examples, that is really the value that minimizes the square distance. So the fact that the K-means algorithm is optimizing a cost function J means that it is guaranteed to converge, that is on every single iteration. The distortion cost function should go down or stay the same, but if it ever fails to go down or stay the same, in the worst case, if it ever goes up. That means there's a bug in the code, it should never go up because every single step of K means is setting the value CI and mu K to try to reduce the cost function.

Also, if the cost function ever stops going down, that also gives you one way to test if K means has converged. Once there's a single iteration where it stays the same. That usually means K means has converged and you should just stop running the algorithm even further or in some rare cases you will run K means for a long time. And the cost function of the distortion is just going down very, very slowly, and that's a bit like gradient descent where maybe running even longer might help a bit. But if the rate at which the cost function is going down has become very, very slow. You might also just say this is good enough. I'm just going to say it's close enough to convergence and not spend even more compute cycles running the algorithm for even longer.

So these are some of the ways that computing the cost function is helpful helps you figure out if the algorithm has converged. It turns out that there's one other very useful way to take advantage of the cost function, which is to use multiple different random initialization of the cluster centroid. It turns out if you do this, you can often find much better clusters using K means, let's take a look at the next video of how to do that.

LECTURE 6

Random Initialization

The very first step of the K means clustering algorithm, was to choose random locations as the initial guesses for the cluster centroids mu one through mu K. But how do you actually take that random guess. Let's take a look at that in this video, as well as how you can take multiple attempts at the initial guesses with mu one through mu K. That will result in your finding a better set of clusters. Let's take a look, here again is the K means algorithm and in this video let's take a look at how you can implement this first step. When running K means you should pretty much always choose the number of cluster centroids K to be lessened to training examples m.

It doesn't really make sense to have K greater than m because then there won't even be enough training examples to have at least one training example per cluster centroids. So in our earlier example we had K equals two and m equals 30. In order to choose the cluster centroids, the most common way is to randomly pick K training examples. So here is a training set where if I were to randomly pick two training examples, maybe I end up picking this one and this one.

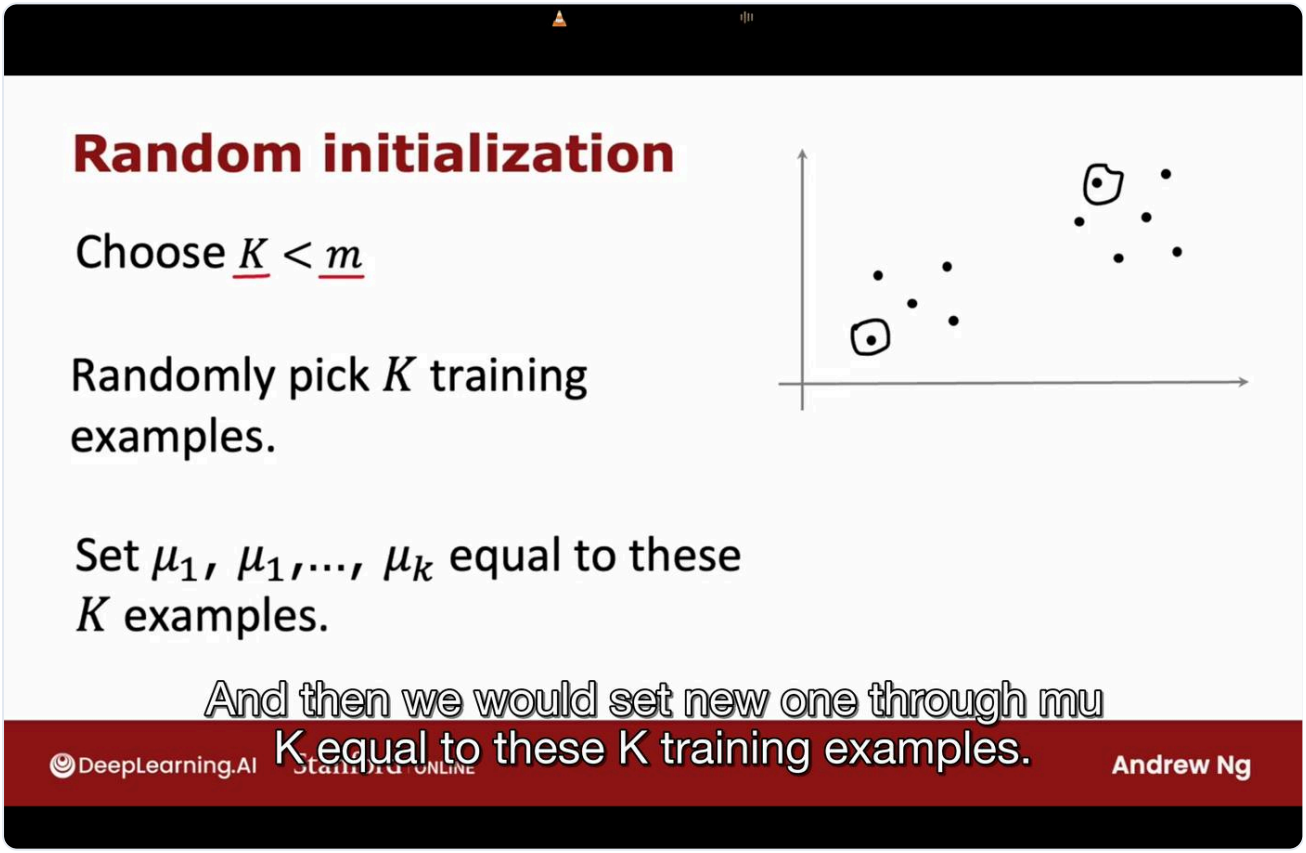


FIGURE 6.1 | Random Initialization

And then we would set new one through mu K equal to these K training examples. So I might initialize my red cluster centroid here, and initialize my blue cluster centroids over here, in the example where K was equal to two. And it turns out that if this was your random initialization and

you were to run K means you pray end up with K means deciding that these are the two classes in the data set. Notes that this method of initializing the cost of central is a little bit different than what I had used in the illustration in the earlier videos.

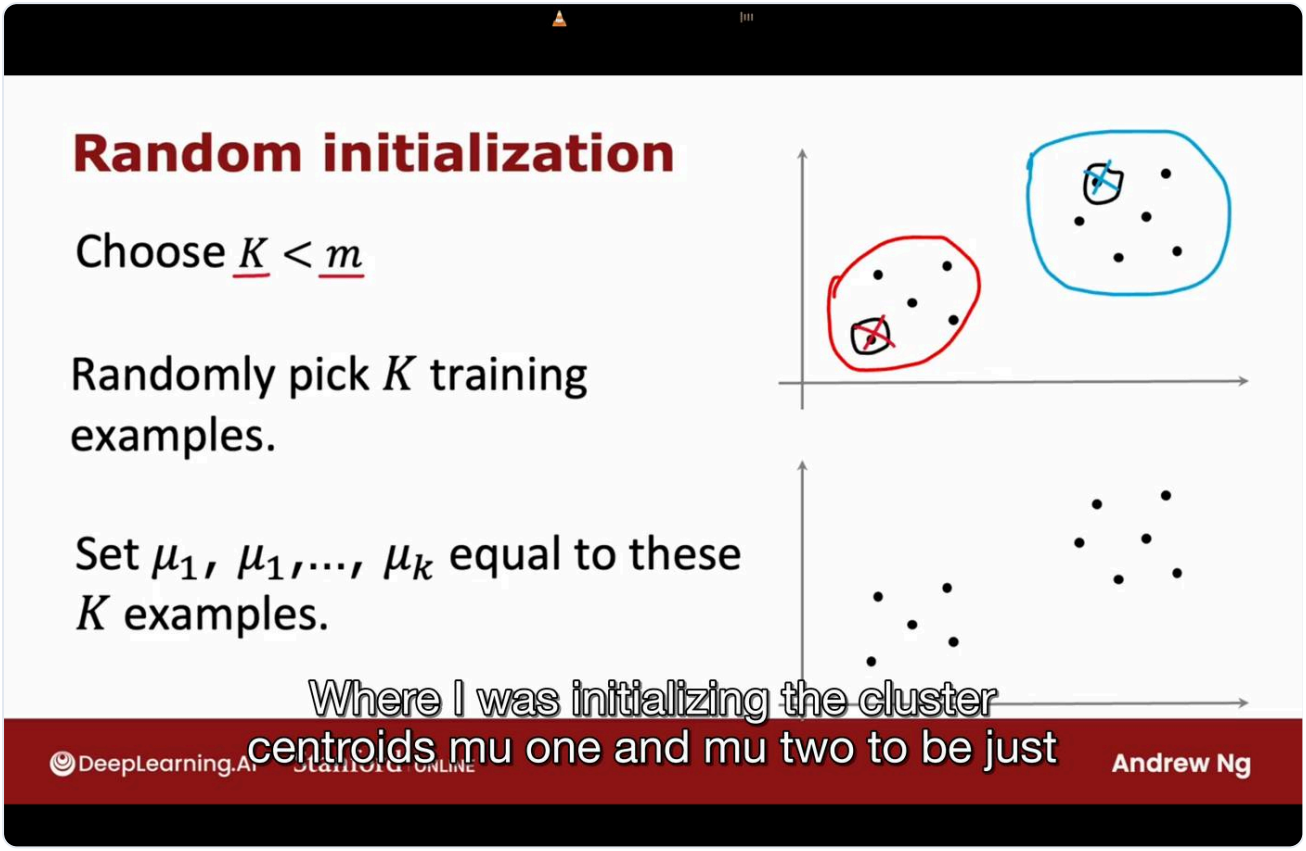


FIGURE 6.2 | Random Initialization

Where I was initializing the cluster centroids mu one and mu two to be just random points rather than sitting on top of specific training examples. I've done that to make the illustrations clearer in the earlier videos. But what I'm showing in this slide is actually a much more commonly used way of initializing the clusters centroids. Now with this method there is a chance that you end up with an initialization of the cluster centroids where the red cross is here and maybe the blue cross is here.

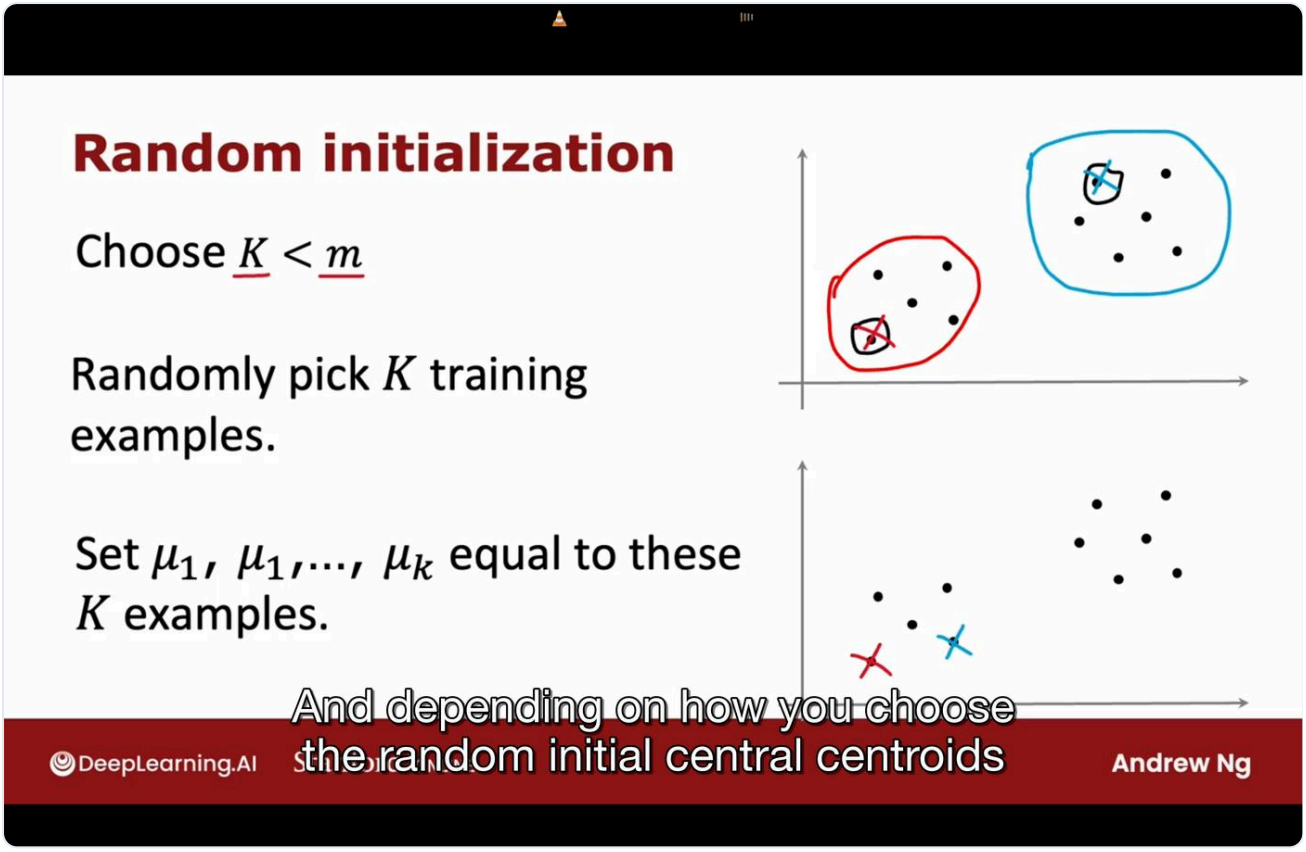


FIGURE 6.3 | Random Initialization

And depending on how you choose the random initial central centroids K-means will end up picking a difference set of causes for your data set. Let's look at a slightly more complex example, where we're going to look at this data set and try to find three clusters so k equals three in this data. If you were to run K means with one random initialization of the cluster centroid, you may get this result up here and this looks like a pretty good choice. Pretty good clustering of the data into three different clusters. But with a different initialization, say you had happened to initialize two of the cluster centroids within this group of points. And one within this group of points, after running k means you might end up with this clustering, which doesn't look as good.

And this turns out to be a local optima, in which K-means is trying to minimize the distortion cost function, that cost function J of C one through CM and mu one through mu K that you saw in the last video.

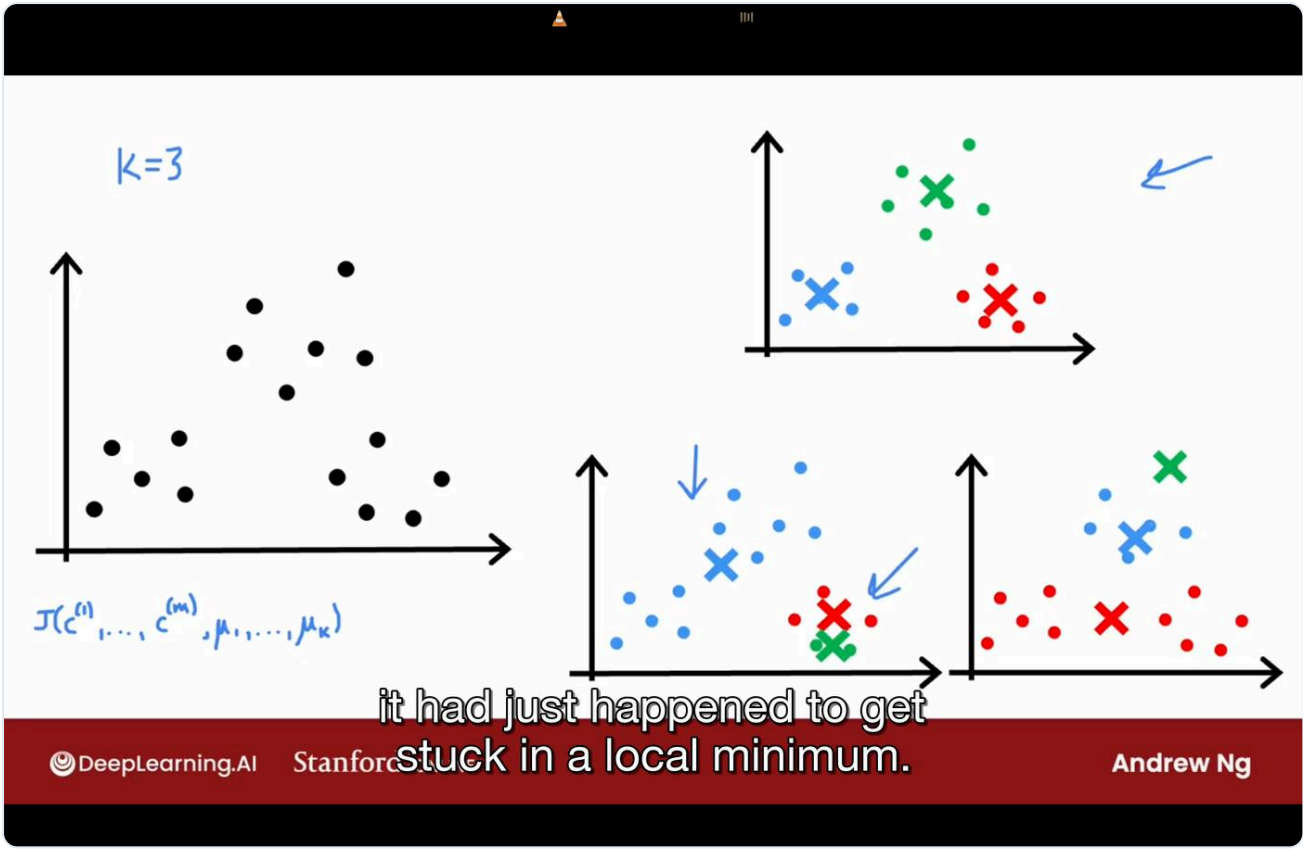


FIGURE 6.4

But with this less fortunate choice of random initialization, it had just happened to get stuck in a local minimum.

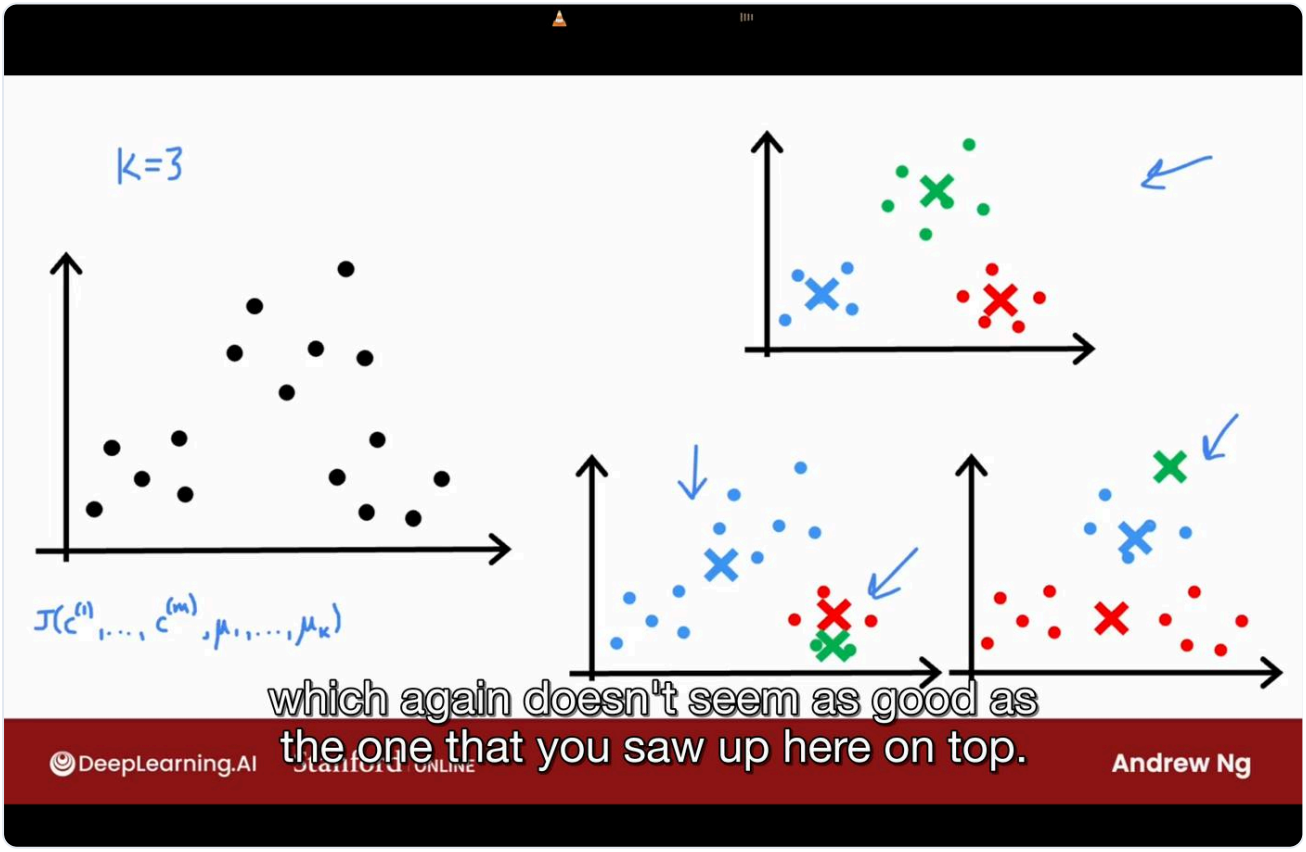


FIGURE 6.5

And here's another example of a local minimum, where a different random initialization course came in to find this clustering of the data into three clusters, which again doesn't seem as good as the one that you saw up here on top. So if you want to give k means multiple shots at finding the best local optimum. If you want to try multiple random initialization, so give it a better chance of finding this good clustering up on top. One other thing you could do with the K-means algorithm is to run it multiple times and then to try to find the best local optima. And it turns out that if you were to run k means three times say, and end up with these three distinct clusterings.

Then one way to choose between these three solutions, is to compute the cost function J for all three of these solutions, all three of these choices of clusters found by k means.

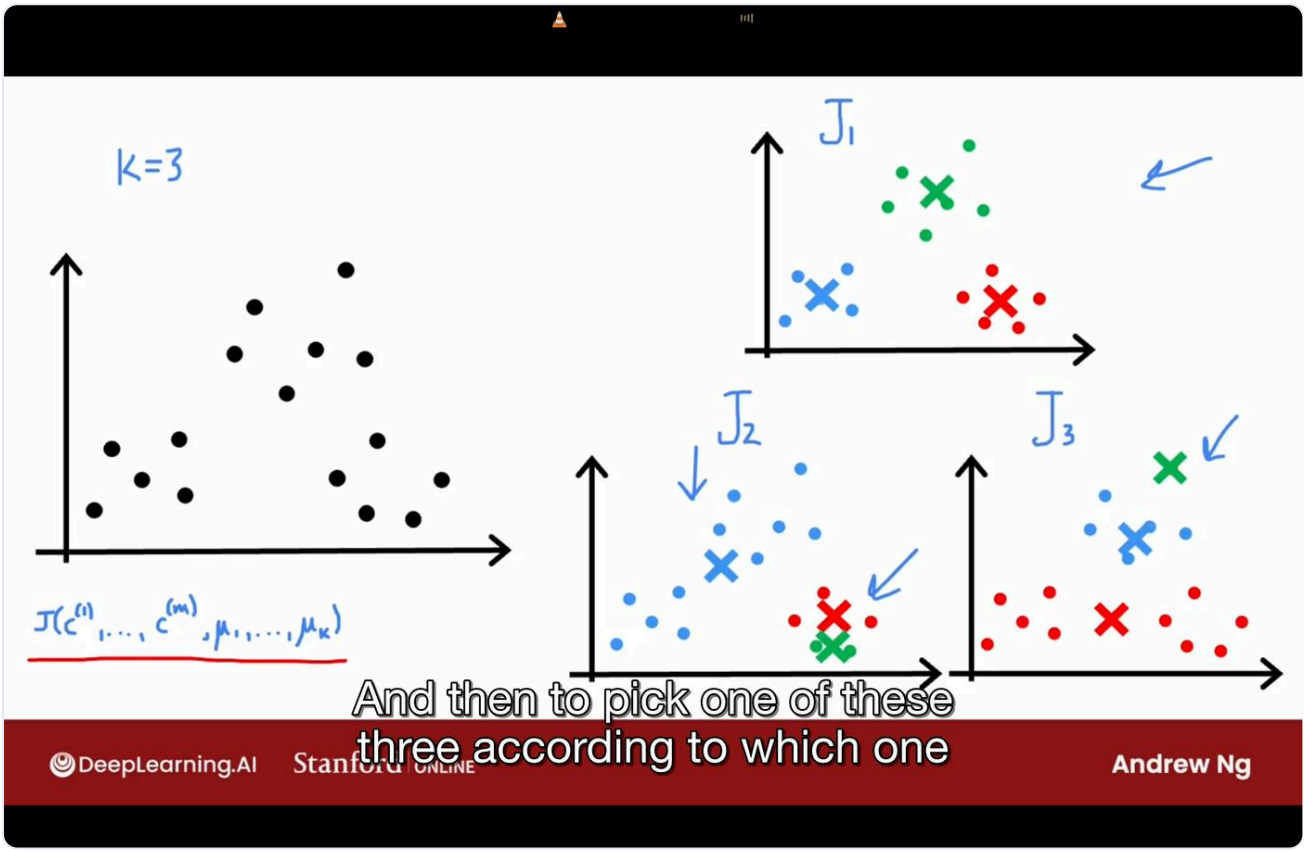


FIGURE 6.6

And then to pick one of these three according to which one of them gives you the lowest value for the cost function J. And in fact, if you look at this grouping of clusters up here, this green cross has relatively small square distances, all the green dots. The red cross is relatively small distance and red dots and similarly the blue cross. And so the cost function J will be relatively small for this example on top. But here, the blue cross has larger distances to all of the blue dots. And here the red cross has larger distances to all of the red dots, which is why the cost function J, for these examples down below would be larger. Which is why if you pick from these three options, the one with the smallest distortion of the smallest cost function J.

You end up selecting this choice of the cluster centroids.

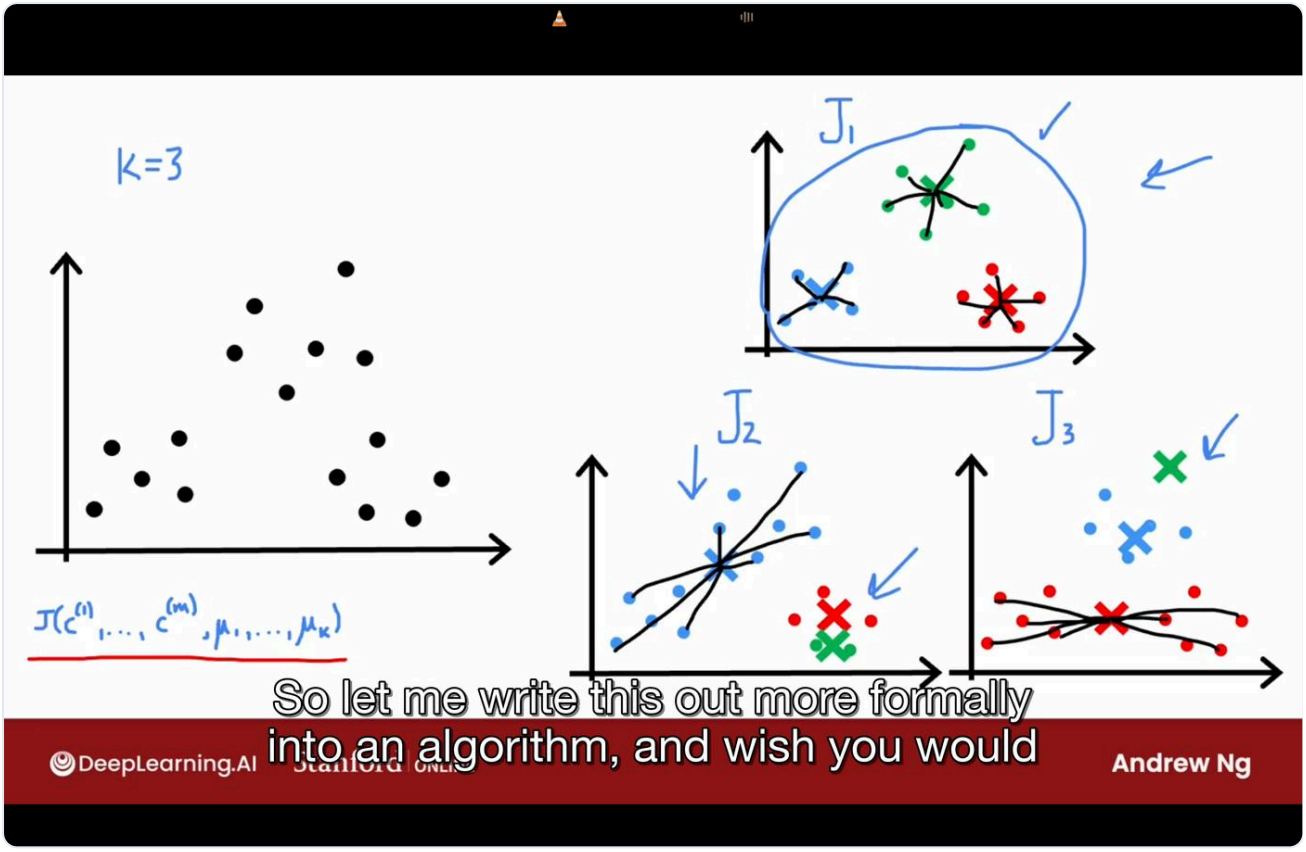


FIGURE 6.7

So let me write this out more formally into an algorithm, and wish you would run K-means multiple times using different random initialization. Here's the algorithm, if you want to use 100 random initialization for K-means, then you would run 100 times randomly initialized K-means using the method that you saw earlier in this video. Pick K training examples and let the cluster centroids initially be the locations of those K training examples.

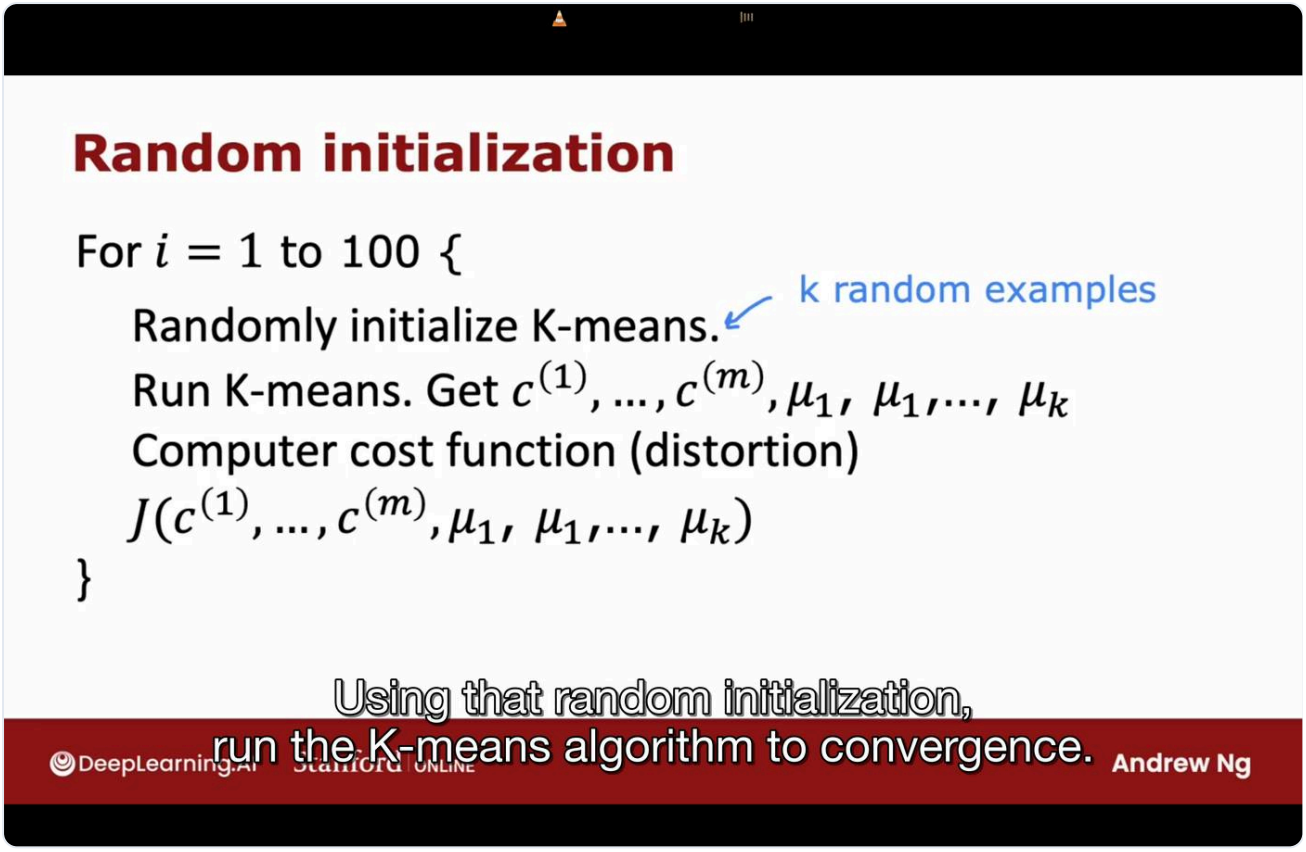


FIGURE 6.8 | Random Initialization

Using that random initialization, run the K-means algorithm to convergence. And that will give you a choice of cluster assignments and cluster centroids. And then finally, you would compute the distortion compute the cost function as follows.

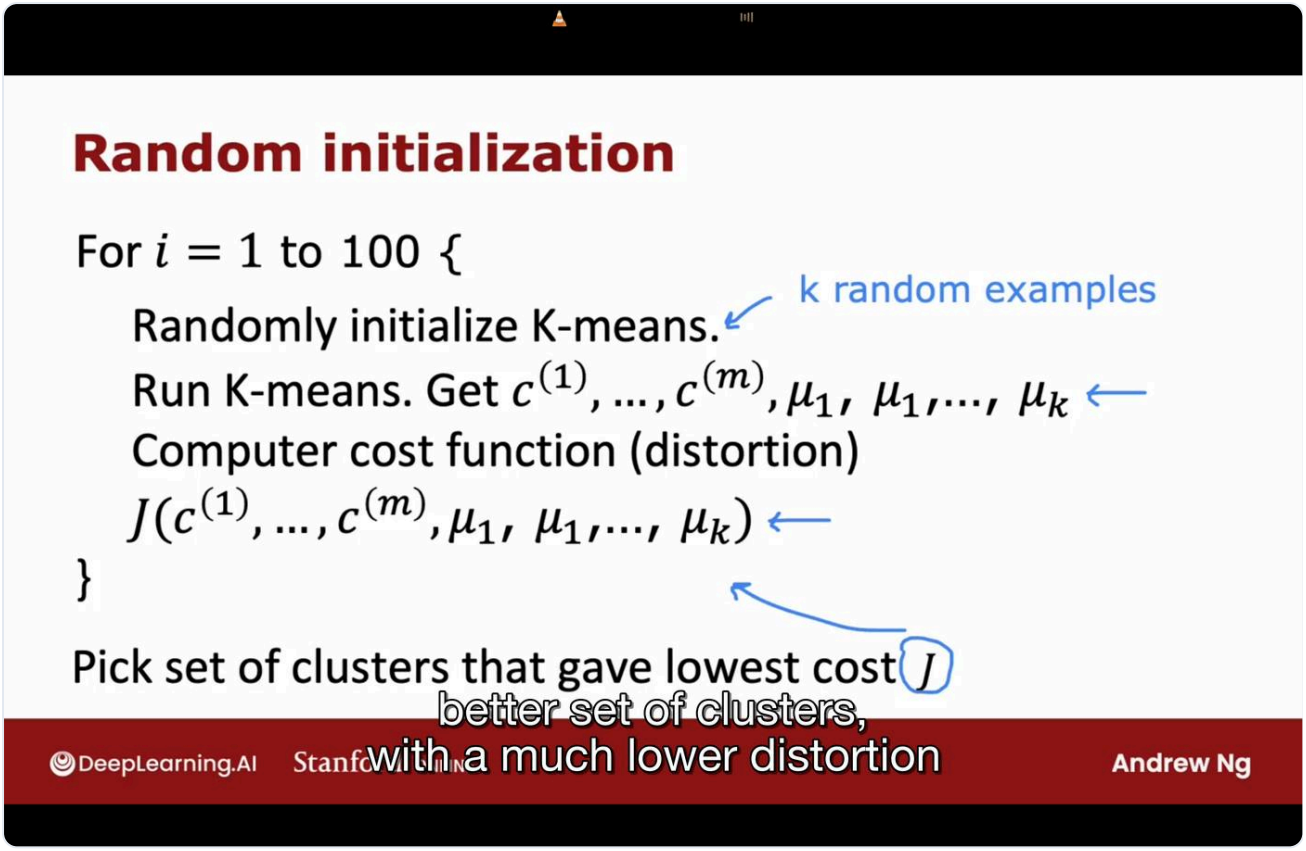


FIGURE 6.9 | Random Initialization

After doing this, say 100 times, you would finally pick the set of clusters, that gave the lowest cost. And it turns out that if you do this will often give you a much better set of clusters, with a much lower distortion function than if you were to run K means only a single time. I plugged in the number up here as 100. When I'm using this method, doing this somewhere between say 50 to 1000 times would be pretty common.

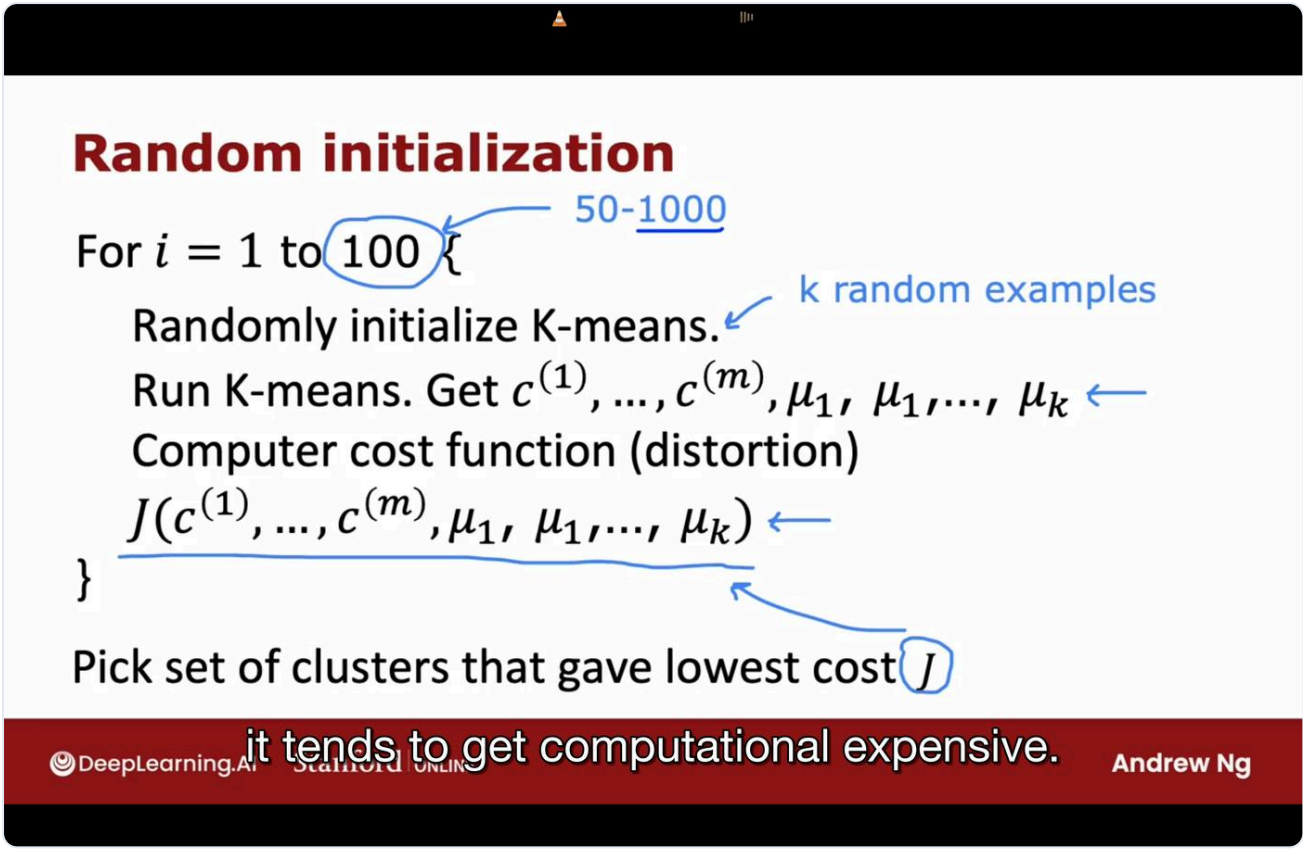


FIGURE 6.10 | Random Initialization

Where, if you run this procedure a lot more than 1000 times, it tends to get computational expensive. And you tend to have diminishing returns when you run it a lot of times. Whereas trying at least maybe 50 or 100 random initializations, will often give you a much better result than if you only had one shot at picking a good random initialization. But with this technique you are much more likely to end up with this good choice of clusters on top. And these less superior local minima down at the bottom. So that's it, when I'm using the K means algorithm myself, I will almost always use more than one random initialization. Because it just causes K means to do a much better job minimizing the distortion cost function and finding a much better choice for the cluster centroids.

Before we wrap up our discussion of K means, there's just one more video in which I hope to discuss with you. The question of how do you choose the number of clusters centroids? How do you choose the value of K? Let's go on to the next video to take a look at that.

LECTURE 7

Clusters

The k-means algorithm requires as one of its inputs, k, the number of clusters you want it to find, but how do you decide how many clusters to used. Do you want two clusters or three clusters of five clusters or 10 clusters? Let's take a look. For a lot of clustering problems, the right value of K is truly ambiguous. If I were to show different people the same data set and ask, how many clusters do you see?

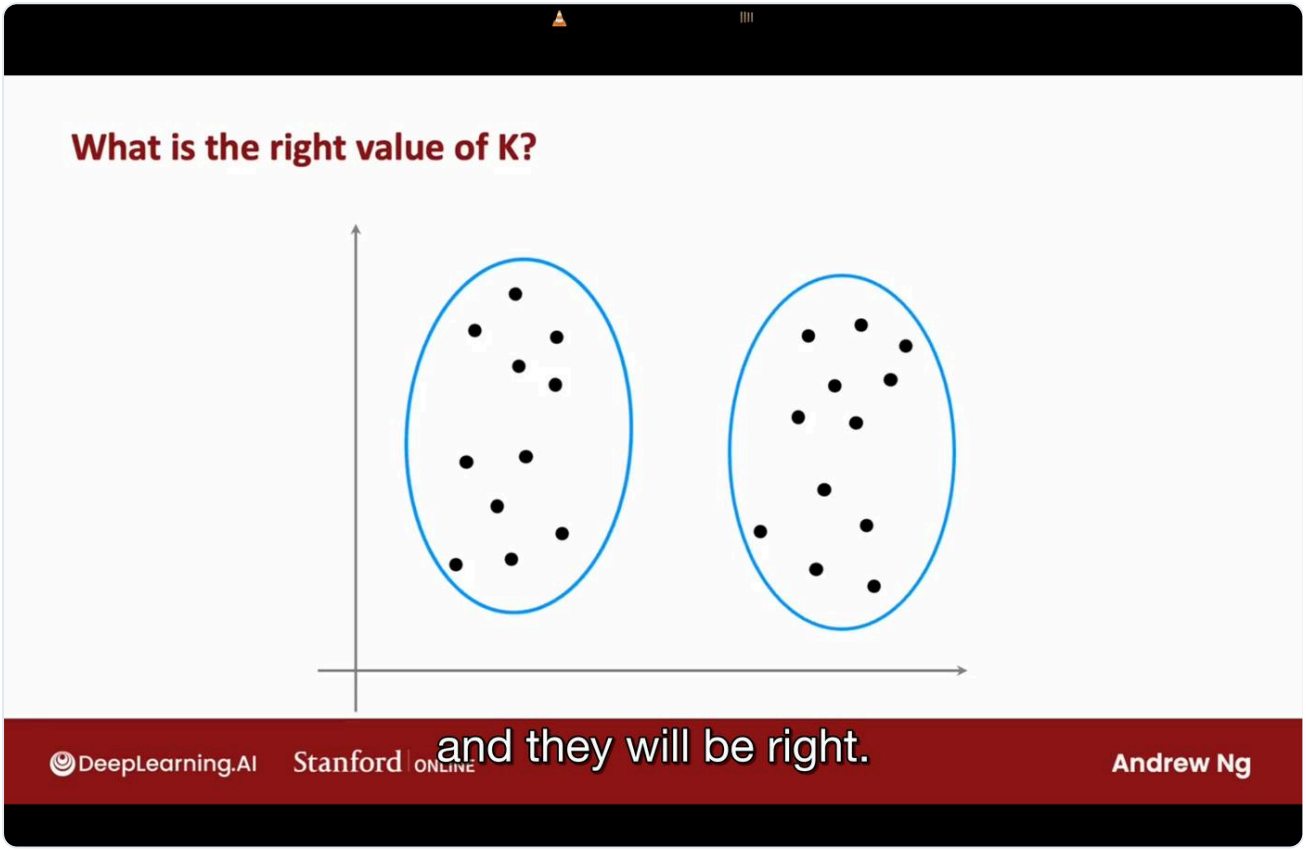


FIGURE 7.1 | What Is the Right Value of K?

There will definitely be people that will say, it looks like there are two distinct clusters and they will be right.

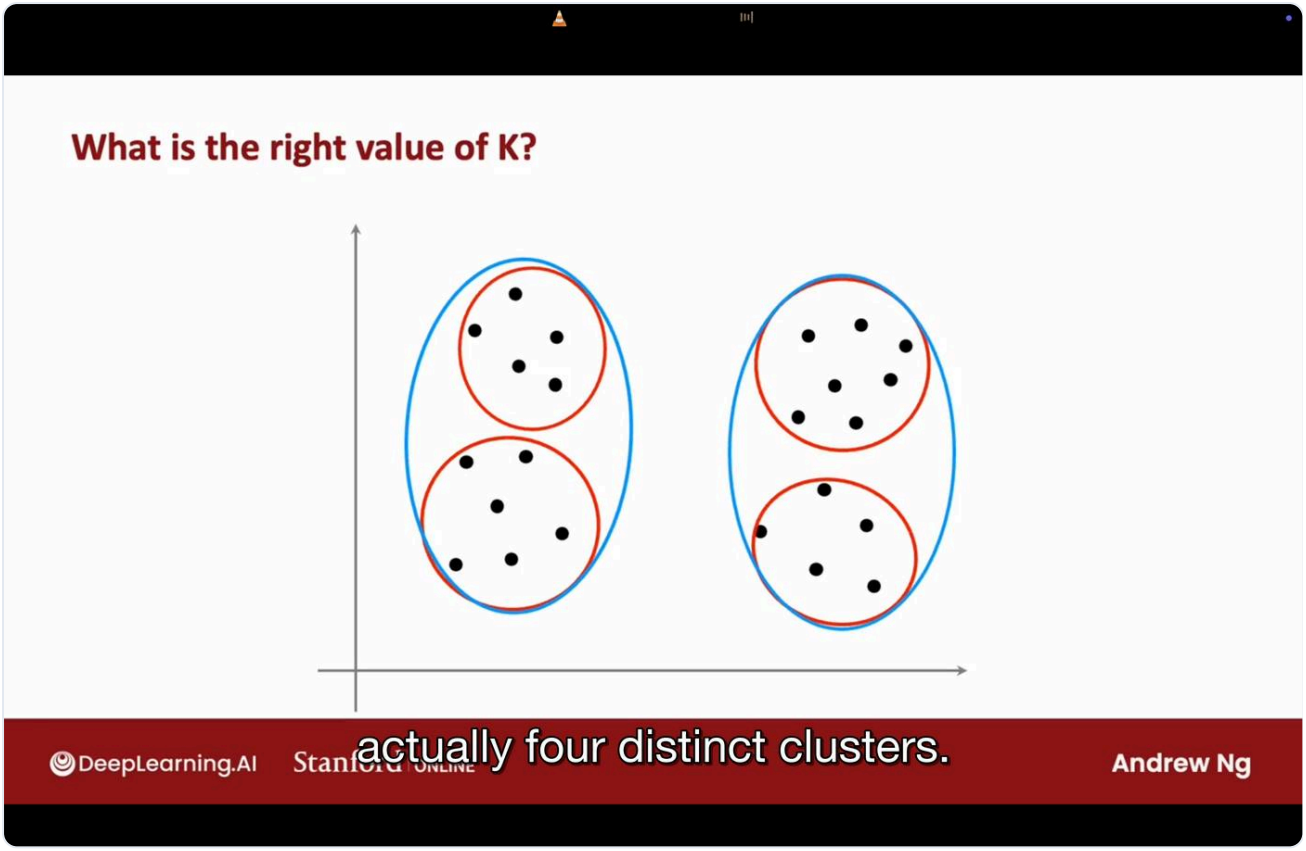


FIGURE 7.2 | What Is the Right Value of K?

There would also be others that will see actually four distinct clusters. They would also be right. Because clustering is unsupervised learning algorithm you're not given the quote right answers in the form of specific labels to try to replicate. There are lots of applications where the data itself does not give a clear indicator for how many clusters there are in it. I think it truly is ambiguous if this data has two or four, or maybe three clusters. If you take say, the red one here and the two blue ones here say. If you look at the academic literature on K-means, there are a few techniques to try to automatically choose the number of clusters to use for a certain application.

I'll briefly mention one here that you may see others refer to, although I had to say, I personally do not use this method myself. But one way to try to choose the value of K is called the elbow method and what that does is you would run K-means with a variety of values of K and plot the cost function or the distortion function J as a function of the number of clusters.

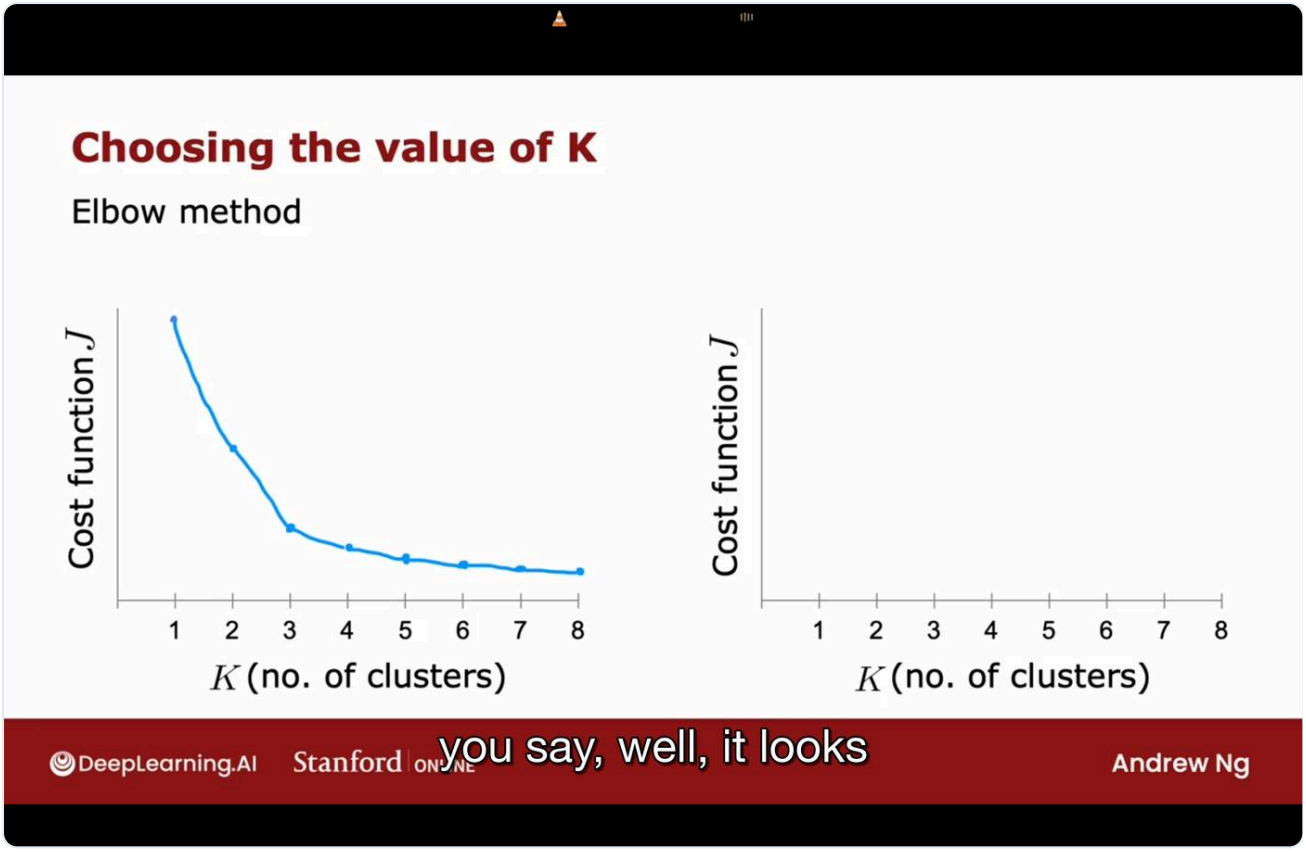


FIGURE 7.3 | Choosing the Value of K

What you find is that when you have very few clusters, say one cluster, the distortion function or the cost function J will be high and as you increase the number of clusters, it will go down, maybe as follows. and if the curve looks like this, you say, well, it looks like the cost function is decreasing rapidly until we get to three clusters but the decrease is more slowly after that.

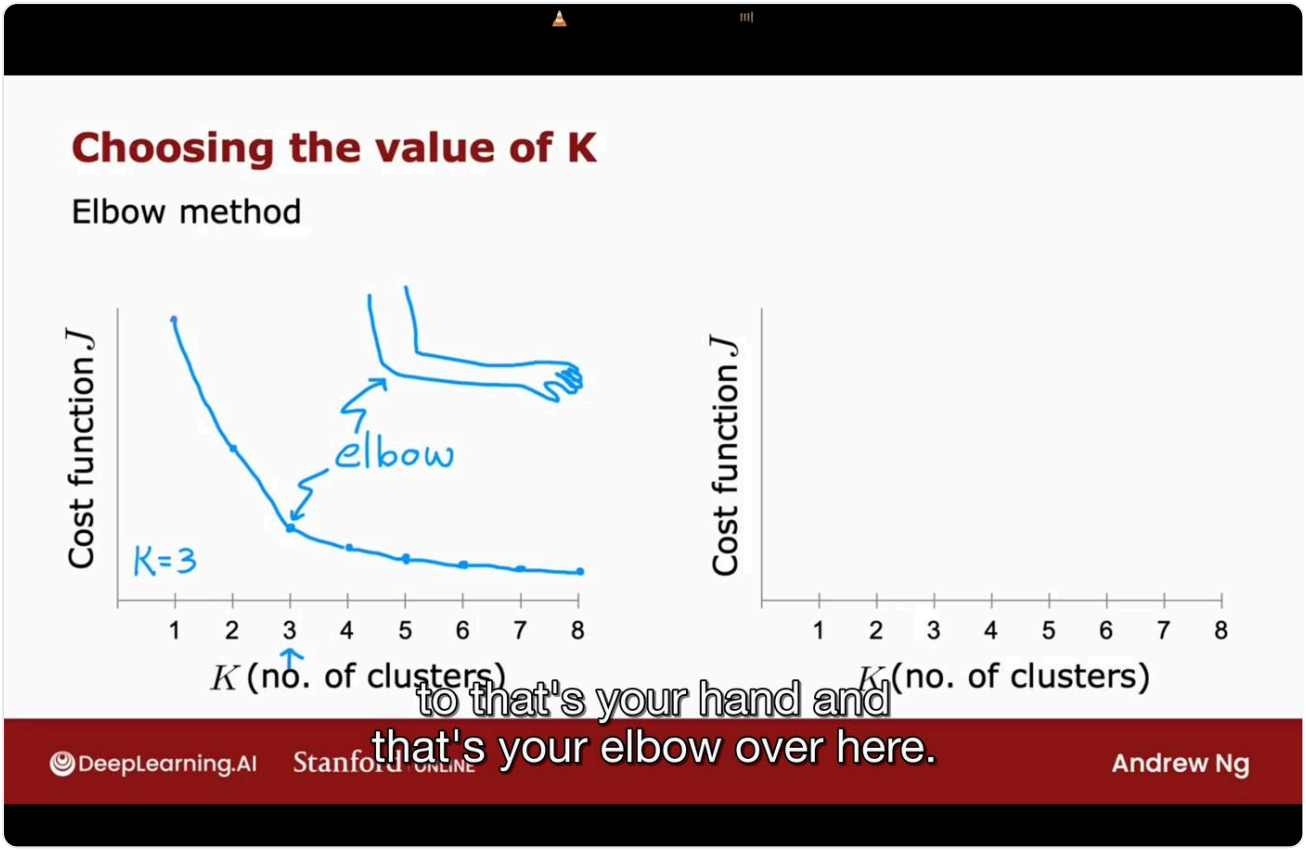


FIGURE 7.4 | Choosing the Value of K

Let's choose K equals 3 and this is called an elbow, by the way, because think of it as analogous to that's your hand and that's your elbow over here. Plotting the cost function as a function of K could help, it could help you gain some insight.

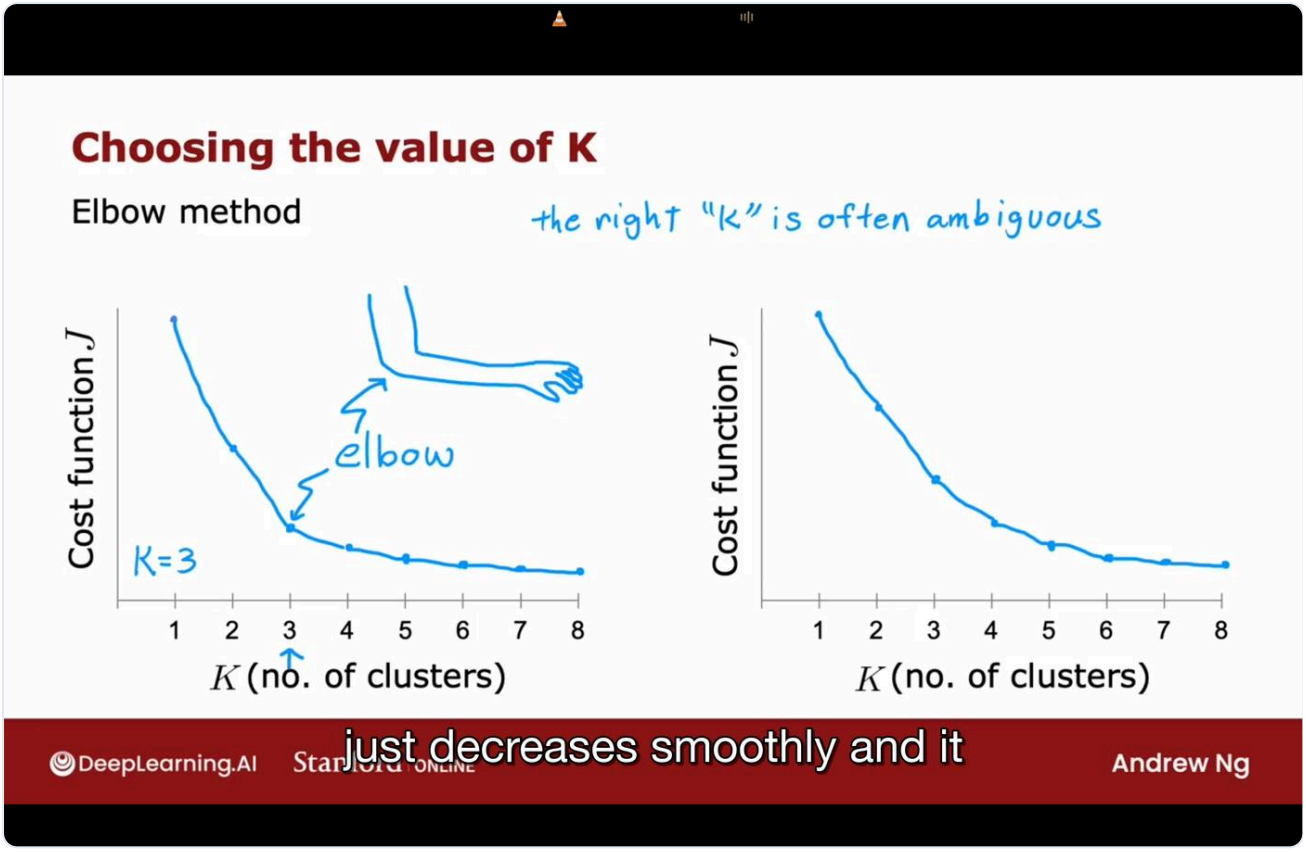


FIGURE 7.5 | Choosing the Value of K

I personally hardly ever use the the elbow method myself to choose the right number of clusters because I think for a lot of applications, the right number of clusters is truly ambiguous and you find that a lot of cost functions look like this with just decreases smoothly and it doesn't have a clear elbow by wish you could use to pick the value of K.

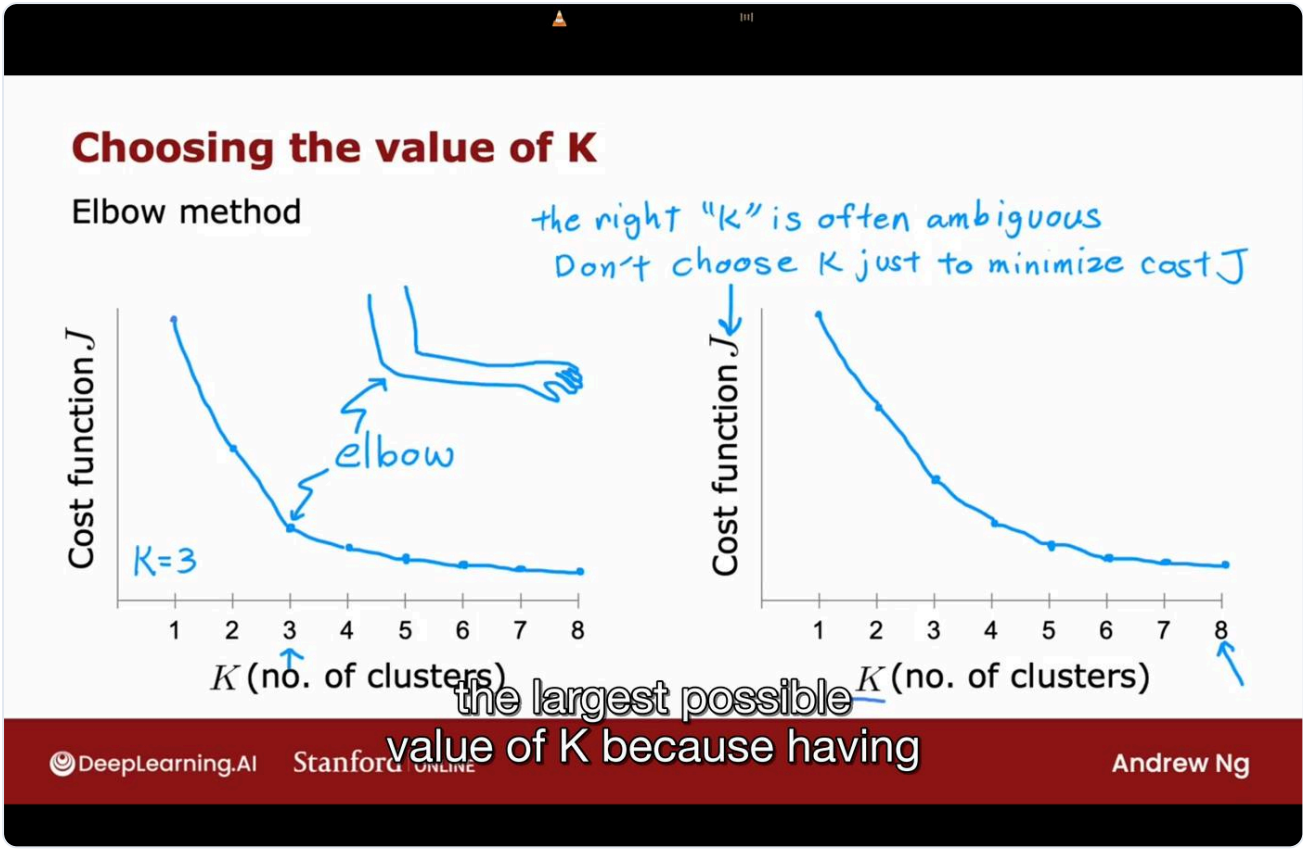


FIGURE 7.6 | Choosing the Value of K

By the way, one technique that does not work is to choose K so as to minimize the cost function J because doing so would cause you to almost always just choose the largest possible value of K because having more clusters will pretty much always reduce the cost function J. Choosing K to minimize the cost function J is not a good technique. How do you choose the value of K and practice? Often you're running K-means in order to get clusters to use for some later or some downstream purpose. That is, you're going to take the clusters and do something with those clusters.

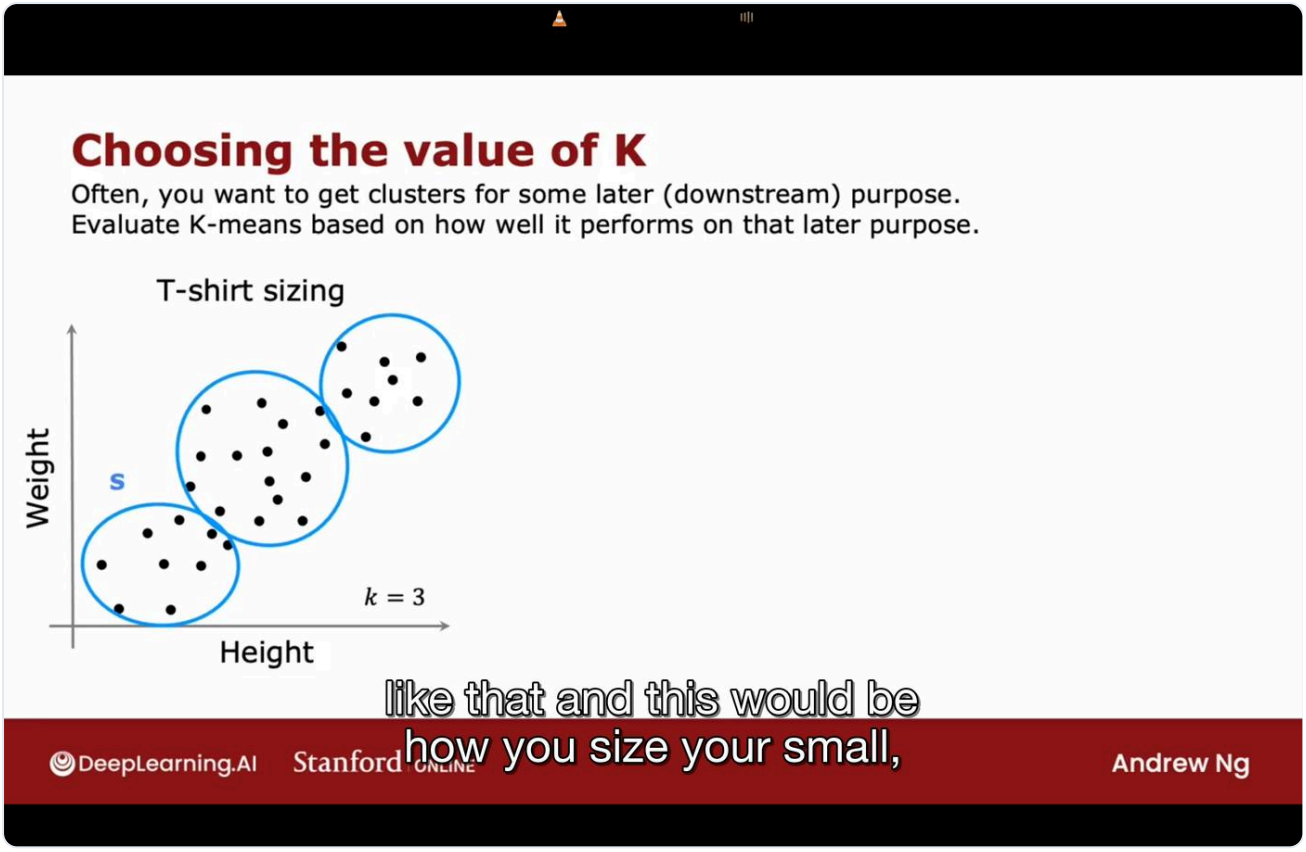


FIGURE 7.7 | Choosing the Value of K

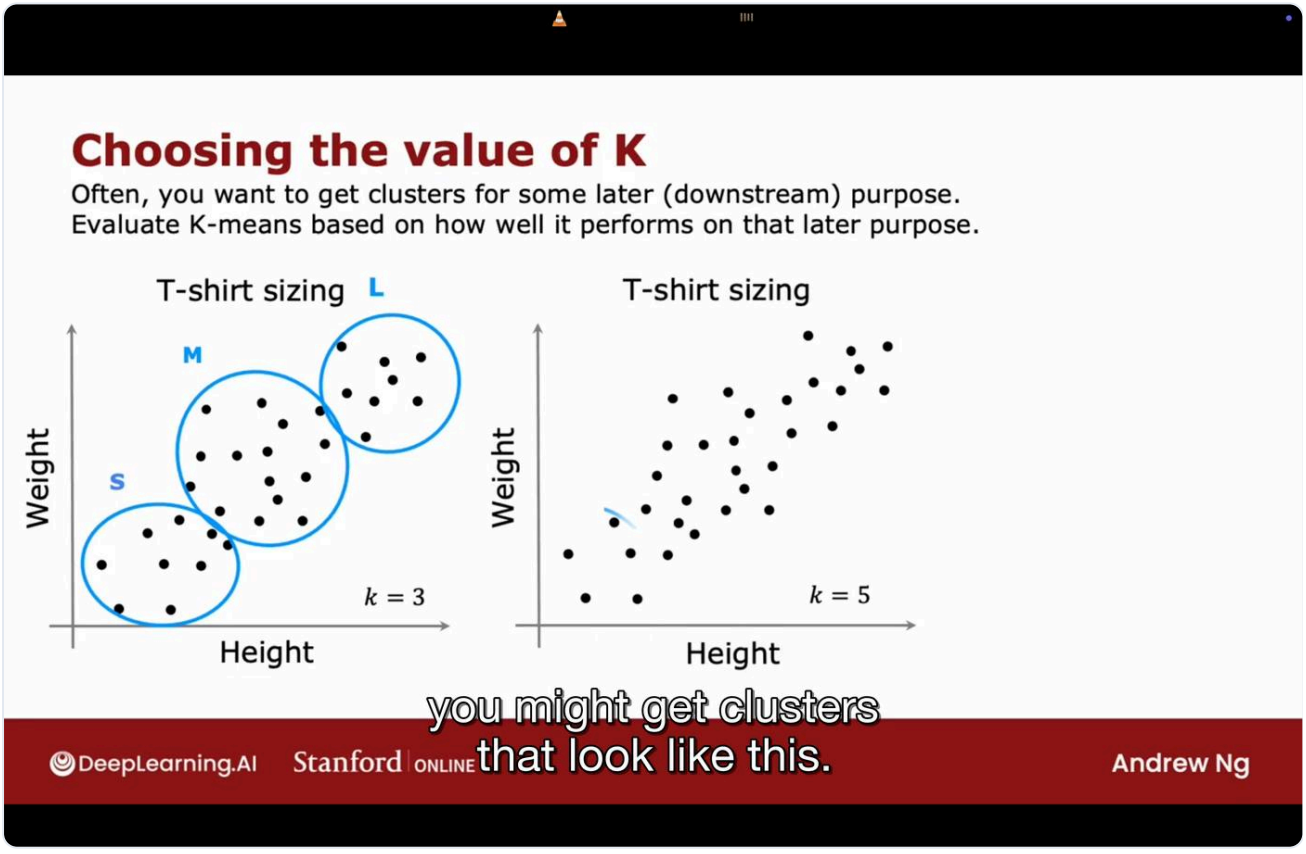


FIGURE 7.8 | Choosing the Value of K

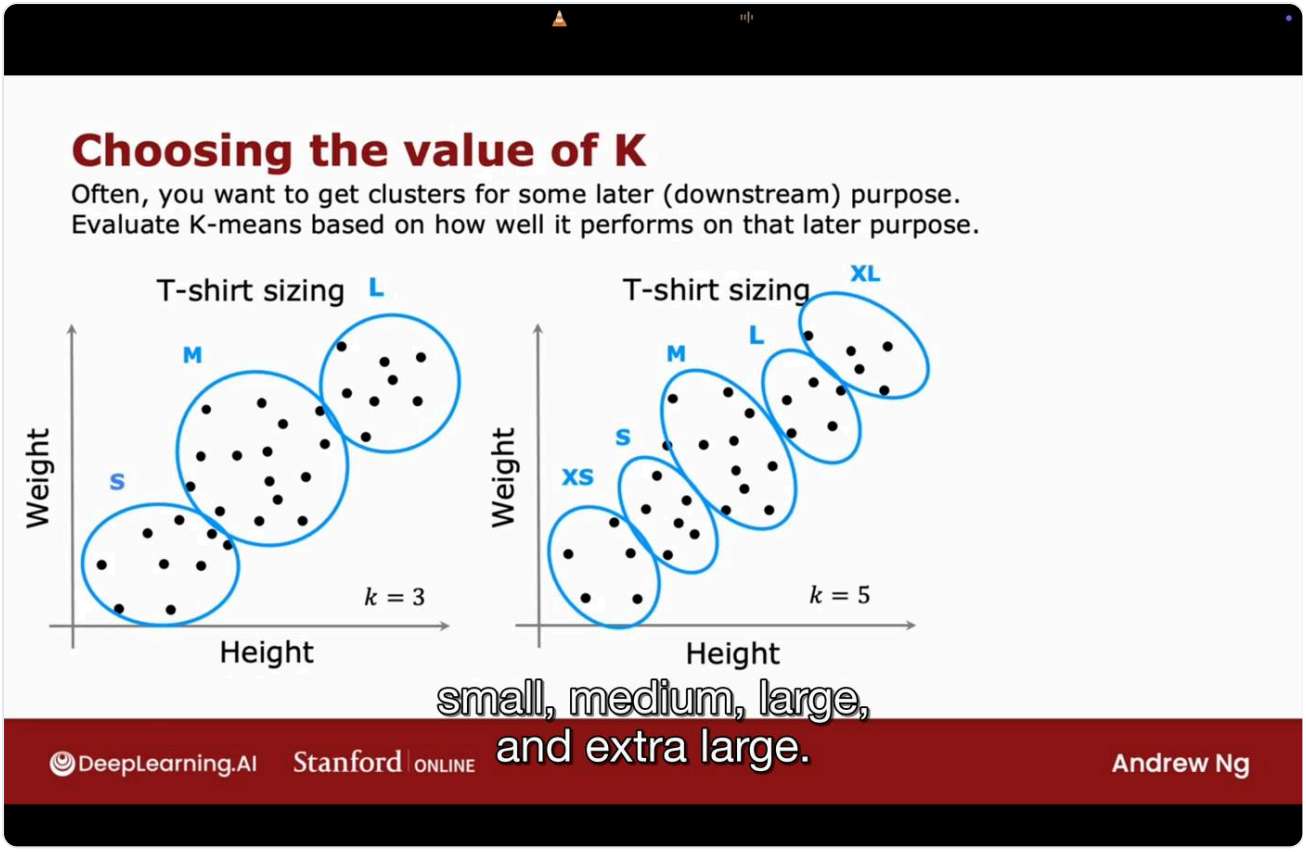


FIGURE 7.9 | Choosing the Value of K

What I usually do and what I recommend you do is to evaluate K-means based on how well it performs for that later downstream purpose. Let me illustrate to the example of t-shirt sizing. One thing you could do is run K-means on this data set to find the clusters, in which case you may find clusters like that and this would be how you size your small, medium, and large t-shirts, but how many t-shirt sizes should there be? Well, it's ambiguous. If you were to also run K-means with five clusters, you might get clusters that look like this. This will let shoe size t-shirts according to extra small, small, medium, large, and extra large. Both of these are completely valid and completely fine groupings of the data into clusters, but whether you want to use three clusters or five clusters can now be decided based on what makes sense for your t-shirt business.

Does a trade-off between how well the t-shirts will fit, depending on whether you have three sizes or five sizes, but there will be extra costs as well associated with manufacturing and shipping five types of t-shirts instead of three different types of t-shirts. What I would do in this case is to run K-means with K equals 3 and K equals 5 and then look at these two solutions to see based on the trade-off between fits of t-shirts with more sizes, results in better fit versus the extra cost of making more t-shirts where making fewer t-shirts is simpler and less expensive to try to decide what makes sense for the t-shirt business. When you get to the programming exercise, you also see there an application of K-means to image compression.

This is actually one of the most fun visual examples of K-means and there you see that there'll be a trade-off between the quality of the compressed image, that is, how good the image looks versus how much you can compress the image to save the space. In that program exercise, you see that you can use that trade-off to maybe manually decide what's the best value of K based on how good do you want the image to look versus how large you want the compress image size to be.

That's it for the K-means clustering algorithm. Congrats on learning your first unsupervised learning algorithm. You now know not just how to do supervised learning, but also unsupervised learning.

I hope you also have fun with the practice lab, is actually one of the most fun exercises I know of the K-means. With that, we're ready to move on to our second unsupervised learning algorithm, which is anomaly detection. How do you look at the data set and find unusual or anomalous things in it. This turns out to be another, one of the most commercially important applications of unsupervised learning. I've used this myself many times in many different applications. Let's go on to the next video to talk about anomaly detection.

LECTURE 8

Density Estimation

Let's look at our second unsupervised learning algorithm. Anomaly detection algorithms look at an unlabeled dataset of normal events and thereby learns to detect or to raise a red flag for if there is an unusual or an anomalous event. Let's look at an example. Some of my friends were working on using anomaly detection to detect possible problems with aircraft engines that were being manufactured. When a company makes an aircraft engine, you really want that aircraft engine to be reliable and function well because an aircraft engine failure has very negative consequences. So some of my friends were using anomaly detection to check if an aircraft engine after it was manufactured seemed anomalous or if there seemed to be anything wrong with it.

Here's the idea, after an aircraft engine rolls off the assembly line, you can compute a number of different features of the aircraft engine. So, say feature x1 measures the heat generated by the engine. Feature x2 measures the vibration intensity and so on and so forth for additional features as well. But to simplify the slide a bit, I'm going to use just two features x1 and x2 corresponding to the heat and the vibrations of the engine. Now, it turns out that aircraft engine manufacturers don't make that many bad engines. And so the easier type of data to collect would be if you have manufactured m aircraft engines to collect the features x1 and x2 about how these m engines behave and probably most of them are just fine that normal engines rather than ones with a defect or flaw in them.

And the anomaly detection problem is, after the learning algorithm has seen these m examples of how aircraft engines typically behave in terms of how much heat is generated and how much they vibrate.

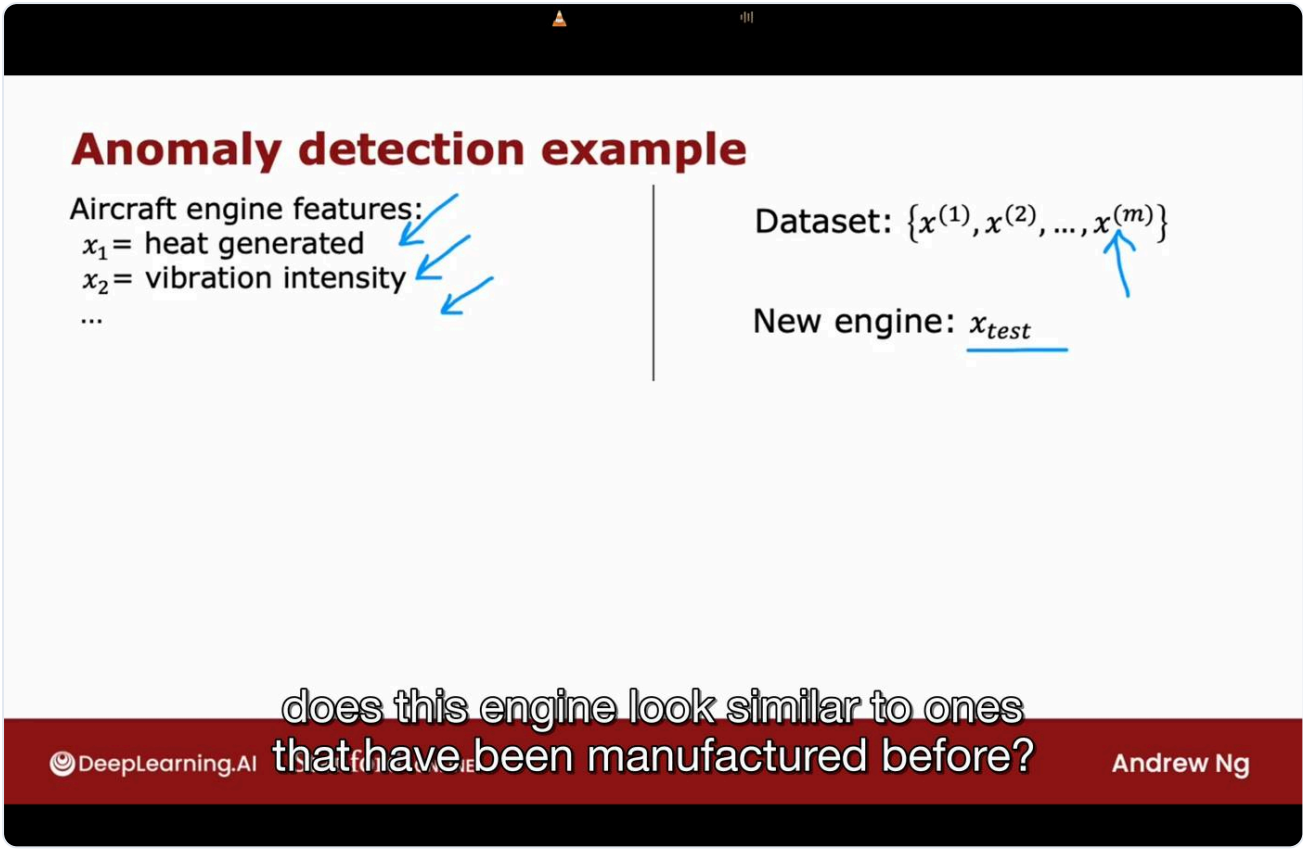


FIGURE 8.1 | Anomaly Detection Example

If a brand new aircraft engine were to roll off the assembly line and it had a new feature vector given by Xtest, we'd like to know does this engine look similar to ones that have been manufactured before? So is this probably okay? Or is there something really weird about this engine which might cause this performance to be suspect, meaning that maybe we should inspect it even more carefully before we let it get shipped out and be installed in an airplane and then hopefully nothing will go wrong with it. Here's how an anomaly detection algorithm works.

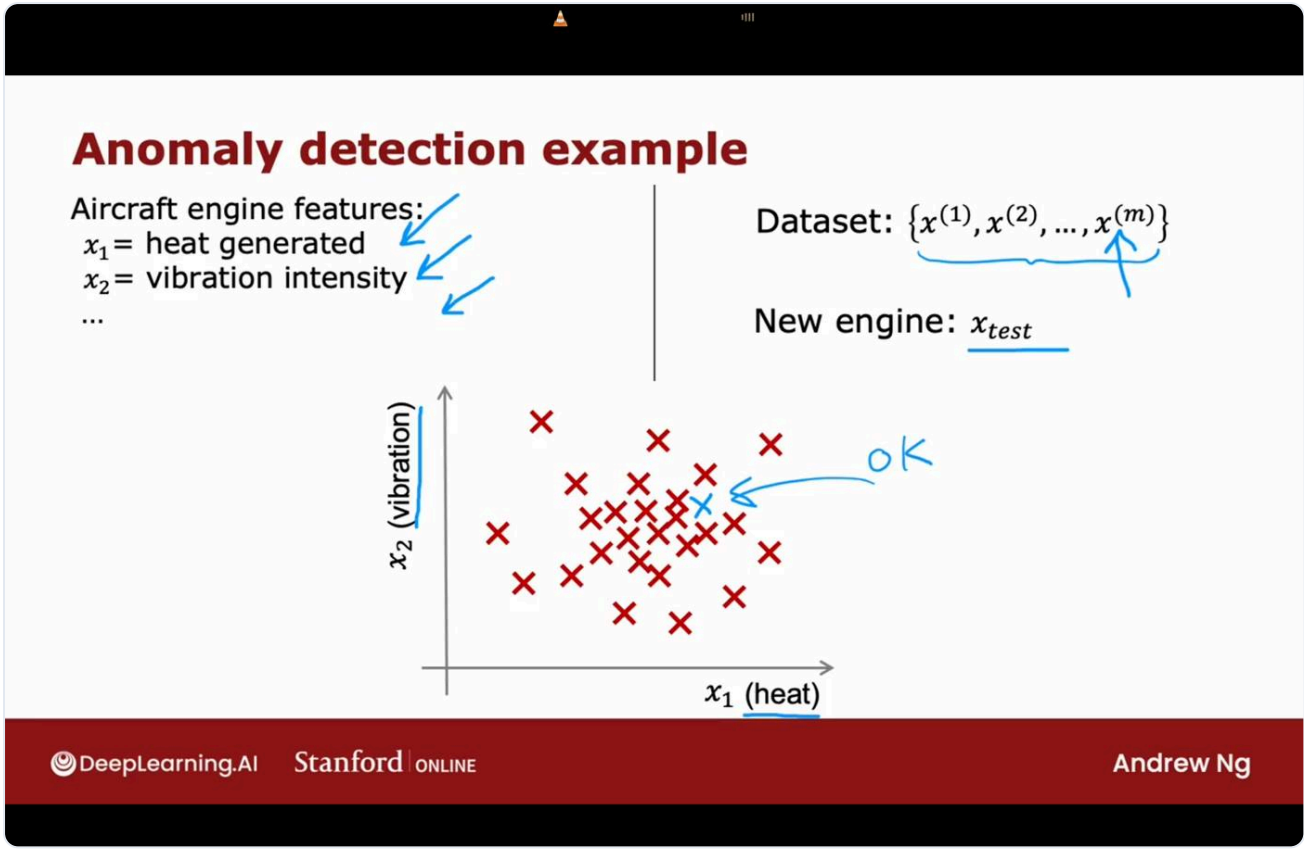


FIGURE 8.2 | Anomaly Detection Example

Let me plot the examples x1 through xm over here via these crosses where each cross each data point in this plot corresponds to a specific engine with a specific amount of heat and specific amount of vibrations. If this new aircraft engine Xtest rolls off the assembly lin,e and if you were to plot these values of x1 and x2 and if it were here, you say, okay, that looks probably okay. Looks very similar to other aircraft engines. Maybe I don't need to worry about this one.

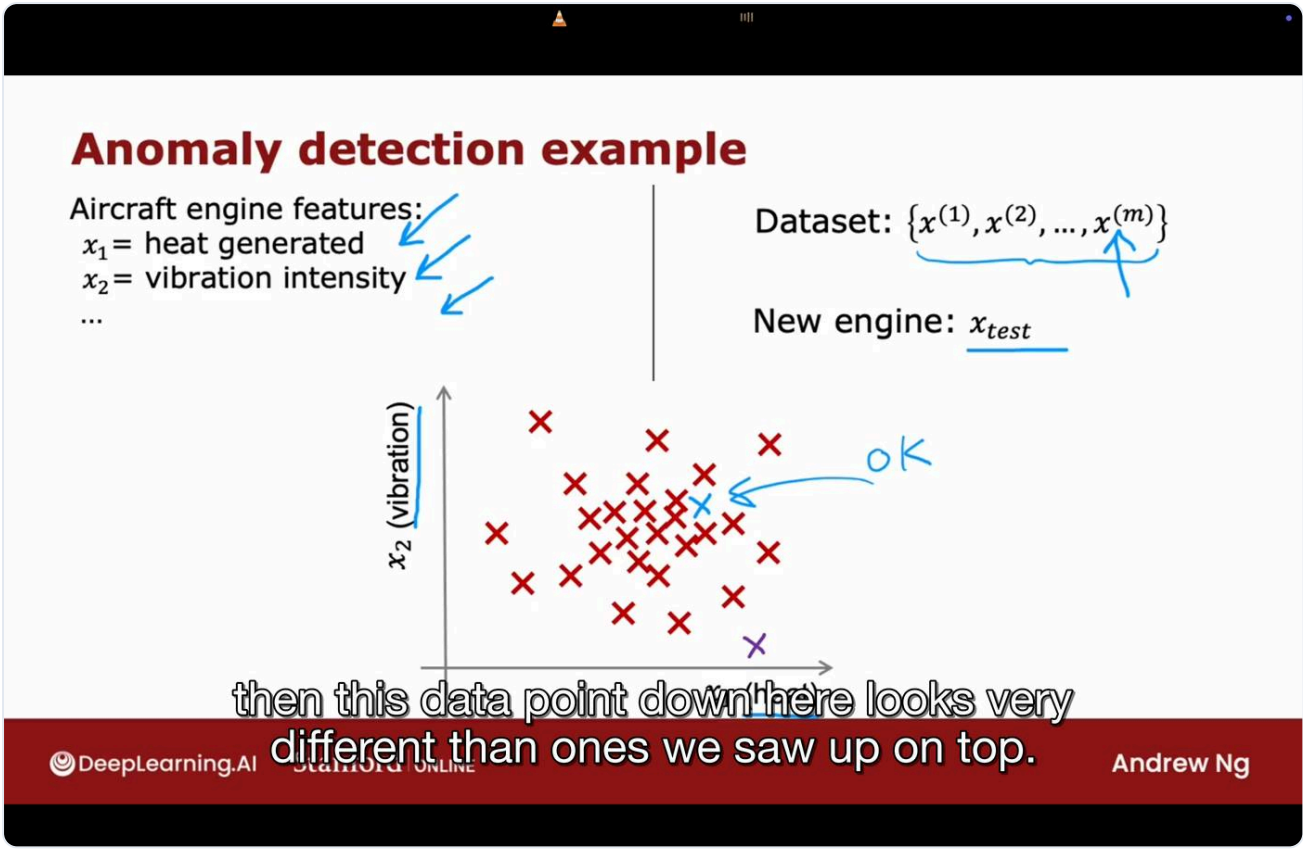


FIGURE 8.3 | Anomaly Detection Example

But if this new aircraft engine has a heat and vibration signature that is say all the way down here, then this data point down here looks very different than ones we saw up on top. And so we will probably say, boy, this looks like an anomaly. This doesn't look like the examples I've seen before, we better inspect this more carefully before we let this engine get installed on an airplane. How can you have an algorithm address this problem? The most common way to carry out anomaly detection is through a technique called density estimation. And what that means is, when you're given your training sets of these m examples, the first thing you do is build a model for the probability of x.

In other words, the learning algorithm will try to figure out what are the values of the features x1 and x2 that have high probability and what are the values that are less likely or have a lower chance or lower probability of being seen in the data set.

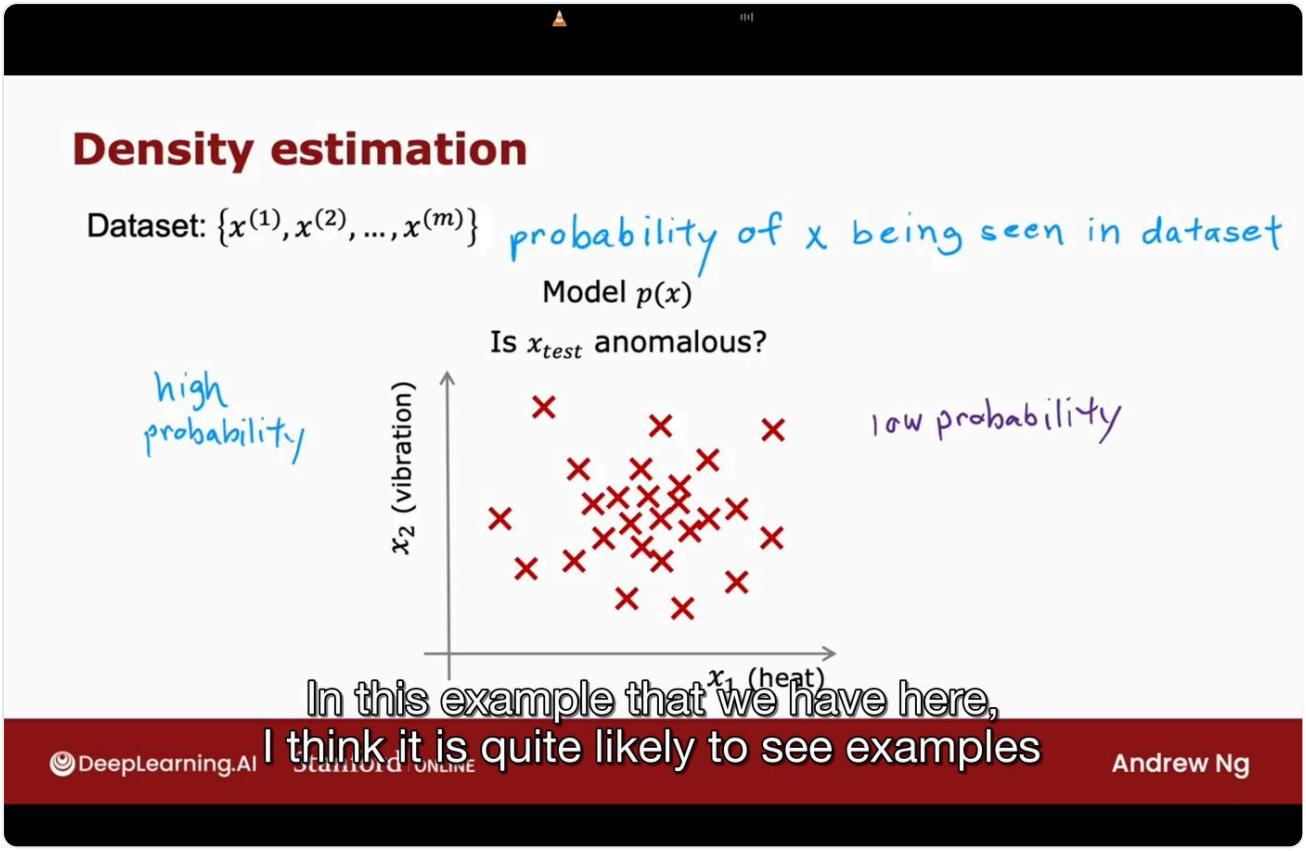


FIGURE 8.4 | Density Estimation

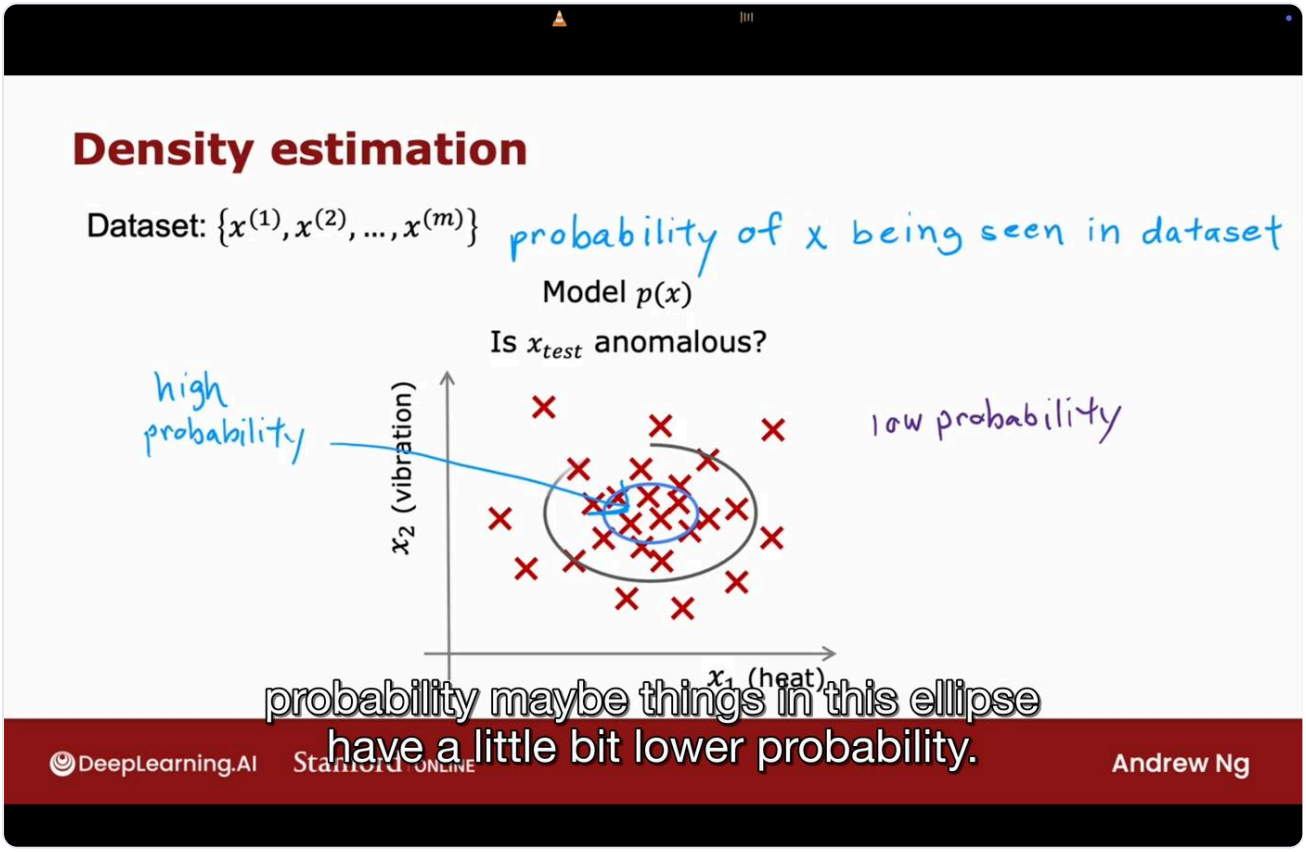


FIGURE 8.5 | Density Estimation

In this example that we have here, I think it is quite likely to see examples in that little ellipse in the middle, so that region in the middle would have high probability maybe things in this ellipse have a little bit lower probability.

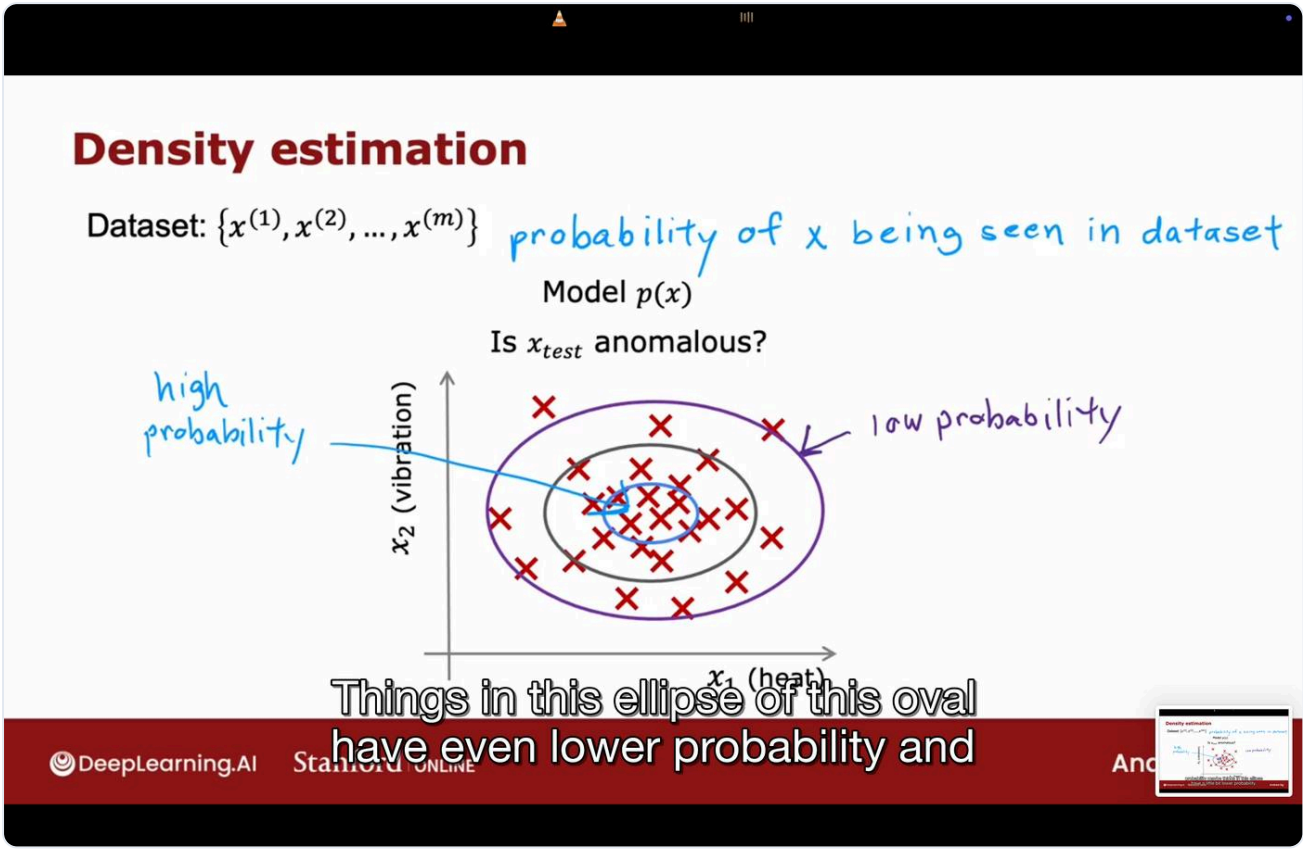


FIGURE 8.6 | Density Estimation

Things in this ellipse of this oval have even lower probability and things outside have even lower probability. The details of how you decide from the training set what regions are higher versus lower probability is something we'll see in the next few videos. And having modeled or having learned to model for p of x When you are given the new test example Xtest. What you will do is then compute the probability of Xtest. And if it is small or more precisely, if it is less than some small number that I'm going to call epsilon, this is a greek alphabet epsilon.

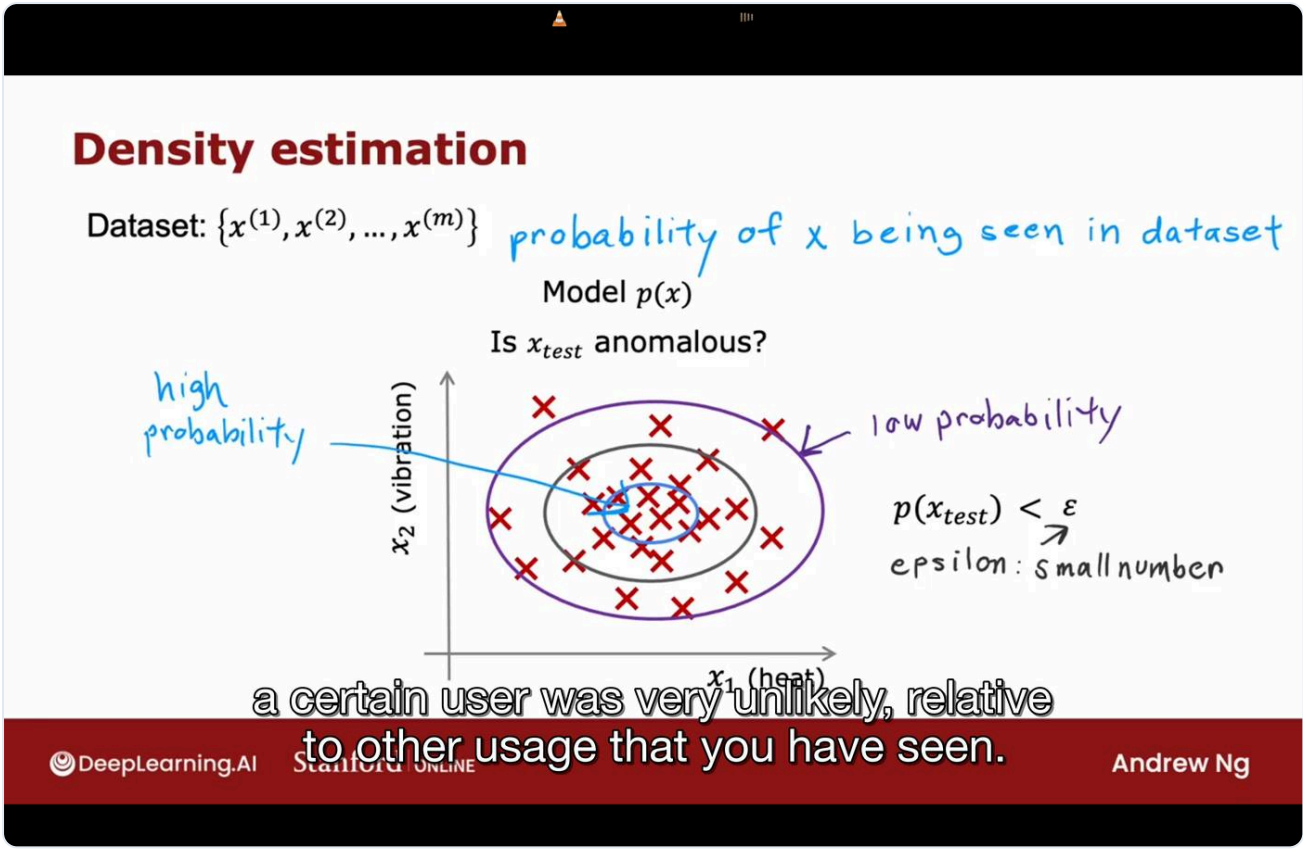


FIGURE 8.7 | Density Estimation

So what you should think of as a small number, which means that p of x is very small or in other words, the specific value of x that you saw for a certain user was very unlikely, relative to other usage that you have seen.

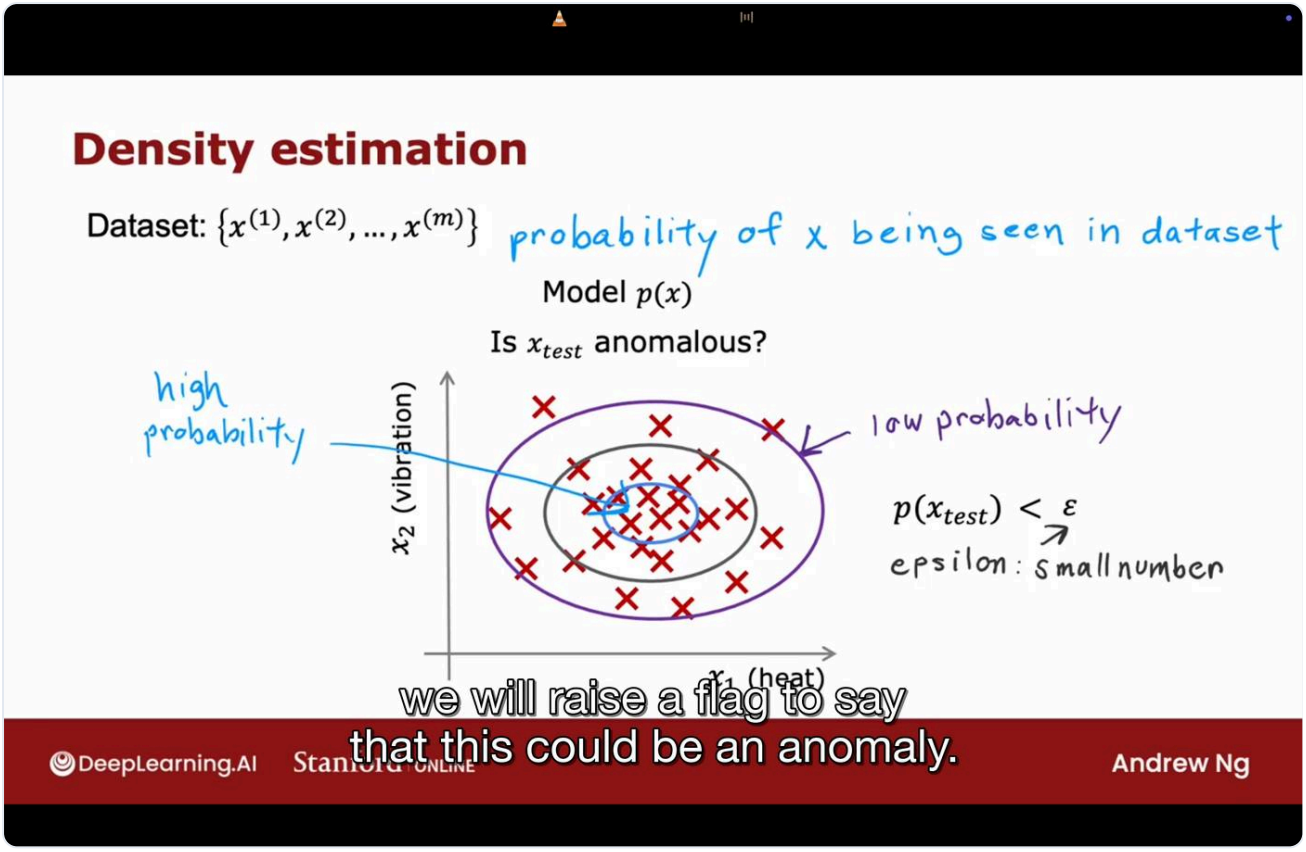


FIGURE 8.8 | Density Estimation

But the p of Xtest is less than some small threshold or some small number epsilon, we will raise a flag to say that this could be an anomaly.

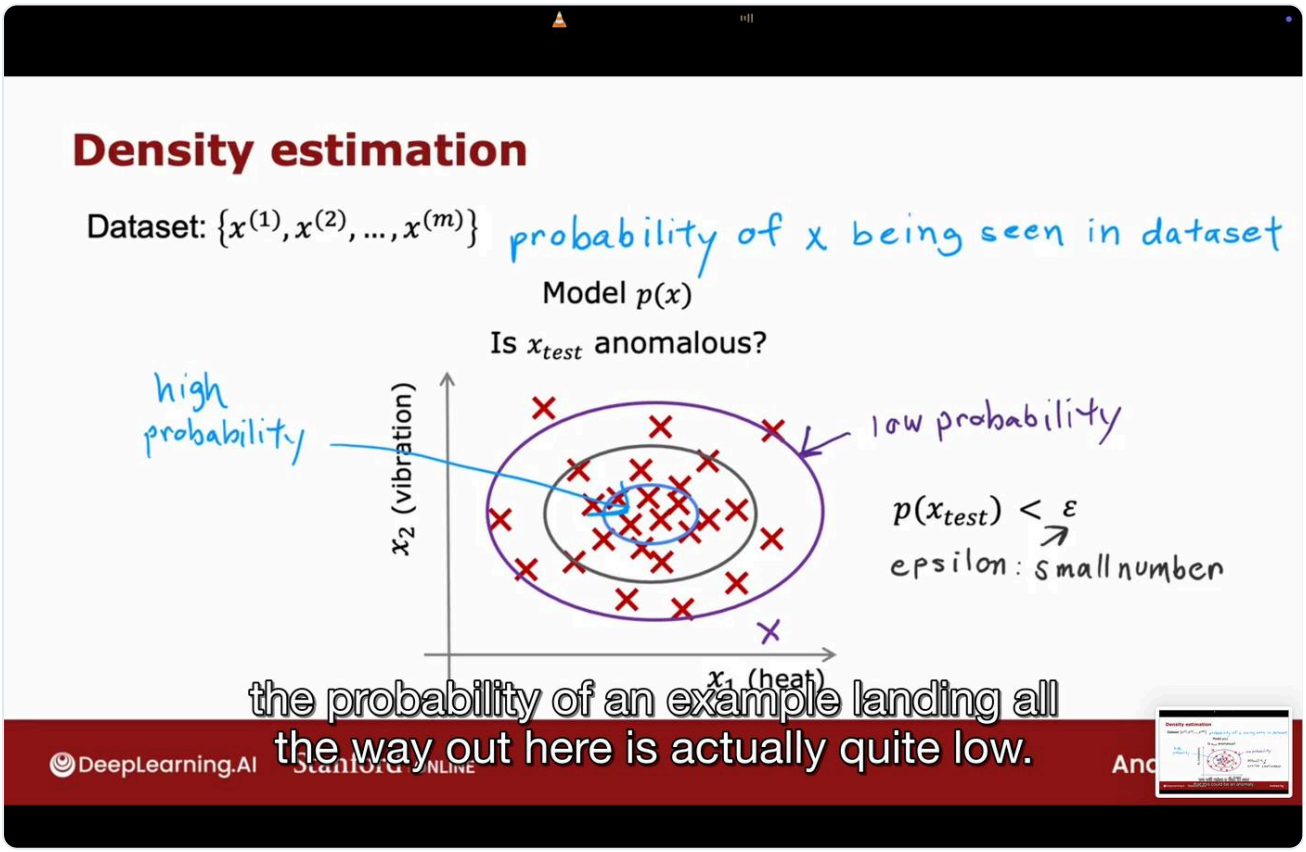


FIGURE 8.9 | Density Estimation

So for example, if Xtest was all the way down here, the probability of an example landing all the way out here is actually quite low.

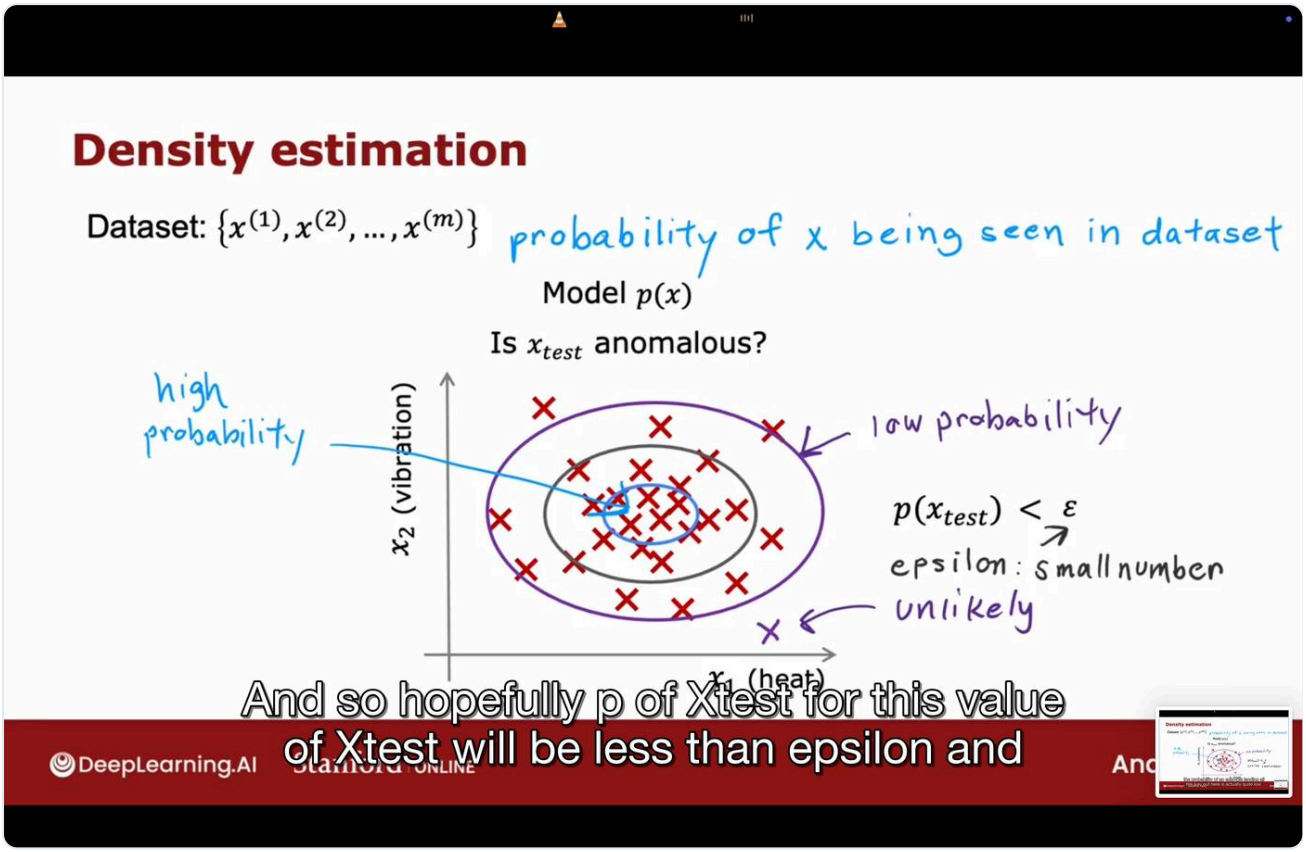


FIGURE 8.10 | Density Estimation

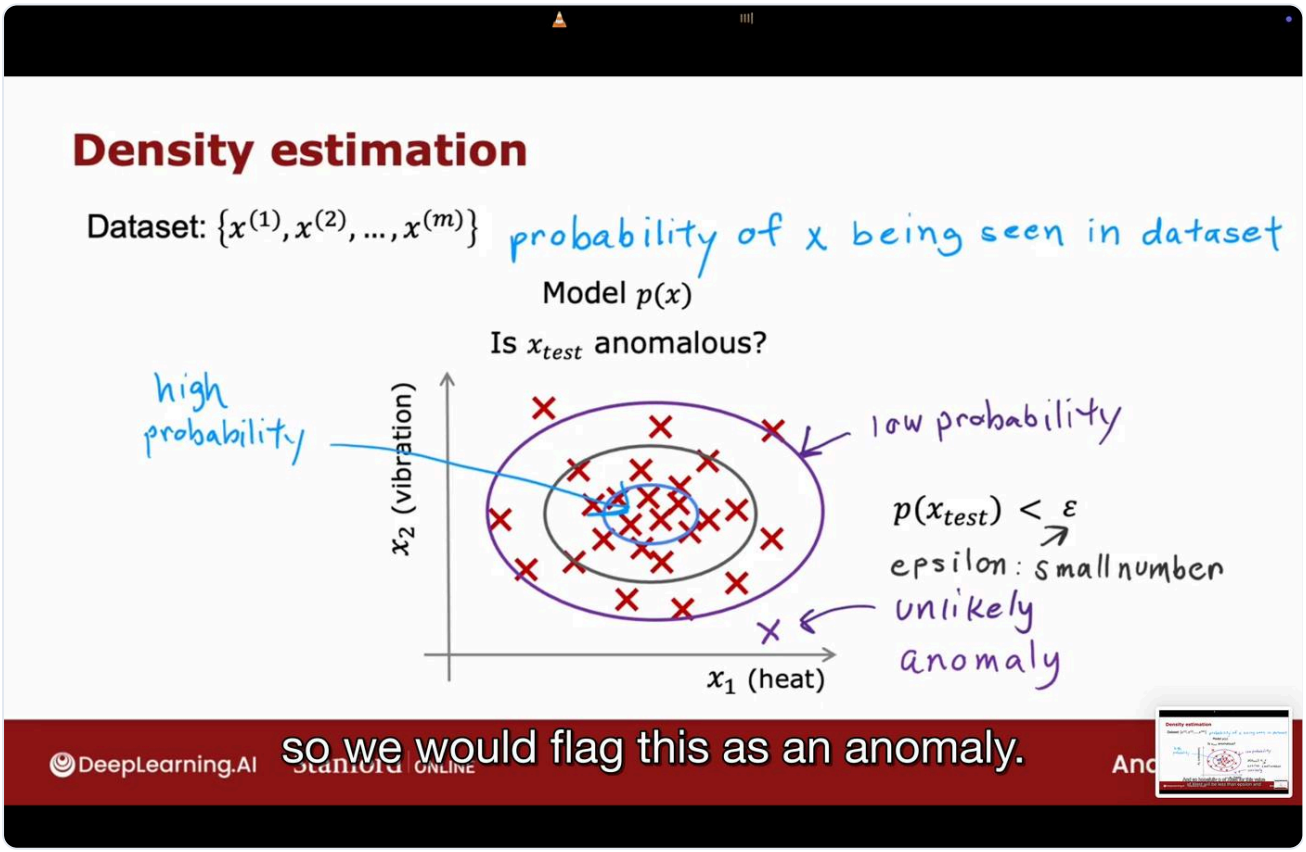


FIGURE 8.11 | Density Estimation

And so hopefully p of Xtest for this value of Xtest will be less than epsilon and so we would flag this as an anomaly. Whereas in contrast, if p of Xtest is not less than epsilon, if p of Xtest is greater than equal to epsilon, then we will say that it looks okay, this doesn't look like an anomaly.

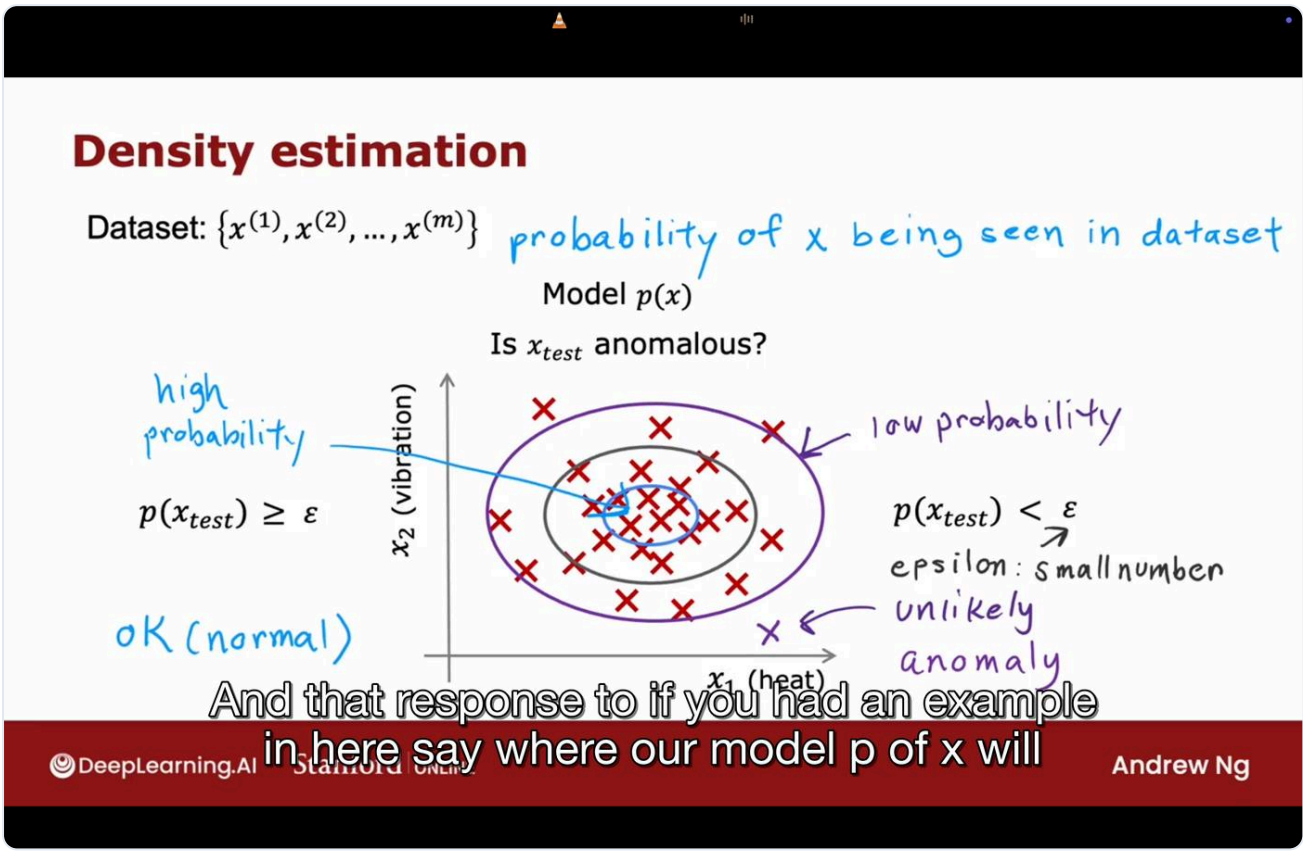


FIGURE 8.12 | Density Estimation

And that response to if you had an example in here say where our model p of x will say that examples near the middle here, they're actually quite high probability. There's a very high chance that the new airplane engine will have features close to these inner ellipses.

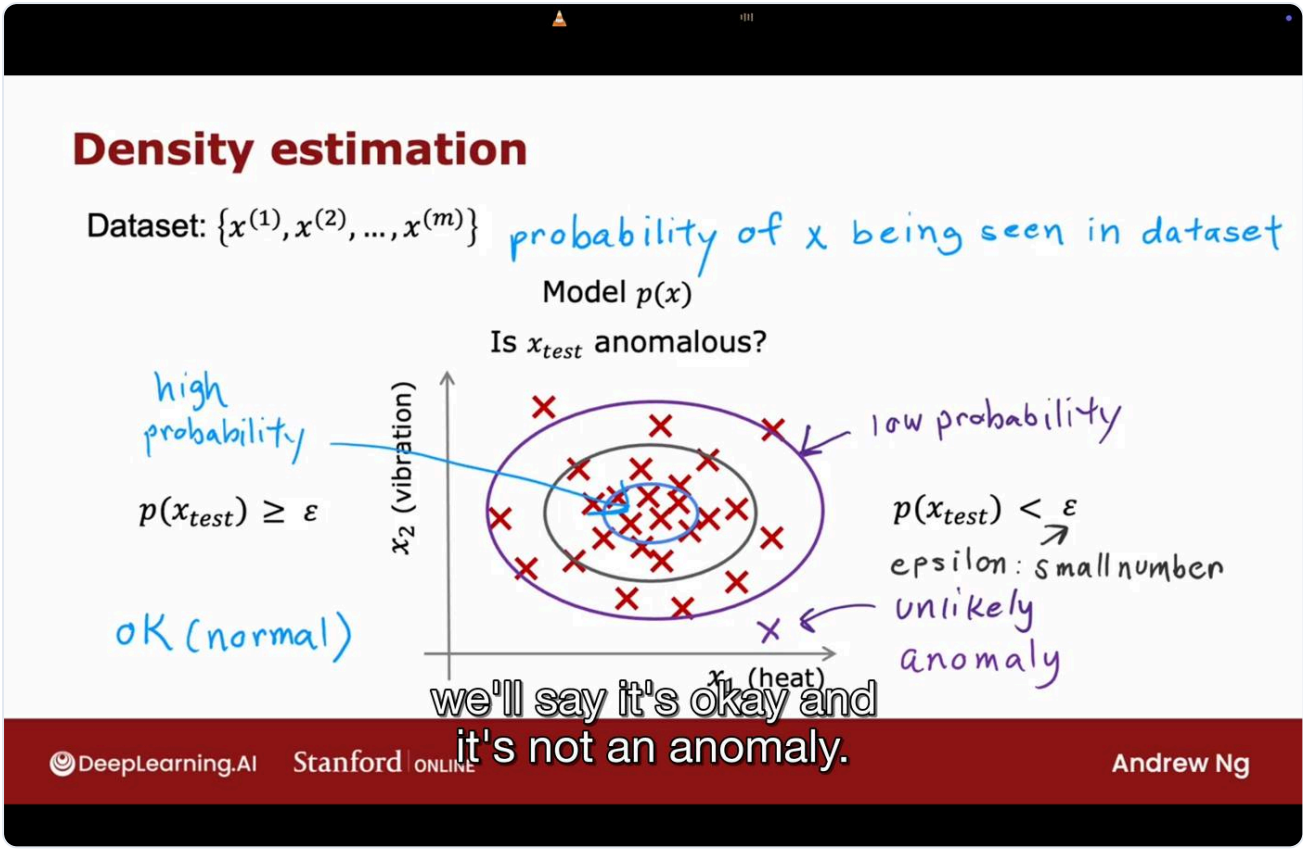


FIGURE 8.13 | Density Estimation

And so p of Xtest will be large for those examples and we'll say it's okay and it's not an anomaly. Anomaly detection is used today in many applications. It is frequently used in fraud detection where for example if you are running a website with many different features. If you compute xi to be the features of user i's activities. And examples of features might include, how often does this user login and how many web pages do they visit? How many transactions are they making or how many posts on the discussion forum are they making to what is their typing speed? How many characters per second do they seem able to type.

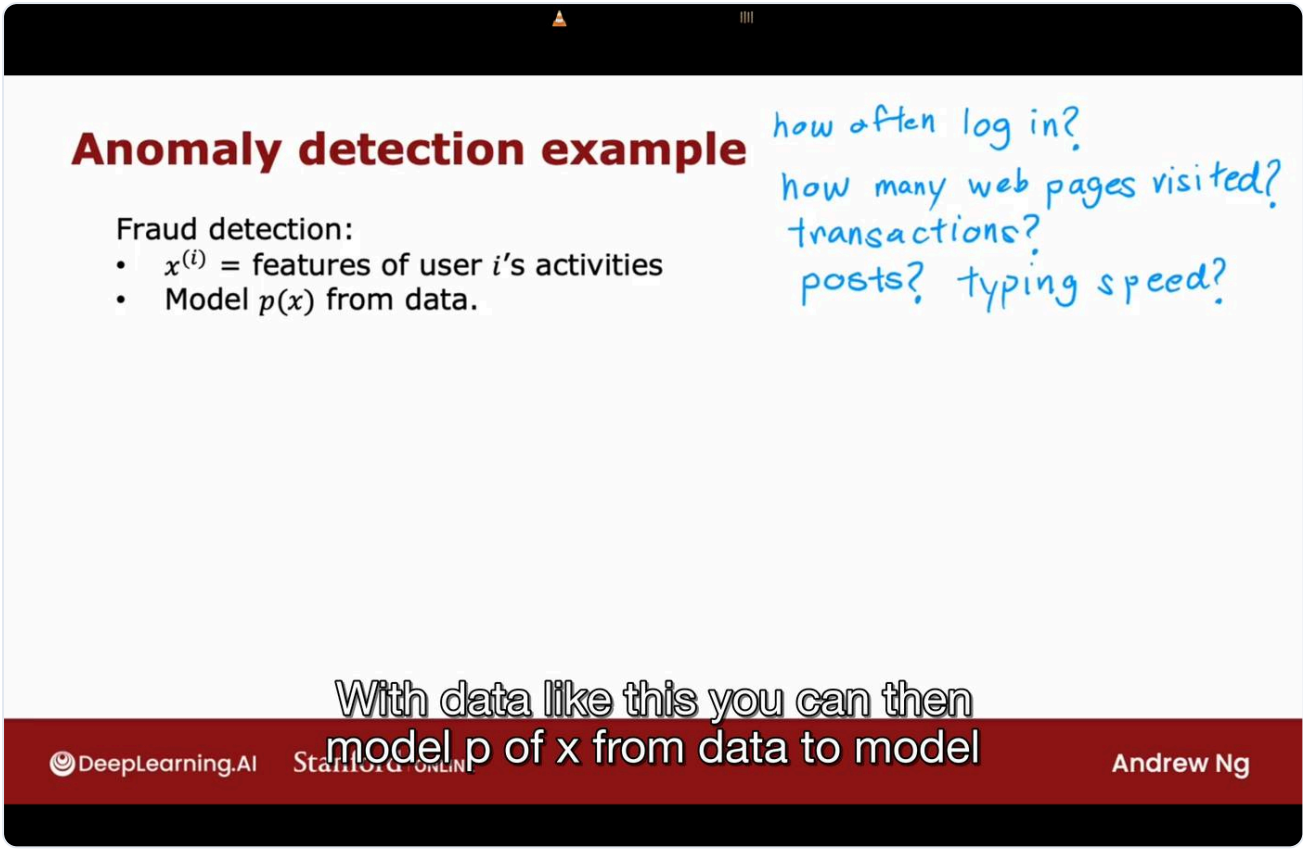


FIGURE 8.14 | 7 How Often Ioq in?

With data like this you can then model p of x from data to model what is the typical behavior of a given user. In the common workflow of fraud detection, you wouldn't automatically turn off an account just because it seemed anomalous. But instead you may ask the security team to take a closer look or put in some additional security checks such as ask the user to verify their identity with a cell phone number or ask them to pass a capture to prove that they're human and so on.

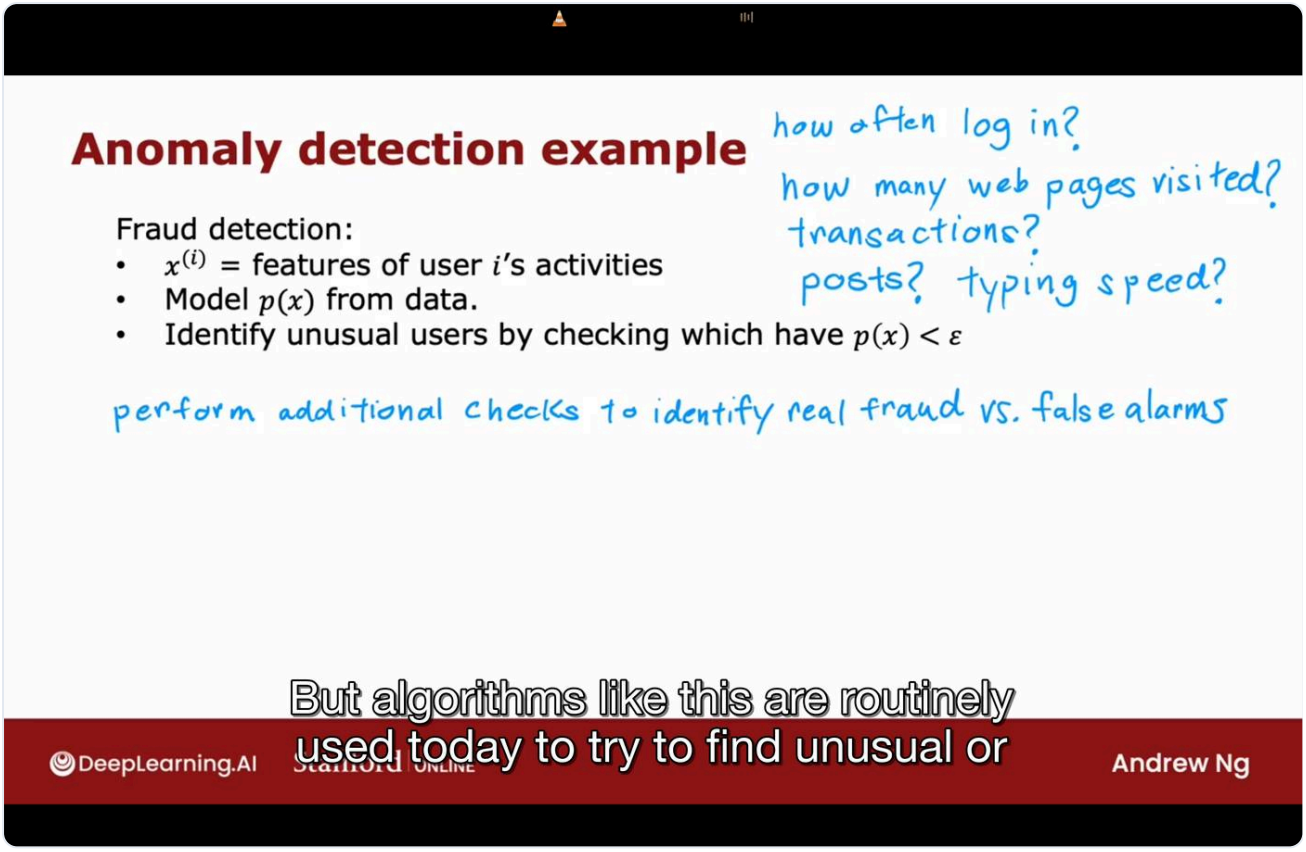


FIGURE 8.15 | Anomaly Detection Example Gu

But algorithms like this are routinely used today to try to find unusual or maybe slightly suspicious activity. So you can more carefully screen those accounts to make sure there isn't something fraudulent. And this type of fraud detection is used both to find fake accounts and this type of algorithm is also used frequently to try to identify financial fraud such as if there's a very unusual pattern of purchases. Then that may be something well worth a security team taking a more careful look at. Anomaly detection is also frequently used in manufacturing. You saw an example on the previous slide with aircraft engine manufacturing. But many manufacturers in multiple continents in many, many factories were routinely use anomaly detection to see if whatever they just manufactured.

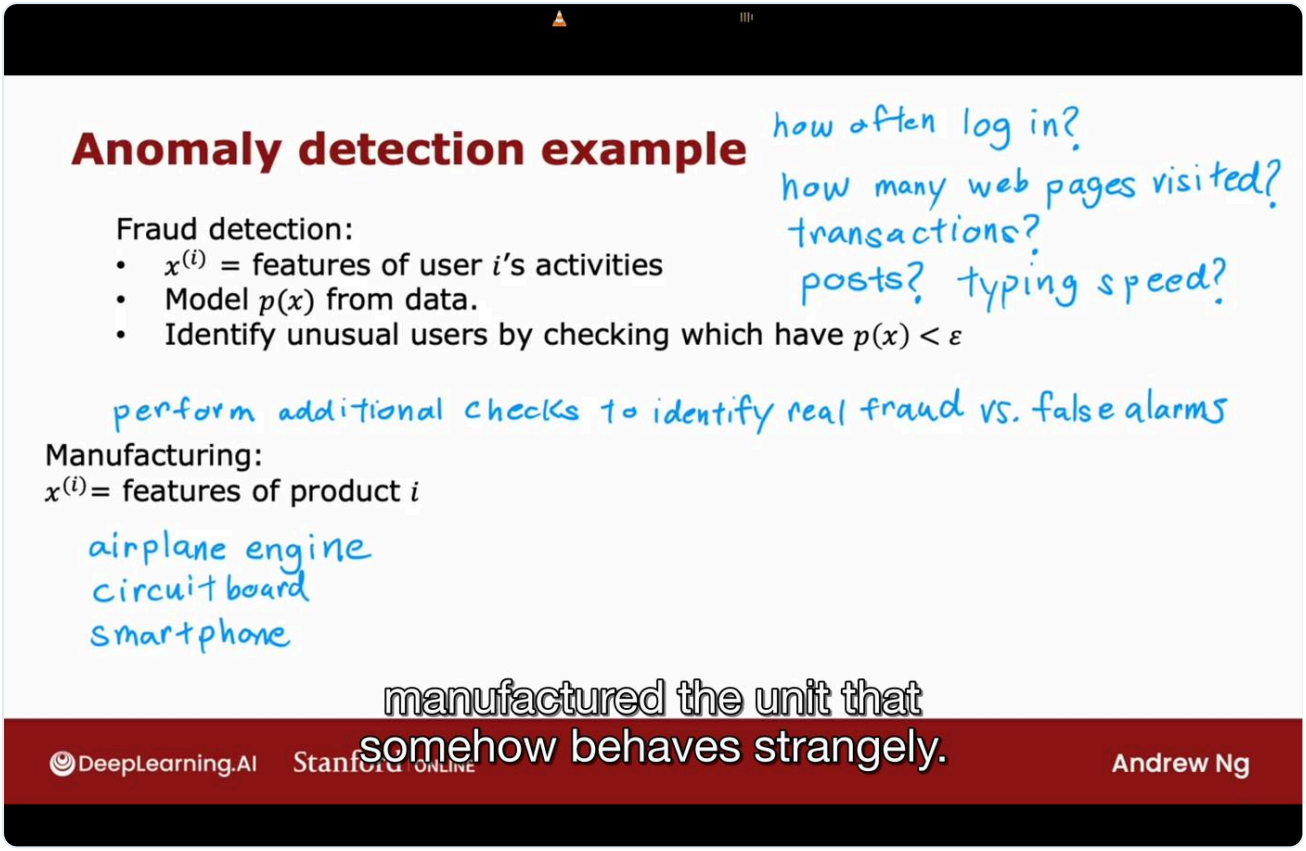


FIGURE 8.16 | Anomaly Detection Example Go

Anything from an airplane engine to a printed circuit board to a smartphone to a motor, to many, many things to see if you've just manufactured the unit that somehow behaves strangely. Because that may indicate that there's something wrong with your airplane engine or printed circuit boards or what have you that might cause you to want to take a more careful look before you ship that object to the customer. Is also used to monitor computers in clusters and in data centers where if X I are the features of a certain machine I such as if the features captured the memory users, the number of disk accesses per second. CPU load features can also be ratios, such as the ratio of CPU load to network traffic.

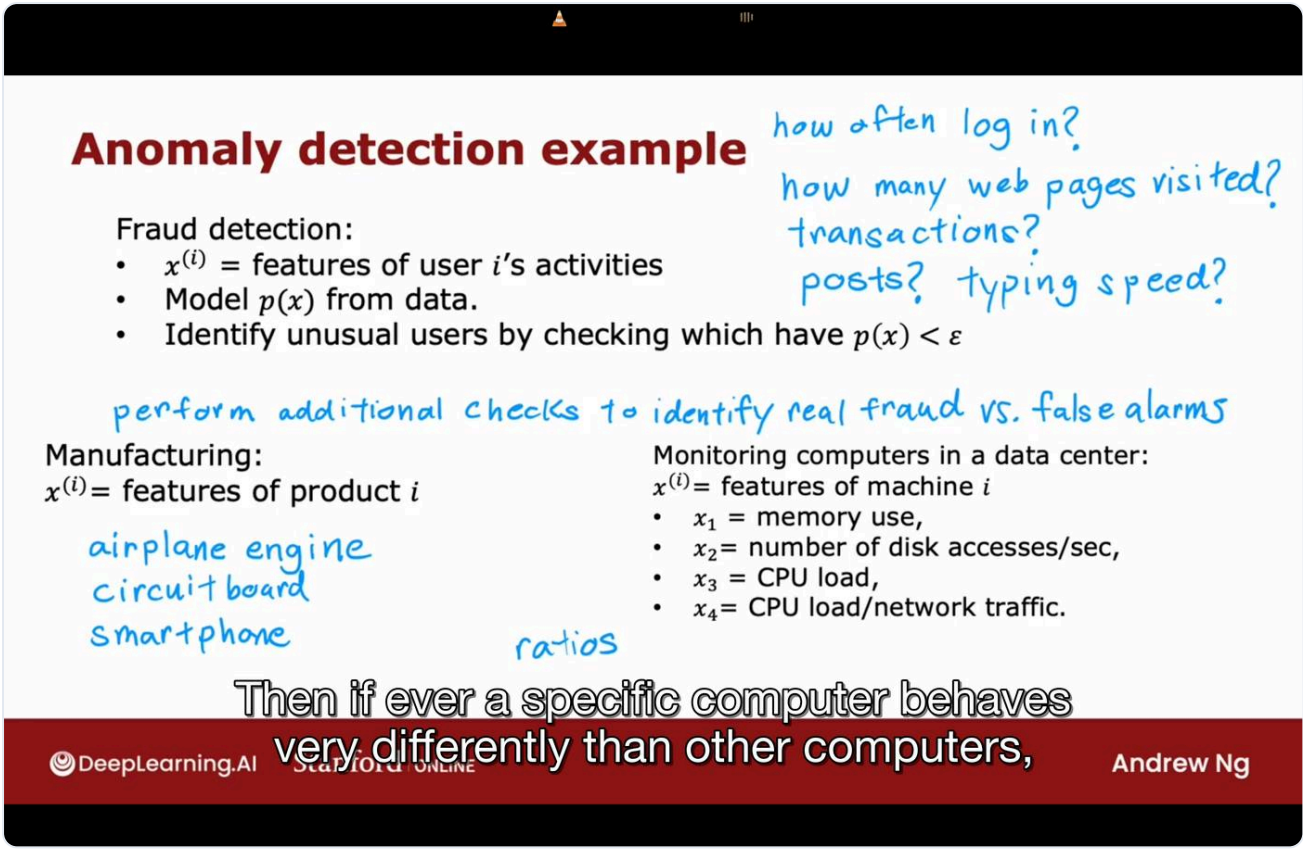


FIGURE 8.17 | Anomaly Detection Example Gu

Then if ever a specific computer behaves very different than other computers, it might be worth taking a look at that computer to see if something is wrong with it. Such as if it has had a hard disk failure or network card failure or something's wrong with it or if maybe it has been hacked into. Anomaly detection is one of those algorithms that is very widely used even though you don't seem to hear people talk about it that much. I remember the first time I worked on the commercial application of anomaly detection was when I was helping a telco company put in place anomaly detection to see when any one of the cell towers was behaving in an unusual way.

Because that probably meant there was something wrong with the cell tower and so they want to get a technician to take so hopefully that helped more people get good cell phone coverage. And I've also used anomaly detection to find fraudulent financial transactions And these days I often use it to help manufacturing companies find anomalous parts that they may have manufactured but should inspect more often. So it is a very useful tool to have in your tool chest. And in the next few videos we'll talk about how you can build and get these algorithms to work for yourself. In order to get anonymous detection algorithms to work, we'll need to use a Gaussian distribution to model the data p of x.

So, let's go on to the next video to talk about Gaussian distributions.

LECTURE 9

Gaussian Distribution Example

In order to apply anomaly detection, we're going to need to use the Gaussian distribution, which is also called the normal distribution. When you hear me say either Gaussian distribution or normal distribution, they mean exactly the same thing. If you've heard the bell-shaped distribution, that also refers to the same thing. But if you haven't heard of the bell-shaped distribution, that's fine too. But let's take a look at what is the Gaussian or the normal distribution. Say x is a number, and if x is a random number, sometimes called the random variable, x can take on random values. If the probability of x is given by a Gaussian or normal distribution with mean parameter Mu, and with variance Sigma squared.

What that means is that the probability of x looks like a curve that goes like this. The center or the middle of the curve is given by the mean Mu, and the standard deviation or the width of this curve is given by that variance parameter Sigma. Technically, Sigma is called the standard deviation and the square of Sigma or Sigma squared is called the variance of the distribution. This curve here shows what is p of x or the probability of x.

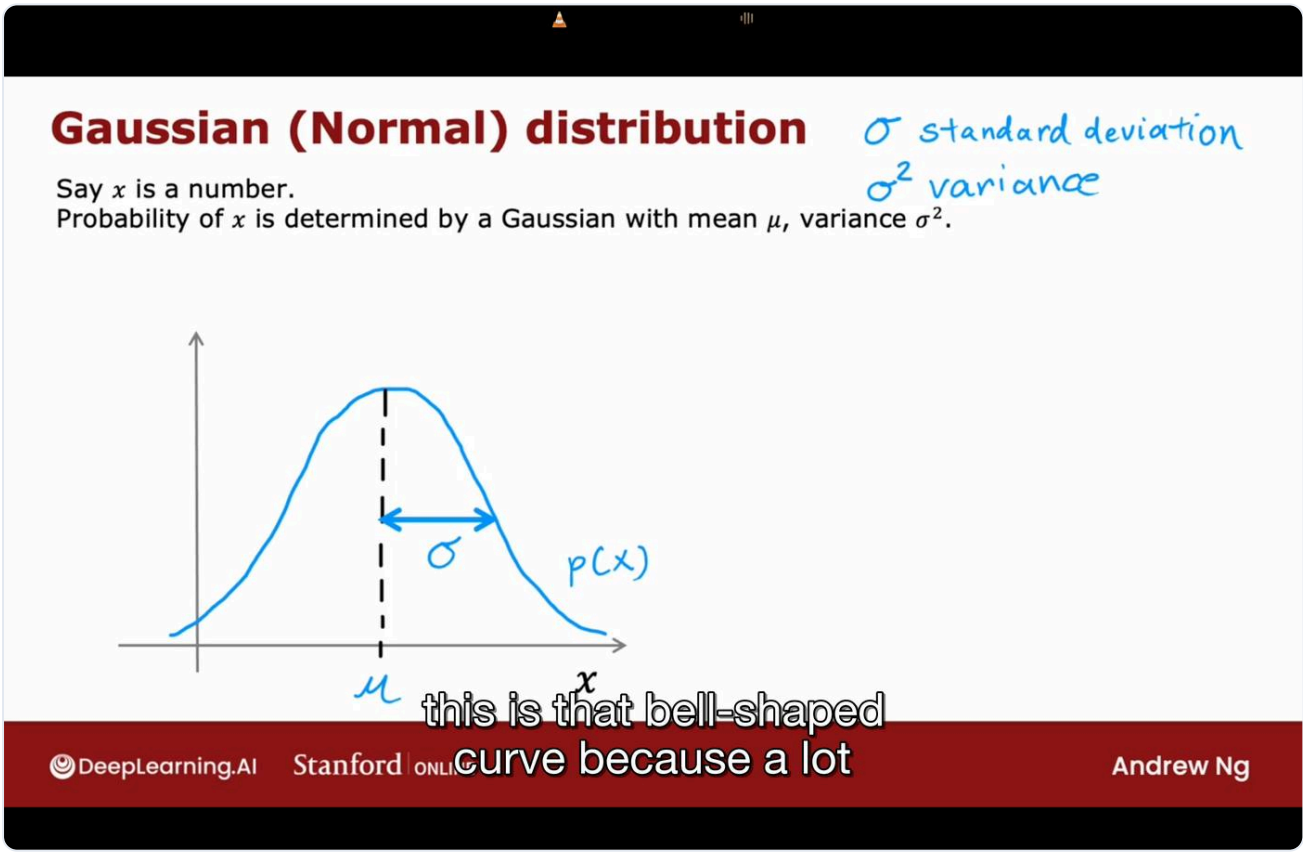


FIGURE 9.1

If you've heard of the bell-shaped curve, this is that bell-shaped curve because a lot of classic bells say in towers were shaped like this with the bell clapper hanging down here, and so the shape of this curve is vaguely reminiscent of the shape of the large bells that you will still find in some old buildings today. Better looking than my hand-drawn one. There's a picture of the

Liberty Bell. Indeed, the Liberty Bell's shape on top is vaguely bell-shaped curve. If you're wondering what does p of x really means? Here's one way to interpret it. It means that if you were to get, say, 100 numbers drawn from this probability distribution, and you were to plot a histogram of these 100 numbers drawn from this distribution, you might get a histogram that looks like this.

It looks vaguely bell-shaped.

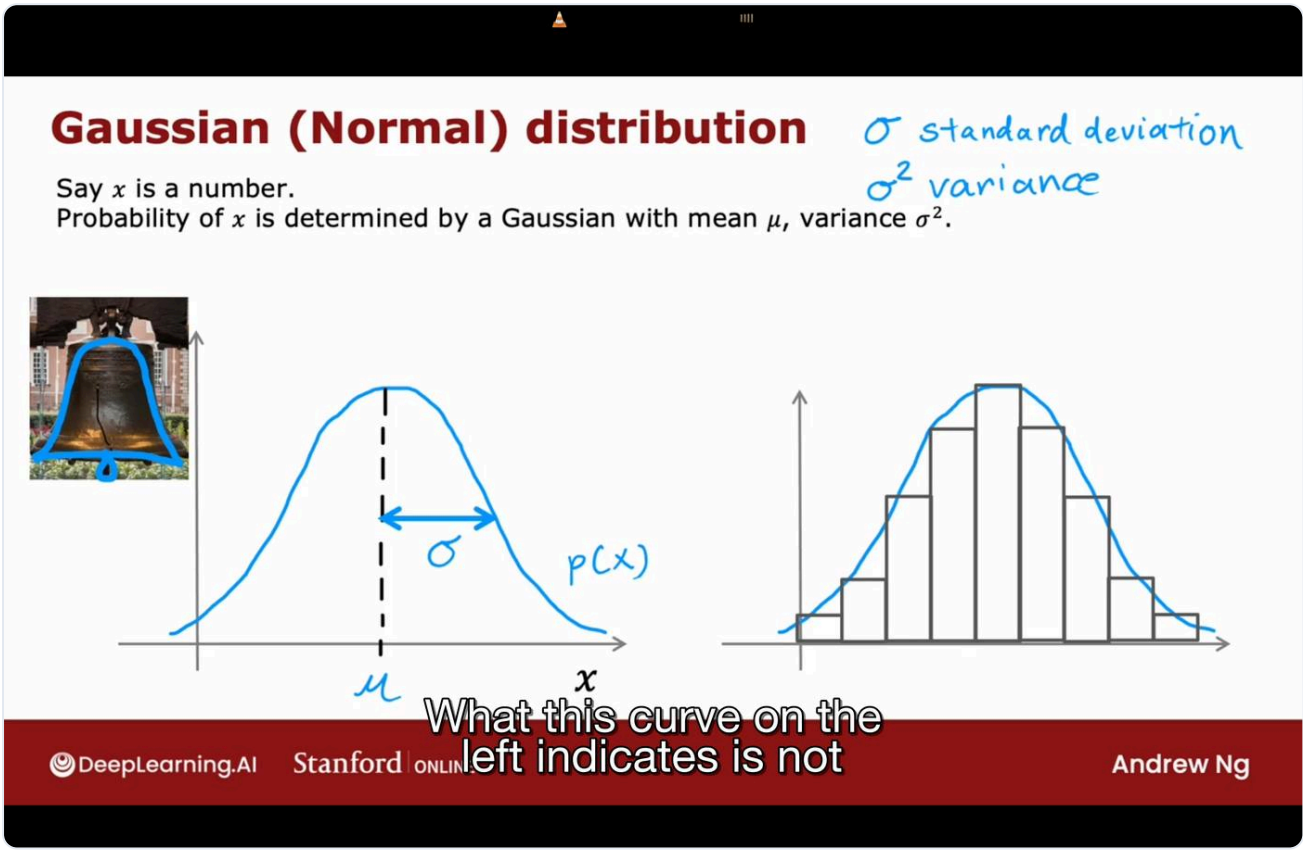


FIGURE 9.2 | Gaussian (Normal) Distribution 0 Standard Devioction

What this curve on the left indicates is not if you have just 100 examples or 1,000 or a million or a billion. But if you had a practically infinite number of examples, and you were to draw a histogram of this practically infinite number of examples with a very fine histogram bin.

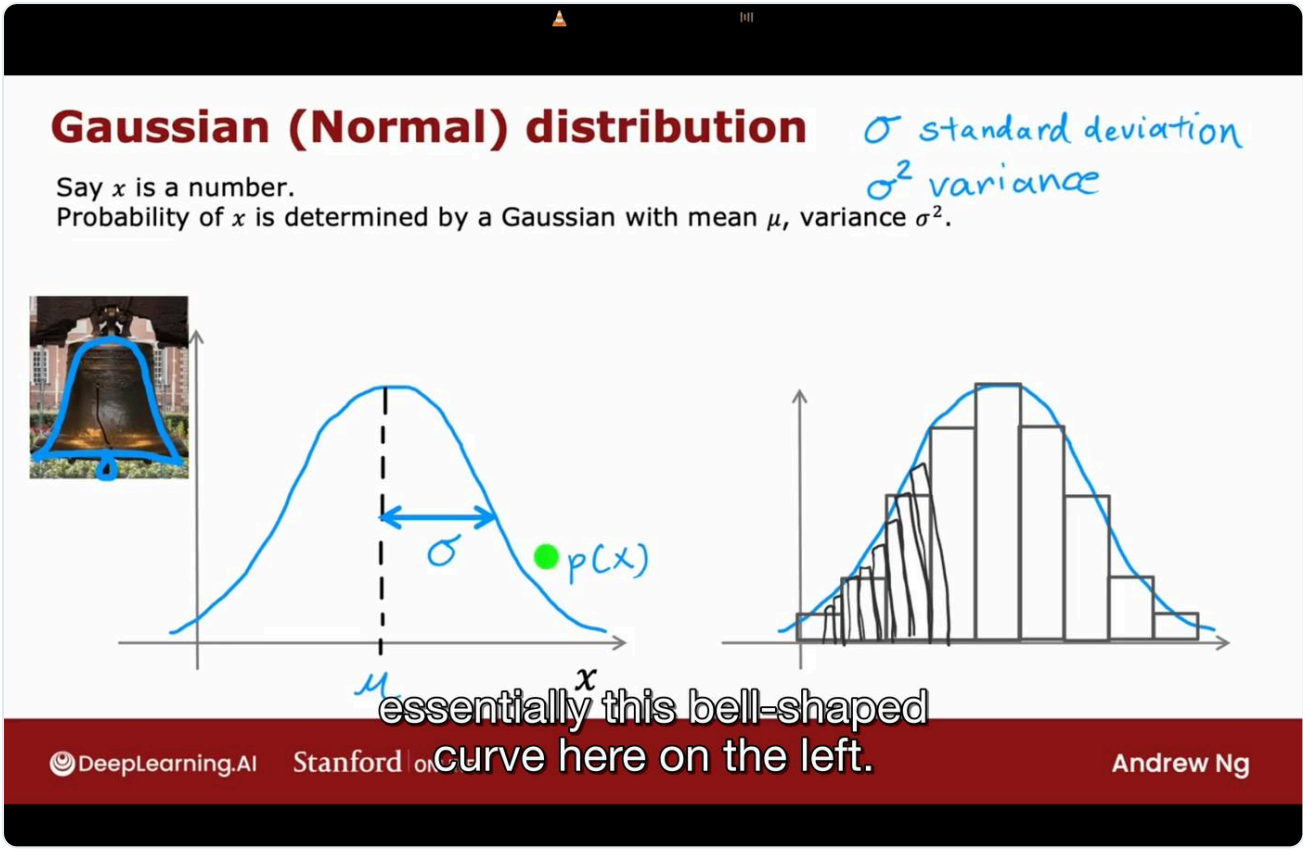


FIGURE 9.3 | Gaussian (Normal) Distribution 0 Standard Deviation

Then you end up with essentially this bell-shaped curve here on the left. The formula for p of x is given by this expression; p of x equals 1 over square root 2 Pi. Pi here is that 3.14159 or it's about 22 over 7.

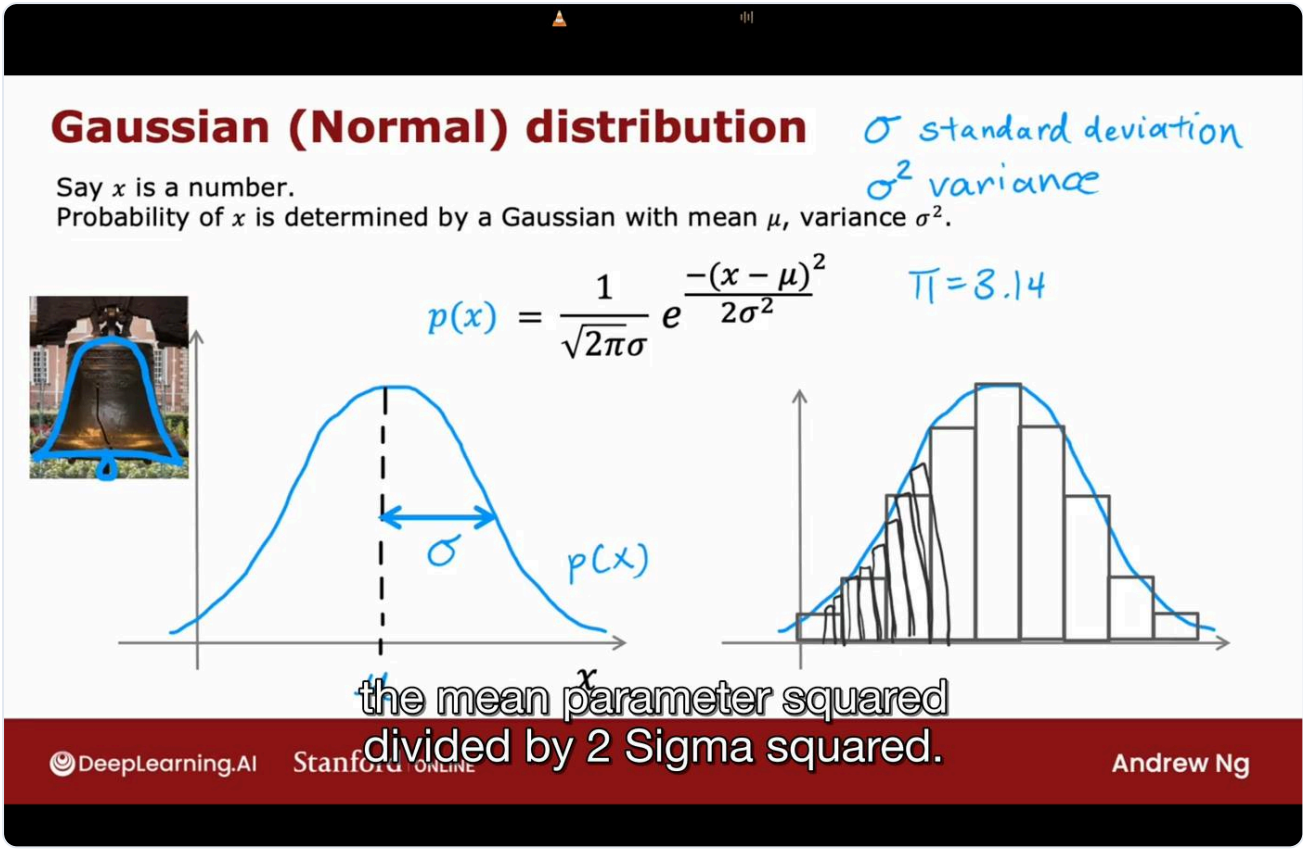


FIGURE 9.4 | Gaussian (Normal) Distribution 0 Standard Deviation

Ratio of a circle's diameter circumference times Sigma times e to the negative x minus Mu, the mean parameter squared divided by 2 Sigma squared. For any given value of Mu and Sigma, if you were to plot this function as a function of x, you get this type of bell-shaped curve that is centered at Mu, and with the width of this bell-shaped curve being determined by the parameter Sigma. Now let's look at a few examples of how changing Mu and Sigma will affect the Gaussian distribution. First, let me set Mu equals 0 and Sigma equals 1. Here's my plot of the Gaussian distribution with mean 0, Mu equals 0, and standard deviation Sigma equals 1. You notice that this distribution is centered at zero and that is the standard deviation Sigma is equal to 1.

Now, let's reduce the standard deviation Sigma to 0.5. If you plot the Gaussian distribution with Mu equals 0 and Sigma equals 0.5, it now it looks like this. Notice that it's still centered at zero because Mu is zero. But it's become a much thinner curve because Sigma is now 0.5. You might recall that Sigma is the standard deviation is 0.5, whereas Sigma squared is also called the variance. That's equal to 0.5 squared or 0.25.

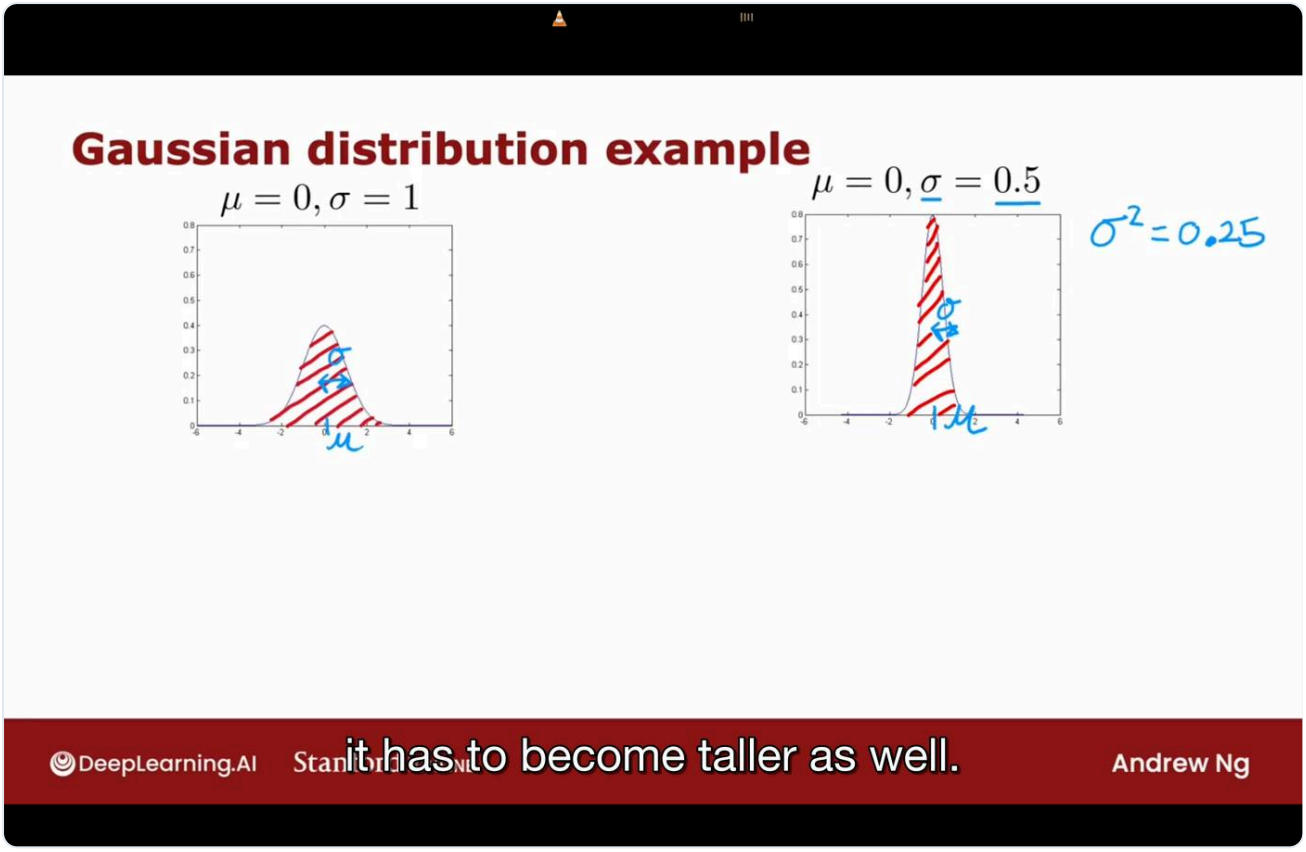


FIGURE 9.5 | Gaussian Distribution Example

You may have heard that probabilities always have to sum up to one, so that's why the area under the curve is always equal to one, which is why when the Gaussian distribution becomes skinnier, it has to become taller as well. Let's look at another value of Mu and Sigma. Now, I'm going to increase Sigma to 2, so the standard deviation is 2 and the variance is 4. This now creates a much wider distribution because Sigma here is now much larger, and because it's now a wider distribution is become shorter as well because the area under the curve is still equals 1. Finally, let's try changing the mean parameter Mu, and I'll leave Sigma equals 0.5. In this case, the center of the distribution Mu moves over here to the right.

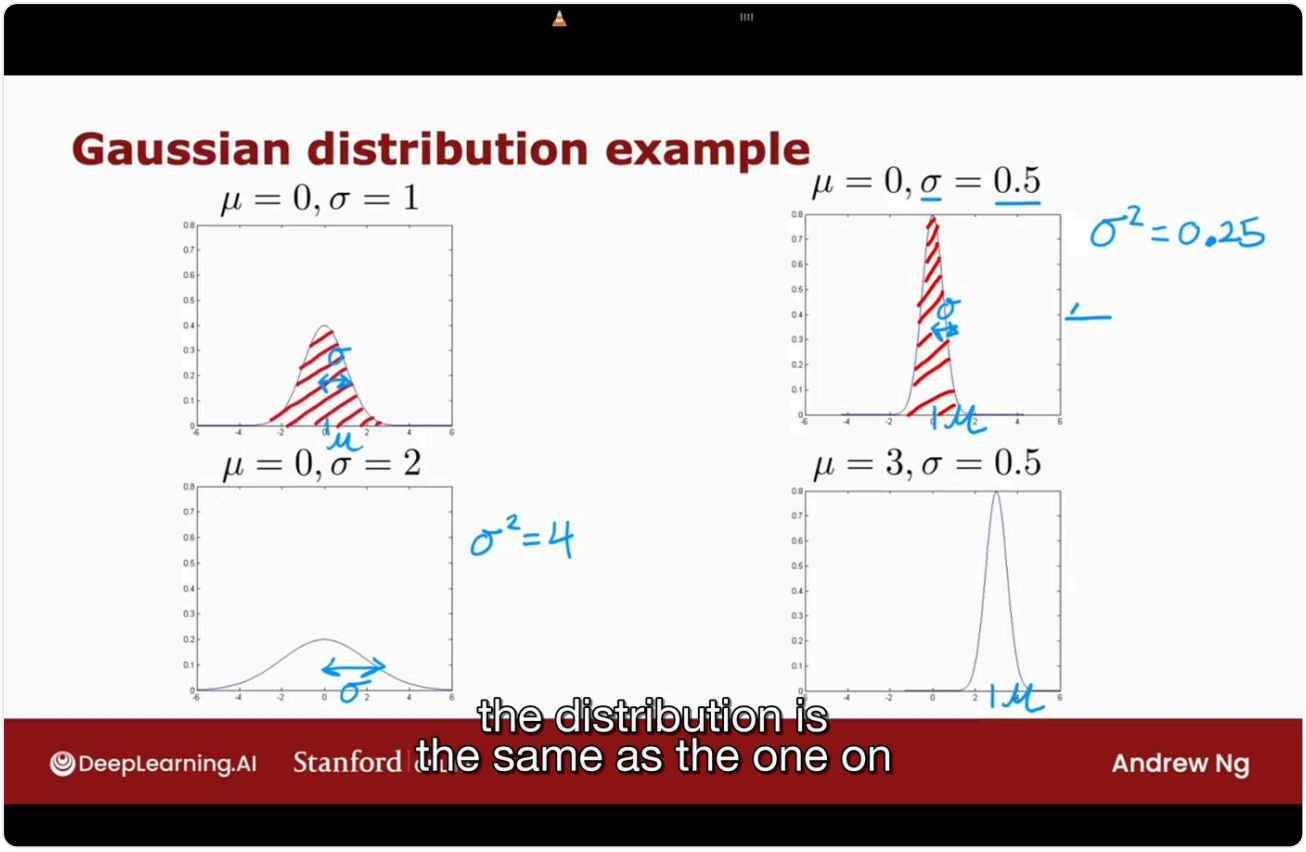


FIGURE 9.6 | Gaussian Distribution Example

But the width of the distribution is the same as the one on top because the standard deviation is 0.5 in both of these cases on the right. This is how different choices of Mu and Sigma affect the Gaussian distribution. When you're applying this to anomaly detection, here's what you have to do. You are given a dataset of m examples, and here x is just a number. Here, are plots of the training sets with 11 examples. What we have to do is try to estimate what a good choice is for the mean parameter Mu, as well as for the variance parameter Sigma squared. Given a dataset like this, it would seem that a Gaussian distribution maybe looking like that with a center here and a standard deviation like that.

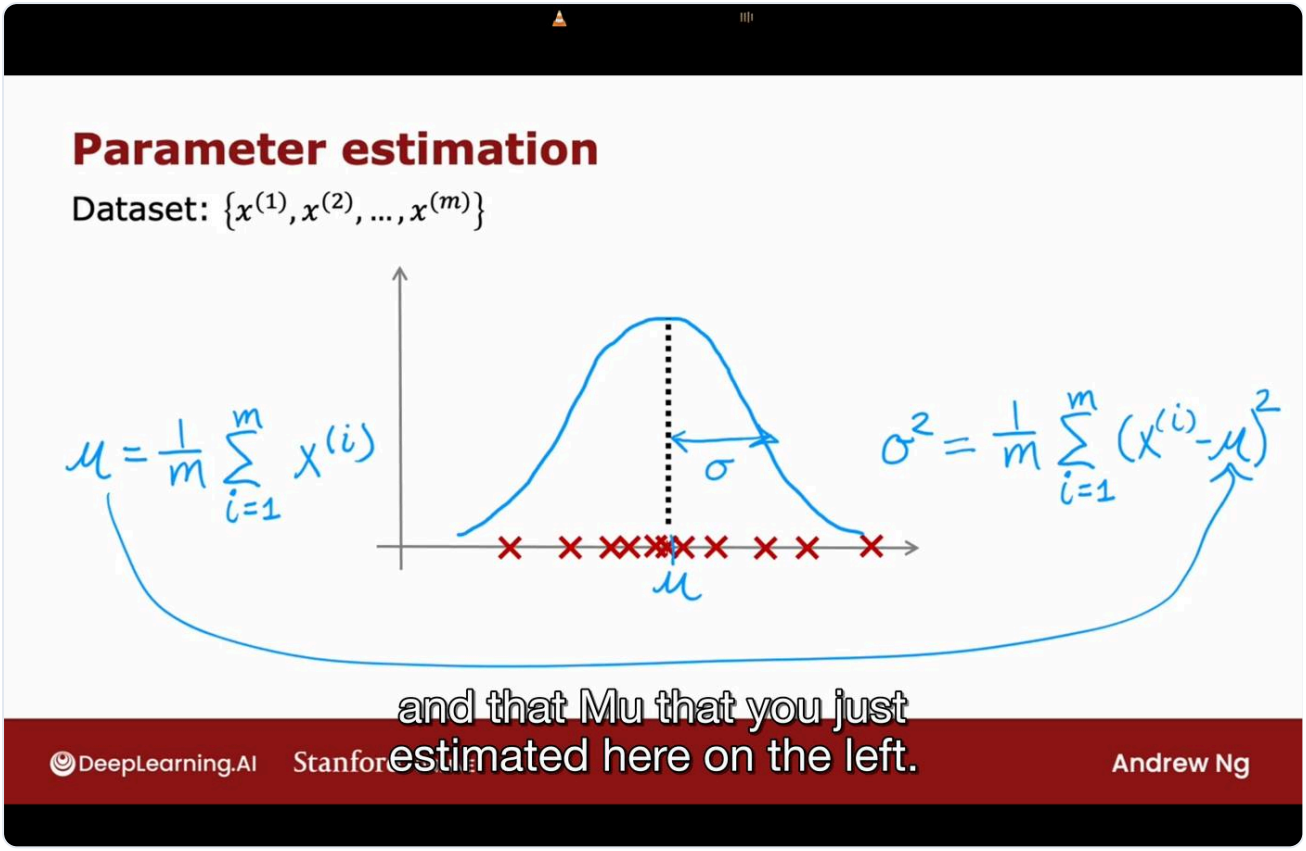


FIGURE 9.7

This might be a pretty good fit to the data. The way you would compute Mu and Sigma squared mathematically is our estimate for Mu will be just the average of all the training examples. It's 1 over m times sum from i equals 1 through m of the values of your training examples. The value we will use to estimate Sigma squared will be the average of the squared difference between two examples, and that Mu that you just estimated here on the left. It turns out that if you implement these two formulas in code with this value for Mu and this value for Sigma squared, then you pretty much get the Gaussian distribution that I hand drew on top. This will give you a choice of Mu and Sigma for a Gaussian distribution so that it looks like the 11 training samples might have been drawn from this Gaussian distribution.

If you've taken an advanced statistics class, you may have heard that these formulas for Mu and Sigma squared are technically called the maximum likelihood estimates for Mu and Sigma.

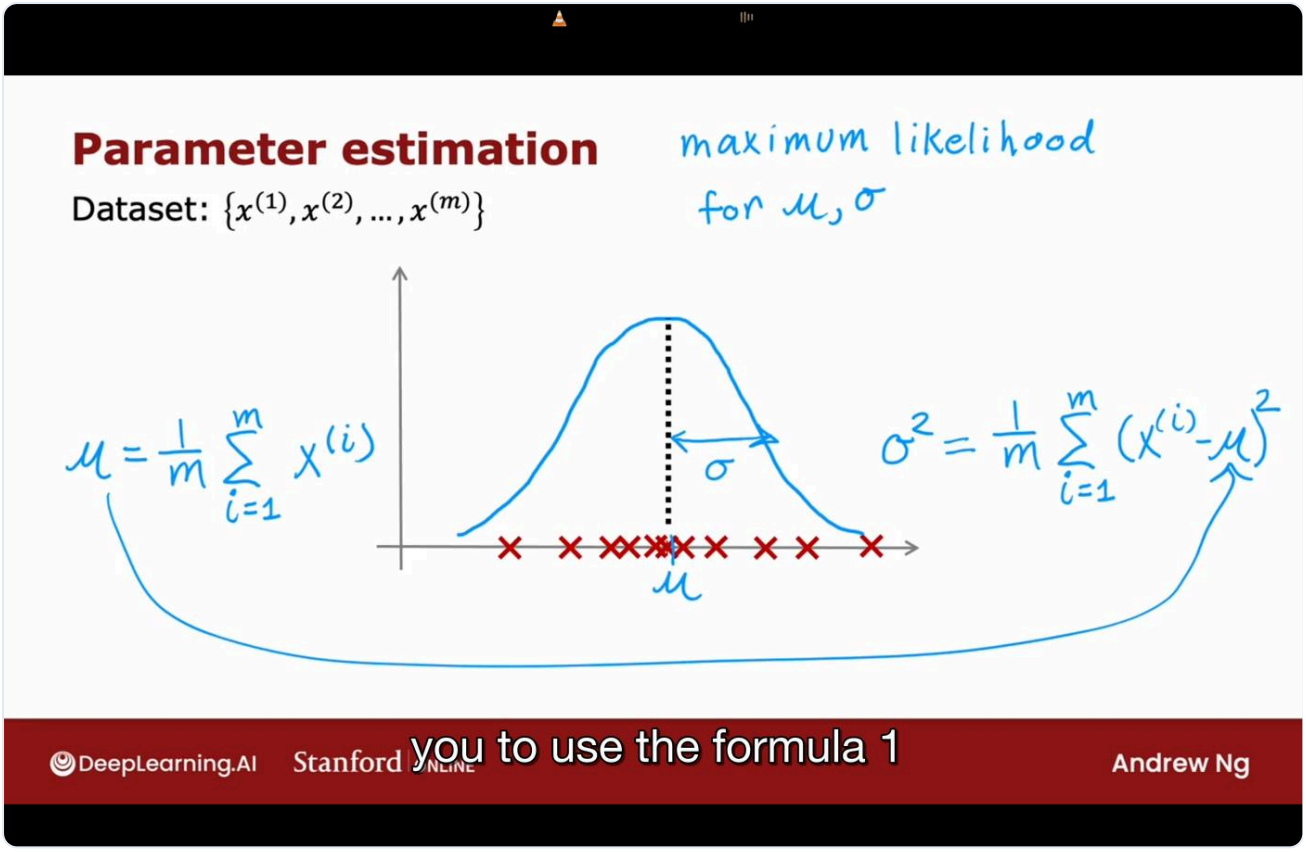


FIGURE 9.8

Some statistics classes will tell you to use the formula 1 over n minus 1 instead of 1 over m.

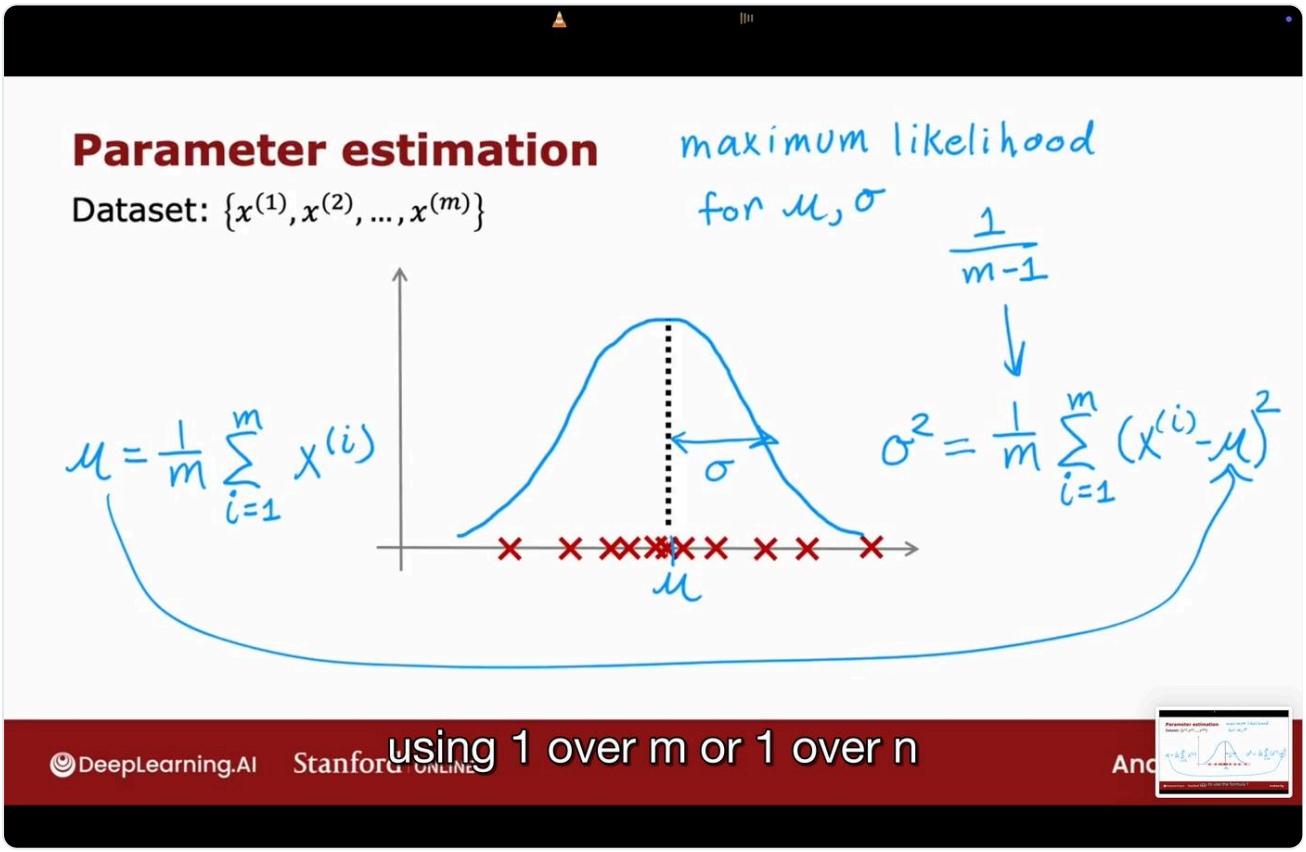


FIGURE 9.9

In practice, using 1 over m or 1 over n minus 1 makes very little difference. I always use 1 over m, but just some other properties of dividing by m minus 1 that some statisticians prefer. But if you don't understand what they just said, don't worry about it. All you need to know is that if you set Mu according to this formula and Sigma squared according to this formula, you'd get a pretty good estimate of Mu and Sigma and in particular, you get a Gaussian distribution that will be a possible probability distribution in terms of what's the probability distribution that the training examples had come from.

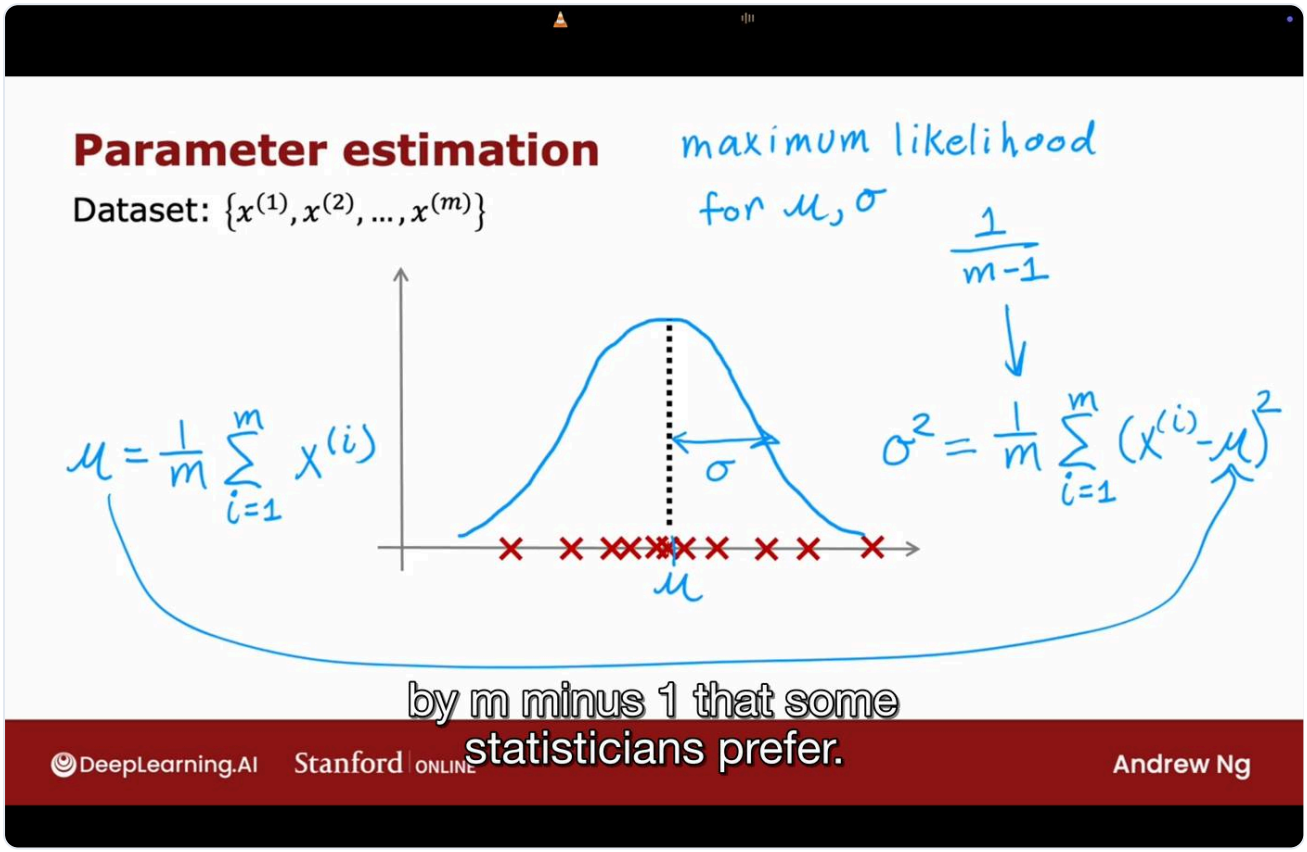


FIGURE 9.10

You can probably guess what comes next. If you were to get an example over here, then p of x is pretty high. Whereas if you were to get an example, we are here, then p of x is pretty low, which is why we would consider this example, okay, not really anomalous, not a lot like the other ones.

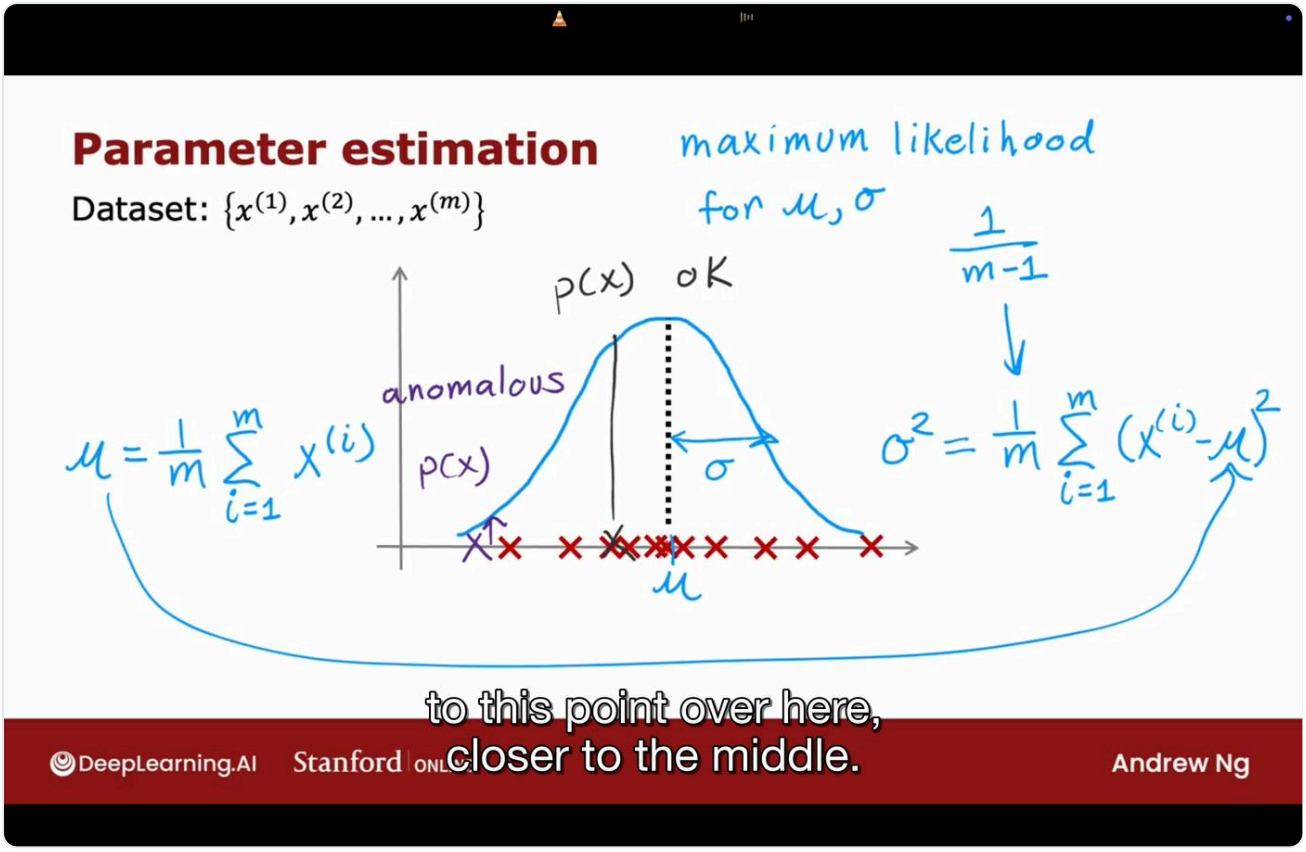


FIGURE 9.11

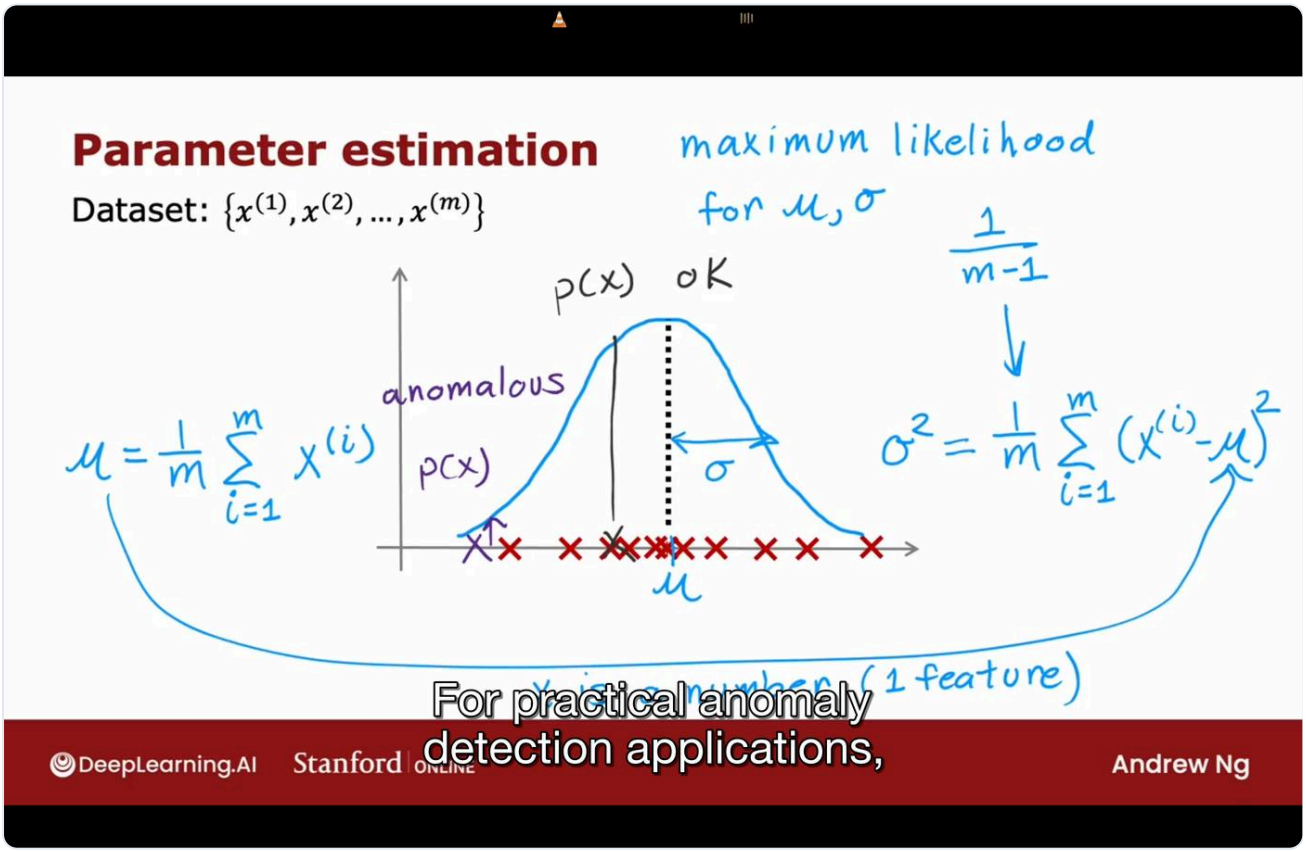


FIGURE 9.12

Whereas an example we are here to be pretty unusual compared to the examples we've seen, and therefore more anomalous because p of x, which is the height of this curve, is much lower over here on the left compared to this point over here, closer to the middle. Now, we've done this only for when x is a number, as if you had only a single feature for your anomaly detection problem. For practical anomaly detection applications, you usually have a lot of different features. You've now seen how the Gaussian distribution works. If x is a single number, this corresponds to if, say you had just one feature for your anomaly detection problem. But for practical anomaly detection applications, you will have many features, two or three or some even larger number n of features.

Let's take what you saw for a single Gaussian and use it to build a more sophisticated anomaly detection algorithm. They can handle multiple features. Let's go do that in the next video.

LECTURE 10

Anomaly Detection Example

Now that you've seen how the Gaussian or the normal distribution works for a single number, we're ready to build our anomaly detection algorithm. Let's dive in. You have a training set x1 through xm, where here each example x has n features. So, each example x is a vector with n numbers. In the case of the airplane engine example, we had two features corresponding to the heat and the vibrations. And so, each of these Xi's would be a two dimensional vector and n would be equal to 2. But for many practical applications n can be much larger and you might do this with dozens or even hundreds of features. Given this training set, what we would like to do is to carry out density estimation and all that means is, we will build a model or estimate the probability for p(x).

What's the probability of any given feature vector?

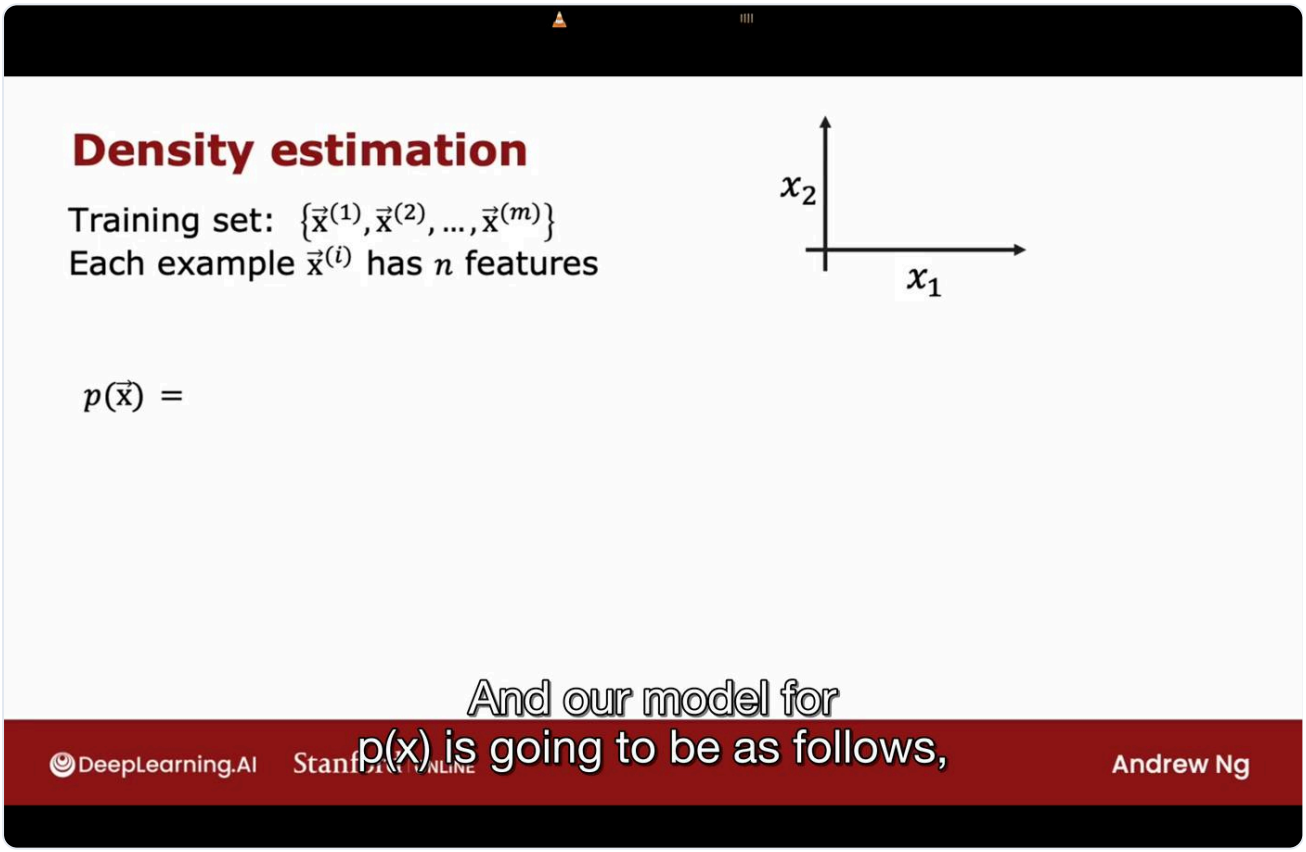


FIGURE 10.1 | Density Estimation

And our model for p(x) is going to be as follows, x is a feature vector with values x1, x2 and so on down to xn. And I'm going to model p(x) as the probability of x1, times the probability of x2, times the probability of x3 times the probability of xn, for the n th features in the feature vectors. If you've taken an advanced class in probably in statistics before, you may recognize that this equation corresponds to assuming that the features x1, x2 and so on up to xm are statistically independent. But it turns out this algorithm often works fine even that the features are not actually statistically independent. But if you don't understand what I just said, don't worry about it.

Understanding statistical independence is not needed to fully complete this class and also, be able to very effectively use anomaly detection algorithm. Now, to fill in this equation a little bit more, we are saying that the probability of all the features of this vector features x, is the product of p(x) 1 and p(x2) and so on up through p(xn). And in order to model the probability of x1, say the heat feature in this example we're going to have two parameters, mu 1 and sigma 1 or sigma squared is 1. And what that means is we're going to estimate, the mean of the feature x1 and also the variance of feature x1 and that will be new 1 and sigma 1. To model p(x2) x2 is a totally different feature measuring the vibrations of the airplane engine.

We're going to have two different parameters, which I'm going to write as mu 2, sigma 2 squared. And it turns out this will correspond to the mean or the average of the vibration feature and the variance of the vibration feature and so on. If you have additional features mu 3 sigma 3 squared up through mu n and sigma n squared. In case you're wondering why we multiply probabilities, maybe here's 1 example that could build intuition.

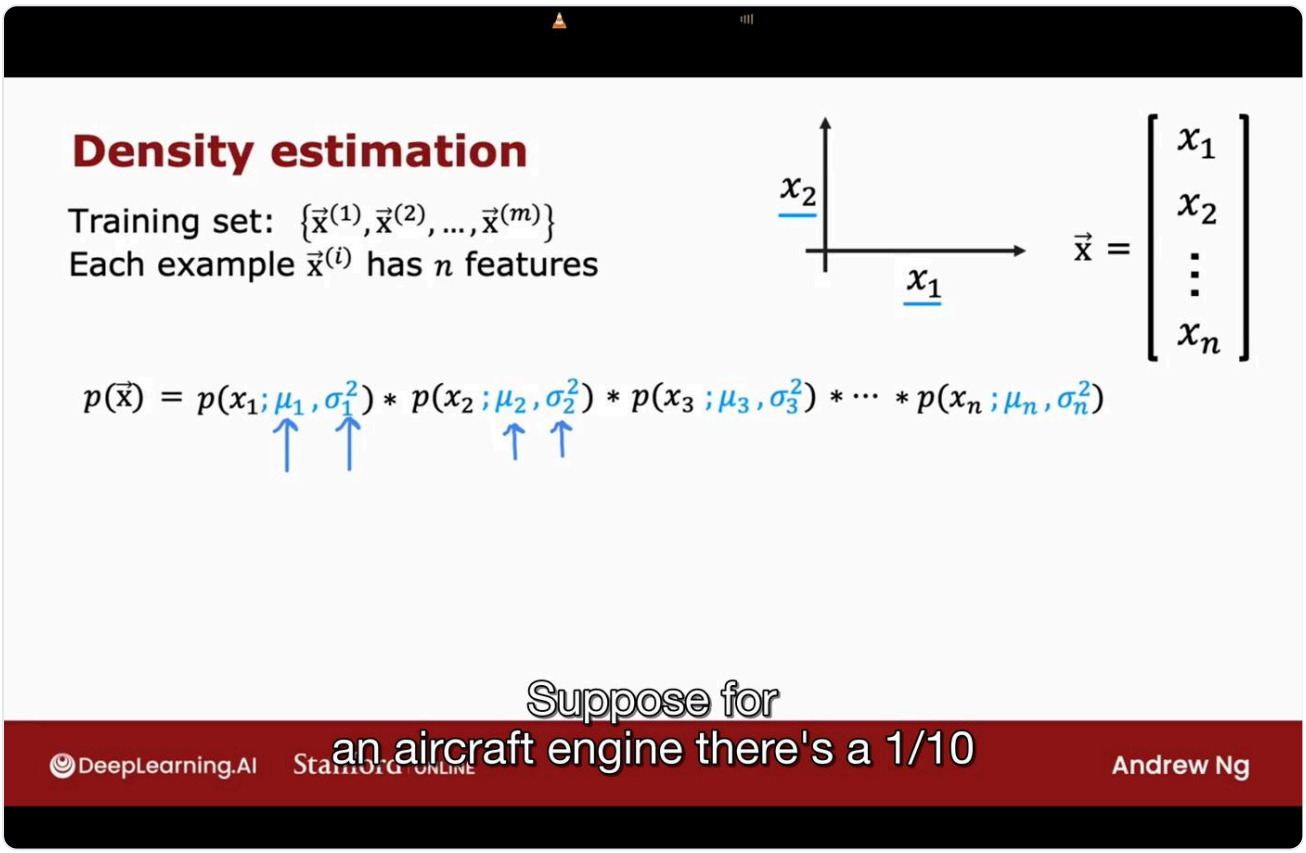


FIGURE 10.2 | Density Estimation 4

Suppose for an aircraft engine there's a 1/10 chance that it is really hot, unusually hot and maybe there is a 1 in 20 chance that it vibrates really hard. Then, what is the chance that it runs really hot and vibrates really hard.

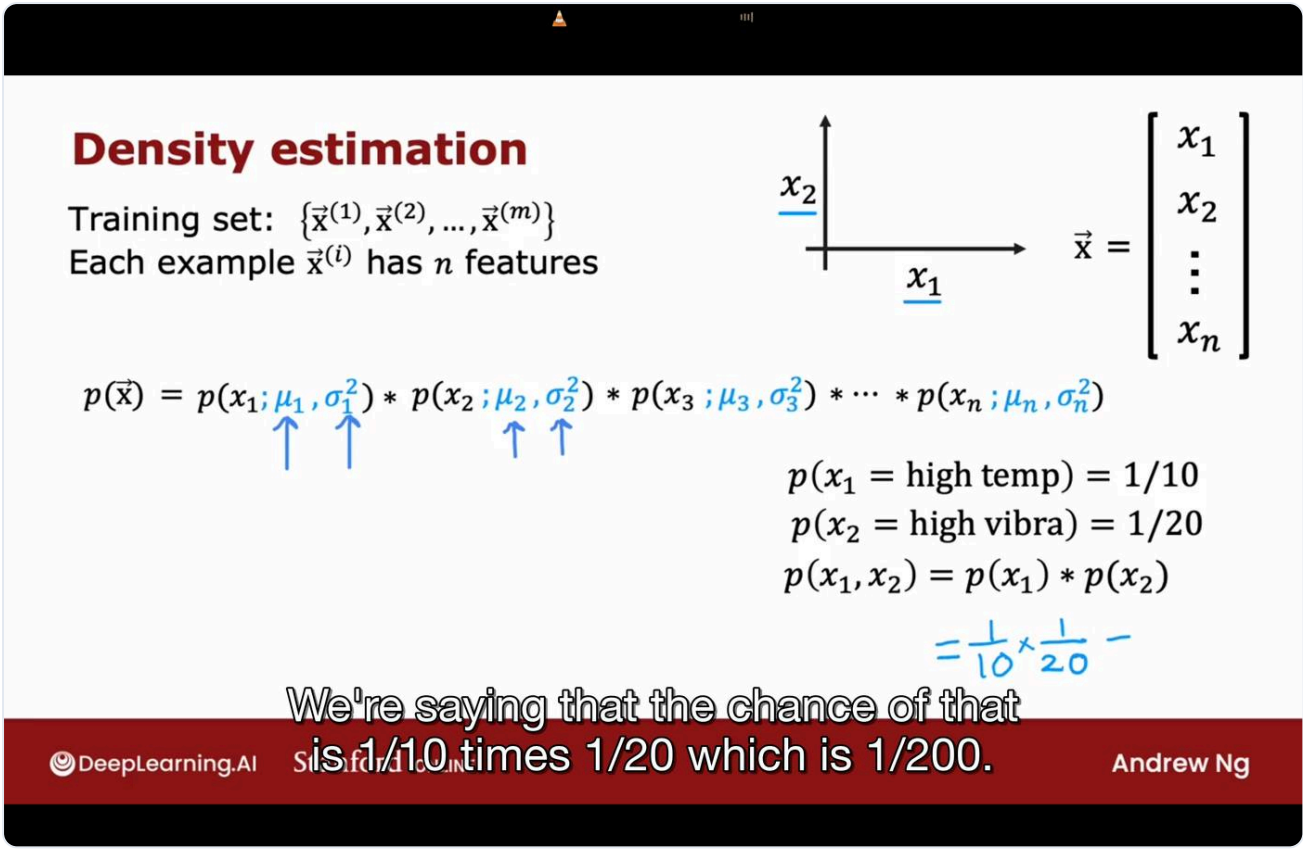


FIGURE 10.3 | Density Estimation

We're saying that the chance of that is 1/10 times 1/20 which is 1/200.

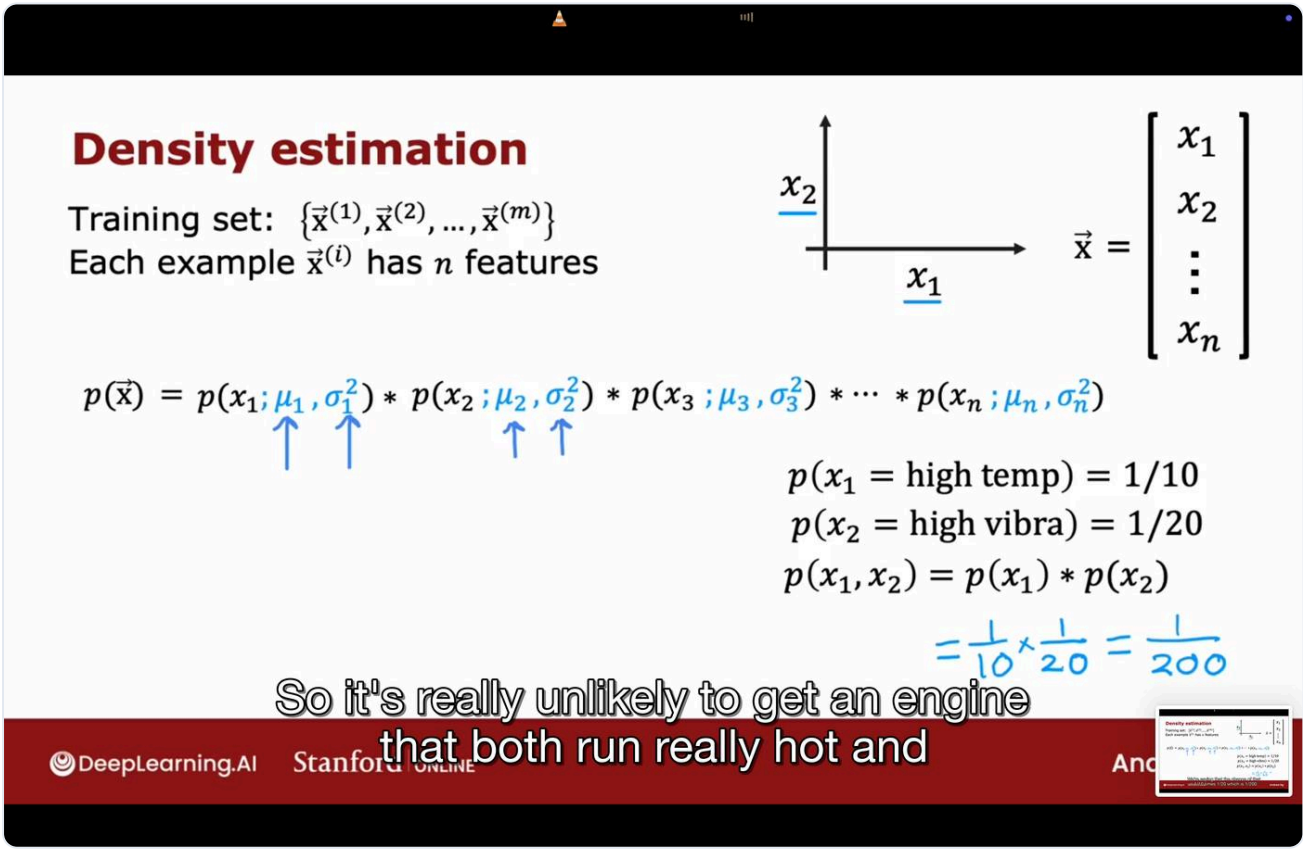


FIGURE 10.4 | Density Estimation

So it's really unlikely to get an engine that both run really hot and vibrates really hard. It's the product of these two probabilities A somewhat more compact way to write this equation up here, is to say that this is equal to, the product from j =1 through n of p(xj).

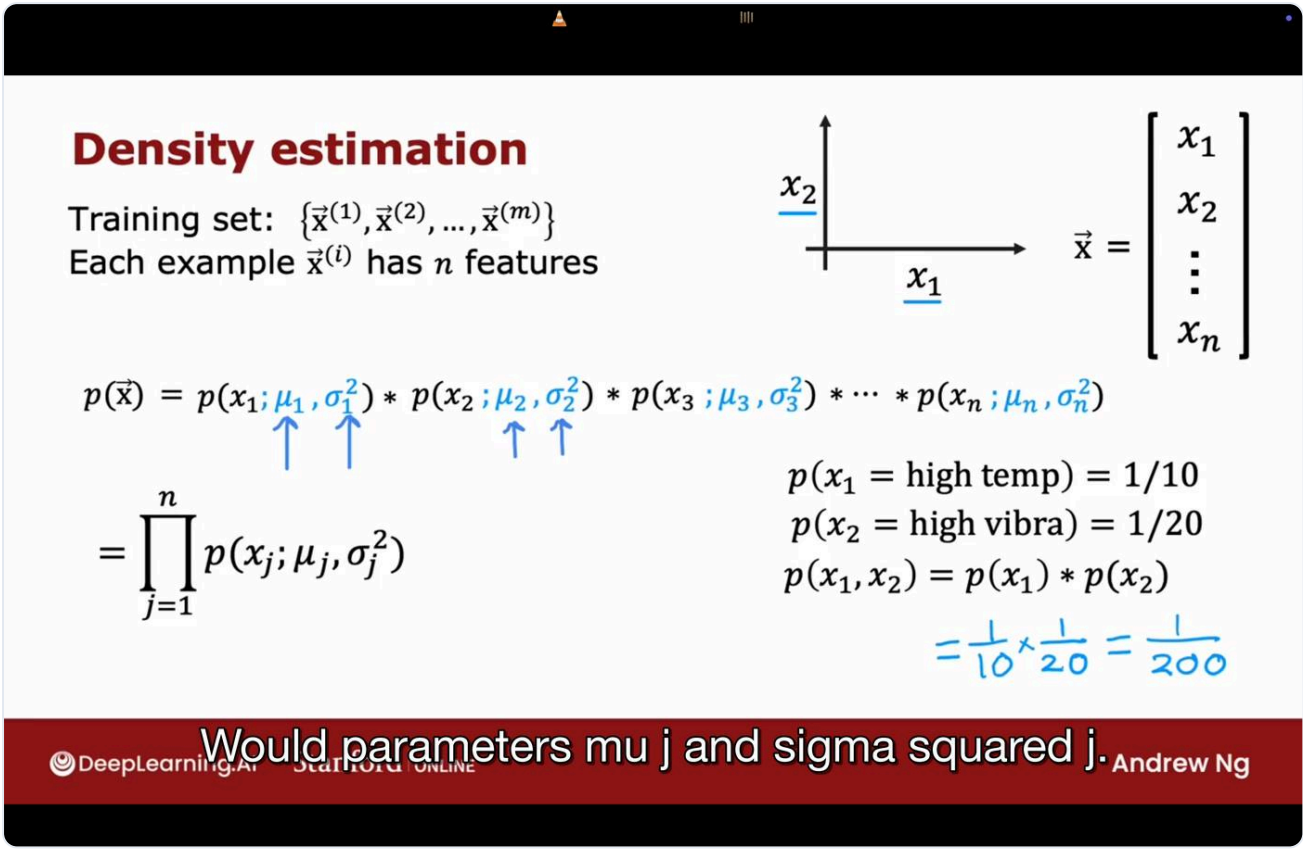


FIGURE 10.5

Would parameters mu j and sigma squared j. And this symbol here is a lot like the summation symbol except that whereas the summation symbol corresponds to addition, this symbol here corresponds to multiplying these terms over here for j =1 through n. So let's put it all together to see how you can build an anomaly detection system. The first step is to choose features xi that you think might be indicative of anomalous examples. Having come up with the features you want to use, you would then fit the parameters mu 1 through mu n and sigma square 1 through sigma squared n, for the n features in your data set. As you might guess, the parameter mu j will be just the average of xj of the feature j of all the examples in your training set.

And sigma square j will be the average of the square difference between the feature and the value mu j, that you just computed. And by the way, if you have a vectorized implementation, you can also compute mu as the average of the training examples as follows, we're here, x and mu are both vectors. And so this would be the vectorized way of computing mu 1 through mu and all at the same time. And by estimating these parameters on your unlabeled training set, you've now computed all the parameters of your model. Finally, when you are given a new example, x test or I'm just going to write a new example as x here, what you would do is compute p(x) and see if it's large or small.

So p(x) as you saw on the last slide is the product from j = 1 through n of the probability of the individual features. So p(x) j with parameters mu j and single square j. And if you substitute in, the formula for this probability, you end up with this expression 1 over root 2 pi sigma j of e to this expression over here. And so xj are the features, this is a j feature of your new example, mu j and sigma j are numbers or parameters you have computed in the previous step. And if you compute out this formula, you get some number for p(x). And the final step is to see a p(x) is less than epsilon. And if it is then you flag that it is an anomaly.

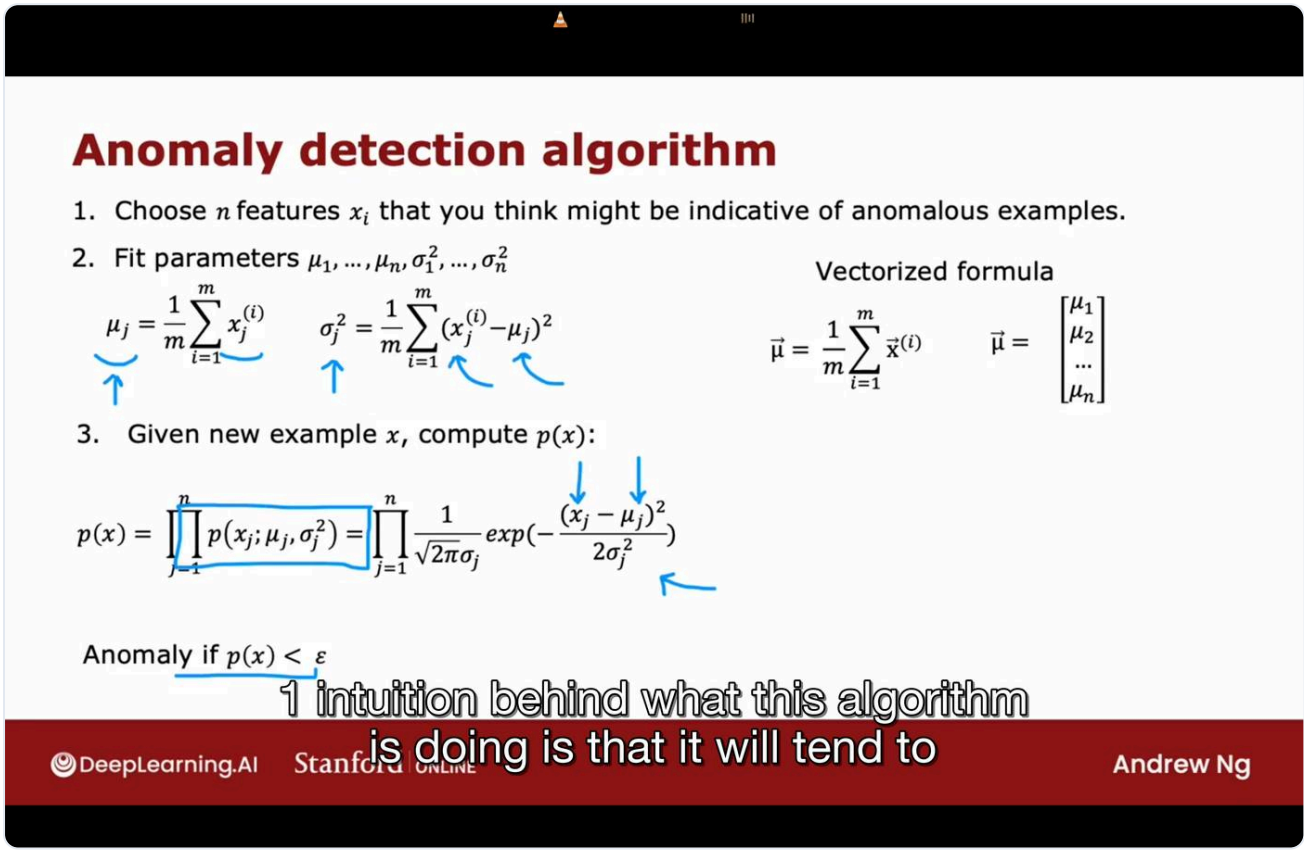


FIGURE 10.6 | Anomaly Detection Algorithm

1 intuition behind what this algorithm is doing is that it will tend to flag an example as anomalous if 1 or more of the features are either very large or very small relative to what it has seen in the training set. So for each of the features x j, you're fitting a Gaussian distribution like this. And so if even 1 of the features of the new example was way out here, say, then P f xJ would be very small. And if just 1 of the terms in this product is very small, then this overall product, when you multiply together will tend to be very small and does p(x) will be small.

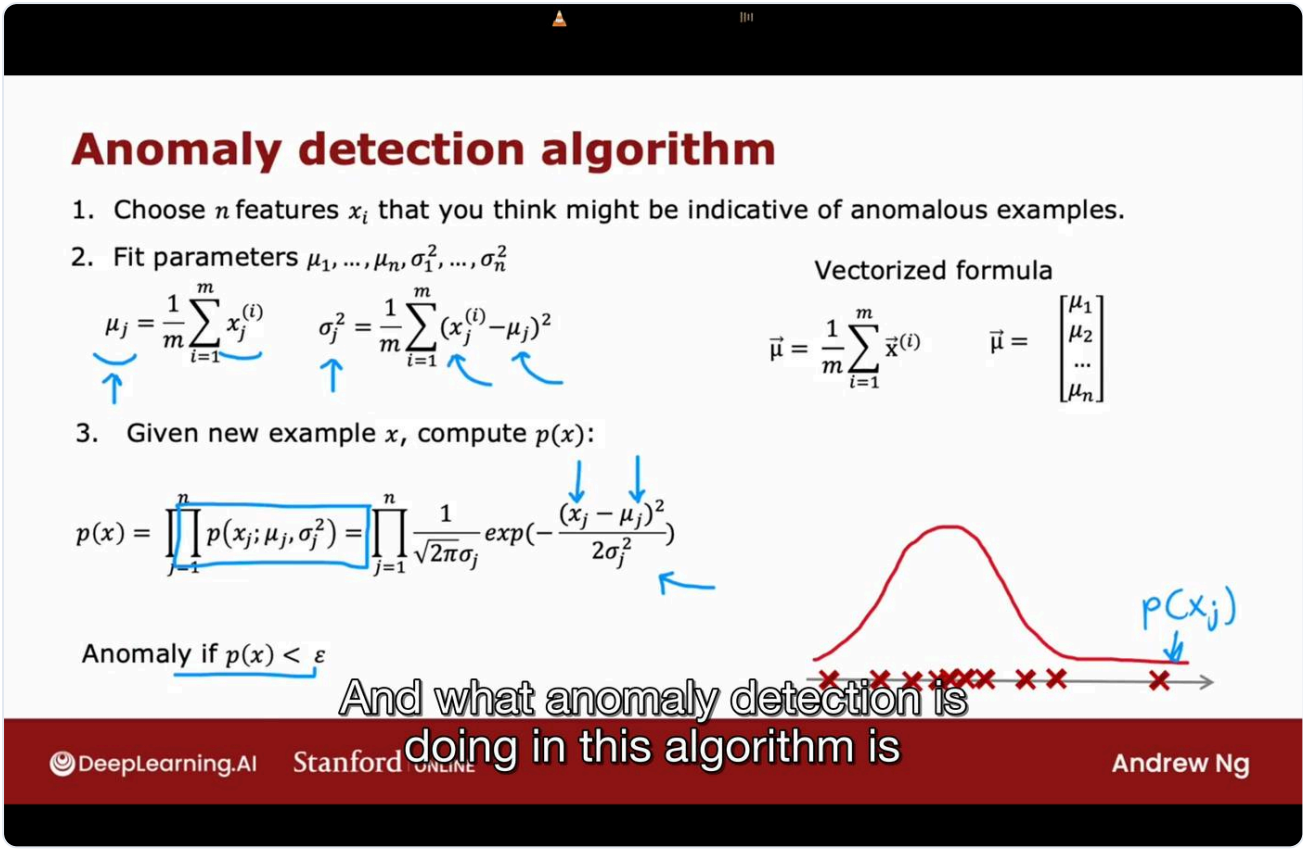


FIGURE 10.7

And what anomaly detection is doing in this algorithm is a systematic way of quantifying whether or not this new example x has any features that are unusually large or unusually small. Now, let's take a look at what all this actually means on 1 example, Here's the data set with features x 1 and x 2. And you notice that the features x 1 take on a much larger range of values than the features x 2. If you were to compute the mean of the Features x 1, you end up with five, which is why you want is equal to 1. And it turns out that for this data said, if you compute sigma 1, it will be equal to about 2. And if you were to compute mu to the average of the features on next to the average is three and similarly is variance or standard deviation is much smaller, which is why Sigma 2 is equal to 1.

So that corresponds to this Gaussian distribution for x 1 and this Gaussian distribution for x 2.

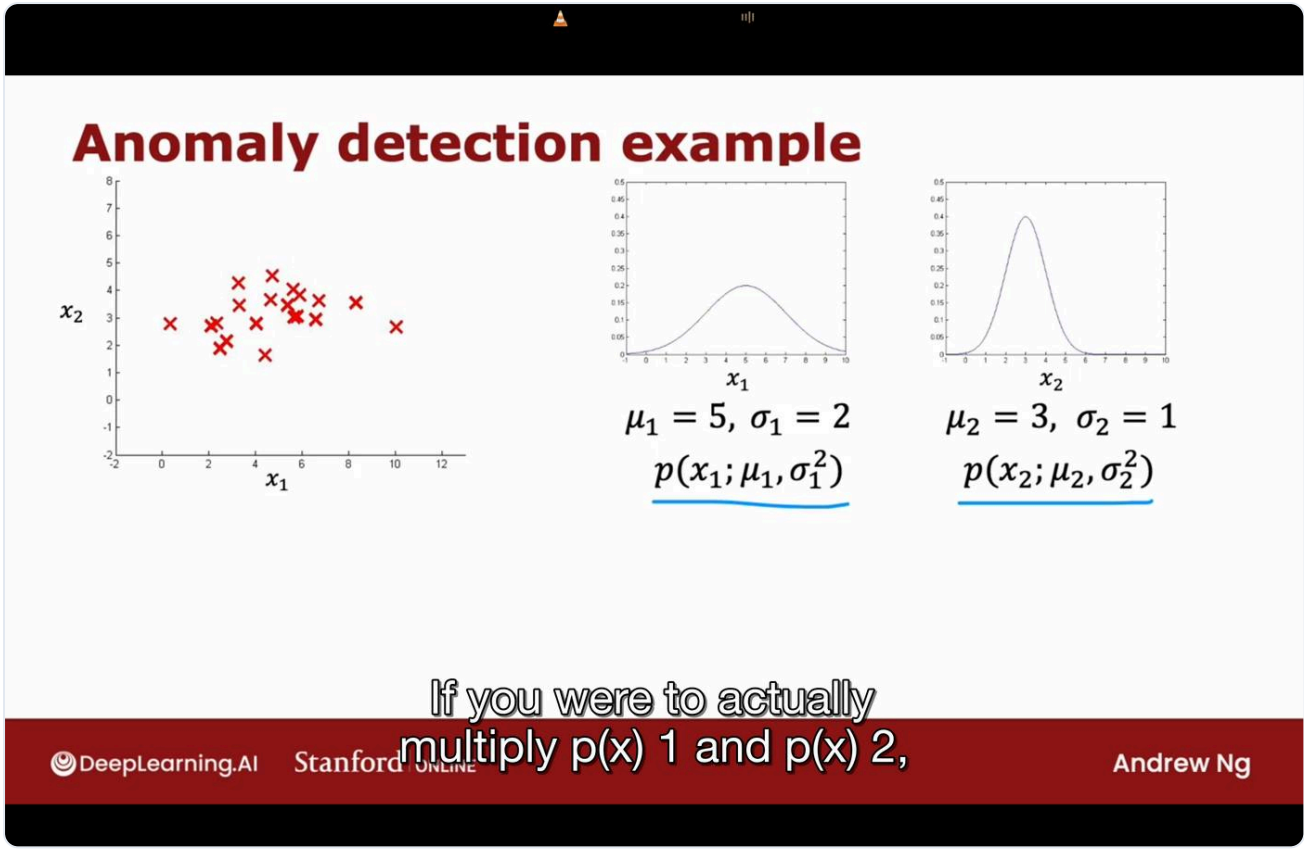


FIGURE 10.8 | Anomaly Detection Example

If you were to actually multiply p(x) 1 and p(x) 2, then you end up with this three D surface plot for p(x) where any point, the height of this is the product of p(x) 1 times p(x) 2. For the corresponding values of x 1 and x 2. And this signifies that values where p(x) is higher or more likely. So, values near the middle kind of here are more likely. Whereas values far out here, values out here are much less likely. I have much lower chance.

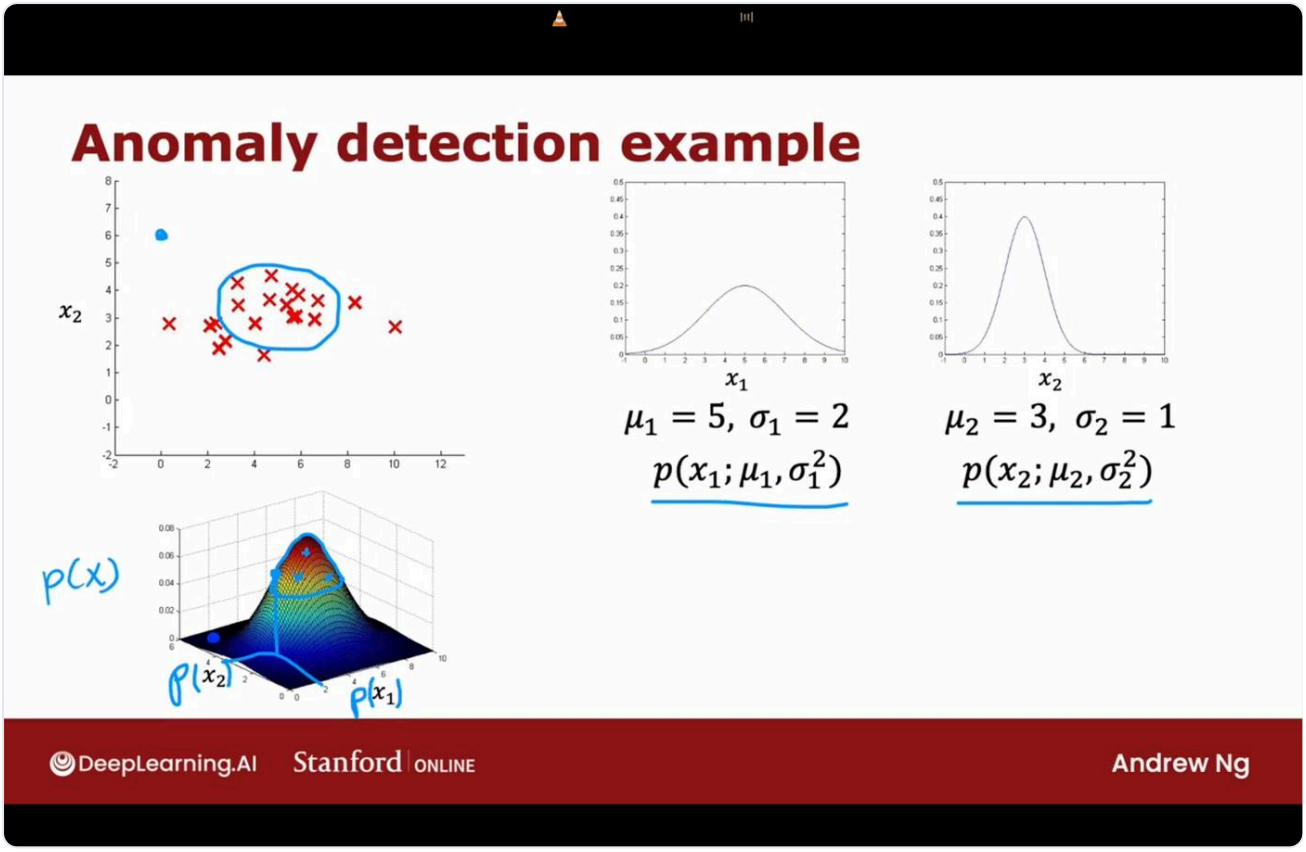


FIGURE 10.9 | Anomaly Detection Example

Now, let me pick two test examples, the first 1 here, I'm going to write this x test 1 and the second 1 down here as x test 2. And let's see which of these 2 examples the algorithm will flag as anomalous. I'm going to pick The Parameter ε to be equal to 0.02. And if you were to compute p(x) test 1, it turns out to be about 0.4 and this is much bigger than epsilon. And so the album will say this looks okay, doesn't look like an anomaly.

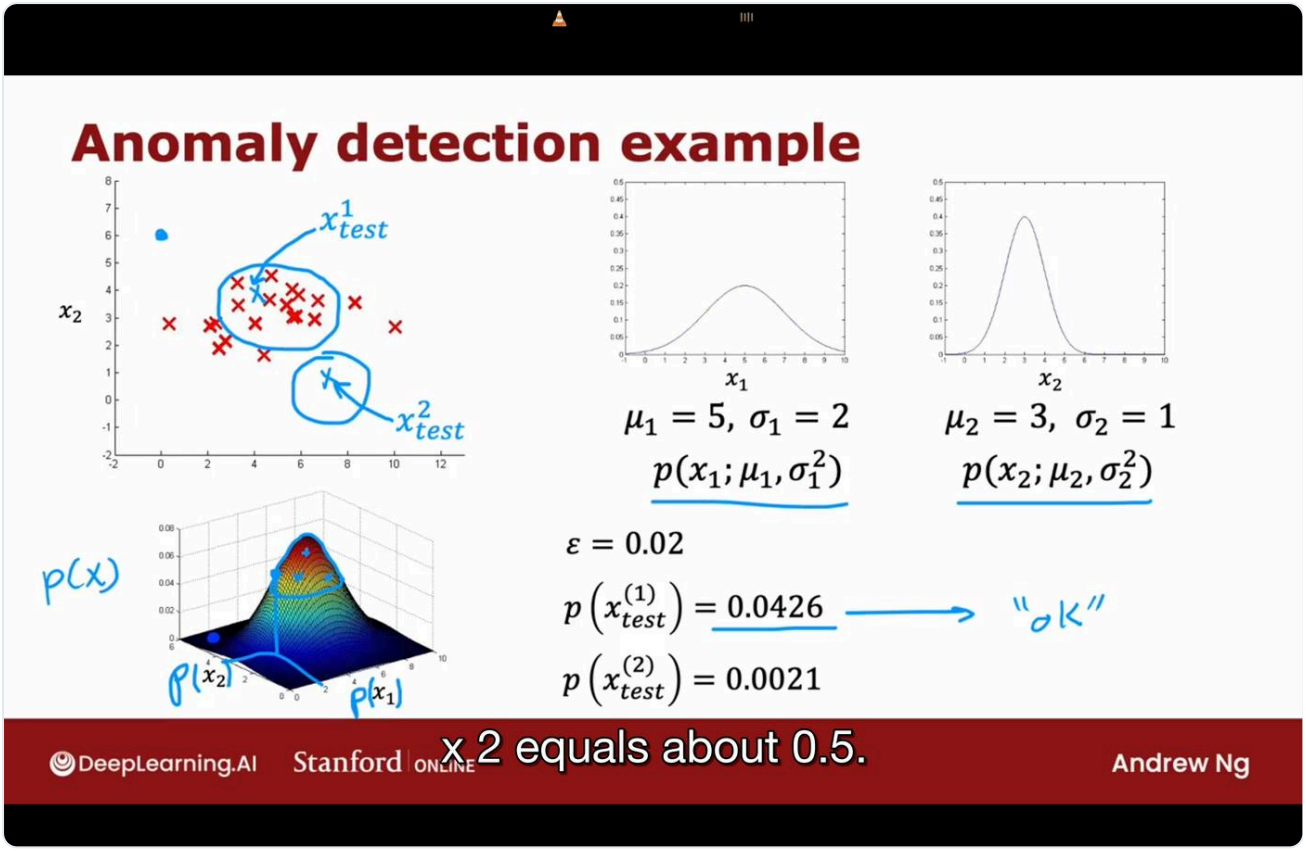


FIGURE 10.10 | Anomaly Detection Example

Whereas in contrast, if you were to compute p(x) for this point down here corresponding to x 1 equals about eight and x 2 equals about 0.5. Kind of down here, then p(x) test 2 is 0.0021. So this is much smaller than epsilon. And so the album will flag this as a likely anomaly. So, pretty much as you might hope it decides that x test 1 looks pretty normal.

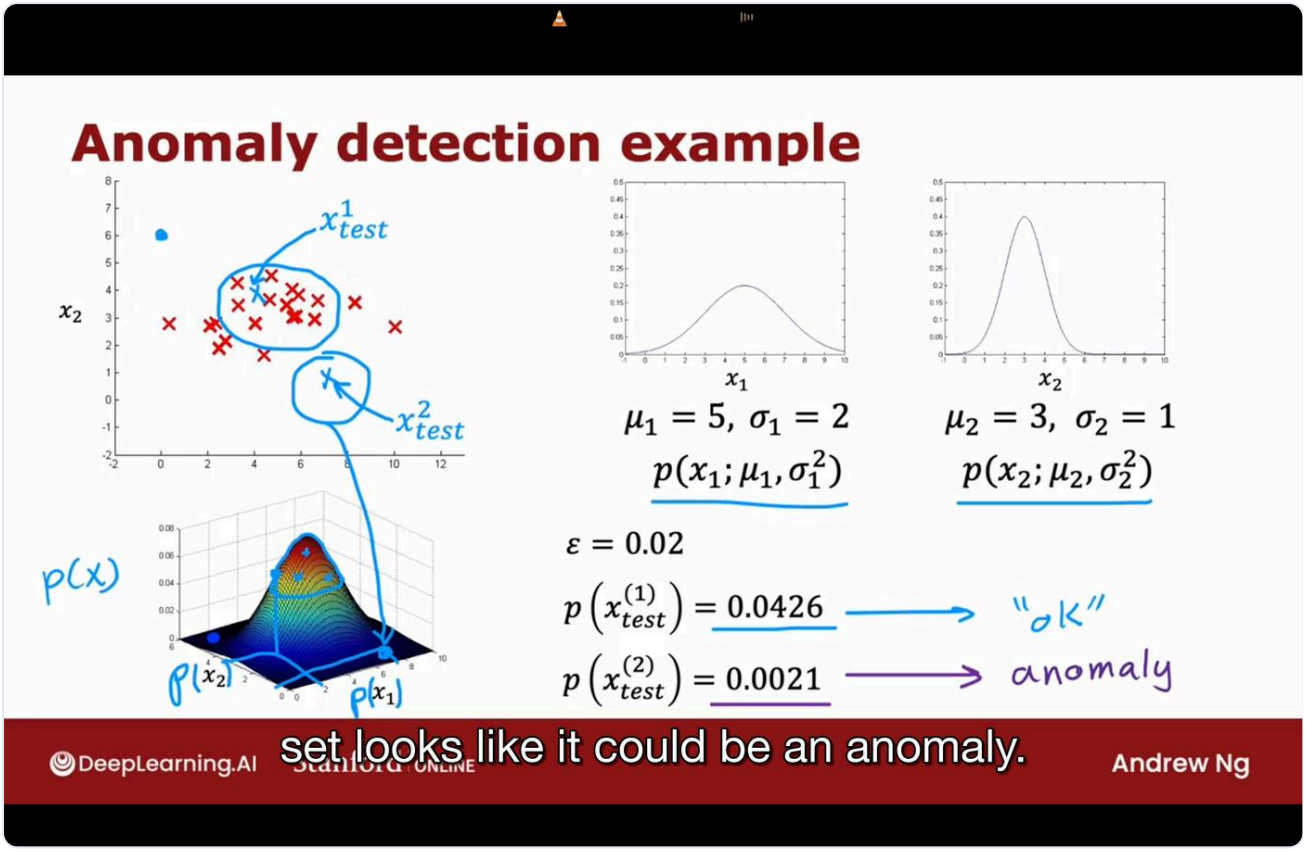


FIGURE 10.11 | Anomaly Detection Example

Whereas excess to which is much further away than anything you see in the training set looks like it could be an anomaly. So you've seen the process of how to build an anomaly detection system. But how do you choose the parameter epsilon? And how do you know if your anomaly detection system is working well in the next video, let's dive a little bit more deeply into the process of developing and evaluating the performance of an anomaly detection system. Let's go on to the next video

LECTURE 11

Aircraft Engines Monitoring Example

I'd like to share with you some practical tips for developing an anomaly detection system. One of the key ideas will be that if you can have a way to evaluate a system, even as it's being developed, you'll be able to make decisions and change the system and improve it much more quickly. Let's take a look at what that means.

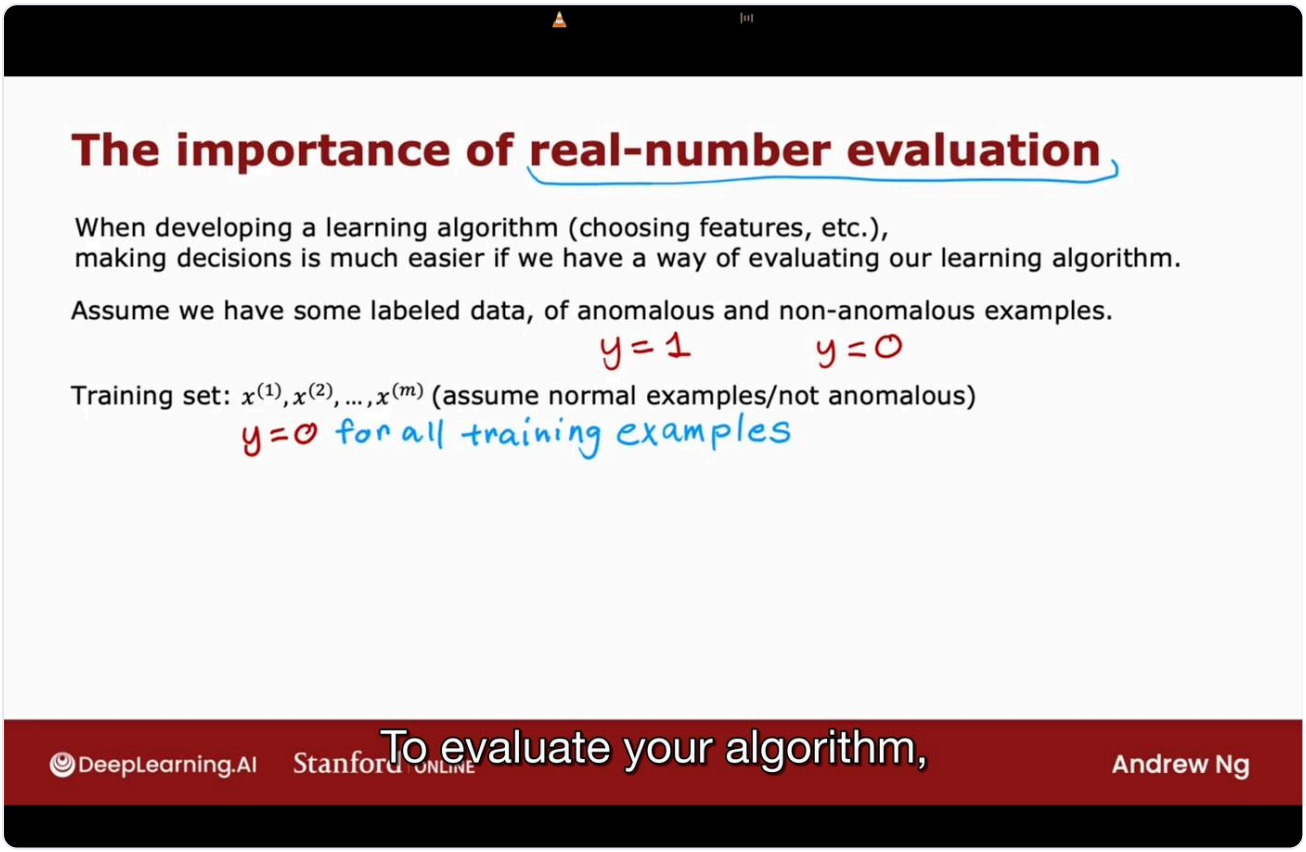


FIGURE 11.1 | When Developing a Learning Algorithm (Choosing Features, Etc.)

When you are developing a learning algorithm, say choosing different features or trying different values of the parameters like epsilon, making decisions about whether or not to change a feature in a certain way or to increase or decrease epsilon or other parameters, making those decisions is much easier if you have a way of evaluating the learning algorithm. This is sometimes called real number evaluation, meaning that if you can quickly change the algorithm in some way, such as change a feature or change a parameter and have a way of computing a number that tells you if the algorithm got better or worse, then it makes it much easier to decide whether or not to stick with that change to the algorithm. This is how it's often done in anomaly detection.

Which is, even though we've mainly been talking about unlabeled data, I'm going to change that assumption a bit and assume that we have some labeled data, including just a small number usually of previously observed anomalies. Maybe after making airplane engines for a few years, you've just seen a few airplane engines that were anomalous, and for examples that you know are anomalous, I'm going to associate a label y equals 1 to indicate this anomalous, and for examples

that we think are normal, I'm going to associate the label y equals 0. The training set that the anomaly detection algorithm will learn from is still this unlabeled training set of x1 through xm, and I'm going to think of all of these examples as ones that we'll just assume are normal and not anomalous, so y is equal to 0.

In practice, you have a few anomalous examples where to slip into this training set, your algorithm will still usually do okay. To evaluate your algorithm, come up with a way for you to have a real number evaluation, it turns out to be very useful if you have a small number of anomalous examples so that you can create a cross validation set, which I'm going to denote x_cv^1, y_cv^1 through x_cv^mcv, y_cv^mcv. This is similar notation as you had seen in the second course of this specialization. As similarly, have a test set of some number of examples where both the cross validation and test sets hopefully includes a few anomalous examples.

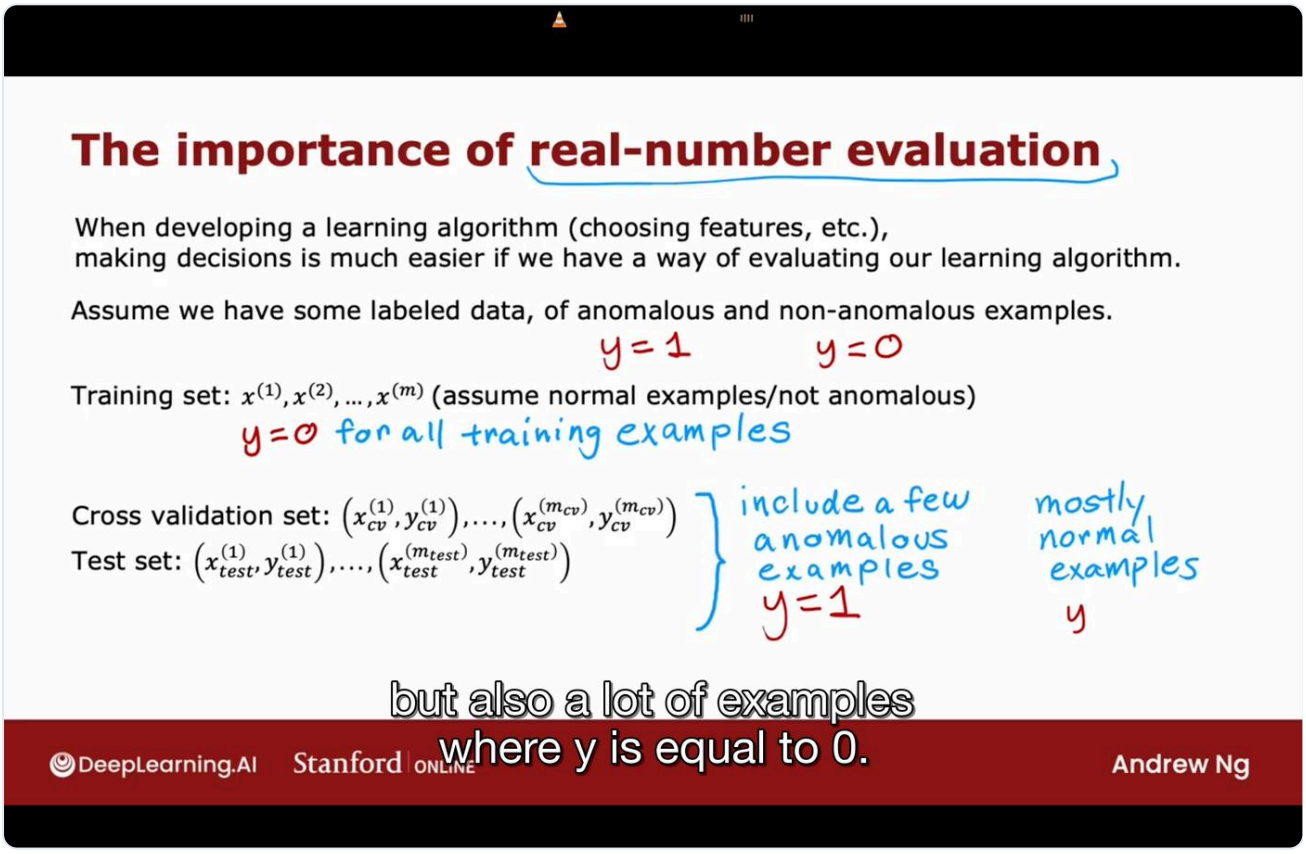


FIGURE 11.2 | When Developing a Learning Algorithm (Choosing Features, Etc.)

In other words, the cross validation and test sets will have a few examples of y equals 1, but also a lot of examples where y is equal to 0.

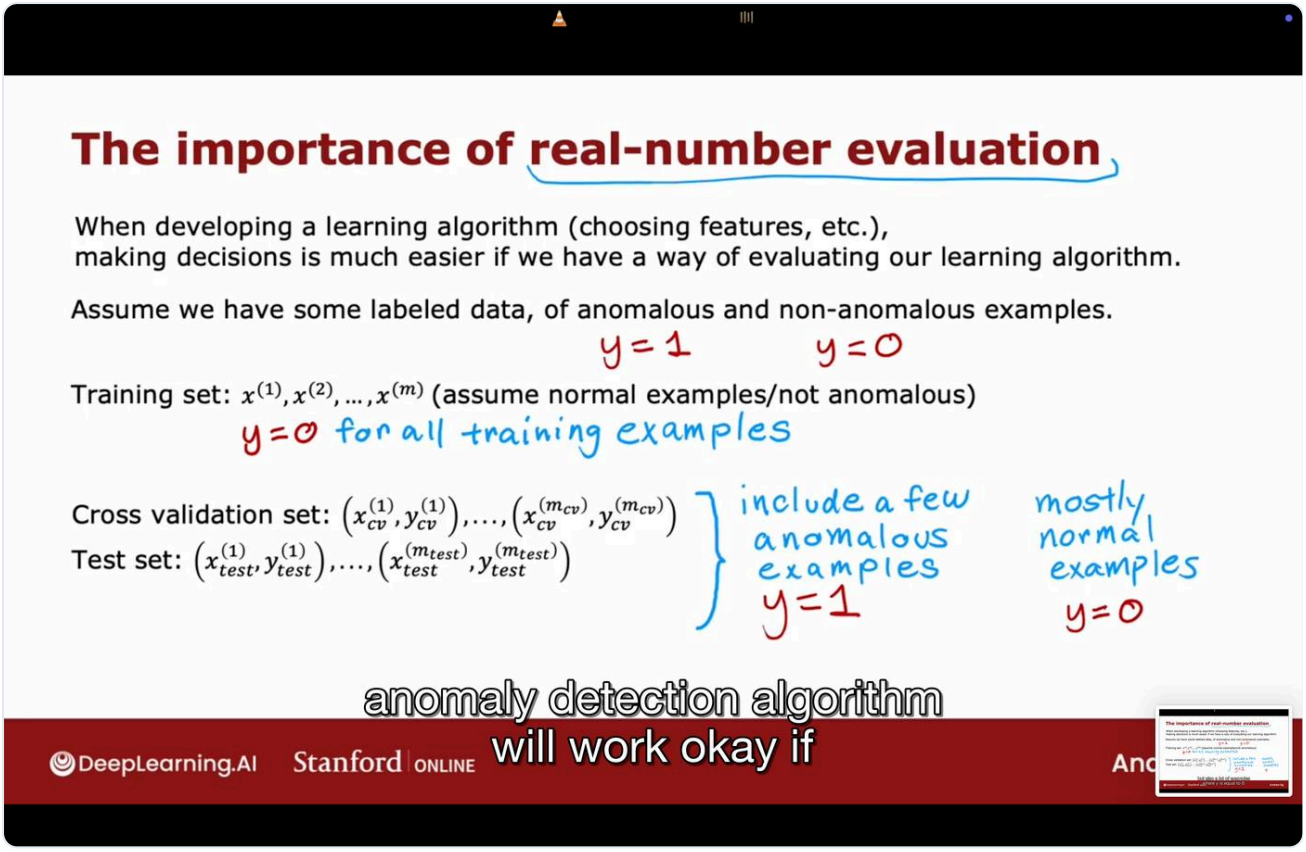


FIGURE 11.3 | When Developing a Learning Algorithm (Choosing Features, Etc.)

Again, in practice, anomaly detection algorithm will work okay if there are some examples that are actually anomalous, but there were accidentally labeled with y equals 0. Let's illustrate this with the aircraft engine example. Let's say you have been manufacturing aircraft engines for years and so you've collected data from 10,000 good or normal engines, but over the years you had also collected data from 20 flawed or anomalous engines. Usually the number of anomalous engines, that is y equals 1, will be much smaller. It will not be a typical to apply this type of algorithm with anywhere from, say, 2-50 known anomalies. We're going to take this dataset and break it up into a training set, a cross validation set, and the test set.

Here's one example. I'm going to put 6,000 good engines into the training set. Again, if there are couple of anomalous engines that got slipped into this set is actually okay, I wouldn't worry too much about that. Then let's put 2,000 good engines and 10 of the known anomalies into the cross-validation set, and a separate 2,000 good and 10 anomalous engines into the test set.

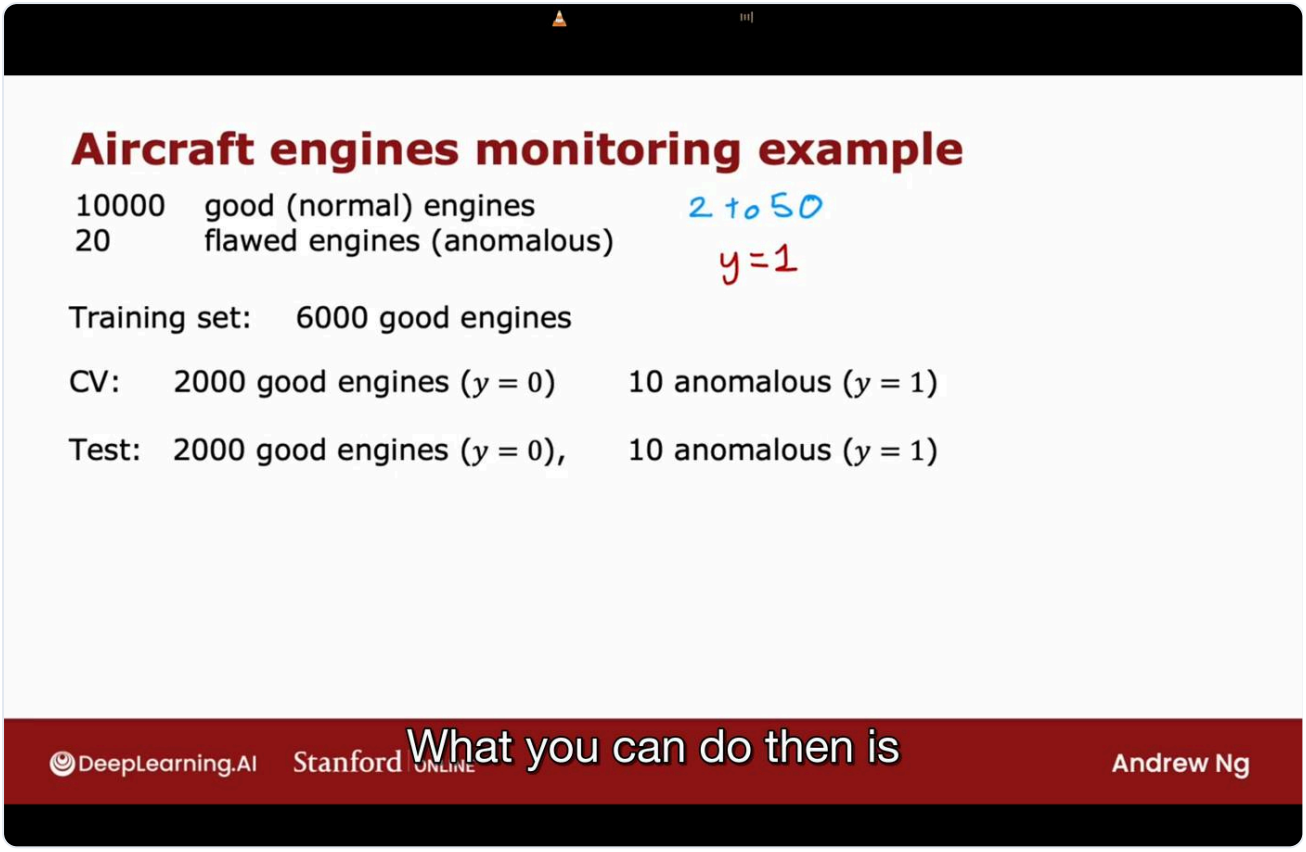


FIGURE 11.4 | Aircraft Engines Monitoring Example

What you can do then is train the algorithm on the training set, fit the Gaussian distributions to these 6,000 examples and then on the cross-validation set, you can see how many of the anomalous engines it correctly flags.

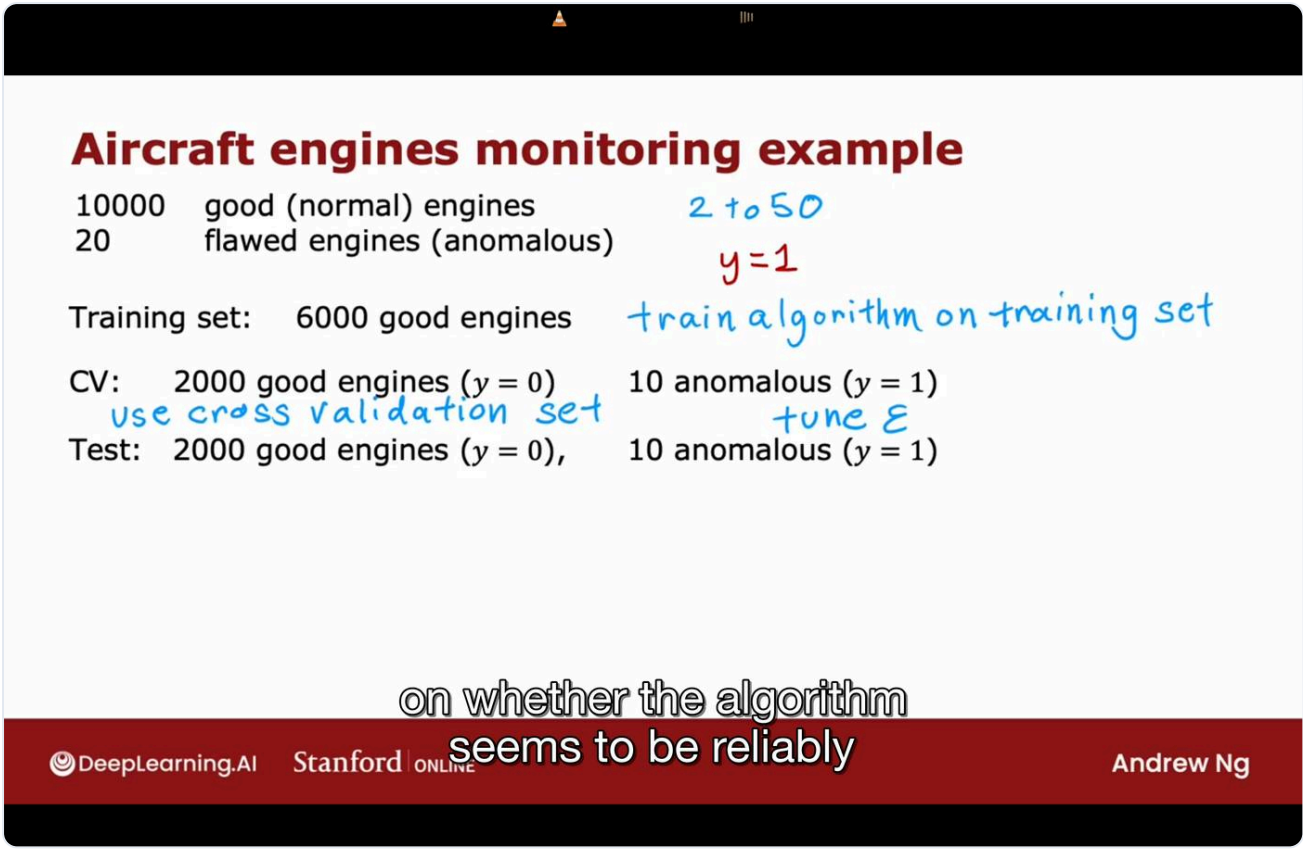


FIGURE 11.5 | Aircraft Engines Monitoring Example

For example, you could use the cross validation set to tune the parameter epsilon and set it higher or lower depending on whether the algorithm seems to be reliably detecting these 10 anomalies without taking too many of these 2,000 good engines and flagging them as anomalies. After you have tuned the parameter epsilon and maybe also added or subtracted or tuned to features X_J you can then take the algorithm and evaluate it on your test set to see how many of these 10 anomalous engines it finds, as well as how many mistakes it makes by flagging the good engines as anomalous ones.

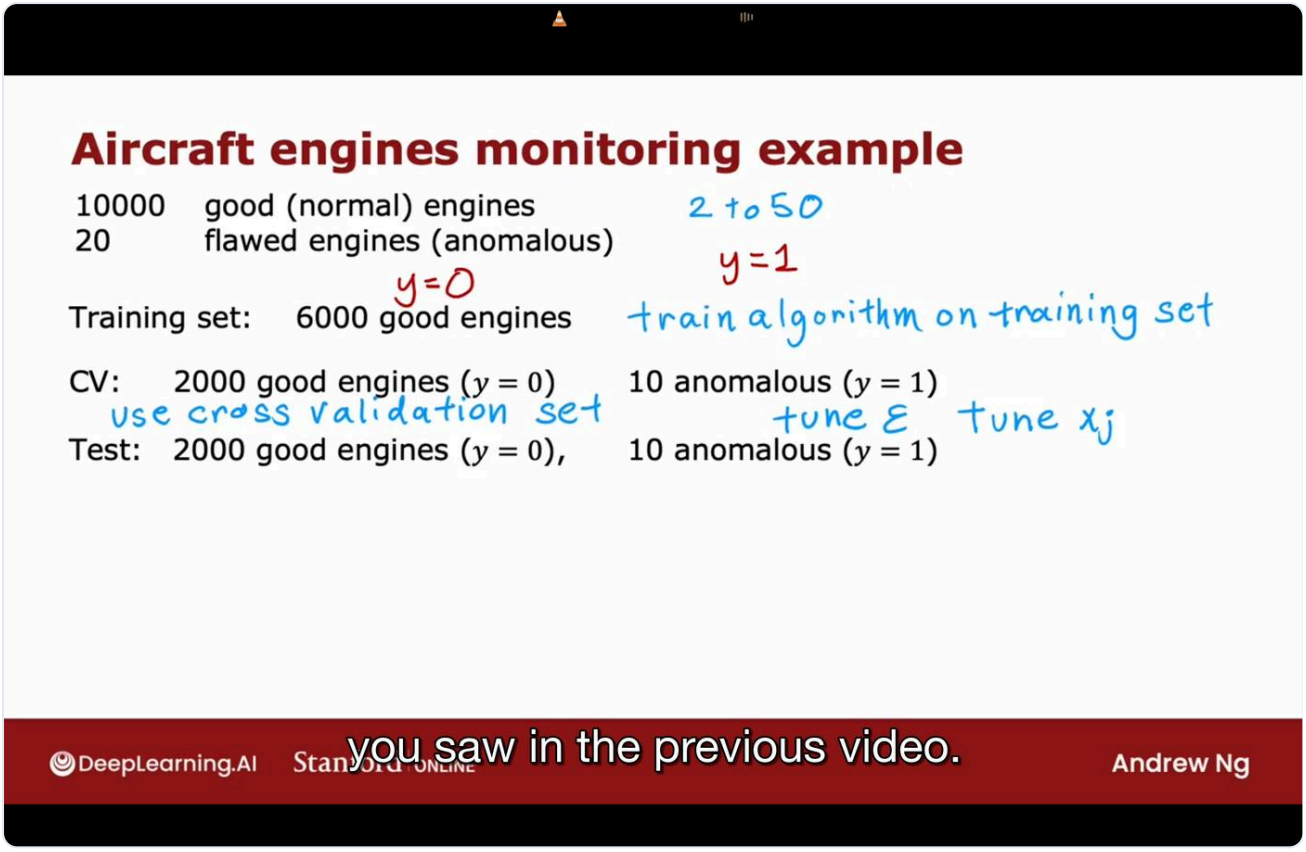


FIGURE 11.6 | Aircraft Engines Monitoring Example

Notice that this is still primarily an unsupervised learning algorithm because the training sets really has no labels or they all have labels that we're assuming to be y equals 0 and so we learned from the training set by fitting the Gaussian distributions as you saw in the previous video. But it turns out if you're building a practical anomaly detection system, having a small number of anomalies to use to evaluate the algorithm that your cross validation and test sets is very helpful for tuning the algorithm. Because the number of flawed engines is so small there's one other alternative that I often see people use for anomaly detection, which is to not use a test set, like to have just a training set and a cross-validation set.

In this example, you will set train on 6,000 good engines, but take the remainder of the data, the 4,000 remaining good engines as well as all the anomalies, and put them in the cross validation set.

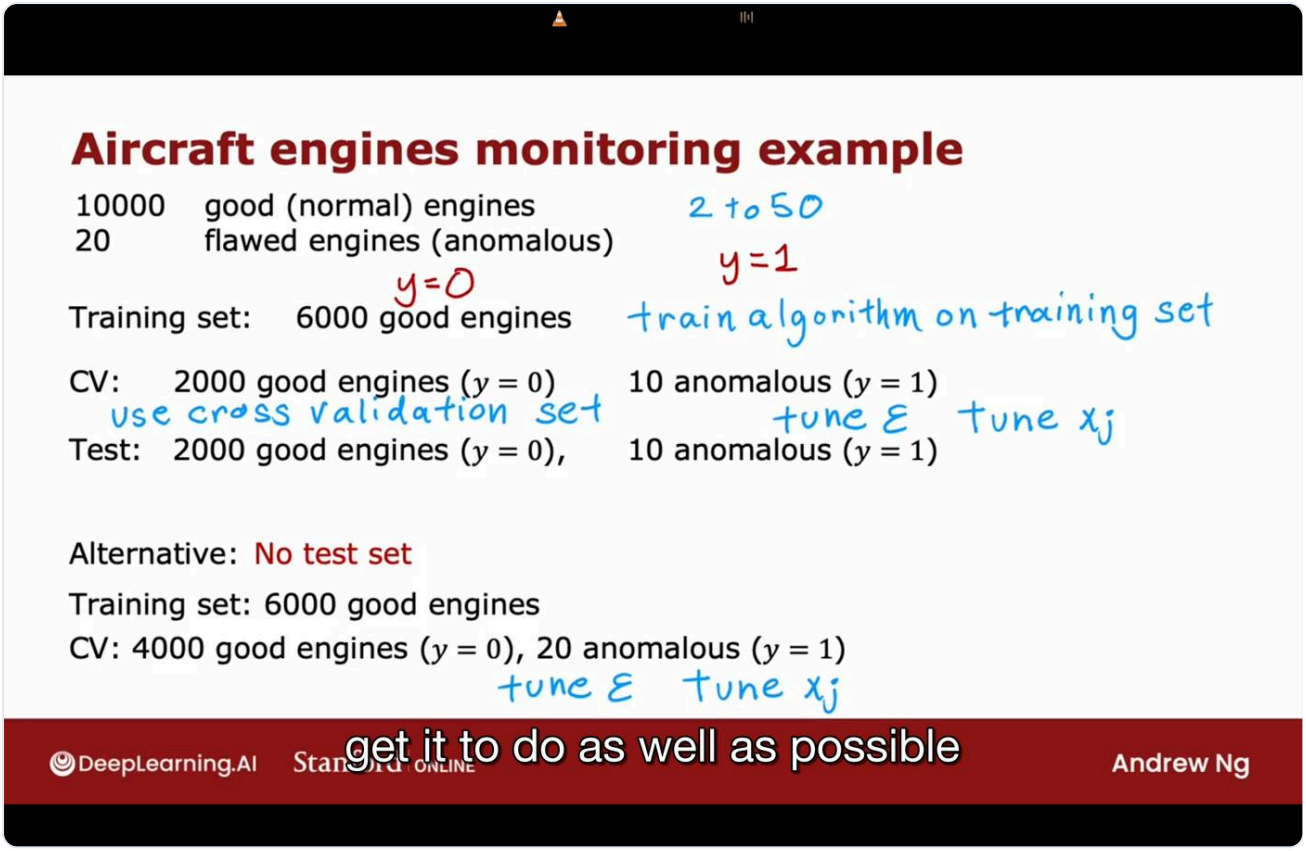


FIGURE 11.7 | Aircraft Engines Monitoring Example

You would then tune the parameters Epsilon and add or subtract features x_j to try to get it to do as well as possible as evaluated on the cross validation set. If you have very few flawed engines, so if you had only two flawed engines, then this really makes sense to put all of that in the cross validation set. You just don't have enough data to create a totally separate test set that is distinct from your cross-validation set.

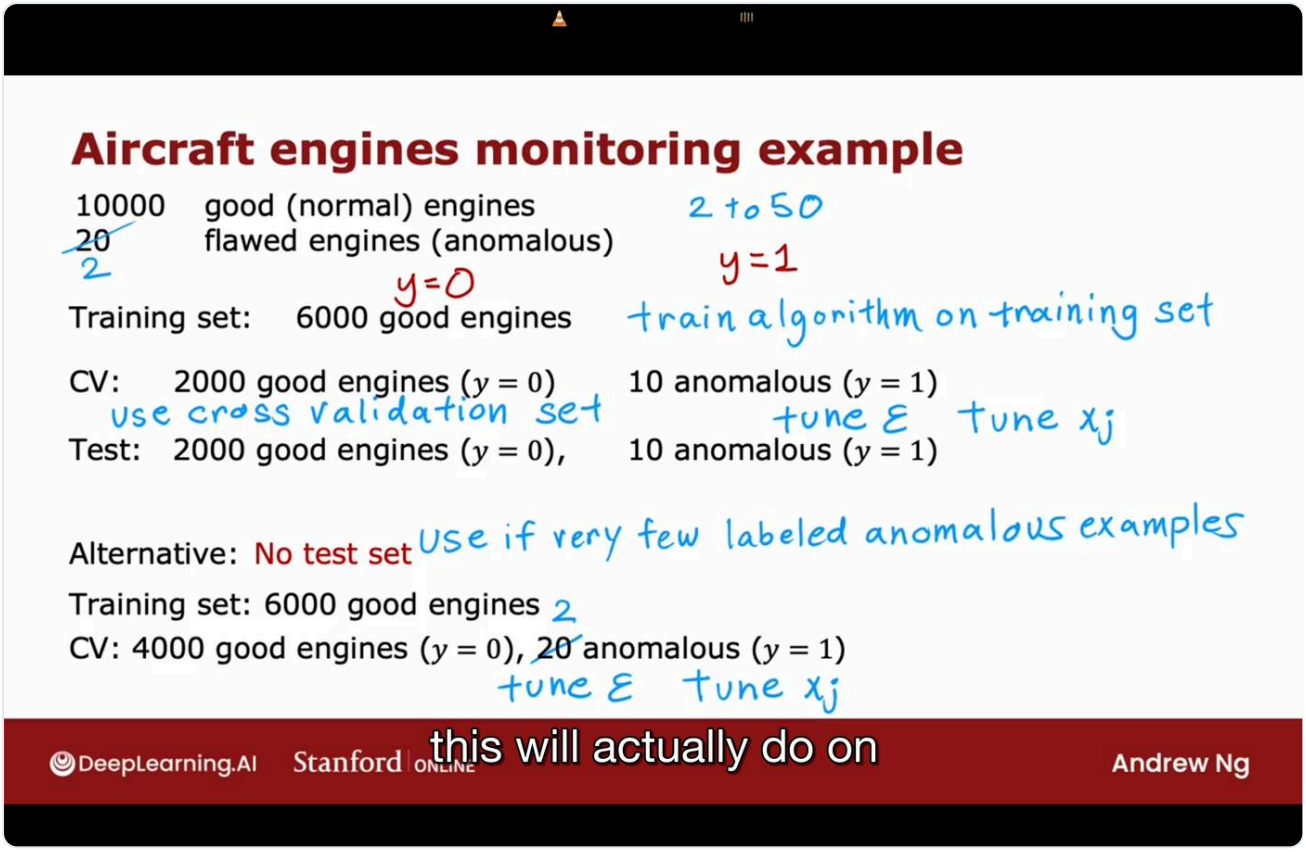


FIGURE 11.8 | Aircraft Engines Monitoring Example

The downside of this alternative here is that after you've tuned your algorithm, you don't have a fair way to tell how well this will actually do on future examples because you don't have the test set. But when your dataset is small, especially when the number of anomalies you have, your dataset is small, this might be the best alternative you have. I see this done quite often as well when you just don't have enough data to create a separate test set.

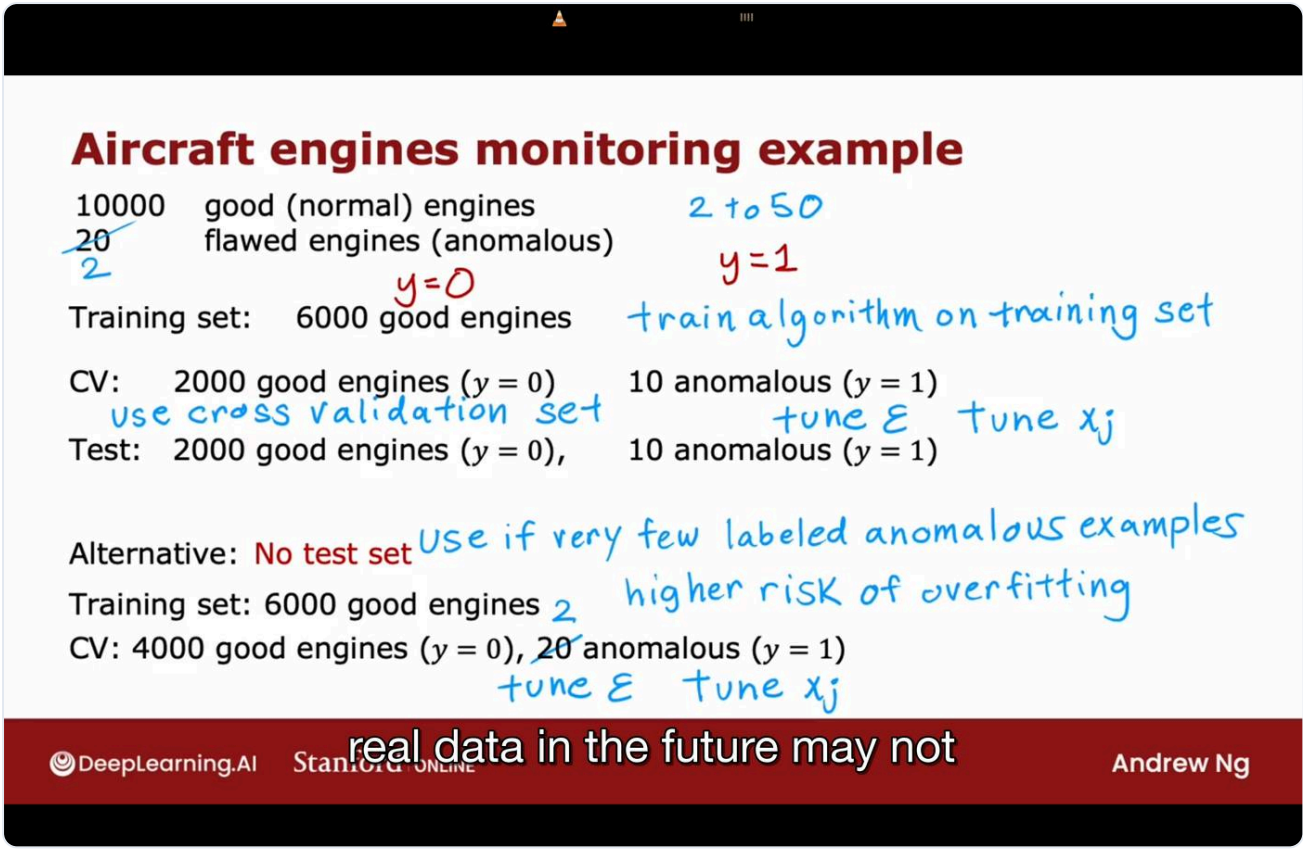


FIGURE 11.9 | Aircraft Engines Monitoring Example

If this is the case, just be aware that there's a higher risk that you will have over-fit some of your decisions around Epsilon and choice of features and so on to the cross-validation set and so its performance on real data in the future may not be as good as you were expecting. Now, let's take a closer look at how to actually evaluate the algorithm on your cross-validation sets or on the test set. Here's what you'd do. You would first fit the model p of x on the training set. This was a 6,000 examples of goods engines. Then on any cross validation or test example x, you would compute p of x and you will predict y equals 1. That is anomalous if p of x is less than Epsilon and you predict y is 0, if p of x is greater than or equal to Epsilon.

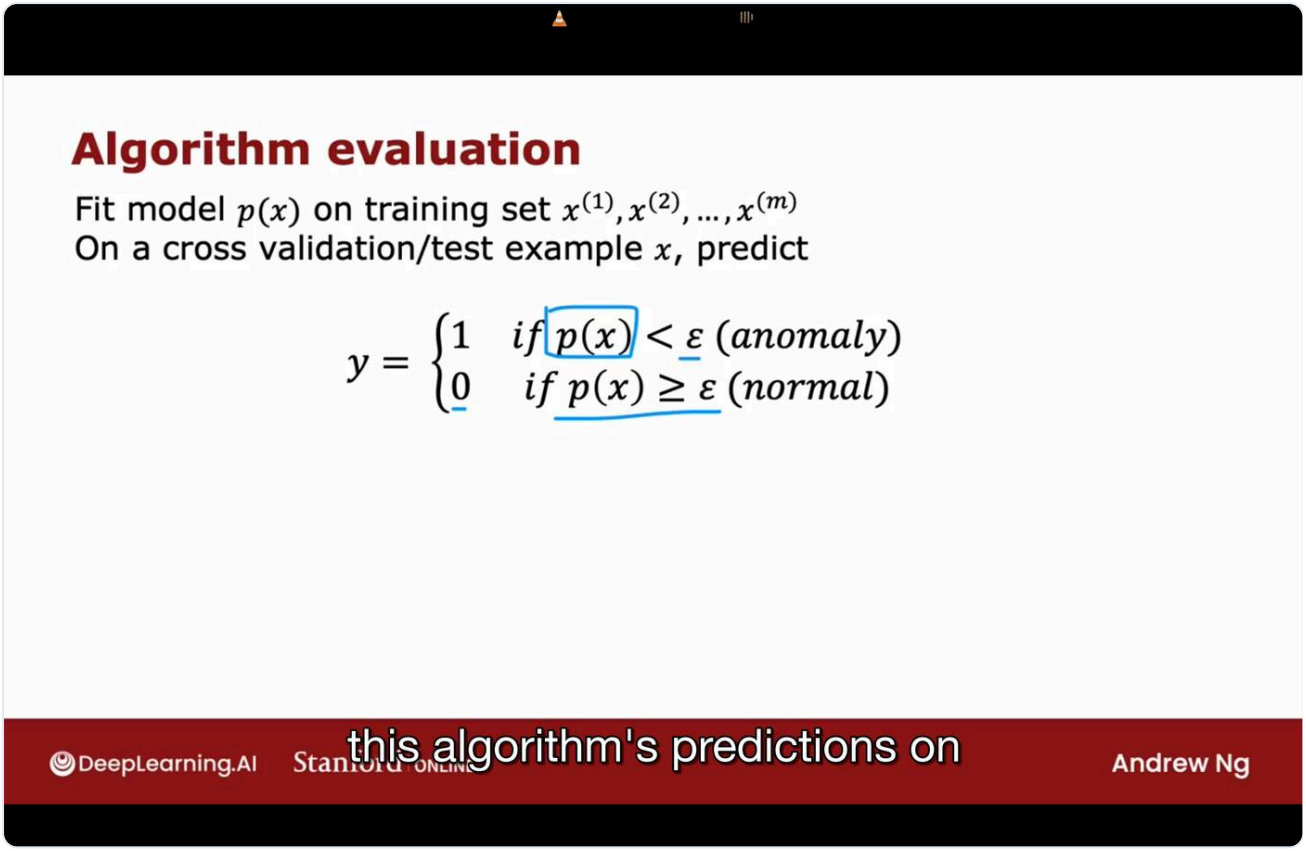


FIGURE 11.10 | Algorithm Evaluation

Based on this, you can now look at how accurately this algorithm's predictions on the cross validation or test set matches the labels, y you have in the cross validation or the test sets. In the third week of the second course, we had had a couple of optional videos on how to handle highly skewed data distributions where the number of positive examples, y equals 1, can be much smaller than the number of negative examples where y equals 0. This is the case as well for many anomaly detection in the applications where the number of anomalies in your cross-validation set is much smaller. In our previous example, we had maybe 10 positive examples and 2,000 negative examples because we had 10 anomalies and 2,000 normal examples.

If you saw those optional videos, you may recall that we saw it can be useful to compute things like the true positive, false positive, false negative, and true negative rates. Also compute precision recall or F_1 score and that these are alternative metrics and classification accuracy that could work better when your data distribution is very skewed.

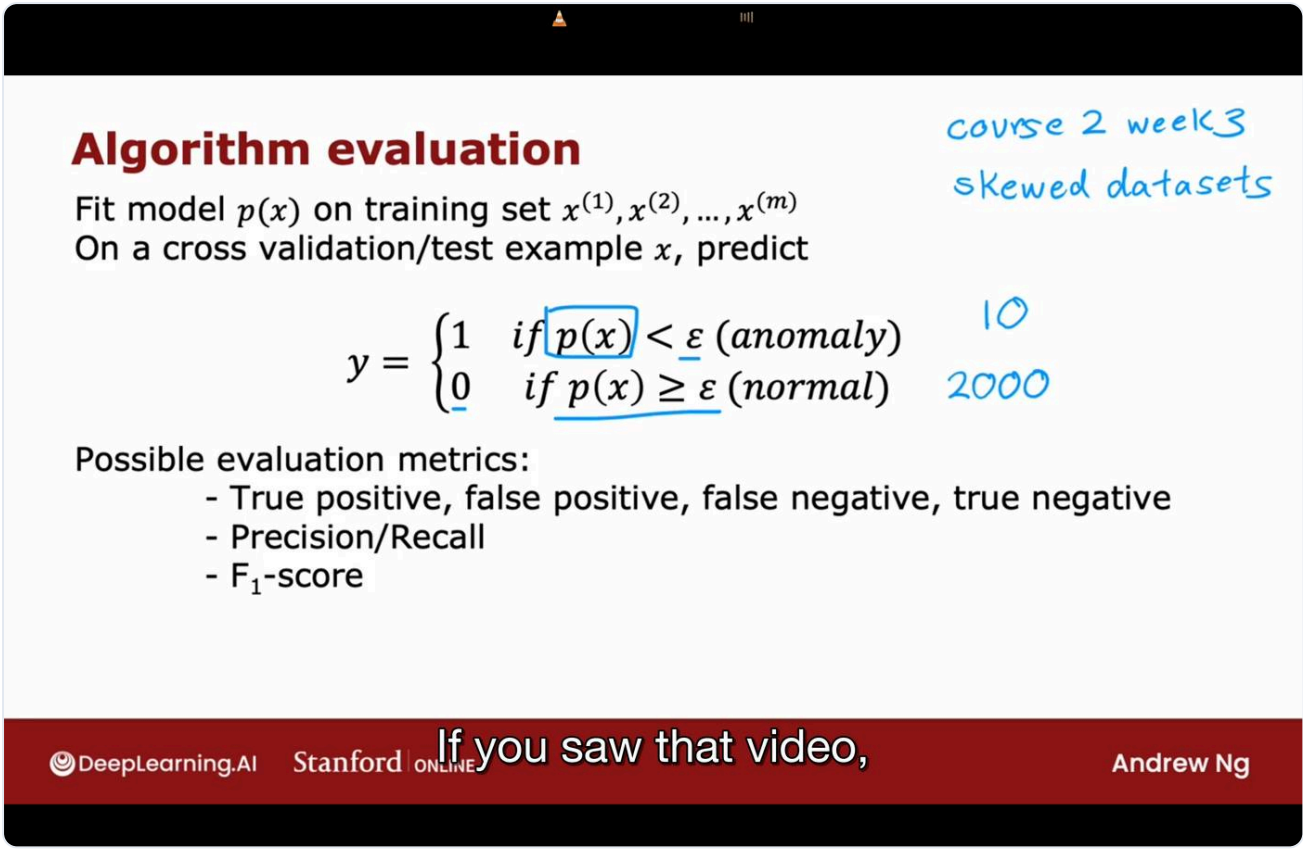


FIGURE 11.11 | Algorithm Evaluation

If you saw that video, you might consider applying those types of evaluation metrics as well to tell how well your learning algorithm is doing at finding that small handful of anomalies or positive examples amidst this much larger set of negative examples of normal plane engines. If you didn't watch that video, don't worry about it. It's okay. The intuition I hope you get is to use the cross-validation set to just look at how many anomalies is finding and also how many normal engines is incorrectly flagging as an anomaly.

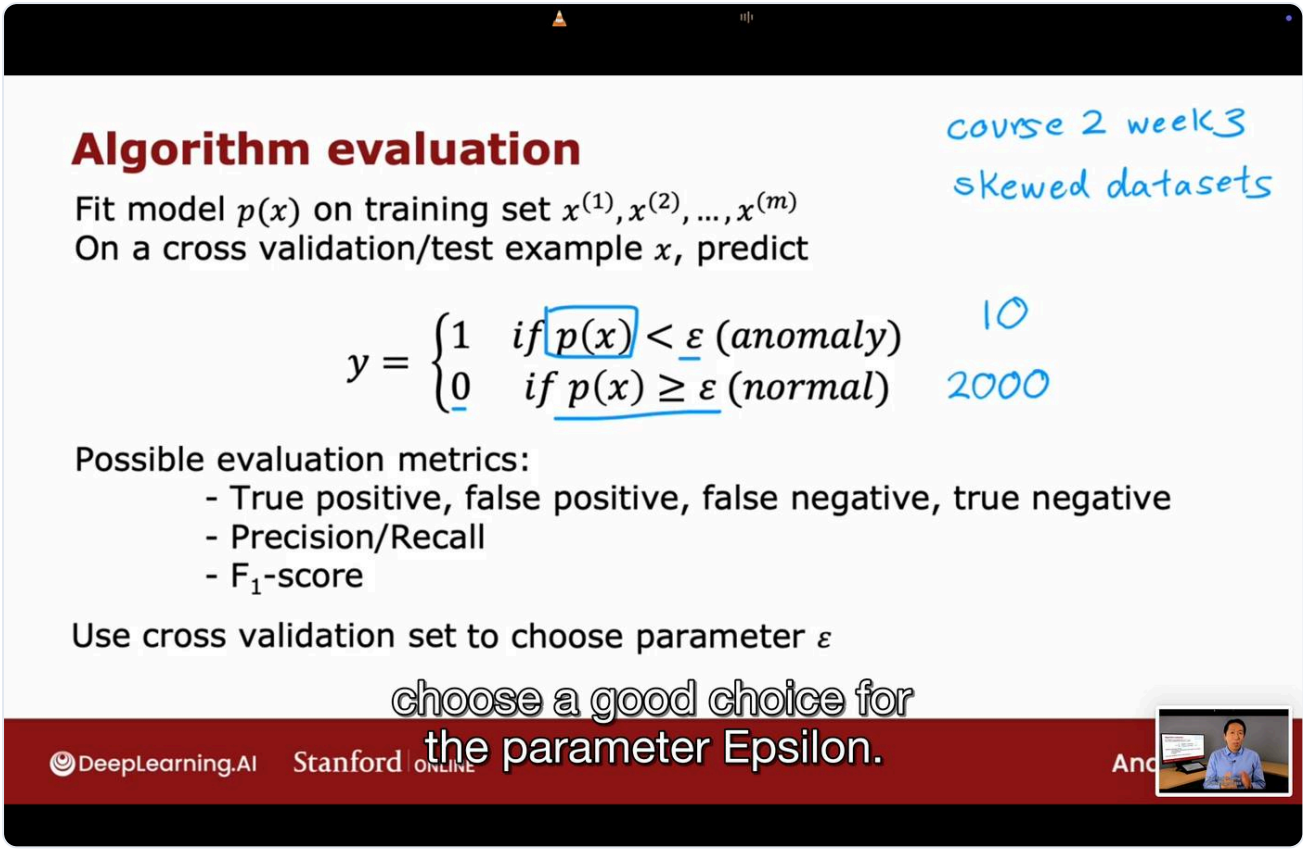


FIGURE 11.12 | Algorithm Evaluation

Then to just use that to try to choose a good choice for the parameter Epsilon. You find that the practical process of building an anomaly detection system is much easier if you actually have just a small number of labeled examples of known anomalies. Now, this does raise the question, if you have a few labeled examples, since you'll still be using an unsupervised learning algorithm, why not take those labeled examples and use a supervised learning algorithm instead? In the next video, let's take a look at a comparison between anomaly detection and supervised learning and when you might prefer one over the other. Let's go on to the next video.

LECTURE 12

Anomaly Detection

When you have a few positive examples with y = 1 and a large number of negative examples say y = 0? When should you use anomaly detection and when should you use supervised learning? The decision is actually quite subtle in some applications. So let me share with you some thoughts and some suggestions for how to pick between these two types of algorithms. And anomaly detection algorithm will typically be the more appropriate choice when you have a very small number of positive examples, 0-20 positive examples is not uncommon. And a relatively large number of negative examples with which to try to build a model for p of x. When you recall that the parameters for p of x are learned only from the negative examples and this much smaller.

So the positive examples is only used in your cross validation set and test set for parameter tuning and for evaluation. In contrast, if you have a larger number of positive and negative examples, then supervised learning might be more applicable. Now, even if you have only 20 positive training examples, it might be okay to apply a supervised learning algorithm. But it turns out that the way anomaly detection looks at the data set versus the way supervised learning looks at the data set are quite different. Here is the main difference, which is that if you think there are many different types of anomalies or many different types of positive examples.

Then anomaly detection might be more appropriate when there are many different ways for an aircraft engine to go wrong. And if tomorrow there may be a brand new way for an aircraft engine to have something wrong with it. Then your 20 say positive examples may not cover all of the ways that an aircraft engine could go wrong. That makes it hard for any algorithm to learn from the small set of positive examples what the anomalies, what the positive examples look like.

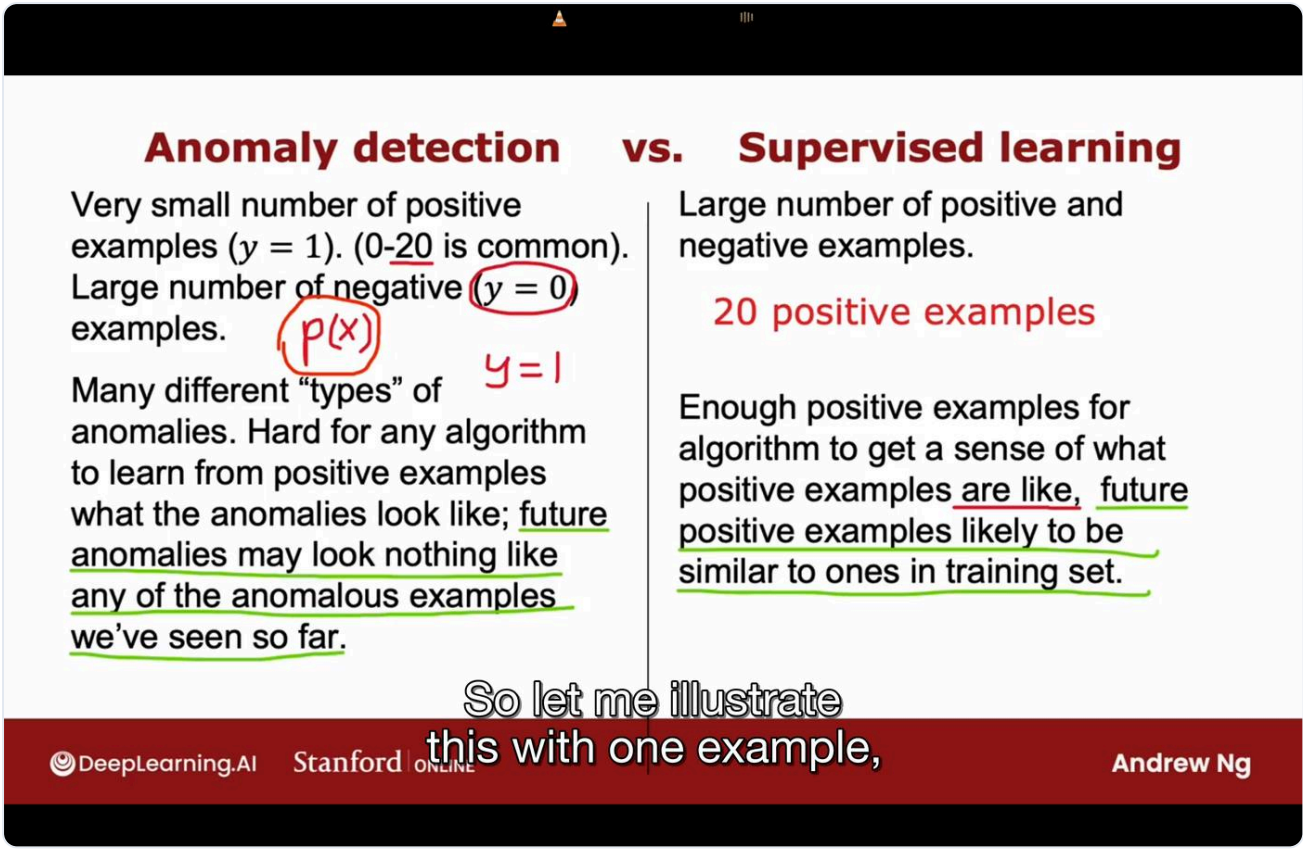


FIGURE 12.1 | Anomaly Detection vs. Supervised Learning

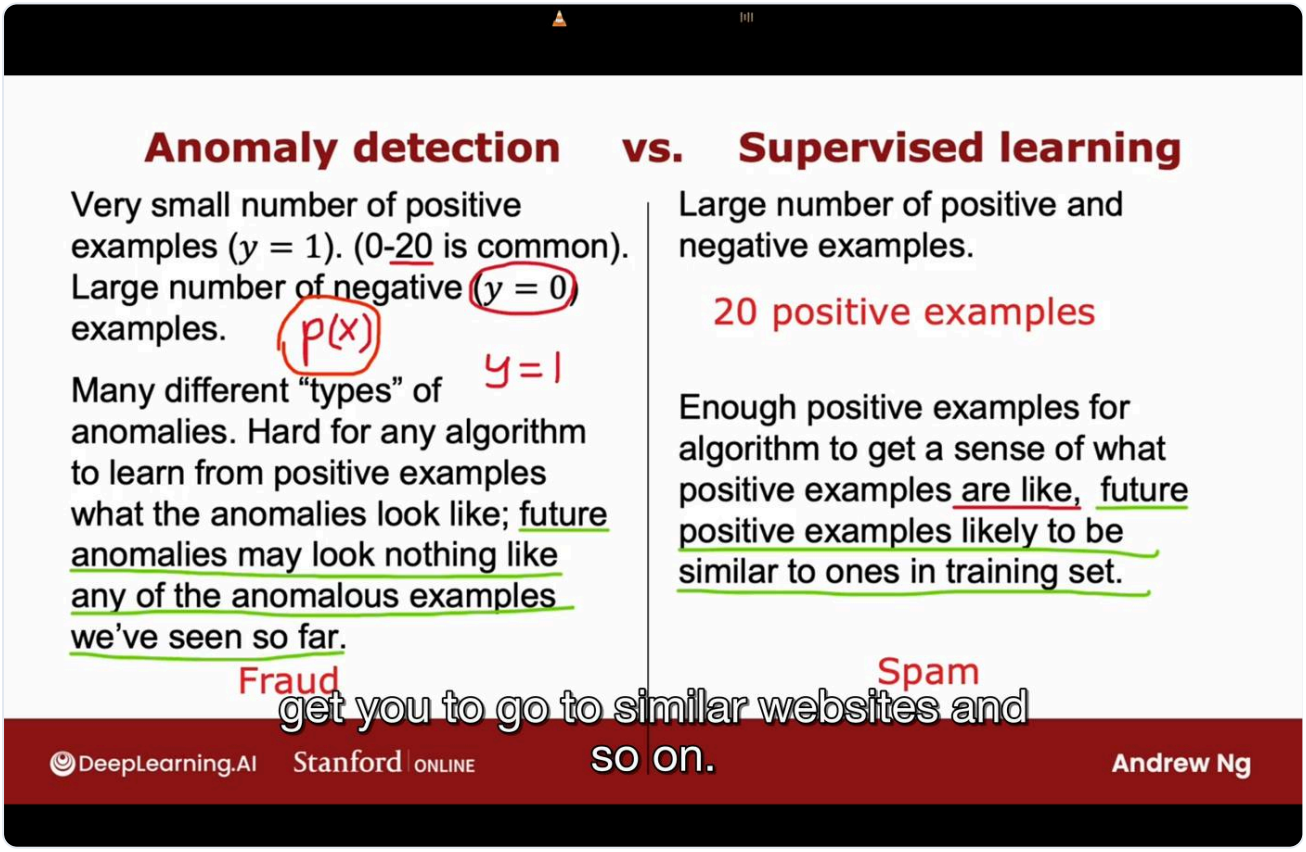


FIGURE 12.2 | Anomaly Detection vs. Supervised Learning

And future anomalies may look nothing like any of the anomalous examples we've seen so far. If you believe this to be true for your problem, then I would gravitate to using an anomaly detection algorithm. Because what anomaly detection does is it looks at the normal examples that is the y = 0 negative examples and just try to model what they look like. And anything that deviates a lot from normal It flags as an anomaly. Including if there's a brand new way for an aircraft engine to fail that had never been seen before in your data set. In contrast, supervised learning has a different way of looking at the problem. When you're applying supervised learning ideally you would hope to have enough positive examples for the average to get a sense of what the positive examples are like.

And with supervised learning, we tend to assume that the future positive examples are likely to be similar to the ones in the training set. So let me illustrate this with one example, if you are using a system to find, say financial fraud. There are many different ways unfortunately that some individuals are trying to commit financial fraud. And unfortunately there are new types of financial fraud attempts every few months or every year. And what that means is that because they keep on popping up completely new. And unique forms of financial fraud anomaly detection is often used to just look for anything that's different, then transactions we've seen in the past.

In contrast, if you look at the problem of email spam detection, well, there are many different types of spam email, but even over many years. Spam emails keep on trying to sell similar things or get you to go to similar websites and so on. Spam email that you will get in the next few days is much more likely to be similar to spam emails that you have seen in the past. So that's why supervised learning works well for spam because it's trying to detect more of the types of spam emails that you have probably seen in the past in your training set. Whereas if you're trying to detect brand new types of fraud that have never been seen before, then anomaly detection maybe more applicable.

Let's go through a few more examples. We have already seen fraud detection being one use case of anomaly detection. Although supervised learning is used to find previously observed forms of fraud. And we've seen email spam classification typically being address using supervised learning. You've also seen the example of manufacturing where you may want to find new previously unseen defects. Such as if there are brand new ways for an aircraft engine to fail in the future that you still want to detect. Even if you don't have any positive example like that in your training set. It turns out that the manufacturing supervised learning is also used to find defects.

The more for finding known and previously seen defects. For example, if you are a smartphone maker, you're making cell phones. And you know that occasionally your machine for making the case of the smartphone will accidentally scratch the cover. So scratches are a common defect on smartphones and so you can get enough training examples of scratched smartphones responding to label y =1. And just train the system to decide if a new smartphone that you just manufactured has any scratches in it. And the difference is if you just see scratched smartphones over and over and you want to check if your phones are scratched, then supervised learning works well.

Whereas if you suspect that they're going to be brand new ways for something to go wrong in the future, then anomaly detection will work well. Some other examples you've heard me talk about monitoring machines in the data center, especially the machine's been hacked. It can behave dif-

ferently in a brand new way unlike any previous way in his behavior. So that would feel more like an anomaly detection application.

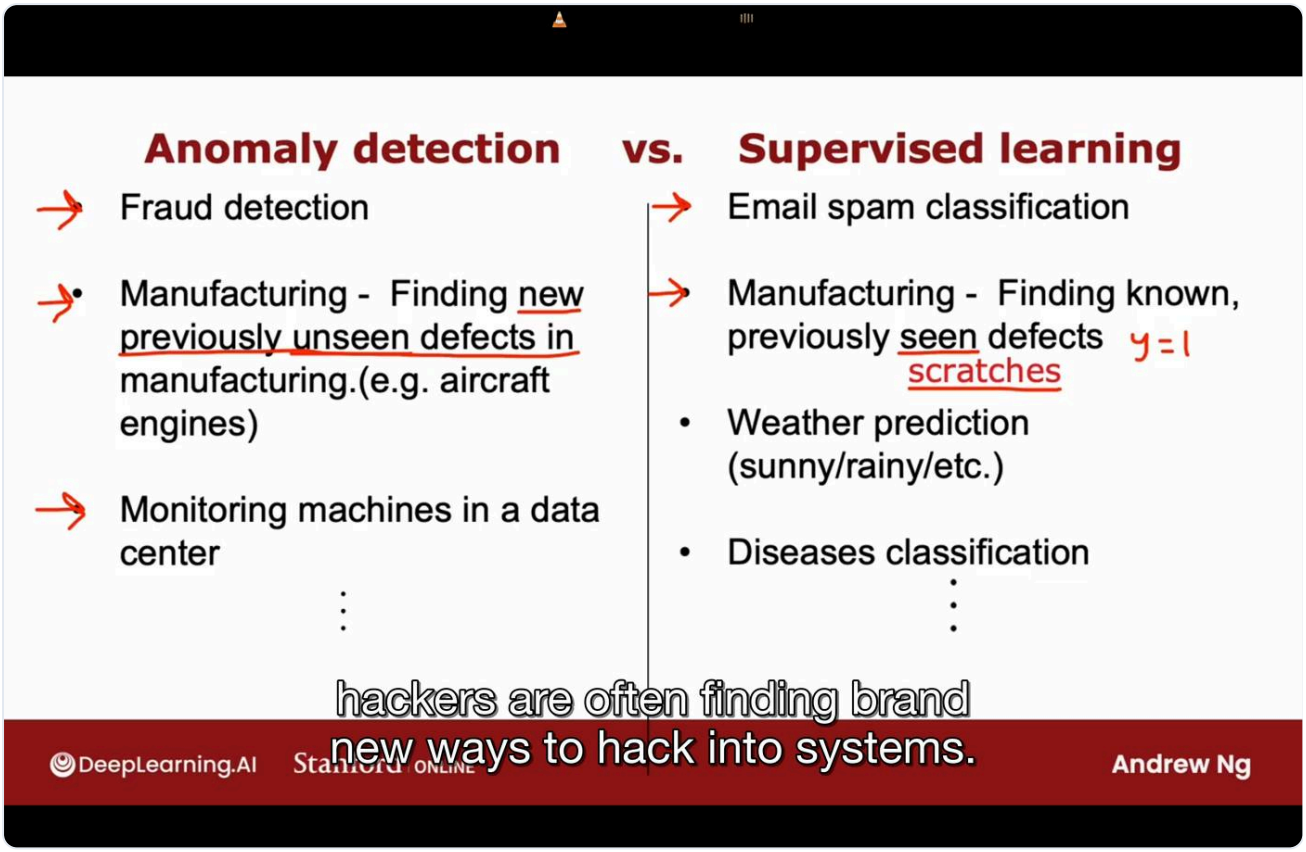


FIGURE 12.3 | Anomaly Detection

In fact, one theme is that many security related applications because hackers are often finding brand new ways to hack into systems. Many security related applications will use anomaly detection. Whereas returning to supervised learning, if you want to learn to predict the weather well, there's only a handful types of weather that you typically see. Is it sunny, rainy, is it going to snow? And so because you see the same output labels over and over, weather prediction would tend to be a supervised learning task. Or if you want to use the symptoms of the patient to see if the patient has a specific disease that you've seen before. Then that would also tend to be a supervised learning application.

So I hope that gives you a framework for deciding when you have a small set of positive examples as well as maybe a large set of negative examples. Whether to use anomaly detection or supervised learning. Anomaly detection tries to find brand new positive examples that may be unlike anything you've seen before. Where supervised learning looks at your positive examples and tries to decide if a future example is similar to the positive examples that you've already seen. Now, it turns out that when building an anomaly detection algorithm, the choice of features is very important and when building anomaly detection systems. I often spend a bit of time trying to tune the features I use for the system in the next video.

Let me share some practical tips on how to tune the features you feed to anomaly detection algorithm.

LECTURE 13

Error Analysis for Anomaly Detection

When building an anomaly detection algorithm, I found that choosing a good choice of features turns out to be really important. In supervised learning, if you don't have the features quite right, or if you have a few extra features that are not relevant to the problem, that often turns out to be okay. Because the algorithm has to supervised signal that is enough labels why for the algorithm to figure out what features ignore, or how to re scale feature and to take the best advantage of the features you do give it. But for anomaly detection which runs, or learns just from unlabeled data, is harder for the algorithm to figure out what features to ignore.

So I found that carefully choosing the features, is even more important for anomaly detection, than for supervised learning approaches. Let's take a look at this video as some practical tips, for how to tune the features for anomaly detection, to try to get you the best possible performance. One step that can help your anomaly detection algorithm, is to try to make sure the features you give it are more or less Gaussian. And if your features are not Gaussian, sometimes you can change it to make it a little bit more Gaussian. Let me show you what I mean. If you have a feature X, I will often plot a histogram of the feature which you can do using the python command PLT.

Though you see this in the practice lab as well, in order to look at the histogram of the data. This distribution here looks pretty Gaussian. So this would be a good candidate feature. If you think this is a feature that helps distinguish between anomalies and normal examples. But quite often when you plot a histogram of your features, you may find that the feature has a distribution like this. This does not at all look like that symmetric bell shaped curve. When that is the case, I would consider if you can take this feature X, and transform it in order to make a more Gaussian. For example, maybe if you were to compute the log of X and plot a histogram of log of X, look like this, and this looks much more Gaussian.

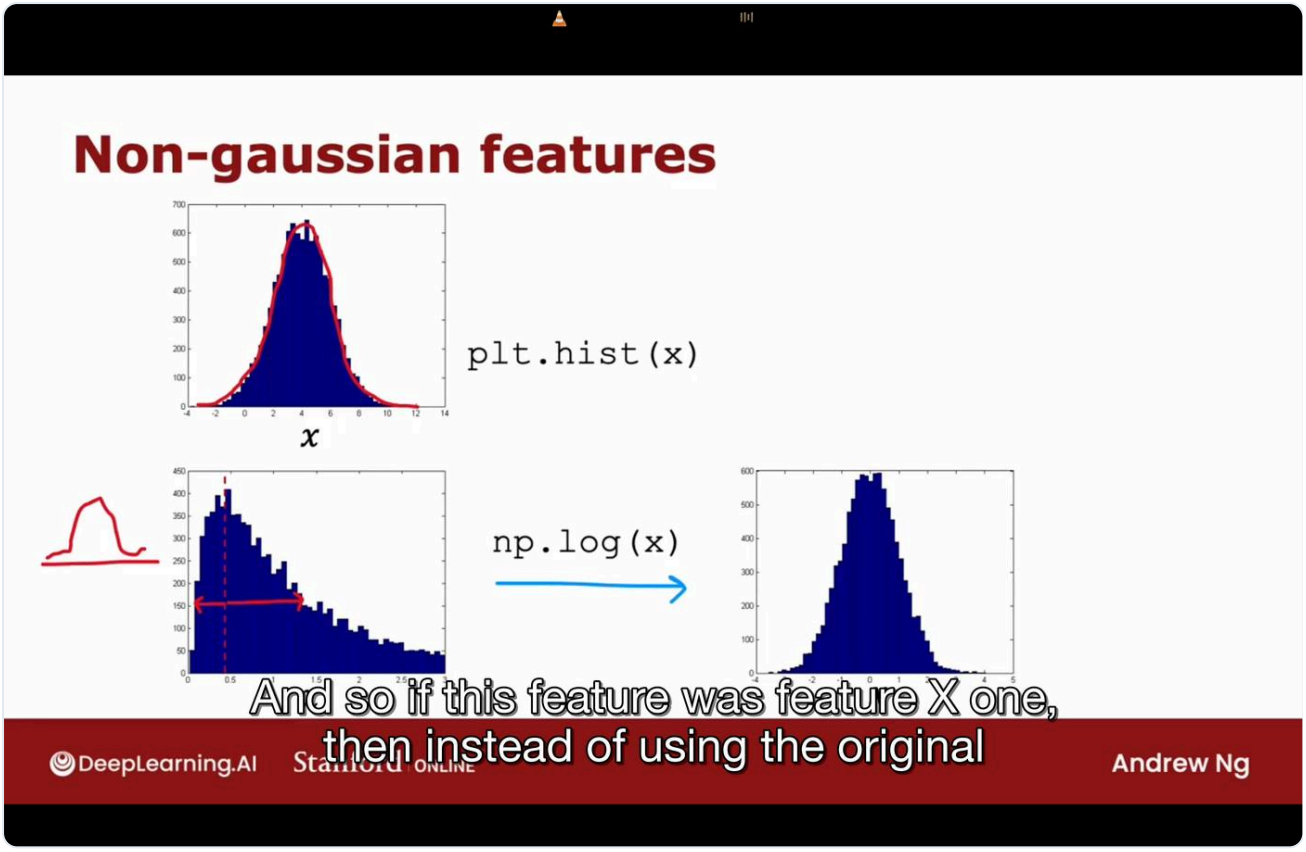


FIGURE 13.1

And so if this feature was feature X one, then instead of using the original feature X one which looks like this on the left, you might instead replace that feature with log of X one, to get this distribution over here. Because when X one is made more Gaussian. When anomaly detection models P of X one using a Gaussian distribution like that, is more likely to be a good fit to the data. Other than the log function, other things you might do is, given a different feature X two, you may replace it with X two, log of X two plus one. This would be a different way of transforming X two. And more generally, log of X two plus C, would be one example of a formula you can use, to change X to try to make it more Gaussian.

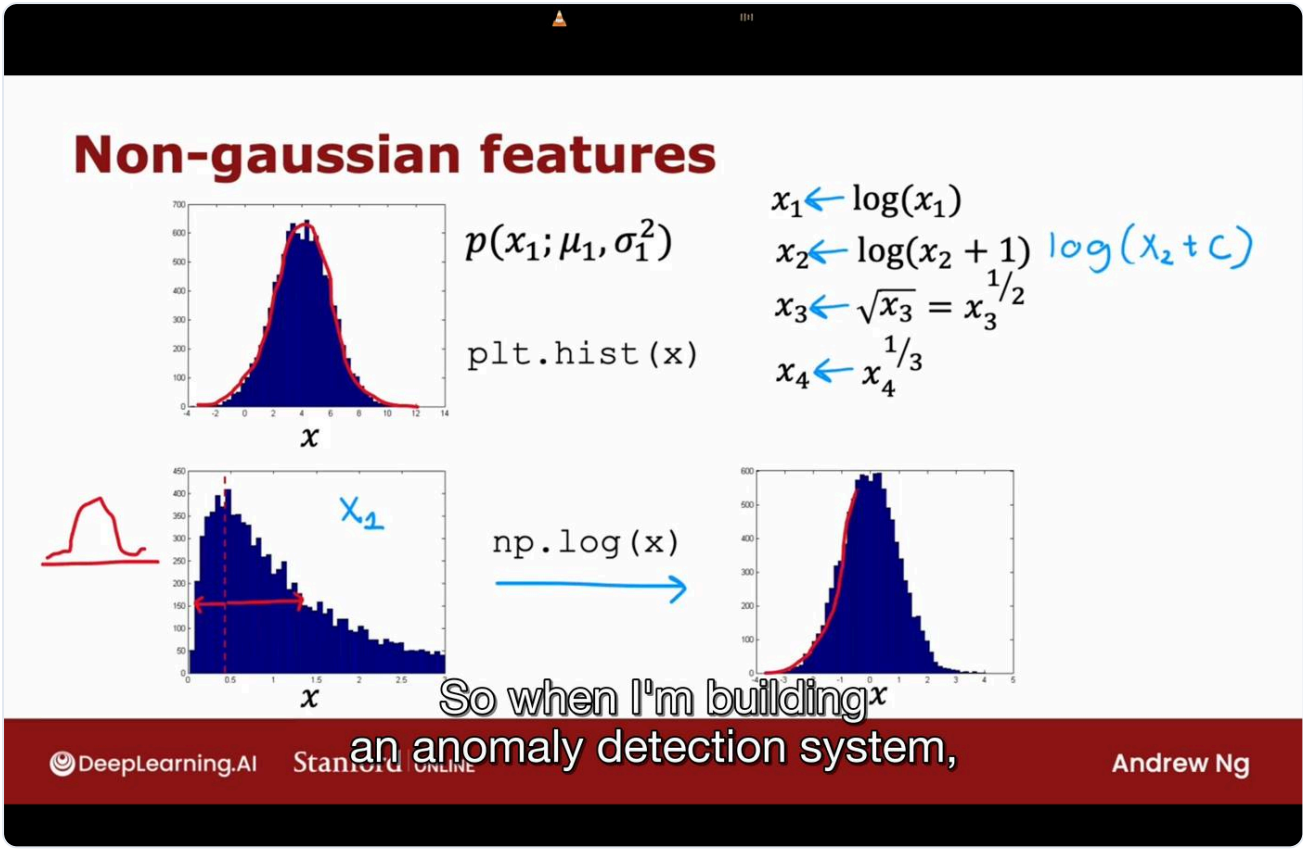


FIGURE 13.2

Or for a different feature, you might try taking the square root or really the square would have executed this X lead to the power of one half,and you may change that exponentially term. So for a different feature X four, you might use X four to the power of one third, for example. So when I'm building an anomaly detection system, I'll sometimes take a look at my features, and if I see any highly non Gaussian by plotting histogram, I might choose transformations like these or others, In order to try to make it more Gaussian. It turns out a larger value of C, will end up transforming this distribution less. But in practice I just try a bunch of different values of C, and then try to take a look to pick one that looks better in terms of making the distribution more Gaussian.

Now, let me illustrate how I actually do this and that you put a notebook. So this is what the process of exploring different transformations in the features might look like.

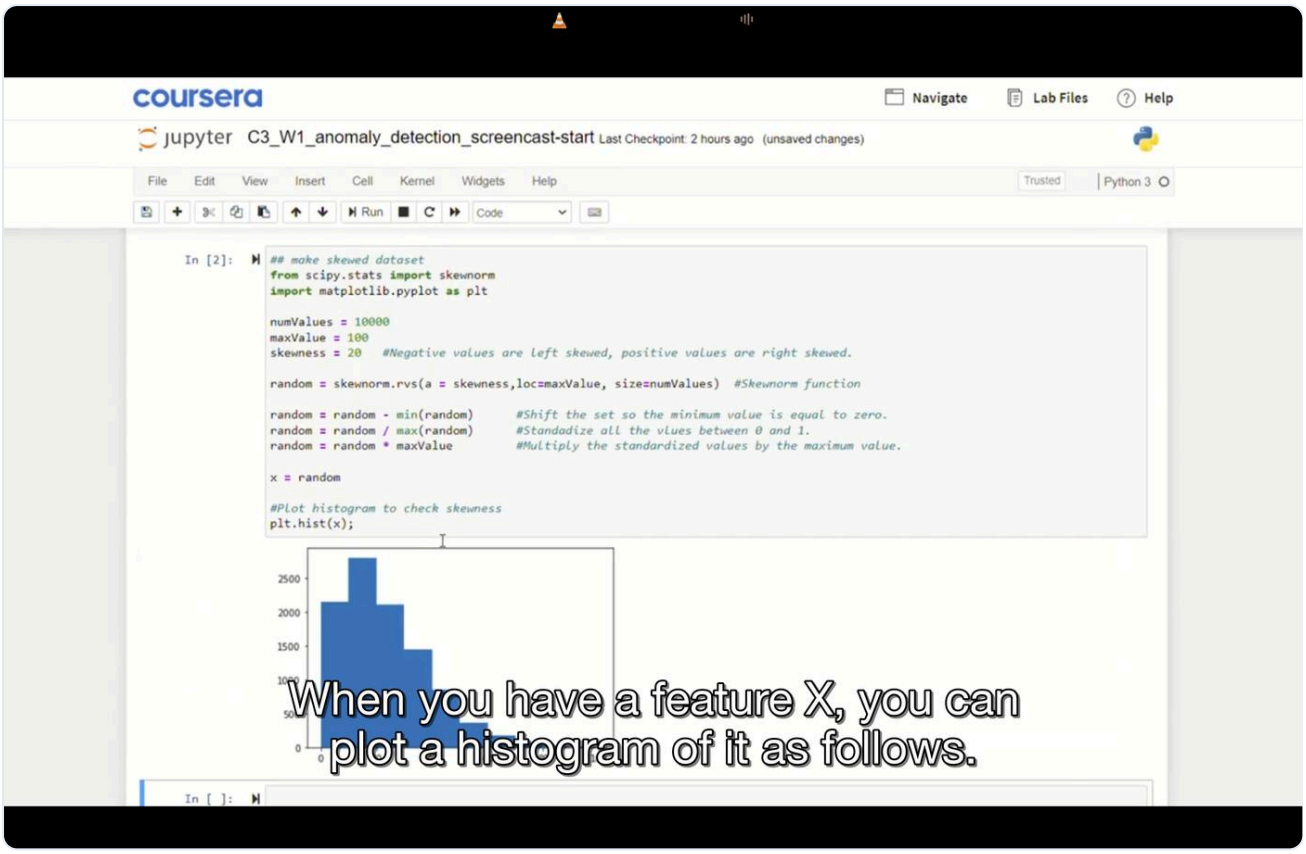


FIGURE 13.3

When you have a feature X, you can plot a histogram of it as follows. It actually looks like there's a pretty cause histogram. Let me increase the number of bins in my history gram to 50.

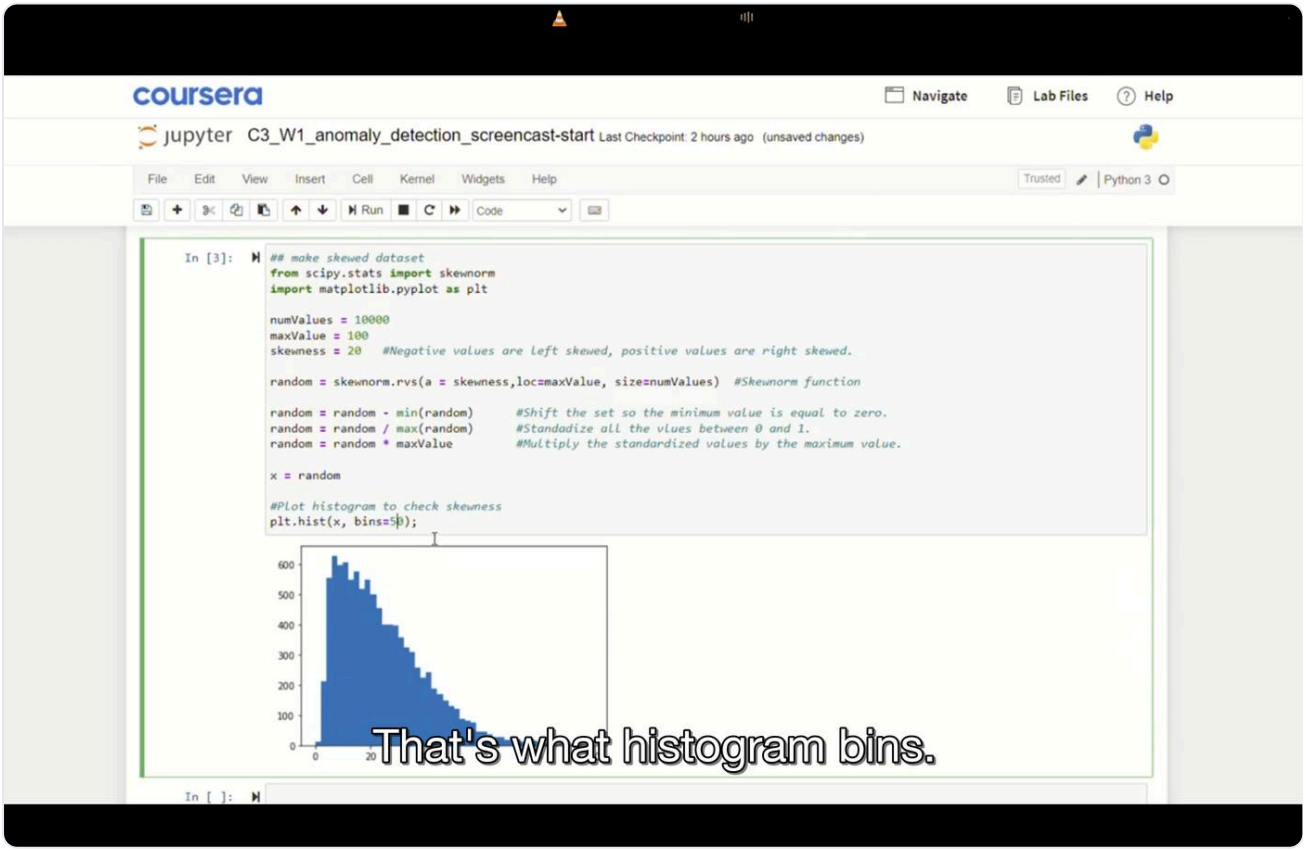


FIGURE 13.4

So bins equals 50 there. That's what histogram bins. And by the way, if you want to change the color, you can also do so as follows.

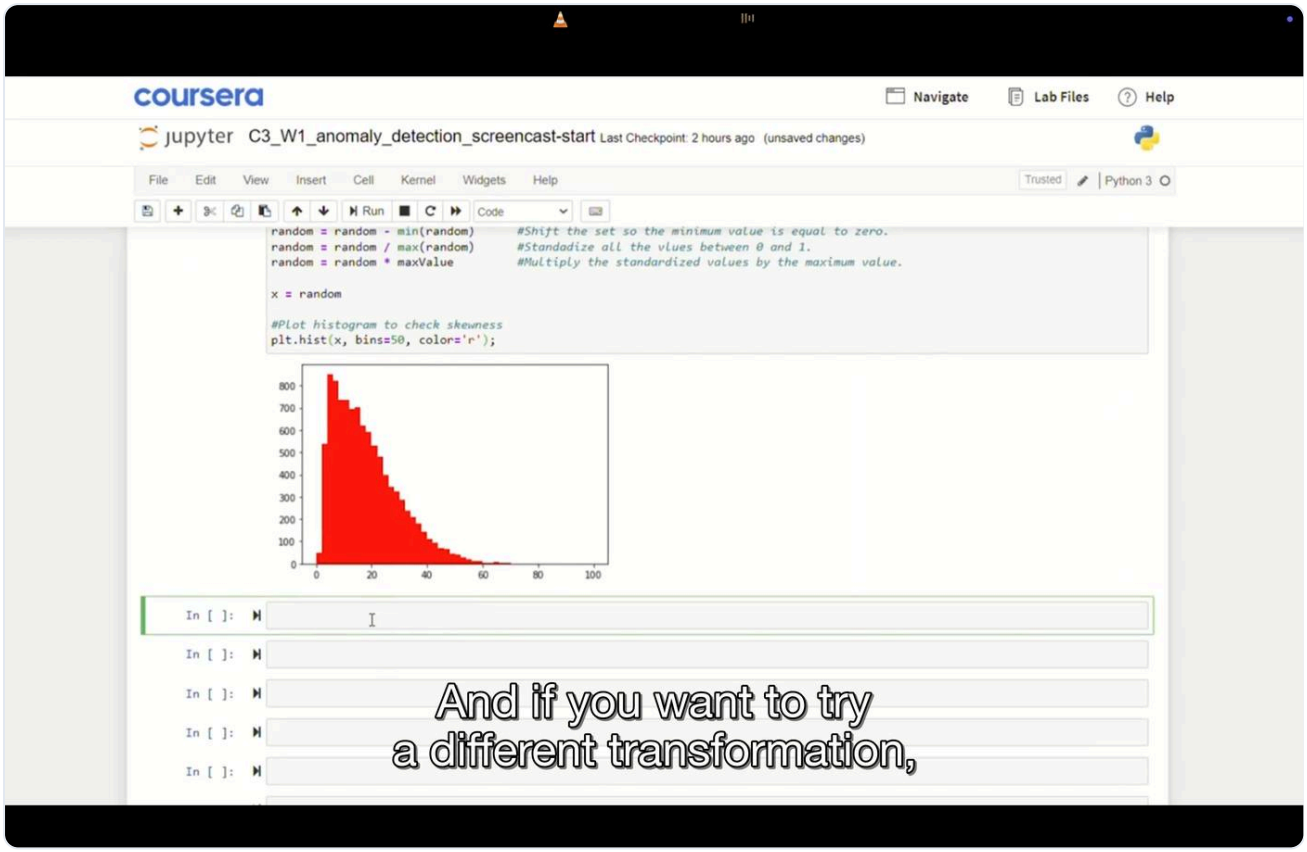


FIGURE 13.5

And if you want to try a different transformation, you can try for example to plot X square root of X. So X to the power of 0.5 with again 50 histogram bins, in which case it might look like this. And this actually looks somewhat more Gaussian.

FIGURE 13.6

But not perfectly, and let's try a different parameter. So let me try to the power of 4.25.

FIGURE 13.7

Maybe I just a little bit too far. It's the old 0.4 that looks pretty Gaussian.

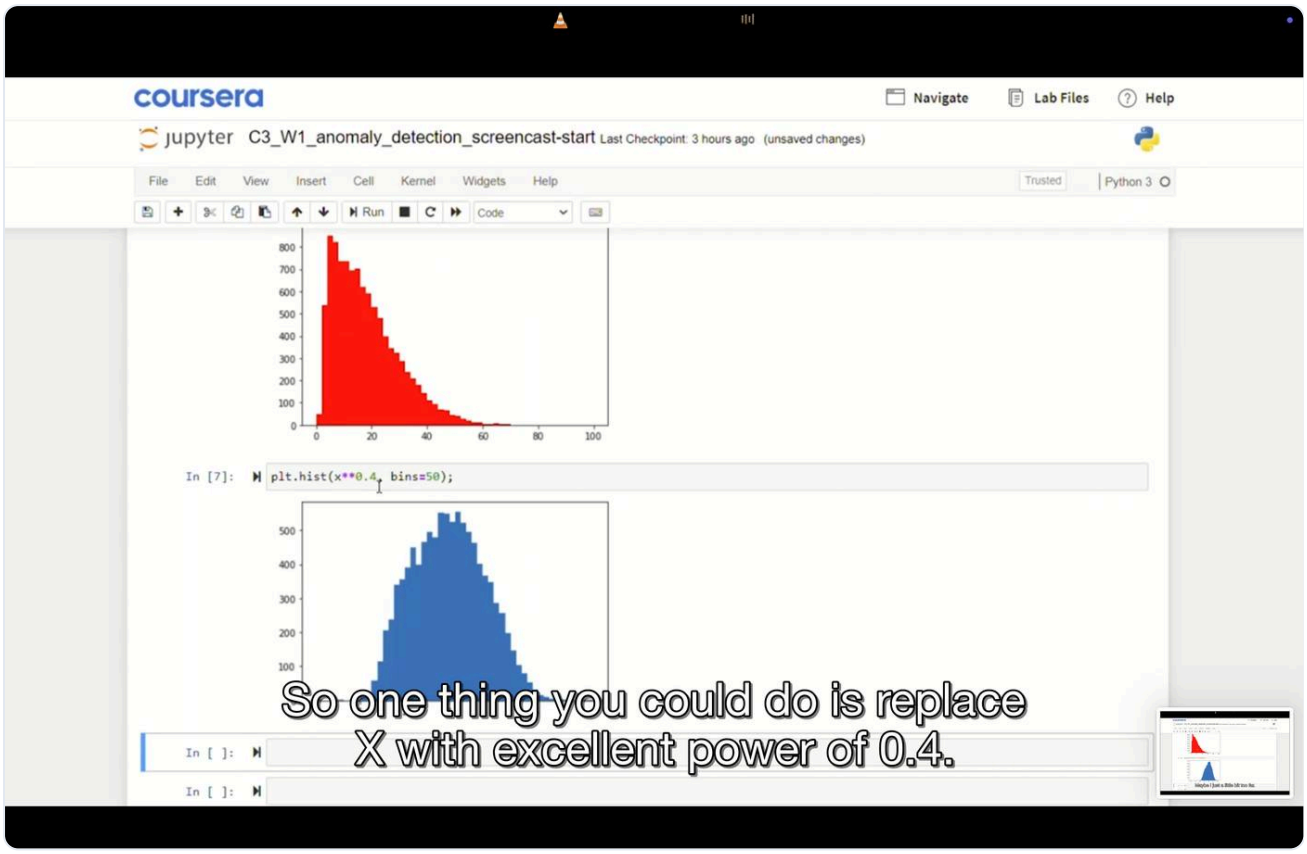


FIGURE 13.8

So one thing you could do is replace X with excellent power of 0.4. And so you would set X to be equal to X to the power of 0.4. And just use the value of X in your training process instead.

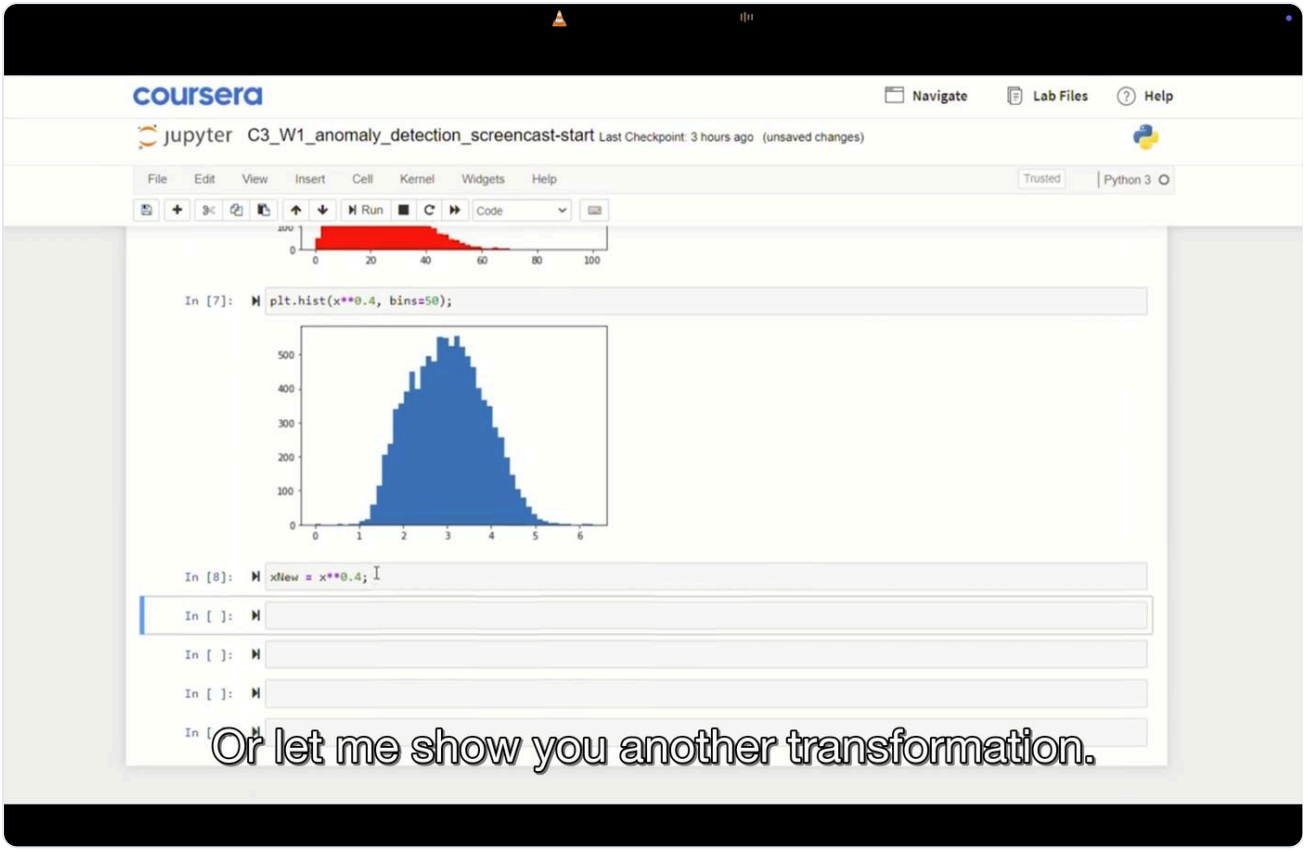


FIGURE 13.9

Or let me show you another transformation. Here, I'm going to try taking the log of X. So log of X spotted with 50 bins, I'm going to use the numpy log function as follows. And it turns out you get an error, because it turns out that excellent. This example has some values that are equal to zero, and we'll log of zero is negative infinity is not defined. So common trick is to add just a very tiny number there.

FIGURE 13.10

So exports 0.001, becomes non negative. And so you get the histogram that looks like this. But if you want the distribution to look more Gaussian, you can also play around with this parameter, to try to see if there's a value of that.

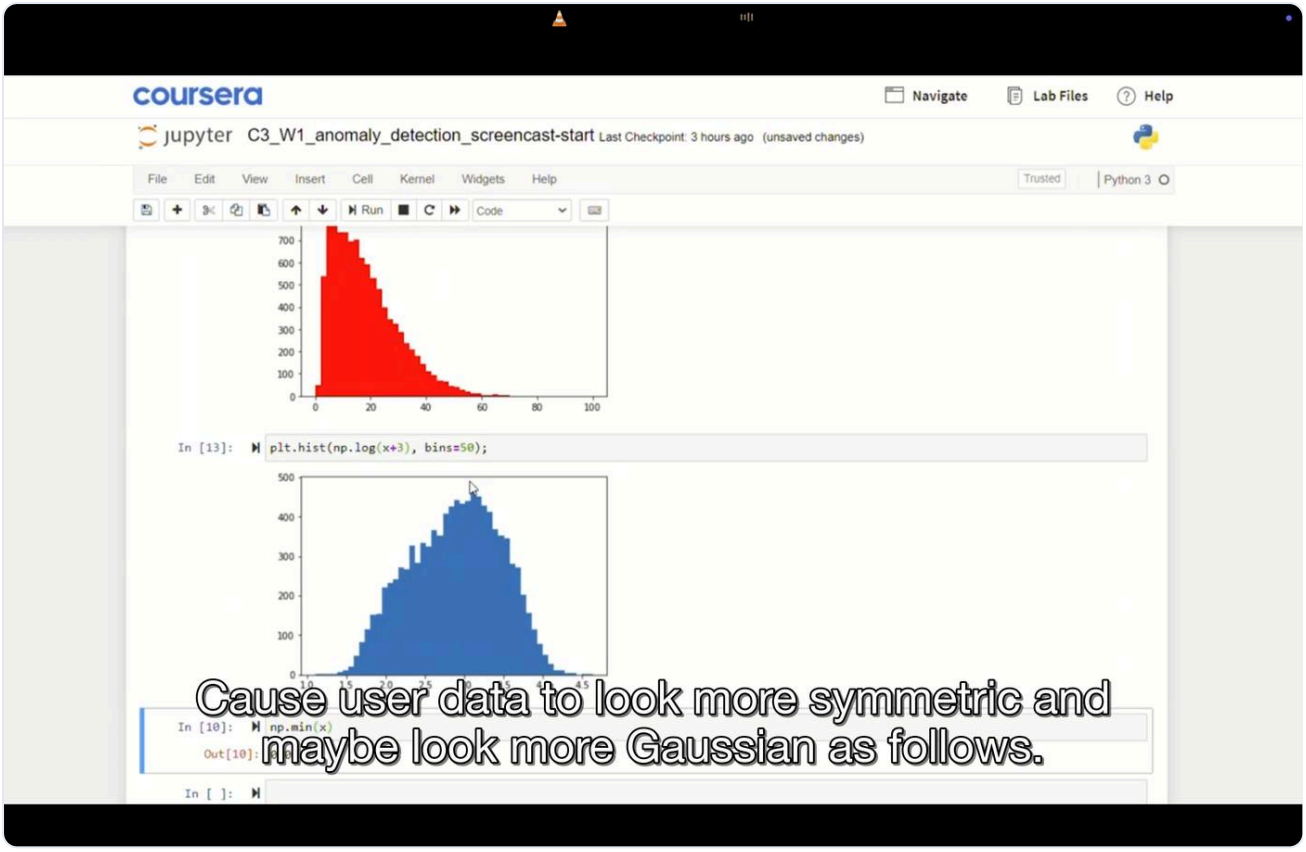


FIGURE 13.11

Cause user data to look more symmetric and maybe look more Gaussian as follows.

FIGURE 13.12

And just as I'm doing right now in real time, you can see that, you can very quickly change these parameters and plot the histogram.

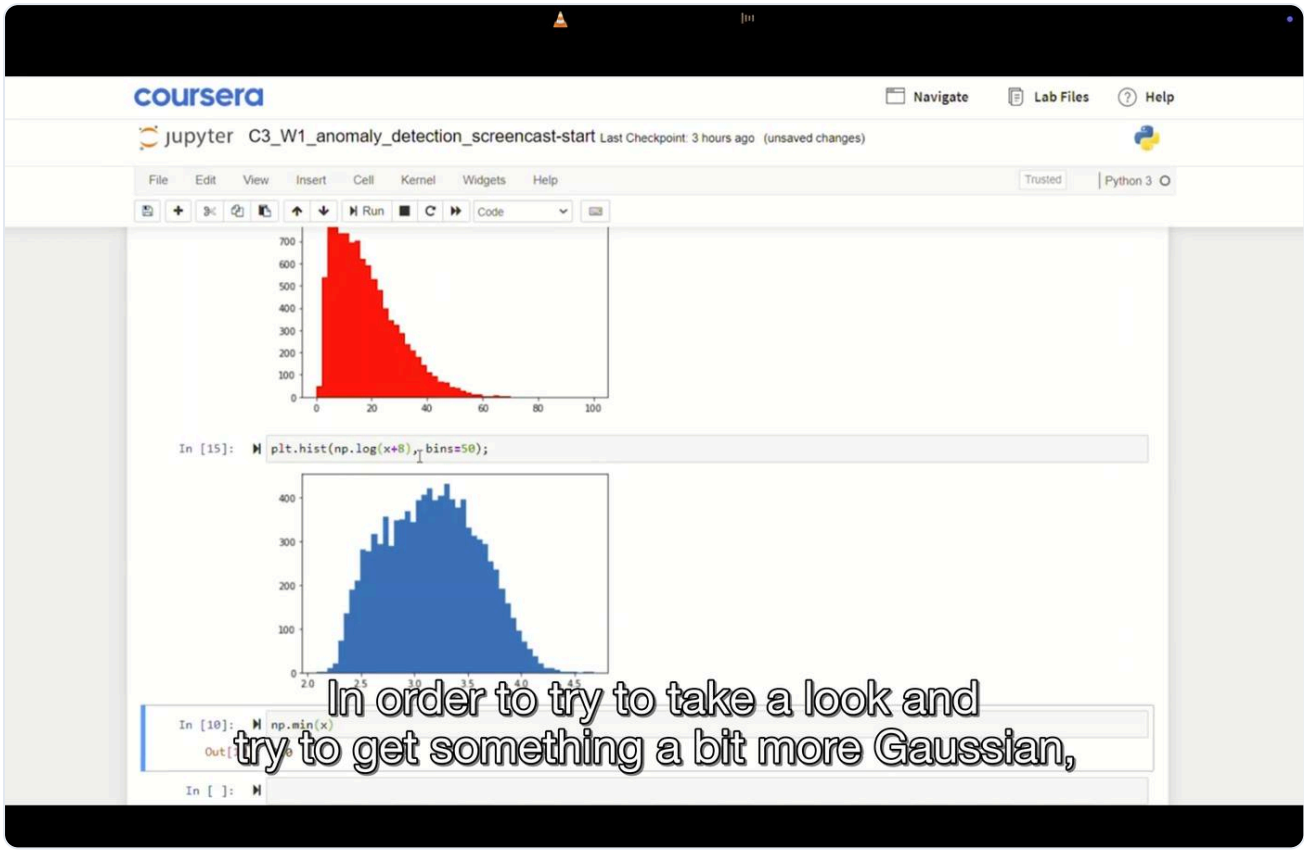


FIGURE 13.13

In order to try to take a look and try to get something a bit more Gaussian, than was the original data next that you saw in this histogram up above. If you read the machine learning literature, there are some ways to automatically measure how close these distributions are to Gaussian. But I found it in practice, it doesn't make a big difference, if you just try a few values and pick something that looks right to you, that will work well for all practical purposes. So, by trying things out in Jupyter notebook, you can try to pick a transformation that makes your data more Gaussian. And just as a reminder, whatever transformation you apply to the training set, please remember to apply the same transformation to your cross validation and test set data as well.

Other than making sure that your data is approximately Gaussian, after you've trained your anomaly detection algorithm, if it doesn't work that well on your cross validation set, you can also carry out an error analysis process for anomaly detection. In other words, you can try to look at where the algorithm is not yet doing well whereas making errors, and then use that to try to come up with improvements. So as a reminder, what we want is for P of X to be large. For normal examples X, so greater than equal to epsilon, and p f X to be small or less than epsilon, for the anomalous examples X. When you've learned the model P of X from your unlabeled data, the most common problem that you may run into is that, P of X is comparable in value say is, large for both normal and for anomalous examples.

As a concrete example, if this is your data set, you might fit that Gaussian into it. And if you have an example in your cross validation set or test set, that is over here, that is anomalous, then this has a pretty high probability.

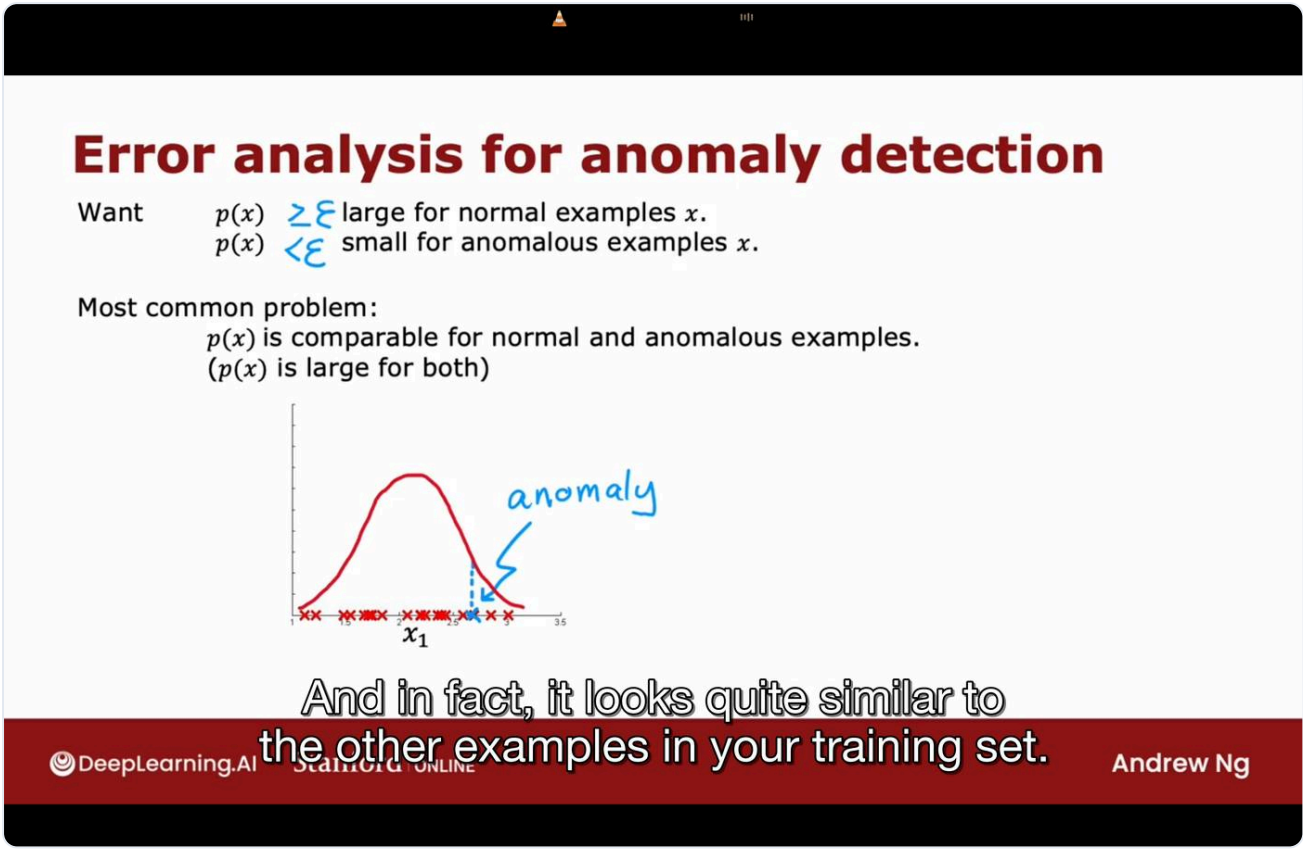


FIGURE 13.14 | Error Analysis for Anomaly Detection

And in fact, it looks quite similar to the other examples in your training set. And so, even though this is an anomaly, P of X is actually pretty large. And so the algorithm will fail to flag this particular example as an anomaly. In that case, what I would normally do is, try to look at that example and try to figure out what is it that made me think is an anomaly, even if this feature X one took on values similar to other training examples. And if I can identify some new feature say X two, that helps distinguish this example from the normal examples. Then adding that feature, can help improve the performance of the algorithm. Here's a picture showing what I mean.

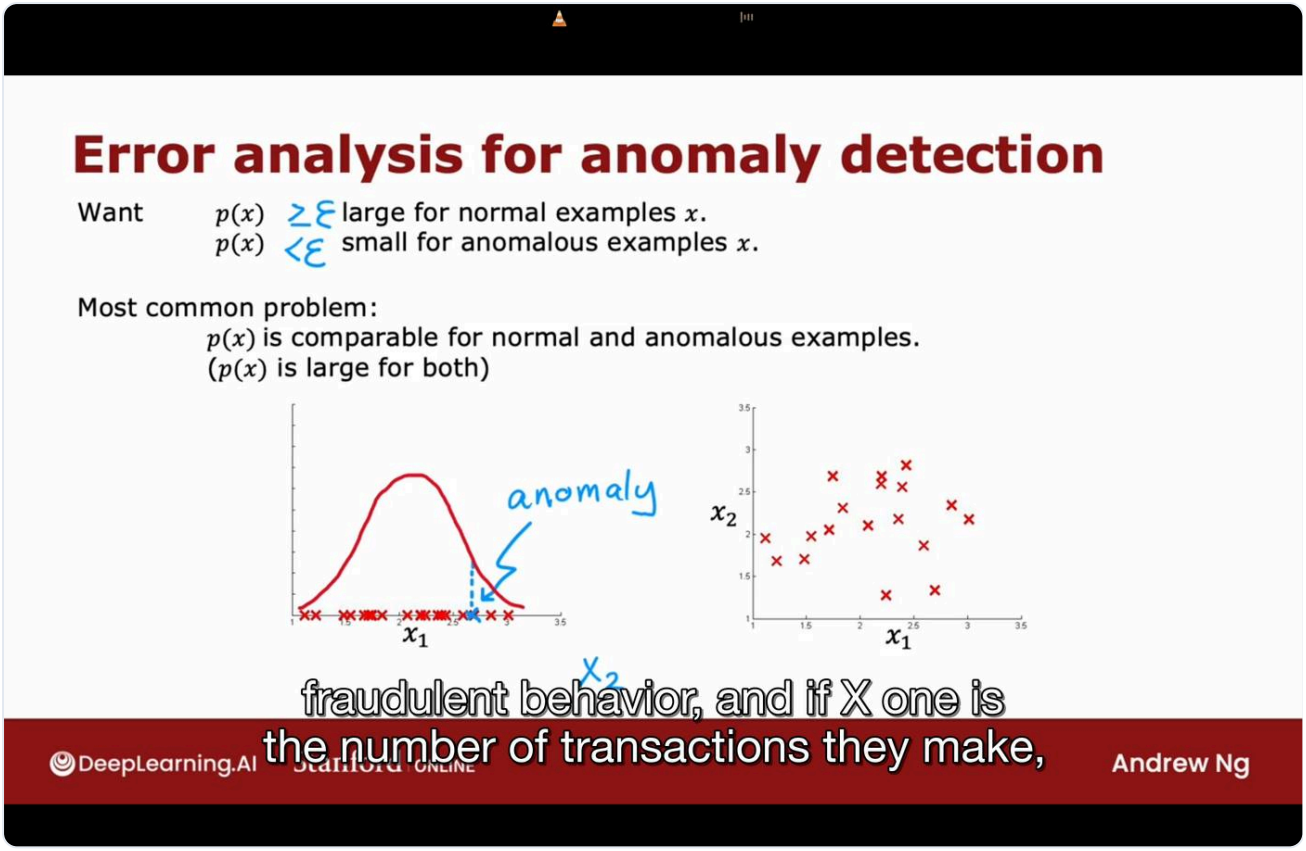


FIGURE 13.15 | Error Analysis for Anomaly Detection

If I can come up with a new feature X two, say, I'm trying to detect fraudulent behavior, and if X one is the number of transactions they make, maybe this user looks like they're making some of the transactions as everyone else. But if I discover that this user has some insanely fast typing speed, and if I were to add a new feature X two, that is the typing speed of this user. And if it turns out that when I plot this data using the old feature X one and this new feature X two, causes X two to stand out over here.

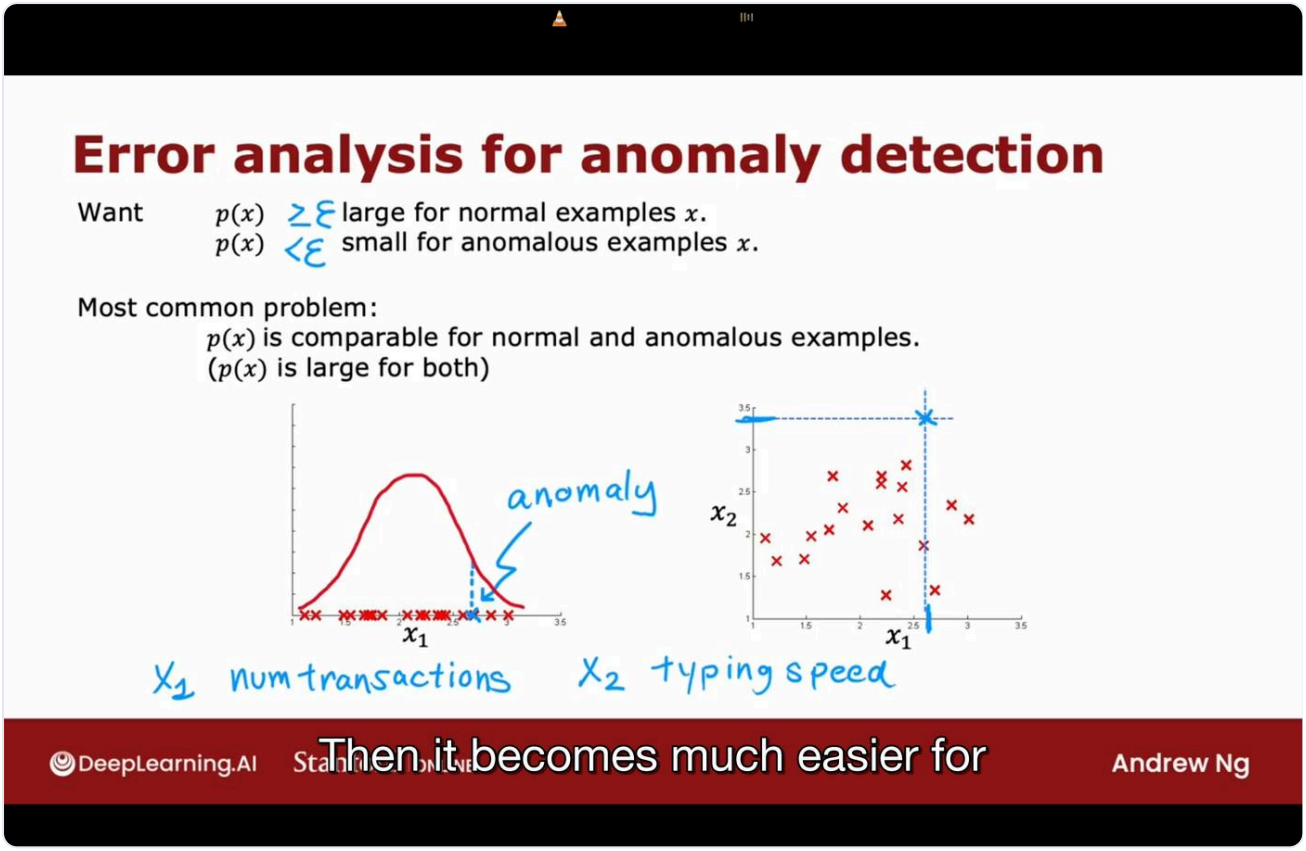


FIGURE 13.16 | Error Analysis for Anomaly Detection

Then it becomes much easier for the anomaly detection algorithm to recognize an X two is an anomalous user. Because when you have this new feature X two, the learning algorithm may fit a Gaussian distribution that assigns high probability to points in this region, a bit lower in this region, and a bit lower in this region.

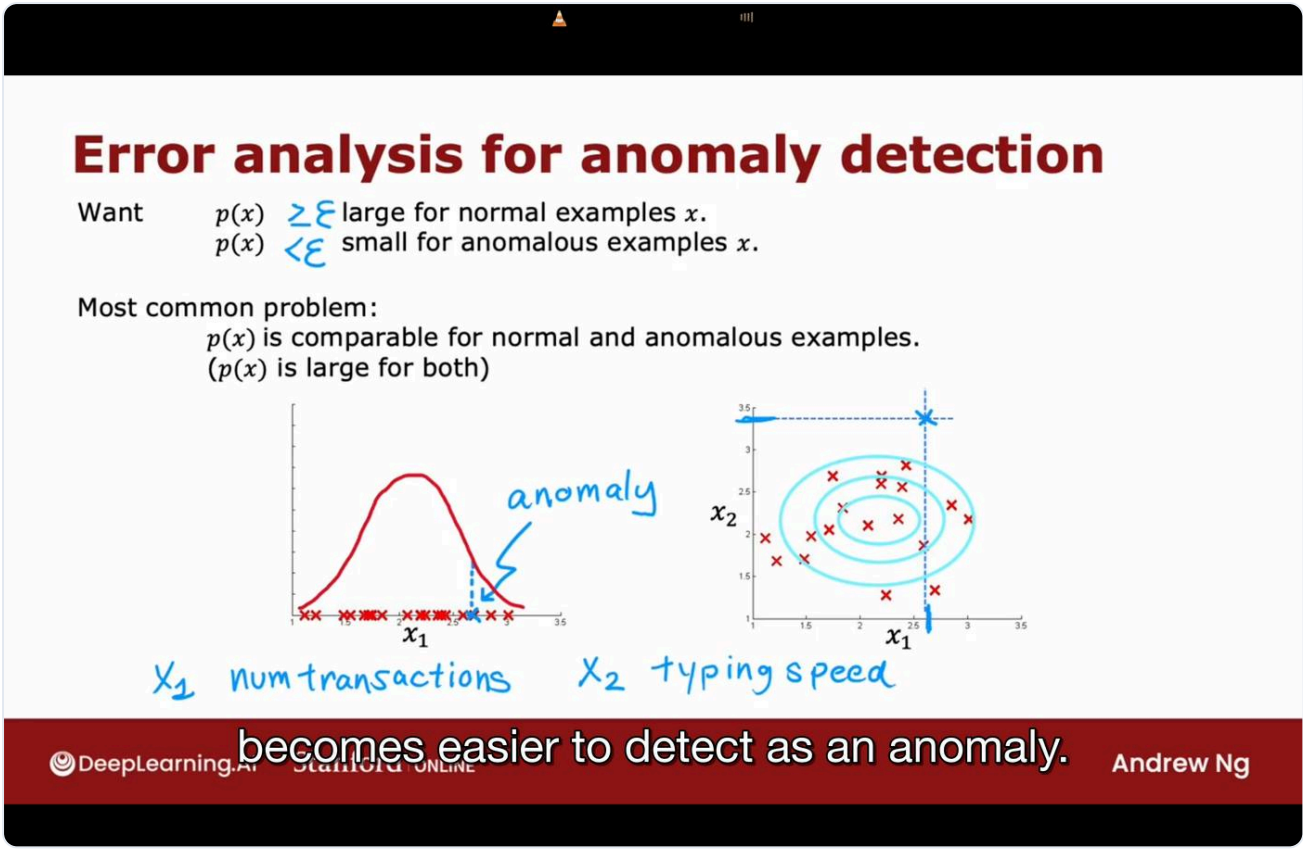


FIGURE 13.17 | Error Analysis for Anomaly Detection

And so this example, because of the very anomalous value of X two, becomes easier to detect as an anomaly. So just to summarize the development process will often go through is, to train the model and then to see what anomalies in the cross validation set the algorithm is failing to detect. And then to look at those examples to see if that can inspire the creation of new features that would allow the algorithm to spot. That example takes on unusually large or unusually small values on the new features, so that you can now successfully flag those examples as anomalies. Just as one more example, let's say you're building an anomaly detection system to monitor computers in the data center.

To try to figure out if a computer may be behaving strangely and deserves a closer look, maybe because of a hardware failure, or because it's been hacked into or something.

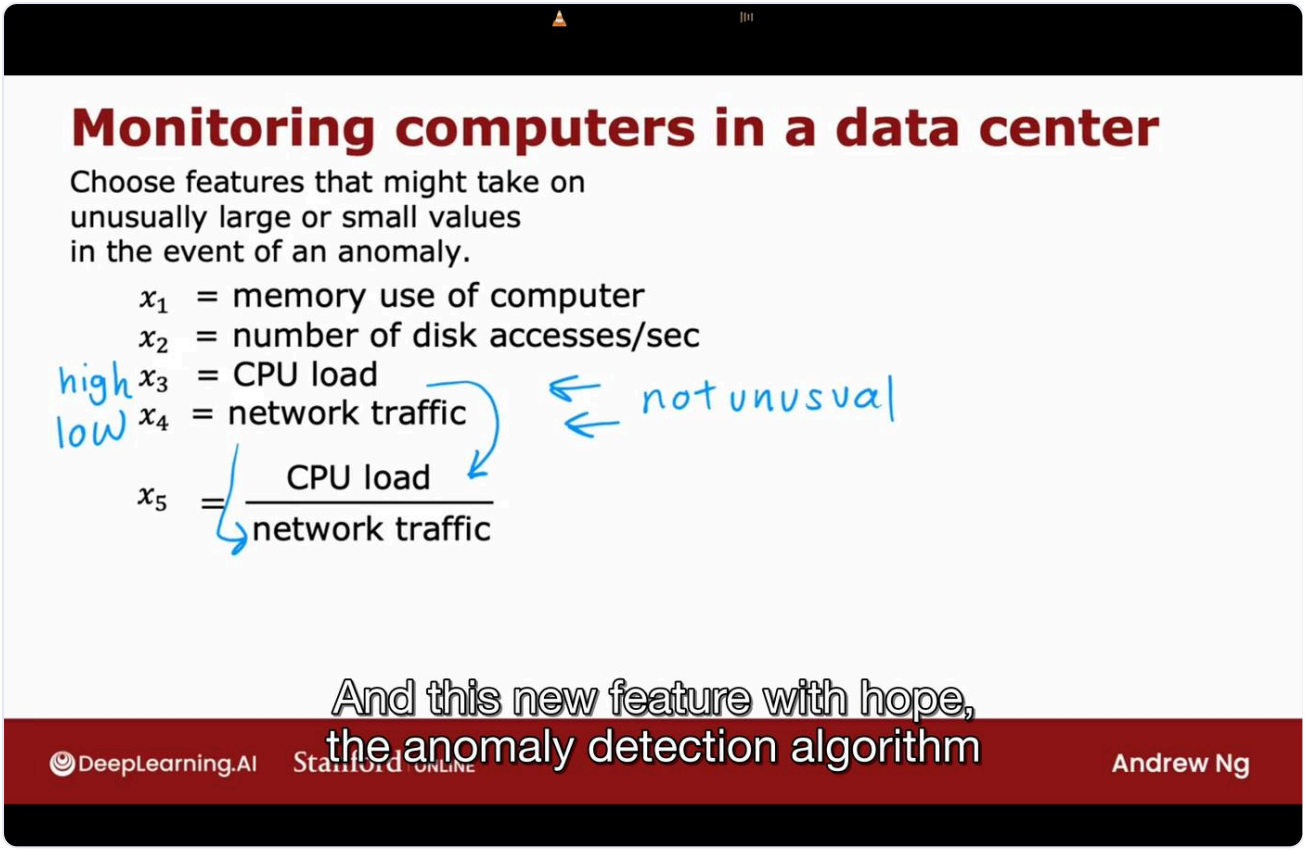


FIGURE 13.18 | Monitoring Computers in a Data Center

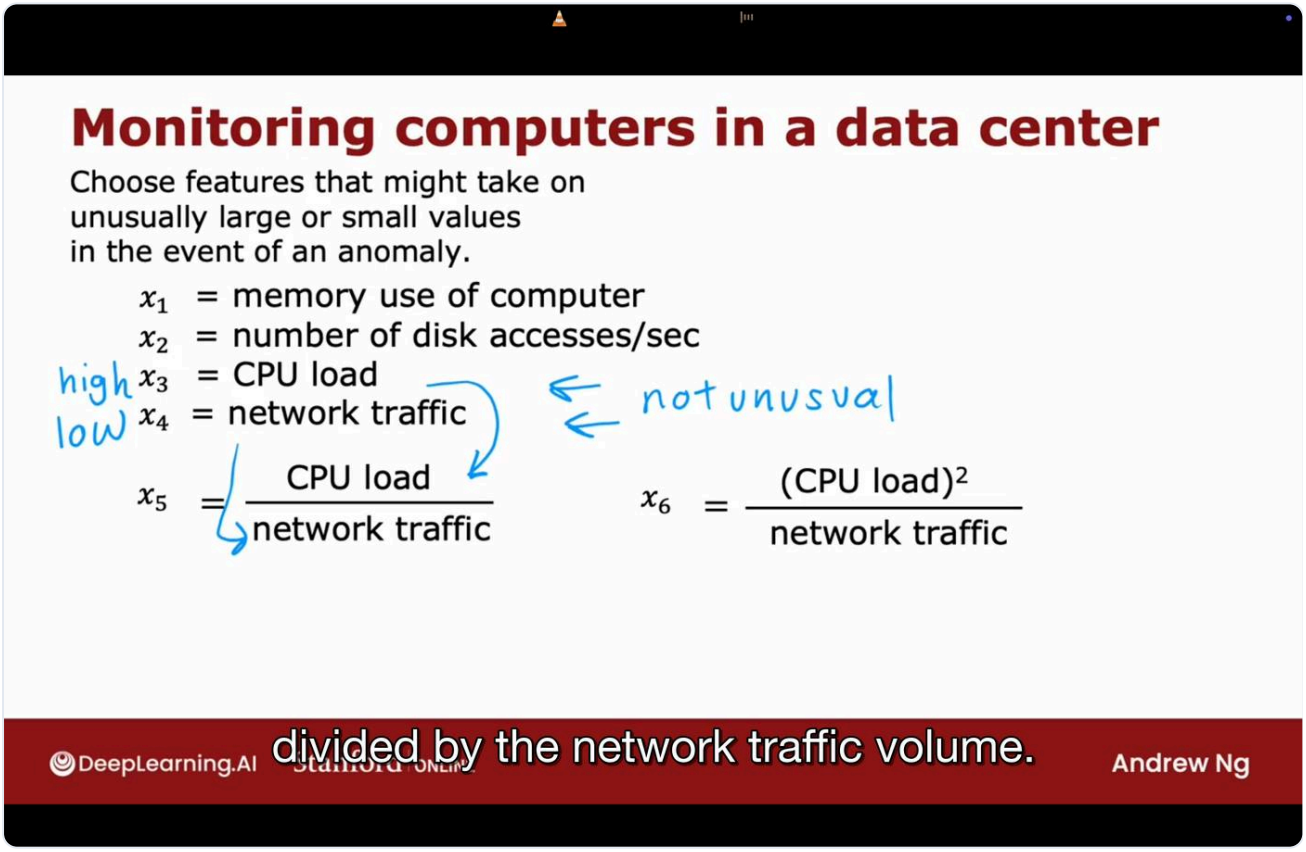


FIGURE 13.19 | Monitoring Computers in a Data Center

So what you'd like to do is, to choose features that might take on unusually large or small values in the event of an anomaly. You might start off with features like X one is the memory use, X two is the number of disk accesses per second, then the CPU load, and the volume of network traffic. And if you train the algorithm, you may find that it detects some anomalies but fails to detect some other anomalies. In that case, it's not unusual to create new features by combining old features. So, for example, if you find that there's a computer that is behaving very strangely, but neither is CPU load nor network traffic is that unusual. But what is unusual is, it has a really high CPU load, while having a very low network traffic volume.

If you're running the data center that streams videos, then computers may have high CPU load and high network traffic, or low CPU load and no network traffic. But what's unusual about this one machine is a very high CPU load, despite a very low traffic volume. In that case, you might create a new feature X five, which is a ratio of CPU load to network traffic. And this new feature with hope, the anomaly detection algorithm flagged future examples like the specific machine you may be seeing as anomalous. Or you can also consider other features like the square of the CPU load, divided by the network traffic volume. And you can play around with different choices of these features.

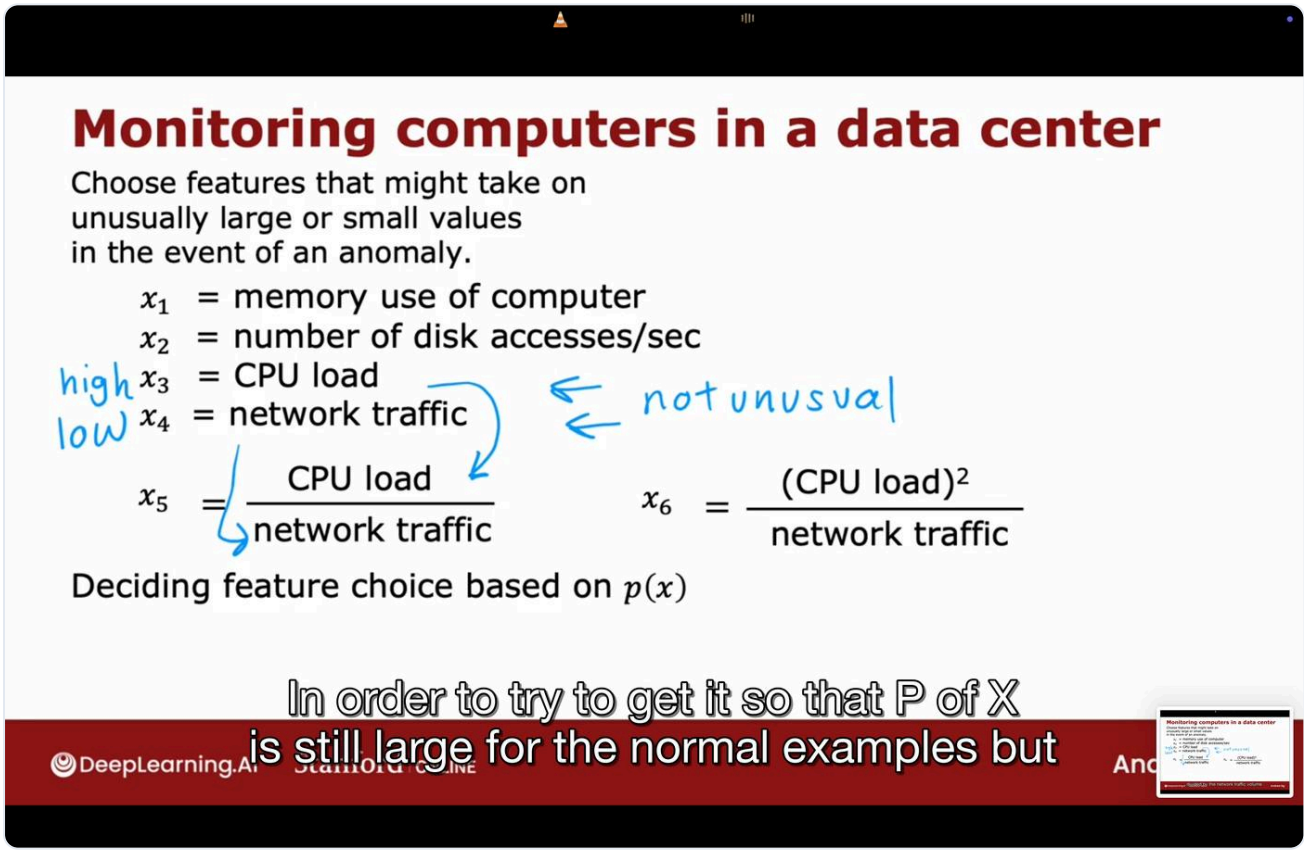


FIGURE 13.20 | Monitoring Computers in a Data Center

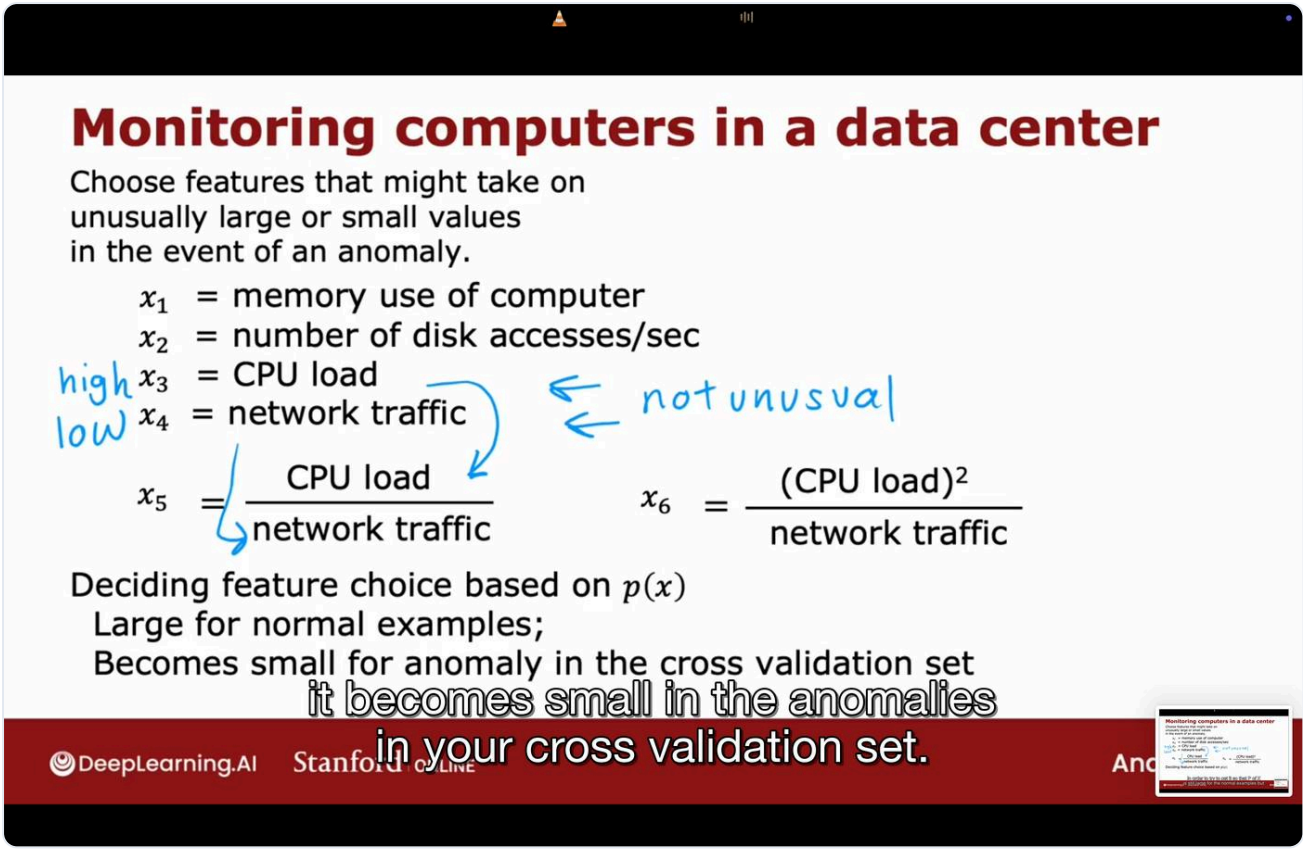


FIGURE 13.21 | Monitoring Computers in a Data Center

In order to try to get it so that P of X is still large for the normal examples but it becomes small in the anomalies in your cross validation set. So that's it. Thanks for sticking with me to the end of this week. I hope you enjoy hearing about both clustering algorithms and anomaly detection algorithms. And that you also enjoy playing with these ideas in the practice labs. Next week, we'll go on to talk about recommender systems. When you go to a website and recommends products, or movies, or other things to you. How does that algorithm actually work? This is one of the most commercially important algorithms in machine learning that gets talked about surprisingly little but next week we'll take a look at how these algorithms work so that you understand the next time you go to the website and then recommend something to you.

Maybe how that came about. As was you'll be able to build other algorithms like that for yourself as well. So have fun with the labs and they look forward to seeing you next week.

LECTURE 14

Predicting Movie Ratings

Welcome to this second to last week of the machine learning specialization. I'm really happy that together, almost all the way to the finish line. What we'll do this week is discuss recommended systems. This is one of the topics that has received quite a bit of attention in academia. But the commercial impact and the actual number of practical use cases of recommended systems seems to me to be even vastly greater than the amount of attention it has received in academia. Every time you go to an online shopping website like Amazon or a movie streaming sites like Netflix or go to one of the apps or sites that do food delivery. Many of these sites will recommend things to you that they think you may want to buy or movies they think you may want to watch or restaurants that they think you may want to try out.

And for many companies, a large fraction of sales is driven by their recommended systems. So today for many companies, the economics or the value driven by recommended systems is very large and so what we're doing this week is take a look at how they work. So with that let's dive in and take a look at what is a recommended system. I'm going to use as a running example, the application of predicting movie ratings. So say you run a large movie streaming website and your users have rated movies using one to five stars. And so in a typical recommended system you have a set of users, here we have four users Alice, Bob Carol and Dave. Which have numbered users 1,2,3,4.

As well as a set of movies Love at last, Romance forever, Cute puppies of love and then Nonstop car chases and Sword versus karate. And what the users have done is rated these movies one to five stars. Or in fact to make some of these examples a little bit easier. I'm not going to let them rate the movies from zero to five stars. So say Alice has rated Love and last five stars, Romance forever five stars. Maybe she has not yet watched cute puppies of love so you don't have a rating for that. And I'm going to denote that via a question mark and she thinks nonstop car chases and sword versus karate deserve zero stars bob. Race at five stars has not watched that, so you don't have a rating race at four stars, 0,0.

Carol on the other hand, thinks that deserve zero stars has not watched that zero stars and she loves nonstop car chases and swords versus karate and Dave rates the movies as follows. In the typical recommended system, you have some number of users as well as some number of items. In this case the items are movies that you want to recommend to the users. And even though I'm using movies in this example, the same logic or the same thing works for recommending anything from products or websites to my self, to restaurants, to even which media articles, the social media articles to show, to the user that may be more interesting for them. The notation I'm going to use is I'm going to use nu to denote the number of users.

So in this example nu is equal to four because you have four users and nm to denote the number of movies or really the number of items. So in this example nm is equal to five because we have five movies. I'm going to set r(i,j)=1, if user j has rated movie i.

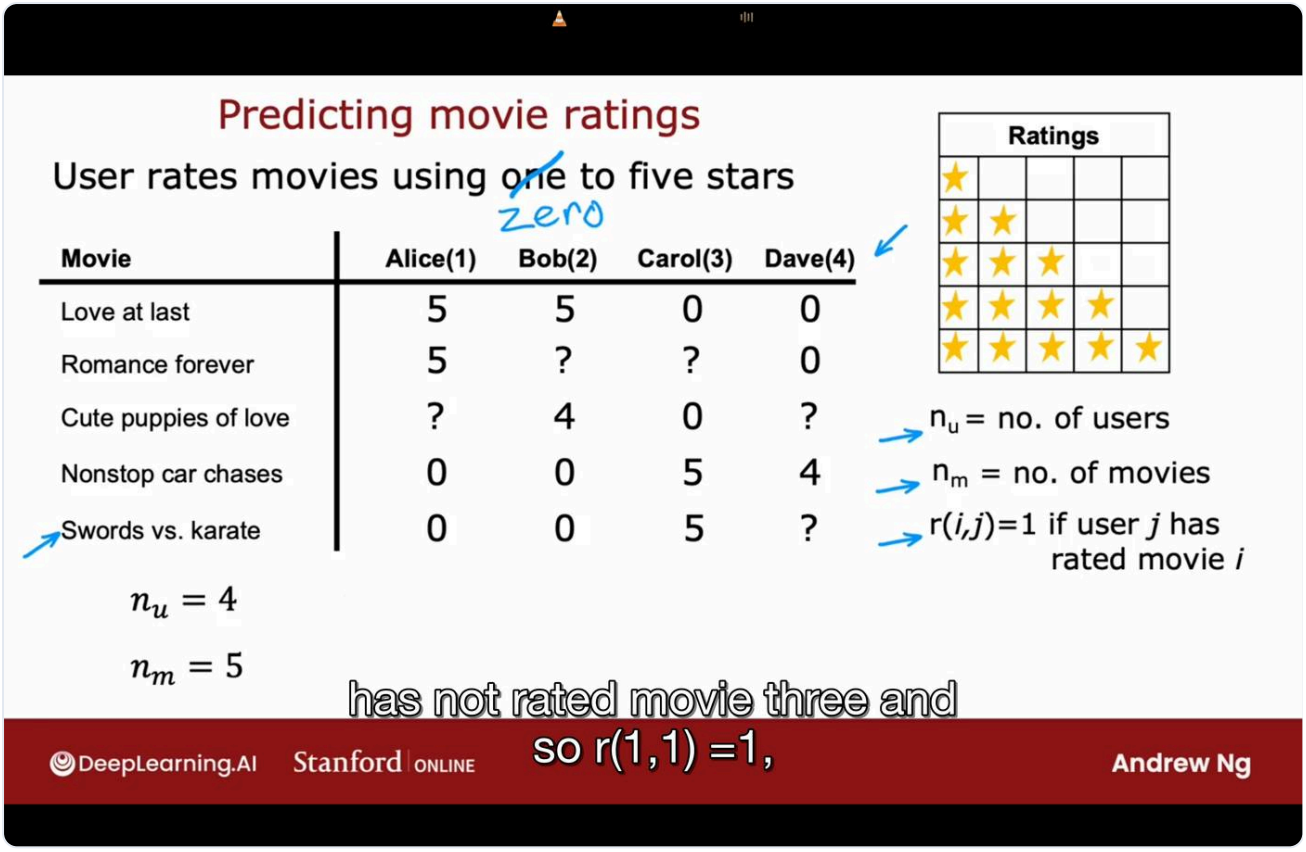


FIGURE 14.1 | User Rates Movies Using Ore to Five Stars

So for example, use a one Dallas Alice has rated movie one but has not rated movie three and so r(1,1) =1, because she has rated movie one, but r(3,1)=0 because she has not rated movie number three. Then finally I'm going to use y(i,j).

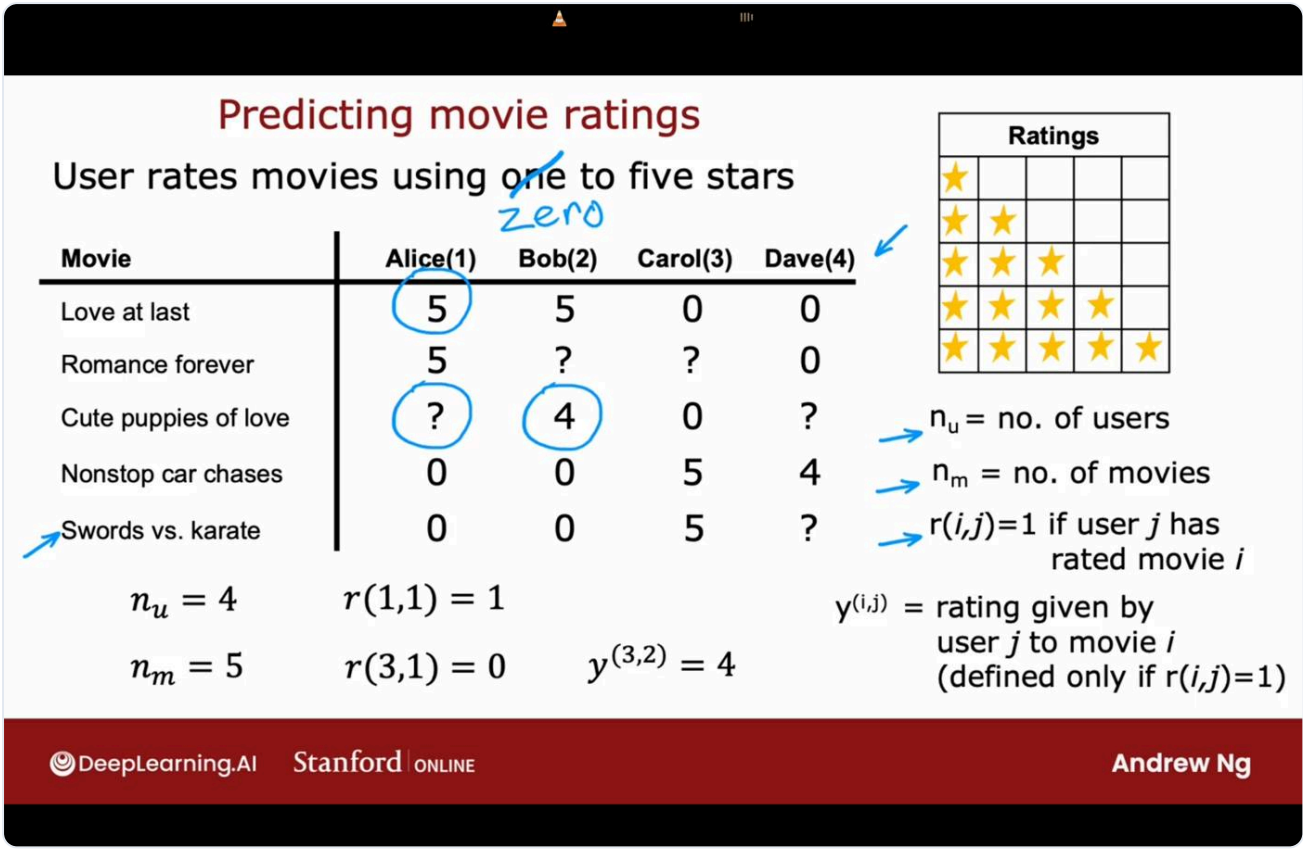


FIGURE 14.2 | Predicting Movie Ratings

J to denote the rating given by user j to movie i. So for example, this rating here would be that movie three was rated by user 2 to be equal to four. Notice that not every user rates every movie and it's important for the system to know which users have rated which movies. That's why we're going to define r(i,j)=1 if user j has rated movie i and will be equal to zero if user j has not rated movie i. So with this framework for recommended systems one possible way to approach the problem is to look at the movies that users have not rated. And to try to predict how users would rate those movies because then we can try to recommend to users things that they are more likely to rate as five stars.

And in the next video we'll start to develop an algorithm for doing exactly that. But making one very special assumption. Which is we're going to assume temporarily that we have access to features or extra information about the movies such as which movies are romance movies, which movies are action movies. And using that will start to develop an algorithm. But later this week will actually come back and ask what if we don't have these features, how can you still get the algorithm to work then? But let's go on to the next video to start building up this algorithm.

LECTURE 15

Cost Function

So let's take a look at how we can develop a recommender system if we had features of each item, or features of each movie. So here's the same data set that we had previously with the four users having rated some but not all of the five movies. What if we additionally have features of the movies? So here I've added two features X1 and X2, that tell us how much each of these is a romance movie, and how much each of these is an action movie.

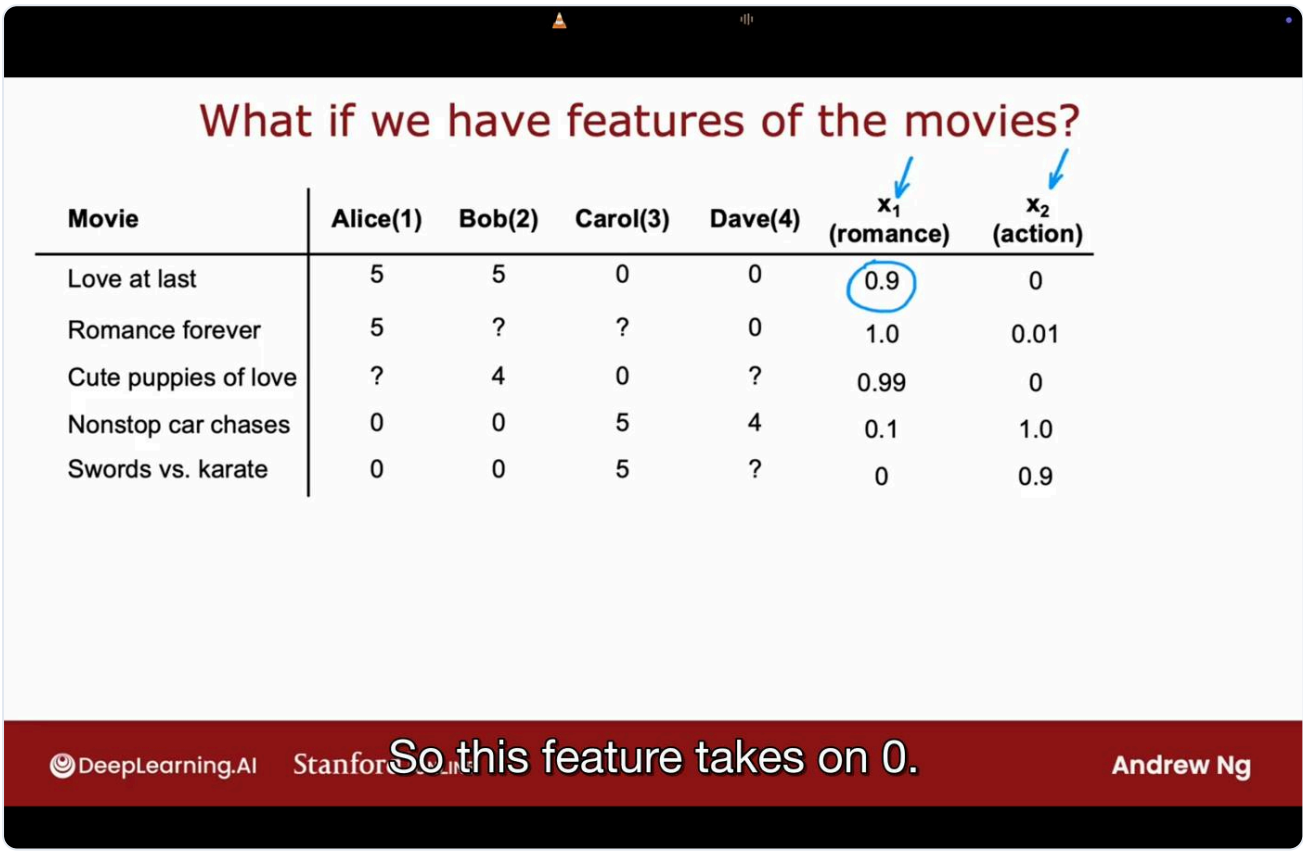


FIGURE 15.1 | What If We Have Features of the Movies?

So for example Love at Last is a very romantic movie, so this feature takes on 0.9, but it's not at all an action movie. So this feature takes on 0. But it turns out Nonstop Car chases has just a little bit of romance in it. So it's 0.1, but it has a ton of action. So that feature takes on the value of 1.0. So you recall that I had used the notation nu to denote the number of users, which is 4 and m to denote the number of movies which is 5. I'm going to also introduce n to denote the number of features we have here. And so n=2, because we have two features X1 and X2 for each movie. With these features we have for example that the features for movie one, that is the movie Love at Last, would be 0.90.

And the features for the third movie Cute Puppies of Love would be 0.99 and 0. And let's start by taking a look at how we might make predictions for Alice's movie ratings. So for user one, that is Alice, let's say we predict the rating for movie i as w.X(i)+b. So this is just a lot like linear regression. For example if we end up choosing the parameter w(1)=[5,0] and say b(1)=0, then the predic-

tion for movie three where the features are 0.99 and 0, which is just copied from here, first feature 0.99, second feature 0. Our prediction would be w.X(3)+b=0.99 times 5 plus 0 times zero, which turns out to be equal to 4.95.

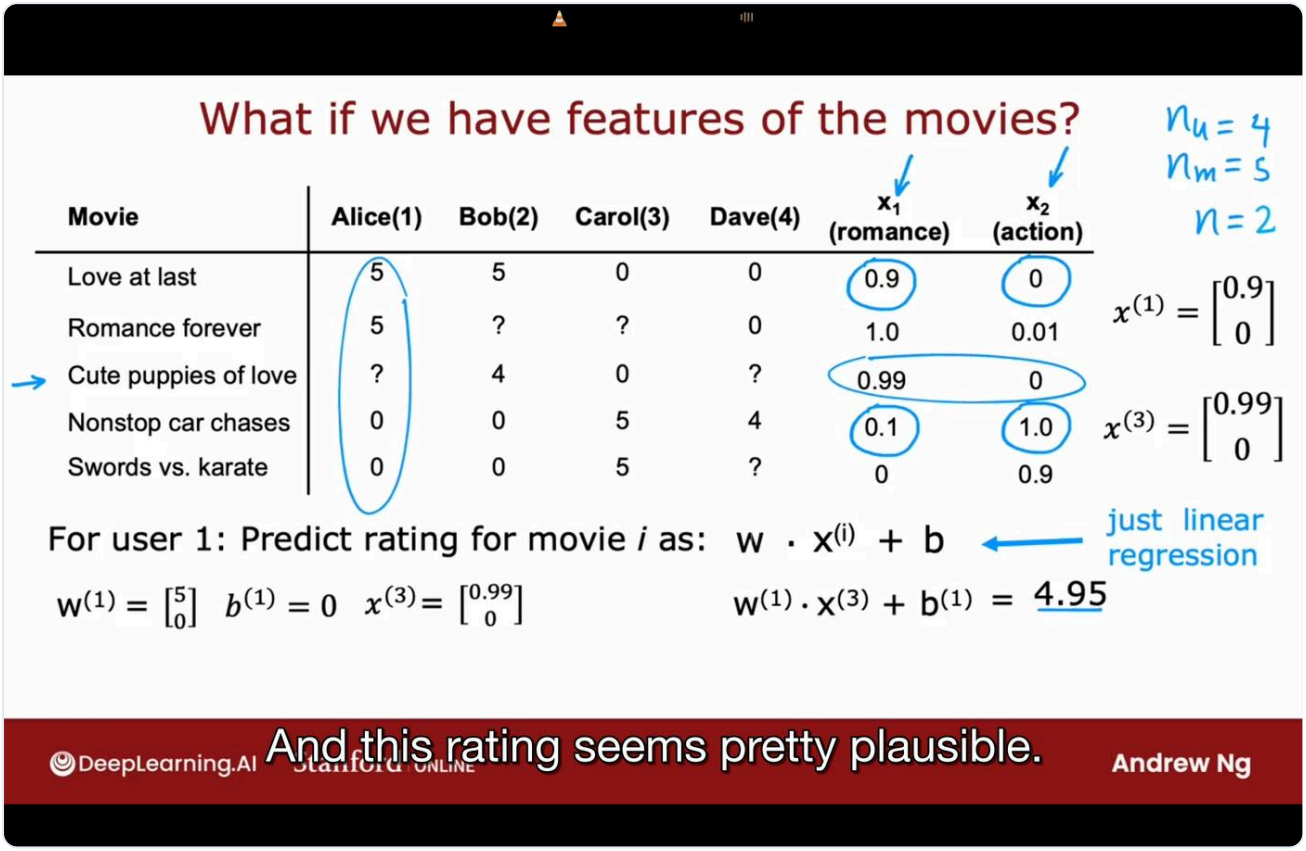


FIGURE 15.2

And this rating seems pretty plausible. It looks like Alice has given high ratings to Love at Last and Romance Forever, to two highly romantic movies, but given low ratings to the action movies, Nonstop Car Chases and Swords vs Karate. So if we look at Cute Puppies of Love, well predicting that she might rate that 4.95 seems quite plausible. And so these parameters w and b for Alice seems like a reasonable model for predicting her movie ratings. Just add a little the notation because we have not just one user but multiple users, or really nu equals 4 users. I'm going to add a superscript 1 here to denote that this is the parameter w(1) for user 1 and add a superscript 1 there as well.

And similarly here and here as well, so that we would actually have different parameters for each of the 4 users on data set.

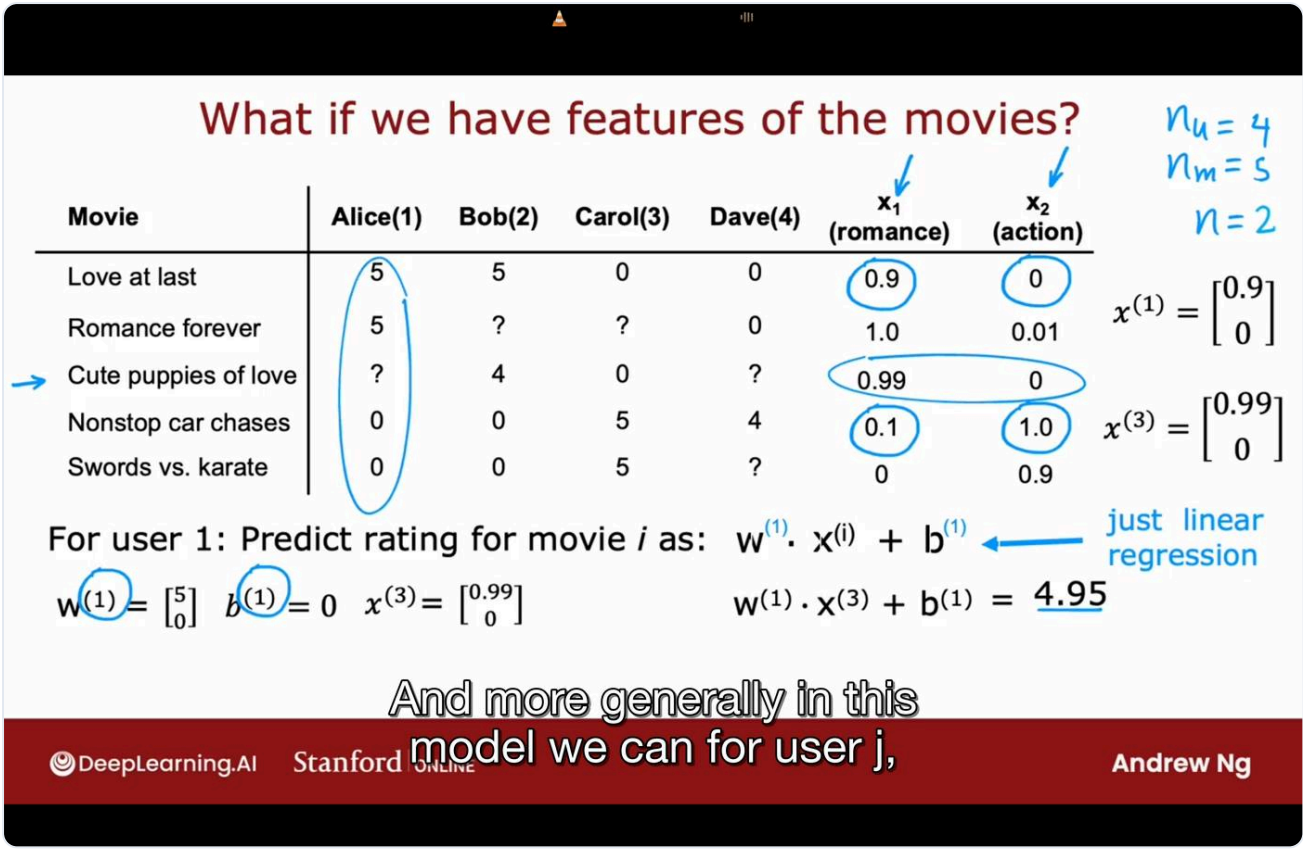


FIGURE 15.3

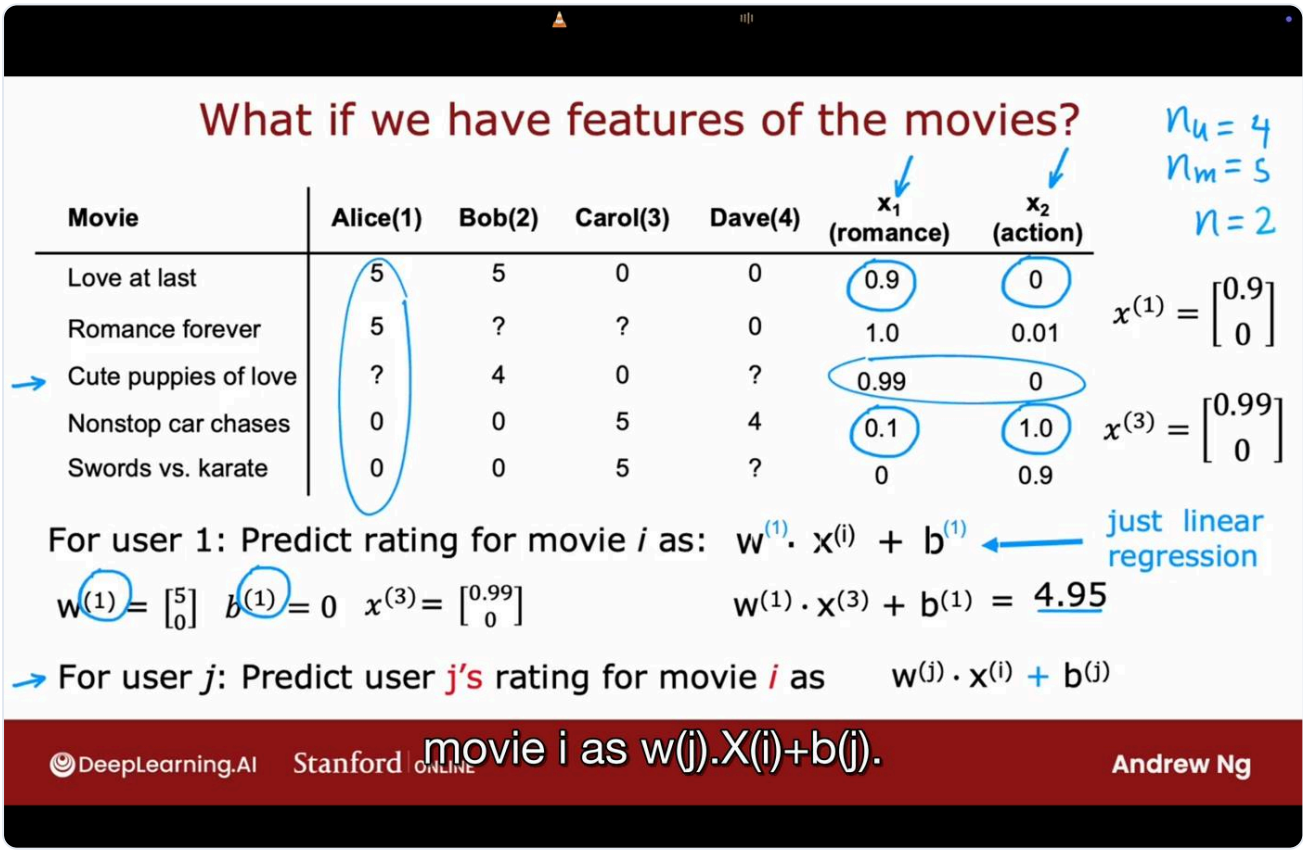


FIGURE 15.4

And more generally in this model we can for user j, not just user 1 now, we can predict user j's rating for movie i as w(j).X(i)+b(j). So here the parameters w(j) and b(j) are the parameters used to predict user j's rating for movie i which is a function of X(i), which is the features of movie i. And this is a lot like linear regression, except that we're fitting a different linear regression model for each of the 4 users in the dataset. So let's take a look at how we can formulate the cost function for this algorithm. As a reminder, our notation is that r(i.,j)=1 if user j has rated movie i or 0 otherwise. And y(i,j)=rating given by user j on movie i.

And on the previous side we defined w(j), b(j) as the parameters for user j. And X(i) as the feature vector for movie i. So the model we have is for user j and movie i predict the rating to be w(j).X(i)+b(j). I'm going to introduce just one new piece of notation, which is I'm going to use m(j) to denote the number of movies rated by user j. So if the user has rated 4 movies, then m(j) would be equal to 4. And if the user has rated 3 movies then m(j) would be equal to 3. So what we'd like to do is to learn the parameters w(j) and b(j), given the data that we have. That is given the ratings a user has given of a set of movies. So the algorithm we're going to use is very similar to linear regression.

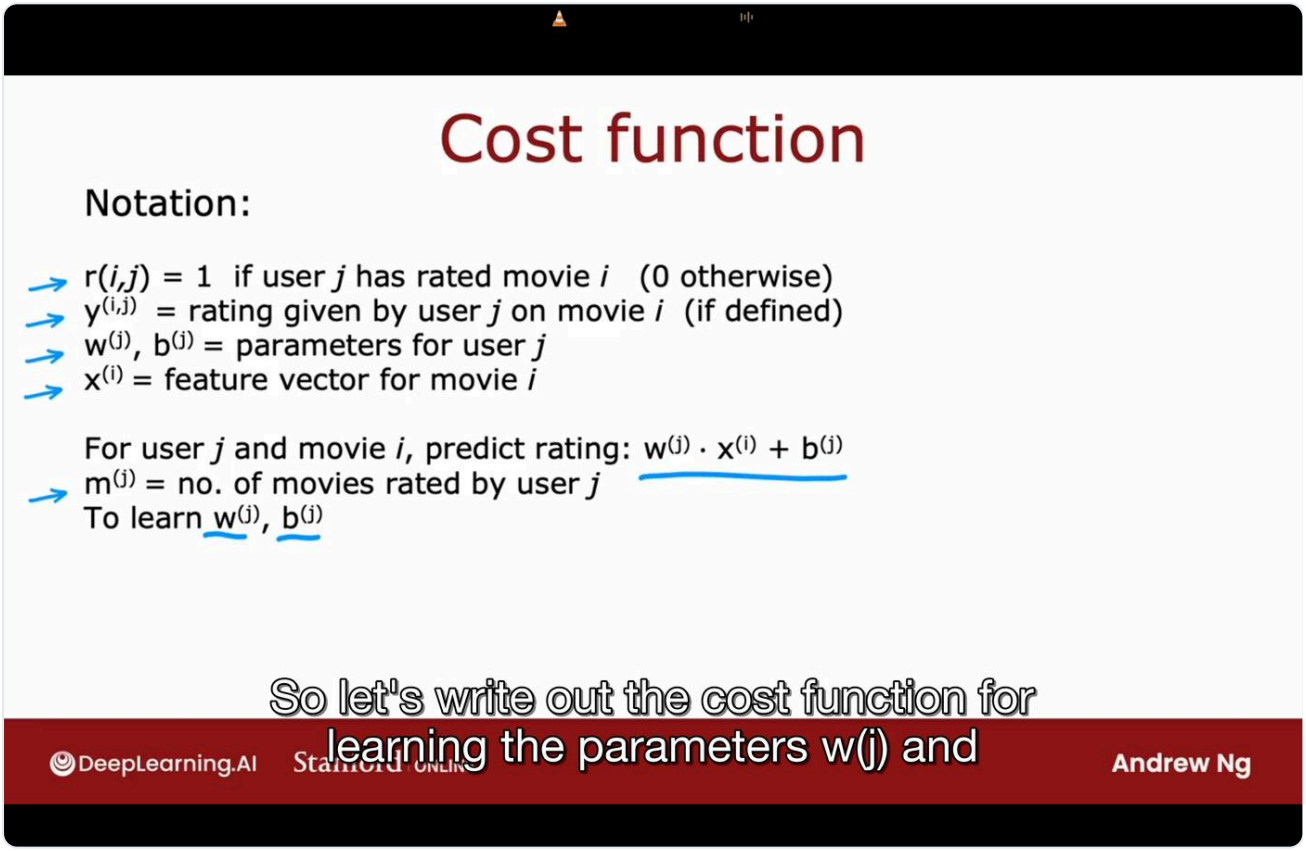


FIGURE 15.5

So let's write out the cost function for learning the parameters w(j) and b(j) for a given user j. And let's just focus on one user on user j for now. I'm going to use the mean squared error criteria.

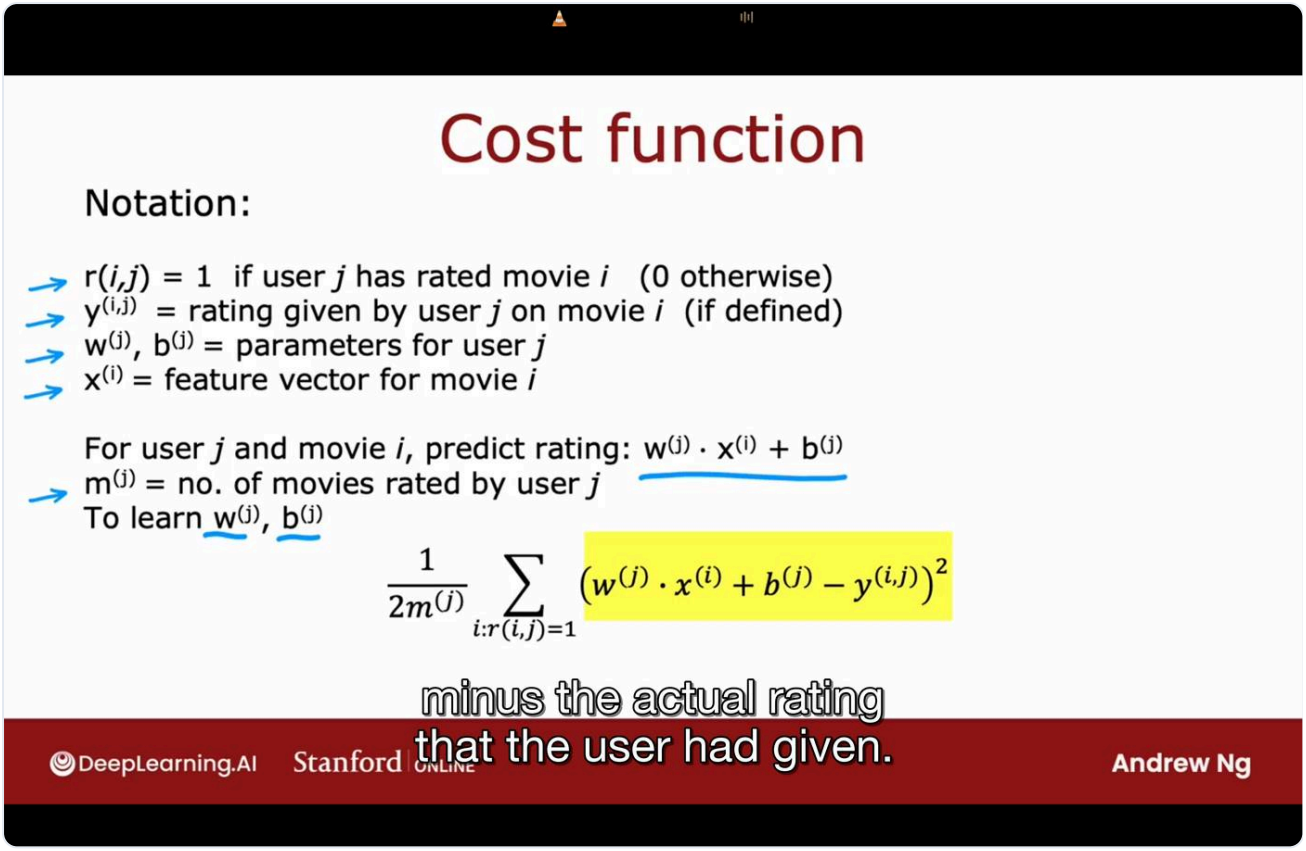


FIGURE 15.6 | Cost Function

So the cost will be the prediction, which is w(j).X(i)+b(j) minus the actual rating that the user had given. So minus y(i,j) squared. And we're trying to choose parameters w and b to minimize the squared error between the predicted rating and the actual rating that was observed.

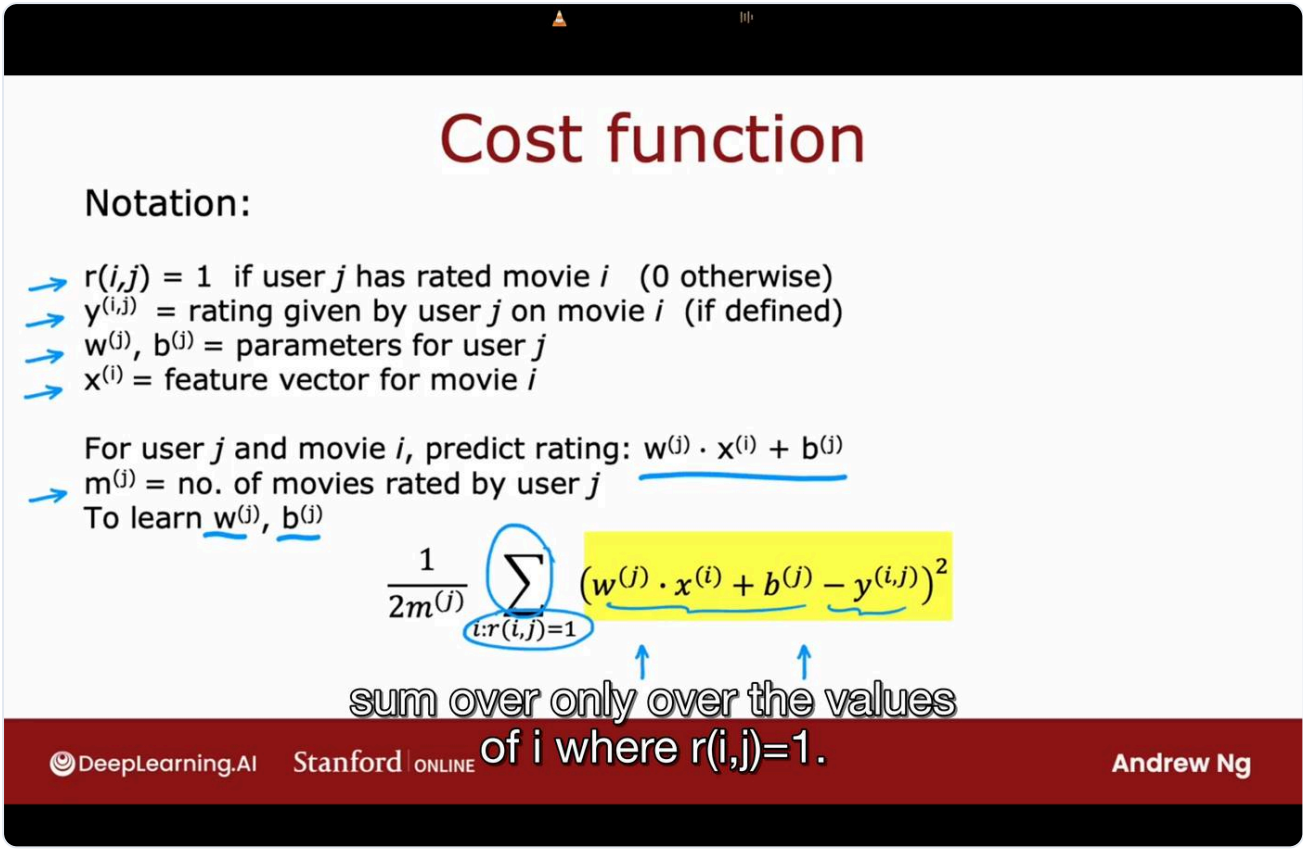


FIGURE 15.7 | Cost Function

But the user hasn't rated all the movies, so if we're going to sum over this, we're going to sum over only over the values of i where r(i,j)=1. So we're going to sum only over the movies i that user j has actually rated. So that's what this denotes, sum of all values of i where r(i,j)=1. Meaning that user j has rated that movie i. And then finally we can take the usual normalization 1 over m(j). And this is very much like the cost function we have for linear regression with m or really m(j) training examples. Where you're summing over the m(j) movies for which you have a rating taking a squared error and the normalizing by this 1 over 2m(j).

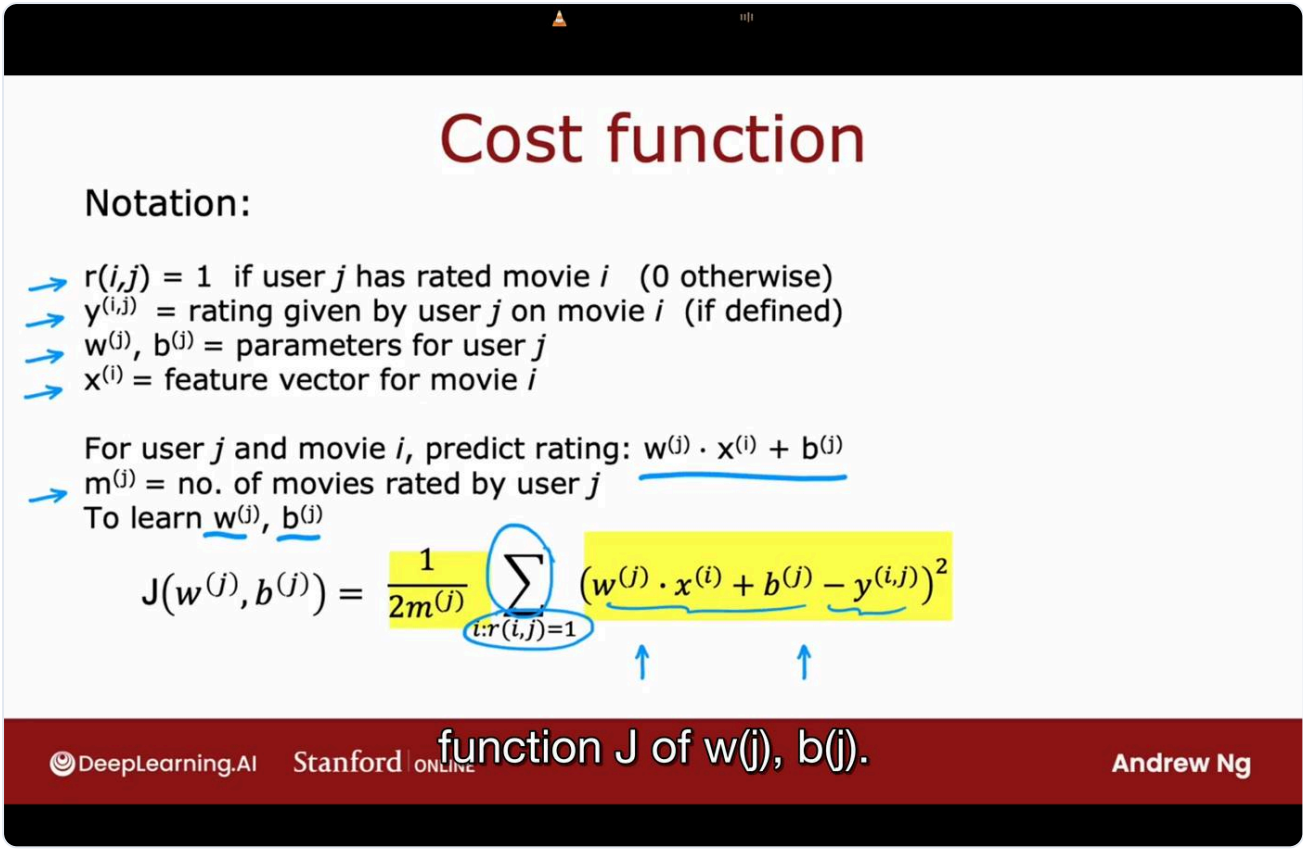


FIGURE 15.8 | Cost Function

And this is going to be a cost function J of w(j), b(j).

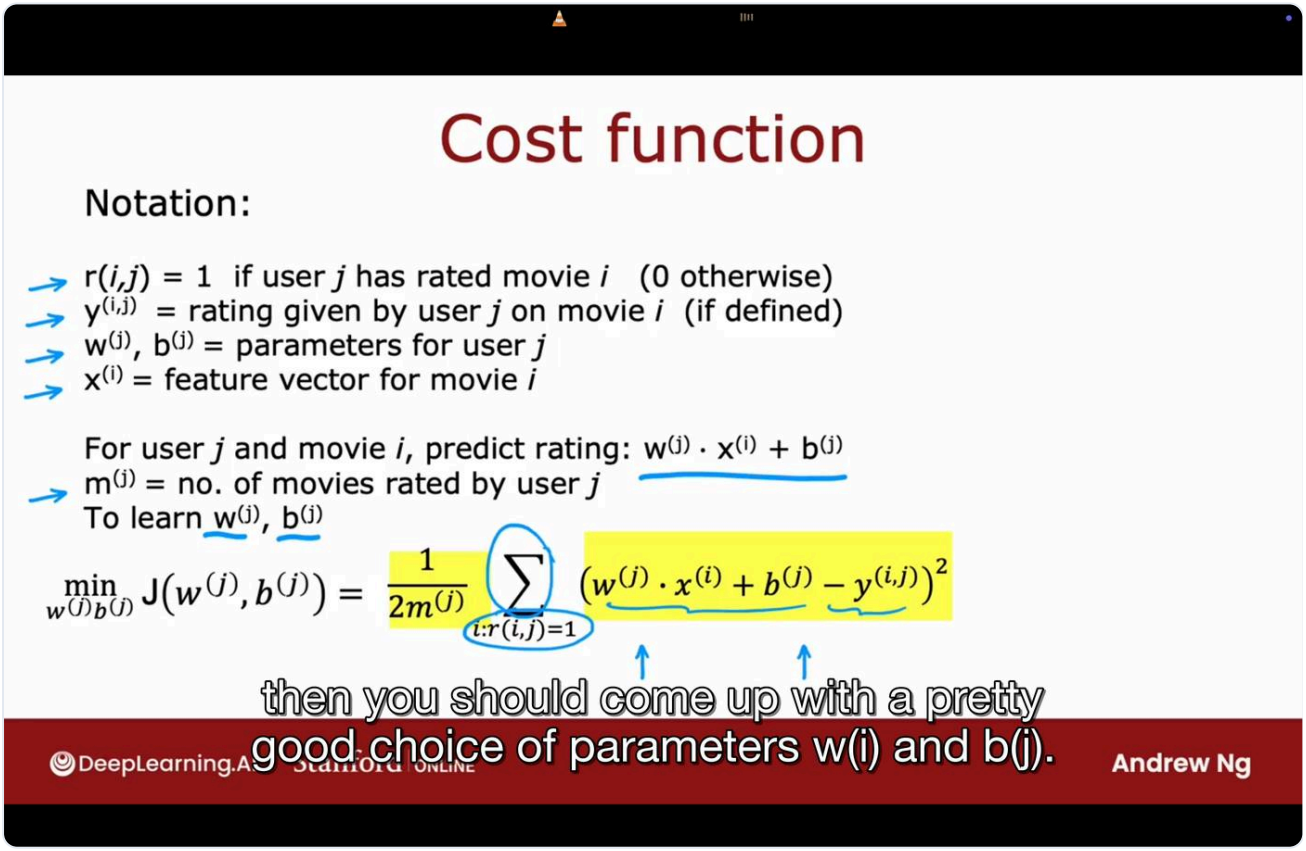


FIGURE 15.9 | Cost Function

And if we minimize this as a function of w(j) and b(j), then you should come up with a pretty good choice of parameters w(i) and b(j). For making predictions for user j's ratings. Let me have just one more term to this cost function, which is the regularization term to prevent overfitting.

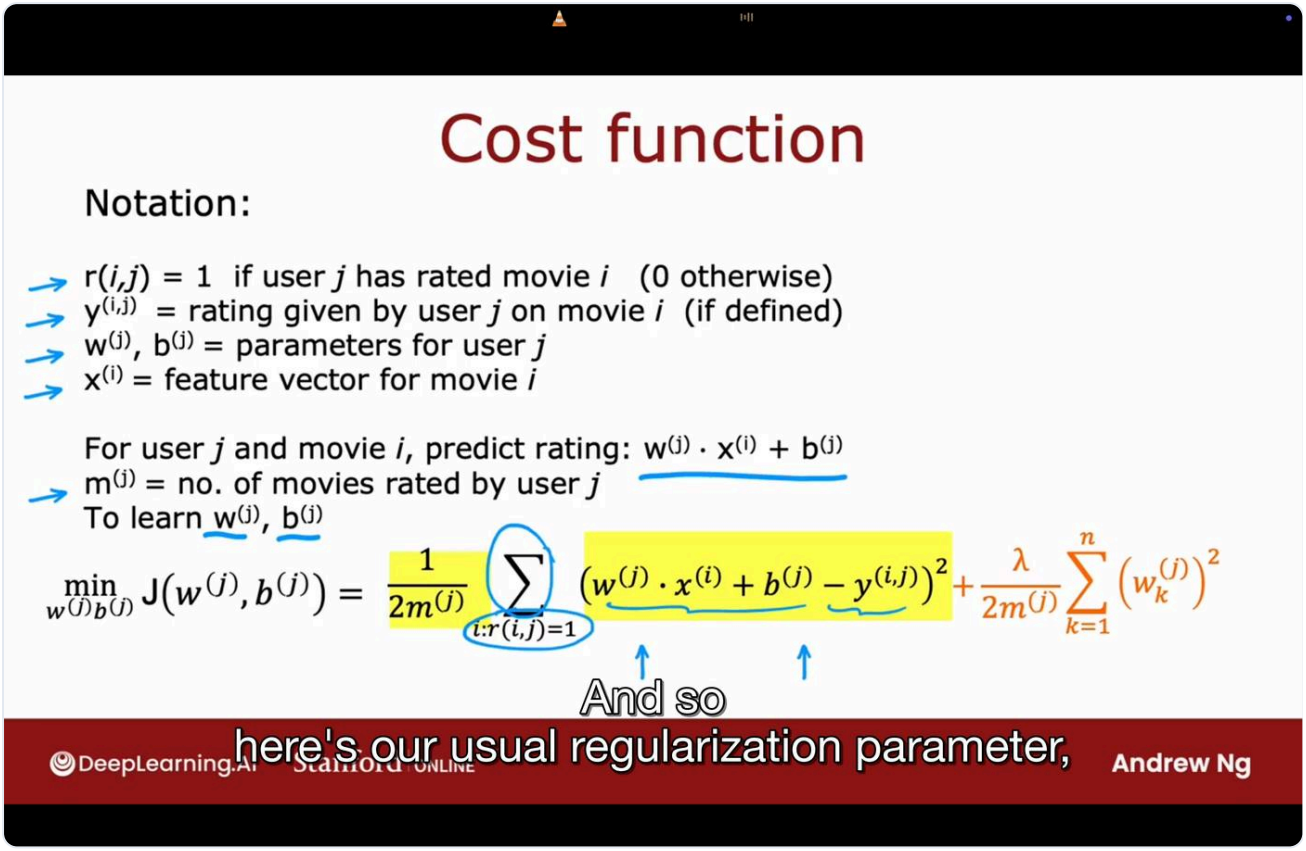


FIGURE 15.10 | Cost Function

And so here's our usual regularization parameter, lambda divided by 2m(j) and then times as sum of the squared values of the parameters w. And so n is a number of numbers in X(i) and that's the same as a number of numbers in w(j). If you were to minimize this cost function J as a function of w and b, you should get a pretty good set of parameters for predicting user j's ratings for other movies. Now, before moving on, it turns out that for recommender systems it would be convenient to actually eliminate this division by m(j) term, m(j) is just a constant in this expression. And so, even if you take it out, you should end up with the same value of w and b.

Now let me take this cost function down here to the bottom and copy it to the next slide. So we have that to learn the parameters w(j), b(j) for user j. We would minimize this cost function as a function of w(j) and b(j). But instead of focusing on a single user, let's look at how we learn the parameters for all of the users.

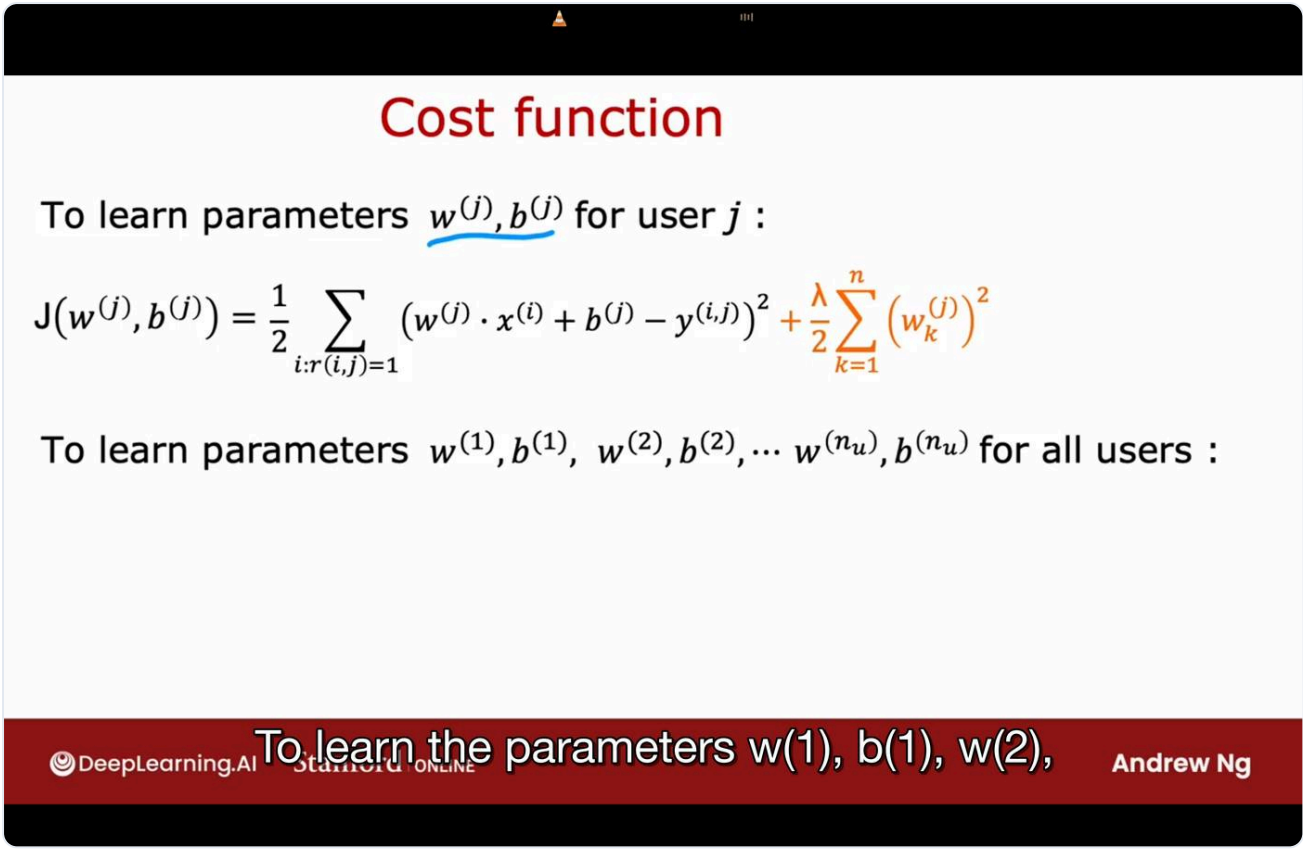


FIGURE 15.11 | Cost Function

To learn the parameters w(1), b(1), w(2), b(2),...,w(nu), b(nu), we would take this cost function on top and sum it over all the nu users.

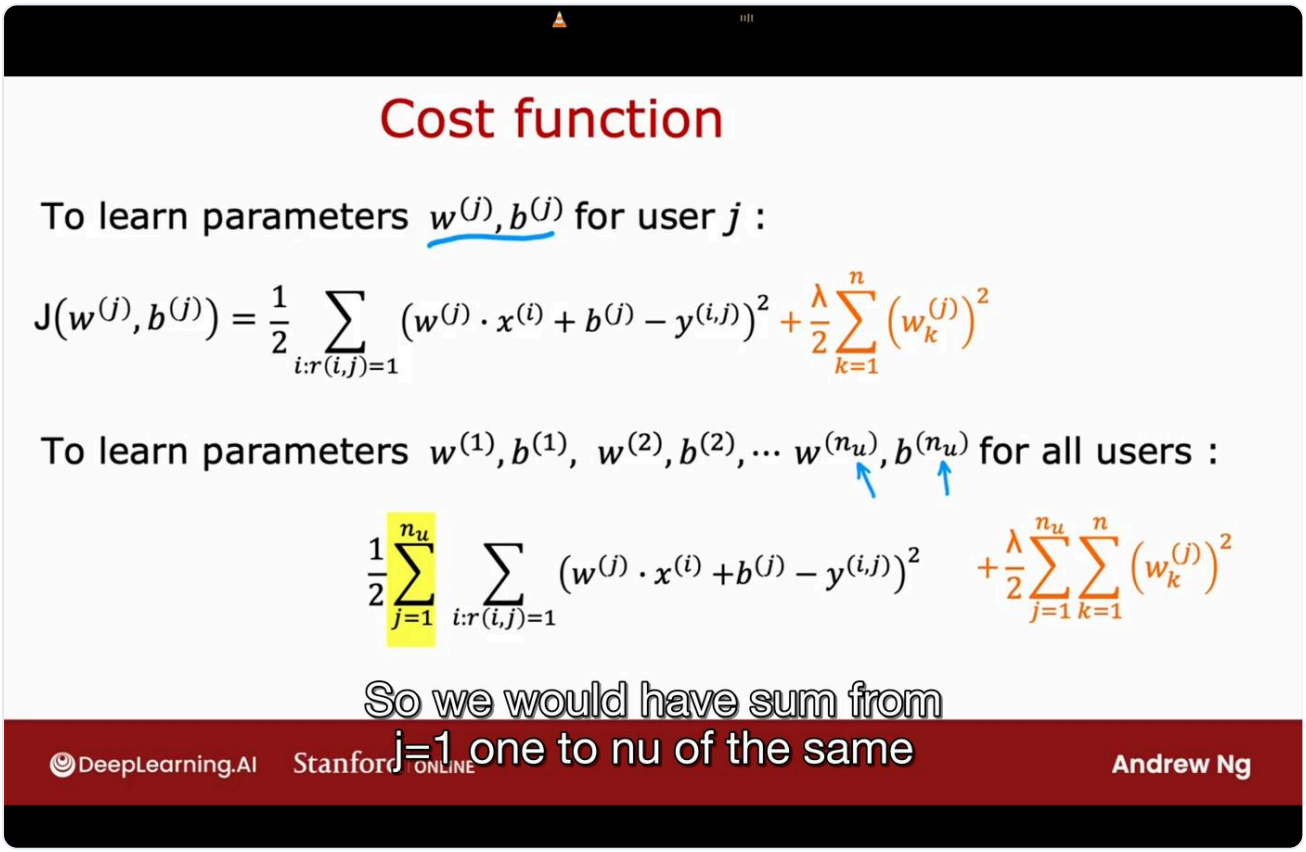


FIGURE 15.12 | Cost Function

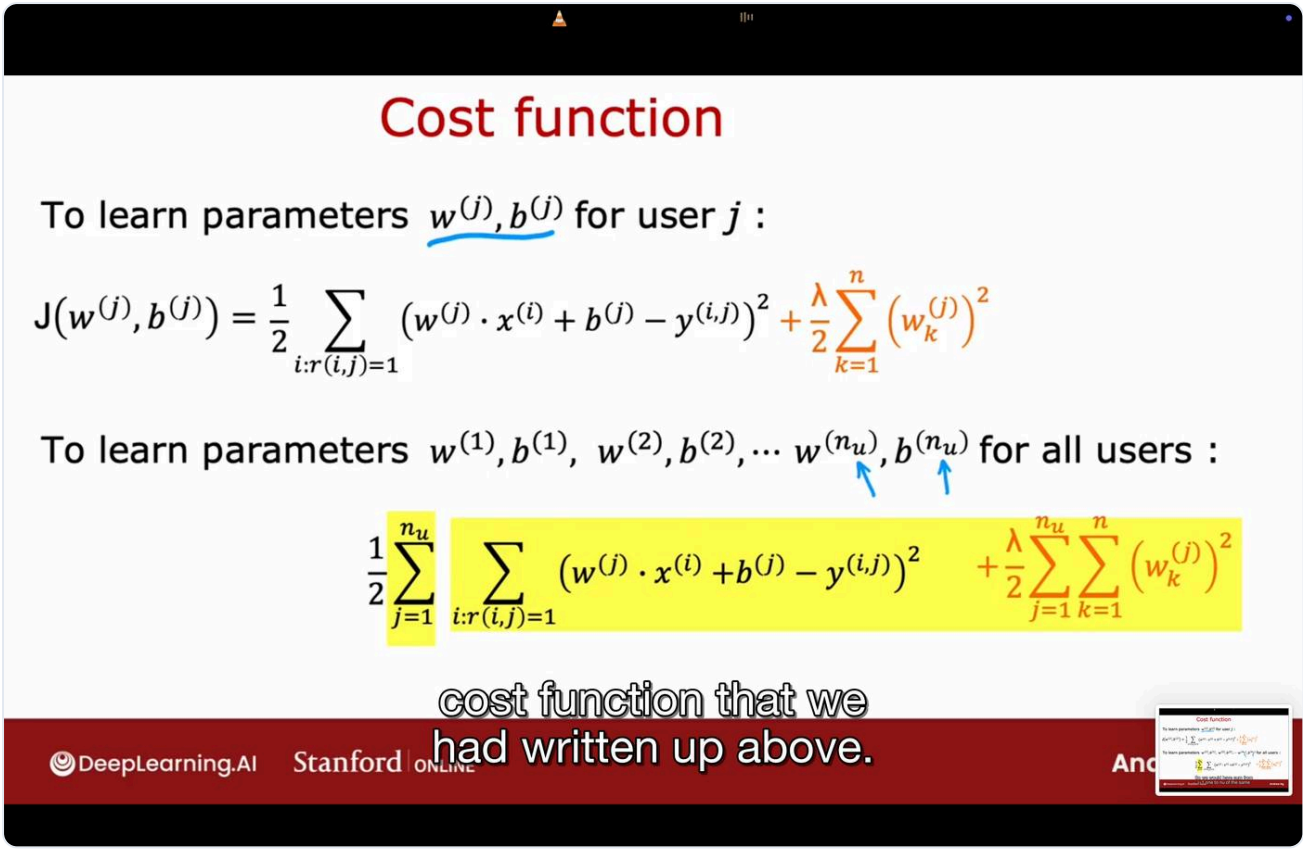


FIGURE 15.13 | Cost Function

So we would have sum from j=1 one to nu of the same cost function that we had written up above.

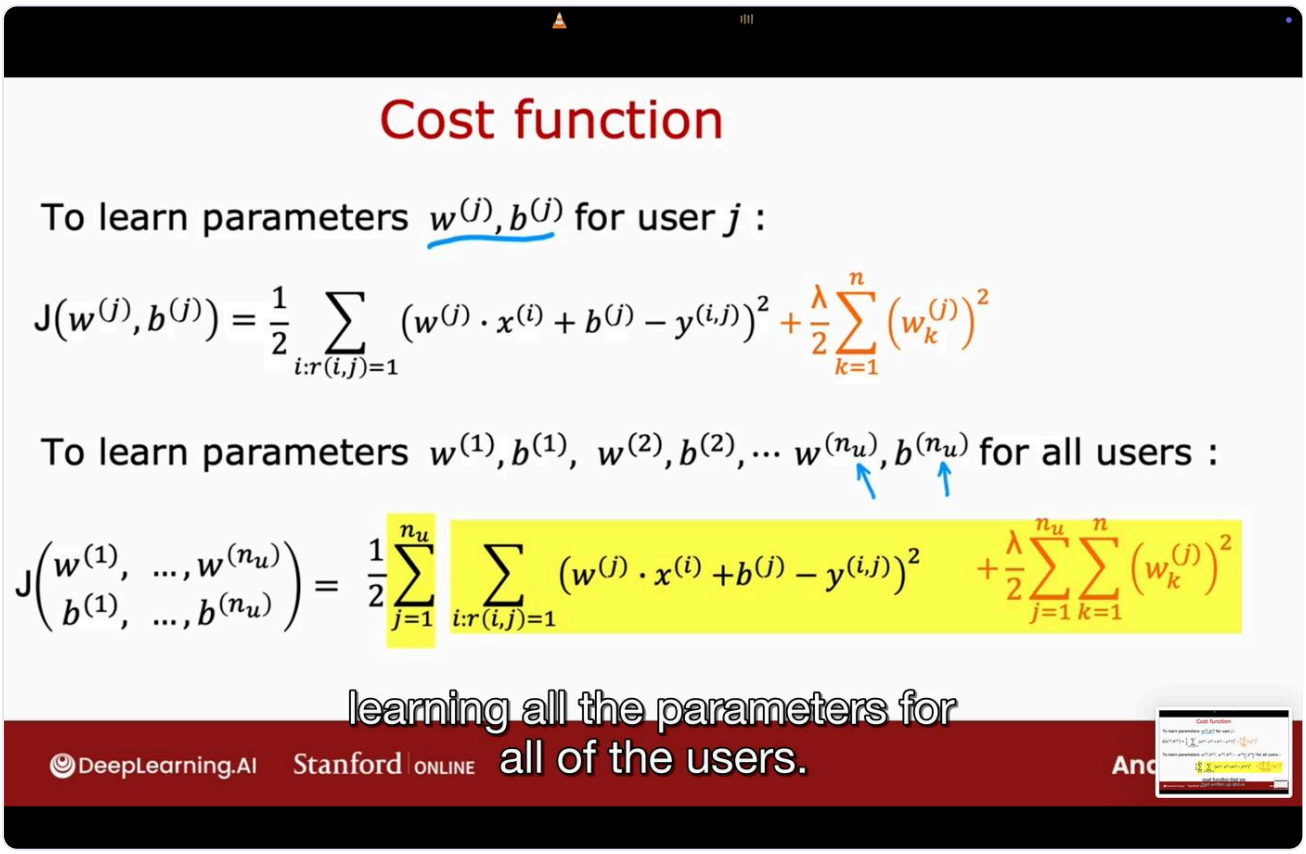


FIGURE 15.14 | Cost Function

And this becomes the cost for learning all the parameters for all of the users. And if we use gradient descent or any other optimization algorithm to minimize this as a function of w(1), b(1) all the way through w(nu), b(nu), then you have a pretty good set of parameters for predicting movie ratings for all the users. And you may notice that this algorithm is a lot like linear regression, where that plays a role similar to the output f(x) of linear regression.

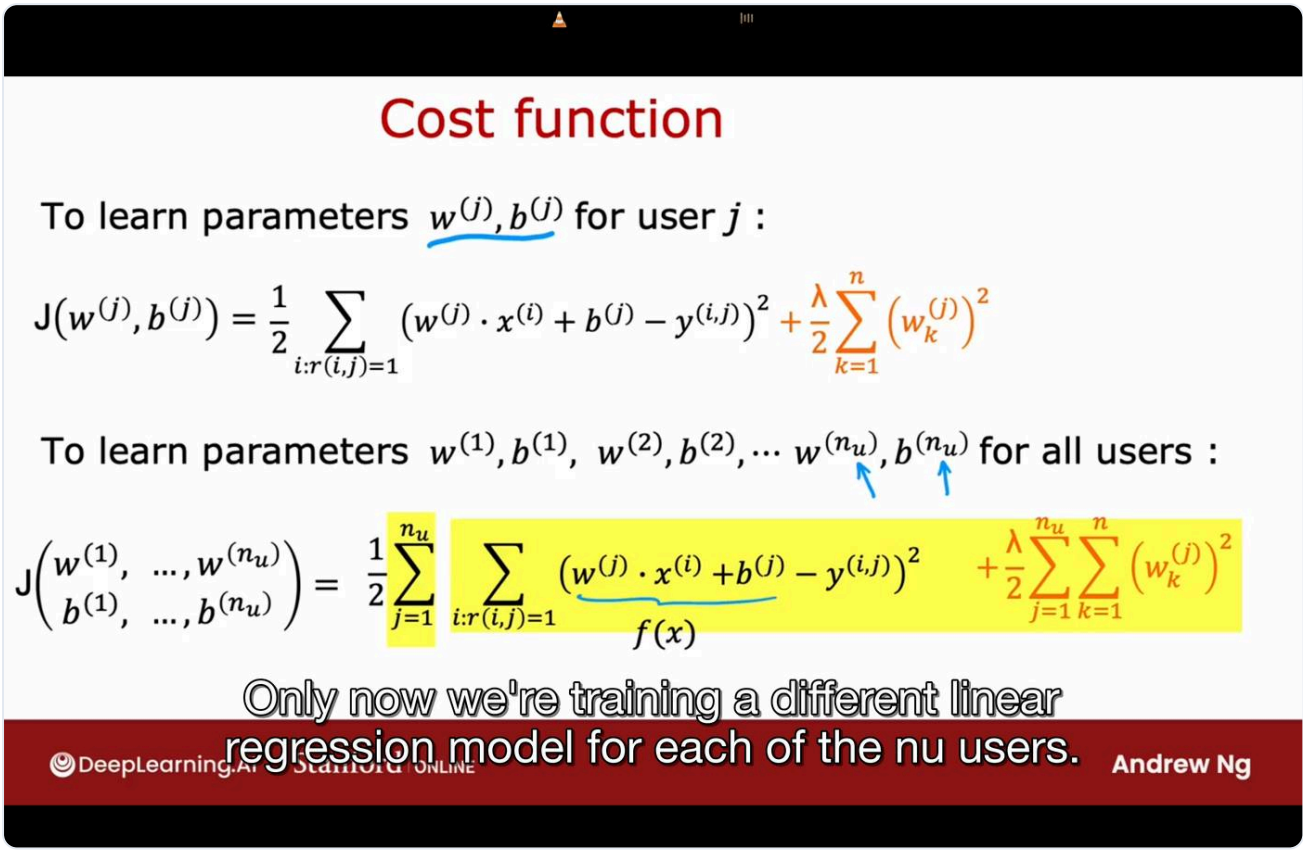


FIGURE 15.15 | Cost Function

Only now we're training a different linear regression model for each of the nu users. So that's how you can learn parameters and predict movie ratings, if you had access to these features X1 and X2. That tell you how much is each of the movies, a romance movie, and how much is each of the movies an action movie? But where do these features come from? And what if you don't have access to such features that give you enough detail about the movies with which to make these predictions? In the next video, we'll look at the modification of this algorithm. They'll let you make predictions that you make recommendations. Even if you don't have, in advance, features that describe the items of the movies in sufficient detail to run the algorithm that we just saw.

Let's go on and take a look at that in the next video

LECTURE 16

Collaborative Filtering

In the last video, you saw how if you have features for each movie, such as features x_1 and x_2 that tell you how much is this a romance movie and how much is this an action movie. Then you can use basically linear regression to learn to predict movie ratings. But what if you don't have those features, x_1 and x_2? Let's take a look at how you can learn or come up with those features x_1 and x_2 from the data. Here's the data that we had before. But what if instead of having these numbers for x_1 and x_2, we didn't know in advance what the values of the features x_1 and x_2 were? I'm going to replace them with question marks over here. Now, just for the purposes of illustration, let's say we had somehow already learned parameters for the four users.

Let's say that we learned parameters w^1 equals 5 and 0 and b^1 equals 0, for user one. W^2 is also 5, 0 b^2, 0. W^3 is 0, 5 b^3 is 0, and for user four W^4 is also 0, 5 and b^4 0, 0. We'll worry later about how we might have come up with these parameters, w and b. But let's say we have them already. As a reminder, to predict user j's rating on movie i, we're going to use w^j dot product, the features of x_i plus b^j. To simplify this example, all the values of b are actually equal to 0. Just to reduce a little bit of writing, I'm going to ignore b for the rest of this example. Let's take a look at how we can try to guess what might be reasonable features for movie one.

If these are the parameters you have on the left, then given that Alice rated movie one, 5, we should have that w^1.x^1 should be about equal to 5 and w^2.x^2 should also be about equal to 5 because Bob rated it 5. W^3.x^1 should be close to 0 and w^4.x^1 should be close to 0 as well. The question is, given these values for w that we have up here, what choice for x_1 will cause these values to be right?

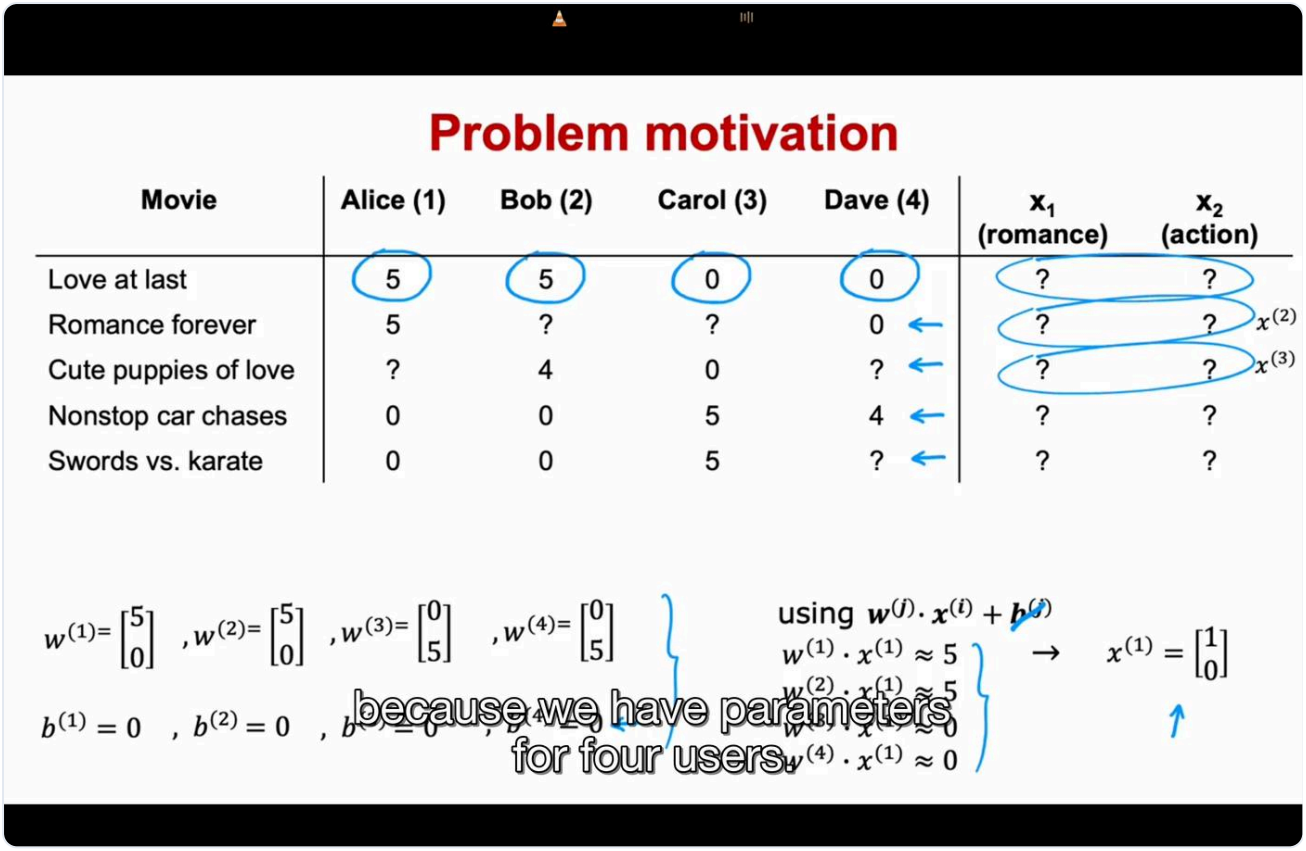


FIGURE 16.1

Well, one possible choice would be if the features for that first movie, were 1, 0 in which case, w^1.x^1 will be equal to 5, w^2.x^1 will be equal to 5 and similarly, w^3 or w^4 dot product with this feature vector x_1 would be equal to 0. What we have is that if you have the parameters for all four users here, and if you have four ratings in this example that you want to try to match, you can take a reasonable guess at what lists a feature vector x_1 for movie one that would make good predictions for these four ratings up on top. Similarly, if you have these parameter vectors, you can also try to come up with a feature vector x_2 for the second movie, feature vector x_3 for the third movie, and so on to try to make the algorithm's predictions on these additional movies close to what was actually the ratings given by the users.

Let's come up with a cost function for actually learning the values of x_1 and x_2. By the way, notice that this works only because we have parameters for four users. That's what allows us to try to guess appropriate features, x_1. This is why in a typical linear regression application if you had just a single user, you don't actually have enough information to figure out what would be the features, x_1 and x_2, which is why in the linear regression contexts that you saw in course 1, you can't come up with features x_1 and x_2 from scratch. But in collaborative filtering, is because you have ratings from multiple users of the same item with the same movie.

That's what makes it possible to try to guess what are possible values for these features. Given w^1, b^1, w^2, b^2, and so on through w^n_u and b^n_u, for the n subscript u users. If you want to learn the features x^i for a specific movie, i is a cost function we could use which is that. I'm going to want to minimize squared error as usual. If the predicted rating by user j on movie i is

given by this, let's take the squared difference from the actual movie rating y,i,j. As before, let's sum over all the users j. But this will be a sum over all values of j, where r, i, j is equal to I. I'll add a 1.5 there as usual. As I defined this as a cost function for x^i.

Then if we minimize this as a function of x^i you be choosing the features for movie i. So therefore all the users J that have rated movie i, we will try to minimize the squared difference between what your choice of features x^i results in terms of the predicted movie rating minus the actual movie rating that the user had given it.

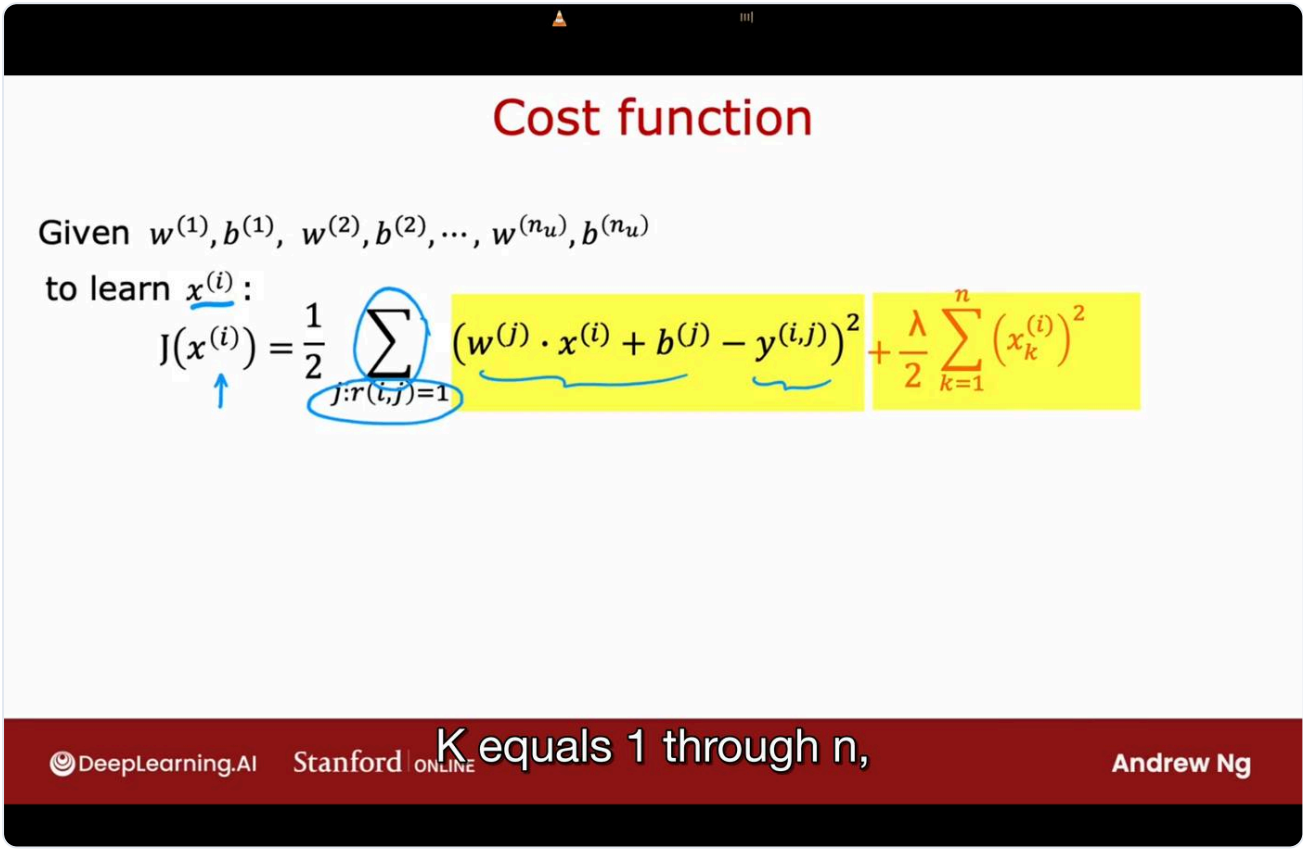


FIGURE 16.2 | Cost Function

Then finally, if we want to add a regularization term, we add the usual plus Lambda over 2, K equals 1 through n, where n as usual is the number of features of x^i squared. Lastly, to learn all the features x1 through x^n_m because we have n_m movies, we can take this cost function on top and sum it over all the movies. Sum from i equals 1 through the number of movies and then just take this term from above and this becomes a cost function for learning the features for all of the movies in the dataset.

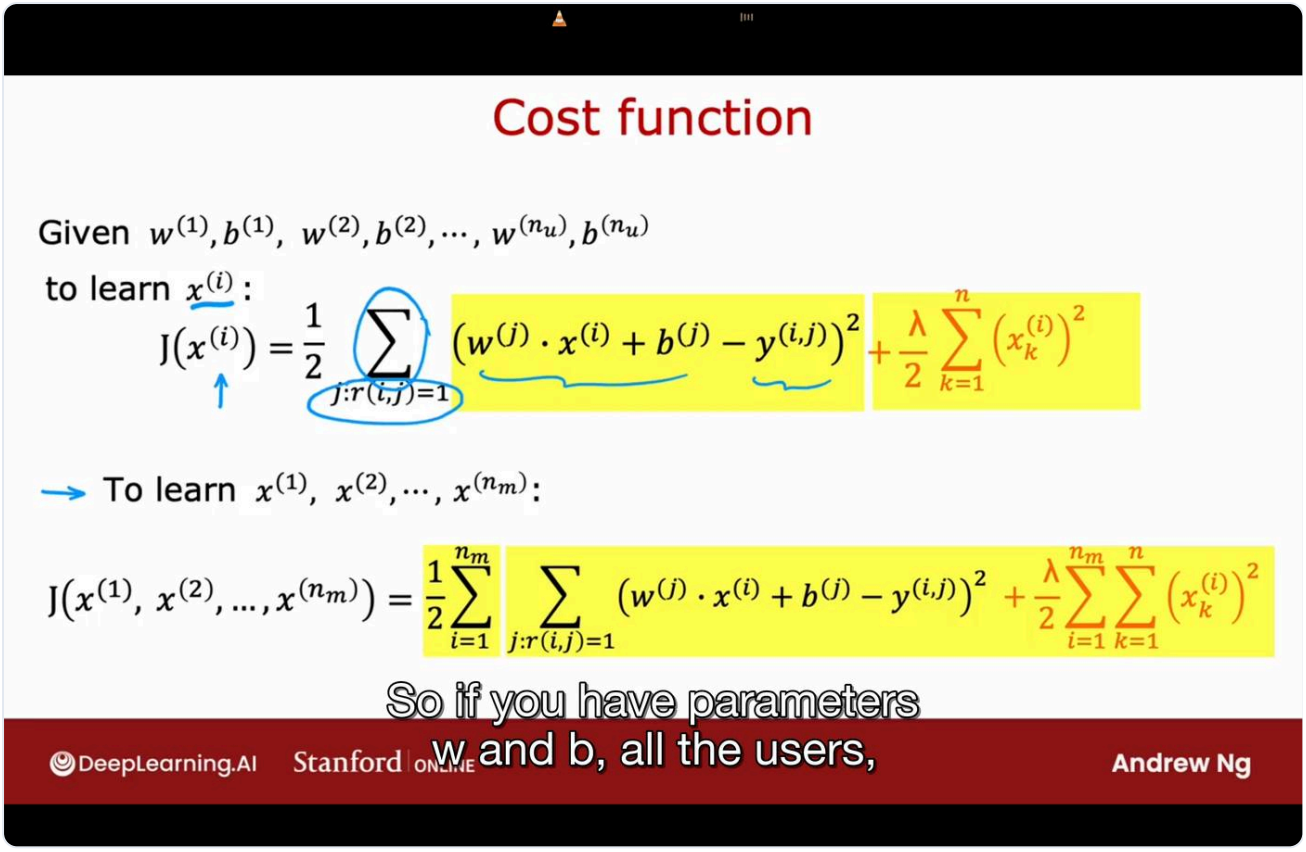


FIGURE 16.3 | Cost Function

So if you have parameters w and b, all the users, then minimizing this cost function as a function of x1 through x^n_m using gradient descent or some other algorithm, this will actually allow you to take a pretty good guess at learning good features for the movies. This is pretty remarkable for most machine learning applications the features had to be externally given but in this algorithm, we can actually learn the features for a given movie. But what we've done so far in this video, we assumed you had those parameters w and b for the different users. Where do you get those parameters from? Well, let's put together the algorithm from the last video for learning w and b and what we just talked about in this video for learning x and that will give us our collaborative filtering algorithm.

Here's the cost function for learning the features. This is what we had derived on the last slide. Now, it turns out that if we put these two together, this term here is exactly the same as this term here.

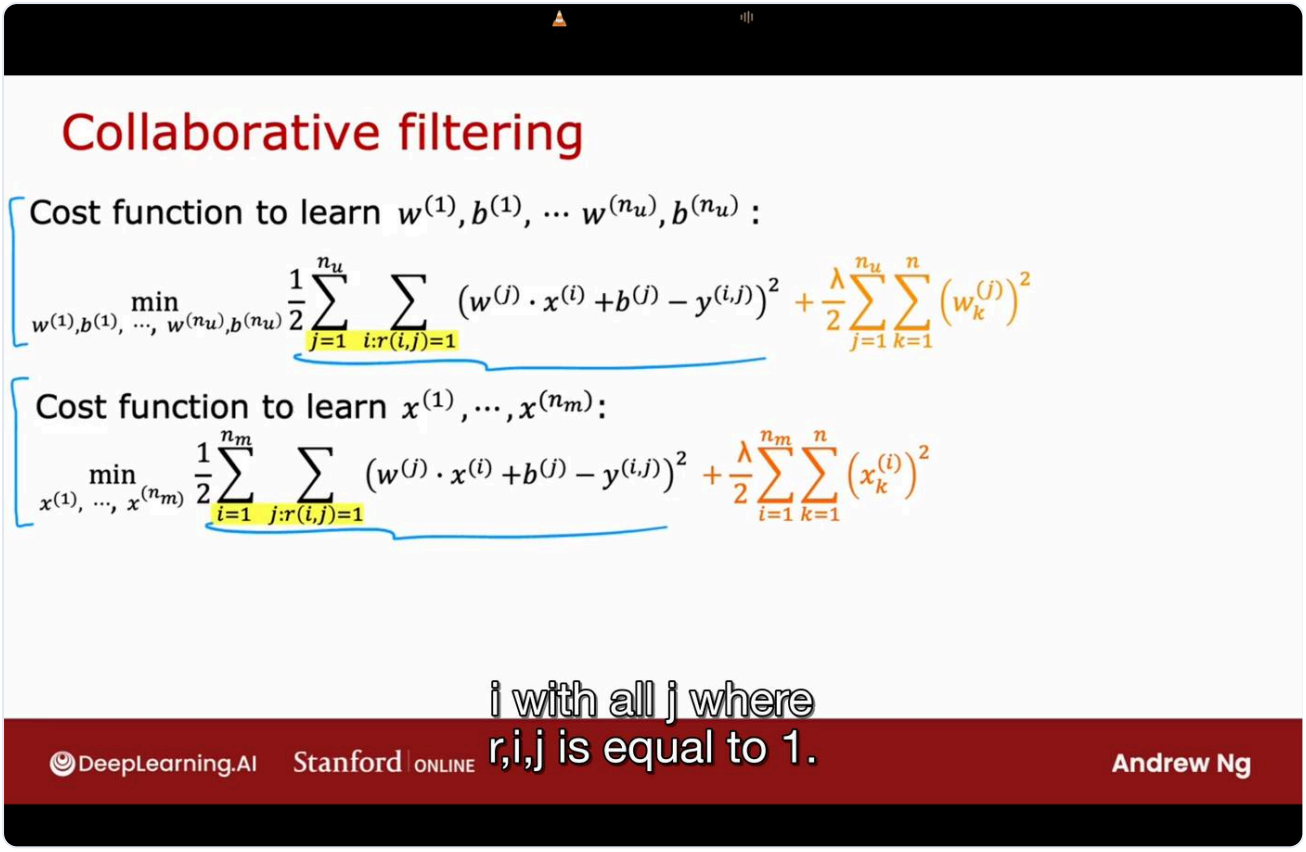


FIGURE 16.4 | Collaborative Filtering

Notice that sum over j of all values of i is that r,i,j equals 1 is the same as summing over all values of i with all j where r,i,j is equal to 1. This summation is just summing over all user movie pairs where there is a rating. What I'm going to do is put these two cost functions together and have this where I'm just writing out the summation more explicitly as summing over all pairs i and j, where we do have a rating of the usual squared cost function and then let me take the regularization term from learning the parameters w and b, and put that here and take the regularization term from learning the features x and put them here and this ends up being our overall cost function for learning w, b, and x.

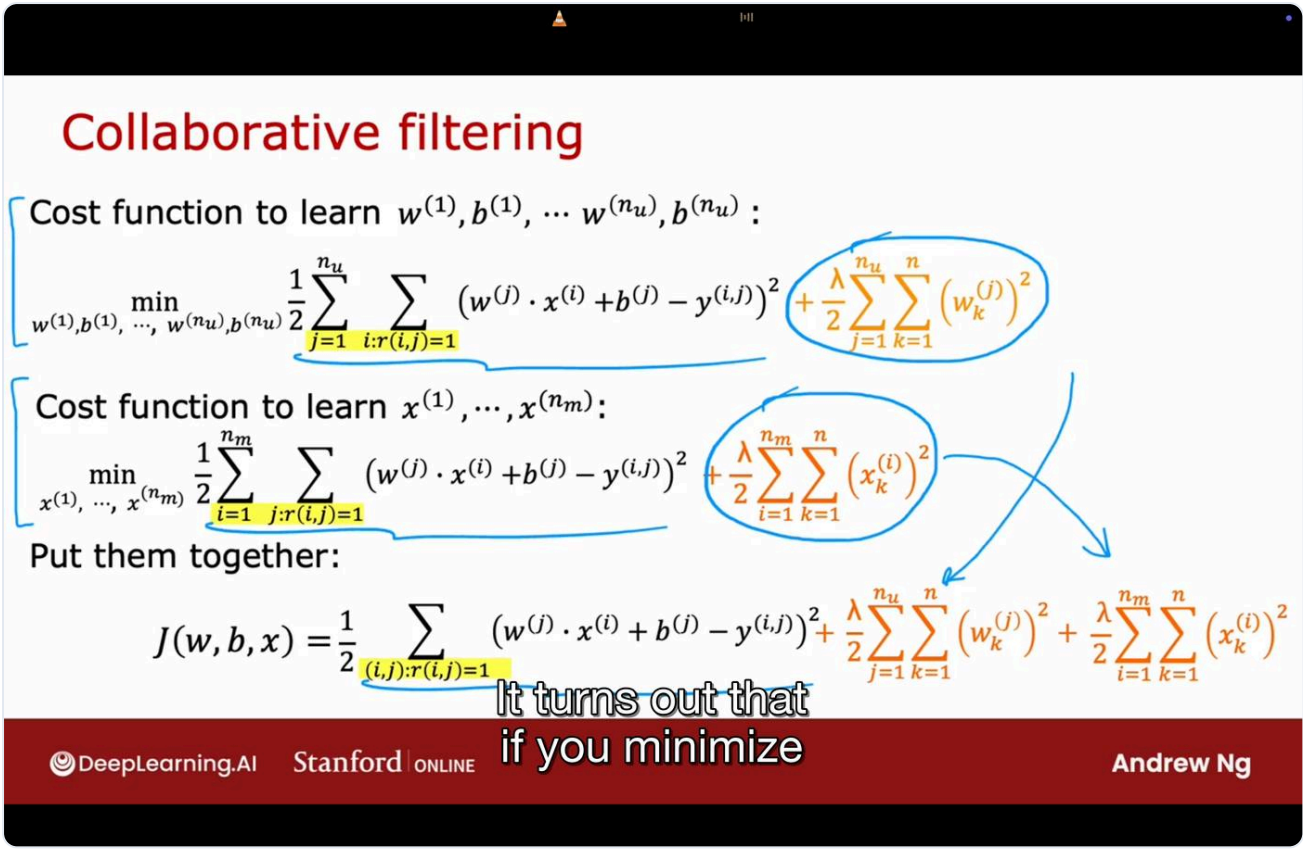


FIGURE 16.5 | Collaborative Filtering

It turns out that if you minimize this cost function as a function of w and b as well as x, then this algorithm actually works. Here's what I mean.

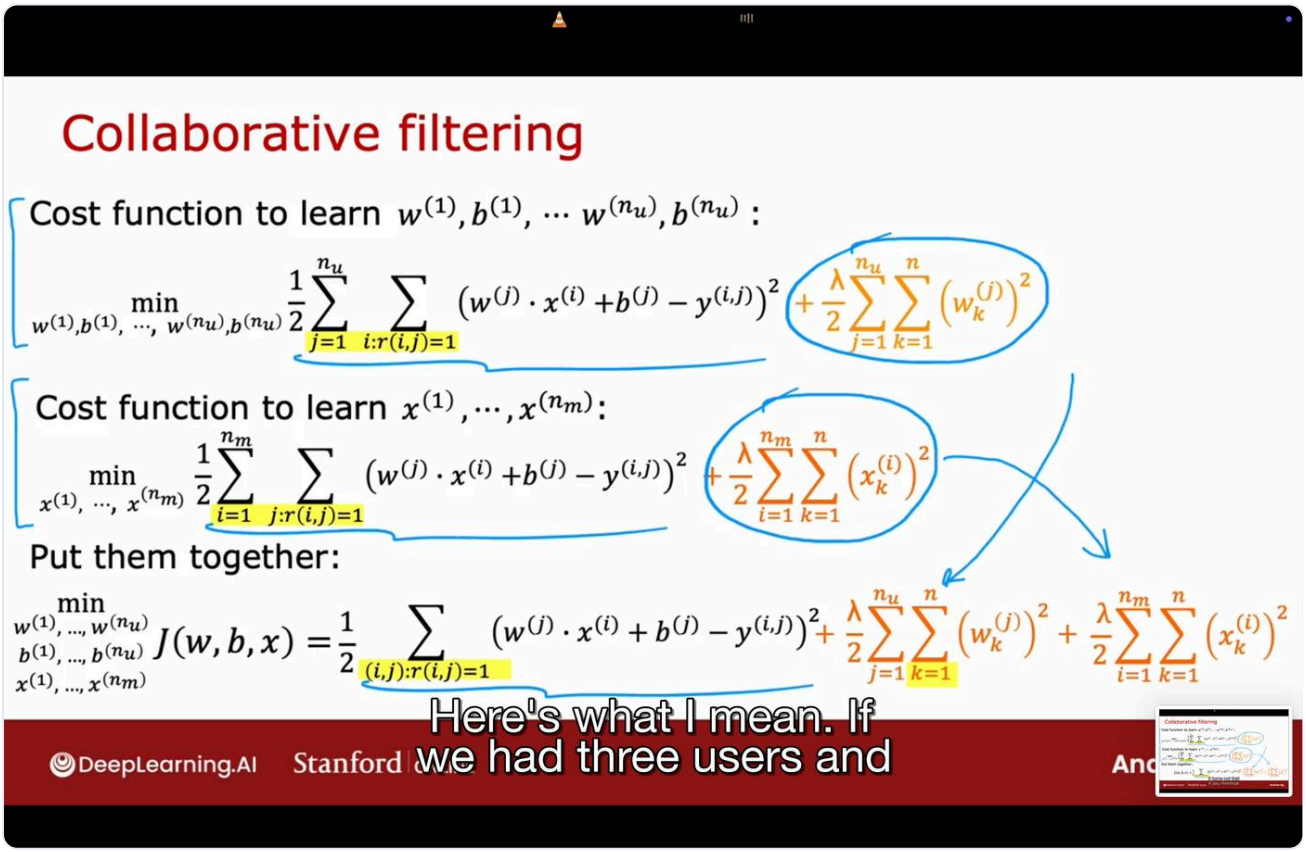


FIGURE 16.6 | Collaborative Filtering

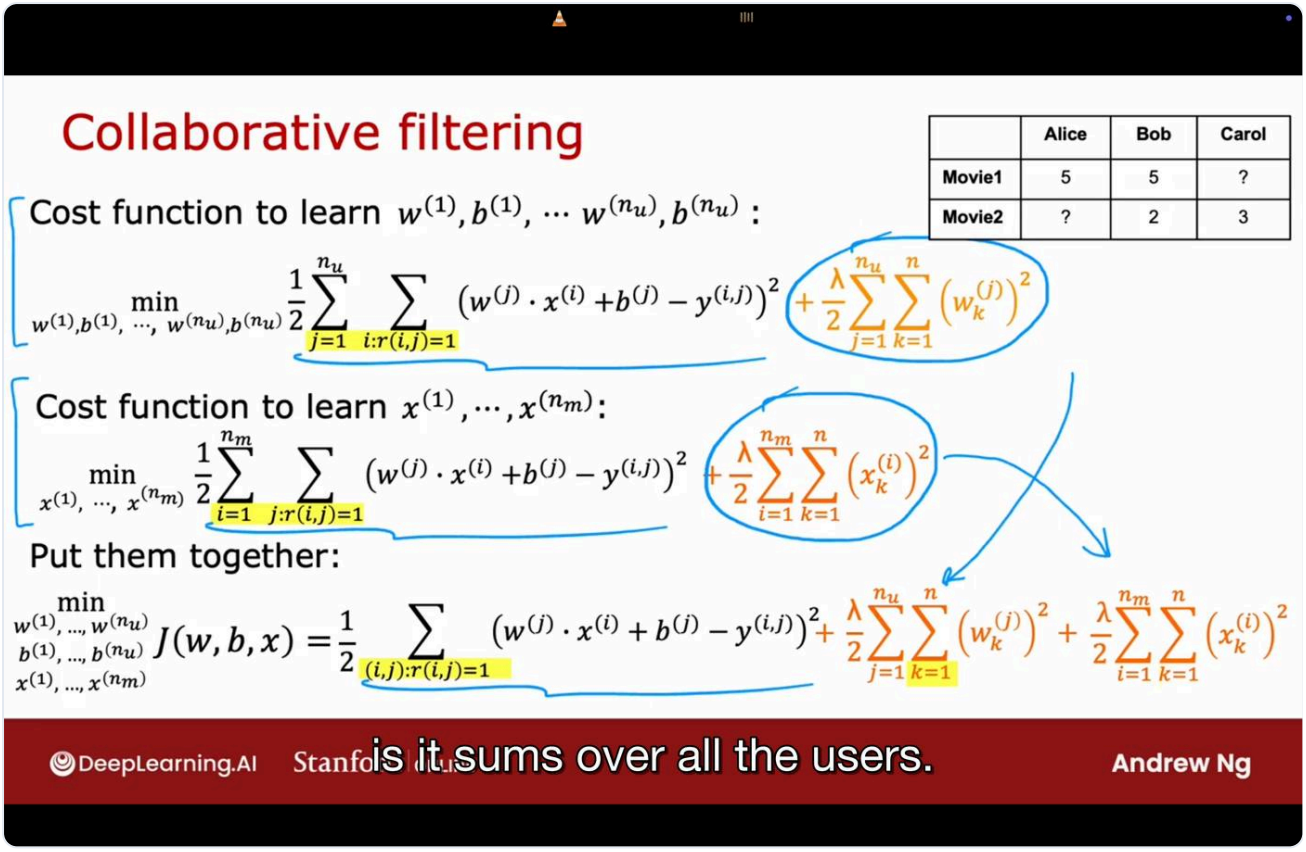


FIGURE 16.7 | Collaborative Filtering

If we had three users and two movies and if you have ratings for these four movies, but not those two, over here does, is it sums over all the users. For user 1 has determined the cost function for this, for user 2 has determined the cost function for this, for user 3 has determined the cost function for this. We're summing over users first and then having one term for each movie where there is a rating.

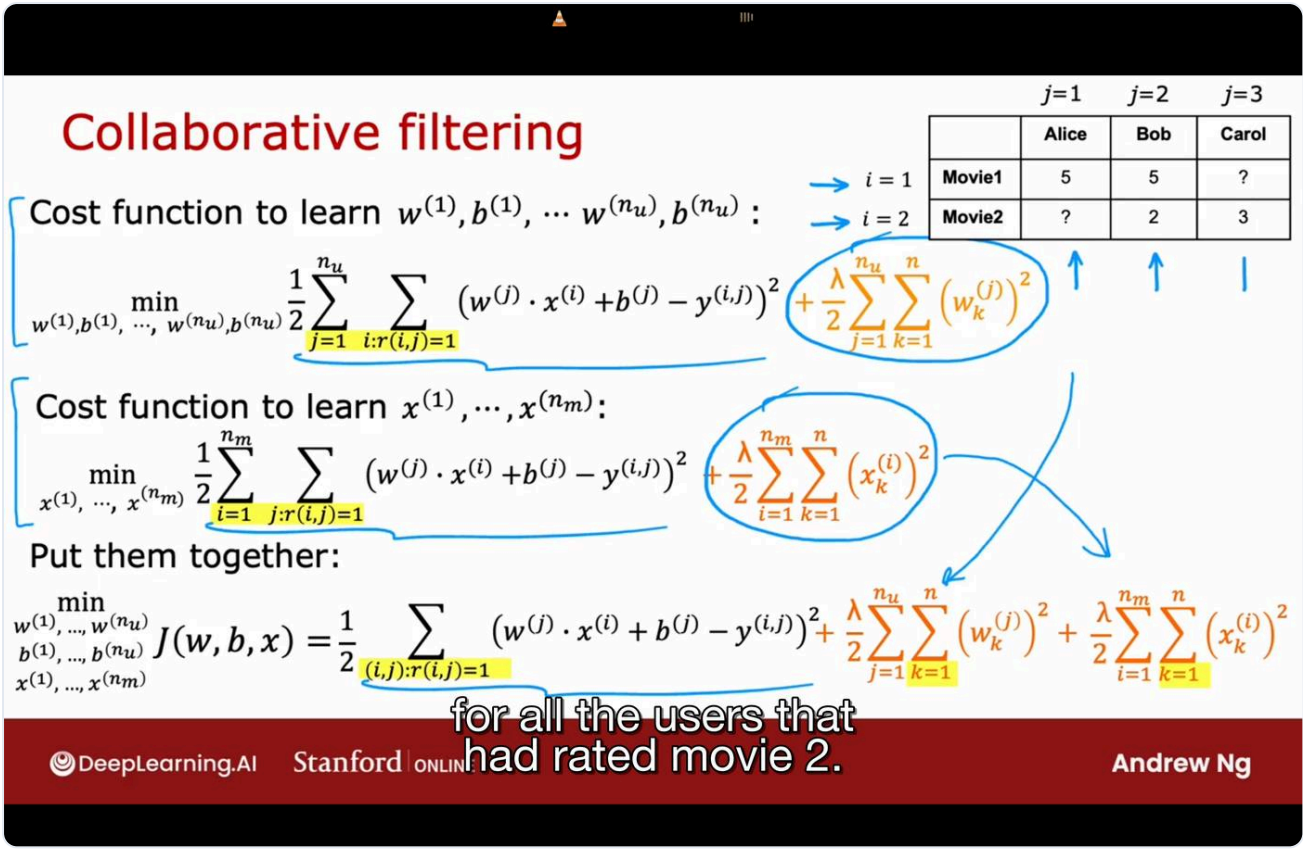


FIGURE 16.8 | Collaborative Filtering

But an alternative way to carry out the summation is to first look at movie 1, that's what this summation here does, and then to include all the users that rated movie 1, and then look at movie 2 and have a term for all the users that had rated movie 2. You see that in both cases we're just summing over these four areas where the user had rated the corresponding movie.

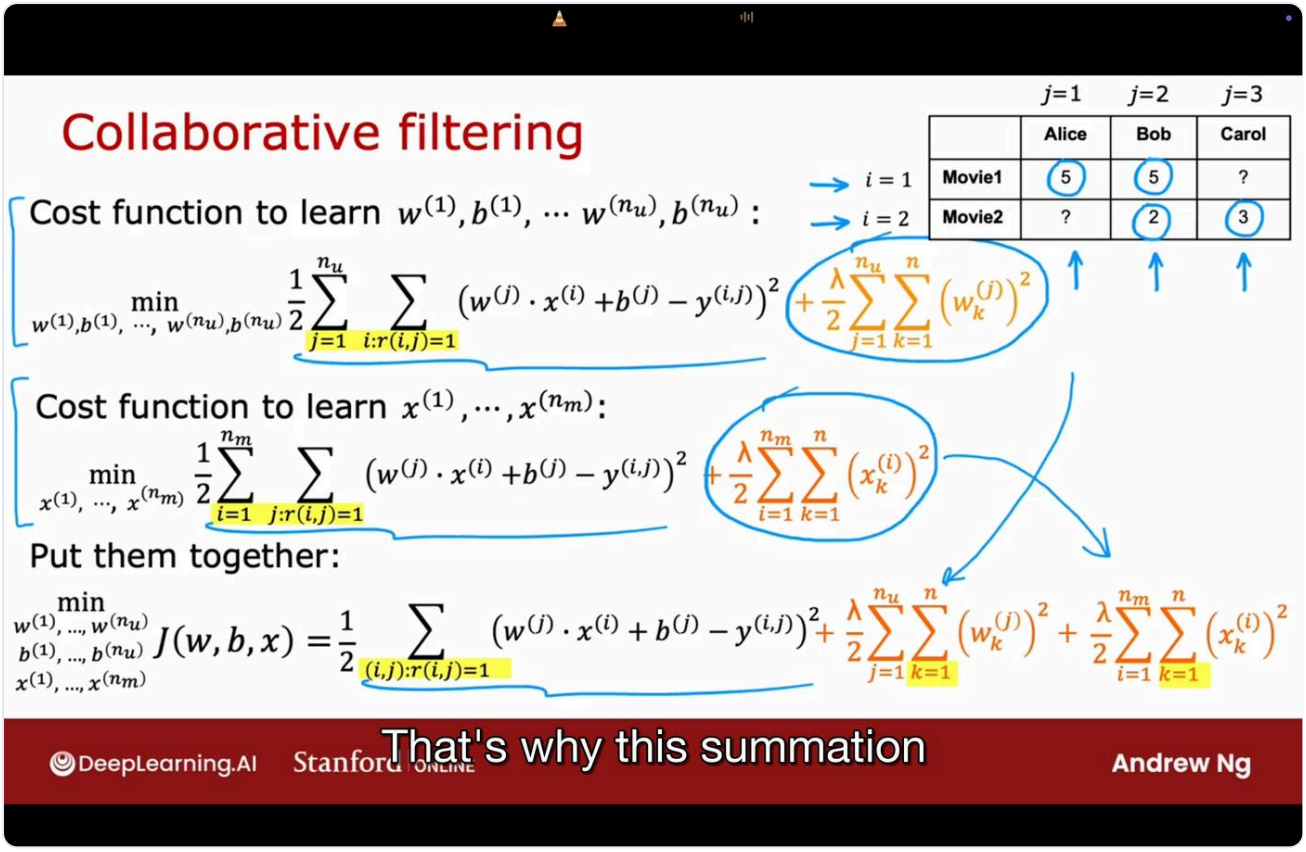


FIGURE 16.9 | Collaborative Filtering

That's why this summation on top and this summation here are the two ways of summing over all of the pairs where the user had rated that movie. How do you minimize this cost function as a function of w, b, and x? One thing you could do is to use gradient descent. In course 1 when we learned about linear regression, this is the gradient descent algorithm you had seen, where we had the cost function J, which is a function of the parameters w and b, and we'd apply gradient descent as follows.

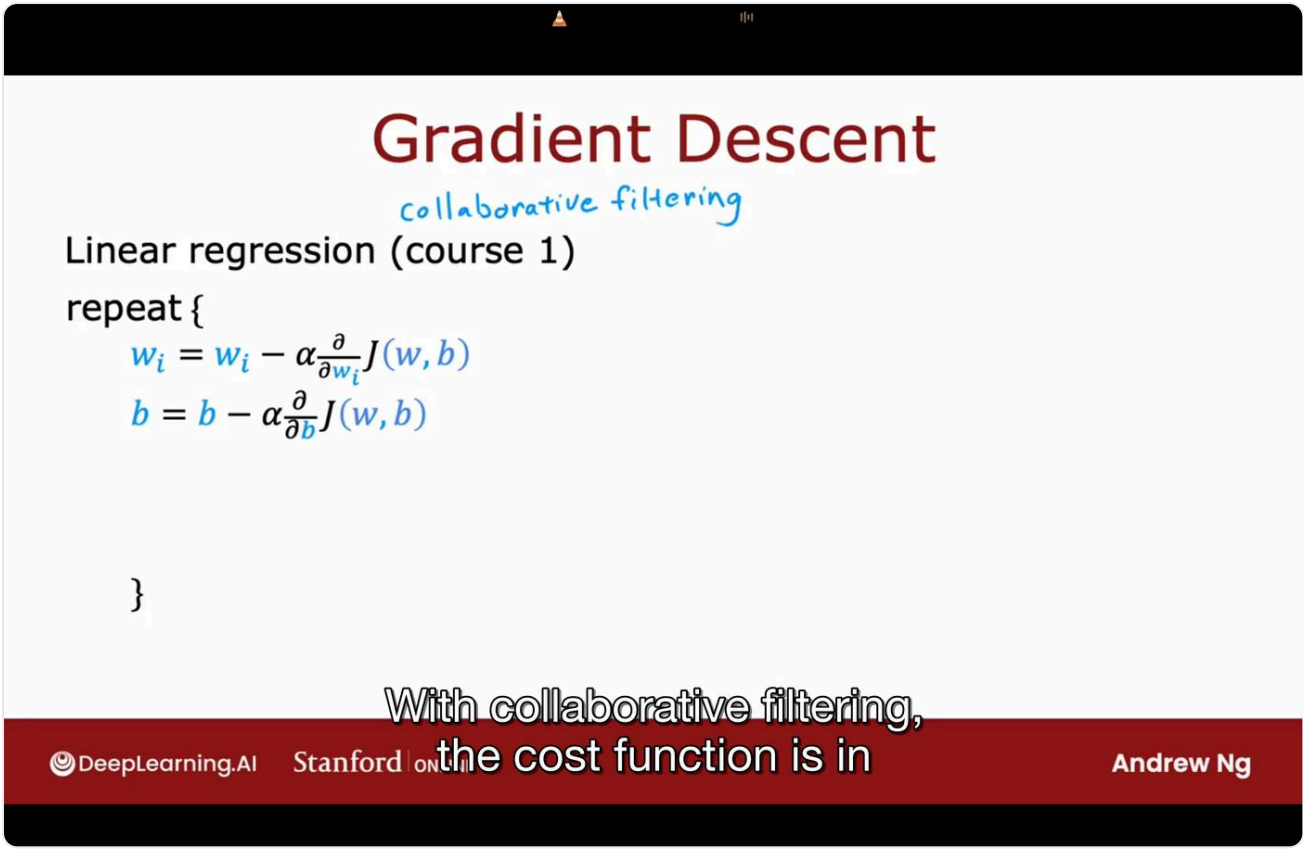


FIGURE 16.10 | Gradient Descent

With collaborative filtering, the cost function is in a function of just w and b is now a function of w, b, and x. I'm using w and b here to denote the parameters for all of the users and x here just informally to denote the features of all of the movies. But if you're able to take partial derivatives with respect to the different parameters, you can then continue to update the parameters as follows. But now we need to optimize this with respect to x as well.

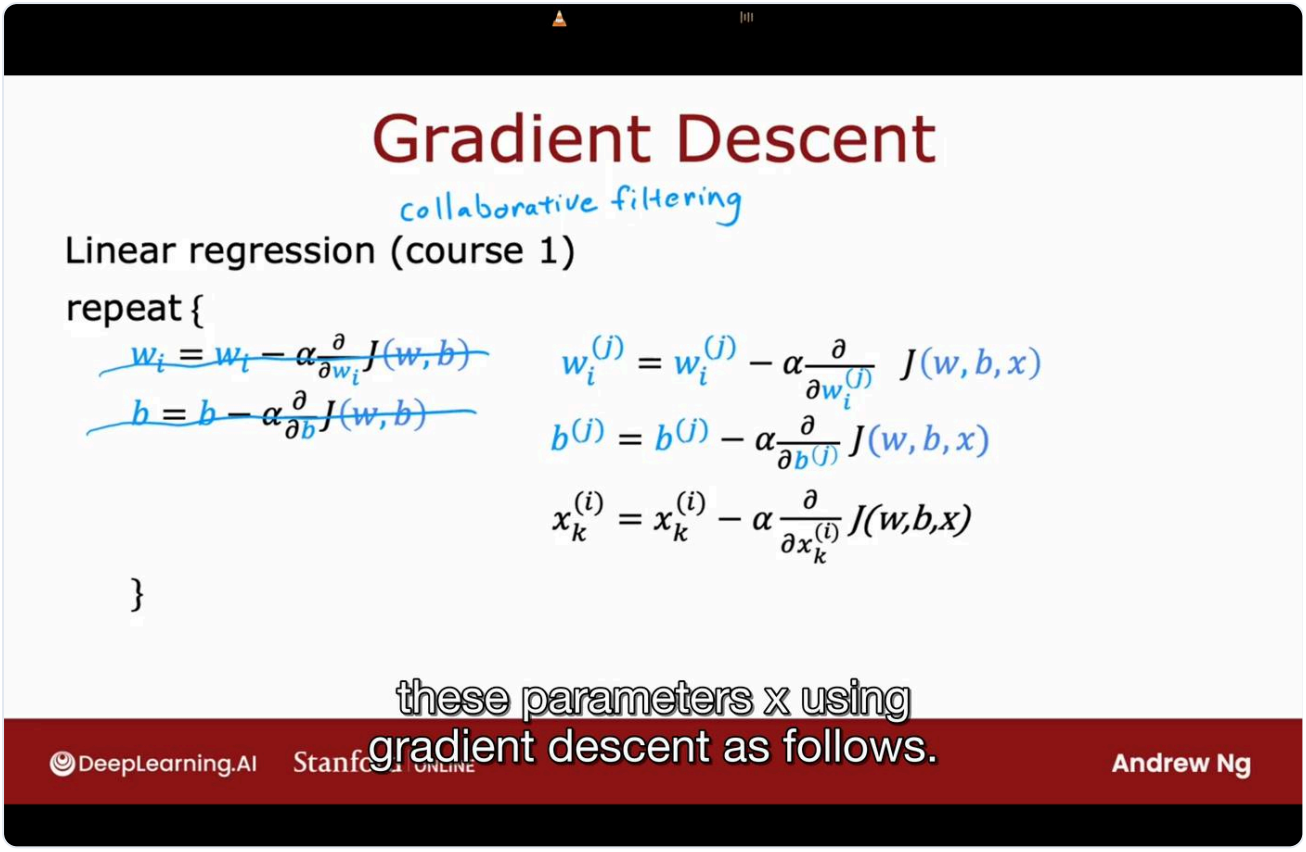


FIGURE 16.11 | Linear Regression (Course 1)

We also will want to update each of these parameters x using gradient descent as follows. It turns out that if you do this, then you actually find pretty good values of w and b as well as x. In this formulation of the problem, the parameters of w and b, and x is also a parameter.

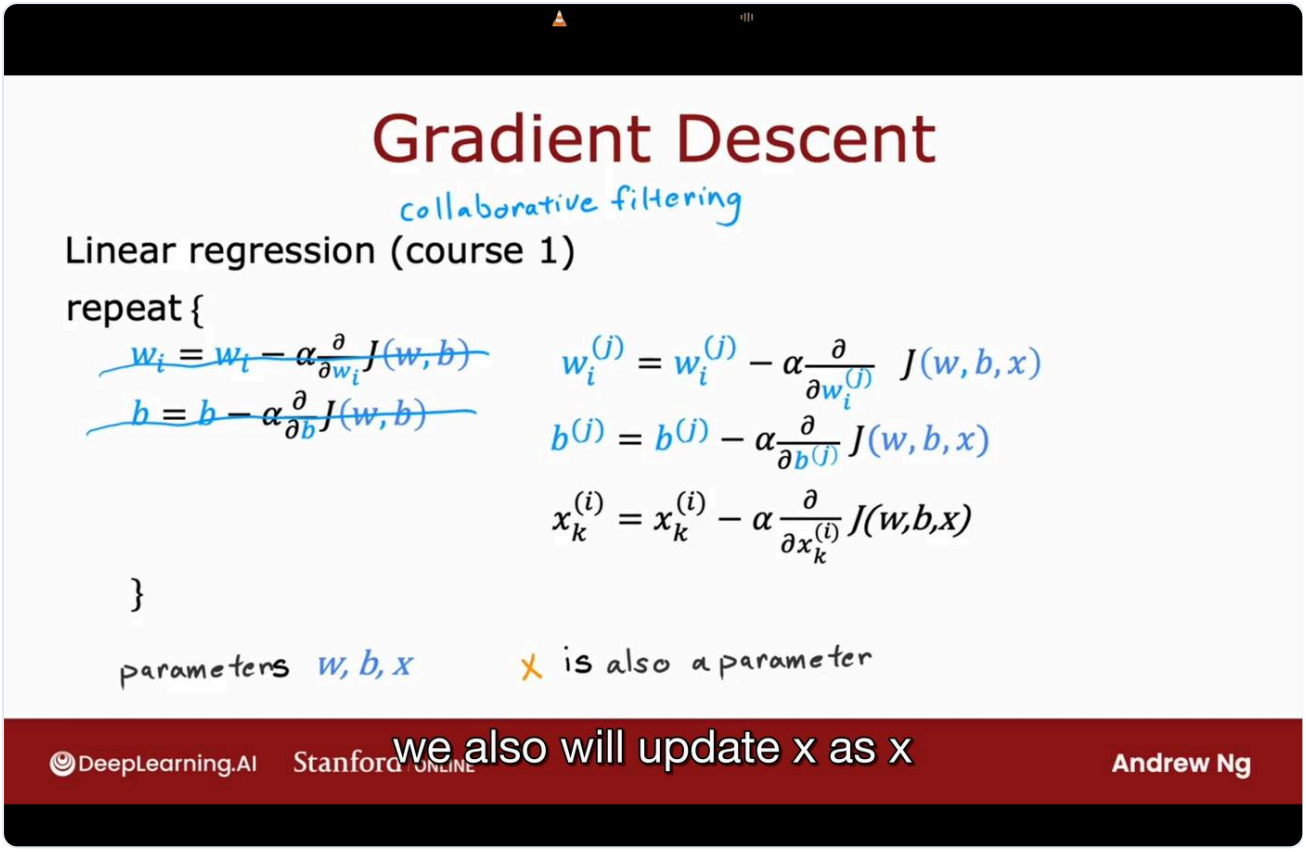


FIGURE 16.12 | Gradient Descent

Then finally, to learn the values of x, we also will update x as x minus the partial derivative respect to x of the cost w, b, x. I'm using the notation here a little bit informally and not keeping very careful track of the superscripts and subscripts, but the key takeaway I hope you have from this is that the parameters to this model are w and b, and x now is also a parameter, which is why we minimize the cost function as a function of all three of these sets of parameters, w and b, as well as x. The algorithm we derived is called collaborative filtering, and the name collaborative filtering refers to the sense that because multiple users have rated the same movie collaboratively, given you a sense of what this movie maybe like, that allows you to guess what are appropriate features for that movie, and this in turn allows you to predict how other users that haven't yet rated that same movie may decide to rate it in the future.

This collaborative filtering is this gathering of data from multiple users. This collaboration between users to help you predict ratings for even other users in the future. So far, our problem formulation has used movie ratings from 1- 5 stars or from 0- 5 stars. A very common use case of recommender systems is when you have binary labels such as that the user favors, or like, or interact with an item. In the next video, let's take a look at a generalization of the model that you've seen so far to binary labels. Let's go see that in the next video.

LECTURE 17

Cost Function for Binary Application

Many important applications of recommender systems or collective filtering algorithms involved binary labels where instead of a user giving you a one to five star or zero to five star rating, they just somehow give you a sense of they like this item or they did not like this item. Let's take a look at how to generalize the algorithm you've seen to this setting. The process we'll use to generalize the algorithm will be very much reminiscent to how we have gone from linear regression to logistic regression, to predicting numbers to predicting a binary label back in course one, let's take a look. Here's an example of a collaborative filtering data set with binary labels.

A one the notes that the user liked or engaged with a particular movie.

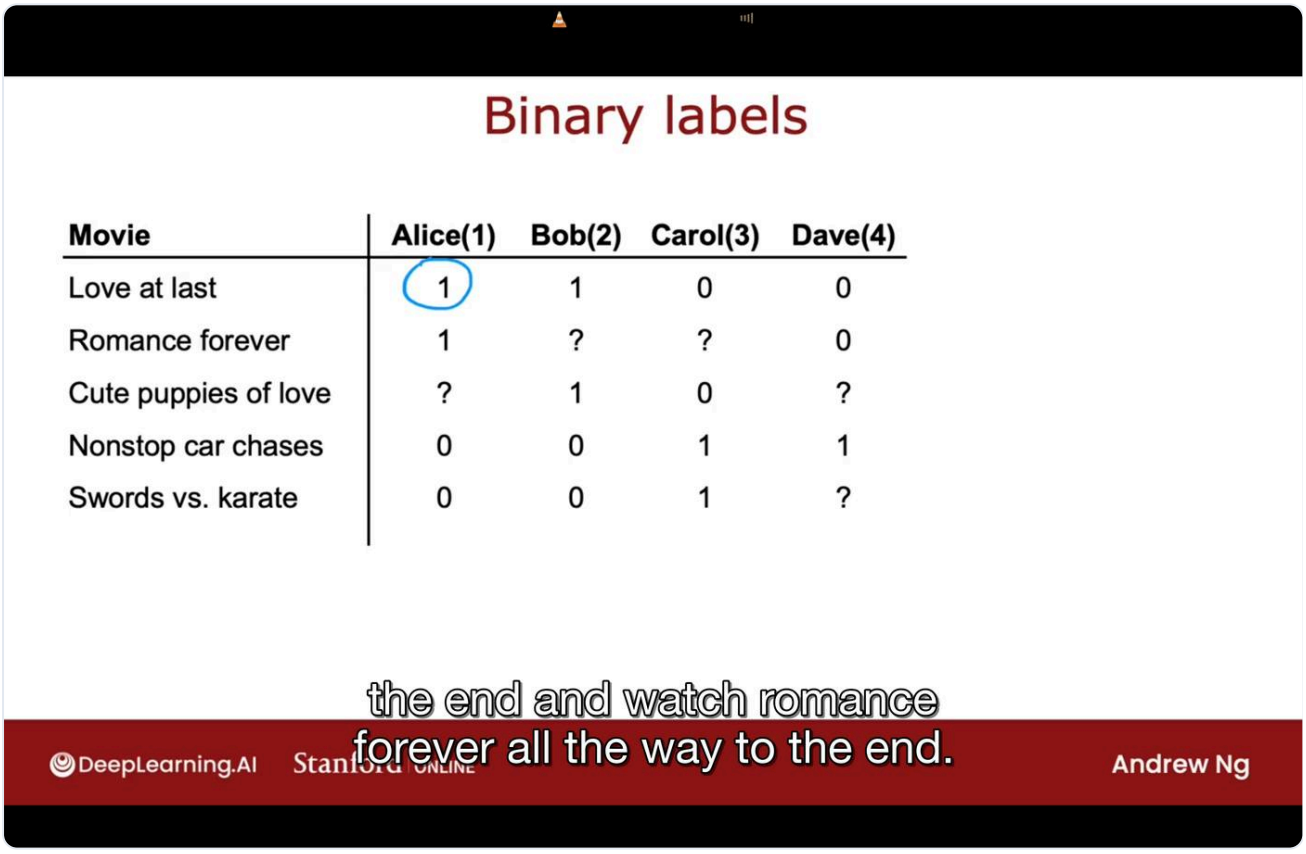


FIGURE 17.1 | Binary Labels

So label one could mean that Alice watched the movie Love at last all the way to the end and watch romance forever all the way to the end. But after playing a few minutes of nonstop car chases decided to stop the video and move on. Or it could mean that she explicitly hit like or favorite on an app to indicate that she liked these movies. But after checking out nonstop car chasers and swords versus karate did not hit like.

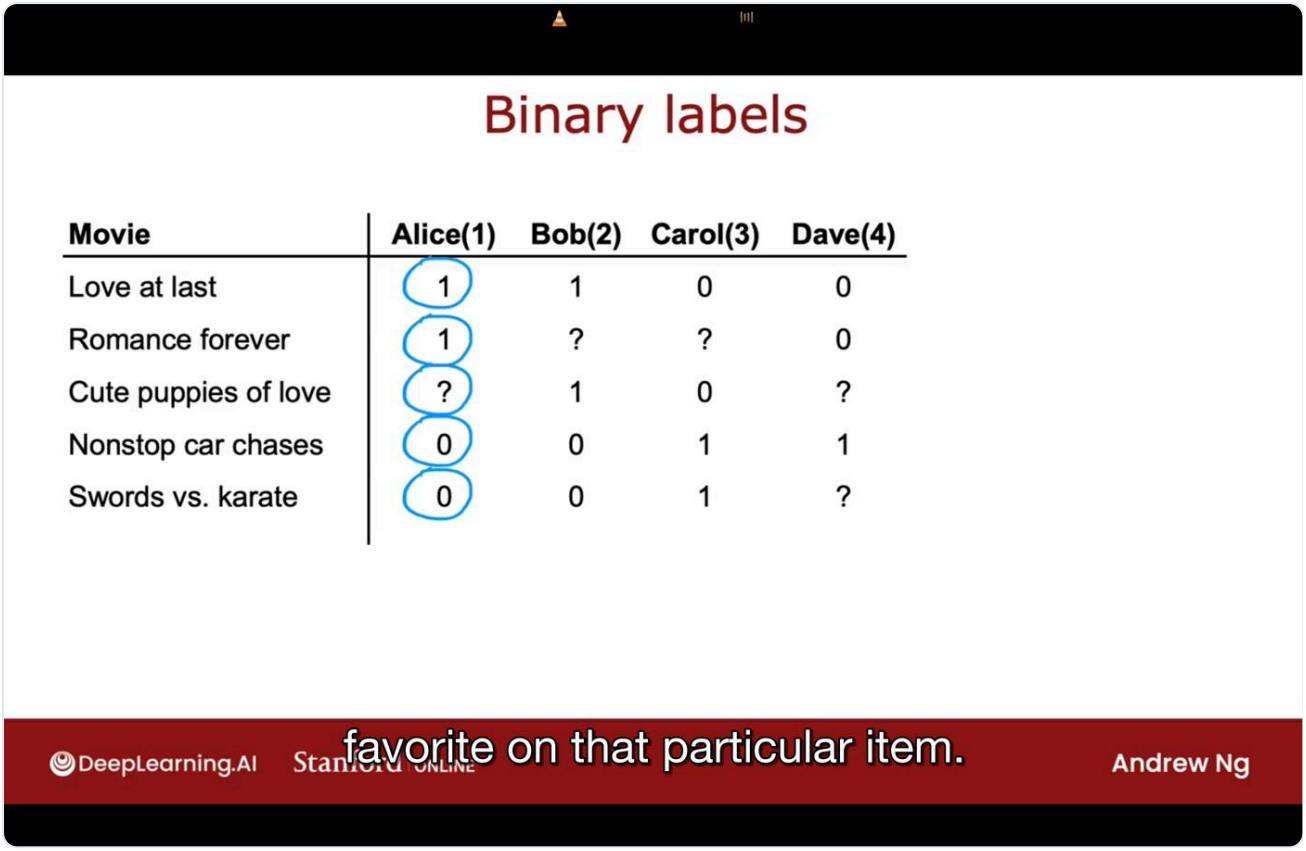


FIGURE 17.2 | Binary Labels

And the question mark usually means the user has not yet seen the item and so they weren't in a position to decide whether or not to hit like or favorite on that particular item. So the question is how can we take the collaborative filtering algorithm that you saw in the last video and get it to work on this dataset. And by predicting how likely Alice, Bob carol and Dave are to like the items that they have not yet rated, we can then decide how much we should recommend these items to them. There are many ways of defining what is the label one and what is the label zero, and what is the label question mark in collaborative filtering with binary labels.

Let's take a look at a few examples. In an online shopping website, the label could denote whether or not user j chose to purchase an item after they were exposed to it, after they were shown the item. So one would denote that they purchase it zero would denote that they did not purchase it. And the question mark would denote that they were not even shown were not even exposed to the item. Or in a social media setting, the labels one or zero could denote did the user favorite or like an item after they were shown it. And question mark would be if they have not yet been shown the item or many sites instead of asking for explicit user rating will use the user behavior to try to guess if the user like the item.

So for example, you can measure if a user spends at least 30 seconds of an item. And if they did, then assign that a label one because the user found the item engaging or if a user was shown an item but did not spend at least 30 seconds with it, then assign that a label zero. Or if the user was not shown the item yet, then assign it a question mark. Another way to generate a rating implicitly as a function of the user behavior will be to see that the user click on an item.

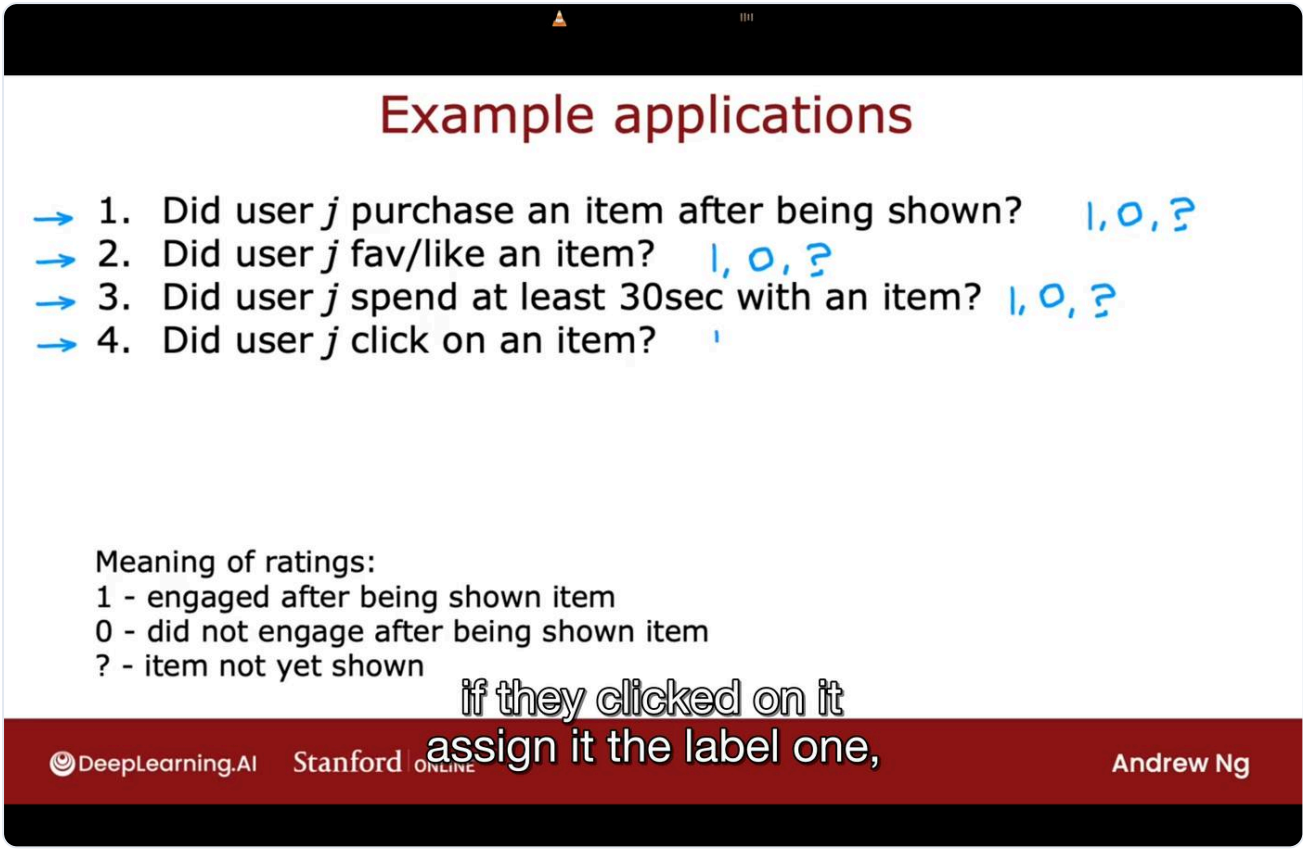


FIGURE 17.3 | Example Applications

This is often done in online advertising where if the user has been shown an ad, if they clicked on it assign it the label one, if they did not click assign it the label zero and the question mark were referred to if the user has not even been shown that ad in the first place. So often these binary labels will have a rough meaning as follows. A labor of one means that the user engaged after being shown an item And engaged could mean that they clicked or spend 30 seconds or explicitly favorite or like to purchase the item. A zero will reflect the user not engaging after being shown the item, the question mark will reflect the item not yet having been shown to the user.

So given these binary labels, let's look at how we can generalize our algorithm which is a lot like linear regression from the previous couple videos to predicting these binary outputs. Previously we were predicting label yij as wj.xi+b. So this was a lot like a linear regression model. For binary labels, we're going to predict that the probability of yijb=1 is given by not wj.xi+b. But it said by g of this formula, where now g(z) 1/1 +e to the -z. So this is the logistic function just like we saw in logistic regression.

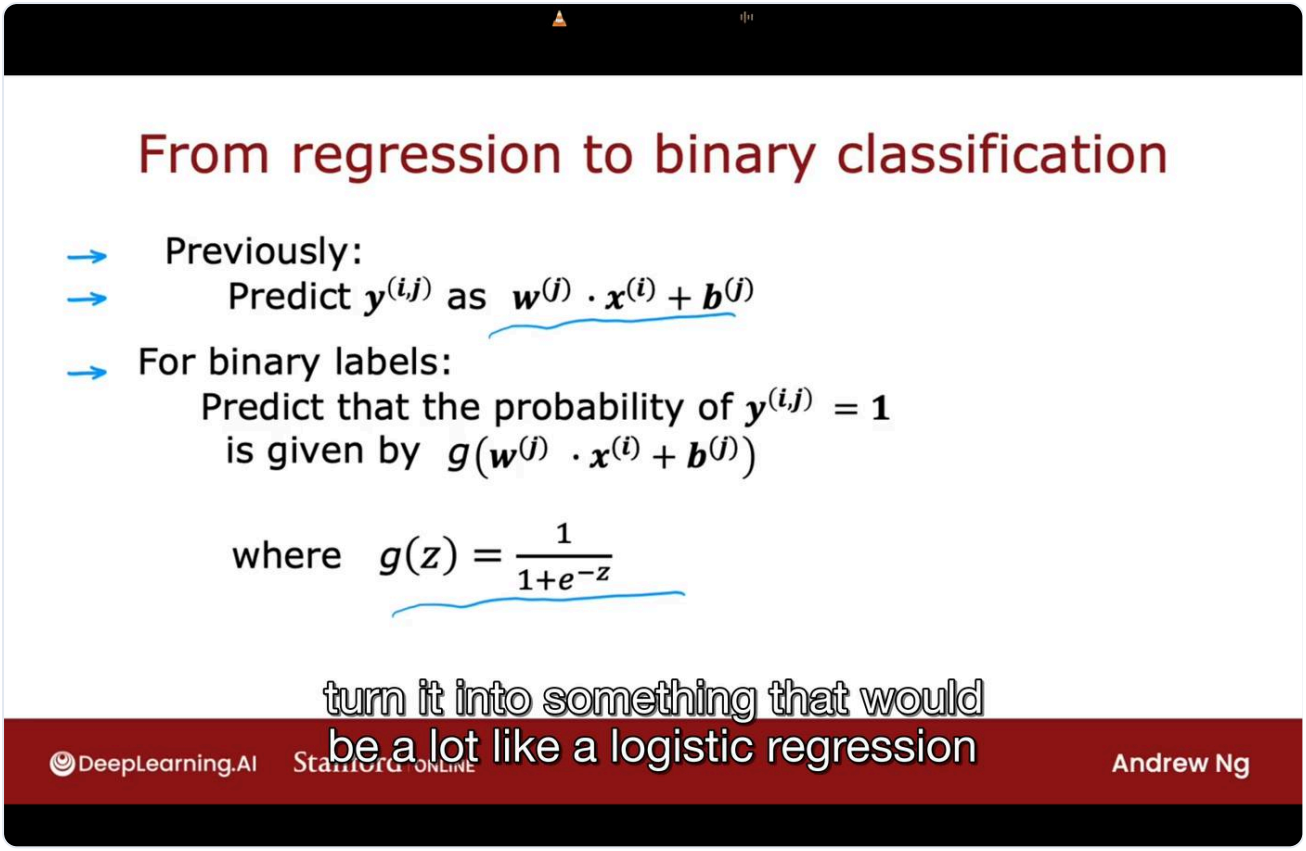


FIGURE 17.4 | From Regression to Binary Classification

And what we would do is take what was a lot like a linear regression model and turn it into something that would be a lot like a logistic regression model where will now predict the probability of yij being 1 that is of the user having engaged with or like the item using this model. In order to build this algorithm, we'll also have to modify the cost function from the squared error cost function to the cost function that is more appropriate for binary labels for a logistic regression like model. So previously, this was the cost function that we had where this term play their role similar to f(x), the prediction of the algorithm. When you now have binary labels, yij when the labels are one or zero or question mark, then the prediction f(x) becomes instead of wj.xi+b j it becomes g of this where g is the logistic function.

And similar to when we had derived logistic regression, we had written out the following loss function for a single example which was at the loss if the algorithm predicts f(x) and the true label was y, the loss was this.

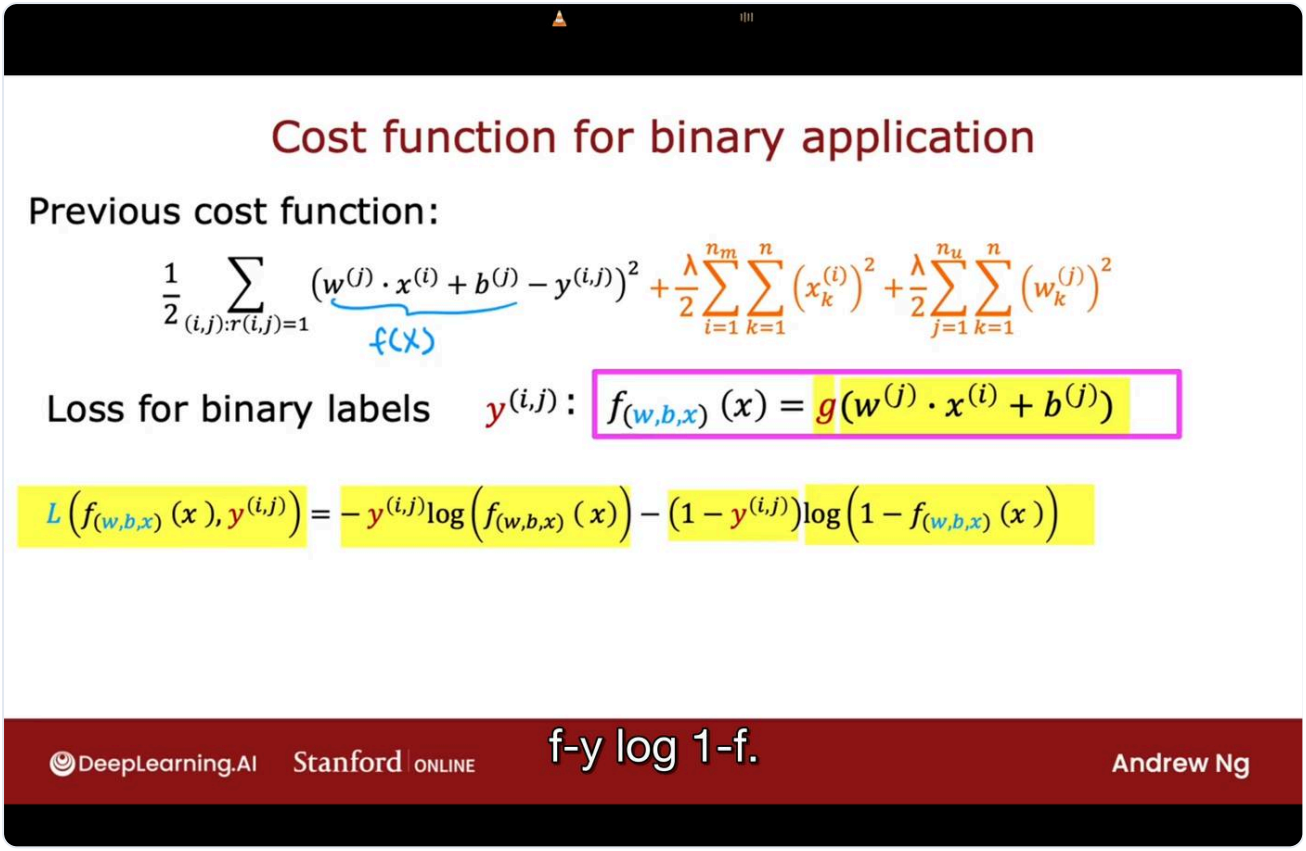


FIGURE 17.5 | Cost Function for Binary Application

It was -y log f-y log 1-f.

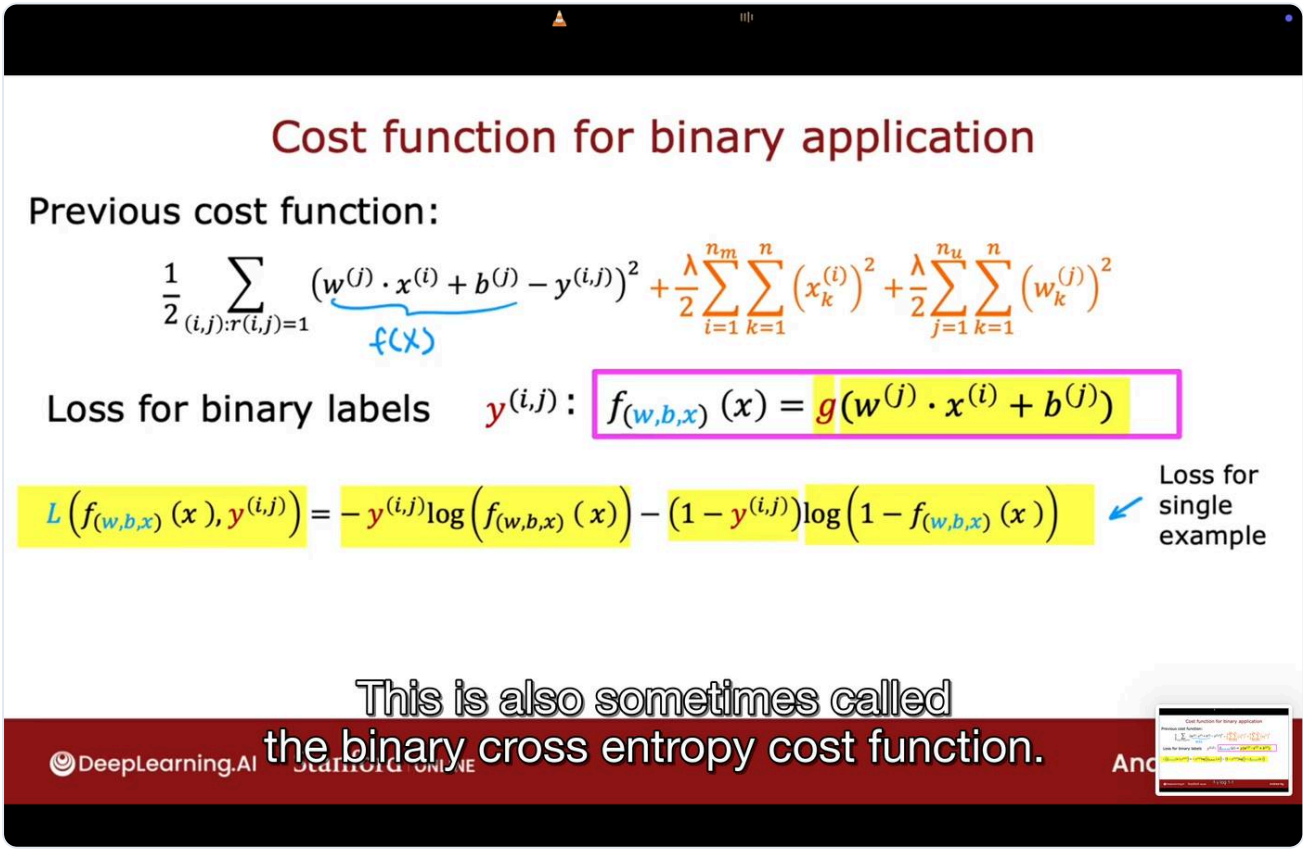


FIGURE 17.6

This is also sometimes called the binary cross entropy cost function. But this is a standard cost function that we used for logistic regression as was for the binary classification problems when we're training neural networks.

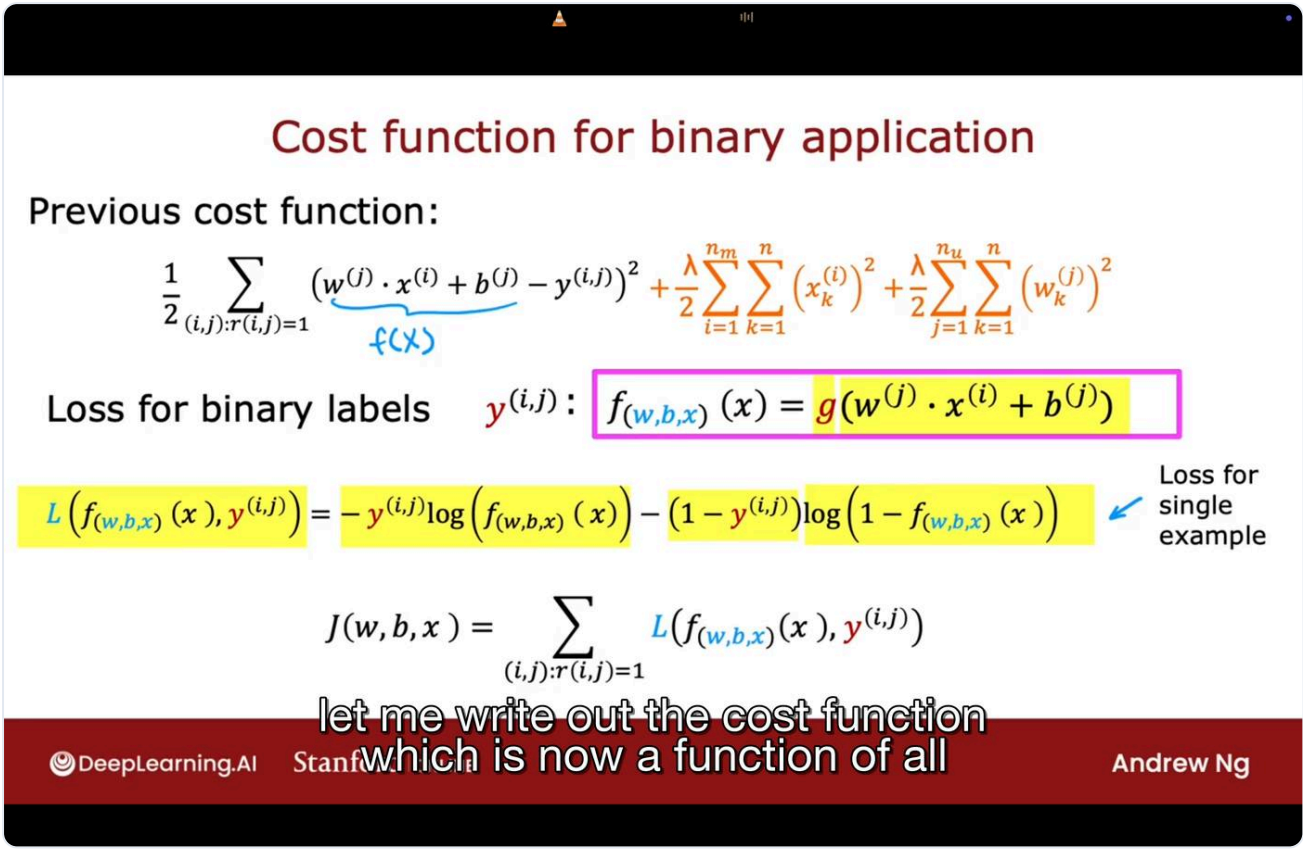


FIGURE 17.7 | Cost Function for Binary Application

And so to adapt this to the collaborative filtering setting, let me write out the cost function which is now a function of all the parameters w and b as well as all the parameters x which are the features of the individual movies or items of. We now need to sum over all the pairs ij where riij=1 notice this is just similar to the summation up on top. And now instead of this squared error cost function, we're going to use that loss function.

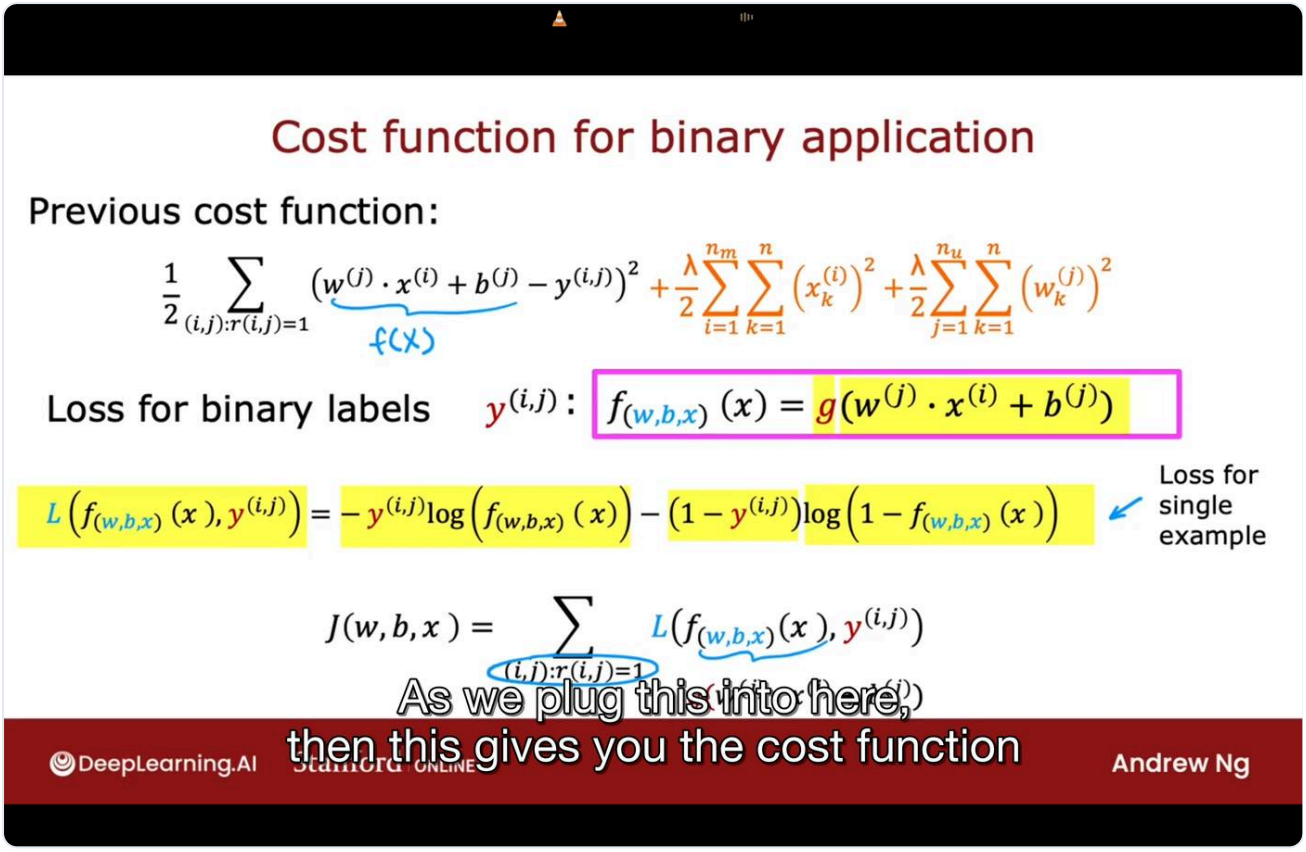


FIGURE 17.8 | Cost Function for Binary Application

There's a function of f(x), yij. Where f(x) here? That's my abbreviation.

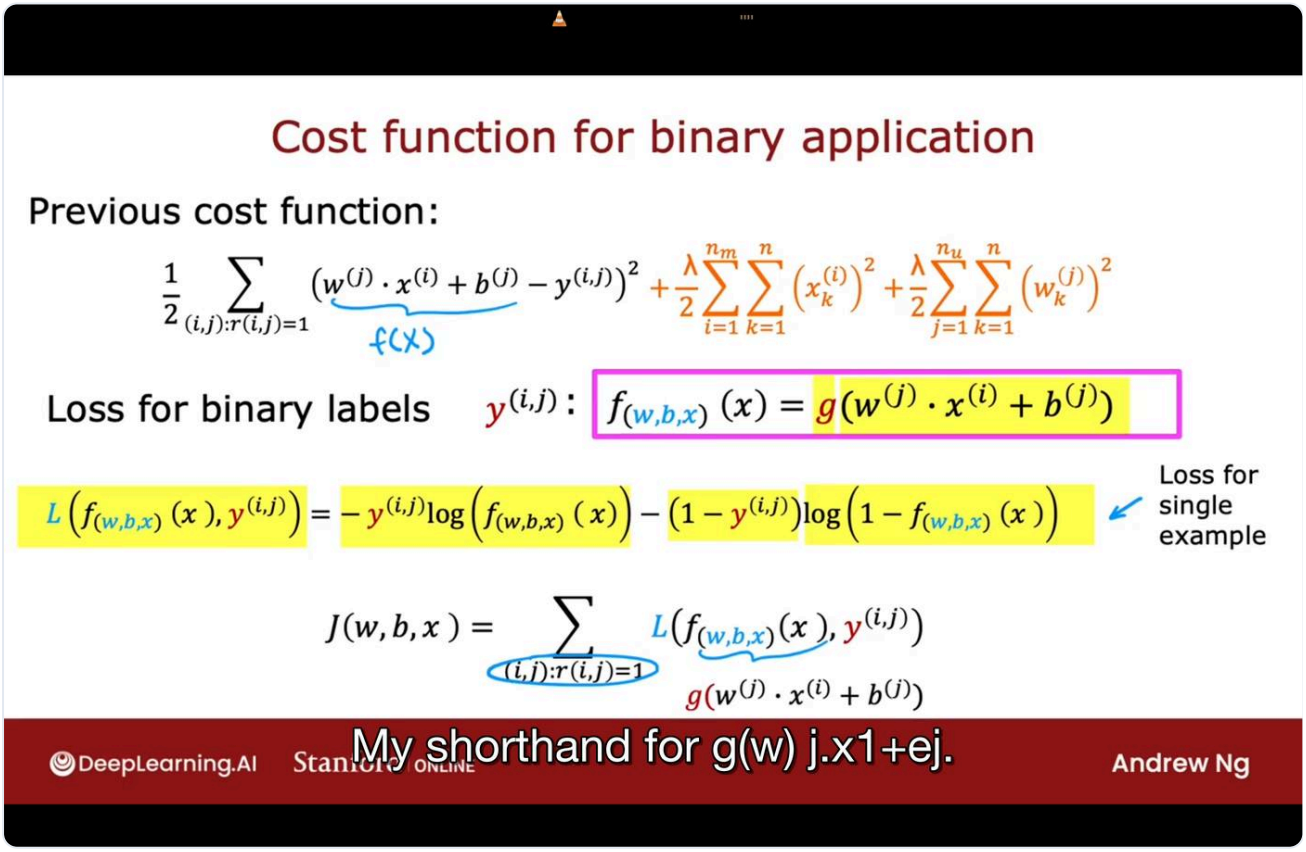


FIGURE 17.9 | Cost Function for Binary Application

My shorthand for g(w) j.x1+ej.

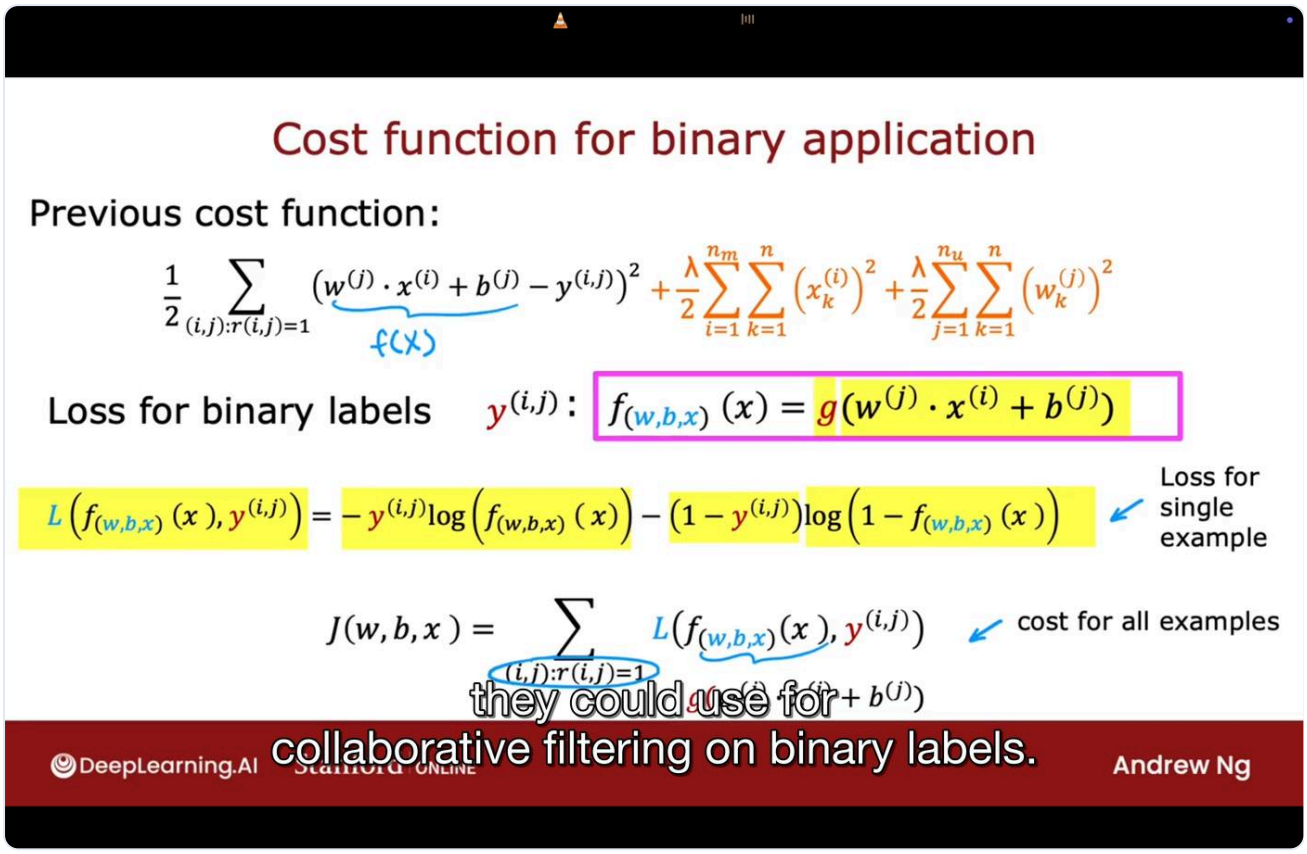


FIGURE 17.10 | Previous Cost Function:

As we plug this into here, then this gives you the cost function they could use for collaborative filtering on binary labels. So that's it. That's how you can take the linear regression, like collaborative filtering algorithm and generalize it to work with binary labels. And this actually very significantly opens up the set of applications you can address with this algorithm. Now, even though you've seen the key structure and cost function of the algorithm, there are also some implementation, all tips that will make your algorithm work much better. Let's go on to the next video to take a look at some details of how you implement it and some little modifications that make the algorithm run much faster.

Let's go on to the next video.

LECTURE 18

Mean Normalization

Back in the first course, you have seen how for linear regression, future normalization can help the algorithm run faster. In the case of building a recommended system with numbers wide such as movie ratings from one to five or zero to five stars, it turns out your algorithm will run more efficiently. And also perform a bit better if you first carry out mean normalization. That is if you normalize the movie ratings to have a consistent average value, let's take a look at what that means. So here's the data set that we've been using.

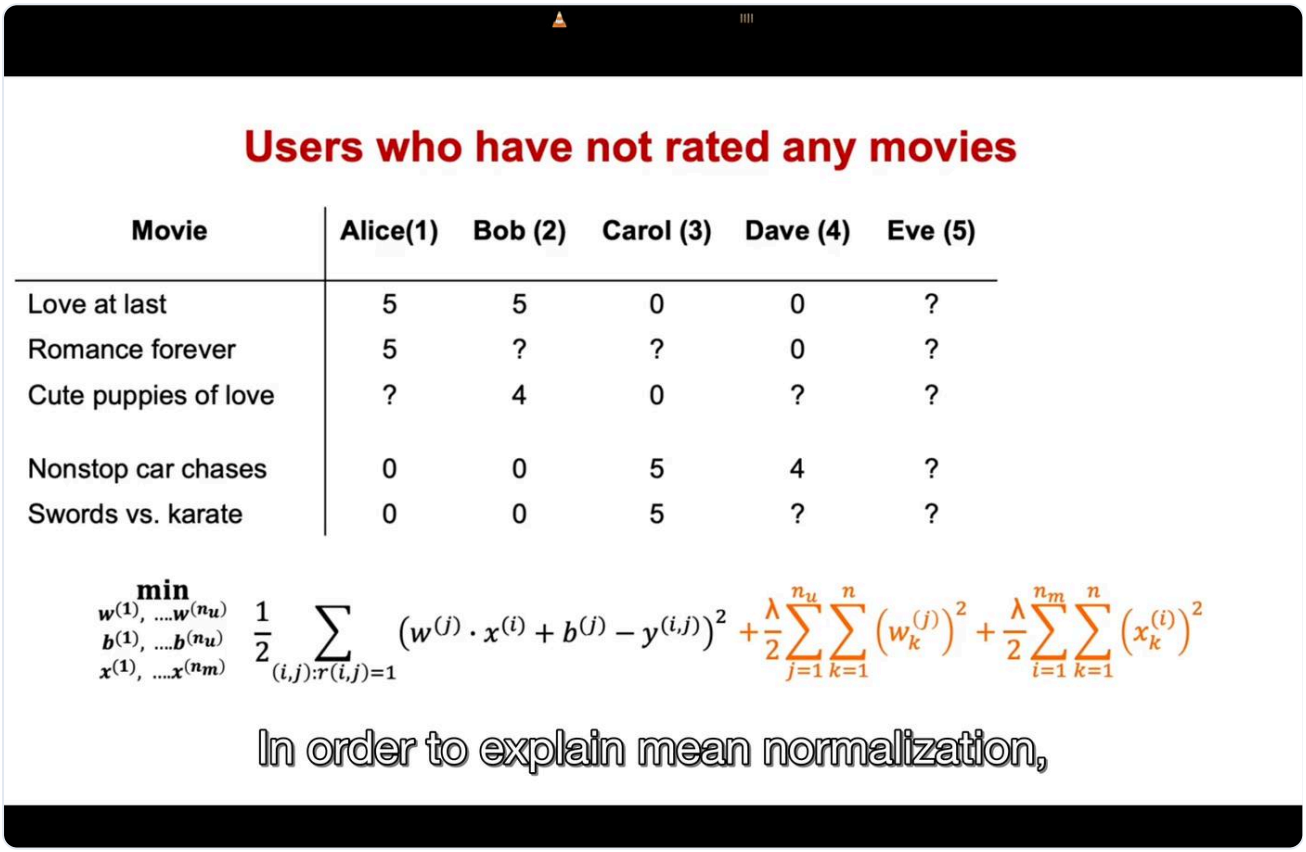


FIGURE 18.1 | Users Who Have Not Rated Any Movies

And down below is the cost function you used to learn the parameters for the model. In order to explain mean normalization, I'm actually going to add fifth user Eve who has not yet rated any movies. And you see in a little bit that adding mean normalization will help the algorithm make better predictions on the user Eve. In fact, if you were to train a collaborative filtering algorithm on this data, then because we are trying to make the parameters w small because of this regularization term. If you were to run the algorithm on this dataset, you actually end up with the parameters w for the fifth user, for the user Eve to be equal to [0 0] as well as quite likely b(5) = 0.

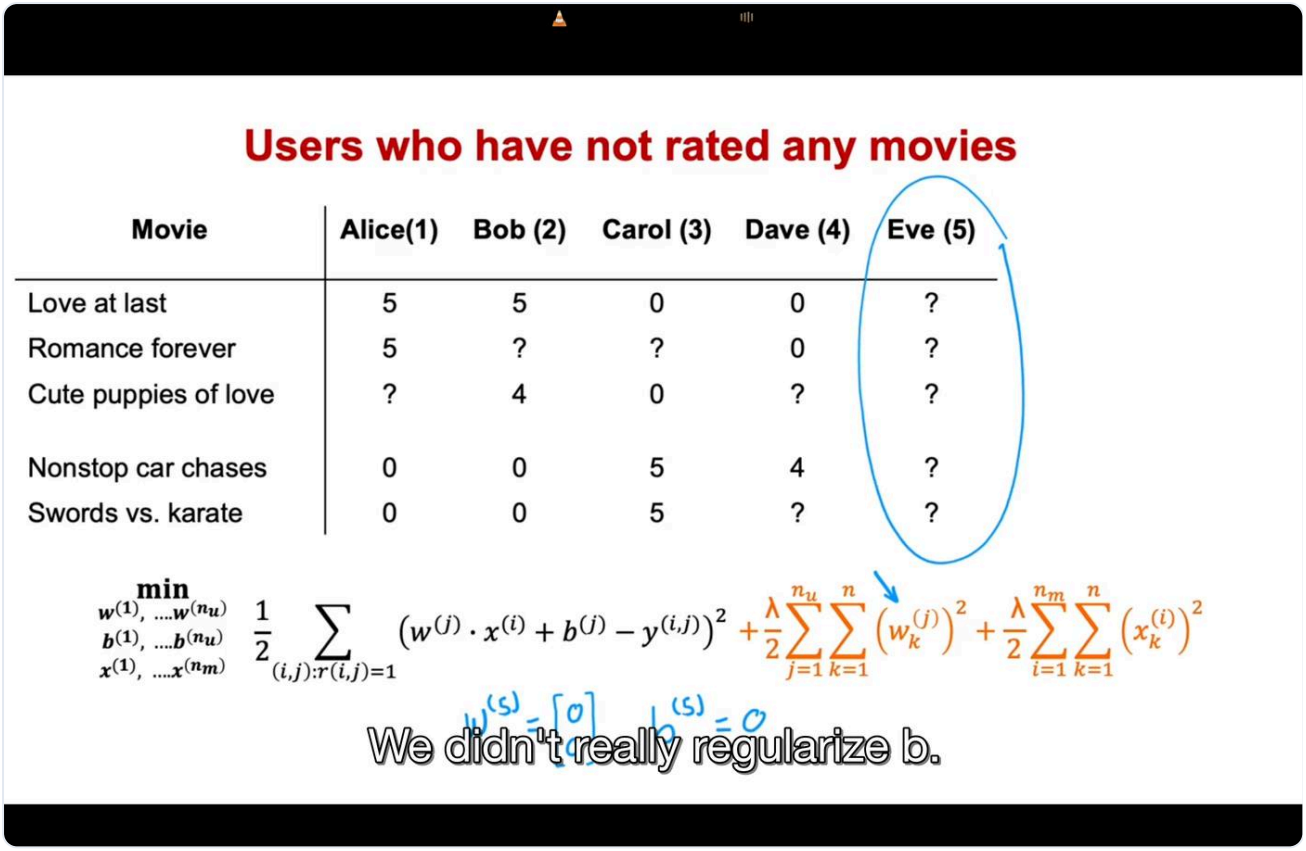


FIGURE 18.2 | Users Who Have Not Rated Any Movies

Because Eve hasn't rated any movies yet, the parameters w and b don't affect this first term in the cost function because none of Eve's movie's rating play a role in this squared error cost function. And so minimizing this means making the parameters w as small as possible. We didn't really regularize b. But if you initialize b to 0 as the default, you end up with b(5) = 0 as well. But if these are the parameters for user 5 that is for Eve, then what the average will end up doing is predict that all of Eve's movies ratings would be w(5) dot x for movie i + b(5).

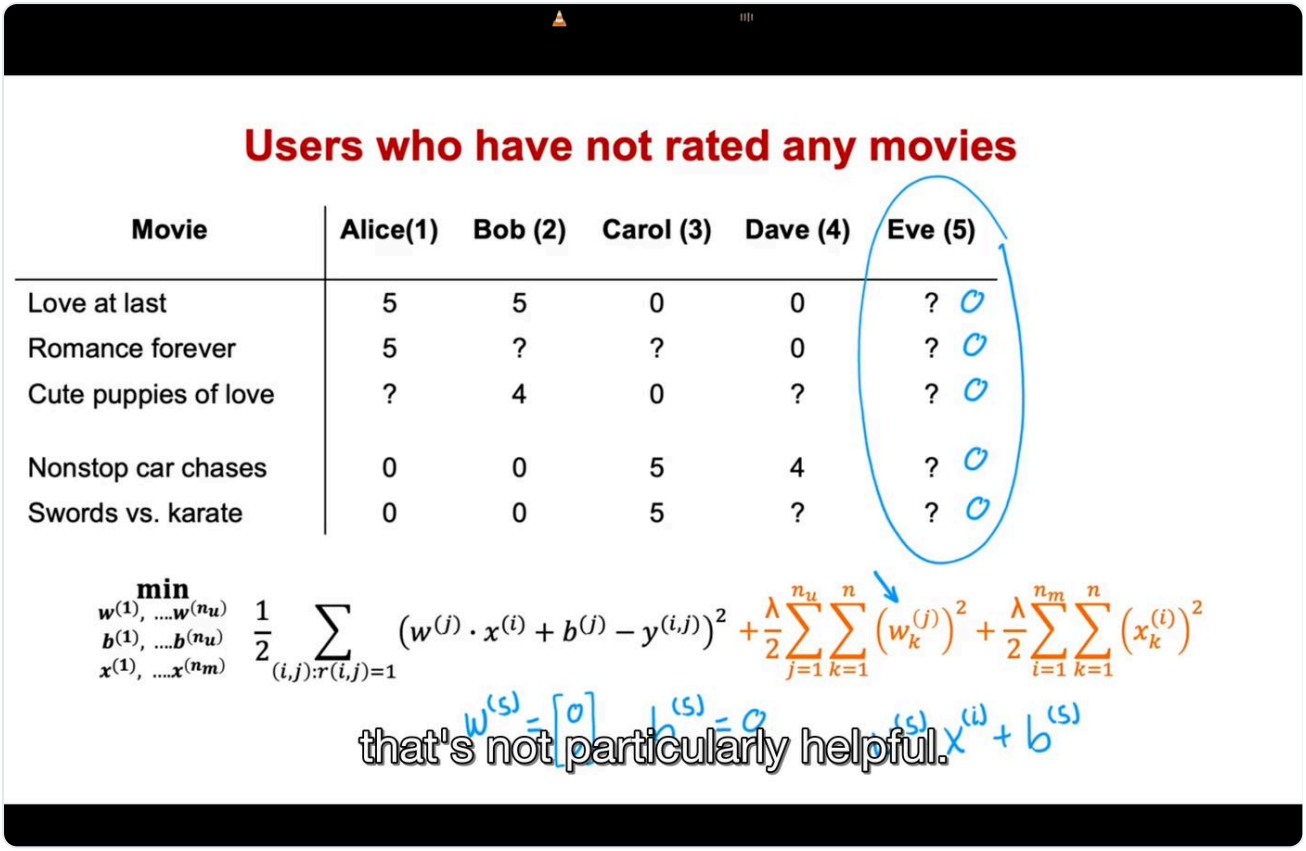


FIGURE 18.3 | Users Who Have Not Rated Any Movies

And this is equal to 0 if w and b above equals 0. And so this algorithm will predict that if you have a new user that has not yet rated anything, we think they'll rate all movies with zero stars and that's not particularly helpful. So in this video, we'll see that mean normalization will help this algorithm come up with better predictions of the movie ratings for a new user that has not yet rated any movies. In order to describe mean normalization, let me take all of the values here including all the question marks for Eve and put them in a two dimensional matrix like this. Just to write out all the ratings including the question marks in a more sustained and more compact way.

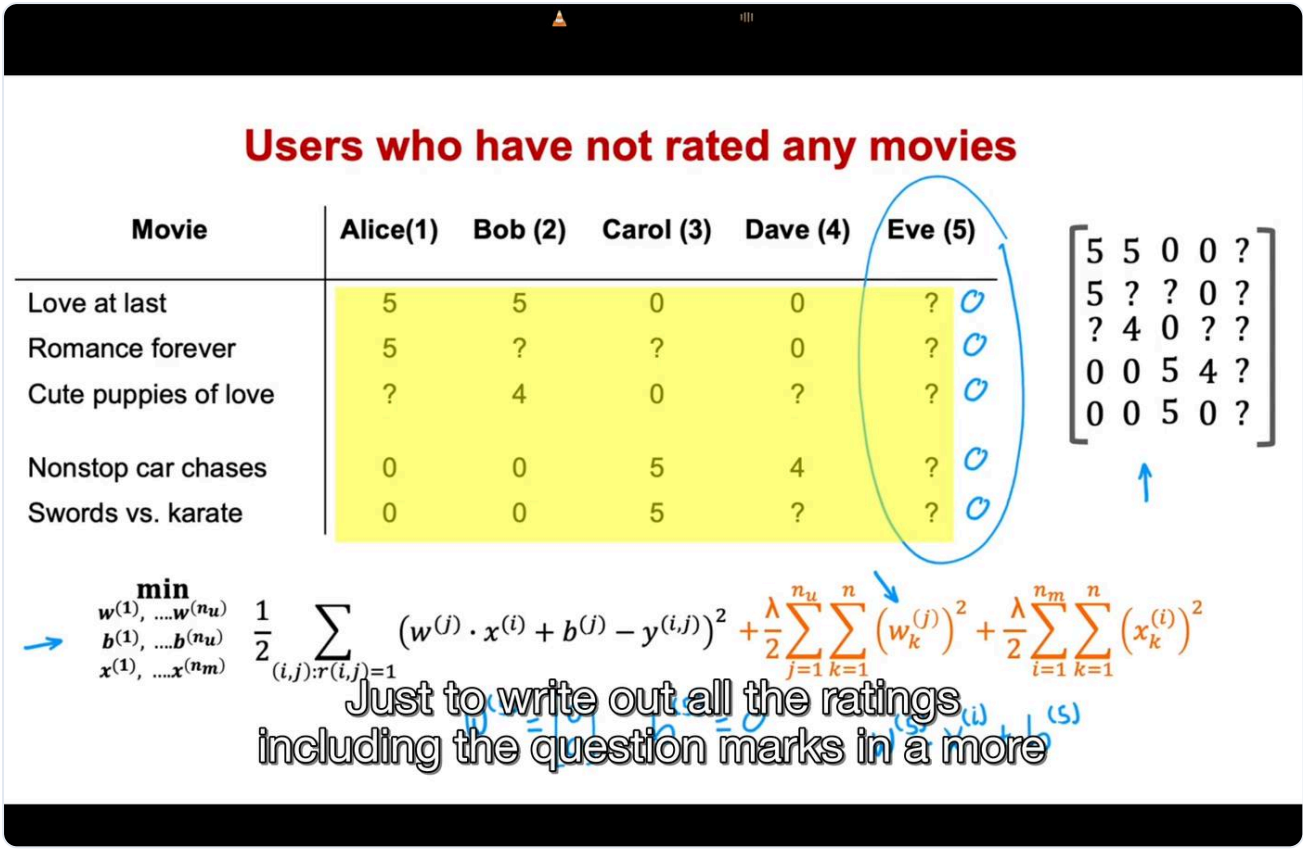


FIGURE 18.4 | Users Who Have Not Rated Any Movies

To carry out mean normalization, what we're going to do is take all of these ratings and for each movie, compute the average rating that was given. So movie one had two 5s and two 0s and so the average rating is 2.5. Movie two had a 5 and a 0, so that averages out to 2.5. Movie three 4 and 0 averages out to 2. Movie four averages out to 2.25 rating.

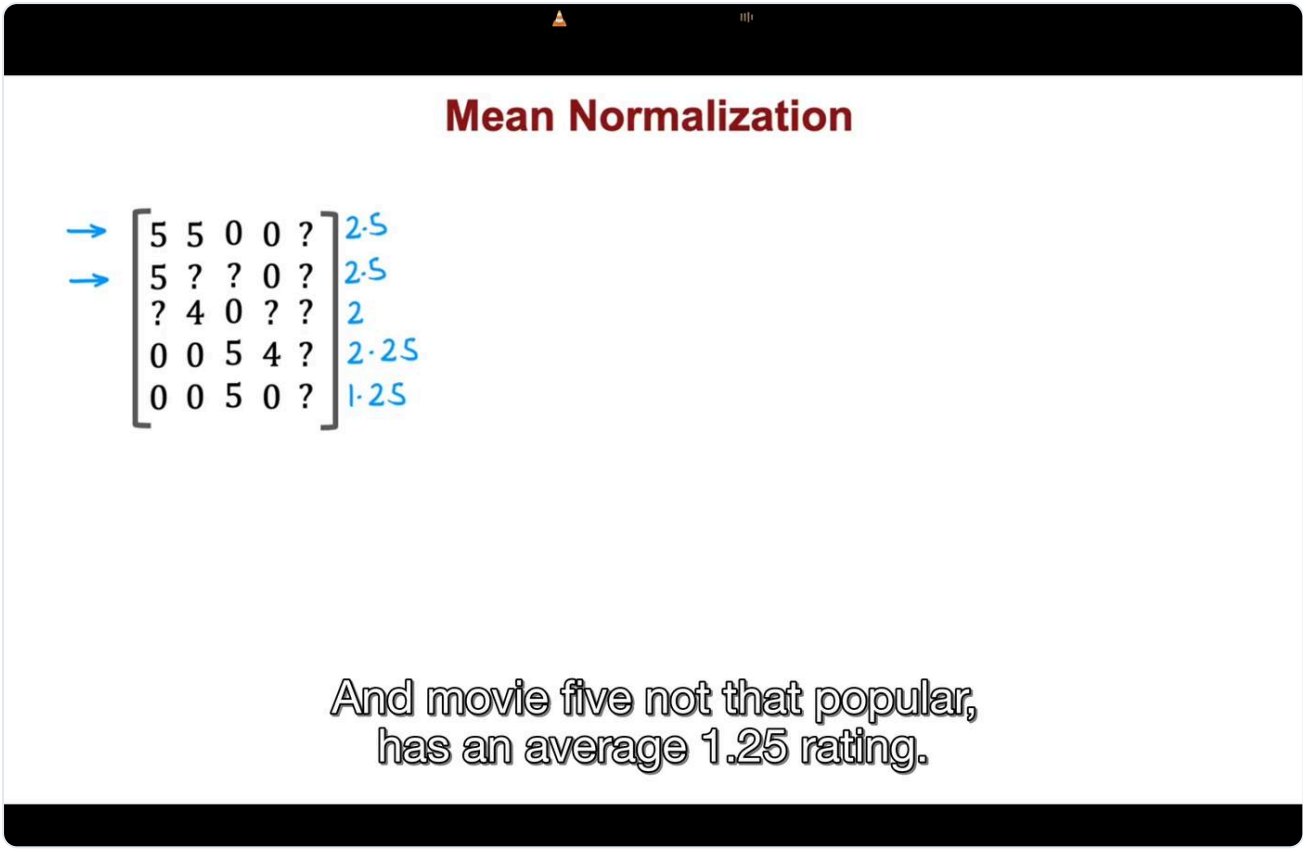


FIGURE 18.5 | Mean Normalization

And movie five not that popular, has an average 1.25 rating.

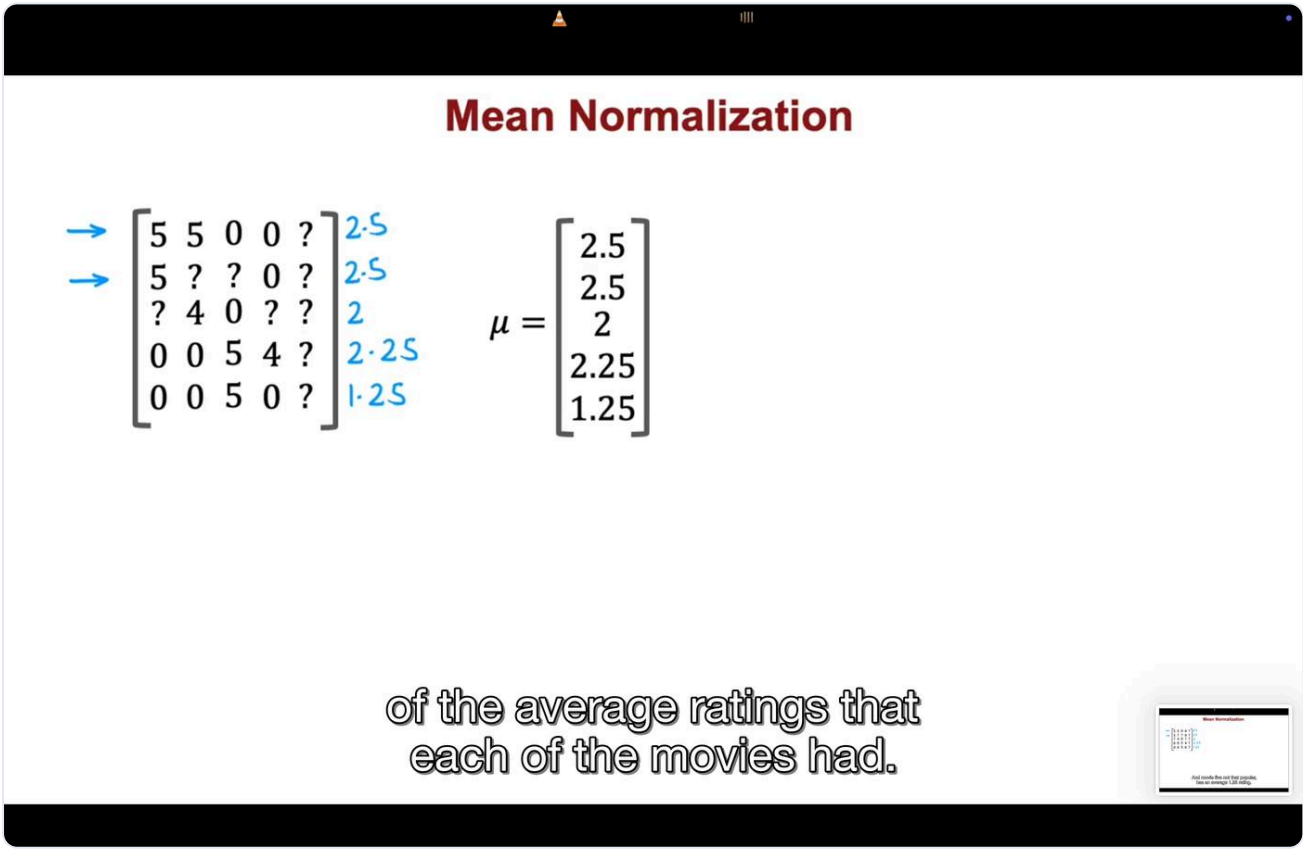


FIGURE 18.6 | Mean Normalization

So I'm going to take all of these five numbers and gather them into a vector which I'm going to call μ because this is the vector of the average ratings that each of the movies had. Averaging over just the users that did read that particular movie. Instead of using these original 0 to 5 star ratings over here, I'm going to take this and subtract from every rating the mean rating that it was given.

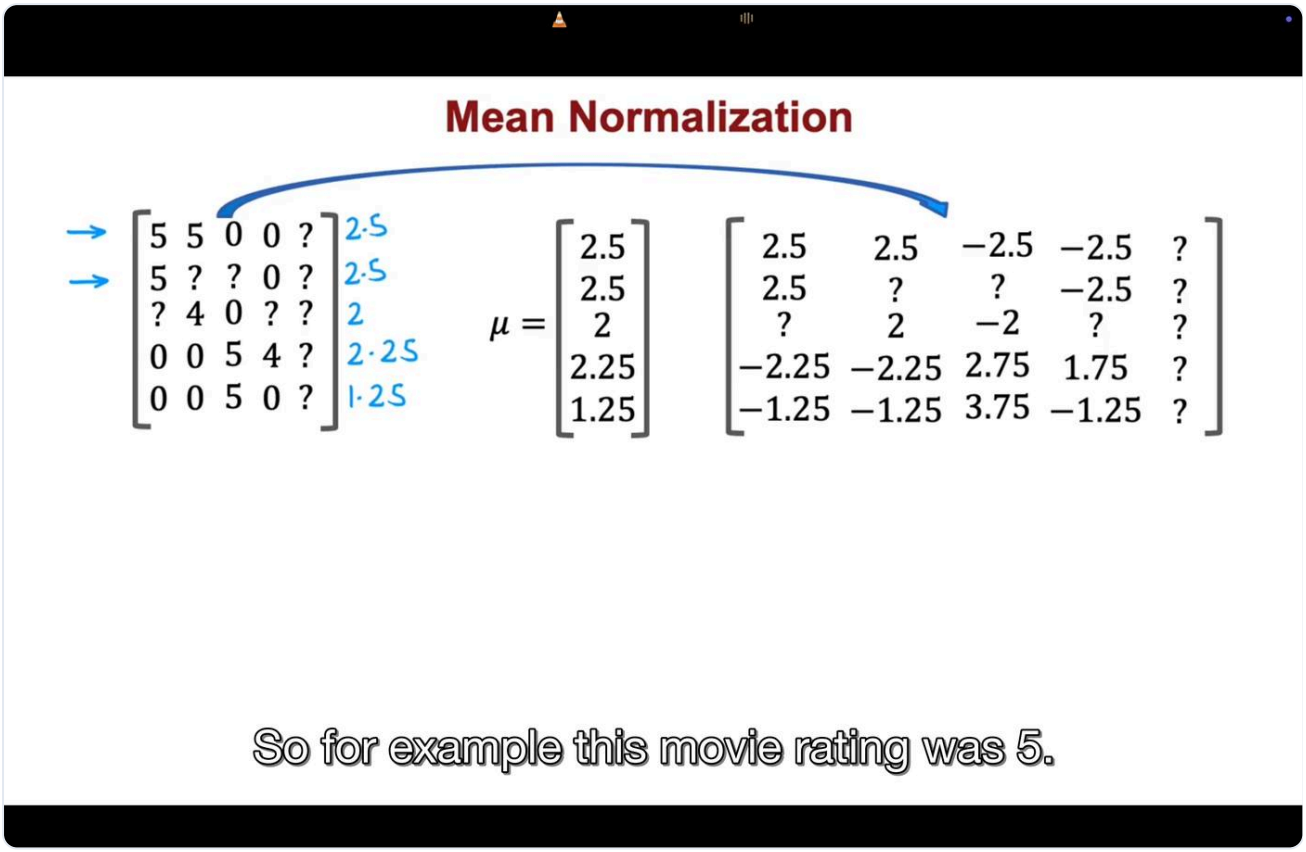


FIGURE 18.7 | Mean Normalization

So for example this movie rating was 5. I'm going to subtract 2.5 giving me 2.5 over here. This movie had a 0 star rating. I'm going to subtract 2.25 giving me a -2.25 rating and so on for all of the now five users including the new user Eve as well as for all five movies. Then these new values on the right become your new values of Y(i,j). We're going to pretend that user 1 had given a 2.5 rating to movie one and the -2.25 rating to movie four. And using this, you can then learn w(j), b(j) and x(i) same as before for user j on movie i, you would predict w(j).x(i) + b(j).

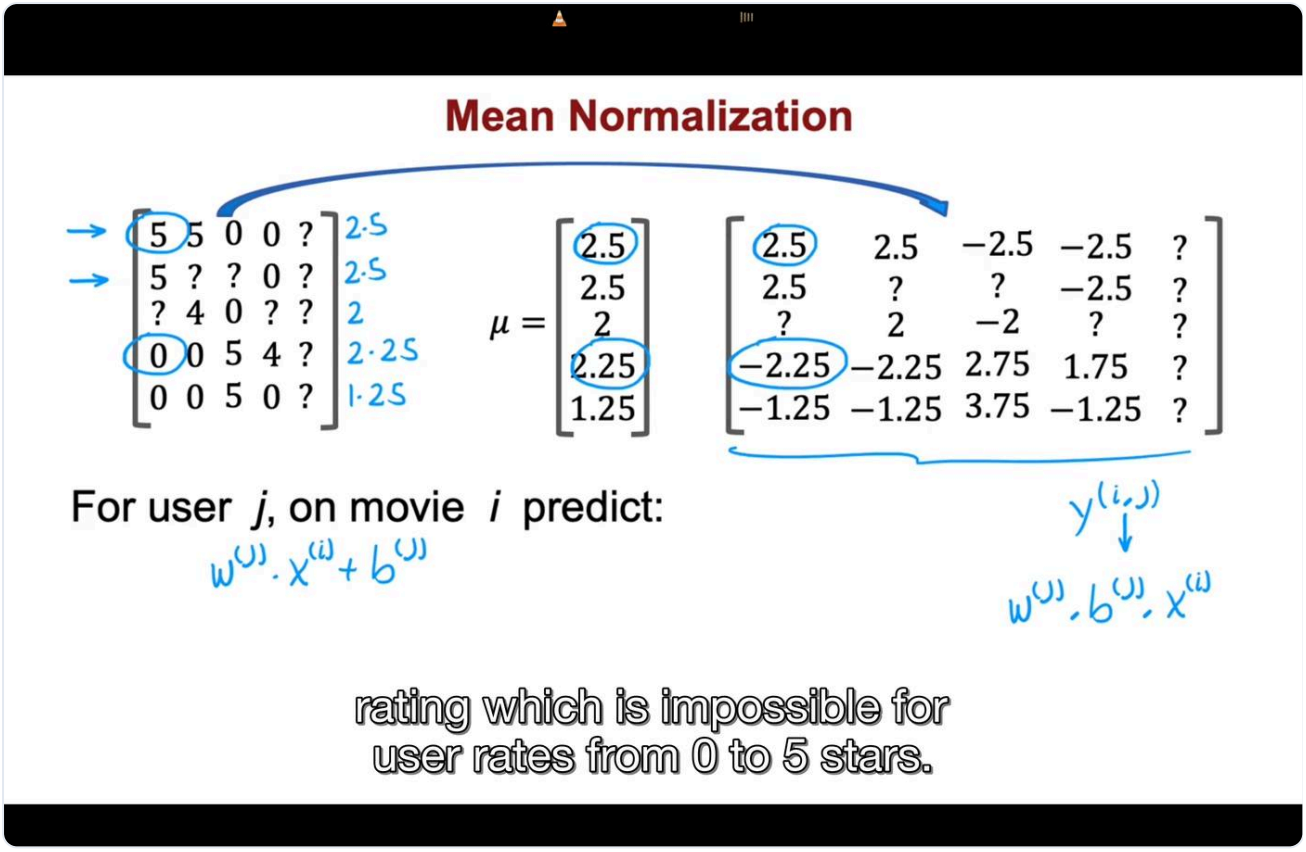


FIGURE 18.8 | Mean Normalization

But because we had subtracted off μi for movie i during this mean normalization step, in order to predict not a negative star rating which is impossible for user rates from 0 to 5 stars. We have to add back this μi which is just the value we have subtracted out.

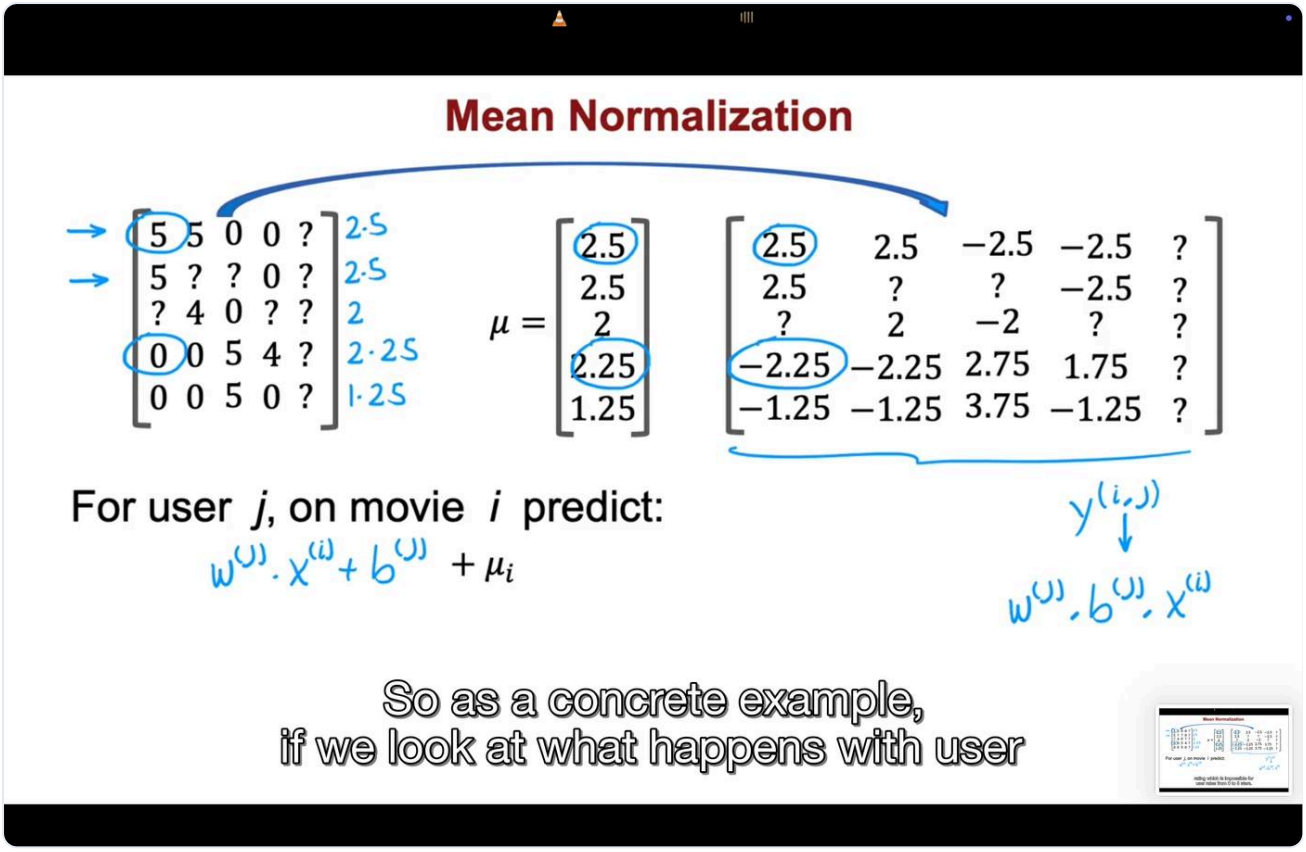


FIGURE 18.9 | Mean Normalization

So as a concrete example, if we look at what happens with user 5 with the new user Eve because she had not yet rated any movies, the average might learn parameters w(5) = [0 0] and say b(5) = 0.

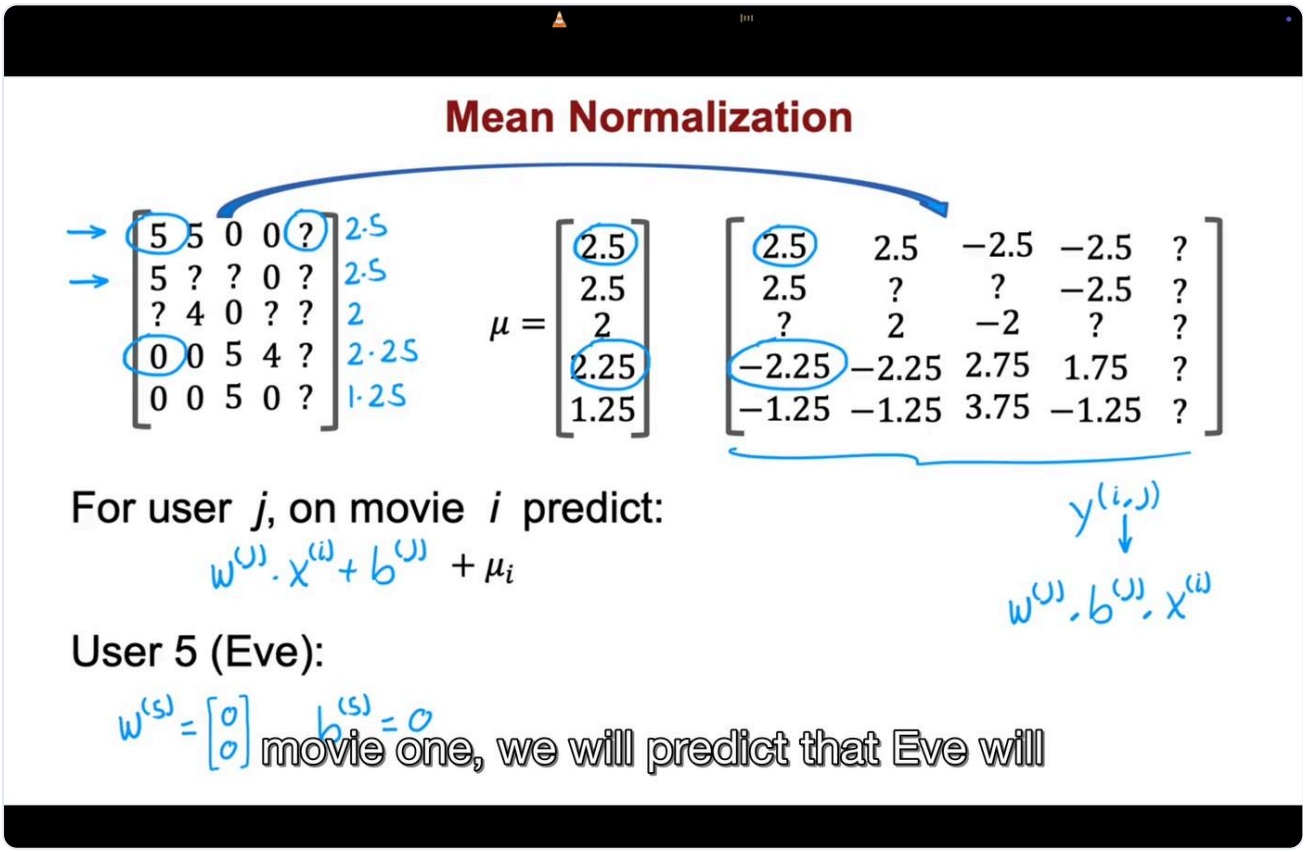


FIGURE 18.10 | Mean Normalization

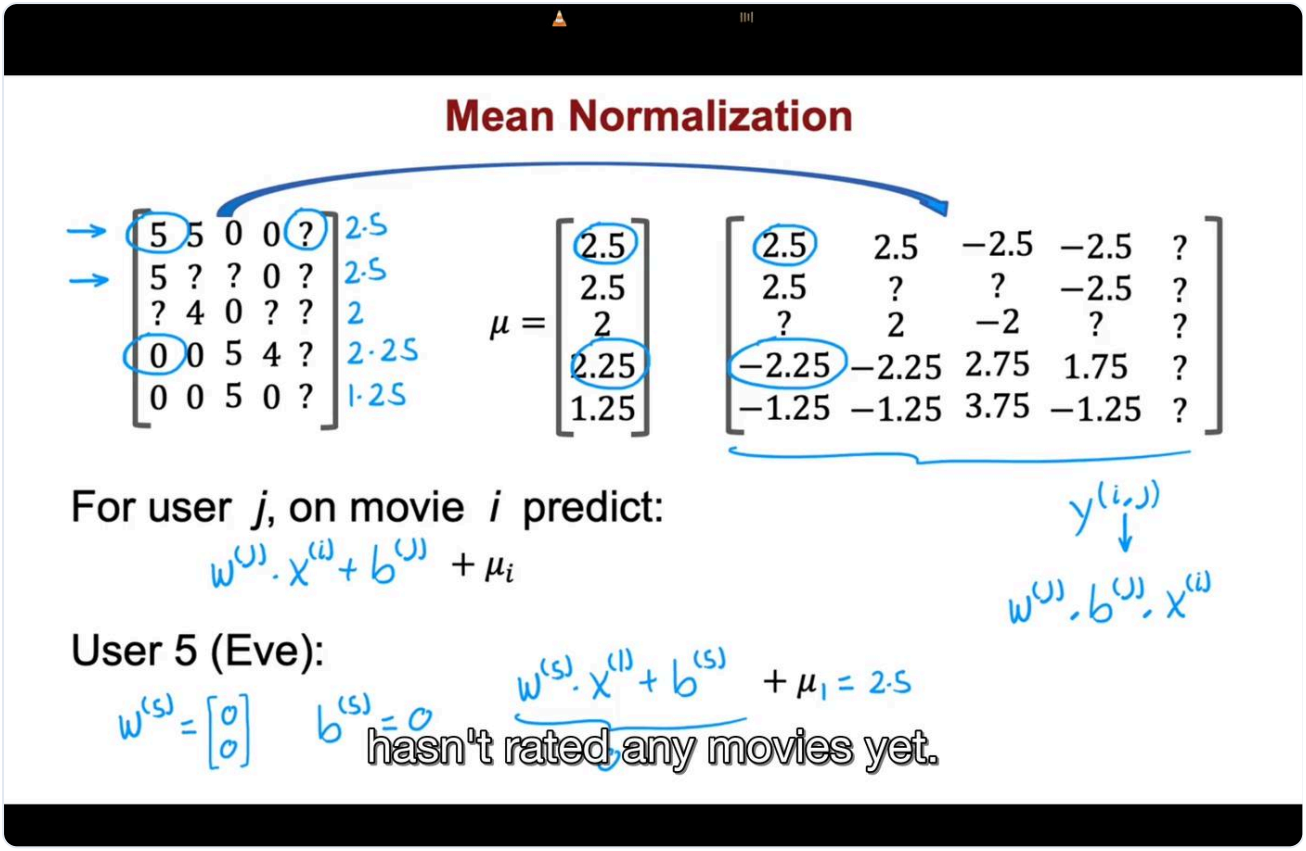


FIGURE 18.11 | Mean Normalization

And so if we look at the predicted rating for movie one, we will predict that Eve will rate it w(5).x1 + b(5) but this is 0 and then + μ1 which is equal to 2.5. So this seems more reasonable to think Eve is likely to rate this movie 2.5 rather than think Eve will rate all movie zero stars just because she hasn't rated any movies yet. And in fact the effect of this algorithm is it will cause the initial guesses for the new user Eve to be just equal to the mean of whatever other users have rated these five movies. And that seems more reasonable to take the average rating of the movies rather than to guess that all the ratings by Eve will be zero. It turns out that by normalizing the mean of the different movies ratings to be zero, the optimization algorithm for the recommender system will also run just a little bit faster.

But it does make the algorithm behave much better for users who have rated no movies or very small numbers of movies. And the predictions will become more reasonable. In this example, what we did was normalize each of the rows of this matrix to have zero mean and we saw this helps when there's a new user that hasn't rated a lot of movies yet. There's one other alternative that you could use which is to instead normalize the columns of this matrix to have zero mean. And that would be a reasonable thing to do too. But I think in this application, normalizing the rows so that you can give reasonable ratings for a new user seems more important than normalizing the columns.

Normalizing the columns would hope if there was a brand new movie that no one has rated yet. But if there's a brand new movie that no one has rated yet, you probably shouldn't show that movie to too many users initially because you don't know that much about that movie. So normalizing columns the hope with the case of a movie with no ratings seems less important to me than normalizing the rules to hope with the case of a new user that's hardly rated any movies

yet. And when you are building your own recommended system in this week's practice lab, normalizing just the roles should work fine. So that's mean normalization. It makes the algorithm run a little bit faster.

But even more important, it makes the algorithm give much better, much more reasonable predictions when there are users that rated very few movies or even no movies at all. This implementation detail of mean normalization will make your recommended system work much better. Next, let's go into the next video to talk about how you can implement this for yourself in TensorFlow.

LECTURE 19

Derivatives in ML

In this video, we'll take a look at how you can use TensorFlow to implement the collaborative filtering algorithm. You might be used to thinking of TensorFlow as a tool for building neural networks. And it is. It's a great tool for building neural networks. And it turns out that TensorFlow can also be very helpful for building other types of learning algorithms as well. Like the collaborative filtering algorithm. One of the reasons I like using TensorFlow for talks like these is that for many applications in order to implement gradient descent, you need to find the derivatives of the cost function, but TensorFlow can automatically figure out for you what are the derivatives of the cost function.

All you have to do is implement the cost function and without needing to know any calculus, without needing to take derivatives yourself, you can get TensorFlow with just a few lines of code to compute that derivative term, that can be used to optimize the cost function. Let's take a look at how all this works. You might remember this diagram here on the right from course one. This is exactly the diagram that we had looked at when we talked about optimizing w. When we were working through our first linear regression example.

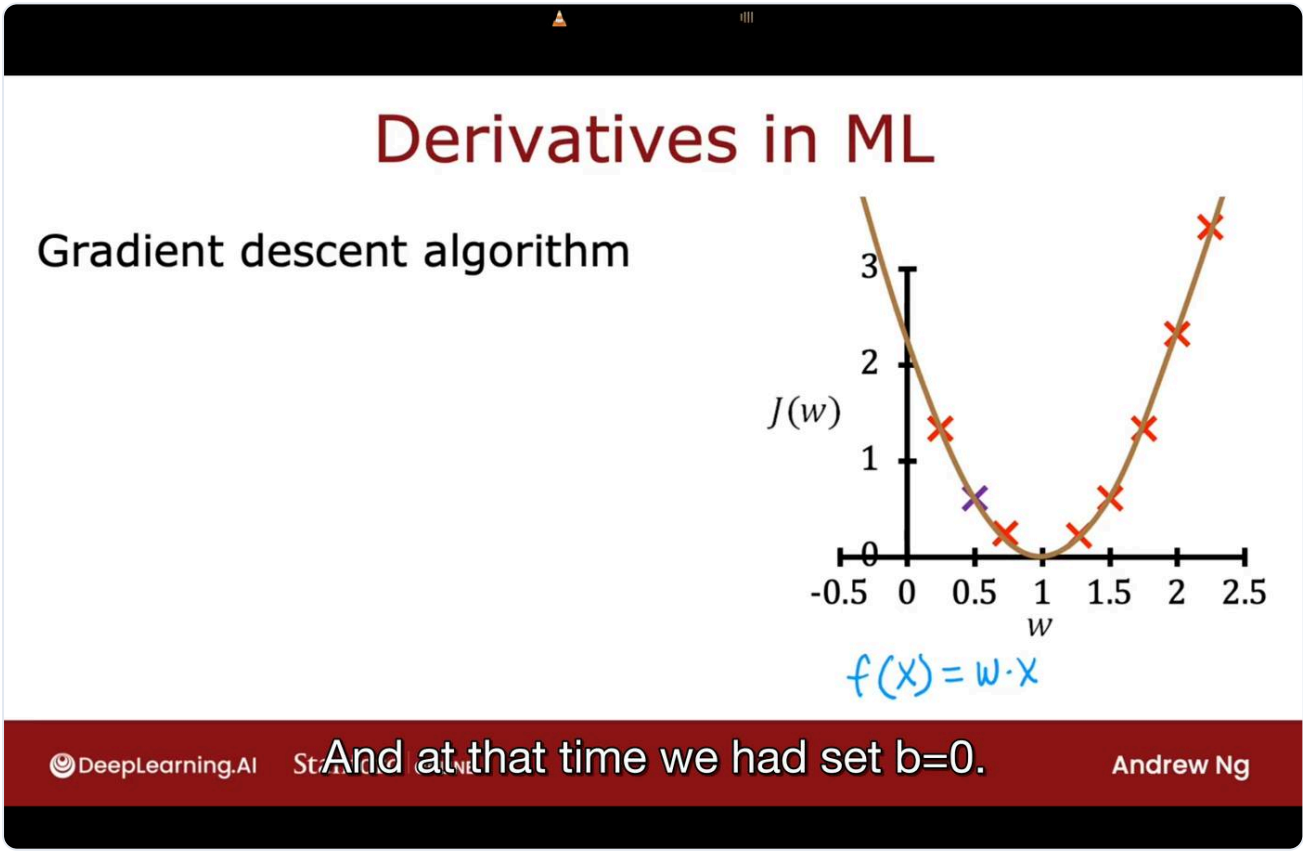


FIGURE 19.1 | Derivatives in ML

And at that time we had set b=0.

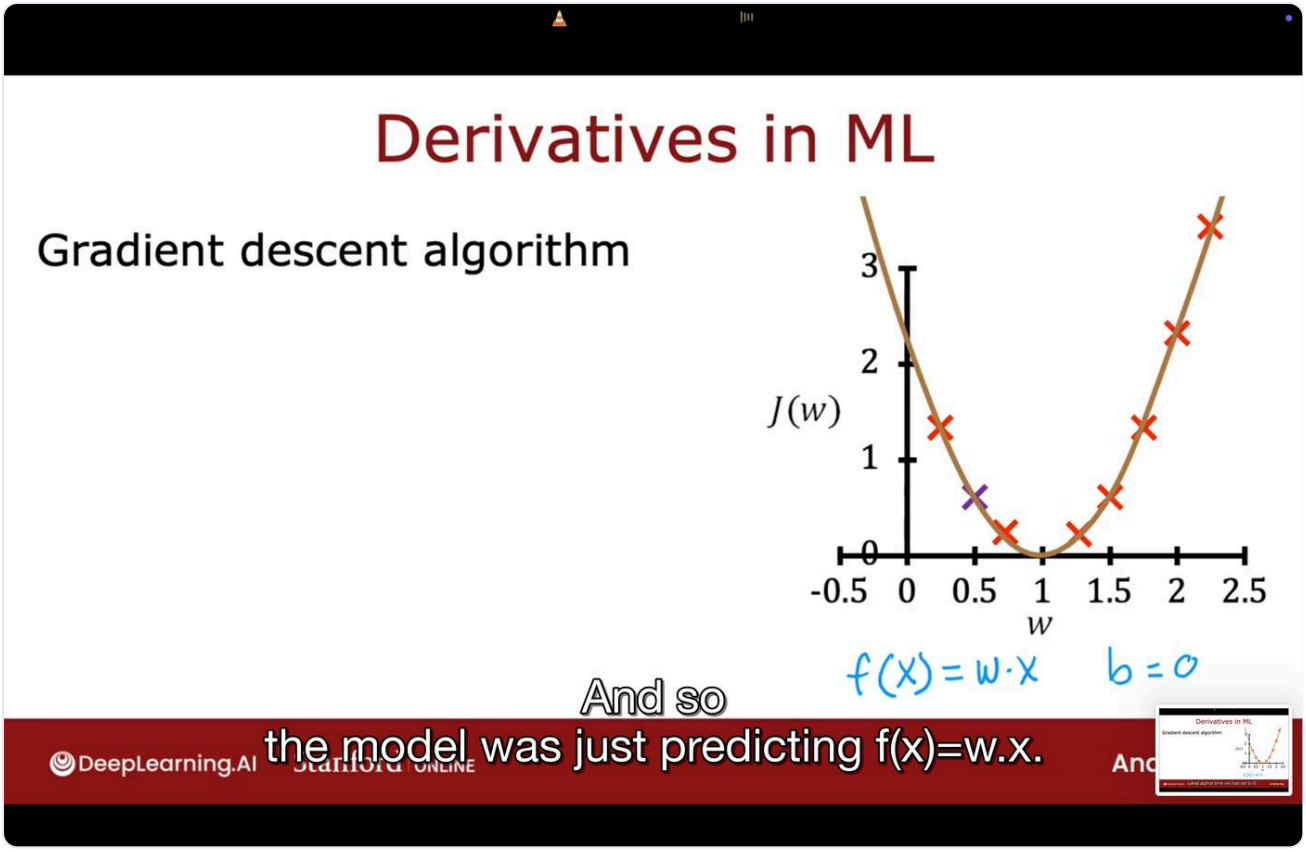


FIGURE 19.2 | Derivatives in ML

And so the model was just predicting f(x)=w.x. And we wanted to find the value of w that minimizes the cost function J. So the way we were doing that was via a gradient descent update, which looked like this, where w gets repeatedly updated as w minus the learning rate alpha times the derivative term. If you are updating b as well, this is the expression you will use. But if you said b=0, you just forgo the second update and you keep on performing this gradient descent update until convergence. Sometimes computing this derivative or partial derivative term can be difficult. And it turns out that TensorFlow can help with that. Let's see how.

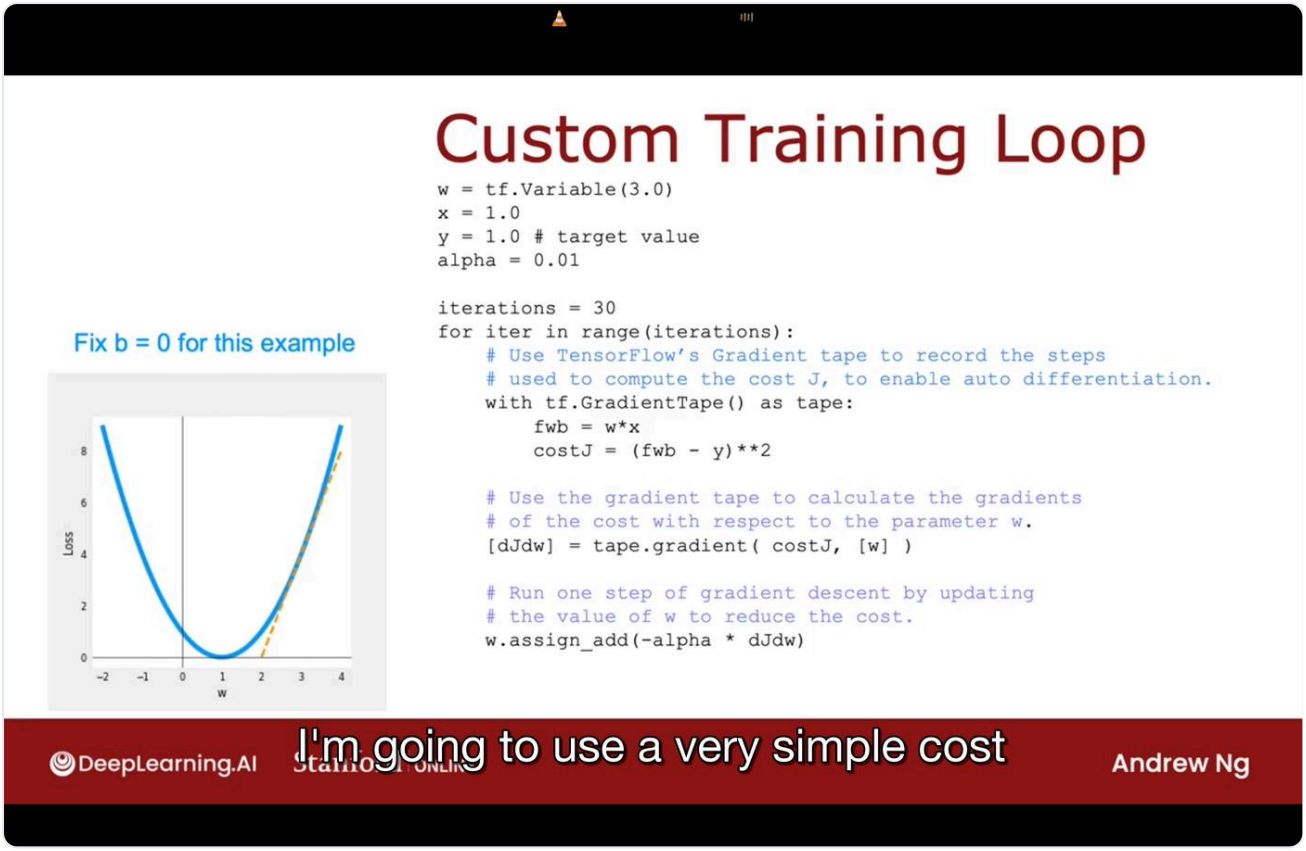


FIGURE 19.3

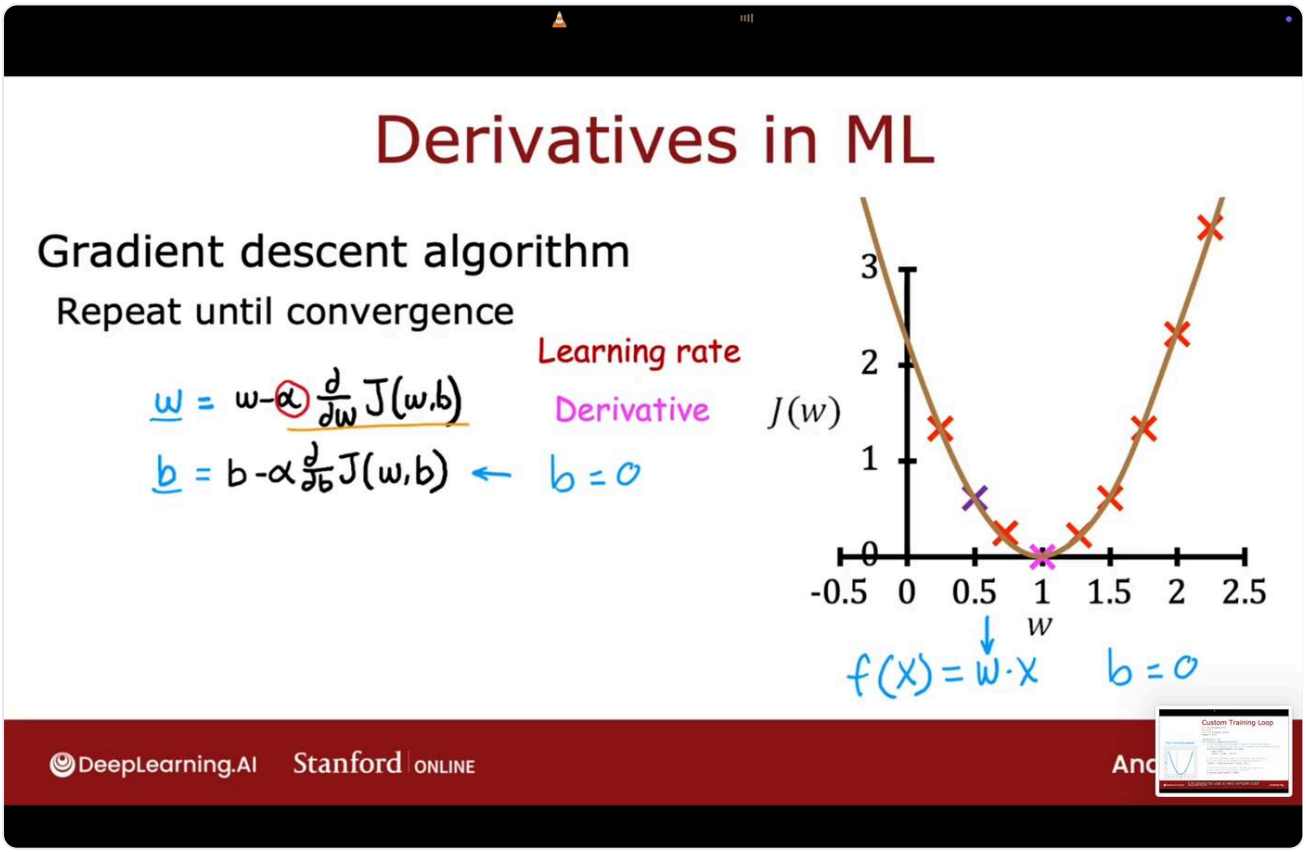


FIGURE 19.4 | Derivatives in ML

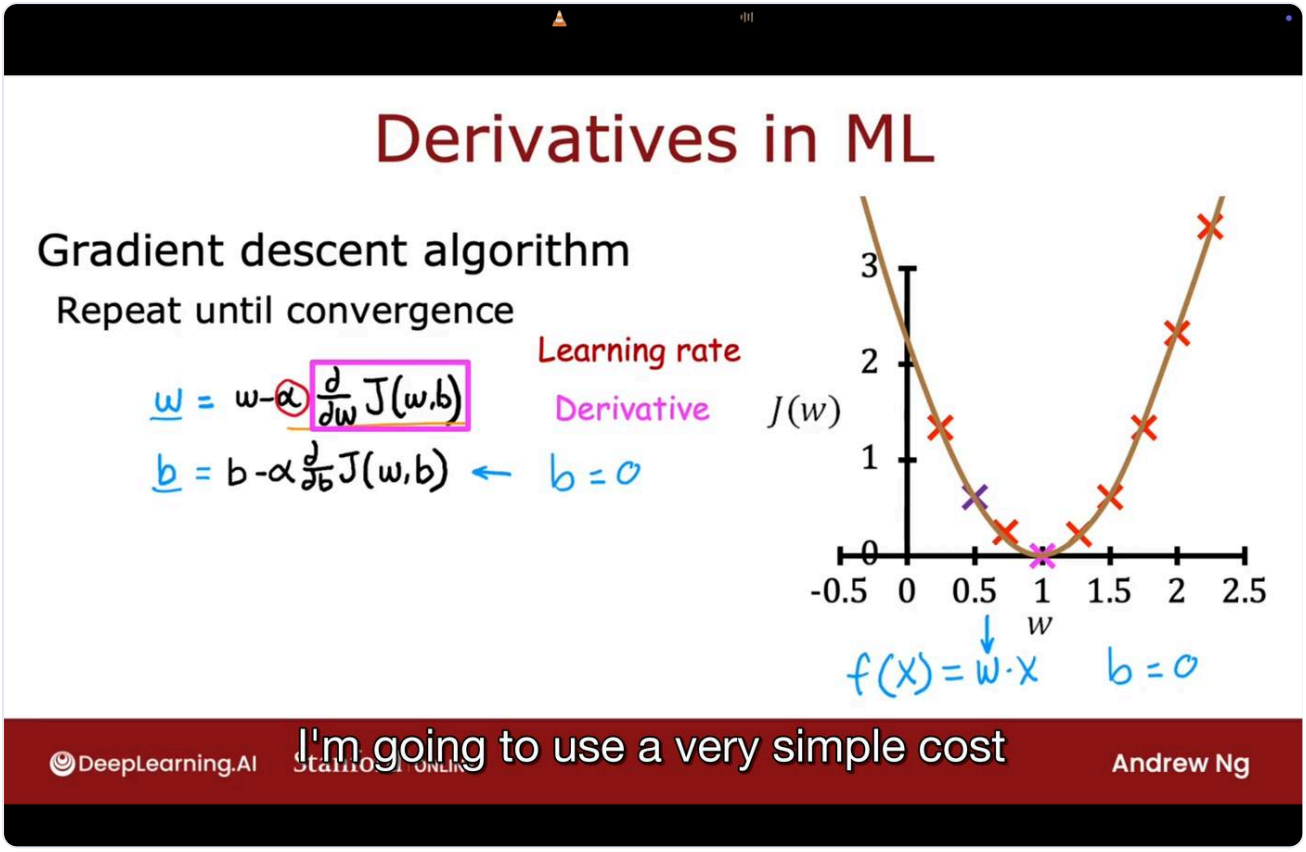


FIGURE 19.5 | Derivatives in ML

I'm going to use a very simple cost function J=(wx-1) squared. So wx is our simplified f w of x and y is equal to 1. And so this would be the cost function if we had f(x) equals wx,y equals 1 for the one training example that we have, and if we were not optimizing this respect to b.

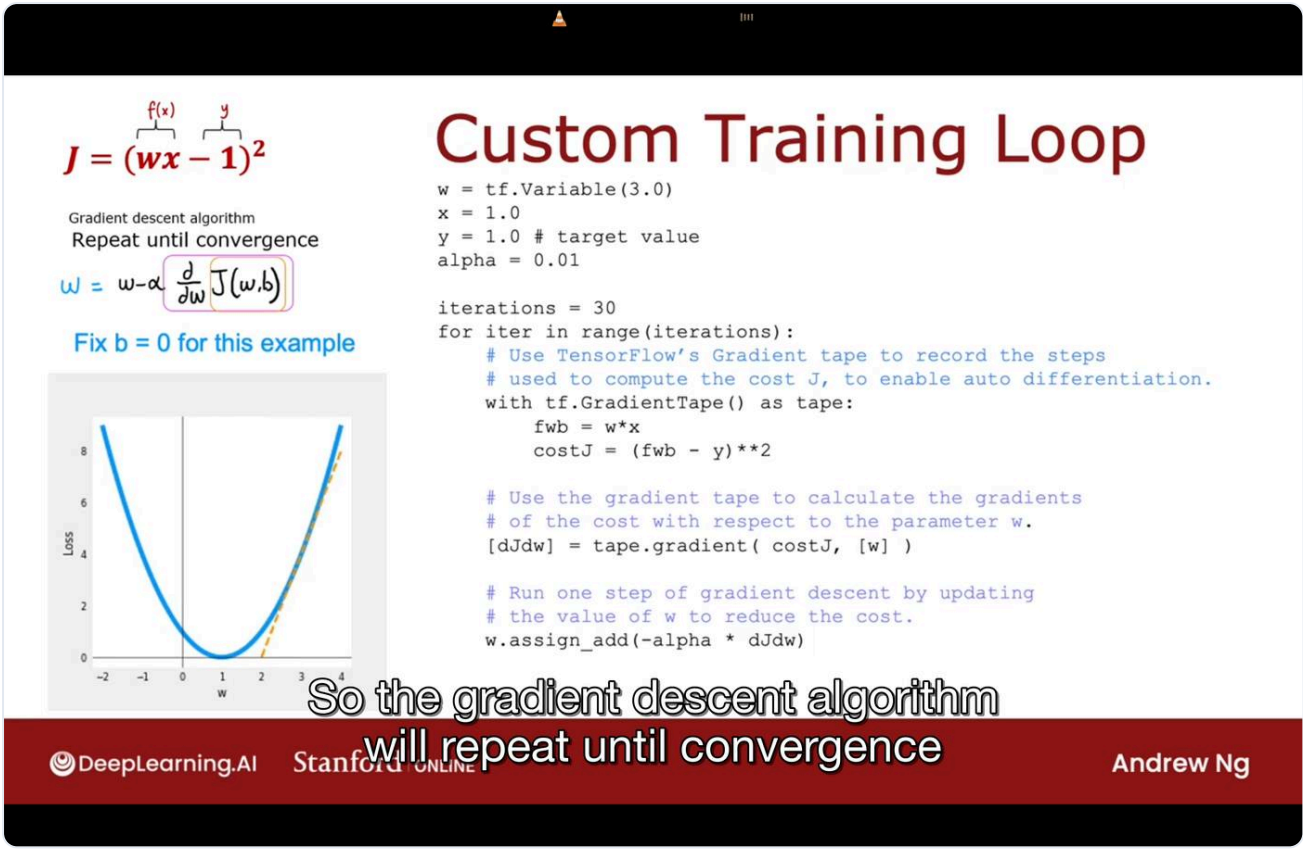


FIGURE 19.6

So the gradient descent algorithm will repeat until convergence this update over here. It turns out that if you implement the cost function J over here, TensorFlow can automatically compute for you this derivative term and thereby get gradient descent to work. I'll give you a high level overview of what this code does, w=tf.variable(3.0). Takes the parameter w and initializes it to the value of 3.0. Telling TensorFlow that w is a variable is how we tell it that w is a parameter that we want to optimize. I'm going to set x=1.0, y=1.0, and the learning rate alpha to be equal to 0.01. And let's run gradient descent for 30 iterations. So in this code will still do for iter in range iterations, so for 30 iterations.

And this is the syntax to get TensorFlow to automatically compute derivatives for you.

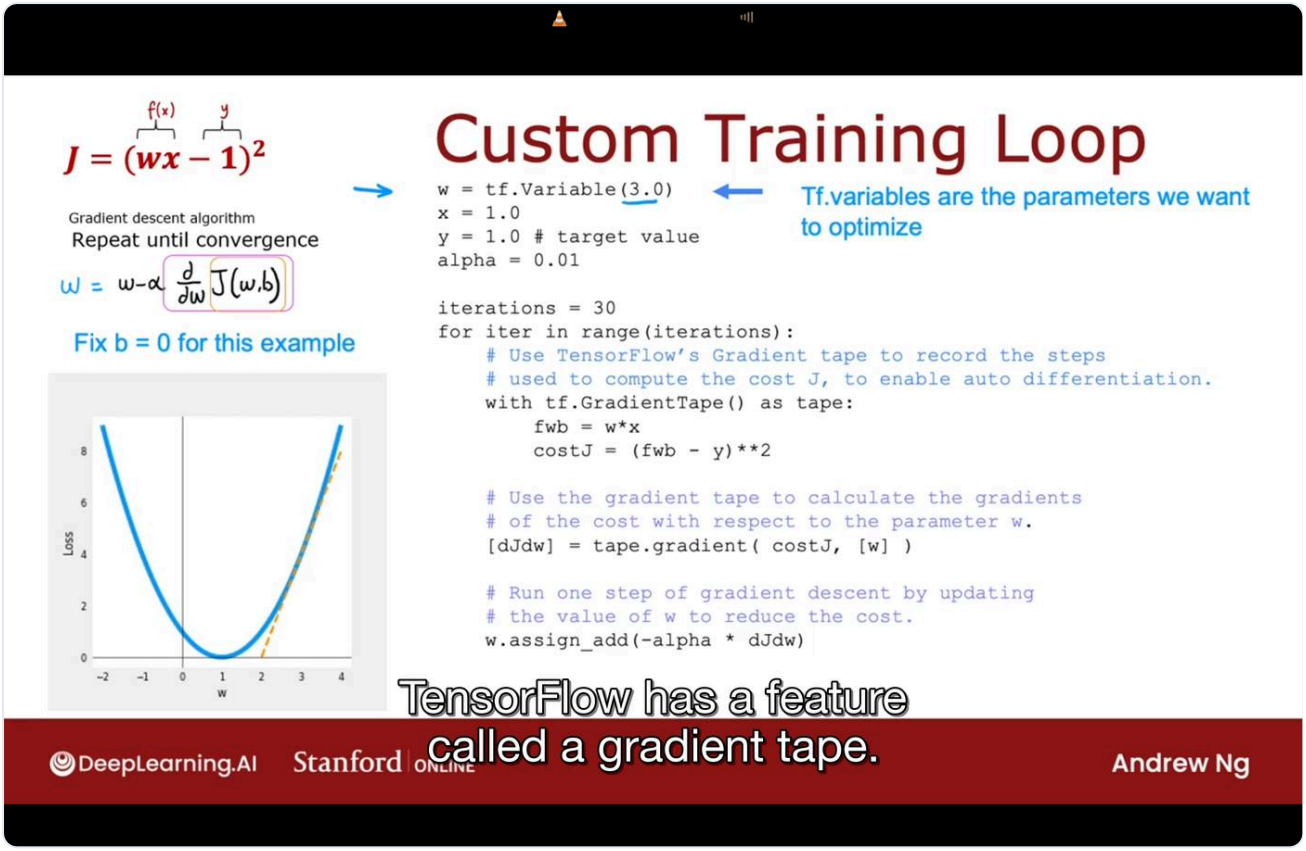


FIGURE 19.7

TensorFlow has a feature called a gradient tape. And if you write this with tf our gradient tape as tape f. This is compute f(x) as w*x and compute J as f(x)-y squared.

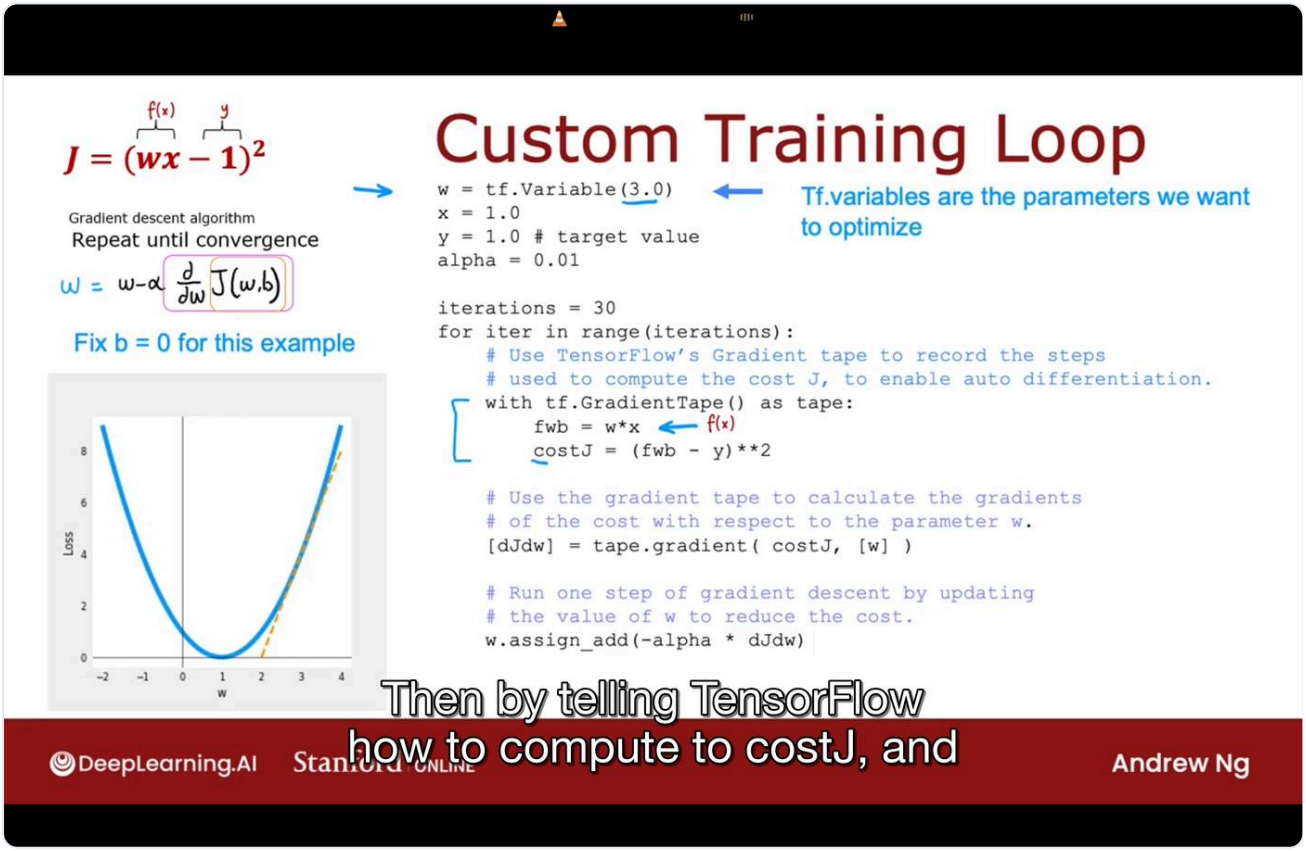


FIGURE 19.8

Then by telling TensorFlow how to compute to costJ, and by doing it with the gradient taped syntax as follows, TensorFlow will automatically record the sequence of steps. The sequence of operations needed to compute the costJ.

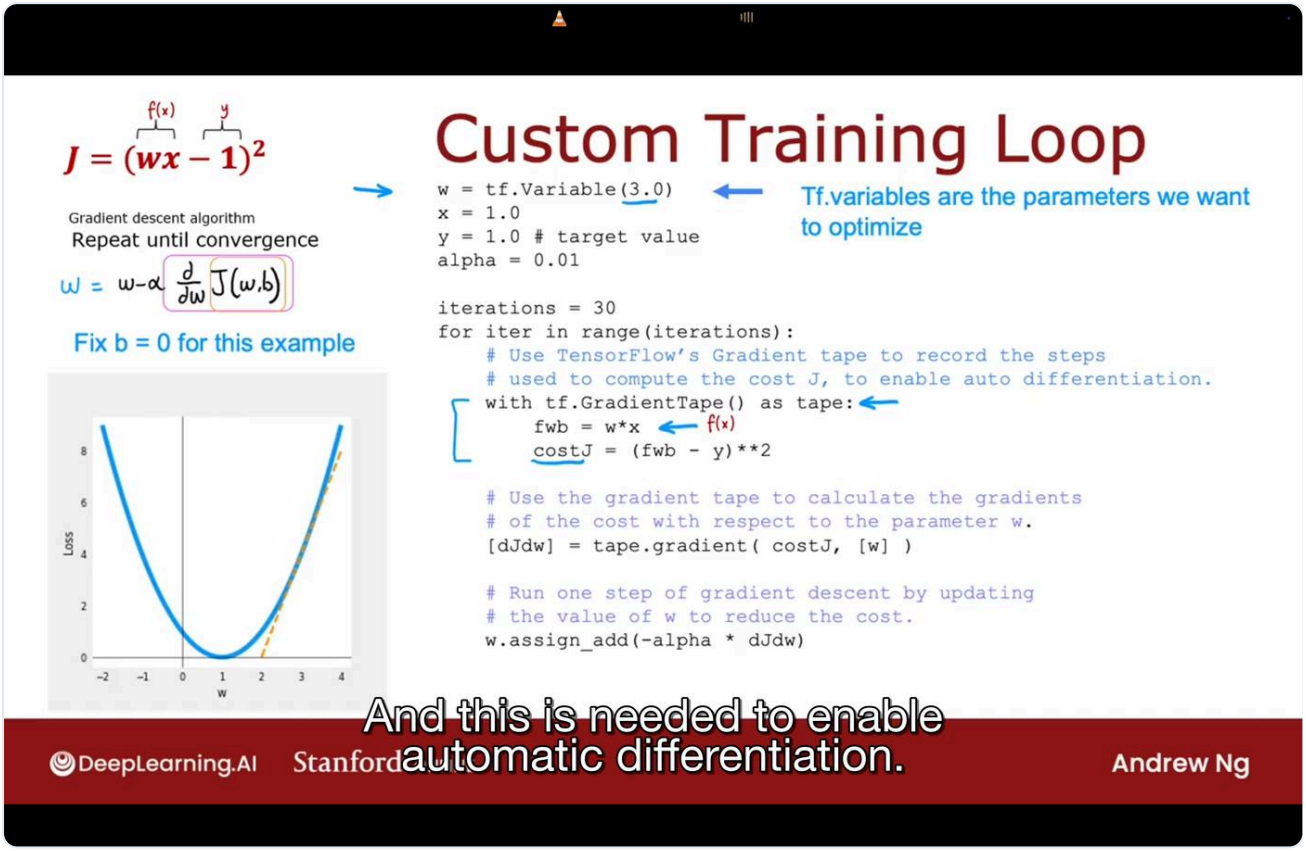


FIGURE 19.9

And this is needed to enable automatic differentiation. Next TensorFlow will have saved the sequence of operations in tape, in the gradient tape. And with this syntax, TensorFlow will automatically compute this derivative term, which I'm going to call dJdw.

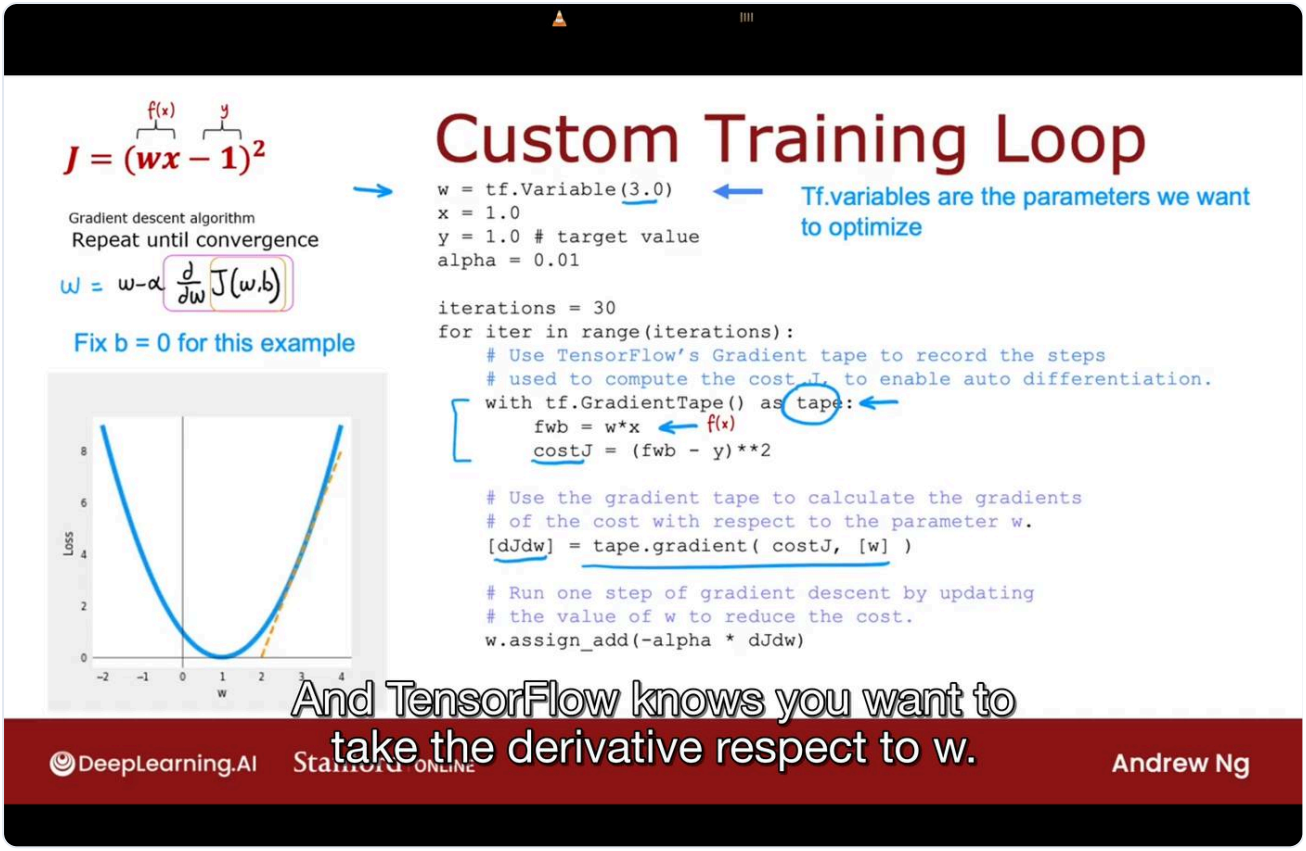


FIGURE 19.10

And TensorFlow knows you want to take the derivative respect to w. That w is the parameter you want to optimize because you had told it so up here. And because we're also specifying it down here.

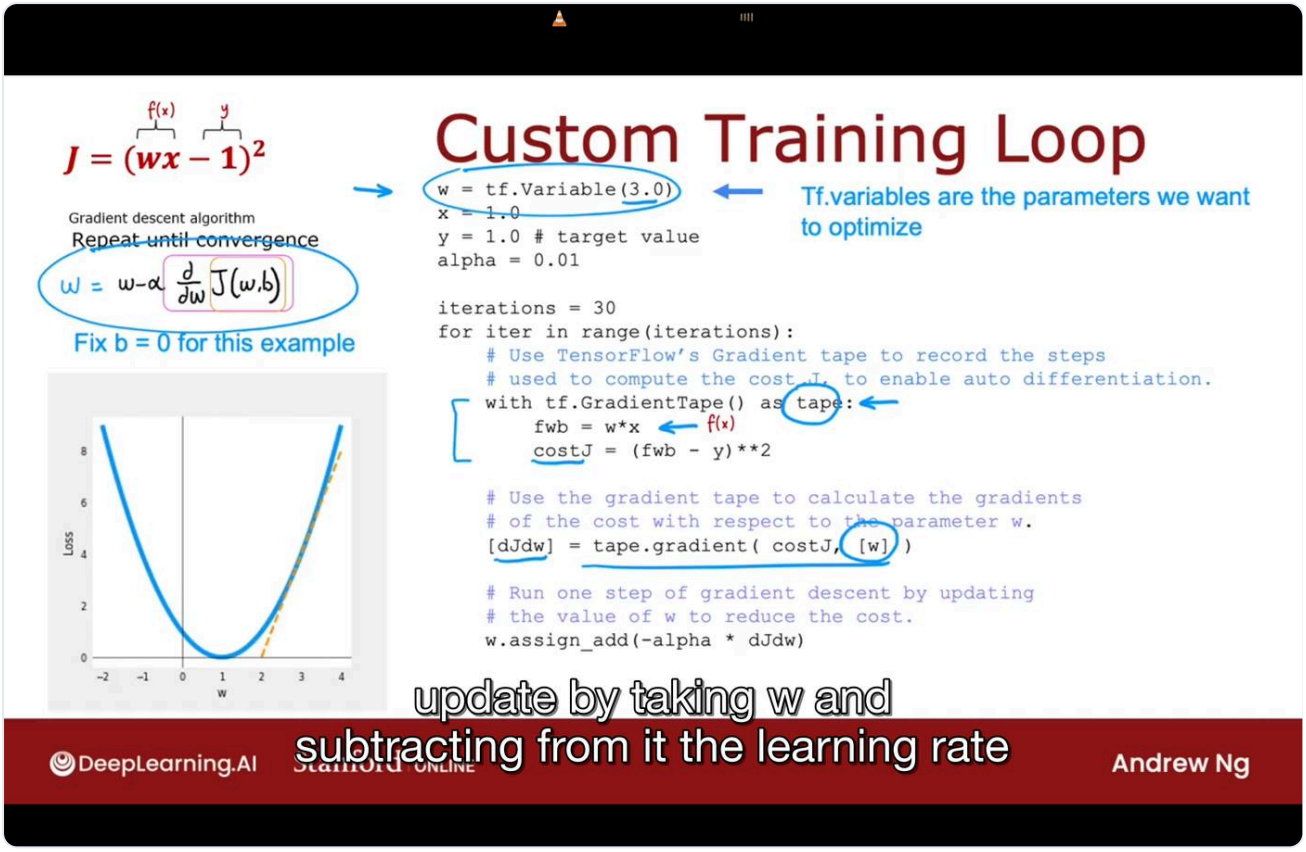


FIGURE 19.11 | Gradient Descent Algorithm X

So now you compute the derivatives, finally you can carry out this update by taking w and subtracting from it the learning rate alpha times that derivative term that we just got from up above. TensorFlow variables, tier variables requires special handling. Which is why instead of setting w to be w minus alpha times the derivative in the usual way, we use this assigned add function. But when you get to the practice lab, don't worry about it.

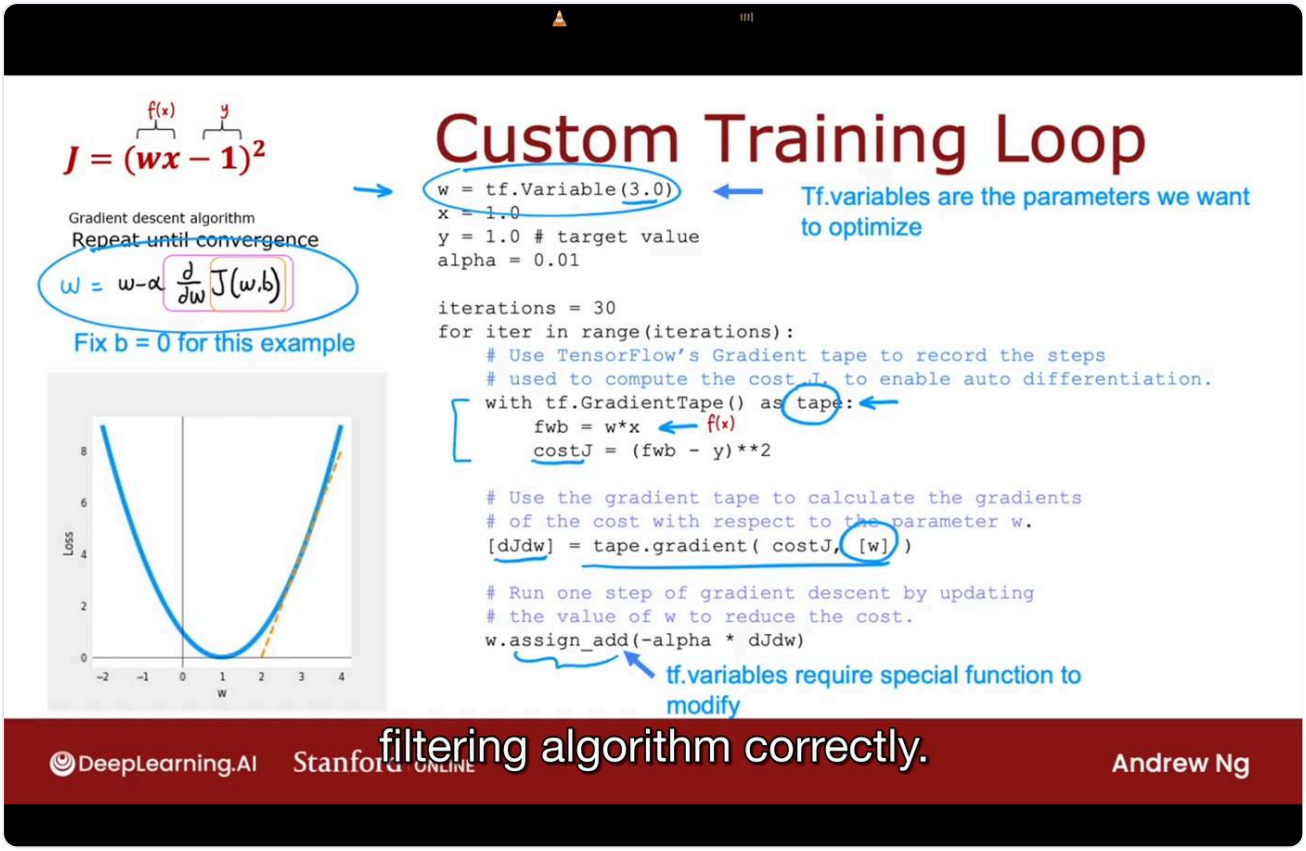


FIGURE 19.12

We'll give you all the syntax you need in order to implement the collaborative filtering algorithm correctly. So notice that with the gradient tape feature of TensorFlow, the main work you need to do is to tell it how to compute the cost function J. And the rest of the syntax causes TensorFlow to automatically figure out for you what is that derivative?

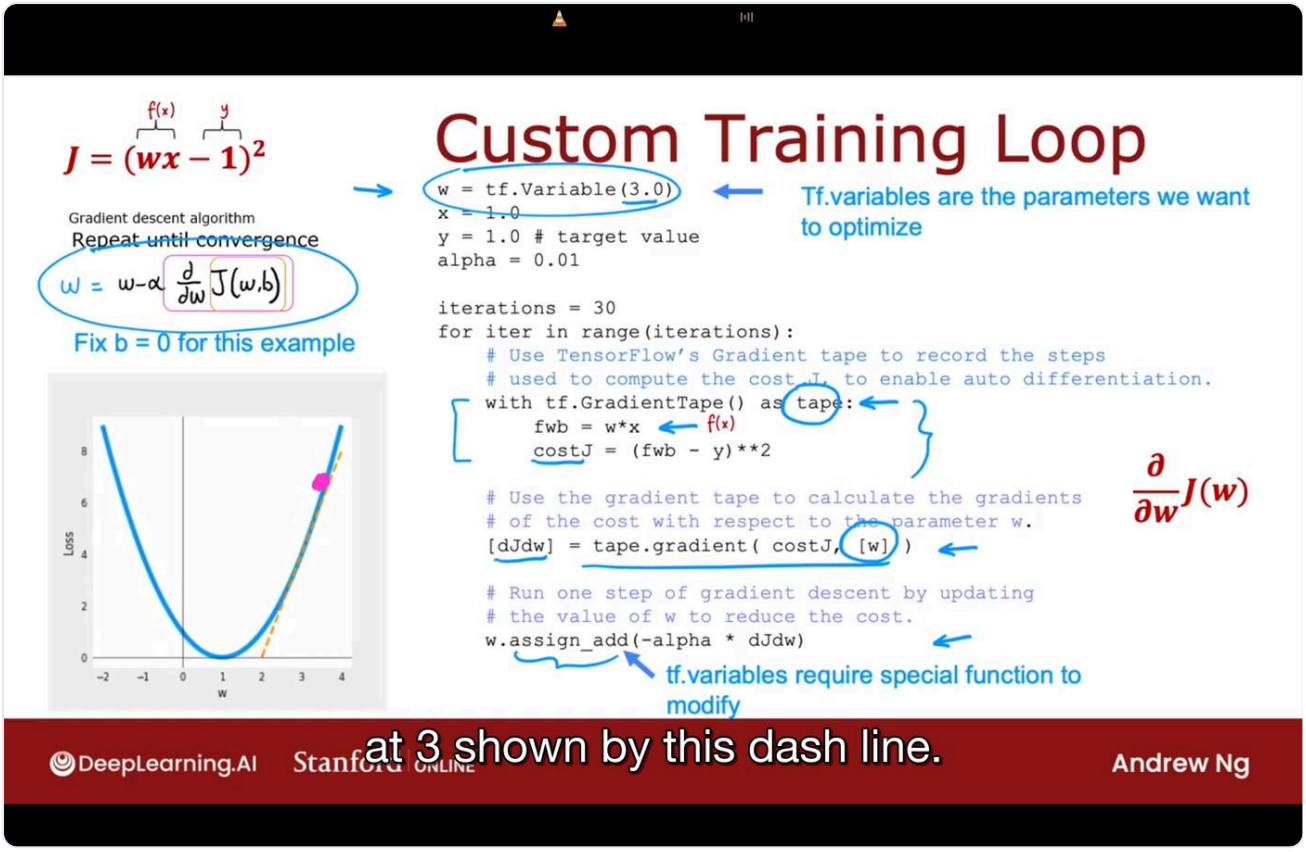


FIGURE 19.13

And with this TensorFlow we'll start with finding the slope of this, at 3 shown by this dash line. Take a gradient step and update w and compute the derivative again and update w over and over until eventually it gets to the optimal value of w, which is at w equals 1. So this procedure allows you to implement gradient descent without ever having to figure out yourself how to compute this derivative term. This is a very powerful feature of TensorFlow called Auto Diff.

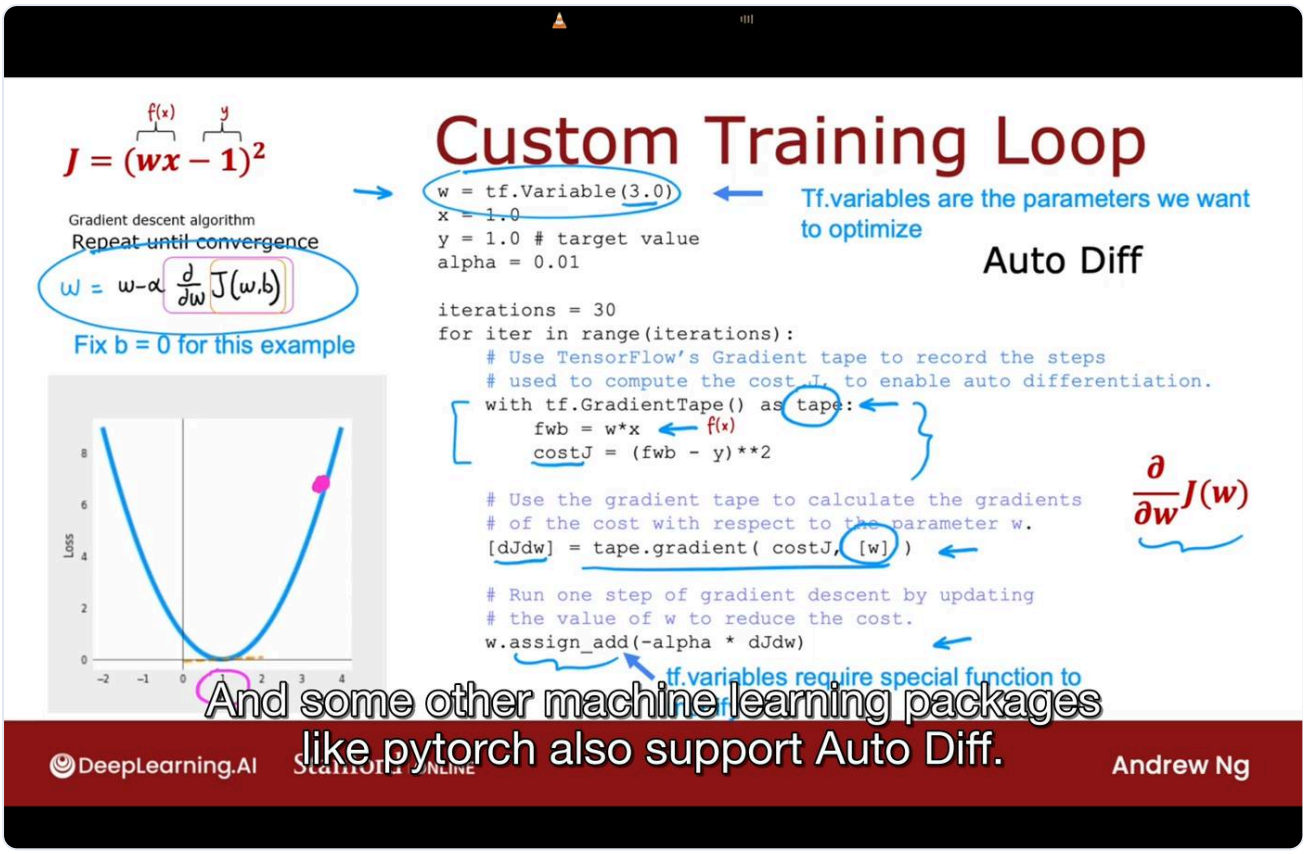


FIGURE 19.14 | Gradient Descent Algorithm

And some other machine learning packages like pytorch also support Auto Diff. Sometimes you hear people call this Auto Grad.

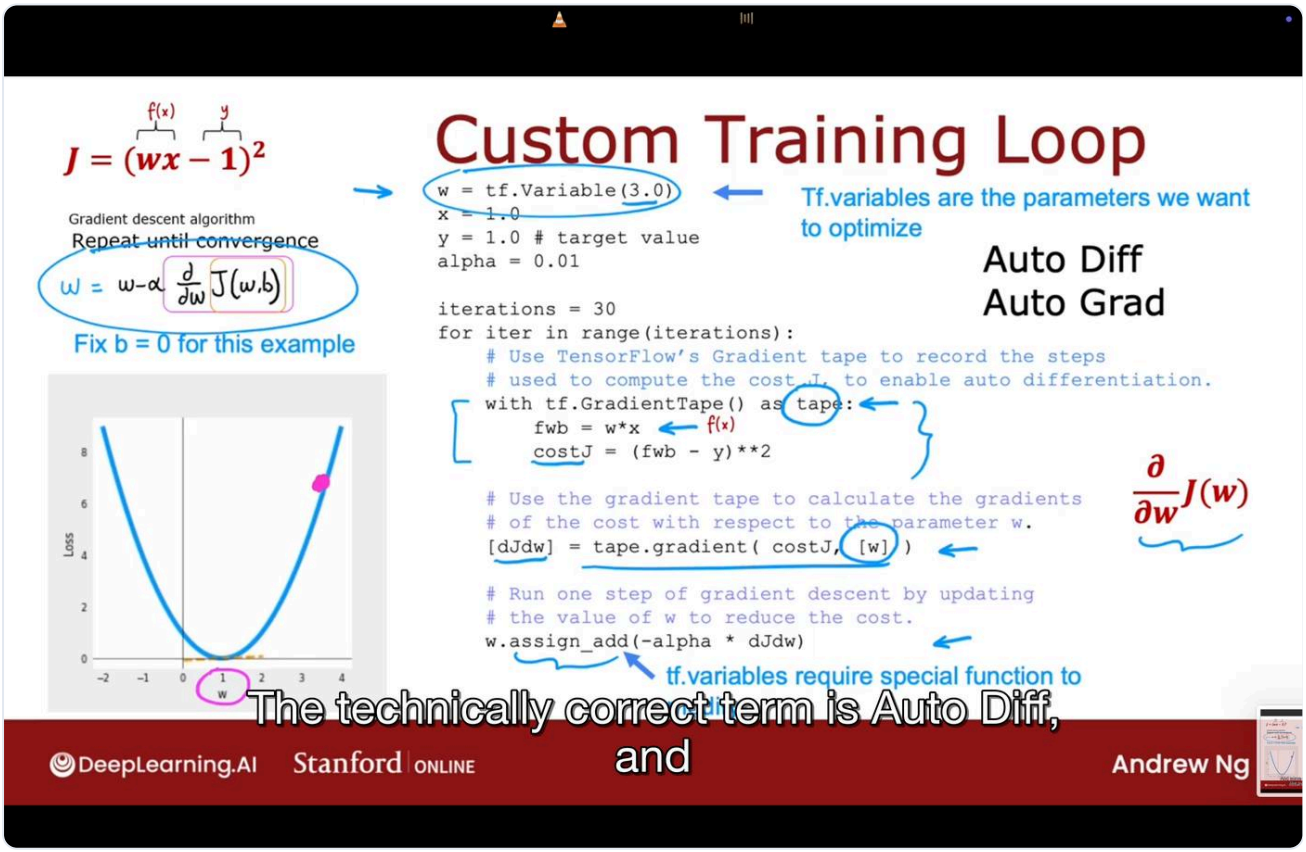


FIGURE 19.15 | Gradient Descent Algorithm

The technically correct term is Auto Diff, and Auto Grad is actually the name of the specific software package for doing automatic differentiation, for taking derivatives automatically. But sometimes if you hear someone refer to Auto Grad, they're just referring to this same concept of automatically taking derivatives.

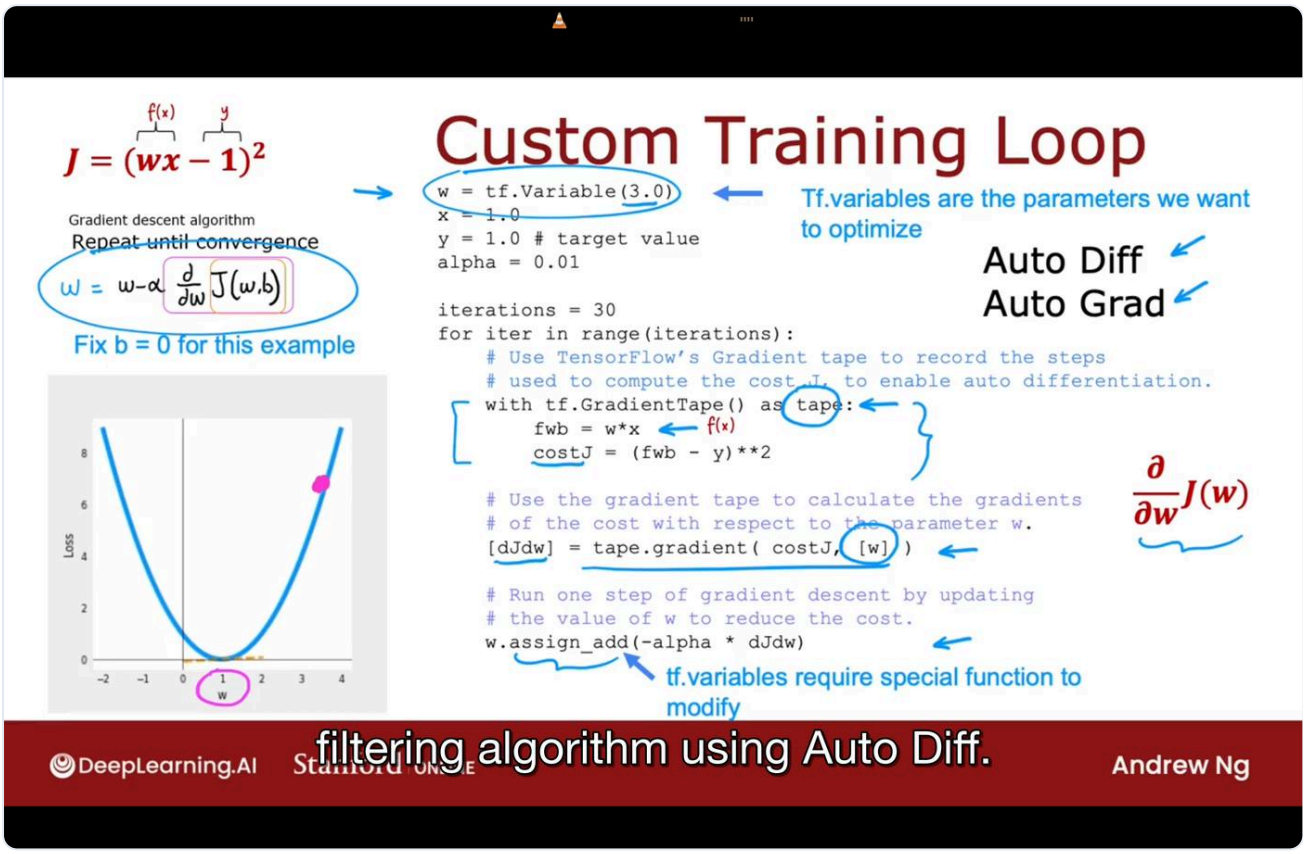


FIGURE 19.16 | Gradient Descent Algorithm

So let's take this and look at how you can implement to collaborative filtering algorithm using Auto Diff. And in fact, once you can compute derivatives automatically, you're not limited to just gradient descent. You can also use a more powerful optimization algorithm, like the adam optimization algorithm. In order to implement the collaborative filtering algorithm TensorFlow, this is the syntax you can use. Let's starts with specifying that the optimizer is keras optimizers adam with learning rate specified here.

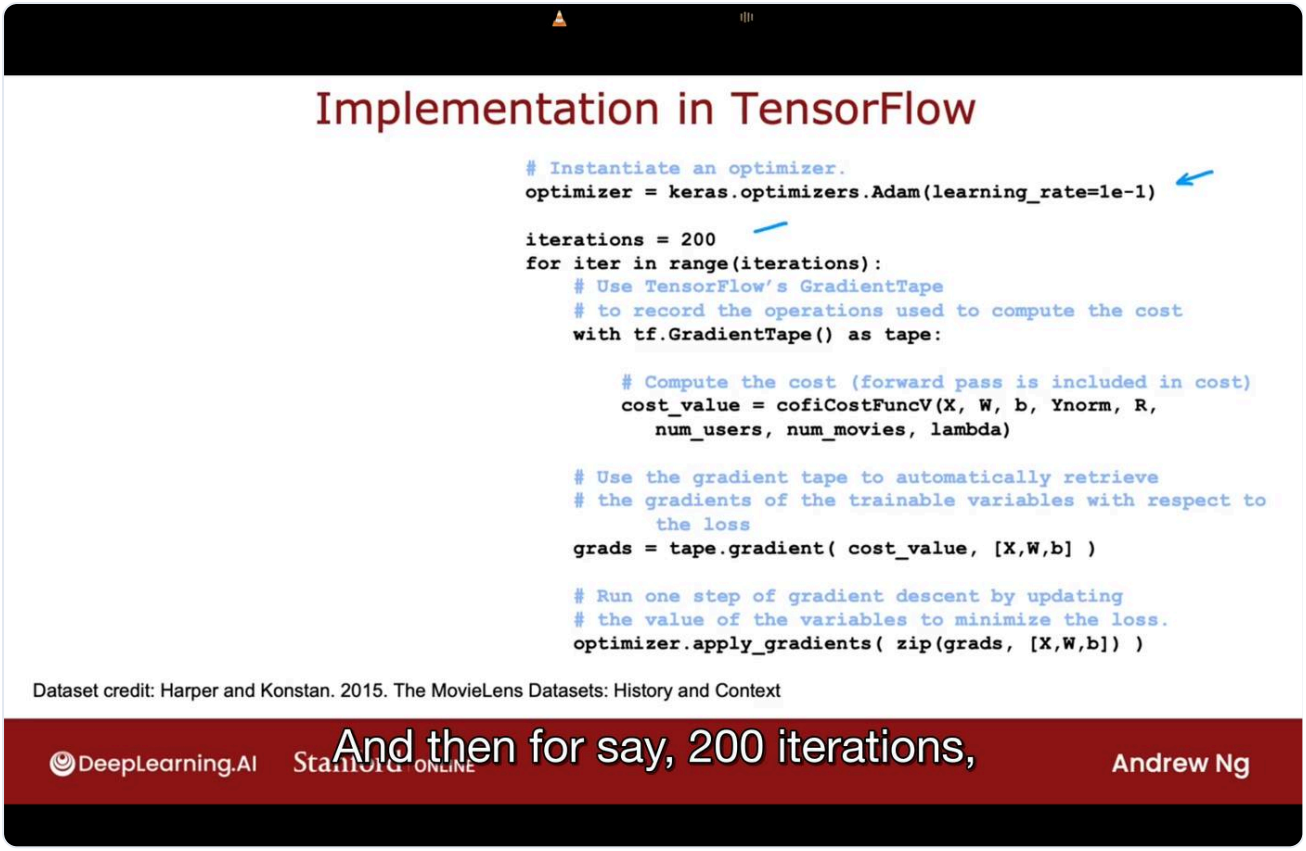


FIGURE 19.17 | Implementation in TensorFlow

And then for say, 200 iterations, here's the syntax as before with tf gradient tape, as tape, you need to provide code to compute the value of the cost function J. So recall that in collaborative filtering, the cost function J takes is input parameters x, w, and b as well as the ratings mean normalized. So that's why I'm writing y norm, r(i,j) specifying which values have a rating, number of users or nu in our notation, number of movies or nm in our notation or just num as well as the regularization parameter lambda.

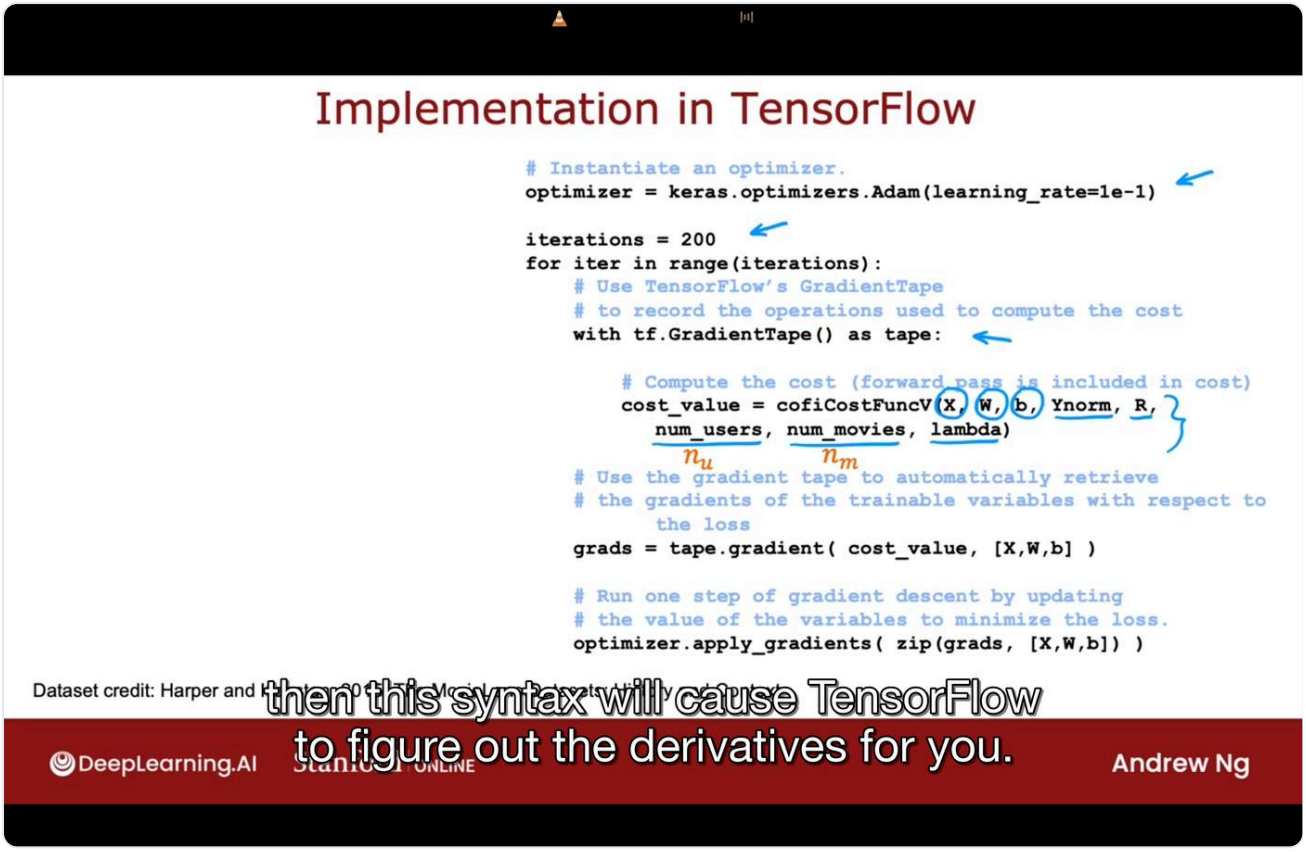


FIGURE 19.18 | Implementation in TensorFlow

And if you can implement this cost function J, then this syntax will cause TensorFlow to figure out the derivatives for you. Then this syntax will cause TensorFlow to record the sequence of operations used to compute the cost. And then by asking it to give you grads equals tape.gradient, this will give you the derivative of the cost function with respect to x, w, and b. And finally with the optimizer that we had specified up on top, as the adam optimizer. You can use the optimizer with the gradients that we just computed. And does it function in python is just a function that rearranges the numbers into an appropriate ordering for the applied gradients function.

If you are using gradient descent for collateral filtering, recall that the cost function J would be a function of w, b as well as x. And if you are applying gradient descent, you take the partial derivative respect the w. And then update w as follows. And you would also take the partial derivative of this respect to b. And update b as follows. And similarly update the features x as follows.

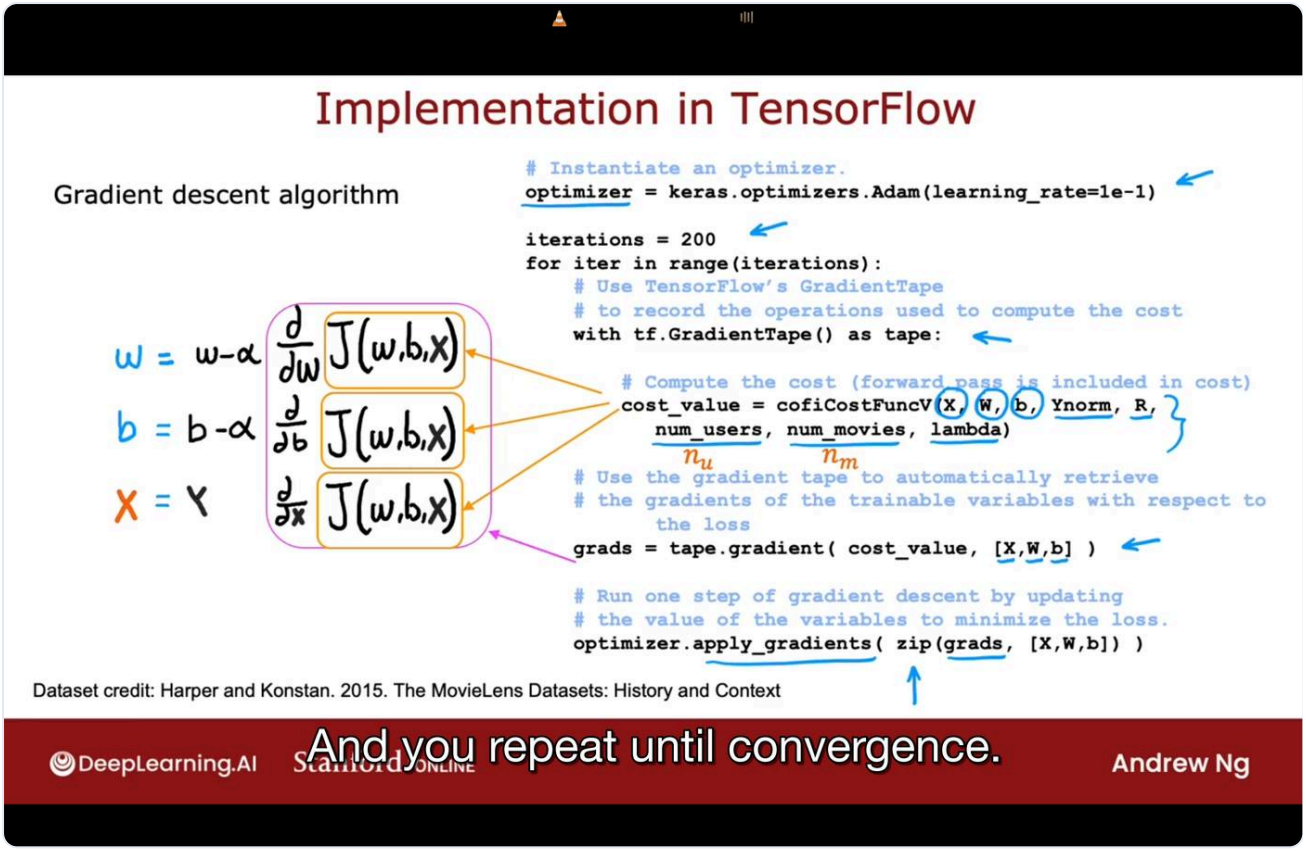


FIGURE 19.19 | Implementation in TensorFlow

And you repeat until convergence. But as I mentioned earlier with TensorFlow and Auto Diff you're not limited to just gradient descent. You can also use a more powerful optimization algorithm like the adam optimizer. The data set you use in the practice lab is a real data set comprising actual movies rated by actual people.

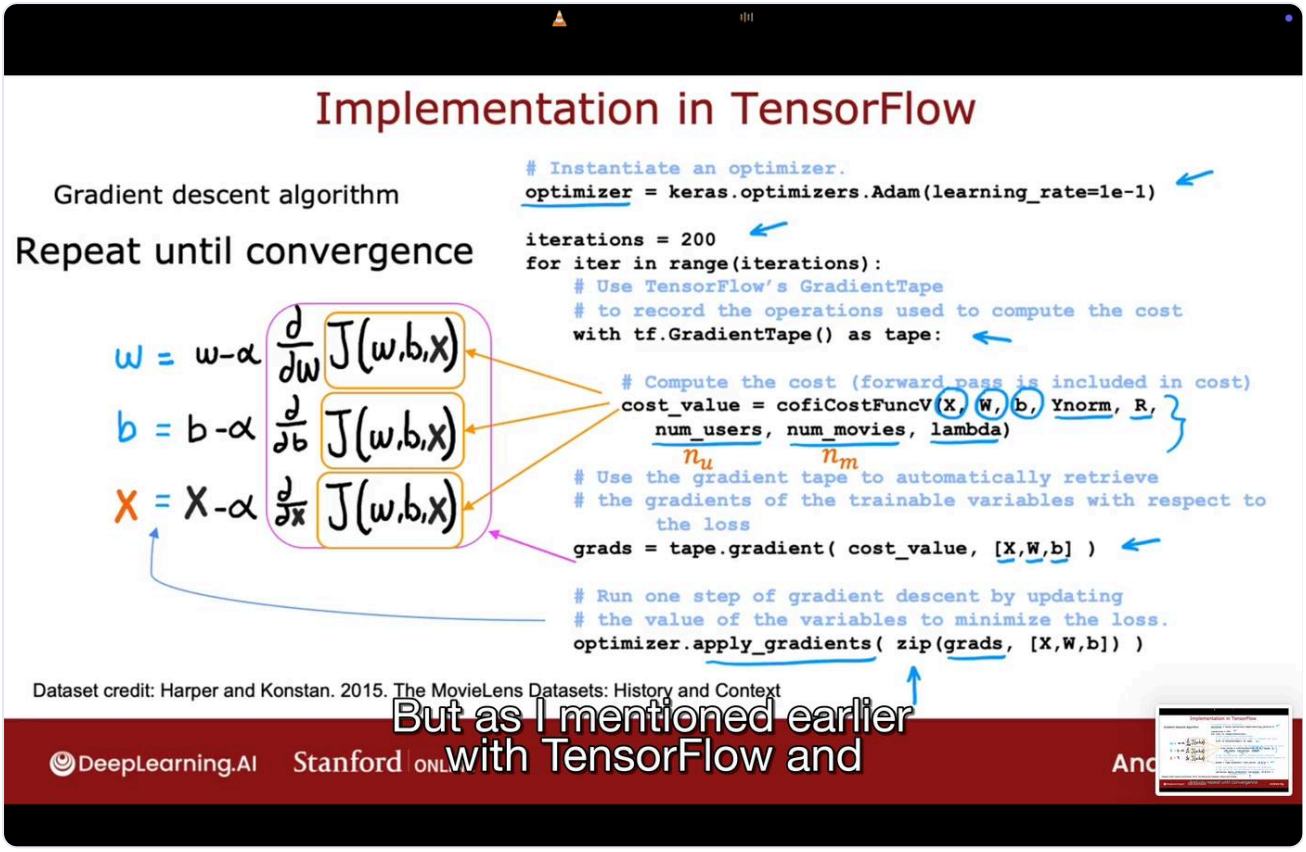


FIGURE 19.20 | Implementation in TensorFlow

This is the movie lens dataset and it's due to Harper and Konstan. And I hope you enjoy running this algorithm on a real data set of movies, and ratings and see for yourself the results that this algorithm can get. So that's it. That's how you can implement the collaborative filtering algorithm in TensorFlow. If you're wondering why do we have to do it this way? Why couldn't we use a dense layer and then model compiler and model fit? The reason we couldn't use that old recipe is, the collateral filtering algorithm and cost function, it doesn't neatly fit into the dense layer or the other standard neural network layer types of TensorFlow. That's why we had to implement it this other way where we would implement the cost function ourselves.

But then use TensorFlow's tools for automatic differentiation, also called Auto Diff. And use TensorFlow's implementation of the adam optimization algorithm to let it do a lot of the work for us of optimizing the cost function. If the model you have is a sequence of dense neural network layers or other types of layers supported by TensorFlow, and the old implementation recipe of model compound model fit works. But even when it isn't, these tools TensorFlow give you a very effective way to implement other learning algorithms as well. And so I hope you enjoy playing more with the collaborative filtering exercise in this week's practice lab. And looks like there's a lot of code and lots of syntax, don't worry about it.

Make sure you have what you need to complete that exercise successfully. And in the next video, I'd like to also move on to discuss more of the nuances of collateral filtering and specifically the question of how do you find related items, given one movie, whether other movies similar to this one. Let's go on to the next video

LECTURE 20

Finding Related Items

If you come to an online shopping website and you're looking at a specific item, say maybe a specific book, the website may show you things like, "Here are some other books similar to this one" or if you're browsing a specific movie, it may say, "Here are some other movies similar to this one. How do the websites do that?, so that when you're looking at one item, it gives you other similar or related items to consider. It turns out the collaborative filtering algorithm that we've been talking about gives you a nice way to find related items. Let's take a look.

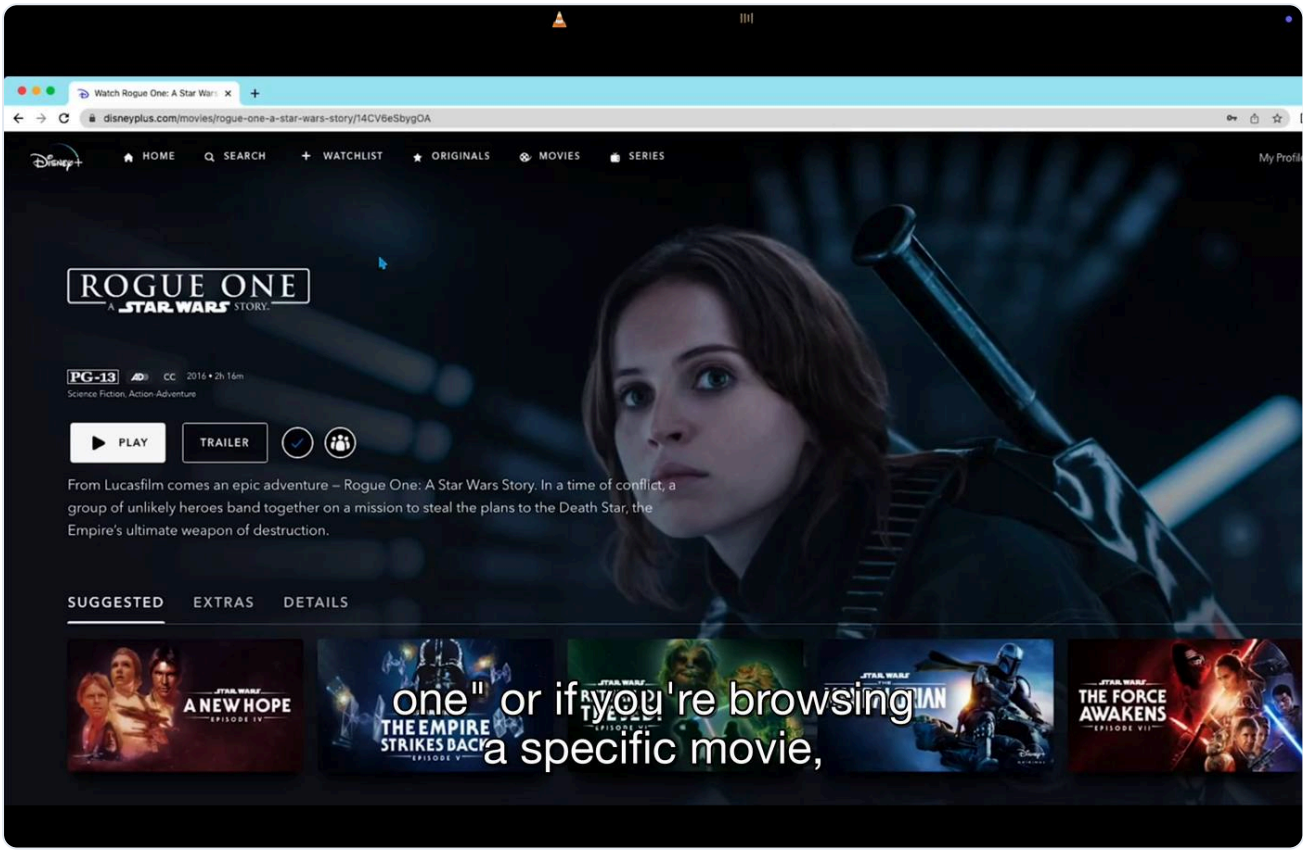


FIGURE 20.1

As part of the collaborative filtering we've discussed, you learned features x^(i) for every item i, for every movie i or other type of item they're recommending to users. Whereas early this week, I had used a hypothetical example of the features representing how much a movie is a romance movie versus an action movie. In practice, when you use this algorithm to learn the features x^(i) automatically, looking at the individual features x_1, x_2, x_3, you find them to be quite hard to interpret. Is quite hard to learn features and say, x_1 is an action movie and x_2 is as a foreign film and so on. But nonetheless, these learned features, collectively x_1, x_2, x_3, other many features, and you have collectively these features do convey something about what that movie is like.

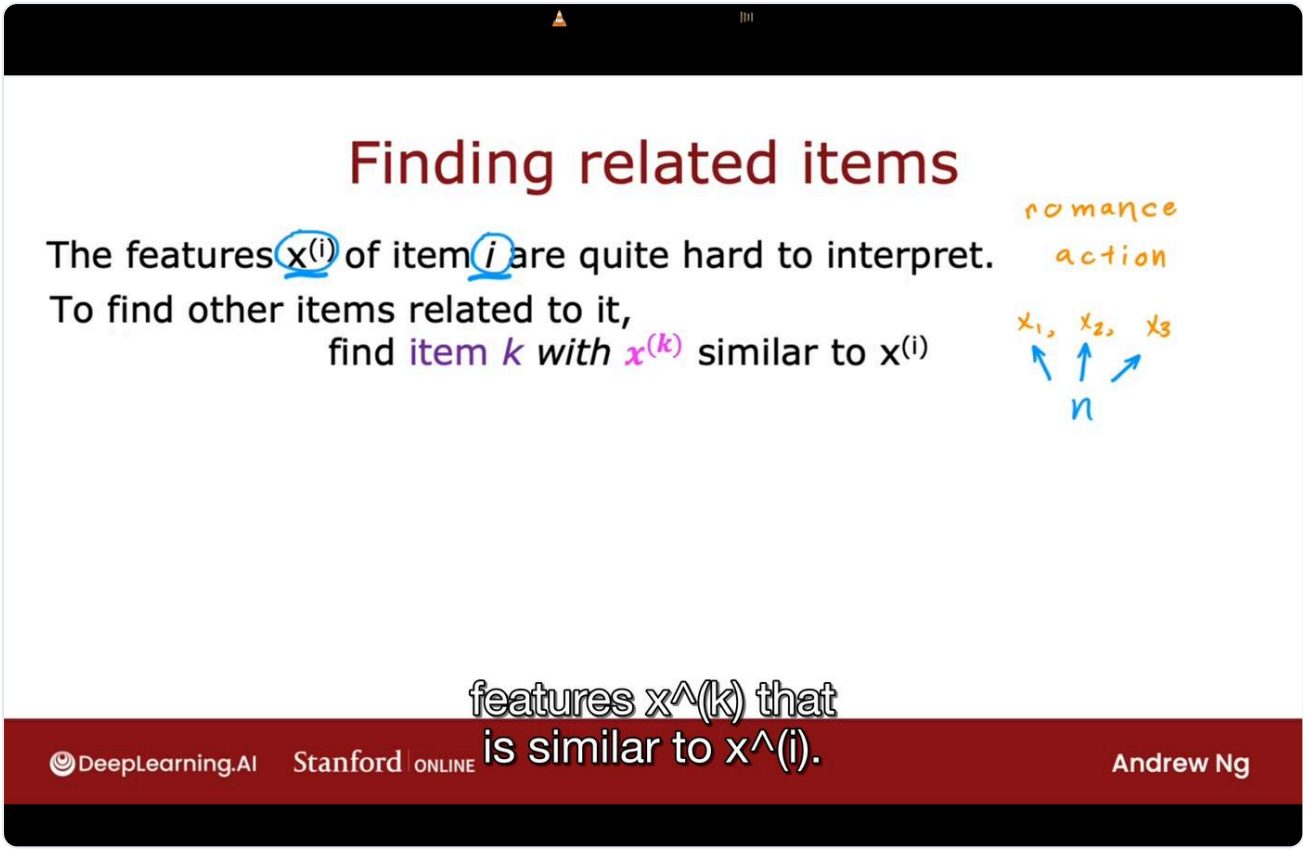


FIGURE 20.2 | Finding Related Items

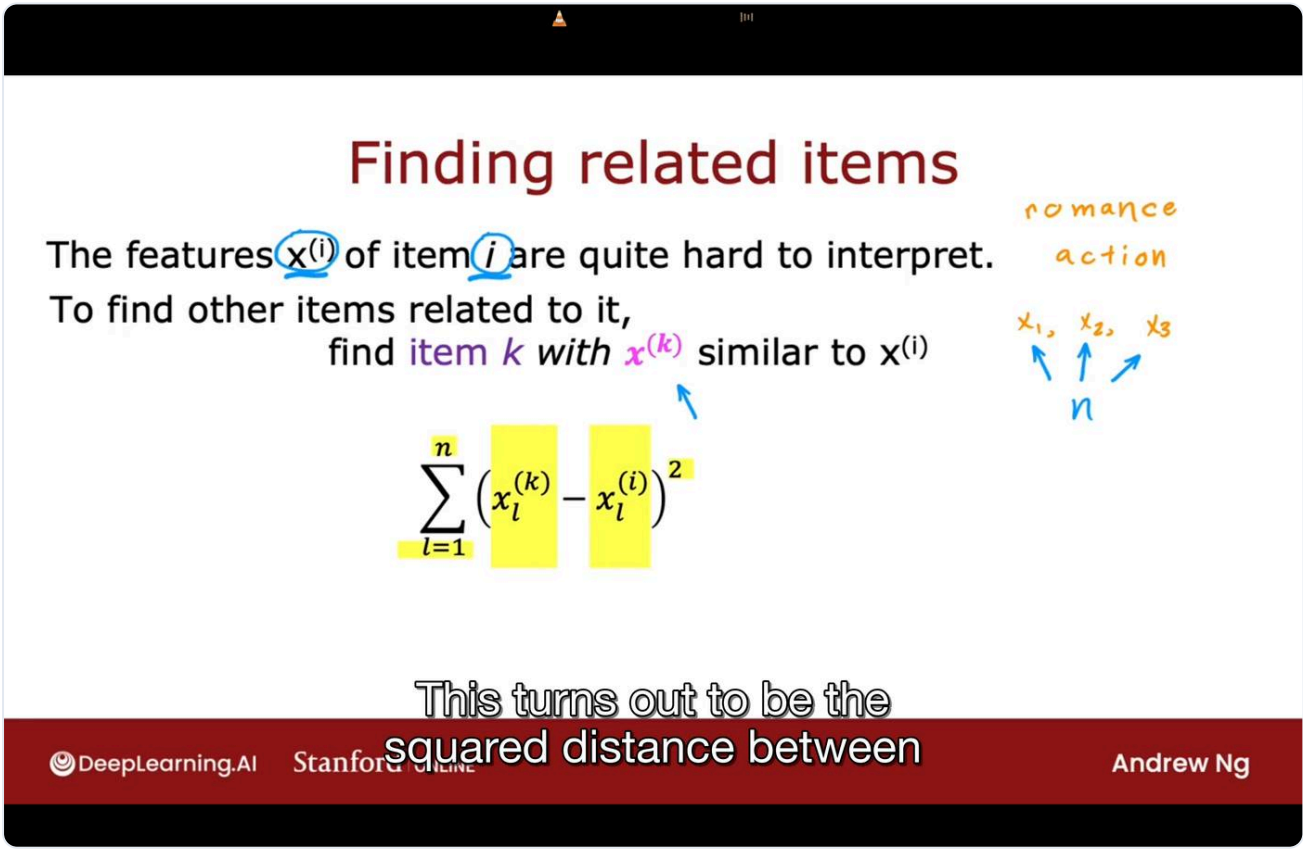


FIGURE 20.3 | Finding Related Items

It turns out that given features x^(i) of item i, if you want to find other items, say other movies related to movie i, then what you can do is try to find the item k with features x^(k) that is similar to x^(i). In particular, given a feature vector x^(k), the way we determine what are known as similar to the feature x^(i) is as follows: is the sum from l equals 1 through n with n features of x^(k)_l minus x^(i)_l square. This turns out to be the squared distance between x^(k) and x^(i) and in math, this squared distance between these two vectors, x^(k) and x^(i), is sometimes written as follows as well.

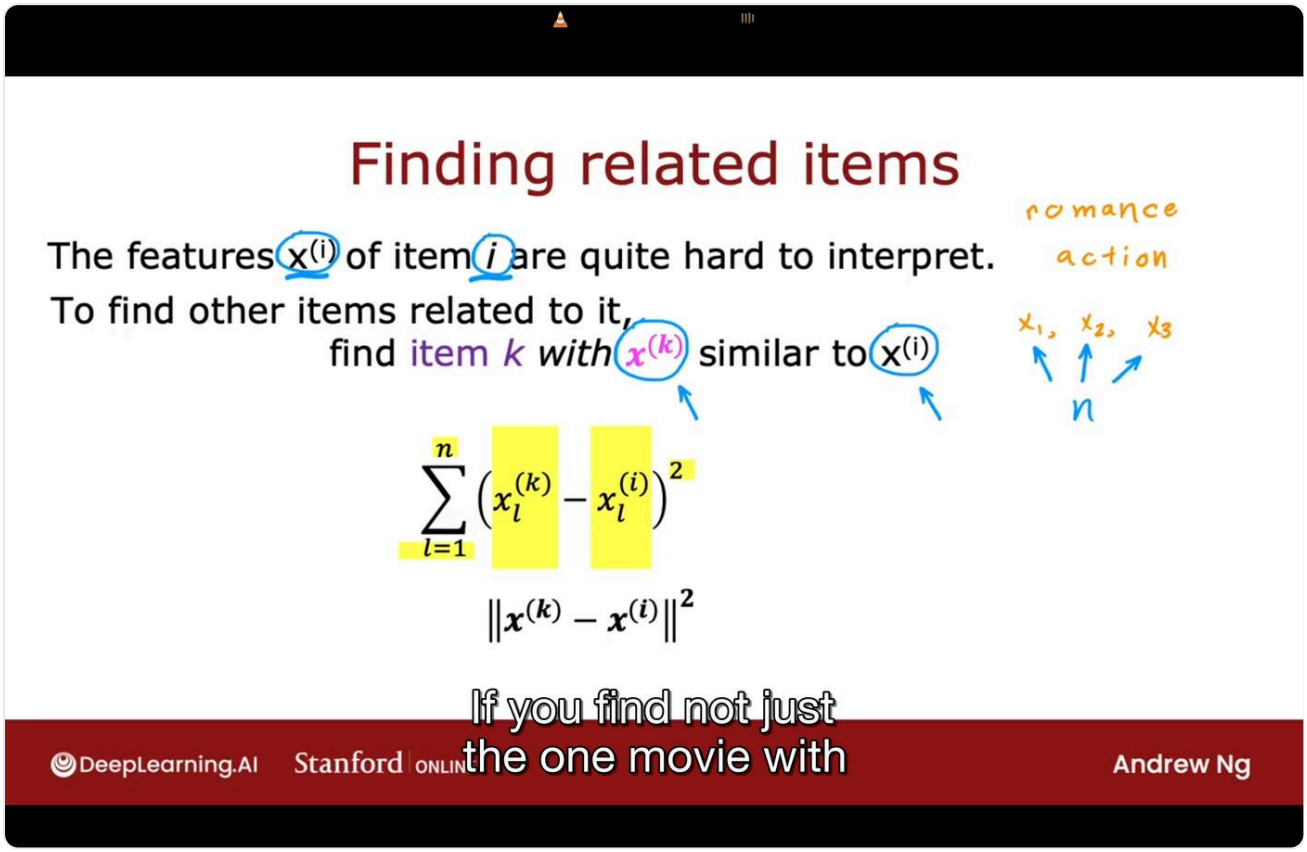


FIGURE 20.4 | Finding Related Items

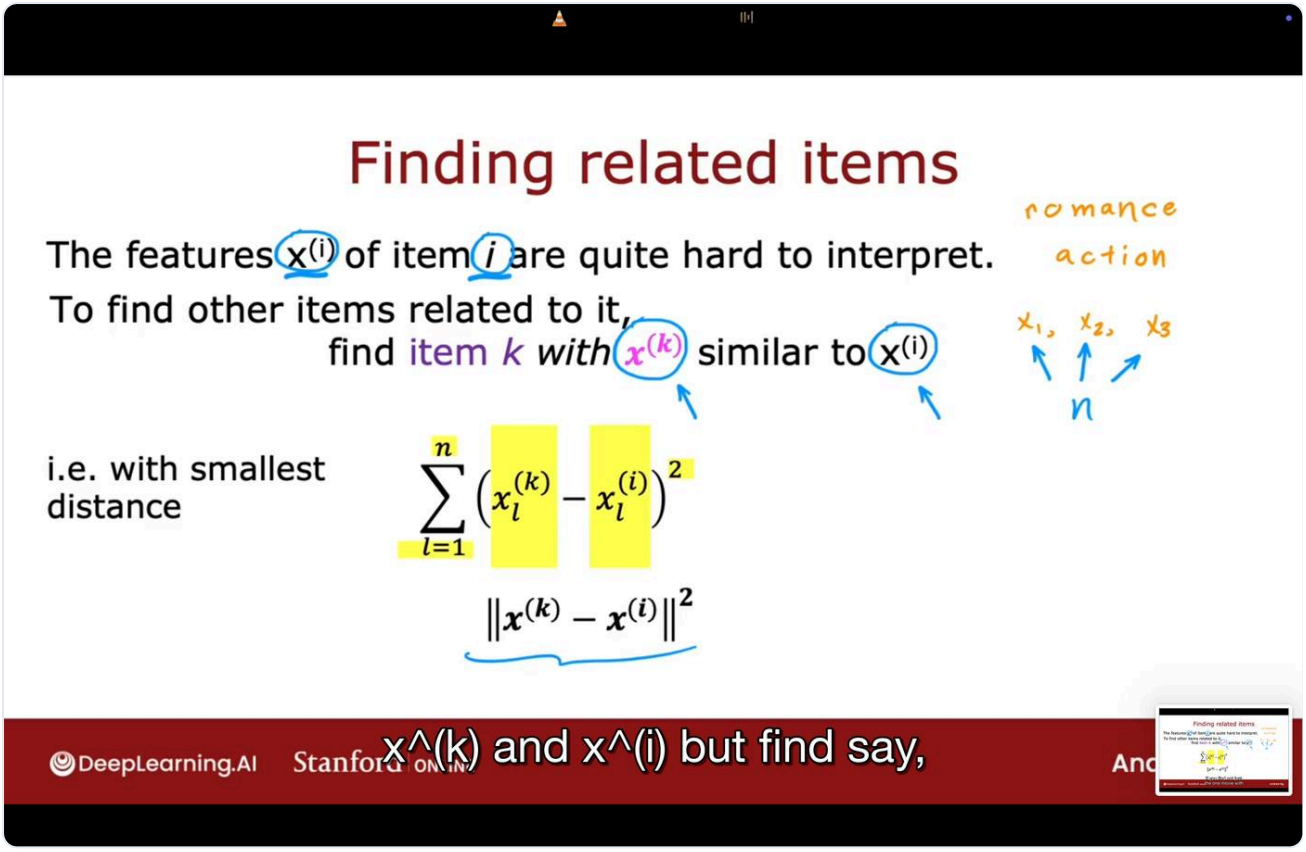


FIGURE 20.5 | Finding Related Items

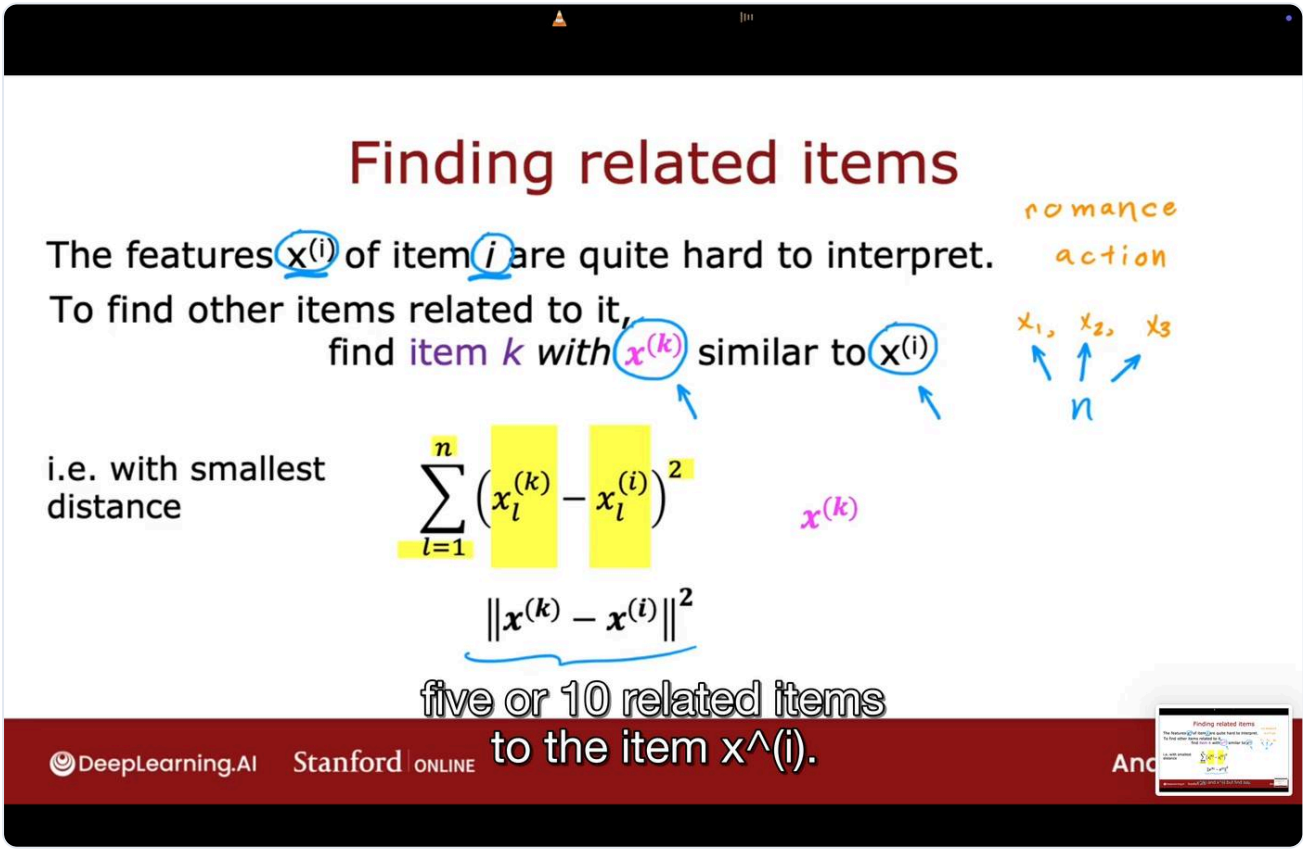


FIGURE 20.6 | Finding Related Items

If you find not just the one movie with the smallest distance between x^(k) and x^(i) but find say, the five or 10 items with the most similar feature vectors, then you end up finding five or 10 related items to the item x^(i). If you're building a website and want to help users find related products to a specific product they are looking at, this would be a nice way to do so because the features x^(i) give a sense of what item i is about, other items x^(k) with similar features will turn out to be similar to item i. It turns out later this week, this idea of finding related items will be a small building blocks that we'll use to get to an even more powerful recommended system as well.

Before wrapping up this section, I want to mention a few limitations of collaborative filtering. In collaborative filtering, you have a set of items and so the users and the users have rated some subset of items. One of this weaknesses is that is not very good at the cold start problem. For example, if there's a new item in your catalog, say someone's just published a new movie and hardly anyone has rated that movie yet, how do you rank the new item if very few users have rated it before? Similarly, for new users that have rated only a few items, how can we make sure we show them something reasonable? We could see in an earlier video, how mean normalization can help with this and it does help a lot.

But perhaps even better ways to show users that rated very few items, things that are likely to interest them.

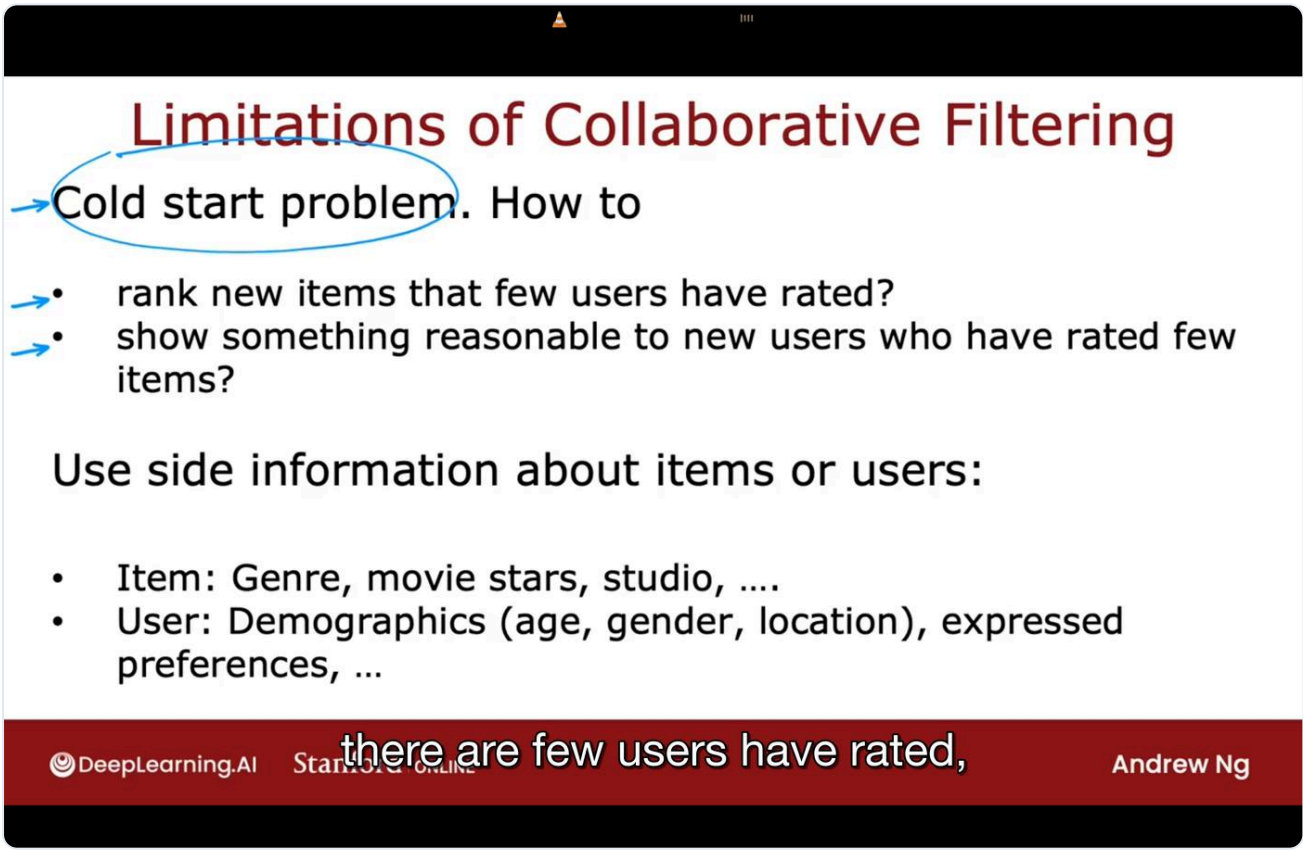


FIGURE 20.7 | Limitations of Collaborative Filtering

This is called the cold start problem, because when you have a new item, there are few users have rated, or we have a new user that's rated very few items, the results of collaborative filtering for that item or for that user may not be very accurate. The second limitation of collaborative fil-

tering is it doesn't give you a natural way to use side information or additional information about items or users.

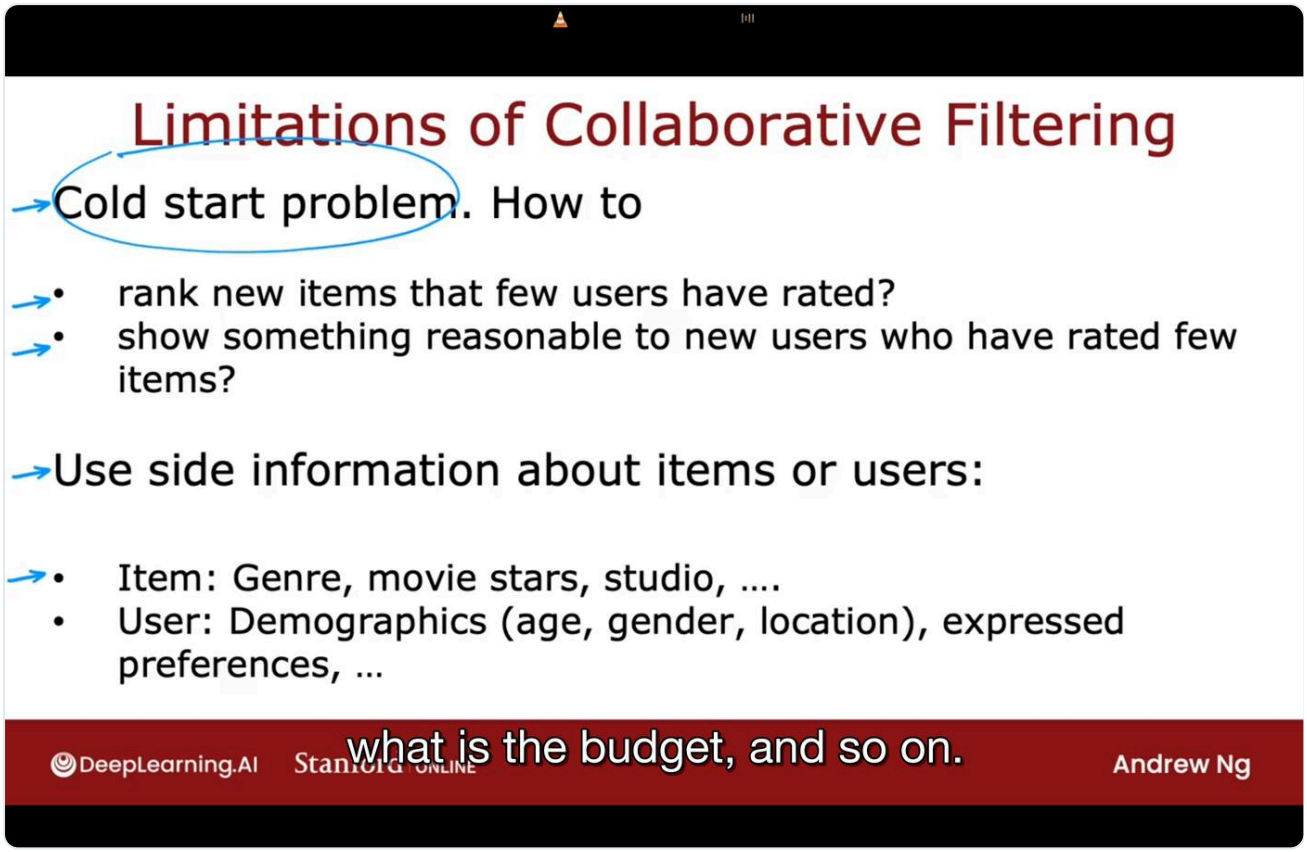


FIGURE 20.8 | Limitations of Collaborative Filtering

For example, for a given movie in your catalog, you might know what is the genre of the movie, who had a movie stars, whether it is a studio, what is the budget, and so on. You may have a lot of features about a given movie. For a single user, you may know something about their demographics, such as their age, gender, location.

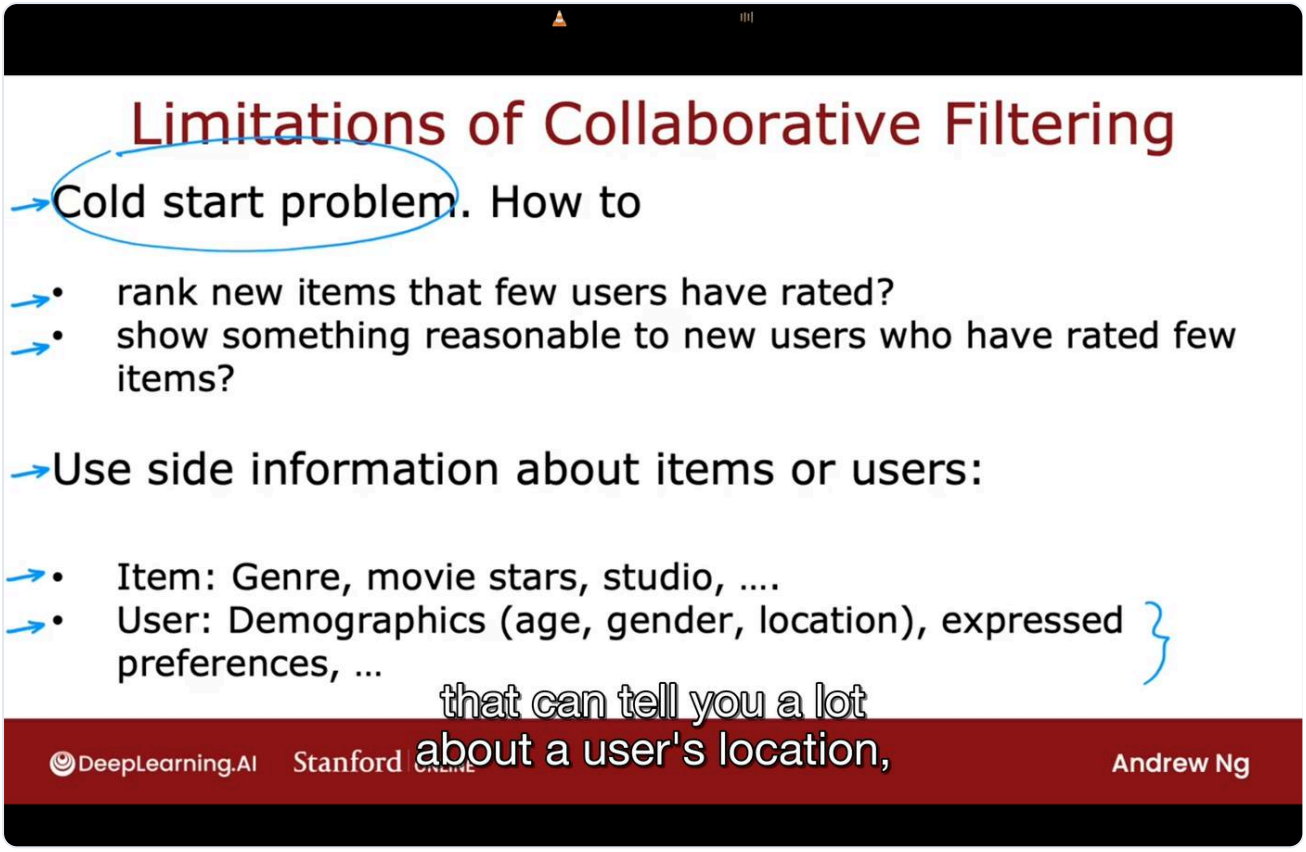


FIGURE 20.9 | Limitations of Collaborative Filtering

They express preferences, such as if they tell you they like certain movies genres but not other movies genres, or it turns out if you know the user's IP address, that can tell you a lot about a user's location, and knowing the user's location might also help you guess what might the user be interested in, or if you know whether the user is accessing your site on a mobile or on a desktop, or if you know what web browser they're using. It turns out all of these are little cues you can get. They can be surprisingly correlated with the preferences of a user. It turns out by the way, that it is known that users, that use the Chrome versus Firefox versus Safari versus the Microsoft Edge browser, they actually behave in very different ways.

Even knowing the user web browser can give you a hint when you have collected enough data of what this particular user may like. Even though collaborative filtering, we have multiple users give you ratings of multiple items, is a very powerful set of algorithms, it also has some limitations. In the next video, let's go on to develop content-based filtering algorithms, which can address a lot of these limitations. Content-based filtering algorithms are a state of the art technique used in many commercial applications today. Let's go take a look at how they work.

LECTURE 21

Content-based Filtering: Learning to Match

In this video, we'll start to develop a second type of recommender system called a content-based filtering algorithm. To get started, let's compare and contrast the collaborative filtering approach that we'll be looking at so far with this new content-based filtering approach. Let's take a look.

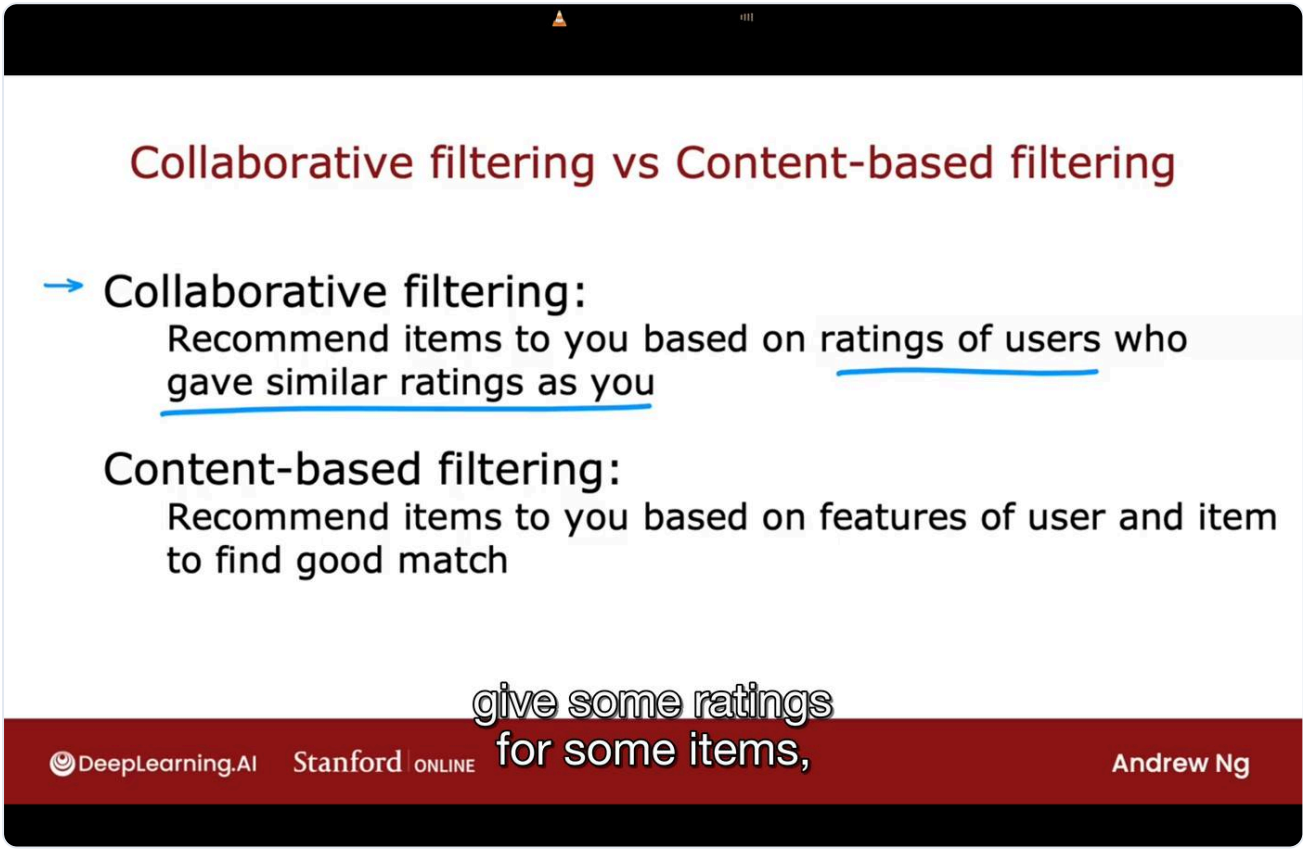


FIGURE 21.1 | Collaborative Filtering vs Content-based Filtering

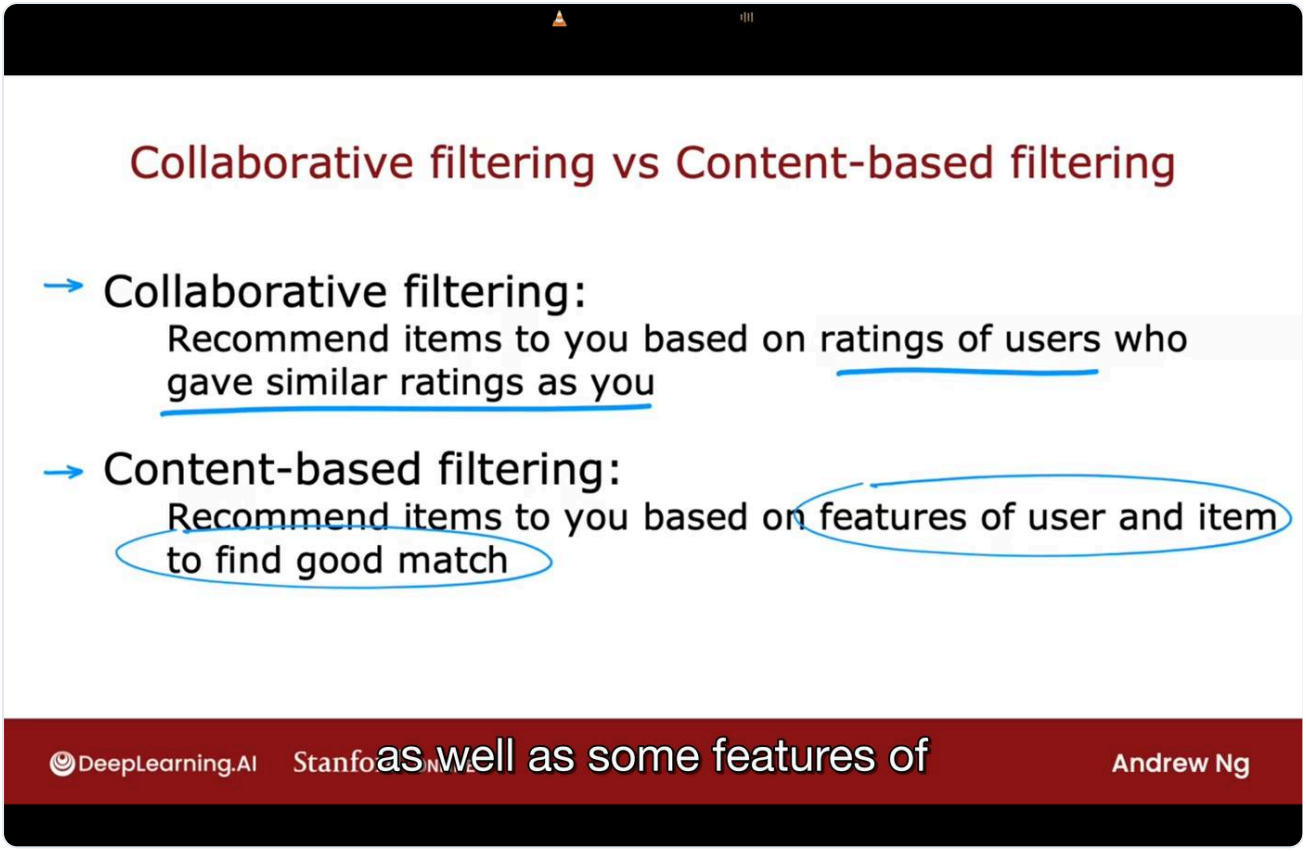


FIGURE 21.2 | Collaborative Filtering vs Content-based Filtering

With collaborative filtering, the general approach is that we would recommend items to you based on ratings of users who gave similar ratings as you. We have some number of users give some ratings for some items, and the algorithm figures out how to use that to recommend new items to you. In contrast, content-based filtering takes a different approach to deciding what to recommend to you. A content-based filtering algorithm will recommend items to you based on the features of users and features of the items to find a good match. In other words, it requires having some features of each user, as well as some features of each item and it uses those features to try to decide which items and users might be a good match for each other.

With a content-based filtering algorithm, you still have data where users have rated some items.

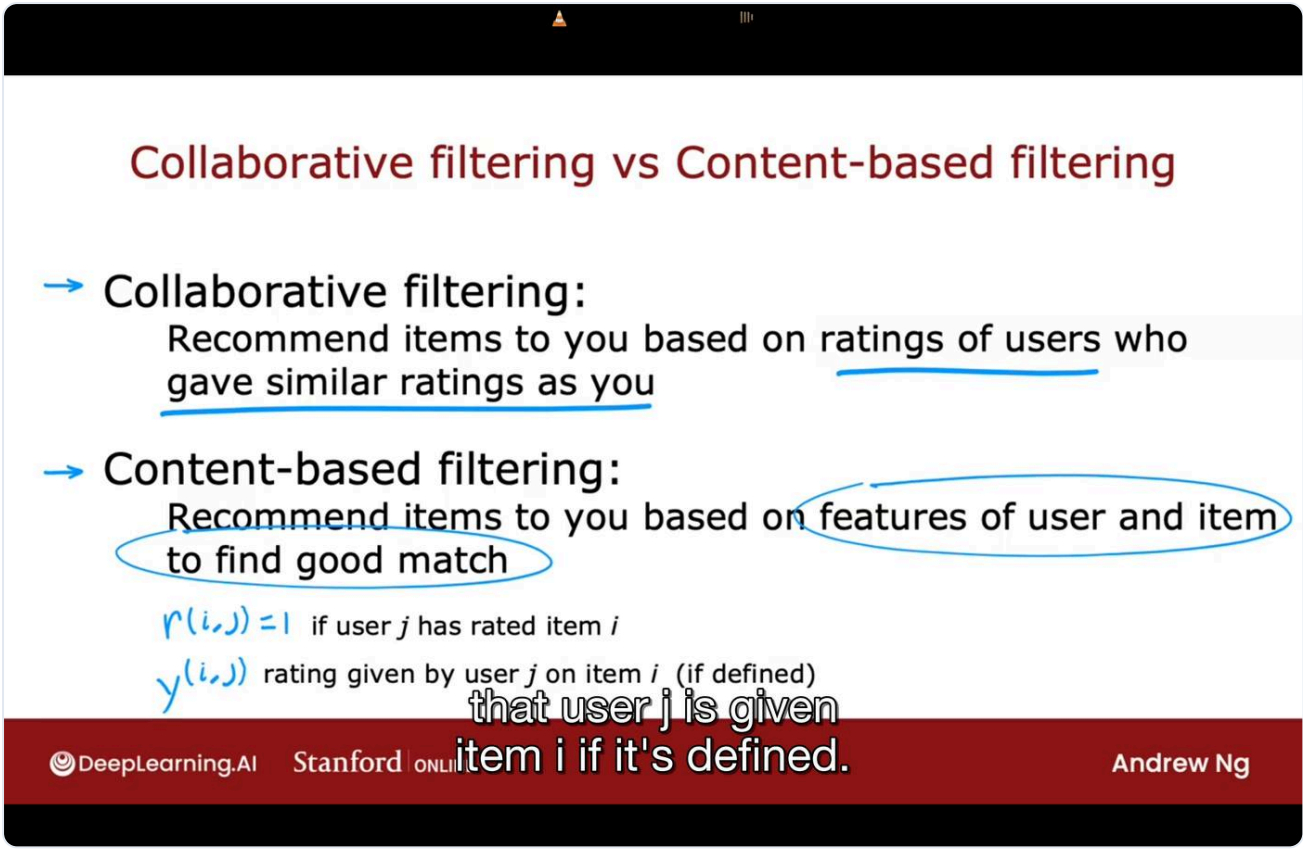


FIGURE 21.3 | Collaborative Filtering vs Content-based Filtering

Well, content-based filtering will continue to use r, i, j to denote whether or not user j has rated item i and will continue to use y i, j to denote the rating that user j is given item i if it's defined. But the key to content-based filtering is that we will be able to make good use of features of the user and of the items to find better matches than potentially a pure collaborative filtering approach might be able to. Let's take a look at how this works. In the case of movie recommendations, here are some examples of features. You may know the age of the user, or you may have the gender of the user. This could be a one-hot feature similar to what you saw when we were talking about decision trees where you could have a one-hot feature with the values based on whether the user's self-identified gender is male or female or unknown, and you may know the country of the user.

If there are about 200 countries in the world then also be a one-hot feature with about 200 possible values. You can also look at past behaviors of the user to construct this feature vector. For example, if you look at the top thousand movies in your catalog, you might construct a thousand features that tells you of the thousand most popular movies in the world which of these has the user watch.

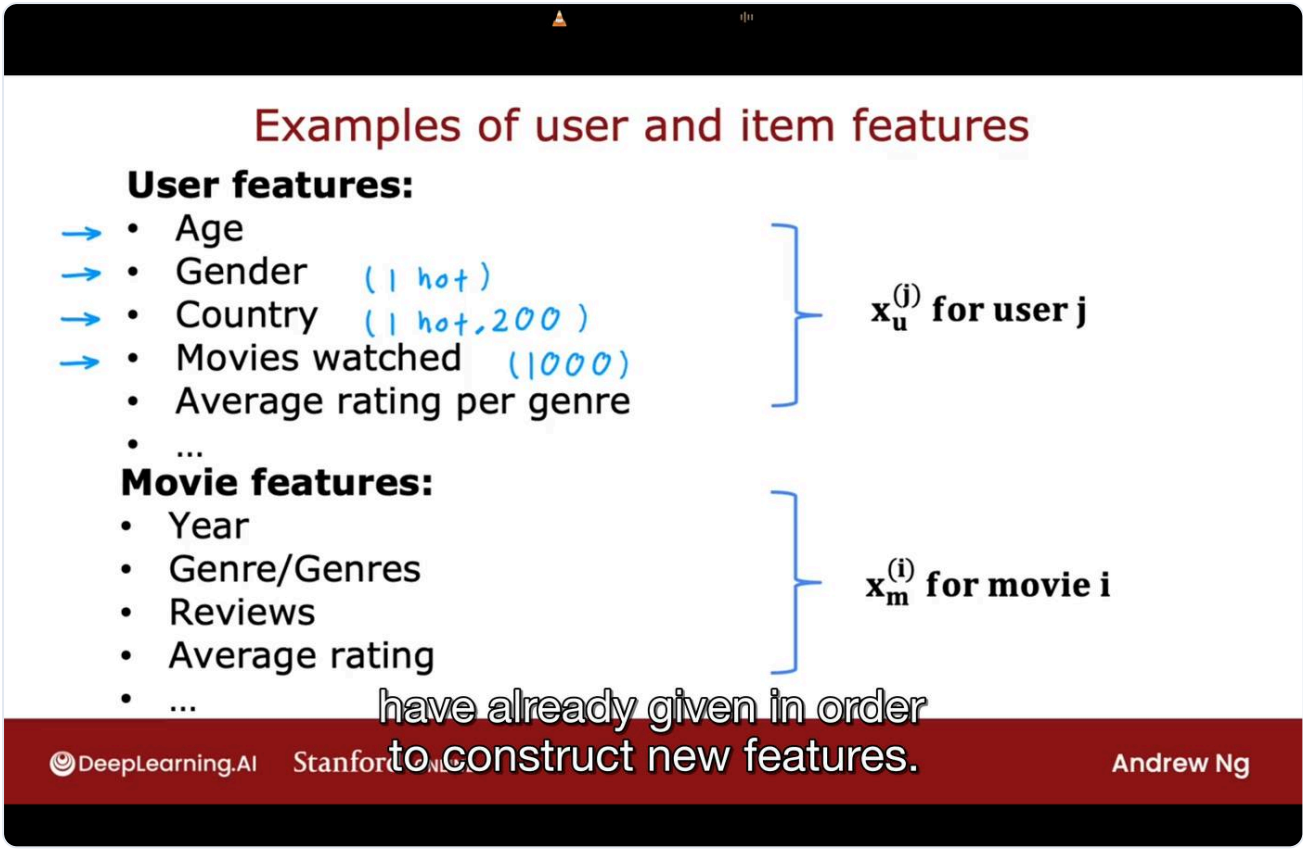


FIGURE 21.4 | Examples of User and Item Features

In fact, you can also take ratings the user might have already given in order to construct new features. It turns out that if you have a set of movies and if you know what genre each movie is in, then the average rating per genre that the user has given. Of all the romance movies that the user has rated, what was the average rating? Of all the action movies that the user has rated, what was the average rating? And so on for all the other genres. This too can be a powerful feature to describe the user. One interesting thing about this feature is that it actually depends on the ratings that the user had given. But there's nothing wrong with that. Constructing a feature vector that depends on the user's ratings is a completely fine way to develop a feature vector to describe that user.

With features like these you can then come up with a feature vector x subscript u, use as a user superscript j for user j. Similarly, you can also come up with a set of features for each movie of each item, such as what was the year of the movie? What's the genre or genres of the movie of known? If there are critic reviews of the movie, you can construct one or multiple features to capture something about what the critics are saying about the movie. Or once again, you can actually take user ratings of the movie to construct a feature of, say, the average rating of this movie. This feature again depends on the ratings that users are given but again, does nothing wrong with that.

You can construct a feature for a given movie that depends on the ratings that movie had received, such as the average rating of the movie. Or if you wish, you can also have average rating per country or average rating per user demographic as they want to construct other types of features of the movies as well. With this, for each movie, you can then construct a feature vector,

which I'm going to denote x subscript m, m stands for movie, and superscript i for movie i. Given features like this, the task is to try to figure out whether a given movie i is going to be good match for user j. Notice that the user features and movie features can be very different in size.

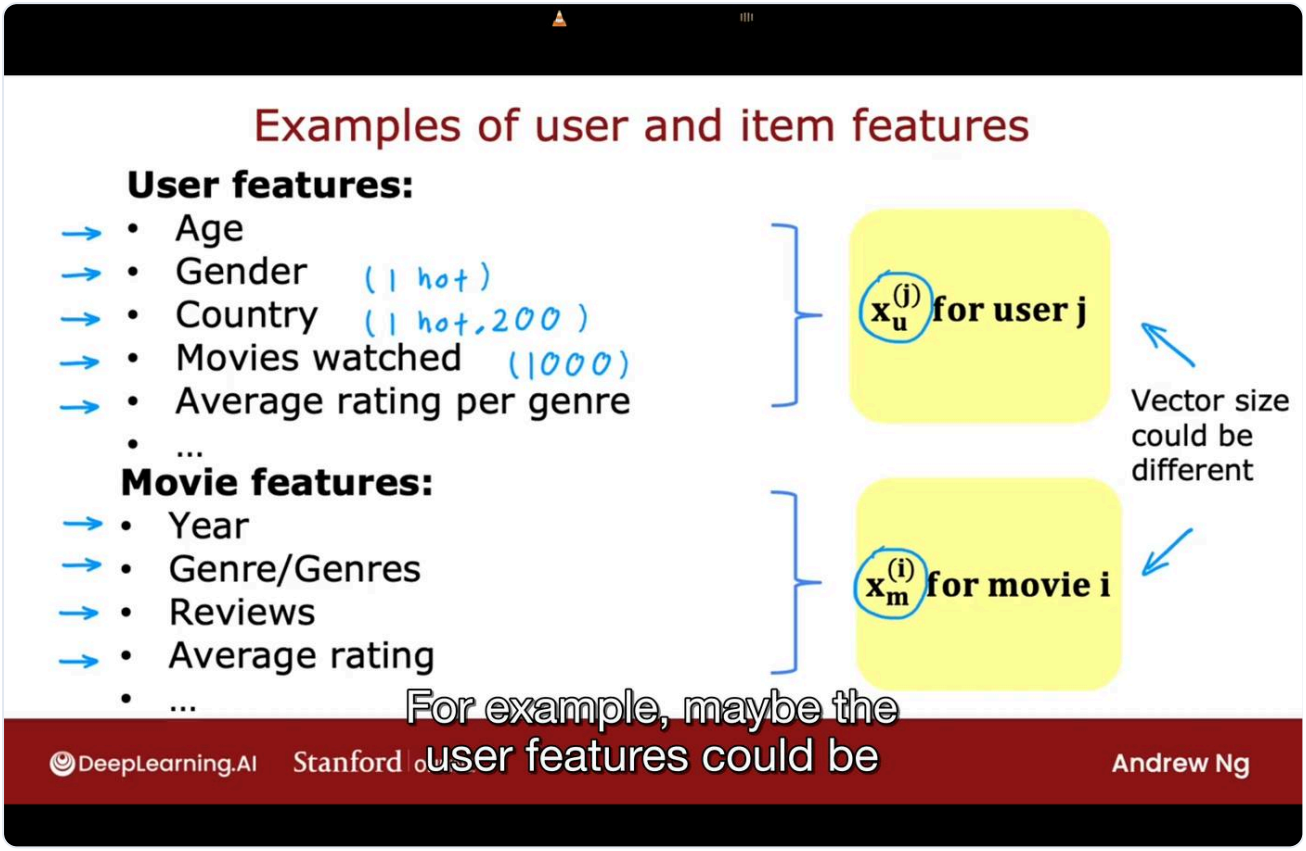


FIGURE 21.5 | Examples of User and Item Features

For example, maybe the user features could be 1500 numbers and the movie features could be just 50 numbers. That's okay too. In content-based filtering, we're going to develop an algorithm that learns to match users and movies. Previously, we were predicting the rating of user j on movie i as wj dot products of xi plus bj. In order to develop content-based filtering, I'm going to get rid of bj. It turns out this won't hurt the performance of the content-based filtering at all. Instead of writing wj for a user j and xi for a movie i, I'm instead going to just replace this notation with vj_u. This v here stands for a vector. There'll be a list of numbers computed for user j and the u subscript here stands for user.

Instead of xi, I'm going to compute a separate vector subscript m, to stand for the movie and for movie is what a superscript stands for. Vj_u as a vector as a list of numbers computed from the features of user j and vi_m is a list of numbers computed from the features like the ones you saw on the previous slide of movie i. If we're able to come up with an appropriate choice of these vectors, vj_u and vi_m, then hopefully the dot product between these two vectors will be a good prediction of the rating that user j gives movie i.

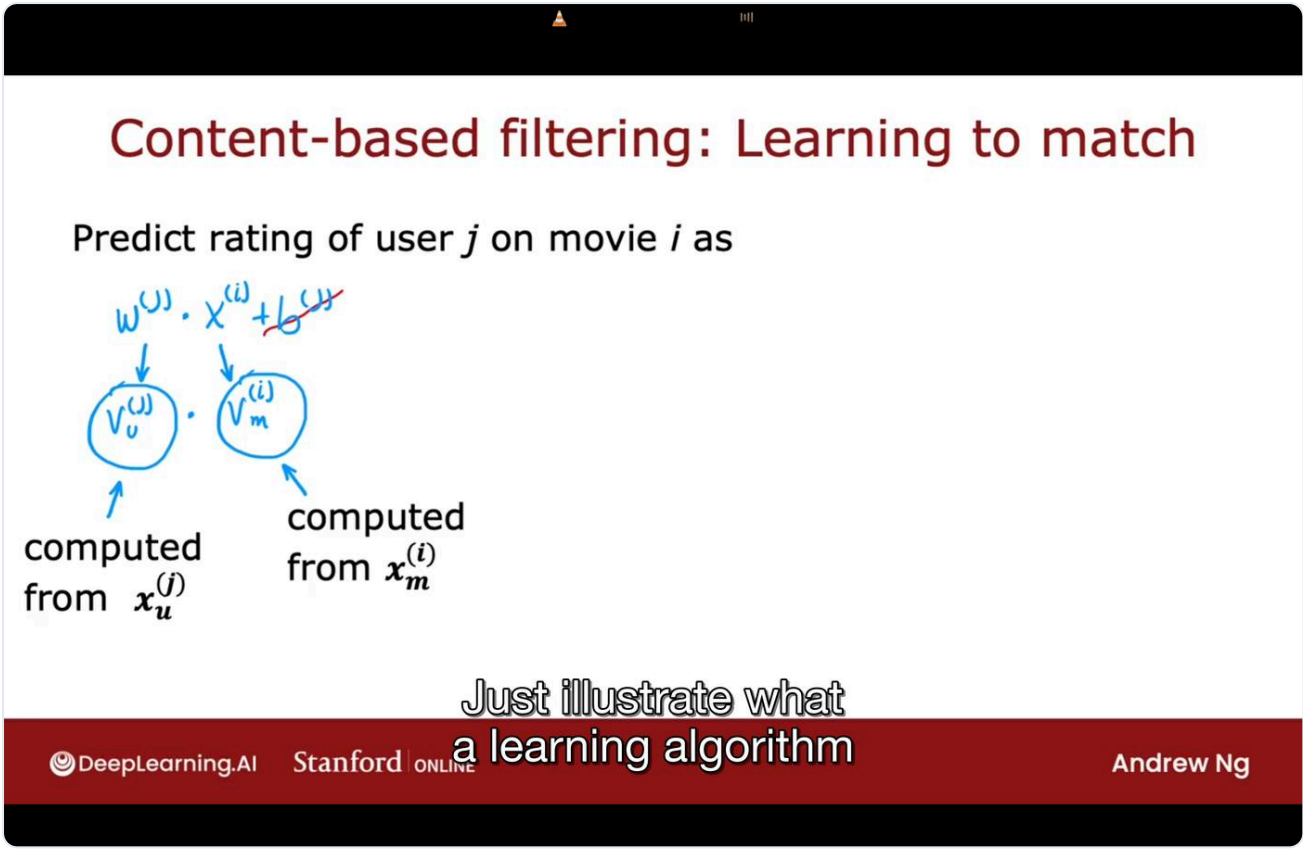


FIGURE 21.6 | Content-based Filtering: Learning to Match

Just illustrate what a learning algorithm could come up with. If v, u, that is a user vector, turns out to capture the user's preferences, say is 4.9, 0.1, and so on. Lists of numbers like that. The first number captures how much do they like romance movies. Then the second number captures how much do they like action movies and so on. Then v_m, the movie vector is 4.5, 0.2, and so on and so forth of these numbers capturing how much is this a romance movie, how much is this an action movie, and so on.

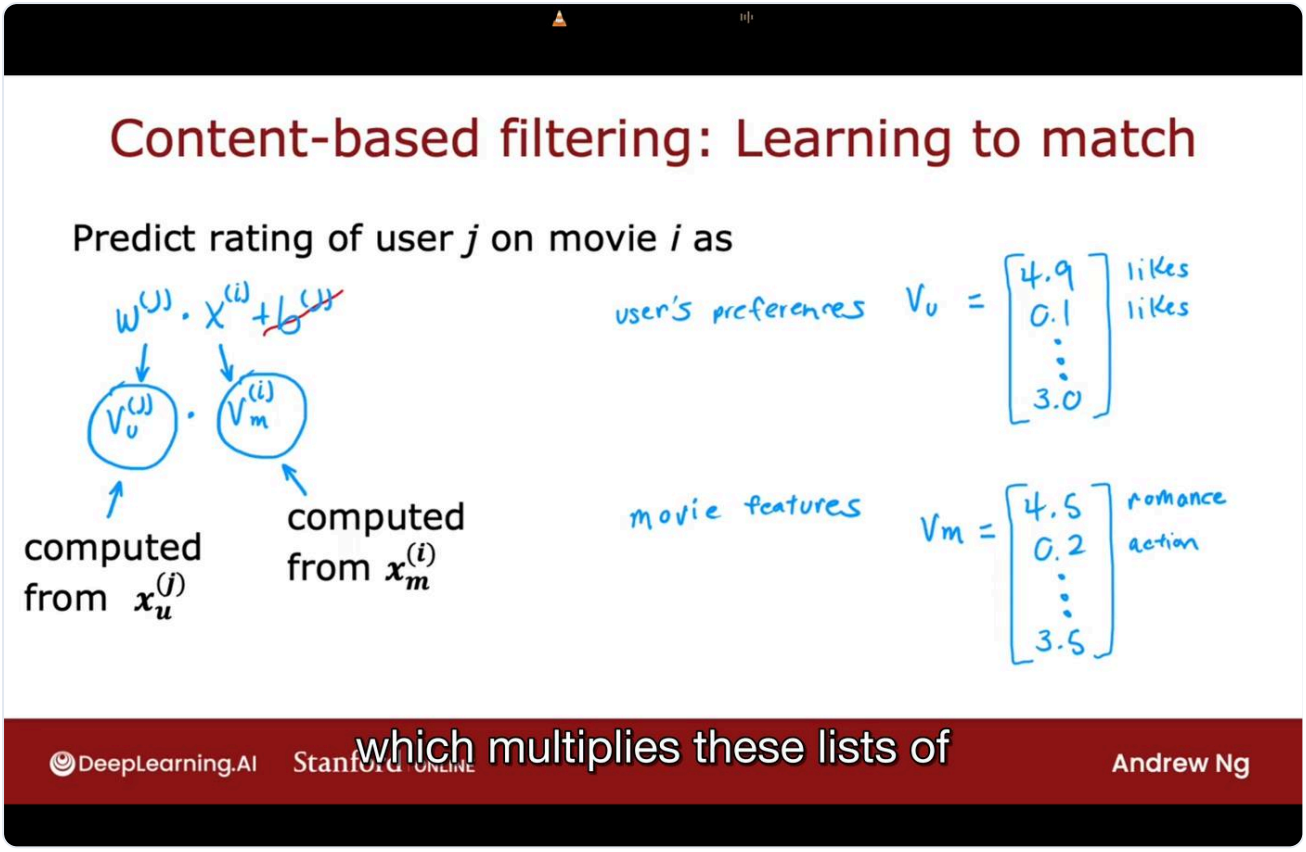


FIGURE 21.7 | Content-based Filtering: Learning to Match

Then the dot product, which multiplies these lists of numbers element-wise and then takes a sum, hopefully, will give a sense of how much this particular user will like this particular movie. The challenges given features of a user, say xj_u, how can we compute this vector vj_u that represents succinctly or compactly the user's preferences? Similarly given features of a movie, how can we compute vi_m?

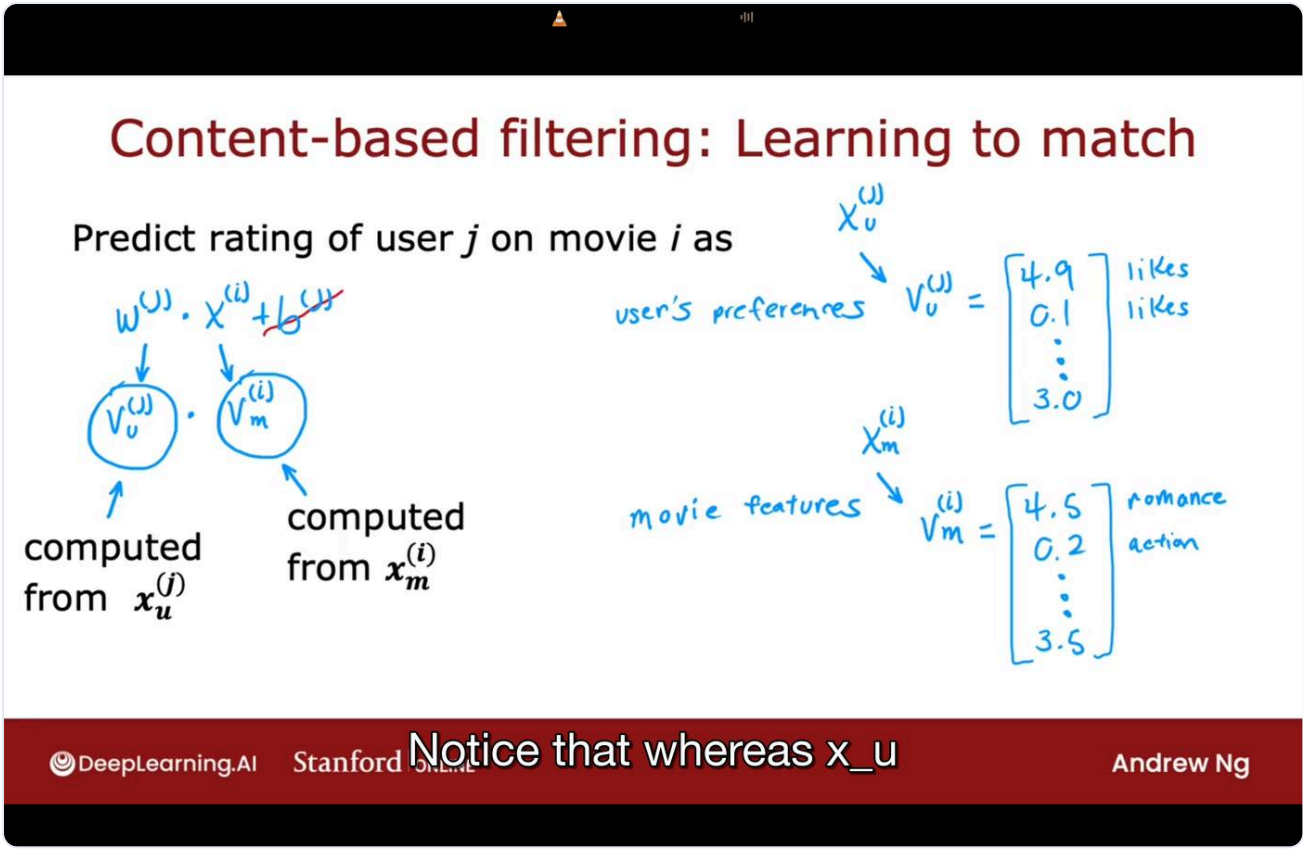


FIGURE 21.8 | Content-based Filtering: Learning to Match

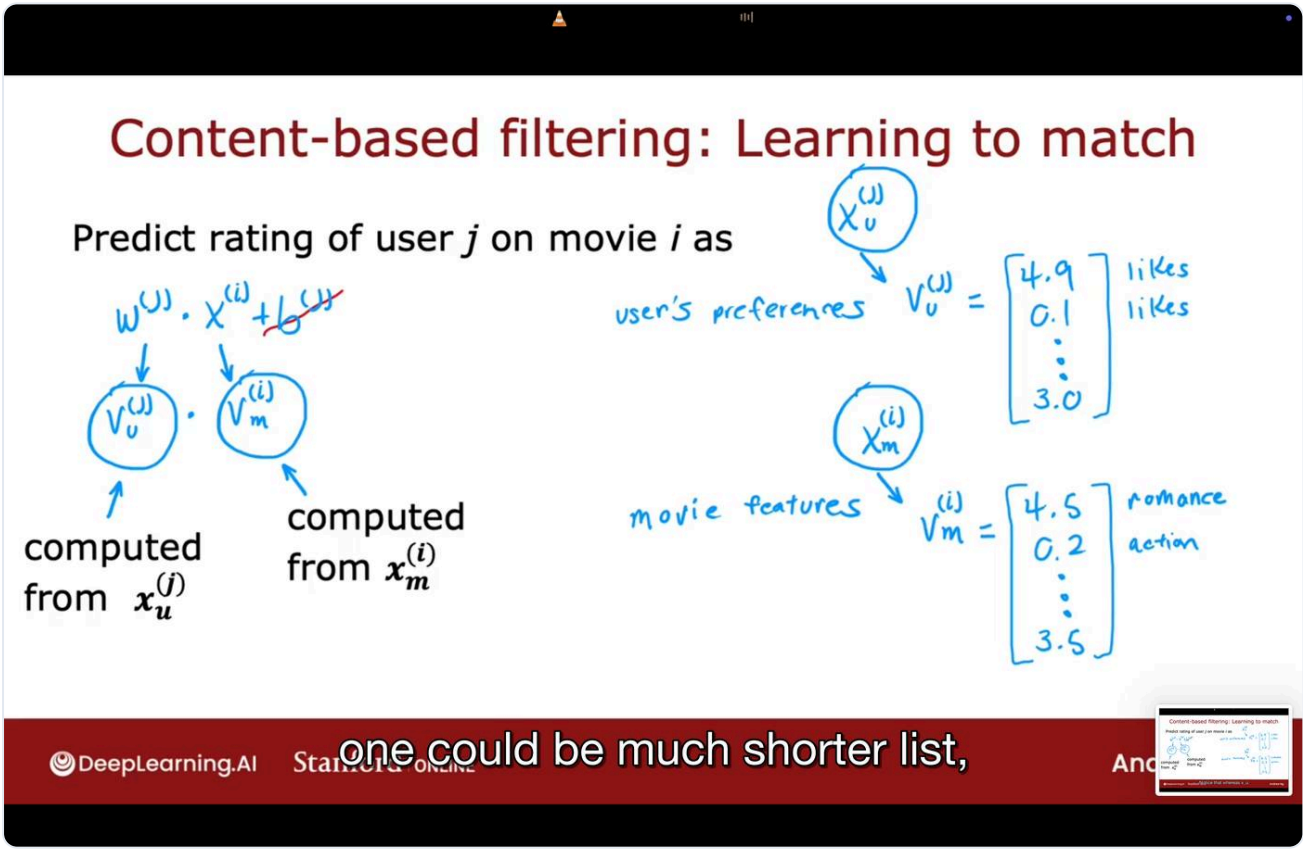


FIGURE 21.9 | Content-based Filtering: Learning to Match

Notice that whereas x_u and x_m could be different in size, one could be very long lists of numbers, one could be much shorter list, v here have to be the same size.

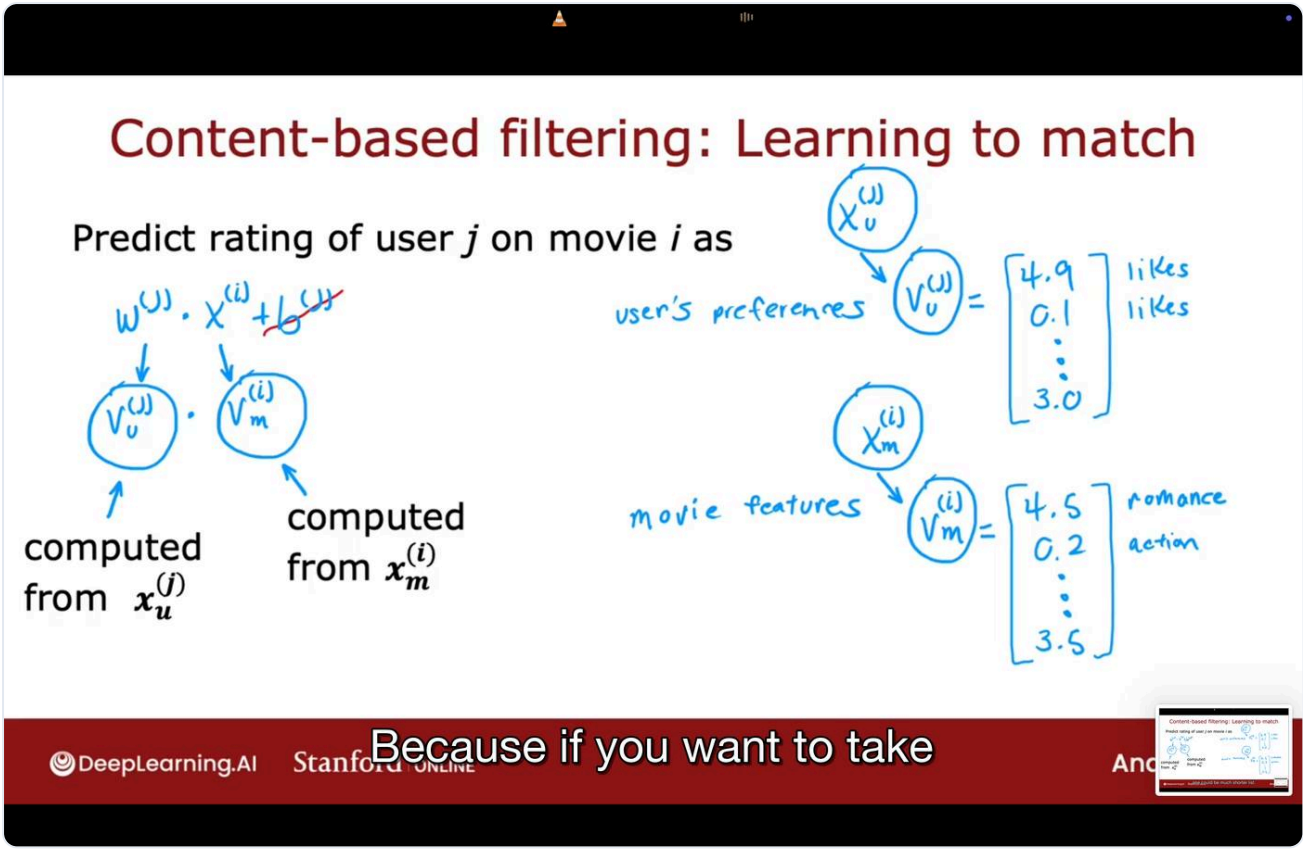


FIGURE 21.10 | Content-based Filtering: Learning to Match

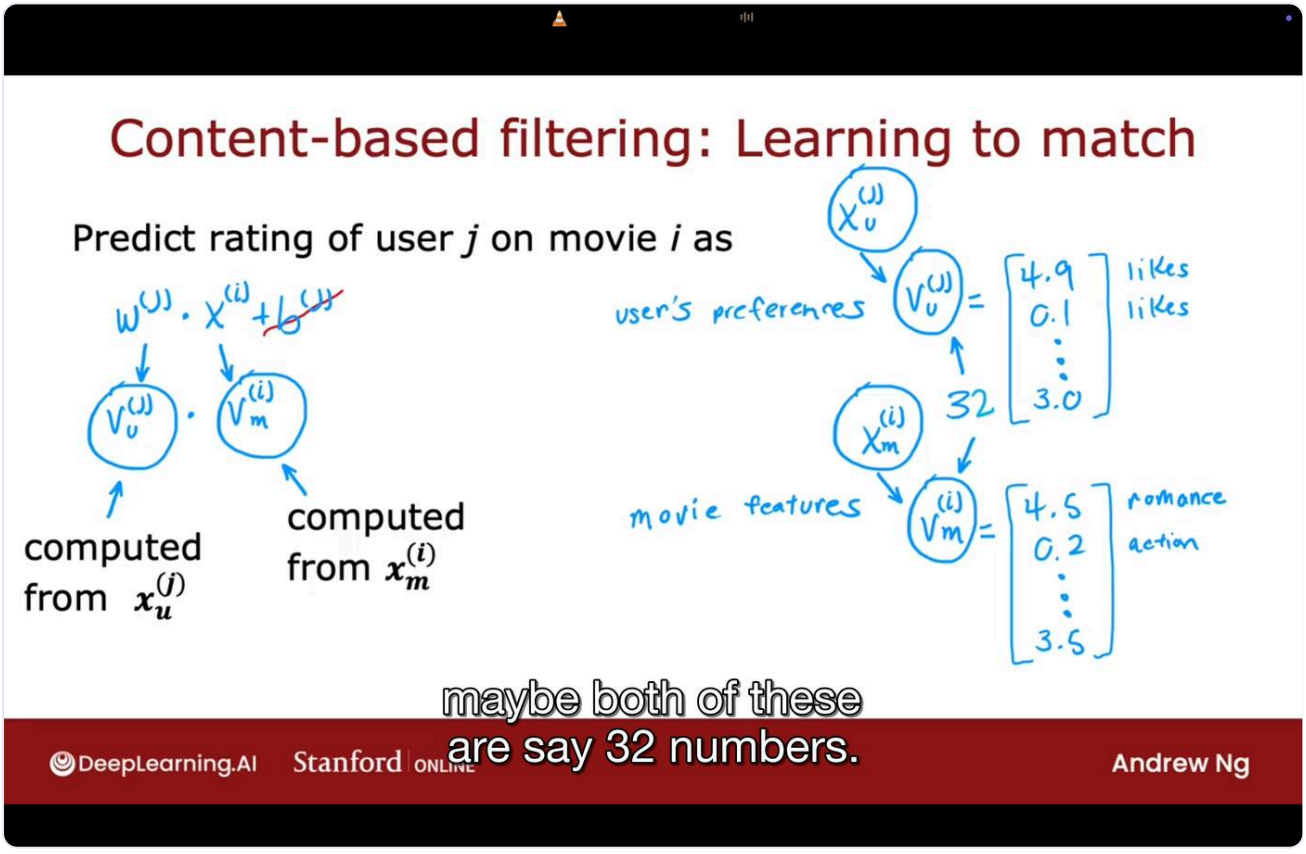


FIGURE 21.11 | Content-based Filtering: Learning to Match

Because if you want to take a dot product between v_u and v_m, then both of them have to have the same dimensions such as maybe both of these are say 32 numbers. To summarize, in collaborative filtering, we had number of users give ratings of different items. In contrast, in content-based filtering, we have features of users and features of items and we want to find a way to find good matches between the users and the items. The way we're going to do so is to compute these vectors, v_u for the users and v_m for the items over the movies, and then take dot products between them to try to find good matches. How do we compute the v_u and v_m? Let's take a look at that in the next video.

LECTURE 22

Neural Network Architecture

A good way to develop a content-based filtering algorithm is to use deep learning. The approach you see in this video is the way that many important commercial state-of-the-art content-based filtering algorithms are built today. Let's take a look. Recall that in our approach, given a feature vector describing a user, such as age and gender, and country, and so on, we have to compute the vector v_u, and similarly, given a vector describing a movie such as year of release, the stars in the movie, and so on, we have to compute a vector v_m. In order to do the former, we're going to use a neural network. The first neural network will be what we'll call the user network.

Here's an example of user network, that takes as input the list of features of the user, x_u, so the age, the gender, the country of the user, and so on. Then using a few layers, say dense neural network layers, it will output this vector v_u that describes the user. Notice that in this neural network, the output layer has 32 units, and so v_u is actually a list of 32 numbers. Unlike most of the neural networks that we were using earlier, the final layer is not a layer with one unit, it's a layer with 32 units. Similarly, to compute v_m for a movie, we can have a movie network as follows, that takes as input features of the movie and through a few layers of a neural network is outputting v_m, that vector that describes the movie.

Finally, we'll predict the rating of this user on that movie as v_ u dot product with v_m.

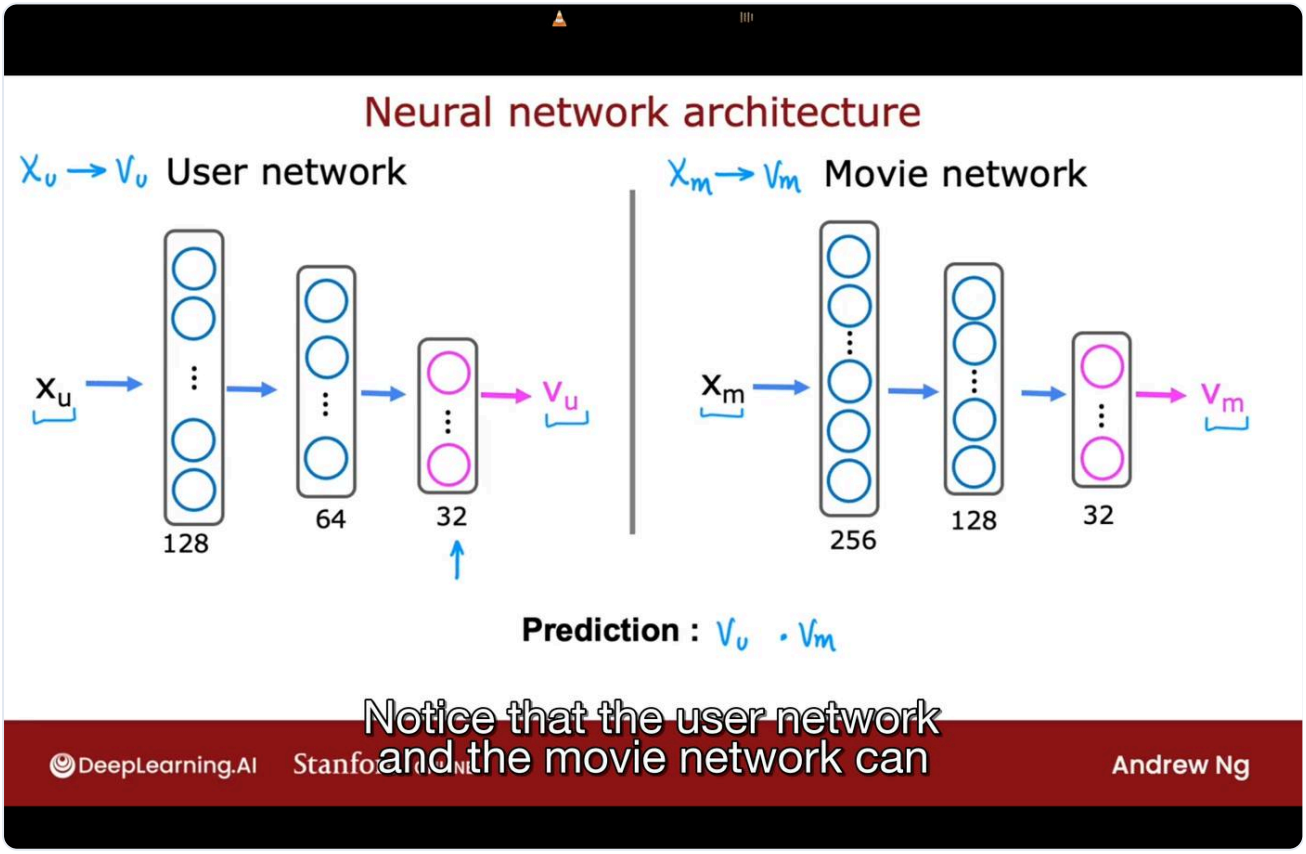


FIGURE 22.1 | Neural Network Architecture

Notice that the user network and the movie network can hypothetically have different numbers of hidden layers and different numbers of units per hidden layer. All the output layer needs to have the same size of the same dimension.

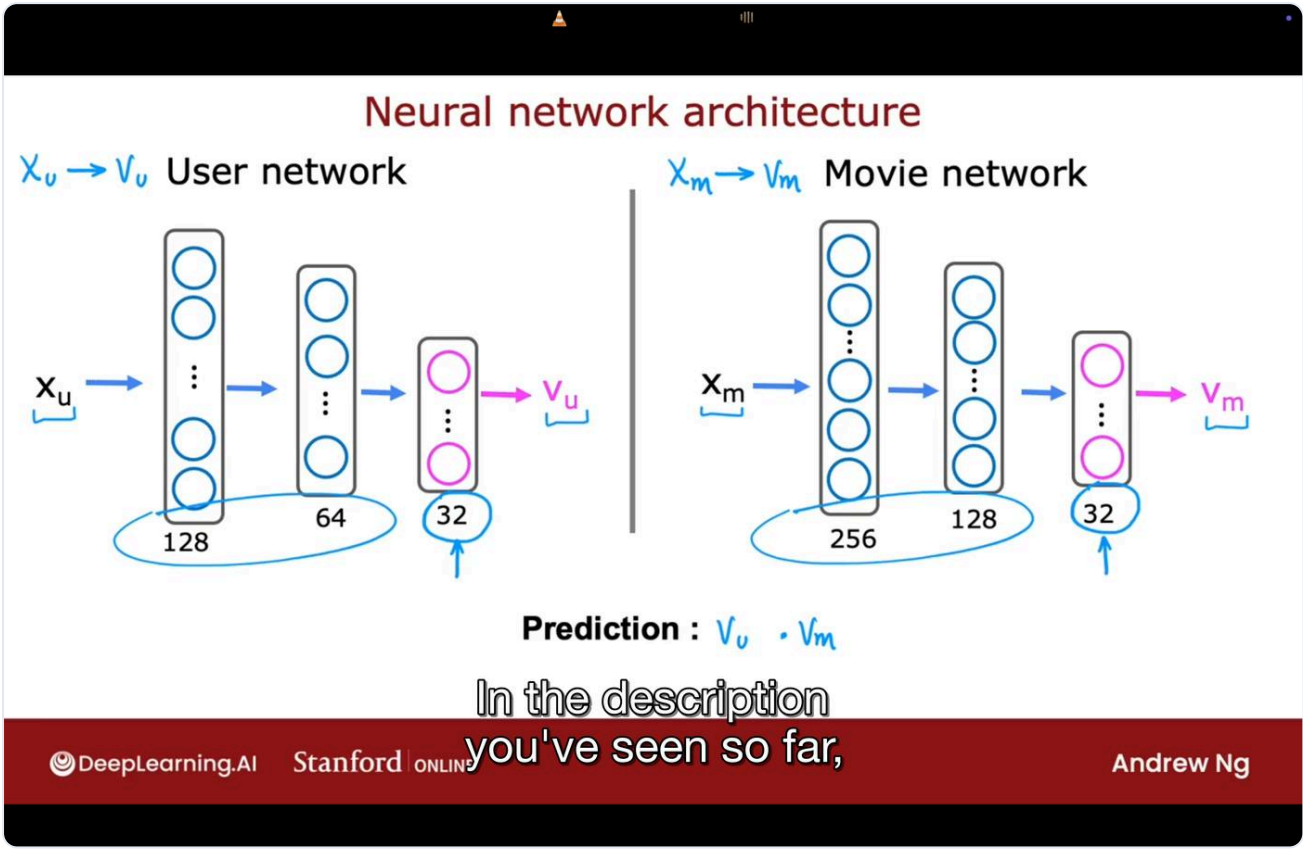


FIGURE 22.2 | Neural Network Architecture

In the description you've seen so far, we were predicting the 1-5 or 0-5 star movie rating. If we had binary labels, if y was to the user like or favor an item, then you can also modify this algorithm to output.

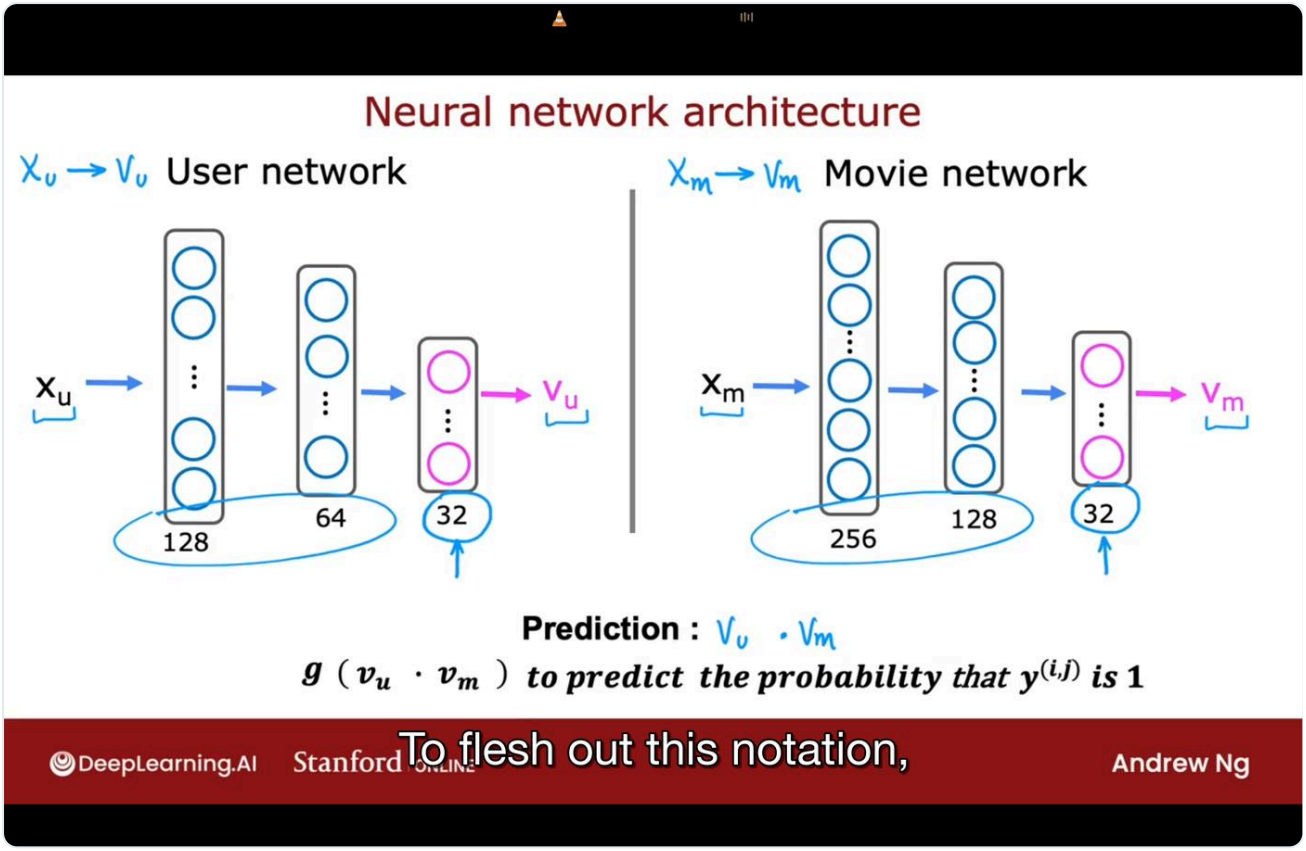


FIGURE 22.3 | Neural Network Architecture

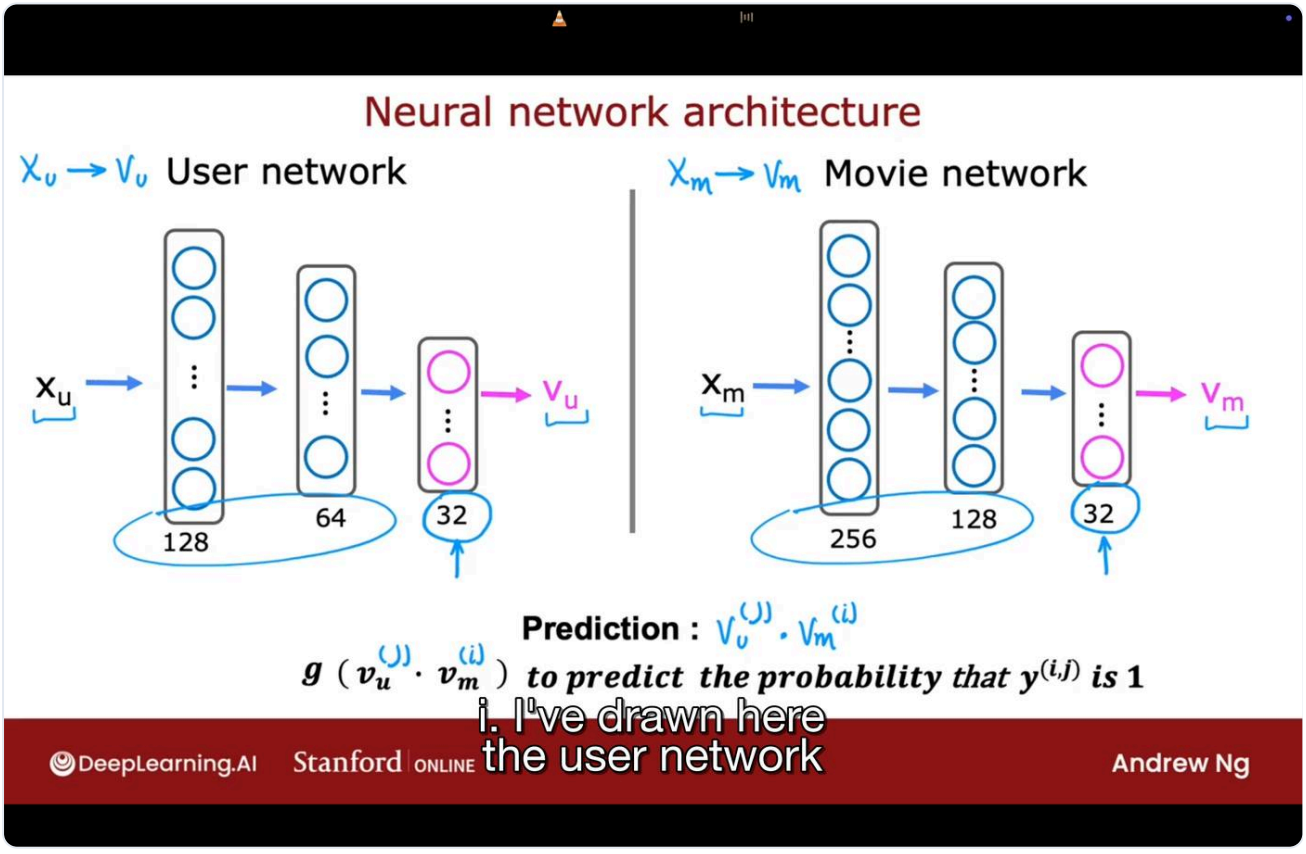


FIGURE 22.4 | Neural Network Architecture

Instead of v_u.v_m, you can apply the sigmoid function to that and use this to predict the probability that's y^i,j is 1. To flesh out this notation, we can also add superscripts i and j here if we want to emphasize that this is the prediction by user j on movie i. I've drawn here the user network and the movie network as two separate neural networks. But it turns out that we can actually draw them together in a single diagram as if it was a single neural network. This is what it looks like. On the upper portion of this diagram, we have the user network which inputs x_u and ends up computing v_u.

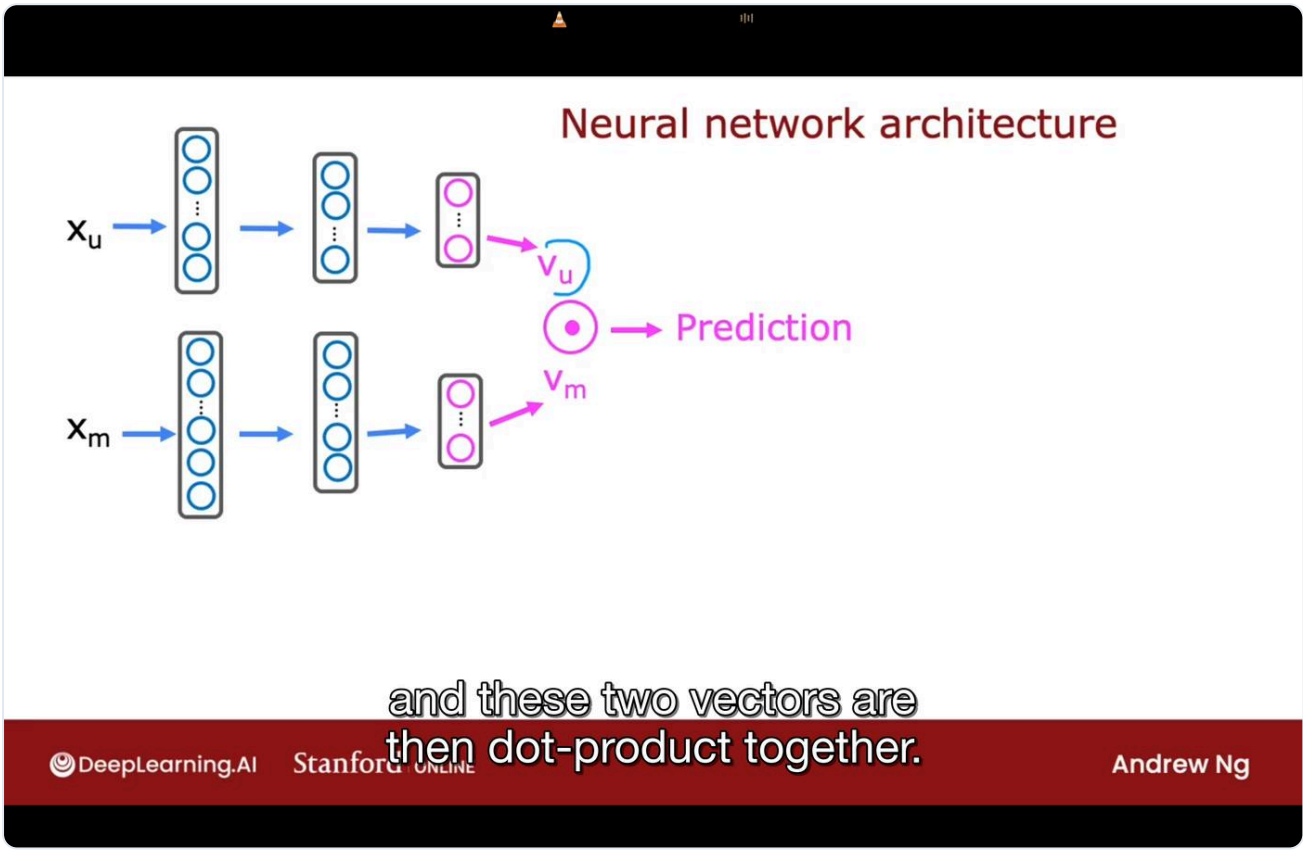


FIGURE 22.5 | Neural Network Architecture

On the lower portion of this diagram, we have what was the movie network, the input is x_m and ends up computing v_m, and these two vectors are then dot-product together. This dot here represents dot product, and this gives us our prediction. Now, this model has a lot of parameters.

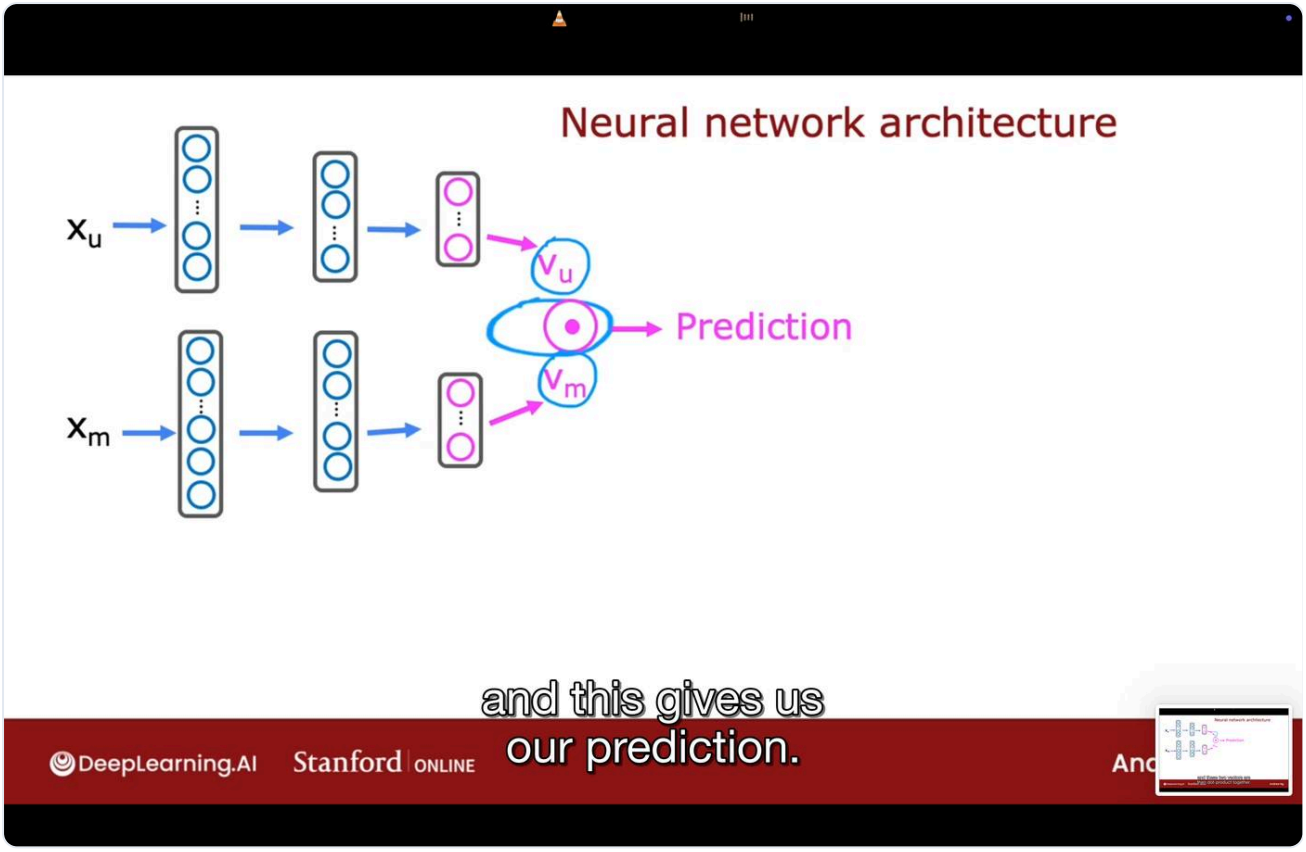


FIGURE 22.6 | Neural Network Architecture

Each of these layers of a neural network has a usual set of parameters of the neural network. How do you train all the parameters of both the user network and the movie network? What we're going to do is construct a cost function J, which is going to be very similar to the cost function that you saw in collaborative filtering, which is assuming that you do have some data of some users having rated some movies, we're going to sum over all pairs i and j of where you have labels, where i,j equals 1 of the difference between the prediction.

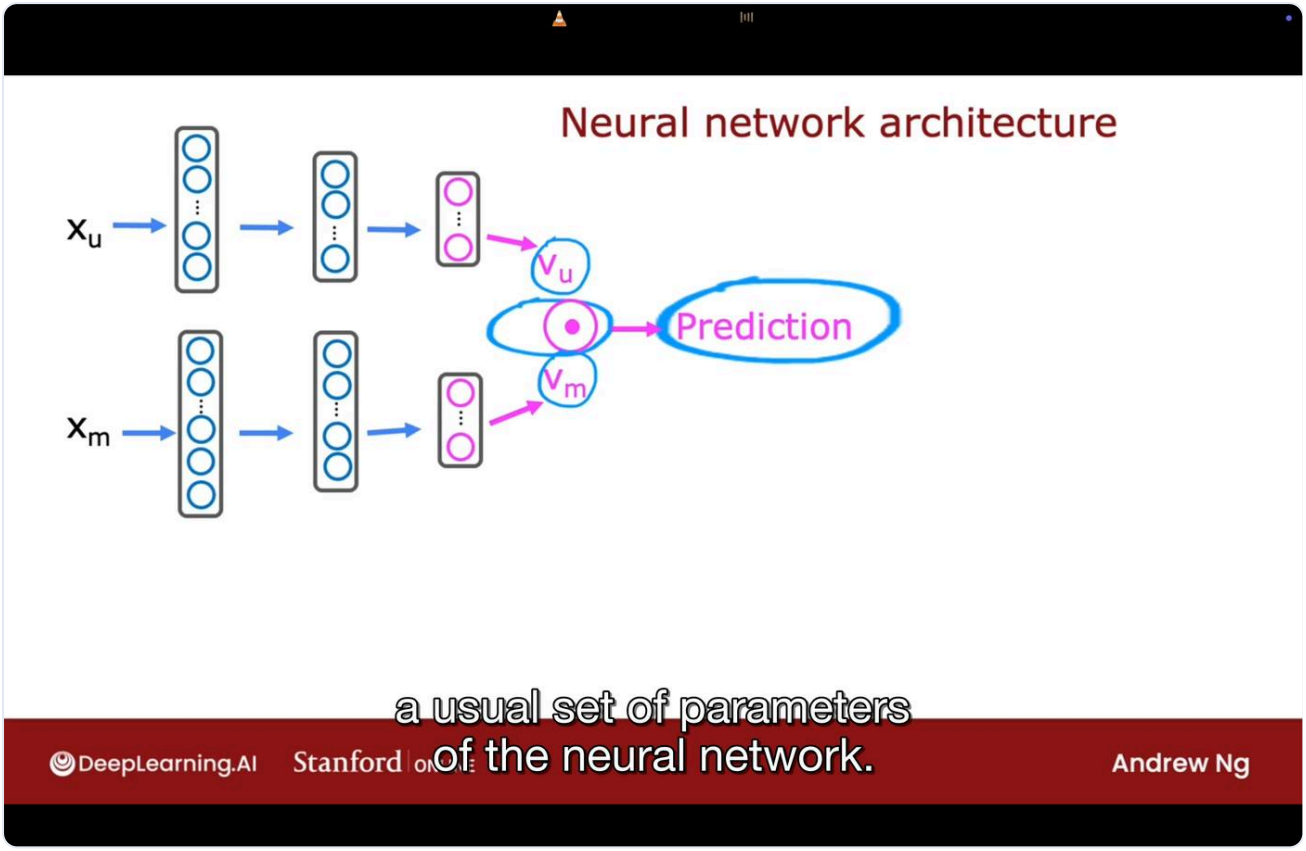


FIGURE 22.7

That would be v_u^j dot product with v_m^i minus y^ij squared. The way we would train this model is depending on the parameters of the neural network, you end up with different vectors here for the users and for the movies. What we'd like to do is train the parameters of the neural network so that you end up with vectors for the users and for the movies that results in small squared error into predictions you get out here.

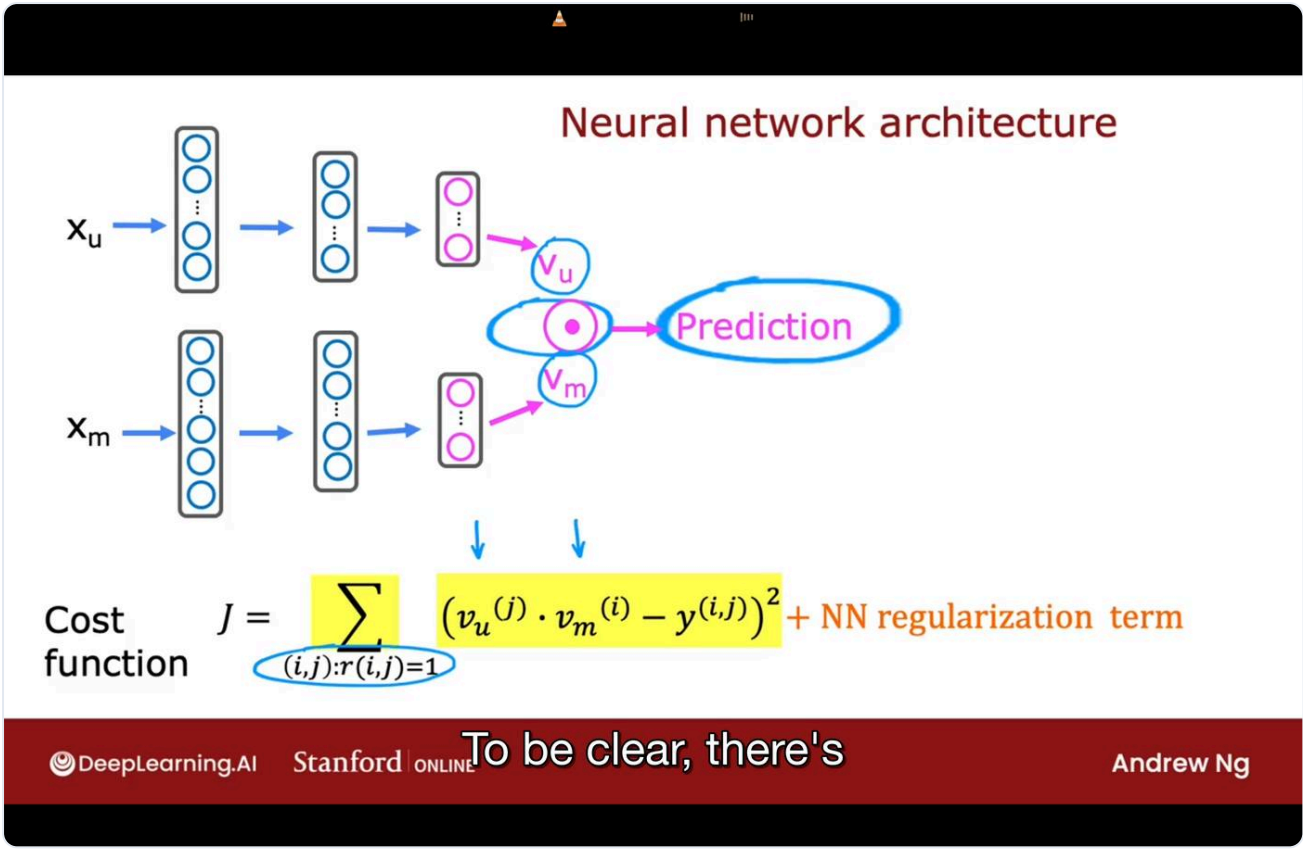


FIGURE 22.8 | Neural Network Architecture

To be clear, there's no separate training procedure for the user and movie networks. This expression down here, this is the cost function used to train all the parameters of the user and the movie networks. We're going to judge the two networks according to how well v_u and v_m predict y^ij, and with this cost function, we're going to use gradient descent or some other optimization algorithm to tune the parameters of the neural network to cause the cost function J to be as small as possible. If you want to regularize this model, we can also add the usual neural network regularization term to encourage the neural networks to keep the values of their parameters small.

It turns out, after you've trained this model, you can also use this to find similar items. This is akin to what we have seen with collaborative filtering features, helping you find similar items as well. Let's take a look. V_u^j is a vector of length 32 that describes a user j that have features x_u^j. Similarly, v^i_m is a vector of length 32 that describes a movie with these features over here. Given a specific movie, what if you want to find other movies similar to it? Well, this vector v^i_m describes the movie i. If you want to find other movies similar to it, you can then look for other movies k so that the distance between the vector describing movie k and the vector describing movie i, that the squared distance is small.

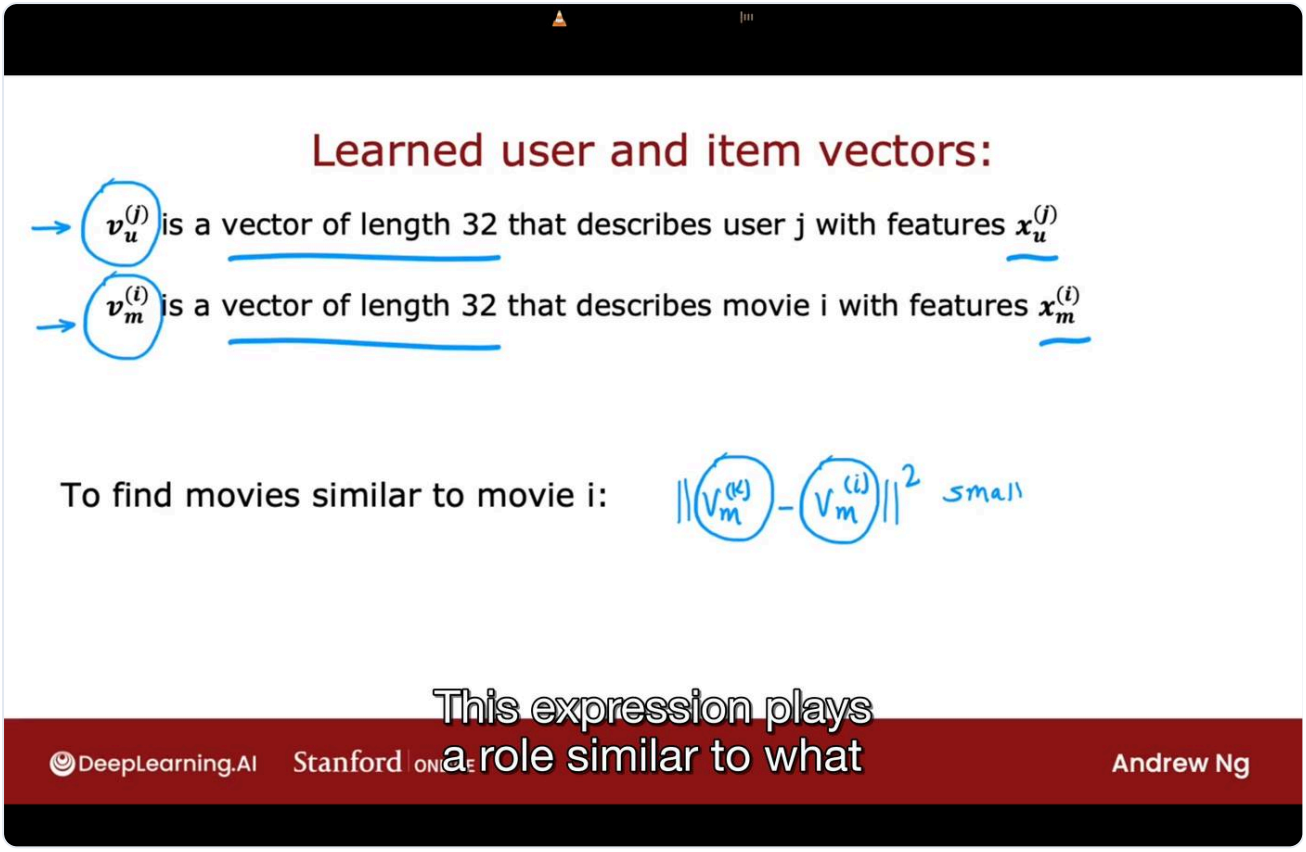


FIGURE 22.9 | Learned User and Item Vectors:

This expression plays a role similar to what we had previously with collaborative filtering, where we talked about finding a movie with features x^k that was similar to the features x^i.

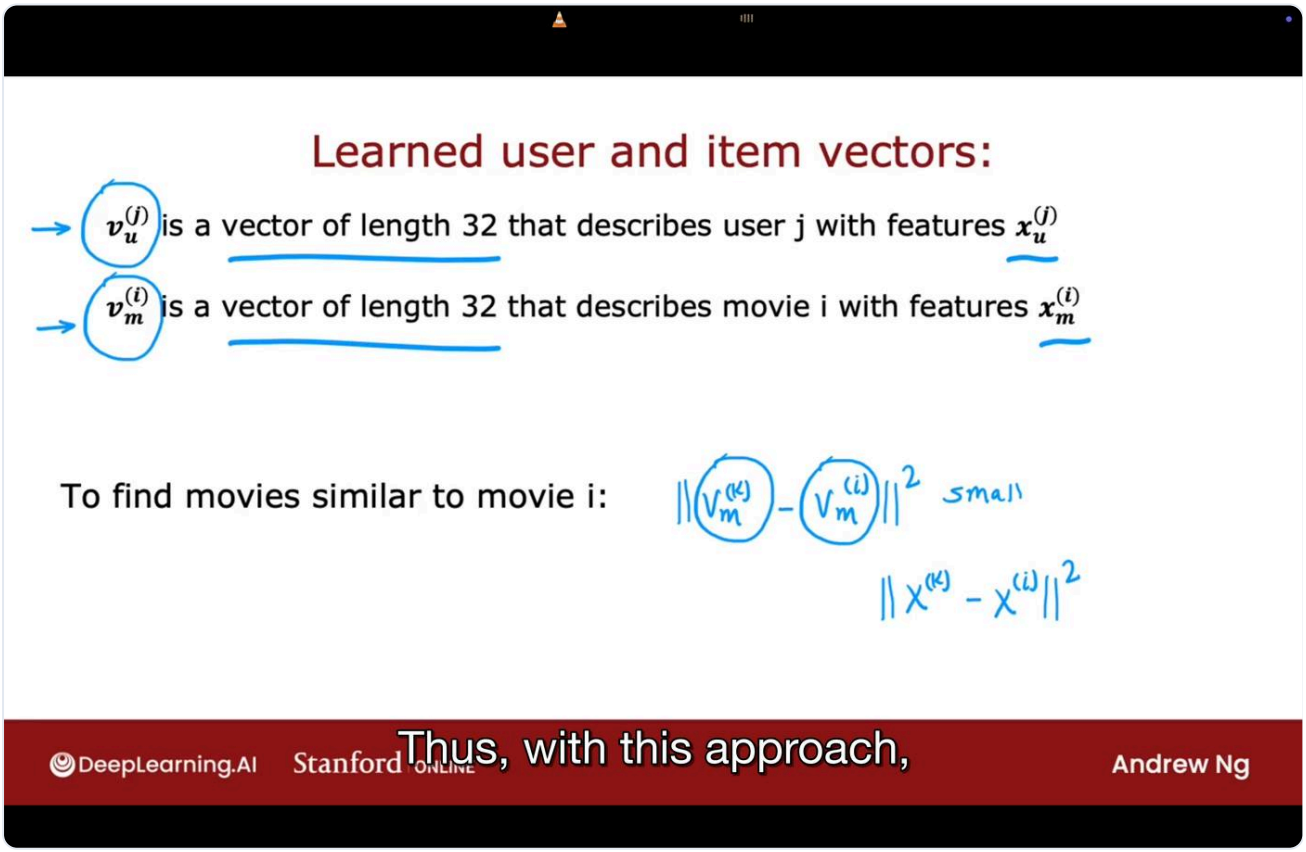


FIGURE 22.10 | Learned User and Item Vectors:

Thus, with this approach, you can also find items similar to a given item.

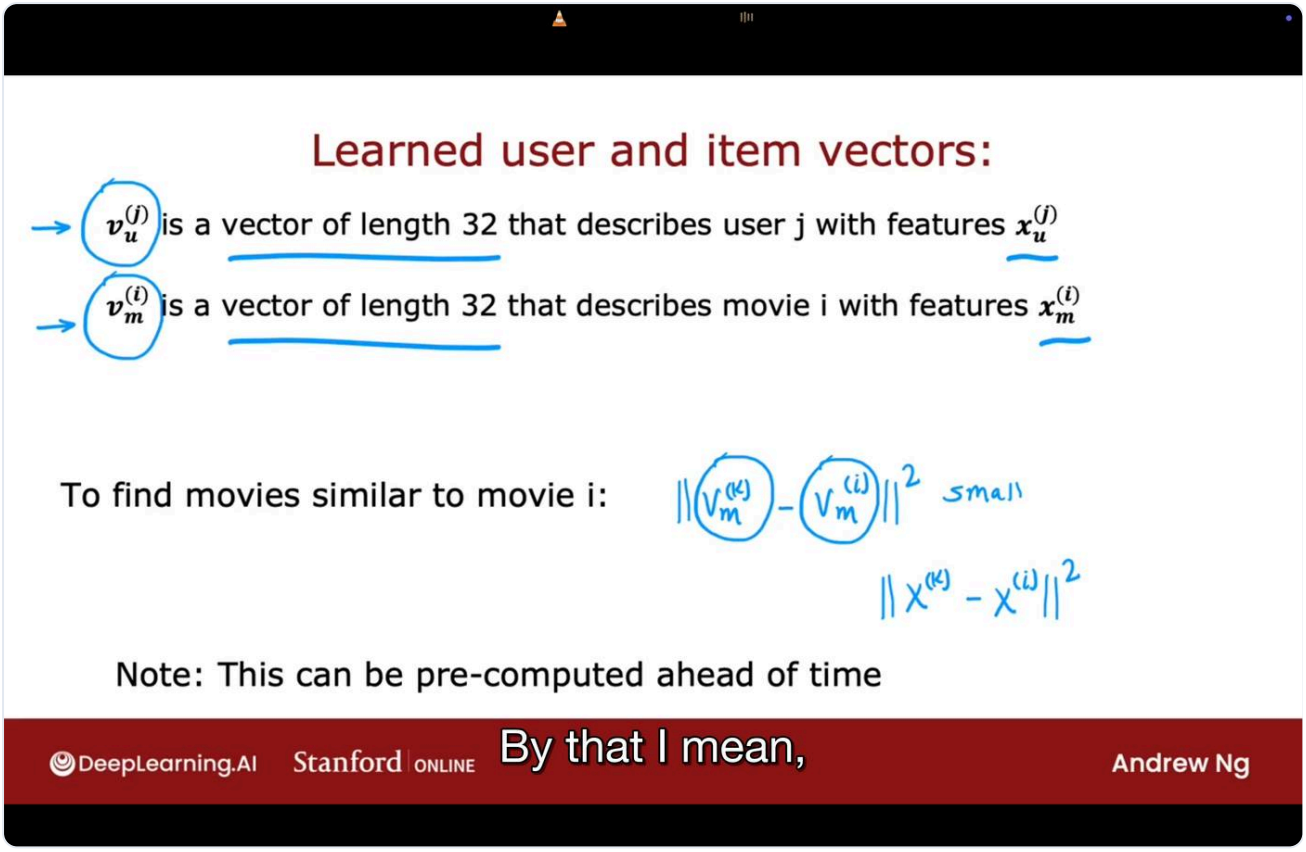


FIGURE 22.11 | Learned User and Item Vectors:

One final note, this can be pre-computed ahead of time. By that I mean, you can run a compute server overnight to go through the list of all your movies and for every movie, find similar movies to it, so that tomorrow, if a user comes to the website and they're browsing a specific movie, you can already have pre-computed to 10 or 20 most similar movies to show to the user at that time. The fact that you can pre-compute ahead of time what's similar to a given movie, will turn out to be important later when we talk about scaling up this approach to a very large catalog of movies. That's how you can use deep learning to build a content-based filtering algorithm.

You might remember when we were talking about decision trees and the pros and cons of decision trees versus neural networks. I mentioned that one of the benefits of neural networks is that it's easier to take multiple neural networks and put them together to make them work in concert to build a larger system. What you just saw was actually an example of that, where we could take a user network and the movie network and put them together, and then take the inner product of the outputs. This ability to put two neural networks together this how we've managed to come up with a more complex architecture that turns out to be quite powerful. One notes, if you're implementing these algorithms in practice, I find that developers often end up spending a lot of time carefully designing the features needed to feed into these content-based filtering algorithms.

If we end up building one of these systems commercially, it may be worth spending some time engineering good features for this application as well. In terms of these applications, one limitation that the algorithm as we've described it is it can be computationally very expensive to run if you have a large catalog of a lot of different movies you may want to recommend. In the next

video, let's take a look at some of the practical issues and how you can modify this algorithm to make it scale that are working on even very large item catalogs. Let's go see that in the next video.

LECTURE 23

Retrieval Step

Today's recommender systems will sometimes need to pick a handful of items to recommend. From a catalog of thousands or millions or 10s of millions or even more items. How do you do this efficiently computationally, let's take a look. Here's in your network we've been using to make predictions about how a user might rate an item. Today a large movie streaming site may have thousands of movies or a system that is trying to decide what ad to show. May have a catalog of millions of ads to choose from. Or a music streaming sites may have 10s of millions of songs to choose from. And large online shopping sites can have millions or even 10s of millions of products to choose from.

When a user shows up on your website, they have some feature Xu. But if you need to take thousands of millions of items to feed through this neural network in order to compute in the product. To figure out which products you should recommend, then having to run neural network inference.

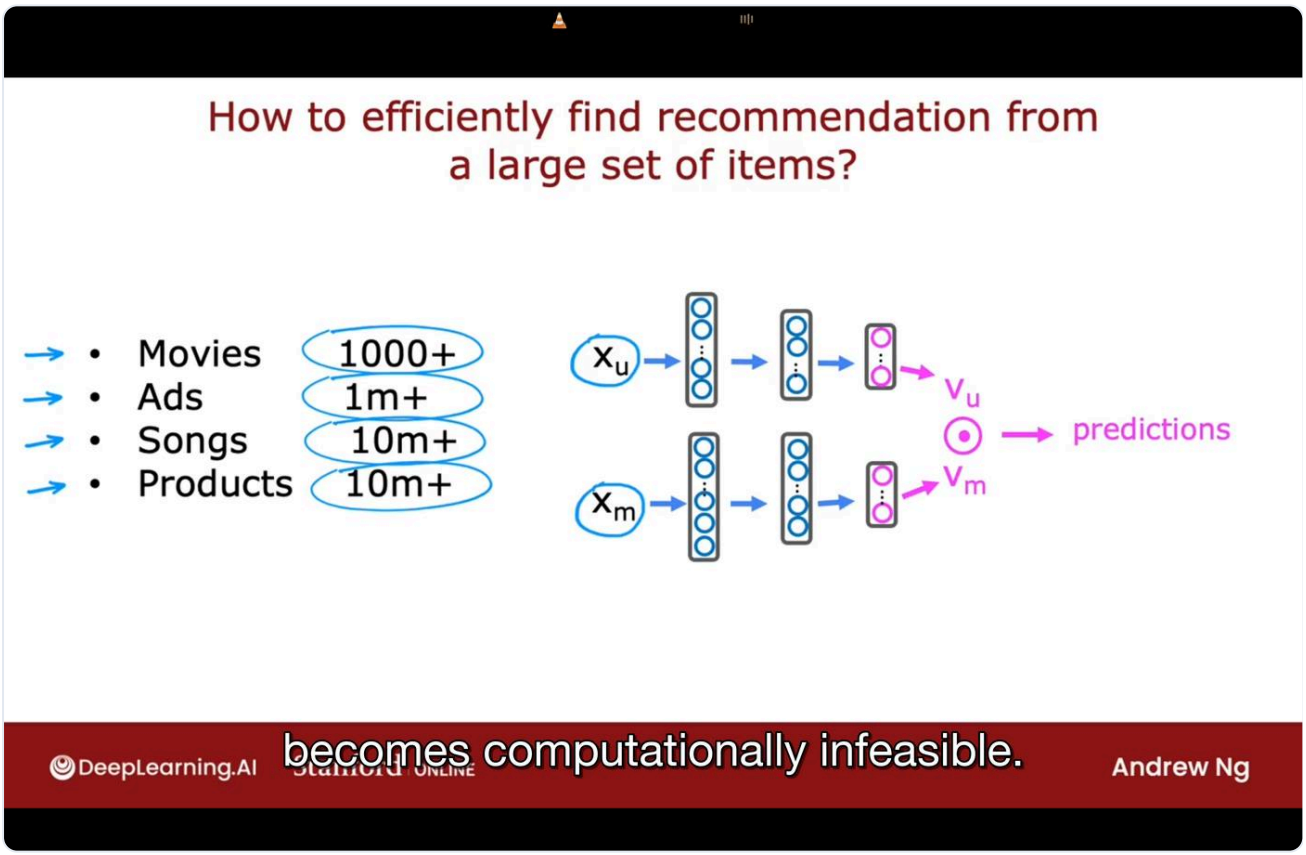


FIGURE 23.1 | How to Efficiently Find Recommendation from

Thousands of millions of times every time a user shows up on your website becomes computationally infeasible. Many law scale recommender systems are implemented as two steps which are called the retrieval and ranking steps. The idea is during the retrieval step will generate a large list of plausible item candidates. That tries to cover a lot of possible things you might recommend to the user and it's okay during the retrieval step. If you include a lot of items that

the user is not likely to like and then during the ranking step will fine tune and pick the best items to recommend to the user. So here's an example, during the retrieval step we might do something like.

For each of the last 10 movies that the user has watched find the 10 most similar movies.

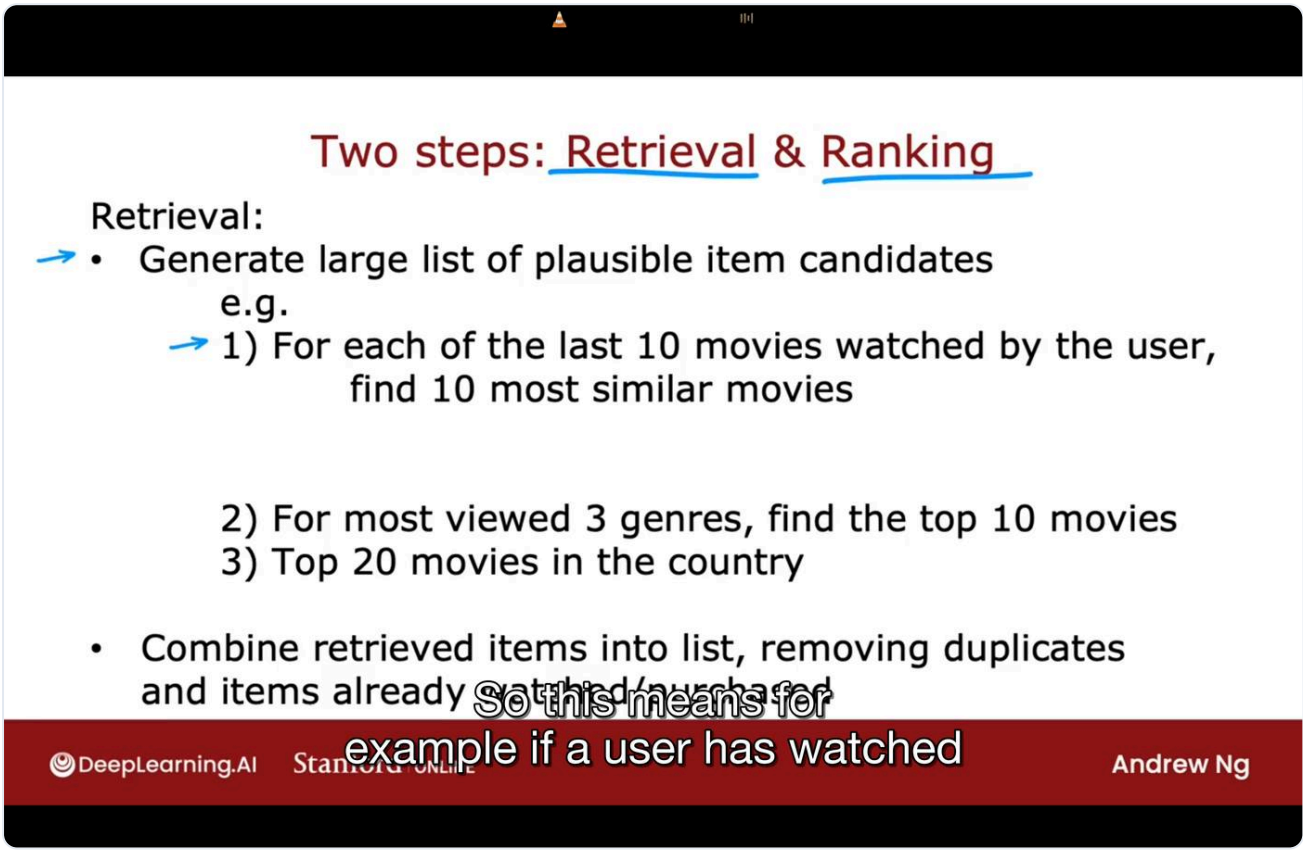


FIGURE 23.2

So this means for example if a user has watched the movie I with vector VIM you can find the movies hey with vector VKM that is similar to that. And as you saw in the last video finding the similar movies, the given movie can be pre computed. So having pre computed the most similar movies to give a movie, you can just pull up the results using a look up table. This would give you an initial set of maybe somewhat plausible movies to recommend to user that just showed up on your website. Additionally you might decide to add to it for whatever are the most viewed three genres of the user. Say that the user has watched a lot of romance movies and a lot of comedy movies and a lot of historical dramas.

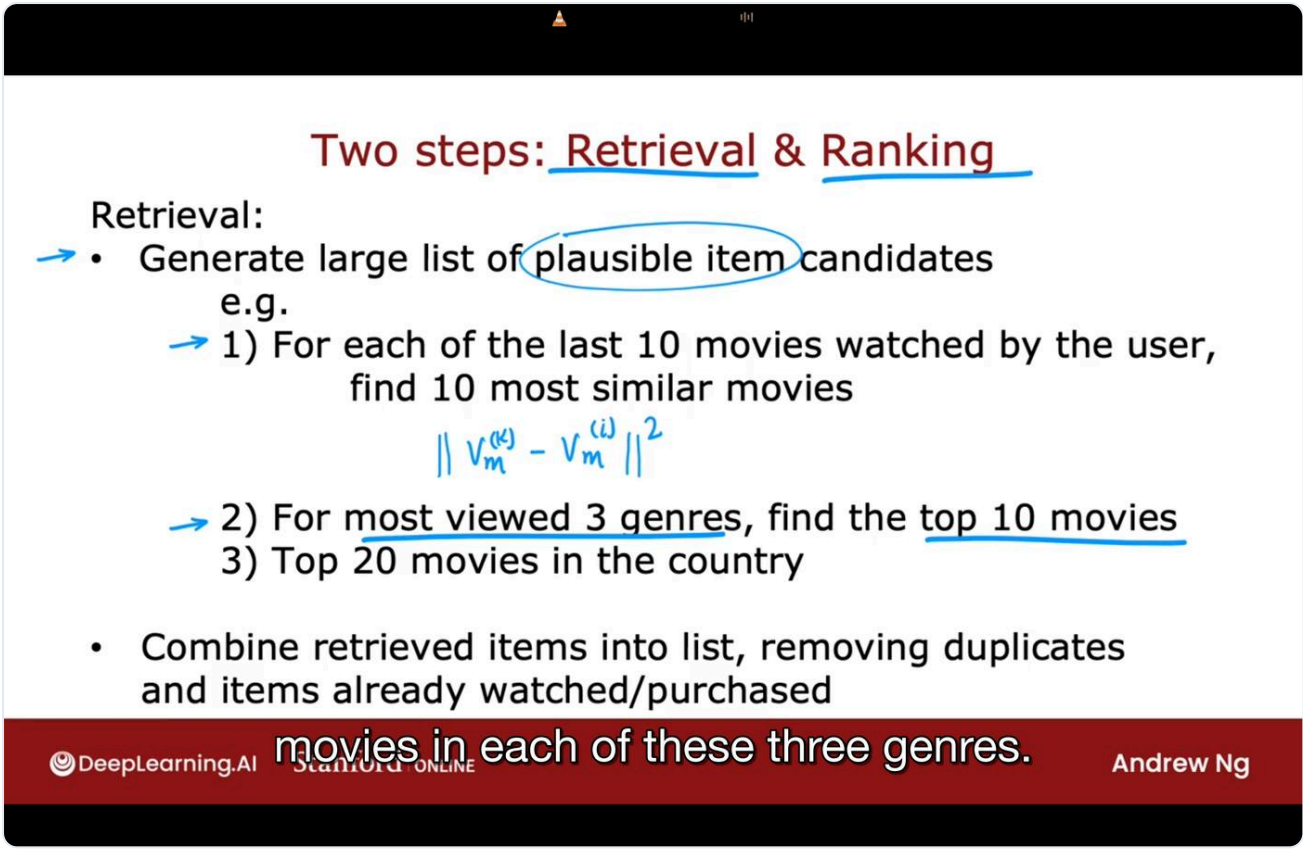


FIGURE 23.3

Then we would add to the list of possible item candidates the top 10 movies in each of these three genres. And then maybe we will also add to this list the top 20 movies in the country of the user. So this retrieval step can be done very quickly and you may end up with a list of 100 or maybe 100s of plausible movies. To recommend to the user and hopefully this list will recommend some good options. But it's also okay if it includes some options that the user won't like at all. The goal of the retrieval step is to ensure broad coverage to have enough movies at least have many good ones in there. Finally, we would then take all the items we retrieve during the retrieval step and combine them into a list.

Removing duplicates and removing items that the user has already watched or that the user has already purchased and that you may not want to recommend to them again. The second step of this is then the ranking step. During the ranking step you will take the list retrieved during the retrieval step. So this may be just hundreds of possible movies and rank them using the learned model. And what that means is you will feed the user feature vector and the movie feature actor into this neural network. And for each of the user movie pairs compute the predicted rating. And based on this, you now have all of the say 100 plus movies, the ones that the user is most likely to give a high rating to.

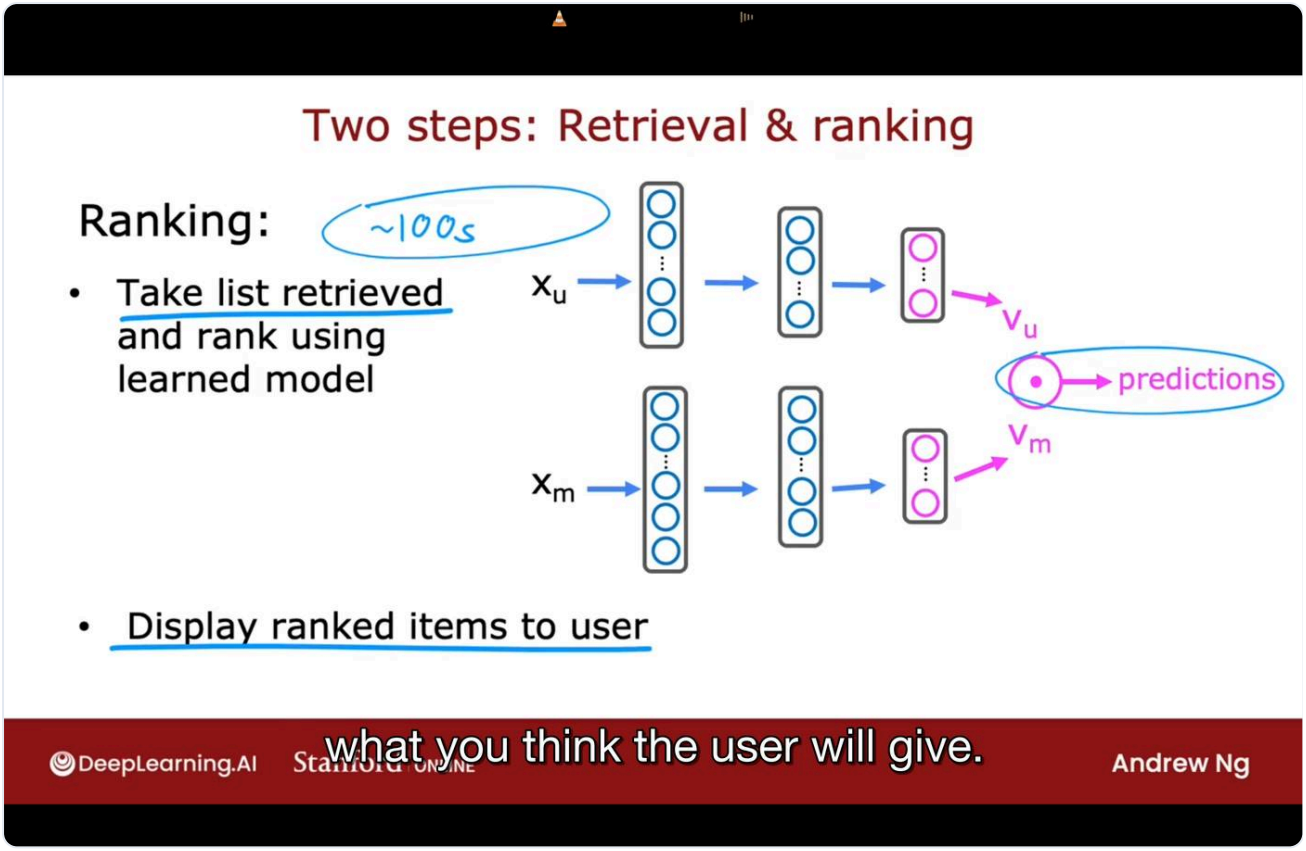


FIGURE 23.4 | Two Steps: Retrieval & Ranking

And then you can just display the rank list of items to the user depending on what you think the user will give. The highest rating to one additional optimization is that if you have computed VM. For all the movies in advance, then all you need to do is to do inference on this part of the neural network a single time to compute VU. And then take that VU they just computed for the user on your website right now. And take the inner product between VU and VM. For the movies that you have retrieved during the retrieval step. So this computation can be done relatively quickly. If the retrieval step just brings up say 100s of movies, one of the decisions you need to make for this algorithm is how many items do you want to retrieve during the retrieval step?

To feed into the more accurate ranking step. During the retrieval step, retrieving more items will tend to result in better performance. But the algorithm will end up being slower to analyze or to optimize the trade off between how many items to retrieve to retrieve 100 or 500 or 1000 items. I would recommend carrying out offline experiments to see how much retrieving additional items results in more relevant recommendations. And in particular, if the estimated probability that YIJ. Is equal to one according to your neural network model.

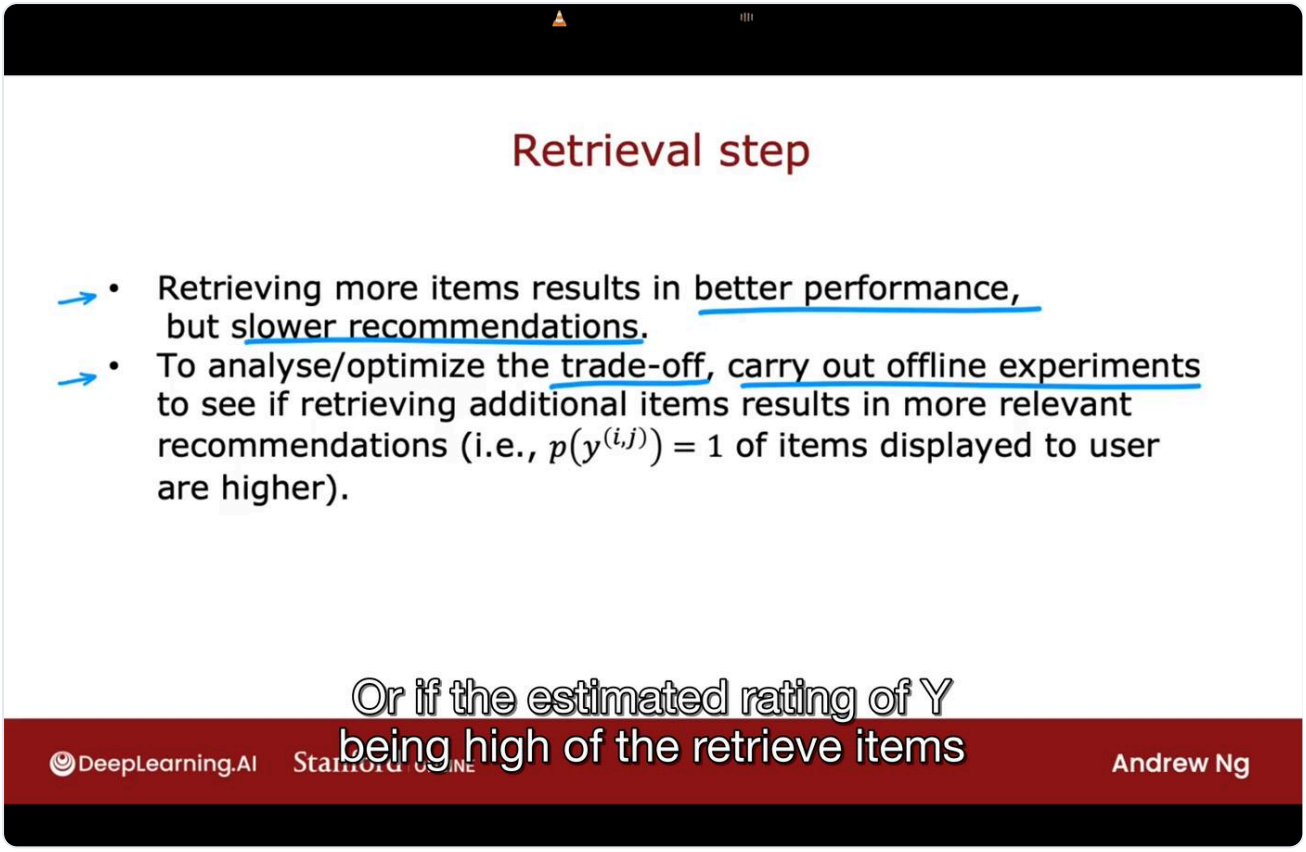


FIGURE 23.5 | Retrieval Step

Or if the estimated rating of Y being high of the retrieve items according to your model's prediction ends up being much higher. If only you were to retrieve say 500 items instead of only 100 items, then that would argue for maybe retrieving more items. Even if it slows down the algorithm a bit. But with the separate retrieval step and the ranking step, this allows many recommender systems today to give both fast as well as accurate results. Because the retrieval step tries to prune out a lot of items that are just not worth doing the more detailed influence and inner product on. And then the ranking step makes a more careful prediction for what are the items that the user is actually likely to enjoy so that's it.

This is how you make your recommender system work efficiently even on very large catalogs of movies or products or what have you. Now, it turns out that as commercially important as our recommender systems, there are some significant ethical issues associated with them as well. And unfortunately there have been recommender systems that have created harm. So as you build your own recommender system, I hope you take an ethical approach and use it to serve your users. And society as large as well as yourself and the company that you might be working for. Let's take a look at the ethical issues associated with recommender systems in the next video

LECTURE 24

Travel Industry Payday Loans

Even though recommender systems have been very profitable for some businesses, that happens, some use cases that have left people and society at large worse off. However, you use recommender systems or for that matter other learning algorithms, I hope you only do things that make society at large and people better off. Let's take a look at some of the problematic use cases of recommender systems, as well as ameliorations to reduce harm or to increase the amount of good that they can do. As you've seen in the last few videos, there are many ways of configuring a recommender system. When we saw binary labels, the label y could be, does a user engage or did they click or did they explicitly like an item?

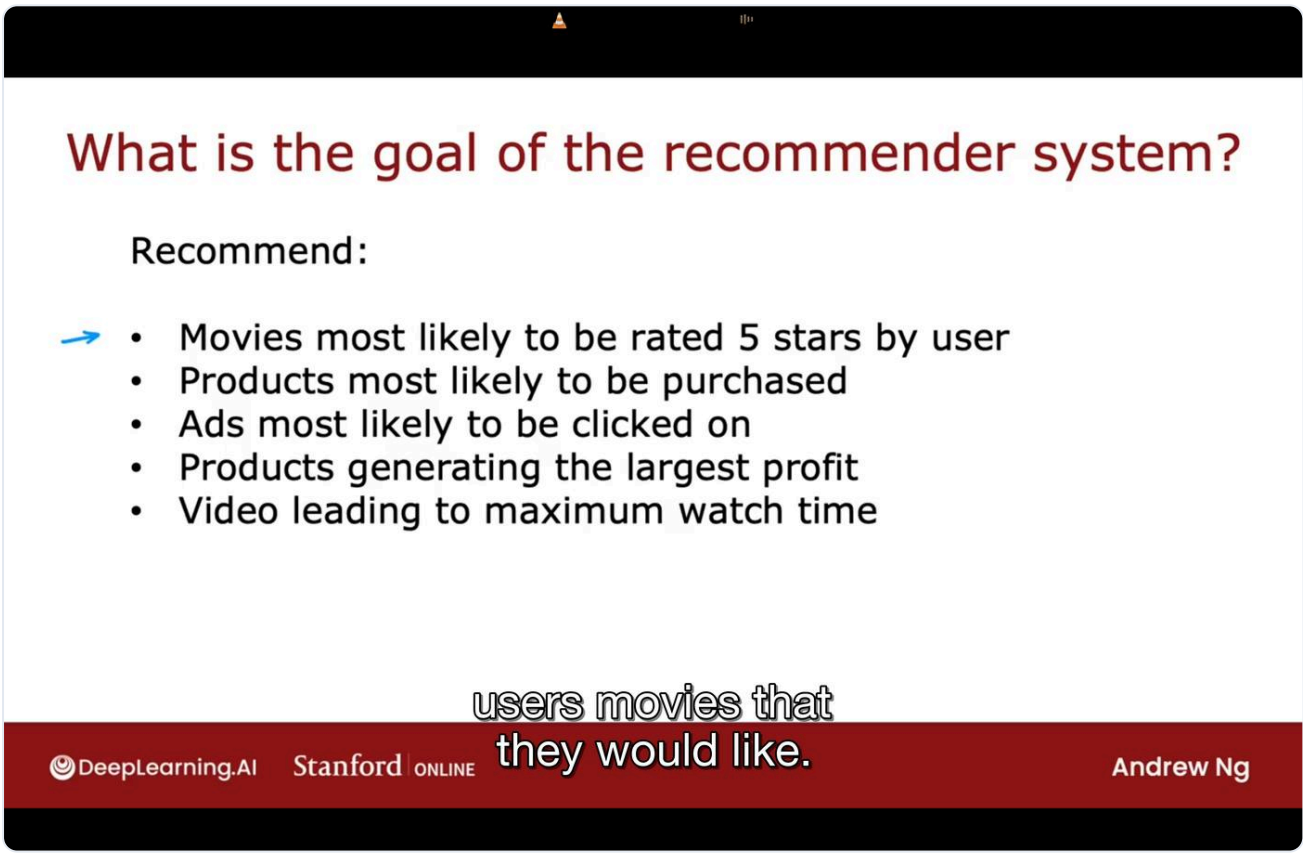


FIGURE 24.1 | What Is the Goal of the Recommender System?

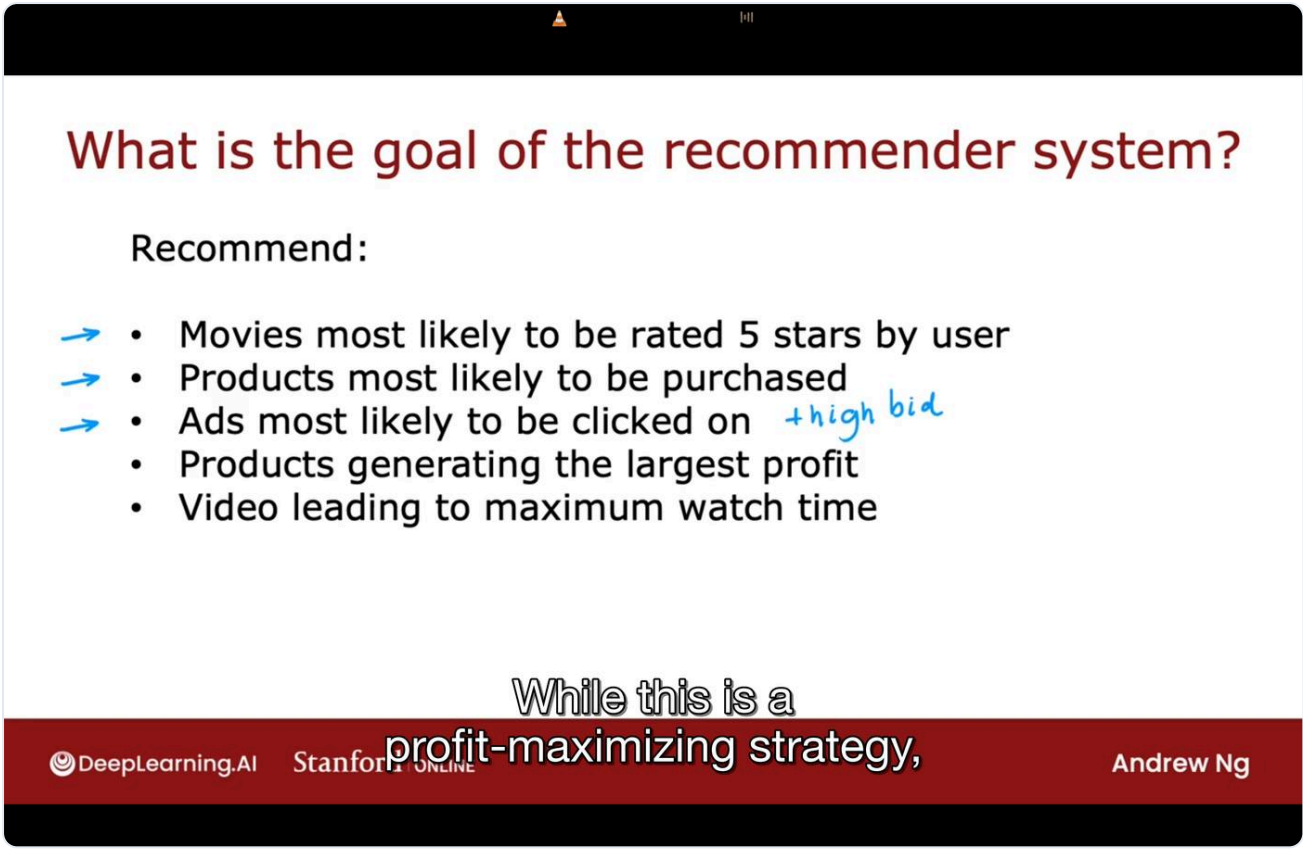


FIGURE 24.2 | What Is the Goal of the Recommender System?

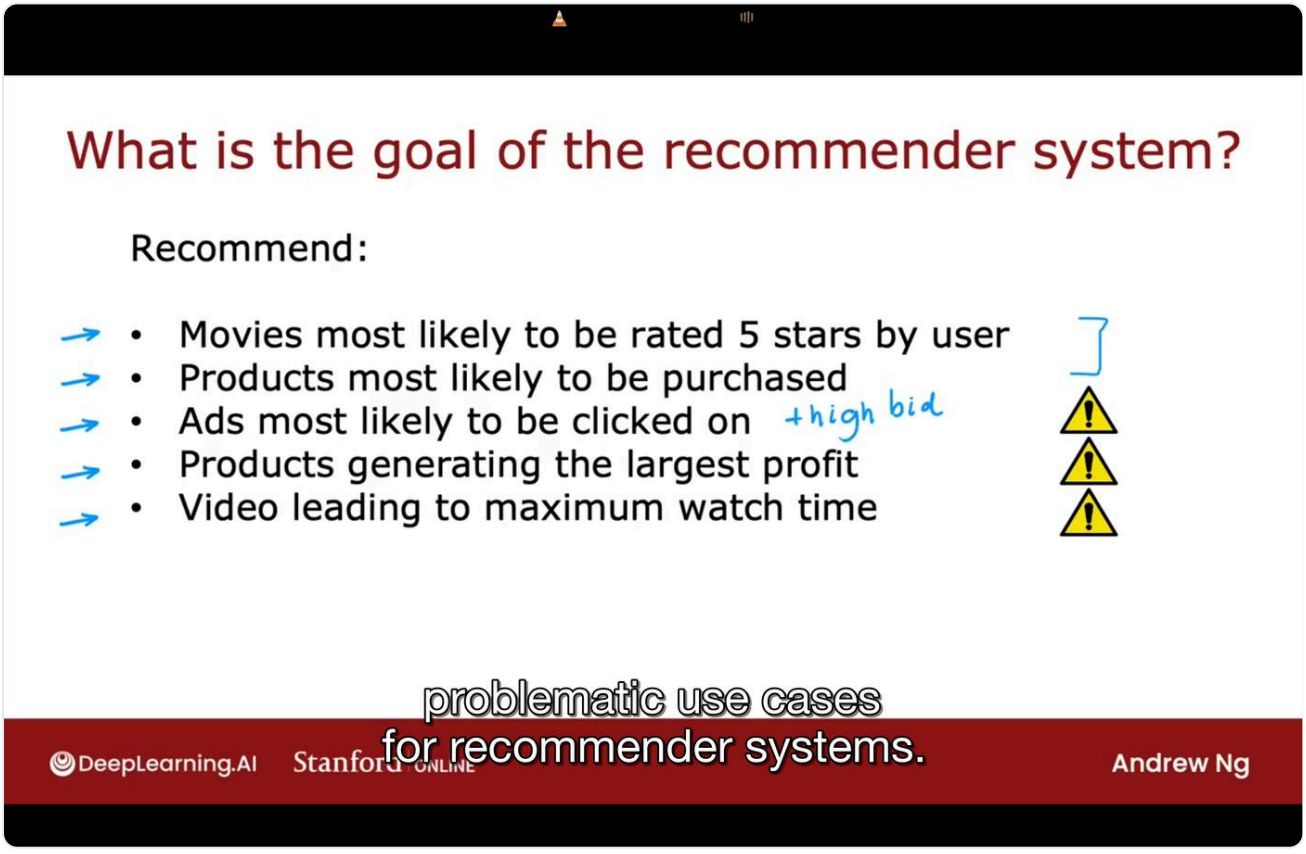


FIGURE 24.3 | What Is the Goal of the Recommender System?

When designing a recommender system, choices in setting the goal of the recommender system and a lot of choices and deciding what to recommend to users. For example, you can decide to recommend to users movies most likely to be rated five stars by that user. That seems fine. That seems like a fine way to show users movies that they would like. Or maybe you can recommend to the user products that they are most likely to purchase. That seems like a very reasonable use of a recommender system as well. Versions of recommender systems can also be used to decide what ads to show to a user. One thing you could do is to recommend or really to show to the user ads that are most likely to be clicked on.

Actually, what many companies will do is try to show ads that are likely to be clicked on and where the advertiser had put in a high bid because for many ad models, the revenue that the company collects depends on whether the ad was clicked on and what the advertiser had bid per-click. While this is a profit-maximizing strategy, there are also some possible negative implications of this type of advertising. I'll give a specific example on the next slide. One other thing that many companies do is try to recommend products that generate the largest profit. If you go to a website and search for a product today, there are many websites that are not showing you the most relevant product or the product that you are most likely to purchase.

But is instead trying to show you the products that will generate the largest profit for the company. If a certain product is more profitable for them, because they can buy it more cheaply and sell it at a higher price, that gets ranked higher in the recommendations. Now, many companies view a pressure to maximize profit. This doesn't seem like an unreasonable thing to do but on the flip side, from the user perspective, when a website recommends to you a product, sometimes it feels it could be nice if the website was transparent with you about the criteria by which it is deciding what to show you. Is it trying to maximize their profits or trying to show you things that are most useful to you?

On video websites or social media websites, a recommender system can also be modified to try to show you the content that leads to the maximum watch time. Specifically, websites that are an ad revenue tend to have an incentive to keep you on the website for a long time. Trying to maximize the time you spend on the site is one way for the site to try to get more of your time so they can show you more ads. Recommender systems today are used to try to maximize user engagement or to maximize the amount of time that someone spends on a site or a specific app. Whereas the first two of these seem quite innocuous, the third, fourth, and fifth, they may be just fine.

They may not cause any harm at all. Or they could also be problematic use cases for recommender systems. Let's take a deeper look at some of these potentially problematic use cases. Let me start with the advertising example. It turns out that the advertising industry can sometimes be an amplifier of some of the most harmful businesses. They can also be an amplifier of some of the best and the most fruitful businesses. Let me illustrate with a good example and a bad example. Take the travel industry. I think in the travel industry, the way to succeed is to try to give good travel experiences to users, to really try to serve users. Now it turns out that if there's a really good travel company, they can sell you a trip to fantastic destinations and make sure you and your friends and family have a lot of fun.

Then a good travel business, I think will often end up being more profitable. The other business is more profitable. They can then bid higher for ads. It can afford to pay more to get users. Because it can afford to bid higher for ads an online advertising site will show its ads more often and drive more users to this good company. This is a virtuous cycle where the more users you serve well, the more profitable the business, and the more you can bid more for ads and the more traffic you get and so on. Just virtuous circle will maybe even tend to help the good travel companies do even better statistically example. Let's look at the problematic example.

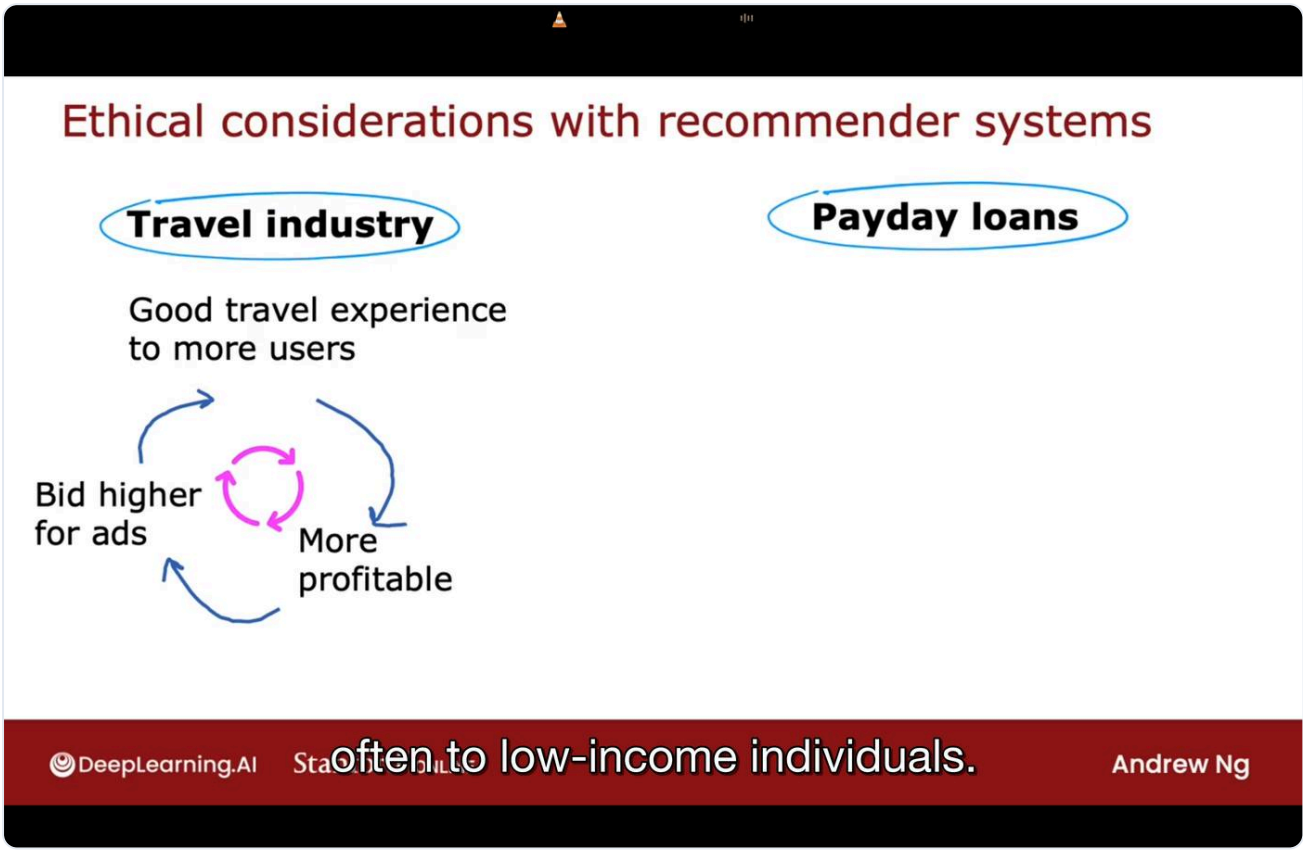


FIGURE 24.4 | Travel Industry Payday Loans

The payday loan industry tends to charge extremely high-interest rates, often to low-income individuals. One of the ways to do well in the payday loan business is to be really efficient as squeezing customers for every single dollar you can get out of them. If there's a payday loan company that is very good at exploiting customers, really squeezing customers for every single dollar, then that company will be more profitable. Thus they can be higher for ads. Because they can get bid higher for ads they will get more traffic sent to them. This allows them to squeeze even more customers and explore even more people for profit.

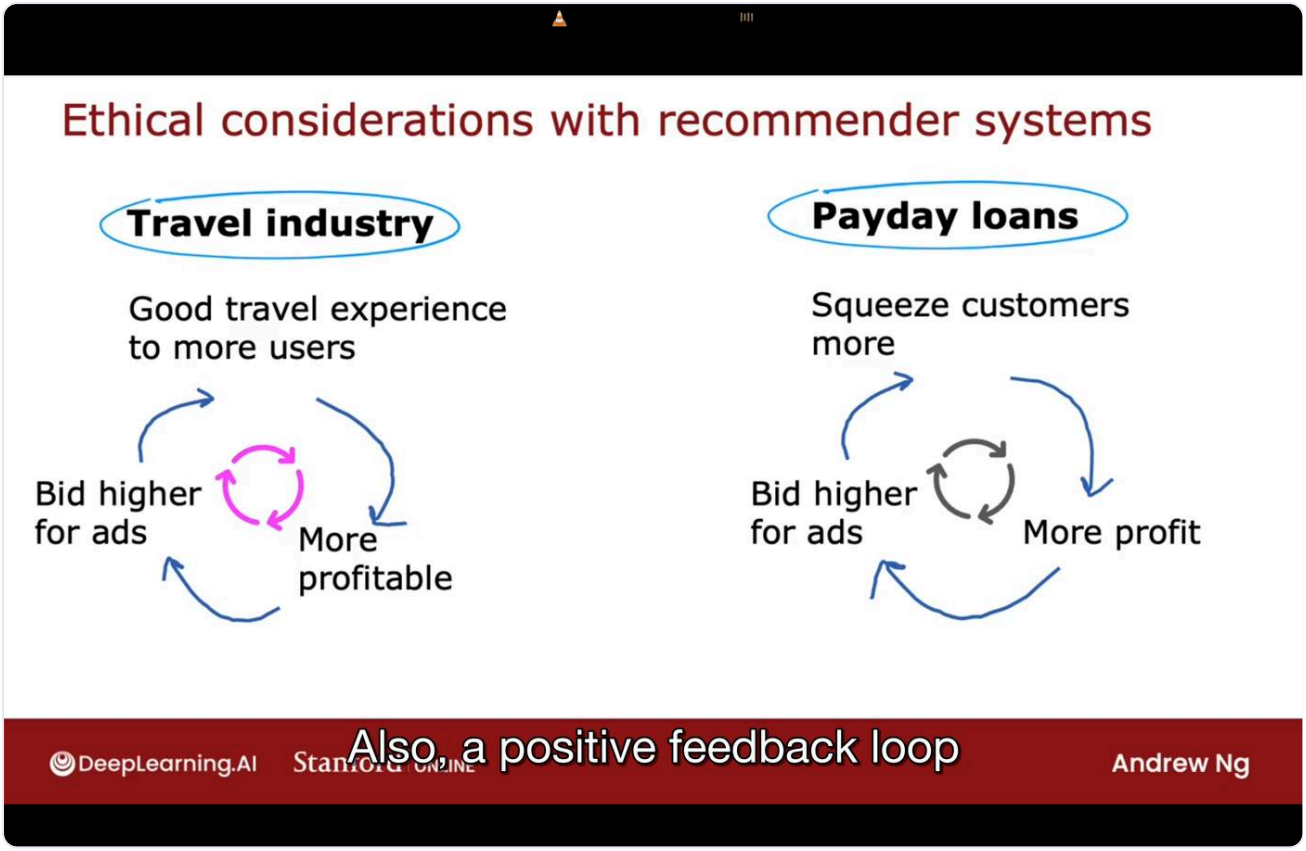


FIGURE 24.5 | Travel Industry Payday Loans

This in turn, also increase a positive feedback loop. Also, a positive feedback loop that can cause the most exploitative, the most harmful payday loan companies to get sent more traffic. This seems like the opposite effect than what we think would be good for society. I don't know that there's an easy solution to this. These are very difficult problems that recommender systems face. One amelioration might be to refuse to set ads from exploitative businesses.

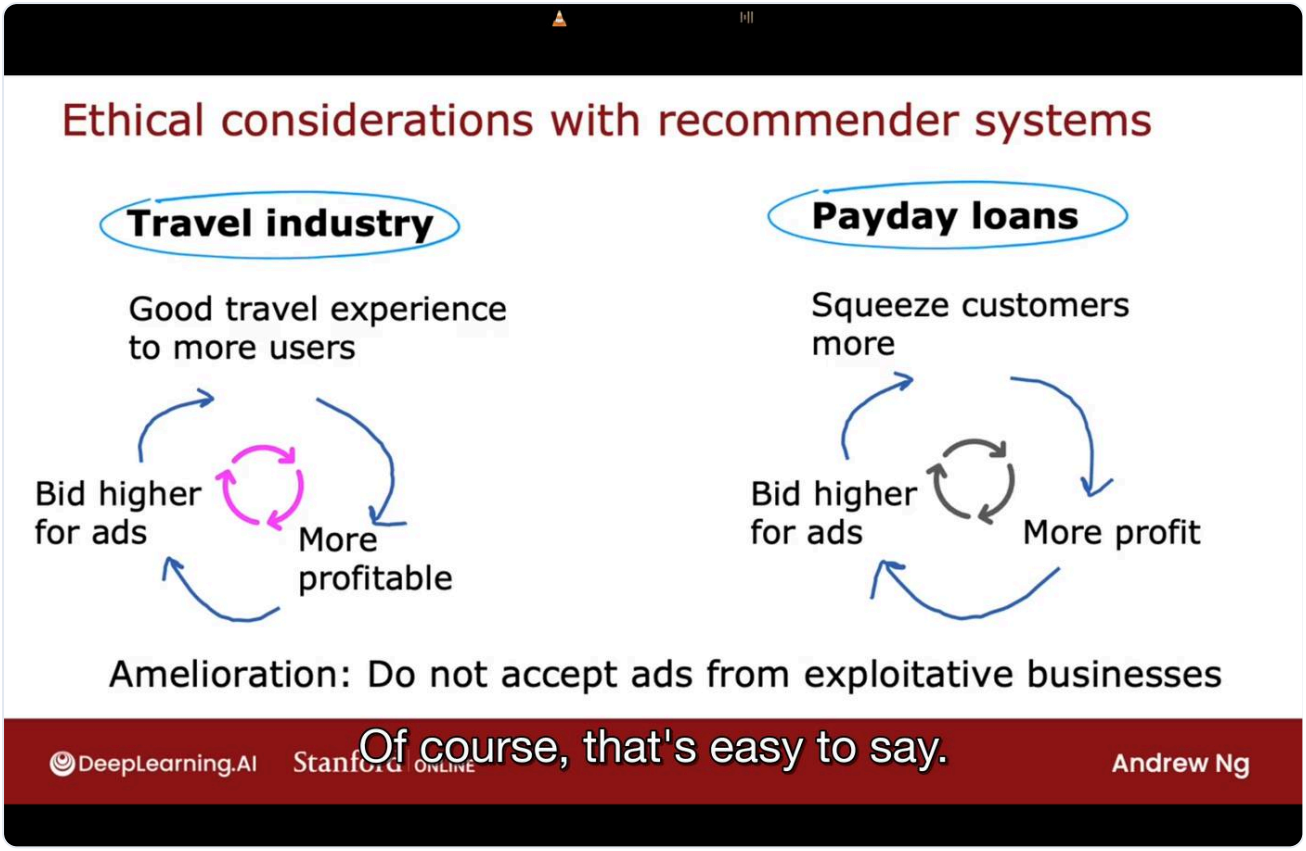


FIGURE 24.6 | Travel Industry Payday Loans

Of course, that's easy to say. But how do you define what is an exploitative business and what is not, is a very difficult question. But as we build recommender systems for advertising or for other things, I think these are questions that each one of us working on these technologies should ask ourselves so that we can hopefully invite open discussion and debate, get multiple opinions from multiple people, and try to come up with design choices that allows our systems to try to do much more good than potential harm. Let's look at some other examples. It's been widely reported in the news that maximizing user engagement such as the amount of time that someone watches videos on a website or the amount of time someone spends on social media.

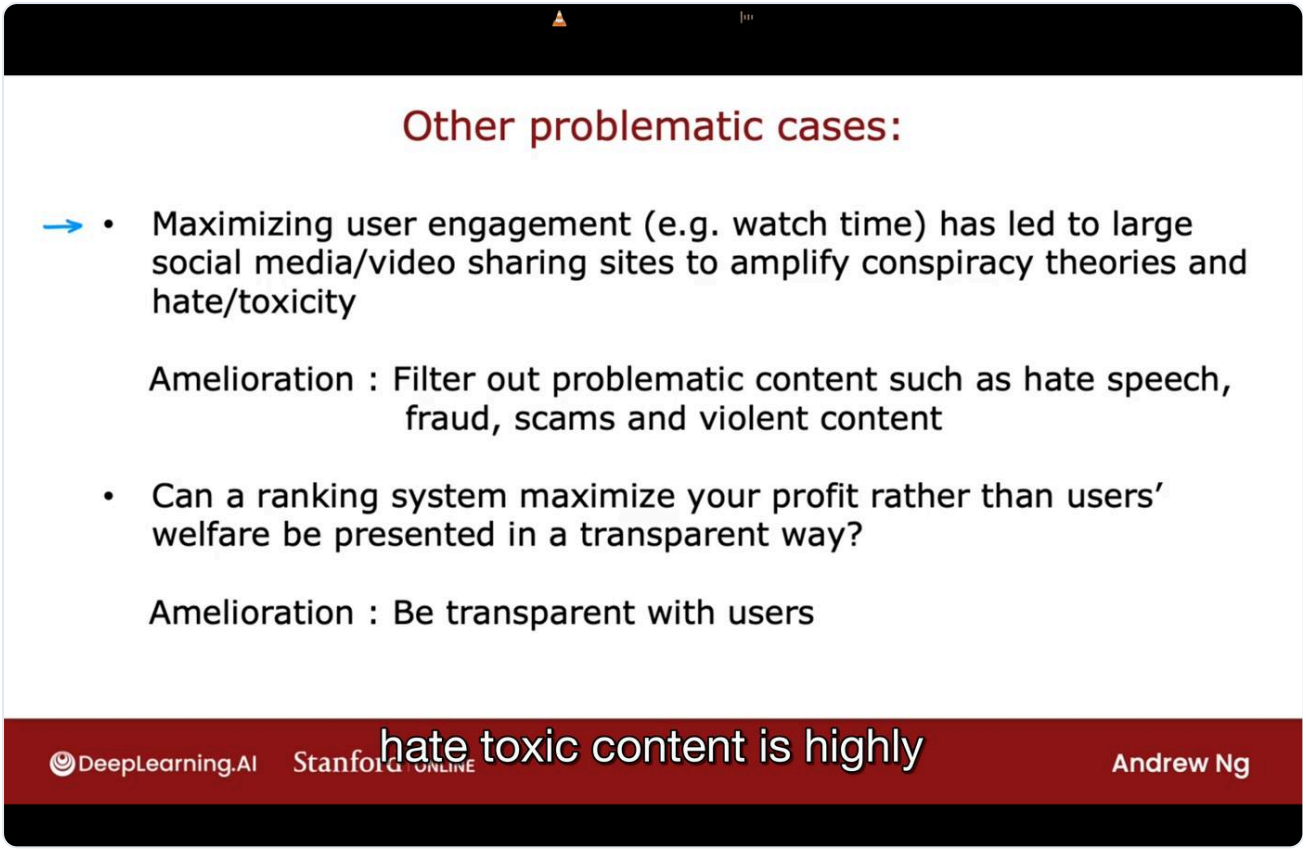


FIGURE 24.7 | Other Problematic Cases:

This has led to large social media and video sharing sites to amplify conspiracy theories or hate and toxicity because conspiracy theories and certain types of hate toxic content is highly engaging and causes people to spend a lot of time on it. Even if the effect of amplifying conspiracy theories amplify hidden toxicity turns out to be harmful to individuals and to society at large. One amelioration for this partial and imperfect is to try to filter out problematic contents such as hate speech, fraud, scams, maybe certain types the violent content. Again, the definitions of what exactly we should filter out is surprisingly tricky to develop. And this is a set of problems that I think companies and individuals and even governments have to continue to wrestle with.

Just one last example. When a user goes to many apps or websites, I think users think the app or website I tried to recommend to the user thinks that they will like. I think many users don't realize that many apps and websites are trying to maximize their profit rather than necessarily the user's enjoyment of the media items that are being recommended. I would encourage you and other companies if at all possible, to be transparent with users about a criteria by which you're deciding what to recommend to them. I know this isn't always easy, but ultimately, I hope that being more transparent with users about why we're showing them and why will increase trust and also cause our systems to do more good for society.

Recommender systems are very powerful technology, a very profitable, a very lucrative technology. There are also some problematic use cases. If you are building one of these systems using recommender technology or really any other machine learning or other technology. I hope you think through not just the benefits you can create, but also the possible harm and invite diverse perspectives and discuss and debate. Please only build things and do things that you really believe can be society better off. I hope that collectively, all of us in AI can only do work that makes

people better off. Thanks for listening. We have just one more video to go in recommender systems in which we take a look at some practical tips for how to implement a content-based filtering algorithm in TensorFlow.

Let's go on to that last video on recommender systems.

LECTURE 25

Item Features

In the practice lab, you see how to implement content-based filtering in TensorFlow. What I'd like to do in this video is just set through of you a few of the key concepts in the code that you get to play with. Let's take a look. Recall that our code has started with a user network as well as a movie that's work. The way you can implement this in TensorFlow is, it's very similar to how we have previously implemented a neural network with a set of dense layers. We're going to use a sequential model. We then in this example have two dense layers with the number of hidden units specified here, and the final layer has 32 units and output's 32 numbers. Then for the movie network, I'm going to call it the item network, because the movies are the items here, this is what the code looks like.

Once again, we have coupled dense hidden layers, followed by this layer, which outputs 32 numbers. For the hidden layers, we'll use our default choice of activation function, which is the relu activation function. Next, we need to tell TensorFlow Keras how to feed the user features or the item features, that is the movie features to the two neural networks. This is the syntax for doing so. That extracts out the input features for the user and then feeds it to the user and that we had defined up here to compute vu, the vector for the user. Then one additional step that turns out to make this algorithm work a bit better is at this line here, which normalizes the vector vu to have length one.

This normalizes the length, also called the l2 norm, but basically the length of the vector vu to be equal to one. Then we do the same thing for the item network, for the movie network. This extract out the item features and feeds it to the item neural network that we defined up there This computes the movie vector vm. Then finally, the step also normalizes that vector to have length one. After having computed vu and vm, we then have to take the dot product between these two vectors. This is the syntax for doing so.

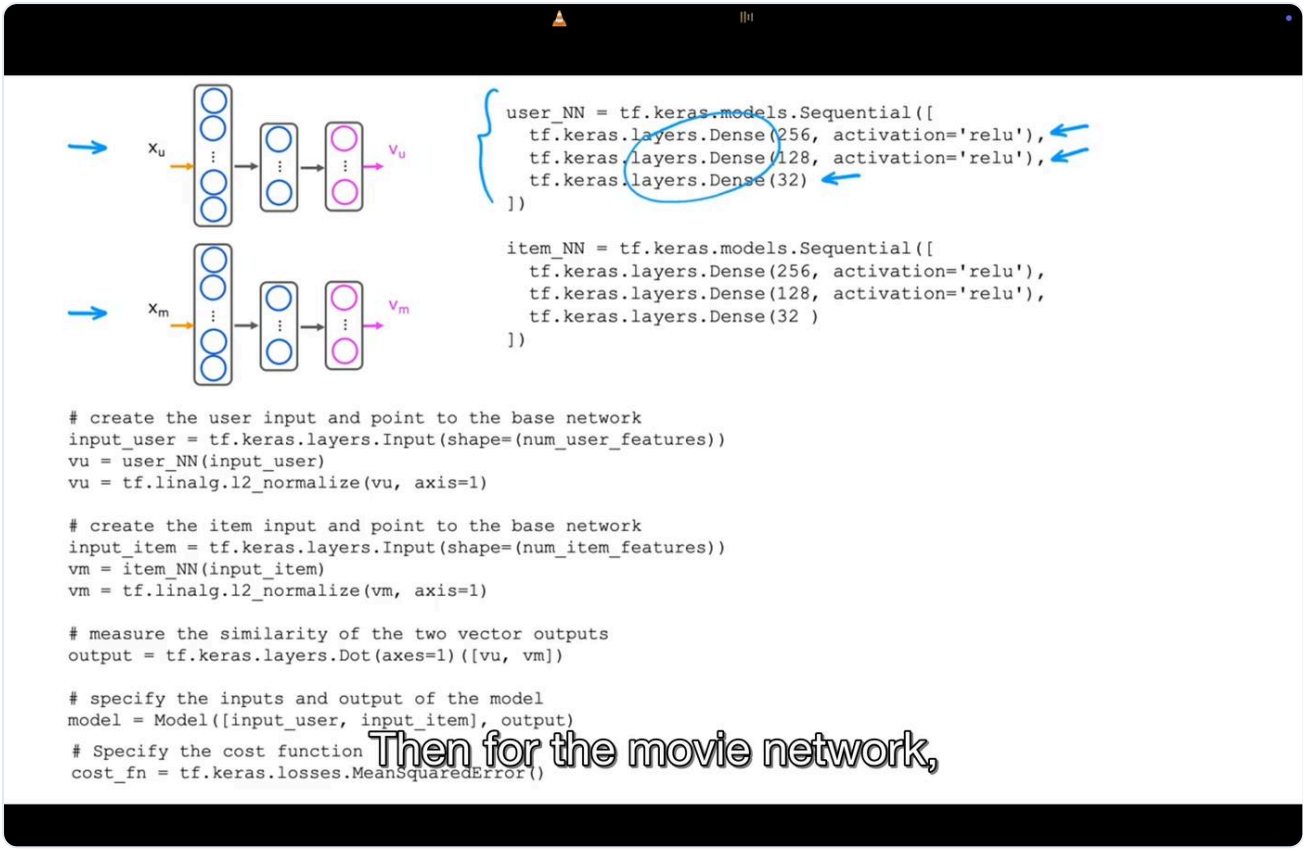


FIGURE 25.1

Keras has a special layer type, notice we had here tf keras layers dense, here this is tf keras layers dot. It turns out that there's a special Keras layer, they just takes a dot product between two numbers. We're going to use that to take the dot product between the vectors vu and vm. This gives the output of the neural network. This gives the final prediction. Finally, to tell keras what are the inputs and outputs of the model, this line tells it that the overall model is a model with inputs being the user features and the movie or the item features and the output, this is output that we just defined up above. The cost function that we'll use to train this model is going to be the mean squared error cost function.

These are the key code snippets for implementing content-based filtering as a neural network. You see the rest of the code in the practice lab but hopefully you'll be able to play with that and see how all these code snippets fit together into working TensorFlow implementation of a content-based filtering algorithm. It turns out that there's one other step that I didn't talk about previously, but if you do this, which is normalize the length of the vector vu, that makes the algorithm work a bit better. TensorFlows has this l2 normalized motion that normalizes the vector, is also called normalizing the l2 norm of the vector, hence the name of the function.

That's it. Thanks for sticking with me through all this material on recommender systems, it is an exciting technology. I hope you enjoy playing with these ideas and codes in the practice labs for this week. That takes us to the lots of these videos on recommender systems and to the end of the next to final week for this specialization. I look forward to seeing you next week as well. We'll talk about the exciting technology of reinforcement learning. Hope you have fun with the quizzes and with the practice labs and I look forward to seeing you next week.

LECTURE 26

PCA: Find New Axis and Coordinates

I hope you enjoyed the videos on how you can build your own recommender system. Before we wrap up this week in this and a few other optional videos I'd like to share with you an unsupervised learning algorithm called principal components analysis. This is an algorithm that is commonly used for visualization. Specifically, if you have a dataset with a lot of features, say 10 features or 50 features or even thousands of features, you can't plot 1,000 dimensional data. PCA, or principal components analysis is an algorithm that lets you take data with a lot of features, 50, 1,000, even more, and reduce the number of features to two features, maybe three features, so that you can plot it and visualize it.

Is commonly used by data scientists to visualize the data, to figure out what might be going on. Let's take a look at how PCA, principal components analysis works. To describe PCA, I'm going to use as a running example, if you have data from a collection of passenger cars, and passenger cars can have a lot of features. You may know the length of the car or the width of the car, maybe the diameter of the wheel, or maybe the height of the car, and many other features of cars. If you want to reduce the number of features so you can visualize it, how can you use PCA to do so? For the first example, let's say you're given a dataset with two features. The feature x_1 is the length of the car, like so, and the second feature x_2, is the width of the car, which is measured like so.

It turns out that in most countries, because of constraints about the width of the road the cars drive on, width of the car which has got to fit within the width of the road of a single lane, tends not to vary that much. For example, in the United States, most cars are, let's call it about 1.8 meters wide, that's just under six feet. If you were to have a collection of cars and the dataset of the length and width of the cars, you will find that the dataset might look like this, where x_1 varies quite a bit because some cars are really long and x_2 varies relatively little. If you want to reduce the number of features, well, one thing you could do is let us take x_1 because x_2 varies relatively little from car to car.

It turns out that PCA is an algorithm that when applied to this data set will more or less automatically decide to just take x_1, but it can do much more than that. Let's look at a second example where here x_1 is again the length of the car, and let's say that in this dataset, x_2 is the diameter of the wheel. The diameter of the wheel does vary a little bit. If you were to plot the data, it might look like this. But again, if you want to simplify this dataset to just one feature, you might decide, let's just take x_1 and forget x_2 and PCA when applied to this dataset. Well, again, more or less, cause you to just check the feature x_1. In both the examples we saw, only one of the two features seemed to have a meaningful degree of variation.

Here's a more complex example. Say the feature x_1 is the length of the car, so that varies quite a bit, and the feature x_2 here is the height of the car, which also varies quite a bit. Some cars are much taller than other cars. If you were to plot the data, you might get a dataset that looks like this, where some cars are bigger and they tend to be longer and taller, and some cars are a little

bit smaller. They tend to be not as long and not as tall. If you wanted to reduce the number of features, what should you pick? You don't want to pick just x_1, the length, and ignore x_2 the height and you also don't want to pick just x_2, the height, and ignore x_1, the length.

It seems as if both x_1 and x_2 have useful information. In this graph, x_1 and x_2 are the two axes of this plot. Instead of being limited to taking either the x_1 axis or the x_2 axis, what if we had a third axis. I'm going to call this new axis the z-axis. To be clear, this is not sticking out of this diagram. This is a combination of x_1 and x_2. This is not a z-axis that's sticking out in the third dimension. This z-axis lies flat within this plot. But why do we have the z-axis which corresponds to something about the size of the car? Given a car like this one over here, its coordinate, meaning the value on the x-axis tells us the length of the car and the coordinate is just, what is this distance?

Similarly its coordinate, meaning, what is this distance on the x_2 axis tells us what is the height of the car.

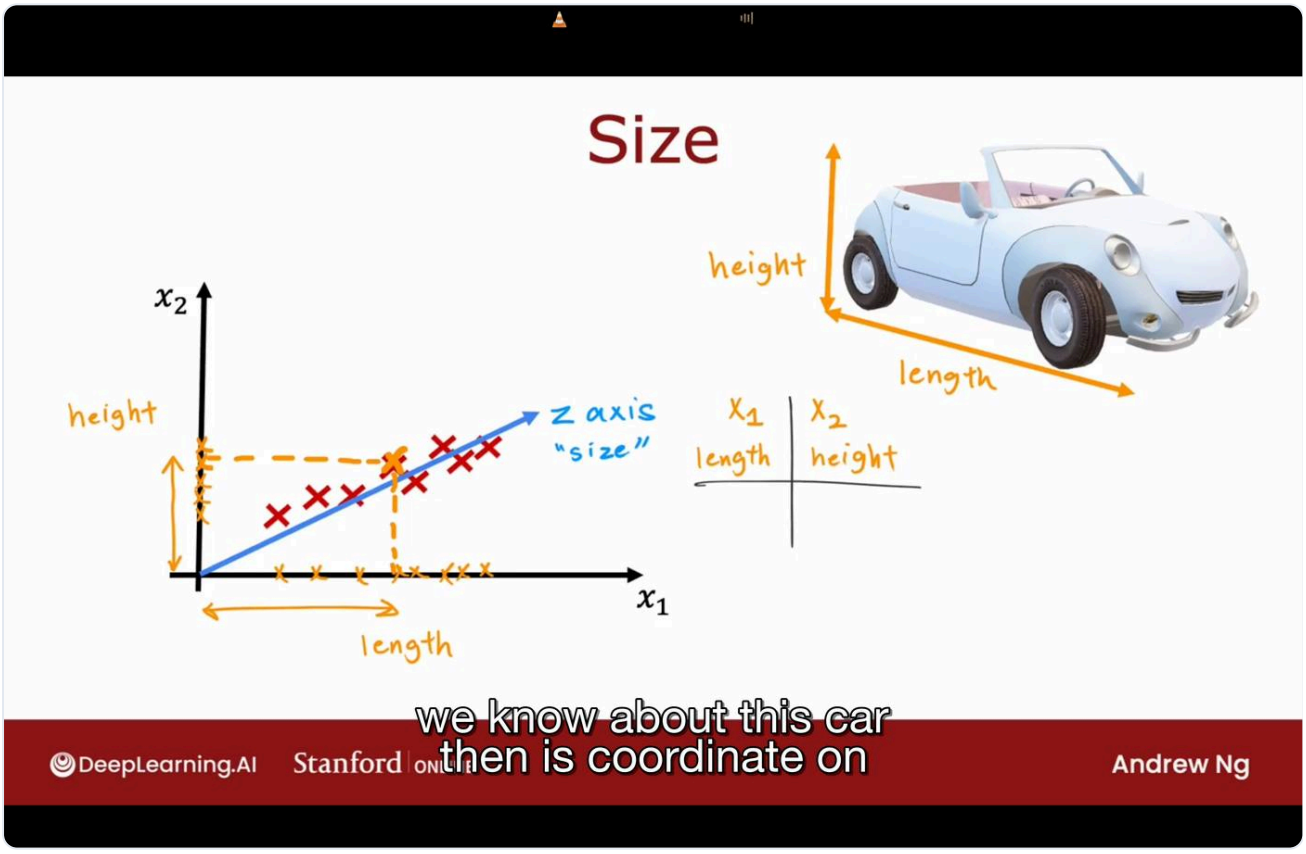


FIGURE 26.1

If we're now going to use the z-axis instead as one feature to capture what we know about this car then is coordinate on the z-axis, meaning this distance. That tells us roughly what is the size of the car. We'll formalize this in the next few videos.

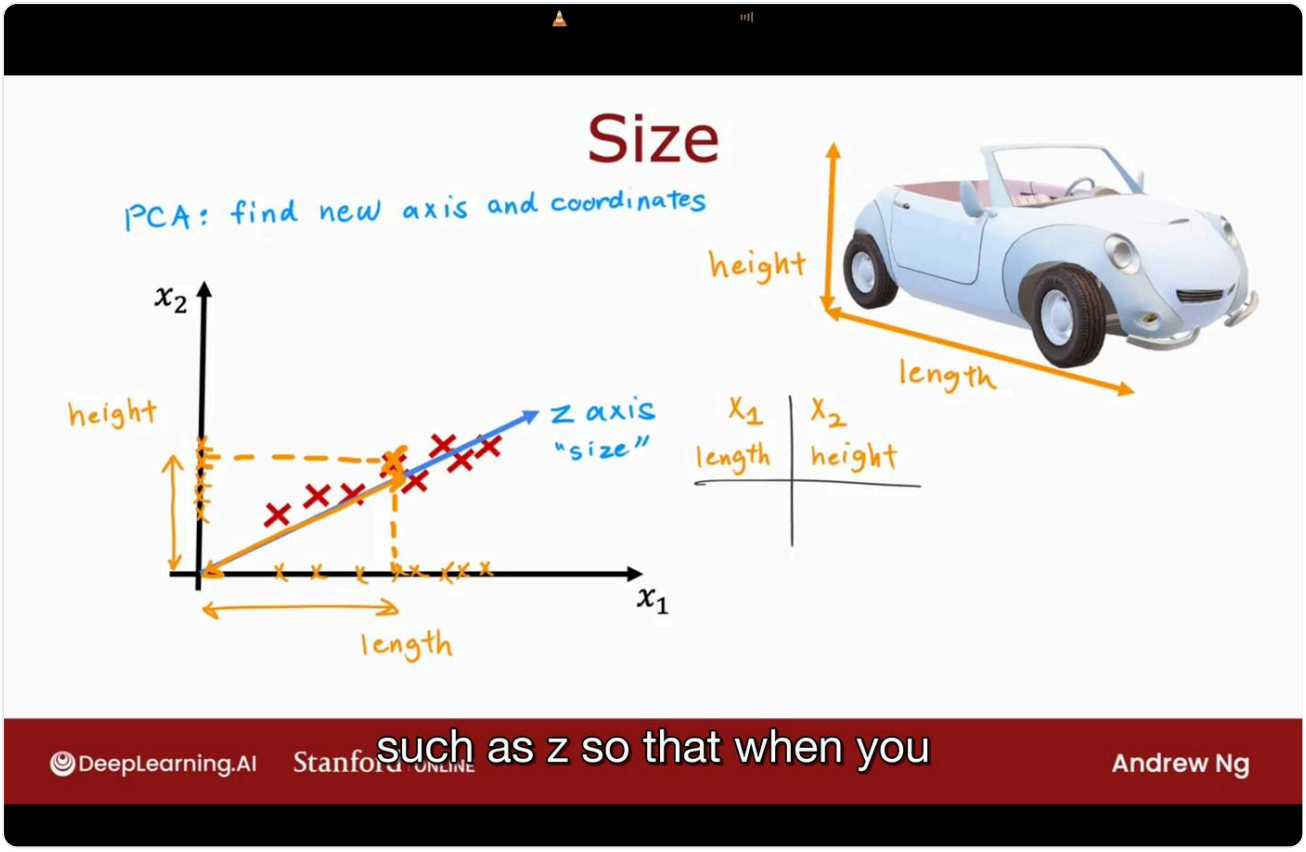


FIGURE 26.2 | PCA: Find New Axis and Coordinates

But the idea of PCA is to find one or more new axes, such as z so that when you measure your data's coordinates on the new axis, you end up still with very useful information about the car.

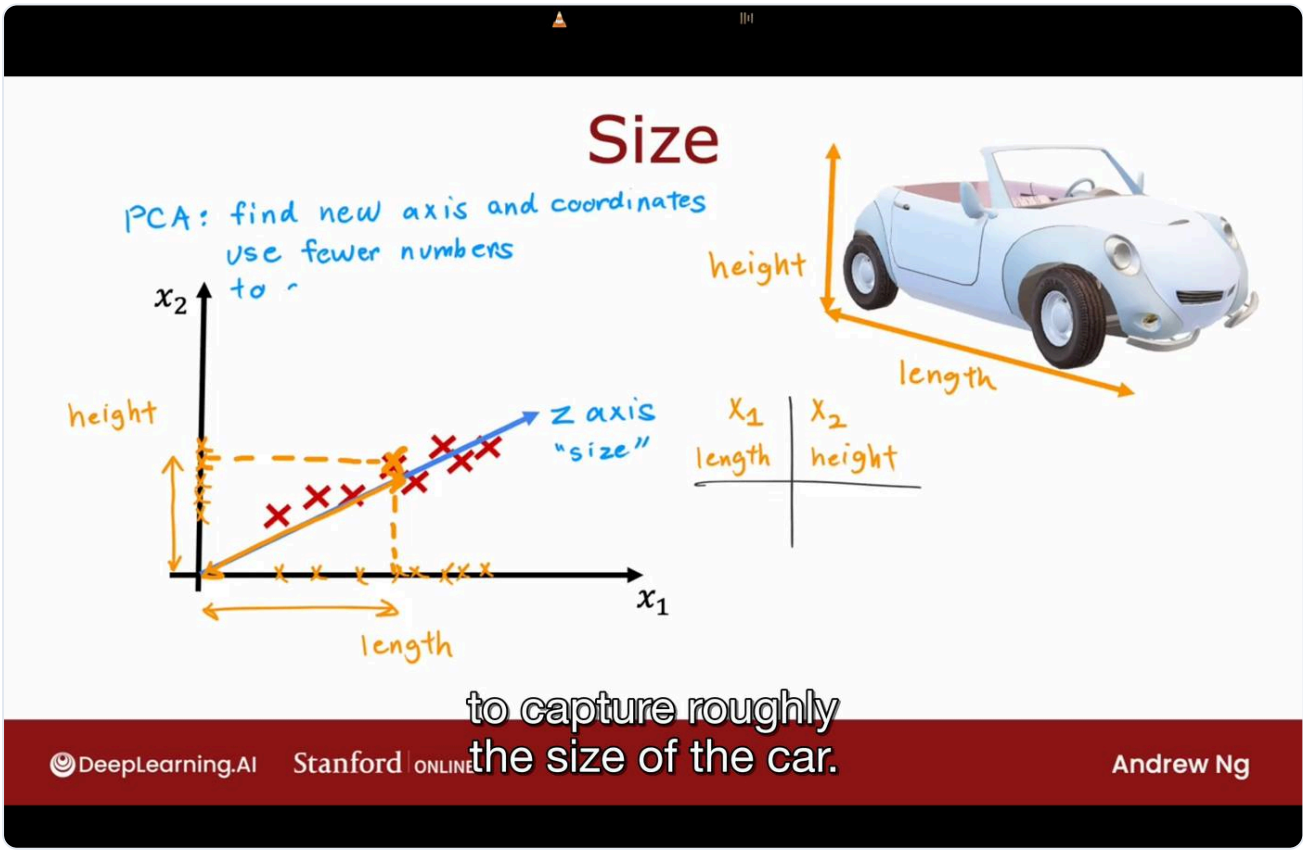


FIGURE 26.3 | PCA: Find New Axis and Coordinates

But maybe now, instead of needing two numbers corresponding to the coordinates on X_1 and X_2 axes, the length and height.

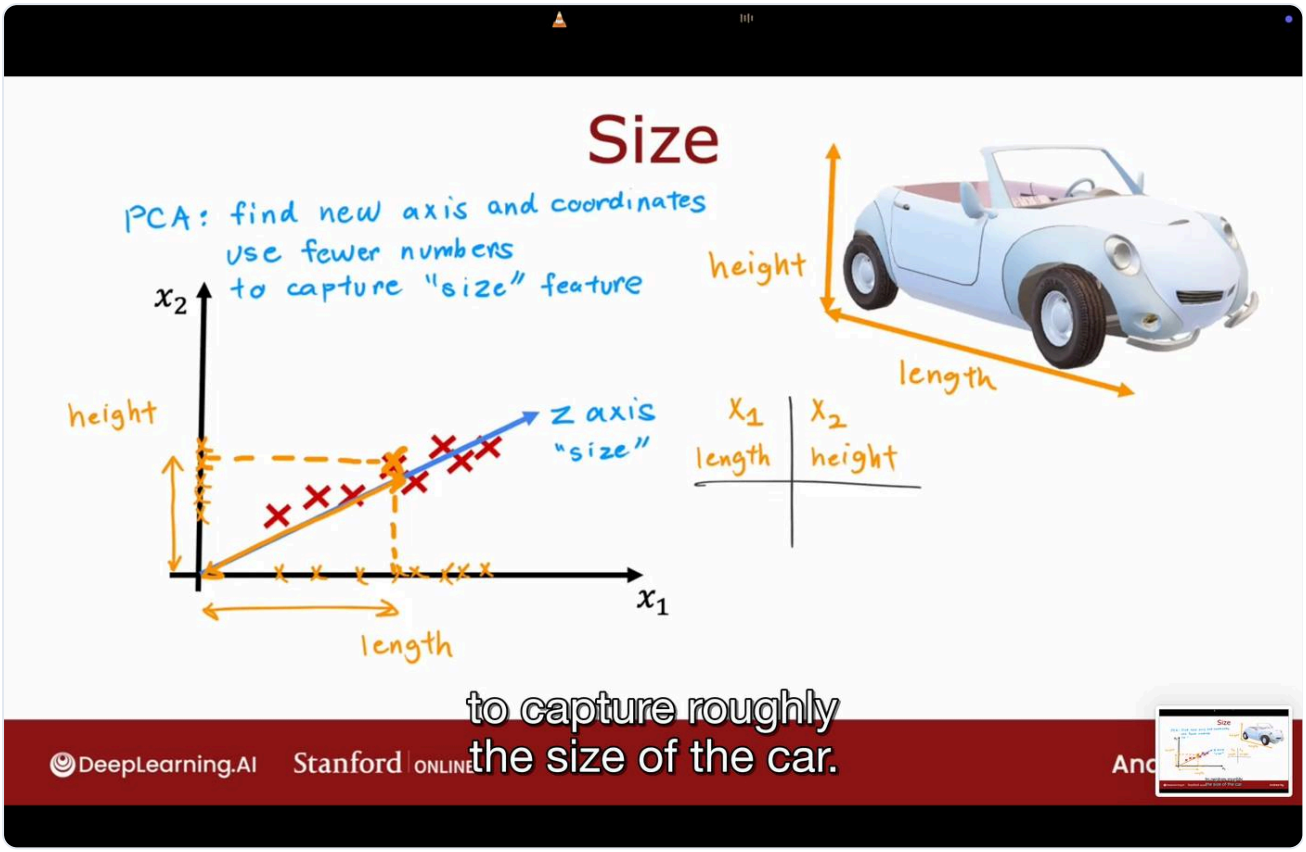


FIGURE 26.4 | PCA: Find New Axis and Coordinates

You now need a few numbers, in this case, only one number instead of two, to capture roughly the size of the car.

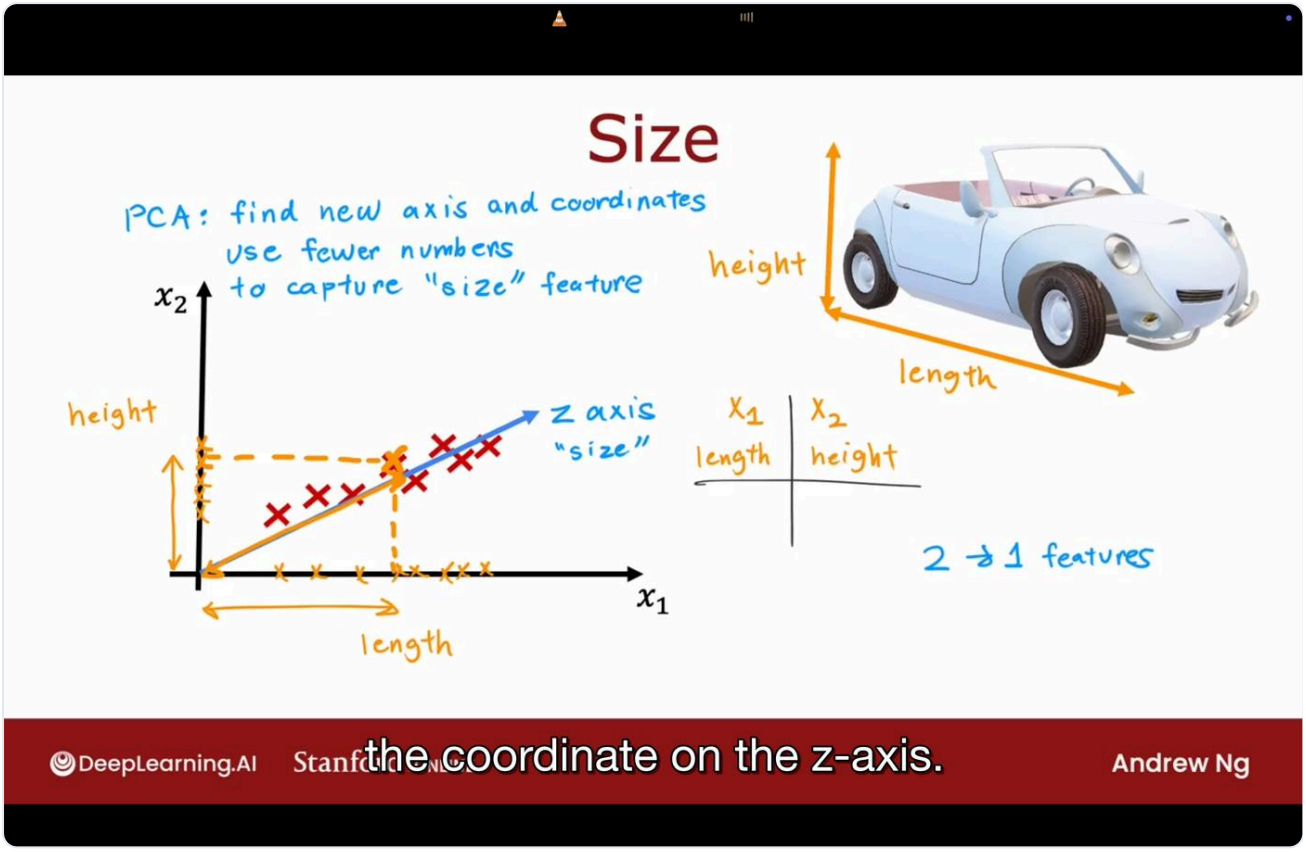


FIGURE 26.5 | PCA: Find New Axis and Coordinates :

In the example, we've used so far, we were trying to reduce the data from two numbers, X_1 and X_2 down to one number, the coordinate on the z-axis.

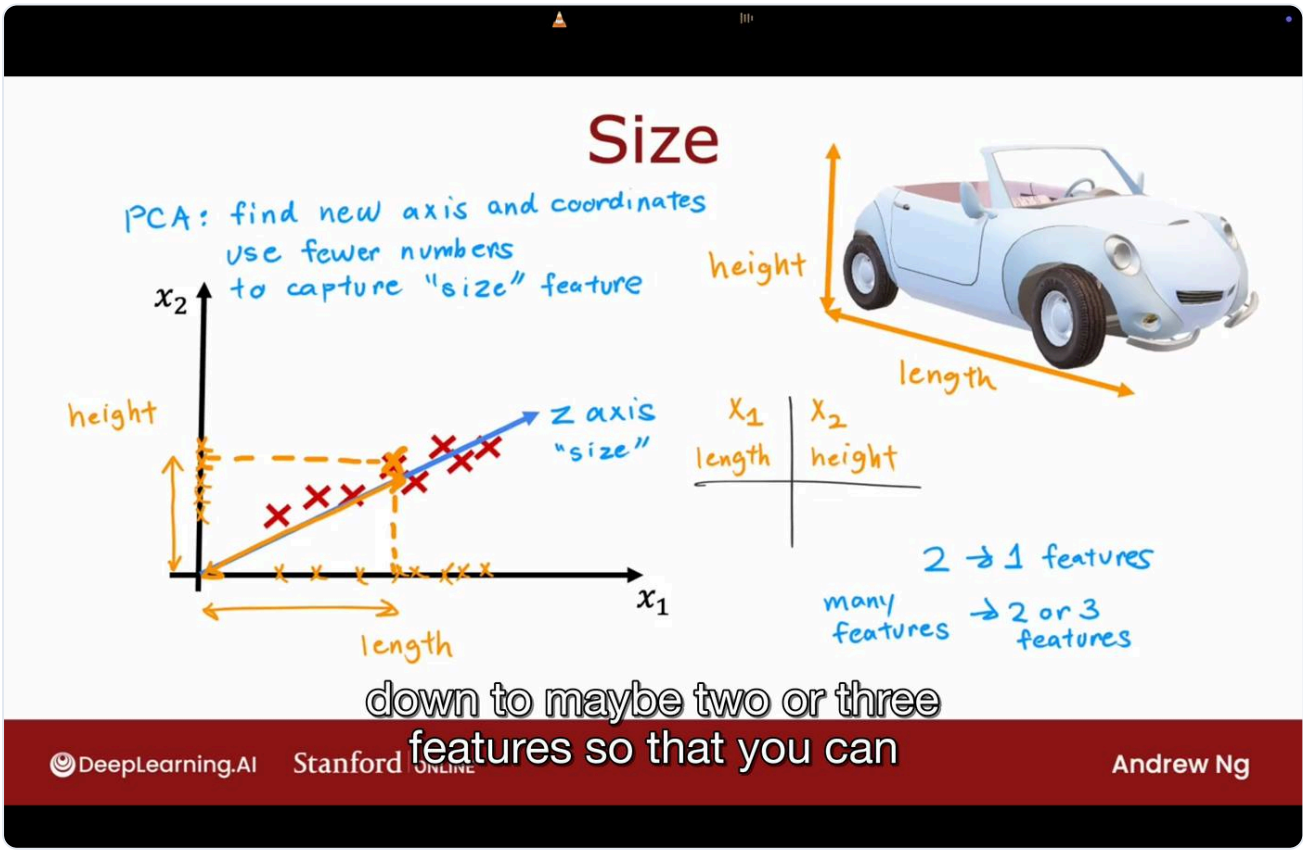


FIGURE 26.6 | PCA: Find New Axis and Coordinates

In practice, PCA is usually used to reduce a very large number of features, say 10, 20, 50, even thousands of features, down to maybe two or three features so that you can visualize the data in a two-dimensional or in a three-dimensional plot. But for this video, because I could only draw on a two-dimensional screen. I'm going to use mainly two or three-dimensional data sets as my examples. Let's look at one more example. In this visualization, we have a three-dimensional data set and notice that I can rotate the data set here, so you can see it in 3D. But notice if I rotate the data set like this, well, most of this data, even though it is in 3D, it actually lives on a very thin surface.

It's almost as if all the data lies on a two-dimensional pancake, even though the pancake lives in this three-dimensional space.

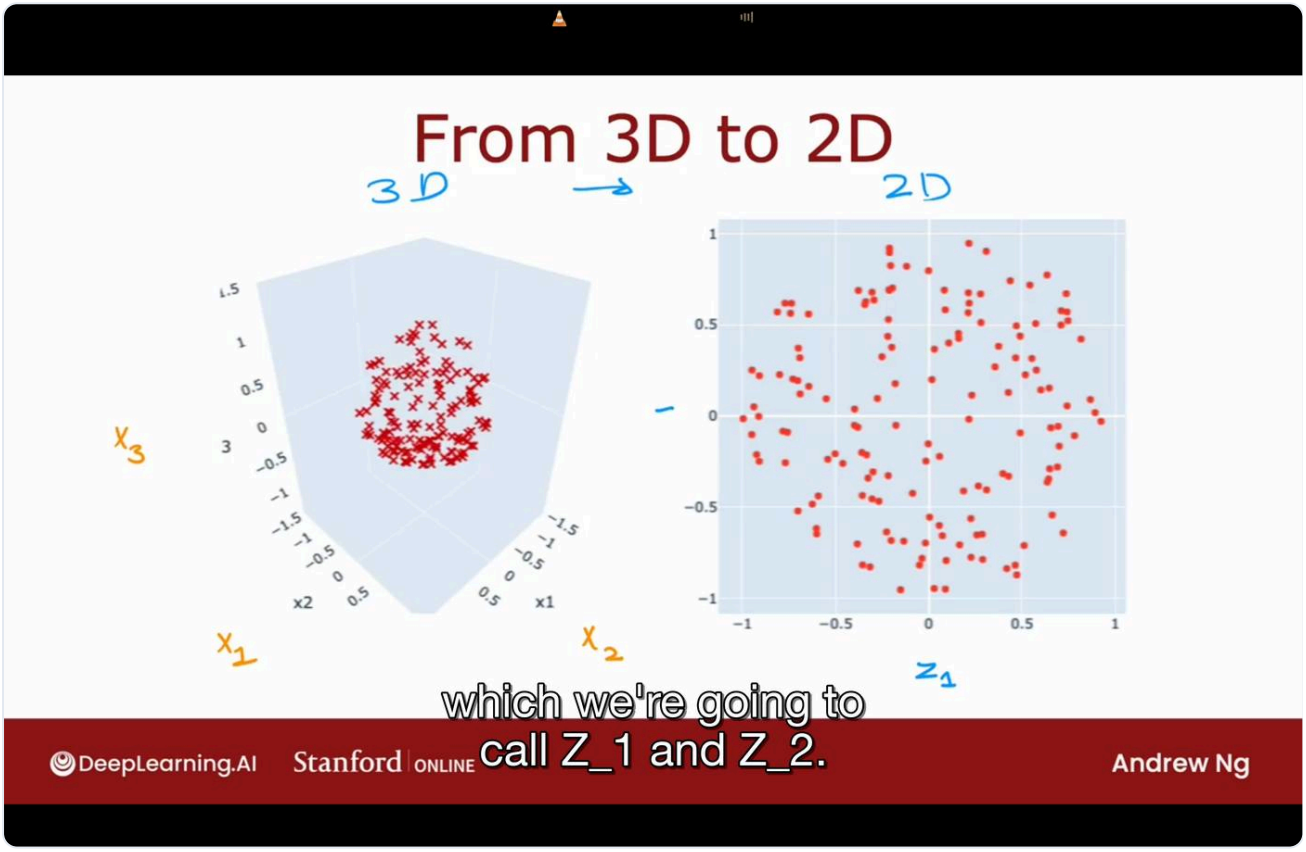


FIGURE 26.7

A PCA, what you can do is, instead of having three features, x_1, x_2, x_3, reduce it to two numbers, which we're going to call Z_1 and Z_2. When you do that, you can then visualize the data on this Z_1, Z_2 axis and this becomes a convenient way to visualize this data if you had to, say print on a piece of paper. I couldn't dynamically rotate it like you'll seeing me do on the screen. Here's one more example. If you have data about the development status of many different countries, you might have for example data about different countries, GDP, and that's feature x_1. In addition, Let's say we also have the per capita GDP and also a measure of their Human Development Index.

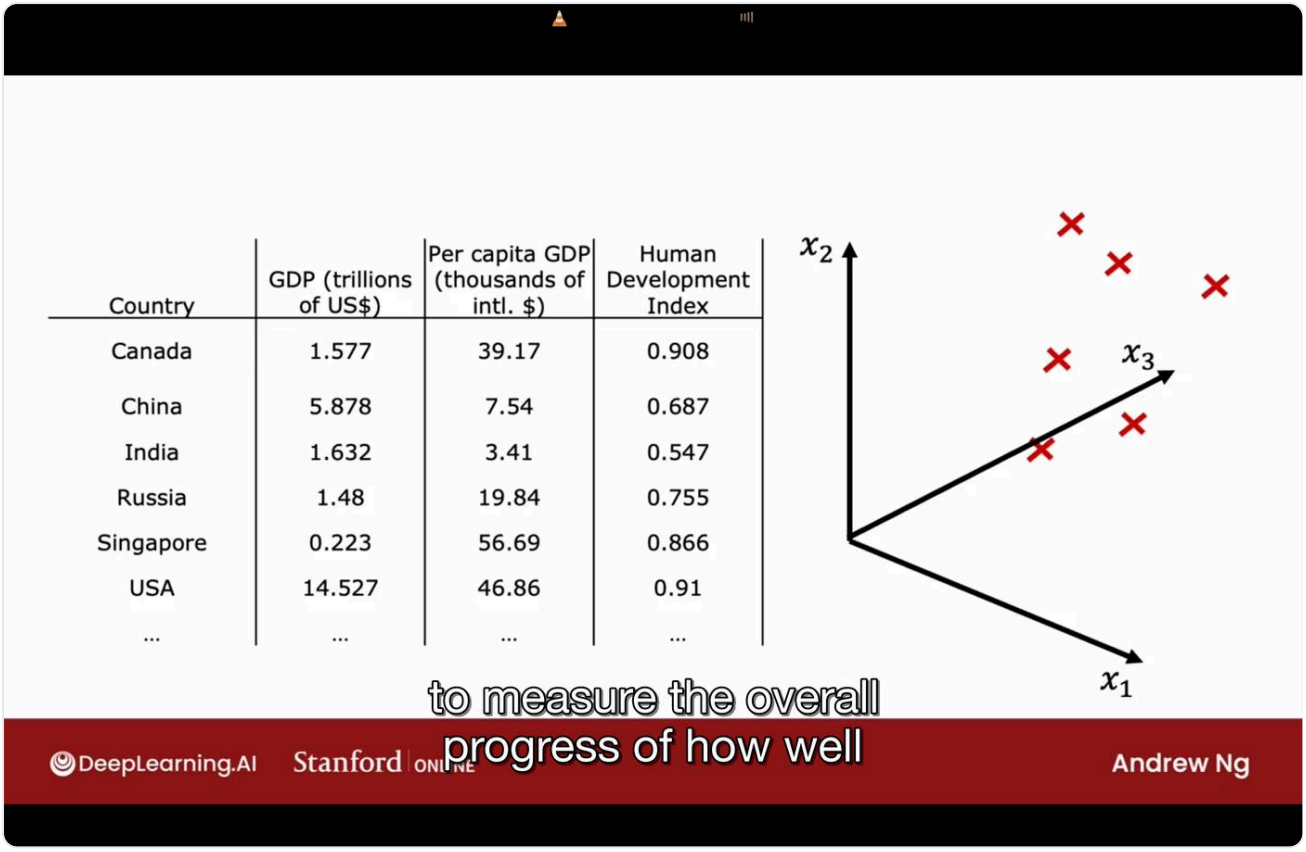
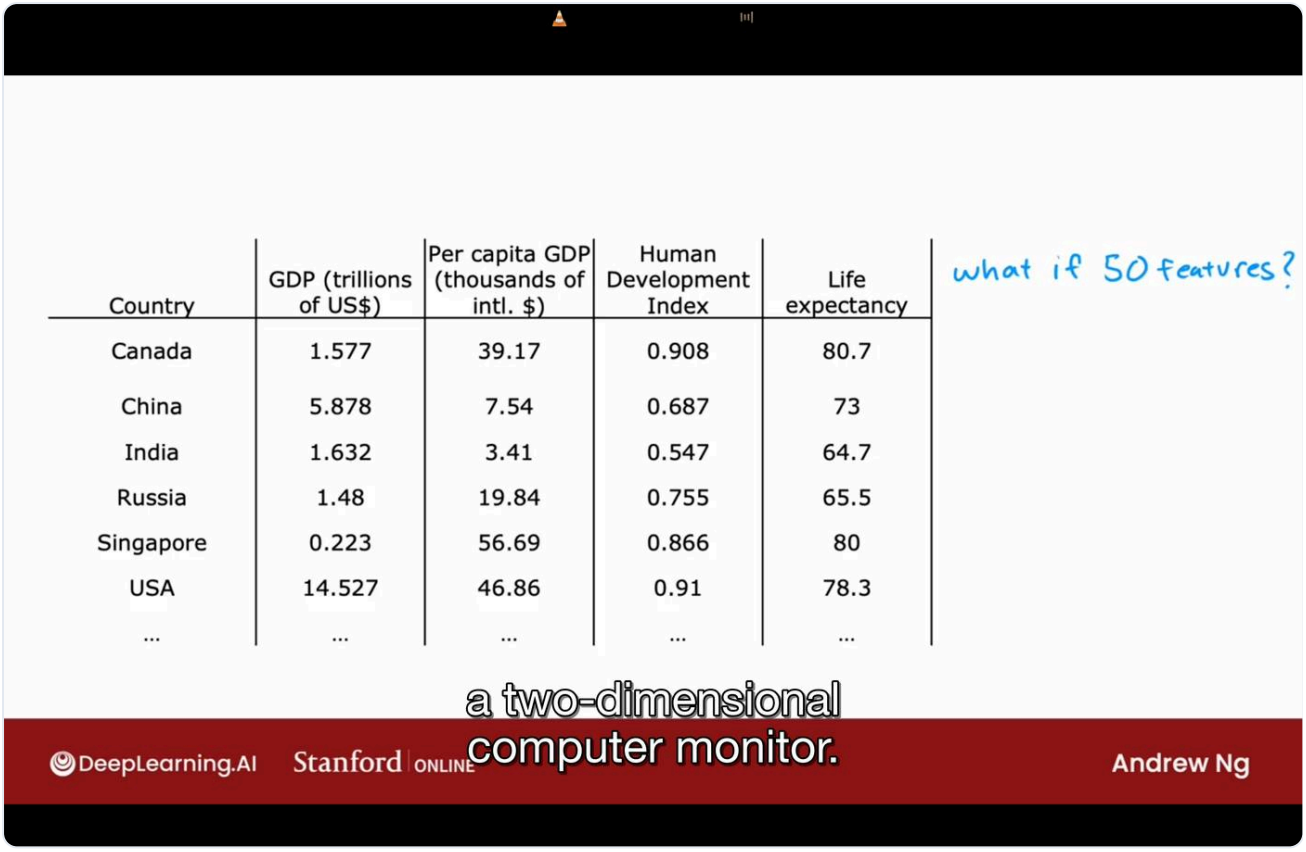


FIGURE 26.8 | Per Capita GDP Human x2 X

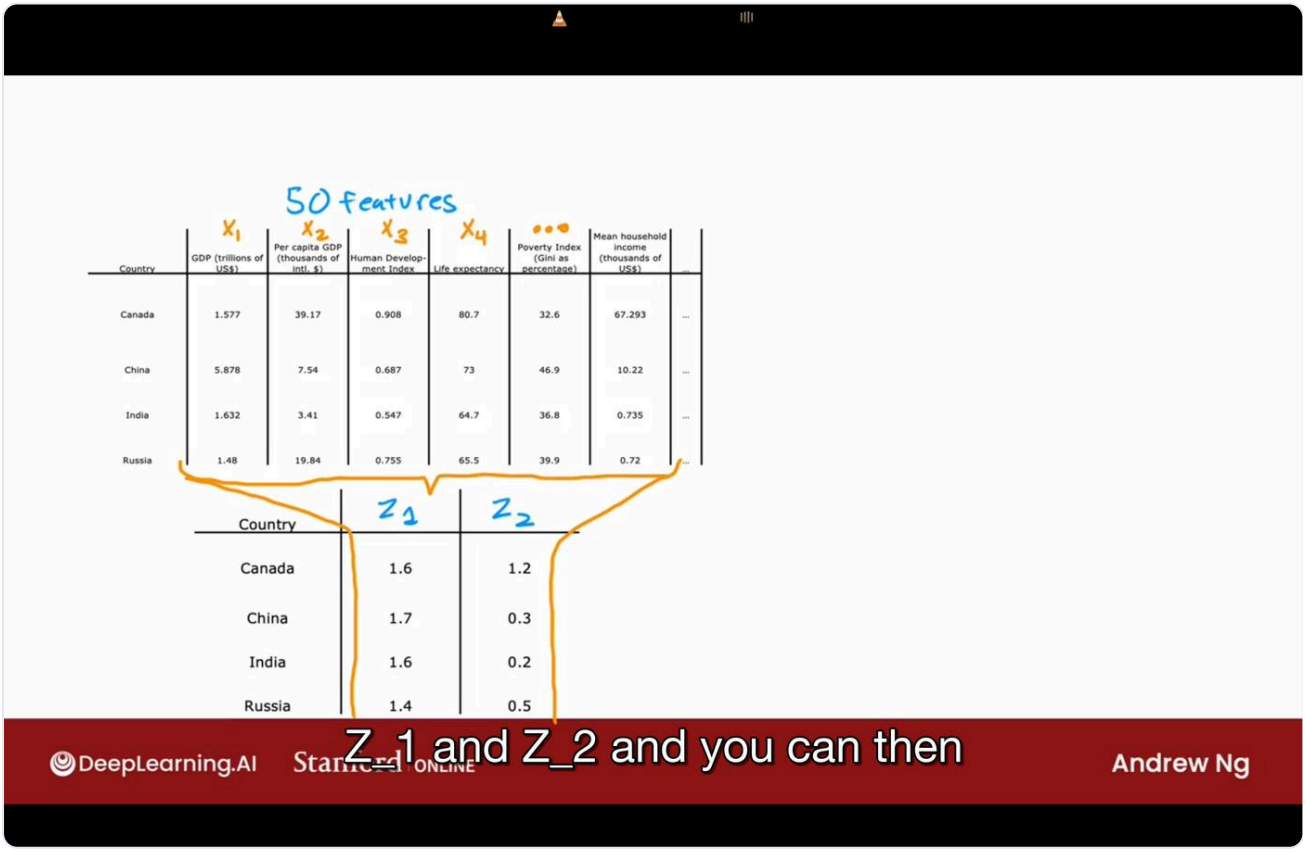
The Human Development Index was developed to measure the overall progress of how well people in a country might be doing based on things like the lifespan and education and so on or you might separately have a feature corresponding to the life expectancy in different countries and so on and so forth.



Country	GDP (trillions of US$)	Per capita GDP (thousands of intl. $)	Human Development Index	Life expectancy
Canada	1.577	39.17	0.908	80.7
China	5.878	7.54	0.687	73
India	1.632	3.41	0.547	64.7
Russia	1.48	19.84	0.755	65.5
Singapore	0.223	56.69	0.866	80
USA	14.527	46.86	0.91	78.3
...	...	...	...	...

FIGURE 26.9

If for each country you have 50 features, how can you visualize this data because you can't plot 50 dimensional data on a two-dimensional computer monitor.



Country	GDP (trillions of US$)	Per capita GDP (thousands of intl. $)	Human Development Index	Life expectancy	Poverty Index (Gini as percentage)	Mean household income (thousands of US$)	...
Canada	1.577	39.17	0.908	80.7	32.6	67.293	...
China	5.878	7.54	0.687	73	46.9	10.22	...
India	1.632	3.41	0.547	64.7	36.8	0.735	...
Russia	1.48	19.84	0.755	65.5	39.9	0.72	...

Country	Z_1	Z_2
Canada	1.6	1.2
China	1.7	0.3
India	1.6	0.2
Russia	1.4	0.5

FIGURE 26.10

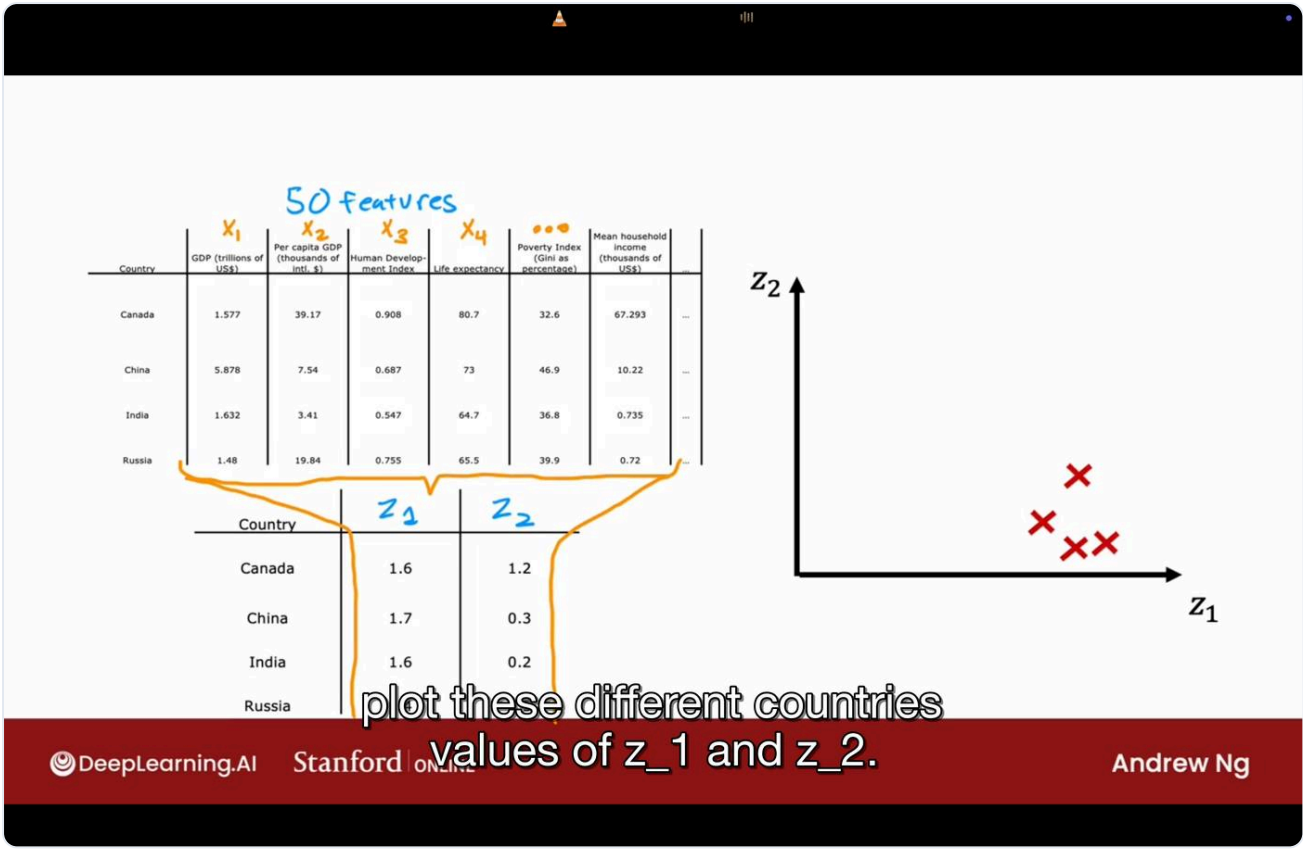


FIGURE 26.11 | Country Z a Z a 2 X

What PCLS will do is take these 50 features, X_1,X_2,X_3,X_4, and so on and compress it down to two features which I'm going to call Z_1 and Z_2 and you can then plot these different countries values of z_1 and z_2. You might find for example that Z_1 loosely corresponds to how big is the country and what is this total GDP? Because larger countries tend to have a higher GDP, because large countries have many people, tend to have a larger economy and perhaps you find that Z_2 corresponds roughly to the per person GDP or the amount of economic activity per person. For example, the United States, which is a relatively large country and has relatively high per person economic activity may be somewhere up here to the up and right of this plot and a country like Singapore, where I live many years as well, is a smaller country but it has relatively high per person economic activity.

Take on a lower value on the z_1 axis, but still a relatively high value on the z_2 axis.

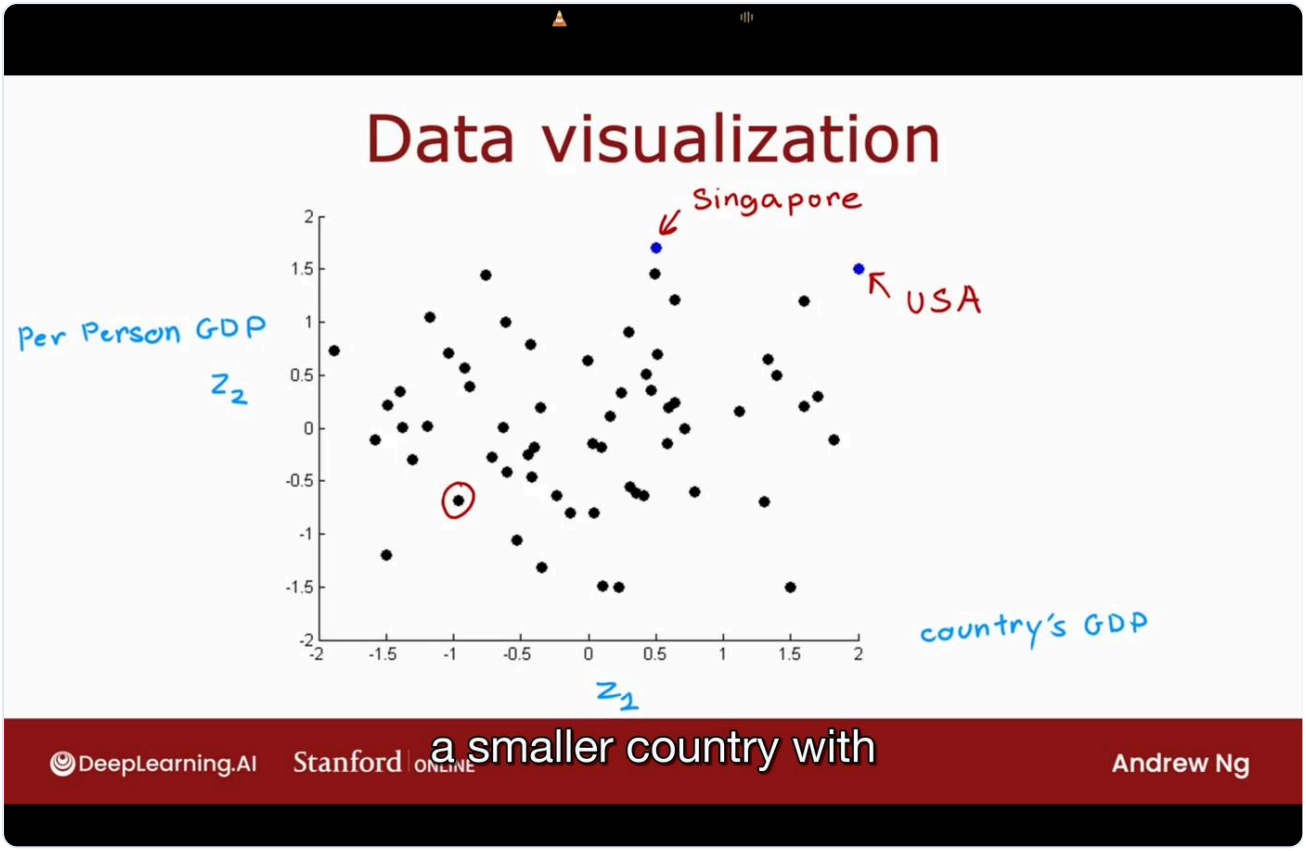


FIGURE 26.12 | Per Person GDP

Whereas a country like this would be maybe a smaller country with lower per person economic activity. Whereas the country like this may be a large country with lower per person economic activity and a figure like this lets you take a large number of features, 50 features. Sometimes we also say that's 50 dimensional data.

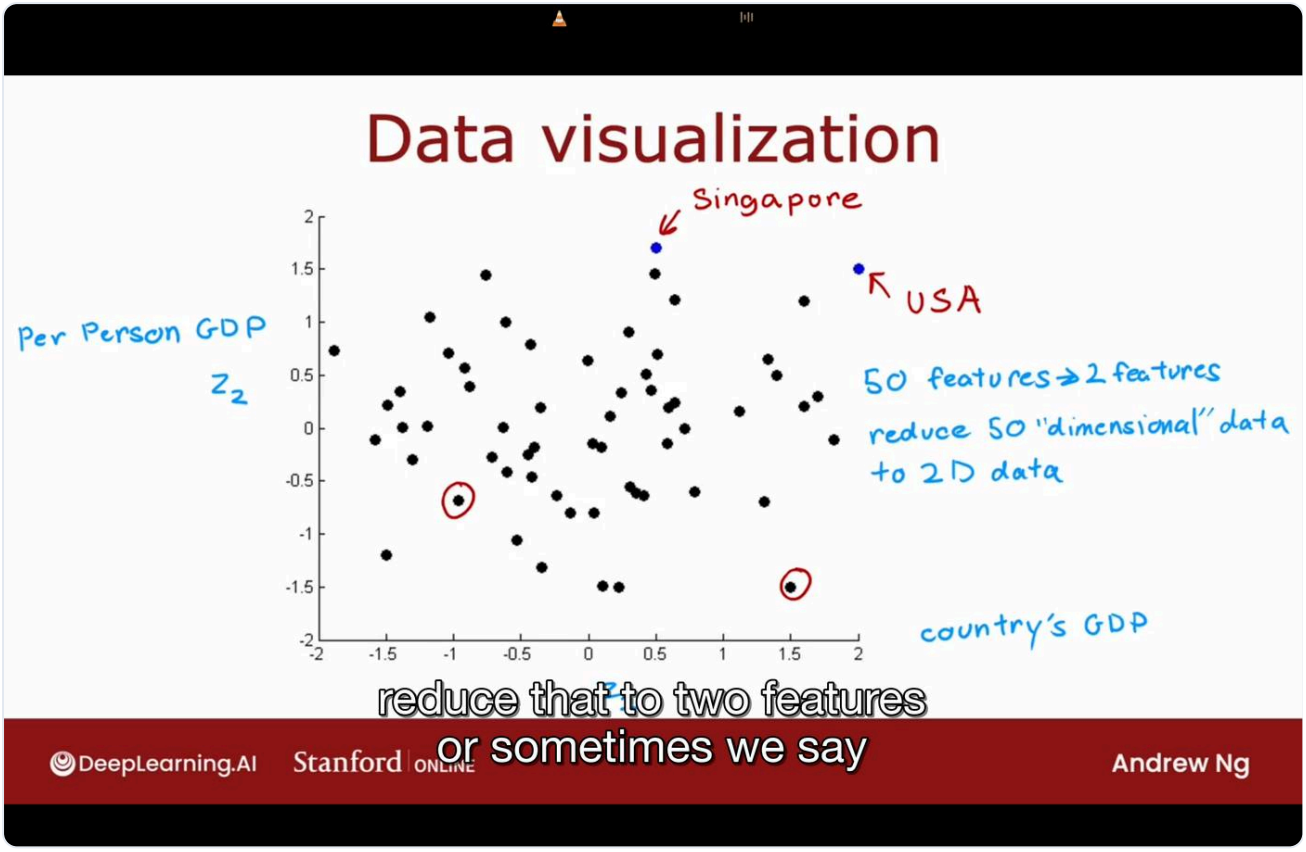


FIGURE 26.13 | Per Person GDP

It just means we have 50 features and reduce that to two features or sometimes we say is two-dimensional data because you can then plot it on this two-dimensional plot like you're seeing here. Whenever I get a new data set, one of the things I'll often want to do is to visualize the data since that helps me understand what the data looks like, what do the countries look like, or what do the cars seem like in this data set or whatever data you may be examining. You find also that visualizing the data set will sometimes help you figure out something funny's going on in this data. Something unexpected is happening. PCA is a powerful algorithm for taking data with a lot of features, with a lot of dimensions or high-dimensional data, and reducing it to two or three features to two or three dimensional data so you can plot it and visualize it and better understand what's in your data.

That's what the PCA algorithm can do for you. In the next video, let's start to take a look at how exactly the PCA algorithm works.

LECTURE 27

Choose an Axis

How does PCA work? If you have a dataset with two features, x_1 and x_2. Initially, your data is plotted or represented using axes x_1 and x_2. But you want to replace these two features with just one feature. How can you choose a new axis, let's call it the z-axis, that is somehow a good feature for capturing, of representing the data? Let's take a look at how PCA does this. Here's the data sets with five training examples. Remember, this is an unsupervised learning algorithm so we just have x_1 and x_2, there is no label y.

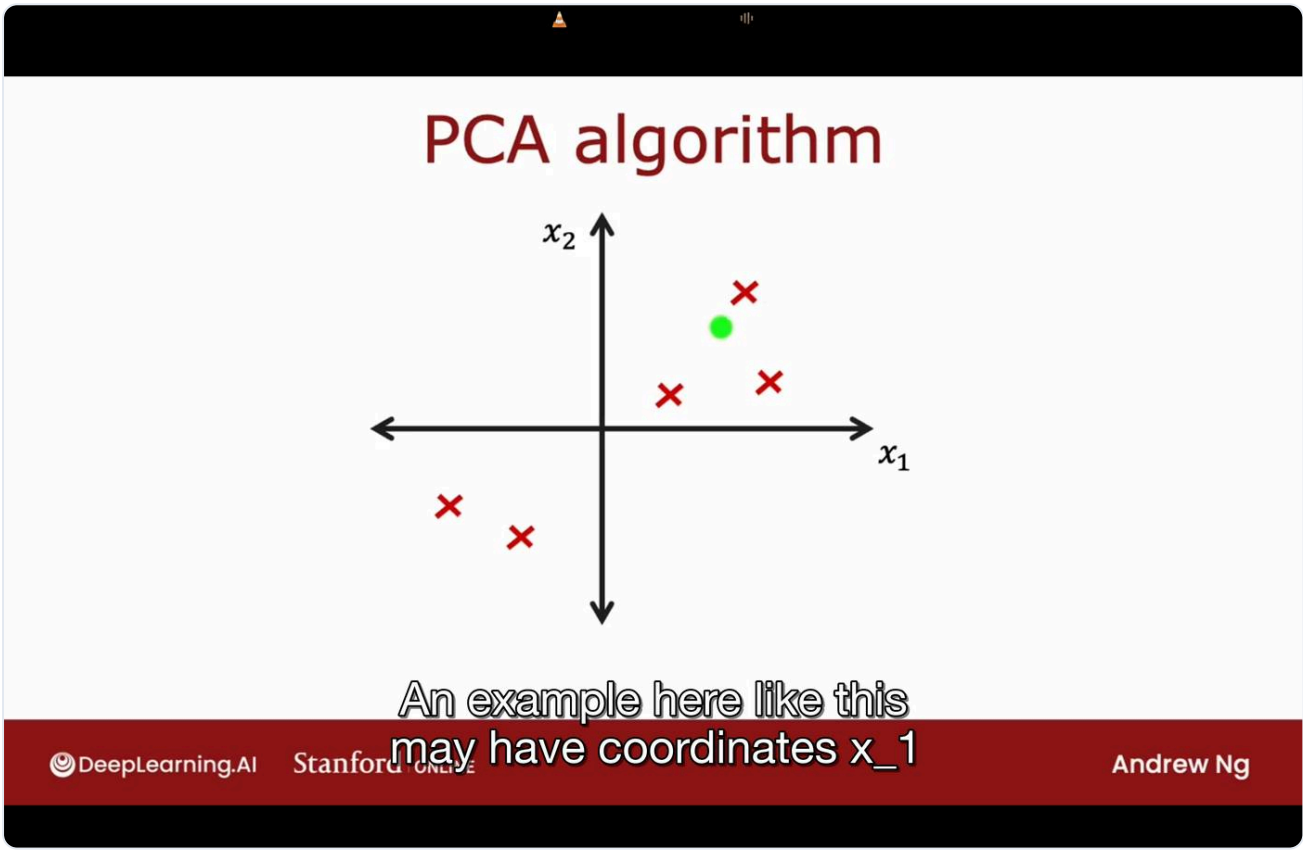


FIGURE 27.1

An example here like this may have coordinates x_1 equals 10 and x_2 equals 8. If we don't want to use the x_1, x_2 axes, how can we pick some different axis with which to capture what's in the data or with which to represent the data? One note on preprocessing, before applying the next few steps of PCA the features should be first normalized to have zero mean and I've already done that here. If the features x_1 and x_2 take on very different scales, for example, if you remember our housing example, if x_1 was the size of a house in square feet, and x_2 was the number of bedrooms then x_1 could be 1,000 or a couple of thousand, whereas x_2 is a small number.

If the features take on very different scales, then you will first perform feature scaling before applying the next few steps of PCA.

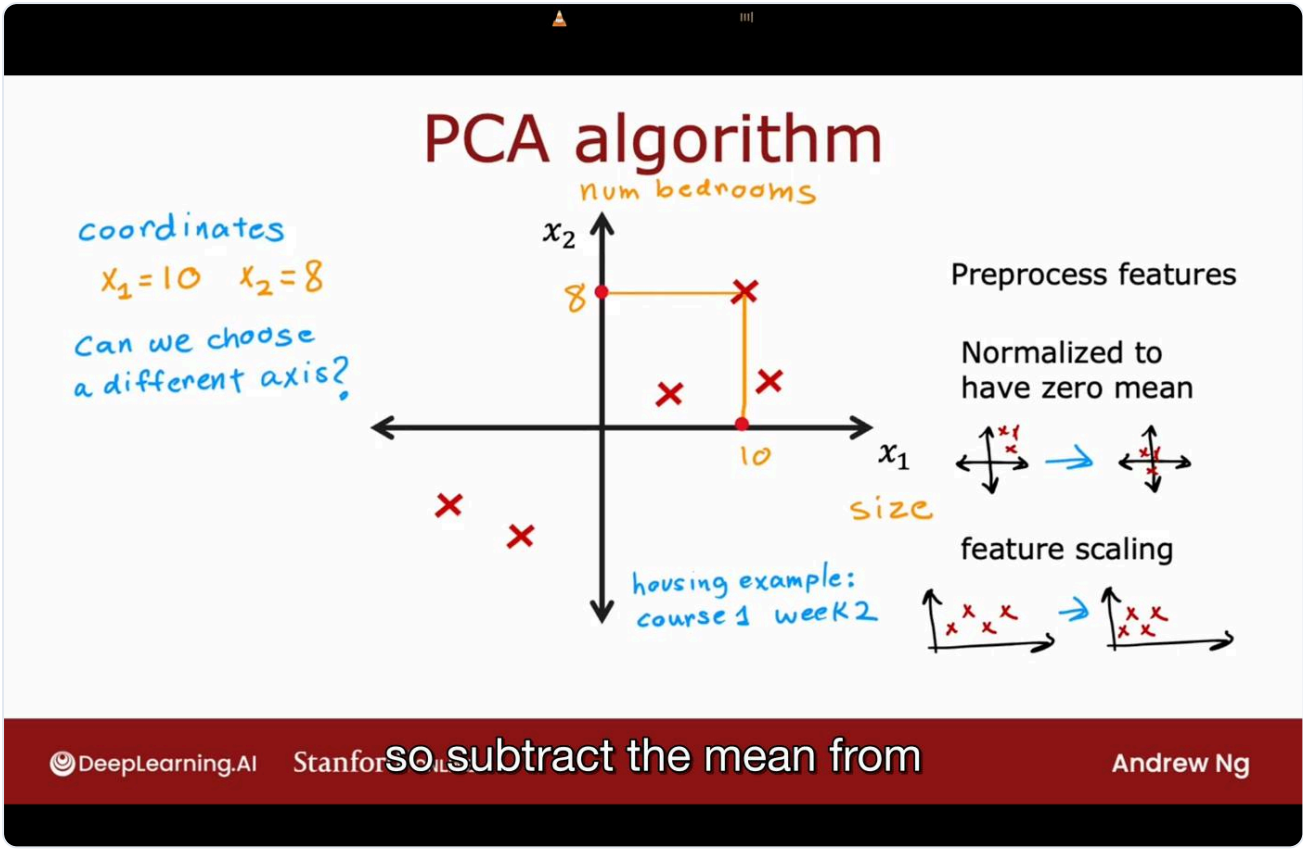


FIGURE 27.2 | Can We Choose

Assuming the features have been normalized to have zero mean, so subtract the mean from each feature and then maybe apply feature scaling as well so the ranges are not too far apart. What does PCA do next? To examine what PCA does, let me remove the x_1 and x_2 axes so that we're just left with the five training examples. This dot here represents the origin. The position of zero on this plot still. What we have to do now with PCA is, pick one axis instead of the two axes that we had previously with which to capture what's important about these five examples. If we were to choose this axis to be our new z-axis, it's actually the same as the x_1 axis just for this example.

Then what we're saying is that for this example, we're going to just capture this value, this coordinate on the z-axis. For the second example, we're going to capture this value, and then this will capture this value, and so on for all five examples. Another way of saying this is that we're going to take each of these examples and project it down to a point on the z-axis. The word project refers to that you're taking this example and bringing it to the z-axis using this line segment that's at a 90-degree angle to the z-axis. This little box here is used to denote that this line segment is at 90 degrees to the z-axis.

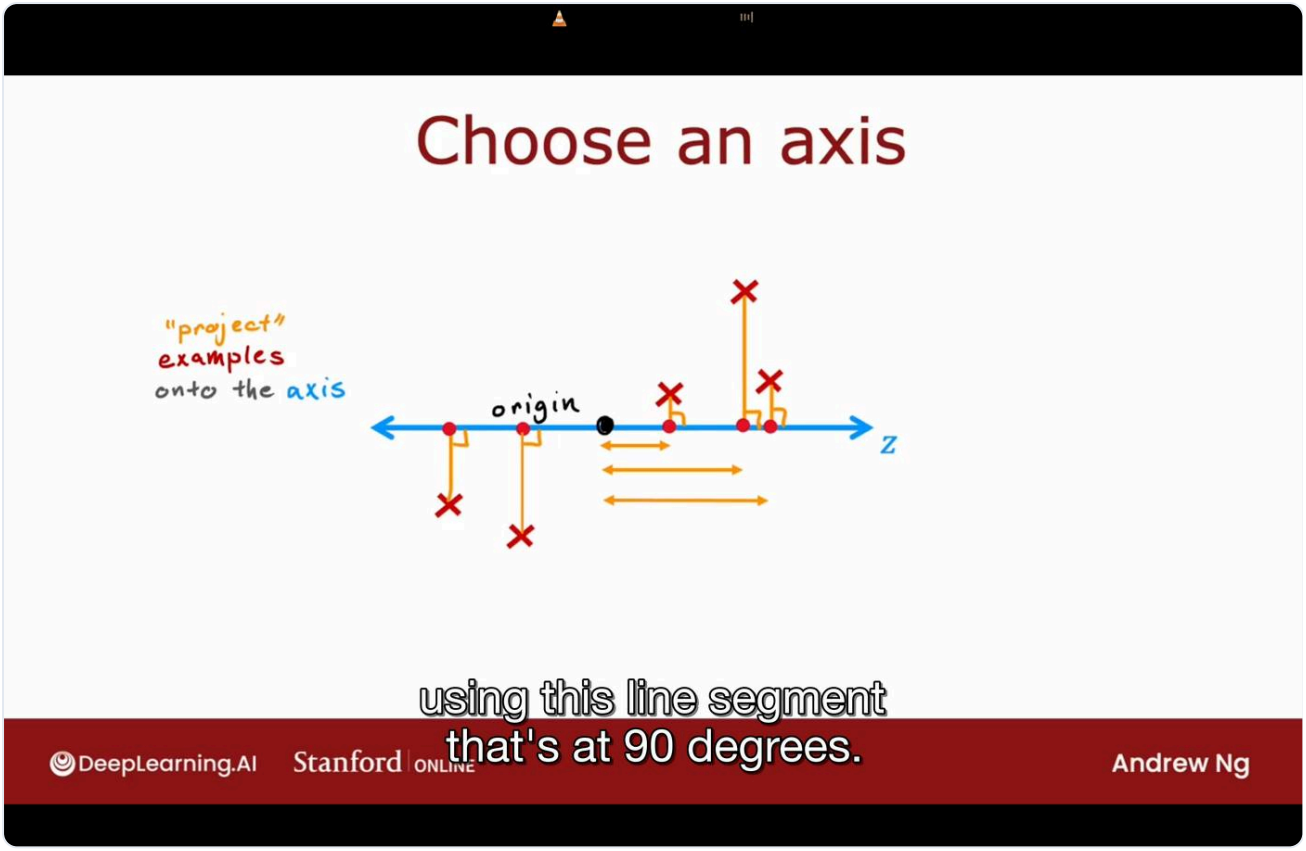


FIGURE 27.3 | Choose an Axis

The term project just means you're taking a point and finding this corresponding point on the z-axis using this line segment that's at 90 degrees. Picking this direction as a z-axis is not a bad choice, but there's some even better choices. This choice isn't too bad because when you project your examples onto the z-axis, you still capture quite a lot of the spread of the data. These five points here, they're pretty spread apart so you're still capturing a lot of the variation or a lot of the variance in the original dataset. By that I mean these five points are quite spread apart and so the variance or variation among these five points, the projections of the data onto the z-axis is decently large.

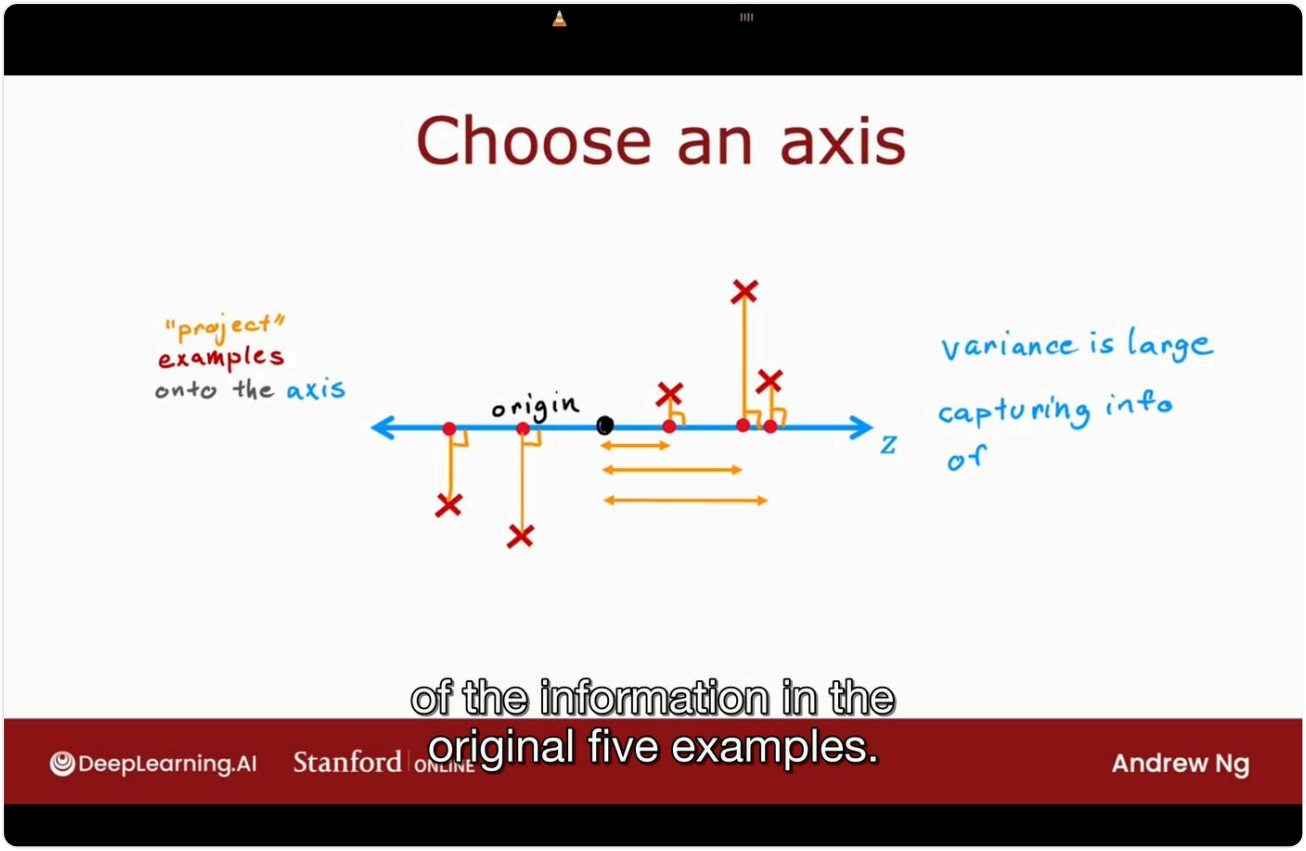


FIGURE 27.4 | Choose an Axis

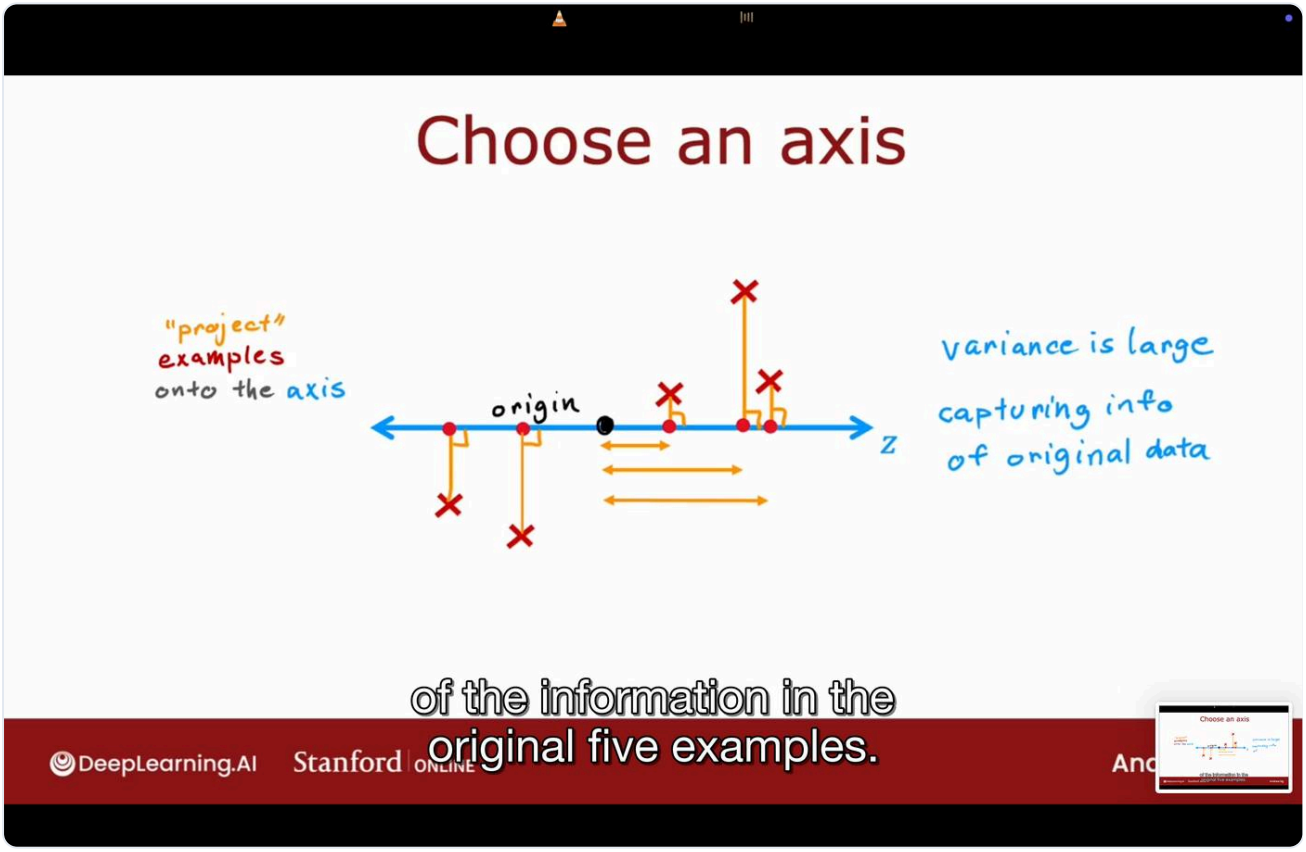


FIGURE 27.5 | Choose an Axis

What that means is we're still capturing quite a lot of the information in the original five examples. Let's look at some other possible choices for the axis z. Here's another choice. This is actually not a great choice. But if I were to choose this as my z axis, then if I take those same five examples and project them down to the z-axis, I end up with these five points. You notice that compare it to the previous choice, these five points are quite squished together. The amount they are differing from each other, or their variance or the variation is much less.

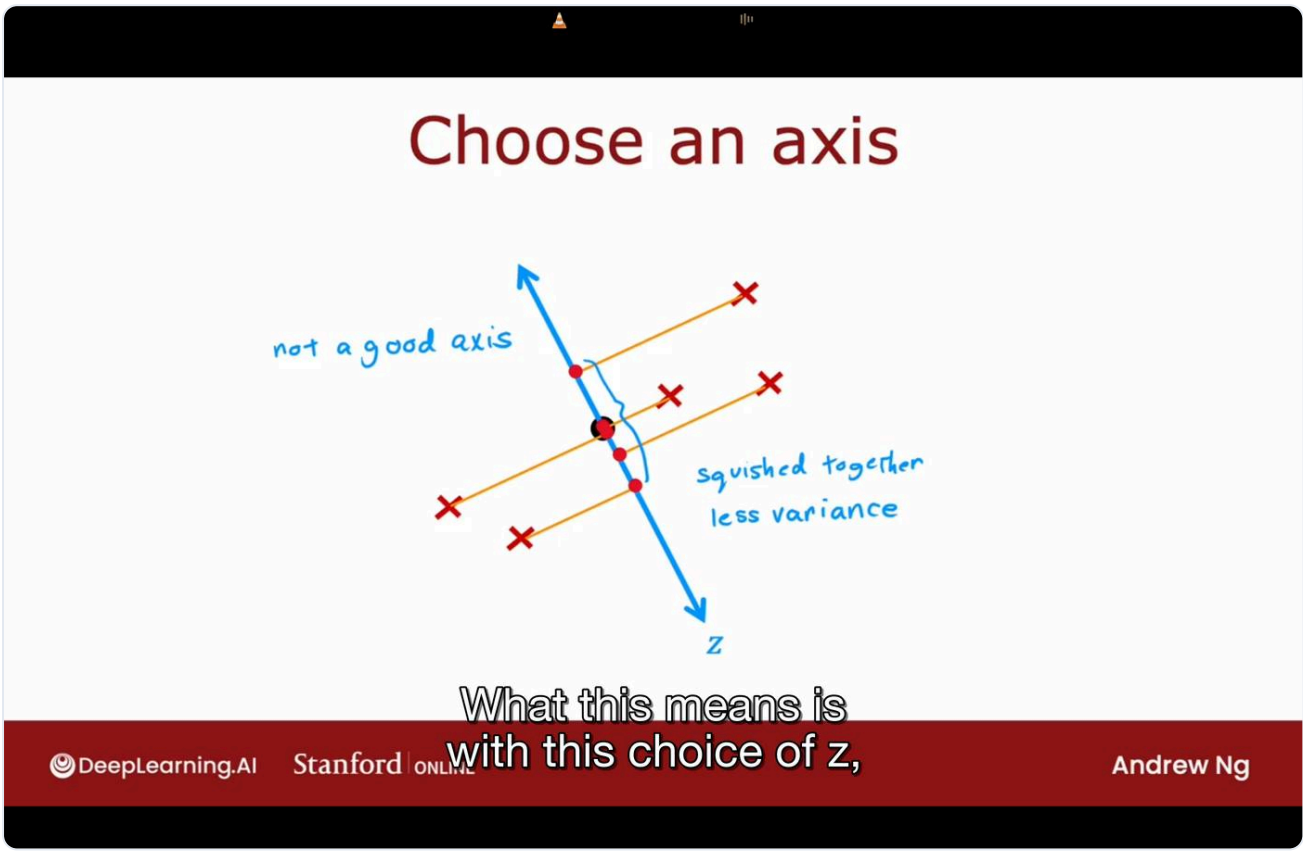


FIGURE 27.6 | Choose an Axis

What this means is with this choice of z, you're capturing much less of the information in the original dataset because you've partially squish all five examples together. Let's look at one last choice, which is if I choose this to be the z-axis. This is actually a better choice than the previous two that we saw, because if we take the data's projections onto the z-axis, we find that these dots over here, they're actually quite far apart.

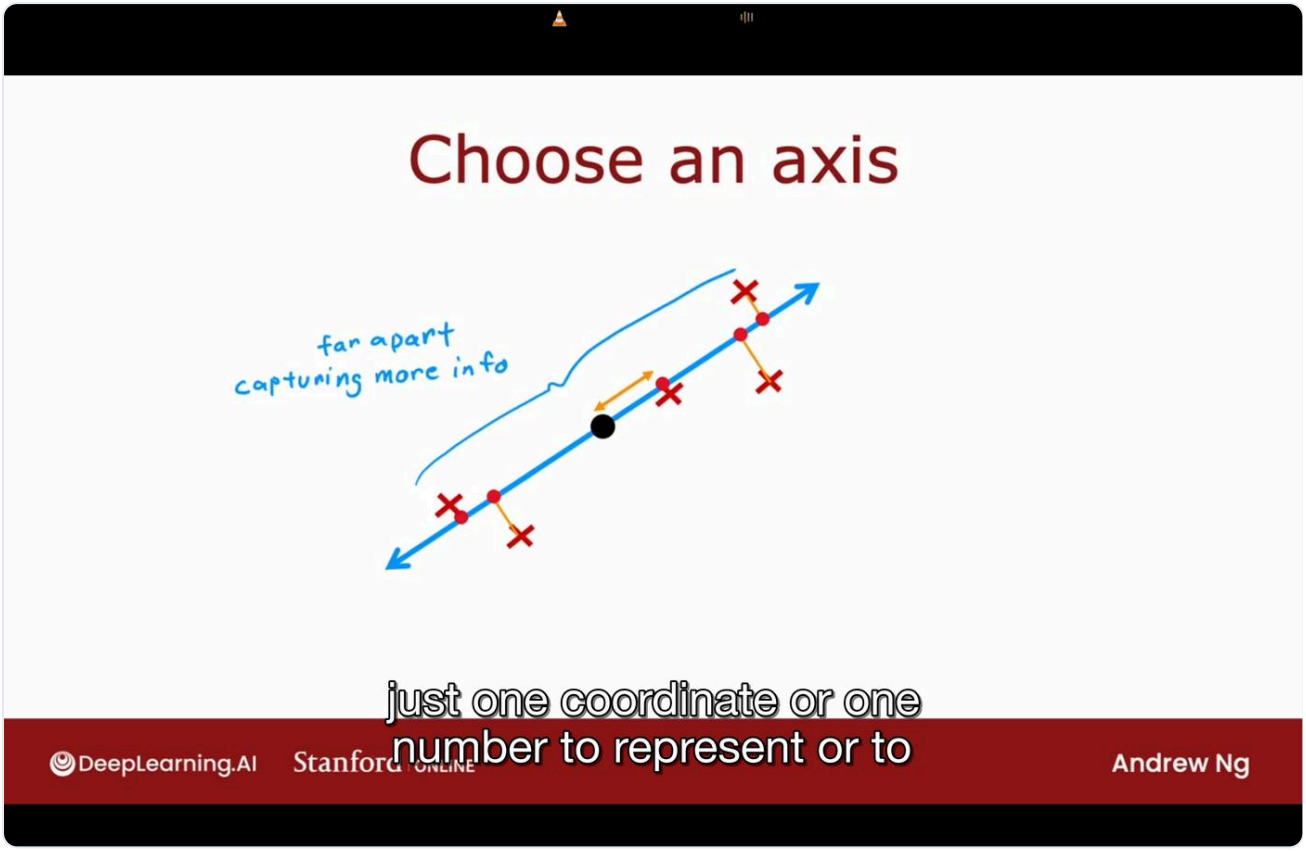


FIGURE 27.7 | Choose an Axis

We're capturing a lot of the variation, a lot of the information in the original dataset, even though we're now using just one coordinate or one number to represent or to capture each of the training examples instead of two numbers or two coordinates, X_1 and X_2. In the PCA algorithm, this axis is called the principal component.

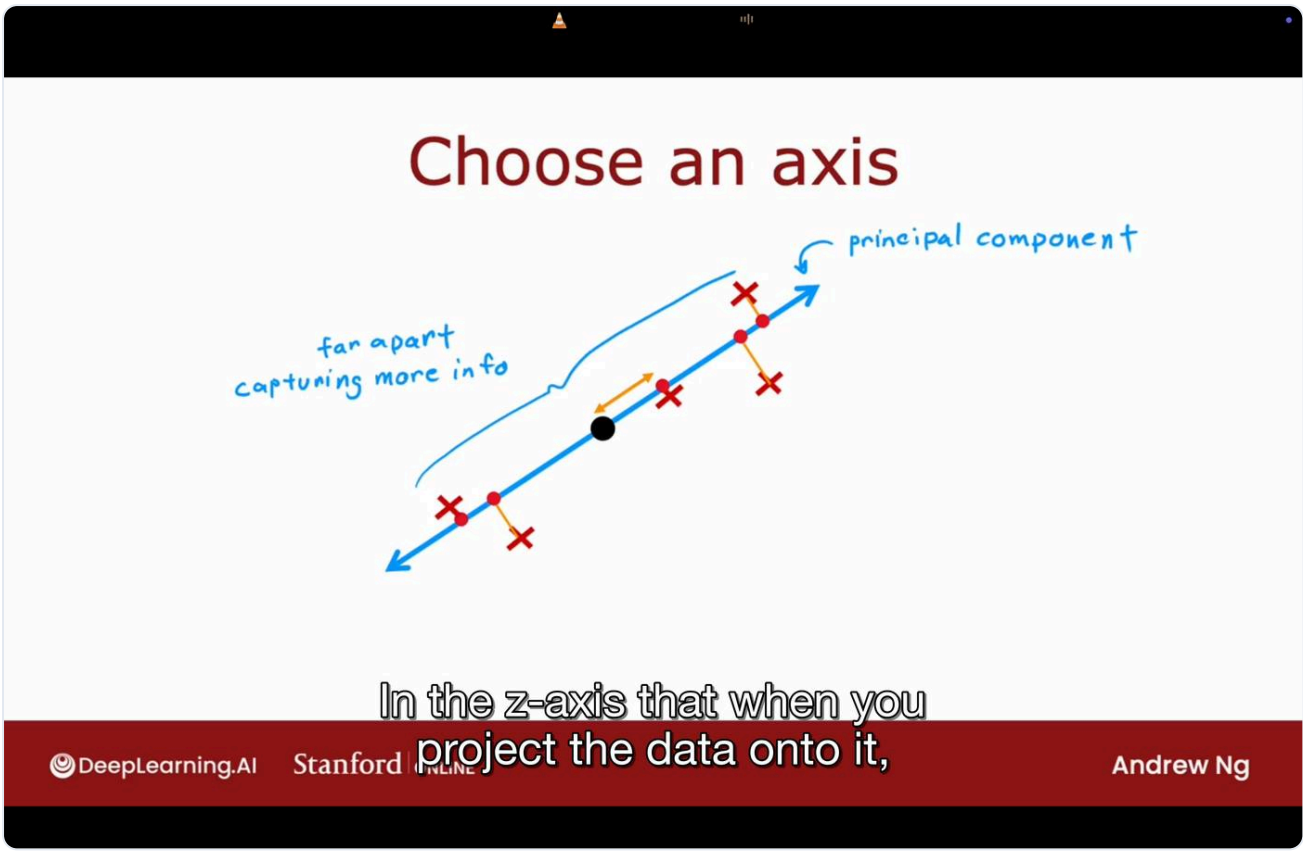


FIGURE 27.8 | Choose an Axis

In the z-axis that when you project the data onto it, you end up with the largest possible amounts of variance.

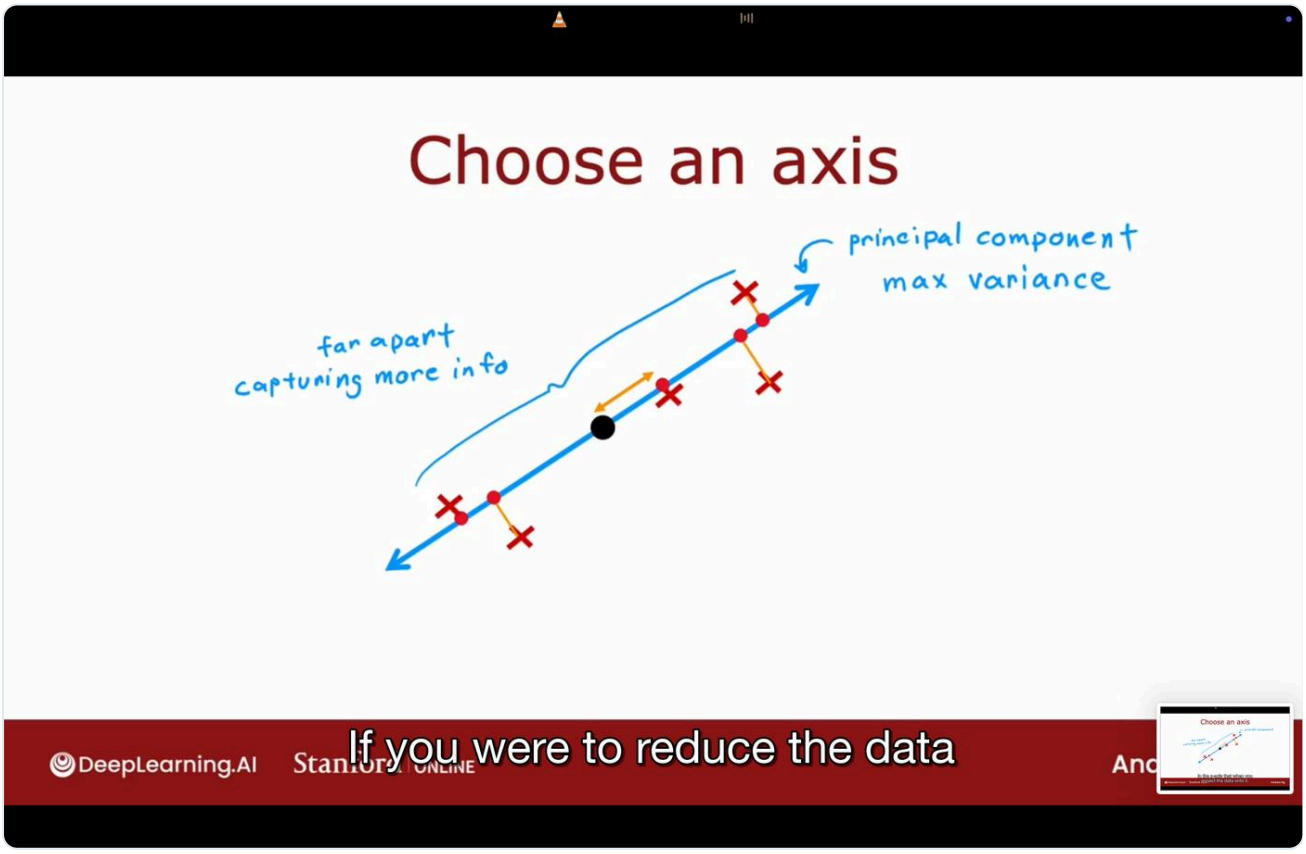


FIGURE 27.9 | Choose an Axis

If you were to reduce the data to one axis or to one feature, this principal component is actually a good choice, and this is what PCA will do. If you want to reduce the data to one-dimensional feature, then it will choose this principal component. Let me show you a visualization of how different choices of the axis affects the projection. Here we have 10 training examples, and as we slide this slider here, and you can play with this in one of the optional labs yourself. As you slide the slider here, the angle of the z-axis changes. What you're seeing on the left is each of the examples projected via that short line segment at 90 degrees to the z-axis.

Here on the right is that projection of the data, meaning the value of these 10 examples, z-coordinate. You notice that when I set the axis to about here, the points are quite squished together. So this posses less of the automation of the original data. Whereas if I set the z-axis say to this, then these points vary much more. This is capturing much more of the information in the original dataset. That's why the principal component corresponds to setting the z-axis to about here. This is the choice that PCA would make if you asked it to reduce the data to one dimension. Machine learning library, like scikit-learn, which you'll hear more about in the next video, can help you automatically find the principal component.

But let's take a little bit deeper into how that works. Here are my x_1 and x_2 axis. Here's one training example with coordinates 2 on the x_1 axis and three on the x_2 axis. Let's say that PCA has found this direction for the z-axis. What I'm drawing here, this little arrow is a length 1 vector pointing in the direction of this z-axis that PCA will choose or that we have chosen. It turns out this length 1 vector is the vector 0.710, 0.71 rounded off a bit. It's actually 0.707 and then a bunch of other digits. Given this example with coordinates 2,3 on the x_1, x_2 axis, how do we project this example onto the z-axis? It turns out the formula for doing so is to take a dot product between the vector 2,3 and this vector 0.71, 0.71.

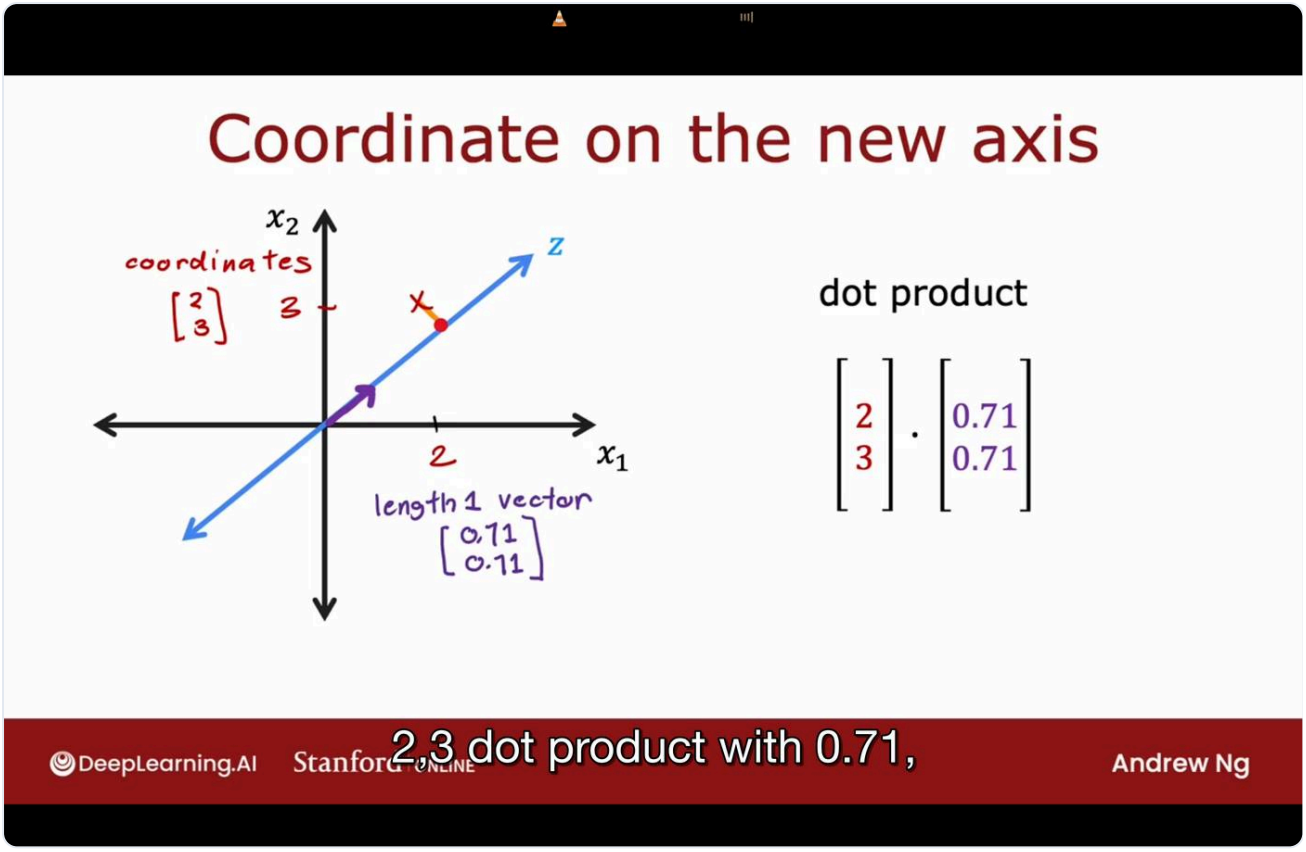


FIGURE 27.10 | Coordinate on the New Axis

If you do that, 2,3 dot product with 0.71, 0.71 turns out to be 2 times 0.71 plus 3 times 0.71, which is equal to 3.55.

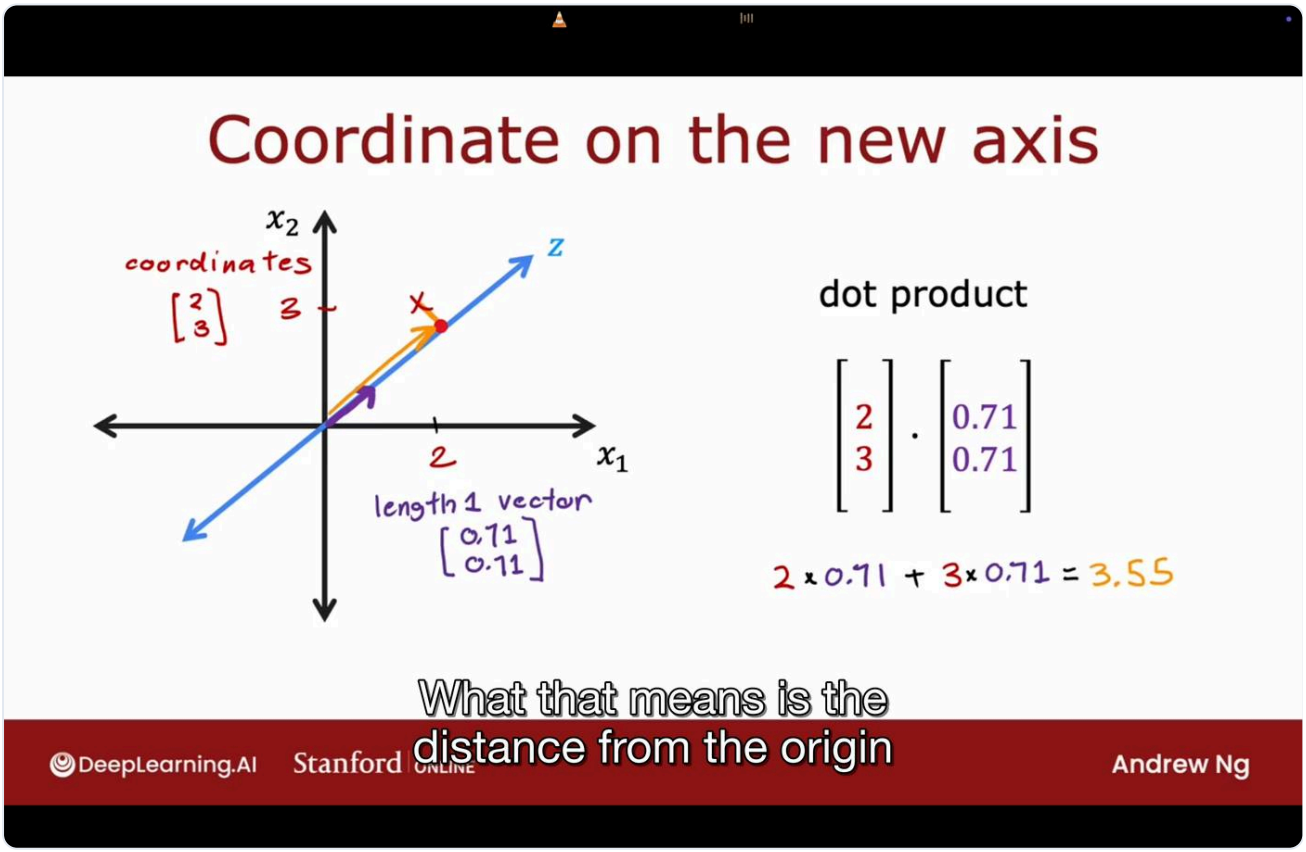


FIGURE 27.11 | Coordinate on the New Axis

What that means is the distance from the origin of this point over here is 3.55, which means that if we were to represent or to use one number to try to capture this example, that one number is 3.55.

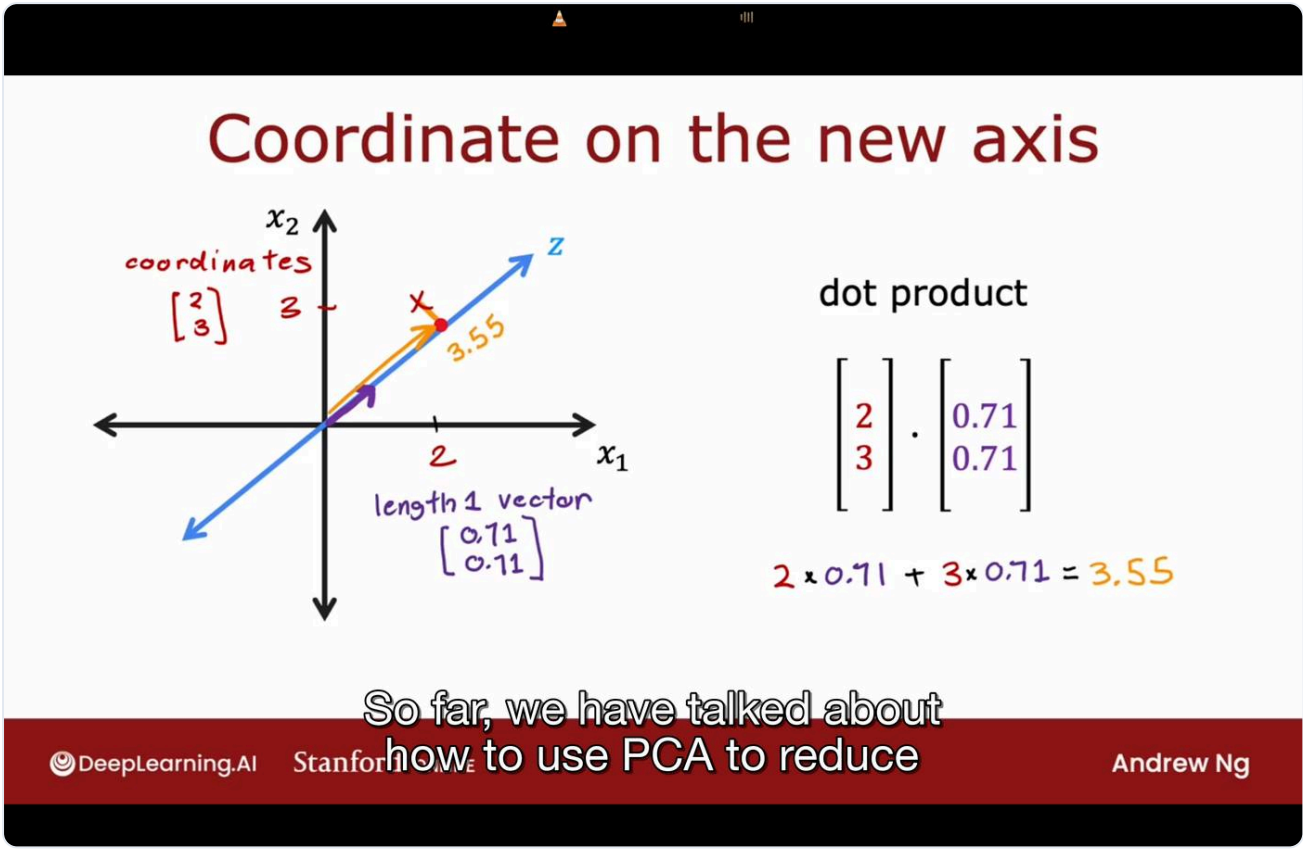


FIGURE 27.12 | Coordinate on the New Axis

So far, we have talked about how to use PCA to reduce data down to one dimension or down to one number. We did so by finding the principal component, also called sometimes the first principal component. In this example, we had found this as the first axis. It turns out that if you were to pick a second axis, the second axis will always be at 90 degrees to the first axis. If you were to choose even a third axis, then the third axis will be at 90 degrees to the first and the second axis.

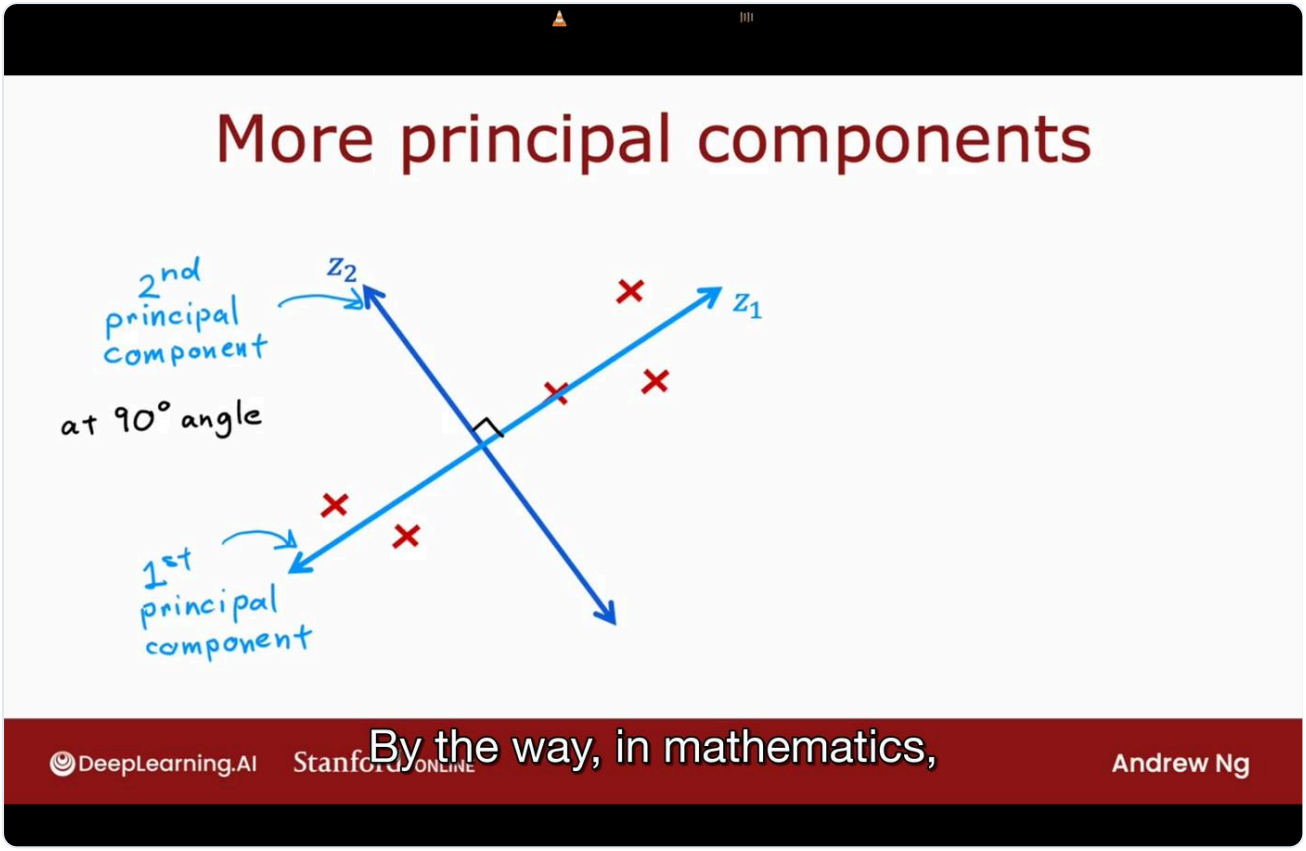


FIGURE 27.13 | More Principal Components

By the way, in mathematics, 90 degrees is sometimes called perpendicular. The term perpendicular just means at 90 degrees. Mathematicians will sometimes say the second axis, z_2, is at 90 degrees or is perpendicular to the first axis, z_1. If you choose additional axes, they're also at 90 degrees or perpendicular to z_1 and z_2 and to any other axes that PCA will choose. If you had 50 features and wanted to find three principal components, then if that's the first axis, the second axis will be at 90 degrees to it. Then the third axis will also be at 90 degrees to the first and the second axis. Now, one question I'm often asked is, how is PCA different from linear regression?

It turns out PCA is not linear regression, is a totally different algorithm. Let me explain why.

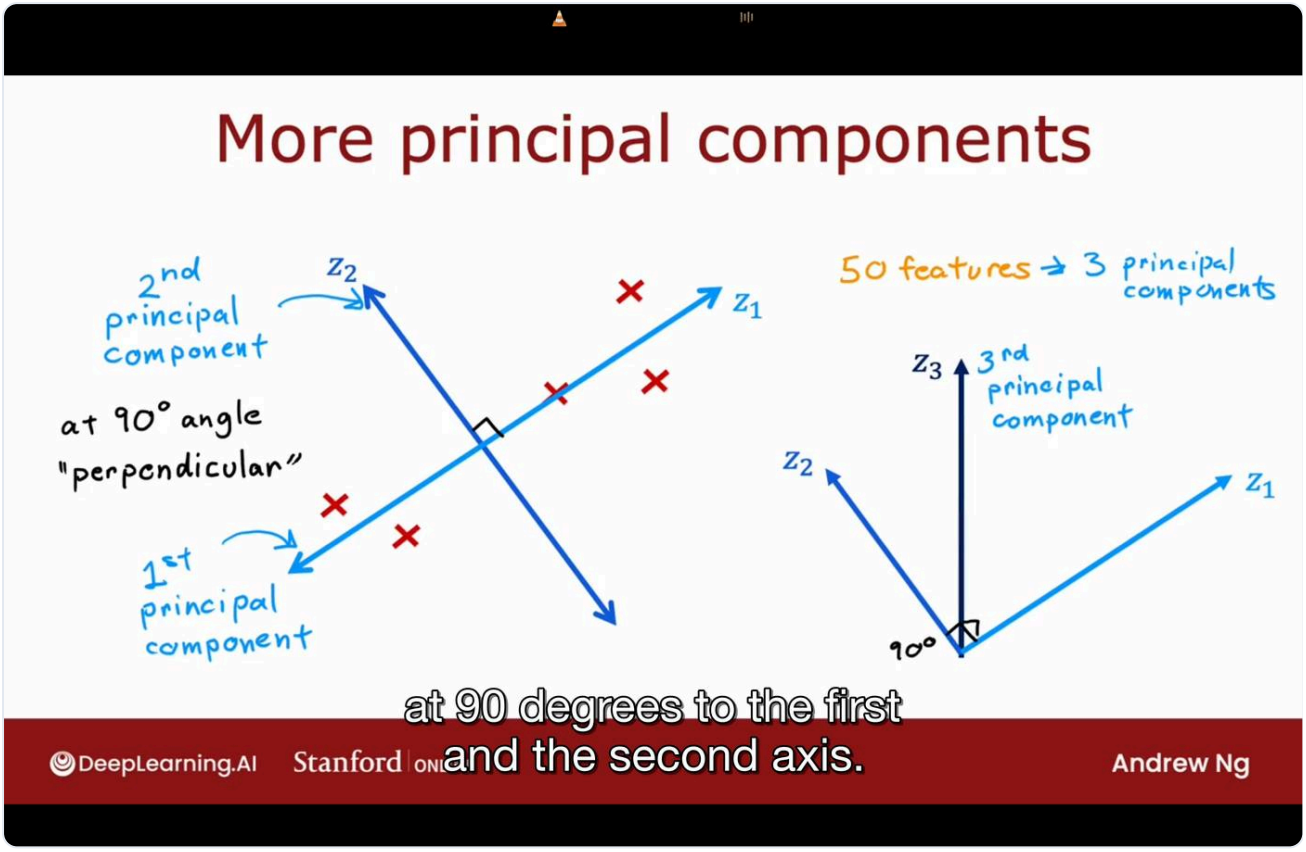


FIGURE 27.14 | More Principal Components

With linear regression, which is a supervised learning algorithm, you have data x and y. Here's a data set where the horizontal axis is the feature x and the vertical axis here is the label y. With linear regression you're trying to fit a straight line so that the predicted value is as close as possible to the ground truth label y. In other words, you're trying to minimize the length of these little line segments which are in the vertical direction. They just aligned with the y axis. In contrast, in PCA, there is no ground truth label y. You just have unlabeled data, X1 and X2, and furthermore, you're not trying to fit a line to use X1 to predict X2.

Instead, the average treats X1 and X2 equally. We're trying to find this axis Z, that it turns out we end up making these little line segments small when you project the data onto Z.

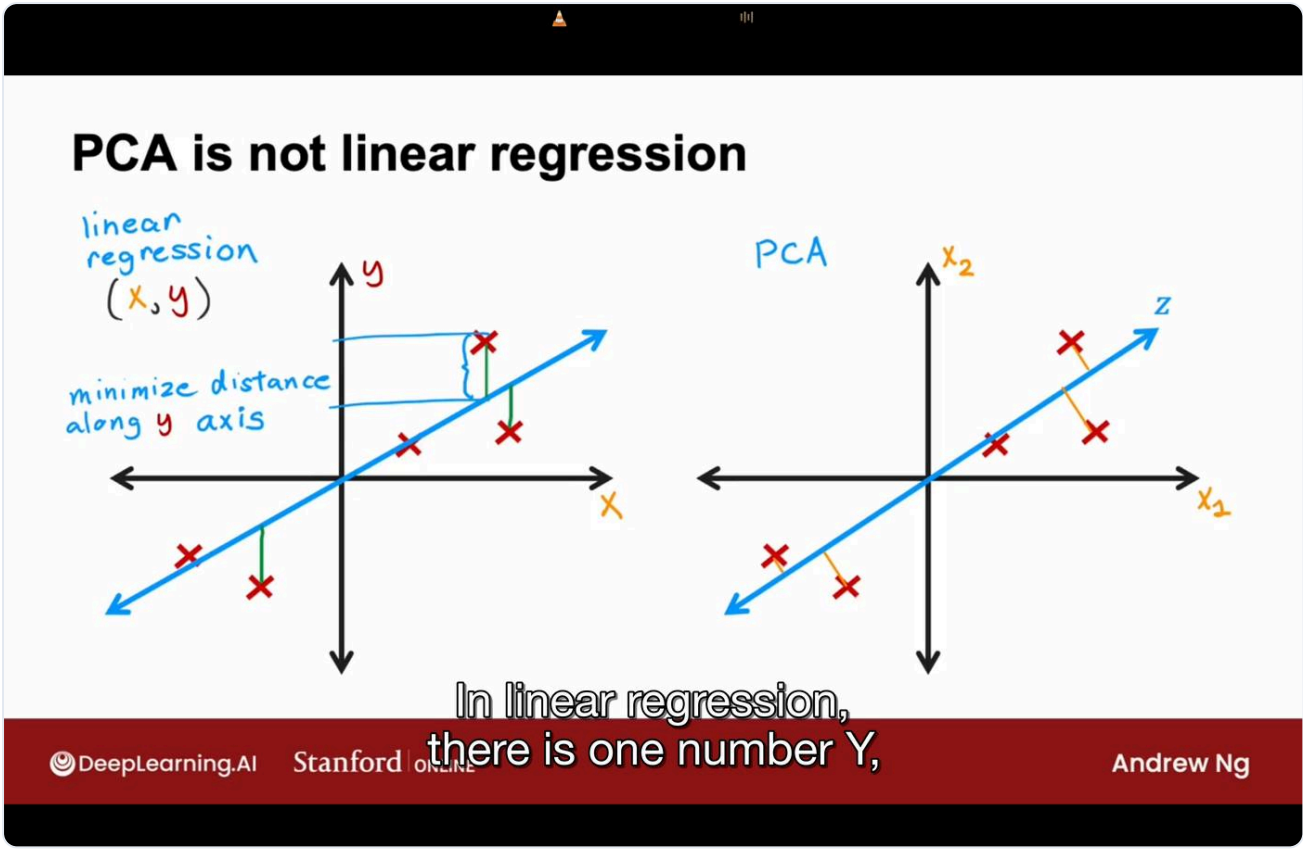


FIGURE 27.15 | PCA Is Not Linear Regression

In linear regression, there is one number Y, which is given very special treatment. We're always trying to measure distance between the fitted line and Y, which is why these distances are measured just in the direction of the y-axis. Whereas in PCA, you can have a lot of features, X1, X2, maybe all the way up to X50 if you have 50 features. All 50 features are treated equally.

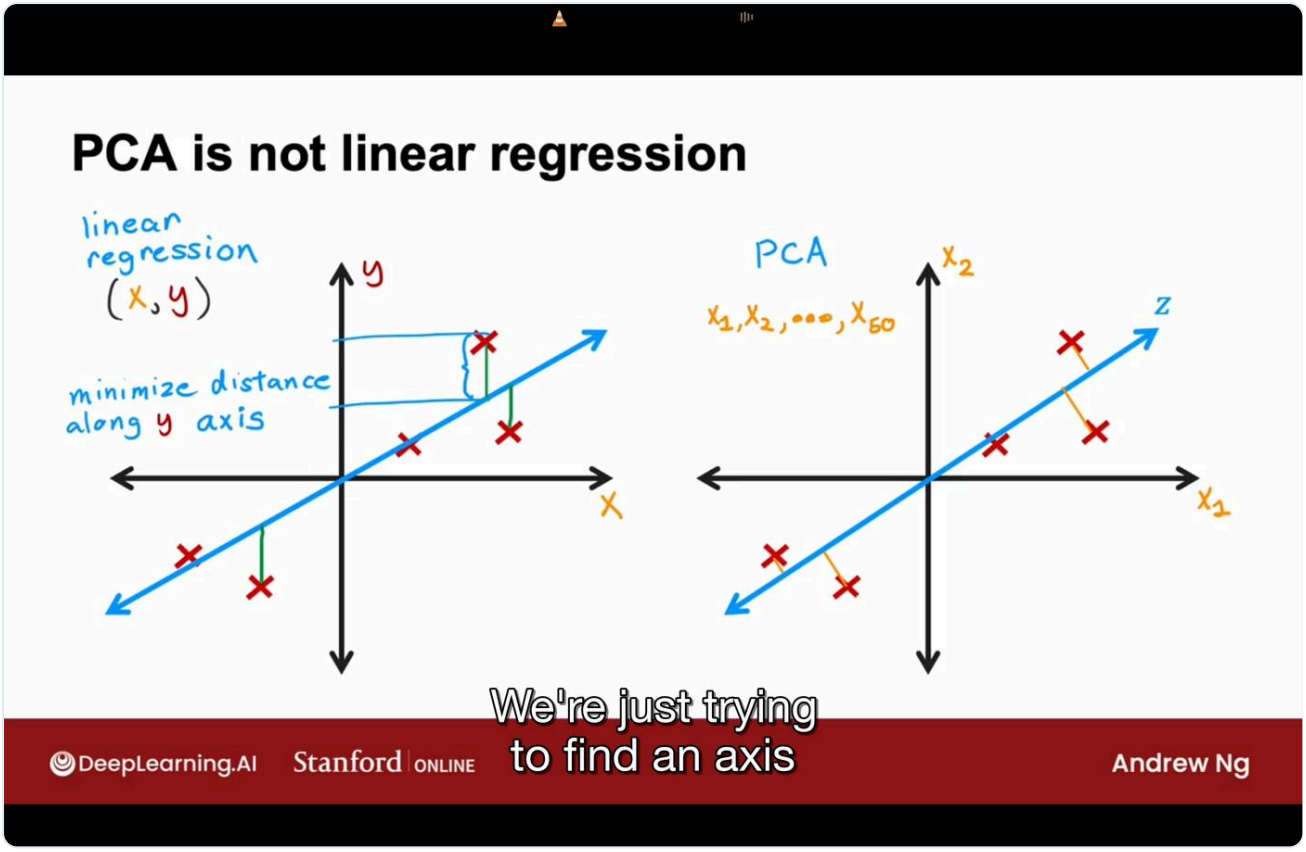


FIGURE 27.16 | PCA Is Not Linear Regression

We're just trying to find an axis Z so that when the data is projected onto the axis Z using these line segments that you still retain as much of the variance of the original data as possible.

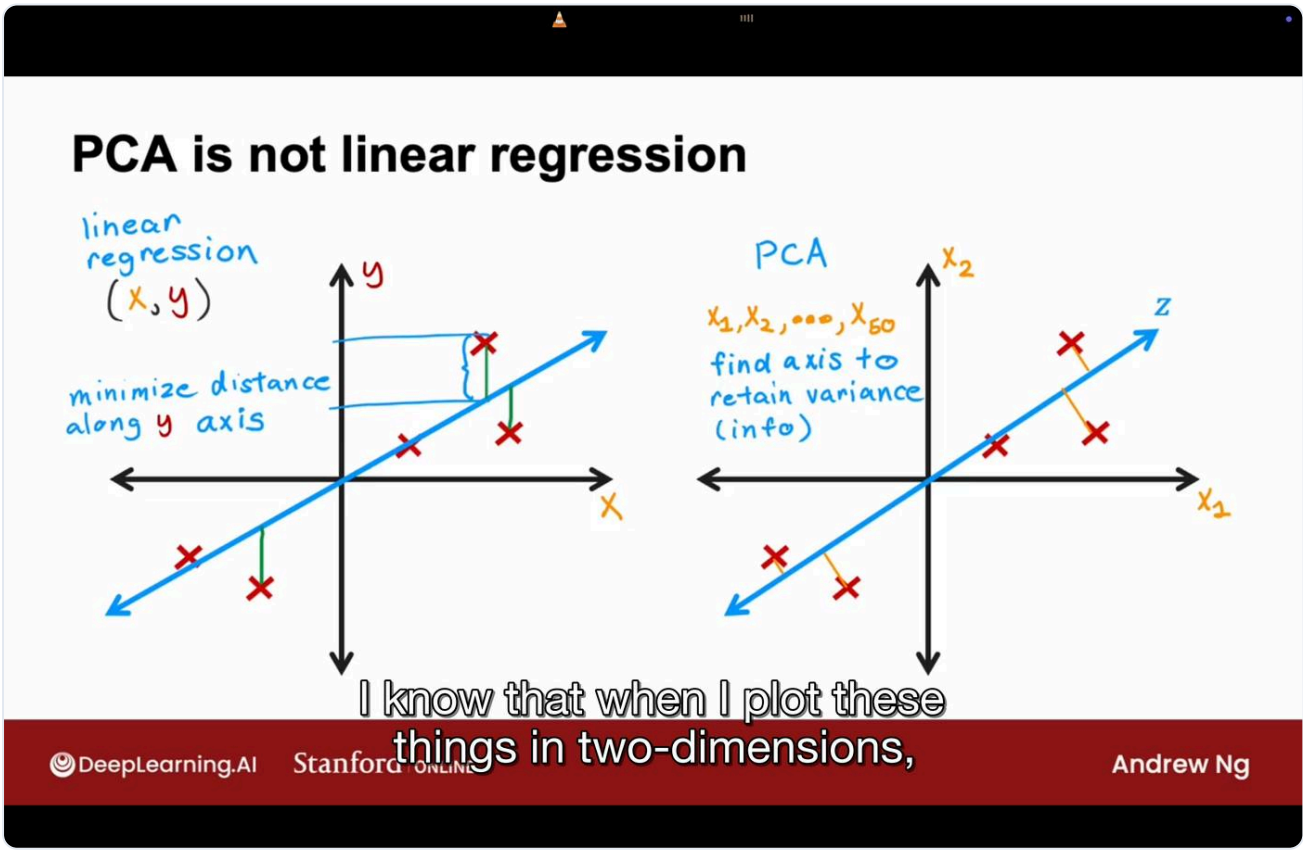


FIGURE 27.17 | PCA Is Not Linear Regression

I know that when I plot these things in two-dimensions, we've just two features, which is, I can draw on a flat computer monitor. These arrows look like maybe they're a little bit similar. But when you have more than two features, which is most of the case, the difference between linear regression and PCA and what the algorithms do is very large. These algorithms are used for totally different purposes and give you very different answers. When linear regression is used to predict a target output Y and PCA is trying to take a lot of features and treat them all equally and reduce the number of axis needed to represent the data well. It turns out that maximizing the spread of these projections will correspond to minimizing the distances of these line segments, the distances to the points have to move to be projected down to Z.

To illustrate the difference between linear regression and PCA in another way, if you have a data set that looks like this, linear regression, all it can do is fit a line that looks like that. Whereas if your data set looks like this, PCA will choose this to be the principal component.

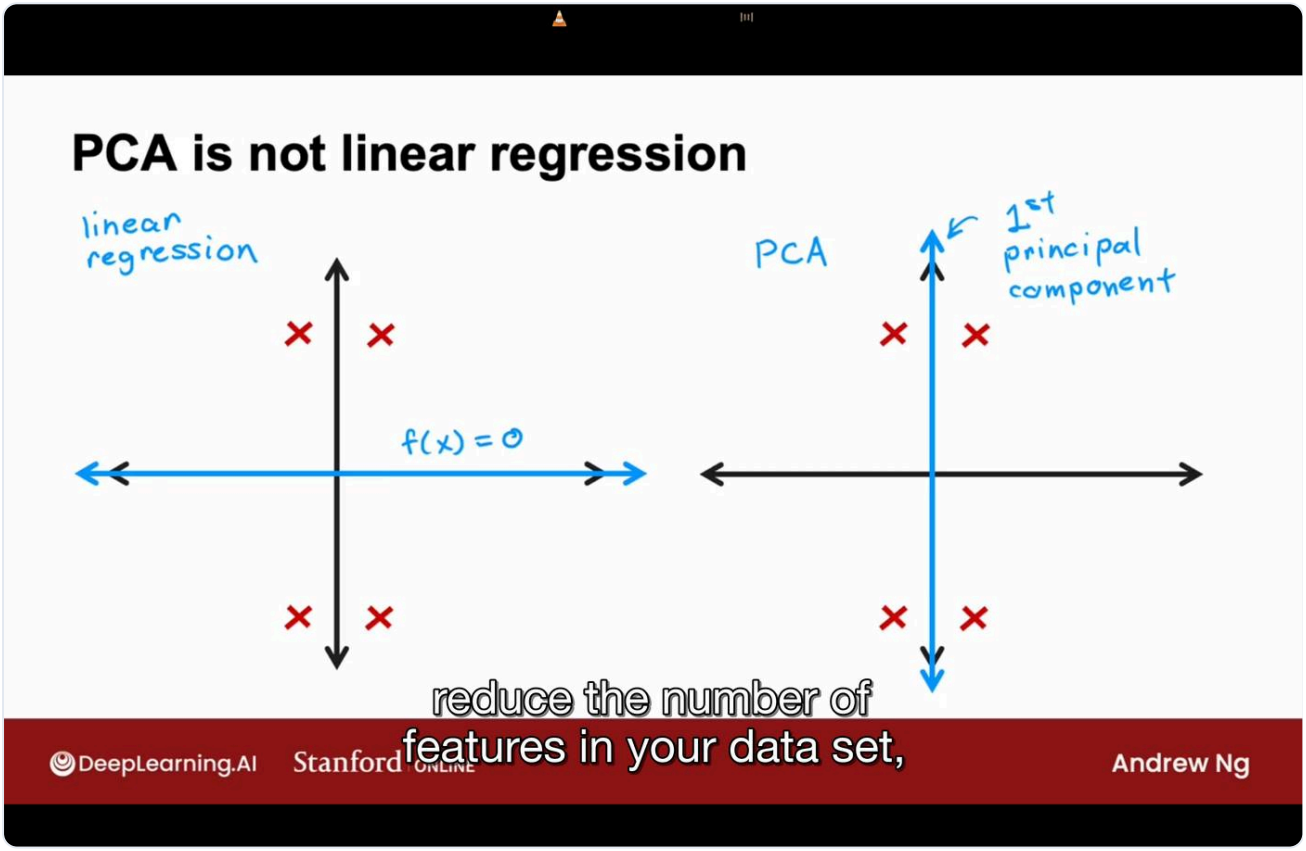


FIGURE 27.18 | PCA Is Not Linear Regression

So you should use linear regression if you're trying to predict the value of y, and you should use PCA if you're trying to reduce the number of features in your data set, say to visualize it. Finally, before we wrap up this video, there's one more thing you could do with PCA, which is, recall this example which was at coordinates 2,3. We found that if you projected to the z-axis, you end up with 3.55. One thing you could do is if you have an example where Z equals 3.55, given just this one number Z, 3.55, can we try to figure out what was the original example? It turns out that there's a step in PCA called reconstruction, which is to try to go from this one number Z equals 3.55 back to the original two numbers, X1 and X2.

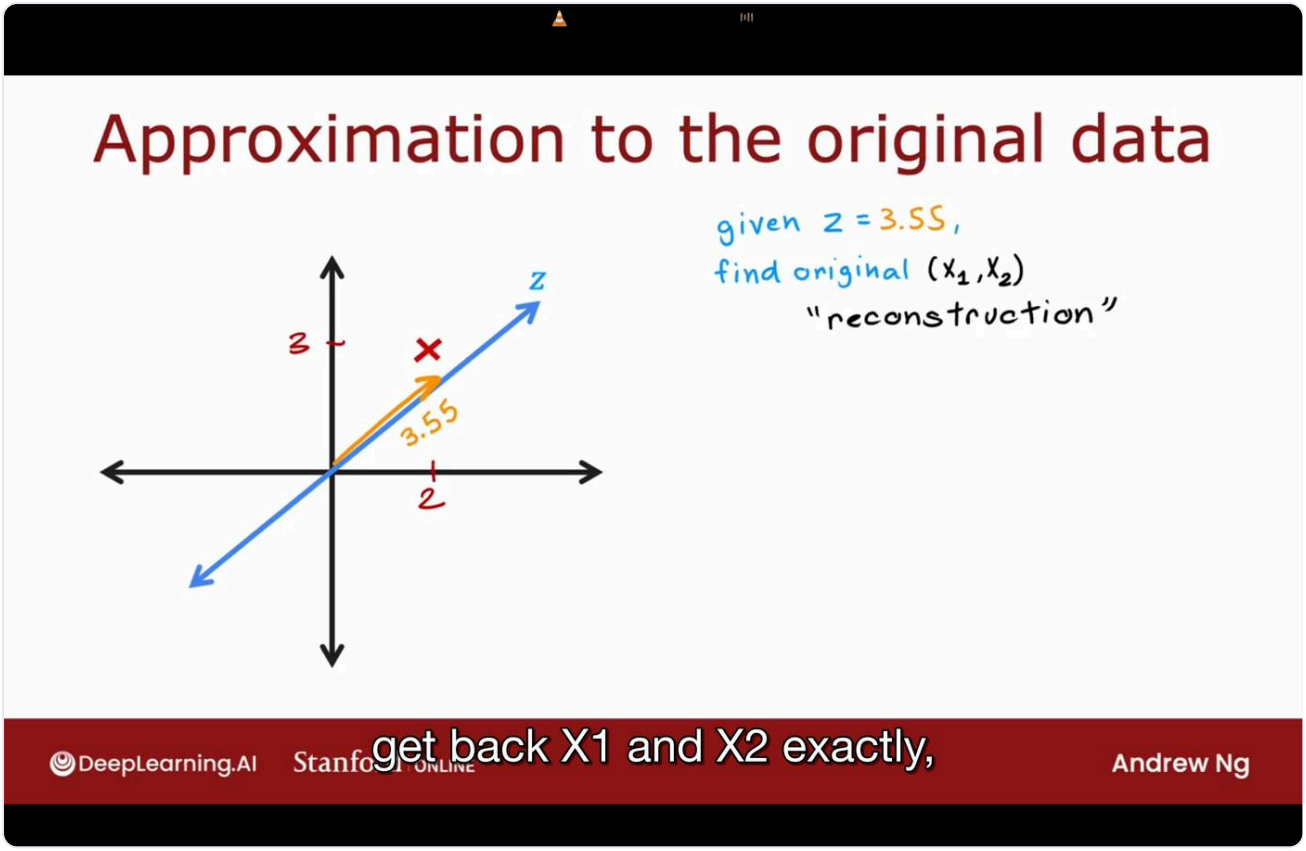


FIGURE 27.19 | Approximation to the Original Data

It turns out you don't have enough information to get back X1 and X2 exactly, but you can try to approximate it. In particular, the formula is, you would take this number 3.55, which is Z, and multiply it by the length one vector that we had just now, which is 0.71, 0.71.

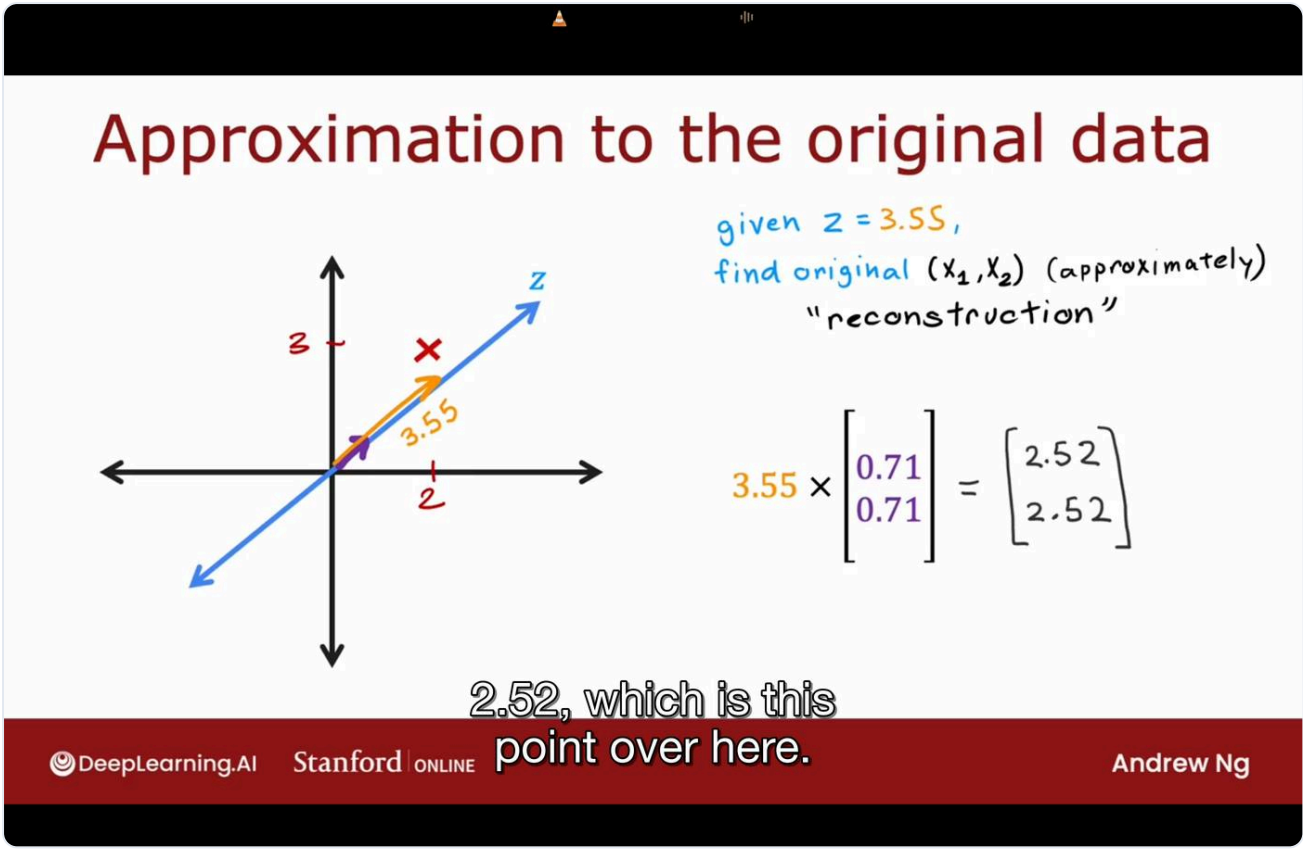


FIGURE 27.20 | Approximation to the Original Data

This ends up to be 2.52, 2.52, which is this point over here. We can approximate the original training example, which was a coordinates 2, 3 with this new point here, which is at 2.52, 2.52. The difference between the original point and the projected point is this little line segment here.

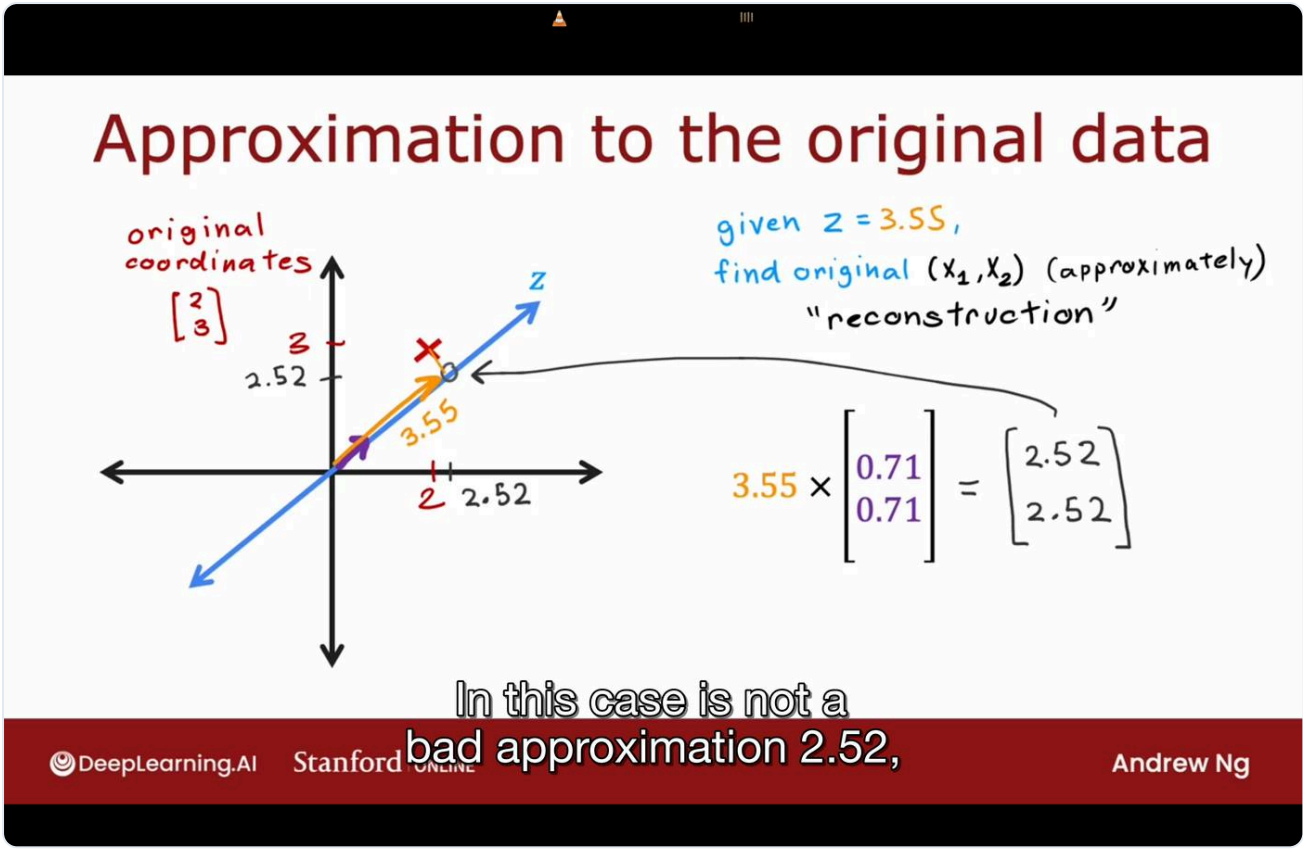


FIGURE 27.21 | Approximation to the Original Data

In this case is not a bad approximation 2.52, 2.52 is not that far from 2, 3. With just one number, we could get a reasonable approximation to the coordinates of the original training example. This is called the reconstruction step of PCA. To summarize, the PCA algorithm looks at your original data and chooses one or more new axis, Z or maybe Z1 and Z2, to represent your data and by taking your original data set and projecting it onto your new axis or axis. This gives you a smaller set of numbers so you can plot if wished to visualize your data. You're seeing the math. Let's now take a look at how you can implement this in code. In the next video, we'll look at how you can use PCA yourself using the scikit-learn library.

Let's go on to the next video.

LECTURE 28

Applications of PCA

In this video, we'll take a look at how you can use the scikit-learn library to implement PCA. These are the main steps. First, if your features take on very different ranges of values, you can perform pre-processing to scale the features to take on comparable ranges of values. If you were looking at the features of different countries, those features take on very different ranges of values. GDP could be in trillions of dollars, whereas other features are less than 100. Feature scaling in applications like that would be important to help PCA find a good choice of axes for you. The next step then is to run the PCA algorithm to "fit" the data to obtain two or three new axes, Z_1, Z_2, and maybe Z_3.

Here I'm assuming you want two or three axes if you want to visualize the data in 2D or 3D. If you have an application where you want more than two or three axes, the PCA implementation can also give you more than two or three axes, it's just that it'd then be harder to visualize. In scikit-learn, you will use the fit function, or the fit method in order to do this. The fit function in PCA automatically carries out mean normalization, it subtracts out the mean of each feature. So you don't need to separately perform mean normalization. After running the fit function, you would get the new axes, Z_1, Z_2, maybe Z_3, and in PCA, we also call these the principal components, where Z_1 is the first principal component, Z_2 the second principal component, and Z_3 the third principal component.

After that, I would recommend taking a look at how much each of these new axes, or each of these new principal components explains the variance in your data.

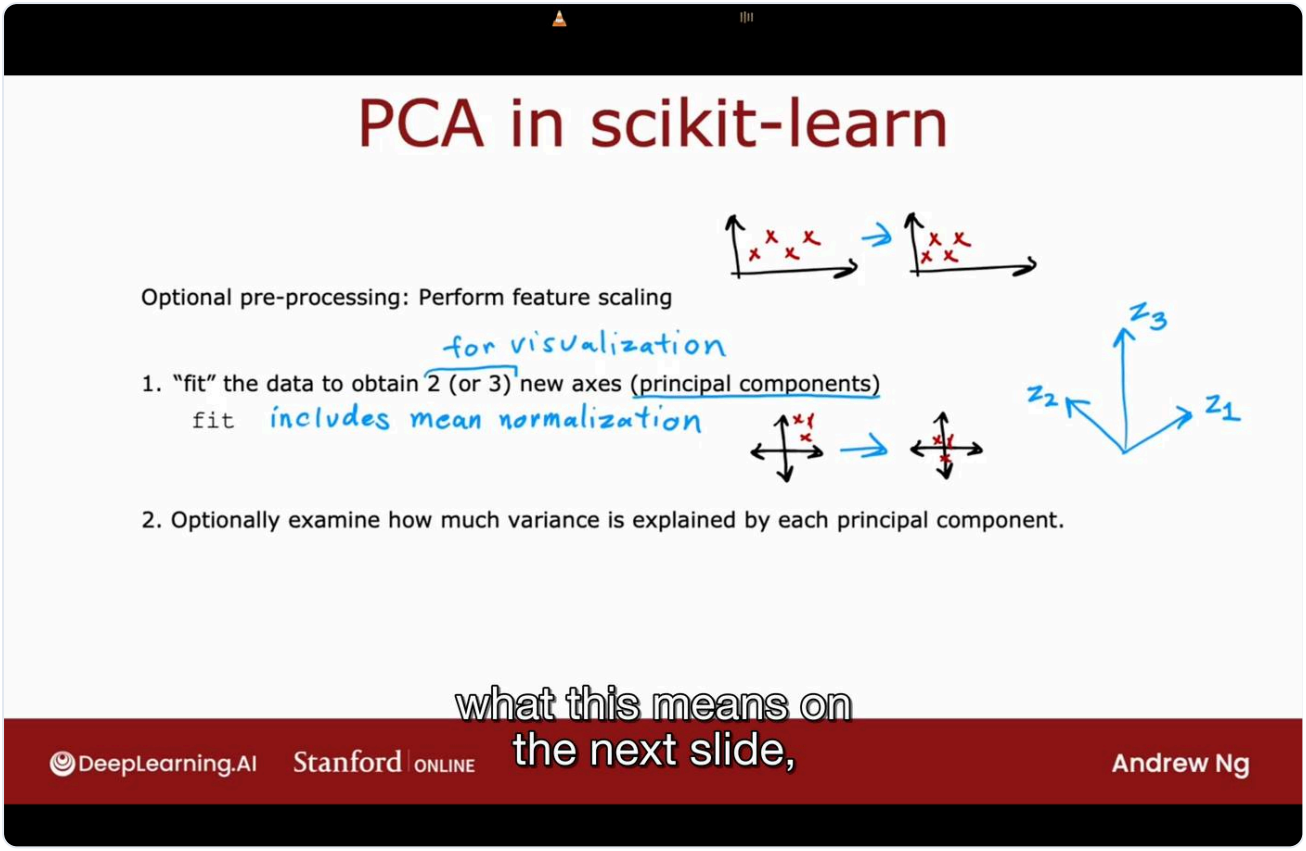


FIGURE 28.1

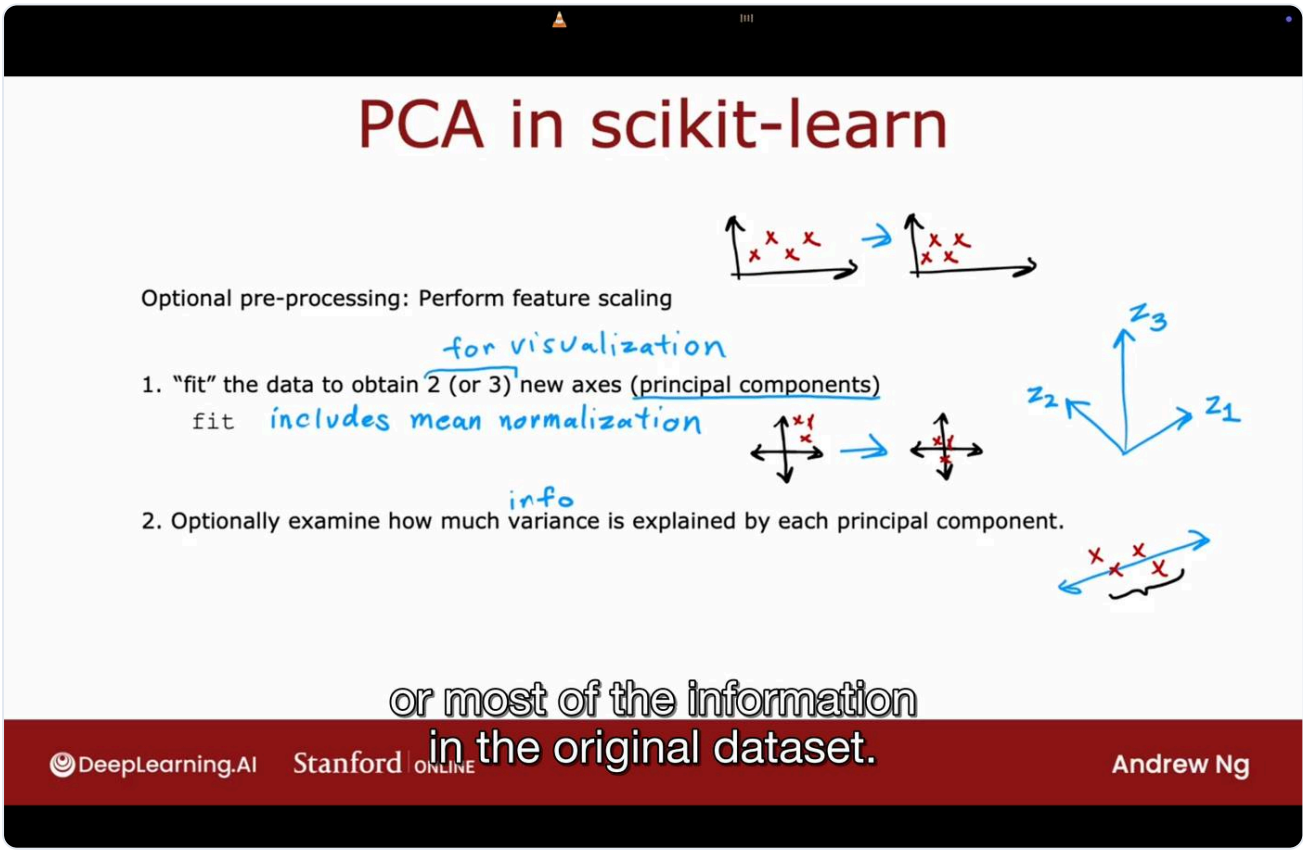


FIGURE 28.2

I'll show a concrete example of what this means on the next slide, but this lets you get a sense of whether or not projecting the data onto these axes help you to retain most of the variability, or most of the information in the original dataset. This is done using the explained variance ratio function.

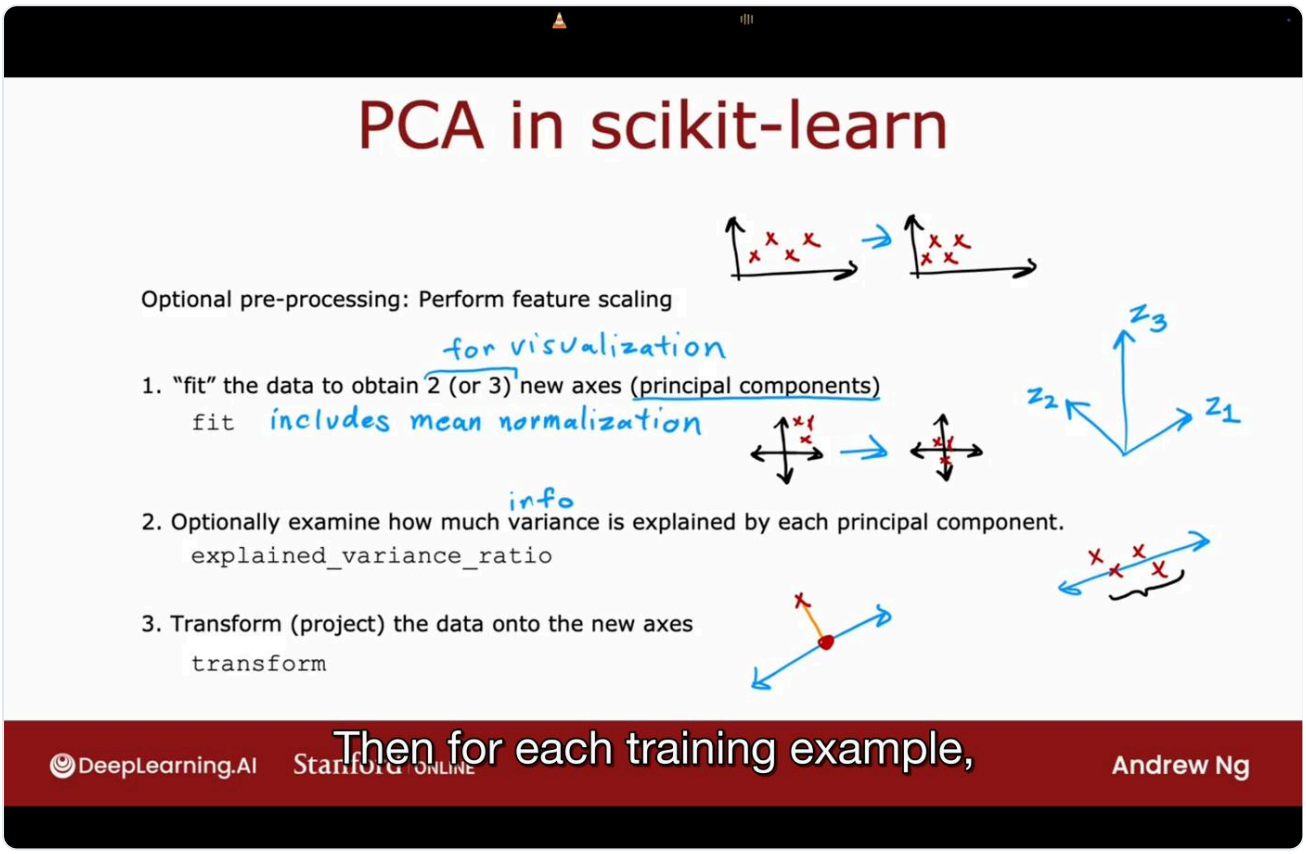


FIGURE 28.3

Finally, you can transform, meaning just project the data onto the new axes, onto the new principal components, which you will do with the transform method. Then for each training example, you would just have two or three numbers, you can then plot those two or three numbers to visualize your data. In detail, this is what PCA in code looks like. Here's the dataset X with six examples.

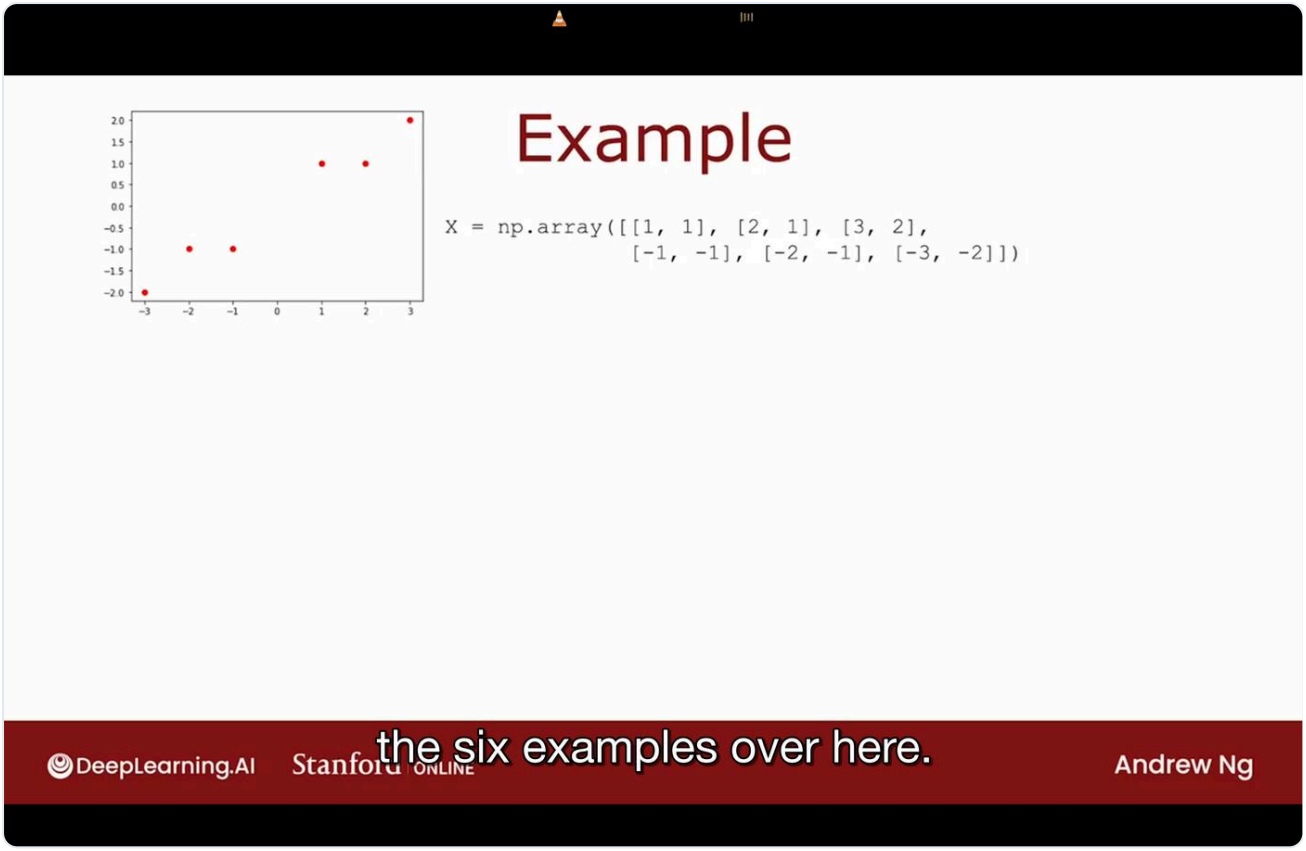


FIGURE 28.4

X equals NumPy array, the six examples over here. To run PCA to reduce this data from two numbers, X_1, X_2 to just one number Z, you would run PCA and ask it to fit one principal component. N components here is equal to one, and fit PCA to X.

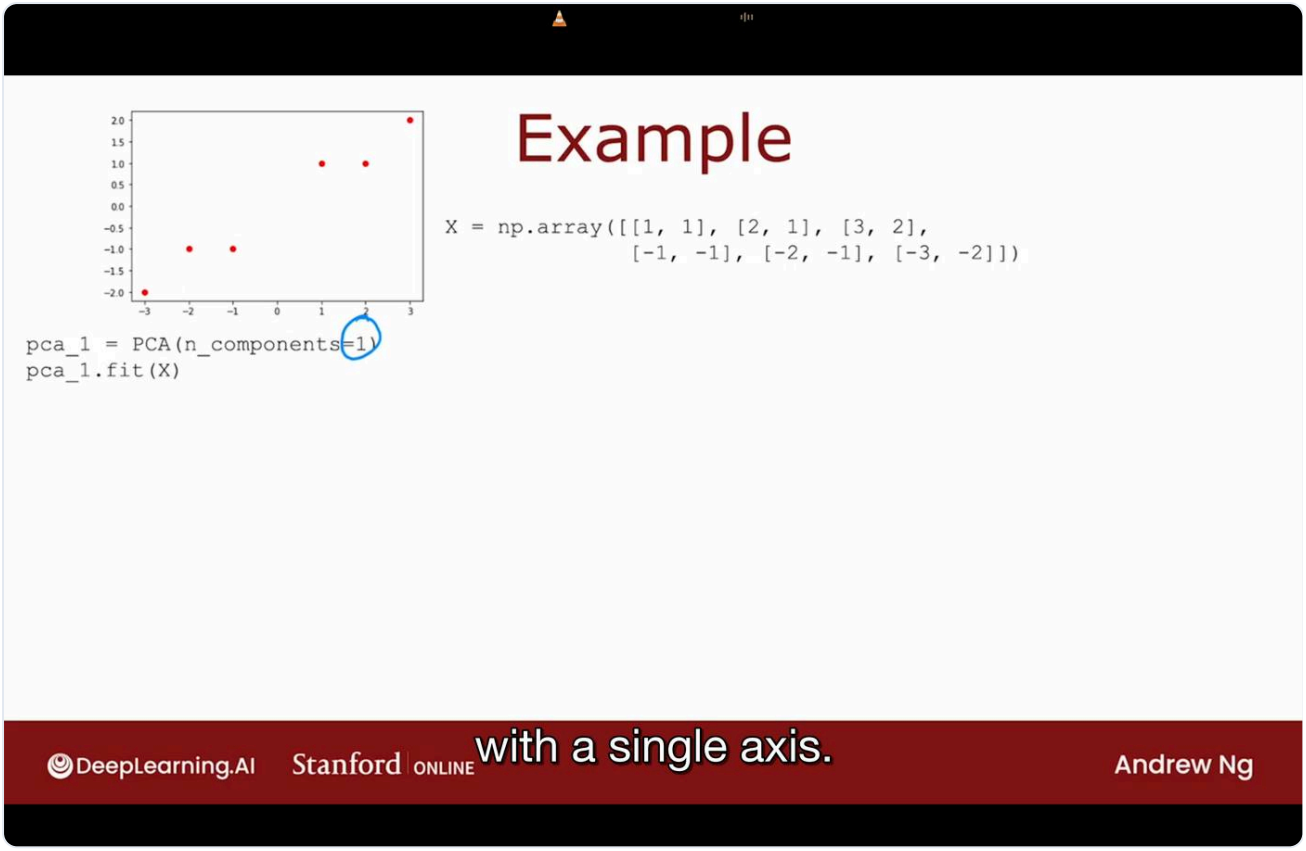


FIGURE 28.5

Pca_1 here is my notation for PCA with a single principle component, with a single axis. It turns out, if you were to print out pca_1.explained_variance_ratio, this is 0.992. This tells you that in this example when you choose one axis, this captures 99.2 percent of the variability or of the information in the original dataset.

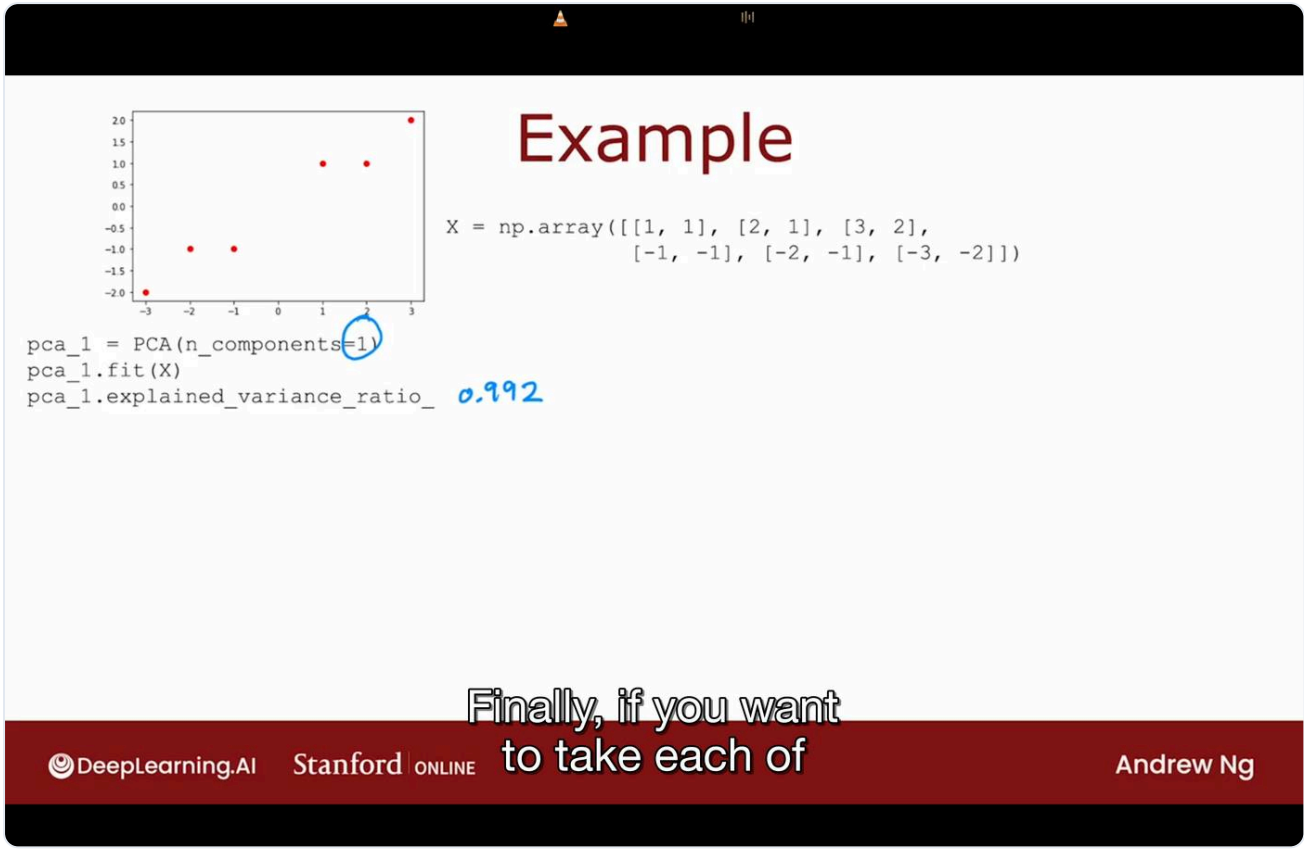


FIGURE 28.6

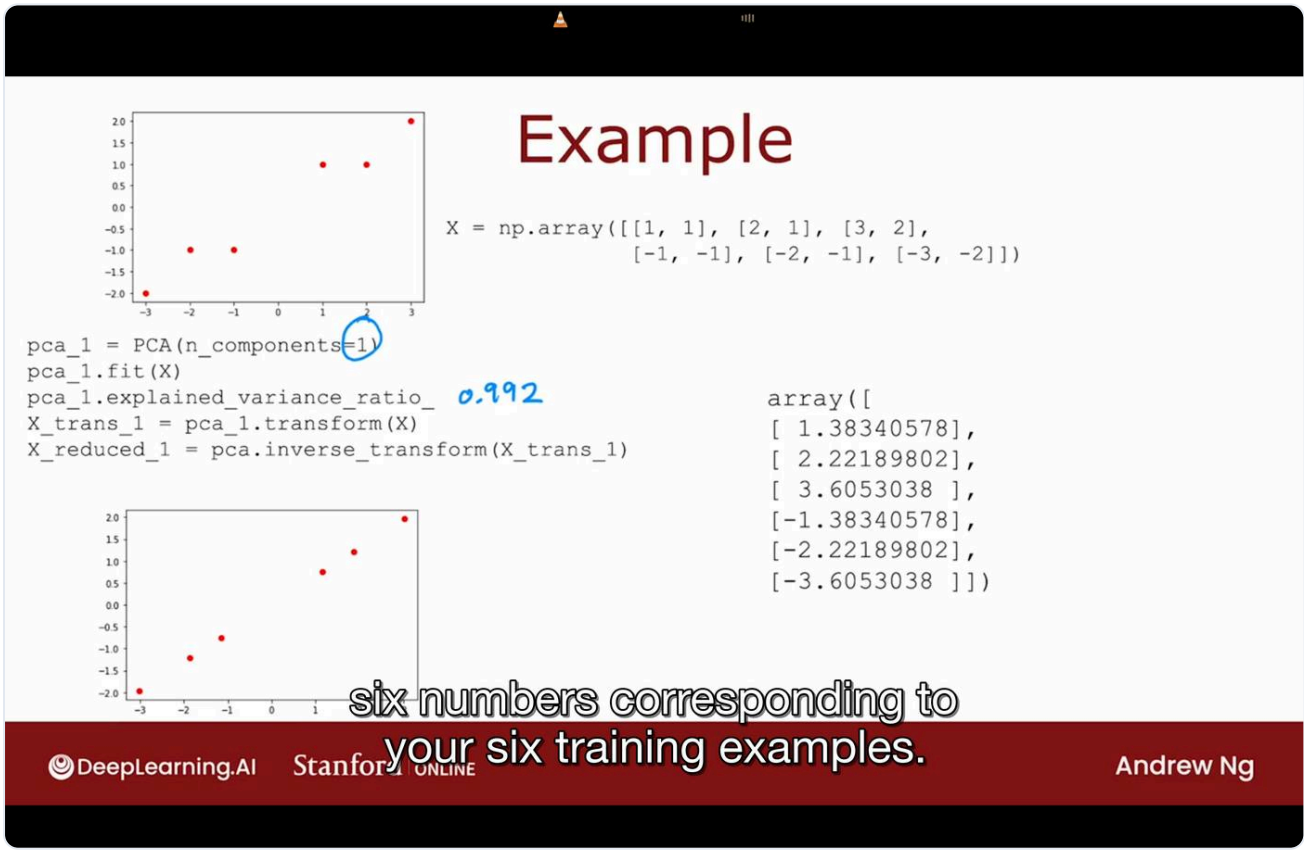


FIGURE 28.7

Finally, if you want to take each of these training samples and project it to a single number, you would then call this, and this will output this array with six numbers corresponding to your six training examples. For example, the first training example 1,1, projected to the Z-axis gives you this number, 1.383, so on. So if you were to visualize this dataset using just one dimension, this will be the number I use to represent the first example. The second example is projected to be this number and so on. I hope you take a look at the optional lab where you see that these six examples have been projected down onto this axis, onto this line which is now Y.

All six examples now lie on this line that looks like this.

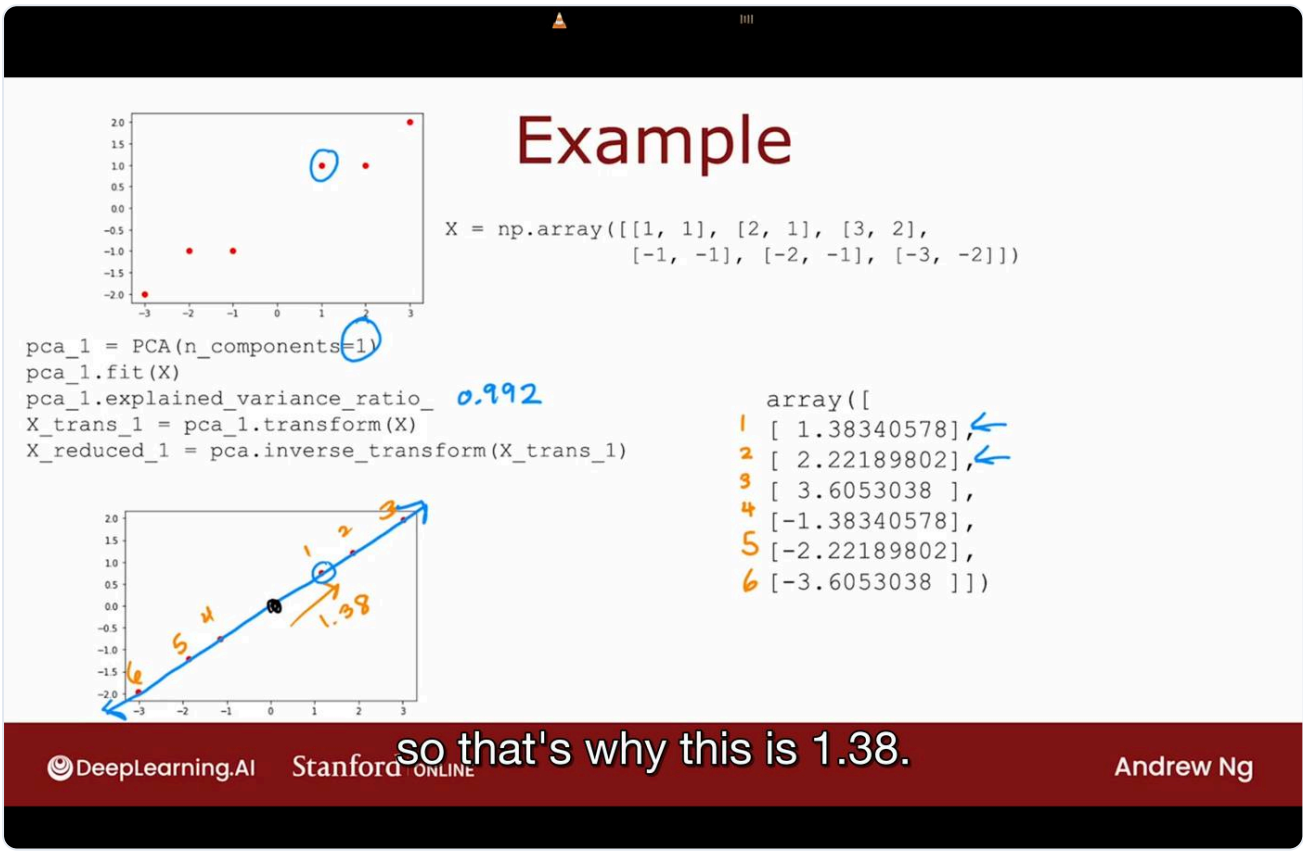


FIGURE 28.8

The first training example, which was 1,1, has been mapped to this example, which has a distance of 1.38 from the origin, so that's why this is 1.38. Just one more quick example. This data is two-dimensional data, and we reduced it to one dimensions. What if you were to compute two principal components? Starts with two-dimensions, and then also end up with two-dimensions. This isn't that useful for visualization but it might help us understand better how PCA and how they code for PCA works.

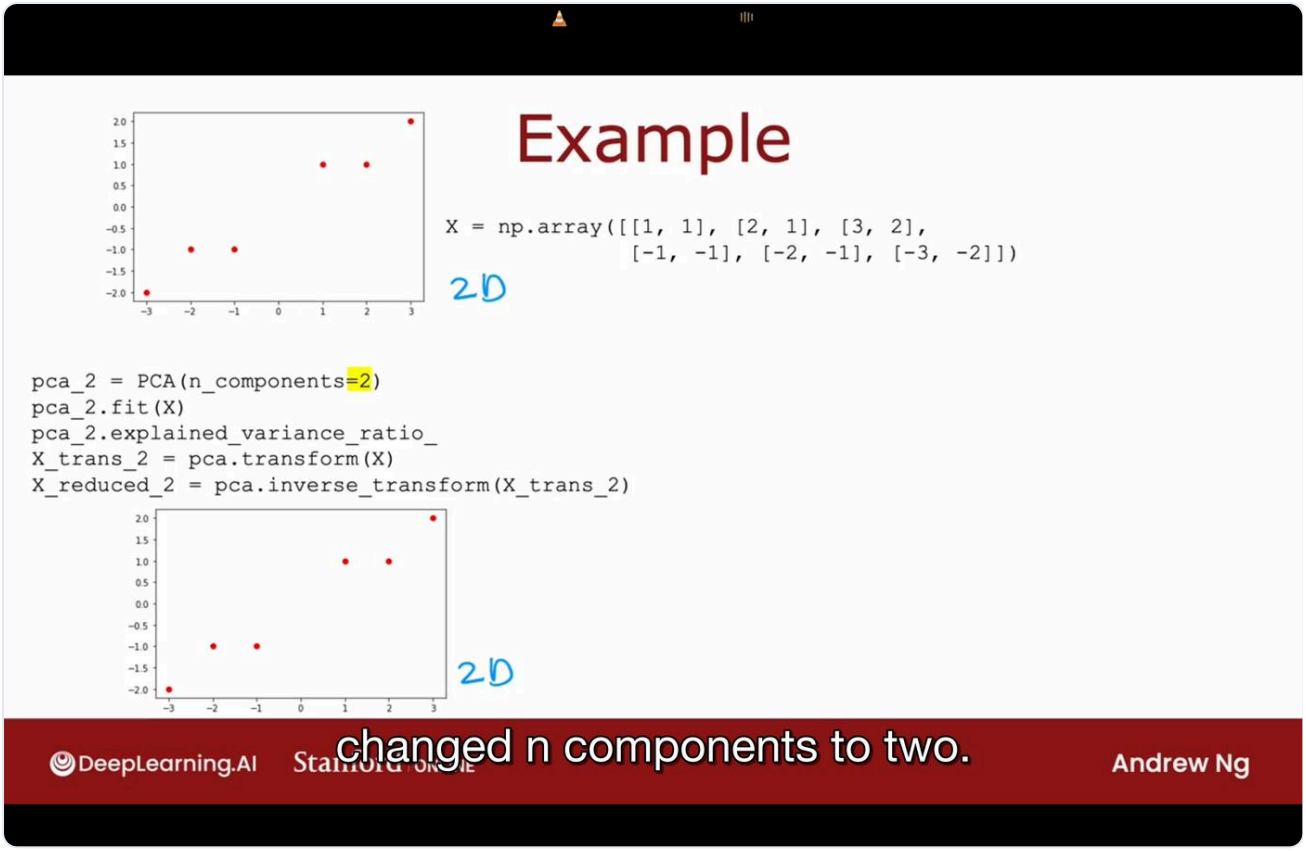


FIGURE 28.9

Here's the same code except that I've changed n components to two. I'm going to ask the algorithm to find two principal components. If you do that the pca_2 explain ratio becomes 0.992, 0.008. What that means is that z_1, the first principle components, still continuous explain 99.2 percent of the variance, Z_2 the second principle components, or the second axis, explains 0.8 percent of the variance.

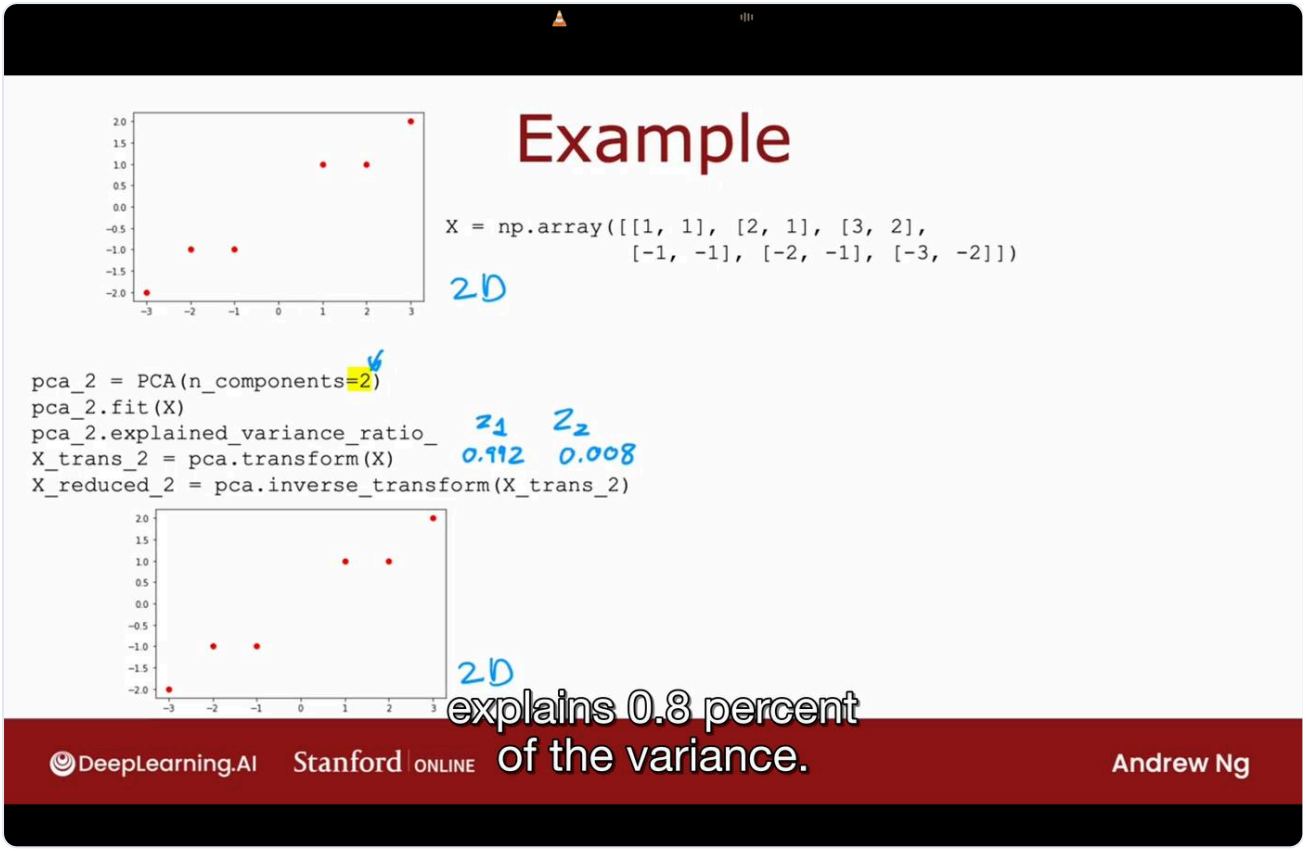


FIGURE 28.10

These two numbers together add up to one. Because while this data is two-dimensional, so the two axes, Z_1 and Z_2, together they explain 100 percent of the variance in the data. If you were to transform our project the data onto the Z_1 and Z_2 axes, this is what you get, with now the first training example is napped too these two numbers, corresponding to its projection onto z_1, and z_2, and the second example, which is this projected onto z_1 and z_2, becomes these two numbers.

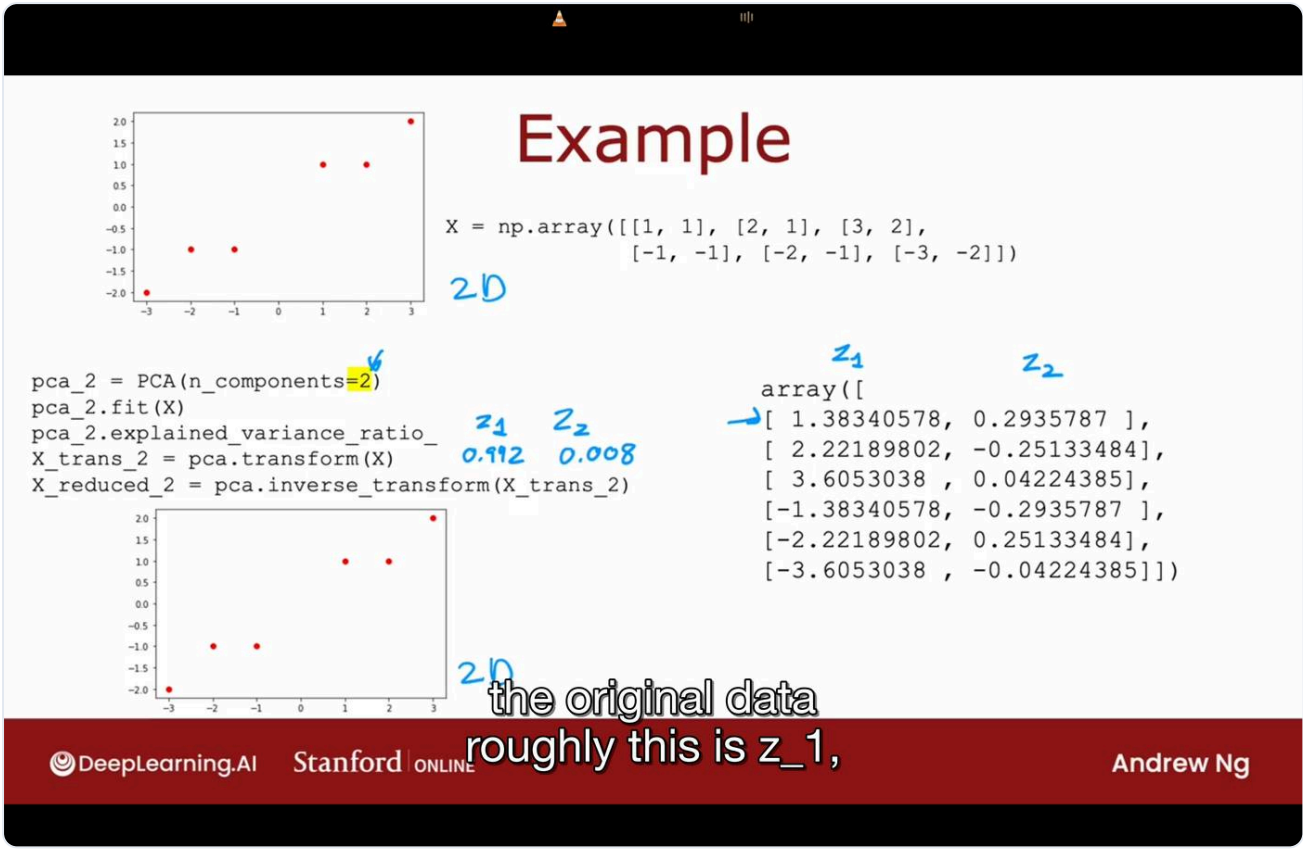


FIGURE 28.11

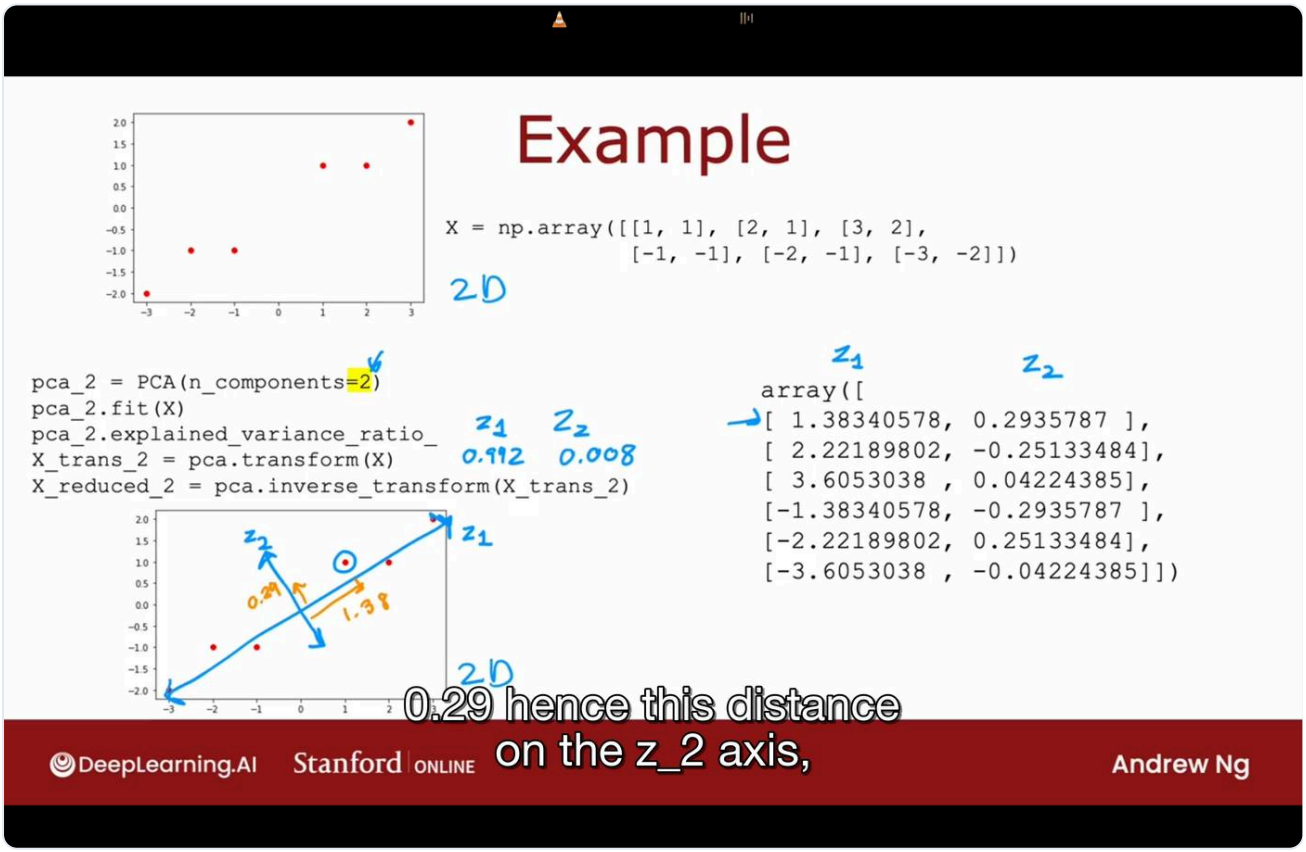


FIGURE 28.12

If you were to reconstruct the original data roughly this is z_1, and this z_2, then the first training example which was a [1, 1] has a distance of 1.38 on the z_1 axis, has this number and the distance here of 0.29 hence this distance on the z_2 axis, and the reconstruction actually looks exactly the same as the original data. Because, if you reduce or not really reduce two-dimensional data to two-dimensional data, there is no approximation and you can get back to your original dataset, with the projections onto z_1 and z_2. This is what the code to run PCA looks like. I hope you take a look at the optional lab where you can play with this more yourself.

Also try varying the parameters look at a specific example to deepen your intuition about how PCA works. Before wrapping up, I'd like to share a little bit of advice for applying PCA. PCA is frequently used for visualization where you reduce data to two or three numbers so you can plot it. Like you saw in an earlier video with the data on different countries so you can visualize different countries. There are some other applications of PCA that you may occasionally hear about. That used to be more popular, maybe 10,15, 20 years ago but much less so now. Another possible use of PCA is data compression. For example, if you have a database of lots of different cars, and you have 50 features per car, but it's just taking up too much space on your database or maybe transmitting 50 numbers over the Internet, just takes too long.

Then one thing you could do is reduce these 50 features to a smaller number of features. It could be 10 features with 10 axes or 10 principal components. You can visualize 10-dimensional data that easily, but this is 1/5 of the storage space, or maybe 1/5 over the network transmission costs needed. Many years ago I saw PCA use for this application and more often, but today with modern storage being able to store pretty large datasets and modern networking, able to transmit faster and more data than ever before. I see this use much less often as an application of PCA.

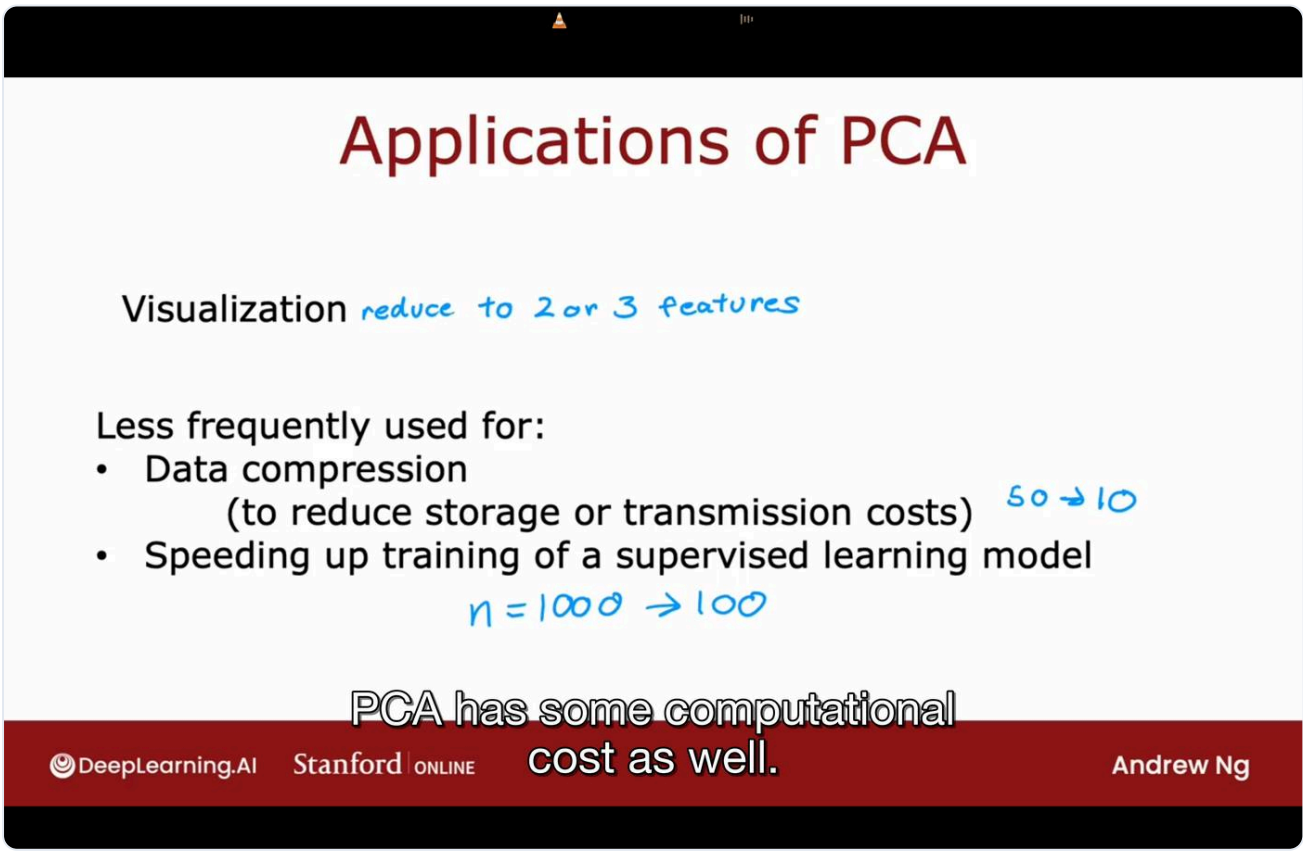


FIGURE 28.13 | Applications of PCA

One of the applications of PCA that again used to be more common maybe 10 years ago, 20 years ago, but much less so now is using it to speed up training of a supervised learning model. Where the idea is, if you had 1,000 features, and having a 1,000 features may the supervised learning algorithm runs too slowly. Maybe you can reduce it to 100 features using PCA and then your dataset is basically smaller and your supervised learning algorithm may run faster. This used to make a difference in the running time of some of the older generations of learning algorithms, such as if you have had a support vector machines. This will speed up a support vector machine.

But it turns out with modern machine learning algorithms, algorithms like deep learning, this doesn't actually help that much, and is much more common to just take the high-dimensional dataset, and feed it into say your neural network. Rather than run PCA because PCA has some computational cost as well. You may hear about this in some of the older research papers, but I don't really see this done much anymore. But the most common thing that I use PCA for today is visualization and then I find it very useful to reduce the dimensional data to visualize it. Thanks for sticking with me through the end of the optional videos for this week, I hope you enjoy learning about PCA and that you find a useful when you get a new dataset for reducing the dimension of the dataset to two or three dimensions so you can visualize it and hopefully gain new insights into your data sets.

There's helped me many times understand my own datasets and I hope that you find it equally useful as well. Thanks for watching these videos and I look forward to seeing you next week.

LECTURE 29

Autonomous Helicopter

Welcome to this final week of the machine learning specialization. It's a little bit bittersweet for me that we're approaching the end of this specialization, but I'm looking forward to this week, sharing with you some exciting ideas about reinforcement learning. In machine learning, reinforcement learning is one of those ideas that while not very widely applied in commercial applications yet today, is one of the pillars of machine learning. And has lots of exciting research backing it up and improving it every single day. So let's start by taking a look at what is reinforcement learning. Let's start with an example. Here's a picture of an autonomous helicopter.

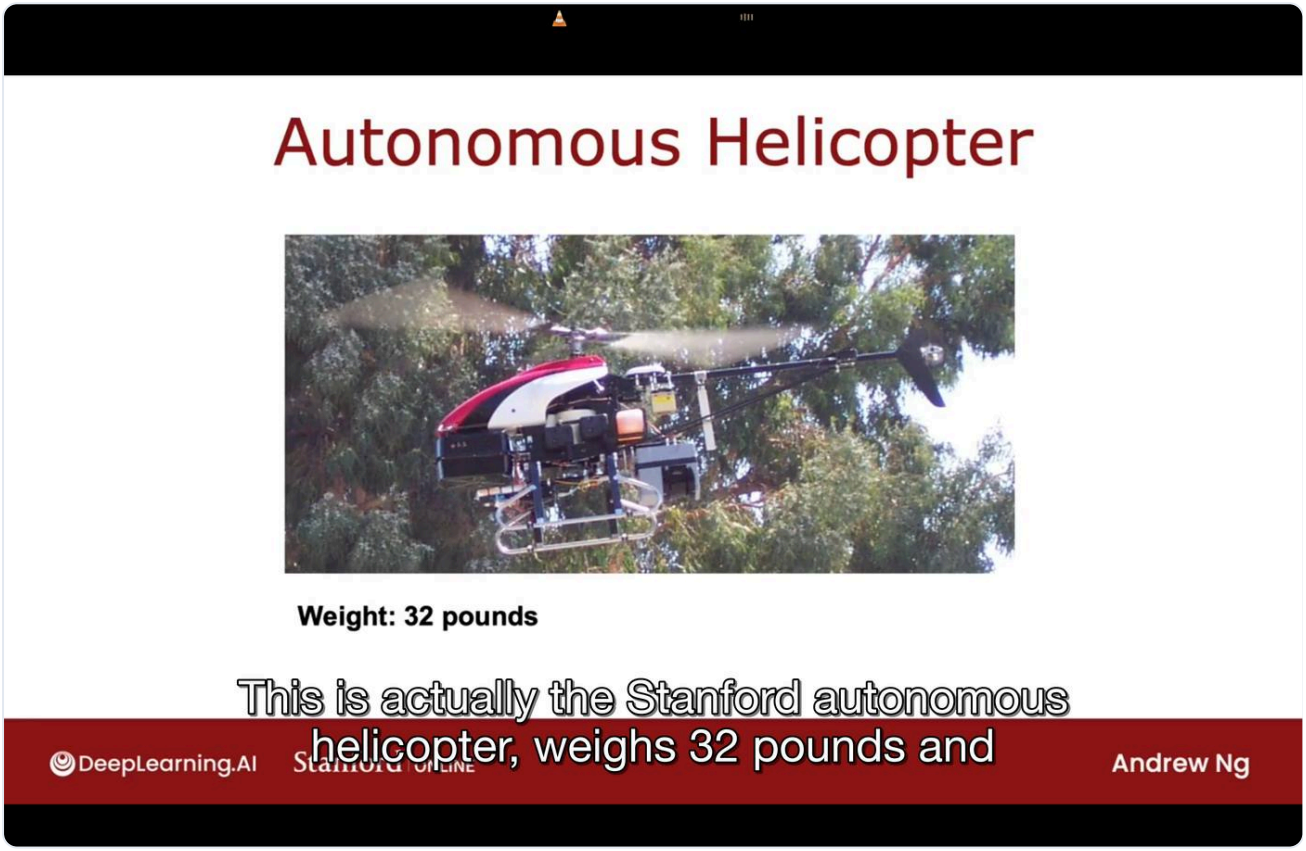


FIGURE 29.1 | Autonomous Helicopter

This is actually the Stanford autonomous helicopter, weighs 32 pounds and it's actually sitting in my office right now. Like many other autonomous helicopters, it's instrumented with an onboard computer, GPS, accelerometers, and gyroscopes and the magnetic compass so it knows where it is at all times quite accurately.

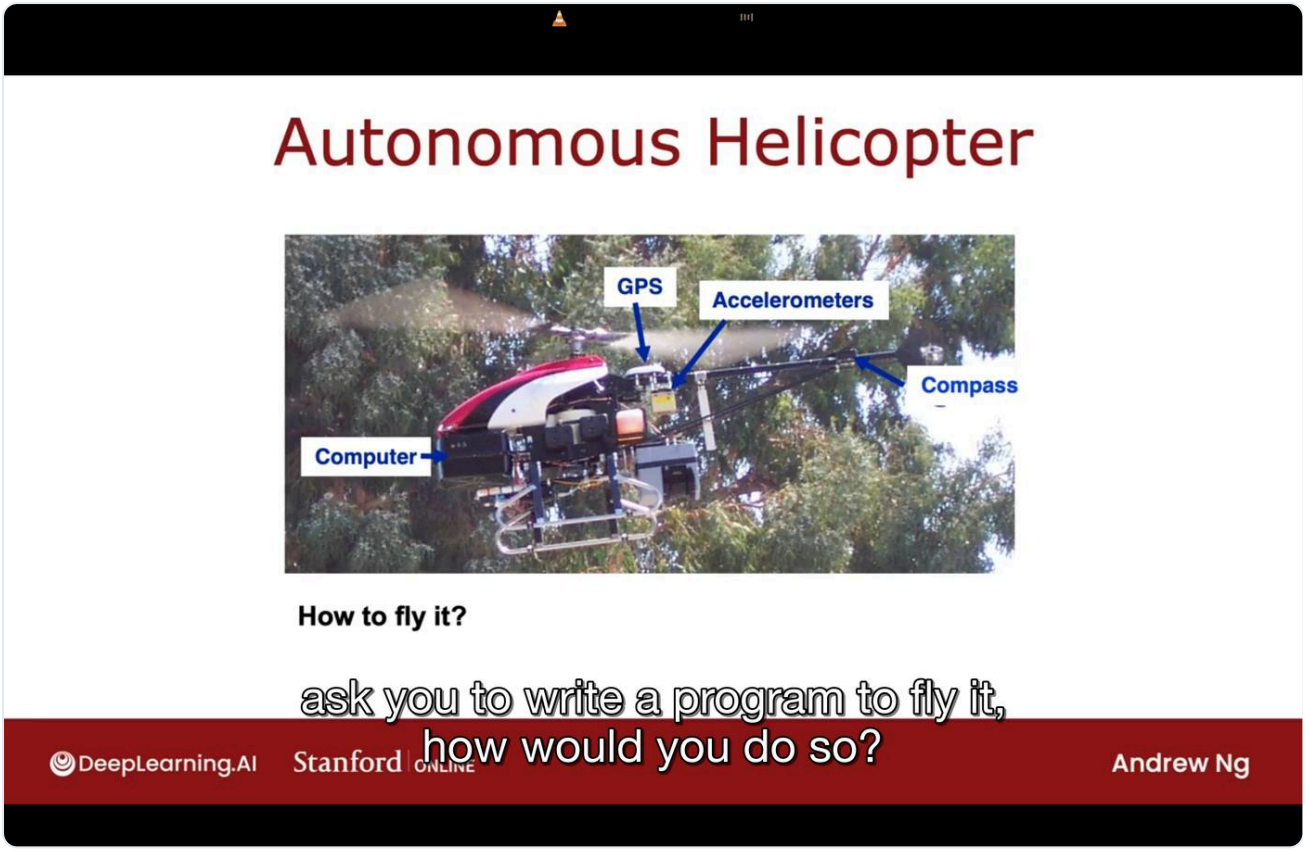


FIGURE 29.2 | Autonomous Helicopter

And if I were to give you the keys to this helicopter and ask you to write a program to fly it, how would you do so? Radio controlled helicopters are controlled with joysticks like these and so the task is ten times per second you're given the position and orientation and speed and so on of the helicopter. And you have to decide how to move these two control sticks in order to keep the helicopter balanced in the air. By the way, I've flown radio controlled helicopters as well as quad rotor drones myself. And radio controlled helicopters are actually quite a bit harder to fly, quite a bit harder to keep balanced in the air. So how do you write a program to do this automatically?

Let me show you a fun video of something we got a Stanford autonomous helicopter to do.

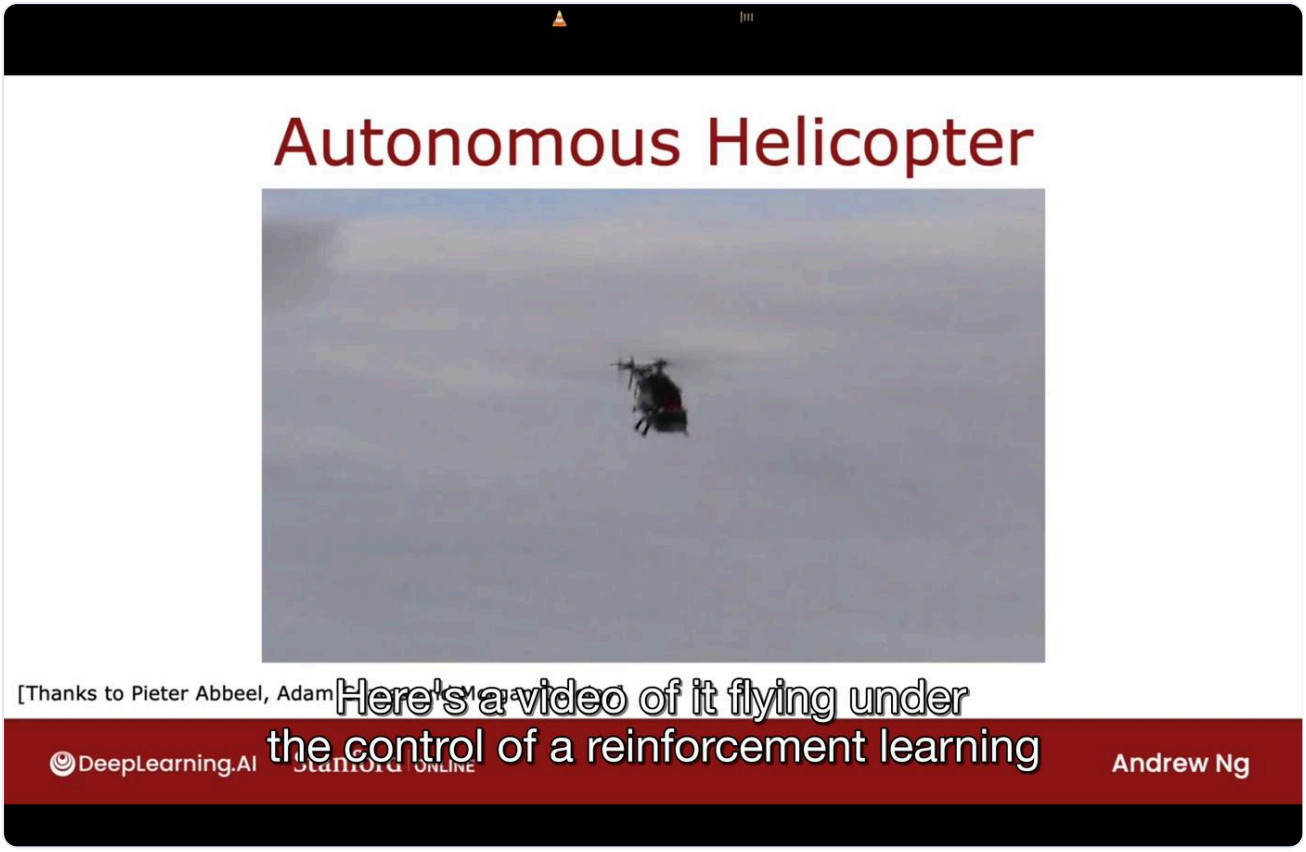


FIGURE 29.3 | Autonomous Helicopter

Here's a video of it flying under the control of a reinforcement learning algorithm. And let me play the video. I was actually the cameraman that day and this is the helicopter flying on the computer control and if I zoom out the video, you see the trees planted in the sky. So using reinforcement learning, we actually got this helicopter to learn to fly upside down. We told it to fly upside down. And so reinforcement learning has been used to get helicopters to fly a wide range of stunts or we call them aerobatic maneuvers.

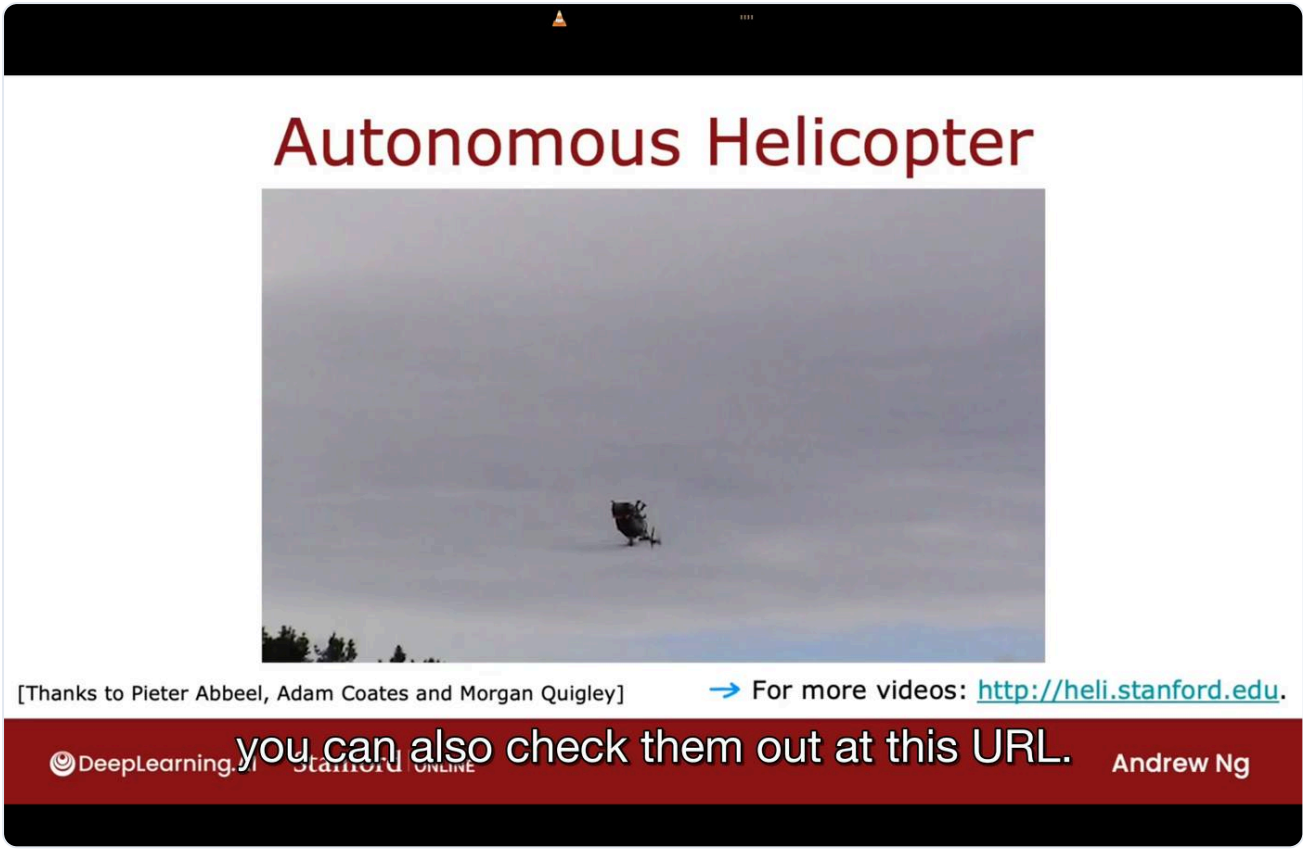


FIGURE 29.4 | Autonomous Helicopter

By the way, if you're interested in seeing other videos, you can also check them out at this URL. So how do you get a helicopter to fly itself using reinforcement learning? The task is given the position of the helicopter to decide how to move the control sticks. In reinforcement learning, we call the position and orientation and speed and so on of the helicopter the state s. And so the task is to find a function that maps from the state of the helicopter to an action a, meaning how far to push the two control sticks in order to keep the helicopter balanced in the air and flying and without crashing.

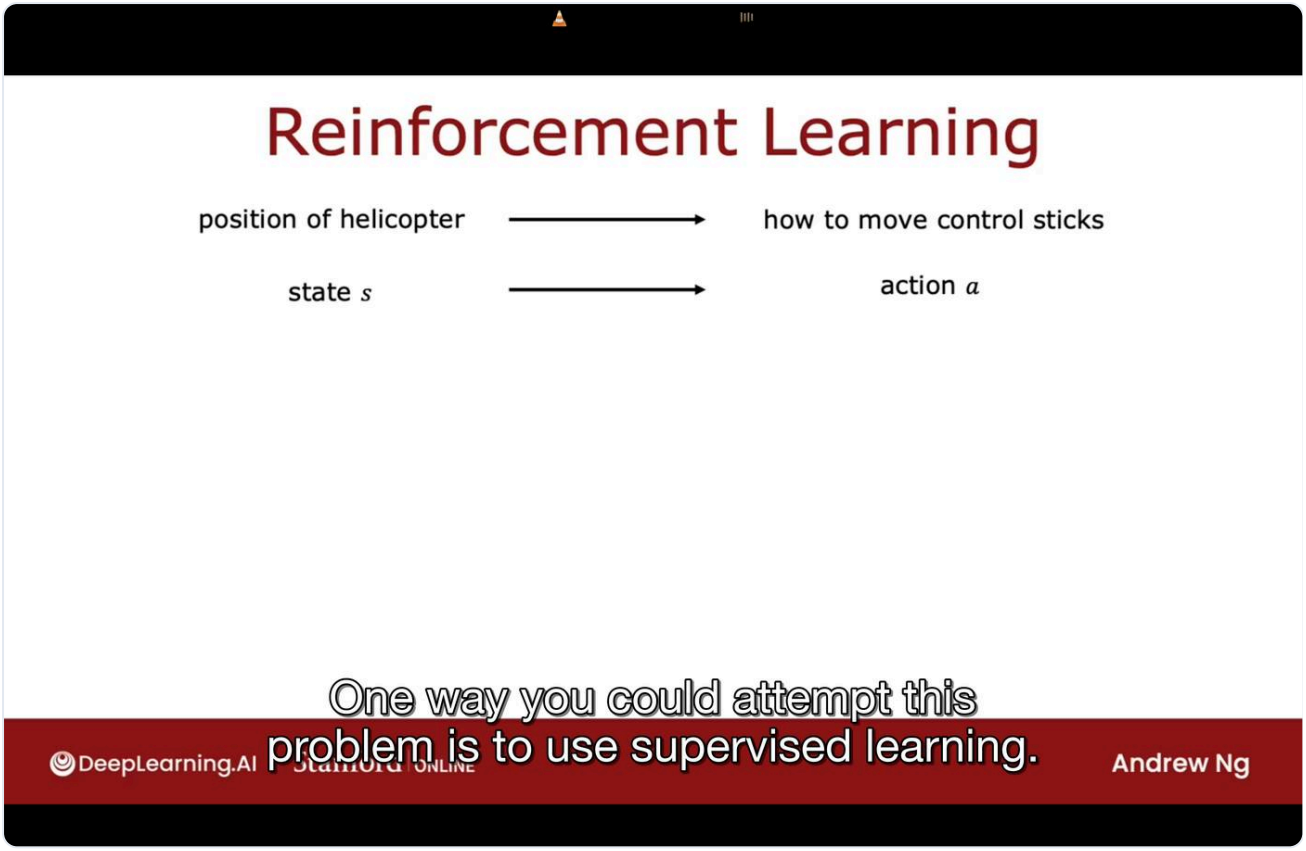


FIGURE 29.5 | Reinforcement Learning

One way you could attempt this problem is to use supervised learning. It turns out this is not a great approach for autonomous helicopter flying. But you could say, well if we could get a bunch of observations of states and maybe have an expert human pilot tell us what's the best action y to take. You could then train a neural network using supervised learning to directly learn the mapping from the states s which I'm calling x here, to an action a which I'm calling the label y here.

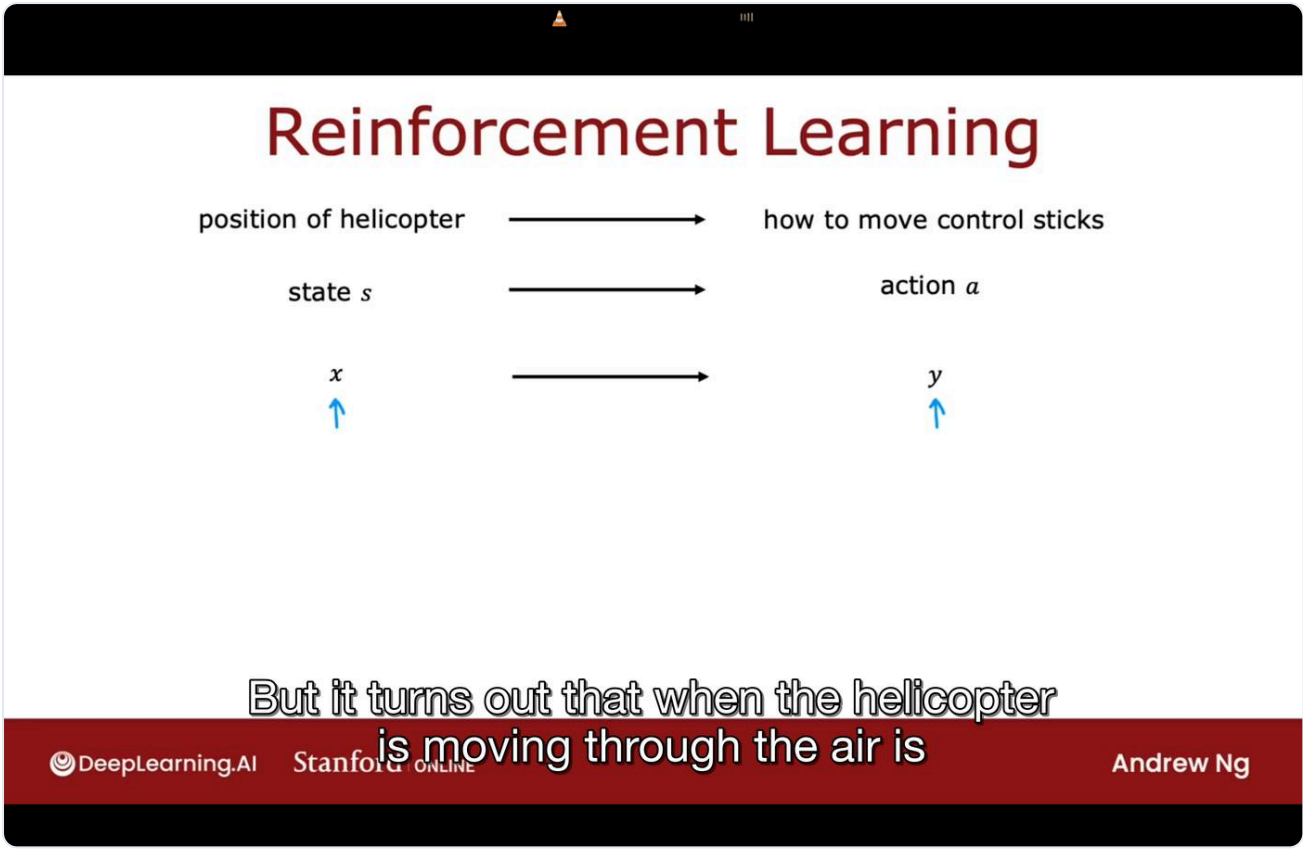


FIGURE 29.6 | Reinforcement Learning

But it turns out that when the helicopter is moving through the air is actually very ambiguous, what is the exact one right action to take. Do you tilt a bit to the left or a lot more to the left or increase the helicopter stress a little bit or a lot? It's actually very difficult to get a data set of x and the ideal action y. So that's why for a lot of task of controlling a robot like a helicopter and other robots, the supervised learning approach doesn't work well and we instead use reinforcement learning.

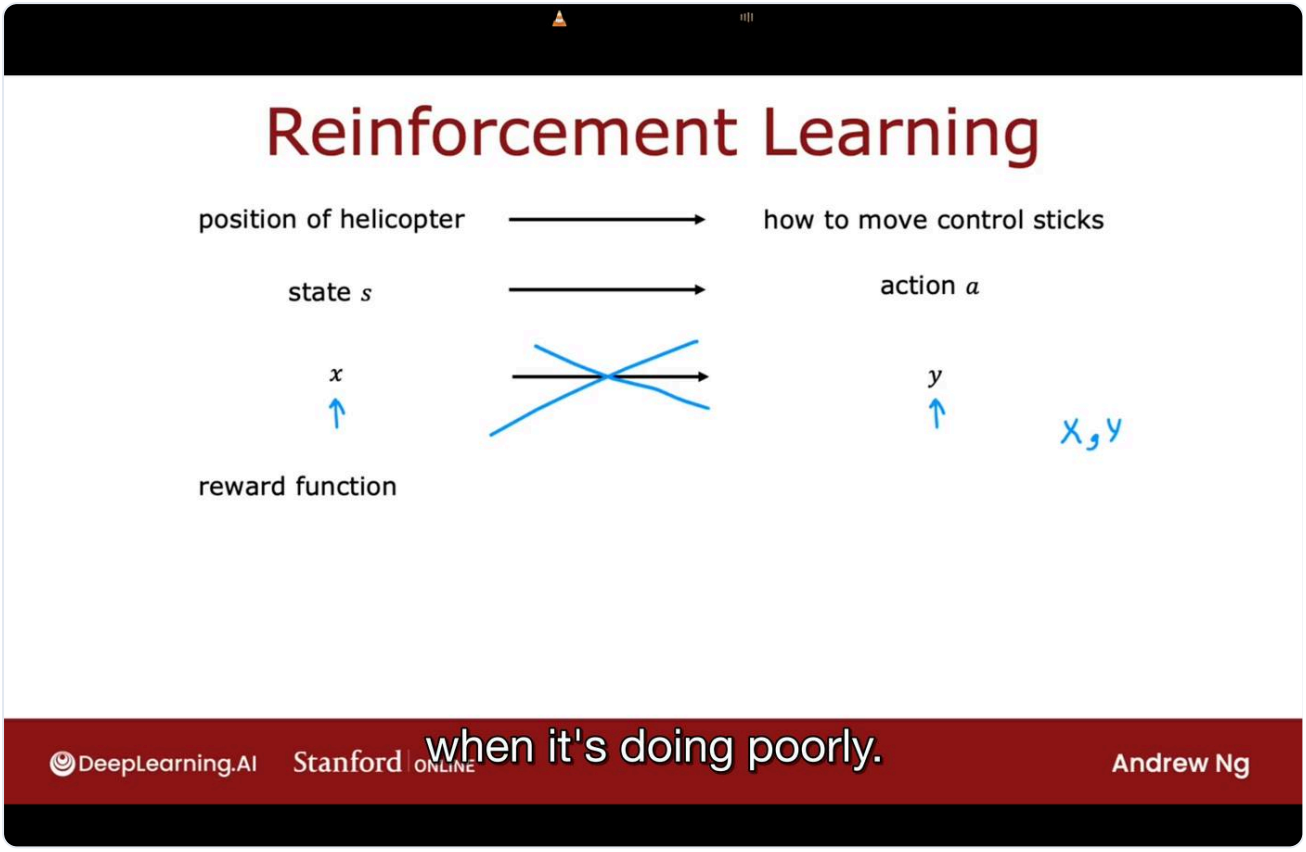


FIGURE 29.7 | Reinforcement Learning

Now a key input to a reinforcement learning is something called the reward or the reward function which tells the helicopter when it's doing well and when it's doing poorly. So the way I like to think of the reward function is a bit like training a dog. When I was growing up, my family had a dog and it was my job to train the dog or the puppy to behave. So how do you get a puppy to behave well? Well, you can't demonstrate that much to the puppy. Instead you let it do its thing and whenever it does something good, you go, good dog. And whenever they did something bad, you go, bad dog. And then hopefully it learns by itself how to do more of the good dog and fewer of the bad dog things.

So training with the reinforcement learning algorithm is like that. When the helicopter's flying well, you go, good helicopter and if it does something bad like crash, you go, bad helicopter. And then it's the reinforcement learning algorithm's job to figure out how to get more of the good helicopter and fewer of the bad helicopter outcomes. One way to think of why reinforcement learning is so powerful is you have to tell it what to do rather than how to do it. And specifying the reward function rather than the optimal action gives you a lot more flexibility in how you design the system. Concretely for flying the helicopter, whenever it is flying well, you may give it a reward of plus one every second it is flying well.

And maybe whenever it's flying poorly you may give it a negative reward or if it ever crashes, you may give it a very large negative reward like negative 1,000.

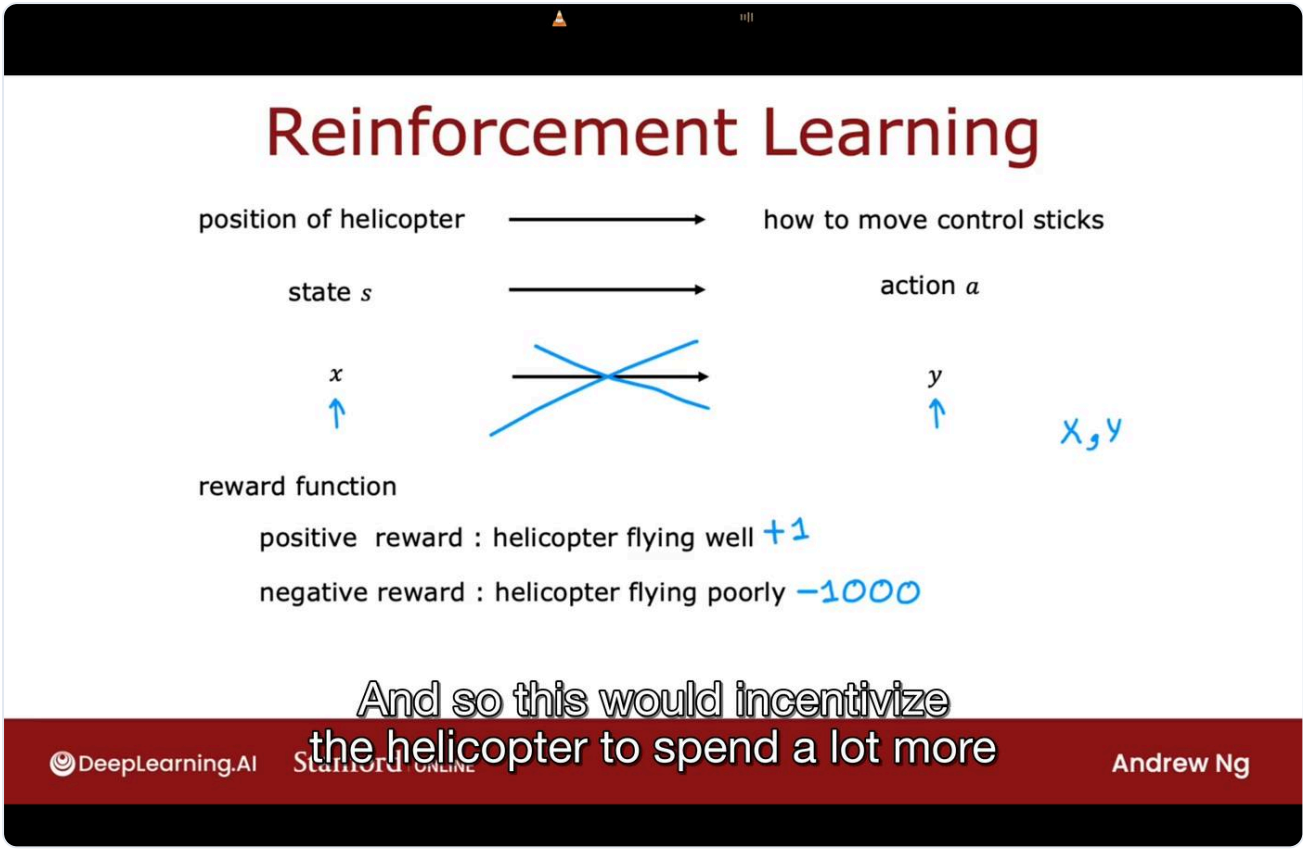


FIGURE 29.8 | Reinforcement Learning

And so this would incentivize the helicopter to spend a lot more time flying well and hopefully to never crash. But here's another fun video. I was using the good dog bad dog analogy for reinforcement learning for many years.

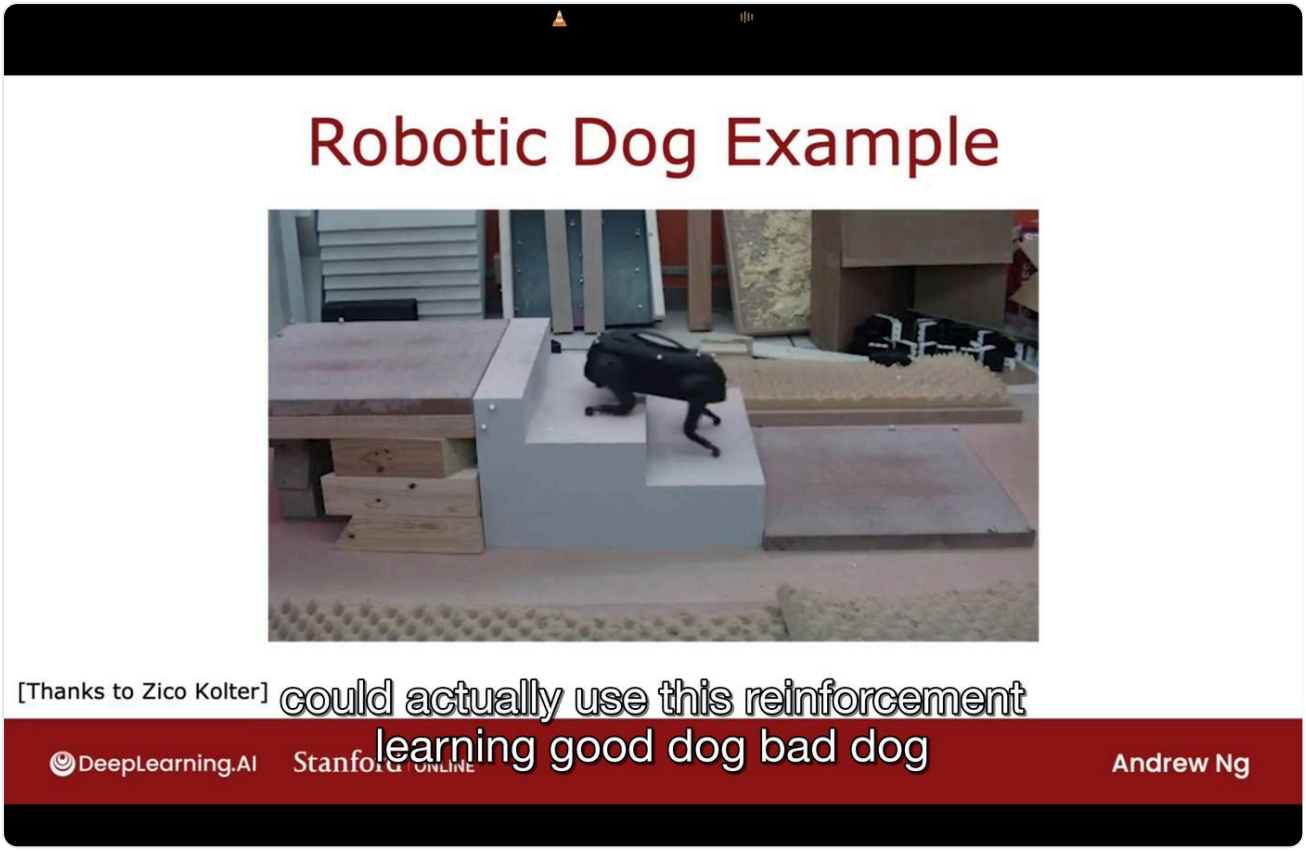


FIGURE 29.9

And then one day I actually managed to get my hands on a robotic dog and could actually use this reinforcement learning good dog bad dog methodology to train a robot dog to get over obstacles. So this is a video of a robot dog that using reinforcement learning, which rewards it, moving toward the left of the screen has learned how to place its feet carefully or climb over a variety of obstacles. And if you think about what it takes to program a dog like this, I have no idea, I really don't know how to tell it what's the best way to place its legs to get over a given obstacle. All of these things were figured out automatically by the robot just by giving it rewards that incentivizes it, making progress toward the goal on the left of the screen.

Today, reinforcement learning has been successfully applied to a variety of applications ranging from controlling robots. And in fact later this week in the practice lab, you implement for yourself a reinforcement learning algorithm to land a lunar lander in simulation. It's also been used for factory optimization.

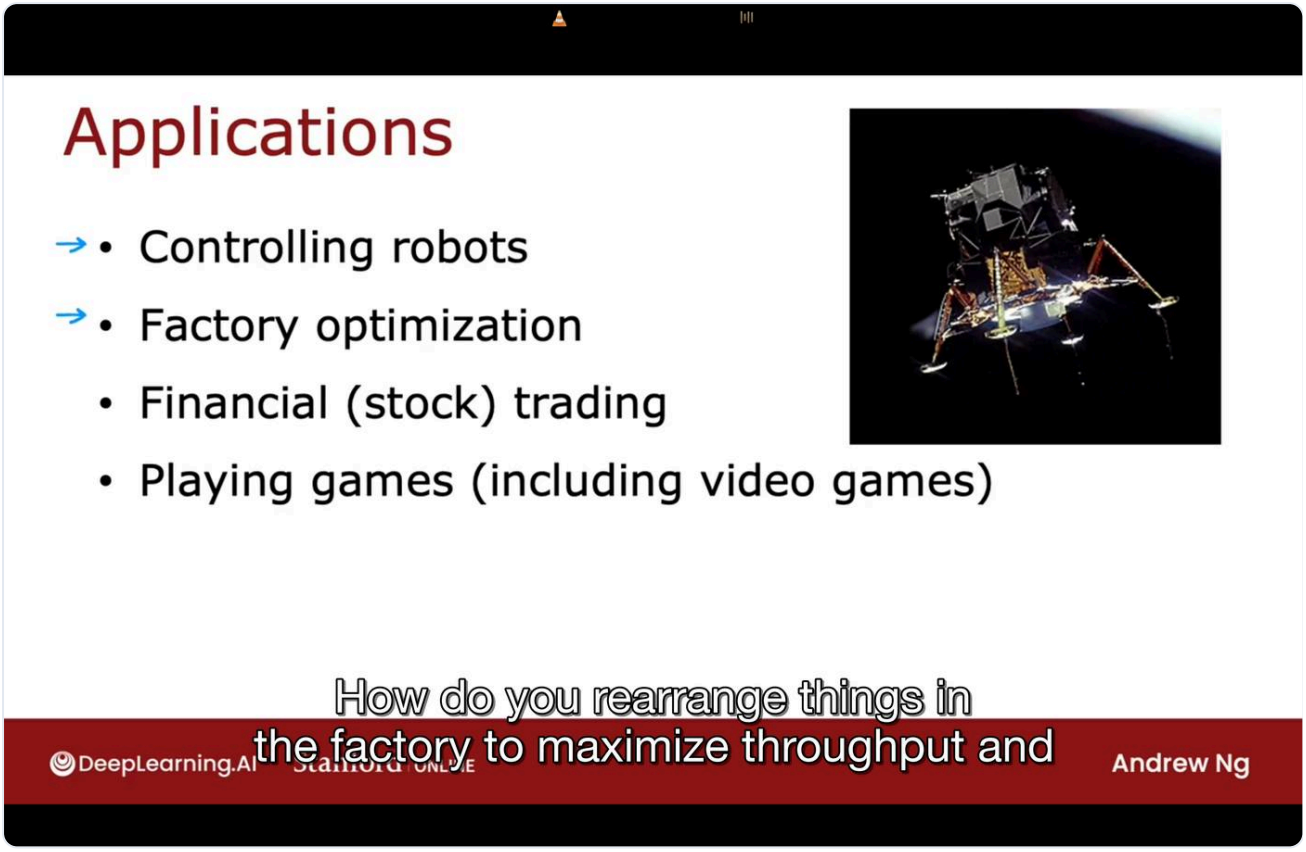


FIGURE 29.10

How do you rearrange things in the factory to maximize throughput and efficiency as well as financial stock trading. For example, one of my friends was working on efficient stock execution. So if you decided to sell a million shares over the next several days, well, you may not want to dump a million shares on the stock market suddenly because that will move prices against you. So what's the best way to sequence out your trades over time so that you can sell the shares you want to sell and hopefully get the best possible price for them? Finally, there have also been many applications of reinforcement learning to playing games, everything from checkers to chess to the card game of bridge to go as well as for playing many video games.

So that's reinforcement learning. Even though reinforcement learning is not used nearly as much as supervised learning, it is still used in a few applications today. And the key idea is rather than you needing to tell the algorithm what is the right output y for every single input, all you have to do instead is specify a reward function that tells it when it's doing well and when it's doing poorly. And it's the job of the algorithm to automatically figure out how to choose good actions. With that, let's now go into the next video where we'll formalize the reinforcement learning problem and also start to develop algorithms for automatically picking good actions

LECTURE 30

Mars Rover Example

To finish out the reinforcement learning formalism, instead of looking at something as complicated as a helicopter or a robot dog, we can use a simplified example that's loosely inspired by the Mars rover. This is adapted from the example due to Stanford professor Emma Brunskill and one of my collaborators, Jagriti Agrawal, who had actually written code that is actually controlling the Mars rover right now that also helped me talk through and helped develop this example. Let's take a look. We'll develop reinforcement learning using a simplified example inspired by the Mars rover. In this application, the rover can be in any of six positions, as shown by the six boxes here.

The rover, it might start off, say, in disposition into fourth box shown here.

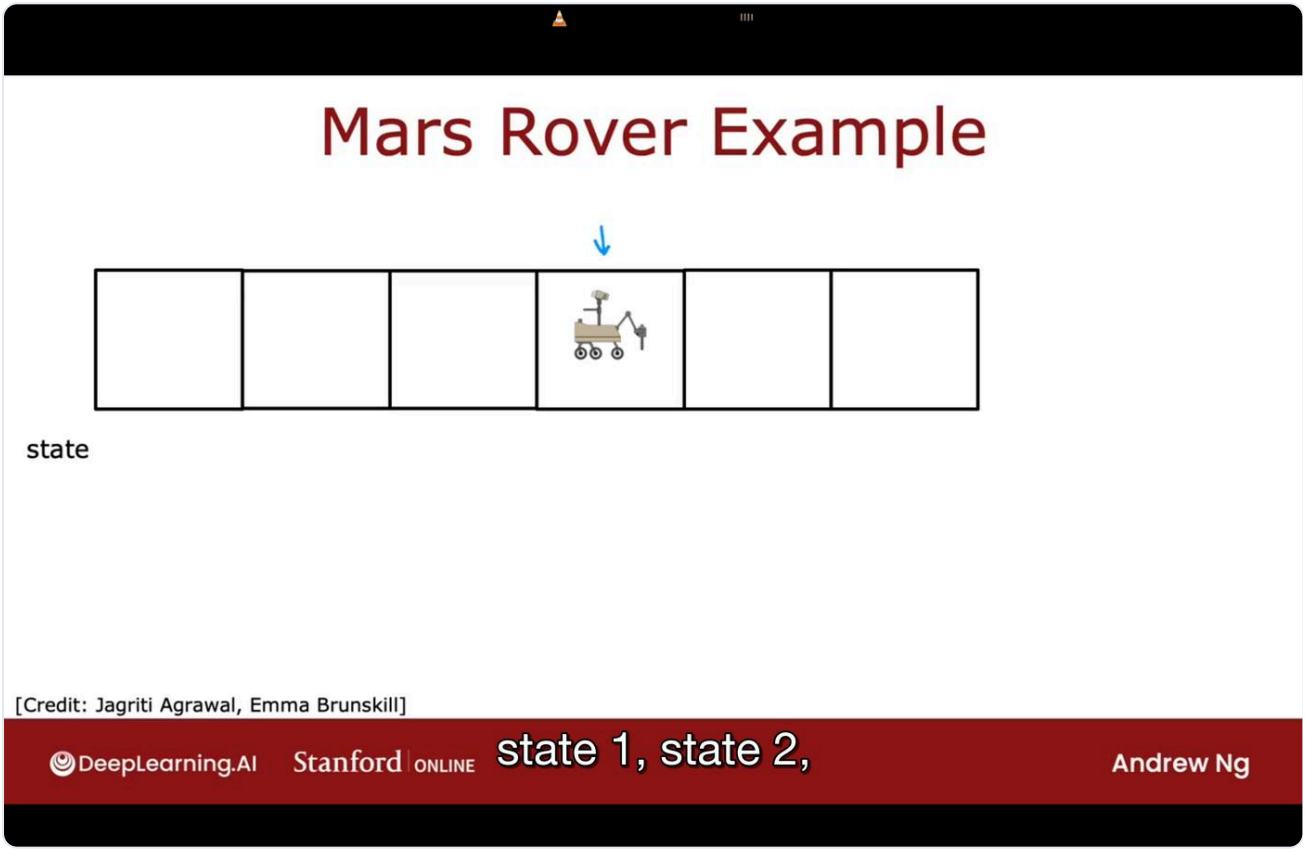


FIGURE 30.1 | Mars Rover Example

The position of the Mars rover is called the state in reinforcement learning, and I'm going to call these six states, state 1, state 2, state 3, state 4, state 5, and state 6, and so the rover is starting off in state 4.

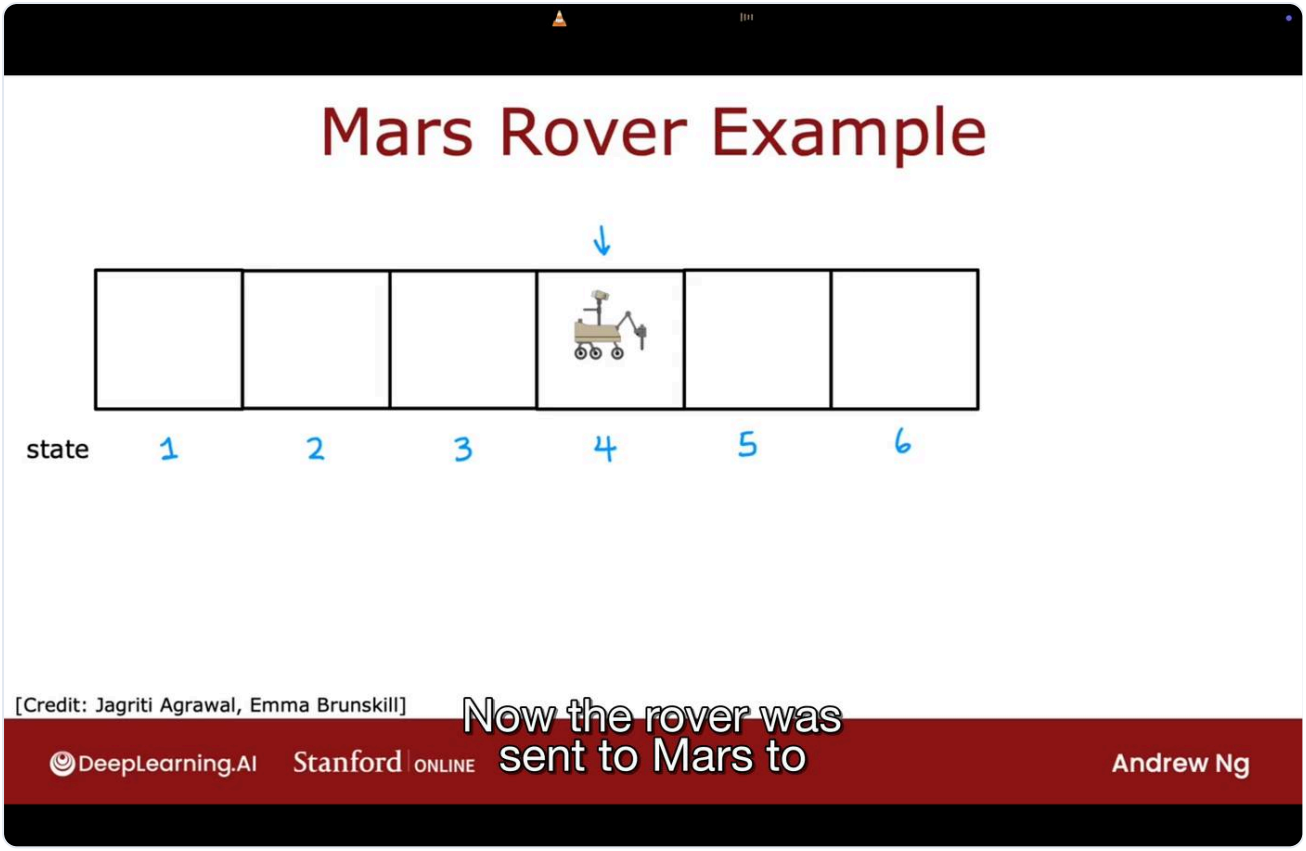


FIGURE 30.2 | Mars Rover Example

Now the rover was sent to Mars to try to carry out different science missions. It can go to different places to use its sensors such as a drill, or a radar, or a spectrometer to analyze the rock at different places on the planet, or go to different places to take interesting pictures for scientists on earth to look at. In this example, state 1 here on the left has a very interesting surface that scientists would love for the rover to sample. State 6 also has a pretty interesting surface that scientists would quite like the rover to sample, but not as interesting as state 1.

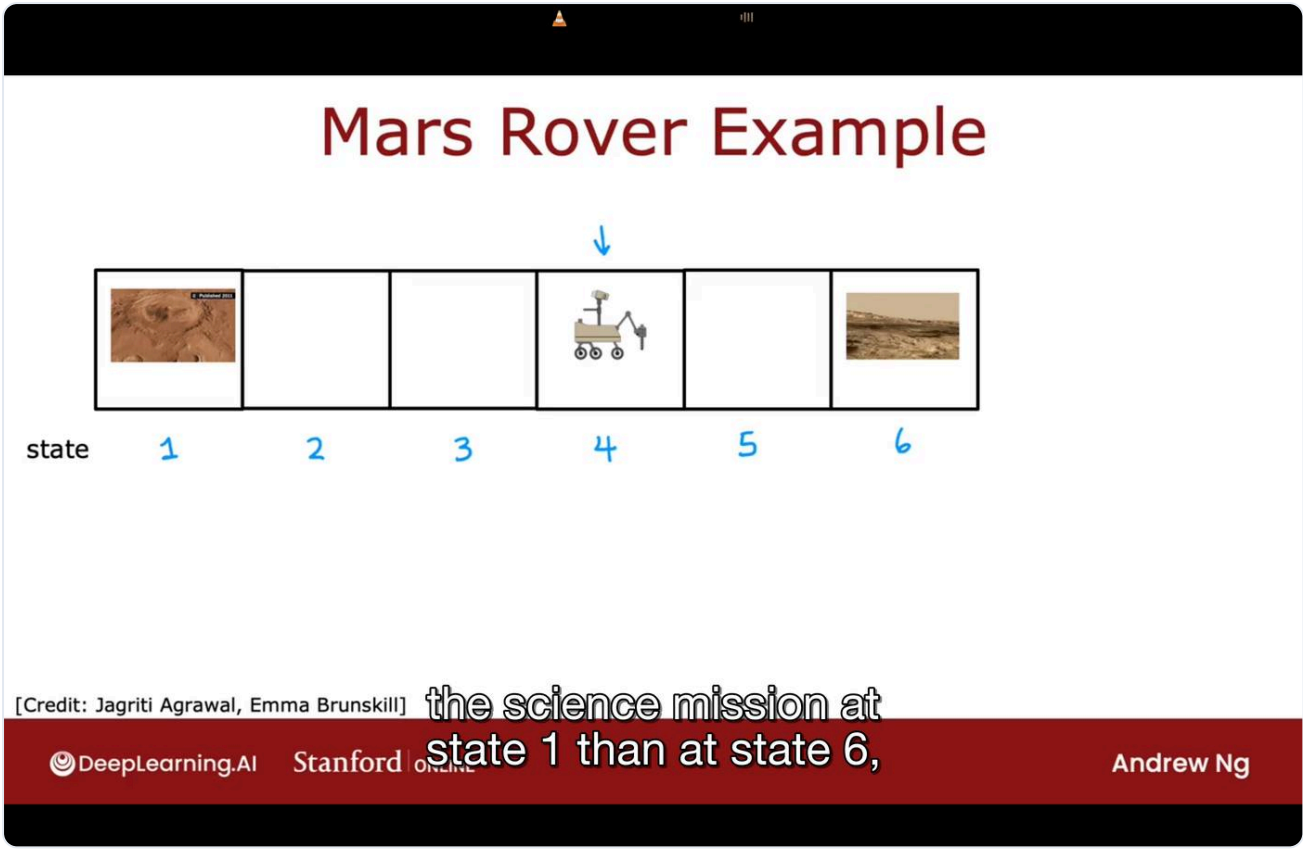


FIGURE 30.3 | Mars Rover Example

We would more likely to carry out the science mission at state 1 than at state 6, but state 1 is further away. The way we will reflect state 1 being potentially more valuable is through the reward function.

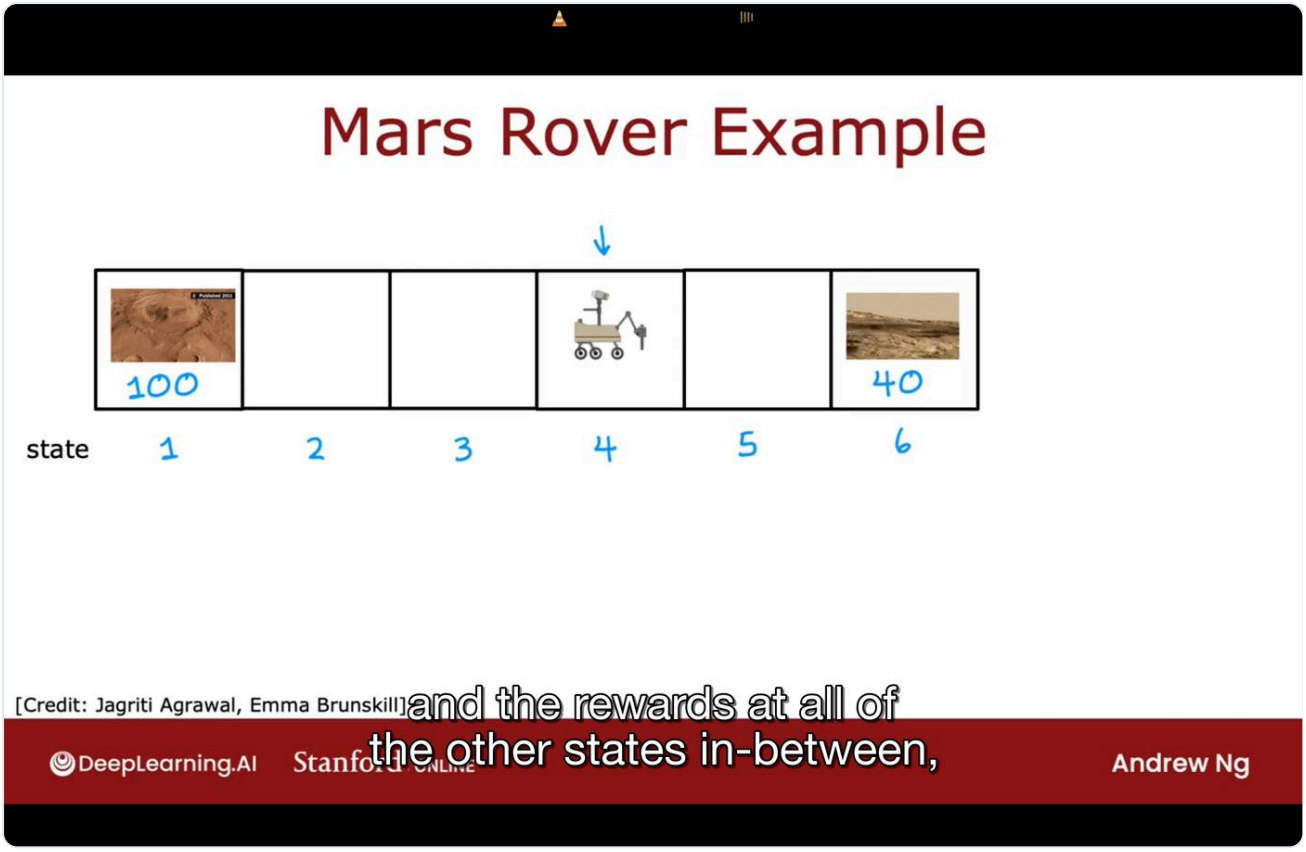


FIGURE 30.4 | Mars Rover Example

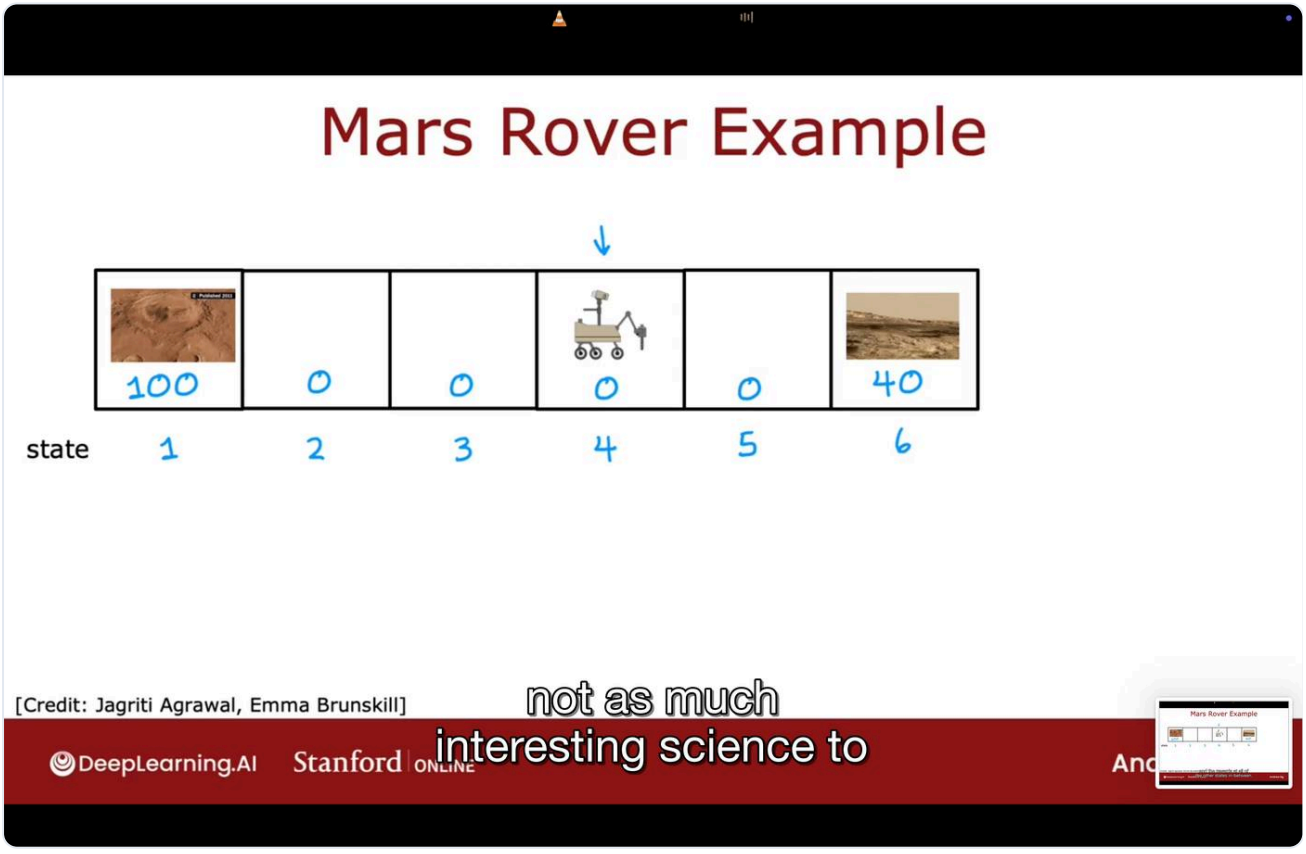


FIGURE 30.5 | Mars Rover Example

The reward at state 1 is a 100, and the reward at stage 6 is 40, and the rewards at all of the other states in-between, I'm going to write as a reward of zero because there's not as much interesting science to be done at these states 2, 3, 4, and 5. On each step, the rover gets to choose one of two actions. It can either go to the left or it can go to the right. The question is, what should the rover do?

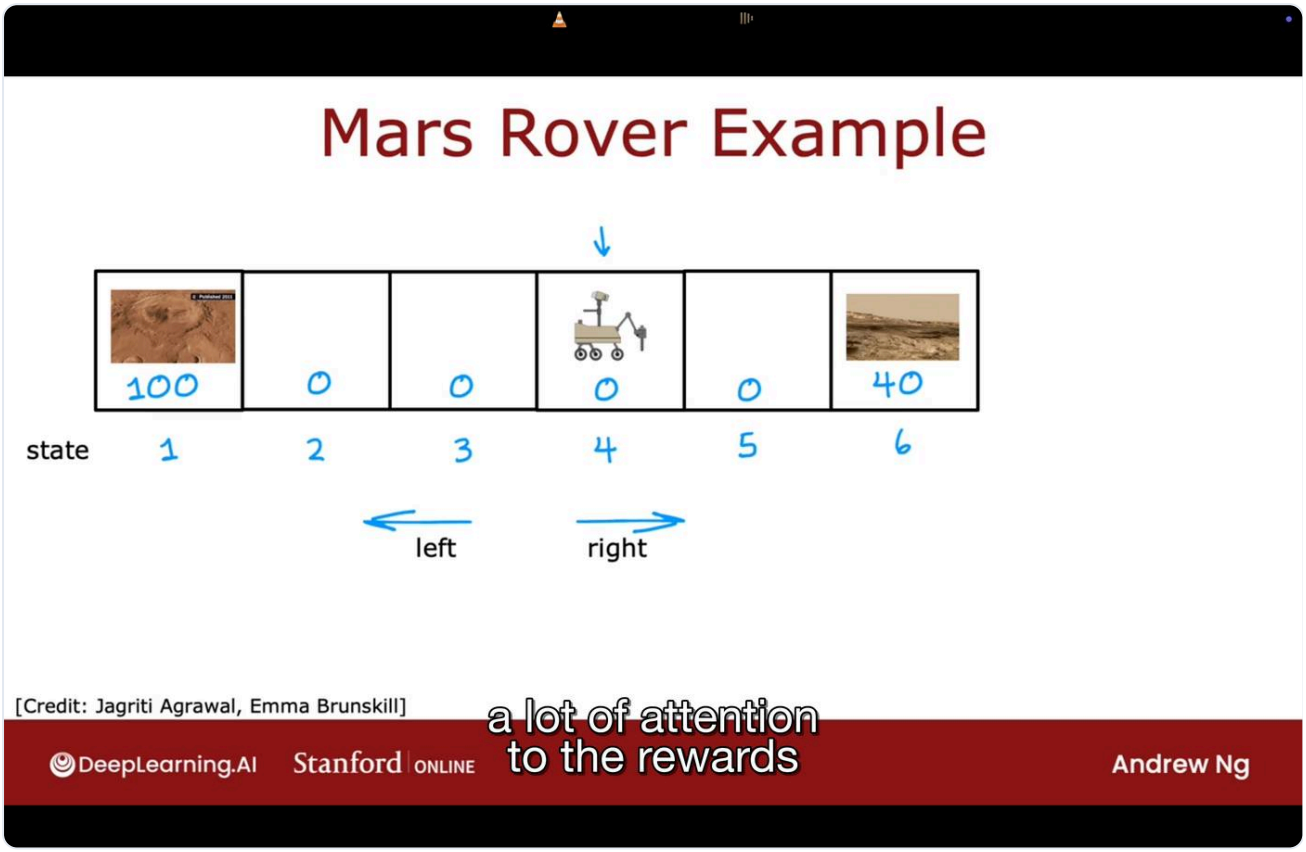


FIGURE 30.6 | Mars Rover Example

In reinforcement learning, we pay a lot of attention to the rewards because that's how we know if the robot is doing well or poorly. Let's look at some examples of what might happen if the robot was to go left, starting from state 4. Then initially starting from state 4, it will receive a reward of zero, and after going left, it gets to state 3, where it receives again a reward of zero.

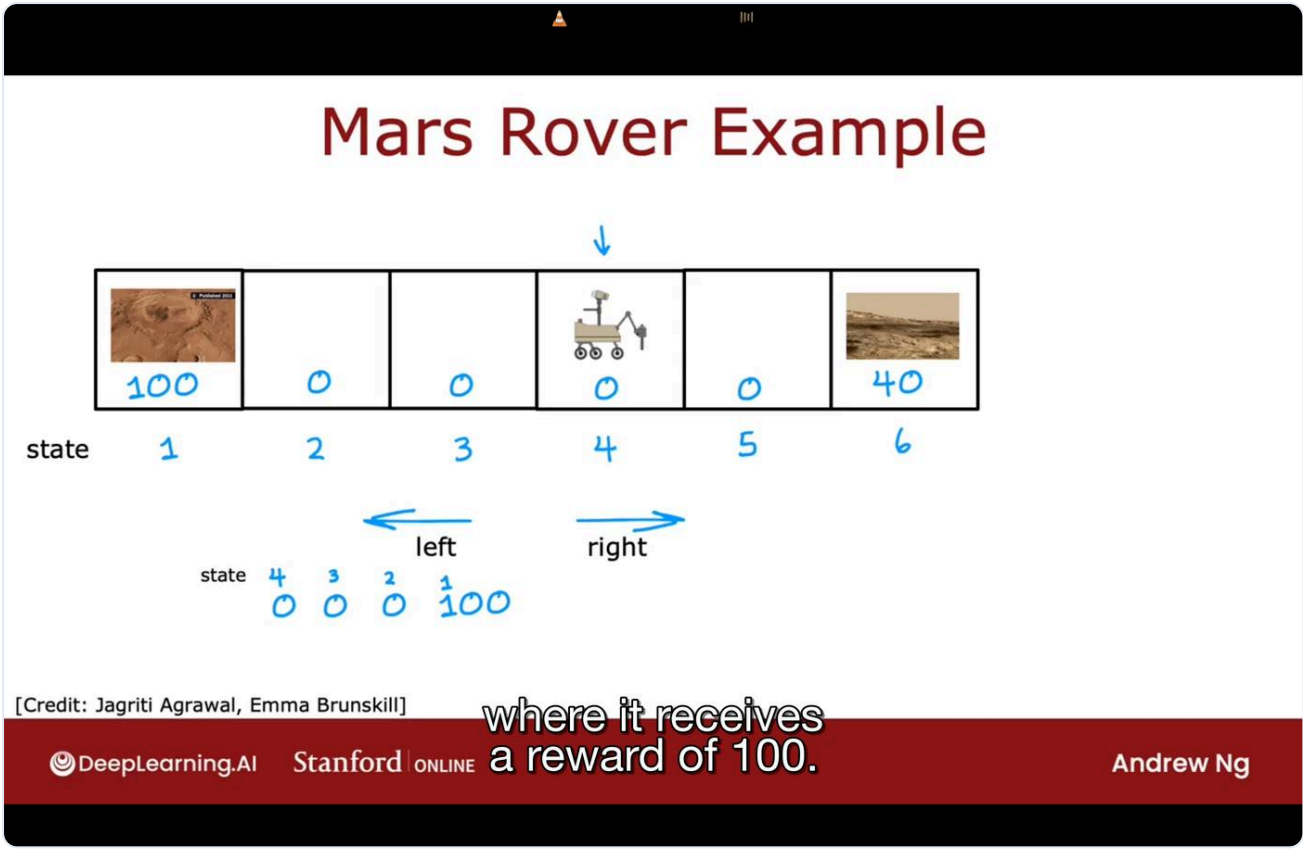


FIGURE 30.7 | Mars Rover Example

Then it gets to state 2, receives the reward is 0, and finally just to state 1, where it receives a reward of 100. For this application, I'm going to assume that when it gets either state 1 or state 6, that the day ends. In reinforcement learning, we sometimes call this a terminal state, and what that means is that, after it gets to one of these terminals states, gets a reward at that state, but then nothing happens after that. Maybe the robots run out of fuel or ran out of time for the day, which is why it only gets to either enjoy the 100 or the 40 reward, but then that's it for the day. It doesn't get to earn additional rewards after that. Now instead of going left, the robot could also choose to go to the right, in which case from state 4, it would first have a reward of zero, and then it'll move right and get to state 5, have another reward of zero, and then it will get to this other terminal state on the right, state 6 and get a reward of 40.

But going left and going right are the only options.

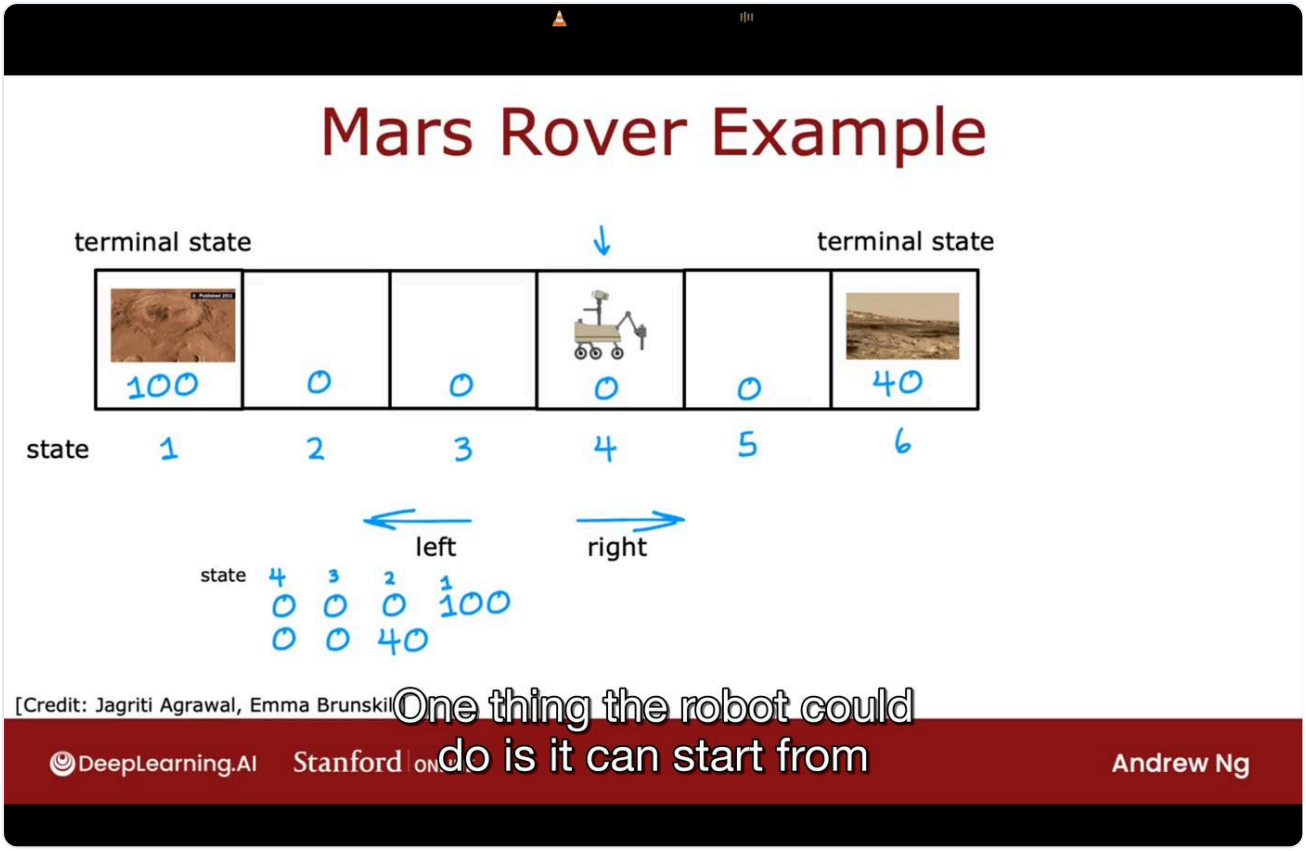


FIGURE 30.8 | Mars Rover Example

One thing the robot could do is it can start from state 4 and decide to move to the right.

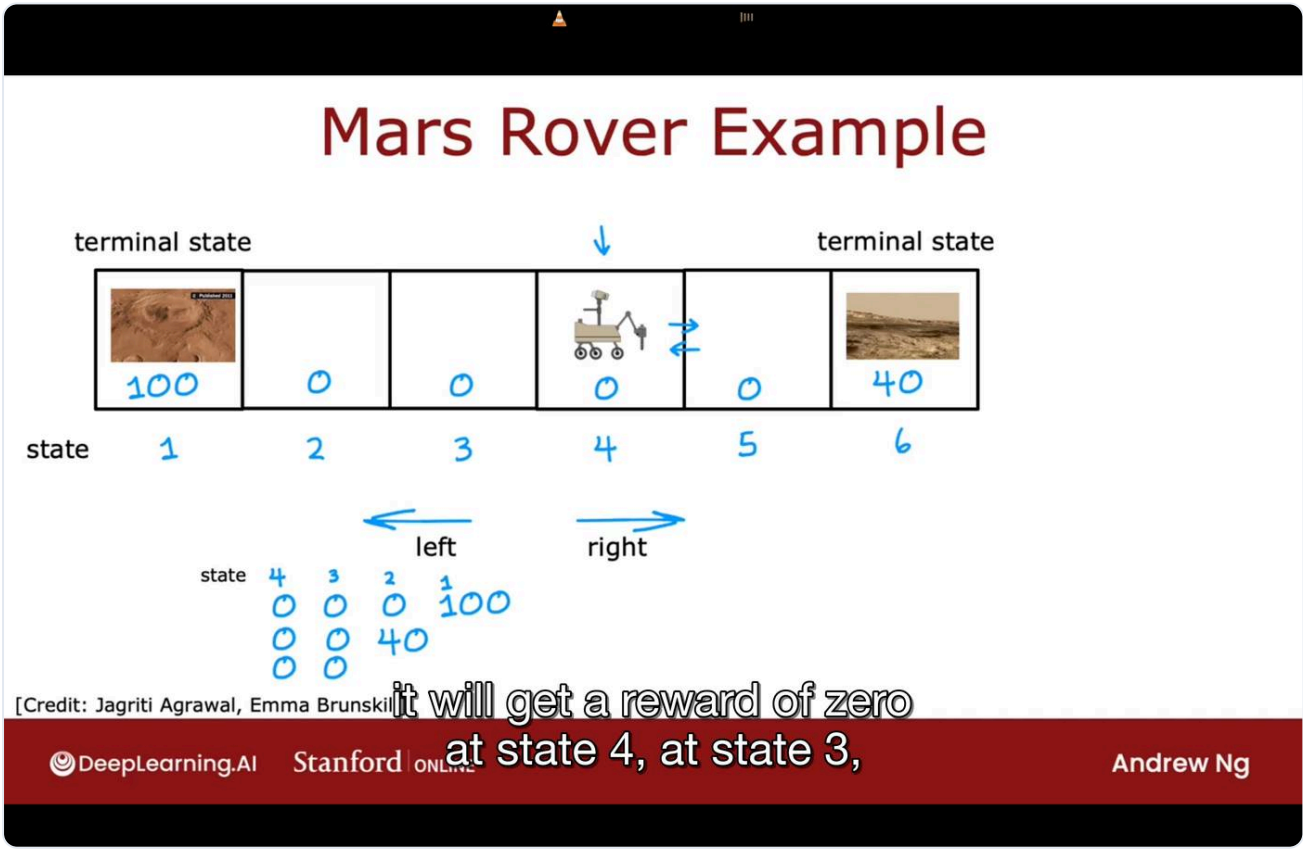


FIGURE 30.9 | Mars Rover Example

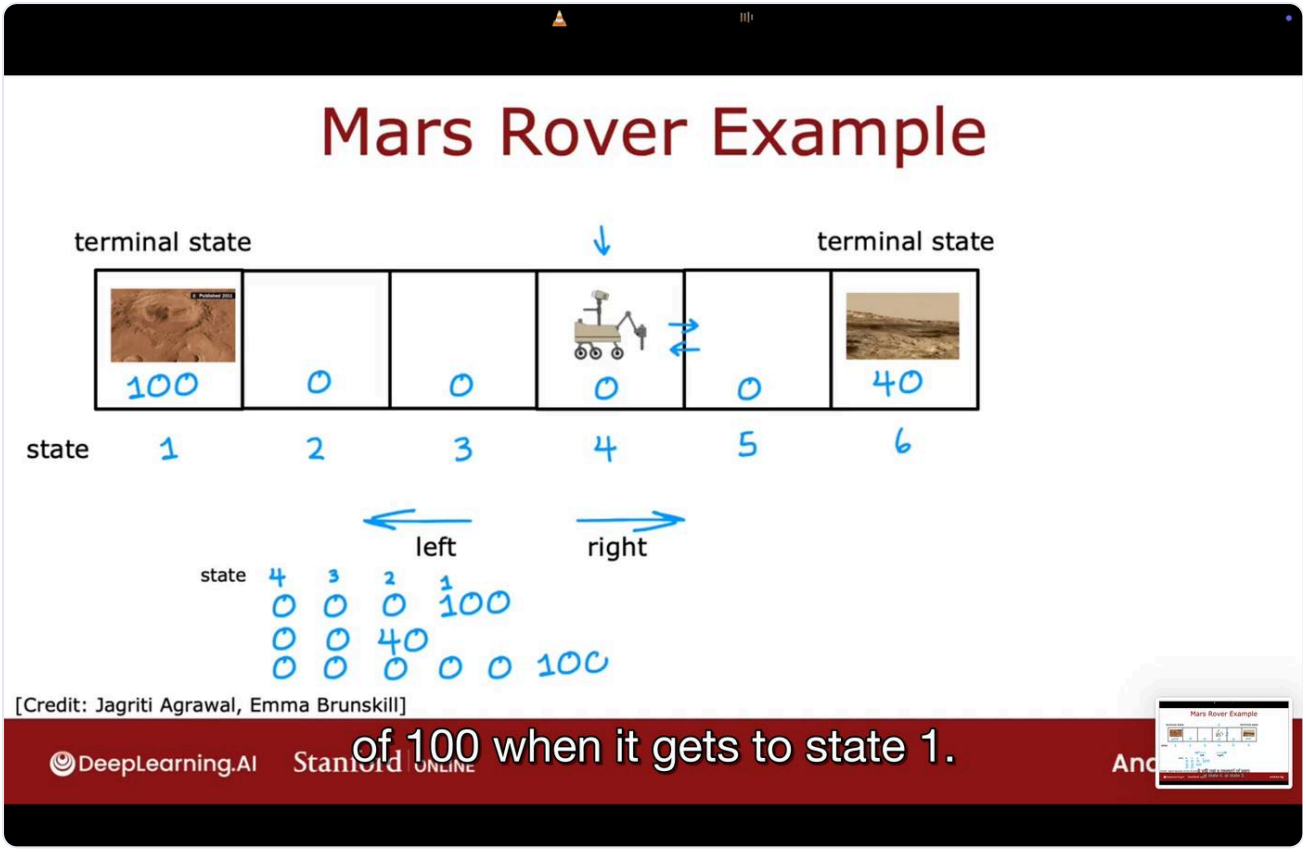


FIGURE 30.10 | Mars Rover Example

It goes from state 4-5, gets a reward of zero in state 4 and a reward of zero in state 5, and then maybe it changes its mind and decides to start going to the left as follows, in which case, it will get a reward of zero at state 4, at state 3, at state 2, and then the reward of 100 when it gets to state 1. In this sequence of actions and states, the robot is wasting a bit of time. So this maybe isn't such a great way to take actions, but it is one choice that the algorithm could pick, but hopefully you won't pick this one. To summarize, at every time step, the robot is in some state, which I'll call S, and it gets to choose an action, and it also enjoys some rewards, R of S that it gets from that state.

As a result of this action, it to some new state S prime.

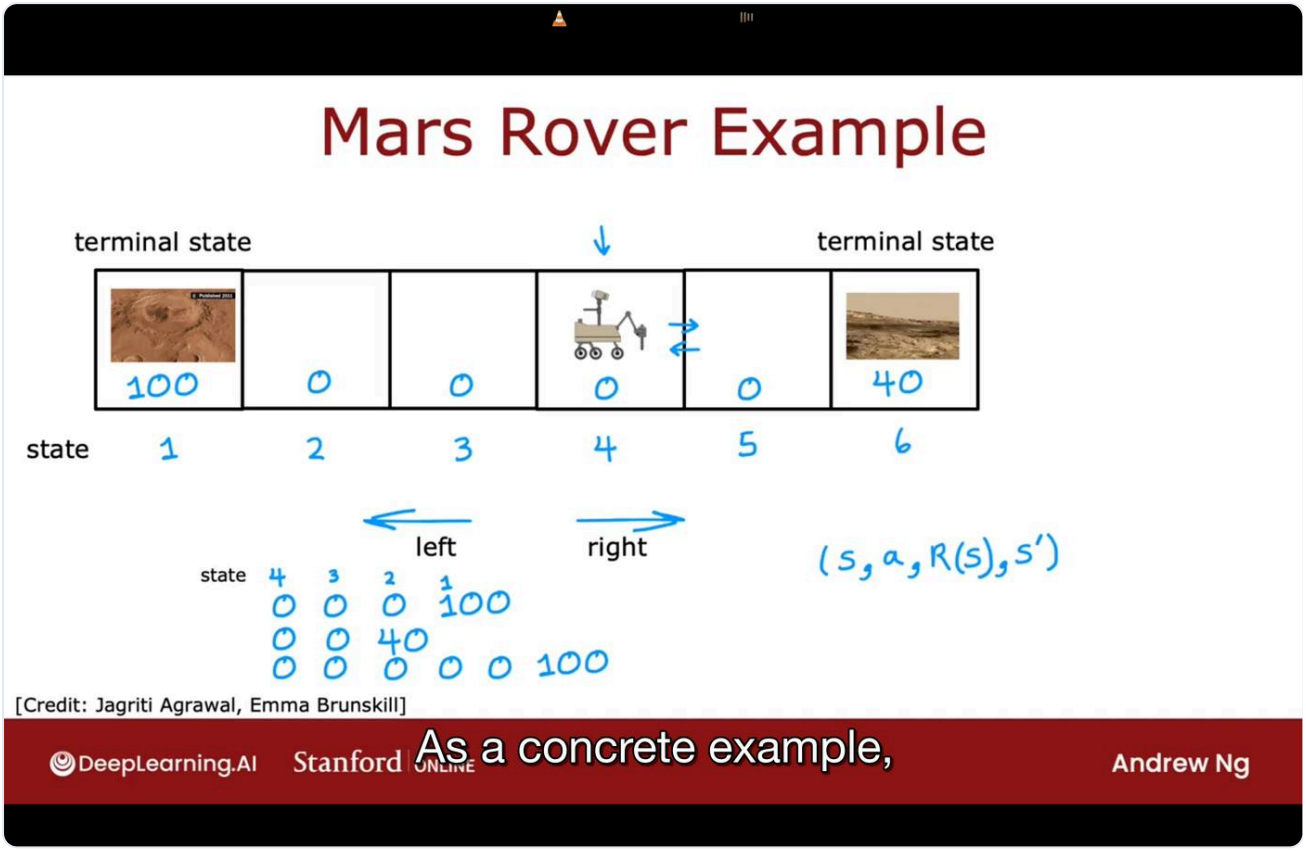


FIGURE 30.11 | Mars Rover Example

As a concrete example, when the robot was in state 4 and it took the action, go left, maybe didn't enjoy the reward of zero associated with that state 4 and it won't have any new state 3.

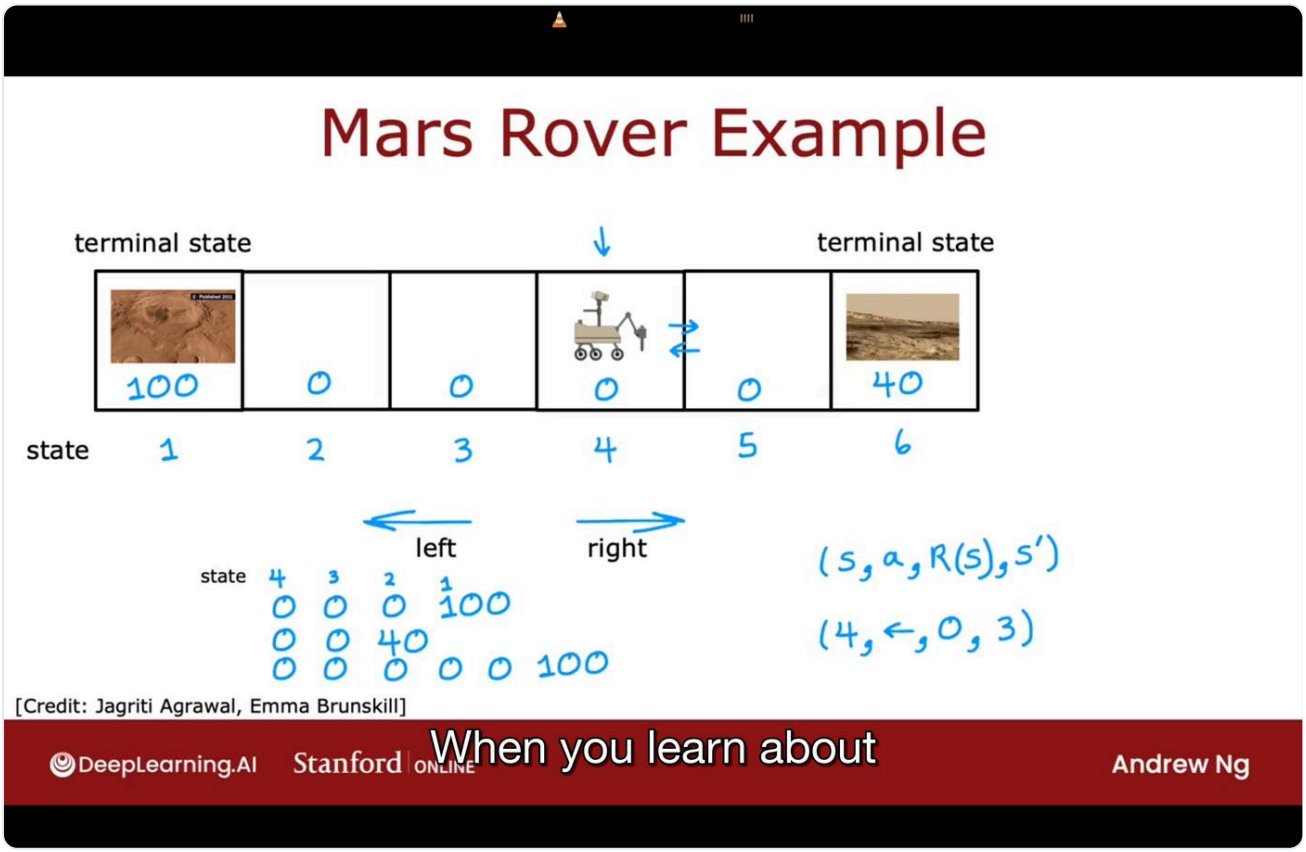


FIGURE 30.12 | Mars Rover Example

When you learn about specific reinforcement learning algorithms, you see that these four things, the state, action, the reward and next state, which is what happens basically every time you take an action that just be a core elements of what reinforcement learning algorithms will look at when deciding how to take actions. Just for clarity, the reward here, R of S, this is the reward associated with this state. This reward of zero is associated with state 4 rather than with state 3. That's the formalism of how a reinforcement learning application works. In the next video, let's take a look at how we specify exactly what we want the reinforcement learning algorithm to do.

In particular, we'll talk about an important idea in reinforcement learning called the return. Let's go on to the next video to see what that means.

LECTURE 31

Example of Return

You saw in the last video, what are the states of reinforcement learning application, as well as how depending on the actions you take you go through different states, and also get to enjoy different rewards. But how do you know if a particular set of rewards is better or worse than a different set of rewards? The return in reinforcement learning, which we'll define in this video, allows us to capture that. As we go through this, one analogy that you might find helpful is if you imagine you have a five-dollar bill at your feet, you can reach down and pick up, or half an hour across town, you can walk half an hour and pick up a 10-dollar bill. Which one would you rather go after?

Ten dollars is much better than five dollars, but if you need to walk for half an hour to go and get that 10-dollar bill, then maybe it'd be more convenient to just pick up the five-dollar bill instead. The concept of a return captures that rewards you can get quicker are maybe more attractive than rewards that take you a long time to get to. Let's take a look at exactly how that works. Here's a Mars Rover example. If starting from state 4 you go to the left, we saw that the rewards you get would be zero on the first step from state 4, zero from state 3, zero from state 2, and then 100 at state 1, the terminal state. The return is defined as the sum of these rewards but weighted by one additional factor, which is called the discount factor.

The discount factor is a number a little bit less than 1. Let me pick 0.9 as the discount factor. I'm going to weight the reward in the first step is just zero, the reward in the second step is a discount factor, 0.9 times that reward, and then plus the discount factor^2 times that reward, and then plus the discount factor^3 times that reward. If you calculate this out, this turns out to be 0.729 times 100, which is 72.9.

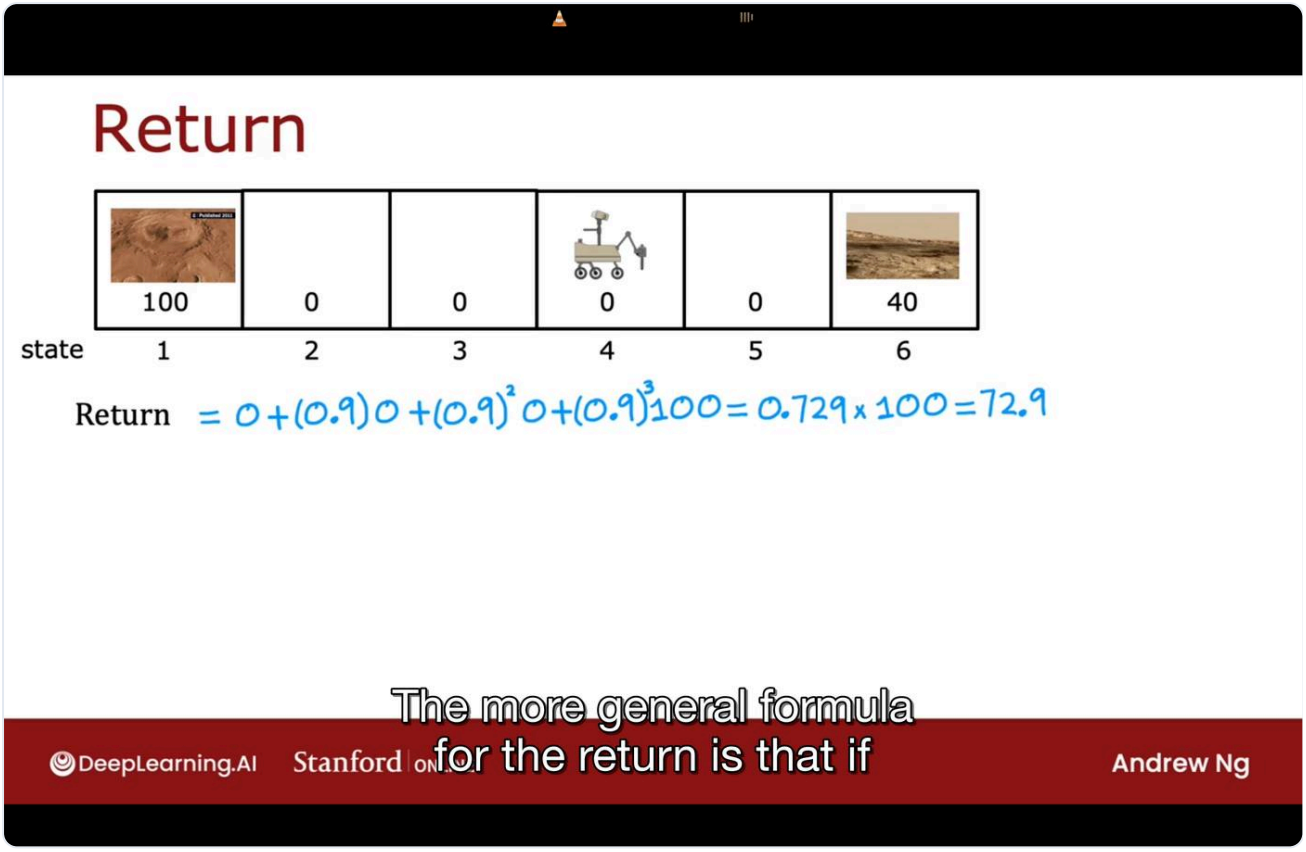


FIGURE 31.1

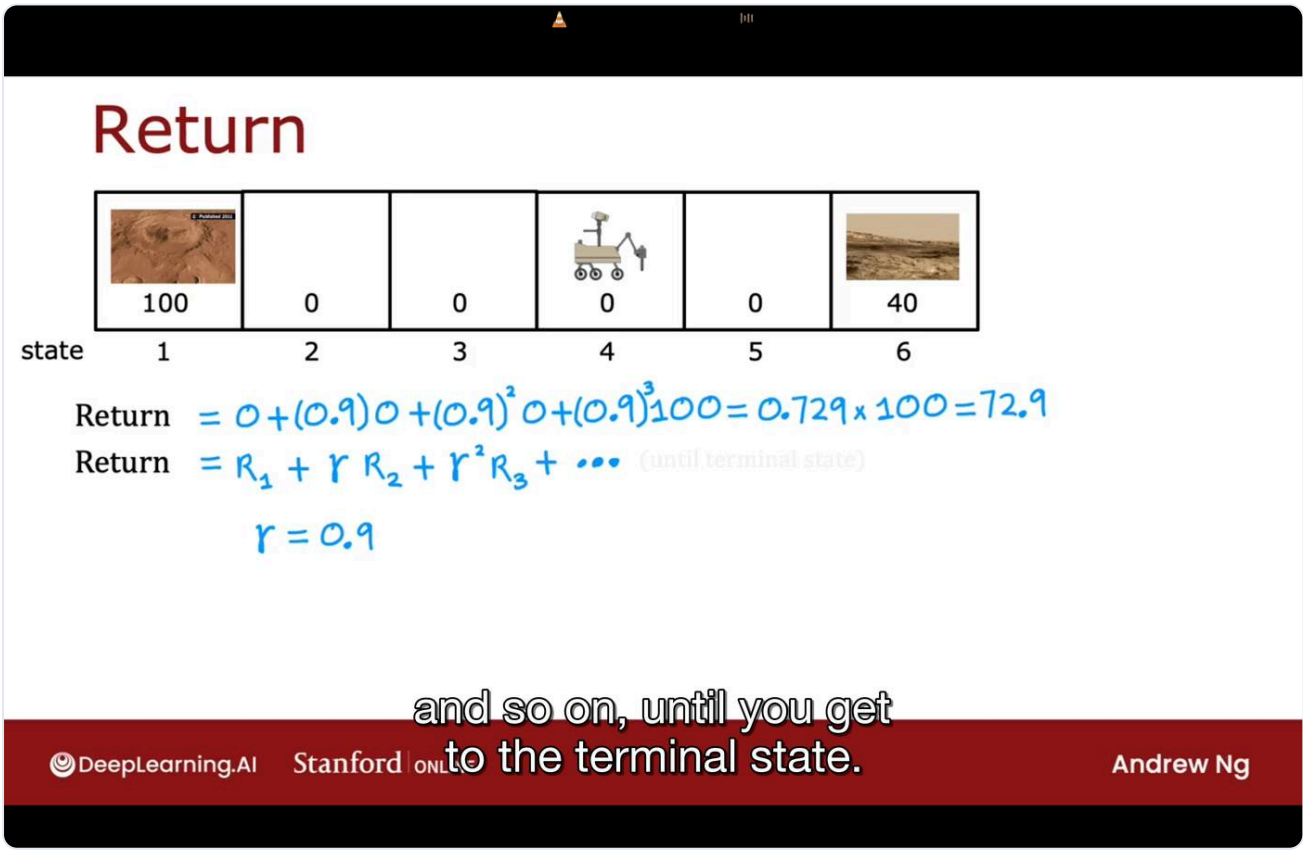


FIGURE 31.2

The more general formula for the return is that if your robot goes through some sequence of states and gets reward R_1 on the first step, and R_2 on the second step, and R_3 on the third step, and so on, then the return is R_1 plus the discount factor Gamma, this Greek alphabet Gamma which I've set to 0.9 in this example, the Gamma times R_2 plus Gamma^2 times R_3 plus Gamma^3 times R_4, and so on, until you get to the terminal state.

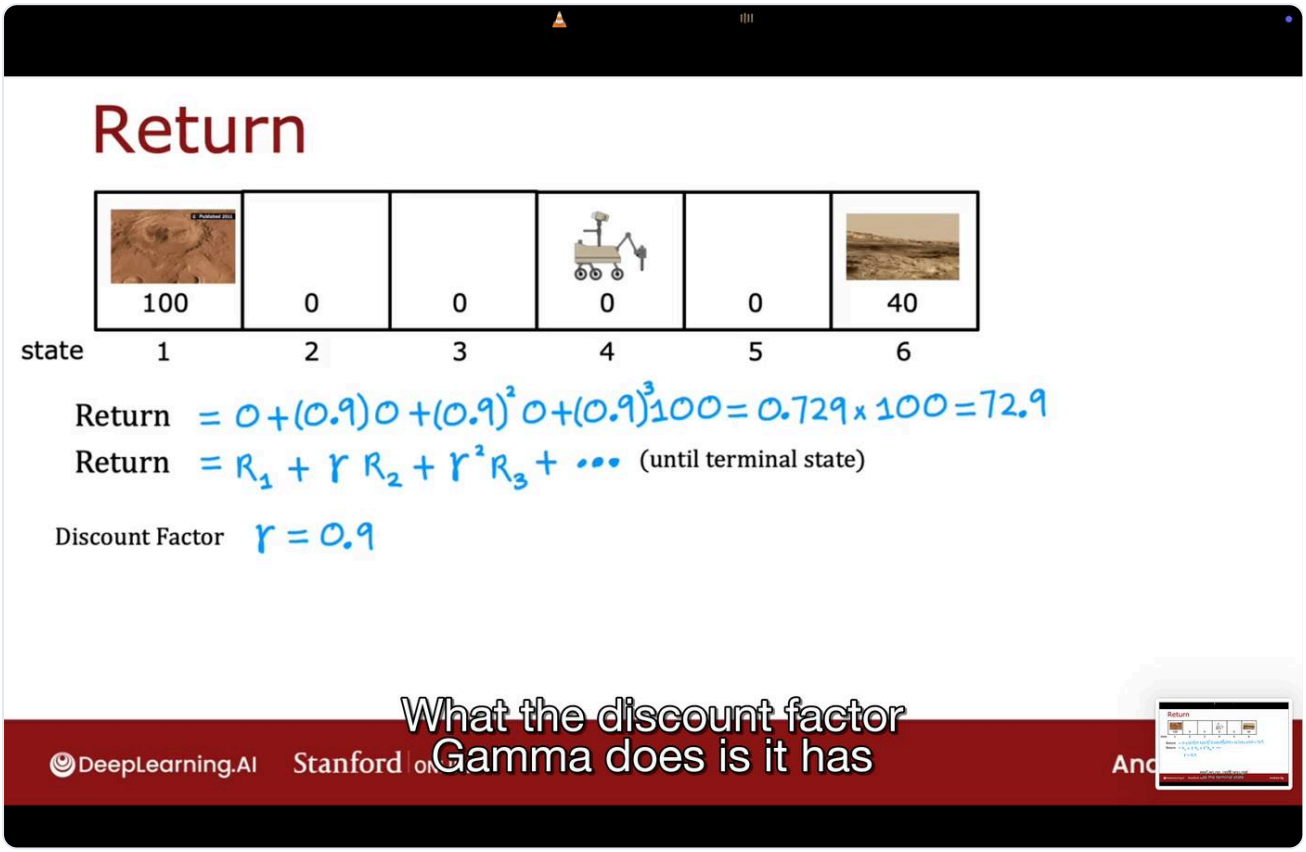


FIGURE 31.3

What the discount factor Gamma does is it has the effect of making the reinforcement learning algorithm a little bit impatient. Because the return gives full credit to the first reward is 100 percent is 1 times R_1, but then it gives a little bit less credit to the reward you get at the second step is multiplied by 0.9, and then even less credit to the reward you get at the next time step R_3, and so on, and so getting rewards sooner results in a higher value for the total return. In many reinforcement learning algorithms, a common choice for the discount factor will be a number pretty close to 1, like 0.9, or 0.99, or even 0.999.

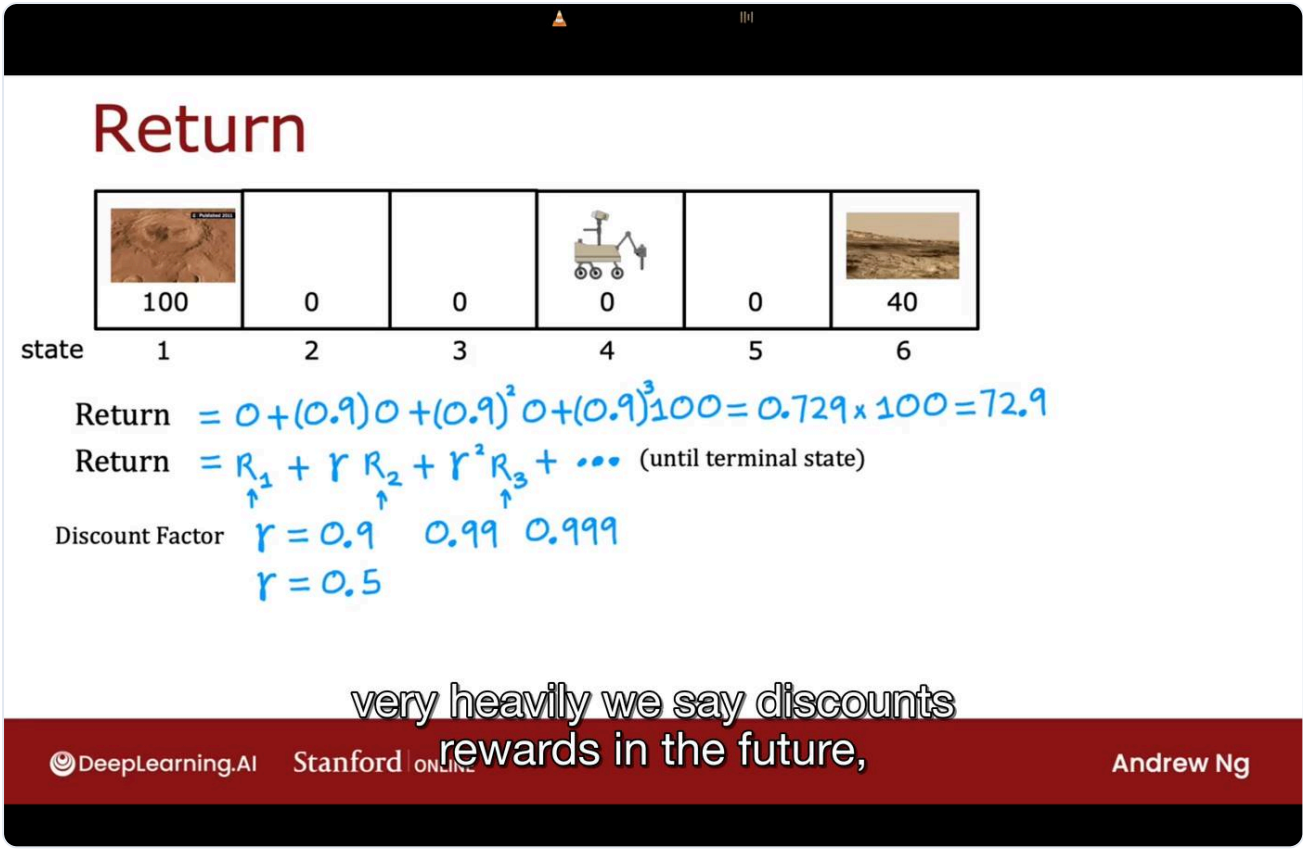


FIGURE 31.4

But for illustrative purposes in the running example I'm going to use, I'm actually going to use a discount factor of 0.5. This very heavily down weights or very heavily we say discounts rewards in the future, because with every additional passing timestamp, you get only half as much credit as rewards that you would have gotten one step earlier. If Gamma were equal to 0.5, the return under the example above would have been 0 plus 0.5 times 0, replacing this equation on top, plus 0.5^2 0 plus 0.5^3 times 100.

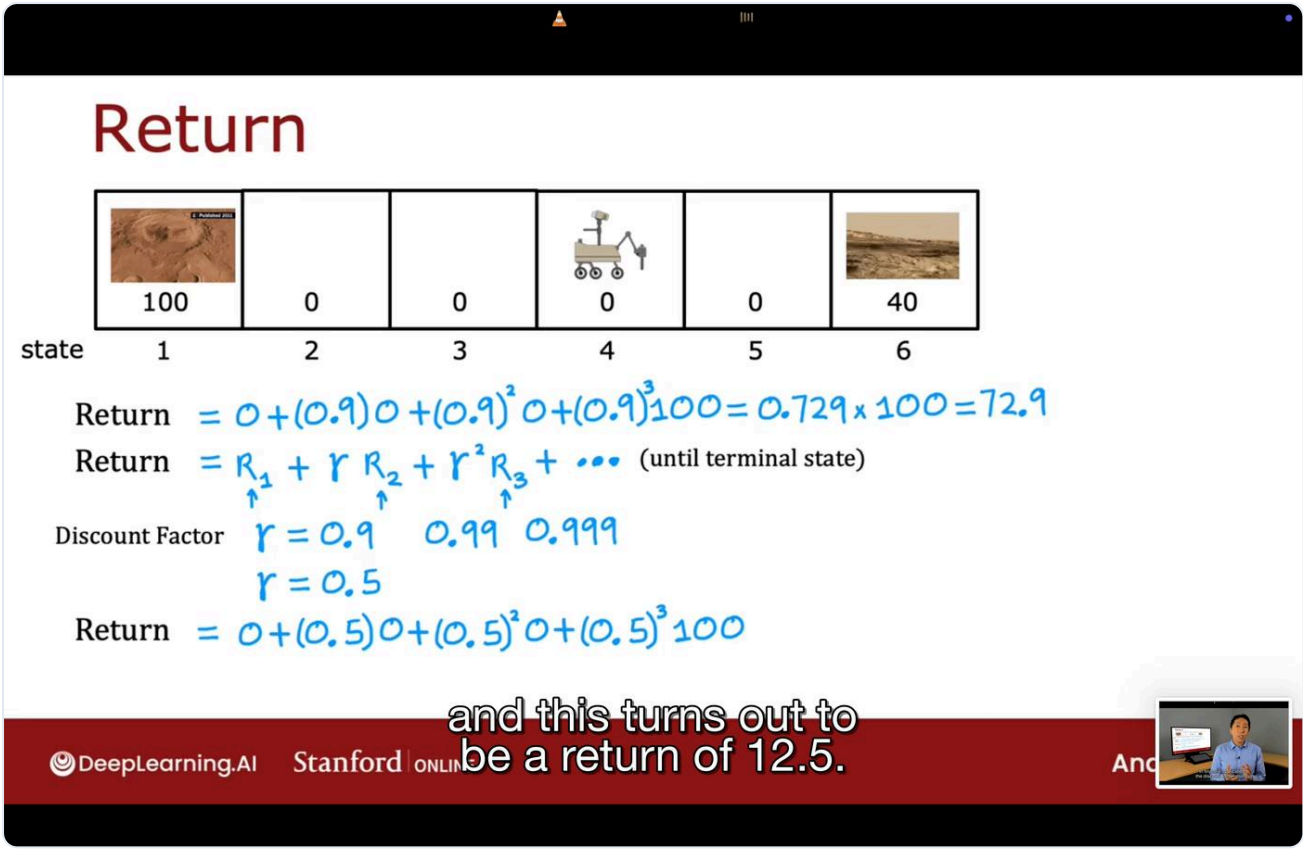


FIGURE 31.5

That's lost reward because state 1 to terminal state, and this turns out to be a return of 12.5.

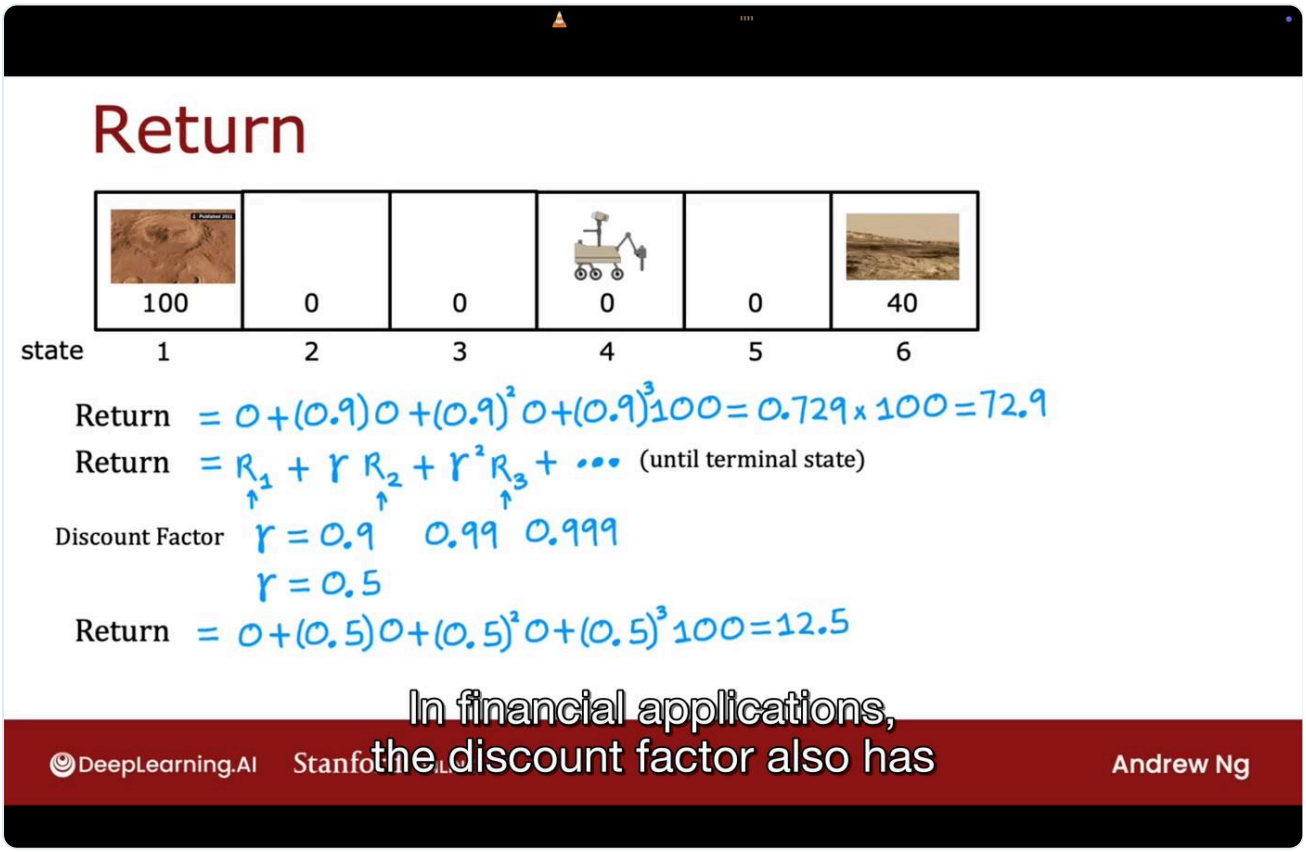


FIGURE 31.6

In financial applications, the discount factor also has a very natural interpretation as the interest rate or the time value of money. If you can have a dollar today, that may be worth a little bit more than if you could only get a dollar in the future. Because even a dollar today you can put in the bank, earn some interest, and end up with a little bit more money a year from now. For financial applications, often, that discount factor represents how much less is a dollar in the future where I've compared to a dollar today. Let's look at some concrete examples of returns. The return you get depends on the rewards, and the rewards depends on the actions you take, and so the return depends on the actions you take.

Let's use our usual example and say for this example, I'm going to always go to the left. We already saw previously that if the robot were to start off in state 4, the return is 12.5 as we worked out on the previous slide. It turns out that if it were to start off in say three, the return would be 25 because it gets to the 100 reward one step sooner, and so it's discounted less. If it were to start off in state 2, the return would be 50. If it were to just start off and state 1, well, it gets the reward of 100 right away, so it's not discounted at all. The return if we were to start out in state 1 will be 100, and then the return in these two states are 6.25.

It turns out if you start off in state 6, which is terminal state, you just get the reward and thus the return of 40.

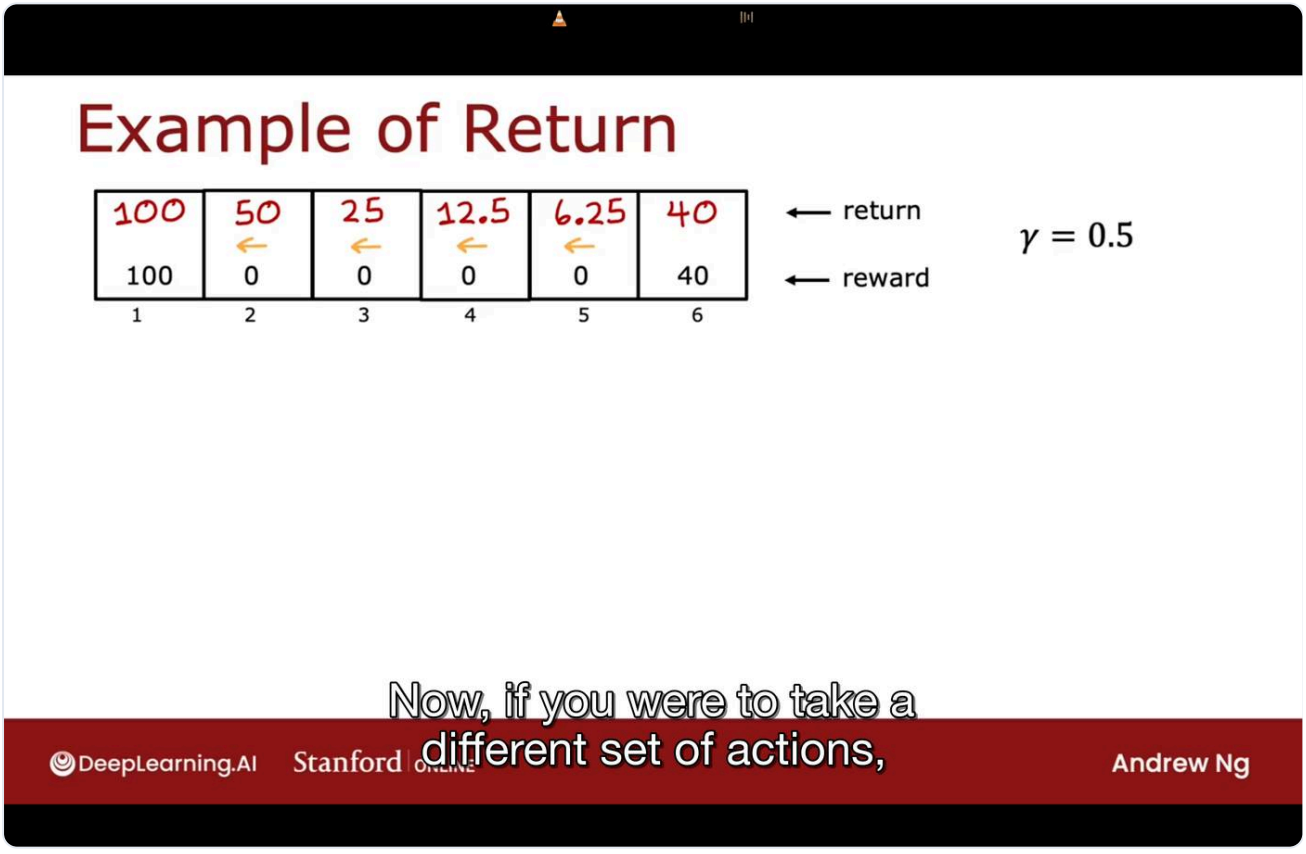


FIGURE 31.7 | Example of Return

Now, if you were to take a different set of actions, the returns would actually be different. For example, if we were to always go to the right, if those were our actions, then if you were to start in state 4, get a reward of 0. Then you get to state 5, get a reward of 0, and it gets to state 6, and get a

reward of 40. In this case, the return would be 0 plus 0.5, the discount factor times 0 plus 0.5 squared times 40, and that turns out to be equal to 0.5 squared is 1/4, so 1/4 of 40 is 10. The return from this state, from state 4 is 10. If you were to take actions, always go to the right.

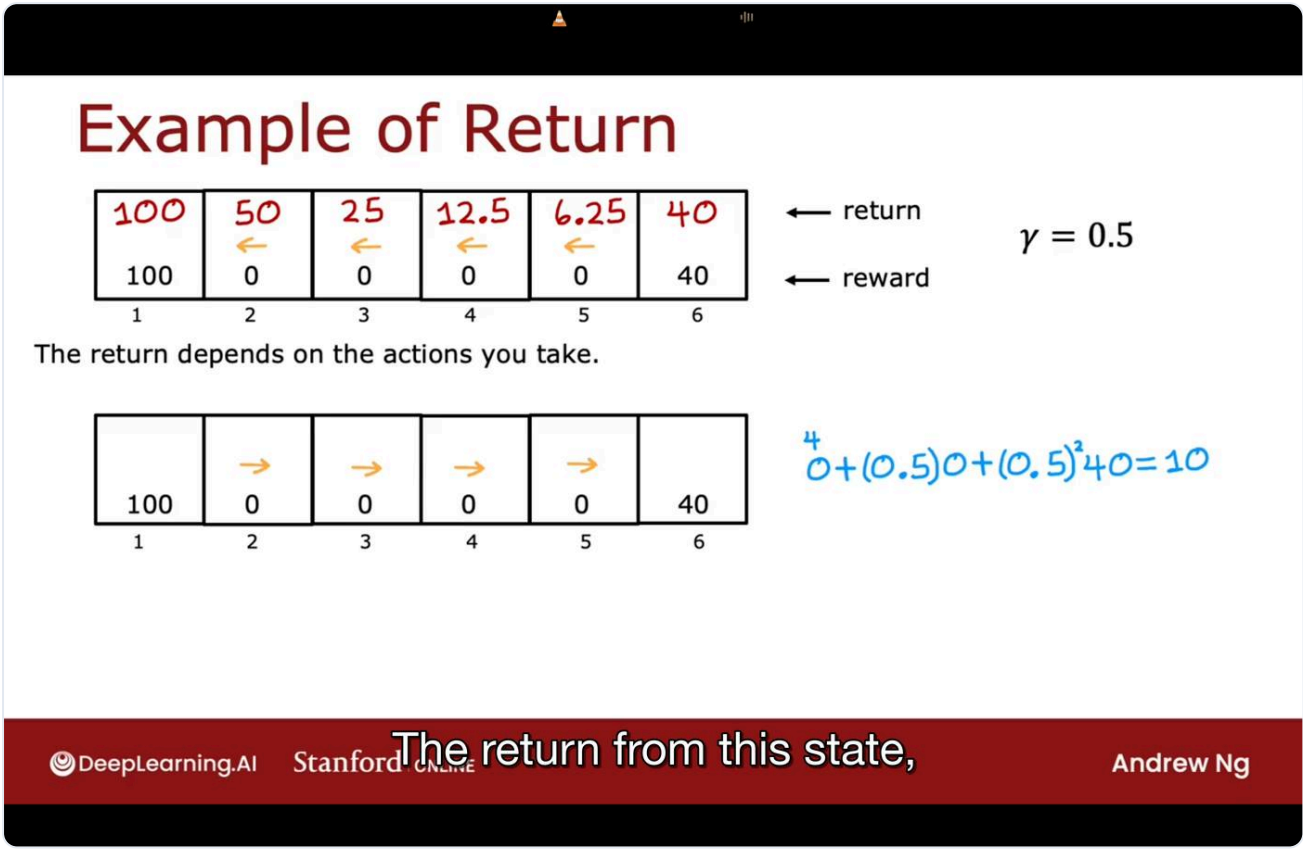


FIGURE 31.8 | Example of Return

Through similar reasoning, the return from this state is 20, the return from this state is five, the return from this state is 2.5, and then the return, the determinant state is is 140.

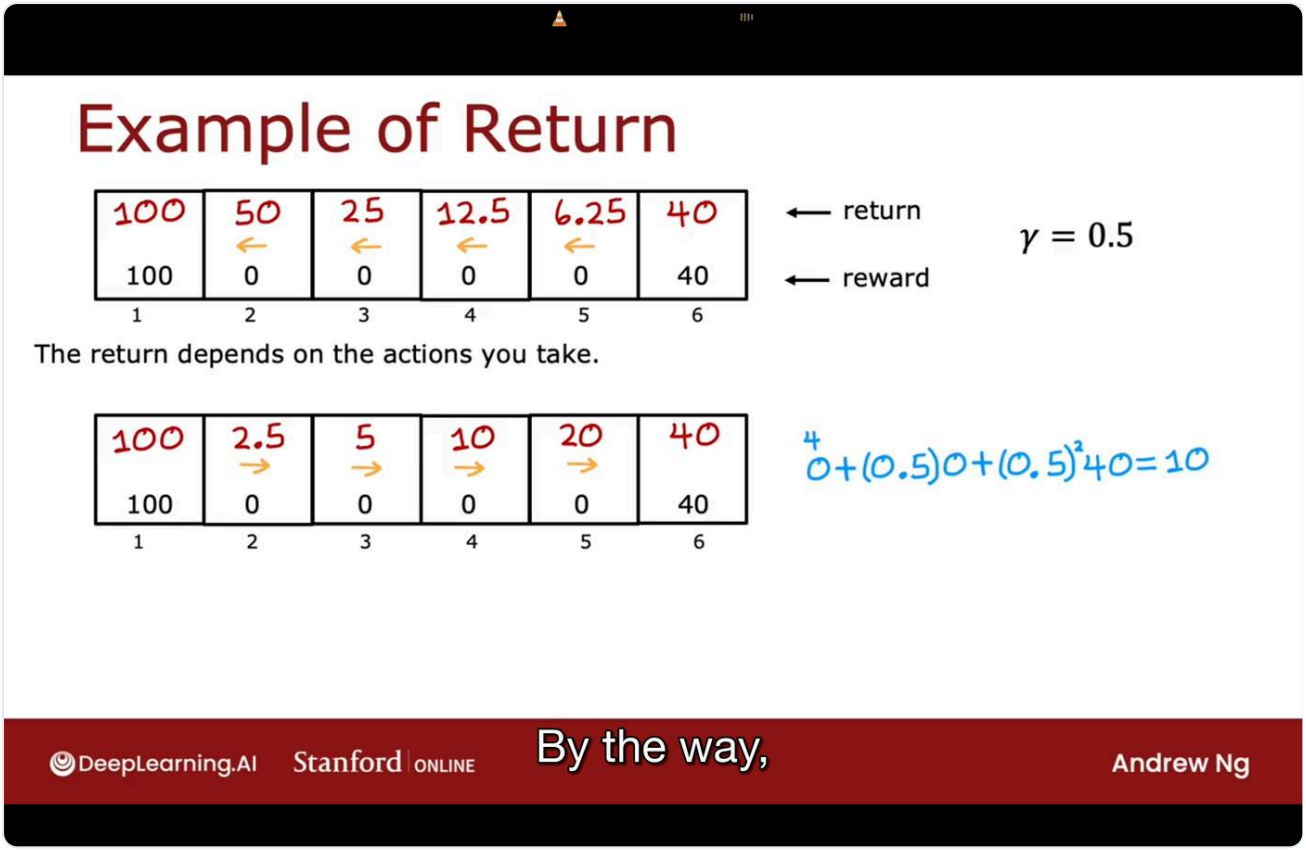


FIGURE 31.9 | Example of Return

By the way, if these numbers don't fully make sense, feel free to pause the video and double-check the math and see if you can convince yourself that these are the appropriate values for the return. For if you start from different states, and if you were to always go to the right. We see that it would always go to the right. The return you expect to get is lower for most states. Maybe always going to the right isn't as good an idea as always going to the left. But it turns out that we don't have to always go to the left, always go to the right. We could also decide if you're in state 2, go left. If your in state 3, go left. If you're in state 4, go left.

But if you're in state 5, then you're so close to this reward. Let's go right. This will be a different way of choosing actions to take based on what state you're in. It turns out that the return you get from the different states will be 100, 50, 25, 12.5, 20, and 40.

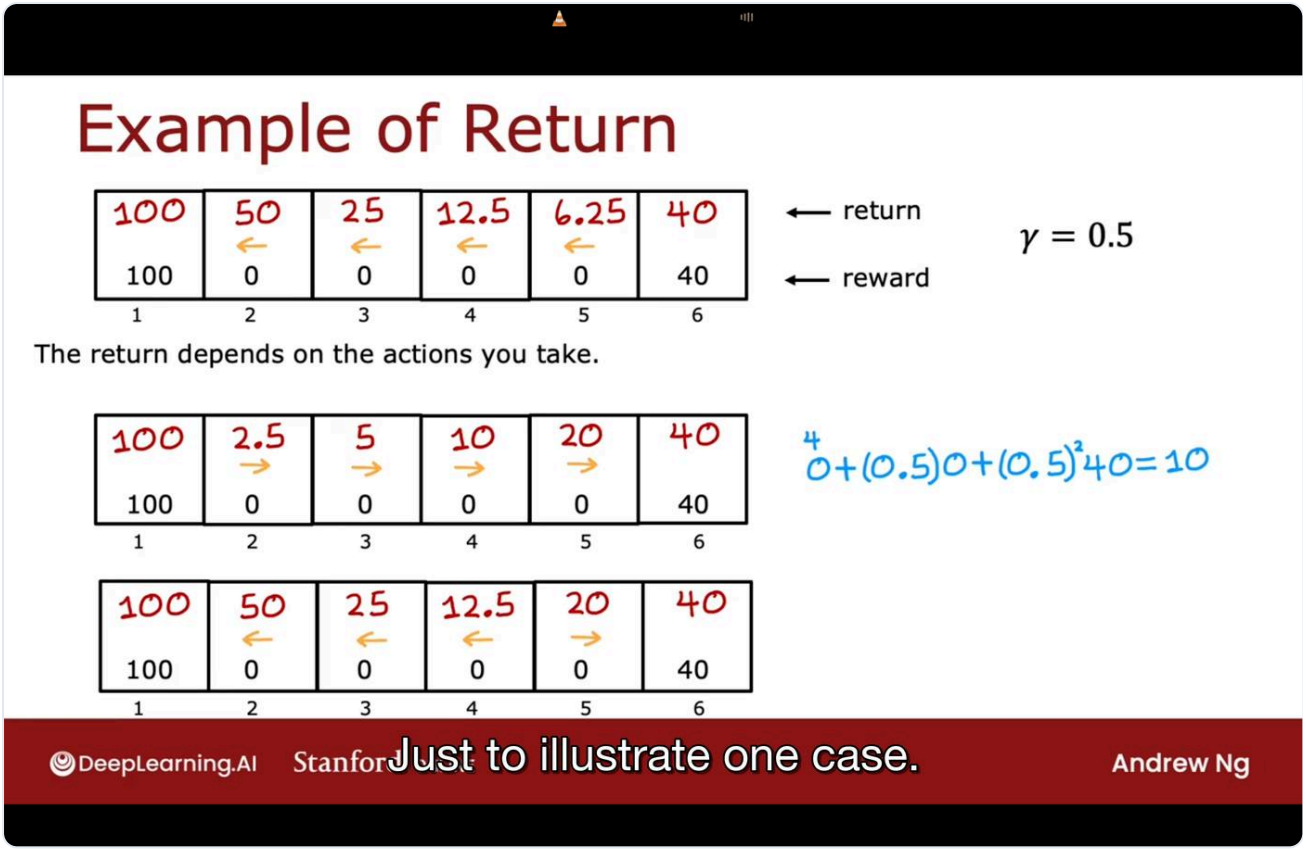


FIGURE 31.10 | Example of Return

Just to illustrate one case. If you were to start off in state 5, here you would go to the right, and so the rewards you get would be zero first in state 5, and then 4.

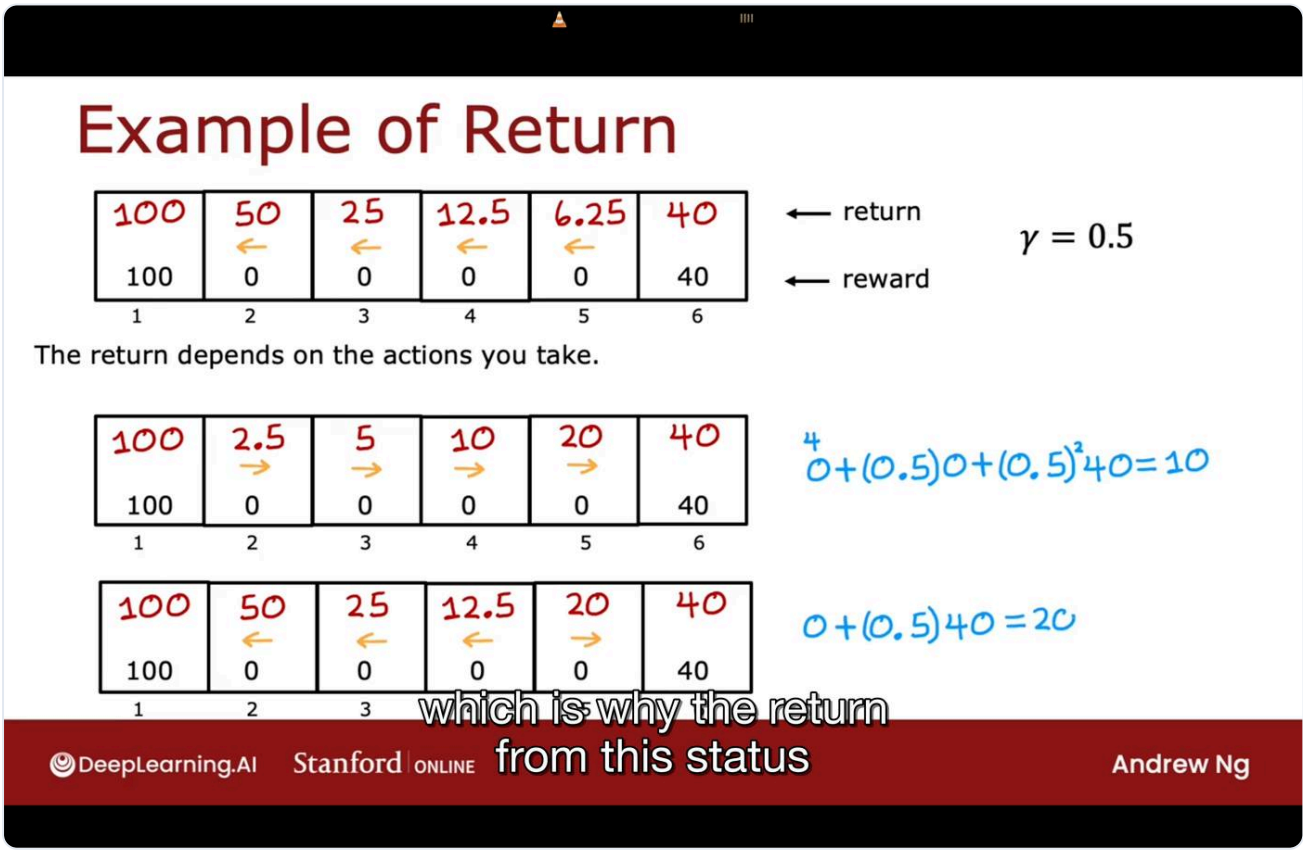


FIGURE 31.11 | Example of Return

The return is zero, the first reward, plus the discount factor is 0.5 times 40, which is 20, which is why the return from this status 20 if you take actions shown here. To summarize, the return in reinforcement learning is the sum of the rewards that the system gets, weighted by the discount factor, where rewards in the far future are weighted by the discount factor raised to a higher power. Now, this actually has an interesting effect when you have systems with negative rewards. In the example we went through, all the rewards were zero or positive. But if there are any rewards are negative, then the discount factor actually incentivizes the system to push out the negative rewards as far into the future as possible.

Taking a financial example, if you had to pay someone $10, maybe that's a negative reward of minus 10. But if you could postpone payment by a few years, then you're actually better off because $10 a few years from now, because of the interest rate is actually worth less than $10 that you had to pay today. For systems with negative rewards, it causes the algorithm to try to push out the make the rewards as far into the future as possible. For financial applications and for other applications, that actually turns out to be right thing for the system to do. You now know what is the return in reinforcement learning, let's go on to the next video to formalize the goal of reinforcement learning algorithm.

LECTURE 32

The Goal of Reinforcement Learning

Let's formalize how a reinforcement learning algorithm picks actions. In this video, you'll learn about what is a policy in reinforcement learning algorithm. Let's take a look.

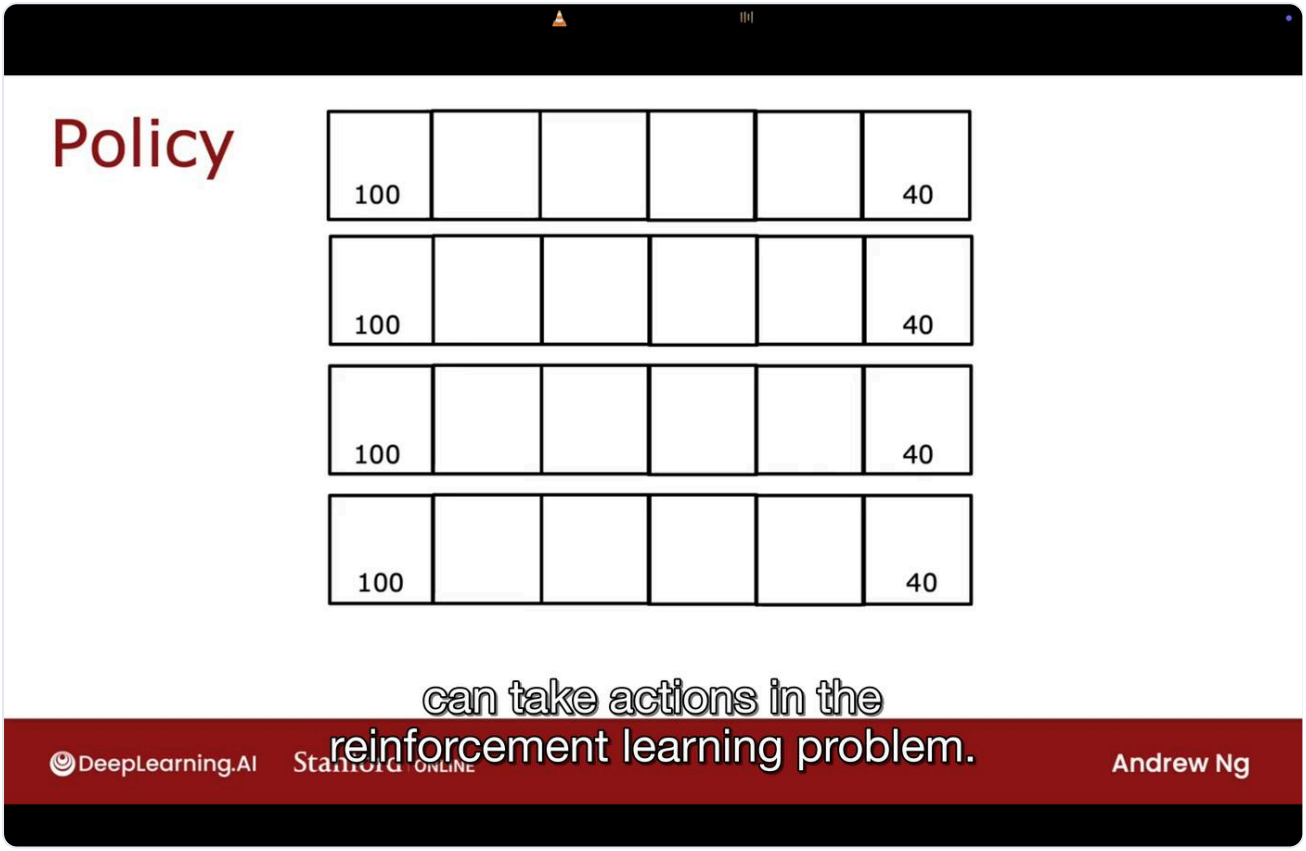


FIGURE 32.1

As we've seen, there are many different ways that you can take actions in the reinforcement learning problem. For example, we could decide to always go for the nearer reward, so you go left if this leftmost reward is nearer or go right if this rightmost reward is nearer.

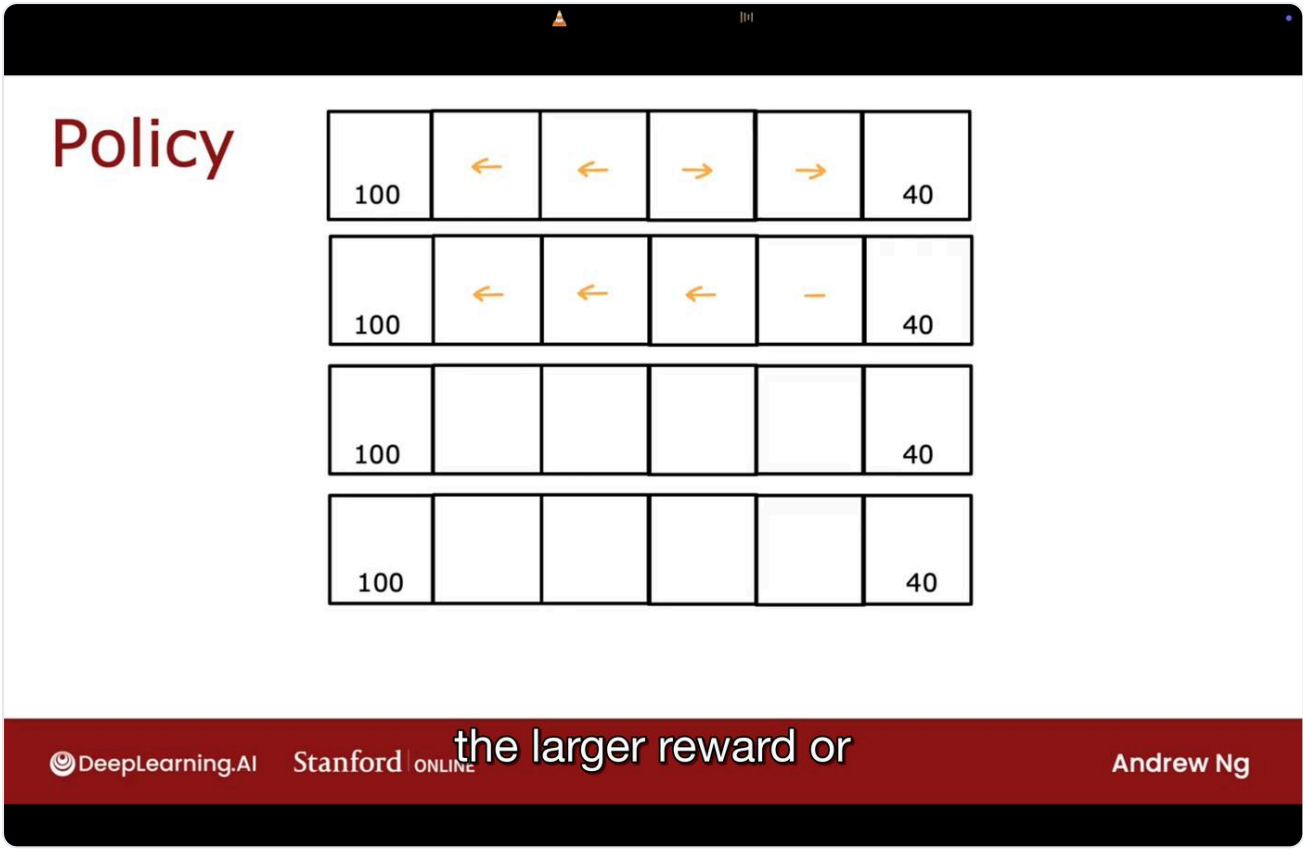


FIGURE 32.2

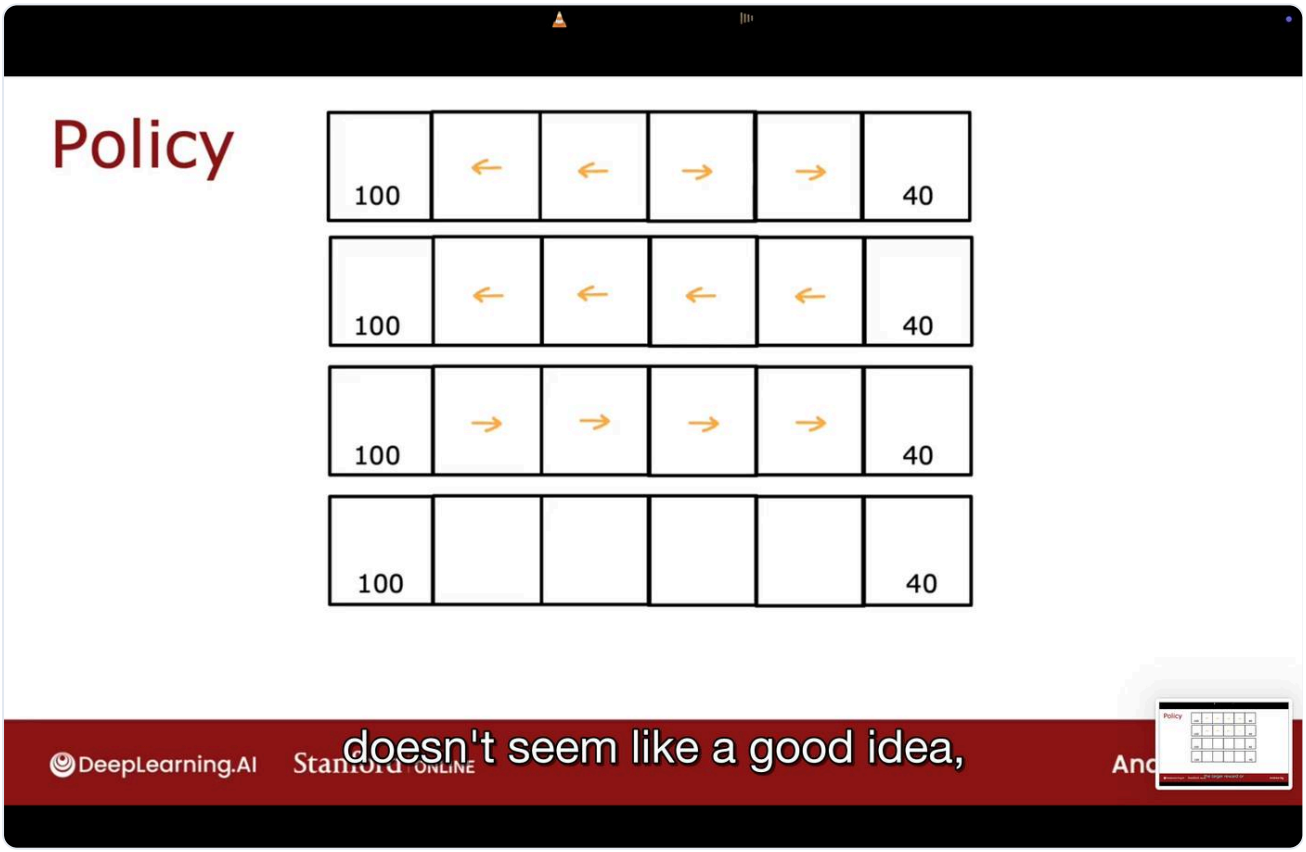


FIGURE 32.3

FIGURE 32.4

Another way we could choose actions is to always go for the larger reward or we could always go for smaller reward, doesn't seem like a good idea, but it is another option, or you could choose to go left unless you're just one step away from the lesser reward, in which case, you go for that one. In reinforcement learning, our goal is to come up with a function which is called a policy Pi, whose job it is to take as input any state s and map it to some action a that it wants us to take.

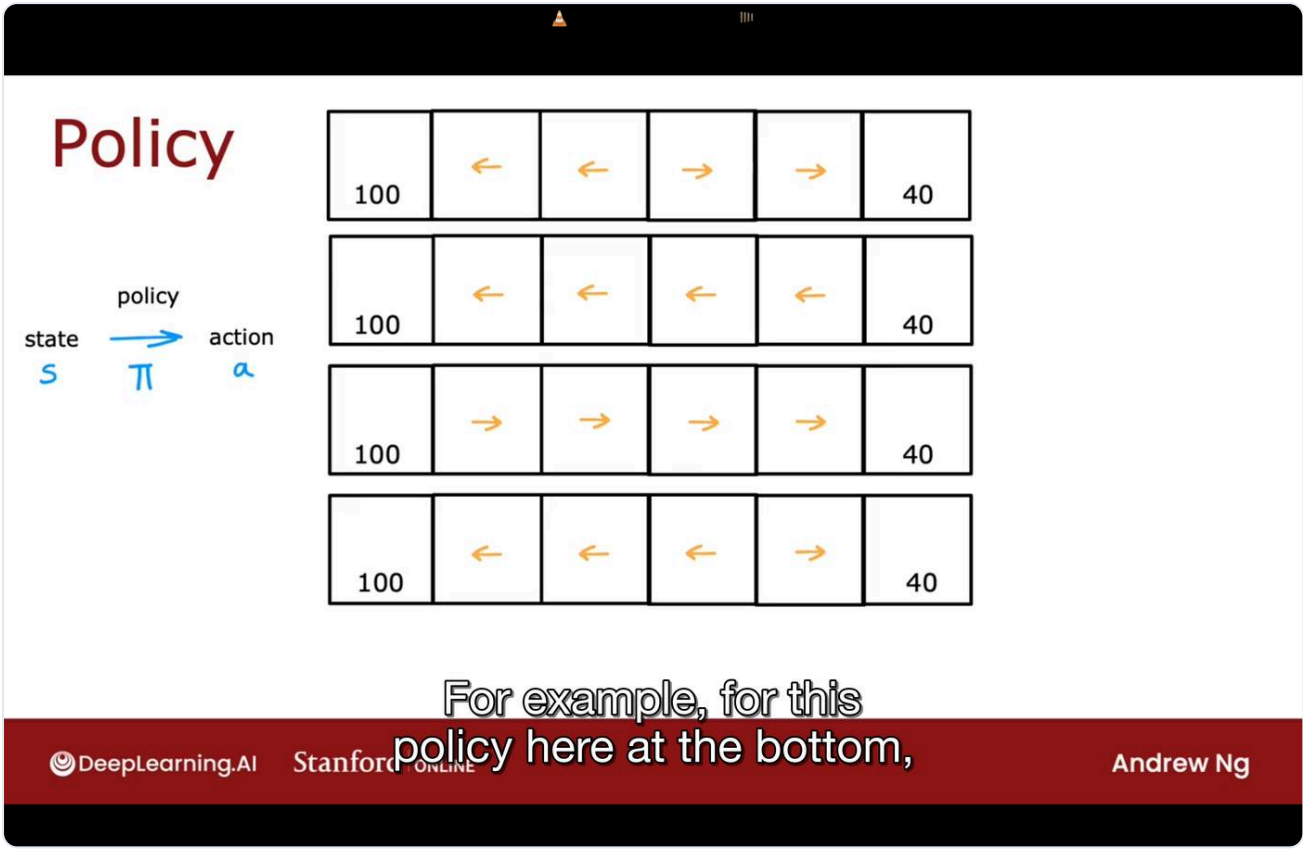


FIGURE 32.5

For example, for this policy here at the bottom, this policy would say that if you're in state 2, then it maps us to the left action. If you're in state 3, the policy says go left.

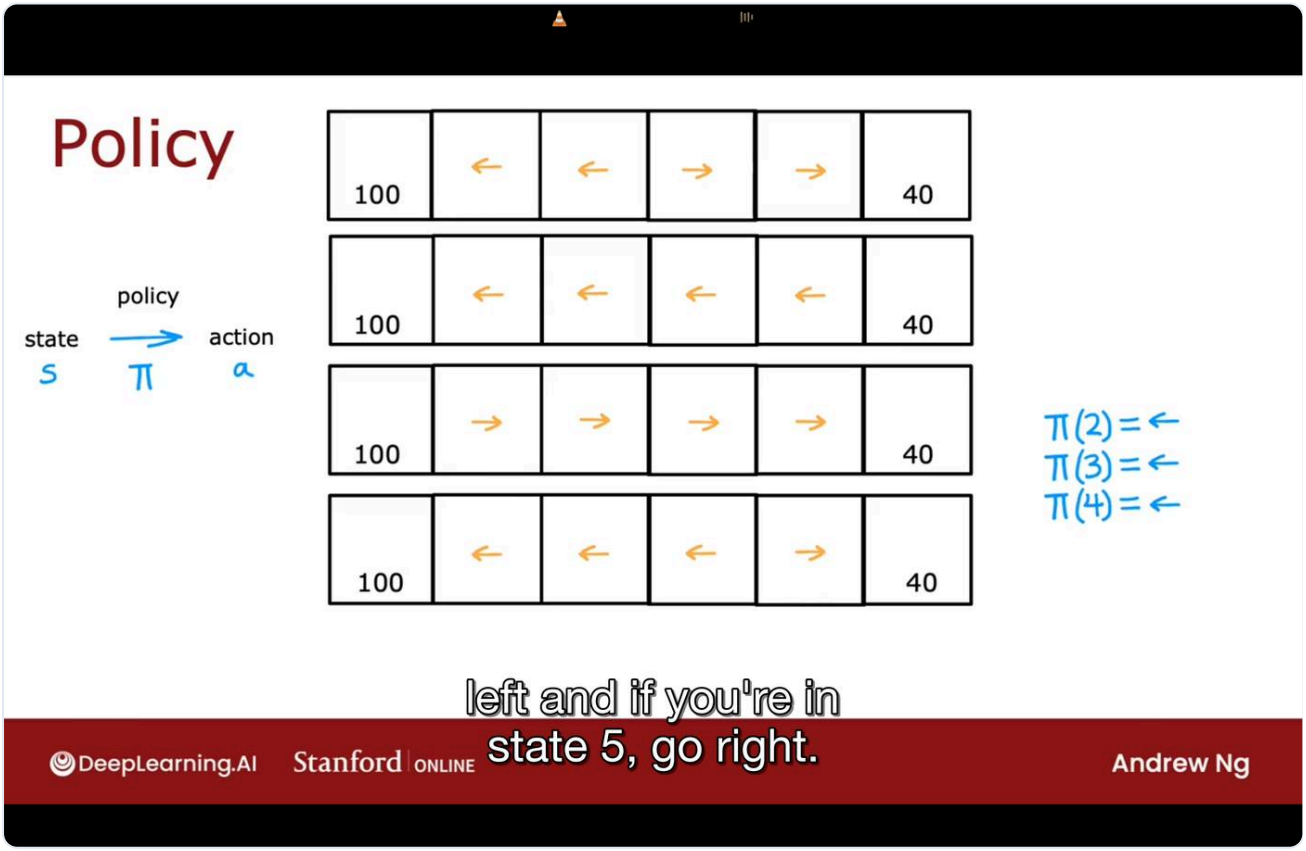


FIGURE 32.6

If you are in state 4 also go left and if you're in state 5, go right.

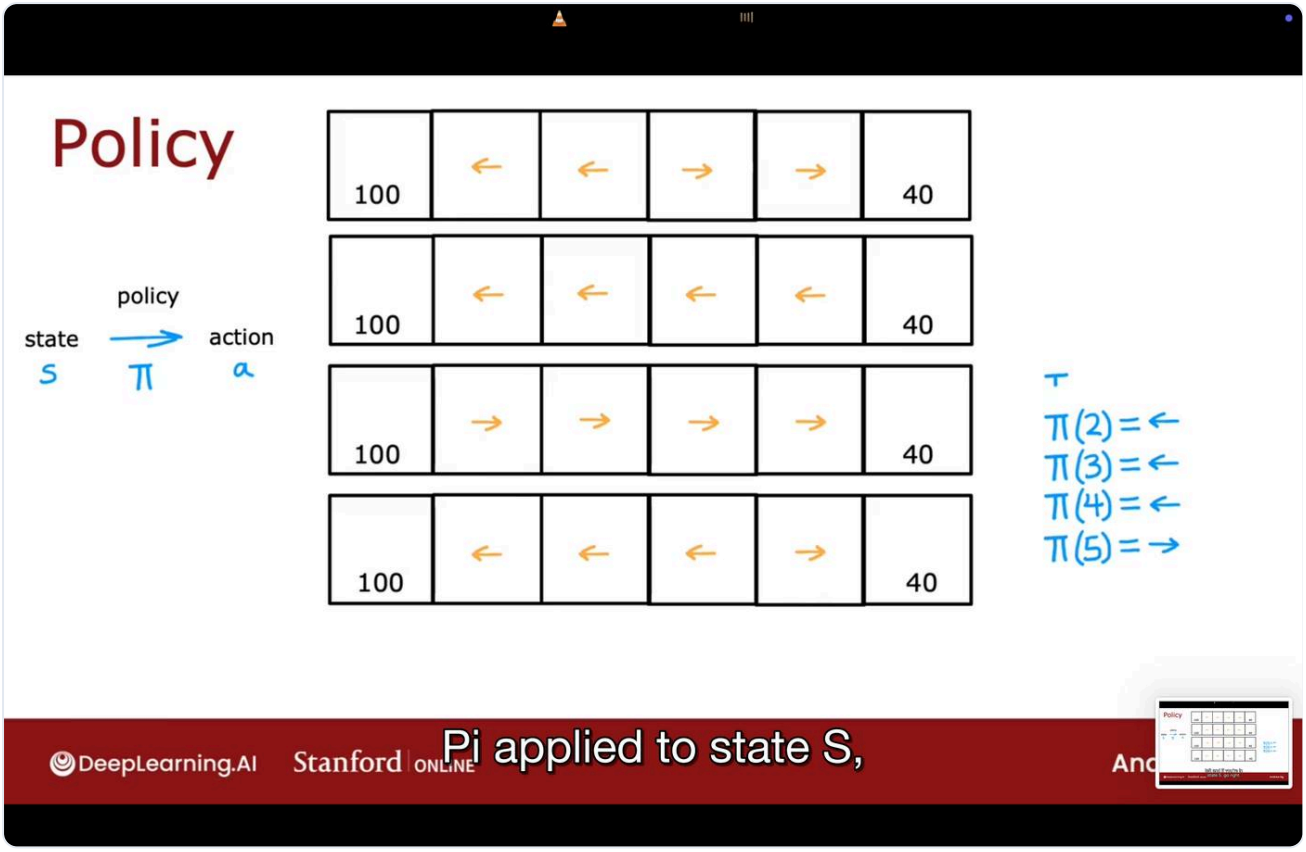


FIGURE 32.7

Pi applied to state S, tells us what action it wants us to take in that state.

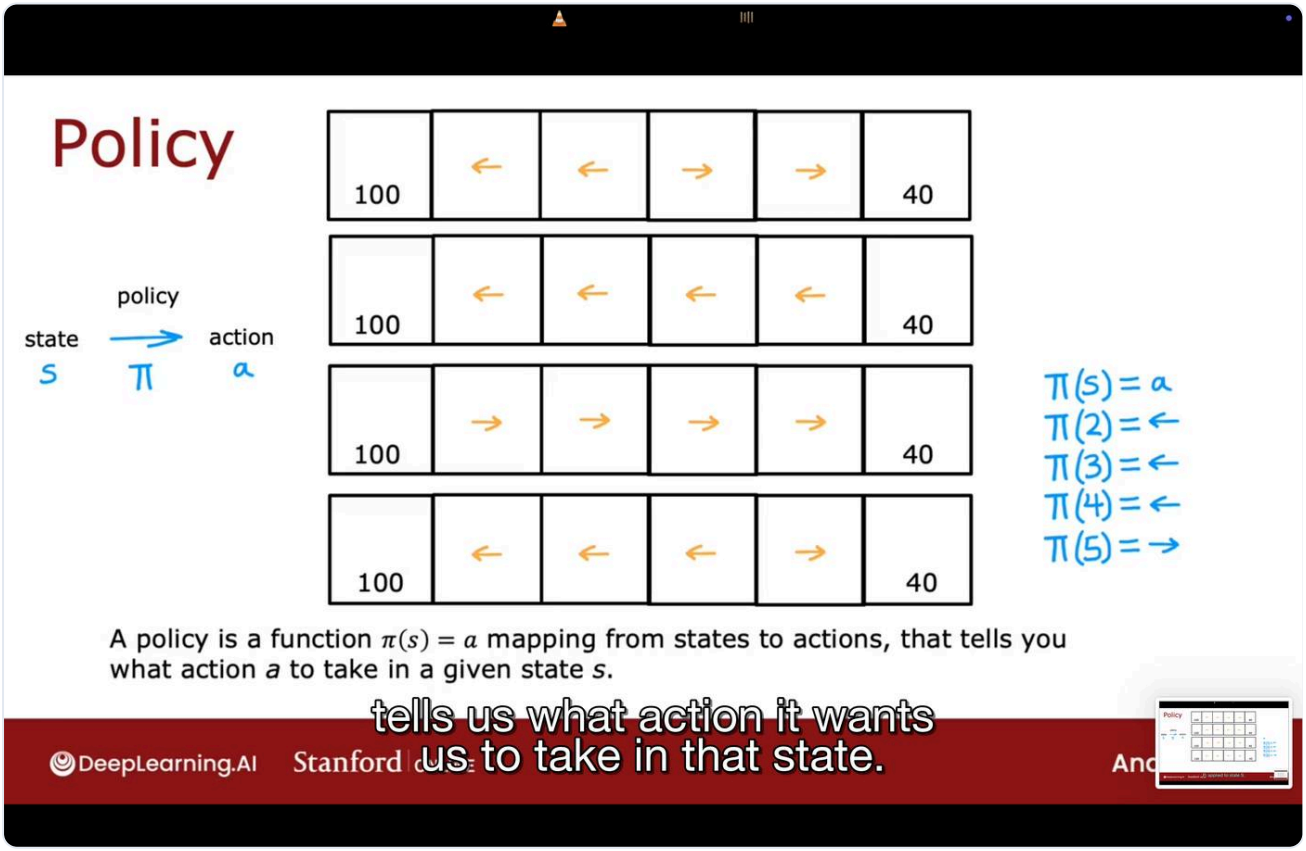


FIGURE 32.8

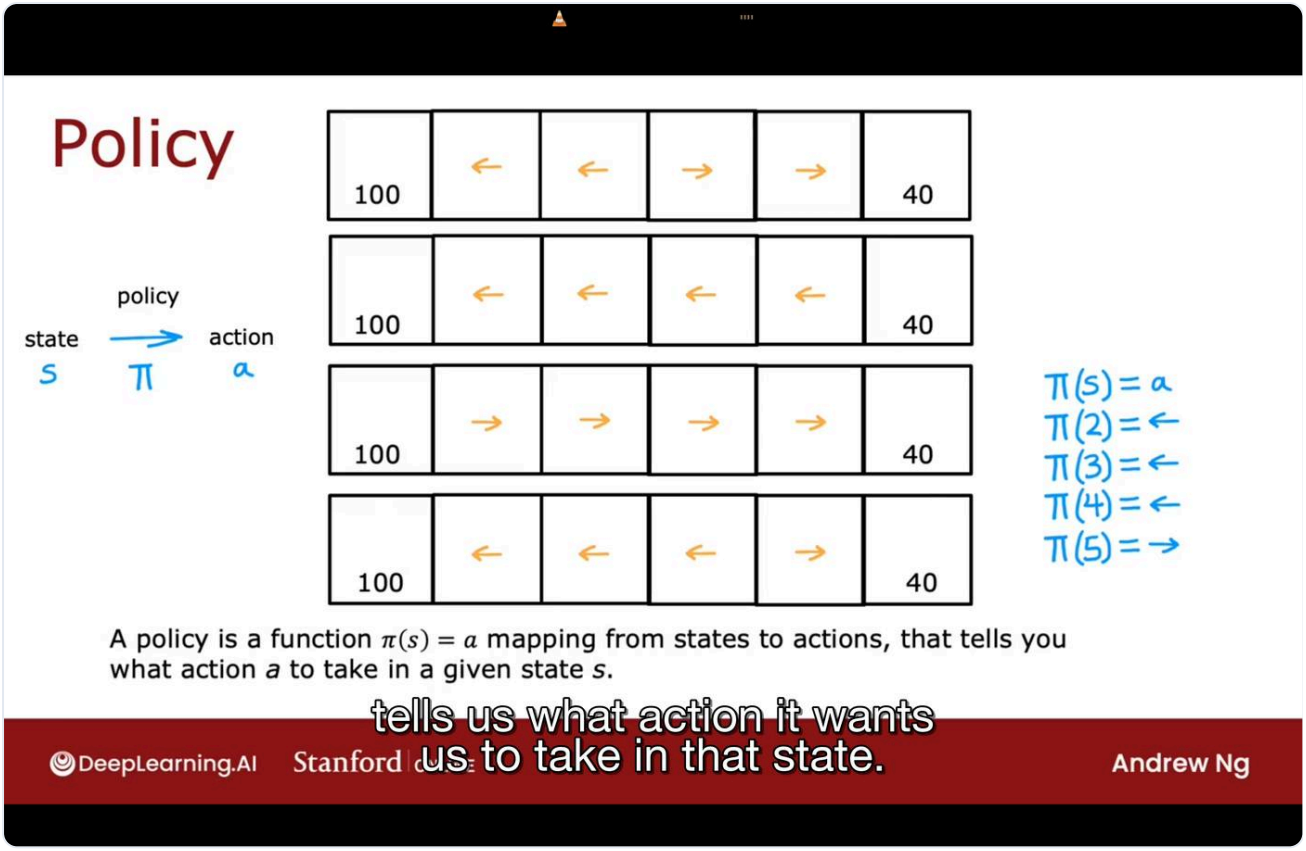


FIGURE 32.9

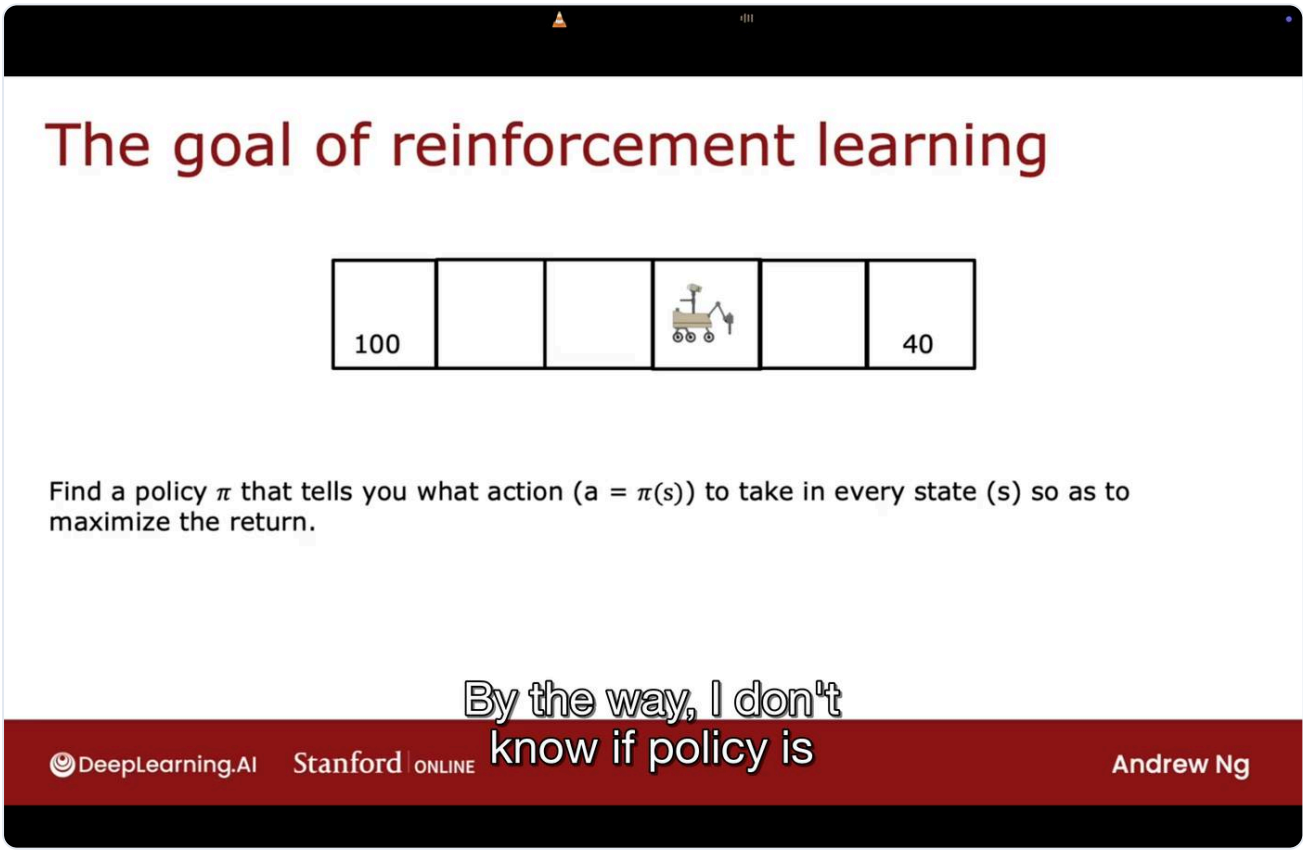


FIGURE 32.10 | The Goal of Reinforcement Learning

The goal of reinforcement learning is to find a policy Pi or Pi of S that tells you what action to take in every state so as to maximize the return. By the way, I don't know if policy is the most descriptive term of what pi is, but it's one of those terms that's become standard in reinforcement learning. Maybe calling Pi a controller rather than a policy would be more natural terminology but policy is what everyone in reinforcement learning now calls this. In the last video, we've gone through quite a few concepts in reinforcement learning from states to actions to reward, to returns, to policies. Let's do a quick review of them in the next video and then we'll go on to start developing algorithms for finding that policies.

Let's go on to the next video.

LECTURE 33

Markov Decision Process (MDP)

We've developed a reinforcement learning formalism using the six state Mars rover example. Let's do a quick review of the key concepts and also see how this set of concepts can be used for other applications as well. Some of the concepts we've discussed are states of a reinforcement learning problem, the set of actions, the rewards, a discount factor, then how rewards and the discount factor altogether use to compute the return, and then finally, a policy whose job it is to help you pick actions so as to maximize the return. For the Mars rover example, we had six states that we numbered 1-6 and the actions were to go left or to go right. The rewards were 100 for the leftmost state, 40 for the rightmost state, and zero in between and I was using a discount factor of 0.5.

The return was given by this formula and we could have different policies Pi depict actions depending on what state you're in. This same formalism or states, actions, rewards, and so on can be used for many other applications as well. Take the problem or find an autonomous helicopter. To set a state would be the set of possible positions and orientations and speeds and so on of the helicopter. The possible actions would be the set of possible ways to move the controls stick of a helicopter, and the rewards may be a plus one if it's flying well, and a negative 1,000 if it doesn't fall really bad or crashes. Reward function that tells you how well the helicopter is flying.

The discount factor, a number slightly less than one maybe say, 0.99 and then based on the rewards and the discount factor, you compute the return using the same formula. The job of a reinforcement learning algorithm would be to find some policy Pi of s so that given as input, the position of the helicopter s, it tells you what action to take. That is, tells you how to move the control sticks. Here's one more example. Here's a game-playing one. Say you want to use reinforcement learning to learn to play chess. The state of this problem would be the position of all the pieces on the board. By the way, if you play chess and know the rules well, I know that's little bit more information than just the position of the pieces is important for chess, but I'll simplify it a little bit for this video.

The actions are the possible legal moves in the game, and then a common choice of reward would be if you give your system a reward of plus one if it wins a game, minus one if it loses the game, and a reward of zero if it ties a game. For chess, usually a discount factor very close to one will be used, so maybe 0.99 or even 0.995 or 0.999 and the return uses the same formula as the other applications.

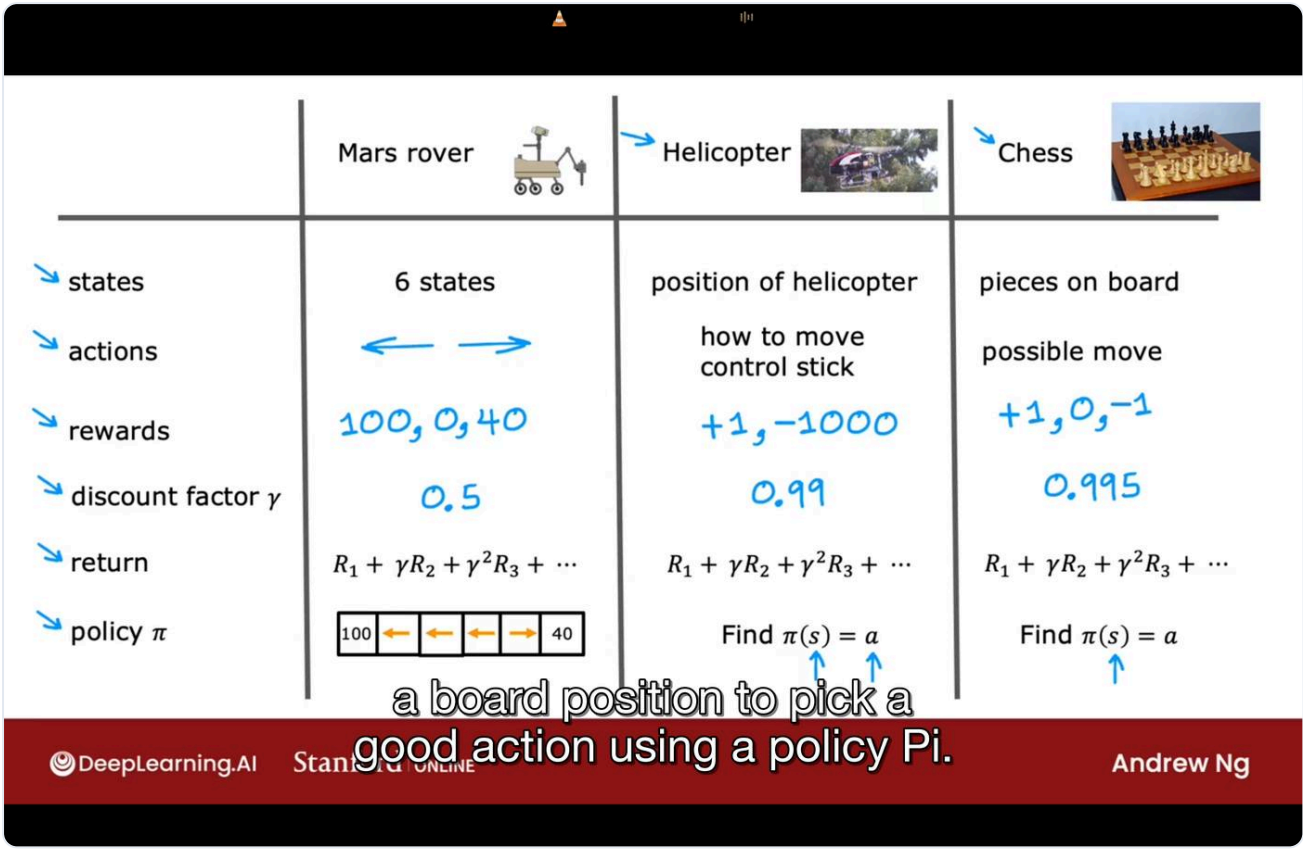


FIGURE 33.1

Once again, the goal is given a board position to pick a good action using a policy Pi. This formalism of a reinforcement learning application actually has a name. It's called a Markov decision process, and I know that sounds like a big technical complicated term. But if you ever hear this term Markov decision process or MDP for short, that's just the formalism that we've been talking about in the last few videos. The term Markov in the MDP or Markov decision process refers to that the future only depends on the current state and not on anything that might have occurred prior to getting to the current state. In other words, in a Markov decision process, the future depends only on where you are now, not on how you got here.

One other way to think of the Markov decision process formalism is that we have a robot or some other agent that we wish to control and what we get to do is choose actions a and based on those actions, something will happen in the world or in the environment, such as our position in the world changes or we get to sample a piece of rock and execute the science mission.

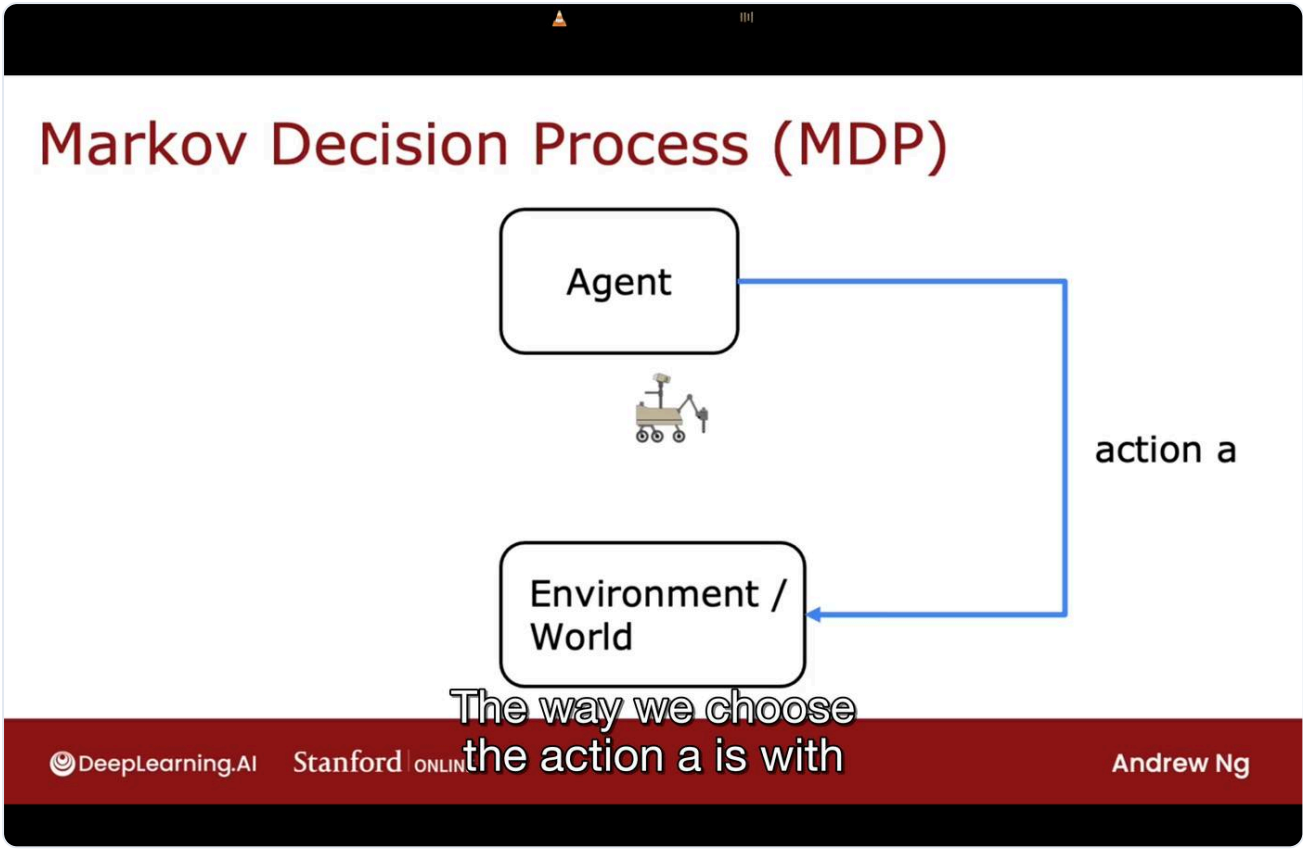


FIGURE 33.2 | Markov Decision Process (MDP)

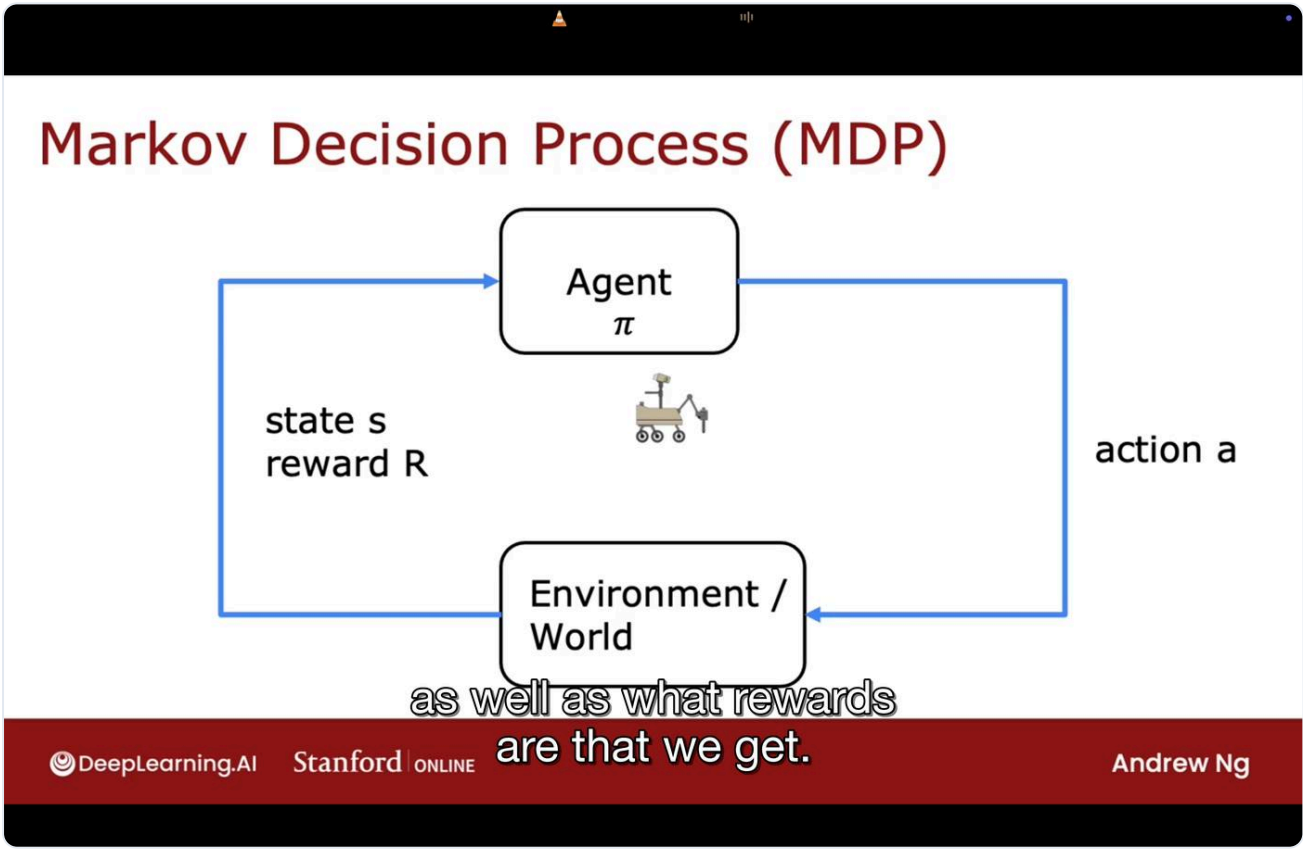


FIGURE 33.3 | Markov Decision Process (MDP)

The way we choose the action a is with a policy Pi and based on what happens in the world, we then get to see or we observe back what state we're in, as well as what rewards are that we get. You sometimes see different authors use a diagram like this to represent the Markov decision process or the MDP formalism but this is just another way of illustrating the set of concepts that you learn about in the last few videos. You now know how a reinforcement learning problem works. In the next video we'll start to develop an algorithm for picking good actions. The first step toward that will be to define and then eventually learn to compute the state action value function.

This turns out to be one of the key quantities for when we want to develop a learning algorithm. Let's go onto the next video to see what is this, state action value function.

LECTURE 34

State Action Value Function

when we start to develop reinforcement learning hours later this week, you see that there's a key quantity that reinforcement learning algorithm will try to compute and that's called the state action value function. Let's take a look at what this function is. The state action value function is a function typically denoted by the letter uppercase Q. And it's a function of a state you might be in as well as the action you might choose to take in that state and Q of s,a. Will give a number that equals the return. If you start in that state. S and take the action A just once and after taking action A once you then behave optimally after that. So after that you take whatever actions will result in the highest possible return.

Now you might be thinking there's something a little bit strange about this definition because how do we know what is the optimal behavior?

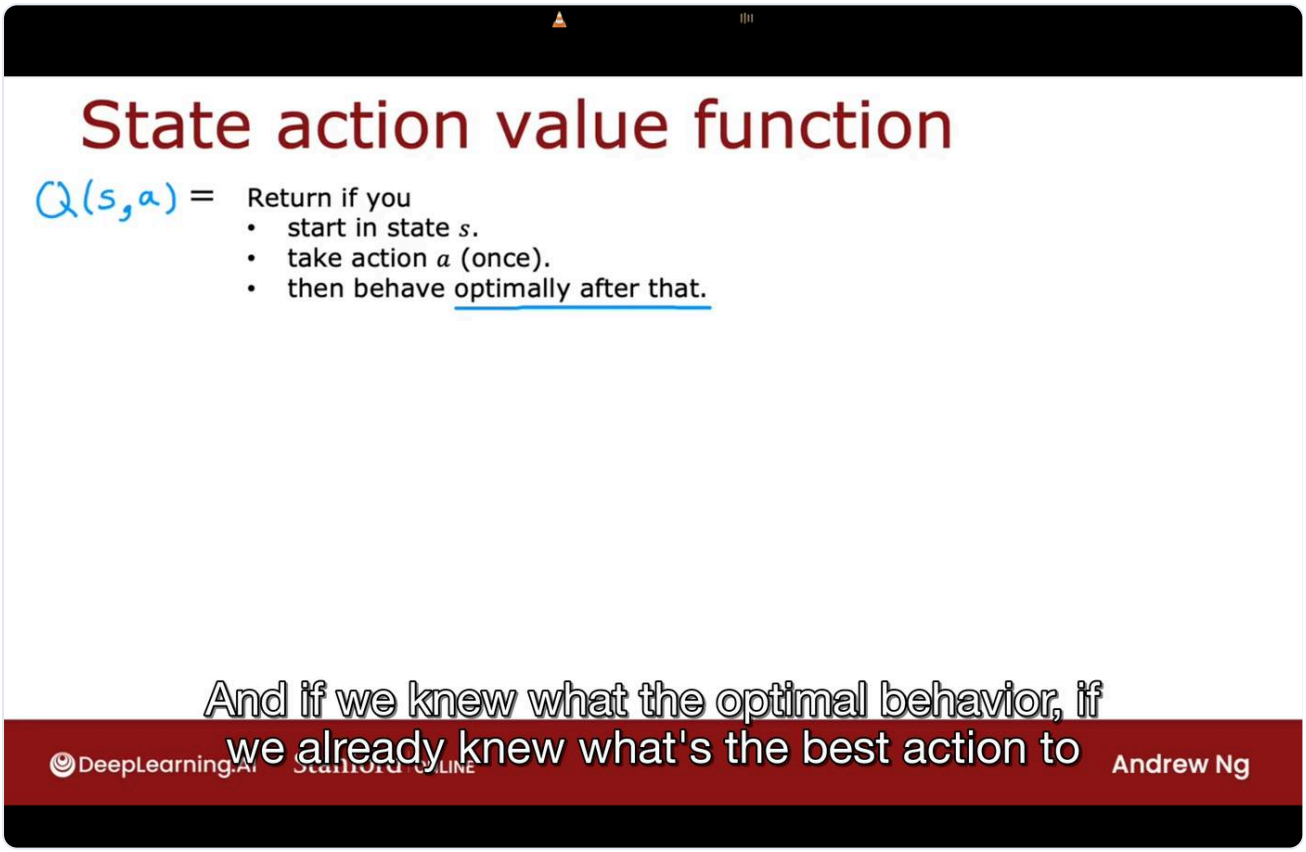


FIGURE 34.1 | State Action Value Function

And if we knew what the optimal behavior, if we already knew what's the best action to take in every state, why do we still need to compute Q of SA. Because we already have the optimal policy. So I do want to acknowledge that there's something a little bit strange about this definition. There's almost something a little bit circular about this definition, but rest assured When we look at specific reinforcement learning algorithms later will resolve this slightly circular definition and will come up with a way to compute the Q function even before we've come up with the optimal policy. But you see that in a later video. So don't worry about this for now.

Let's look at an example we saw previously that this is a pretty good policy Go left from stage 2, 3 and four and go right from State five.

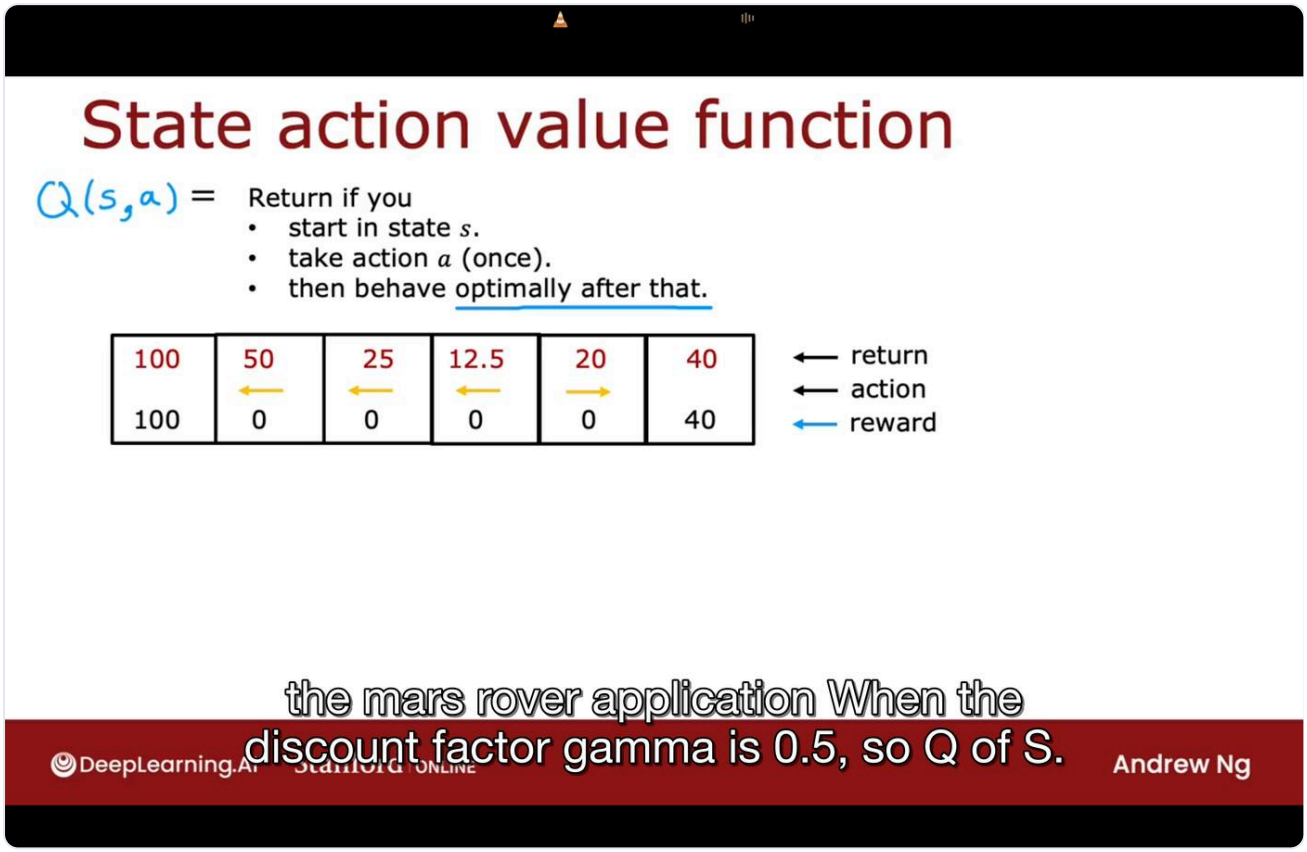


FIGURE 34.2 | State Action Value Function

It turns out that this is actually the optimal policy for the mars rover application When the discount factor gamma is 0.5, so Q of S. A will be equal to the total return If you start from say that take the action A and then behave optimally after that. Meaning take actions according to this policy. Shown over here, let's figure out what Q of s,a. Is for a few different states. Let's look at say Q of state too. And what if we take the action to go right well if you're in state two and you go right then you end up at state three And then after that you behave optimally you're going to go left from ST three and then go left from state to and then eventually you get the reward of 100.

In this case, the rewards you get would be zero from state to zero when you get to stay three zero when you get back to state two and then 100 when you finally get to the terminal state one and so the return will be zero plus 0.5 times that plus 0.5 squared times ac plus 0.5 cubed times 100. And this turns out to be 12.5 And so Q of ST two of going right as equal to 12.5.

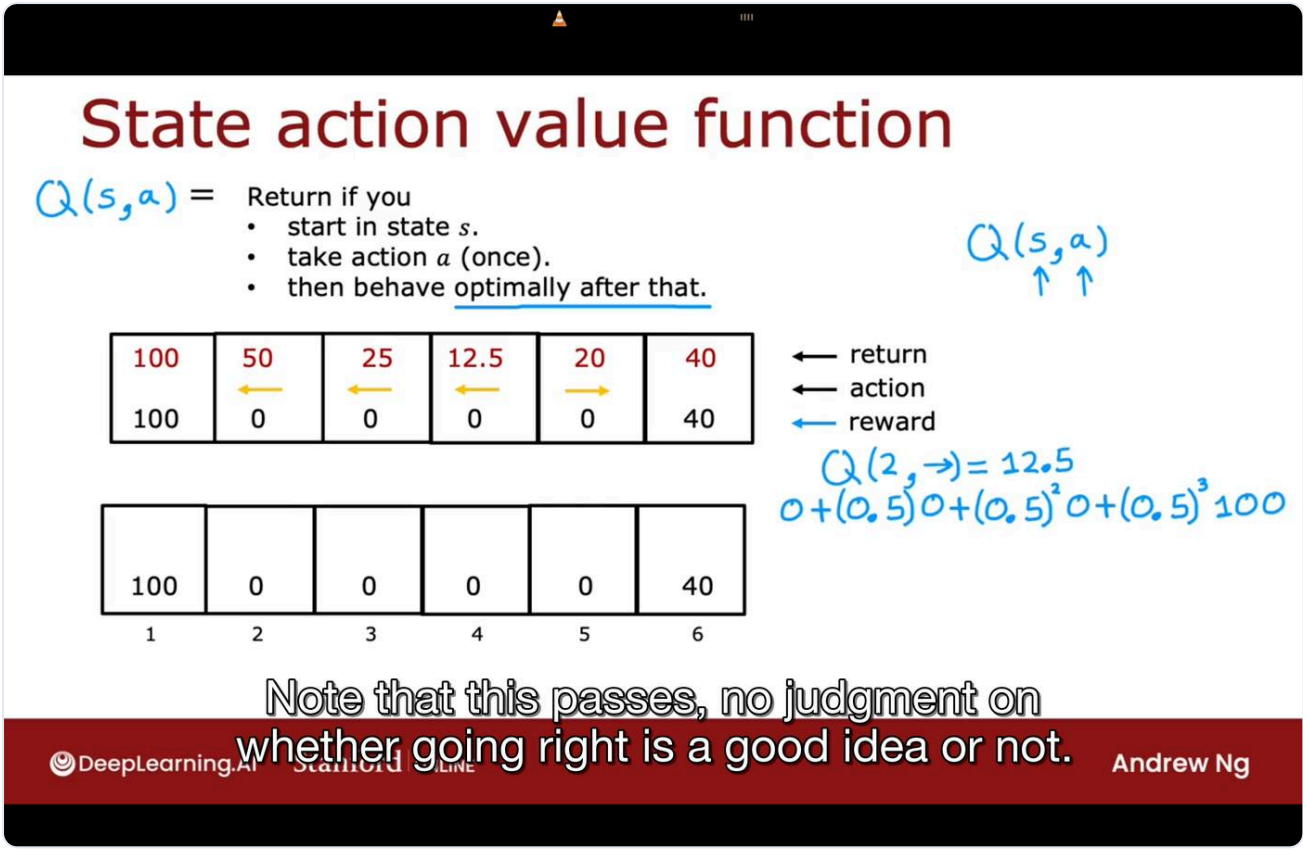


FIGURE 34.3 | State Action Value Function

Note that this passes, no judgment on whether going right is a good idea or not. It's actually not that good an idea from state two to go right, but it just faithfully reports out the return if you take action A and then behave optimally afterwards. Here's another example. If you're in state to and you were to go left, then the sequence of rewards you get will be zero when you're in state two followed by 100. And so the return is zero plus 0.5 times 100 that's equal to 50 in order to write down The values of Q(s,a). In this diagram, I'm going to write 12.5 here on the right to denote that this is Q of state two going to the right.

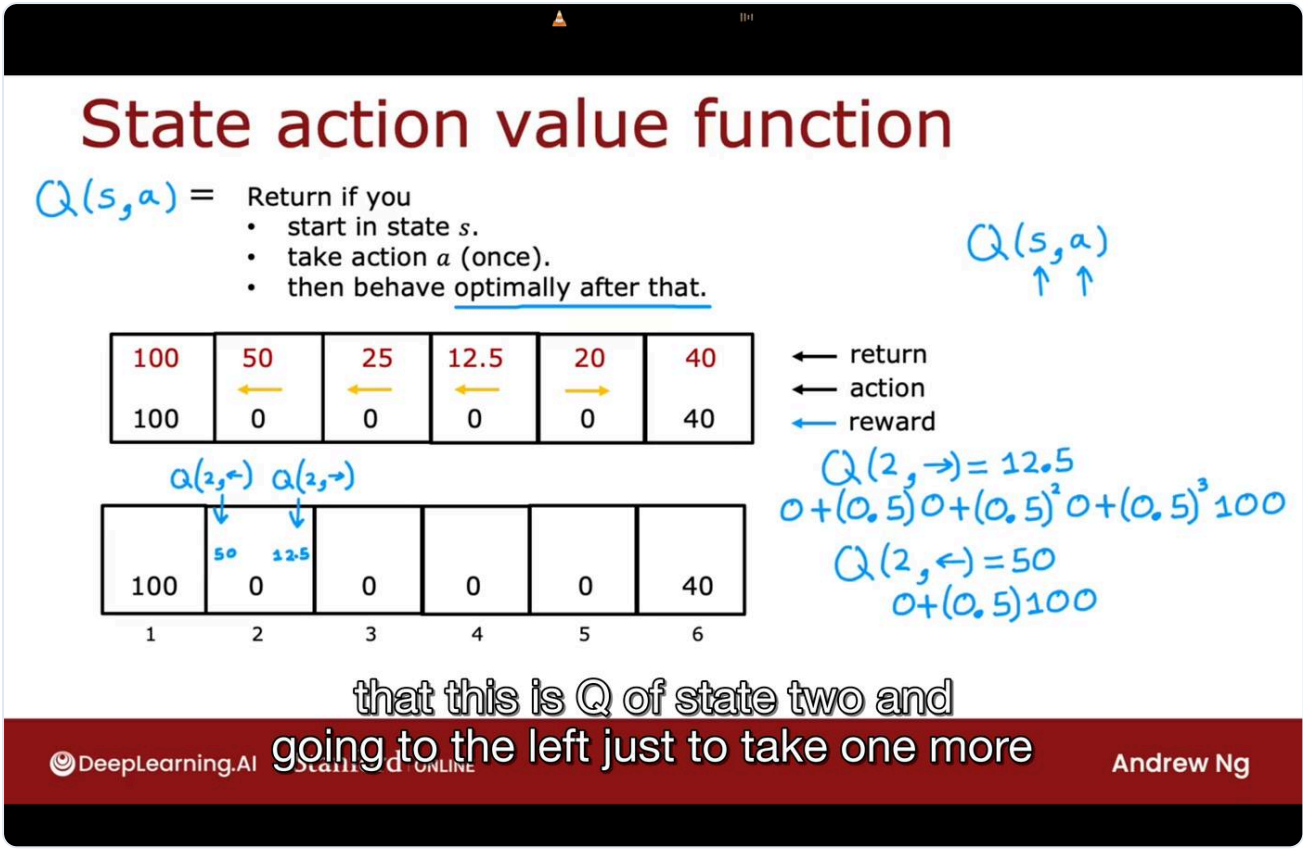


FIGURE 34.4 | State Action Value Function

And then when I write a little 50 here on the left to denote that this is Q of state two and going to the left just to take one more example What if we're in state four and we decide to go left. Well if you're in state four you go left, you get rewards zero and then you take action left here. So zero gain, take action left here, zero and then 100. So Q of four Left results in rewards zero because the first action is left and then because we followed the optimal policy afterwards You can reward 00 100. And so the return is zero plus 00.5 times that. Plus 4.5 squared times that plus 0.5 cubed times that. Which is therefore equal to 12.5.

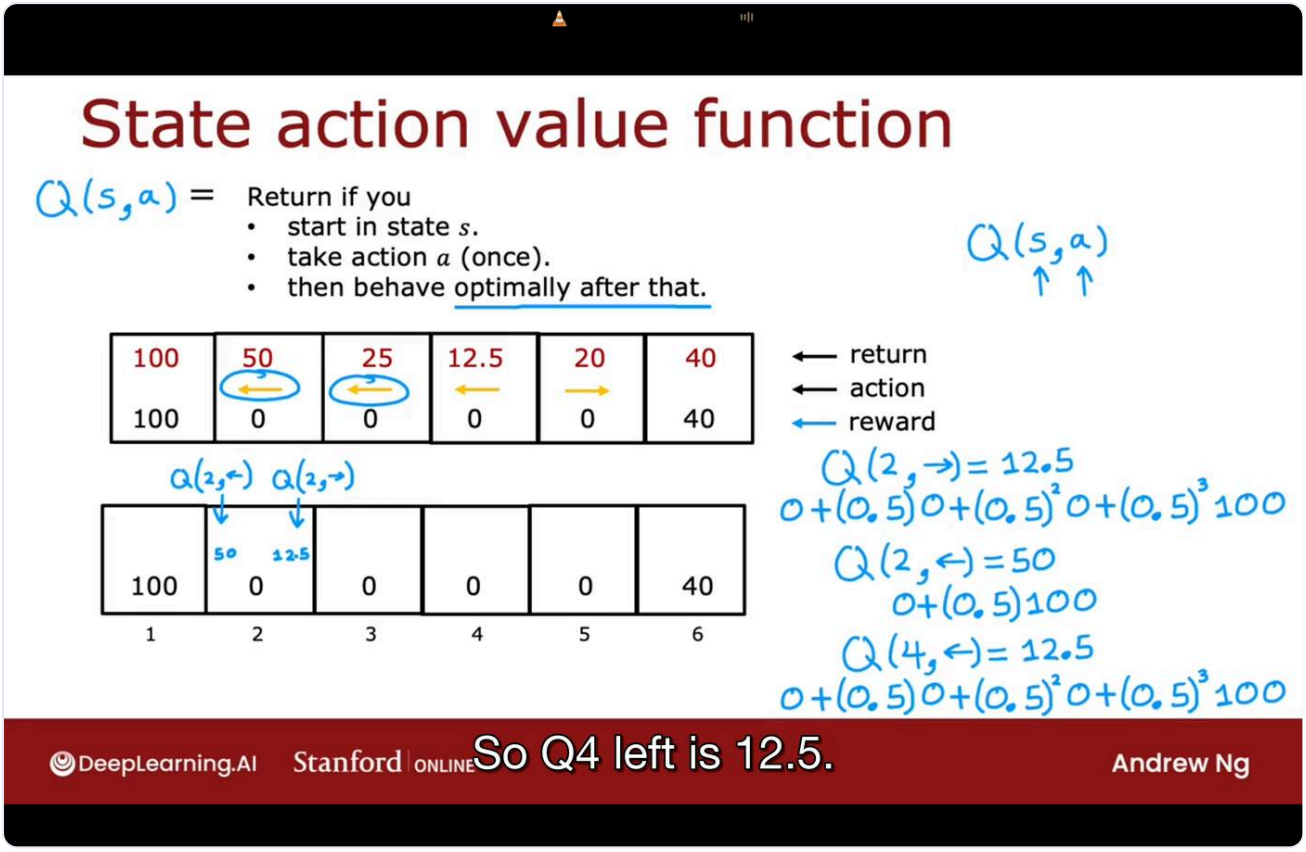


FIGURE 34.5 | State Action Value Function

So Q4 left is 12.5. I'm going to write this here as 12.5. And it turns out if you were to carry out this exercise for all of the other states and all of the other actions, you end up with this being the Q of s,a. For different states and different actions And then finally at the Terminal State.

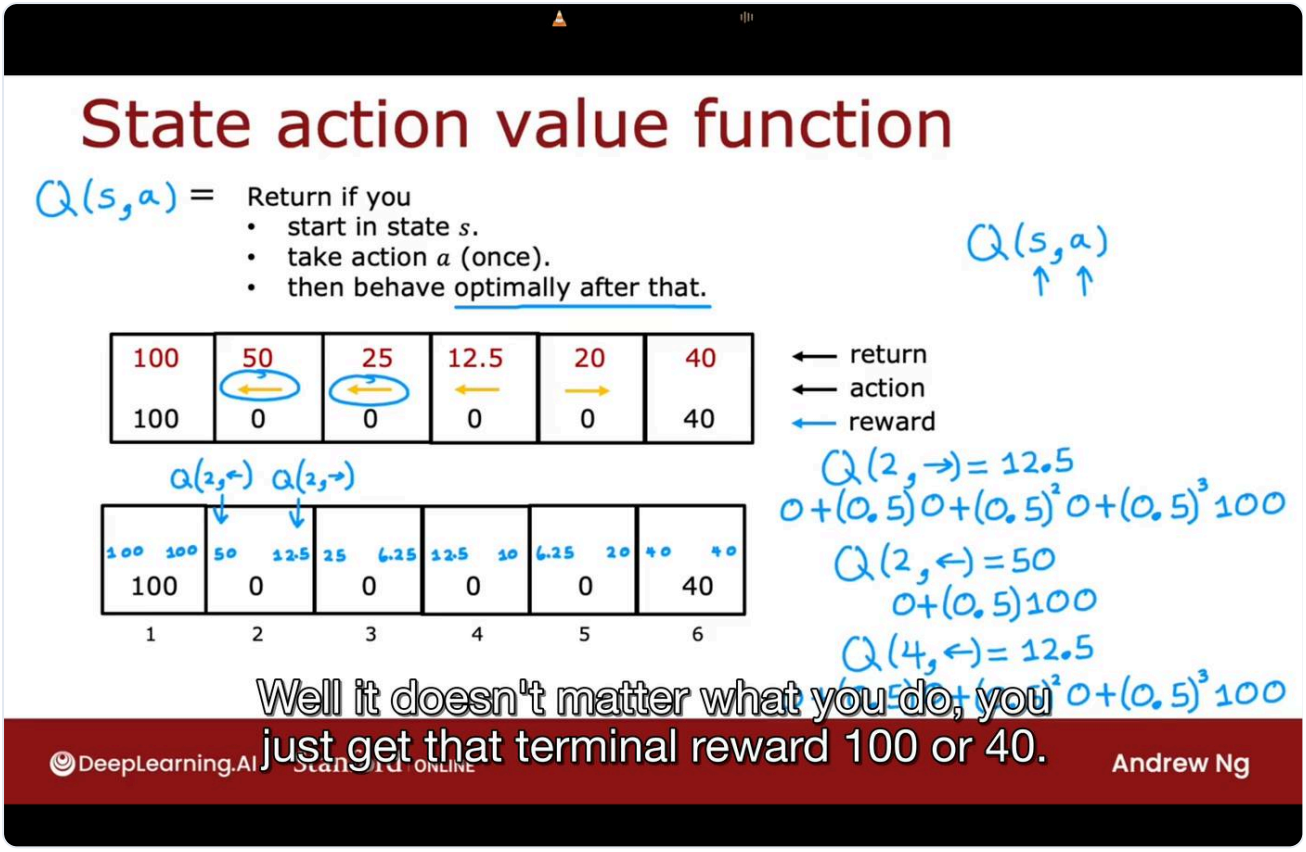


FIGURE 34.6 | State Action Value Function

Well it doesn't matter what you do, you just get that terminal reward 100 or 40. So just write down those terminal awards over here. So this is Q of s,a.

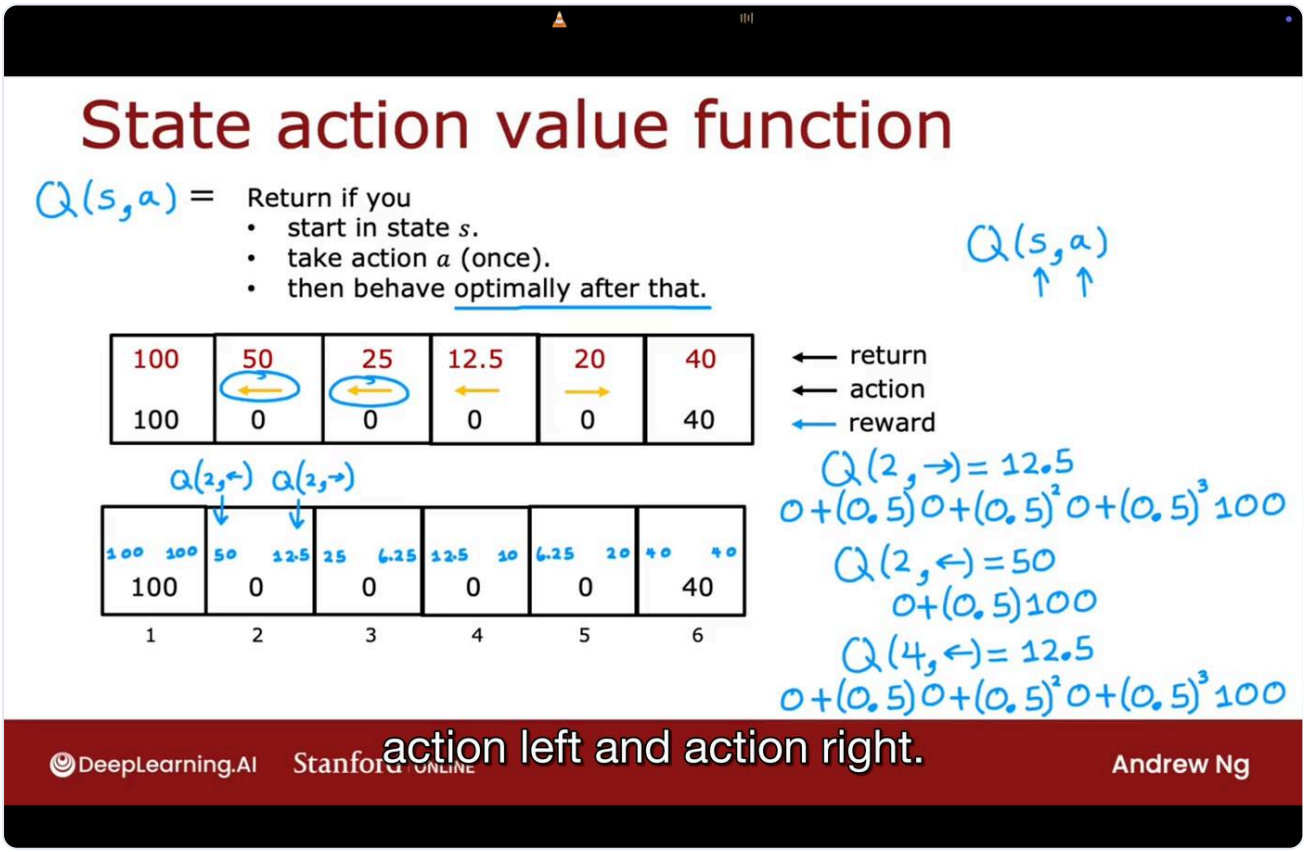


FIGURE 34.7 | State Action Value Function

For every state state one through six and for the two actions, action left and action right. Because the state action value function is almost always denoted by the letter Q. This is also often called the Q function. So the terms Q. Function and state action value function are used interchangeably and it tells you what are your returns or really what is the value? How good is it? Just take action A and state S and then behave optimally after that. Now it turns out that once you can compute the Q function this will give you a way to pick actions as well. Here's the policy and return. And here are the values Q of s,a. From the previous slide. You notice one interesting thing when you look at the different states which is that if you take state two taking the action left results in a,q.

Value or state action value of 50 which is actually the best possible return you can get from that state. In state three two of s,a. for the action left also gives you that higher return in state four the action left gives you the return you want. And in state five is actually the action going to the right that gives you that higher return of 20.

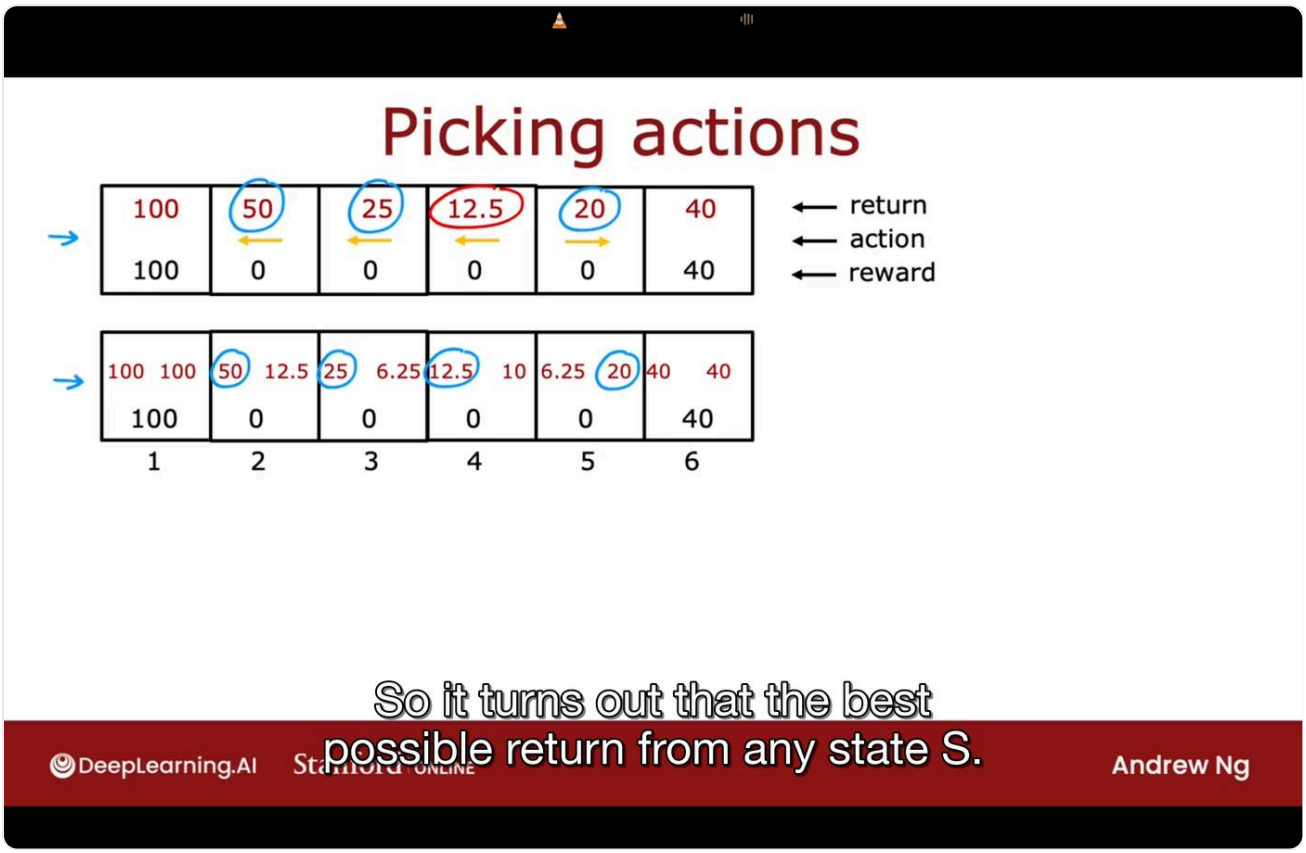


FIGURE 34.8

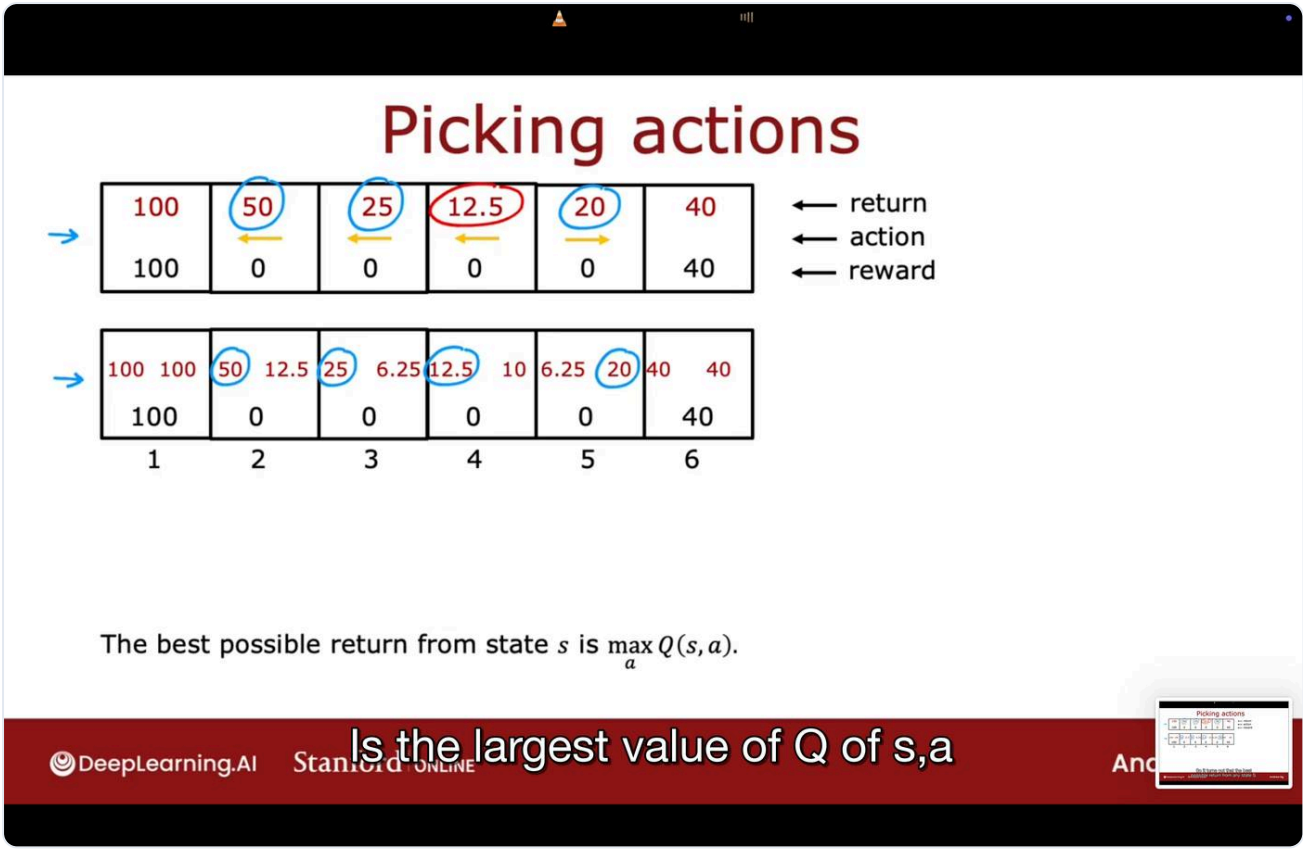


FIGURE 34.9

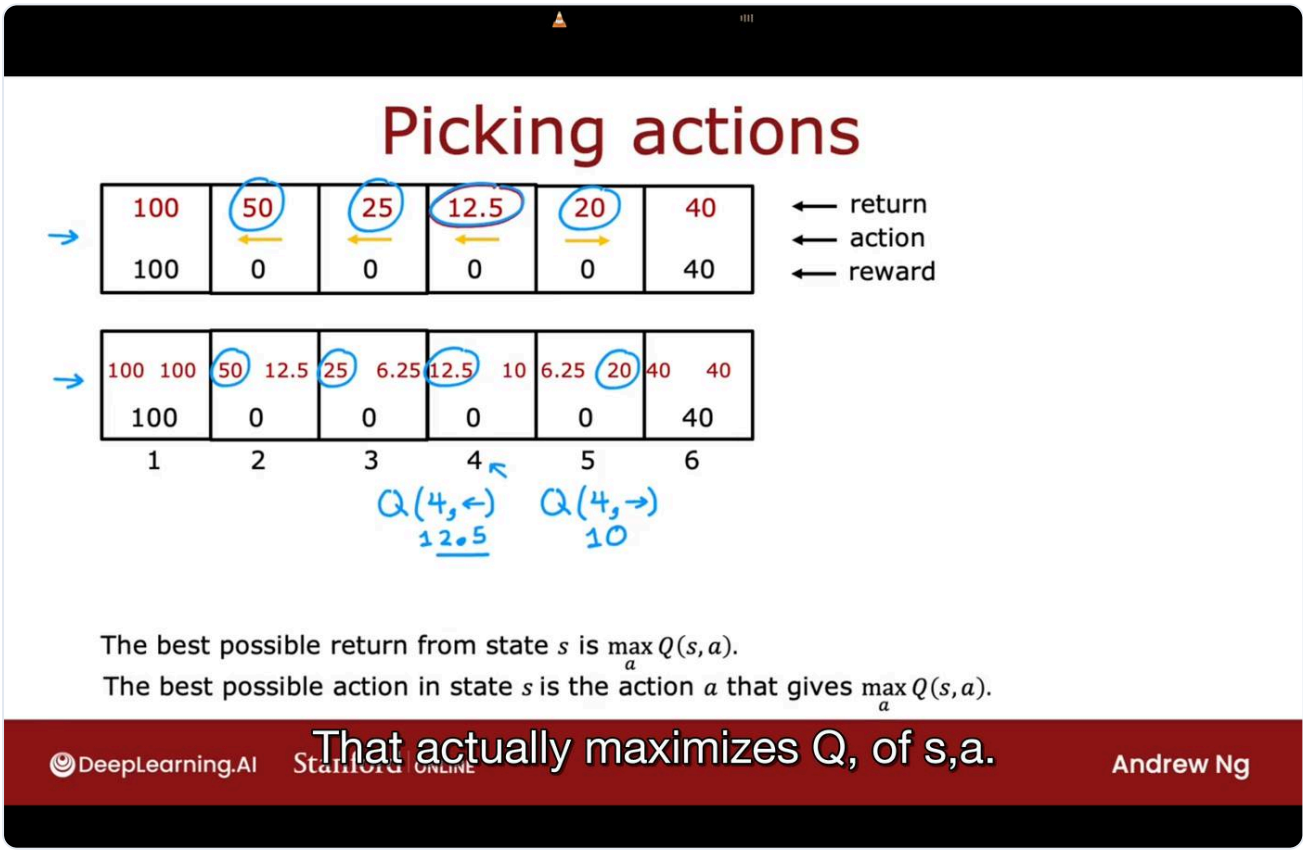


FIGURE 34.10

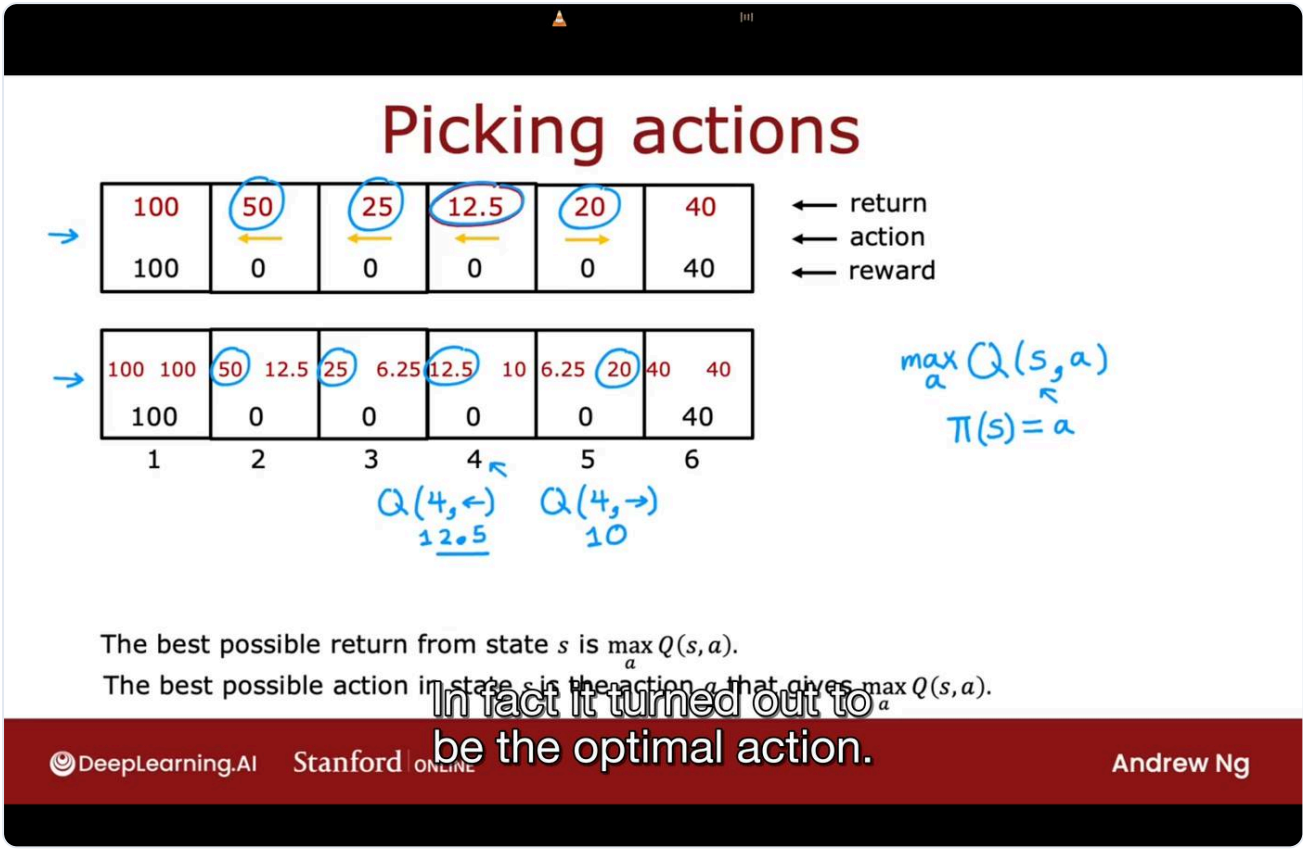


FIGURE 34.11

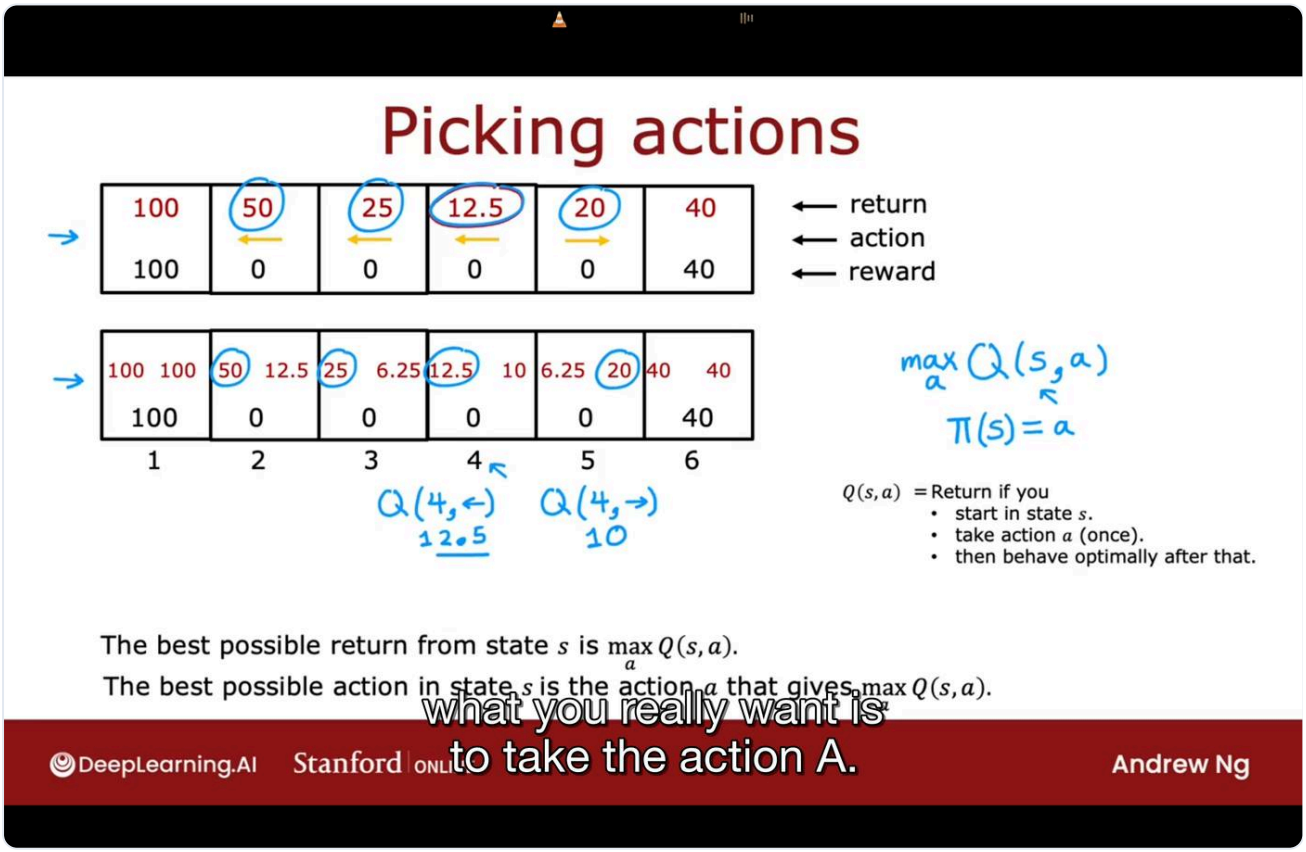


FIGURE 34.12

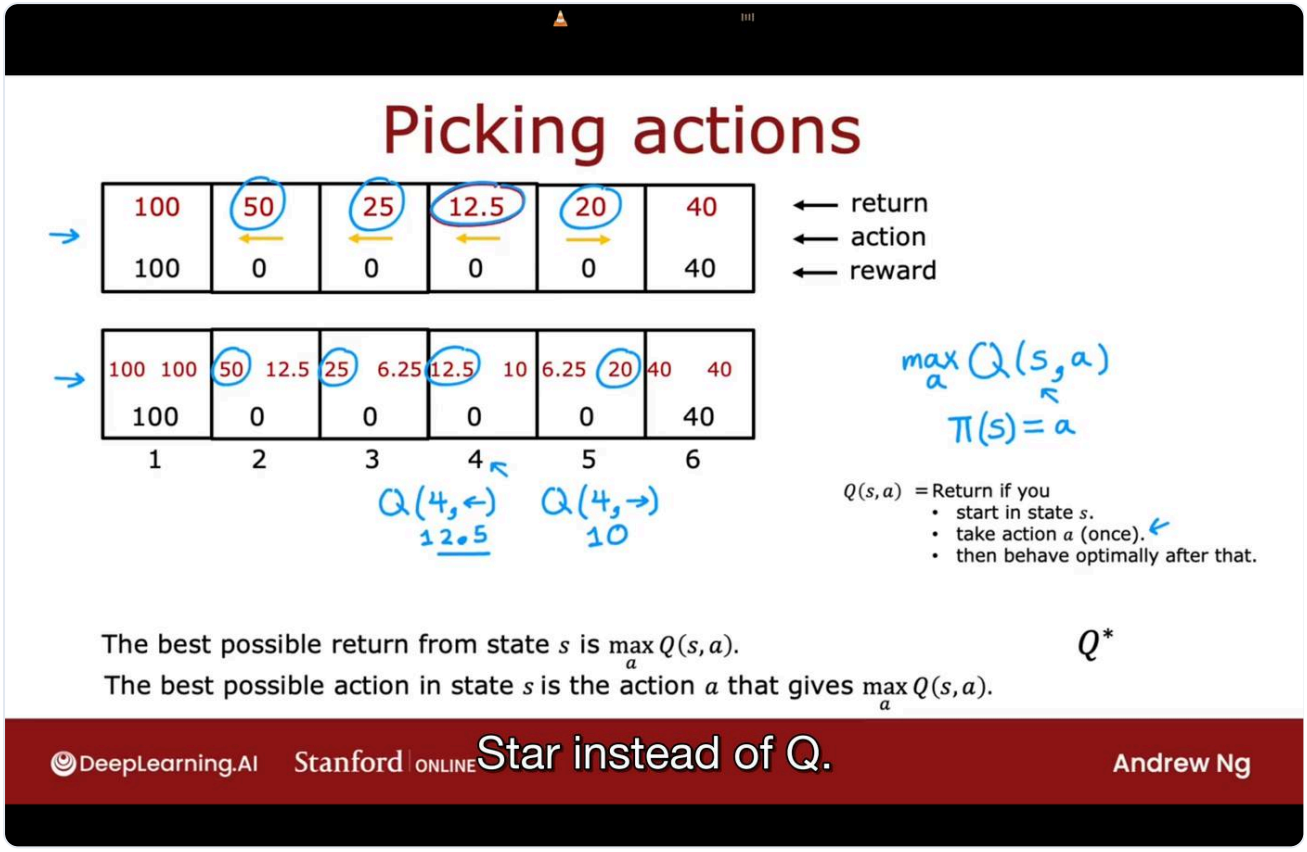


FIGURE 34.13

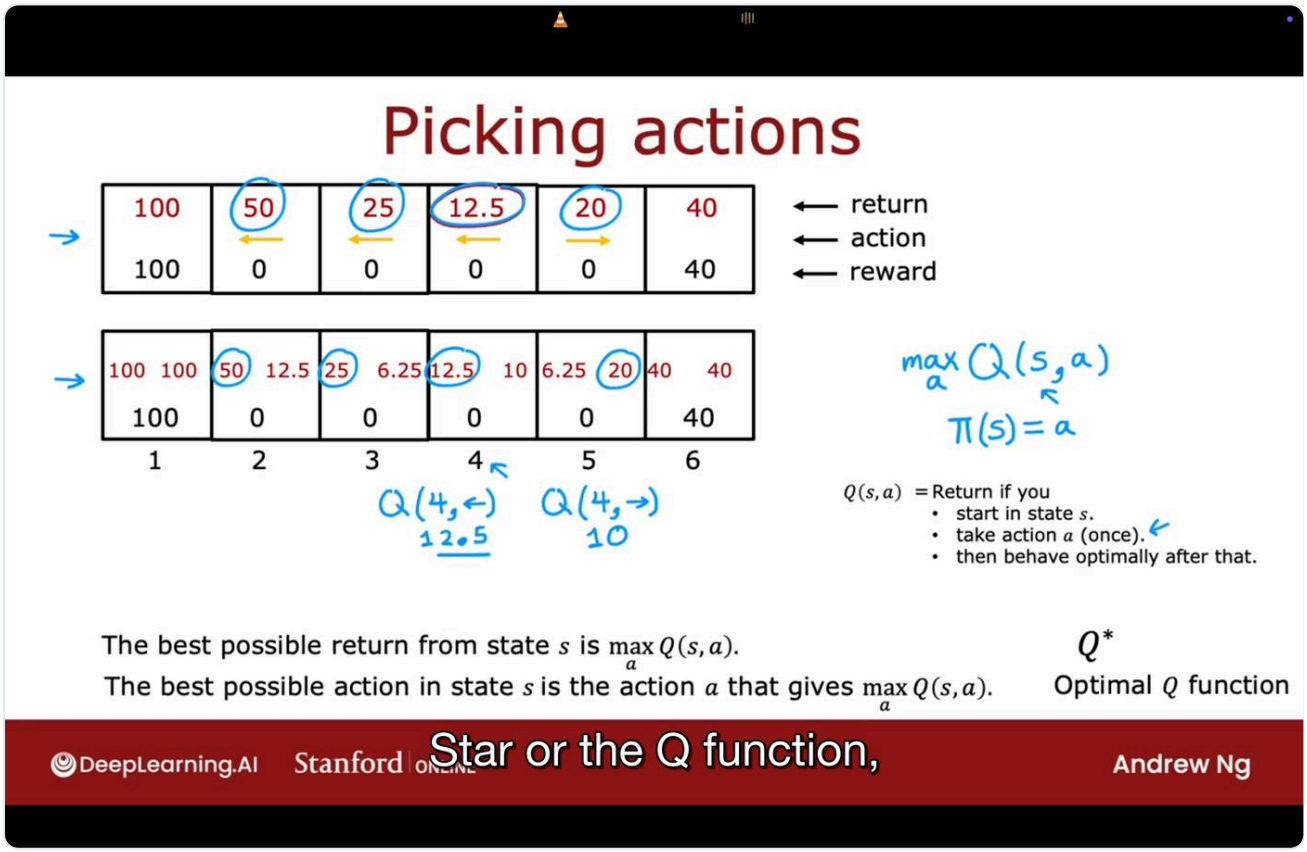


FIGURE 34.14

So it turns out that the best possible return from any state S. Is the largest value of Q of s,a A maximizing over A. Just to make sure this is clear what I'm saying is that in say state for There is Q of state four left which is 12.5 And q. of state four right, Which turns out to be 10. And the larger of these two values which is 12.5 Is the best possible return from that state four. In other words the highest return you can hope to get from State four is 12.5. And it's actually the larger of these two numbers 12.5 and 10. And moreover, if you want your Mars Rover to enjoy a return of 12.5 rather than say 10 then the action you should take is the action A.

That gives you the larger value of Q of s,a. So the best possible action status is the action A. That actually maximizes Q, of s,a. So this might give you a hint for why computing Q, of s,a. Is an important part of the reinforcement learning algorithm that will build later. Namely if you have a way of computing Q of s,a. For every state and for every action then when you're in some state s all you have to do is look at the different actions A. And pick the action A. That maximizes Q of s,a. And so pi of s. S can just pick the action A. That gives the largest value of Q of s,a. And that will turn out to be a good action. In fact it turned out to be the optimal action.

Another intuition about why this makes sense is Qof s,a. Is returned if you start in the state S and take the action A. And then behave optimally after that. So in order to earn the biggest possible return, what you really want is to take the action A. That results in the biggest total return. That's why if only we have a way of computing Q f s,a. For every state taking the action A that maximizes return under these circumstances seems like it's the best action to take in that state. Although this isn't something you need to know for this course, I want to mention also that if you look online or look at the reinforcement learning literature, sometimes you also see this Q function written as Q.

Star instead of Q. And this Q function is sometimes also called the optimal Q function. These terms just refer to the Q function exactly as we've defined it. So if you look at the reinforcement learning literature and read about Q. Star or the Q function, that just means the state action value function that we've been talking about. But for the purposes of this course you don't need to worry about this. So to summarize if you can compute Q of s,a. For every state and every action, then that gives us a good way to compute the optimal policy pi of S. So that's the state action value function or the Q function. We'll talk later about how to come up with an algorithm to compute them despite the slightly circular aspect of the definition of the Q function.

But first let's take a look at the next video at some specific examples of what these values Q of s,a. Actually look like

LECTURE 35

State Action Value Function Example

Using the state-action value function example. You're seeing what the values of QSA are like. In order to keep holding our intuition about reinforcement learning problems and how the values of QSA change depending on the problem will provided an optional lab. That lets you play around modify the mars rover example and see for yourself how QSA will change. Let's take a look. Here's a Jupyter Notebook that I hope you play with after watching this video.

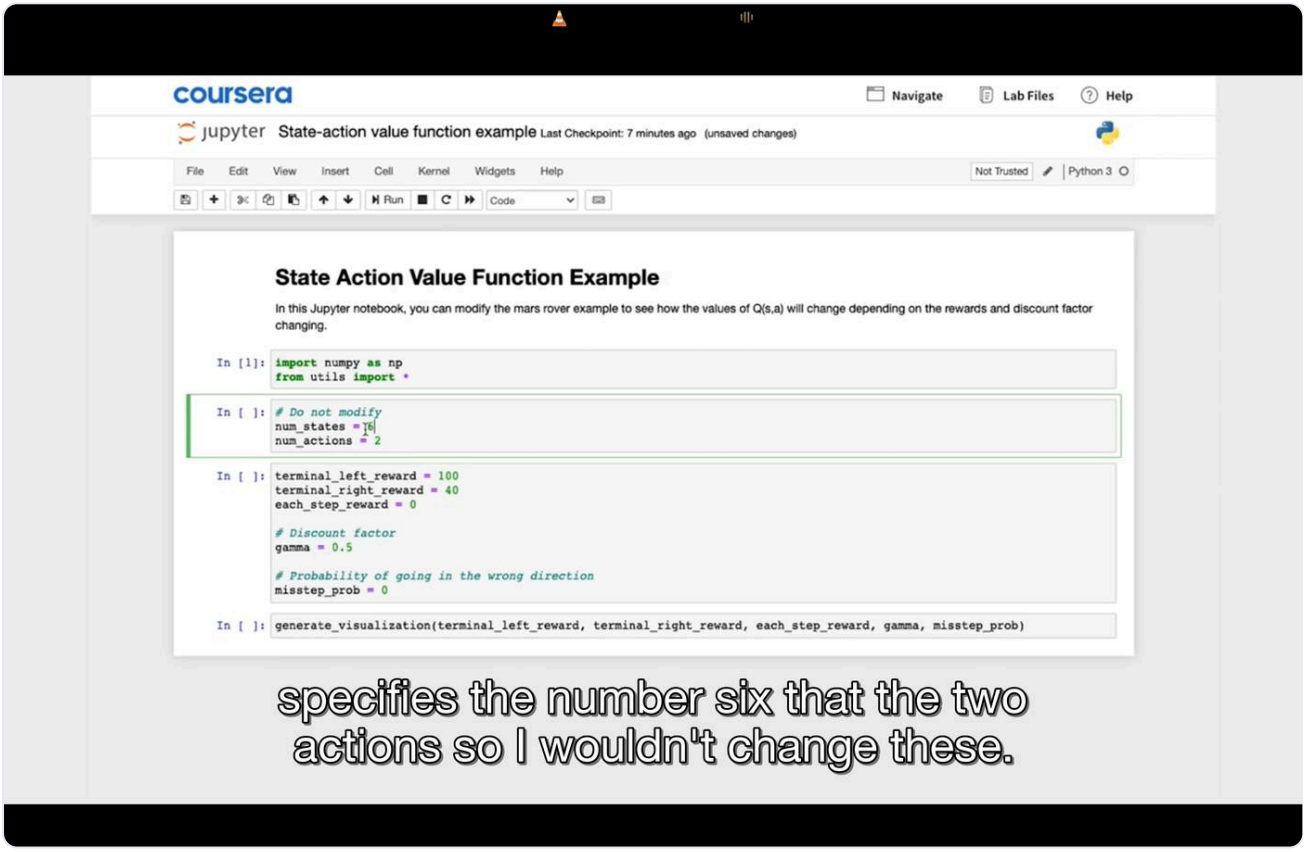


FIGURE 35.1

I'm going to run these helper functions, now notice here that this specifies the number six that the two actions so I wouldn't change these. And this specifies the terminal left in the terminal right rewards which has been 100 and 40 and then zero was the rewards of the intermediate states. The discount factor gamma 0.5. And let's ignore the misstep probability for now we'll talk about that in a later video. And with these values if you run this code this will compute and visualize the optimal policy as well as the Q function Q of SA. You learn later about how to develop a learning algorithm to estimate or compute Q of SA yourself. So for now don't worry about what code we have written to compute Q of SA.

But you see that the values here Q of SA are the values we saw in the lecture. Now here's where the fun starts. Let's change around some of the values and see how these things change. I'm going to update the terminal right reward to a much smaller value says only 10. If I now rerun the code, look at how Q of SA changes and now thinks that if you're in state 5. Then if you go left and be-

have optimally you get 6.25. Whereas if you go right and behave also the after that you get a return of only five. So now when the reward at the right is so small it's only 10. Even when you're so close to you rather go left all the way. And in fact the auto policy is now to go left from every single state.

Let's make some other changes. I'm going to change the terminal right reward back to 40. But let me change the discount factor to 0.9, with a discount factor that's closer to one. This makes the Mars Rover less impatient is willing to take longer to hold out for a higher reward because rewards in the future are not multiplied by 0.5 to some high power is multiplied by 0.9 to some high power. And so is willing to be more patients, because rewards in the future are not discounted or multiplied by as small a number as when the discount was 0.5. So let's rerun the code. And now you see this is Q of SA for the different states and now for state 5 going left actually gives you a higher reward of 65.61 compared to 36.

Notice by the way that 36 is 0.9 times this terminal reward of 40. So these numbers make sense. But when a small patient is willing to go to the left, even when you're in state 5. Now let's change gamma to a much smaller number like 0 .3. So this very heavily discounts rewards in the future. This makes it incredibly impatient. So let me rerun this code and now the behavior has changed. Noticed that now in state 4 is not going to have the patience to go for the larger 100 reward, because the discount factor gamma is now so small is 0.3. It would rather go for the reward of 40 even though it's a much smaller reward is closer and that's whether we choose to do.

So I hope that you can get a sense by playing around with these numbers yourself and running this code. How the values of Q of SA change as was how the optimal return which you notice is the larger of these two numbers QSA. How that changes as well as how the optimal policy also changes. So I hope you go and play with the optional lab and change the reward function and change the discount factor gamma and try a different values. And see for yourself how the values of Q of SA change, how the optimal return from different states change and how the auto policy changes depending on these different values.

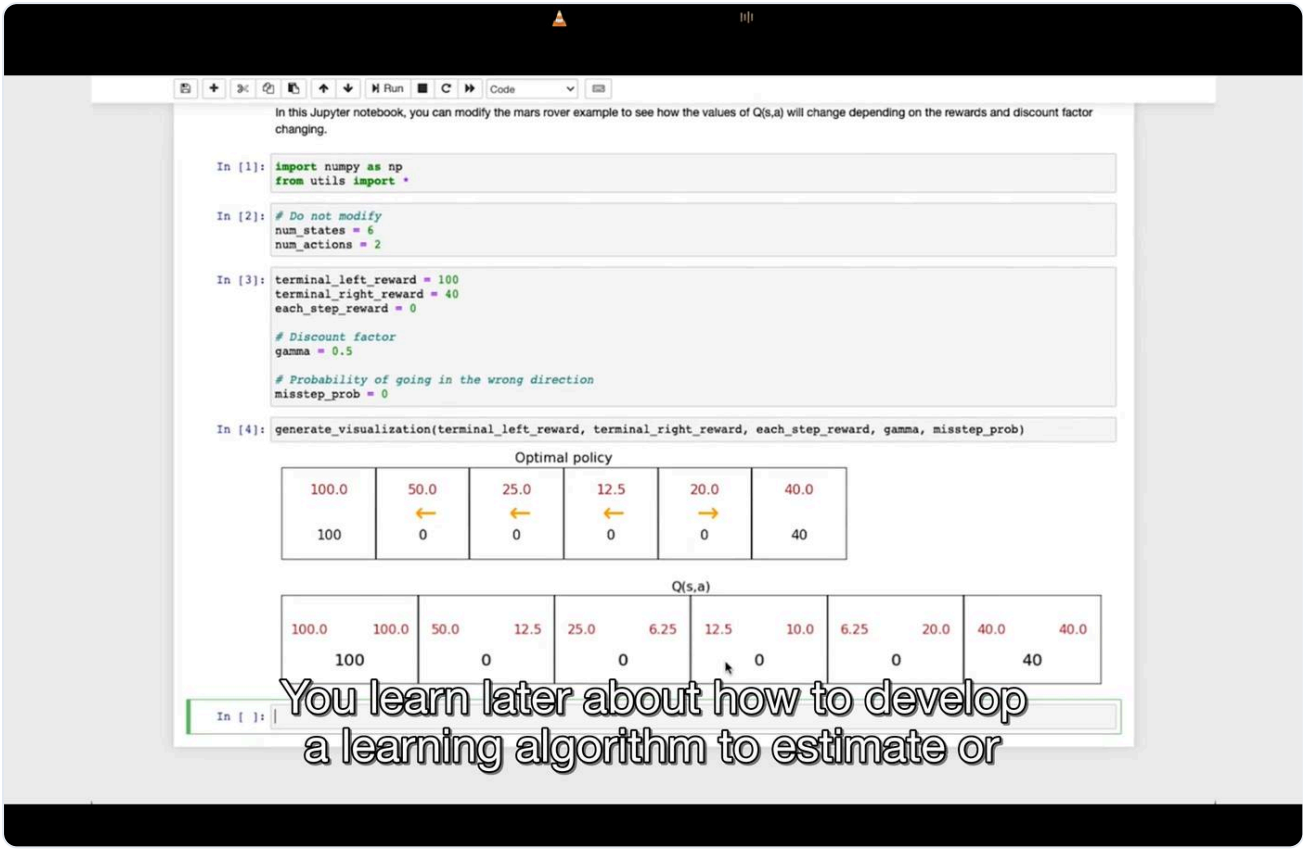


FIGURE 35.2

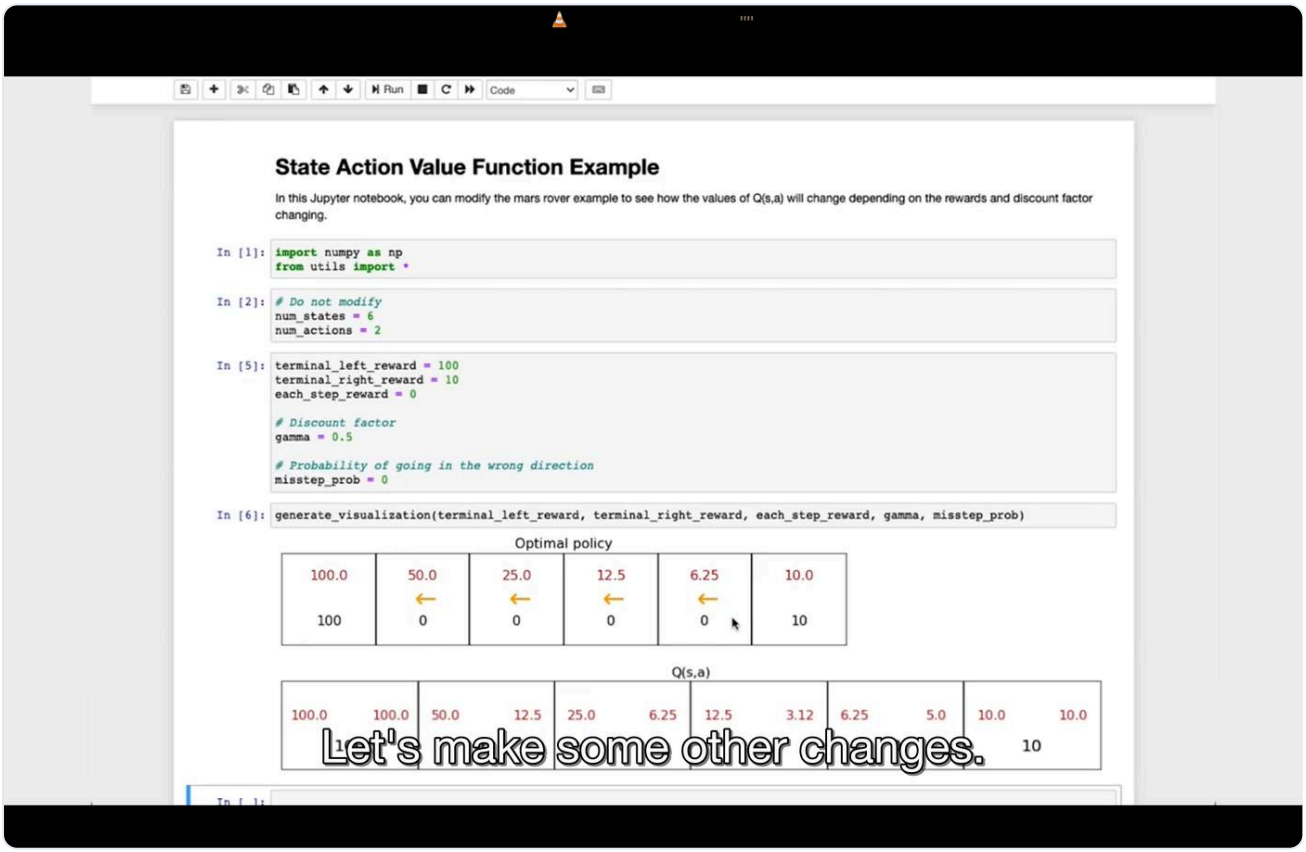


FIGURE 35.3 | State Action Value Function Example

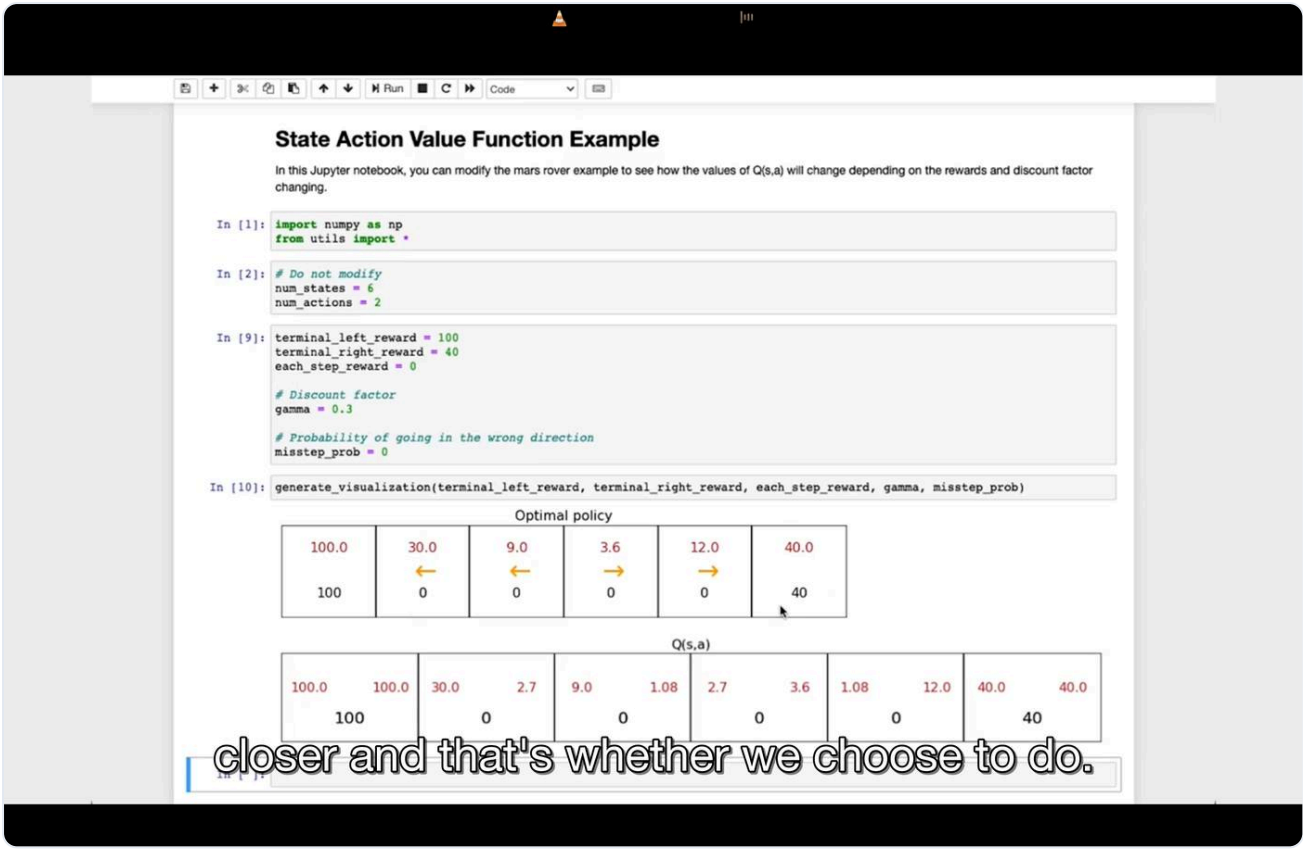


FIGURE 35.4 | State Action Value Function Example

And by doing so, I hope that will sharpen your intuition about how these different quantities are affected depending on the rewards and so on in reinforcement learning application. After you play to the lab, we then be ready to come back and talk about what's probably the single most important equation in reinforcement learning, which is something called the bellman equation. So I hope you have fun playing with the optional lab and after that let's come back to talk about bellman equations.

LECTURE 36

Bellman Equation

Let me summarize where we are. If you can compute the state action value function Q of S, A, then it gives you a way to pick a good action from every scene. Just pick the action A, that gives you the largest value of Q of S,A. The question is, how do you compute these values Q of S,A? In reinforcement learning, there's a key equation called the Bellman equation that will help us to compute the state action value function. Let's take a look at what is this equation. As a reminder, this is the definition of Q of S, A.

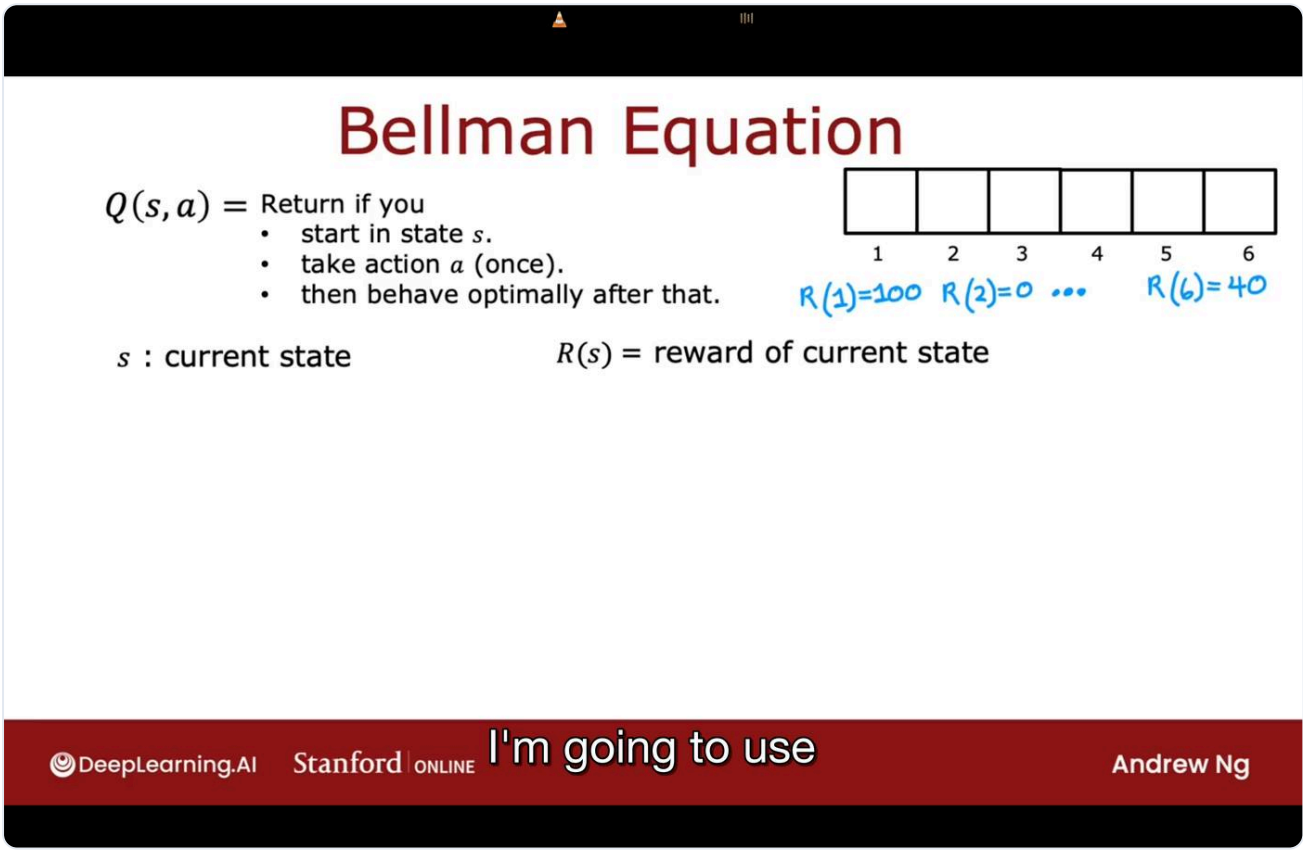


FIGURE 36.1 | Bellman Equation

It's return if we start in state S, take the action a once and they behave optimally after that. In order to describe the Bellman equation, I'm going to use the following notation. I'm going to use S to denote the current state. Next, I'm going to use R of S to denote the rewards of the current state. For our little MDP example, we will have that r of one State1 is 100. The reward of State 2 is 0, and so on. The reward of State 6 is 40. I'm going to use the alphabet A to denote the current action, the action that you take in the state S. After you take the action a, you get to some new state. For example, if you're in State 4 and you take the action left, then you get to State 3.

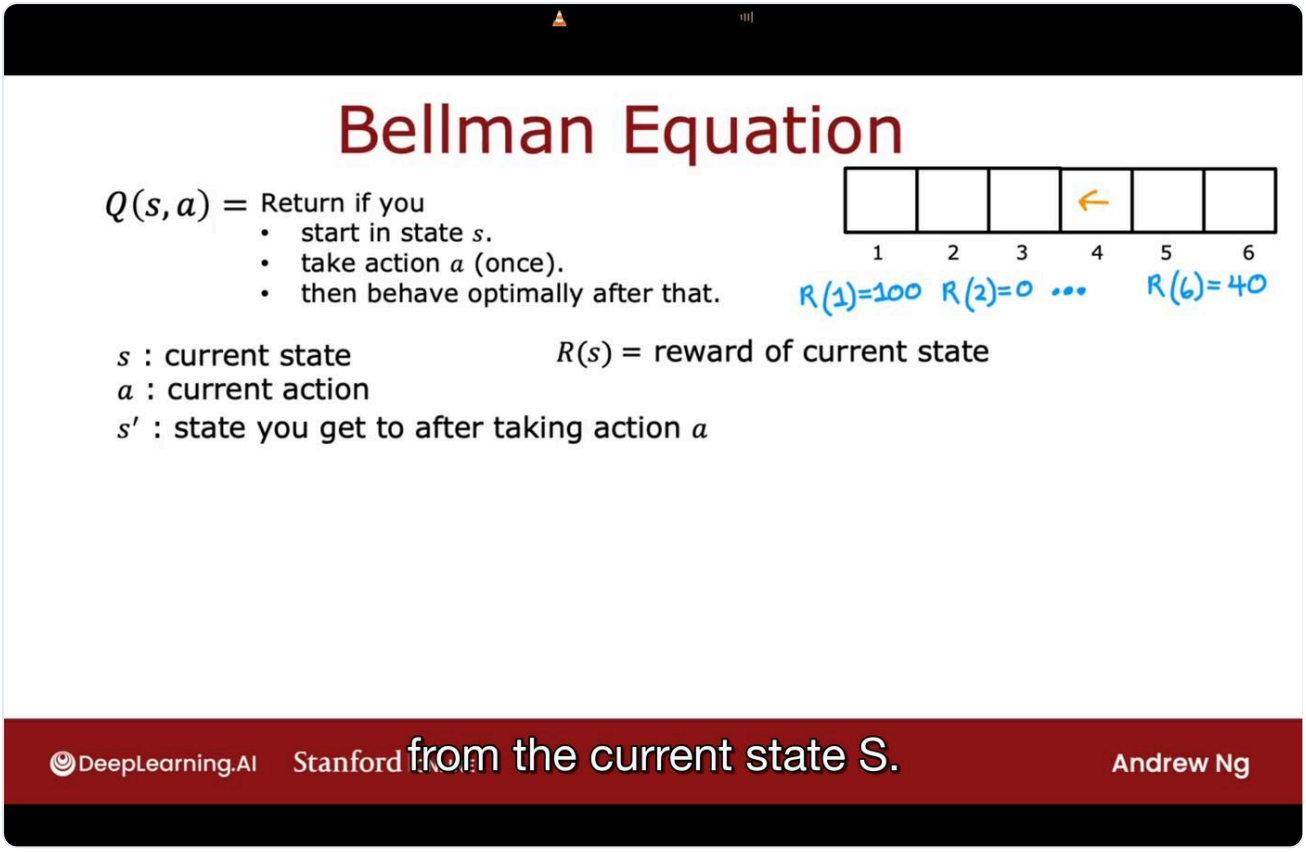


FIGURE 36.2 | Bellman Equation

I'm going to use S prime to denote the state you get to after taking that action a from the current state S. I'm also going to use A prime to denote the action that you might take in state S prime, the new state you got to.

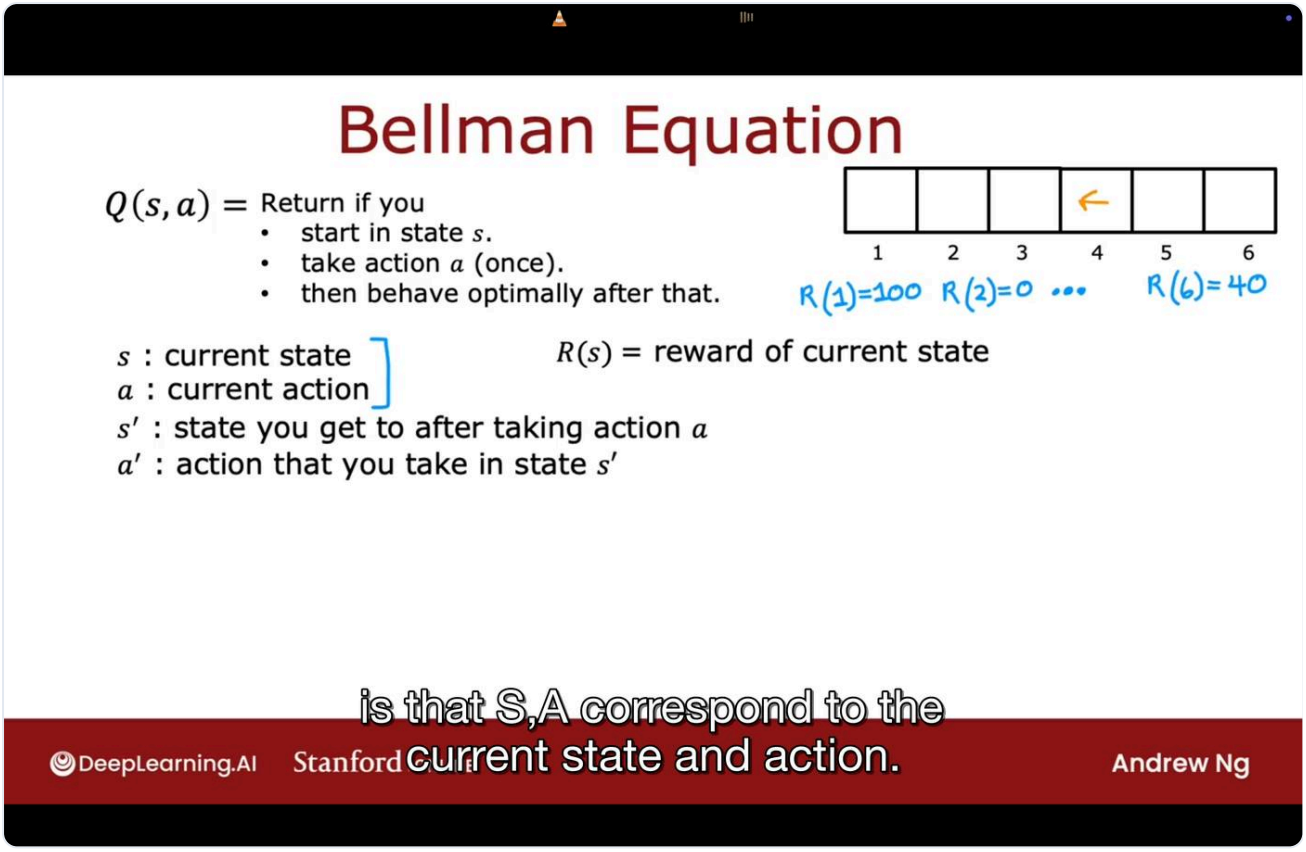


FIGURE 36.3 | Bellman Equation

The notation convention, by the way, is that S,A correspond to the current state and action.

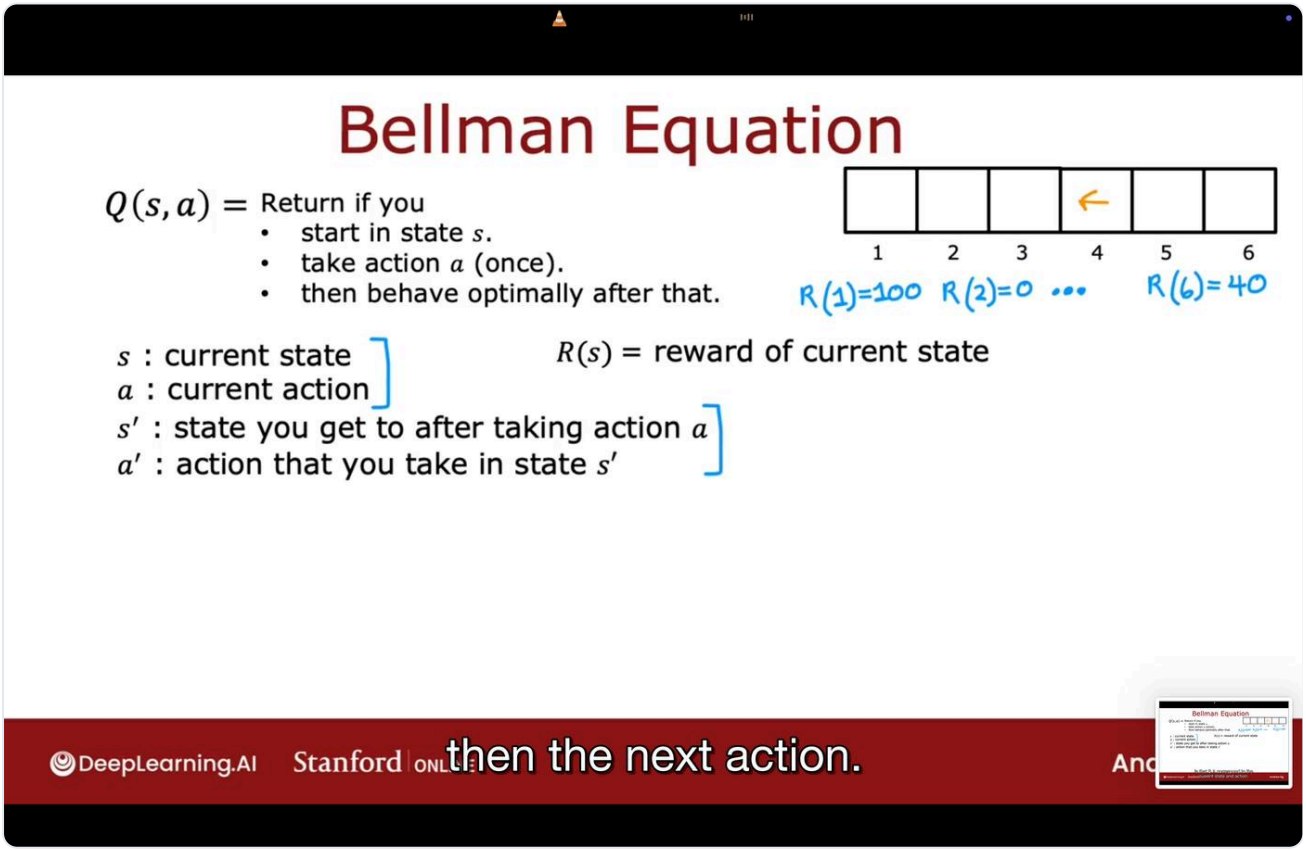


FIGURE 36.4 | Bellman Equation

When we add the prime, that's the next state, then the next action. The Bellman equation is the following. It says that Q of S,A, that is the return under this set of assumptions that's equal to r of S, says reward you get for being in that state plus the discount factor Gamma times max over all possible actions, a prime of q of S prime, the new state you just got to, and then a prime. There's a lot going on in this equation.

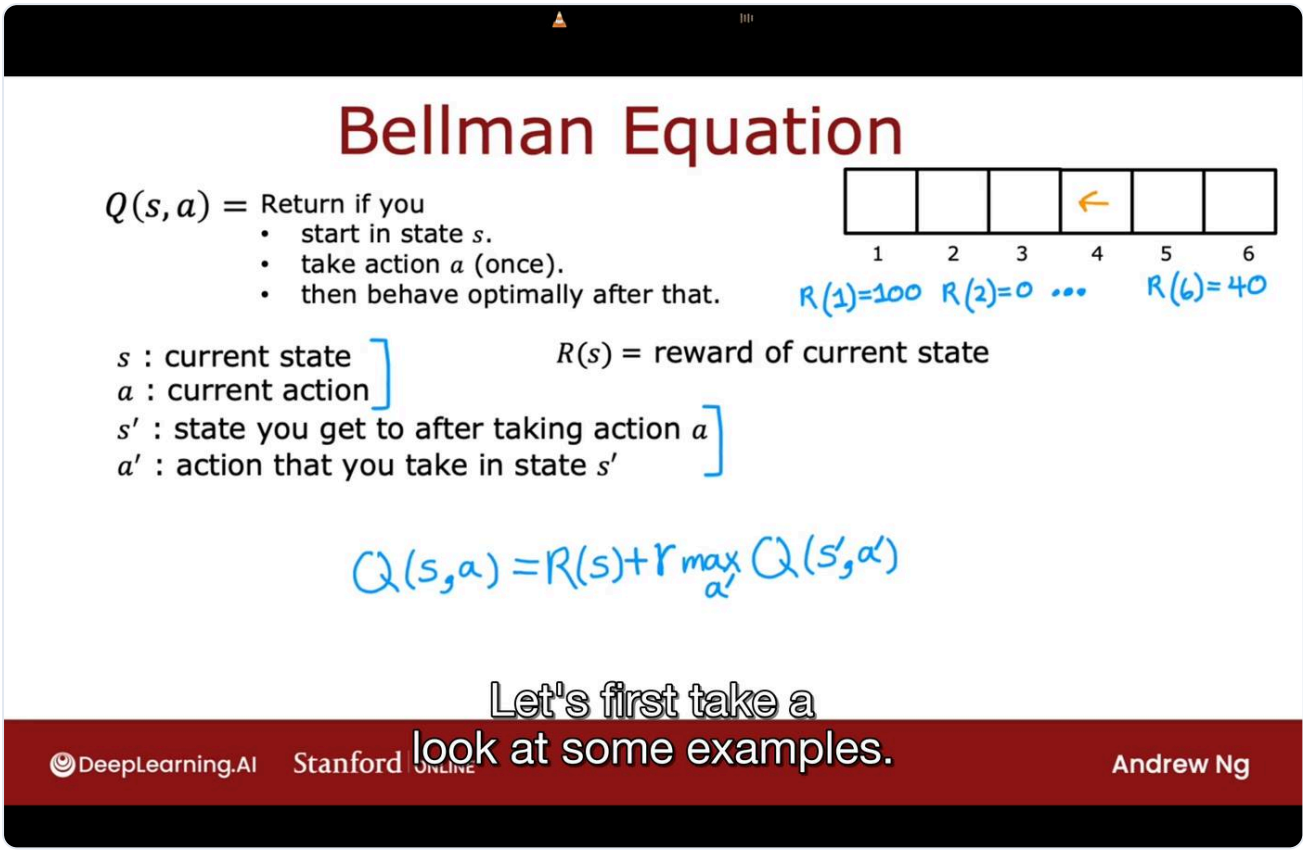


FIGURE 36.5 | Bellman Equation

Let's first take a look at some examples. We'll come back to see why this equation might make sense. Let's look at an example. Let's look at Q of State 2 and action. Apply Bellman Equation to this to see what value it gives us. If the current state is state two and that the action is to go right, then the next day you get to, after going write S prime would be the State 3. The Bellman equation says Q of 2, right is R of S. This R State 2, which is just the rewards zero plus the discount factor Gamma, which we set to 0.5 in this example, times max of the Q values in state S prime in State 3. This is going to be the max of 25 and 6.25, since this is max over a prime of q of S prime comma a prime.

This is taking the larger of 25 or 6.25 because those are the two choices for State 3.

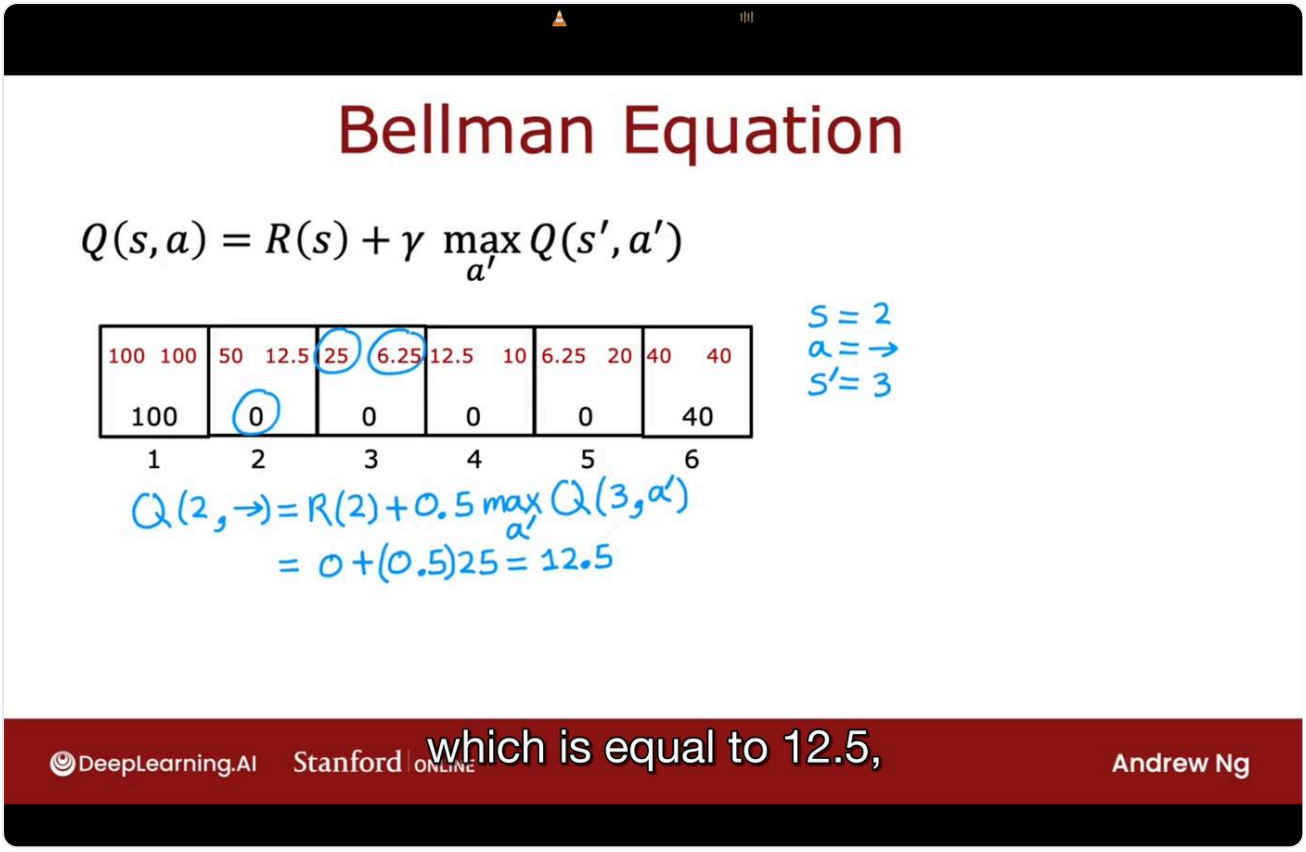


FIGURE 36.6 | Bellman Equation

This turns out to be equal to zero plus 0.5 times 25, which is equal to 12.5, which fortunately is Q of two and then the action right. Let's look at just one more example. Let me take the State 4 and see what is Q of State 4 if you decide to go left. In this case, the current state is four current action is to go left. The next state, if you can start from four going left. You end up also at State 3. Let us prime this three again, the Bellman Equation, we'll say this is equal to R of S. Our State four, which is zero plus 0.5 the discount factor Gamma of max over a prime of q of S prime. That is the State 3 again, comma a prime. Once again, the Q values for State 3 are 25 and 6.25 and the larger of these is 25.

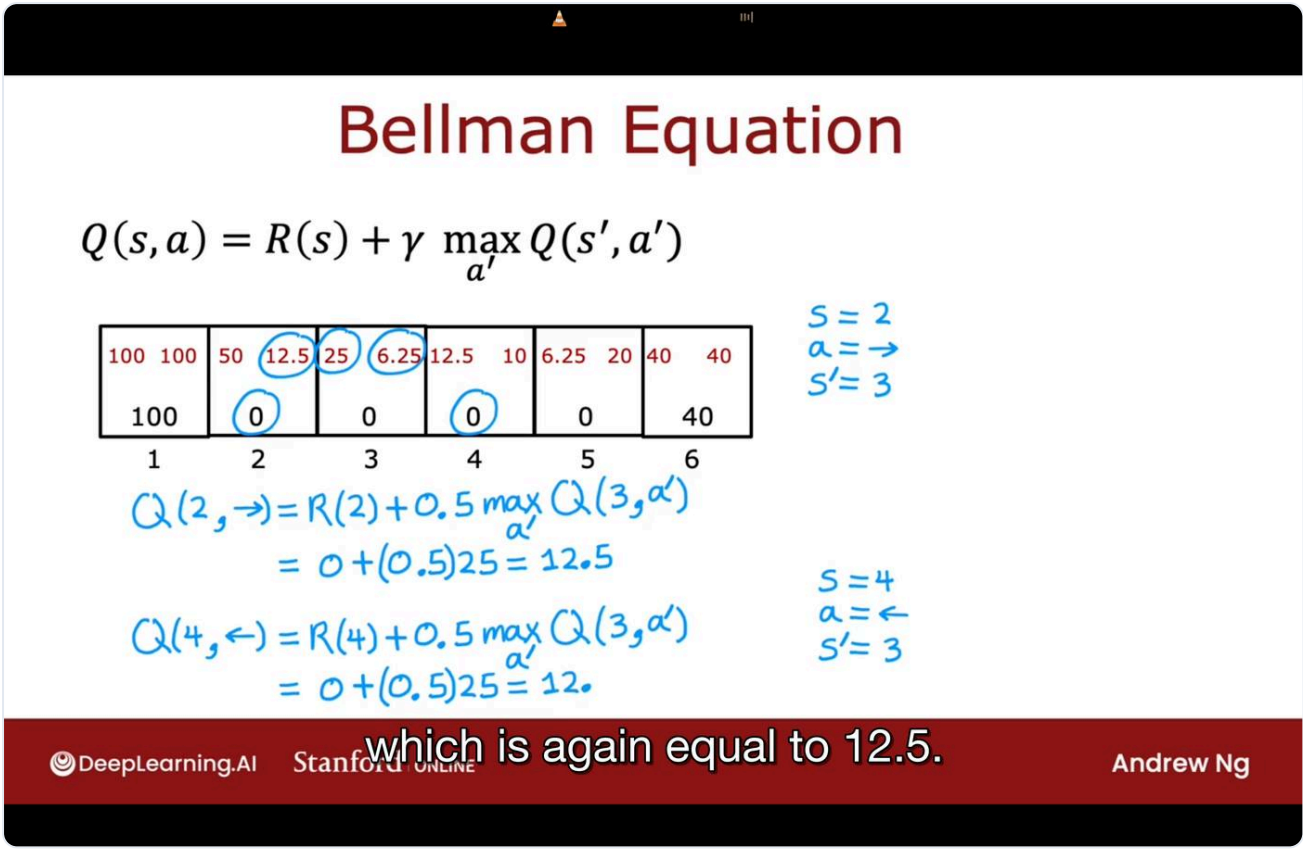


FIGURE 36.7 | Bellman Equation

This works out to be R(4) is 0 plus 0.5 times 25, which is again equal to 12.5.

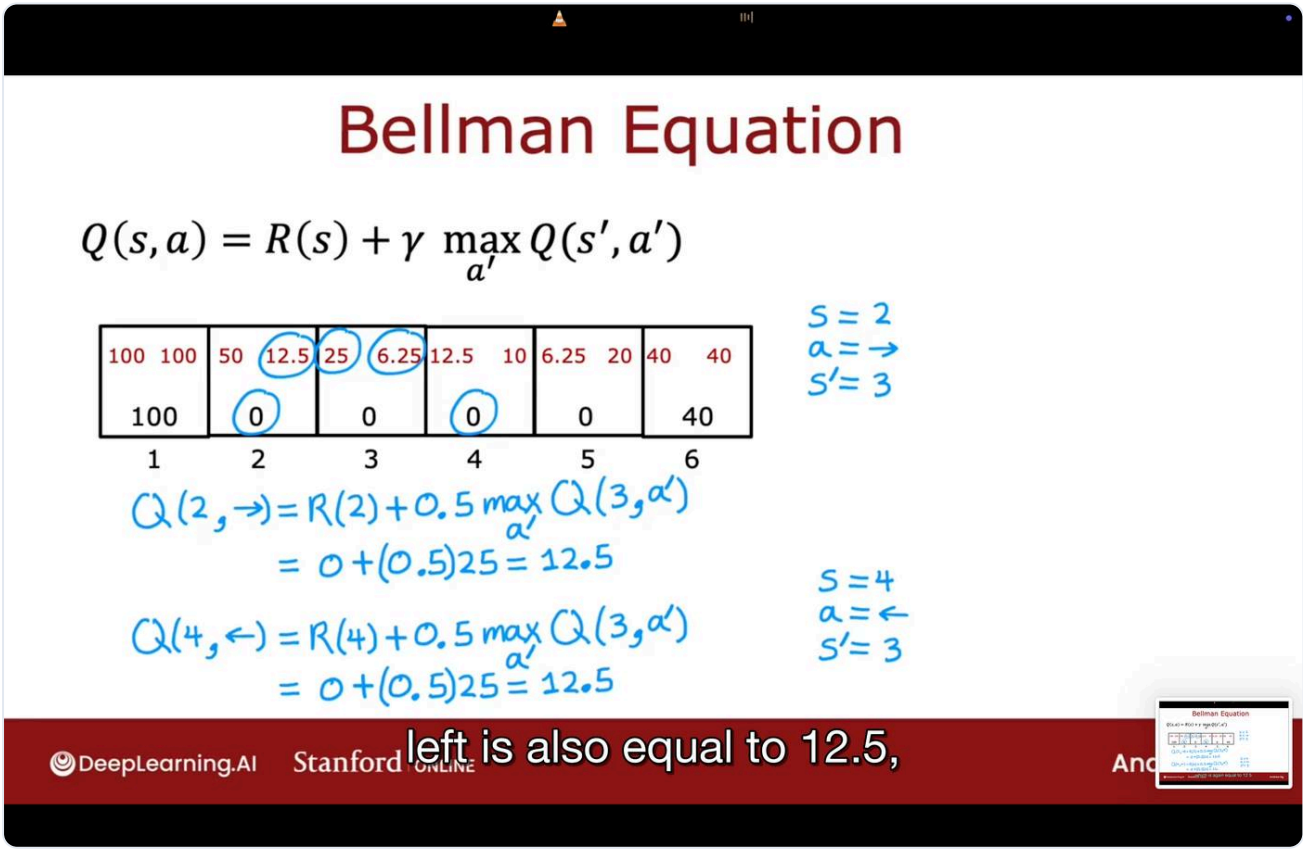


FIGURE 36.8 | Bellman Equation

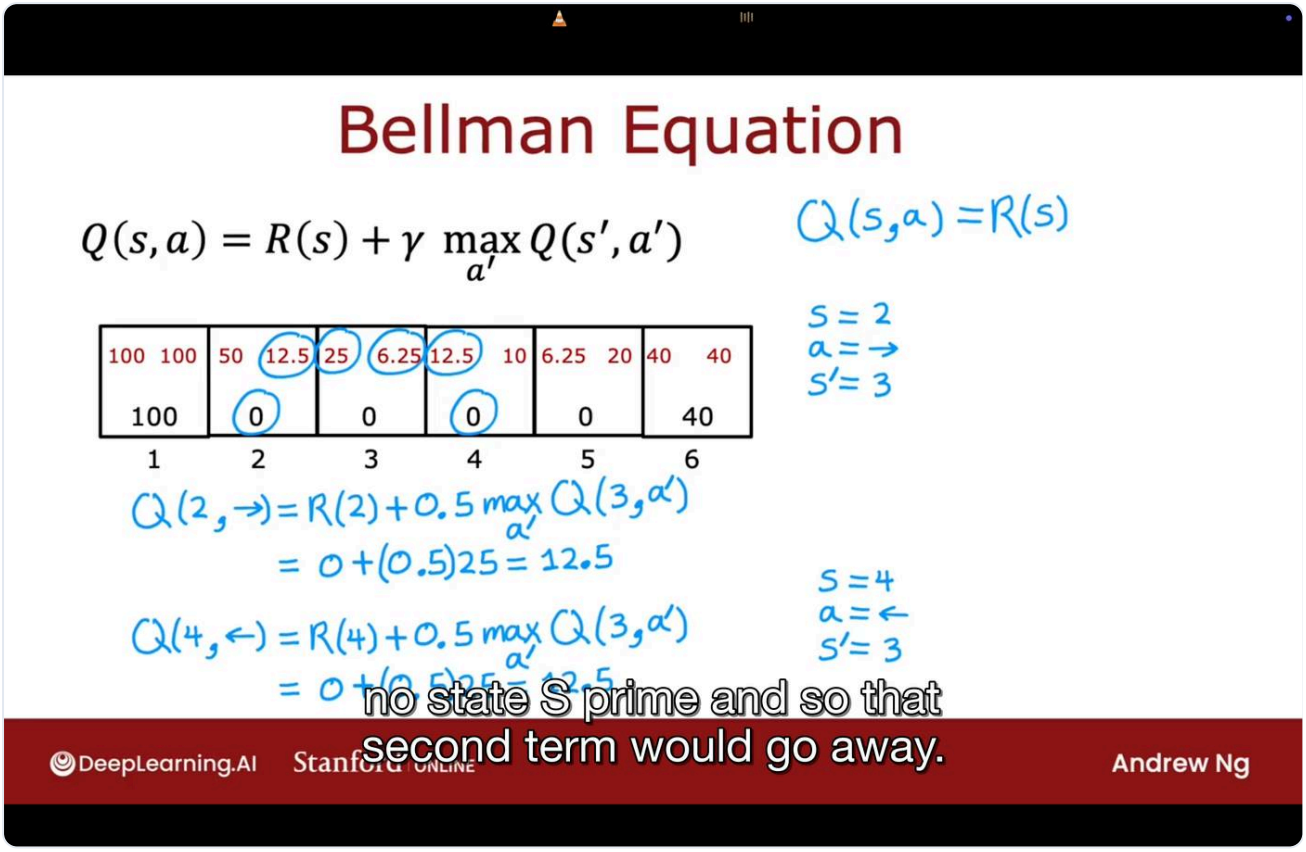


FIGURE 36.9 | Bellman Equation

That's why q of four with the action left is also equal to 12.5, just one note, if you're in a terminal state, then Bellman Equation simplifies to q of SA equals to r of S because there's no state S prime and so that second term would go away. Which is why Q of S,A in the terminal states is just 100, 100 or 40 40. If you wish feel free to pause the video and apply the Bellman Equation to any other state action in this MDP and check for yourself if this math works out. Just to recap, this is how we had define Q of S,A. We saw earlier that the best possible return from any state S is max over a Q of S,A. In fact, just to rename SNA, it turns out that the best possible return from a state S prime, is max over S prime of a prime.

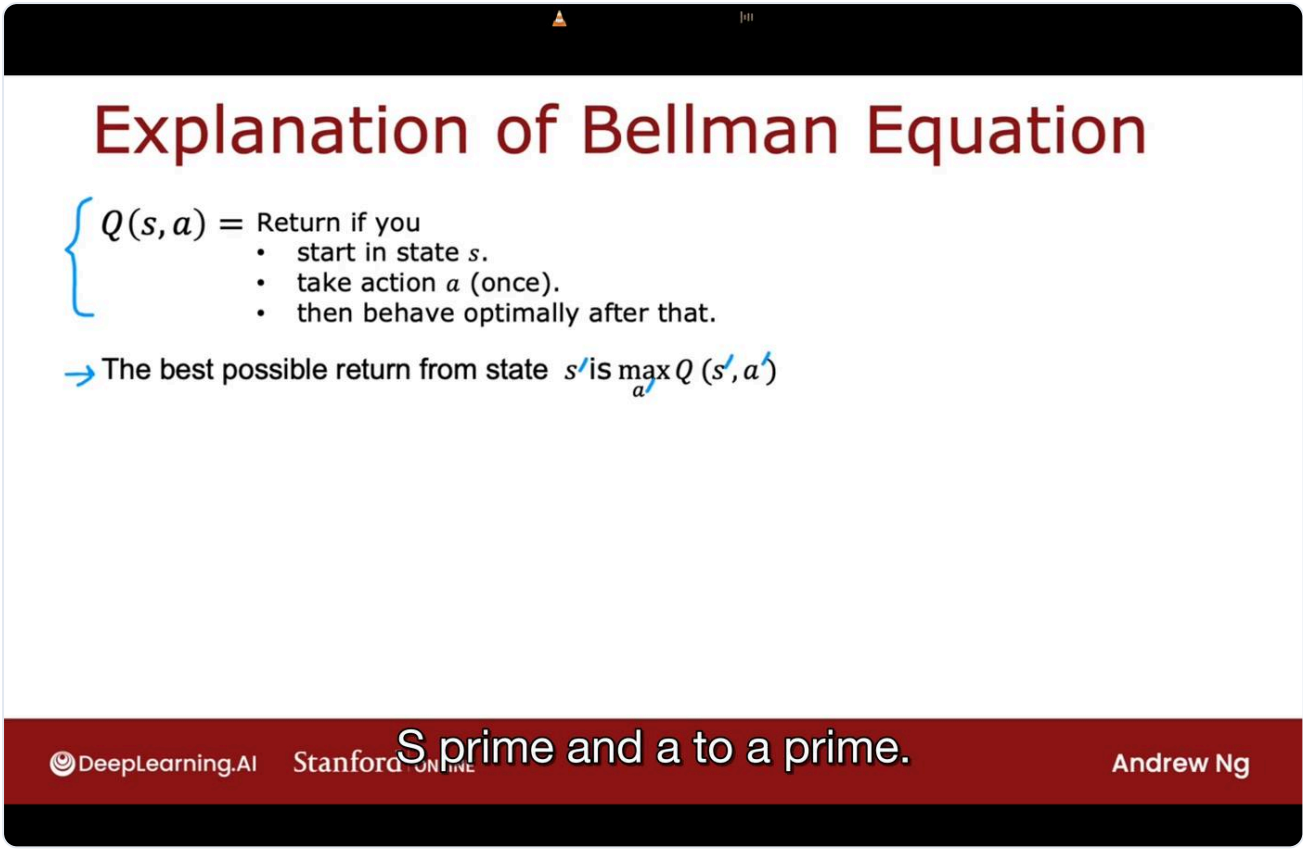


FIGURE 36.10 | Explanation of Bellman Equation

I didn't really do anything other than rename S, S prime and a to a prime. But this will make some of the intuitions a little bit easier later. But for any state S prime, like State 3, the best possible return from, say, State 3 is the max over all possible actions of Q of S prime a prime. Here again is the Bellman equation. The intuition that this captures is if you're starting from state s and you're going to take action a and then act optimally after that, then you're going to see some sequence of rewards over time. In particular, the return will be computed from the reward at the first step, plus Gamma times reward at the second step plus Gamma squared times reward at the third step, and so on.

Plus dot, dot, dot until you get to the terminal state. What Bellman equation says is this sequence of rewards, what the discount factor is, can be broken down into two components. First, this R of s, that's the reward you get right away. In the reinforcement learning literature, this is sometimes also called the immediate reward, but that's what R_1 is. It's the reward you get for starting out in some state s. The second term then is the following; after you start in state s and take action a, you get to some new state s prime. The definition of Q of s, a assumes we're going to behave optimally after that. After we get to s prime, we are going to behave optimally and get the best possible return from the state s prime.

What this is, max of a prime of Q of s prime a prime, this is the return from behaving optimally, starting from the state s prime.

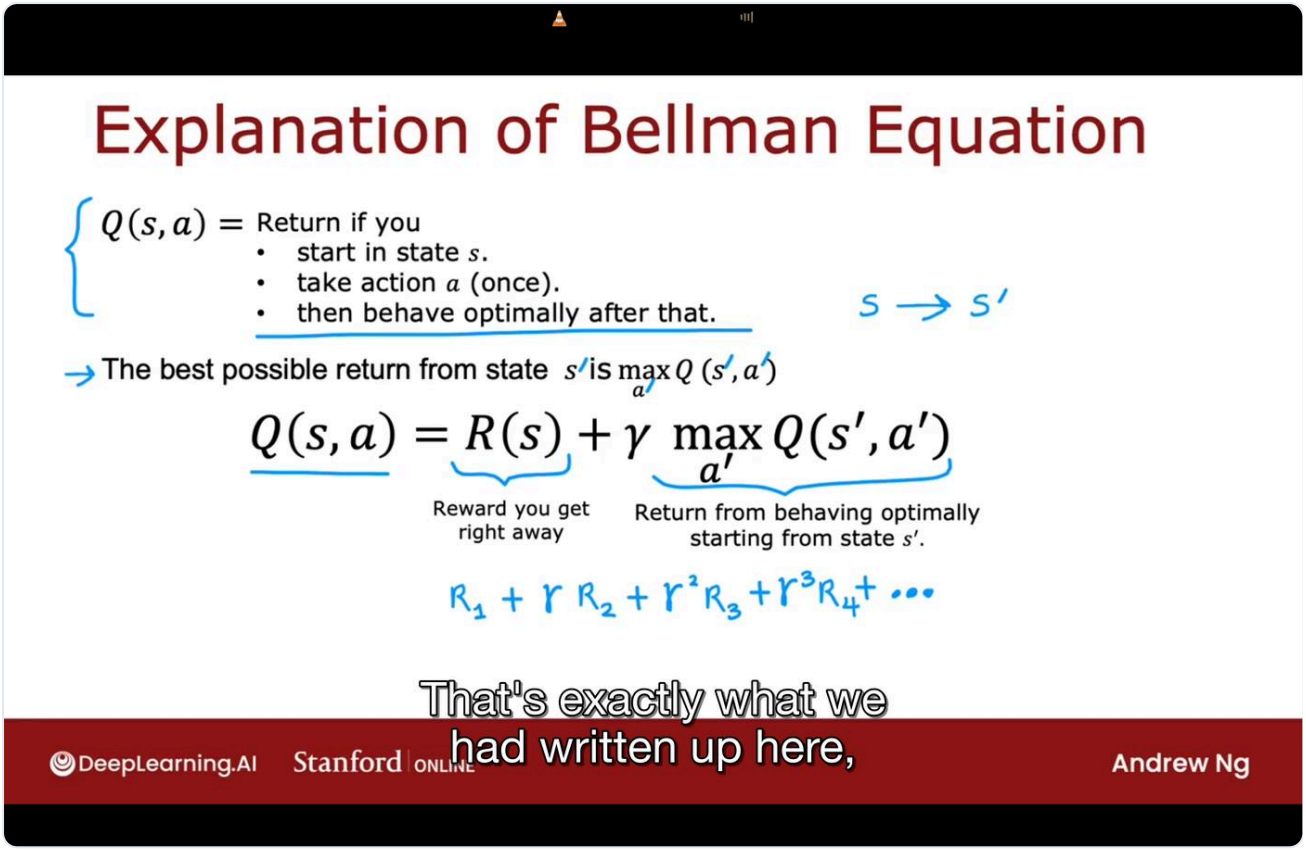


FIGURE 36.11 | Explanation of Bellman Equation

That's exactly what we had written up here, is the best possible return for when you start from state s prime. Another way of phrasing this is this total return down here is also equal to R_1 plus, and then we're going to factor out Gamma in the map, is Gamma times R_2 plus, and instead of Gamma squared is just Gamma times R_3 plus Gamma squared times R_4 plus dot dot dot. Notice that if you were starting from state s prime, the sequence of rewards you get will be R_2, R_3, then R_4, and so on. That's why this expression here, that's the total return if you were to start from state s prime.

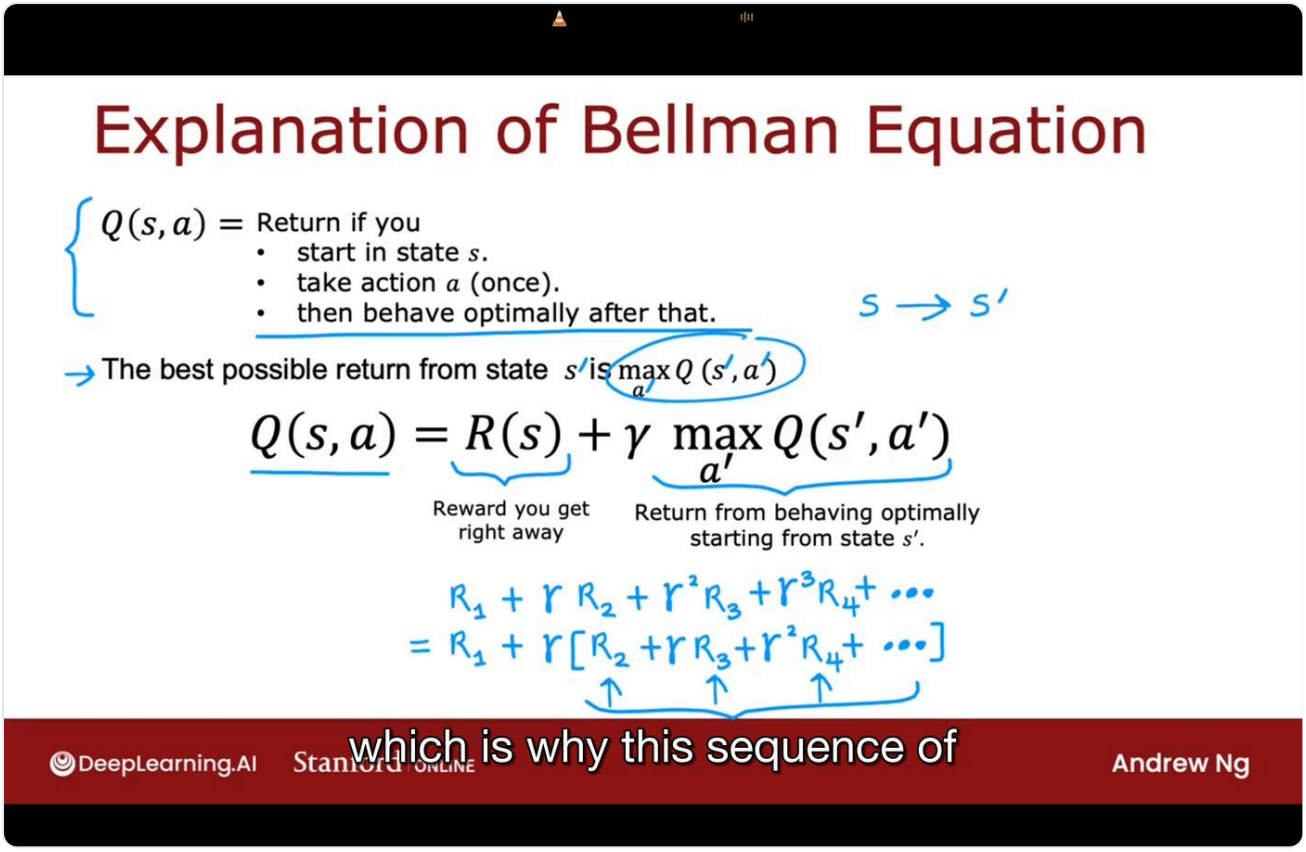


FIGURE 36.12 | Explanation of Bellman Equation

If you were to behave optimally, then this expression should be the best possible return for starting from state s prime, which is why this sequence of discount rewards equals that max of a prime of Q of s prime a prime and there were also leftover with this extra discount factor Gamma there, which is why Q of s, a is also equal to this expression over here. In case you think this is quite complicated and you aren't following all the details, don't worry about it. So long as you apply this equation, you will manage to get the right results. But the high level intuition I hope you take away is that the total return you get in the reinforcement learning problem has two parts.

The first part is this reward that you get right away, and then the second part is Gamma times the return you get starting from the next state s prime. As these two components together, R of s plus Gamma times the return from the next state, that is equal to the total return from the current state s. That is the essence of the Bellman equation. Just to relate this back to our earlier example, Q of 4, left.

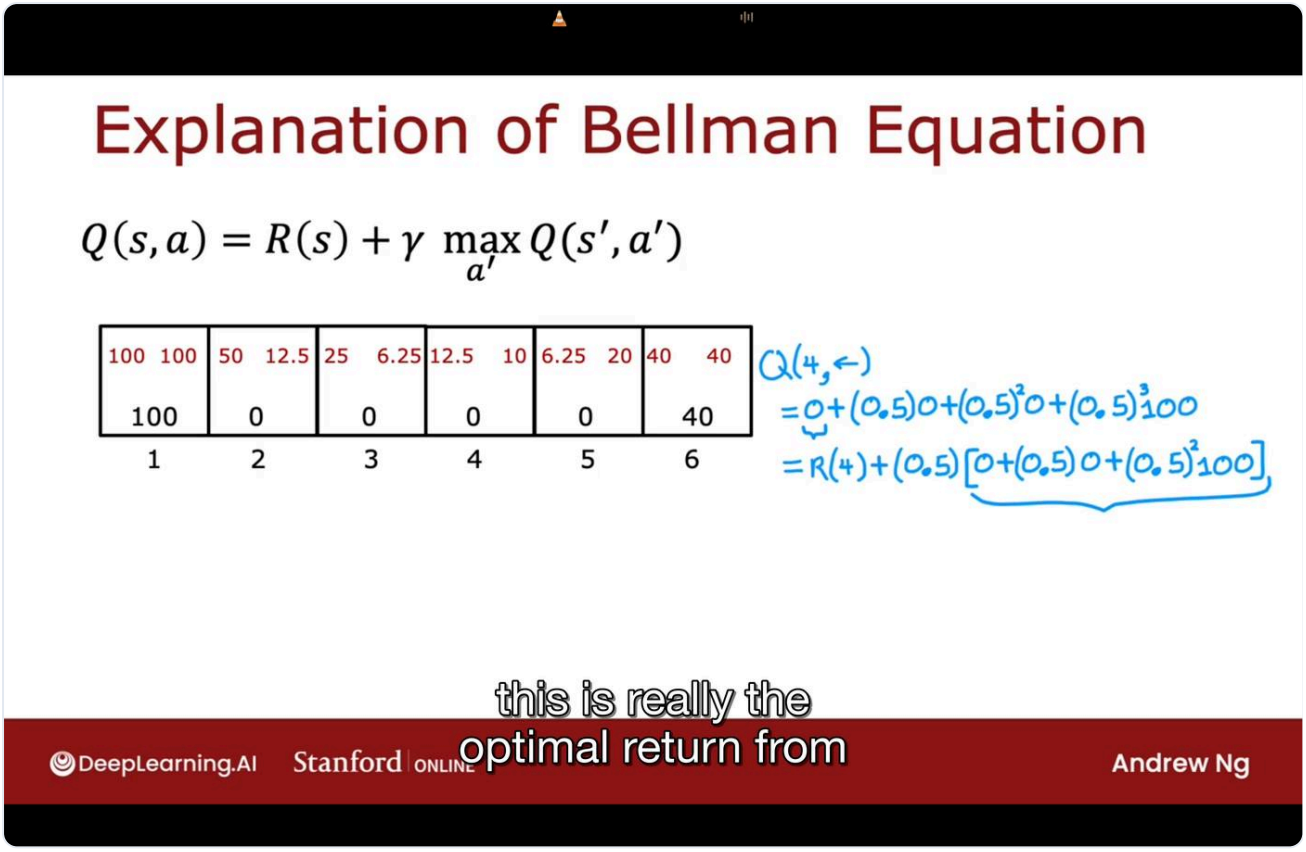


FIGURE 36.13 | Explanation of Bellman Equation

That's the total return for starting State 4 and going left. If you were to go left in State 4 the rewards you get are 0 in State 4, 0 in State 3, 0 in State 2, and then 100, which is why the total return is this; 0.5 squared plus 0.5 cubed, which was 12.5. What Bellman equation is saying is that we can break this up into two pieces. There is this zero, which is R of the state four, and then plus 0.5 times this other sequence, 0 plus 0.50 plus 0.5 squared times 100. But if you look at what this sequence is, this is really the optimal return from the next state s prime that you got to after taking the action left from state four. That's why this is equal to the reward 4 plus 0.5 times the optimal return from State 3.

Because if you were to start from State 3, the rewards you get would be zero followed by zero followed by 100, so this is optimal return from State 3 and that's why this is just R of 4 plus 0.5 max over a prime Q of State 3, a prime.

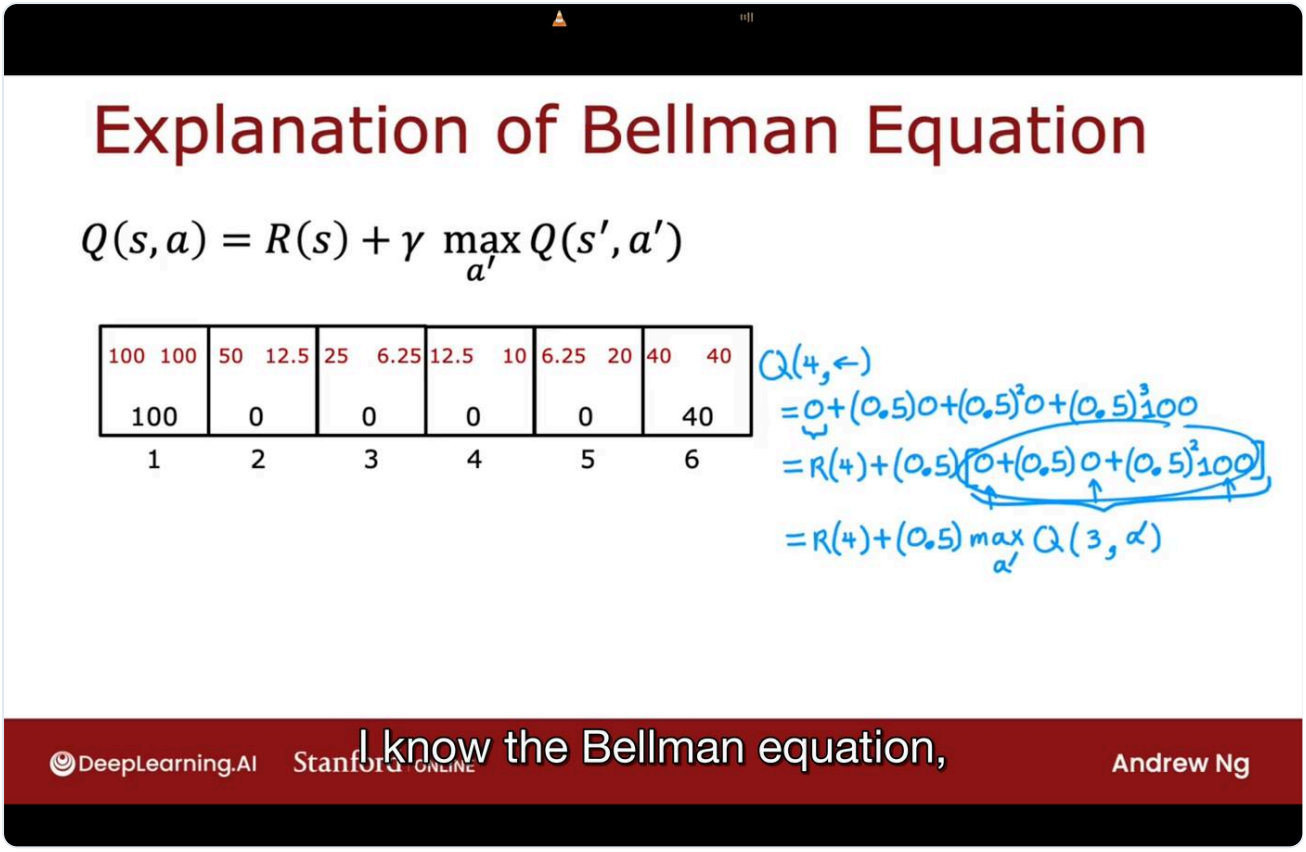


FIGURE 36.14 | Explanation of Bellman Equation

I know the Bellman equation, this is somewhat complicated equation breaking down your total returns into the reward you're getting right away. The immediate reward plus Gamma times the returns from the next state s prime. If it makes sense to you, but not fully, it's okay. Don't worry about it. You can still apply Bellman's equations to get a reinforcement learning algorithm to work correctly, but I hope that at least the high level intuition of why breaking down the rewards into what you get right away plus what you get in the future. I hope that makes sense. Before moving on to develop a reinforcement learning algorithm, we have coming up next an optional video on Stochastic Markov decision processes or on reinforcement learning applications where the actions, if you take, can have a slightly random effect.

Take look at the optional video if you wish. Then after that, we'll start to develop a reinforcement learning algorithm.

LECTURE 37

Expected Return

In some applications, when you take an action, the outcome is not always completely reliable. For example, if you command your Mars rover to go left maybe there's a little bit of a rock slide, or maybe the floor is really slippery and so it slips and goes in the wrong direction. In practice, many robots don't always manage to do exactly what you tell them because of wind blowing and off course and the wheel slipping or something else. There's a generalization of the reinforcement learning framework we've talked about so far, which models random or stochastic environments. In this optional video, we'll talk about how these reinforcement learning problems work, continuing with our simplifying Mars Rover example, let's say you take the action and command it to go left.

Most of the time you'll succeed but what if 10 percent of the time or 0.1 of the time, it actually ends up accidentally slipping and going in the opposite direction? If you command it to go left, it has a 90 percent chance or 0.9 chance of correctly going in the left direction. But the 0.1 chance of actually heading to the right so that it has a 90 percent chance of ending up in state three in this example and a 10 percent chance of ending up in state five.

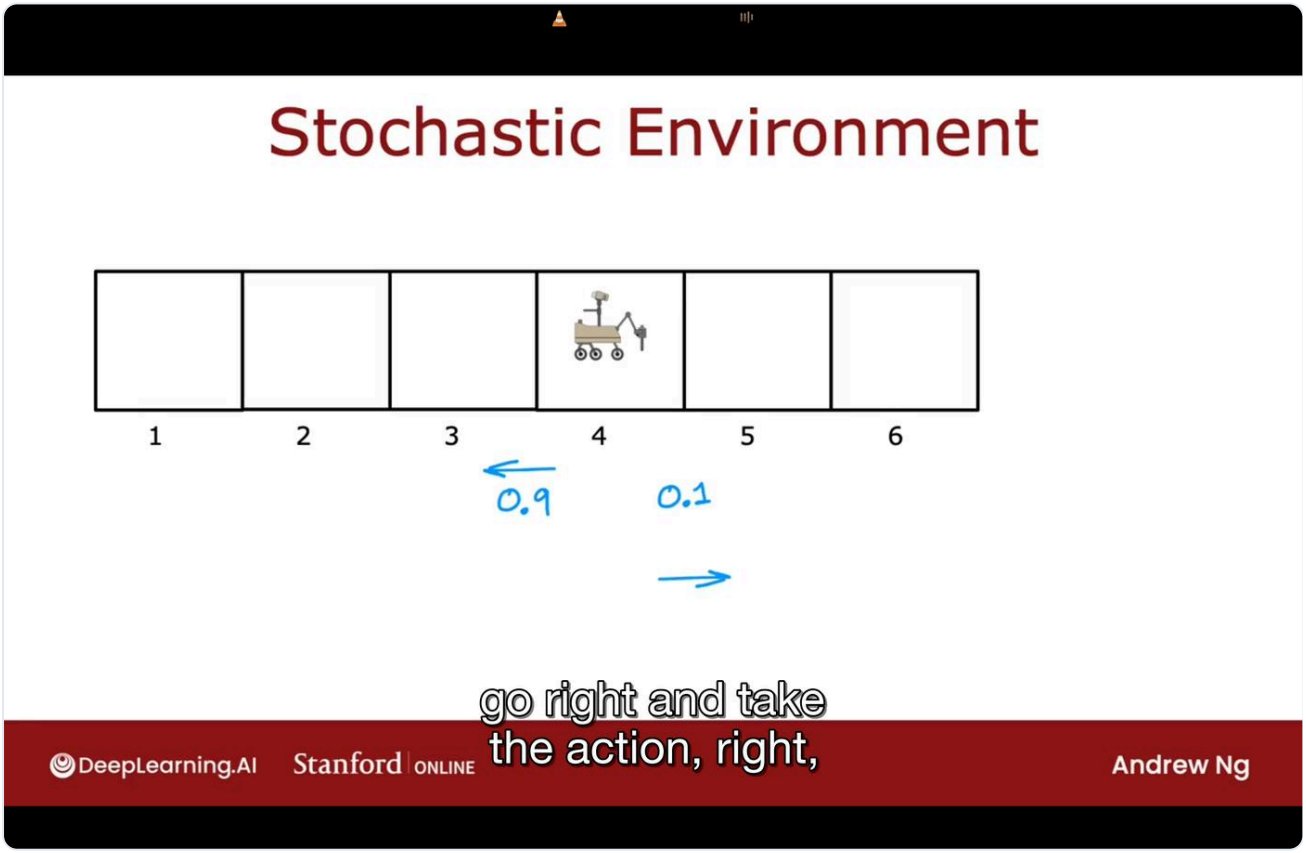


FIGURE 37.1 | Stochastic Environment

Conversely, if you were to command it to go right and take the action, right, it has a 0.9 chance of ending up in state five and 0.1 chance of ending up in state three. This would be an example of a stochastic environment.

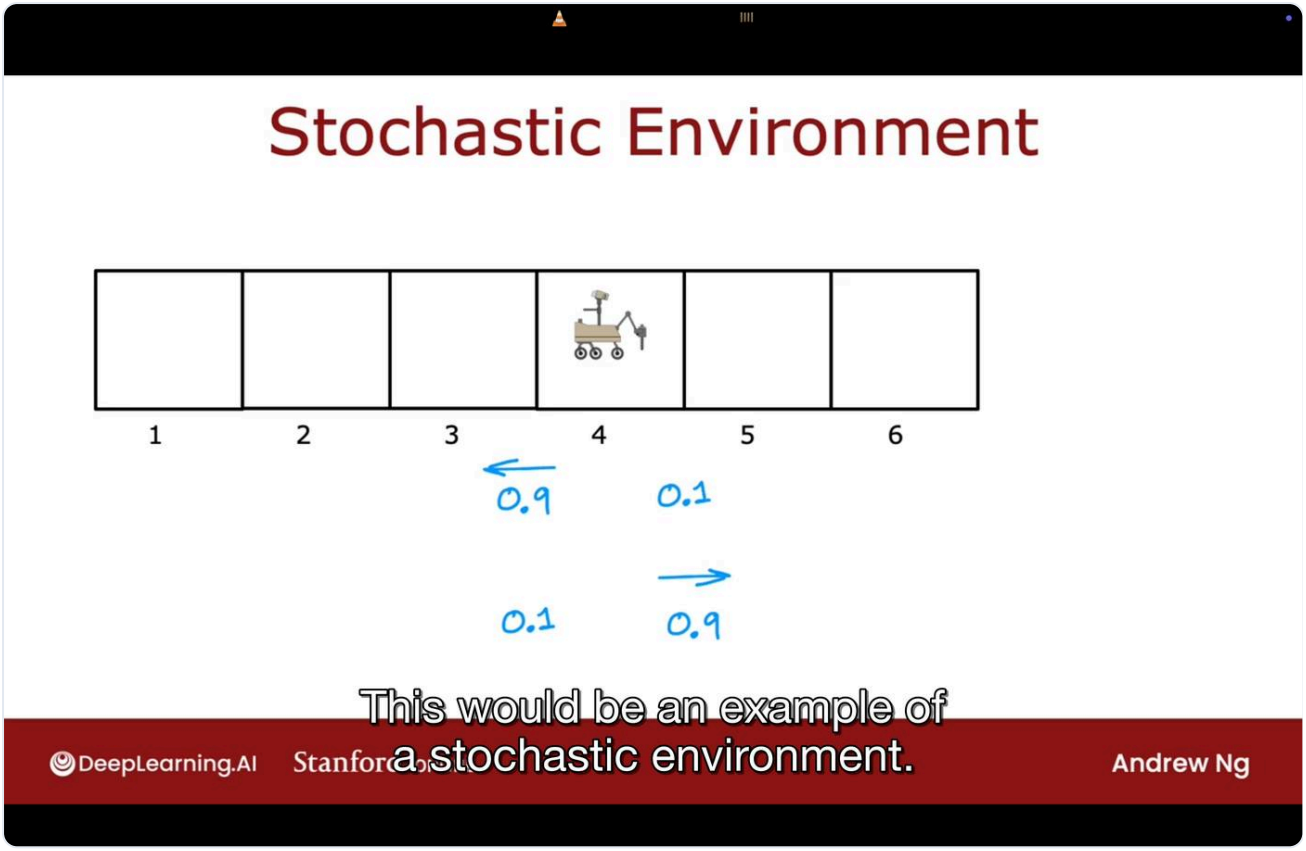


FIGURE 37.2

Let's see what happens in this reinforcement learning problem. Let's say you use this policy shown here, where you go left in states 2 3 4 and go rights or try to go right in state five.

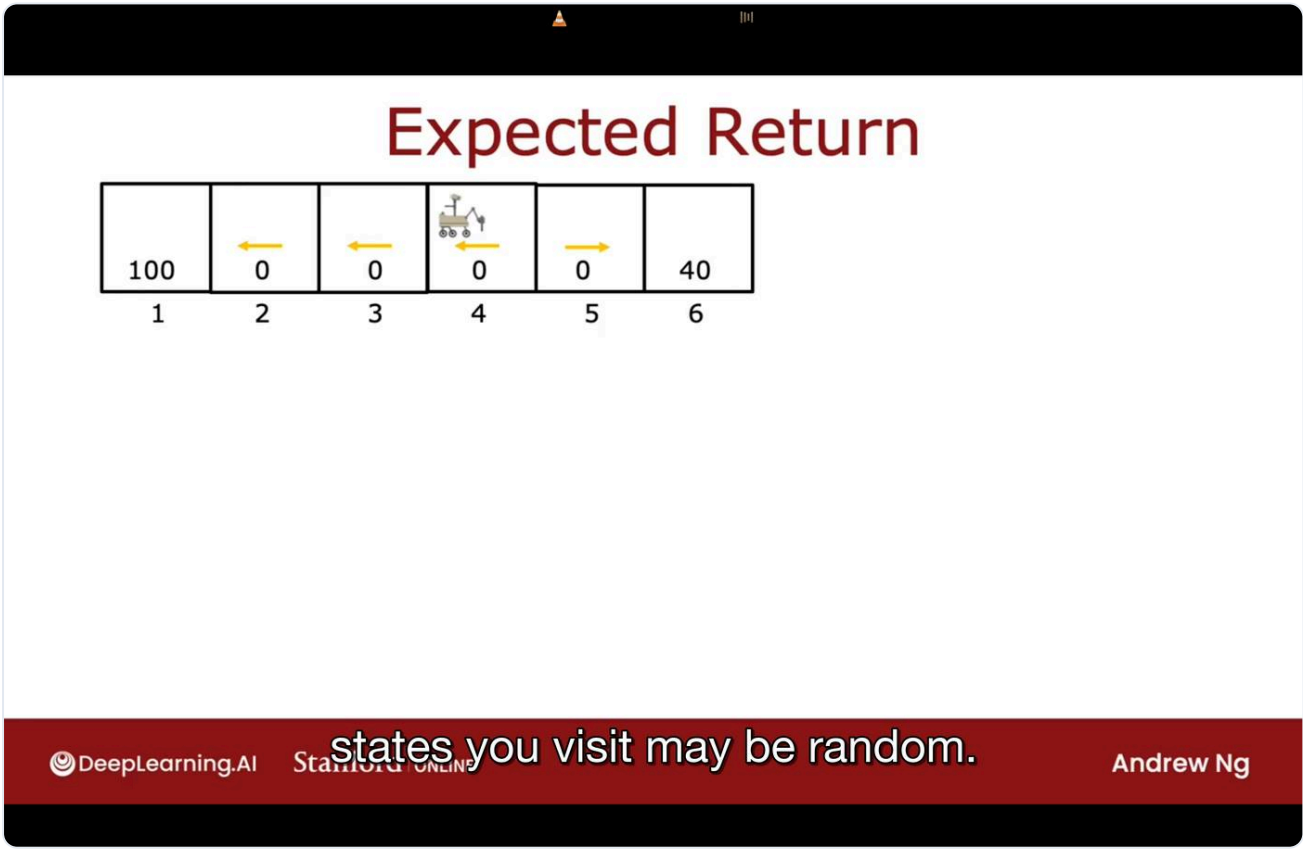


FIGURE 37.3 | Expected Return

If you were to start in state four and you were to follow this policy, then the actual sequence of states you visit may be random.

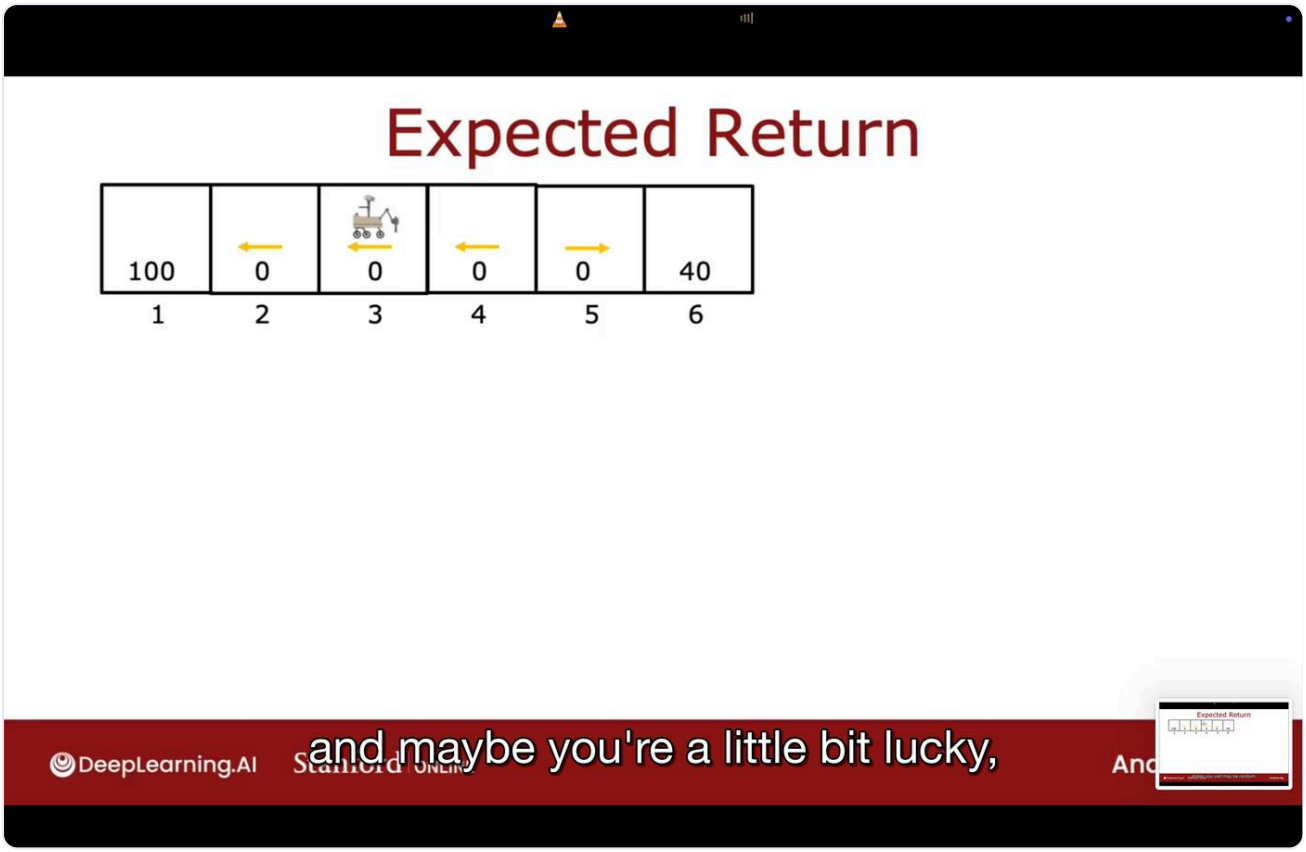


FIGURE 37.4 | Expected Return

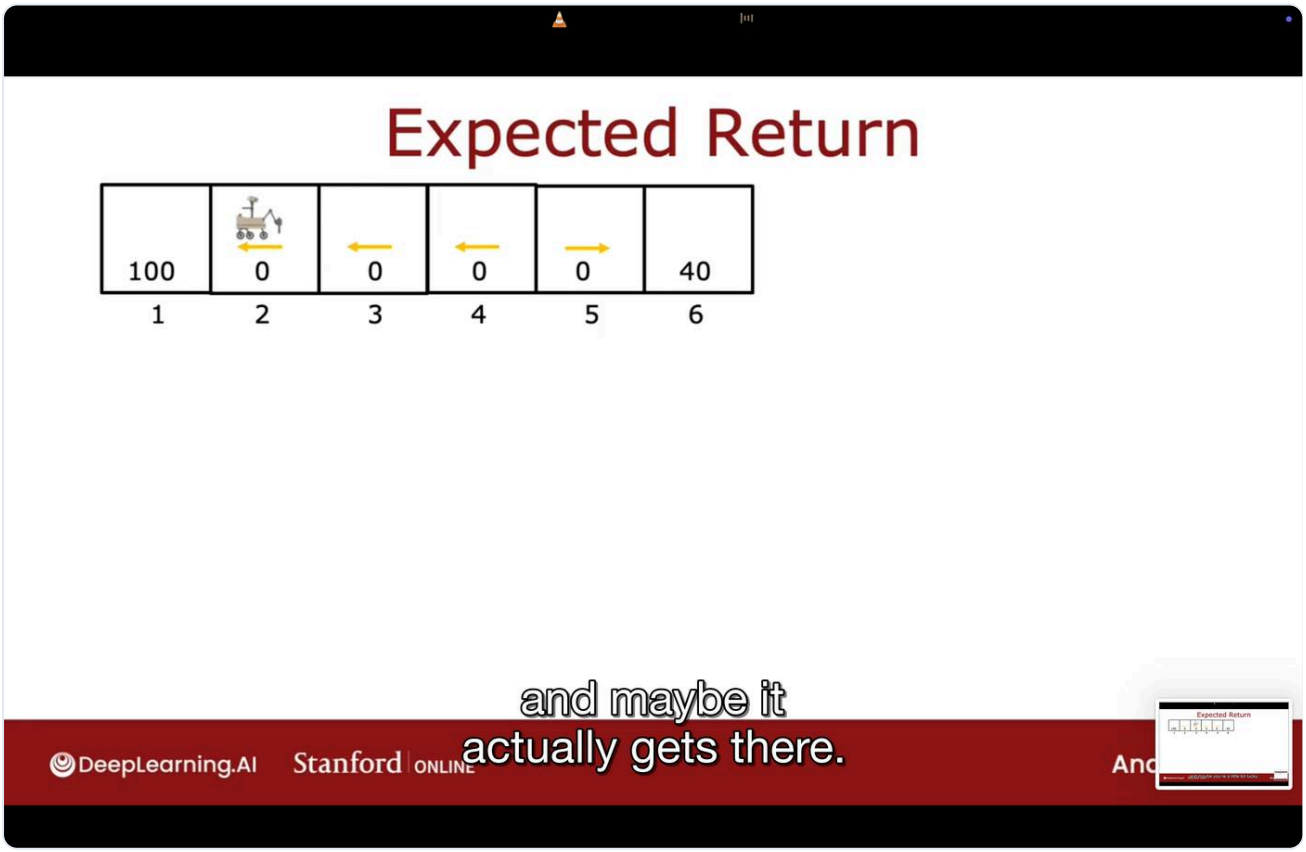


FIGURE 37.5 | Expected Return

For example, in state four, you will go left, and maybe you're a little bit lucky, and it actually gets the state three, and then you try to go left again, and maybe it actually gets there.

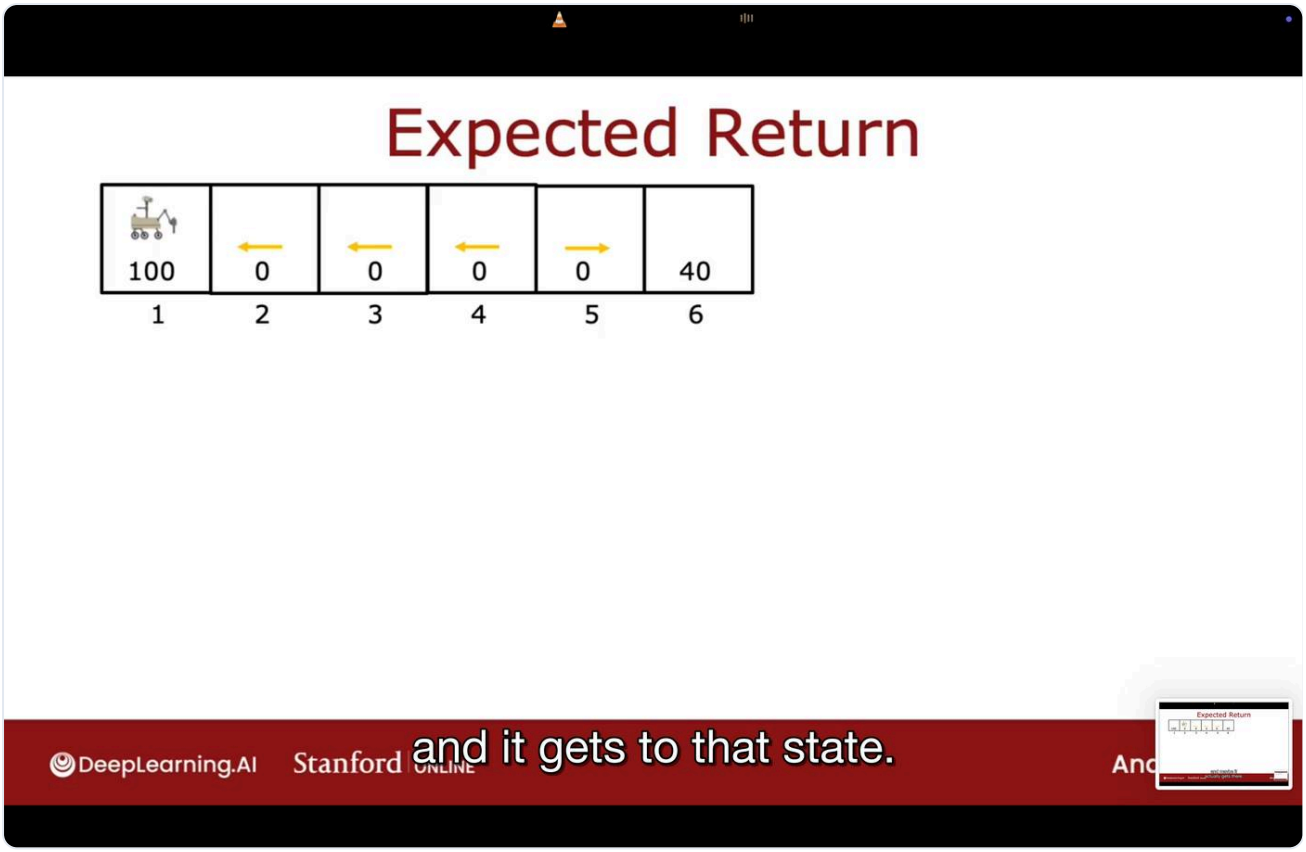


FIGURE 37.6 | Expected Return

You tell it to go left again, and it gets to that state.

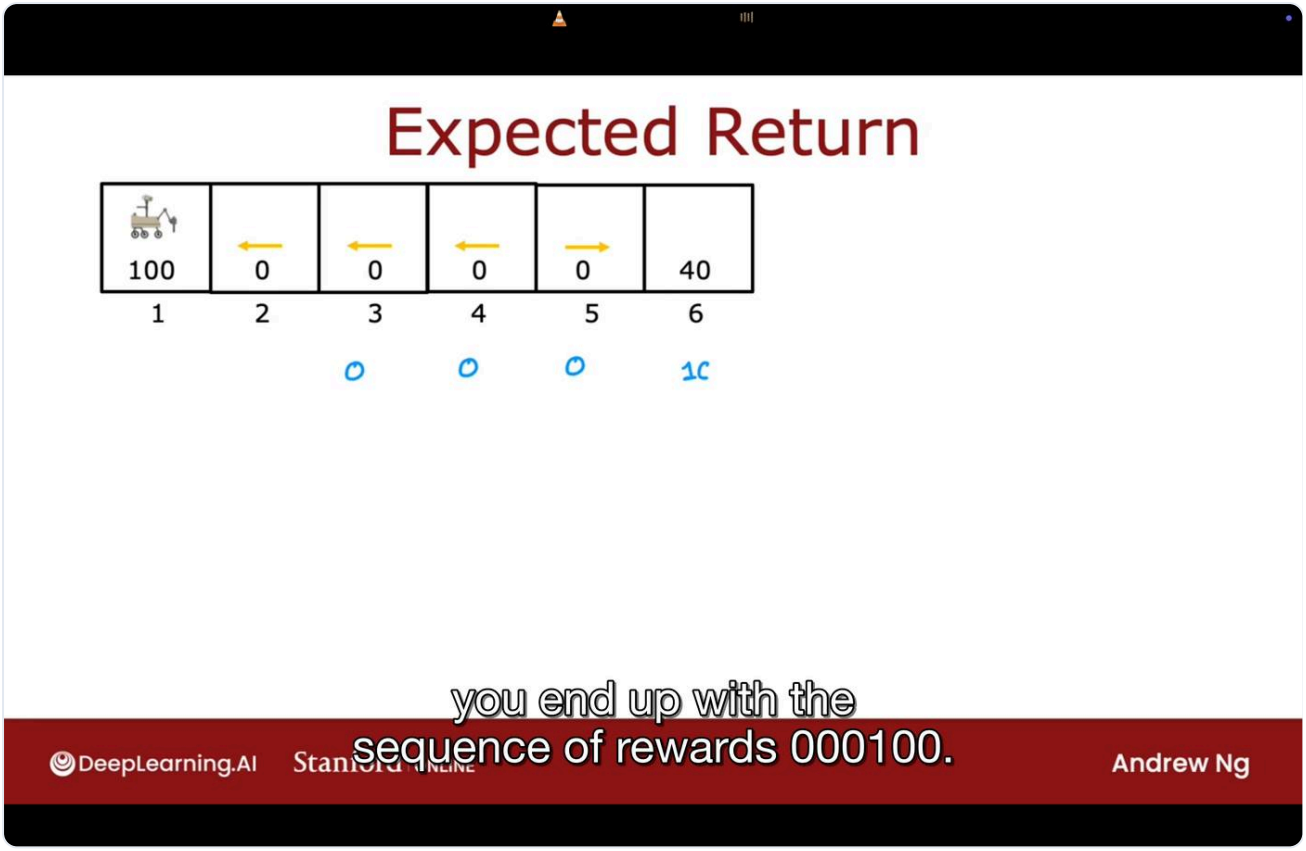


FIGURE 37.7 | Expected Return

If this is what happens, you end up with the sequence of rewards 000100.

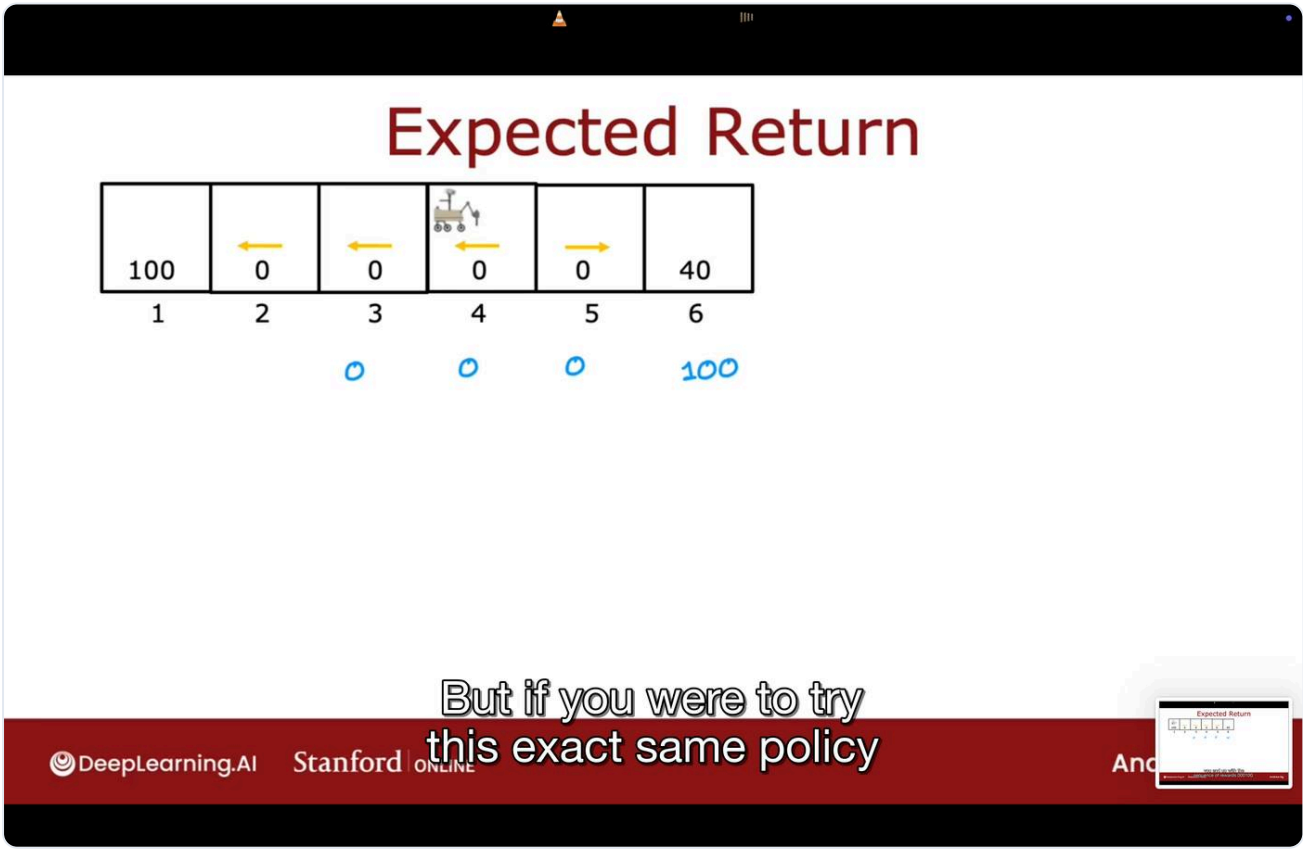


FIGURE 37.8

But if you were to try this exact same policy a second time, maybe you're a little less lucky, the second time you start here.

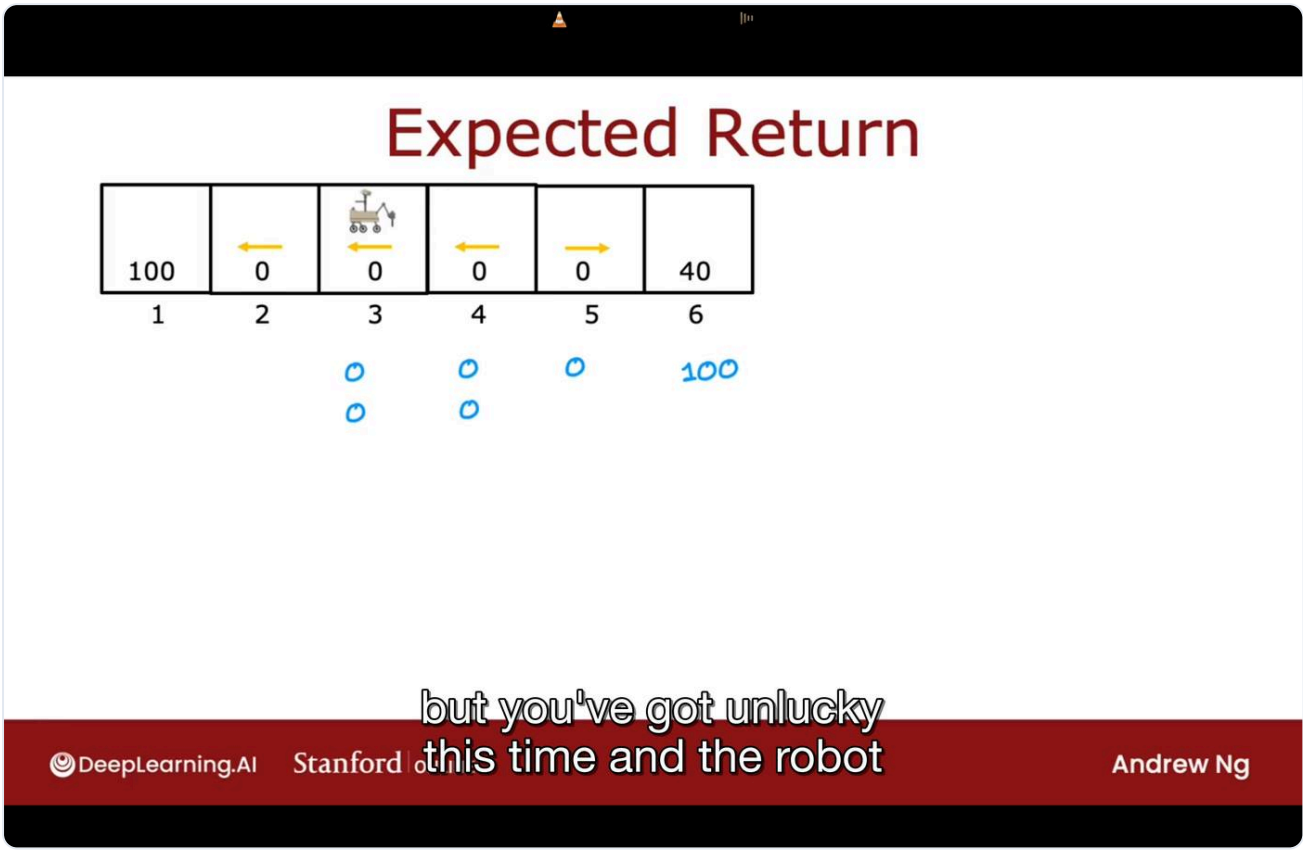


FIGURE 37.9 | Expected Return

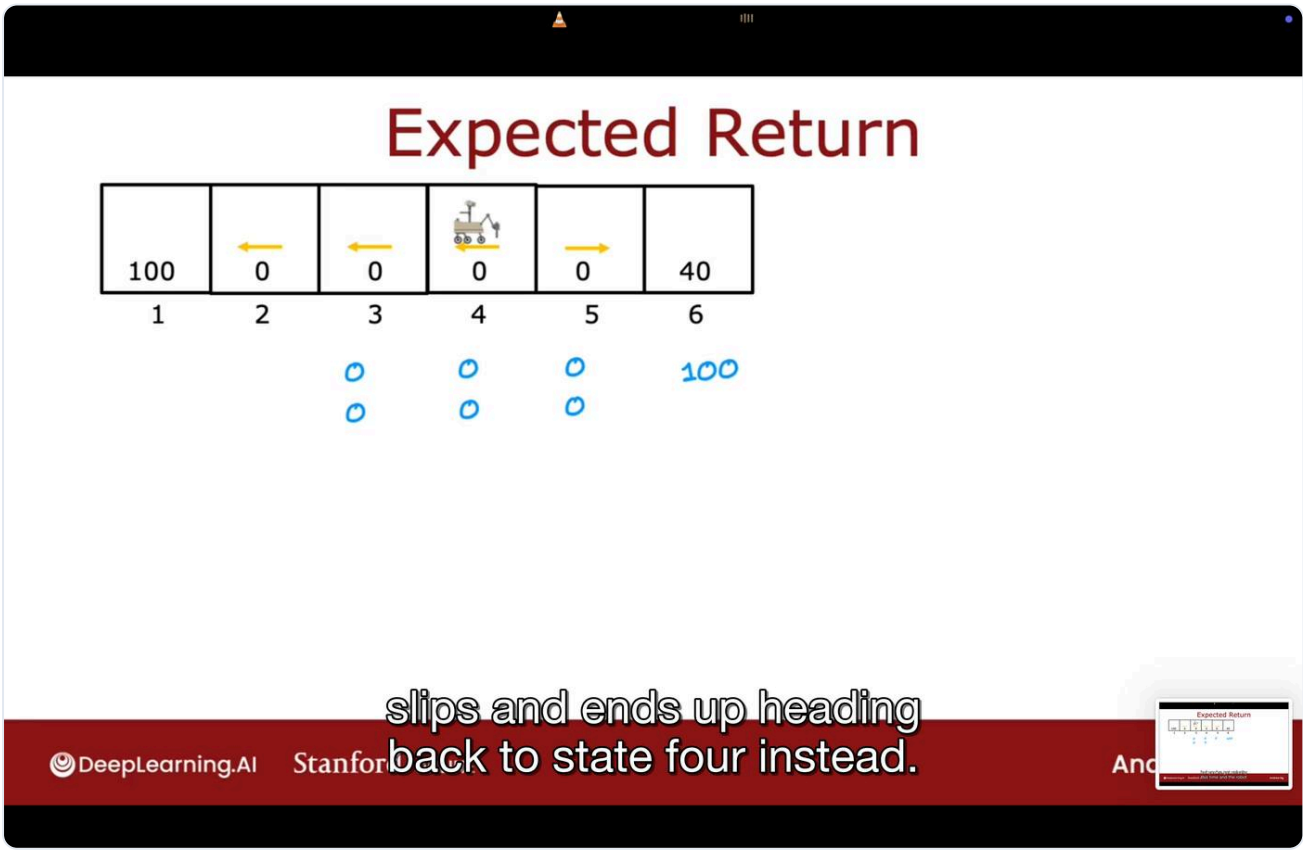


FIGURE 37.10 | Expected Return

Try to go left and say it succeeds so a zero from state four zero from state three, here you tell it to go left, but you've got unlucky this time and the robot slips and ends up heading back to state four instead.

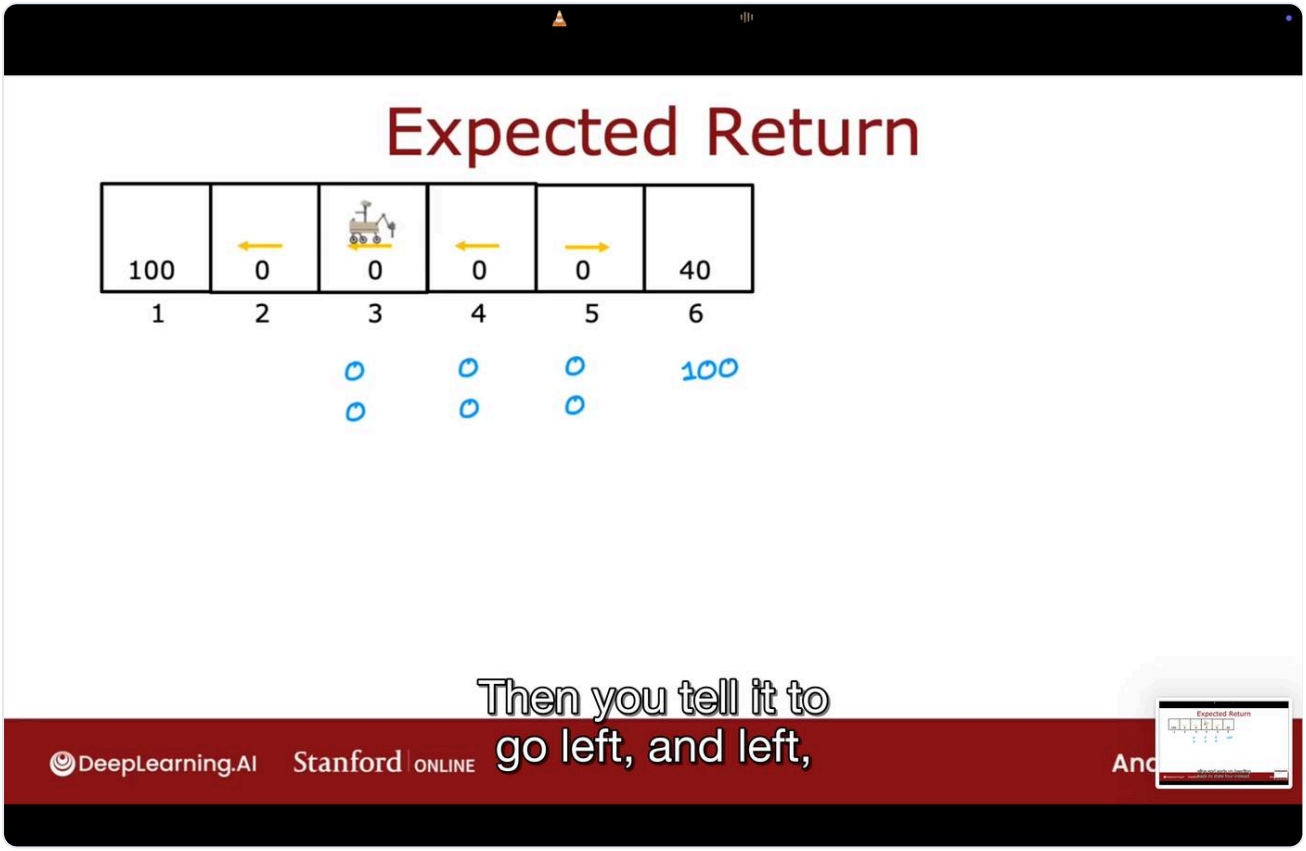


FIGURE 37.11 | Expected Return

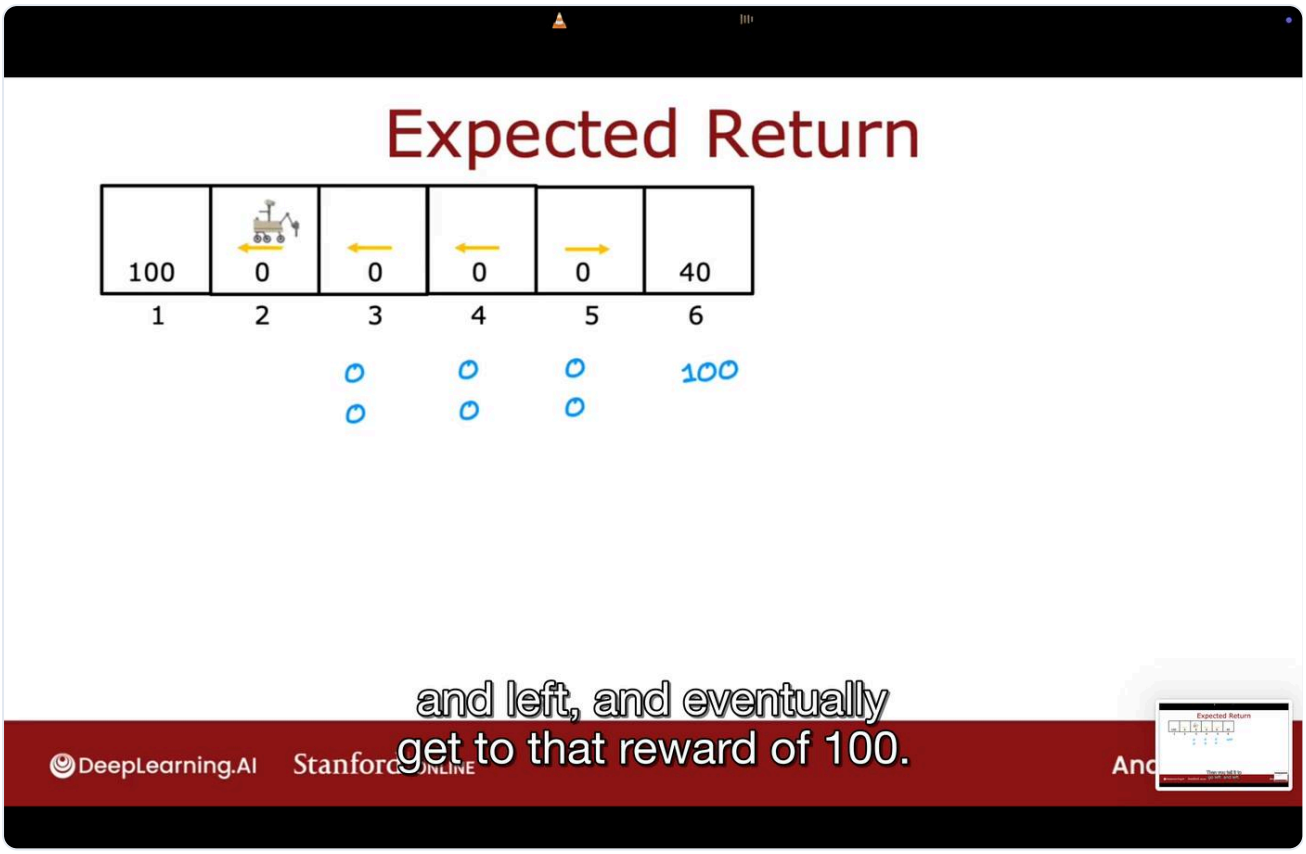


FIGURE 37.12 | Expected Return

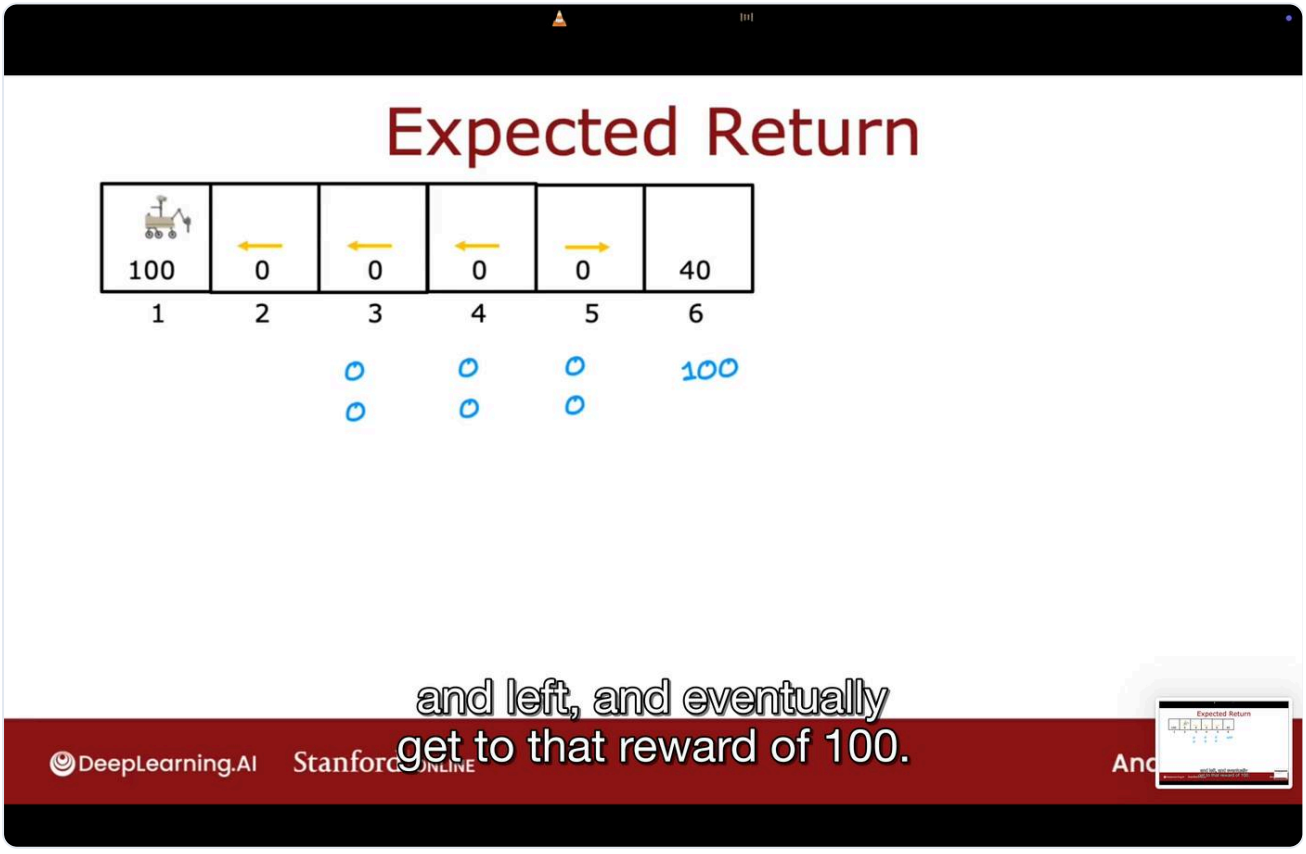


FIGURE 37.13 | Expected Return

Then you tell it to go left, and left, and left, and eventually get to that reward of 100. In that case, this will be the sequence of rewards you observe.

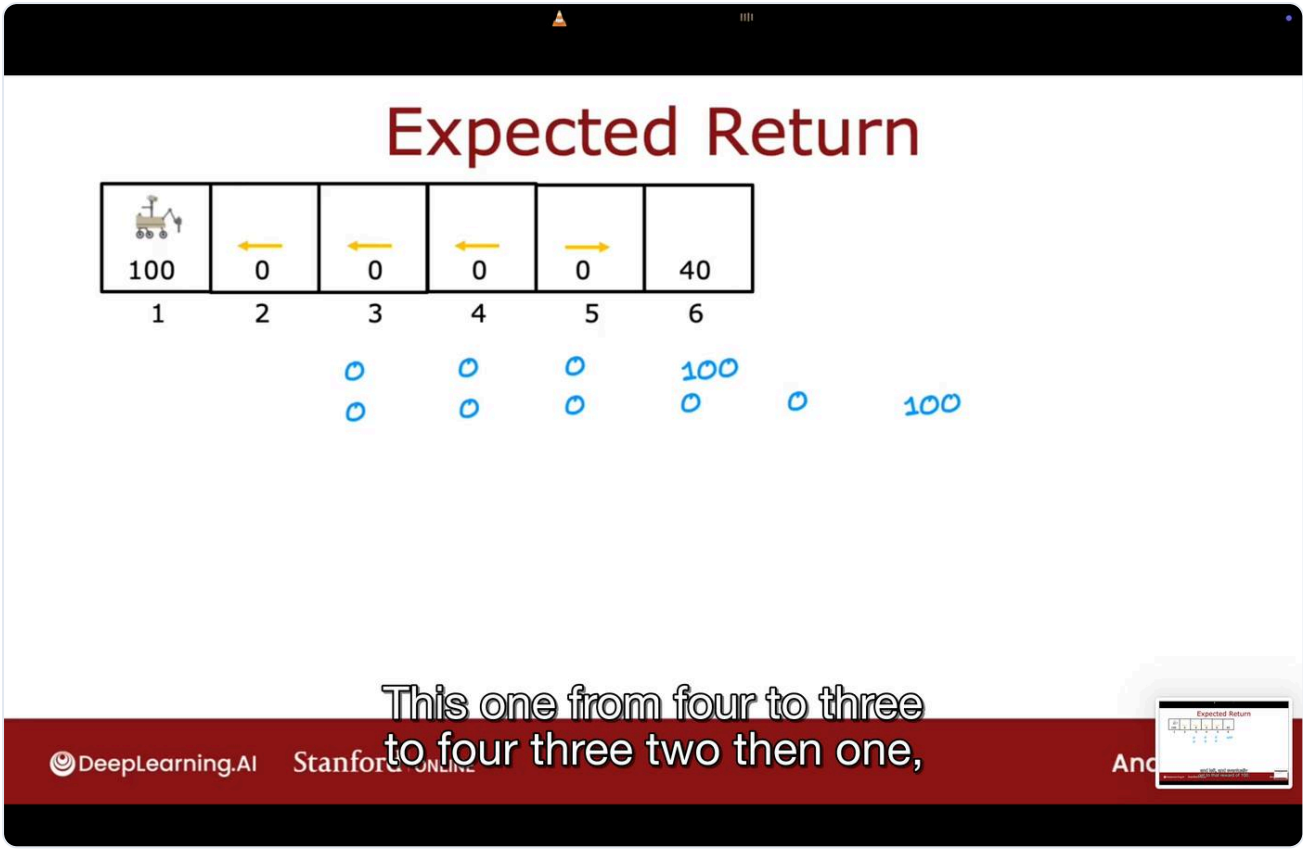


FIGURE 37.14 | Expected Return

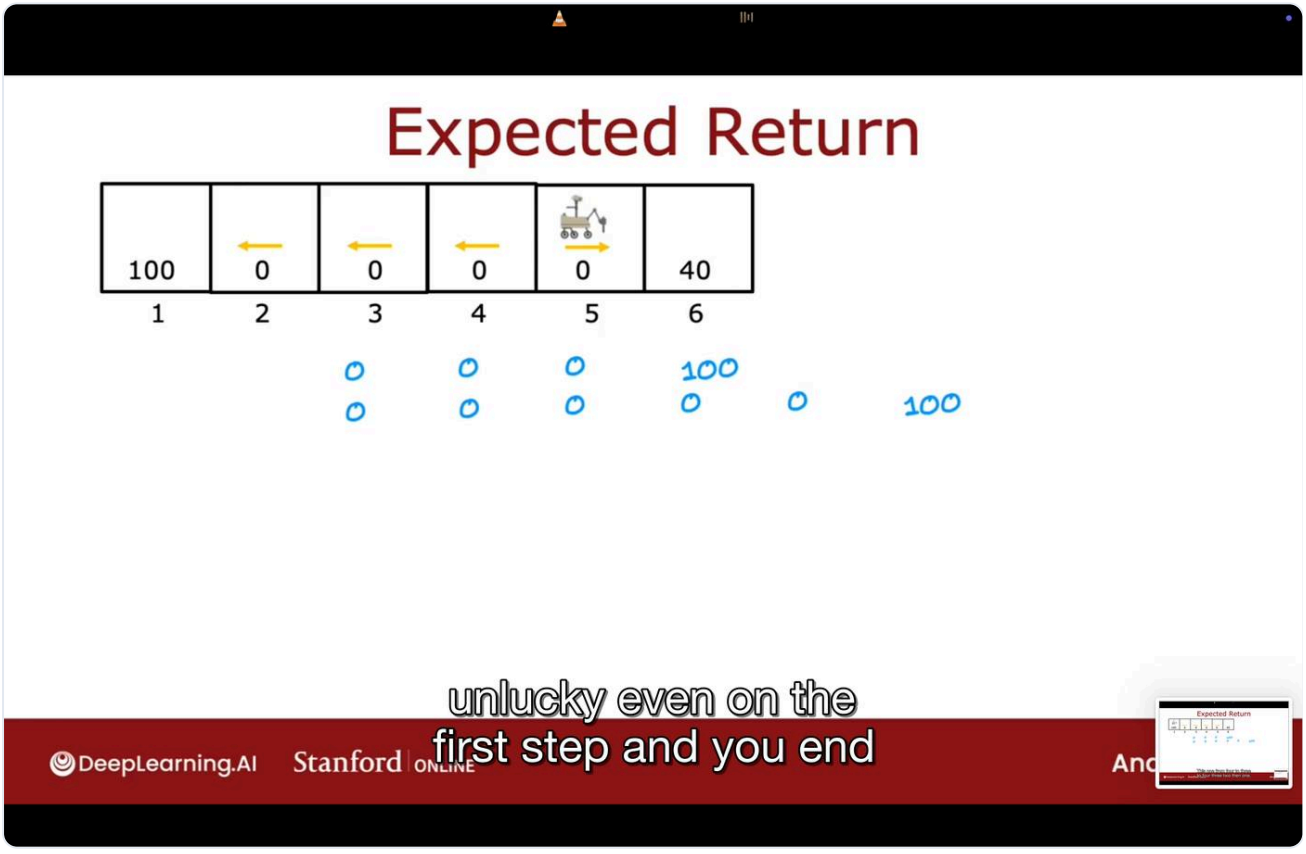


FIGURE 37.15 | Expected Return

This one from four to three to four three two then one, or is even possible, if you tell from state four to go left following the policy you may get unlucky even on the first step and you end up going to state five because it slipped.

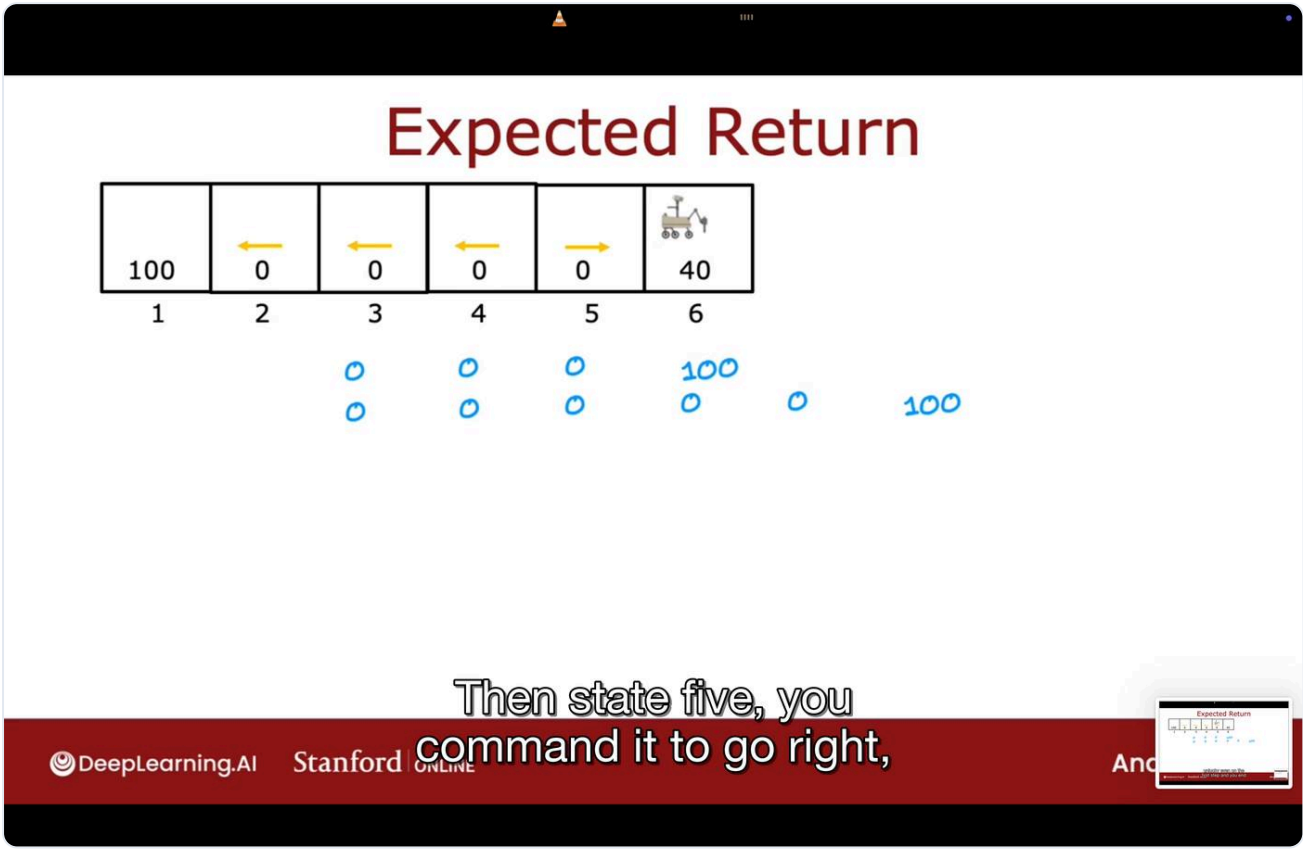


FIGURE 37.16 | Expected Return

Then state five, you command it to go right, and it succeeds as you end up here.

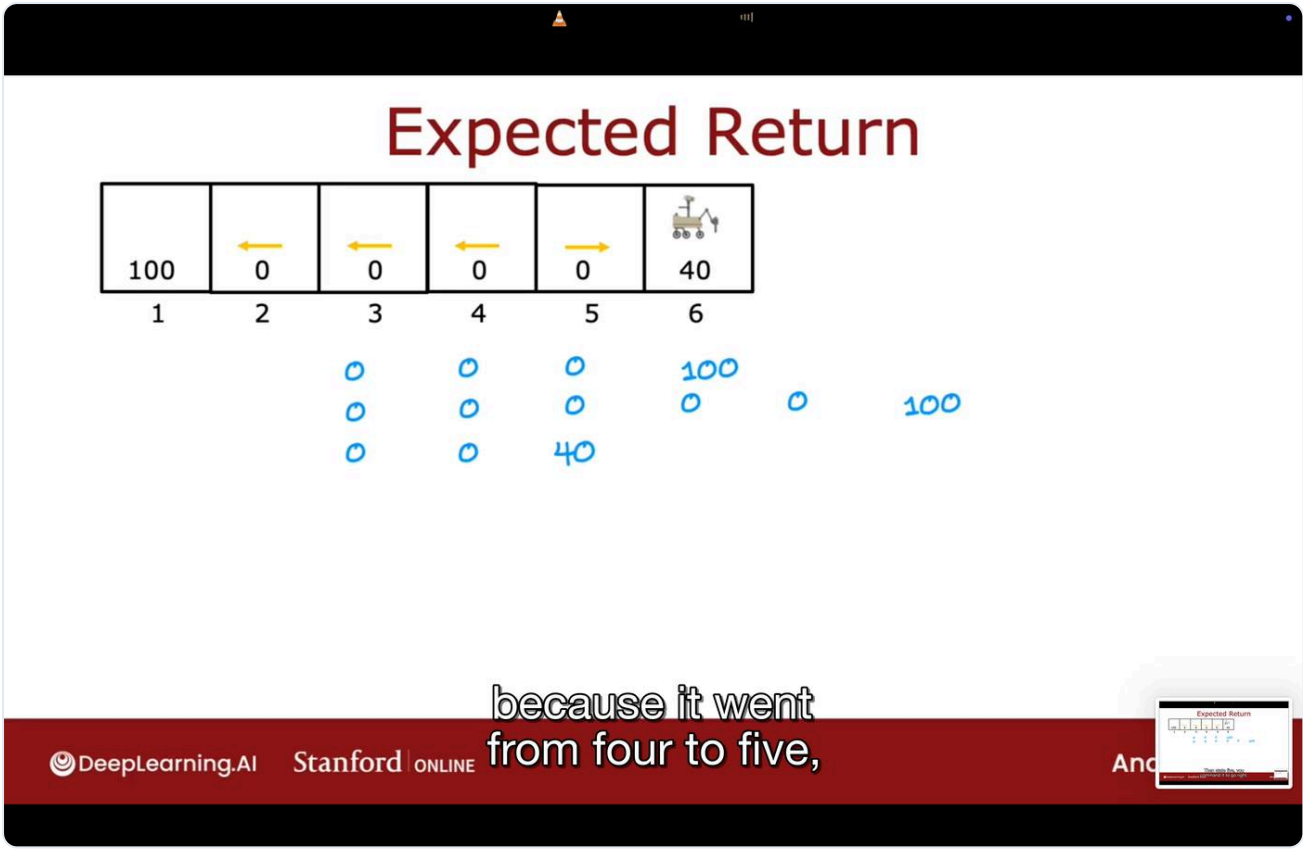


FIGURE 37.17 | Expected Return

In this case, the sequence of rewards you see will be 0040, because it went from four to five, and then states six, we had previously written out the return as this sum of discounted rewards.

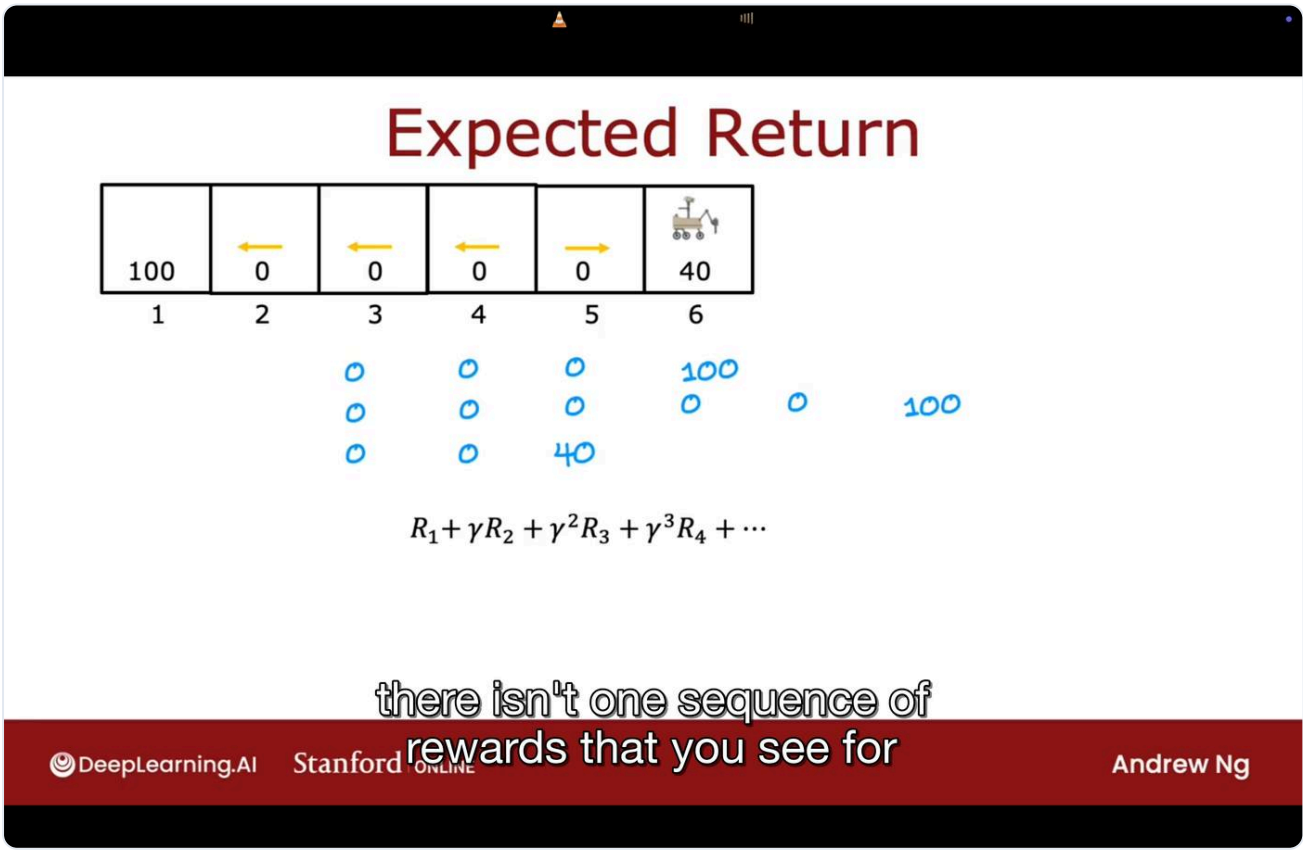


FIGURE 37.18 | Expected Return

But when the reinforcement learning problem is stochastic, there isn't one sequence of rewards that you see for sure instead you see this sequence of different rewards. In a stochastic reinforcement learning problem, what we're interested in is not maximizing the return because that's a random number.

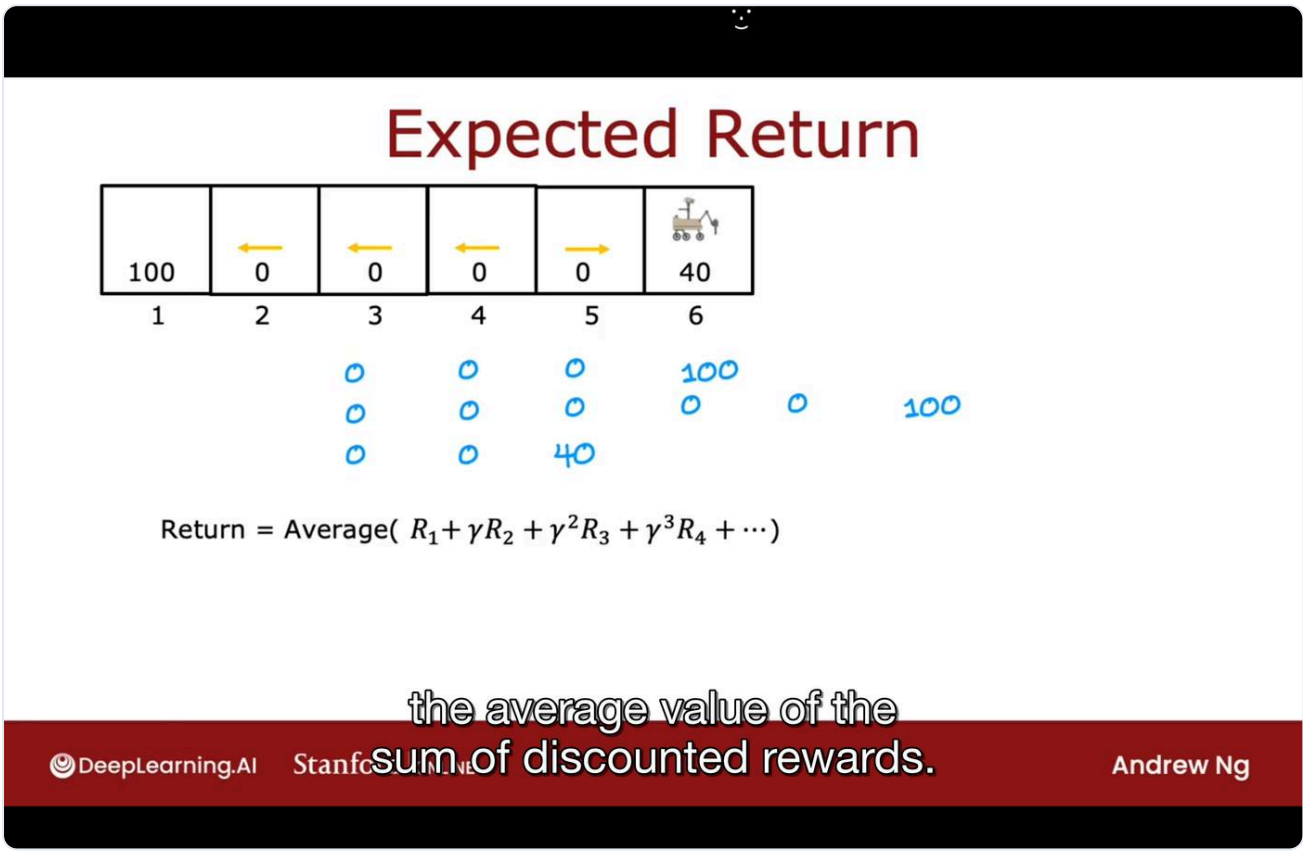


FIGURE 37.19 | Expected Return

What we're interested in is maximizing the average value of the sum of discounted rewards. By average value, I mean if you were to take your policy and try it out a thousand times or a 100,000 times or a million times, you get lots of different reward sequences like that and if you were to take the average over all of these different sequences of the sum of discounted rewards, then that's what we call the expected return.

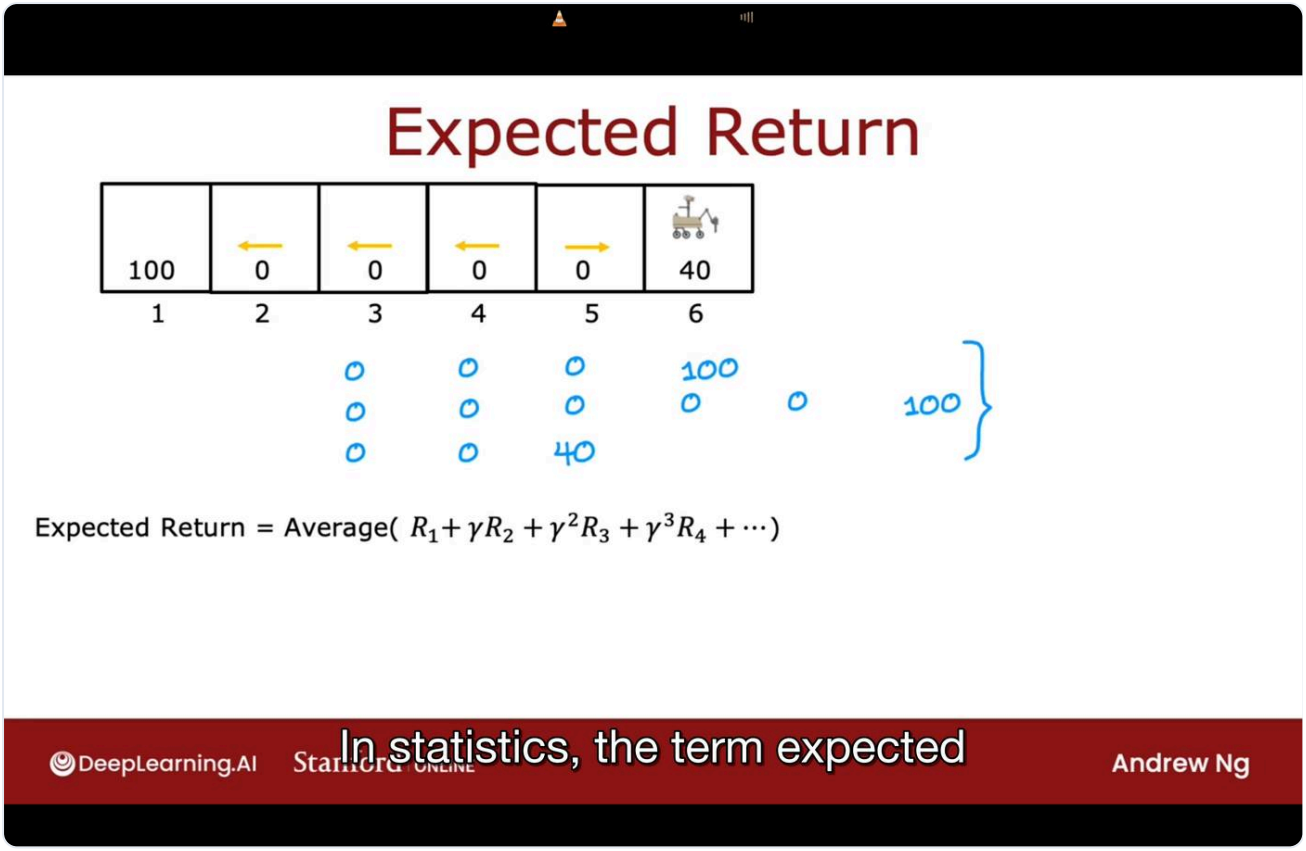


FIGURE 37.20 | Expected Return

In statistics, the term expected is just another way of saying average. But what this means is we want to maximize what we expect to get on average in terms of the sum of discounted rewards. The mathematical notation for this is to write this as E.

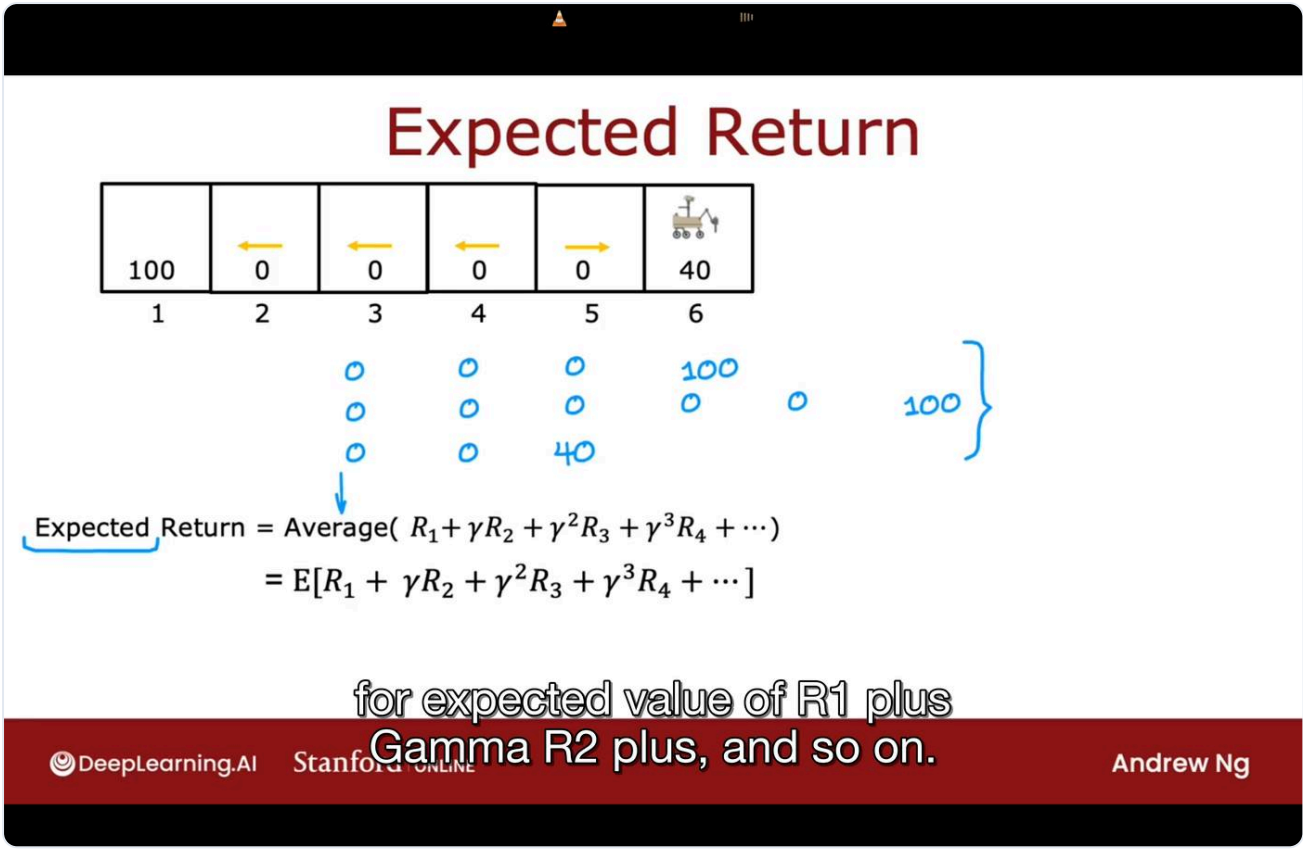


FIGURE 37.21 | Expected Return

E stands for expected value of R1 plus Gamma R2 plus, and so on. The job of reinforcement learning algorithm is to choose a policy Pi to maximize the average or the expected sum of discounted rewards.

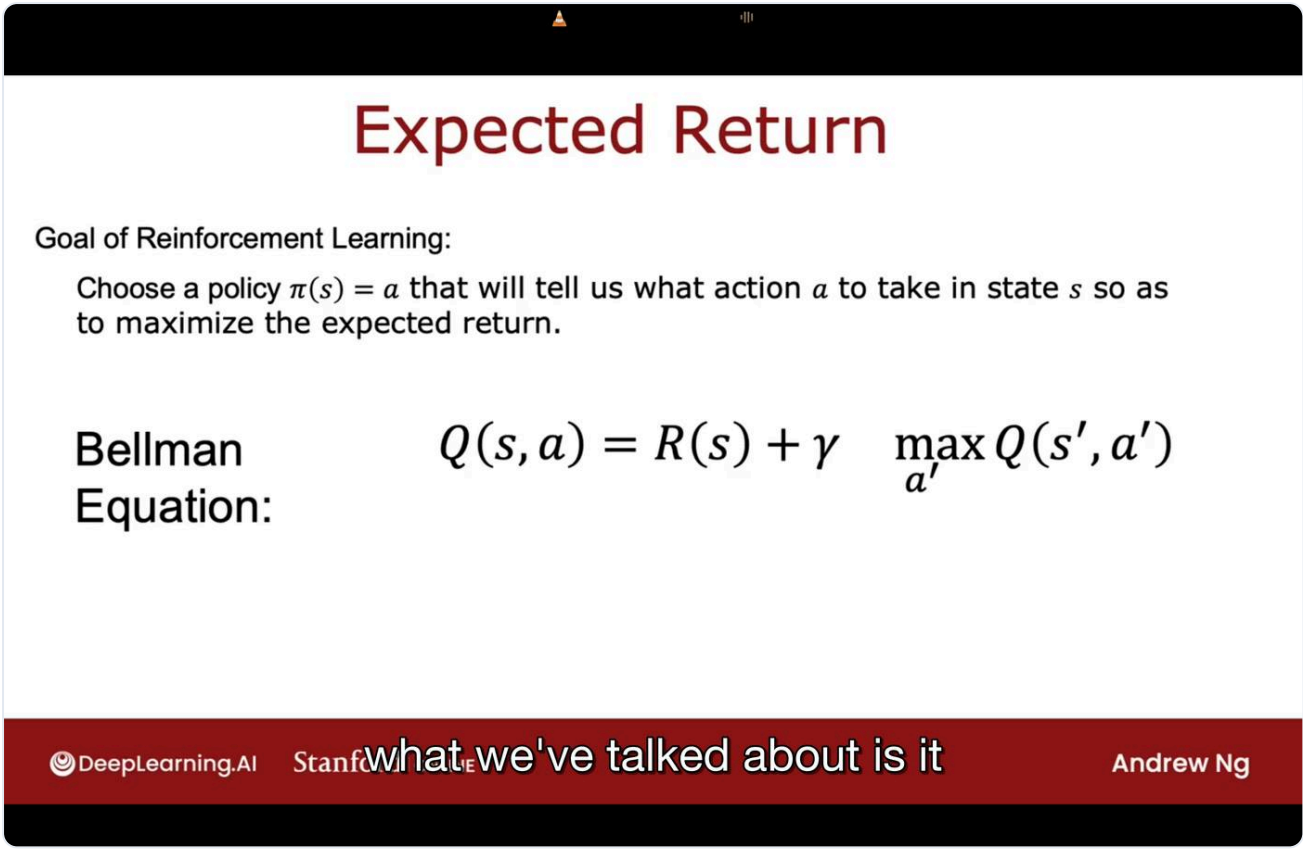


FIGURE 37.22 | Expected Return

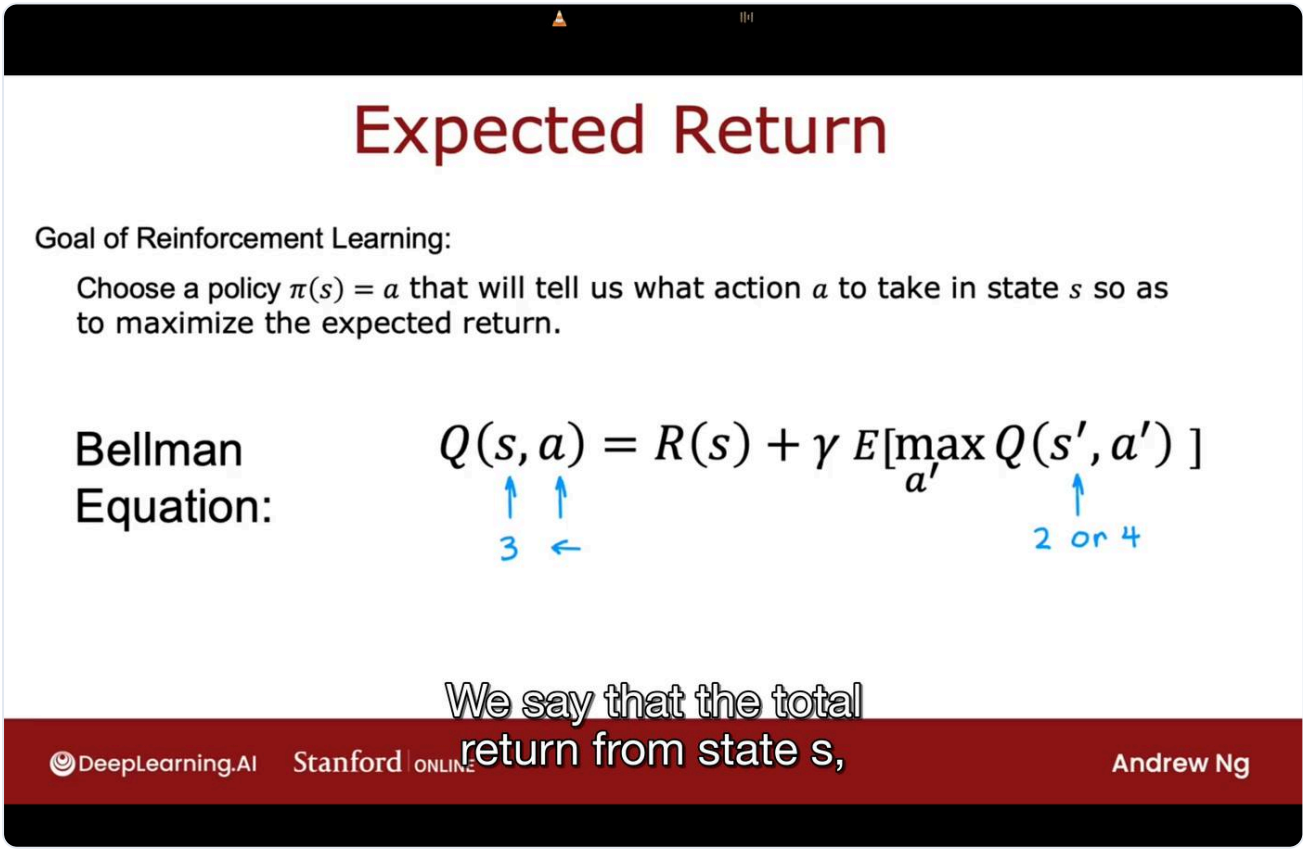


FIGURE 37.23 | Expected Return

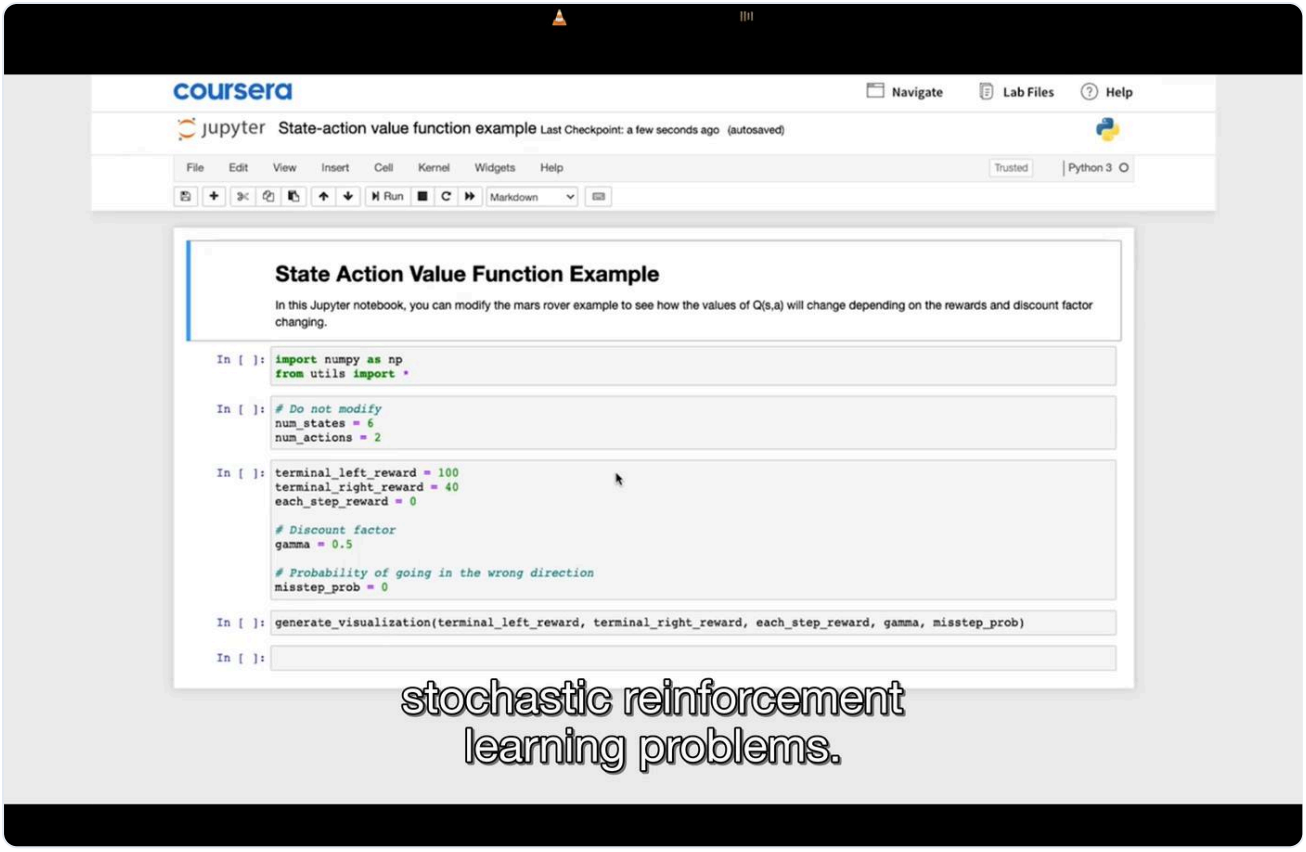


FIGURE 37.24

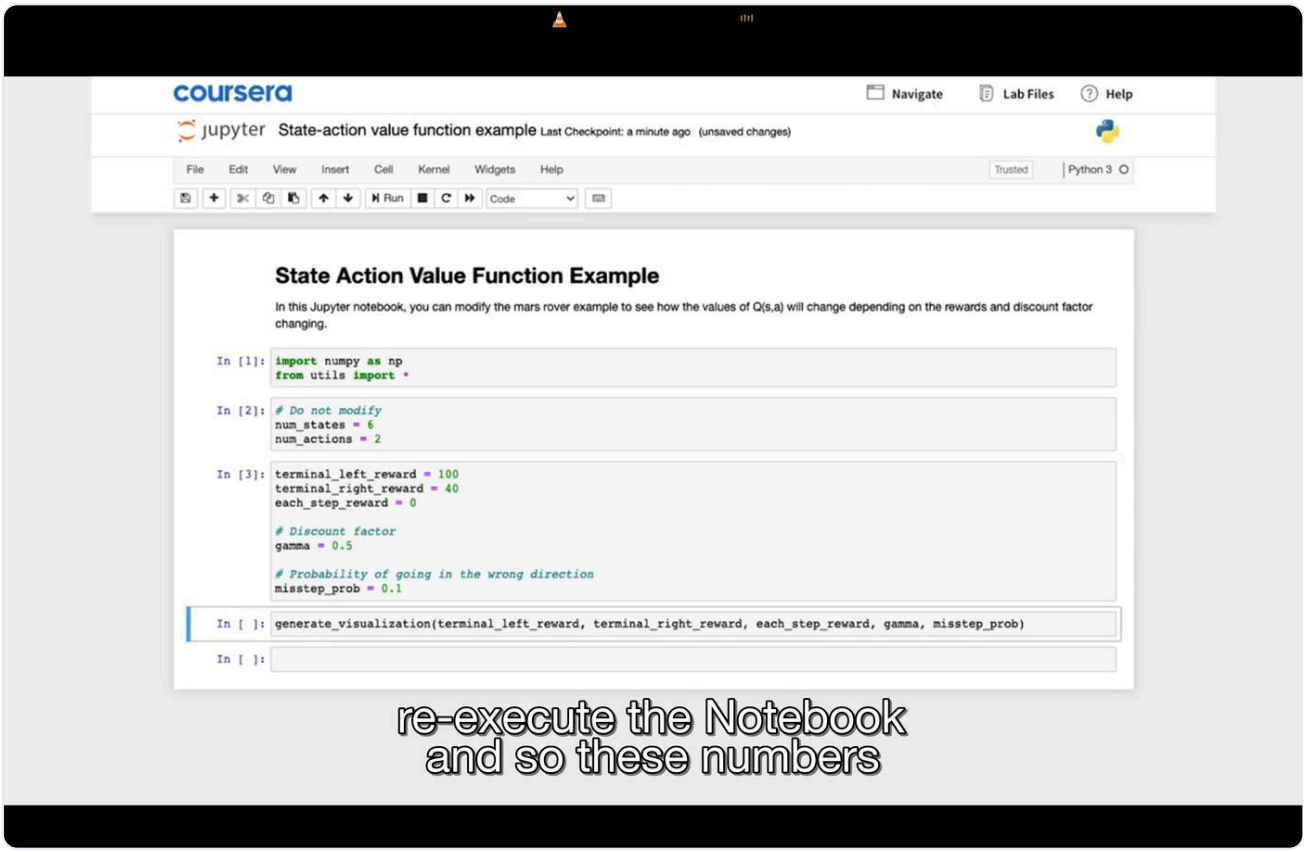


FIGURE 37.25

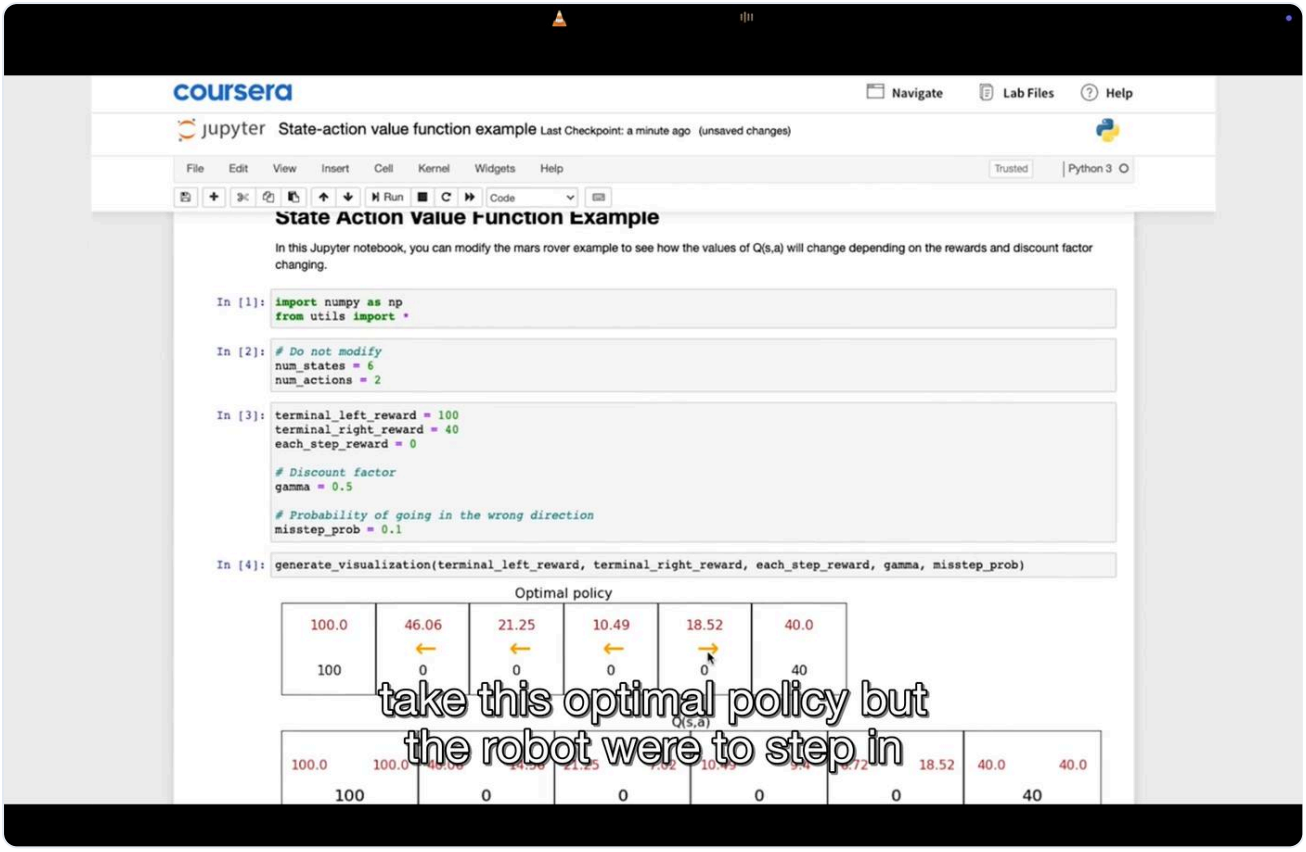


FIGURE 37.26

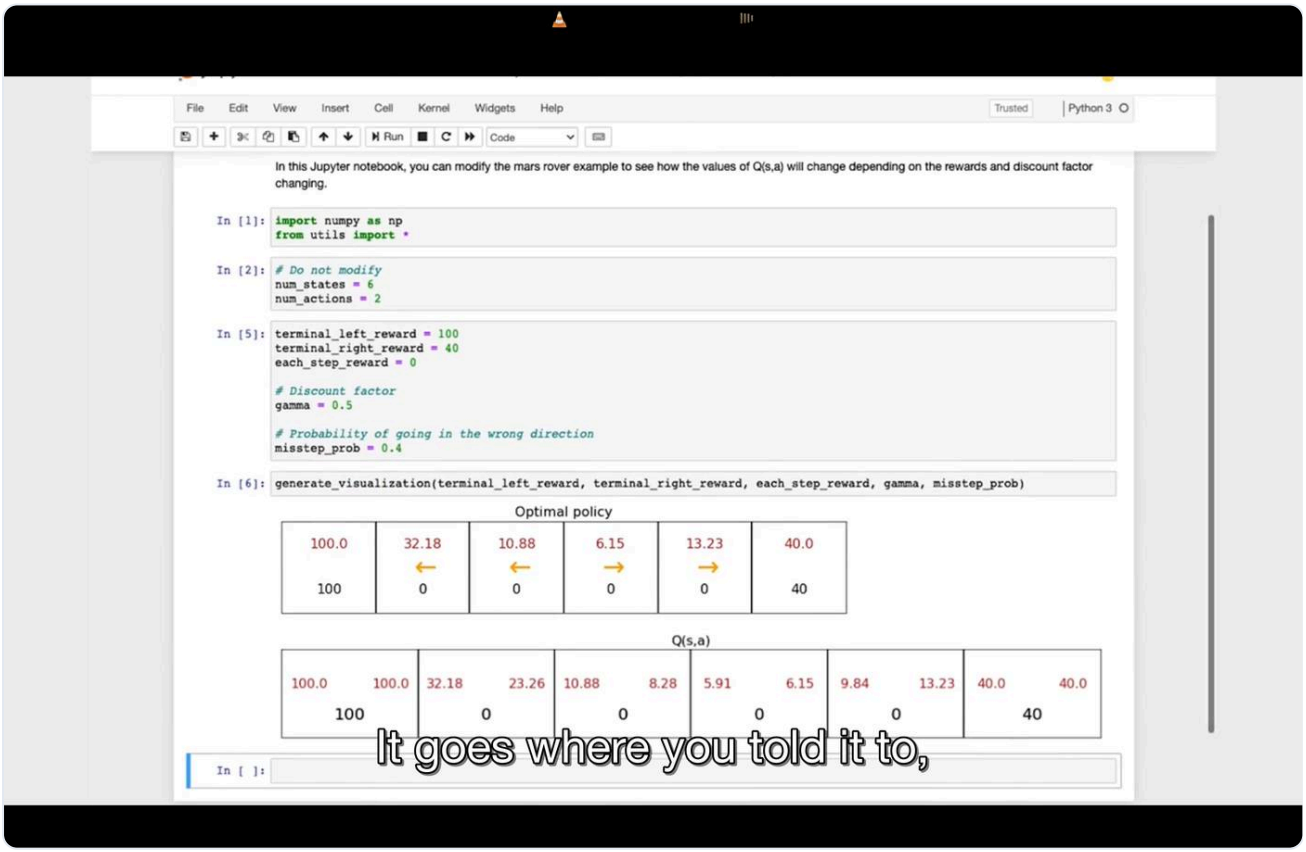


FIGURE 37.27

To summarize, when you have a stochastic reinforcement learning problem or a stochastic Markov decision process the goal is to choose a policy to tell us what action to take in state S so as to maximize the expected return. The last way that this changes, what we've talked about is it modifies Bellman equation a little bit. Here's the Bellman equation exactly as we've written down. But the difference now is that when you take the action a in state s, the next state s prime you get to is random. When you're in state 3 and you tell it to go left the next state s prime it could be the state 2, or it could be the state 4. S prime is now random, which is why we also put an average operator or unexpected operator here.

We say that the total return from state s, taking action a, once in a behaving optimally, is equal to the reward you get right away, also called the immediate reward plus the discount factor, Gamma plus what you expect to get on average of the future returns. If you want to sharpen your intuition about what happens with these stochastic reinforcement learning problems. You'd go back to the optional lab that I had shown you just now, where this parameter misstep problem is the probability of your Mars Rover going in the opposite direction, than you had commanded it to. If we said misstep prop two is 0.1 and re-execute the Notebook and so these numbers up here are the optimal return if you were to take the best possible actions, take this optimal policy but the robot were to step in the wrong direction 10 percent of the time and these are the q values for this stochastic NTP.

Notice that these values are now a little bit lower because you can't control the robot as well as before. The q values, as well as the optimal returns, have gone down a bit. In fact, if you were to increase the misstep probability, say 40 percent of the time the robot doesn't even go into directions. You're commanding it to only 60 percent of the time. It goes where you told it to, then these values end up even lower because your degree of control over the robot has decreased. I encourage you to play with the optional lab and change the value of the misstep probability and see how that affects the for return or the auto expected return, as well as the Q values, q of s a.

Now, in everything we've done so far, we've been using this Markov decision process, this Mars rover with just six states. For many practical applications, the number of states will be much larger. In the next video, we'll take the reinforcement learning or Markov decision process framework we've talked about so far and generalize it to this much richer and maybe even more interesting set of problems with much larger and in particular with continuous state spaces, let's take a look at that in the next video.

LECTURE 38

Discrete vs Continuous State

Many robotic control applications, including the lunar lander application that you work on in the practice lab, have continuous state spaces. Let's take a look at what that means and how to generalize the concept we've talked about to these continuous state spaces. A simplify Mars rover example we use, I use a discrete set of states, and what that means is that simplify Mars rover could only be in one of six possible positions. But most robots can be in more than one of six or any discrete number of positions, instead, they can be in any of a very large number of continuous value positions. For example, if the Mars rover could be anywhere on a line, so its position was indicated by a number ranging from 0-6 kilometers where any number in between is valid.

That would be an example of a continuous state space, because the position would be represented by a number such as that is 2.7 kilometers along or 4.8 kilometers or any other number between zero and six. Let's look at another example. I'm going to use for this example, the application of controlling a car or a truck.

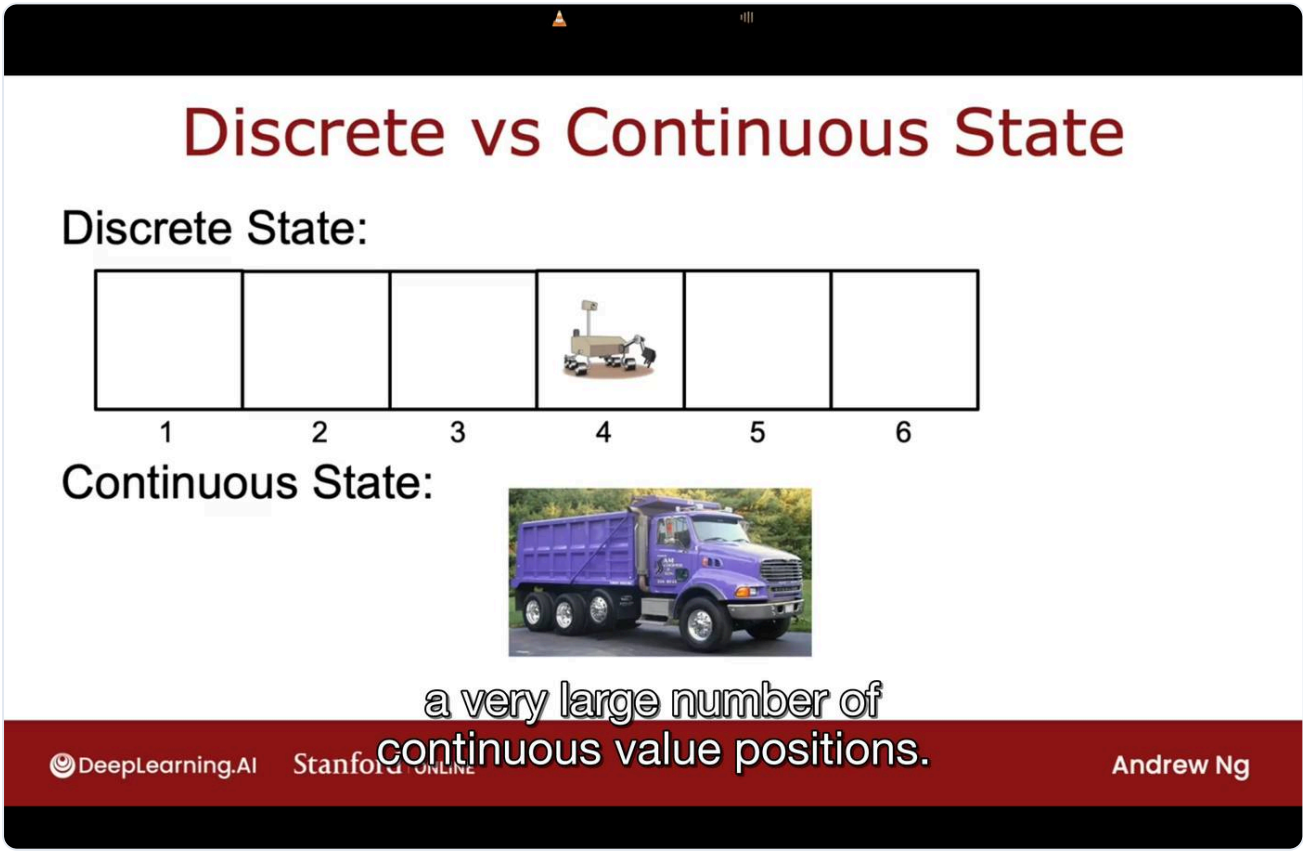


FIGURE 38.1 | Discrete vs Continuous State

Here's a toy car, or actually a toy truck. This one belongs to my daughter. If you're building a self-driving car or self-driving truck and you want to control this to drive smoothly, then the state of this truck might include a few numbers such as this, x position, its y position, maybe it's orientation. What way is it facing? Assuming the truck stays on the ground, you probably don't need to worry about how tall is, how high up it is. This is state will include x, y, and is angle Theta, as

well as maybe its speeds in x-direction, the speed in the y-direction, and how quickly it's turning. Is it turning at one degree per second or is it turning at 30 degrees per second or is it turning really quickly at 90 degrees per second?

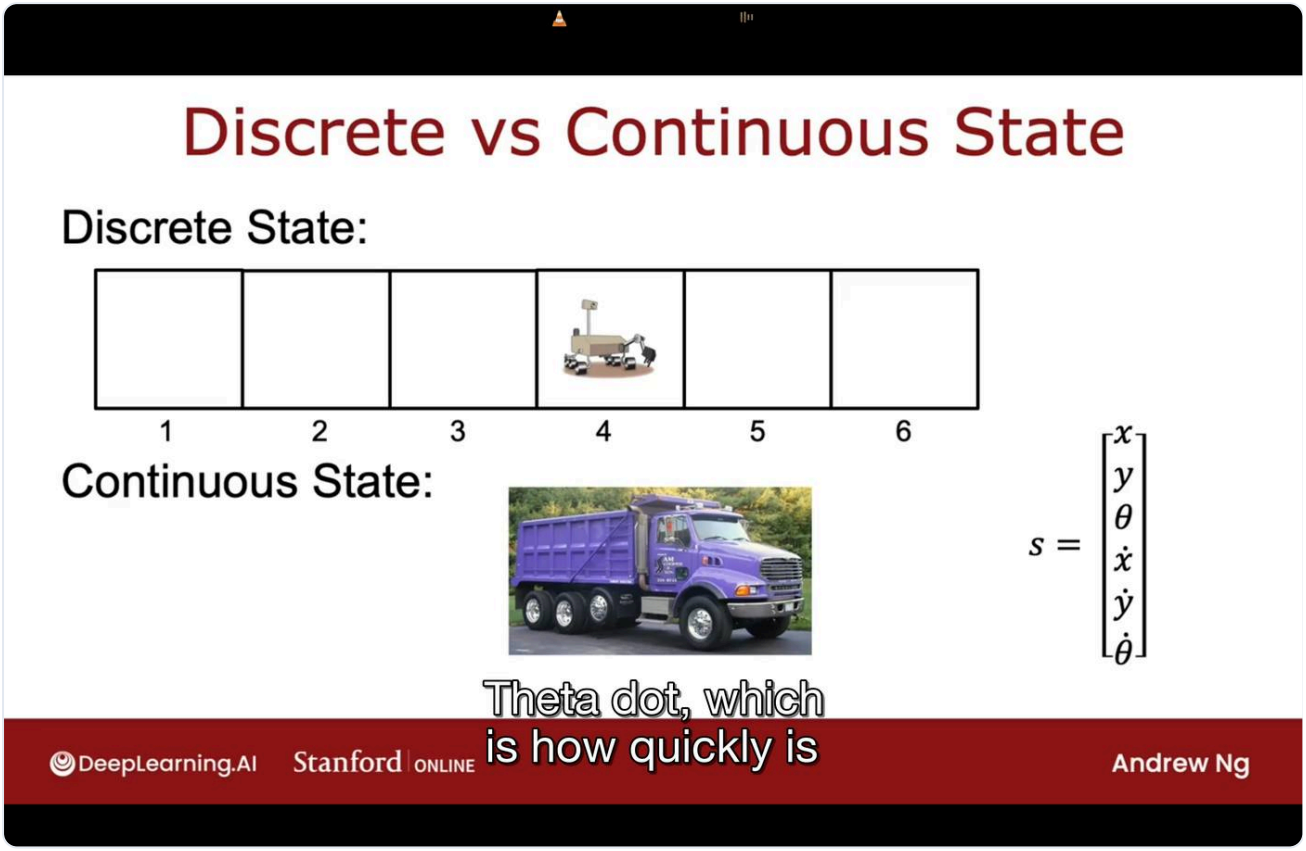


FIGURE 38.2 | Discrete vs Continuous State

For a truck or a car, the state might include not just one number, like how many kilometers of this along on this line, but they might includes six numbers, is x position, is y position, is orientation, which I'm going to denote using Greek alphabet Theta, as well its velocity in the x-direction, which I will denote using x dot, so that means how quickly is this x-coordinate changing, y dot how quickly is the y coordinate changing, and then finally, Theta dot, which is how quickly is the angle of the car changing. Whereas for the 60 Mars rover example, the state was just one of six possible numbers. It could be one, two, three, four, five or six.

For the car, the state would comprise this vector of six numbers, and any of these numbers can take on any value within is valid range. For example, Theta should range between zero and 360 degrees. Let's look at another example. What if you're building a reinforcement learning algorithm to control an autonomous helicopter, how would you characterize the position of a helicopter? To illustrate, I have with me here a small toy helicopter. The positioning of the helicopter would include is x position, such as how far north or south is a helicopter, is y position. Maybe how far on the east-west axis is the helicopter, and then also z, the height of the helicopter above ground.

But other than the position, the helicopter also has an orientation, and conventionally, one way to capture its orientation is with three additional numbers, one of which captures of the row of the helicopter. Is it rolling to the left or the right? The pitch, is it pitching forward or pitching up,

pitching back, and then finally the yaw which is west the compass orientation is it facing. If facing north or east or south or west? To summarize, the state of the helicopter includes is position in the say, north-south direction, is positioned in the east-west direction, y is height above ground, and also the row, the pitch, and also that yaw of helicopter.

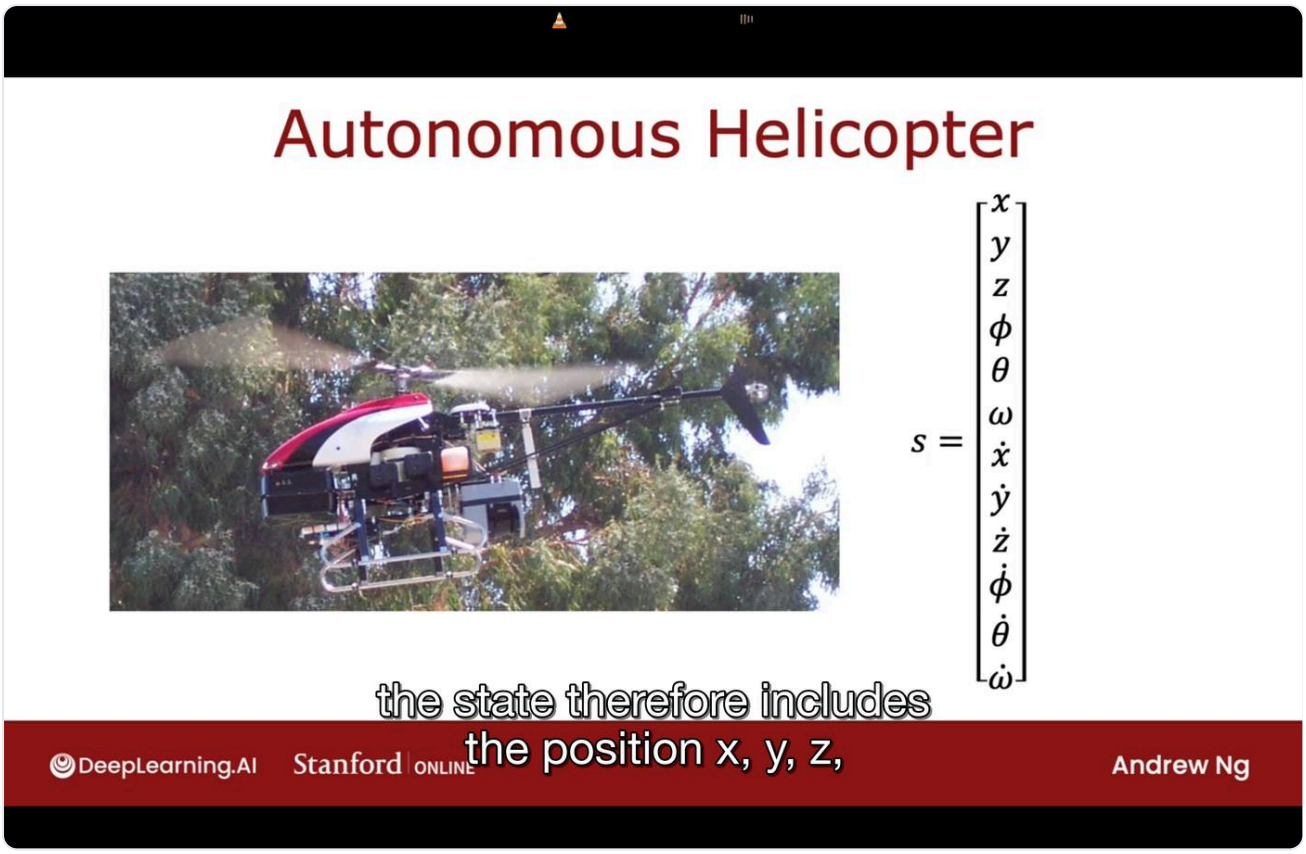


FIGURE 38.3 | Autonomous Helicopter

To write this down, the state therefore includes the position x, y, z, and then the row pitch, and yaw denoted with the Greek alphabets Phi, Theta and Omega. But to control the helicopter, we also need to know its speed in the x-direction, in the y-direction, and in the z direction, as well as its rate of turning, also called the angular velocity. How fast is this row changing and how fast is this pitch changing and how fast is its yaw changing? This is actually the state used to control autonomous helicopters. Is this list of 12 numbers that is input to a policy, and the job of a policy is look at these 12 numbers and decide what's an appropriate action to take in the helicopter.

So any continuous state reinforcement learning problem or a continuous state Markov decision process, continuously MTP. The state of the problem isn't just one of a small number of possible discrete values, like a number from 1-6. Instead, it's a vector of numbers, any of which could take any of a large number of values. In the practice lab for this week, you get to implement for yourself a reinforcement learning algorithm applied to a simulated lunar lander application. Landing something on the moon is simulation. Let's take a look in the next video at what that application entails, since there will be another continuous state application.

LECTURE 39

Lunar Lander

The lunar lander lets you land a simulated vehicle on the moon. It's like a fun little video game that's been used by a lot of reinforcement learning researchers. Let's take a look at what it is. In this application you're in command of a lunar lander that is rapidly approaching the surface of the moon. And your job is the fire thrusters at the appropriate times to land it safely on the landing pad. To give you a sense of what it looks like.

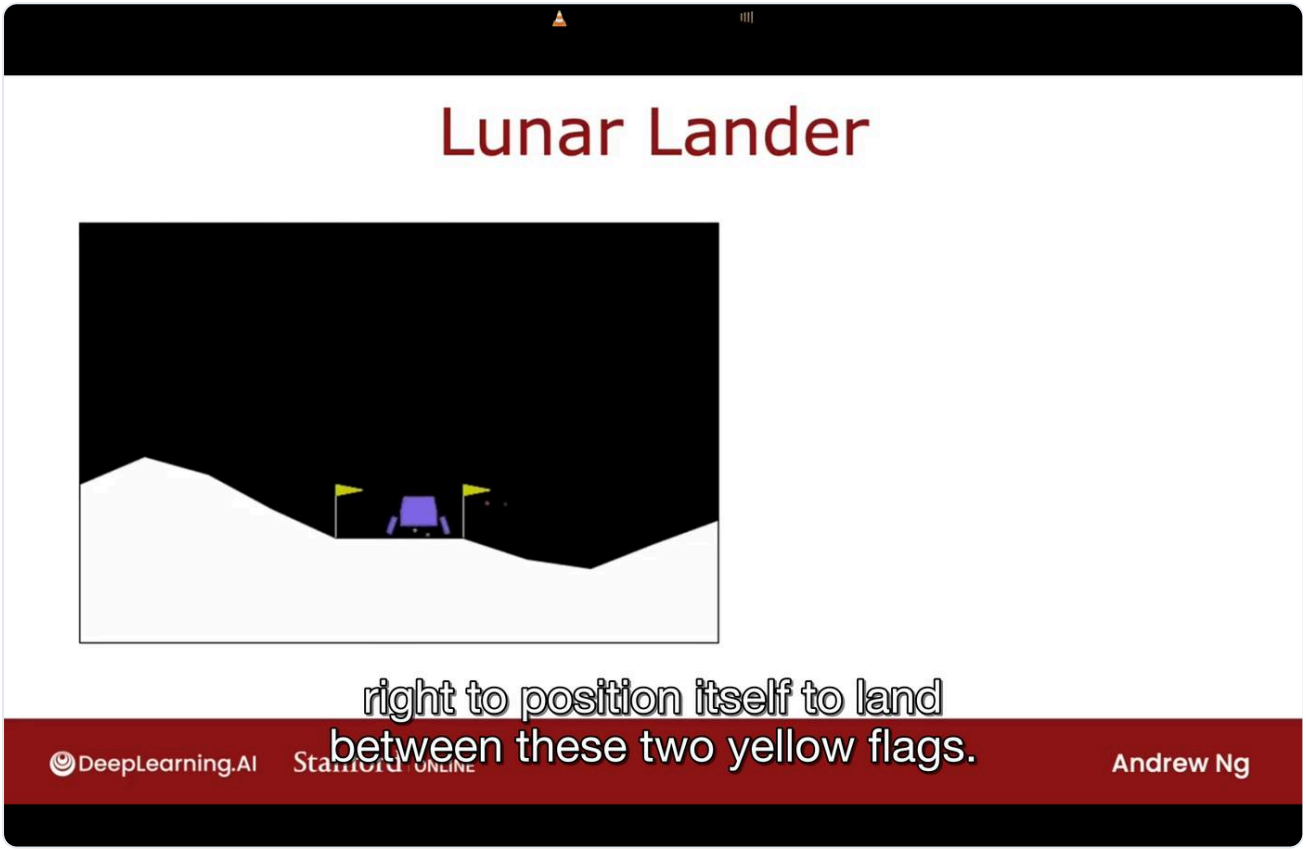


FIGURE 39.1 | Lunar Lander

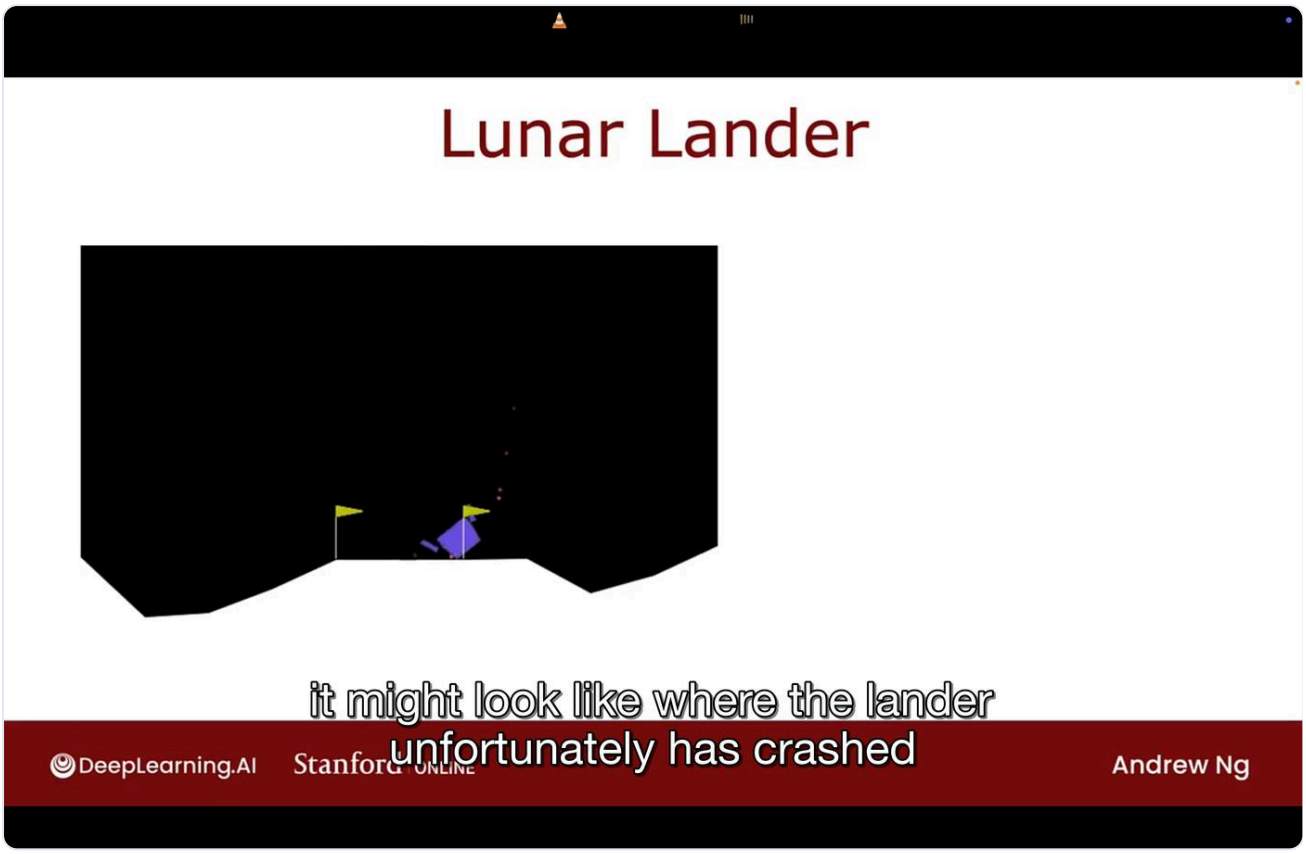


FIGURE 39.2 | Lunar Lander

This is the lunar lander landing successfully and it's firing thrusters downward and to the left and right to position itself to land between these two yellow flags.

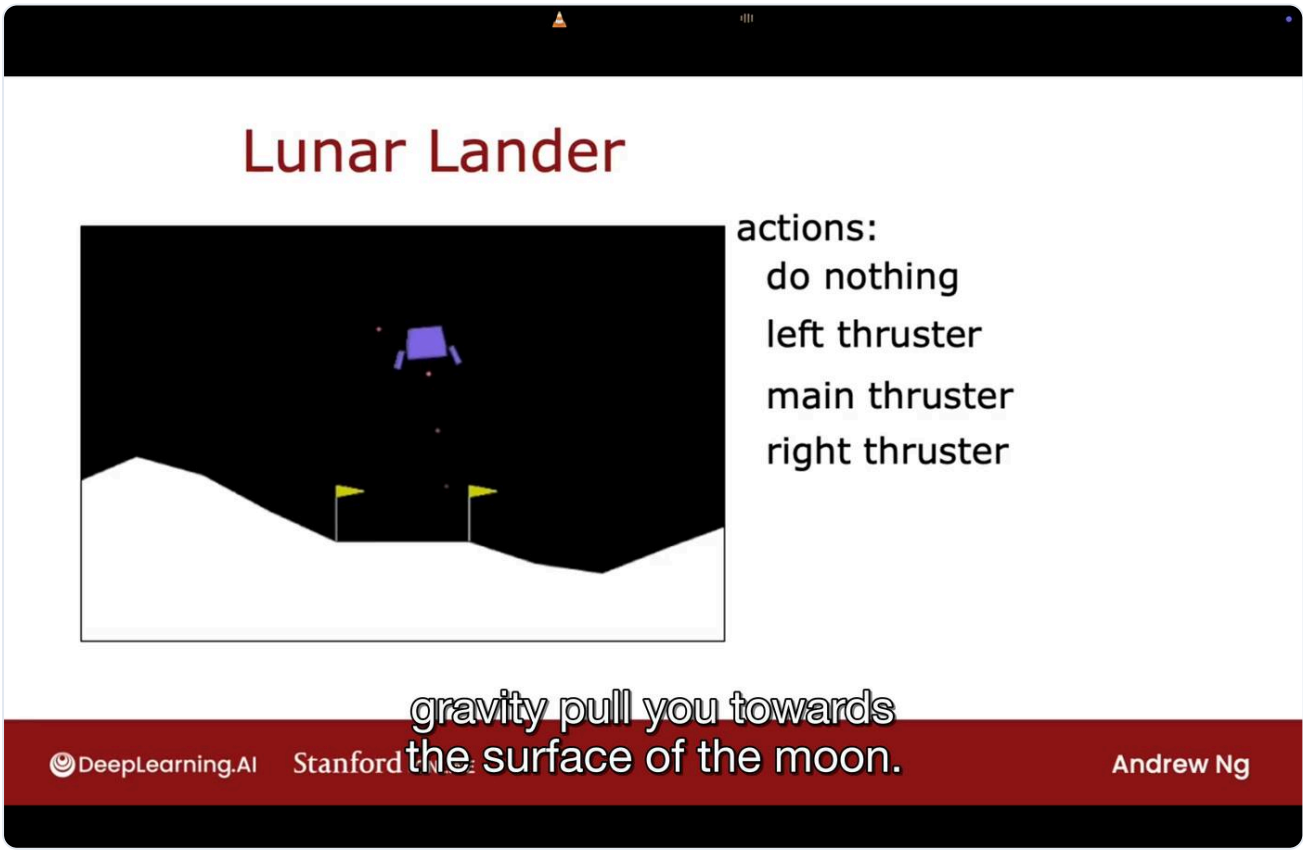


FIGURE 39.3 | Lunar Lander

Or if the reinforcement landing algorithm policy does not do well then this is what it might look like where the lander unfortunately has crashed on the surface of the moon. In this application you have four possible actions on every time step. You could either do nothing, in which case the forces of inertia and gravity pull you towards the surface of the moon. Or you can fire a left thruster when you see a little red dot come out on the left, that's firing the left.

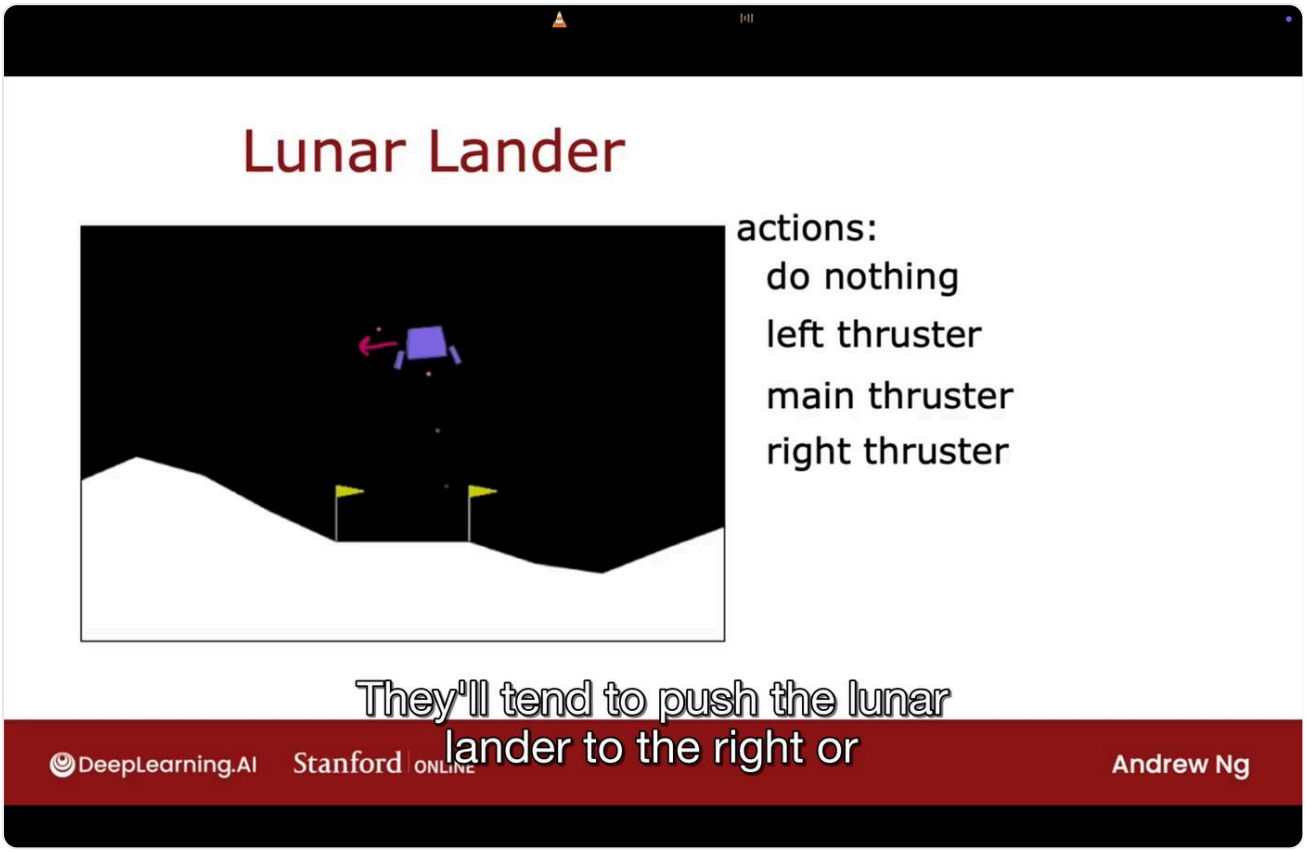


FIGURE 39.4 | Lunar Lander

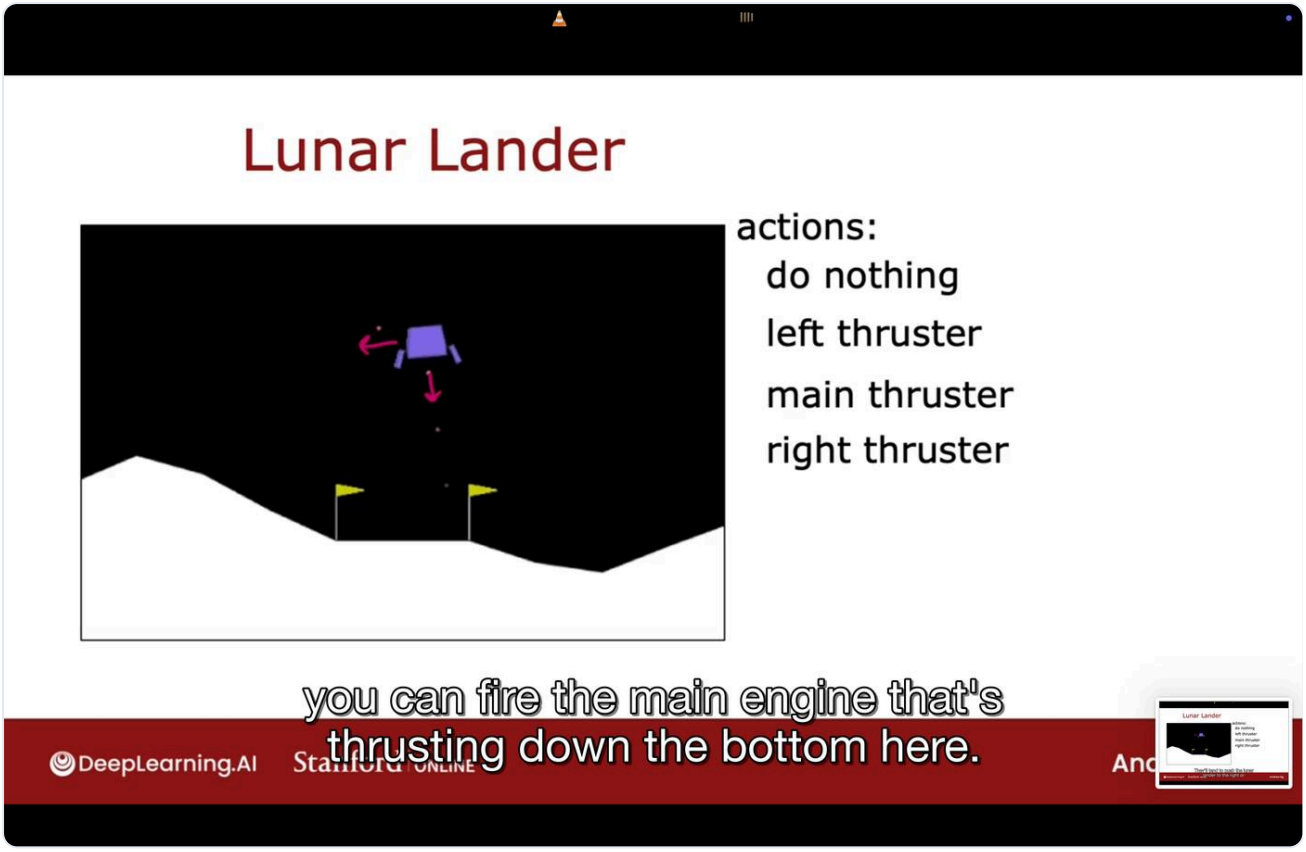


FIGURE 39.5 | Lunar Lander

They'll tend to push the lunar lander to the right or you can fire the main engine that's thrusting down the bottom here.

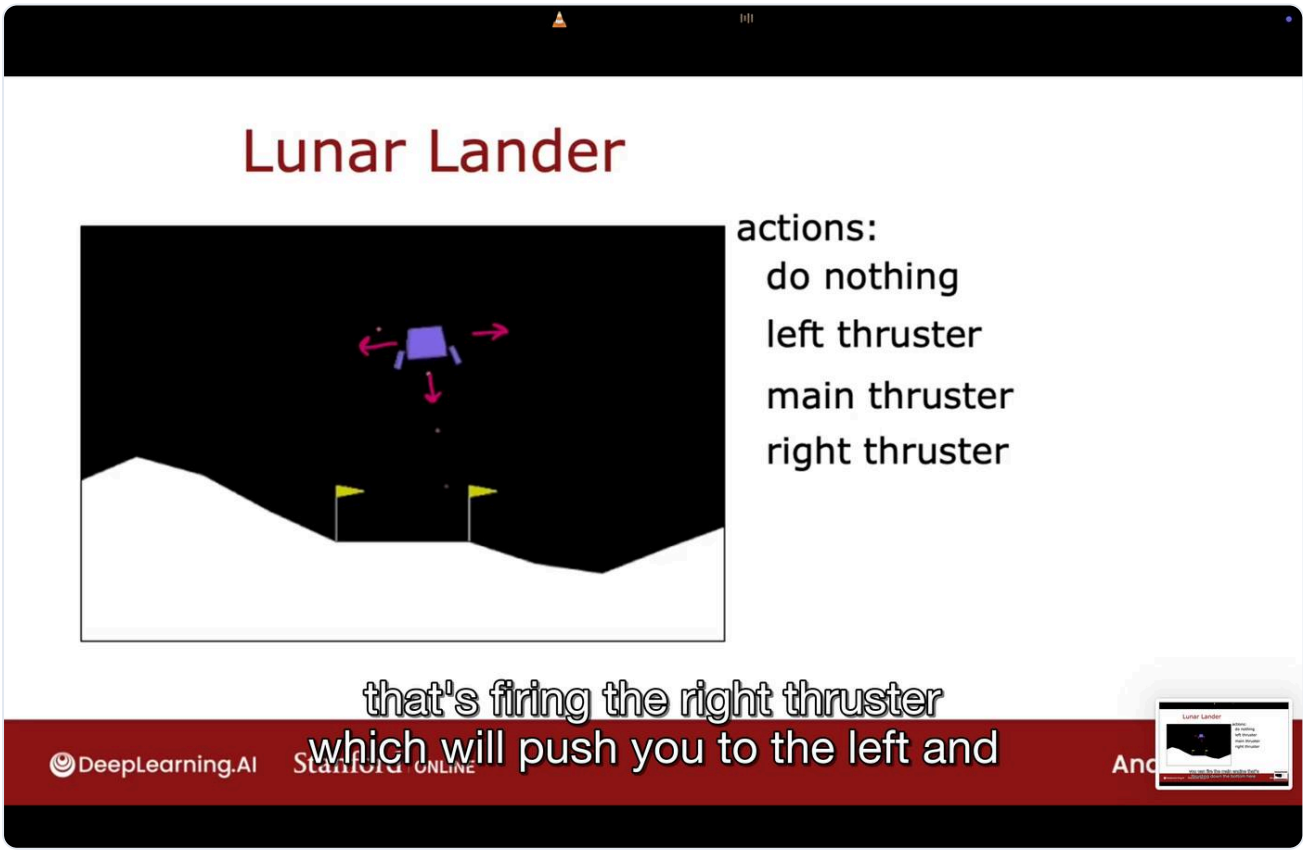


FIGURE 39.6 | Lunar Lander

Or you can fire the right thruster and that's firing the right thruster which will push you to the left and your job is to keep on picking actions over time. So it's the lunar lander safely between these two flags here on the landing pad. In order to give the actions a shorter name I'm sometimes going to call the actions nothing meaning do nothing or left meaning fire left thruster or main meaning fire the main engine downward or right. So I'm going to call the actions nothing left. main and right for short later in this video. How about the states space of this? Mtp the states are its position X and Y. So how far to the left or right and how high up is it as well as velocity x.y how fast is it moving in the horizontal and vertical directions and then also is angle.

So how far is the lunar lander tilted to the left or tilted to the right? Is angular velocity theta dot. And then finally, because a small difference in positioning makes a big difference in whether or not it's landed. We're going to have two other variables in the state vector which we call l and r.

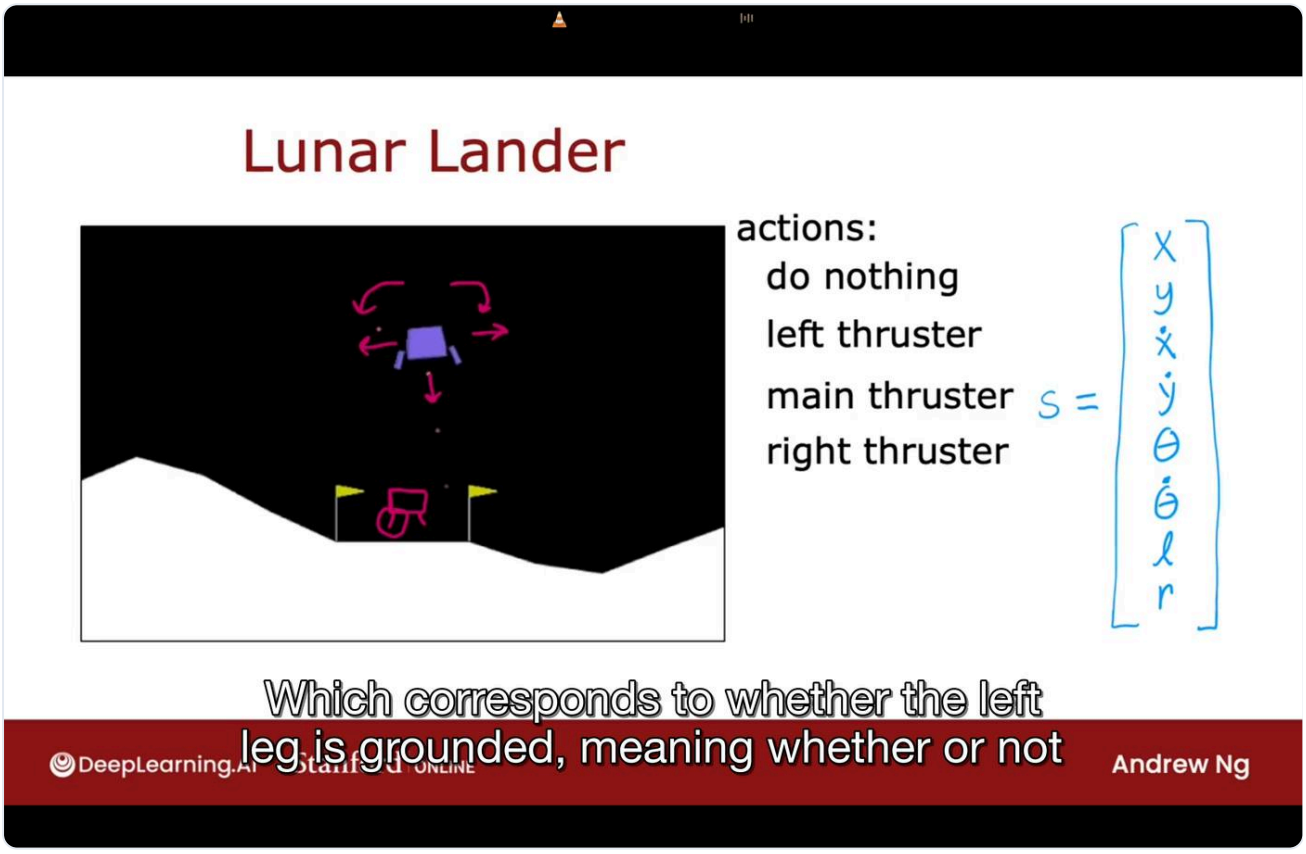


FIGURE 39.7 | Lunar Lander

Which corresponds to whether the left leg is grounded, meaning whether or not the left leg is sitting on the ground as well as r which corresponds to whether or not the right leg is sitting on the ground. So whereas xy x.theta theta.our numbers l and r will be binary valued and can take on only values zero or one depending on whether the left and right legs are touching the ground. Finally his reward function for the lunar lander. If it manages to get to the landing pad, didn't receive the reward between 100 and 140 depending on how well it's flown when gotten to the center of the landing pad.

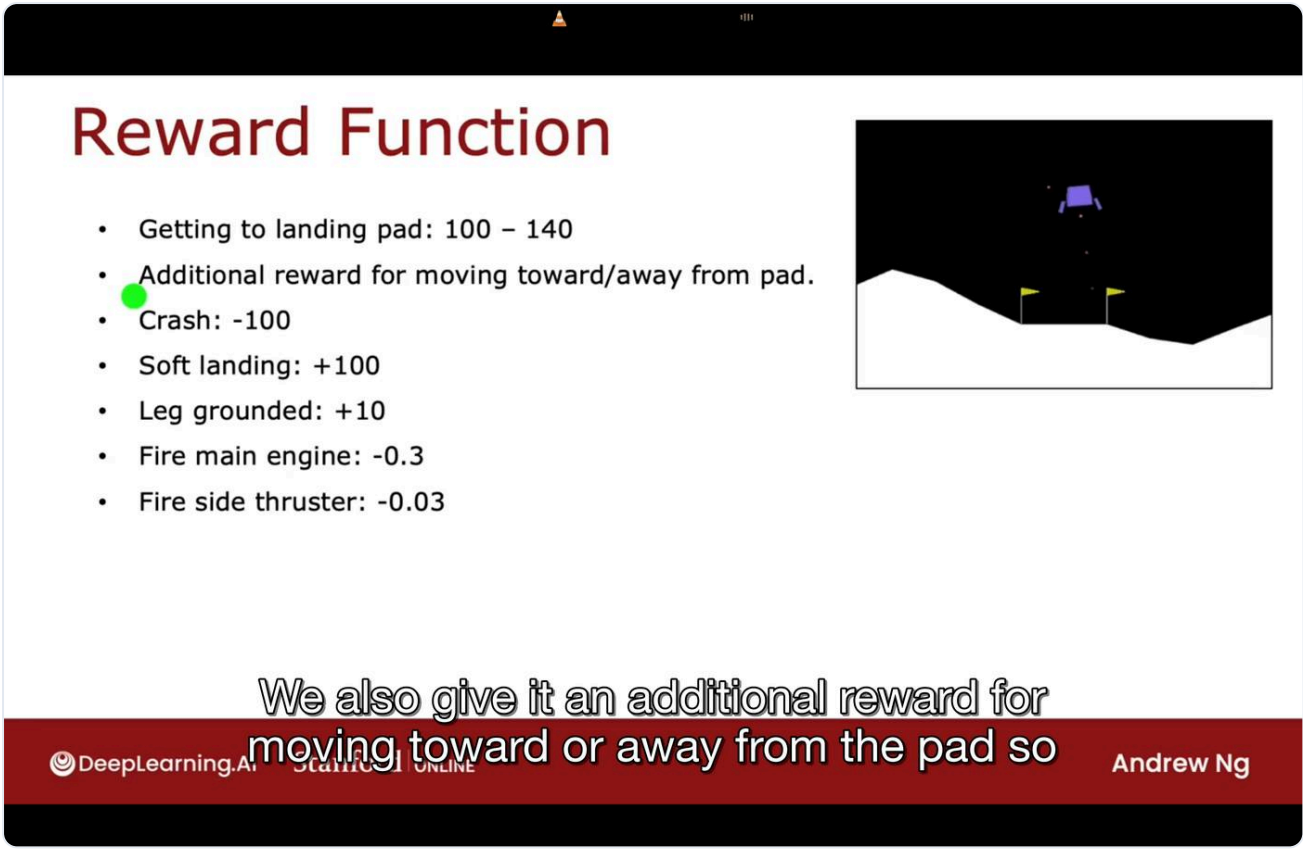


FIGURE 39.8 | Reward Function

We also give it an additional reward for moving toward or away from the pad so it moves closer to the pad it receives a positive reward and it moves away and drifts away. It receives a negative reward. If it crashes it gets a large -100 reward, it achieves a soft landing, that is a landing. There's another crash, it gets a +100 reward for each leg, the left leg or the right link that it gets grounded. It receives a +10 reward and finally to encourage it not to waste too much fuel and fire thrusters aren't necessarily. Every time it fires the main engine we give it a -0.3 rewards and every time it fires the left or the right side thrusters we give it a -0.03 reward.

Notice that this is a moderately complex reward function. The designers of the lunar lander application actually put some thought into exactly what behavior you want and codified it in the reward function. To incentivize more of the behaviors you want and fewer of the behaviors like crashing that you don't want. You find when you're building your own reinforcement learning application usually takes some thought to specify exactly what you want or don't want and to codify that in the reward function. But specify the reward function should still turn out to be much easier to specify the exact right action to take from every single state. Which is much harder for this and many other reinforcement learning applications.

So the lunar lander problem is as follows.

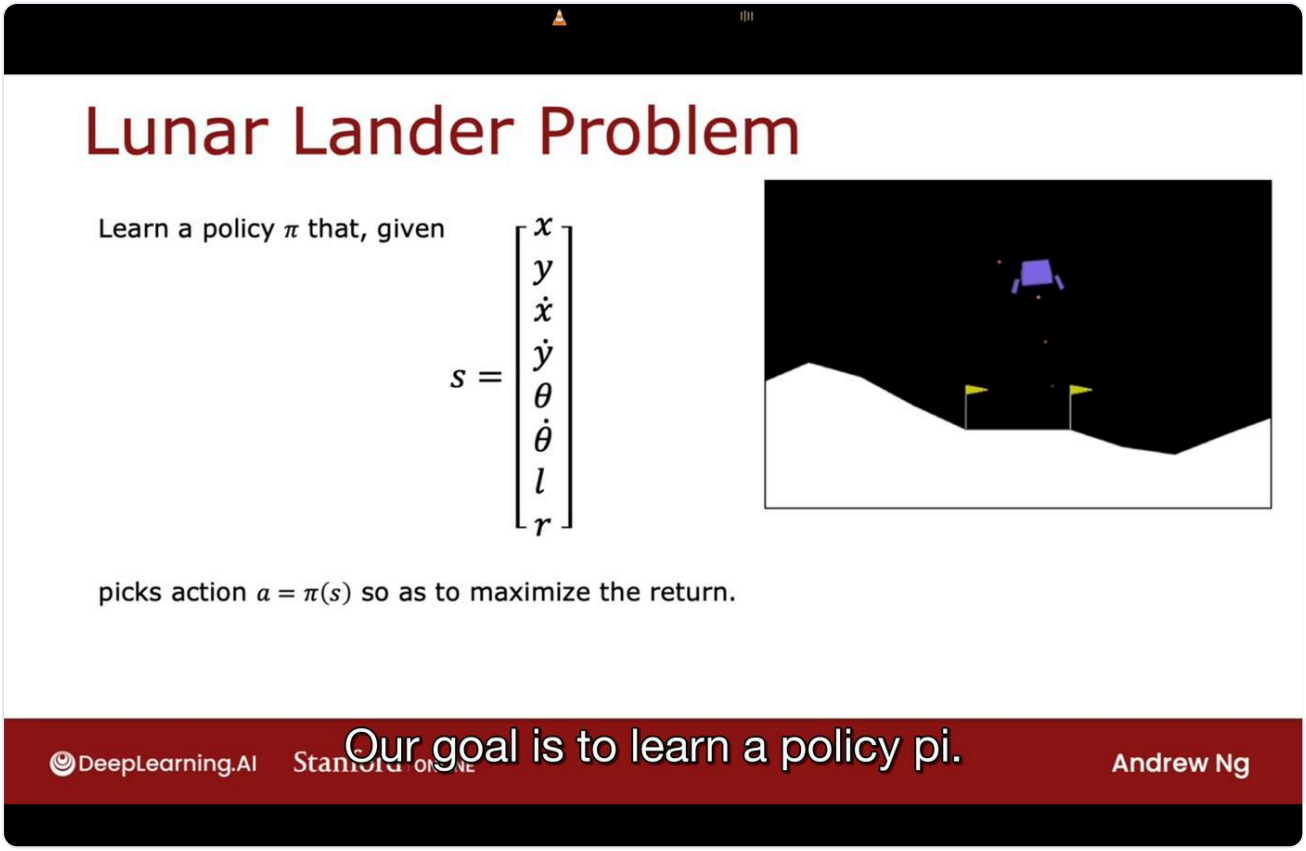


FIGURE 39.9 | Lunar Lander Problem

Our goal is to learn a policy pi. That when given a state S as written here picks an action a equals pi of S. So as to maximize the return the sum of discounted rewards. And usually for the lunar lander would use a fairly large value for gamma ra.

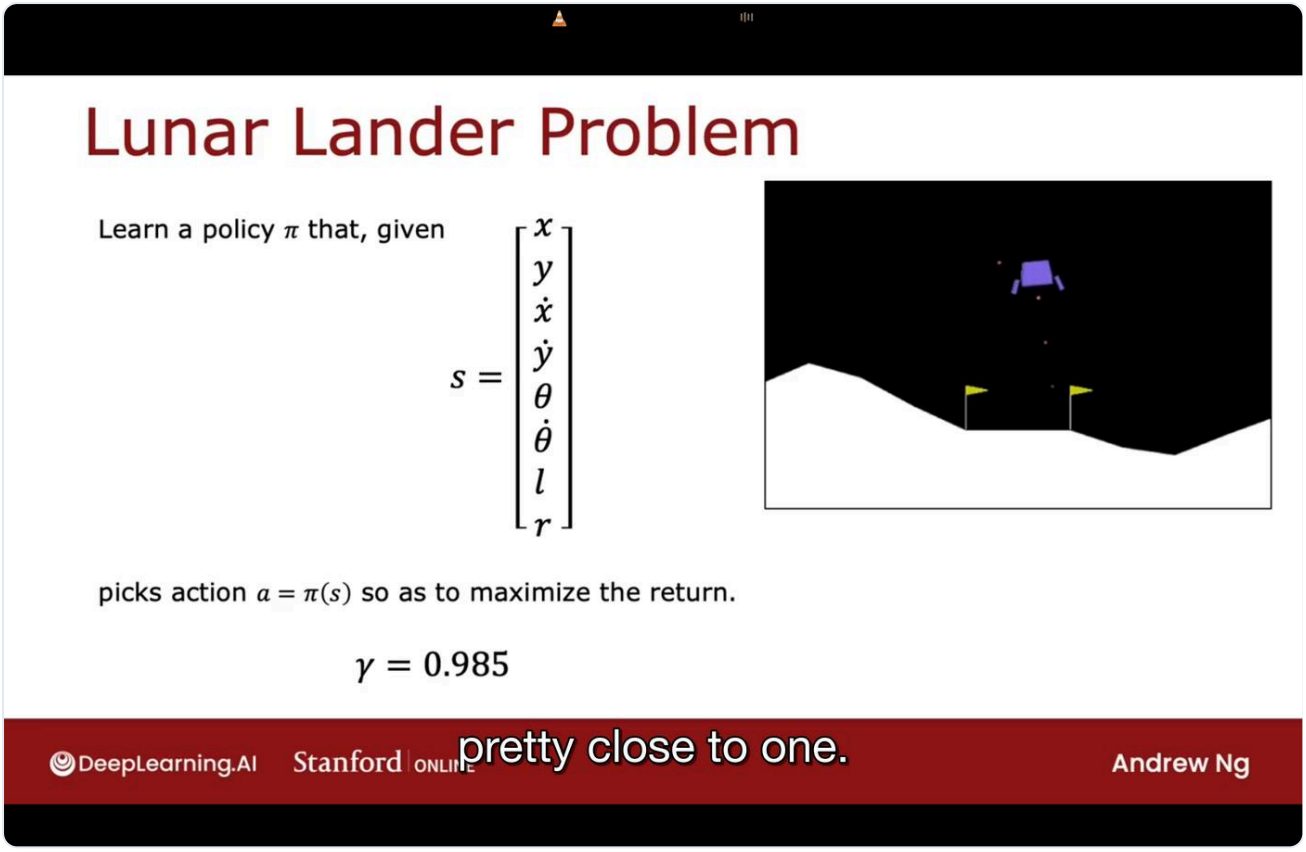


FIGURE 39.10 | Lunar Lander Problem

In fact for the would use the value of gamma that's equal to 0.985 so pretty close to one. And if you can learn a policy pi that does this then you successfully land this lunar lander exciting application and we're now finally ready to develop a learning algorithm which will turn out to use deep learning or neural networks to come up with a policy to land the lunar lander. Let's go into the next video where we start to learn about deep reinforcement.

LECTURE 40

Bellman Equation

Let's see how we can use reinforcement learning to control the Lunar Lander or for other reinforcement learning problems. The key idea is that we're going to train a neural network to compute or to approximate the state action value function Q of s, a and that in turn will let us pick good actions. Let's see how it works. The heart of the learning algorithm is we're going to train a neural network that inputs the current state and the current action and computes or approximates Q of s, a. In particular, for the Lunar Lander, we're going to take the state s and any action a and put them together. Concretely, the state was that list of eight numbers that we saw previously, so you have xy, x dot, y dot, Theta, Theta dot, and then LR for where the legs are grounded, so that's a list of eight numbers that describe the state.

Then finally, we have four possible actions: nothing, left, main, a main engine, and right. We can encode any of those four actions using a one-hot feature vector. If action were the first action, we may encode it using 1, 0, 0, 0 or if it was the second action to find the left cluster, we may encode it as 0, 1, 0, 0. This list of 12 numbers, eight numbers for the state and then four numbers, a one-hot encoding of the action is the inputs we'll have to the neural network, and I'm going to call this X. We'll then take these 12 numbers and feed them to a neural network with say, 64 units in the first hidden layer, 64 units in the second hidden layer, and then a single output in the output layer.

The job of the neural network is the output Q of s, a. The state action-value function for the Lunar Lander given the input s and a. Because we'll be using neural network training algorithms in a little bit, I'm also going to refer to this value Q of s, a as the target value Y that were training the neural network to approximate. Notice that I did say reinforcement learning is different from supervised learning, but what we're going to do is not input a state and have it output an action. What we're going to do is input a state action pair and have it try to output Q of s, a, and using a neural network inside the reinforcement learning algorithm this way will turn out to work pretty well.

We'll see the details in a little bit so don't worry about it if it doesn't make sense yet. But if you can train a neural network with appropriate choices of parameters in the hidden layers and in the upper layer to give you a good estimates of Q of s, a, then whenever you're Lunar Lander is in some state s, you can then use the neural network to compute Q of s, a.

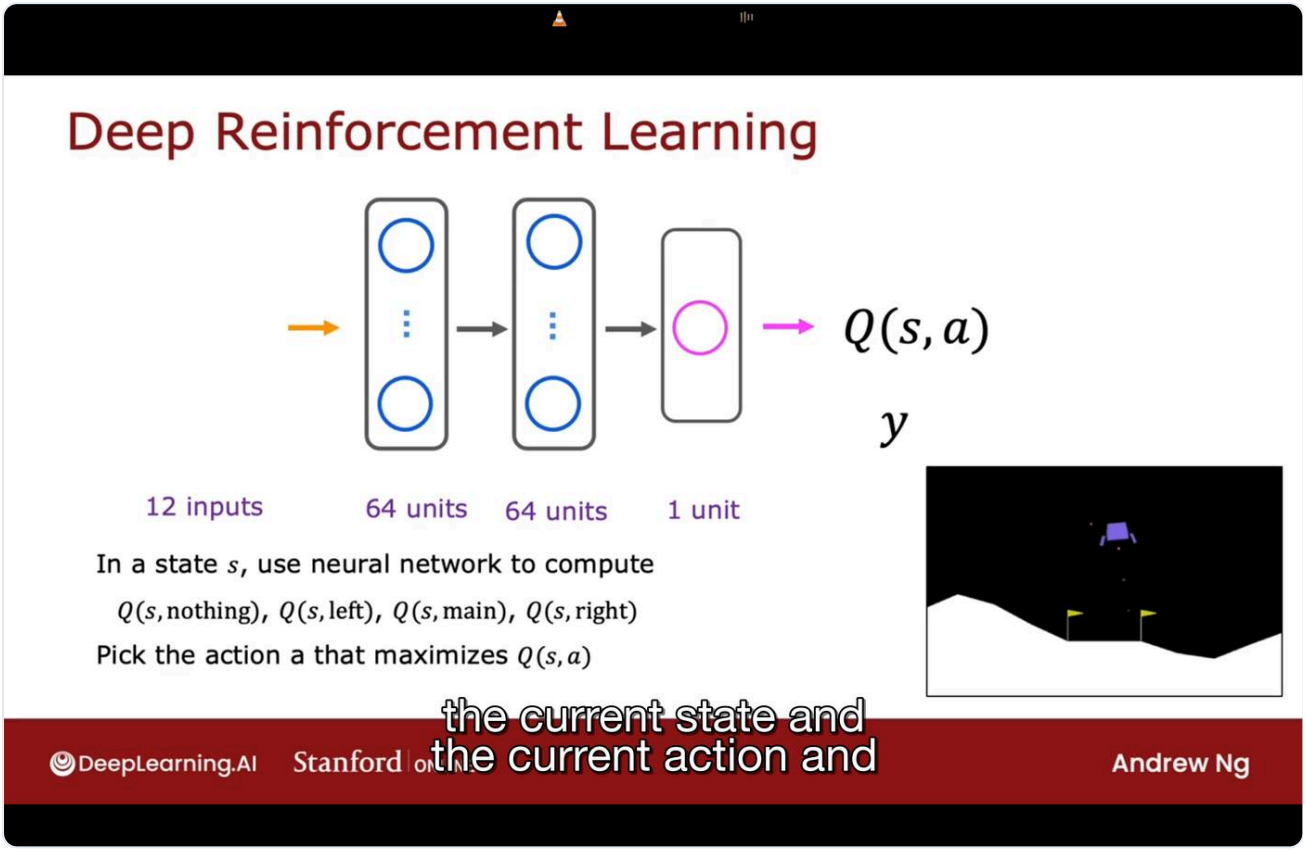


FIGURE 40.1 | Deep Reinforcement Learning

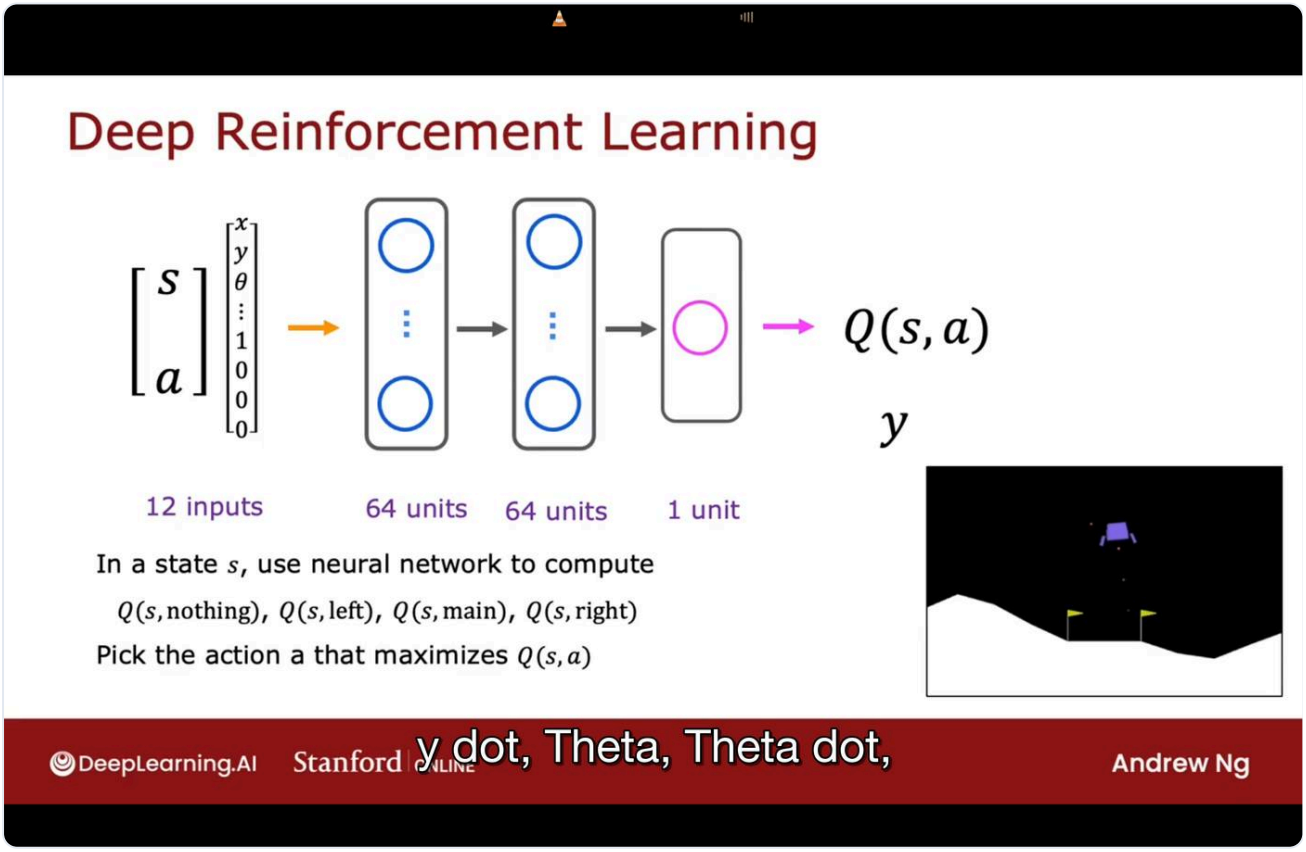


FIGURE 40.2 | Deep Reinforcement Learning

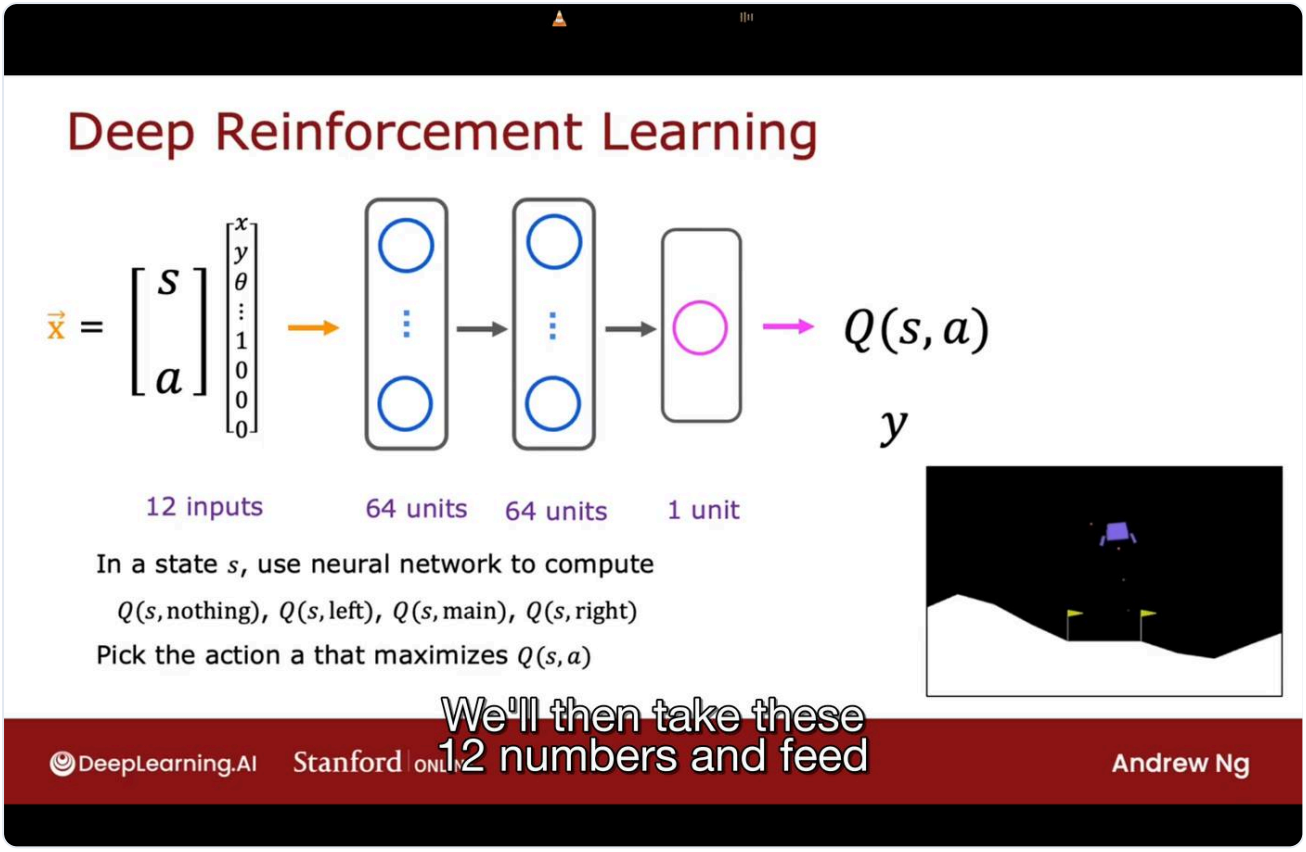


FIGURE 40.3 | Deep Reinforcement Learning

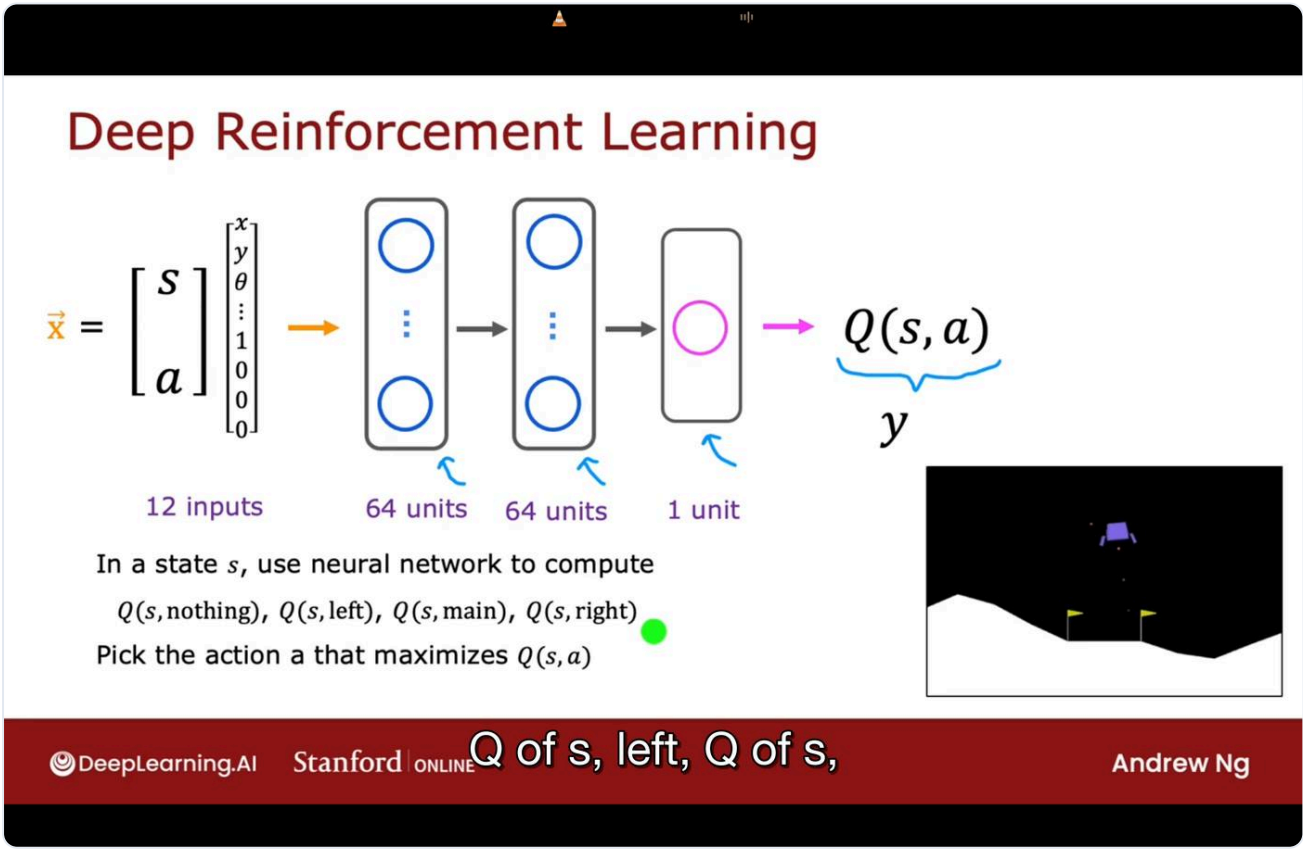


FIGURE 40.4 | Deep Reinforcement Learning

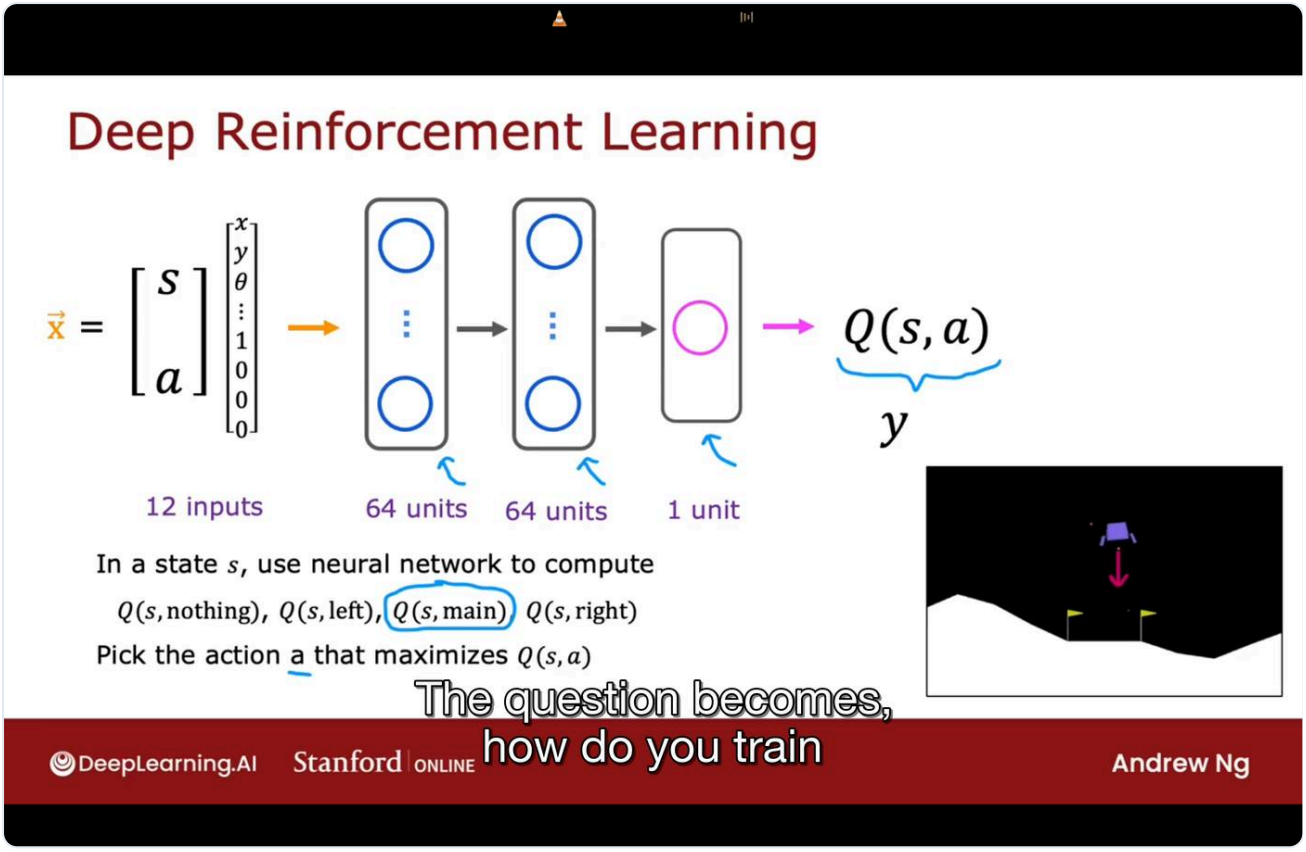


FIGURE 40.5 | Deep Reinforcement Learning

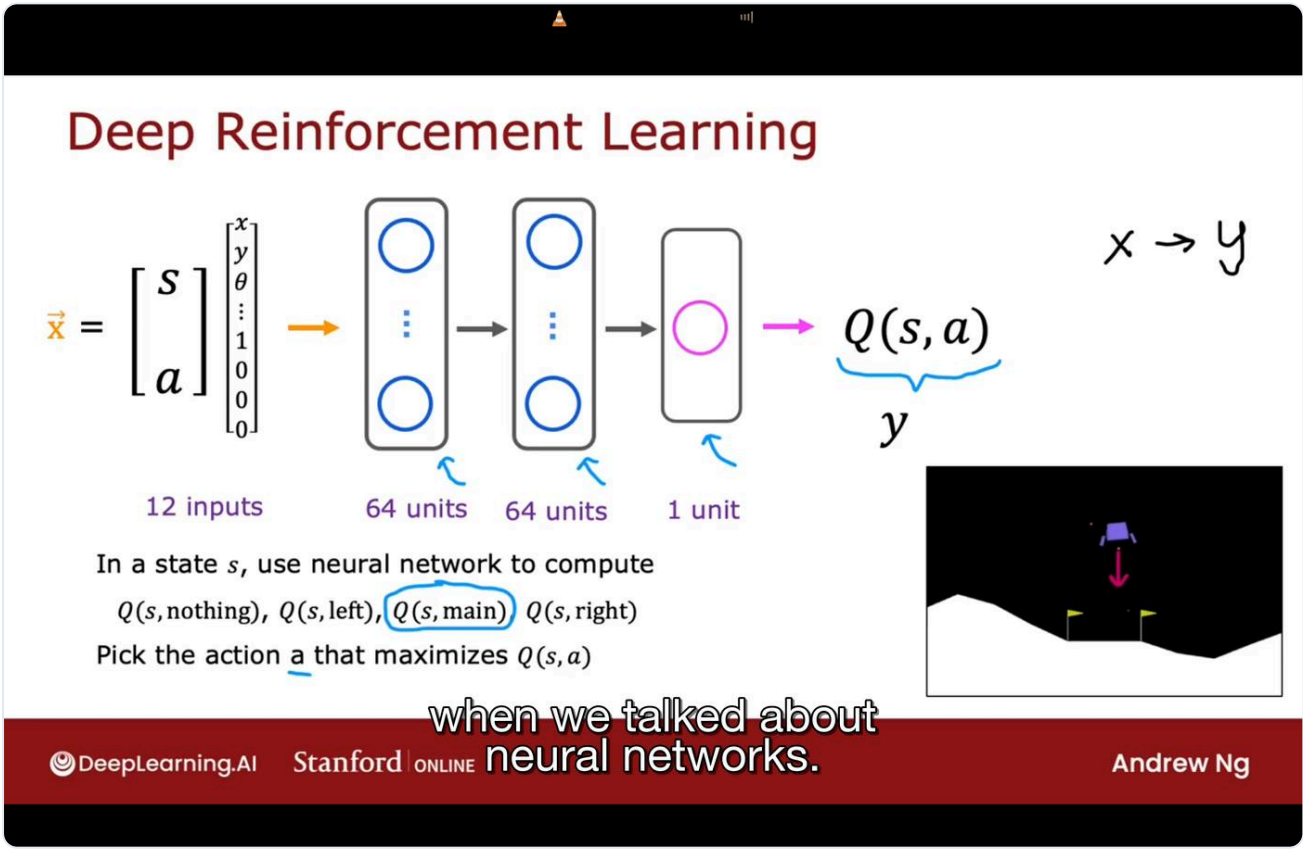


FIGURE 40.6 | Deep Reinforcement Learning

For all four actions, you can compute Q of s, nothing, Q of s, left, Q of s, main, Q of s, right, and then finally, whichever of these has the highest value, you pick the corresponding action a. For example, if out of these four values, Q of s, main is largest, then you would decide to go and fire the main engine of the Lunar Lander. The question becomes, how do you train a neural network to output Q of s, a? It turns out the approach will be to use Bellman's equations to create a training set with lots of examples, x and y, and then we'll use supervised learning exactly as you learned in the second course when we talked about neural networks. To learn using supervised learning, a mapping from x to y, that is a mapping from the state action pair to this target value Q of s, a.

But how do you get the training set with values for x and y that you can then train a neural network on? Let's take a look. Here's the Bellman equation, Q of s, a equals R of s plus Gamma, max of a prime, Q of s prime, a prime.

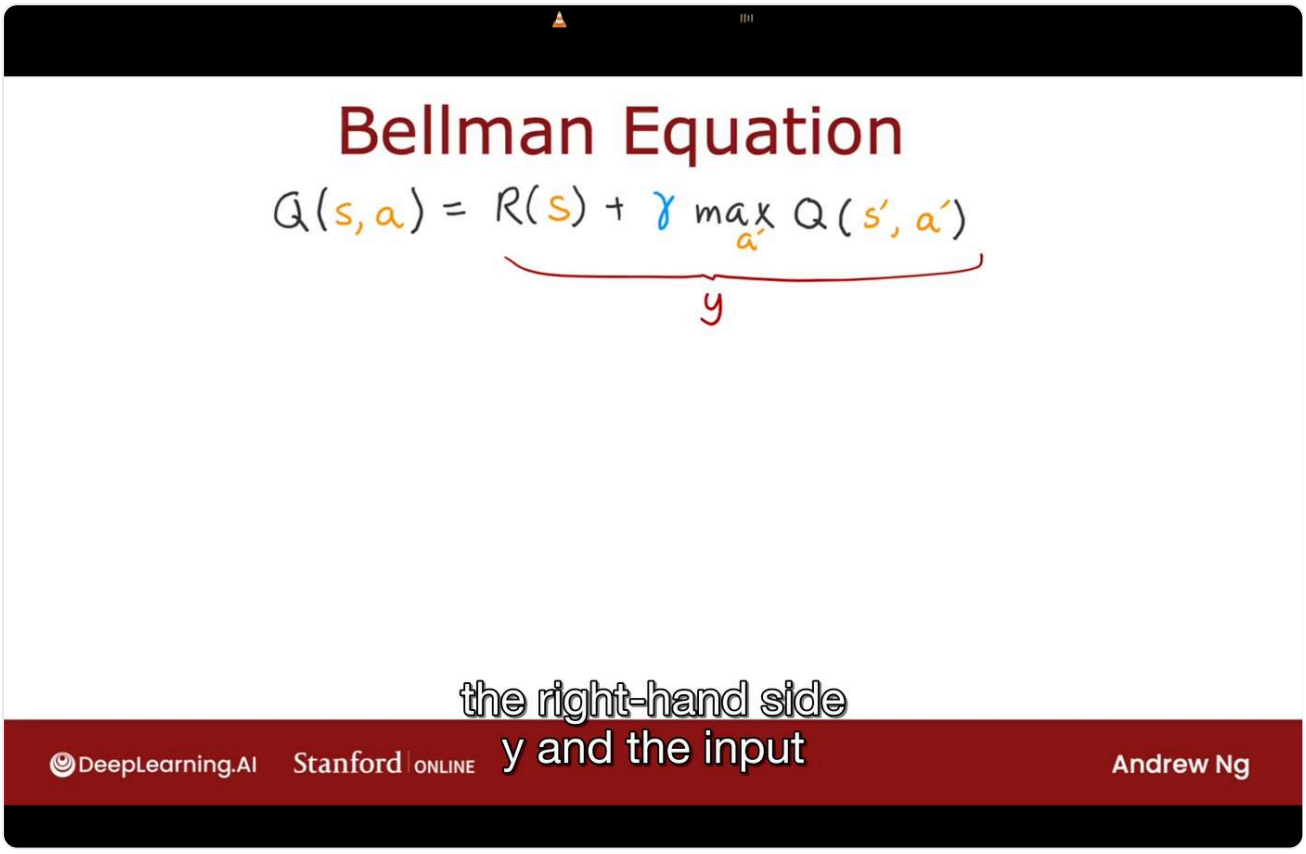


FIGURE 40.7 | Bellman Equation

The right-hand side is what you want Q of s, a to be equal to, so I'm going to call this value on the right-hand side y and the input to the neural network is a state and an action so I'm going to call that x.

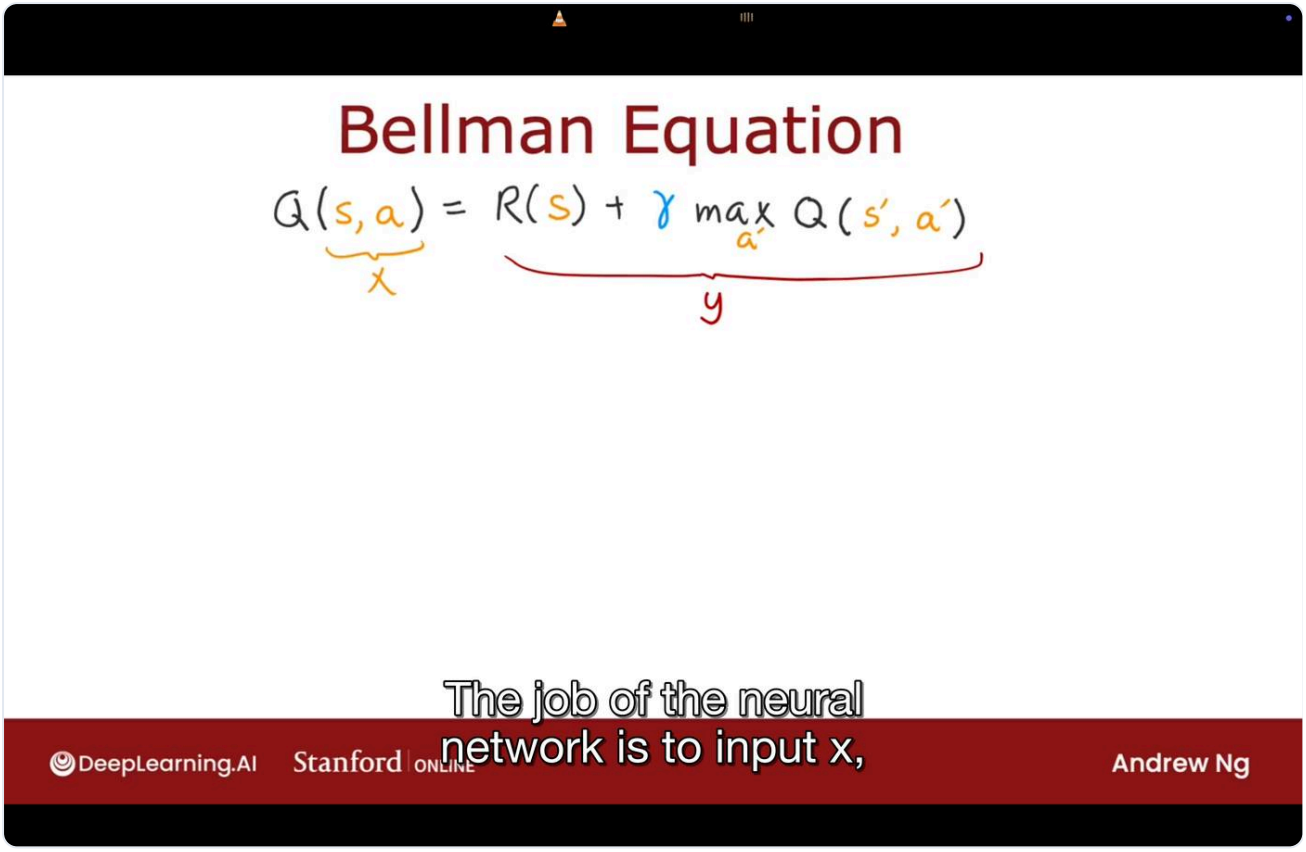


FIGURE 40.8 | Bellman Equation

The job of the neural network is to input x, that is input the state action pair, and try to accurately predict what will be the value on the right. In supervised learning, we were training a neural network to learn a function f, which depends on a bunch of parameters, W and B, the parameters of the various layers of the neural network and it was the job of the neural network to input x. Hopefully I'll put something close to the target value y.

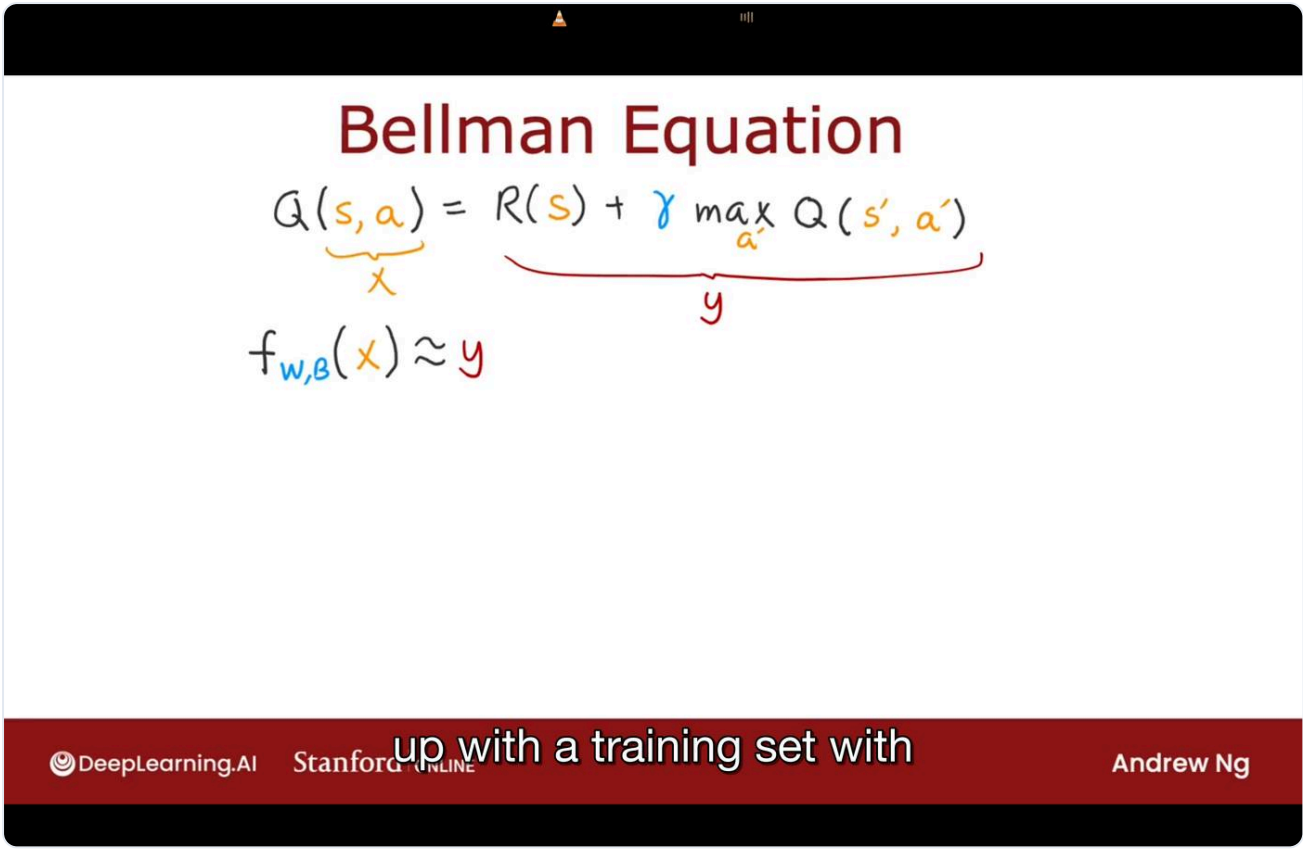


FIGURE 40.9 | Bellman Equation

The question is, how can we come up with a training set with values x and y for a neural network to learn from. Here's what we're going to do.

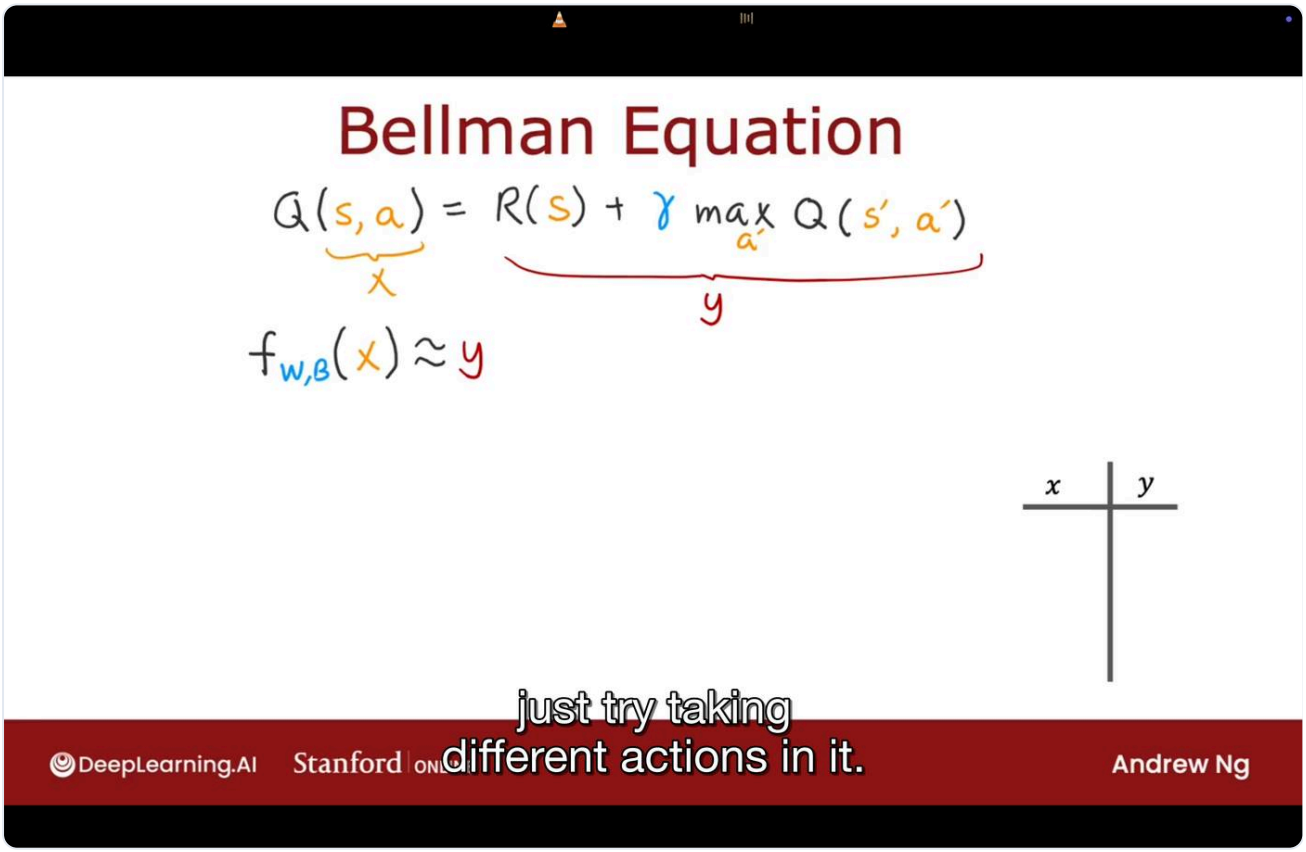


FIGURE 40.10 | Bellman Equation

We're going to use the lunar lander and just try taking different actions in it. If we don't have a good policy yet, we'll take actions randomly, fire the left thruster, fire the, right thruster, fire the main engine, do nothing. By just trying out different things in the lunar lander simulator we'll observe a lot of examples of when we're in some state and we took some action, may be a good action, maybe a terrible action either way. Then we got some reward R of s for being in that state, and as a result of our action, we got to some new state, S'. As you take different actions in the lunar lander, you see this S, a, R of s, S', and we call them tuples in Python code many times.

For example, maybe one time you're in some state S and just to give this an index and we call this S^1, and you happen to take some action a^1, this could be nothing left, main thrusters or right. As a result of which you've got some reward, and you wound up at some state S^'1.

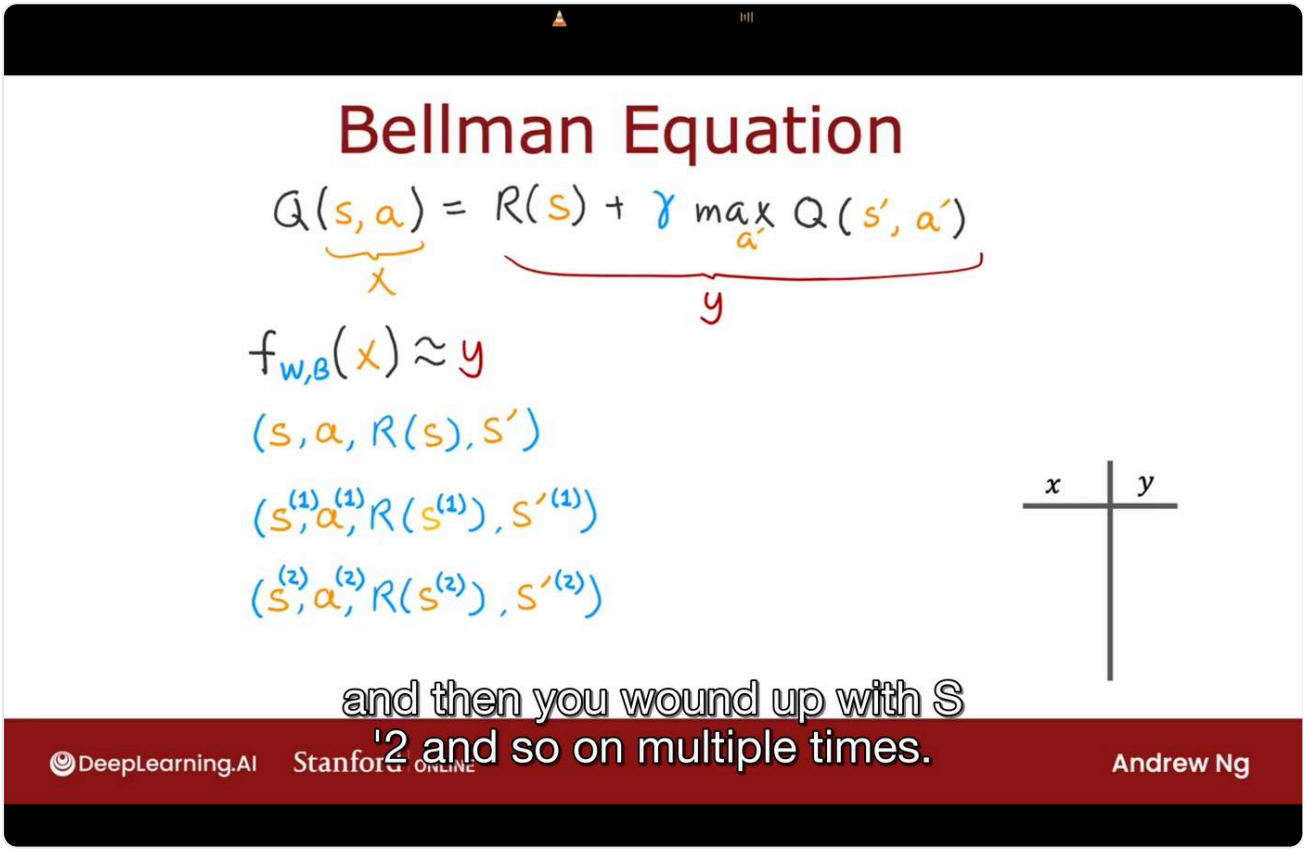


FIGURE 40.11 | Bellman Equation

Maybe at different times you're in some other state S2, you took some other action, could be good action, could be bad action, could be any of the four actions, and you got the reward, and then you wound up with S '2 and so on multiple times.

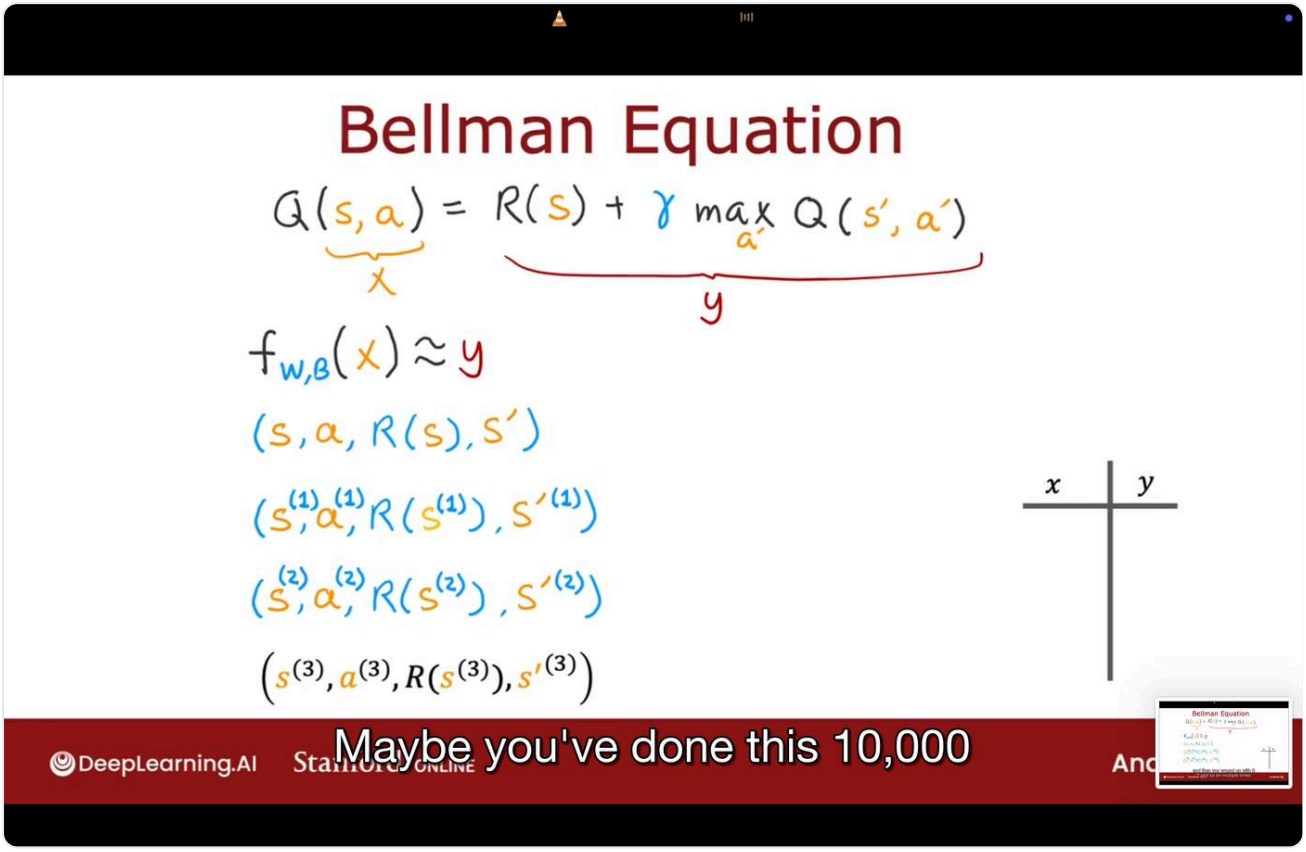


FIGURE 40.12 | Bellman Equation

Maybe you've done this 10,000 times or even more than 10,000 times, so you would have to save the way with not just S^1, a^1 and so on, but up to S^10,000, a^10,000. It turns out that each of these tuples will be enough to create a single training example, x^1, y^1. In particular, here's how you do it. There are four elements in this first tuple. The first two will be used to compute x^1, and the second two would be used to compute y^1. In particular, x^1 is just going to be S^1, a^1 put together. S^1 would be eight numbers, the state of the lunar lander, a^1 would be four numbers, the one-hot encoding of whatever action this was and y^1 would be computed using the right-hand side of the Bellman equation.

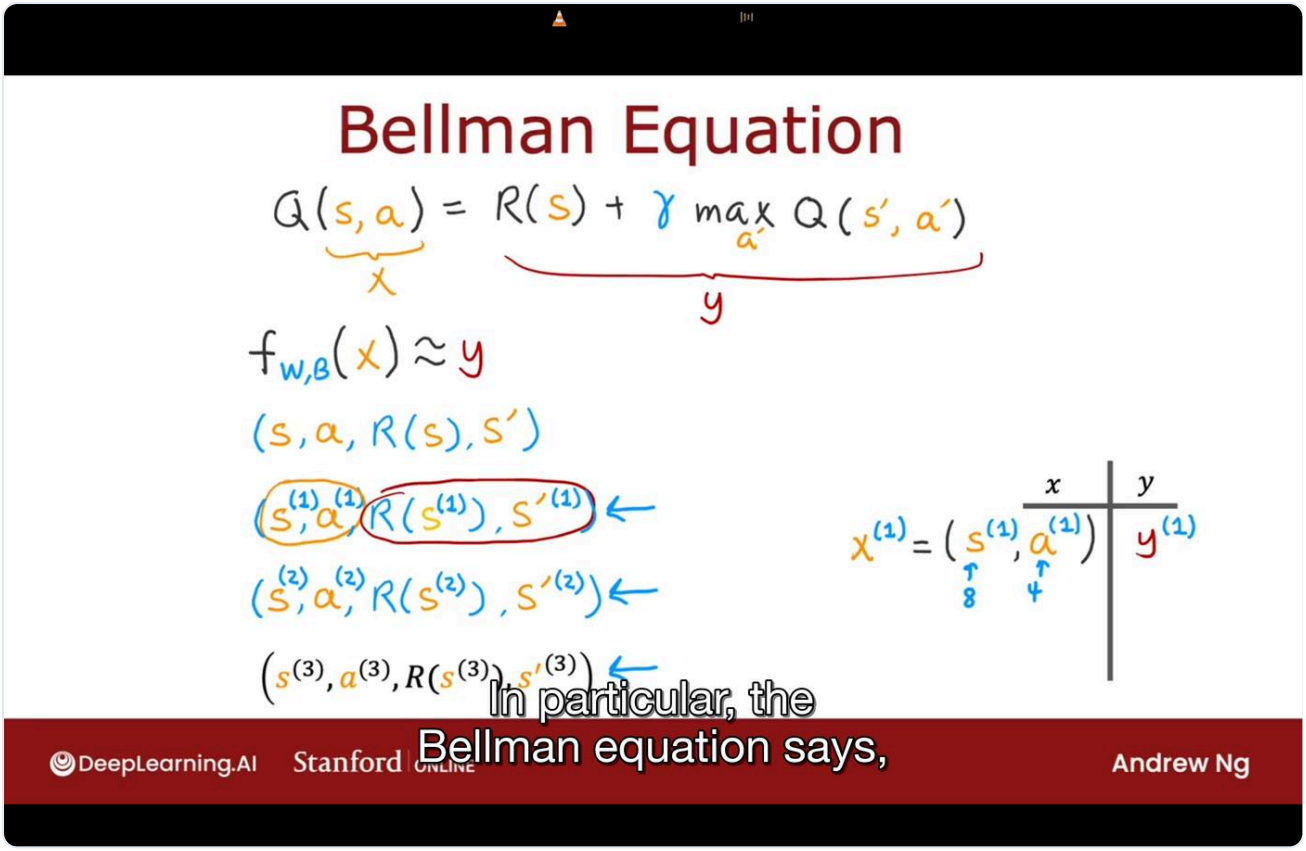


FIGURE 40.13

In particular, the Bellman equation says, when you input S^1, a^1, you want Q of S^1, a^1 to be this right hand side, to be equal to R of S^1 plus Gamma max over a' of Q of S^1'a' prime.

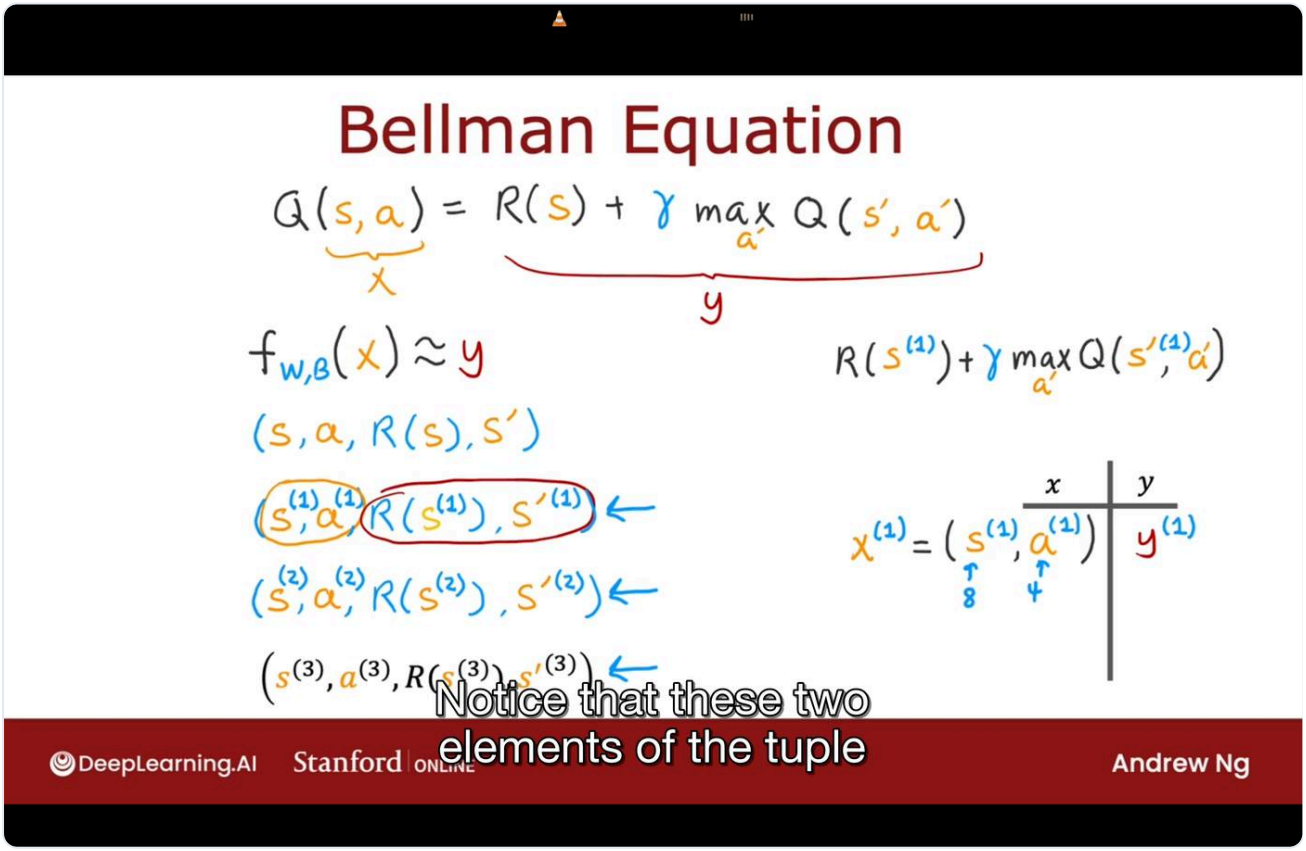


FIGURE 40.14 | Bellman Equation

Notice that these two elements of the tuple on the right give you enough information to compute this. You know what is R of S^1? That's the reward you've saved the way here. Plus the discount factor gamma times max over all actions, a' of Q of S'^1, that's a state you got to in this example, and then take the maximum over all possible actions, a'. I'm going to call this y^1.

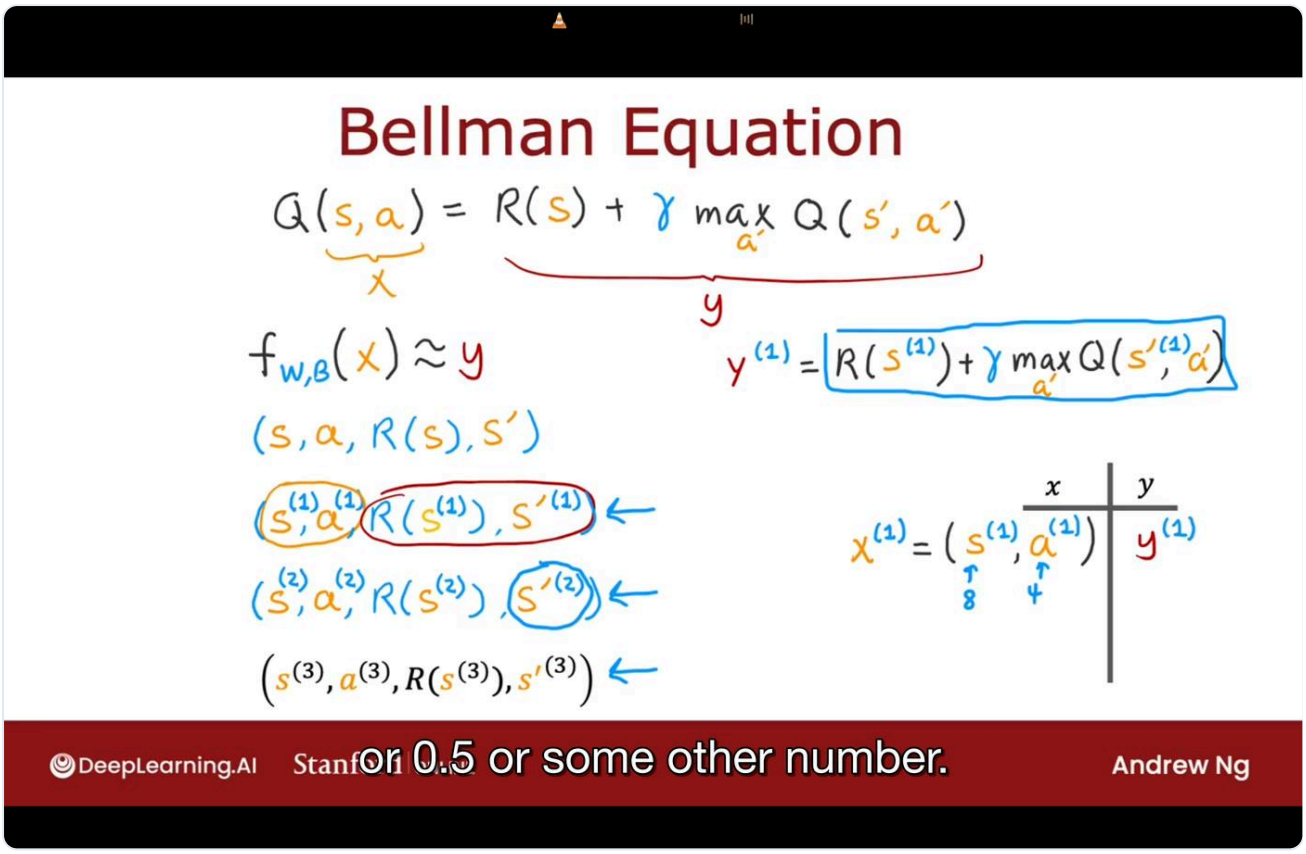


FIGURE 40.15 | Bellman Equation

When you compute this, this will be some number like 12.5 or 17, or 0.5 or some other number. We'll save that number here as y^1, so that this pair x^1, y^1 becomes the first training example in this little dataset we're computing. Now, you may be wondering, where does Q of S', a' or Q of S'^1, a' come from. Well, initially we don't know what is the Q function. But it turns out that when you don't know what is the Q function, you can start off with taking a totally random guess. What is the Q function? We'll see on the next slide that the algorithm will work nonetheless. But in every step Q here is just going to be some guess. They'll get better over time it turns out of what is the actual Q function.

Let's look at the second example. If you had a second experience where you are in state S^2 to got to a^2, got that reward and then got to that state. Then we would create a second training example in this dataset, x^2, where the input is now S^2, a^2, so the first two elements go to computing the input x, and then y^2 will be equal to R of s^2 plus gamma max of a' Q of S' to a', and whatever this number is, y^2.

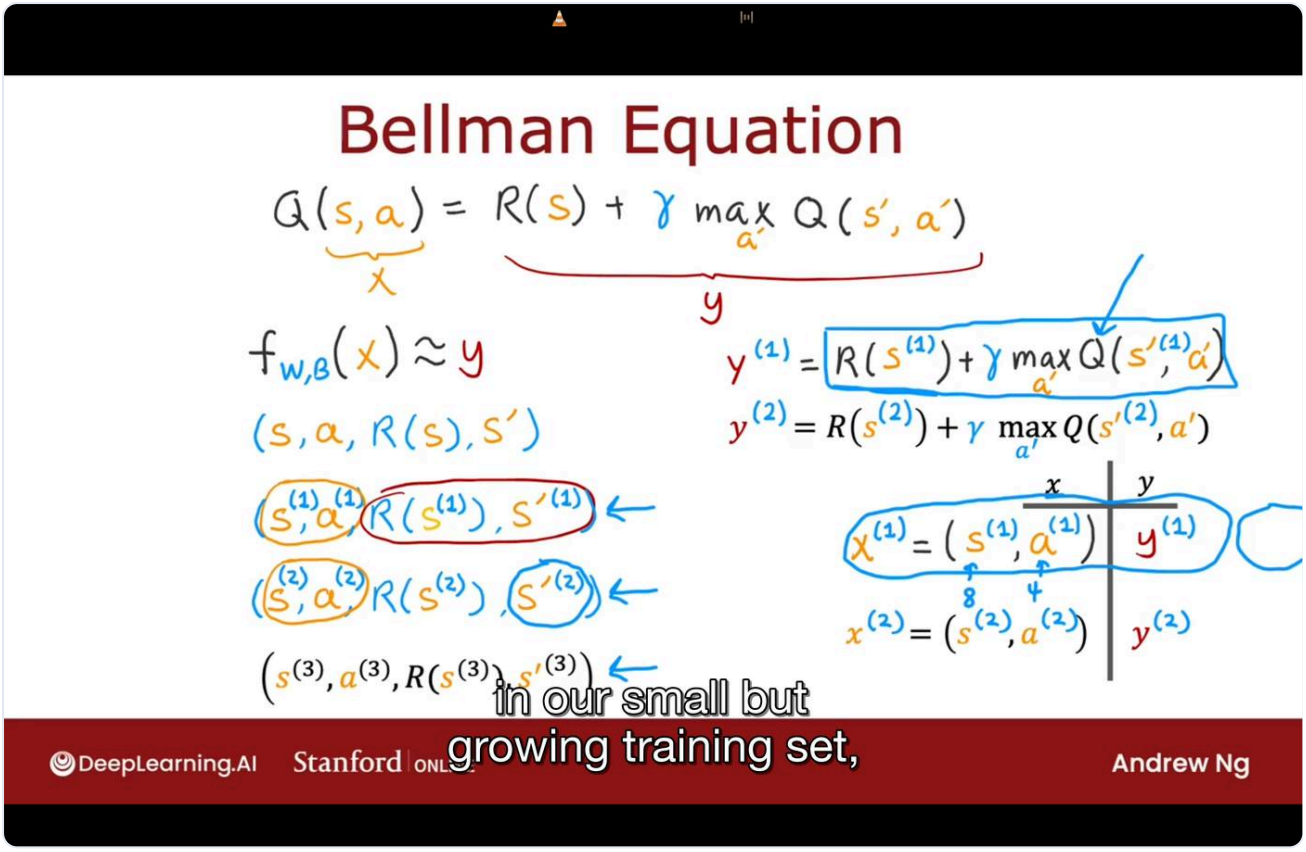


FIGURE 40.16 | Bellman Equation

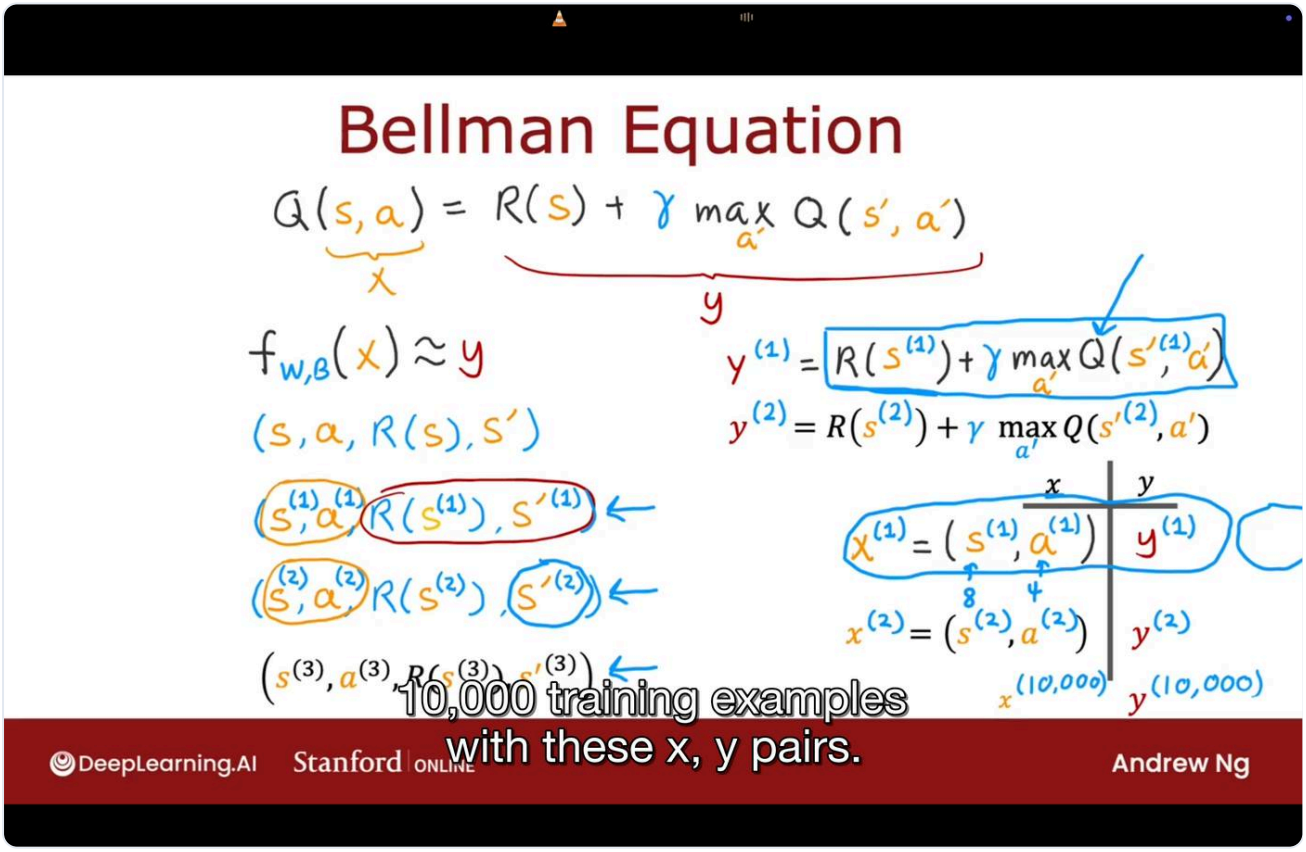


FIGURE 40.17 | Bellman Equation

We put this over here in our small but growing training set, and so on and so forth, until maybe you end up with 10,000 training examples with these x, y pairs. What we'll see later is that we'll actually take this training set where the x's are inputs with 12 features and the y's are just numbers. We'll train a neural network with, say, the mean squared error loss to try to predict y as a function of the input x. What I describe here is just one piece of the learning algorithm we'll use. Let's put it all together on the next slide and see how it all comes together into a single algorithm. Let's take a look at what a full algorithm for learning the Q-function is like.

First, we're going to take our neural network and initialize all the parameters of the neural network randomly. Initially we have no idea, whether it's a Q function, let's just pick totally random values of the weights. We'll pretend that this neural network is our initial random guess for the Q-function. This is a little bit like when you are training linear regression and you initialize all the parameters randomly and then use gradient descent to improve the parameters. Initializing it randomly for now is fine. What's important is whether the algorithm can slowly improve the parameters to get a better estimate. Next, we will repeatedly do the following; We will take actions in the lunar lander, so float around randomly, take some good actions, take some bad actions.

It's okay either way. But you get lots of these tuples of when it was in some state, you took some action a get a reward R of S and you got to some state s prime. What we will do is store to 10,000 most recent examples of these tuples.

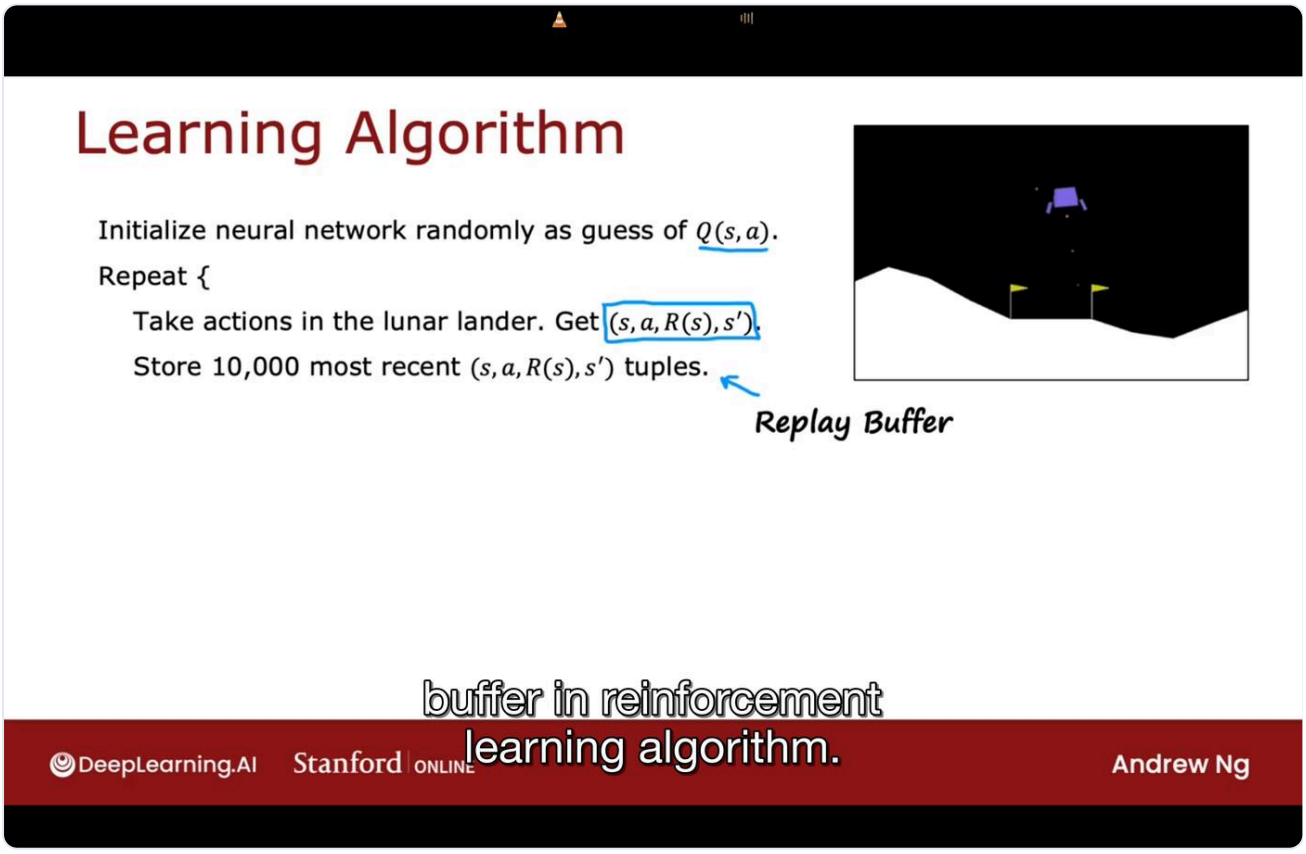


FIGURE 40.18

As you run this algorithm, you will see many steps in the lunar lander, maybe hundreds of thousands of steps. But to make sure we don't end up using excessive computer memory, common practice is to just remember the 10,000 most recent such tuples that we saw taking actions in the

MTP. This technique of storing the most recent examples only is sometimes called the replay buffer in reinforcement learning algorithm. For now, we're just flying the lunar lander randomly, sometimes crashing, sometimes not and getting these tuples as experienced for our learning algorithm. Occasionally then we will train the neural network. In order to train the neural network, here's what we'll do.

We'll look at these 10,000 most recent tuples we had saved and create a training set of 10,000 examples. Training set needs lots of pairs of x and y. For our training examples, x will be the s, a from this part of the tuple. There'll be a list of 12 numbers, the 8 numbers for the state and the 4 numbers for the one-hot encoding of the action. The target value that we want a neural network to try to predict would be y equals R of S plus Gamma max of a prime, Q of s prime a prime. How do we get this value of Q? Well, initially is this neural network that we had randomly initialized. It may not be a very good guess, but it's a guess. After creating these 10,000 training examples we'll have training examples x1, y1 through x10,000, y10,000.

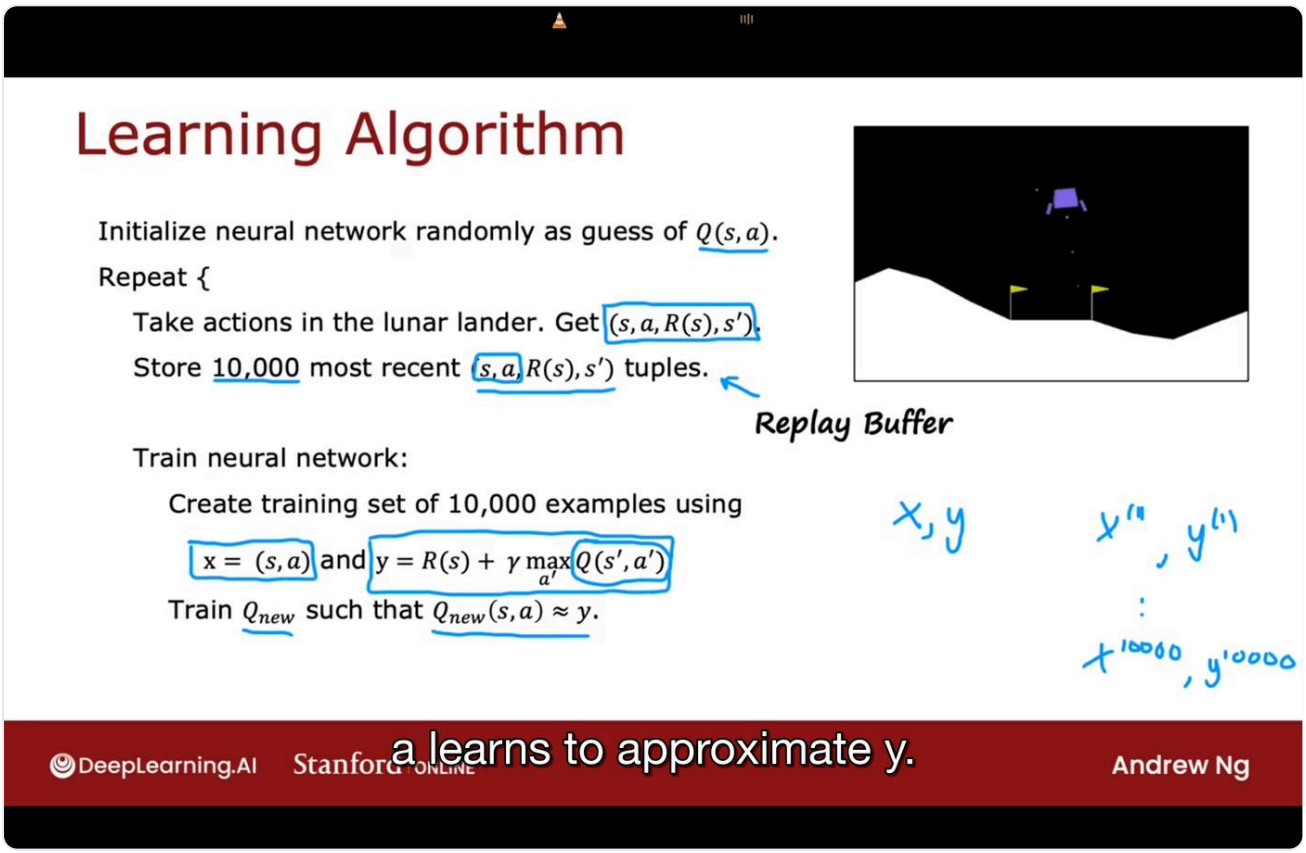


FIGURE 40.19 | Learning Algorithm

We'll train a neural network and I'm going to call the new neural network Q new, such that Q new of s, a learns to approximate y. This is exactly training that neural network to output f with parameters w and b, to input x to try to approximate the target value y.

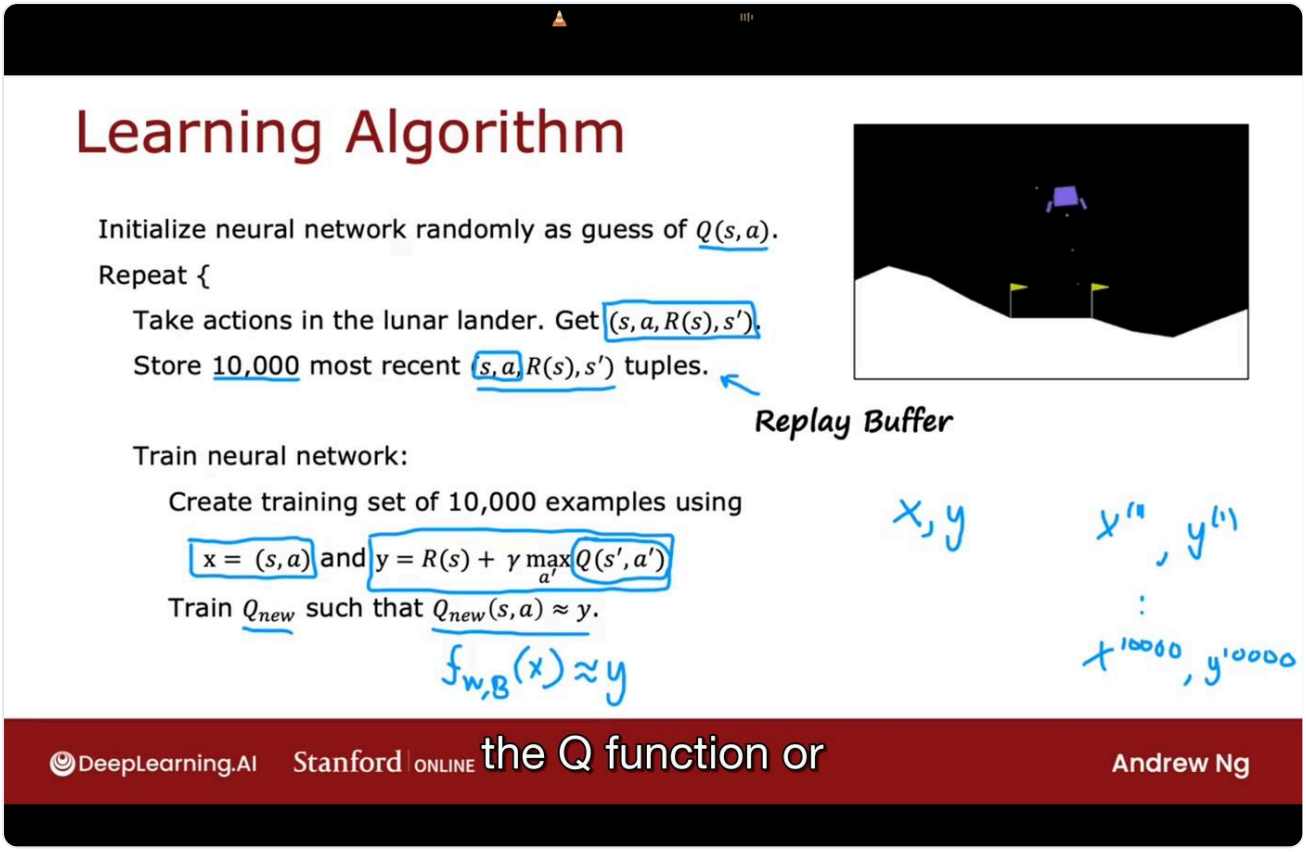


FIGURE 40.20 | Learning Algorithm

Now, this neural network should be a slightly better estimates of what the Q function or the state action value function should be.

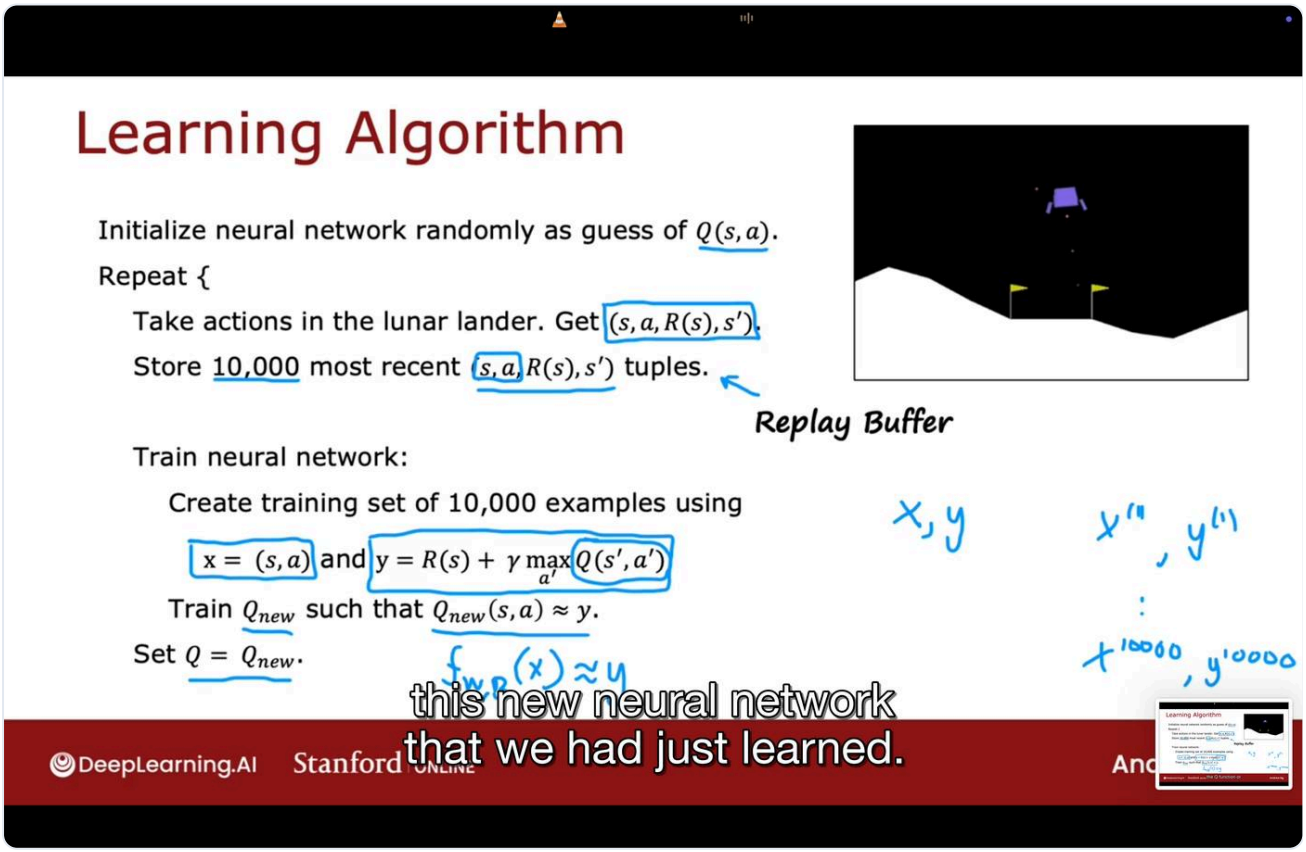


FIGURE 40.21 | Learning Algorithm

What we'll do is we're going to take Q and set it to this new neural network that we had just learned. Many of the ideas in this algorithm are due to Mnih et al. It turns out that if you run this algorithm where you start with a really random guess of the Q function, then use Bellman's equations to repeatedly try to improve the estimates of the Q function. Then by doing this over and over, taking lots of actions, training a model, that will improve your guess for the Q-function. For the next model you train, you now have a slightly better estimate of what is the Q function. Then the next model you train will be even better. When you update Q equals Q new.

Then for the next time you train a model Q of s prime a prime will be an even better estimate.

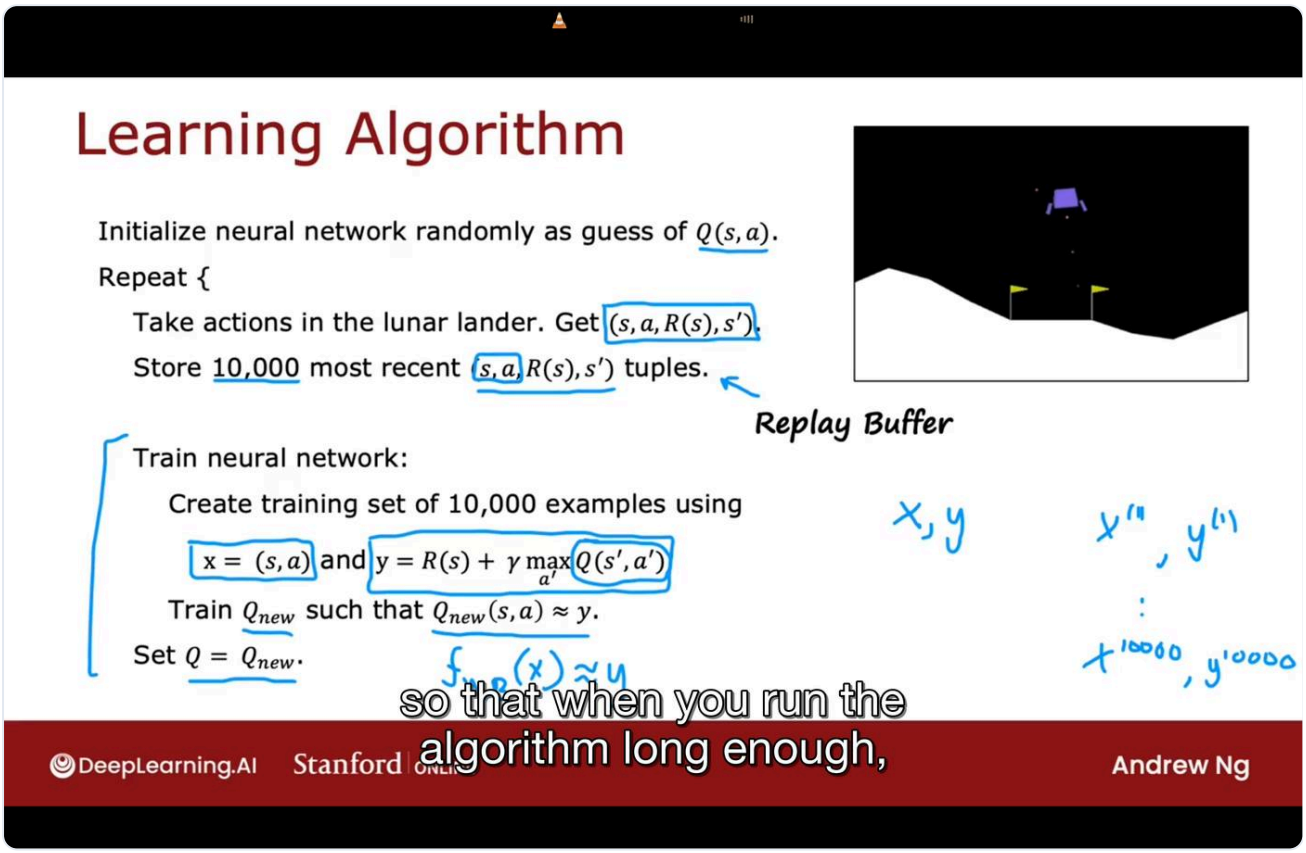


FIGURE 40.22 | Learning Algorithm

As you run this algorithm on every iteration, Q of s prime, a prime hopefully becomes an even better estimate of the Q function so that when you run the algorithm long enough, this will actually become a pretty good estimate of the true value of Q of s, a, so that you can then use this to pick, hopefully good actions or the MTP. The algorithm you just saw is sometimes called the DQN algorithm which stands for Deep Q-Network because you're using deep learning and neural network to train a model to learn the Q functions. Hence DQN or DQ using a neural network. If you use the algorithm as I described it, it will work, okay, on the lunar lander. Maybe it'll take a long time to converge, maybe it won't land perfectly, but it'll work.

But it turns out that with a couple of refinements to the algorithm, it can work much better. In the next few videos, let's take a look at some refinements to the algorithm that you just saw.

LECTURE 41

Neural Network

In the last video, we saw a neural network architecture that will input the state and action and attempt to output the Q function, Q of s, a. It turns out that there's a change to neural network architecture that make this algorithm much more efficient. Most implementations of DQN actually use this more efficient architecture that we'll see in this video. Let's take a look. This was the neural network architecture we saw previously, where it would input 12 numbers and output Q of s, a. Whenever we are in some state s, we would have to carry out inference in the neural network separately four times to compute these four values so as to pick the action a that gives us the largest Q value.

This is inefficient because we have to carry our inference four times from every single state. Instead, it turns out to be more efficient to train a single neural network to output all four of these values simultaneously. This is what it looks like. Here's a modified neural network architecture where the input is eight numbers corresponding to the state of the lunar lander. It then goes through the neural network with 64 units in the first hidden layer, 64 units in the second hidden layer. Now the output unit has four output units, and the job of the neural network is to have the four output units output Q of s, nothing, Q of s, left, Q of s, main, and q of s, right.

The job of the neural network is to compute simultaneously the Q value for all four possible actions for when we are in the state s.

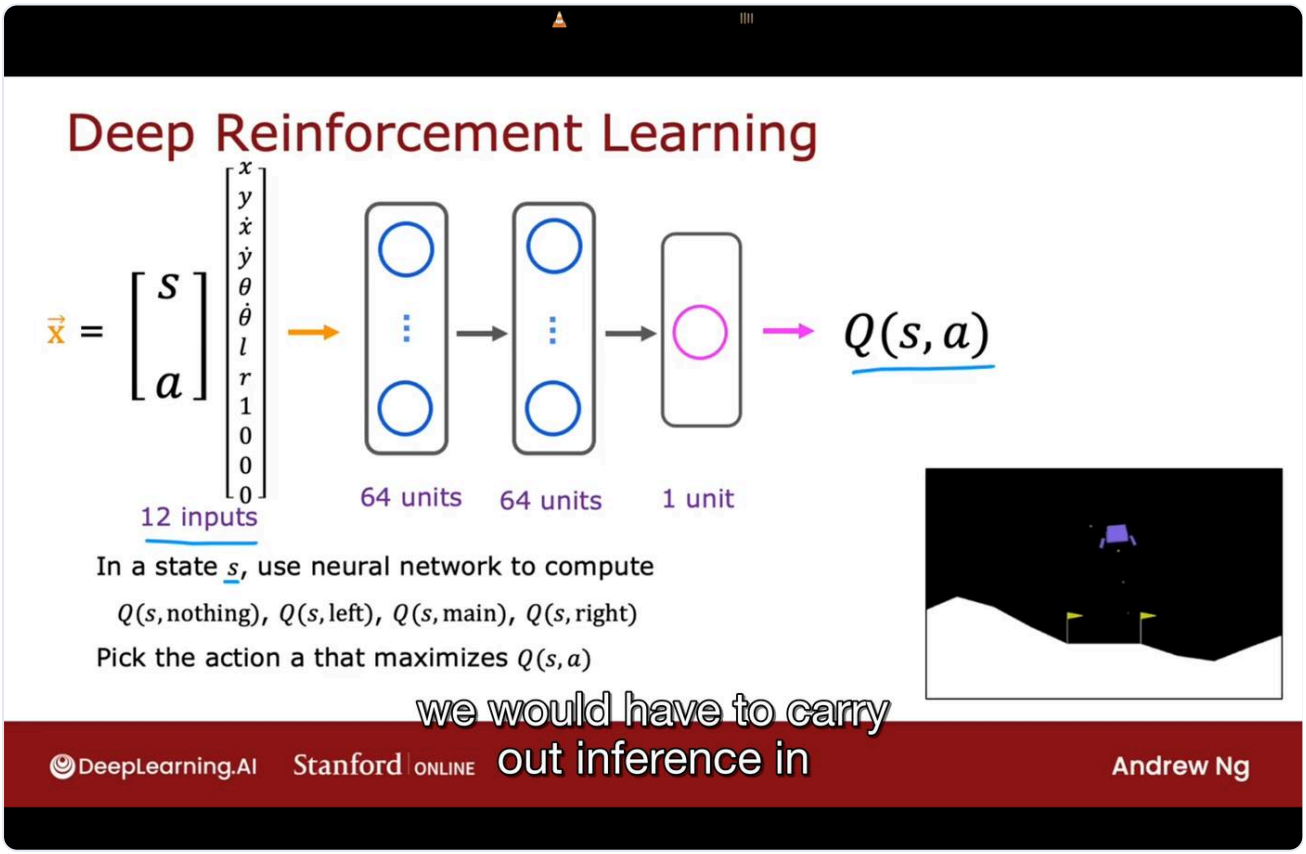


FIGURE 41.1

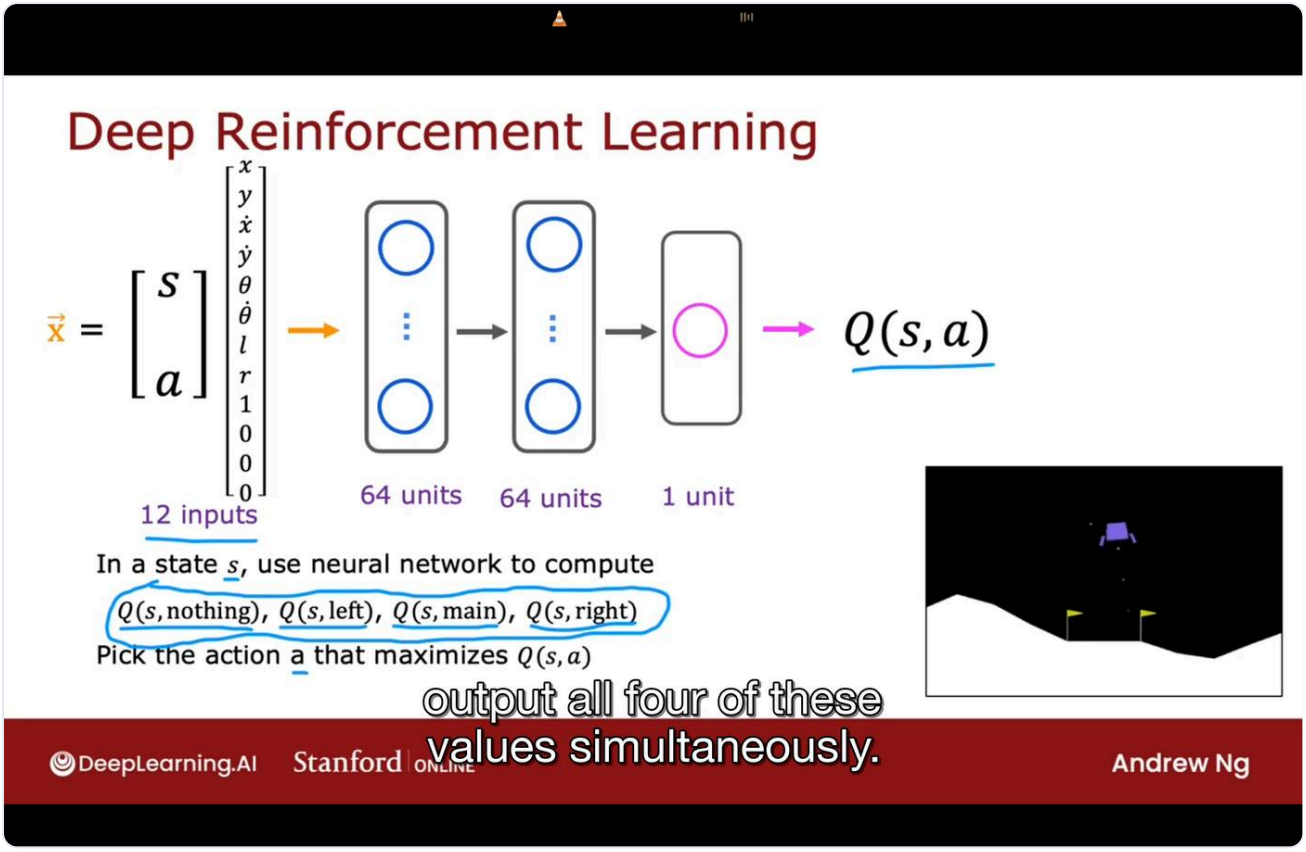


FIGURE 41.2

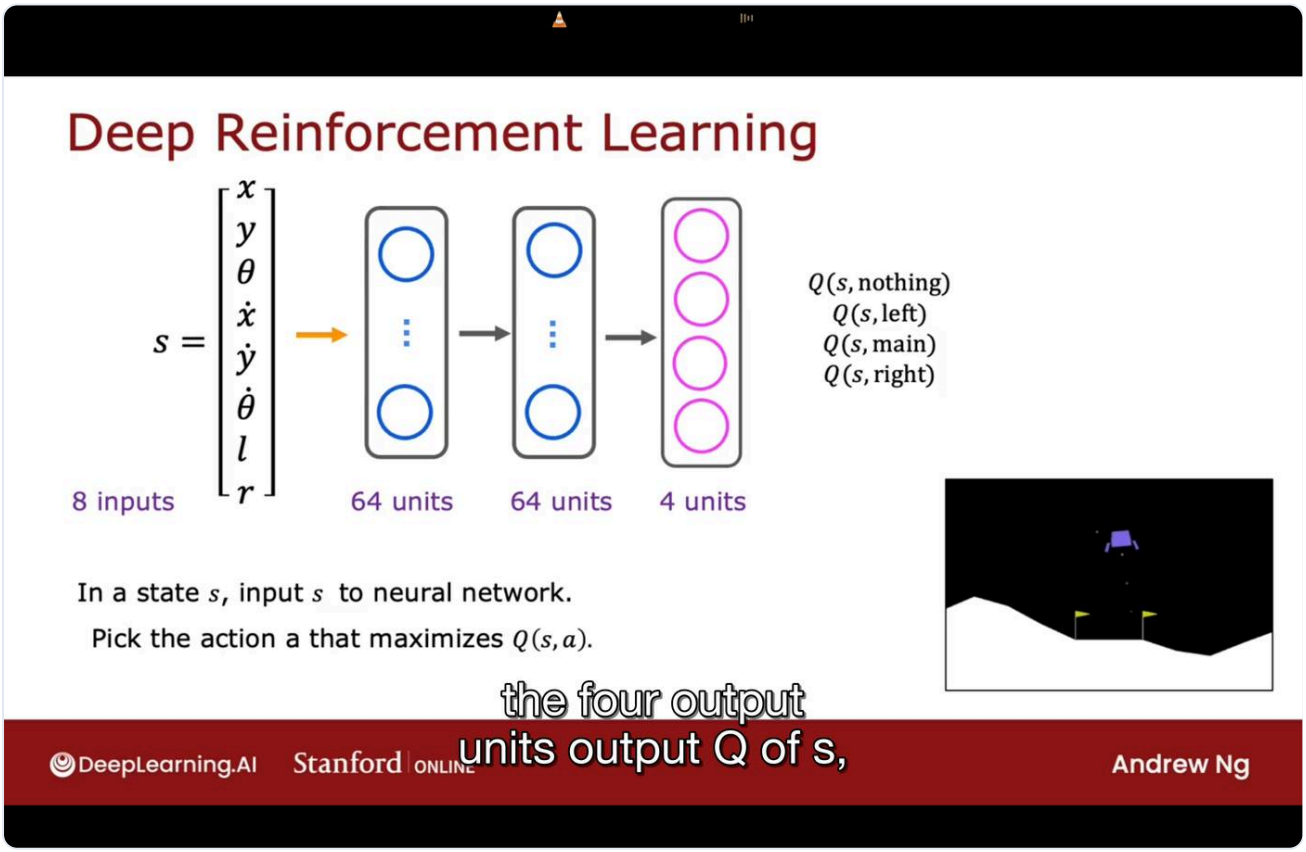


FIGURE 41.3

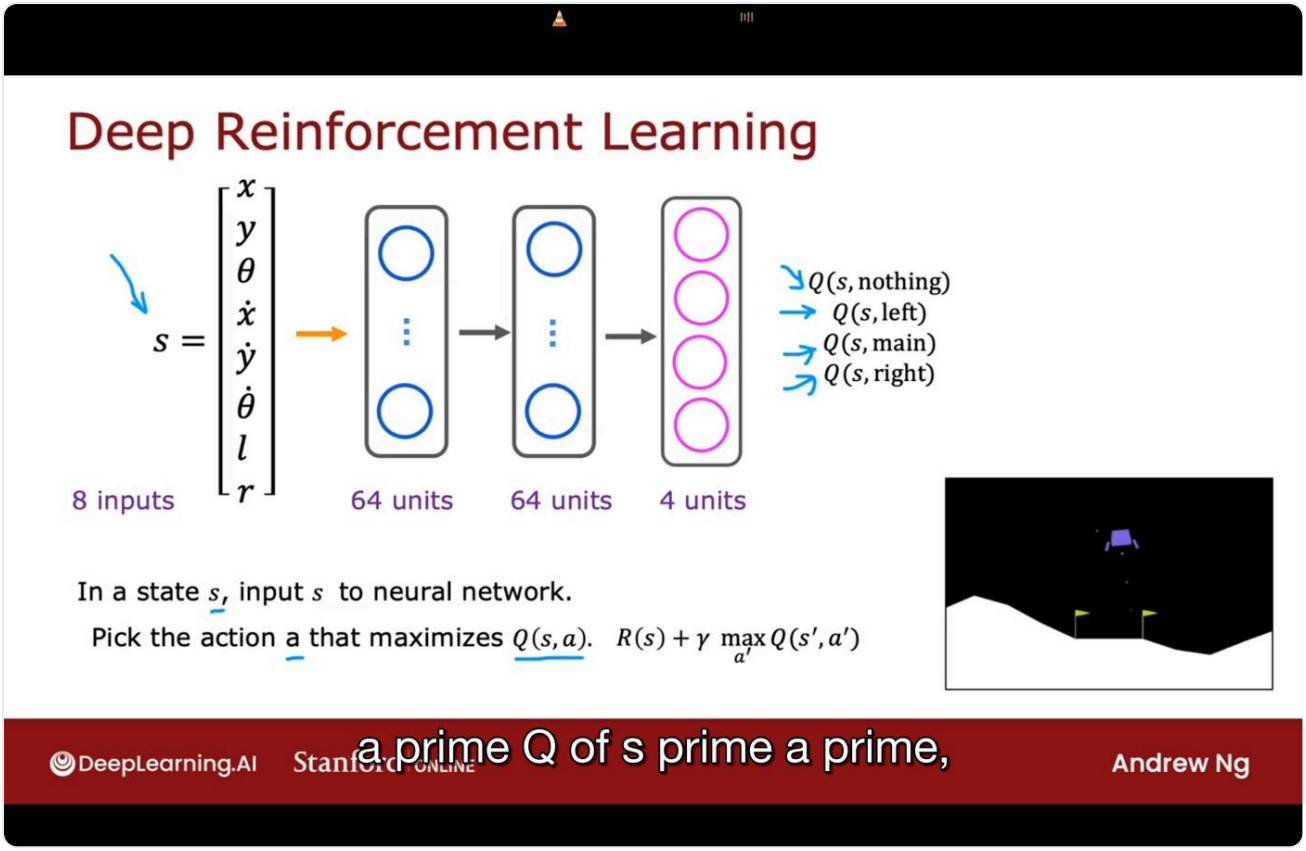


FIGURE 41.4

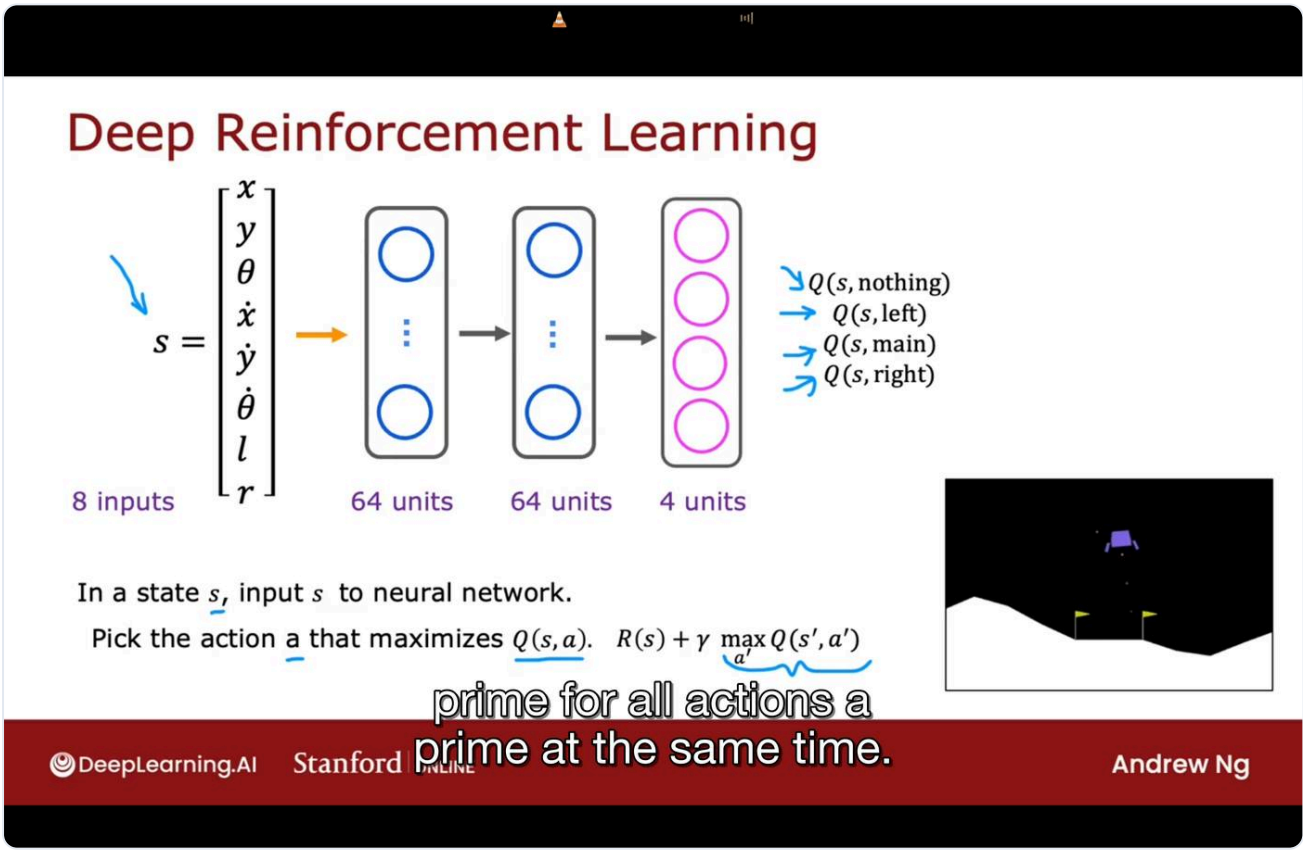


FIGURE 41.5

This turns out to be more efficient because given the state s we can run inference just once and get all four of these values, and then very quickly pick the action a that maximizes Q of s, a. You notice also in Bellman's equations, there's a step in which we have to compute max over a prime Q of s prime a prime, this multiplied by gamma and then there was plus R of s up here. This neural network also makes it much more efficient to compute this because we're getting Q of s prime a prime for all actions a prime at the same time. You can then just pick the max to compute this value for the right-hand side of Bellman's equations. This change to the neural network architecture makes the algorithm much more efficient, and so we will be using this architecture in the practice lab.

Next, there's one other idea that'll help the algorithm a lot which is something called an Epsilon-greedy policy, which affects how you choose actions even while you're still learning. Let's take a look at the next video , and what that means.

LECTURE 42

Learning Algorithm

The learning algorithm that we developed, even while you're still learning how to approximate Q(s,a), you need to take some actions in the lunar lander. How do you pick those actions while you're still learning? The most common way to do so is to use something called an Epsilon-greedy policy. Let's take a look at how that works. Here's the algorithm that you saw earlier.

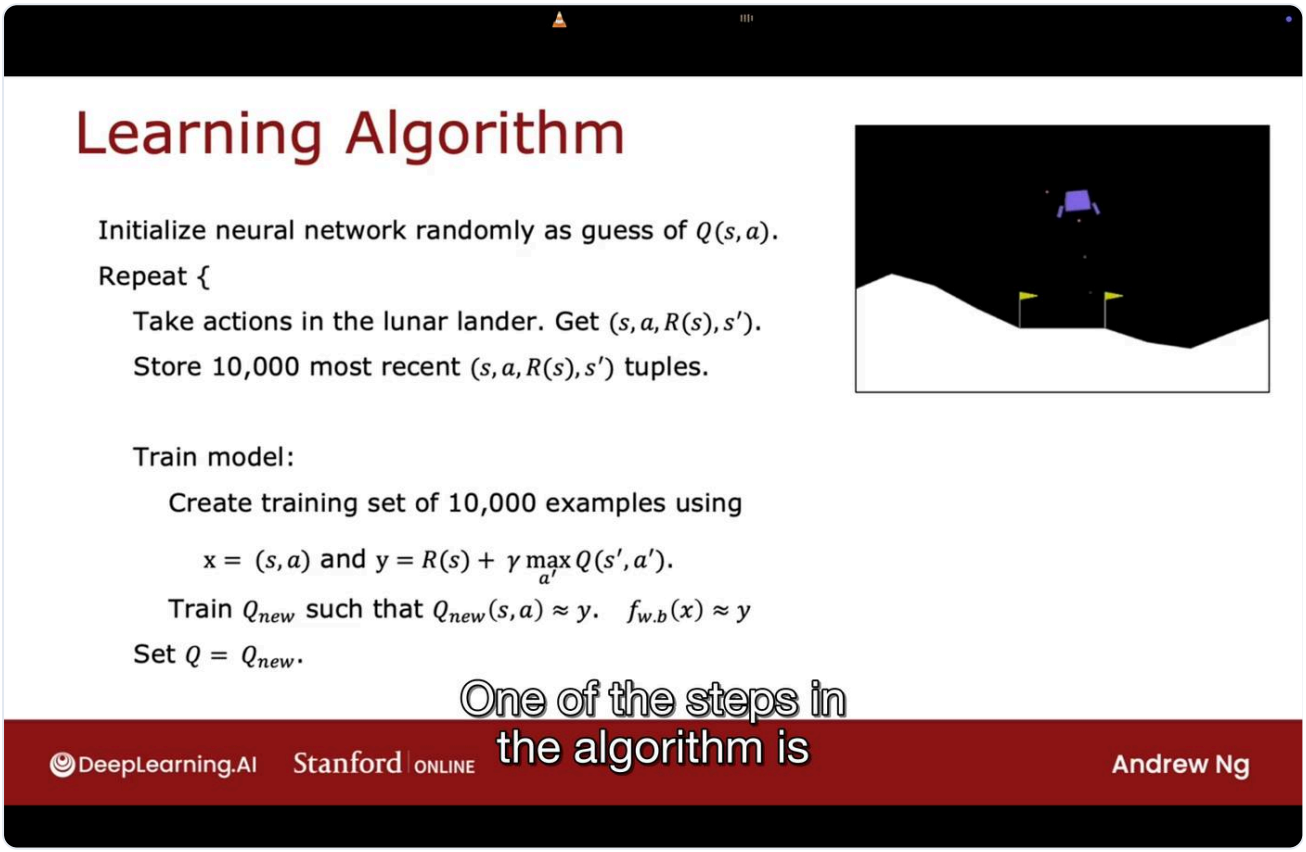


FIGURE 42.1 | Learning Algorithm

One of the steps in the algorithm is to take actions in the lunar lander. When the learning algorithm is still running, we don't really know what's the best action to take in every state.

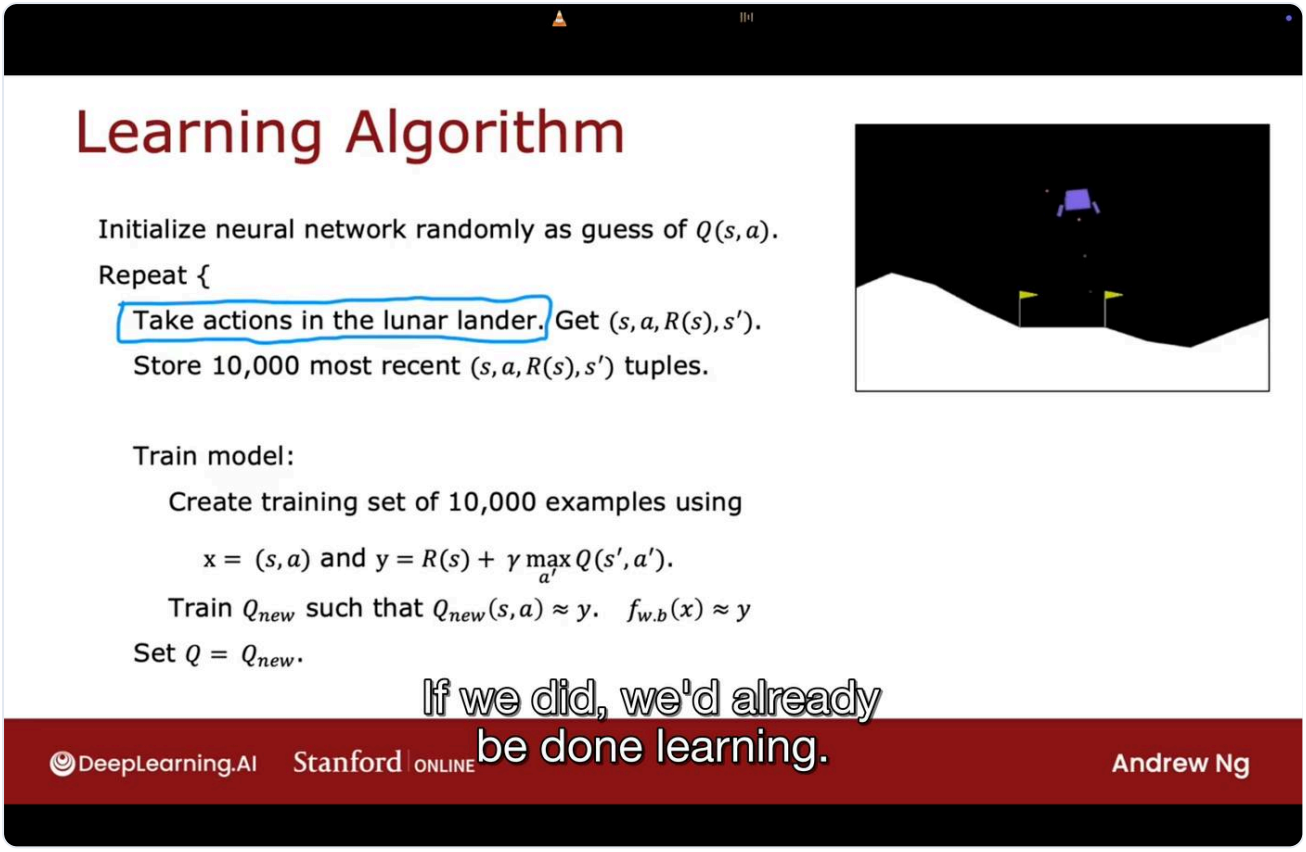


FIGURE 42.2 | Learning Algorithm

If we did, we'd already be done learning. But even while we're still learning and don't have a very good estimate of Q(s,a) yet, how do we take actions in this step of the learning algorithm? Let's look at some options. When you're in some state s, we might not want to take actions totally at random because that will often be a bad action. One natural option would be to pick whenever in state s, pick an action a that maximizes Q(s,a). We may say, even if Q(s,a) is not a great estimate of the Q function, let's just do our best and use our current guess of Q(s,a) and pick the action a that maximizes it. It turns out this may work okay, but isn't the best option.

Instead, here's what is commonly done. Here's option 2, which is most of the time, let's say with probability of 0.95, pick the action that maximizes Q(s,a). Most of the time we try to pick a good action using our current guess of Q(s,a). But the small fraction of the time, let's say, five percent of the time, we'll pick an action a randomly.

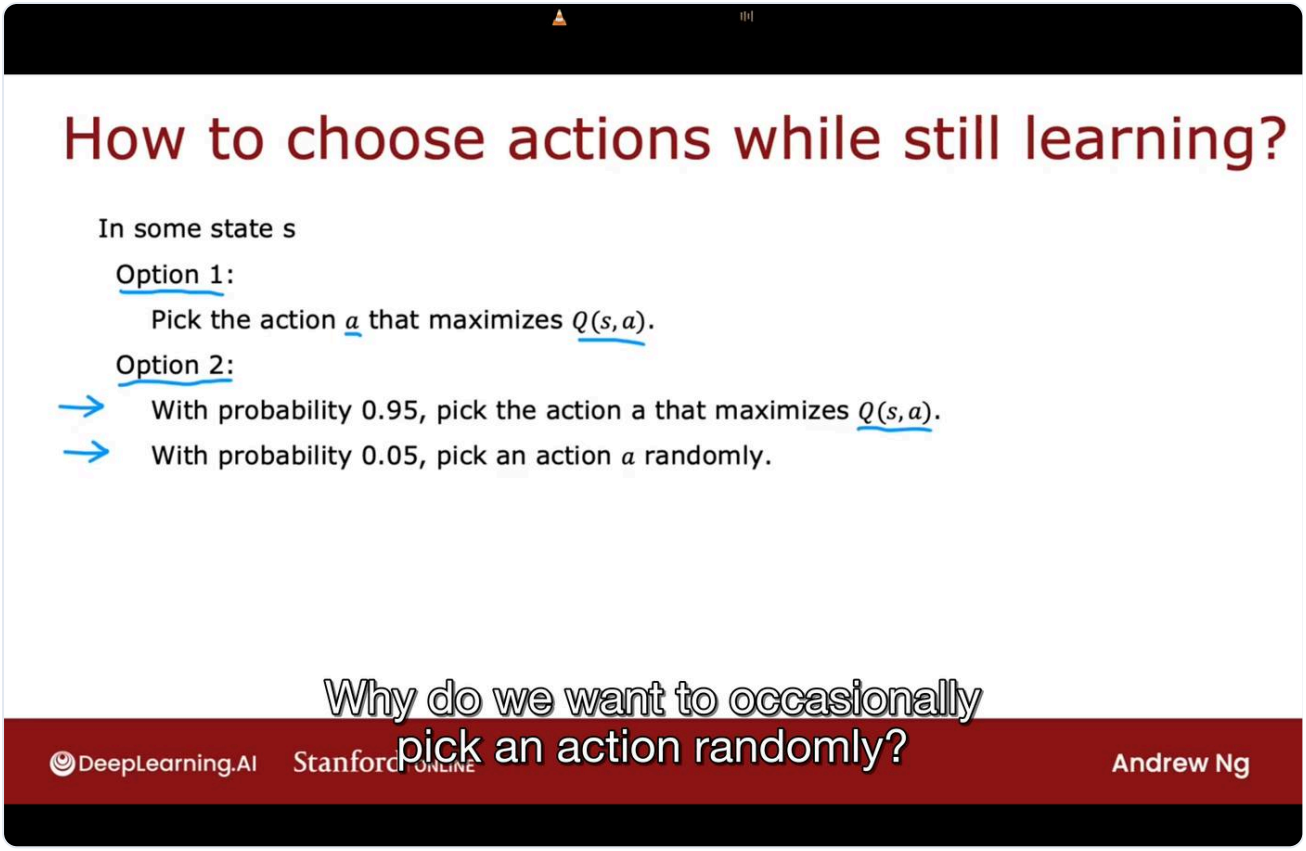


FIGURE 42.3 | How to Choose Actions While Still Learning?

Why do we want to occasionally pick an action randomly? Well, here's why. Suppose there's some strange reason that Q(s,a) was initialized randomly so that the learning algorithm thinks that firing the main thruster is never a good idea. Maybe the neural network parameters were initialized so that Q(s, main) is always very low. If that's the case, then the neural network, because it's trying to pick the action a that maximizes Q(s,a), it will never ever try firing the main thruster. Because it never ever tries firing the main thruster, it will never learn that firing the main thruster is actually sometimes a good idea. Because of the random initialization, if the neural network somehow initially gets stuck in this mind that some things are bad idea, just by chance, then option 1, it means that it will never try out those actions and discover that maybe is actually a good idea to take that action, like fire the main thrusters sometimes.

Under option 2 on every step, we have some small probability of trying out different actions so that the neural network can learn to overcome its own possible preconceptions about what might be a bad idea that turns out not to be the case. This idea of picking actions randomly is sometimes called an exploration step. Because we're going to try out something that may not be the best idea, but we're going to just try out some action in some circumstances, explore and learn more about an action in the circumstance where we may not have had as much experience before.

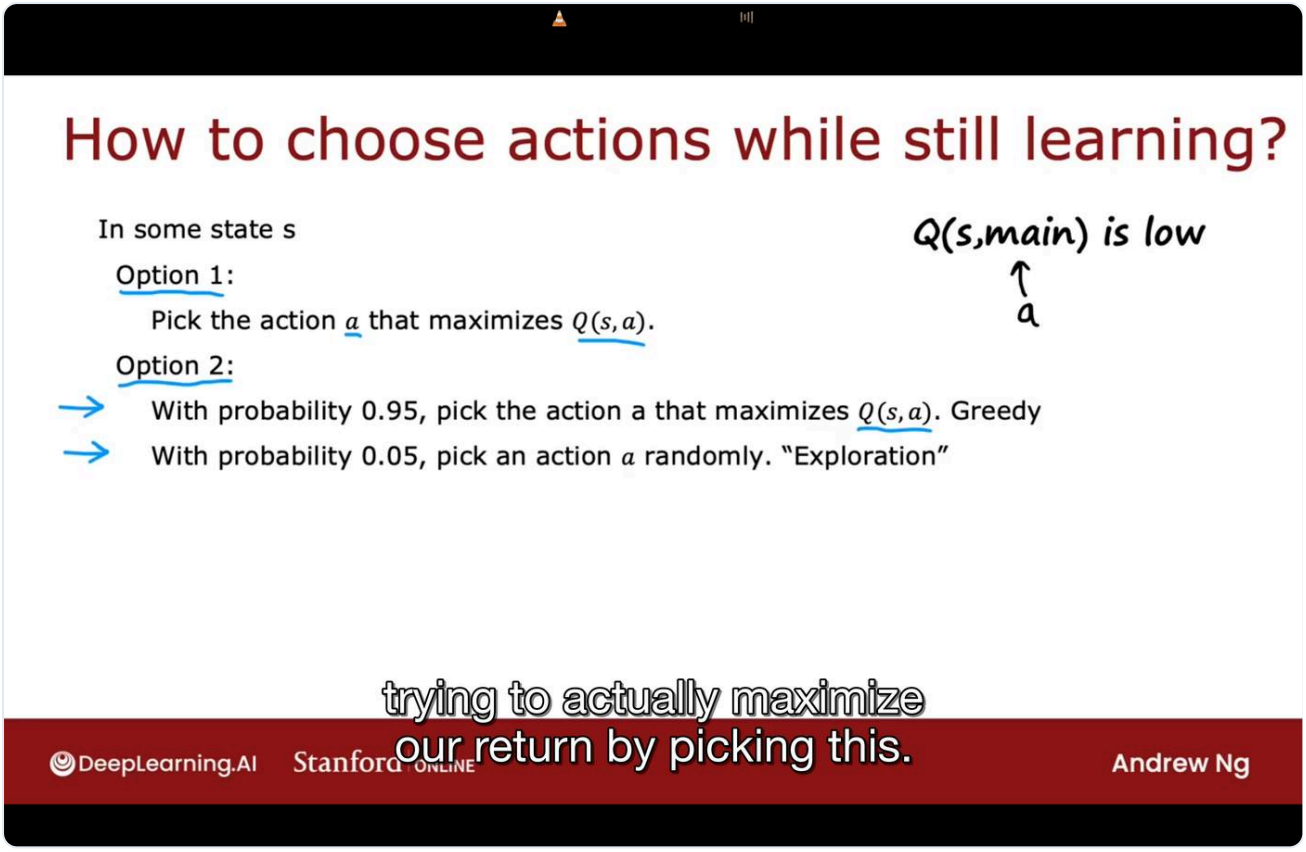


FIGURE 42.4 | How to Choose Actions While Still Learning?

Taking an action that maximizes Q(s,a), sometimes this is called a greedy action because we're trying to actually maximize our return by picking this. Or in the reinforcement learning literature, sometimes you'll also hear this as an exploitation step.

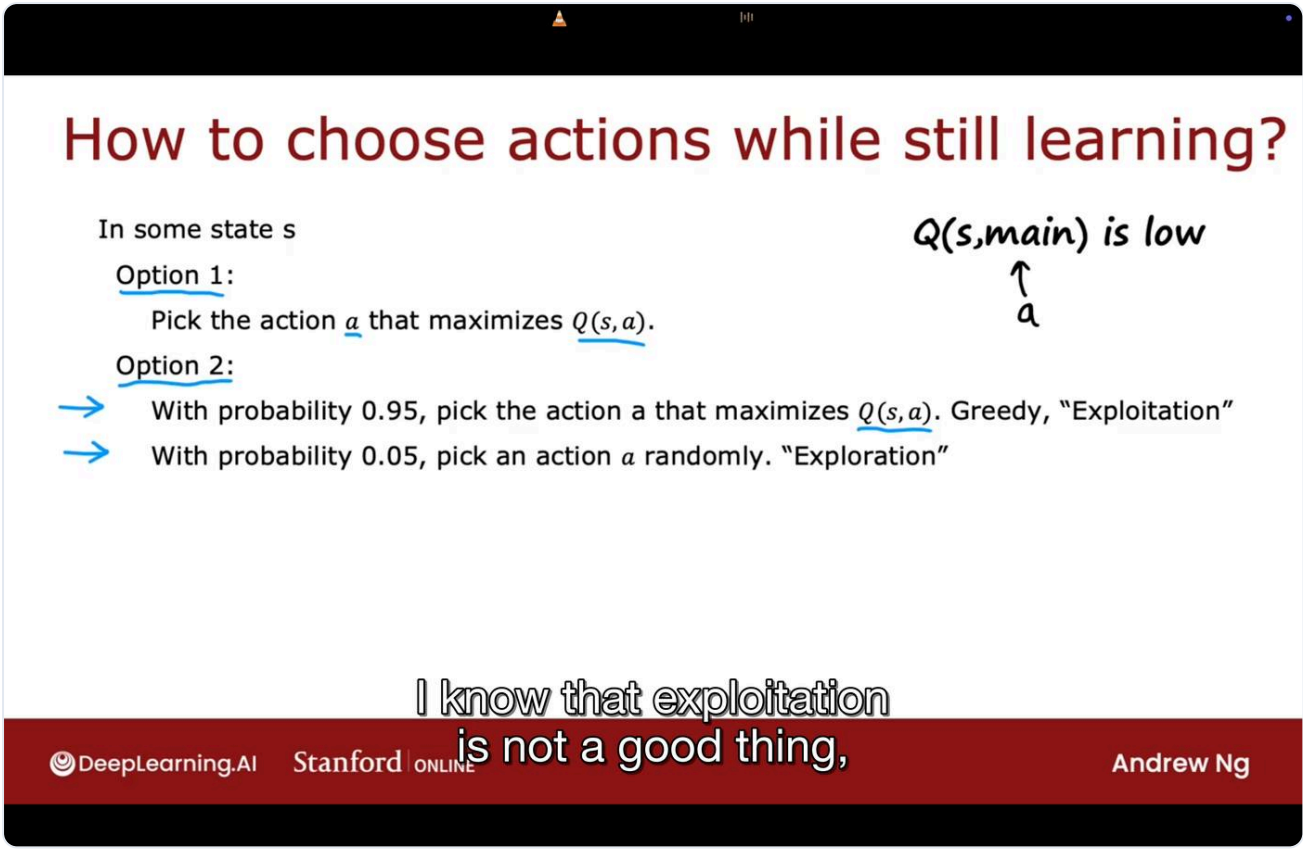


FIGURE 42.5 | How to Choose Actions While Still Learning?

I know that exploitation is not a good thing, nobody should ever explore anyone else. But historically, this was the term used in reinforcement learning to say, let's exploit everything we've learned to do the best we can. In the reinforcement learning literature, sometimes you hear people talk about the exploration versus exploitation trade-off, which refers to how often do you take actions randomly or take actions that may not be the best in order to learn more, versus trying to maximize your return by say, taking the action that maximizes Q (s,a).

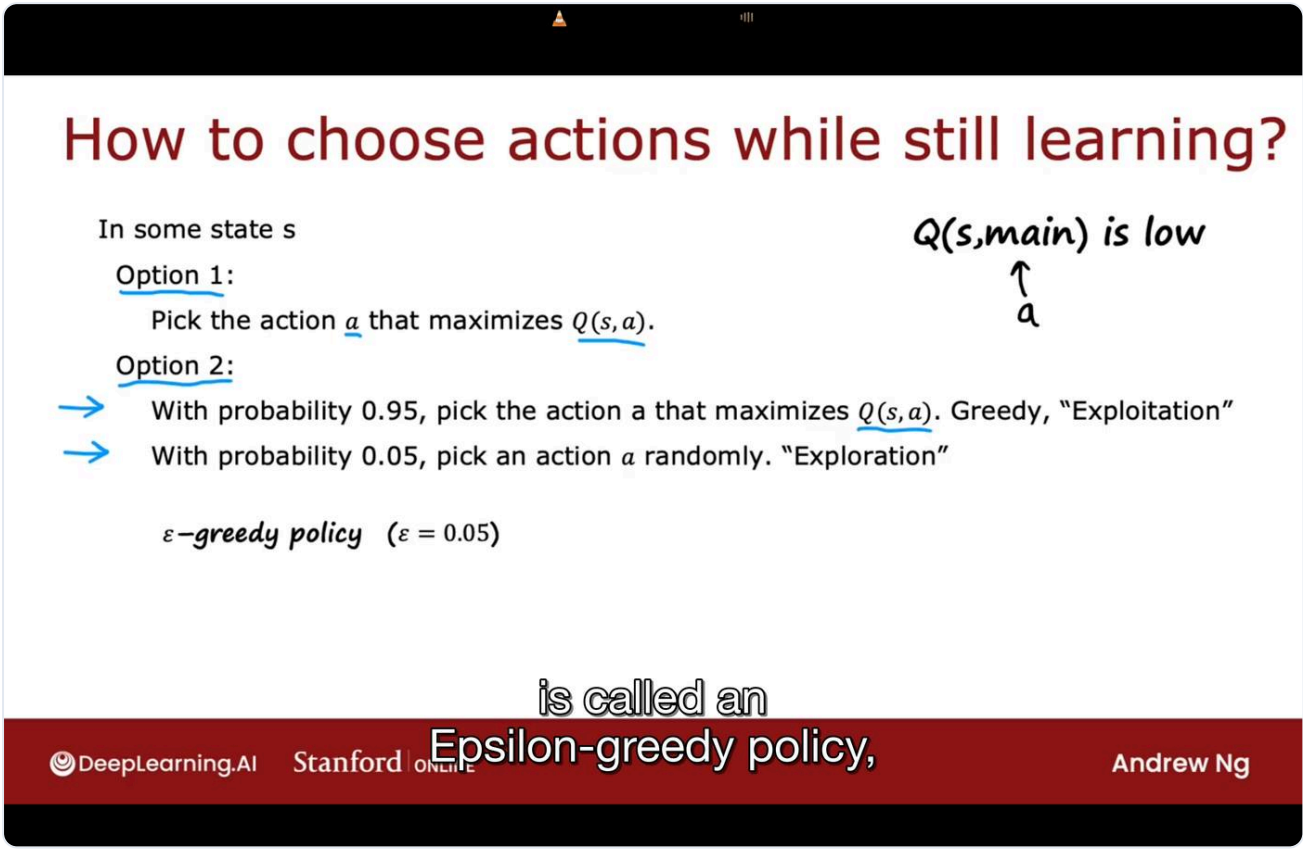


FIGURE 42.6 | How to Choose Actions While Still Learning?

This approach, that is option 2, has a name, is called an Epsilon-greedy policy, where here Epsilon is 0.05 is the probability of picking an action randomly. This is the most common way to make your reinforcement learning algorithm explore a little bit, even whilst occasionally or maybe most of the time taking greedy actions. By the way, lot of people have commented that the name Epsilon-greedy policy is confusing because you're actually being greedy 95 percent of the time, not five percent of the time. So maybe 1 minus Epsilon-greedy policy, because it's 95 percent greedy, five percent exploring, that's actually a more accurate description of the algorithm.

But for historical reasons, the name Epsilon-greedy policy is what has stuck.

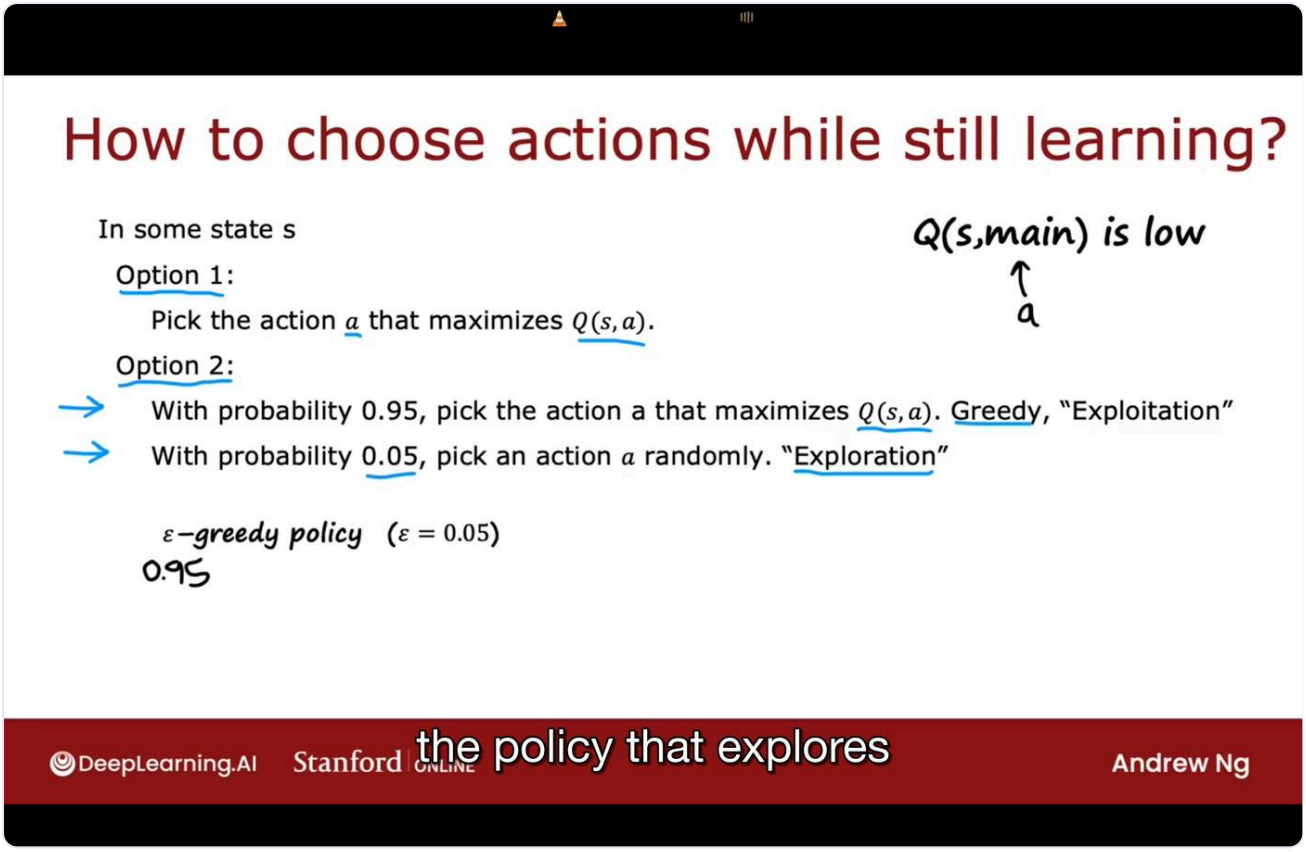


FIGURE 42.7 | How to Choose Actions While Still Learning?

This is the name that people use to refer to the policy that explores actually Epsilon fraction of the time rather than this greedy Epsilon fraction of the time. Lastly, one of the trick that's sometimes used in reinforcement learning is to start off Epsilon high.

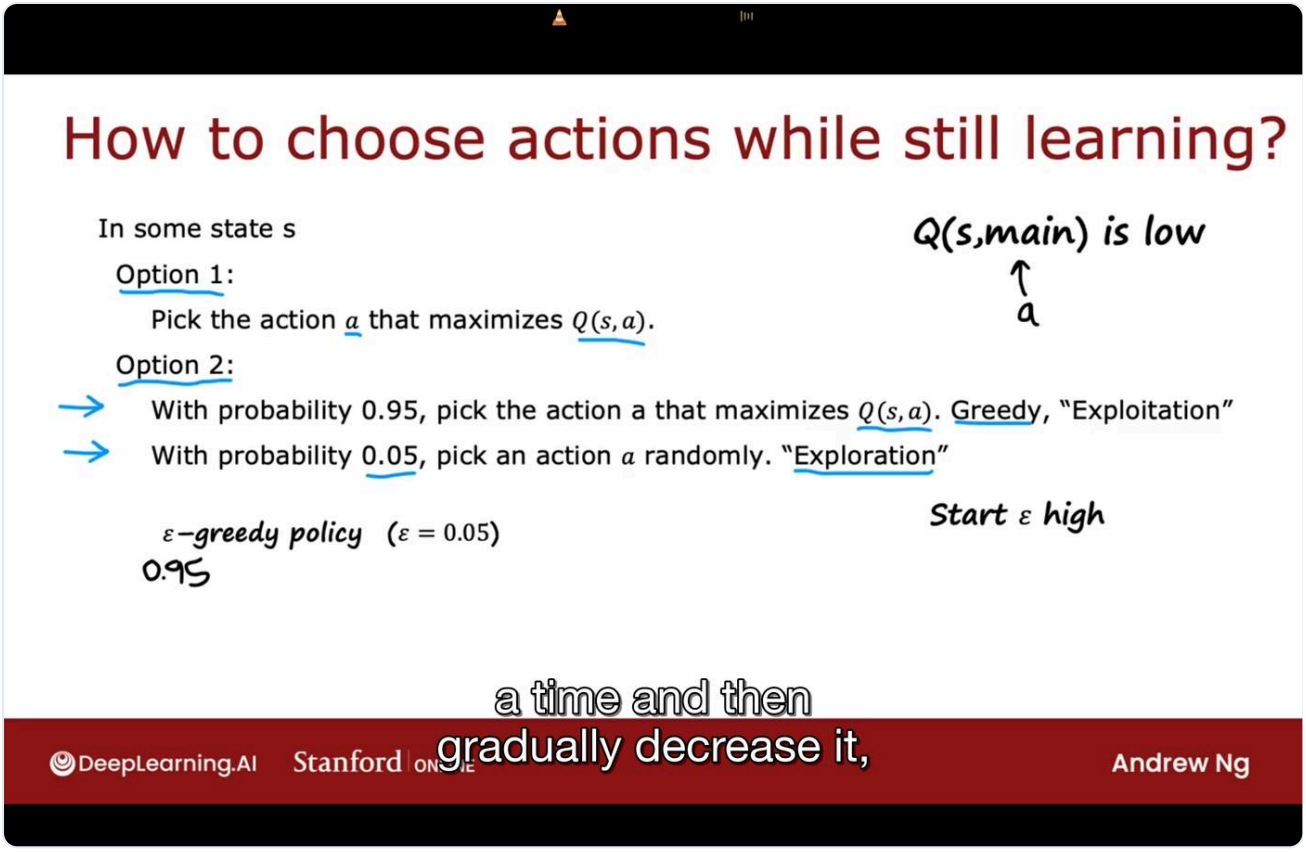


FIGURE 42.8 | How to Choose Actions While Still Learning?

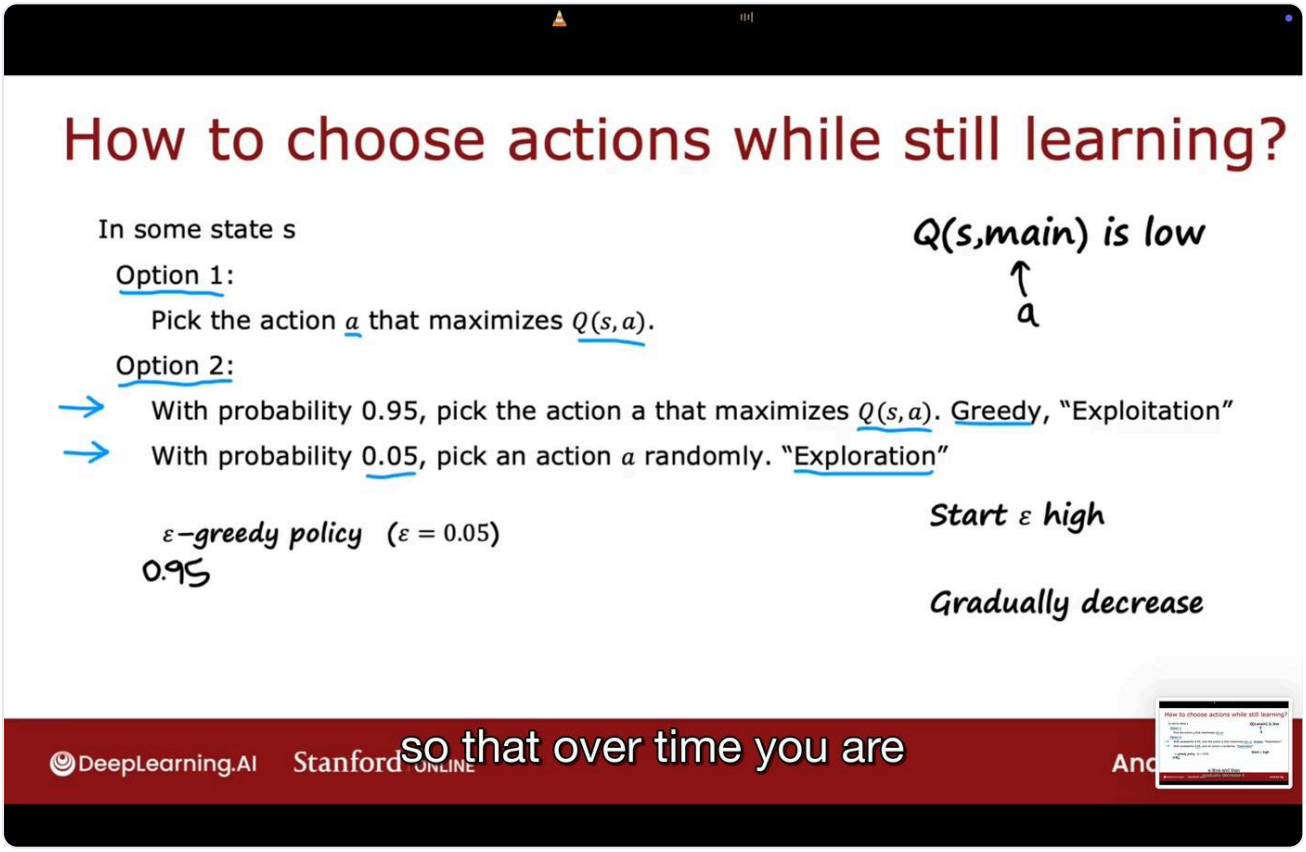


FIGURE 42.9 | How to Choose Actions While Still Learning?

Initially, you are taking random actions a lot at a time and then gradually decrease it, so that over time you are less likely to take actions randomly and more likely to use your improving estimates of the Q-function to pick good actions. For example, in the lunar lander exercise, you might start off with Epsilon very, very high, maybe even Epsilon equals 1.0.

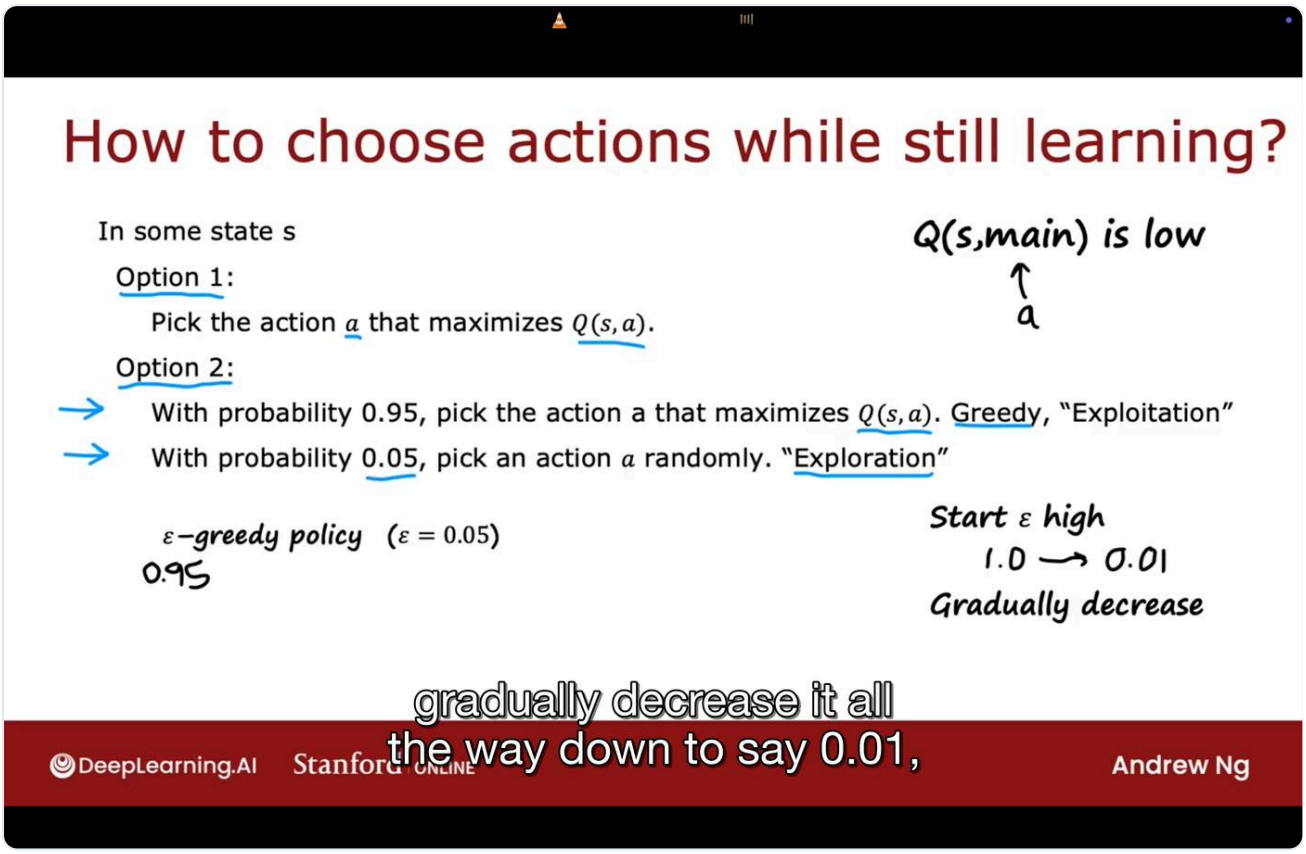


FIGURE 42.10 | How to Choose Actions While Still Learning?

You're just picking actions completely at random initially and then gradually decrease it all the way down to say 0.01, so that eventually you're taking greedy actions 99 percent of the time and acting randomly only a very small one percent of the time. If this seems complicated, don't worry about it. We'll provide the code in the Jupyter lab that shows you how to do this. If you were to implement the algorithm as we've described it with the more efficient neural network architecture and with an Epsilon-greedy exploration policy, you find that they work pretty well on the lunar lander. One of the things that I've noticed for reinforcement learning algorithm is that compared to supervised learning, they're more finicky in terms of the choice of hyper parameters.

For example, in supervised learning, if you set the learning rate a little bit too small, then maybe the algorithm will take longer to learn. Maybe it takes three times as long to train, which is annoying, but maybe not that bad. Whereas in reinforcement learning, find that if you set the value of Epsilon not quite as well, or set other parameters not quite as well, it doesn't take three times as long to learn. It may take 10 times or a 100 times as long to learn. Reinforcement learning algorithms, I think because they're are less mature than supervised learning algorithms, are much more finicky to little choices of parameters like that, and it actually sometimes is frankly more frustrating to tune these parameters with reinforcement learning algorithm compared to a supervised learning algorithm.

But again, if you're worried about the practice lab, the program exercise, we'll give you a sense of good parameters to use in the program exercise so that you should be able to do that and successfully learn the lunar lander, hopefully without too many problems. In the next optional video, I want us to drive a couple more algorithm refinements, mini batching, and also using soft updates. Even without these additional refinements, the algorithm will work okay, but these are additional refinements that make the algorithm run much faster. It's okay if you skip this video, we've provided everything you need in the practice lab to hopefully successfully complete it.

But if you're interested in learning about more of these details of two named reinforcement learning algorithms, then come with me and let's see in the next video, mini batching and soft updates.

LECTURE 43

Learning Algorithm

In this video, we'll look at two further refinements to the reinforcement learning algorithm you've seen. The first idea is called using mini-batches, and this turns out to be an idea they can both speedup your reinforcement learning algorithm and it's also applicable to supervised learning. They can help you speed up your supervised learning algorithm as well, like training a neural network, or training a linear regression, or logistic regression model. The second idea we'll look at is soft updates, which it turns out will help your reinforcement learning algorithm do a better job to converge to a good solution. Let's take a look at mini-batches and soft updates.

To understand mini-batches, let's just look at supervised learning to start.

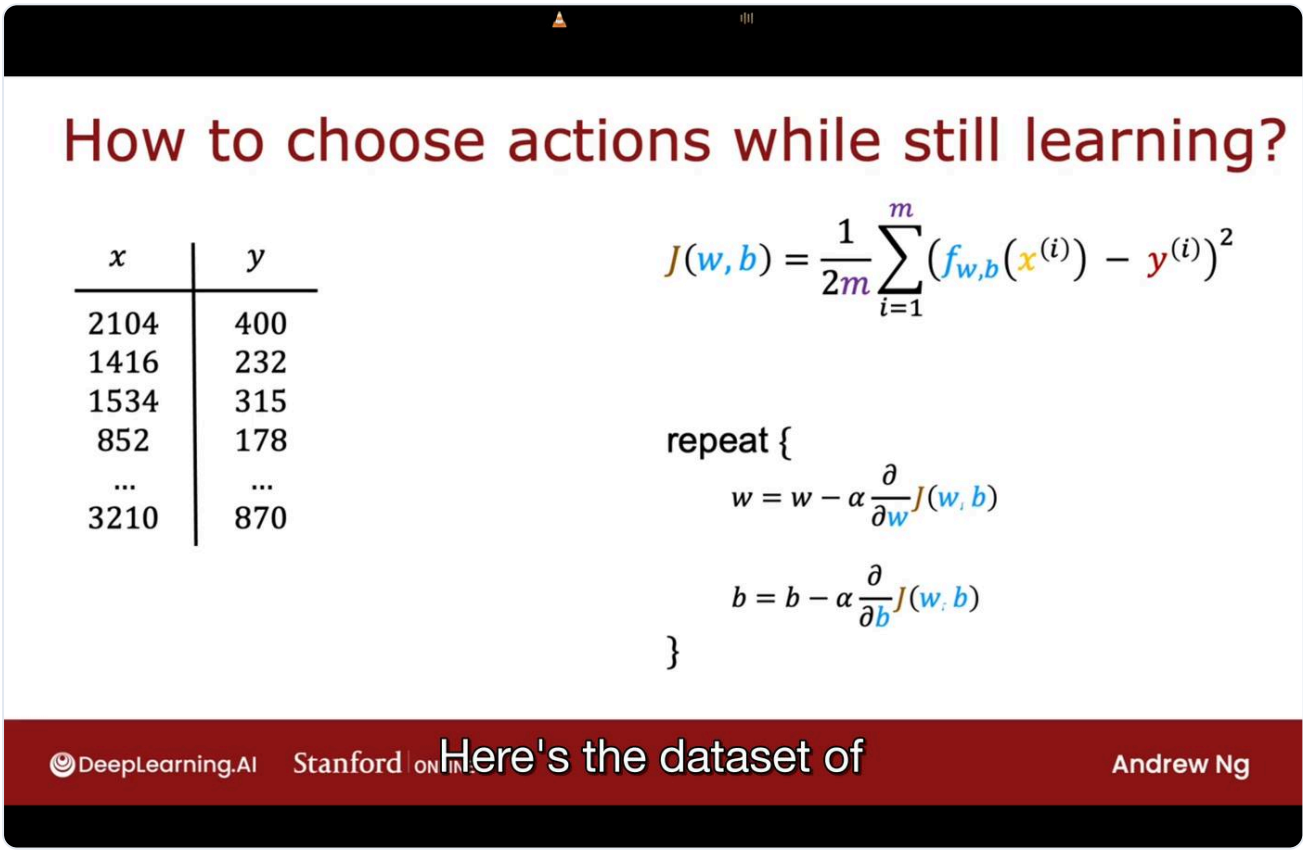


FIGURE 43.1

Here's the dataset of housing sizes and prices that you had seen way back in the first course of this specialization on using linear regression to predict housing prices. There we had come up with this cost function for the parameters w and b, it was 1 over 2m, sum of the prediction minus the actual value y^2. The gradient descent algorithm was to repeatedly update w as w minus the learning rate alpha times the partial derivative respect to w of the cost J of wb, and similarly to update b as follows. Let me just take this definition of J of wb and substitute it in here.

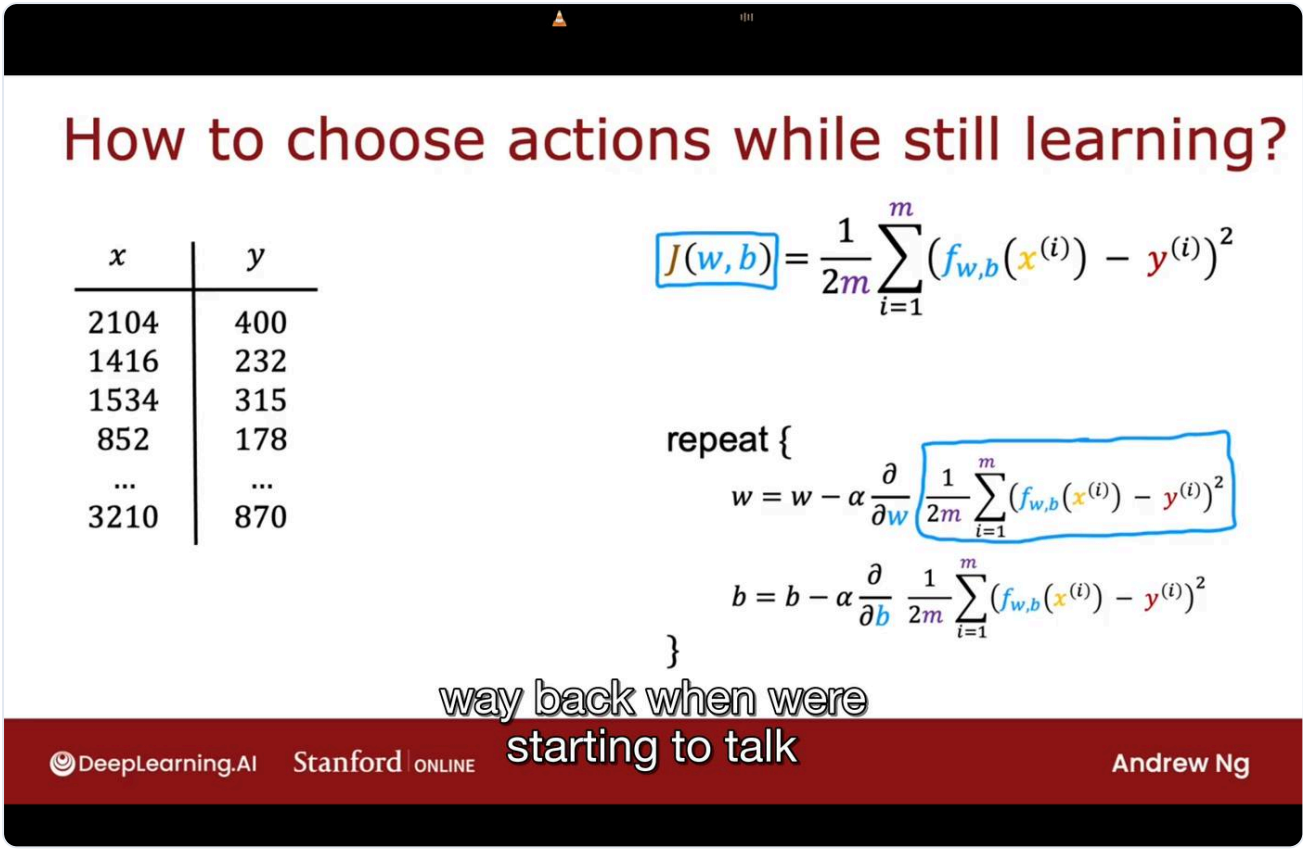


FIGURE 43.2

Now, when we looked at this example, way back when were starting to talk about linear regression and supervised learning, the training set size m was pretty small. I think we had 47 training examples. But what if you have a very large training set? Say m equals 100 million. There are many countries including the United States with over a 100 million housing units, and so a national census will give you a dataset that is this order of magnitude or size.

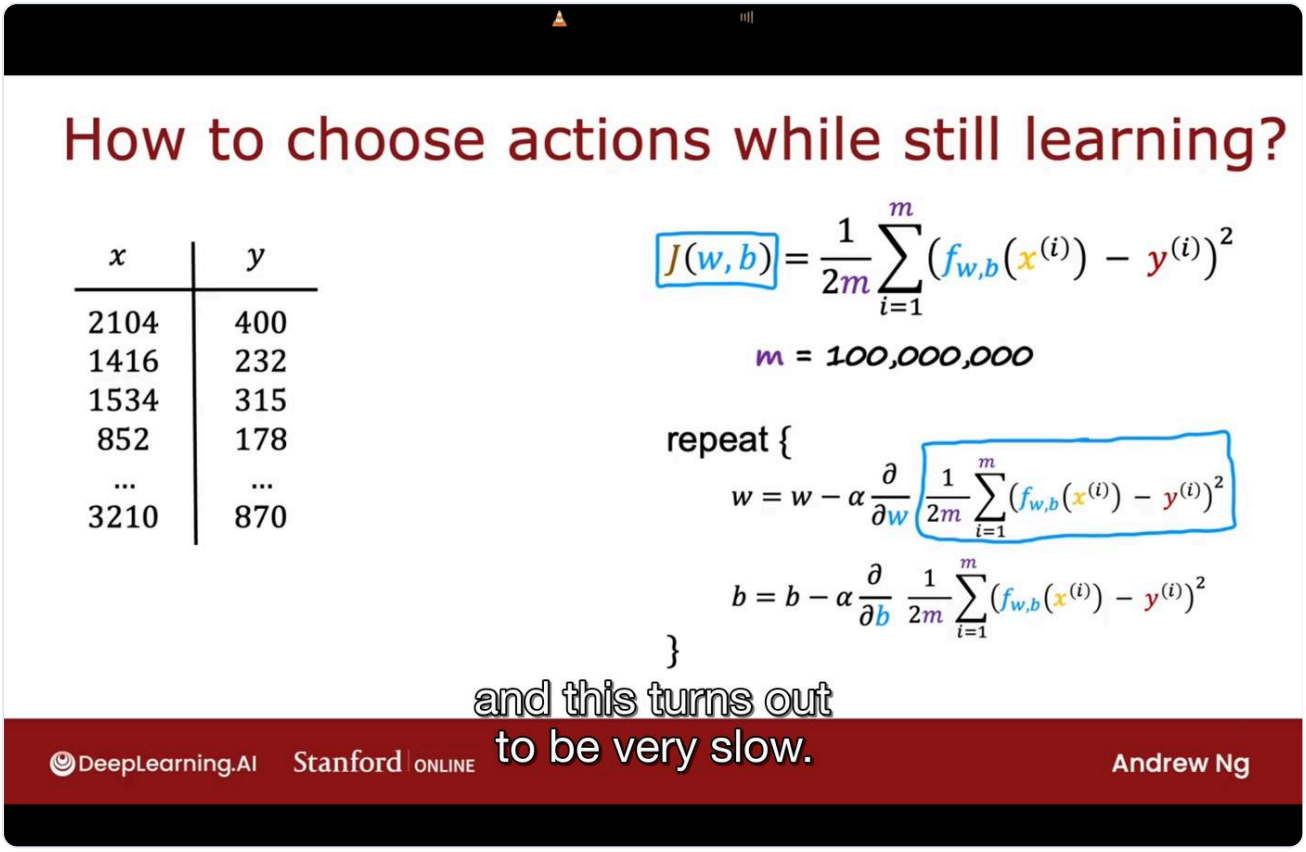


FIGURE 43.3

The problem with this algorithm when your dataset is this big, is that every single step of gradient descent requires computing this average over 100 million examples, and this turns out to be very slow. Every step of gradient descent means you would compute this sum or this average over 100 million examples. Then you take one tiny gradient descent step and you go back and have to scan over your entire 100 million example dataset again to compute the derivative on the next step, they take another tiny gradient descent step and so on and so on.

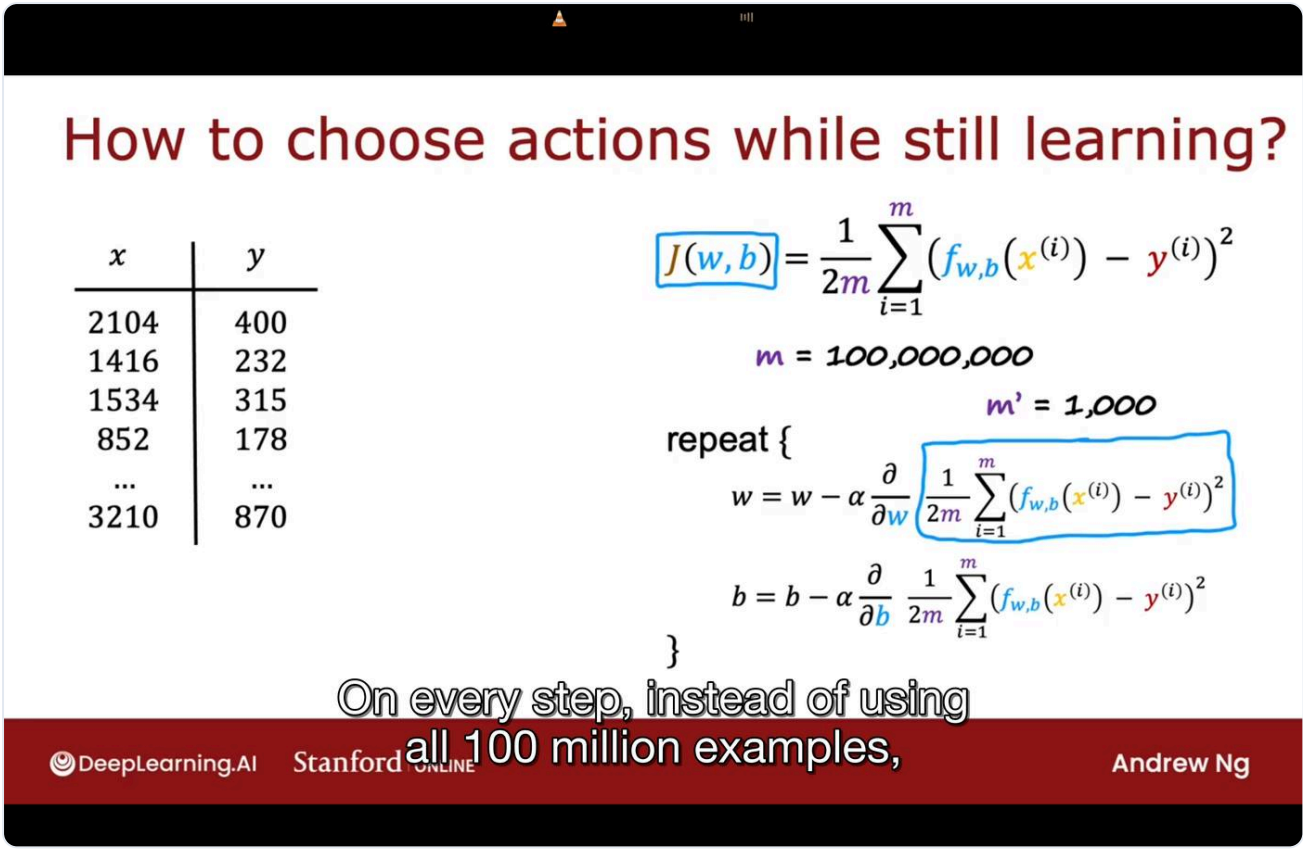


FIGURE 43.4

When the training set size is very large, this gradient descent algorithm turns out to be quite slow. The idea of mini-batch gradient descent is to not use all 100 million training examples on every single iteration through this loop. Instead, we may pick a smaller number, let me call it m prime equals say, 1,000.

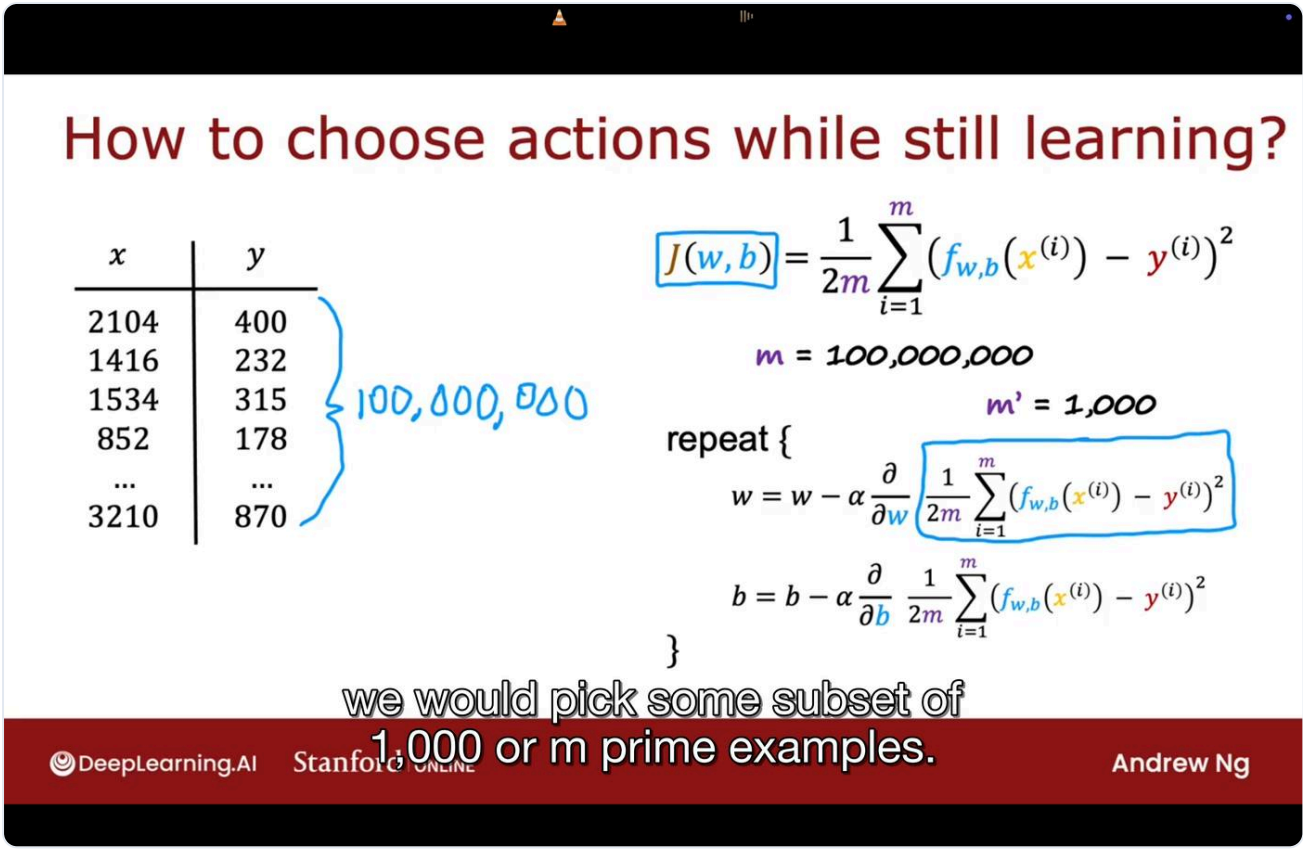


FIGURE 43.5

On every step, instead of using all 100 million examples, we would pick some subset of 1,000 or m prime examples.

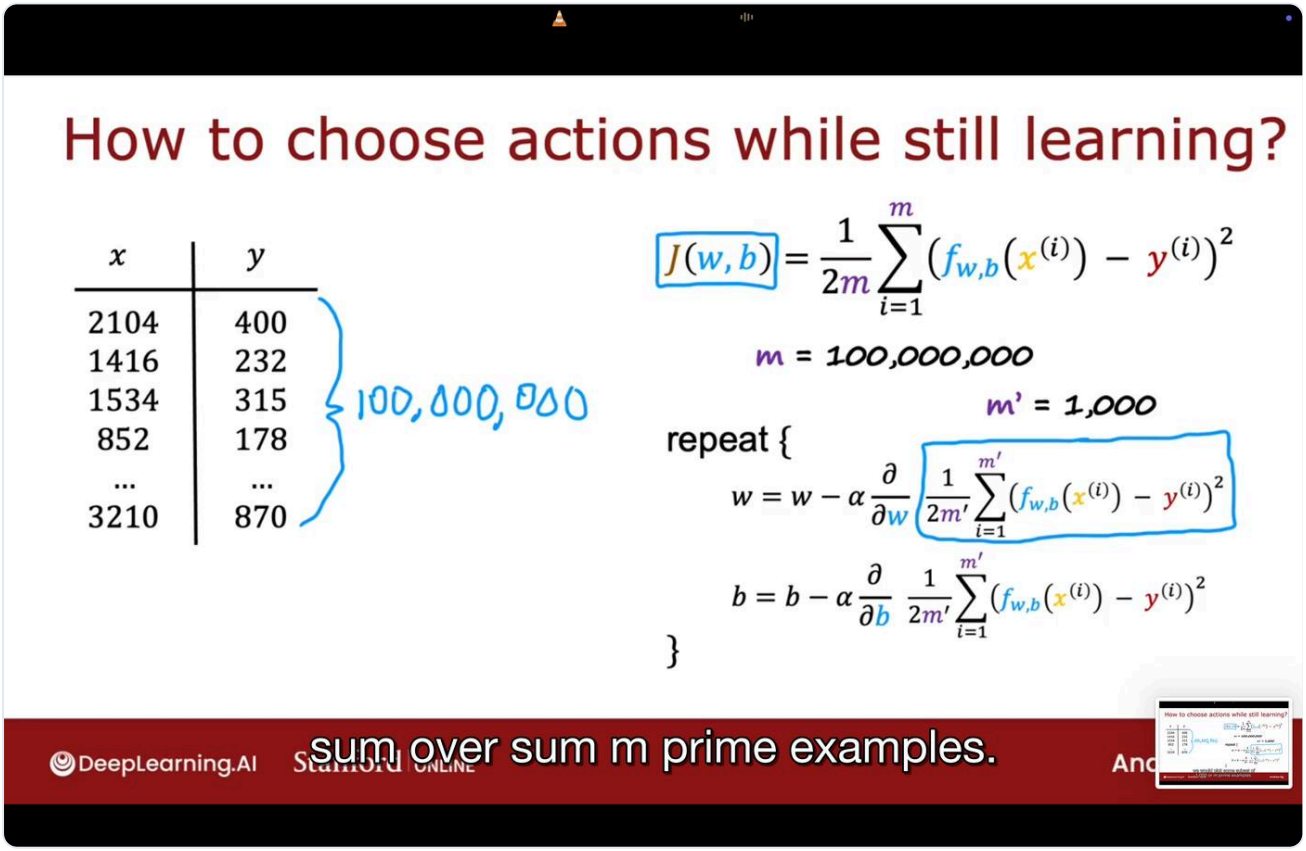


FIGURE 43.6

This inner term becomes 1 over 2m prime is sum over sum m prime examples. Now each iteration through gradient descent requires looking only at the 1,000 rather than 100 million examples, and every step takes much less time and just leads to a more efficient algorithm. What mini-batch gradient descent does is on the first iteration through the algorithm, may be it looks at that subset of the data. On the next iteration, maybe it looks at that subset of the data, and so on.

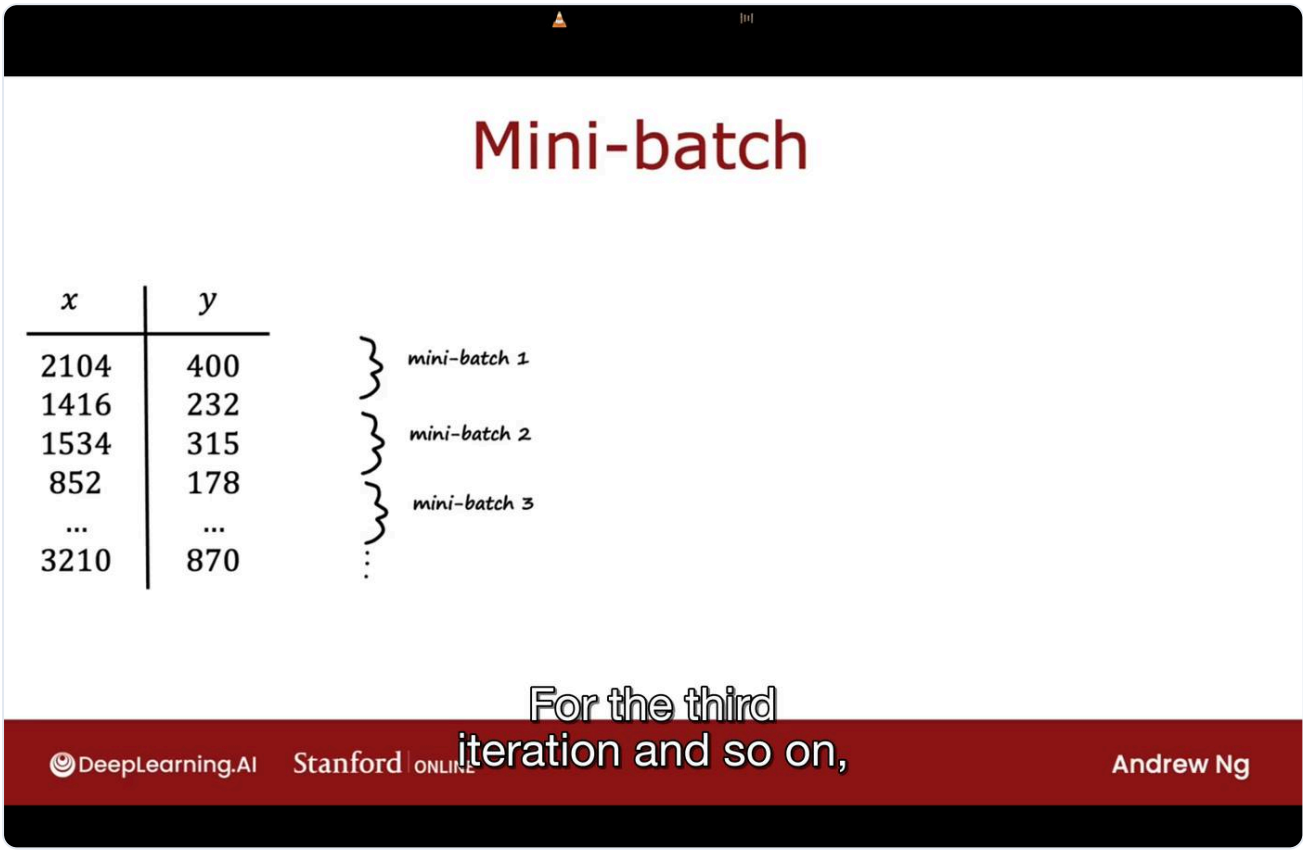


FIGURE 43.7

For the third iteration and so on, so that every iteration is looking at just a subset of the data so each iteration runs much more quickly. To see why this might be a reasonable algorithm, here's the housing dataset. If on the first iteration we were to look at just say five examples, this is not the whole dataset but it's slightly representative of the string line you might want to fit in the end, and so taking one gradient descent step to make the algorithm better fit these five examples is okay. But then on the next iteration, you take a different five examples like that shown here. You take one gradient descent step using these five examples, and on the next iteration you use a different five examples and so on and so forth.

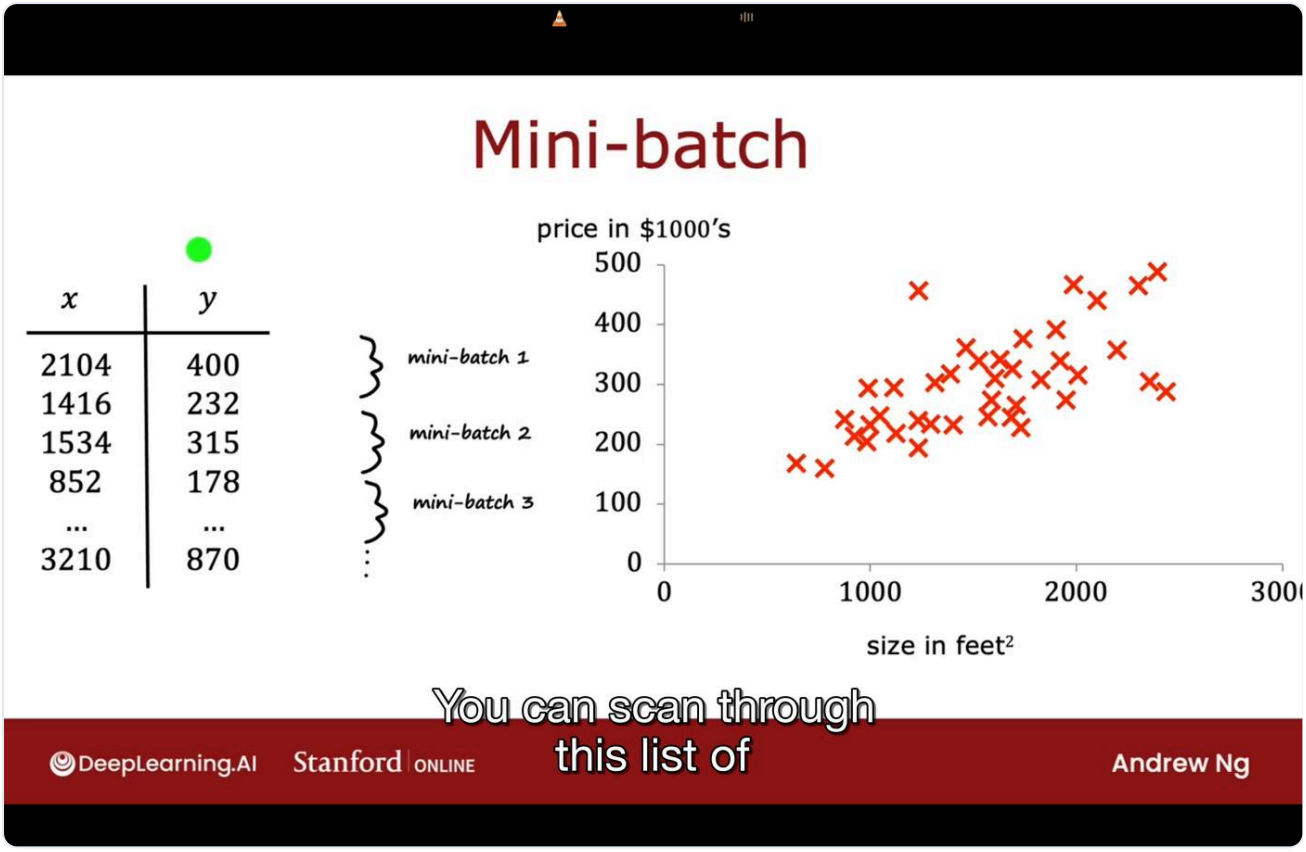


FIGURE 43.8

You can scan through this list of examples from top to bottom. That would be one way.

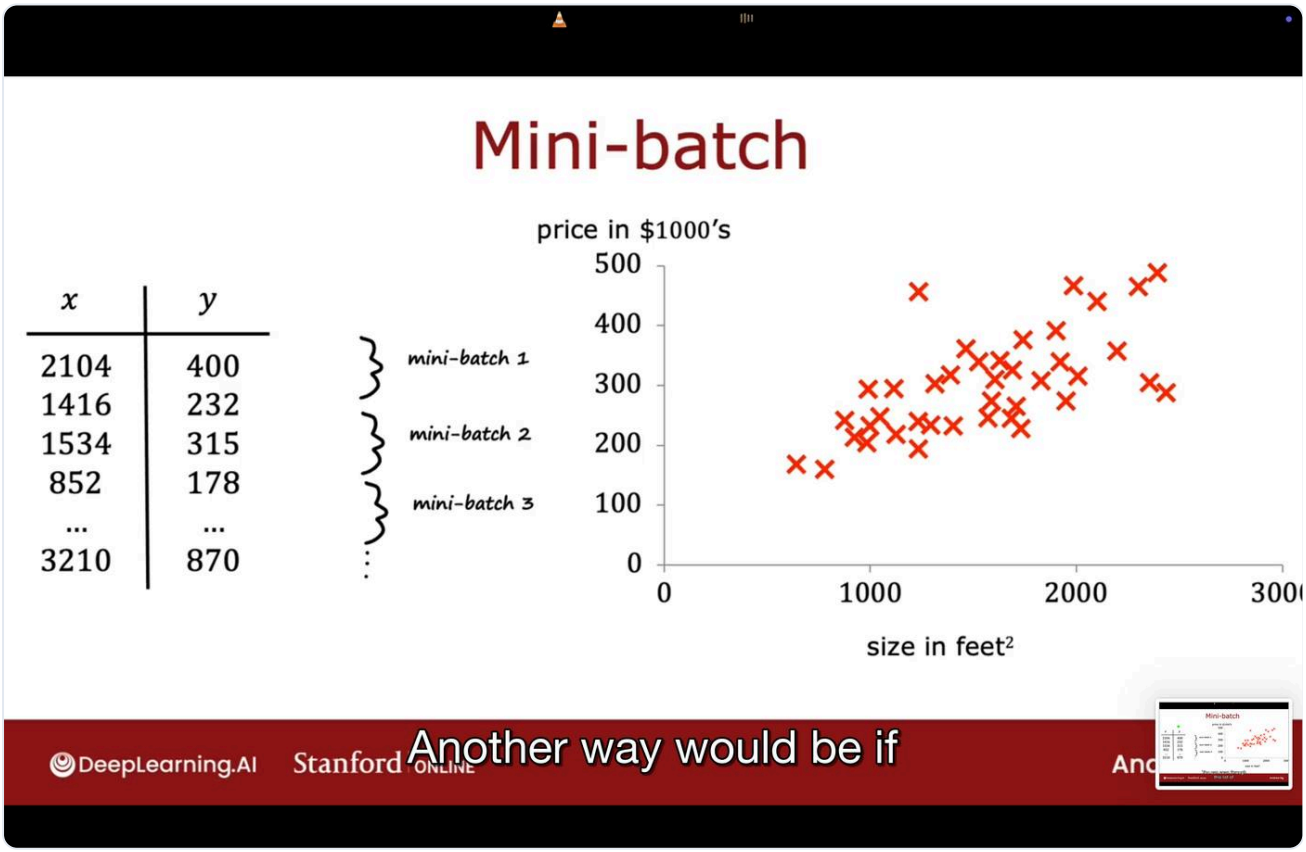


FIGURE 43.9

Another way would be if on every single iteration you just pick a totally different five examples to use. You might remember with batch gradient descent, if these are the contours of the cost function J.

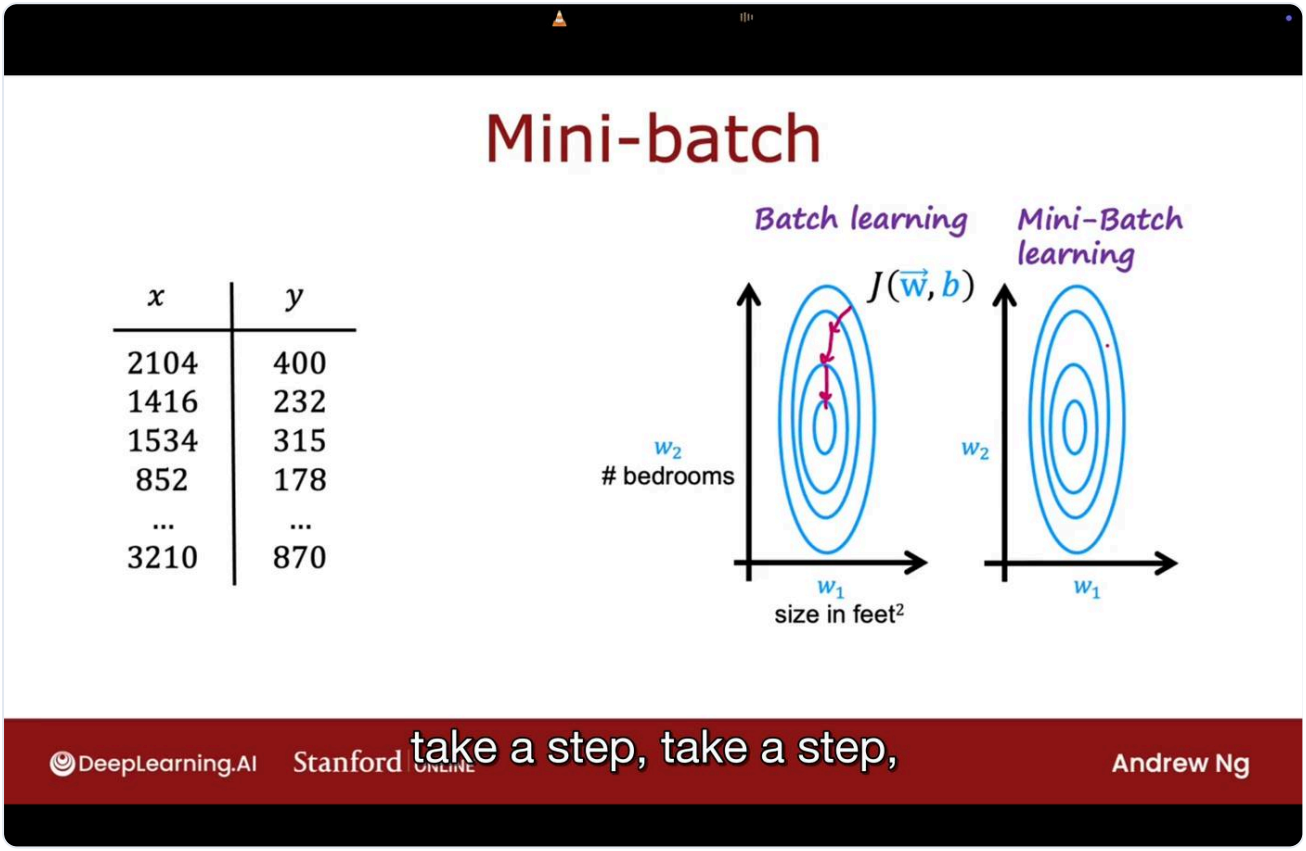


FIGURE 43.10

Then batch gradient descent would say, start here and take a step, take a step, take a step, take a step, take a step. Every step of gradient descent causes the parameters to reliably get closer to the global minimum of the cost function here in the middle.

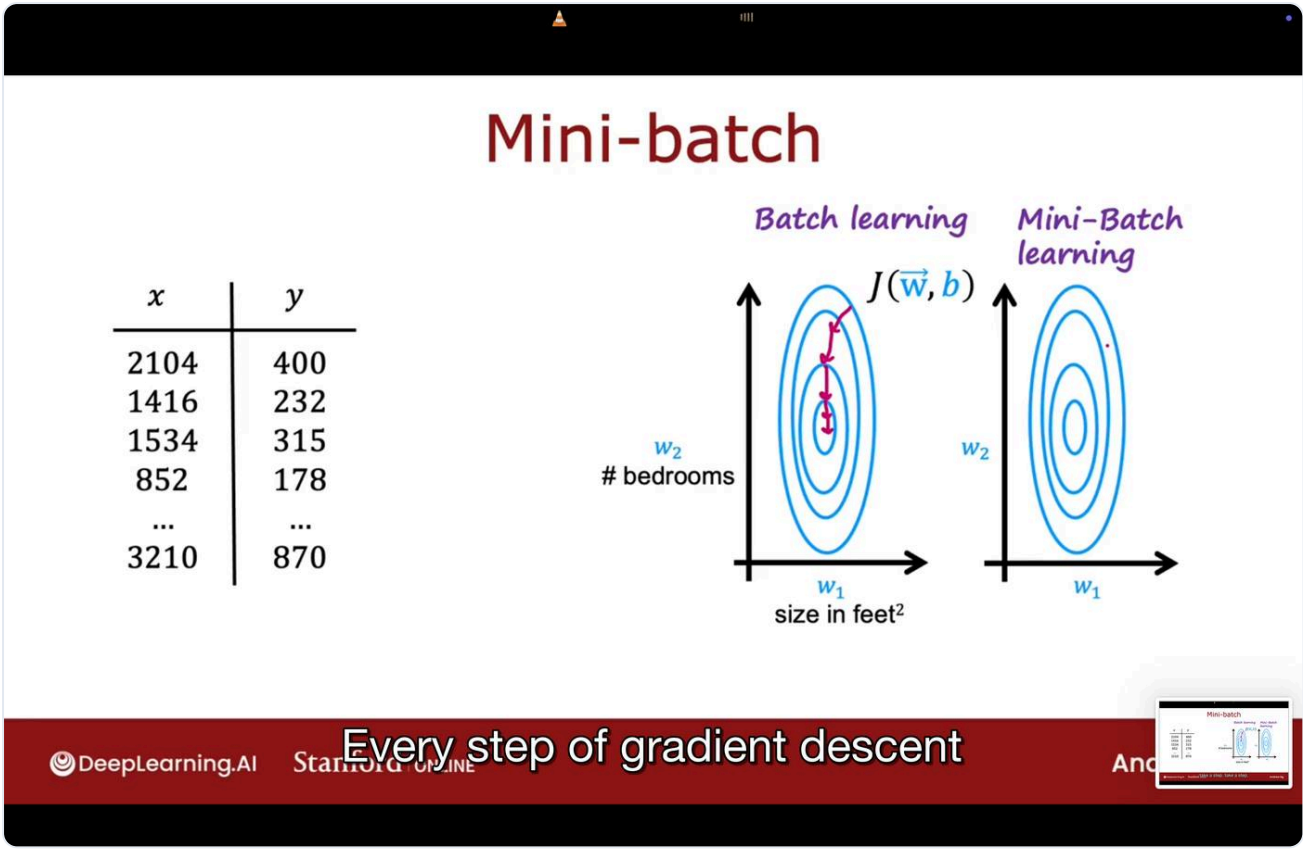


FIGURE 43.11

In contrast, mini-batch gradient descent or a mini-batch learning algorithm will do something like this.

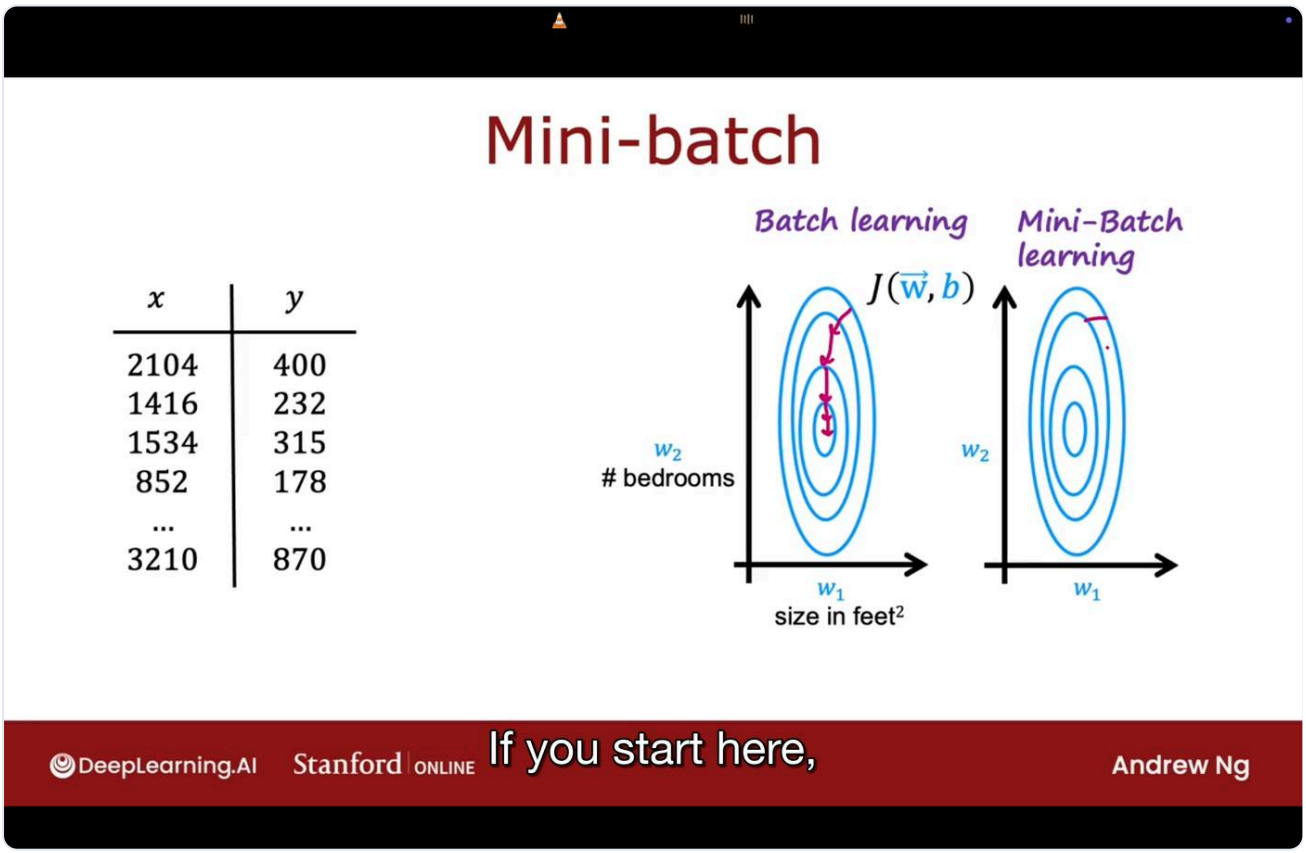


FIGURE 43.12

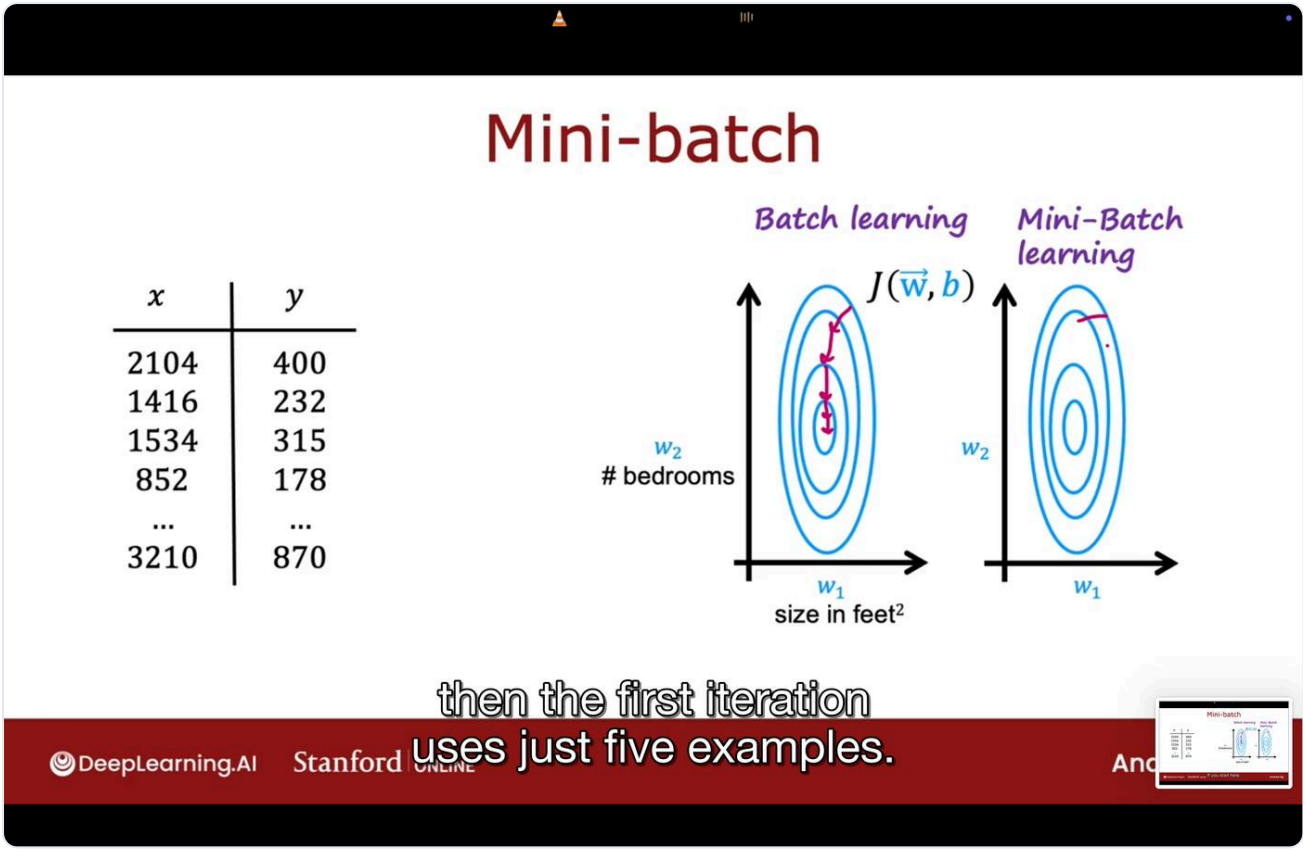


FIGURE 43.13

If you start here, then the first iteration uses just five examples. It'll hit in the right direction but maybe not the best gradient descent direction.

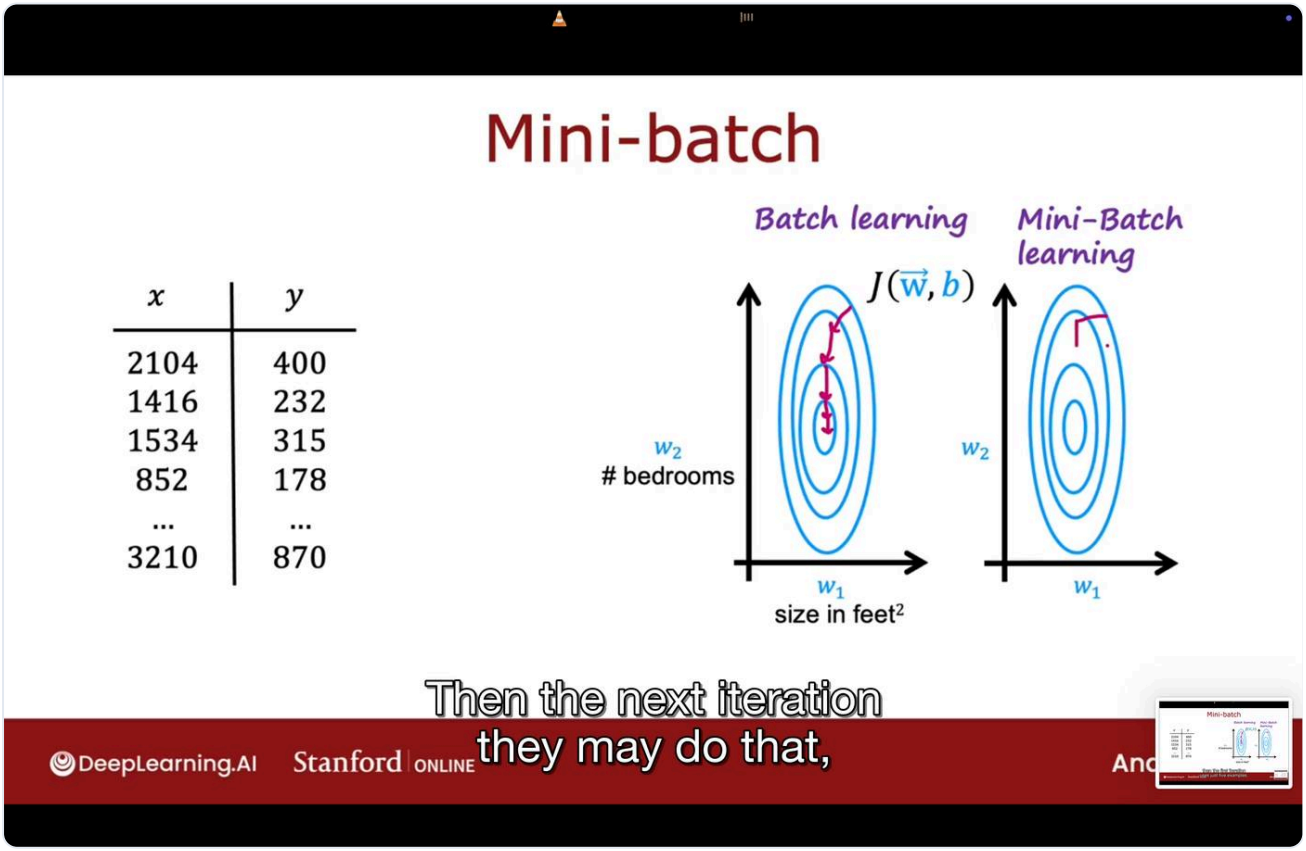
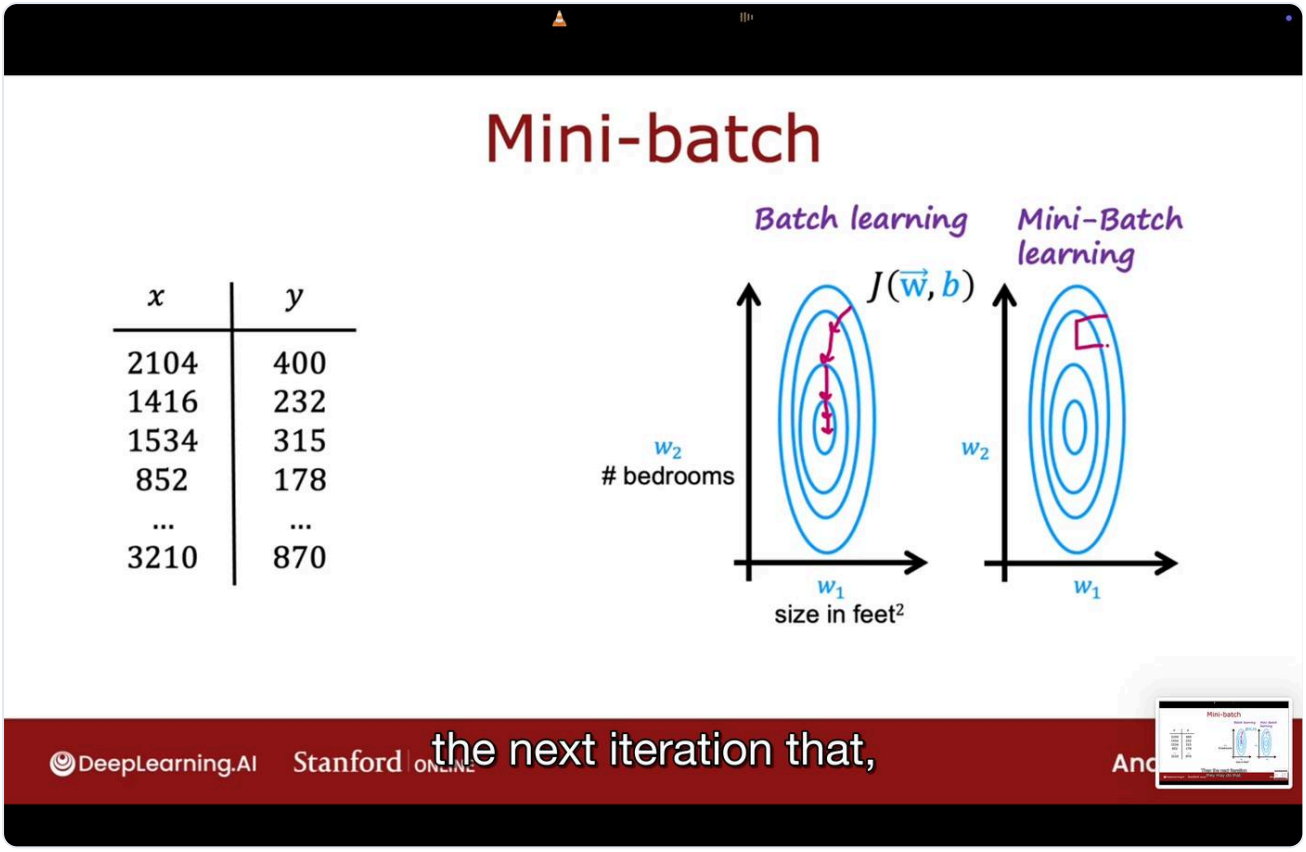
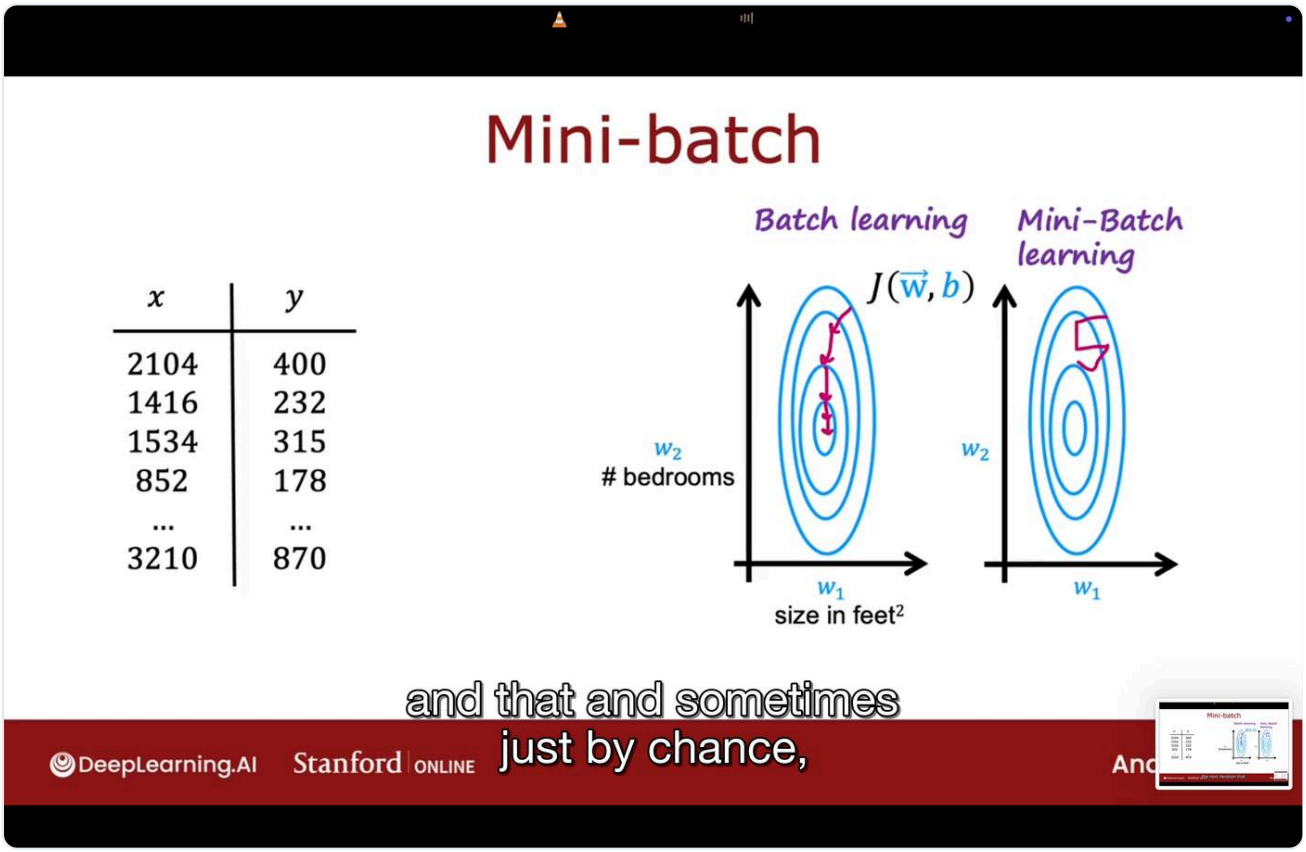


FIGURE 43.14



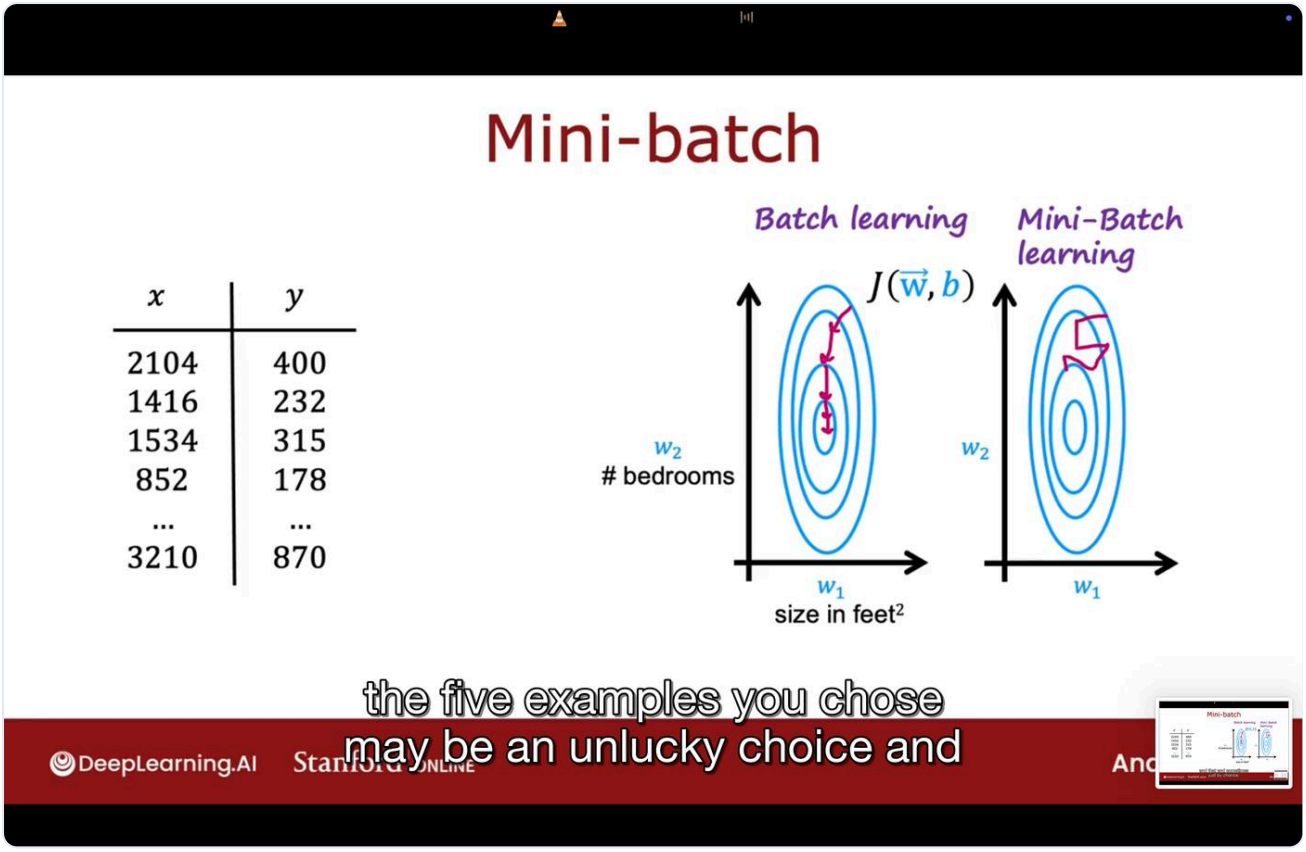
x	y
2104	400
1416	232
1534	315
852	178
...	...
3210	870

FIGURE 43.15



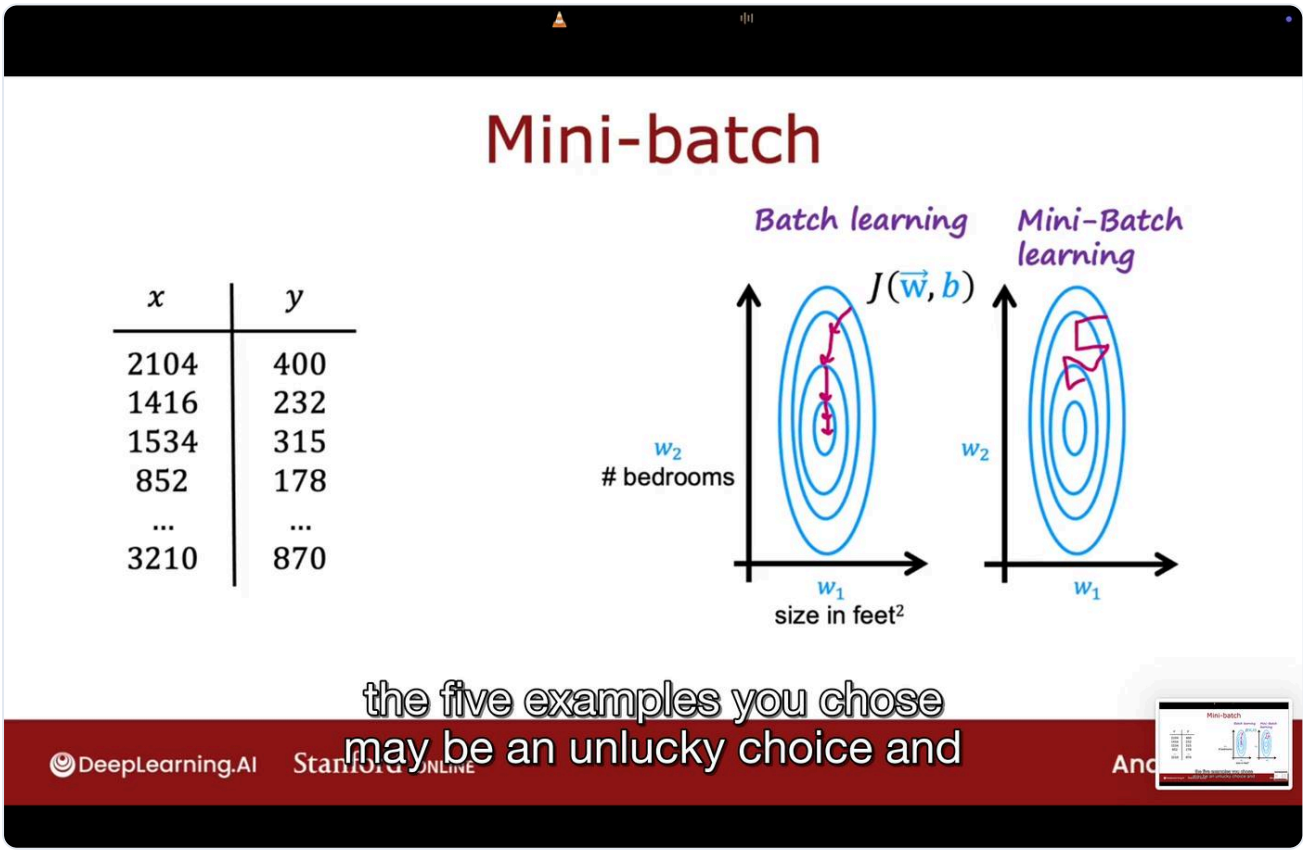
x	y
2104	400
1416	232
1534	315
852	178
...	...
3210	870

FIGURE 43.16



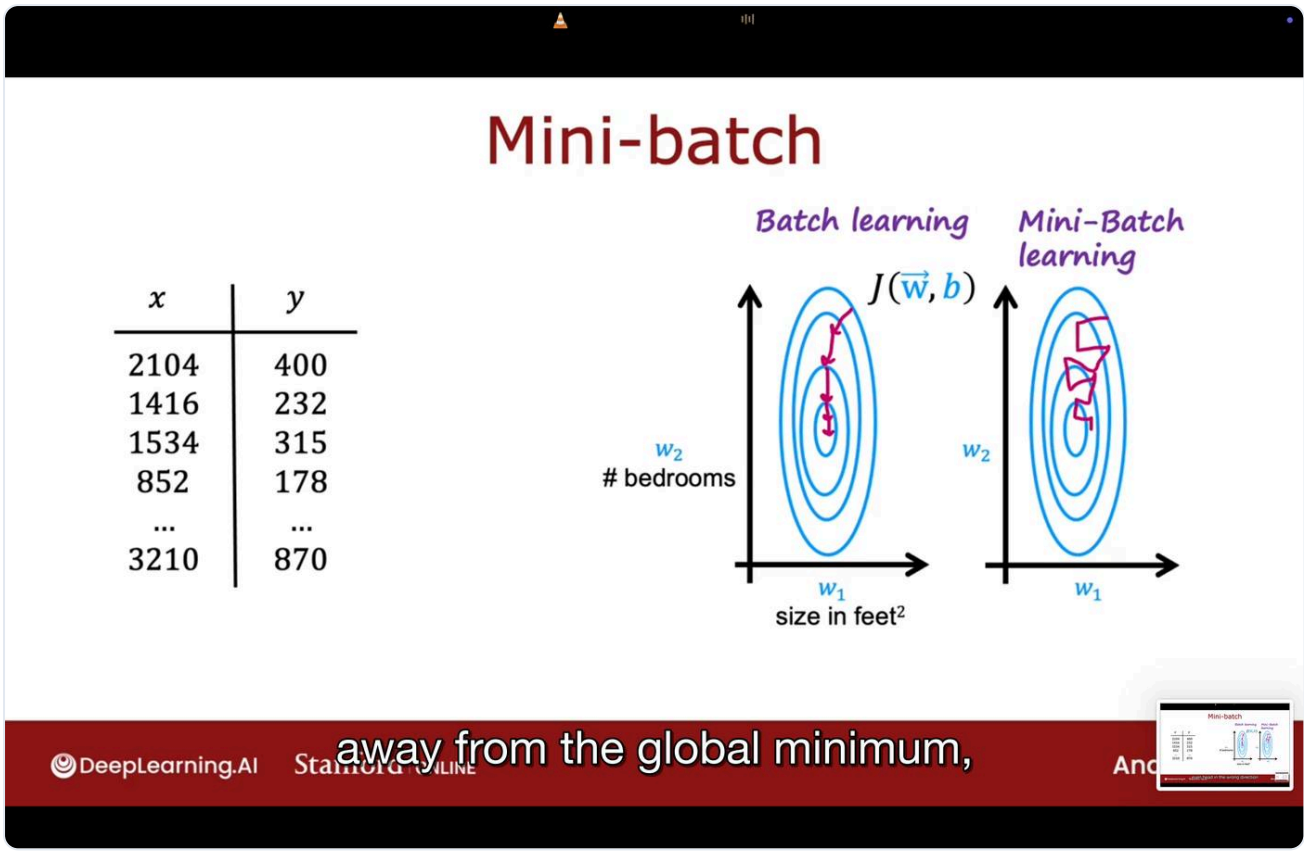
x	y
2104	400
1416	232
1534	315
852	178
...	...
3210	870

FIGURE 43.17



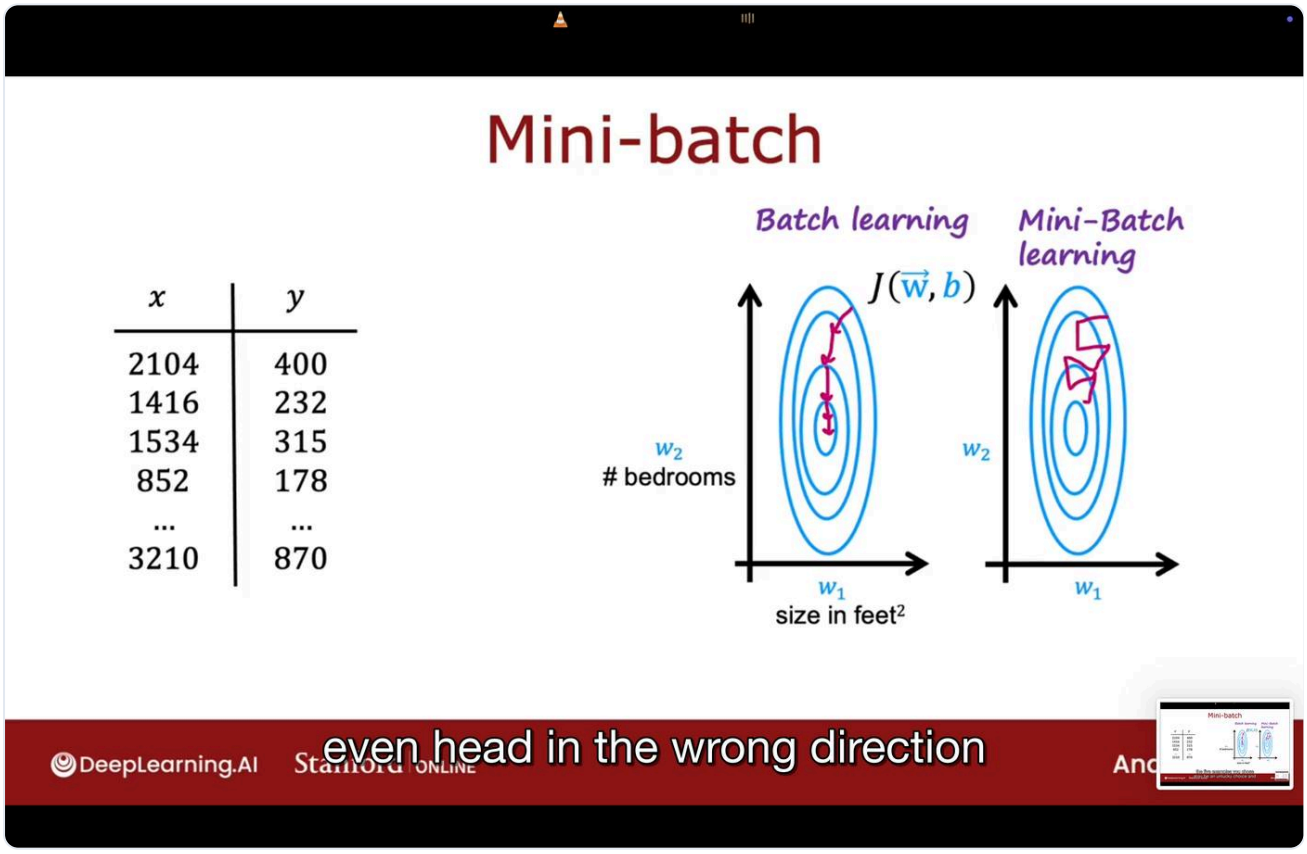
x	y
2104	400
1416	232
1534	315
852	178
...	...
3210	870

FIGURE 43.18



x	y
2104	400
1416	232
1534	315
852	178
...	...
3210	870

FIGURE 43.19



x	y
2104	400
1416	232
1534	315
852	178
...	...
3210	870

FIGURE 43.20

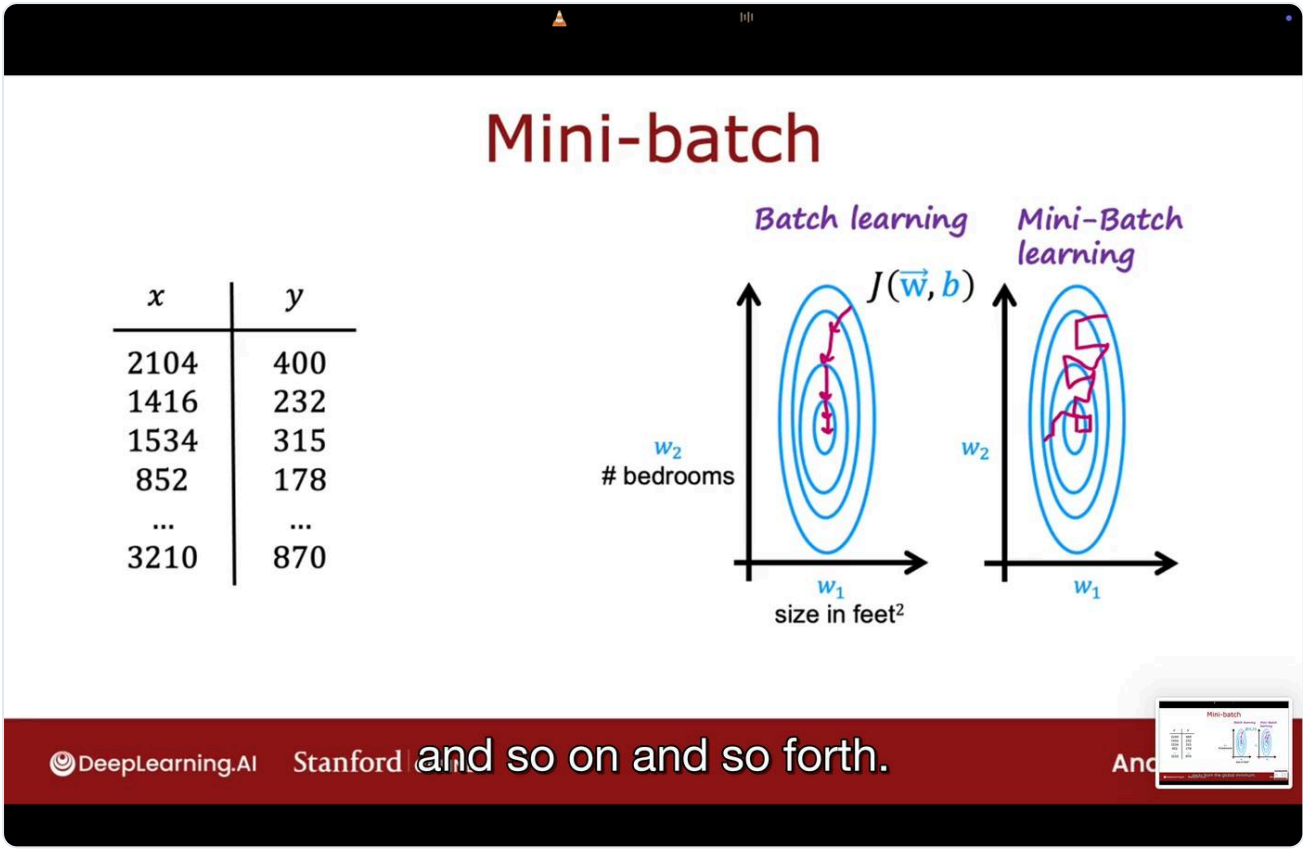


FIGURE 43.21

Then the next iteration they may do that, the next iteration that, and that and sometimes just by chance, the five examples you chose may be an unlucky choice and even head in the wrong direction away from the global minimum, and so on and so forth.

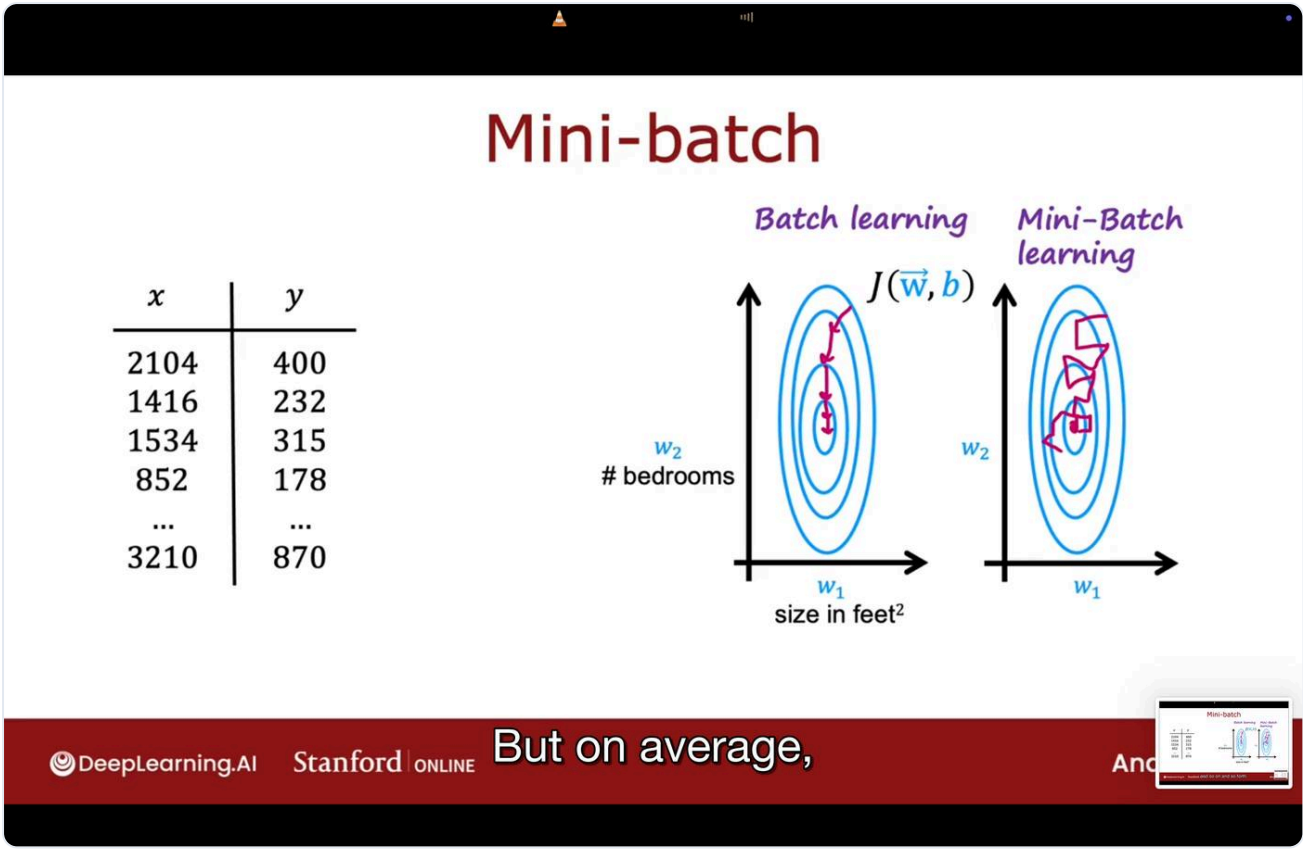


FIGURE 43.22

But on average, mini-batch gradient descent will tend toward the global minimum, not reliably and somewhat noisily, but every iteration is much more computationally inexpensive and so mini-batch learning or mini-batch gradient descent turns out to be a much faster algorithm when you have a very large training set. In fact, for supervised learning, where you have a very large training set, mini-batch learning or mini-batch gradient descent, or a mini-batch version with other optimization algorithms like Atom, is used more common than batch gradient descent.

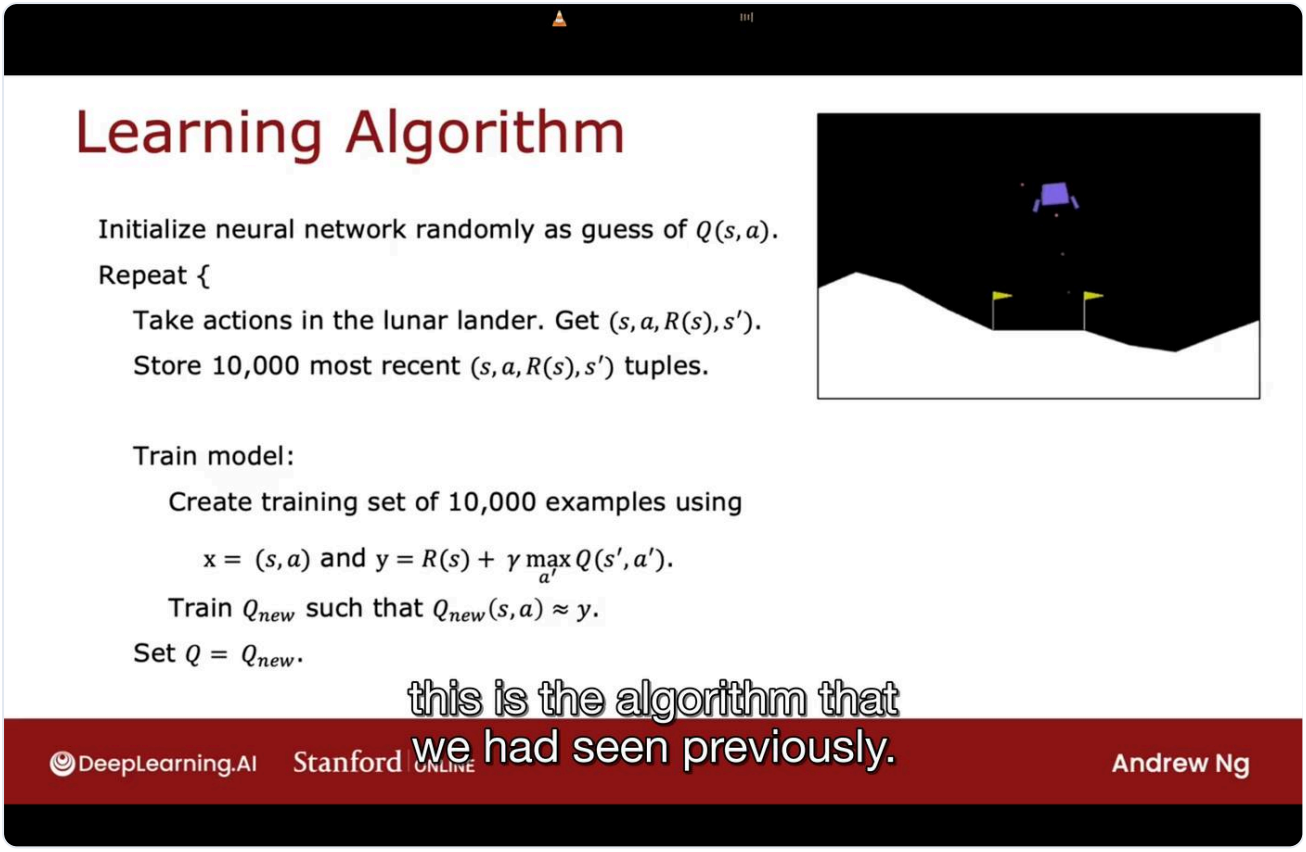


FIGURE 43.23 | Learning Algorithm

Going back to our reinforcement learning algorithm, this is the algorithm that we had seen previously.

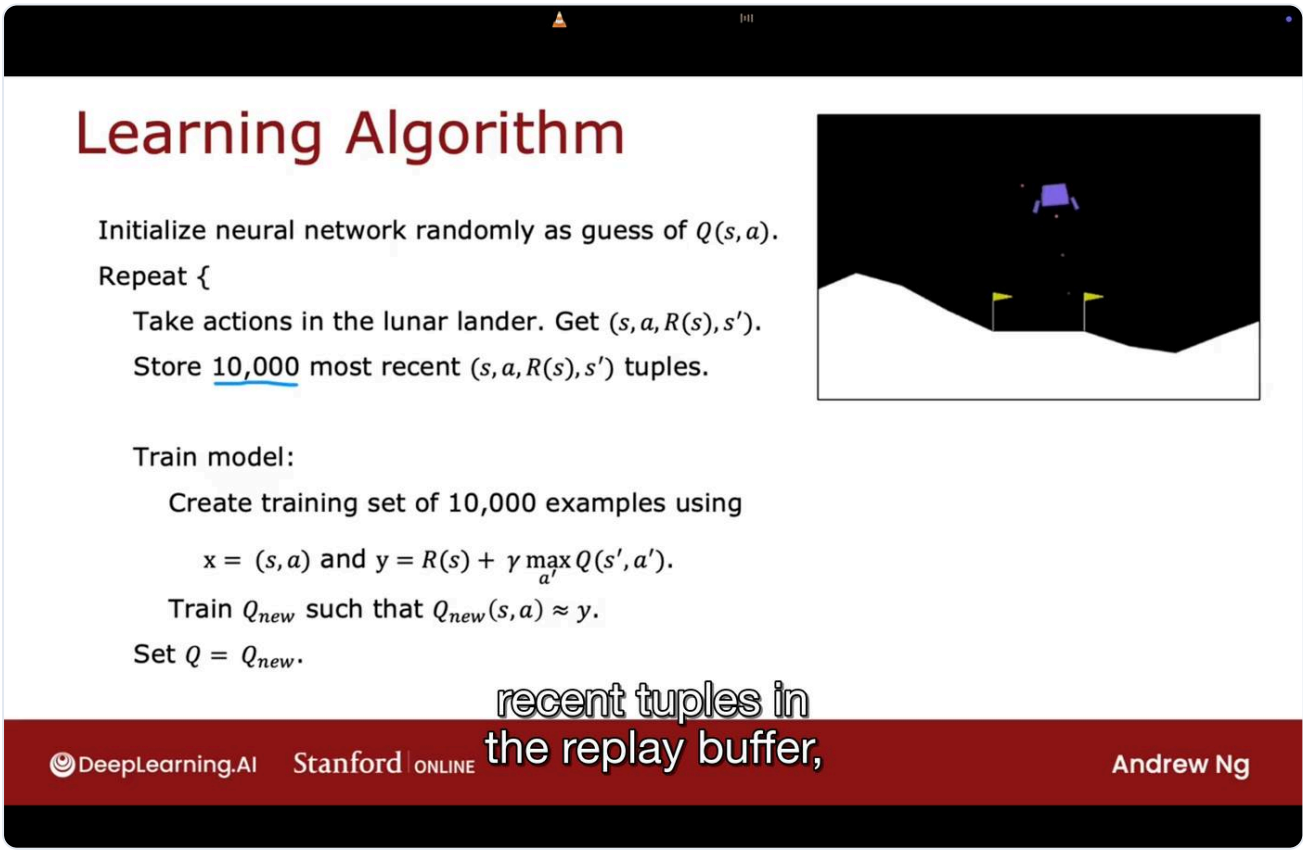


FIGURE 43.24 | Learning Algorithm

The mini-batch version of this would be, even if you have stored the 10,000 most recent tuples in the replay buffer, what you might choose to do is not use all 10,000 every time you train a model. Instead, what you might do is just take the subset.

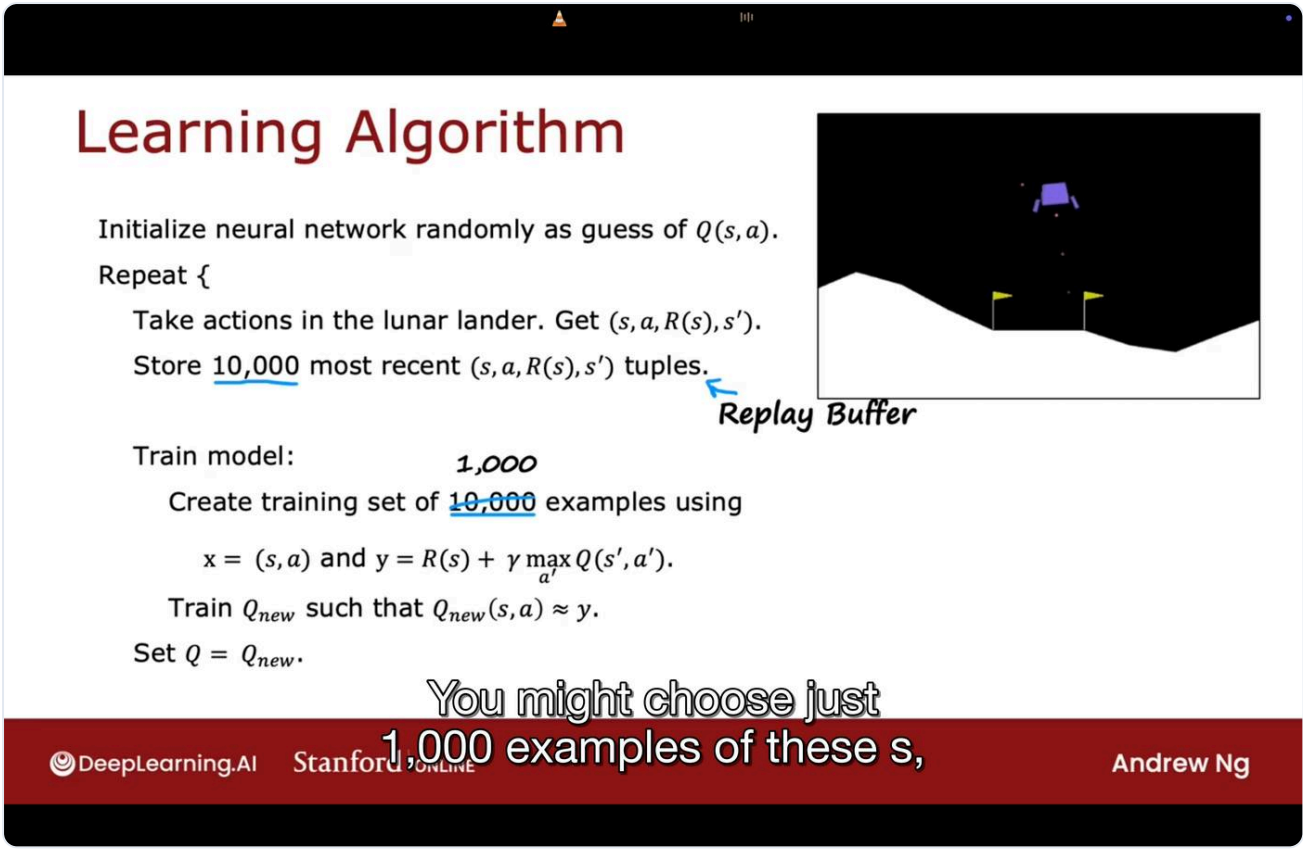


FIGURE 43.25 | Learning Algorithm

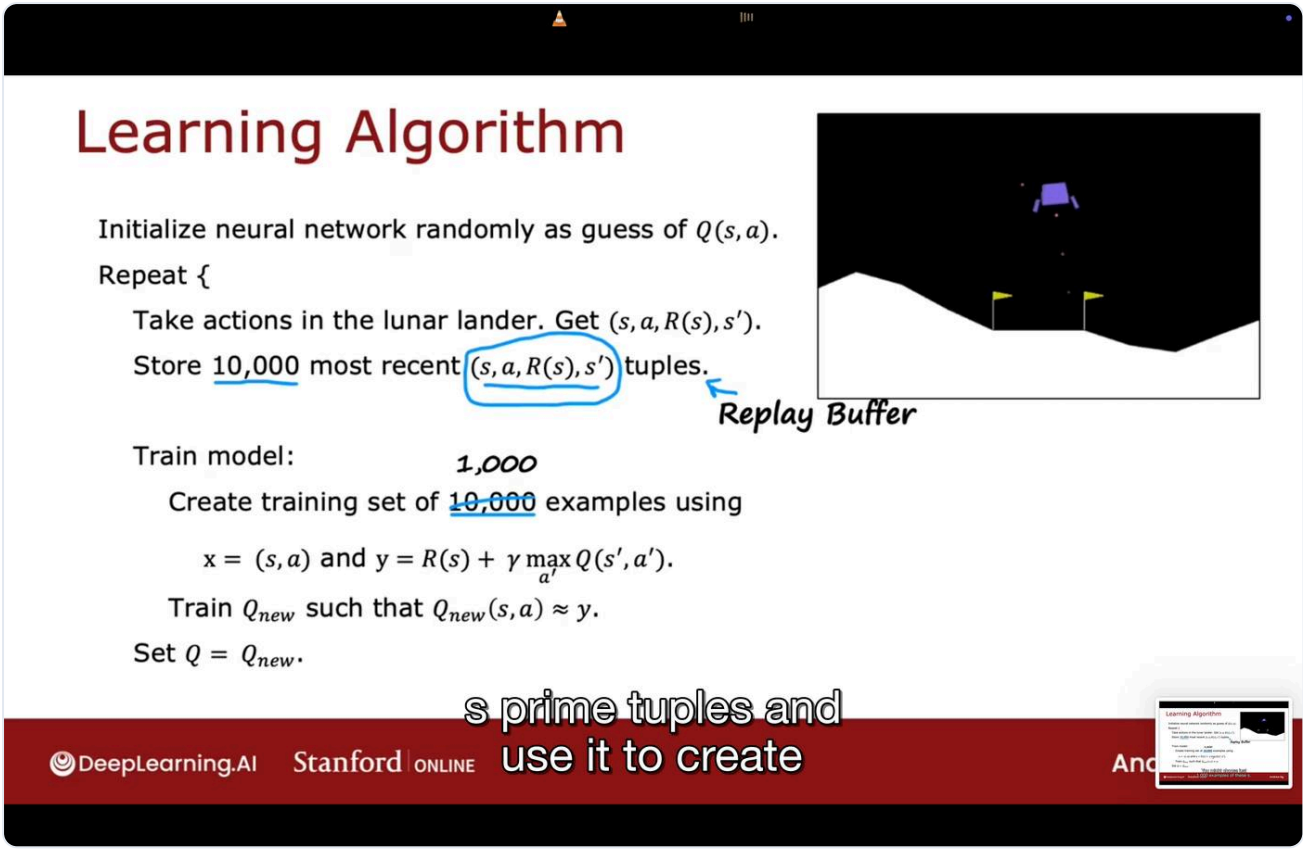


FIGURE 43.26 | Learning Algorithm

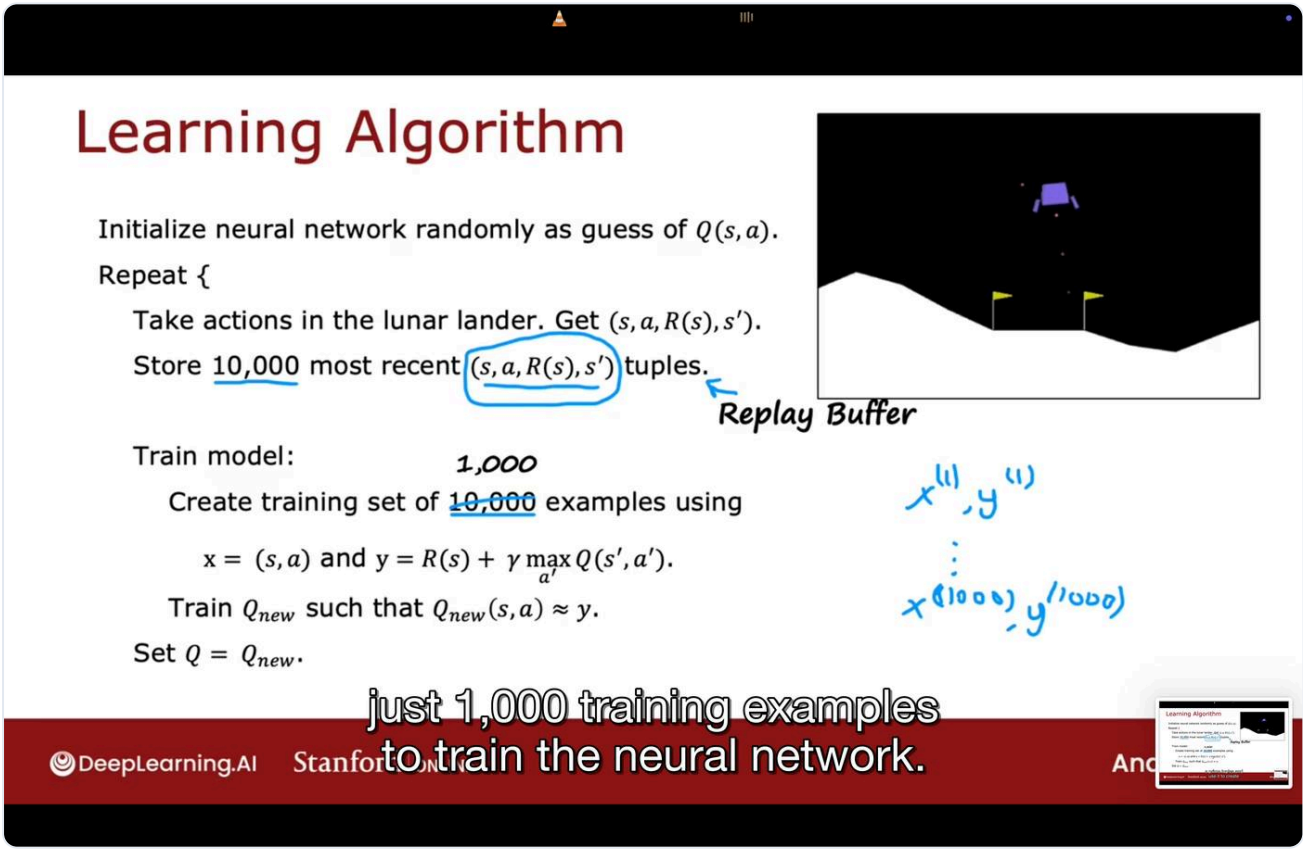


FIGURE 43.27 | Learning Algorithm

You might choose just 1,000 examples of these s, a, R of s, s prime tuples and use it to create just 1,000 training examples to train the neural network.

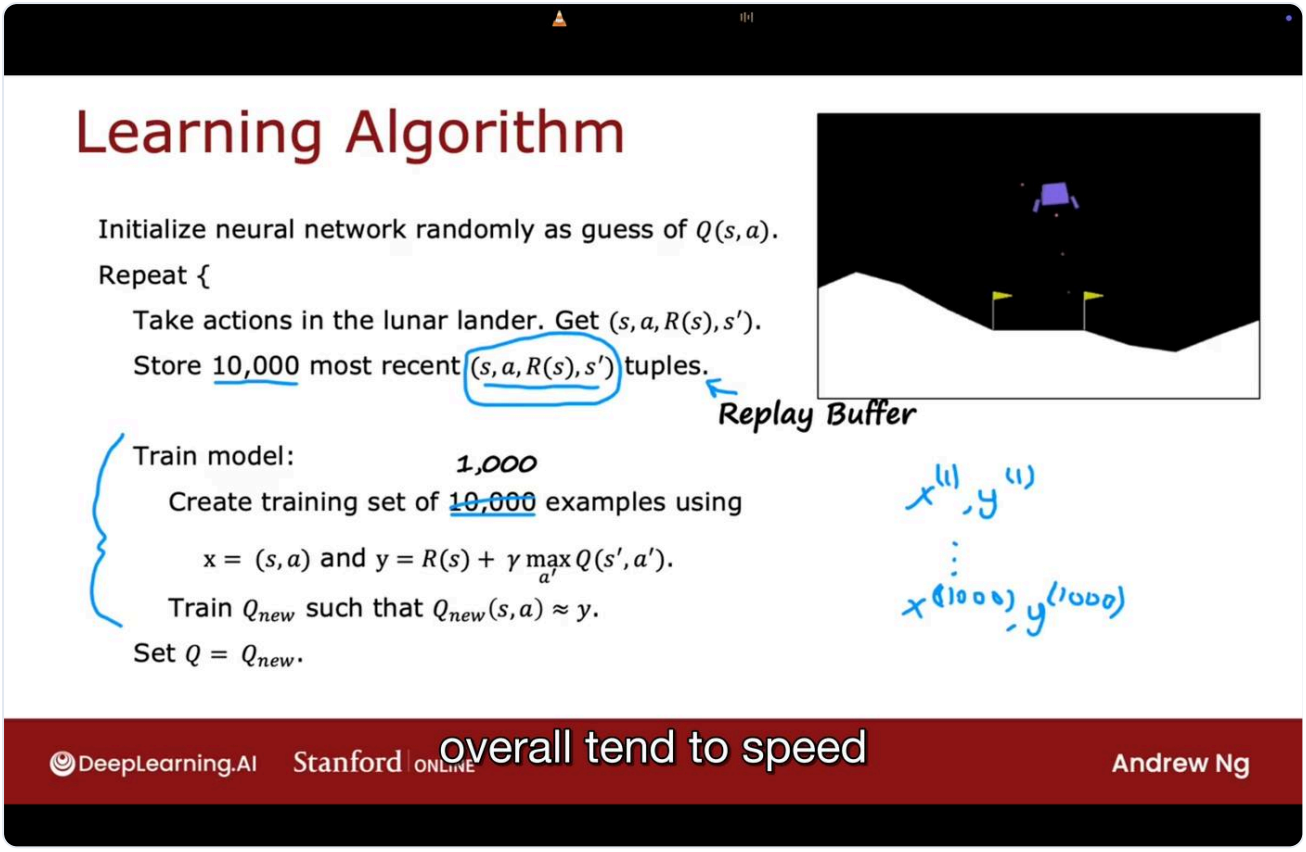


FIGURE 43.28 | Learning Algorithm

It turns out that this will make each iteration of training a model a little bit more noisy but much faster and this will overall tend to speed up this reinforcement learning algorithm. That's how mini-batching can speed up both a supervised learning algorithm like linear regression as well as this reinforcement learning algorithm where you may use a mini-batch size of say, 1,000 examples, even if you store it away, 10,000 of these tuples in your replay buffer. Finally, there's one other refinement to the algorithm that can make it converge more reliably, which is, I've written out this step here of Set Q equals Q_new. But it turns out that this can make a very abrupt change to Q.

If you train a new neural network to new, maybe just by chance is not a very good neural network. Maybe is even a little bit worse than the old one, then you just overwrote your Q function with a potentially worse noisy neural network.

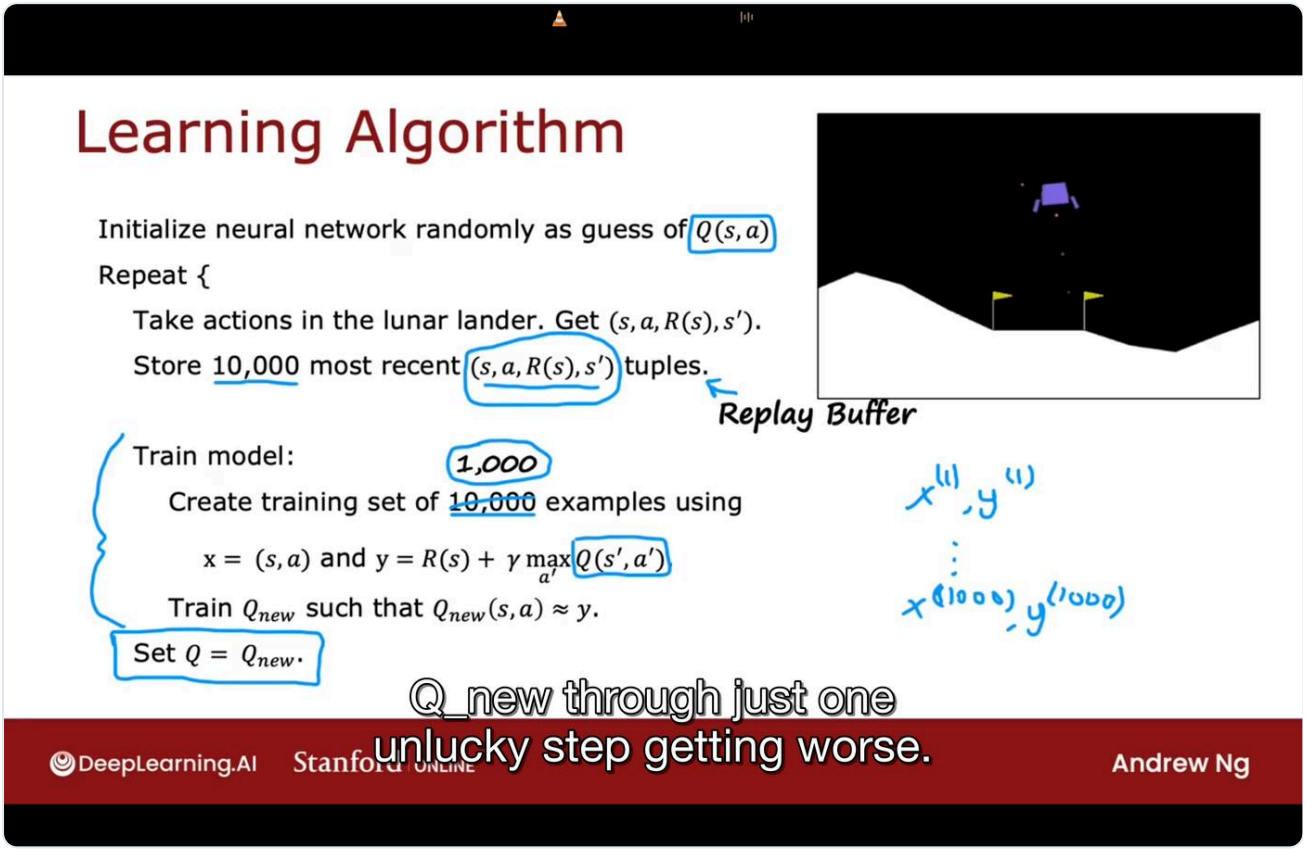


FIGURE 43.29 | Learning Algorithm

The soft update method helps to prevent Q_new through just one unlucky step getting worse. In particular, the neural network Q will have some parameters, W and B, all the parameters for all the layers in the neural network. When you train the new neural network, you get some parameters W_new and B_new. In the original algorithm S as described on that slide, you would set W to be equal to W_new and B equals B_new. That's what set Q equals Q_new means. With the soft update, what we do is instead Set W equals 0.01 times W_new plus 0.99 times W. In other words, we're going to make W to be 99 percent the old version of W plus one percent of the new version W_new.

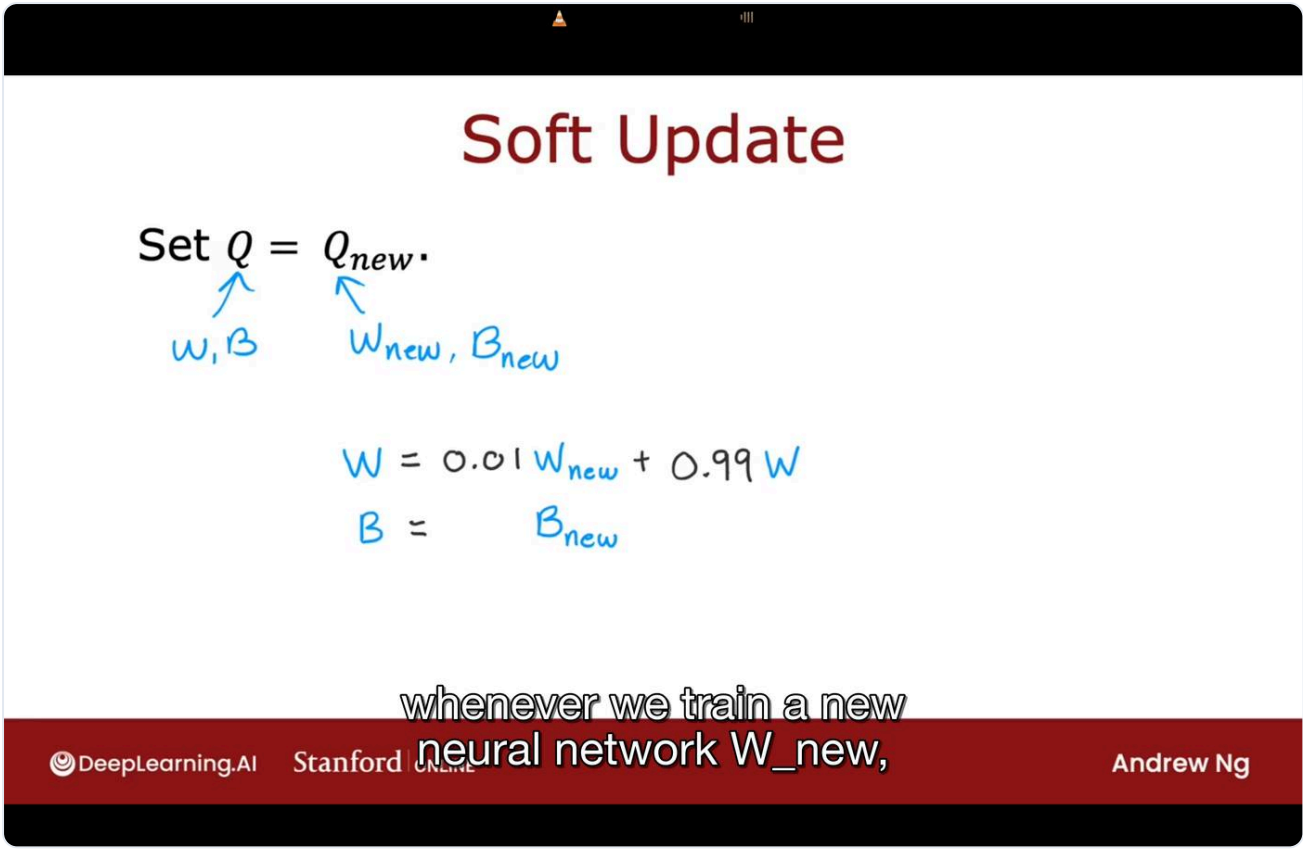


FIGURE 43.30 | Soft Update

This is called a soft update because whenever we train a new neural network W_new, we're only going to accept a little bit of the new value. As similarly, B equals 0.01 times B_new plus 0.99 times B.

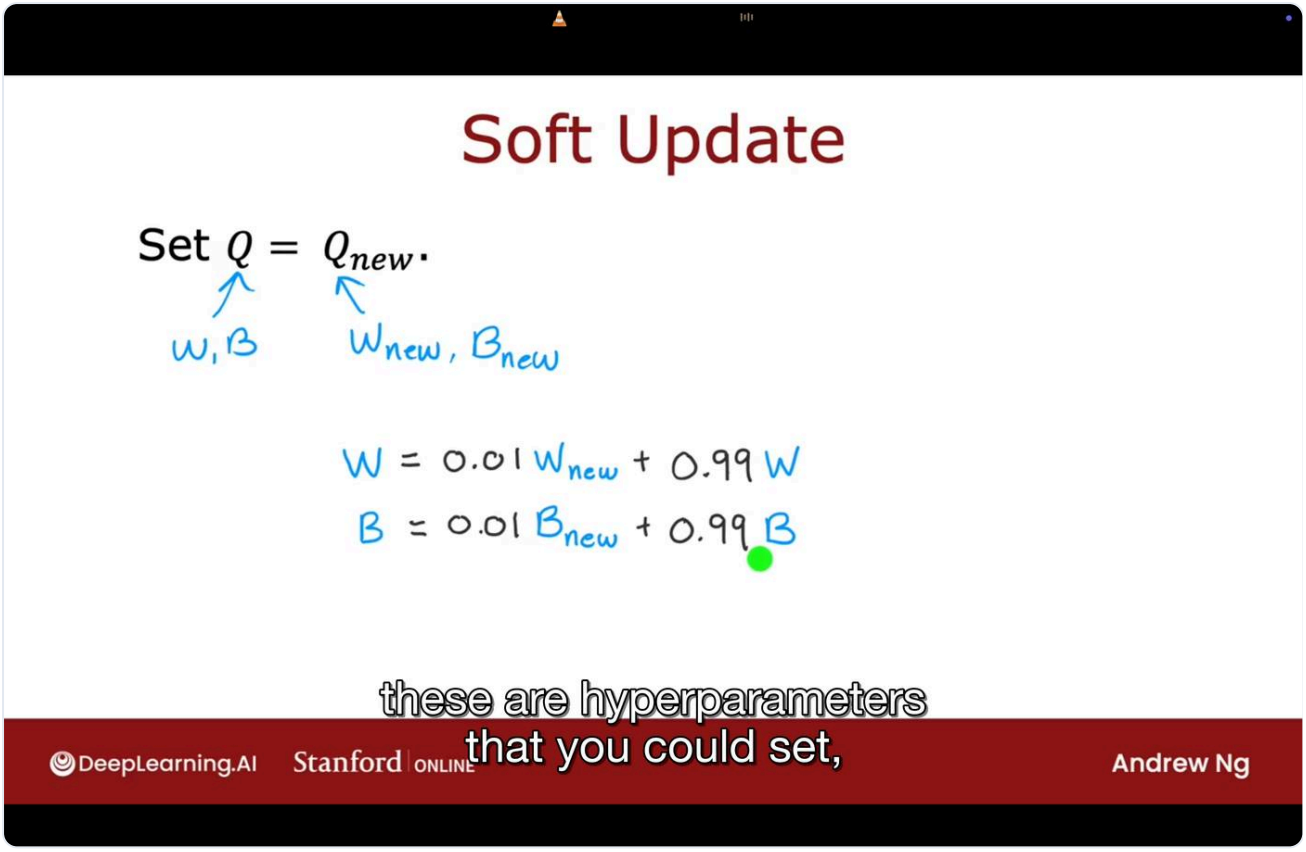


FIGURE 43.31 | Soft Update

These numbers, 0.01 and 0.99, these are hyperparameters that you could set, but it controls how aggressively you move W to W_new and these two numbers are expected to add up to one.

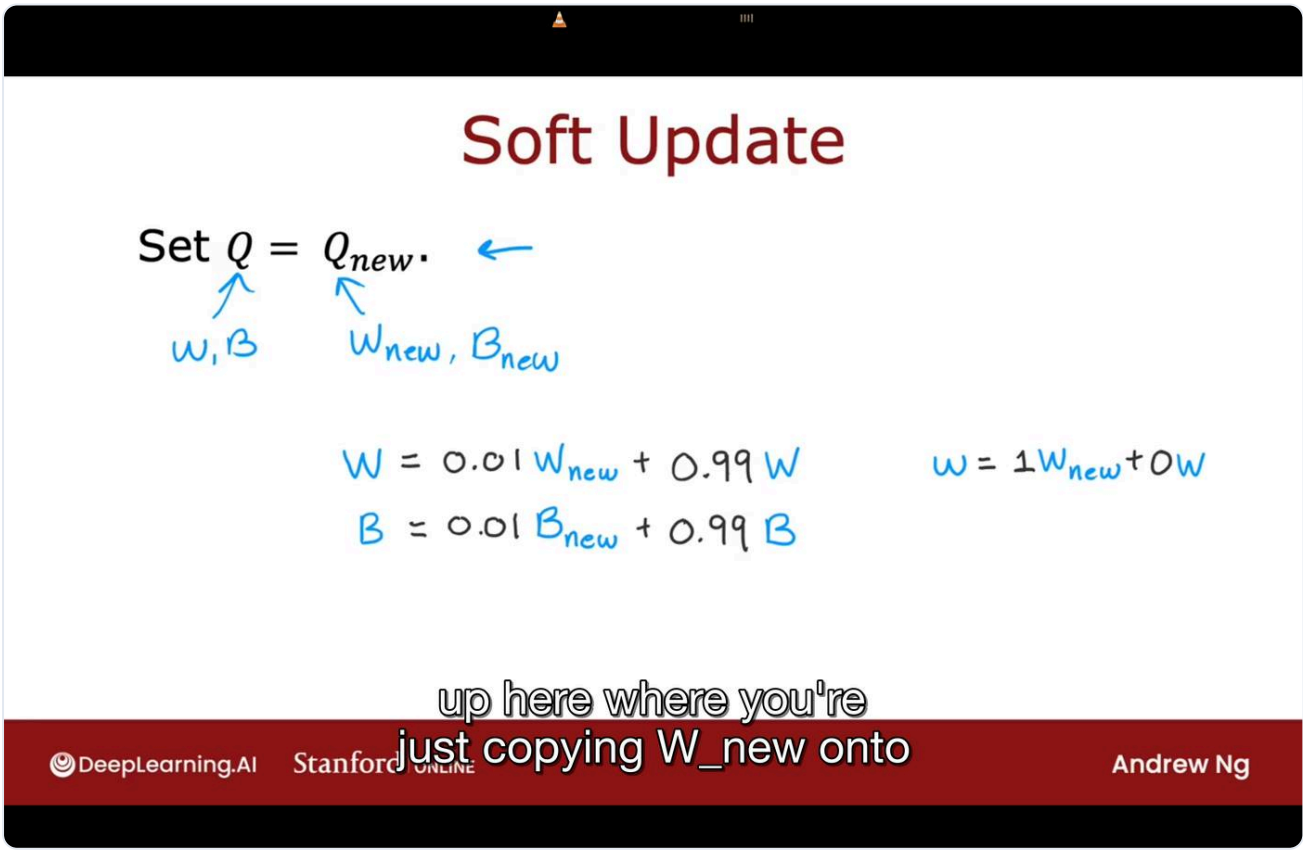


FIGURE 43.32 | Soft Update

One extreme would be if you were to set W equals one times W_new plus 0 times W, in which case, you're back to the original algorithm up here where you're just copying W_new onto W. But a soft update allows you to make a more gradual change to Q or to the neural network parameters W and B that affect your current guess for the Q function Q of s, a.

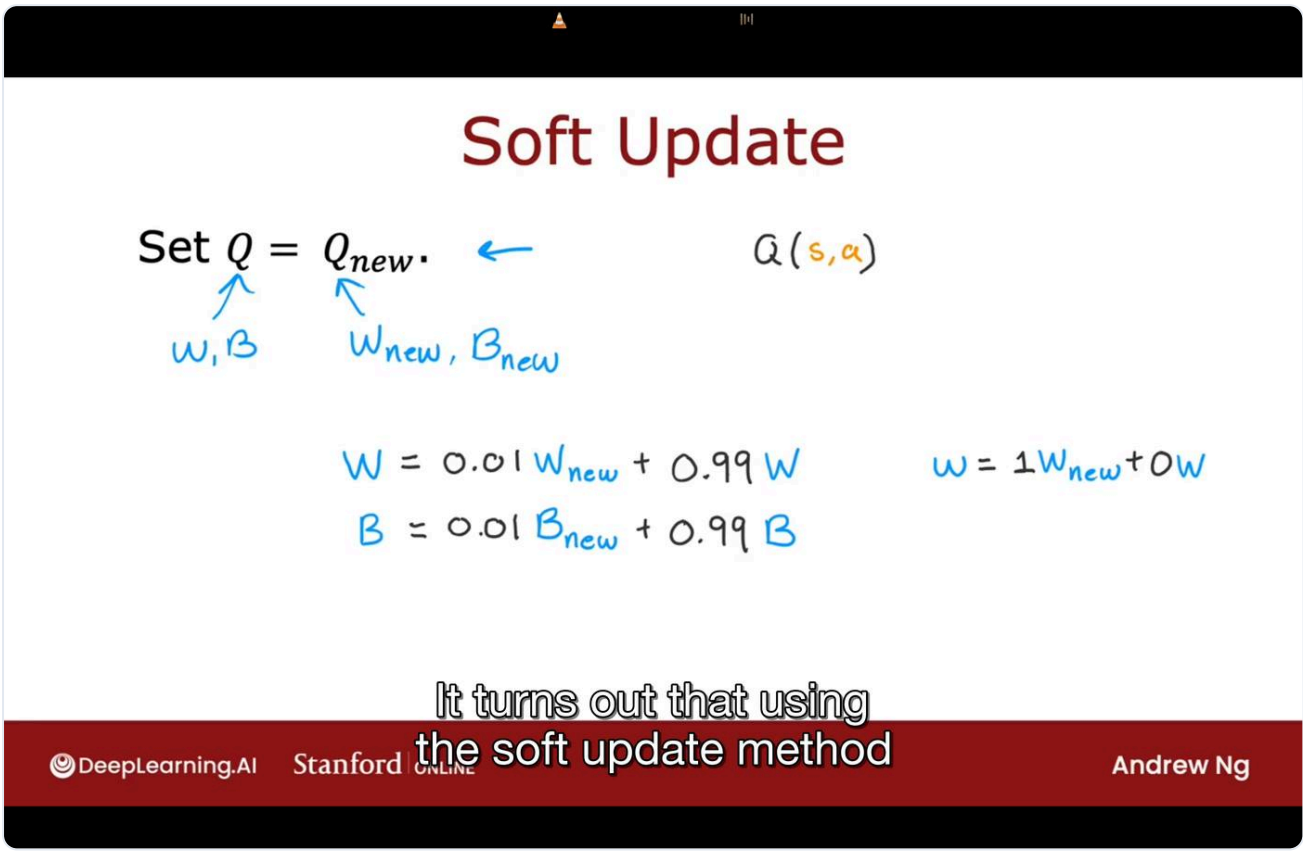


FIGURE 43.33

It turns out that using the soft update method causes the reinforcement learning algorithm to converge more reliably. It makes it less likely that the reinforcement learning algorithm will oscillate or divert or have other undesirable properties. With these two final refinements to the algorithm, mini-batching, which actually applies very well to supervise learning as well, not just reinforcement learning, as well as the idea of soft updates, you should be able to get your lunar algorithm to work really well on the Lunar Lander. The Lunar Lander is actually a decently complex, decently challenging application and so that you can get it to work and land safely on the moon.

I think that's actually really cool and I hope you enjoy playing with the practice lab. Now, we've talked a lot about reinforcement learning. Before we wrap up, I'd like to share with you my thoughts on the state of reinforcement learning so that as you go out and build applications using different machine learning techniques via supervised, unsupervised, reinforcement learning techniques that you have a framework for understanding where reinforcement learning fits in to the world of machine learning today. Let's go take a look at that in the next video.

LECTURE 44

Limitations of Reinforcement Learning

Reinforcement learning is an exciting set of technologies. In fact, when I was working on my PhD thesis reinforcement learning was the subject of my thesis. So I was and still am excited about these ideas. Despite all the research momentum and excitement behind reinforcement learning though, I think there is a bit or maybe sometimes a lot of hype around it. So what I hope to do is share with you a practical sense of where reinforcement learning is today in terms of its utility for applications. One of the reasons for some of the hype about reinforcement learning is, it turns out many of the research publications have been on simulated environments.

And having worked in both simulations and on real robots myself, I can tell you that it's much easier to get a reinforcement learning album to work in a simulation or in a video game than in a real robot. So a lot of developers have commented that even after they got it to work in simulation, it turned out to be surprisingly challenging to get something to work in the real world or the real robot. And so if you apply these algorithms to a real application, this is one limitation that I hope you pay attention to to make sure what you do does work on the real application. Second, despite all the media coverage about reinforcement learning, today there are far fewer applications of reinforcement learning than supervised and unsupervised learning.

So if you are building a practical application, the odds that you will find supervised learning or unsupervised learning useful or the right tool for the job, is much higher than the odds that you would end up using reinforcement learning. I have used reinforcement learning a few times myself especially on robotic control applications, but in my day to day applied work, I end up using supervised and unsupervised learning much more. There is a lot of exciting research in reinforcement learning right now, and I think the potential of reinforcement learning for future applications is very large. And reinforcement learning still remains one of the major pillars of machine learning.

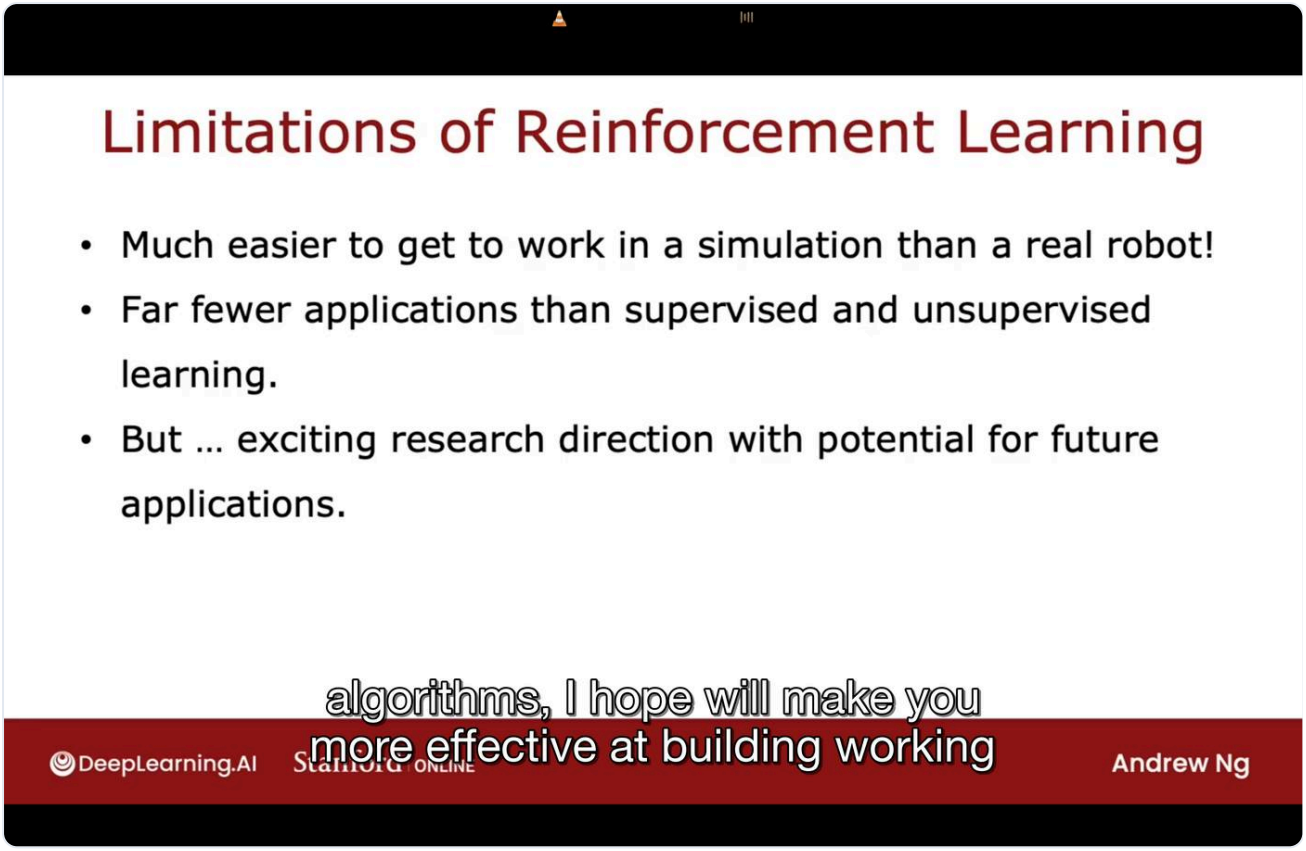


FIGURE 44.1 | Limitations of Reinforcement Learning

And so having it as a framework as you develop your own machine learning algorithms, I hope will make you more effective at building working machine learning systems as well. So I hope you've enjoyed this week's materials on reinforcement learning, and specifically I hope you have fun getting the lunar lander to land for yourself. I hope will be a satisfying experience when you implement an algorithm and then see that lunar lander land safely on the moon because of code that you wrote. That brings us towards the end of this specialization. Let's go on to the last video where we wrap up

LECTURE 1

Machine Learning

Welcome to the final video of this machine learning specialization. We've been through a lot of videos together and this is the last one. Let's summarize the main topics we've gone over, and then I like to say a few words and then we'll wrap up the class. Looking back, I think we've been through a lot together. The first course we went through was on supervised machine learning, including regression and classification. Where you learned about linear regression, logistic regression, cost functions, and the gradient descent algorithm. In the second course, we then looked at more advanced learning algorithms, including neural networks, decision tree ensembles, and also went through advice for machine learning such as bias and variance and how to use a train hold-out cross validation and test sets and how to improve your learning algorithm efficiently.

Then the third course was on unsupervised learning recommenders and reinforcement learning, where we talked about clustering algorithms, anomaly detection algorithms, collaborative filtering and content-based filtering, and then in this past week, reinforcement learning. With this broad set of tools, you are now well qualified to build a huge range possible machine learning applications. Congratulations on making it all the way to this last video. If you've worked all the way through this specialization, you now have a very solid foundation in machine learning. I think you've made a great start to become an expert in machine learning. As you know, machine learning is having a huge impact on society as a powerful tool used by billions of people every day through web search, product recommendations, speech recognition, and many other applications.

There's even improving human knowledge through hoping with scientific discovery is driving billions of dollars of value and is enabling new applications that were unimaginable just a few years ago. But I think the best application of machine learning are still yet to be invented, and that brings us to you. You are now well-qualified to wield the tools of machine learning, to build applications yourself, and to create things. I hope that you will use these skills to make other people's lives better. Before wrap up this class, I want to say just one last thing to you. This class has been fun for me to teach, but not long ago, I was a student myself, and so I know how time-consuming it is to learn this stuff.

I know you're a busy person with many other things going on in your life, so that you took the time to watch the videos, go through the quizzes and labs. I know you've put a lot of time and put a lot of yourself into this class. I just want to say, thank you very much for having been a student in this class. I'm very grateful to you and appreciate all the time you spent with me and with the specialization. Thank you.

LECTURE 46

Robotics

[MUSIC] Thanks for joining me here at Stanford University, with Professor Chelsea Finn, who is with both the computer science and the electrical engineering departments. Chelsea is a research lab that does cutting edge work, applying machine learning, especially reinforcement learning, to a range of robotics applications. So I'm really glad to have you here Chelsea. >> Happy to be here. >> So, I know that before you settled into working on computer science and machine learning, you actually considered a lot of career options ranging from biology to aerospace to computer science and electrical engineering. So, what made you make this choice? Yeah, so I was always drawn to engineering because I really like solving puzzles and solving problems.

And when I arrived at MIT for my undergraduate, I was considering different engineering majors. And what really drew me first to computer science was that, it gives you a lot of flexibility in terms of the different paths that you can do. Essentially computer science empowers you to build lots of different things with software, with code and with that, I could eventually go into biology. If I wanted to, I could go into robotics, there's all sorts of really exciting things you can do with computer science. So that landed me in computer science and then once I decided on that, I really enjoyed a lot of the math that goes behind AI like probability and statistics.

And I was also really drawn to this challenge of how can we build a computer, that can see like humans do. That can perceive things and images and can take actions in the world, yeah. >> So it was almost as if your choice was if you do computer science, you could do anything. And so that makes it, I guess what Marc Andreessen said software is eating the world. And I think Jen Huang said AI is eating software, and it feels like when you work in machine learning, you get to poke your nose in all corners of the world and do fun things. >> Absolutely, I think machine learning is a really exciting place to be. And I mentioned that computer science is very flexible and opens lots of doors, and I think that machine learning is very similar to that.

There's so many different things you can do with machine learning, and so many different applications. I'm really excited about applications in robotics. But, we've seen lots of different applications in language processing in biology and in other fields. >> So your specialization, of the many things you do, a large fraction of your work is in robotics. And, when people look at the popular media or social media, you see these videos of for example, humanoid robots jumping up and down and doing and they do incredible things. How should one think about what robots really can and cannot do today? Because there are these amazing demo videos that looks like can do almost anything.

But, how should one decide what is actually doable? >> Yeah, the videos that we see online from Boston Dynamics for example of robots doing back flips or doing Parkour and of course. Are really impressive, because they look really complicated. It's really easy in some ways anthropomorphize the robot and think that if you saw a person doing that, that would be very impressive.

And they would probably be able to do lots of other really impressive things. And the catch is that in robotics in those videos and in many impressive demonstrations of robotics. The robot was set up for that particular environment, and it was tuned to do well with that set up.

And if you change something about the environment, about the setup about what it's supposed to do, then the demo no longer works. The the robot will fall flat. So, the really the big challenge with robotics is getting robots to generalize, to be able to handle lots of different scenarios with lots of different objects, with lots of different environments. And right now, robots can do things in very controlled environments like in factories and they are very useful in factories. And, what I'm really excited about and what's really hard in robotics is giving them the flexibility in generality that of the skills that humans can do. >> Just to dig a little bit more into the specific one off one environment thing before going into generality.

When someone sees the next cool robotics demo, how should they think about it or what the questions they should ask themselves? >> Yeah, so I think that the first question that someone can ask is, what will happen if something changes in the environment. What will happen if you move a block or if you put the robot in a slightly different position when it starts. Oftentimes you don't see those kinds of tests of the system's robustness or resilience to those variations. And, if you do start seeing those those demos where actually you kind of poke the robot, or you change the environment a little bit and it still works. That's when I think we can be really impressed. >> Cool, so a lot of your work has specifically been on making robots robust, to generalize or apply to lots of different environments.

Just tell me more about that work and how you approach this problem. >> Definitely, so in other areas of machine learning we've seen huge successes where we take a very large data set. Like you take all of Wikipedia, and you train a neural network to be able to model the data in that very large and broad data set. And in robotics, one thing that I'm excited about is if we can somewhat replicate that sort of success. If we can collect a similarly diverse data set and allow robots to learn from that data set. Now, this is challenging for a couple of reasons, one is that we don't already have an existing data set, we don't have Wikipedia for robot motor control.

There isn't data lying on the internet, of how exactly to move a robot's fingers to tie shoes, for example. So we don't have tons of diverse data already, but fortunately, robots in principle should be able to collect data themselves. And, that is an opportunity because it means that we might be able to collect a large amount of diverse data, by allowing the robots to collect their own data. But it's also a challenge because it means that we need to figure out how to allow robots to collect data that's useful, and diverse. And so, there's lots of different things that we explore in terms of approaching this problem, some is allowing robots to collect their own data in lots of different environments.

Some is showing demonstrations to a robot, of how to perform a task. We've also explored a little bit of how to use data from the internet, like videos of humans to show a robot how to do a task. And incorporating all of these different data sources to allow robots to, well, once they've hopefully seen very diverse data sets like this. They might be able to generalize not just to the one scenario that they saw, in a lab environment but possibly the real world in the long term. >> One

of the things we've seen is machine learning reinforcement learning, algorithms are great at playing video games. Because you can get, essentially infinite data playing a video game with yourself.

What do you think of the role of simulation. Yeah, so we've definitely seen lots of successes of AI systems doing very well at video games. And simulation is somewhat similar because we can collect lots of data in simulation as well. One challenge with simulation is that, the physics in the simulation engine are not exactly the same as the real world. We know generally the laws of physics but, the friction on a table, for example, may not be exactly known. Or the friction between the grooves on a cap and a water bottle if you try to screw a water bowl, modeling that is very difficult and there are it's prone to errors. And then when you try to train a policy in that simulator to close a water bottle, for example, And then deploy it in the real world.

It actually doesn't work because of those errors in the simulation. So I think it's a promising data source. But because of these inaccuracies and this challenge of transferring something that was learned in simulation to the real world. I don't think it's the only approach that we should bet on. I think that we should try to leverage lots of real data. >> And I wonder if it is surprising to someone watching this, why is it so hard to simulate the real world? It's just a lot of bottles two pieces of plastic and somehow we can't seem to figure out what's going on with two pieces of plastic. >> Yeah. So I think that there's lots of challenges. I mean, first is just modeling the low level physics.

A lot of simulation engines out there, they will model time at a certain and in reality, time is much more fluid than operating at like 100 Hertz or something like that. And to actually accurately model these things, you may need to actually model at a very fine time discrimination and that gets very expensive. And then your simulator may actually be slower than actually running things in the real world. And then it's no longer useful in comparison to collecting data in the real world. And then the second thing is the real world is super diverse. And so actually creating the content of all of the kind of objects that you encounter even in everyday life.

When you're eating lunch or when you're kind of cleaning up a table, creating all of that content is actually a really manual effort. And usually in machine learning systems, you want to kind of do away with the things that require a lot of manual effort. And try to automate and automate things so that they can be scaled up. >> Yeah. And in fact, in terms of the diversity of environments today, factories work great in robots where the environment is relatively controlled, relatively few unexpected things happening on average. I know some of your work has been looking to send robots into people's homes and do things in people's kitchens. That's one of the most uncontrolled environments in the world to say, you say a bit more about that. >> Yeah, absolutely.

So I think that the controlled environments are the ones where we've seen a lot of success. And in the long term, I would love to see robots that could go into any environment like a home or an office and be useful to people. And to do that, we want to actually start collecting data in those environments. Often time in robotics research, we collect the data once in a lab environment and learn based on that data. But if we only collect real data in a lab, we're never going to actually get the robots out into the real world. So we're trying to start data collection efforts, actually putting robots into the real world so that they see the diversity of all these different homes.

Rather than kind of the narrow data that you see in the lab. >> So I want to get your gut about one thing which is you talked about wanting to get a robot to go into a brand new home and make a bowl of cereal, right? Challenging to liquids, cereal fridge. What's your gut about how many different homes, how many different kitchens a robot will need to see. So how many training examples do you think a robot will need in order to have a good chance of doing on the N plus first on the next kitchen? >> Yeah. So I think it really depends on how hard the task is. If the task is as simple as I don't know, maybe pressing a button on the microwave, then I think that you may not need quite as much data for that.

But I think even for a new kitchen, you probably need still quite a large number of kitchens in order to generalize to a new one, maybe 20 maybe 100. If it's a task that's more complex, like making a bowl of cereal, it's something that's super simple to people. But it's actually really hard for robots because there's lots of different steps involved. For that, I think you need more data. And probably also more kitchens as well, maybe in the thousands. And I also think that even if we had thousands of kitchens of data or tens of thousands might guess is that the problem still wouldn't be solved fully. We've seen applications like autonomous driving where there's tons of data and the problem isn't quite yet solved.

So I think we also need to try to go beyond the data distribution and also allow robots to adapt on the fly in a new environment. >> Well, thousands, that's higher than my gut but that's interesting to see how it plays out and so on the theme of making robots more robust. I know a lot of your research work was also on meta learning or learning how to learn rather than us telling computer this is a learning algorithm to how they get a computer to even get better at learning. Can you say a bit about that. >> Yeah, absolutely. So, part of the motivation is that if a robot encounters a new kitchen, we don't want it to just run what it learned in that new kitchen.

It may actually need to adapt and learn on the fly. It may need to, for example, if it's trying to open the door to a pantry and the door is a little bit different. It may need to take a little bit of experience and trial and error on that door in order to figure out how to open it. And we want to allow systems to do this sort of learning process on the fly at test time in new scenarios. And unfortunately, if you just run machine learning from scratch at test time on a very small amount of data that won't work very well. You need large amounts of data in order to learn something. So what meta learning tries to do is it tries to take experience in previous kitchens or previous tasks.

And optimize for the ability to learn new things, the ability to learn new things based on having learned lots of previous things. >> So for example, what best learning rate which wish to adapt to new kitchen setting and lots of other things, one could tune. >> Yeah, absolutely. So the simplest setting is if you're only optimizing for one parameter, like the learning rate in a new situation. And in more complex meta learning algorithms, you can sort of optimize for the entire learning algorithm. >> So I think if you as one of the world experts in meta learning, in fact, if I remember this is a big part of your PhD thesis. Which you did way back with Peter, who was my PhD student at Stanford.

And so just say more about how meta learning works. >> Yeah. So there's a few different approaches to metal learning. But the approach that was in my PhD thesis was essentially putting a learning problem inside another learning problem. So it's actually forms this bilevel optimiza-

tion, this two level learning problem where you are actually trying to optimize for all of the parameters in this inner learning problem. Such that you could achieve faster learning or more efficient learning. >> Yeah, I remember reading some early papers and very impressive. So lots of software algorithms, machine learning algorithms, reinforcement learning algorithms. One part of your work is you also run a robotics lab with robots running around robot arms.

I'm curious for someone that has not yet spent a lot of time in a robotics lab. Can you tell us what is it like if a student were to join your lab, what's it like to show up and work in the morning and work in a robotics lab? >> Yeah. So in the lab, it's not all that different from running machine learning experiments where you run code, you try to train a model and then evaluate that model. But in addition to those everyday things, there's also interfacing with the real robot. So oftentimes a lot of projects might start with the simulation and start iterating on algorithms. Using a physics engine that might involve actually kind of designing aspects of the simulation to test the hypotheses you want to test.

And also sometimes products won't involve that simulation step or maybe you've already iterated in simulation. And on the real robot, there is everything from setting up the control stack. Such that you can actually command the robot to move to a certain position, to setting up the cameras, to collecting data with the robot. You might be collecting data by using like a V R controller, for example, to tell the robot to move to certain positions. And then also actually when it comes time to evaluate a model, it involves actually running that on the robot and seeing what the robot does in real life. It's a pretty fun process at least for me, I think it's very rewarding to actually see the robot do something rather than just see some numbers or some accuracy of your model.

But it's also requires a little bit of persistence because robots can break down. You might have a motor that breaks, you may not have set up the camera correctly. It might be giving you all zeros for example. There's many more bugs that can come up than in a system without a robot. >> So, number of people have had experience debugging software and then there's debugging machine learning algorithms which has different practices and ideas. And then can you say a bit about what debugging hardware in a robotics lab is like? >> Definitely. I mean, in general, with debugging, you want to rule out possible sources of errors, possible sources of bugs and often times you might have a guess that it is from the software or the hardware.

And from there, I think that many of the machine learning things will apply. You want to look at your data. And because a lot of the data is actually collected in the lab, it may not be from a data set, then there may be something that is wrong about it. And you may want to actually look at the images that the robot is seeing, maybe visualize some of the features that your model has learned. I guess one thing that we did recently is we wanted to actually deploy a policy on multiple different robots. And so we would actually replay actions on a robot and make sure that the robot was reproducing a certain motion that we wanted it to reproduce. How you debug depends a lot on the problem.

I don't know if there's any one way or any kind of single bug that comes up the most because there's all sorts of sources of bugs. But yeah, that gives us a little bit of a sense for what it looks like. >> And I thought one of the lovely and frustrating things about robotics as opposed to software is in software. If you make a mistake, you can always revert back to the early version of soft-

ware. But that time that you do something and your robot breaks and then you go, boy, that's another two months of work or something to rebuild that. That's a unique experience in hardware. >> Definitely. And because we're doing a lot of work on the software side of things, we typically try to get robots that won't break, like nice high end robots so that we can worry more about the software and less about the hardware.

But if there are cases where a robot might break, we try to get robots where you can pretty easily swap out parts where if one motor breaks, you could just swap in a new motor rather than having to repurpose the whole thing. But certainly there, there's lots of challenges that come up. >> Yeah. And you know, I used to fly autonomous helicopters. So every time a helicopter crash, fortunately, we manage to keep it safe and away from people, then this is a well frankly destroyed helicopter and then we have to just go build a new one but it's nice when you go to do that too often. >> Yeah. Fortunately, arms do not crash too much. >> I see. Yeah. Yeah, maybe I should have chosen, working on that instead isn't it.

So in your robotics lab, one tool of choice that you've often used is reinforcement learning. Can you say a bit about the practical lessons, tips and tricks for how to apply reinforcement learning amazing algorithms to your hardware robots. >> Yeah. So, reinforcement learning is a really appealing framework for robotics because in principle, it would allow a robot to learn something completely on its own through trial and error from data that it collected itself. And so it's something that we're really excited about for that reason, but it's also really challenging for a number of reasons. So first reinforcement learning typically assumes that you're given some reward signal, some feedback.

And if you're trying to learn how to play a game, like play a pong or another Atari game, you can just use the score as a reward function and just try to maximize your score. But the real world doesn't give you a score. It doesn't give the robot a score when it tries to learn something. And instead the robot might need to learn its own reward function. It might need to learn what it means to accomplish a task at the same time as also trying to learn how to accomplish that task in the physical world. So that's one big challenge and one way that we approach it. Another challenge is typically in this trial and error. >> And so how do you learn the reward?

Like from a demonstration? How, how do you learn the reward? Yeah. >> So we learned it in a few different ways. We often give it at least give it some examples of what success looks like. And we also sometimes give it a full demonstration of this is an example of what we want you to do. And then you can get into techniques like inverse reinforcement learning, which you worked on a long time ago as well and actually try to invert the reinforcement learning process and try to figure out what reward function underlies this behavior. So beyond learning, reward functions, another challenge is autonomy in the process of reinforcement learning. So typically, you think of it as a trial-and-error process where you try to solve the task and then you want to try it again.

And in between trying the task and trying it again, you need to somehow get back to some starting point. And maybe for example, if you're trying to learn how to do a back flip, trying it again will involve standing up after you fall down. But actually, sometimes learning that resetting behavior is also quite challenging. And there is this challenging interplay between learning how to

do the task and also learning how to recover from all the ways that you failed at the task in the entire process of doing that. >> It's really automating the research process so that you run repeat experiments without having to lug a robot around every single time to put it back. >> Exactly.

Yeah. >> Cool, really cool. And in fact, in your application of reinforcement learning, I think you're among the earliest researchers to do more or less or completely end to end robotics. End to end reinforcement learning. Can you say a bit about what that was like in the early days and why you went that route? >> Yeah. So at the start of my PhD, deep learning was just starting to become popular. After AlexNet was showing that you can do very well with deep neural networks trained end to end for computer vision. And I had just arrived at Berkeley for my phD. There were some people that were working on reinforcement learning with neural networks. And I was pretty excited about that work and I wanted to see in the past work, it was all blind.

The robot wasn't using cameras in any way. And I wanted to see if we could learn a neural network that directly maps from the images taken by the robot's camera to the torques applied to the robots joints. And in that first project, it was a lot of work to try to actually get that initial result. But we were excited to see that we actually could get a single neural network that could learn something like screwing a cap on a bottle or putting a coat hanger on a coat rack with a single neural network mapping directly from pixels to torques. And it was a pretty exciting result. It was definitely very different from what the rest of the robotics community was working on.

And I think the, the paper that we worked on was actually rejected twice because people didn't like deep learning, roboticists didn't like that these neural networks were very black box and you couldn't understand them. But eventually it got published and it was accepted. And now it's actually become a much more prominent paradigm in robotics where a lot more people are now training neural networks to represent the policy, to represent how the robot chooses actions based on the camera images. >> I think that early work you did was quite visionary and set an important help, set an important new direction for the field. What's your prediction for where this will go?

Do you think the percentage of practical robotics work that this train end to end? How much do you think that will continue to go up or not? >> Yeah, I certainly think that it will continue to grow quite a bit. Still not all of the robotics research community is looking at these kinds of approaches, in part because there are useful techniques in control theory that are useful, especially in controlled scenarios, like we talked about. But I think that it will certainly grow. I think that especially as the machine learning field continues to make really impressive advances, I hope that that will convince more people that this sort of paradigm is really promising for robotics as well, especially in terms of handling the vast degree of variety in objects and environments in the world.

In terms of looking forward, in terms of understanding what might happen in the next 10 or 20 years in robotics, one thing that I'm really excited about that we touched on a little bit, is trying to train robots on broader datasets. A lot of the field of robot learning is still collecting a dataset for a project in a lab and then training on that small data set. It's small because it has to be, because you collected it for that project. And I think if we imagine that if computer vision researchers had to collect ImageNet for every single project that they did, they probably wouldn't

be making that much progress. And so, my prediction at least for for the coming years is to move towards a paradigm where we reuse data sets a lot, we use much larger datasets, and ideally are kind of sharing data sets across institutions, across robot platforms.

And scaling up the data that these systems are trained on such that they can generalize more broadly. >> That's an exciting vision. And how do we get there? For images, everyone uses PNG files, and GIF files, and JPEG files, so, the data is highly portable and highly shareable. If different research labs just have different pieces of hardware, how do you make that happen for robotics? >> Yeah, so, it's definitely challenging. The place that we're starting that I'm quite excited about is, first, trying to actually have a little bit of consensus, bring the community together to say, okay, let's start with this robot platform with roughly this setup so that we can at least standardize on some of it, and then try to collect lots and lots of diverse data on that fairly standardized platform.

Start from there, and then if we're able to get some signs of life and some positive success from collecting a broad dataset with those standardized choices, then starting to standardize less and less and move on. Maybe the same robot but with a different control stack, and then maybe starting to move towards different versions of that robot, different grippers, and ultimately lots of different robots and lots of environments. >> Cool, yep, great, this is exciting vision. So, this is one of the things you're doing in robotics. Just looking more broadly, with all the work you do in reinforcement learning, some not even apply to robotics but other applications, where do you hope for reinforcement learning and robotics will go in the next many years? >> Yeah, so, I think that, I mean, I'm hoping that it will move towards these reusable datasets and very broad datasets.

I also think that even beyond robotics, I think that there's lots of other applications of reinforcement learning as well. So, for example, in some of our recent work, we applied reinforcement learning, and actually meta reinforcement learning, a combination of meta learning and reinforcement learning, to education to give feedback on student work, where students would program a game like Breakout and then the system would actually play the game and find the bugs that the students made and give them feedback on how to improve their program and how to continue to learn how to code. >> Now, that's very cool. In fact, robots are so visual, sometimes it's easy to forget that reinforcement learning is a general technique for decision making with delayed rewards.

That's useful for many things other than robots. Frankly, when I see more robots, I used to try to get all my students to learn to solder. I don't know if that was a- >> I learned how to solder an undergrad, although I don't think I actually ever used that in my robotics work. Wow, I see. Maybe things have moved on, but I had to, yeah. >> I think we would just buy robots that didn't need some soldering. >> So, your journey has led you to do cutting edge work in reinforcement learning and robotics. I'm curious what advice you would have for someone looking to break into reinforcement learning or to robotics. Not everyone owns a cool robot, that an easy program, but how should someone get started? >> Yeah, my main advice would just be to try to get your feet wet, and just try to build something, try to program and code something up.

There's lots of different things that you can do. And I think that learning by trying to build something, by trying to do something, is one of the coolest ways to learn because you have a certain goal in mind. >> But would you build something using a simulated robot or do you think you should just build a little robot? >> I think that starting with a simulated robot is probably the easiest and there's less of a learning curve and less things that you have to do, you don't have to go out and buy hardware. That said, I think that also trying to work a little bit with hardware right off the bat is great, and it definitely teaches you about how hard robotics is, robotics is really hard.

But also, by kind of struggling to get the robot to do something, it makes it all that much more rewarding when it actually does something cool, when it does something that you wanted it to do. And, I don't know, some of the robots that we use are fairly expensive. But nowadays, there's actually lots of robots that are fairly inexpensive that people can buy, or you can kind of build your own robot by connecting wheels and a camera or maybe even a smartphone together to build something. I actually got started with LEGO robotics in middle school and you could also build robots out of LEGO and get started that way. >> Yes, actually, I remember I went to LEGO robotics competitions when I was in middle school or something.

That was interesting set of experiences. So, to someone taking the machine learning specialization, if their goal is to work in enforcement learning and robotics, what else would you recommend they look at? What are the other resources you think would be helpful? >> Definitely, so there are resources in terms of just learning about reinforcement learning, and there are kind of courses online, there's also kind of course videos, there's textbooks like Sutton Barto. And so, those are lots of resources. I also think that there are lots of, I don't know, videos online, tutorials ,code bases, you can actually check out a physics engine like the MuJoCo Physics Engine, which is freely available, and try to kind of explore with that.

So, yeah, those are a few resources. There's also one thing that we use a lot to interface with robots is called the Robot Operating System, or ROS. And so, learning about that can also be useful. >> That came out of my research lab wayback working with with a garage. Yeah, good times. Sorry, I didn't mean to interrupt. >> Yeah, so, I think that those are some of the resources, and yeah, not too much more than that. >> Cool, yeah, very cool. With the world moving forward and evolving, it feels like the number of path for someone wanting to get involved in this is even growing and it probably gets a little bit easier every year for someone to enter this field.

So, this is exciting time to be in. >> Yeah, definitely. >> Thanks for joining me here at Stanford University. I'm with Professor Chelsea Finn who is with both the computer science and the electrical engine departments, and leads a research group that does cutting edge work on applying reinforcement learning and machine learning to robotics and other applications. Thanks for joining us, Chelsea. >> Yeah, happy to be here. [MUSIC]